

Description of STM32F2 HAL and low-layer drivers

Introduction

STM32Cube is an STMicroelectronics original initiative to significantly improve developer productivity by reducing development effort, time and cost. STM32Cube covers the STM32 portfolio.

STM32Cube includes:

- [STM32CubeMX](#), a graphical software configuration tool that allows the generation of C initialization code using graphical wizards.
- A comprehensive embedded software platform, delivered per Series (such as [STM32CubeF2](#) for STM32F2)
 - The STM32Cube HAL, STM32 abstraction layer embedded software ensuring maximized portability across the STM32 portfolio. HAL APIs are available for all peripherals.
 - Low-layer APIs (LL) offering a fast light-weight expert-oriented layer which is closer to the hardware than the HAL. LL APIs are available only for a set of peripherals.
 - A consistent set of middleware components such as RTOS, USB, TCP/IP and Graphics.
 - All embedded software utilities, delivered with a full set of examples.

The HAL driver layer provides a simple, generic multi-instance set of APIs (application programming interfaces) to interact with the upper layer (application, libraries and stacks).

The HAL driver APIs are split into two categories: generic APIs, which provide common and generic functions for all the STM32 series and extension APIs, which include specific and customized functions for a given line or part number. The HAL drivers include a complete set of ready-to-use APIs that simplify the user application implementation. For example, the communication peripherals contain APIs to initialize and configure the peripheral, manage data transfers in polling mode, handle interrupts or DMA, and manage communication errors.

The HAL drivers are feature-oriented instead of IP-oriented. For example, the timer APIs are split into several categories following the IP functions, such as basic timer, capture and pulse width modulation (PWM). The HAL driver layer implements run-time failure detection by checking the input values of all functions. Such dynamic checking enhances the firmware robustness. Run-time detection is also suitable for user application development and debugging.

The LL drivers offer hardware services based on the available features of the STM32 peripherals. These services reflect exactly the hardware capabilities, and provide atomic operations that must be called by following the programming model described in the product line reference manual. As a result, the LL services are not based on standalone processes and do not require any additional memory resources to save their states, counter or data pointers. All operations are performed by changing the content of the associated peripheral registers. Unlike the HAL, LL APIs are not provided for peripherals for which optimized access is not a key feature, or for those requiring heavy software configuration and/or a complex upper-level stack (such as USB).

The HAL and LL are complementary and cover a wide range of application requirements:

- The HAL offers high-level and feature-oriented APIs with a high-portability level. These hide the MCU and peripheral complexity from the end-user.
- The LL offers low-level APIs at register level, with better optimization but less portability. These require deep knowledge of the MCU and peripheral specifications.

The HAL- and LL-driver source code is developed in Strict ANSI-C, which makes it independent of the development tools. It is checked with the CodeSonar® static analysis tool. It is fully documented.

It is compliant with MISRA C®:2004 standard.

This user manual is structured as follows:

- Overview of HAL drivers
- Overview of low-layer drivers
- Cohabiting of HAL and LL drivers
- Detailed description of each peripheral driver: configuration structures, functions, and how to use the given API to build your application



1 General information

The [STM32CubeF2](#) MCU Package runs on STM32F2 32-bit microcontrollers based on the Arm® Cortex®-M processor.

Note: Arm is a registered trademark of Arm Limited (or its subsidiaries) in the US and/or elsewhere.



2 Acronyms and definitions

Table 1. Acronyms and definitions

Acronym	Definition
ADC	Analog-to-digital converter
AES	Advanced encryption standard
ANSI	American national standards institute
API	Application programming interface
BSP	Board support package
CAN	Controller area network
CEC	Consumer electronic controller
CMSIS	Cortex microcontroller software interface standard
COMP	Comparator
CORDIC	Trigonometric calculation unit
CPU	Central processing unit
CRC	CRC calculation unit
CRYP	Cryptographic processor
CSS	Clock security system
DAC	Digital to analog converter
DLYB	Delay block
DCMI	Digital camera interface
DFSDM	Digital filter sigma delta modulator
DMA	Direct memory access
DMAMUX	Direct memory access request multiplexer
DSI	Display serial interface
DTS	Digital temperature sensor
ETH	Ethernet controller
EXTI	External interrupt/event controller
FDCAN	Flexible data-rate controller area network unit
FLASH	Flash memory
FMAC	Filtering mathematical calculation unit
FMC	Flexible memory controller
FW	Firewall
GFXMMU	Chrom-GRC™
GPIO	General purpose I/Os
GTZC	Global TrustZone controller
GTZC-MPCBB	GTZC block-based memory protection controller
GTZC-MPCWM	GTZC watermark memory protection controller
GTZC-TZIC	GTZC TrustZone illegal access controller
GTZC-TZSC	GTZC TrustZone security controller

Acronym	Definition
HAL	Hardware abstraction layer
HASH	Hash processor
HCD	USB host controller driver
HRTIM	High-resolution timer
I2C	Inter-integrated circuit
I2S	Inter-integrated sound
ICACHE	Instruction cache
IRDA	Infrared data association
IWDG	Independent watchdog
JPEG	Joint photographic experts group
LCD	Liquid crystal display controller
LTDC	LCD TFT Display Controller
LPTIM	Low-power timer
LPUART	Low-power universal asynchronous receiver/transmitter
MCO	Microcontroller clock output
MDIOS	Management data input/output (MDIO) slave
MDMA	Master direct memory access
MMC	MultiMediaCard
MPU	Memory protection unit
MSP	MCU specific package
NAND	NAND Flash memory
NOR	NOR Flash memory
NVIC	Nested vectored interrupt controller
OCTOSPI	Octo-SPI interface
OPAMP	Operational amplifier
OTFDEC	On-the-fly decryption engine
OTG-FS	USB on-the-go full-speed
PKA	Public key accelerator
PCD	USB peripheral controller driver
PSSI	Parallel synchronous slave interface
PWR	Power controller
QSPI	QuadSPI Flash memory
RAMECC	RAM ECC monitoring
RCC	Reset and clock controller
RNG	Random number generator
RTC	Real-time clock
SAI	Serial audio interface
SD	Secure digital
SDMMC	SD/SDIO/MultiMediaCard card host interface
SMARTCARD	Smartcard IC

Acronym	Definition
SMBUS	System management bus
SPI	Serial peripheral interface
SPDIFRX	SPDIF-RX Receiver interface
SRAM	SRAM external memory
SWPMI	Serial wire protocol master interface
SysTick	System tick timer
TIM	Advanced-control, general-purpose or basic timer
TSC	Touch sensing controller
UART	Universal asynchronous receiver/transmitter
UCPD	USB Type-C and power delivery interface
USART	Universal synchronous receiver/transmitter
VREFBUF	Voltage reference buffer
WWDG	Window watchdog
USB	Universal serial bus
PPP	STM32 peripheral or block

3 Overview of HAL drivers

The HAL drivers are designed to offer a rich set of APIs and to interact easily with the application upper layers. Each driver consists of a set of functions covering the most common peripheral features. The development of each driver is driven by a common API which standardizes the driver structure, the functions and the parameter names.

The HAL drivers include a set of driver modules, each module being linked to a standalone peripheral. However, in some cases, the module is linked to a peripheral functional mode. As an example, several modules exist for the USART peripheral: UART driver module, USART driver module, SMARTCARD driver module and IRDA driver module.

The HAL main features are the following:

- Cross-family portable set of APIs covering the common peripheral features as well as extension APIs in case of specific peripheral features.
- Three API programming models: polling, interrupt and DMA.
- APIs are RTOS compliant:
 - Fully reentrant APIs
 - Systematic usage of timeouts in polling mode.
- Support of peripheral multi-instance allowing concurrent API calls for multiple instances of a given peripheral (USART1, USART2...)
- All HAL APIs implement user-callback functions mechanism:
 - Peripheral Init/DelInit HAL APIs can call user-callback functions to perform peripheral system level Initialization/De-Initialization (clock, GPIOs, interrupt, DMA)
 - Peripherals interrupt events
 - Error events.
- Object locking mechanism: safe hardware access to prevent multiple spurious accesses to shared resources.
- Timeout used for all blocking processes: the timeout can be a simple counter or a timebase.

3.1 HAL and user-application files

3.1.1 HAL driver files

A HAL drivers are composed of the following set of files:

Table 2. HAL driver files

File	Description
<i>stm32f2xx_hal_ppp.c</i>	Main peripheral/module driver file. It includes the APIs that are common to all STM32 devices. <i>Example: stm32f2xx_hal_adc.c, stm32f2xx_hal_irda.c, ...</i>
<i>stm32f2xx_hal_ppp.h</i>	Header file of the main driver C file It includes common data, handle and enumeration structures, define statements and macros, as well as the exported generic APIs. <i>Example: stm32f2xx_hal_adc.h, stm32f2xx_hal_irda.h, ...</i>
<i>stm32f2xx_hal_ppp_ex.c</i>	Extension file of a peripheral/module driver. It includes the specific APIs for a given part number or family, as well as the newly defined APIs that overwrite the default generic APIs if the internal process is implemented in different way. <i>Example: stm32f2xx_hal_adc_ex.c, stm32f2xx_hal_flash_ex.c, ...</i>
<i>stm32f2xx_hal_ppp_ex.h</i>	Header file of the extension C file. It includes the specific data and enumeration structures, define statements and macros, as well as the exported device part number specific APIs

File	Description
	<i>Example: stm32f2xx_hal_adc_ex.h, stm32f2xx_hal_flash_ex.h, ...</i>
<i>stm32f2xx_hal.c</i>	This file is used for HAL initialization and contains DBGMCU, Remap and Time Delay based on SysTick APIs.
<i>stm32f2xx_hal.h</i>	stm32f2xx_hal.c header file
<i>stm32f2xx_hal_msp_template.c</i>	Template file to be copied to the user application folder. It contains the MSP initialization and de-initialization (main routine and callbacks) of the peripheral used in the user application.
<i>stm32f2xx_hal_conf_template.h</i>	Template file allowing to customize the drivers for a given application.
<i>stm32f2xx_hal_def.h</i>	Common HAL resources such as common define statements, enumerations, structures and macros.

3.1.2 User-application files

The minimum files required to build an application using the HAL are listed in the table below:

Table 3. User-application files

File	Description
<i>system_stm32f2xx.c</i>	This file contains SystemInit() which is called at startup just after reset and before branching to the main program. It does not configure the system clock at startup (contrary to the standard library). This is to be done using the HAL APIs in the user files. It allows relocating the vector table in internal SRAM and configuring FSMC peripheral to use as data memory the external SRAM mounted on the evaluation board.
<i>startup_stm32f2xx.s</i>	Toolchain specific file that contains reset handler and exception vectors. For some toolchains, it allows adapting the stack/heap size to fit the application requirements.
<i>stm32f2xx_flash.icf</i> (optional)	Linker file for EWARM toolchain allowing mainly adapting the stack/heap size to fit the application requirements.
<i>stm32f2xx_hal_msp.c</i>	This file contains the MSP initialization and de-initialization (main routine and callbacks) of the peripheral used in the user application.
<i>stm32f2xx_hal_conf.h</i>	This file allows the user to customize the HAL drivers for a specific application. It is not mandatory to modify this configuration. The application can use the default configuration without any modification.
<i>stm32f2xx_it.c/h</i>	This file contains the exceptions handler and peripherals interrupt service routine, and calls HAL_IncTick() at regular time intervals to increment a local variable (declared in <i>stm32f2xx_hal.c</i>) used as HAL timebase. By default, this function is called each 1ms in SysTick ISR. . The PPP_IRQHandler() routine must call HAL_PPP_IRQHandler() if an interrupt based process is used within the application.
<i>main.c/h</i>	This file contains the main program routine, mainly: • Call to HAL_Init() • assert_failed() implementation • system clock configuration • peripheral HAL initialization and user application code.

The STM32Cube package comes with ready-to-use project templates, one for each supported board. Each project contains the files listed above and a preconfigured project for the supported toolchains.

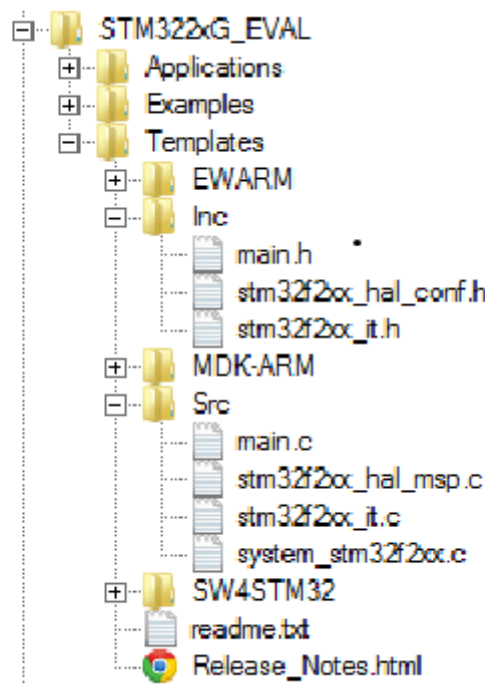
Each project template provides empty main loop function and can be used as a starting point to get familiar with project settings for STM32Cube. Its features are the following:

- It contains the sources of HAL, CMSIS and BSP drivers which are the minimal components to develop a code on a given board.
- It contains the include paths for all the firmware components.

- It defines the STM32 device supported, and allows configuring the CMSIS and HAL drivers accordingly.
- It provides ready to use user files preconfigured as defined below:
 - HAL is initialized
 - SysTick ISR implemented for HAL_Delay()
 - System clock configured with the maximum device frequency.

Note: *If an existing project is copied to another location, then include paths must be updated.*

Figure 1. Example of project template



3.2 HAL data structures

Each HAL driver can contain the following data structures:

- Peripheral handle structures
- Initialization and configuration structures
- Specific process structures.

3.2.1 Peripheral handle structures

The APIs have a modular generic multi-instance architecture that allows working with several IP instances simultaneously.

PPP_HandleTypeDef *handle is the main structure that is implemented in the HAL drivers. It handles the peripheral/module configuration and registers and embeds all the structures and variables needed to follow the peripheral device flow.

The peripheral handle is used for the following purposes:

- Multi-instance support: each peripheral/module instance has its own handle. As a result instance resources are independent.
- Peripheral process intercommunication: the handle is used to manage shared data resources between the process routines.
Example: global pointers, DMA handles, state machine.
- Storage : this handle is used also to manage global variables within a given HAL driver.

An example of peripheral structure is shown below:

```
typedef struct
{
    USART_TypeDef *Instance; /* USART registers base address */
    USART_InitTypeDef Init; /* Usart communication parameters */
    uint8_t *pTxBuffPtr; /* Pointer to Usart Tx transfer Buffer */
    uint16_t TxXferSize; /* Usart Tx Transfer size */
    __IO uint16_t TxXferCount; /* Usart Tx Transfer Counter */
    uint8_t *pRxBuffPtr; /* Pointer to Usart Rx transfer Buffer */
    uint16_t RxXferSize; /* Usart Rx Transfer size */
    __IO uint16_t RxXferCount; /* Usart Rx Transfer Counter */
    DMA_HandleTypeDef *hdmatx; /* Usart Tx DMA Handle parameters */
    DMA_HandleTypeDef *hdmarx; /* Usart Rx DMA Handle parameters */
    HAL_LockTypeDef Lock; /* Locking object */
    __IO HAL_USART_StateTypeDef State; /* Usart communication state */
    __IO HAL_USART_ErrorTypeDef ErrorCode; /* USART Error code */
}USART_HandleTypeDef;
```

Note:

1. *The multi-instance feature implies that all the APIs used in the application are reentrant and avoid using global variables because subroutines can fail to be reentrant if they rely on a global variable to remain unchanged but that variable is modified when the subroutine is recursively invoked. For this reason, the following rules are respected:*
 - *Reentrant code does not hold any static (or global) non-constant data: reentrant functions can work with global data. For example, a reentrant interrupt service routine can grab a piece of hardware status to work with (e.g. serial port read buffer) which is not only global, but volatile. Still, typical use of static variables and global data is not advised, in the sense that only atomic read-modify-write instructions should be used in these variables. It should not be possible for an interrupt or signal to occur during the execution of such an instruction.*
 - *Reentrant code does not modify its own code.*
2. *When a peripheral can manage several processes simultaneously using the DMA (full duplex case), the DMA interface handle for each process is added in the PPP_HandleTypeDef.*
3. *For the shared and system peripherals, no handle or instance object is used. The peripherals concerned by this exception are the following:*
 - GPIO
 - SYSTICK
 - NVIC
 - PWR
 - RCC
 - FLASH

3.2.2 Initialization and configuration structure

These structures are defined in the generic driver header file when it is common to all part numbers. When they can change from one part number to another, the structures are defined in the extension header file for each part number.

```
typedef struct
{
    uint32_t BaudRate; /*!< This member configures the UART communication baudrate.*/
    uint32_t WordLength; /*!< Specifies the number of data bits transmitted or received in a frame.*/
    uint32_t StopBits; /*!< Specifies the number of stop bits transmitted.*/
    uint32_t Parity; /*!< Specifies the parity mode.*/
    uint32_t Mode; /*!< Specifies whether the Receive or Transmit mode is enabled or disabled.*/
    uint32_t HwFlowCtl; /*!< Specifies whether the hardware flow control mode is enabled or disabled.*/
    uint32_t OverSampling; /*!< Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8).*/
}UART_InitTypeDef;
```

Note: *The config structure is used to initialize the sub-modules or sub-instances. See below example:*

```
HAL_ADC_ConfigChannel (ADC_HandleTypeDef* hadc, ADC_ChannelConfTypeDef* sConfig)
```

3.2.3 Specific process structures

The specific process structures are used for specific process (common APIs). They are defined in the generic driver header file.

Example:

```
HAL_PPP_Process (PPP_HandleTypeDef* hadc, PPP_ProcessConfig* sConfig)
```

3.3 API classification

The HAL APIs are classified into three categories:

- **Generic APIs:** common generic APIs applying to all STM32 devices. These APIs are consequently present in the generic HAL driver files of all STM32 microcontrollers.

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_DeInit(ADC_HandleTypeDef *hadc);
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Start_IT(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADC_Stop_IT(ADC_HandleTypeDef* hadc);
void HAL_ADC_IRQHandler(ADC_HandleTypeDef* hadc);
```

- **Extension APIs:**
 - **Family specific APIs:** APIs applying to a given family. They are located in the extension HAL driver file (see example below related to the ADC).

```
HAL_StatusTypeDef HAL_ADCEX_InjectedStop(ADC_HandleTypeDef* hadc); HAL_StatusTypeDef
HAL_ADCEX_InjectedStop_IT(ADC_HandleTypeDef* hadc); HAL_StatusTypeDef HAL_ADCEX_Inj
ectedStart(ADC_HandleTypeDef* hadc); HAL_StatusTypeDef HAL_ADCEX_InjectedStart_IT(AD
C_HandleTypeDef* hadc);
```

The following table summarizes the location of the different categories of HAL APIs in the driver files.

Table 4. API classification

	Generic file	Extension file
Common APIs	X	X ⁽¹⁾
Family specific APIs		X
Device specific APIs		X

1. In some cases, the implementation for a specific device part number may change. In this case the generic API is declared as weak function in the extension file. The API is implemented again to overwrite the default function.

Note: Family specific APIs are only related to a given family. This means that if a specific API is implemented in another family, and the arguments of this latter family are different, additional structures and arguments might need to be added.

Note: The IRQ handlers are used for common and family specific processes.

3.4 Devices supported by HAL drivers

Table 5. List of devices supported by HAL drivers

IP/Module	STM32F205xx	STM32F215xx	STM32F207xx	STM32F217xx
stm32f2xx_hal.c	Yes	Yes	Yes	Yes
stm32f2xx_adc.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_adc_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_can.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_cortex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_crc.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_cryp.c	NO	Yes	NO	Yes
stm32f2xx_hal_dac.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_dac_ex.c	NO	NO	Yes	Yes
stm32f2xx_hal_dcmi.c	NO	NO	Yes	Yes
stm32f2xx_hal_dma.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_dma_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_eth.c	NO	NO	Yes	Yes
stm32f2xx_hal_flash.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_flash_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_gpio.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_hash.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_hcd.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_i2c.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_i2s.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_irda.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_iwdg.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_nand.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_nor.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_pccard.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_pcd.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_pcd_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_pwr.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_pwr_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_rcc.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_rcc_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_rng.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_rtc.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_rtc_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_sd.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_smartcard.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_spi.c	Yes	Yes	Yes	Yes

IP/Module	STM32F205xx	STM32F215xx	STM32F207xx	STM32F217xx
stm32f2xx_hal_sram.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_tim.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_tim_ex.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_uart.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_usart.c	Yes	Yes	Yes	Yes
stm32f2xx_hal_wwdg.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_fsmc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_sdmmc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_usb.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_adc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_crc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_dac.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_dma.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_exti.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_gpio.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_i2c.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_pwr.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_rcc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_rng.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_rtc.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_spi.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_tim.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_usart.c	Yes	Yes	Yes	Yes
stm32f2xx_ll_utils.c	Yes	Yes	Yes	Yes

3.5 HAL driver rules

3.5.1 HAL API naming rules

The following naming rules are used in HAL drivers:

Table 6. HAL API naming rules

	Generic	Family specific	Device specific
File names	<i>stm32f2xx_hal_ppp (c/h)</i>	<i>stm32f2xx_hal_ppp_ex (c/h)</i>	<i>stm32f2xx_hal_ppp_ex (c/h)</i>
Module name	<i>HAL_PPP_MODULE</i>		
Function name	<i>HAL_PPP_Function</i> <i>HAL_PPP_FeatureFunction_MODE</i>	<i>HAL_PPPEX_Function</i> <i>HAL_PPPEX_FeatureFunction_MODE</i>	<i>HAL_PPPEX_Function</i> <i>HAL_PPPEX_FeatureFunction_MODE</i>
Handle name	<i>PPP_HandleTypeDef</i>	NA	NA
Init structure name	<i>PPP_InitTypeDef</i>	NA	<i>PPP_InitTypeDef</i>
Enum name	<i>HAL_PPP_StructnameTypeDef</i>	NA	NA

- The **PPP** prefix refers to the peripheral functional mode and not to the peripheral itself. For example, if the USART, PPP can be USART, IRDA, UART or SMARTCARD depending on the peripheral mode.
- The constants used in one file are defined within this file. A constant used in several files is defined in a header file. All constants are written in uppercase, except for peripheral driver function parameters.
- typedef variable names should be suffixed with *_TypeDef*.
- Registers are considered as constants. In most cases, their name is in uppercase and uses the same acronyms as in the STM32F2 reference manuals.
- Peripheral registers are declared in the *PPP_TypeDef* structure (e.g. *ADC_TypeDef*) in the *stm32f2xxx.h* header file:
stm32f2xxx.h corresponds to *stm32f205xx.h*, *stm32f215.h*, *stm32f207xx.h* or *stm32f217xx.h*.
- Peripheral function names are prefixed by *HAL_*, then the corresponding peripheral acronym in uppercase followed by an underscore. The first letter of each word is in uppercase (e.g. *HAL_UART_Transmit()*). Only one underscore is allowed in a function name to separate the peripheral acronym from the rest of the function name.
- The structure containing the PPP peripheral initialization parameters are named *PPP_InitTypeDef* (e.g. *ADC_InitTypeDef*).
- The structure containing the Specific configuration parameters for the PPP peripheral are named *PPP_xxxxConfTypeDef* (e.g. *ADC_ChannelConfTypeDef*).
- Peripheral handle structures are named *PPP_HandleTypeDef* (e.g. *DMA_HandleTypeDef*)
- The functions used to initialize the PPP peripheral according to parameters specified in *PPP_InitTypeDef* are named *HAL_PPP_Init* (e.g. *HAL_TIM_Init()*).
- The functions used to reset the PPP peripheral registers to their default values are named *HAL_PPP_DeInit* (e.g. *HAL_TIM_DeInit()*).
- The **MODE** suffix refers to the process mode, which can be polling, interrupt or DMA. As an example, when the DMA is used in addition to the native resources, the function should be called: *HAL_PPP_Function_DMA()*.
- The **Feature** prefix should refer to the new feature.
Example: *HAL_ADC_Start()* refers to the injection mode

3.5.2

HAL general naming rules

- For the shared and system peripherals, no handle or instance object is used. This rule applies to the following peripherals:
 - GPIO
 - SYSTICK
 - NVIC
 - RCC
 - FLASH.

Example: The *HAL_GPIO_Init()* requires only the GPIO address and its configuration parameters.

```
HAL_StatusTypeDef HAL_GPIO_Init (GPIO_TypeDef* GPIOx, GPIO_InitTypeDef *Init)
{
  /*GPIO Initialization body */
}
```

- The macros that handle interrupts and specific clock configurations are defined in each peripheral/module driver. These macros are exported in the peripheral driver header files so that they can be used by the extension file. The list of these macros is defined below:

Note: *This list is not exhaustive and other macros related to peripheral features can be added, so that they can be used in the user application.*

Table 7. Macros handling interrupts and specific clock configurations

Macros	Description
<code>__HAL_PPP_ENABLE_IT(__HANDLE__, __INTERRUPT__)</code>	Enables a specific peripheral interrupt
<code>__HAL_PPP_DISABLE_IT(__HANDLE__, __INTERRUPT__)</code>	Disables a specific peripheral interrupt
<code>__HAL_PPP_GET_IT(__HANDLE__, __INTERRUPT__)</code>	Gets a specific peripheral interrupt status
<code>__HAL_PPP_CLEAR_IT(__HANDLE__, __INTERRUPT__)</code>	Clears a specific peripheral interrupt status
<code>__HAL_PPP_GET_FLAG(__HANDLE__, __FLAG__)</code>	Gets a specific peripheral flag status
<code>__HAL_PPP_CLEAR_FLAG(__HANDLE__, __FLAG__)</code>	Clears a specific peripheral flag status
<code>__HAL_PPP_ENABLE(__HANDLE__)</code>	Enables a peripheral
<code>__HAL_PPP_DISABLE(__HANDLE__)</code>	Disables a peripheral
<code>__HAL_PPP_XXXX(__HANDLE__, __PARAM__)</code>	Specific PPP HAL driver macro
<code>__HAL_PPP_GET_IT_SOURCE(__HANDLE__, __INTERRUPT__)</code>	Checks the source of specified interrupt

- NVIC and SYSTICK are two Arm® Cortex® core features. The APIs related to these features are located in the `stm32f2xx_hal_cortex.c` file.
- When a status bit or a flag is read from registers, it is composed of shifted values depending on the number of read values and of their size. In this case, the returned status width is 32 bits. Example : `STATUS = XX | (YY << 16)` or `STATUS = XX | (YY << 8) | (YY << 16) | (YY << 24)`.
- The PPP handles are valid before using the `HAL_PPP_Init()` API. The init function performs a check before modifying the handle fields.

```
HAL_PPP_Init(PPP_HandleTypeDef)
if(hppp == NULL)
{
    return HAL_ERROR;
}
```

- The macros defined below are used:

- Conditional macro:

```
#define ABS(x) ((x) > 0) ? (x) : -(x)
```

- Pseudo-code macro (multiple instructions macro):

```
#define __HAL_LINKDMA(__HANDLE__, __PPP_DMA_FIELD__, __DMA_HANDLE__) \
do{ \
    (__HANDLE__)->__PPP_DMA_FIELD__ = &(__DMA_HANDLE__); \
    (__DMA_HANDLE__)->Parent = (__HANDLE__); \
}while(0)
```

3.5.3 HAL interrupt handler and callback functions

Besides the APIs, HAL peripheral drivers include:

- `HAL_PPP_IRQHandler()` peripheral interrupt handler that should be called from `stm32f2xx_it.c`
- User callback functions.

The user callback functions are defined as empty functions with “weak” attribute. They have to be defined in the user code.

There are three types of user callbacks functions:

- Peripheral system level initialization/ de-Initialization callbacks: HAL_PPP_MspInit() and HAL_PPP_MspDeInit
- Process complete callbacks : HAL_PPP_ProcessCpltCallback
- Error callback: HAL_PPP_ErrorCallback.

Table 8. Callback functions

Callback functions	Example
HAL_PPP_MspInit() / _DeInit()	Example: HAL_USART_MspInit() Called from HAL_PPP_Init() API function to perform peripheral system level initialization (GPIOs, clock, DMA, interrupt)
HAL_PPP_ProcessCpltCallback	Example: HAL_USART_TxCpltCallback Called by peripheral or DMA interrupt handler when the process completes
HAL_PPP_ErrorCallback	Example: HAL_USART_ErrorCallback Called by peripheral or DMA interrupt handler when an error occurs

3.6 HAL generic APIs

The generic APIs provide common generic functions applying to all STM32 devices. They are composed of four APIs groups:

- **Initialization and de-initialization functions:** HAL_PPP_Init(), HAL_PPP_DeInit()
- **IO operation functions:** HAL_PPP_Read(), HAL_PPP_Write(), HAL_PPP_Transmit(), HAL_PPP_Receive()
- **Control functions:** HAL_PPP_Set (), HAL_PPP_Get ().
- **State and Errors functions:** HAL_PPP_GetState (), HAL_PPP_GetError ().

For some peripheral/module drivers, these groups are modified depending on the peripheral/module implementation.

Example: in the timer driver, the API grouping is based on timer features (PWM, OC, IC...).

The initialization and de-initialization functions allow initializing a peripheral and configuring the low-level resources, mainly clocks, GPIO, alternate functions (AF) and possibly DMA and interrupts. The *HAL_DeInit()* function restores the peripheral default state, frees the low-level resources and removes any direct dependency with the hardware.

The IO operation functions perform a row access to the peripheral payload data in write and read modes.

The control functions are used to change dynamically the peripheral configuration and set another operating mode.

The peripheral state and errors functions allow retrieving in run time the peripheral and data flow states, and identifying the type of errors that occurred. The example below is based on the ADC peripheral. The list of generic APIs is not exhaustive. It is only given as an example.

Table 9. HAL generic APIs

Function group	Common API name	Description
Initialization group	<i>HAL_ADC_Init()</i>	This function initializes the peripheral and configures the low -level resources (clocks, GPIO, AF..)
	<i>HAL_ADC_DeInit()</i>	This function restores the peripheral default state, frees the low-level resources and removes any direct dependency with the hardware.
IO operation group	<i>HAL_ADC_Start ()</i>	This function starts ADC conversions when the polling method is used

Function group	Common API name	Description
<i>IO operation group</i>	<i>HAL_ADC_Stop ()</i>	This function stops ADC conversions when the polling method is used
	<i>HAL_ADC_PollForConversion()</i>	This function allows waiting for the end of conversions when the polling method is used. In this case, a timeout value is specified by the user according to the application.
	<i>HAL_ADC_Start_IT()</i>	This function starts ADC conversions when the interrupt method is used
	<i>HAL_ADC_Stop_IT()</i>	This function stops ADC conversions when the interrupt method is used
	<i>HAL_ADC_IRQHandler()</i>	This function handles ADC interrupt requests
	<i>HAL_ADC_ConvCpltCallback()</i>	Callback function called in the IT subroutine to indicate the end of the current process or when a DMA transfer has completed
	<i>HAL_ADC_ErrorCallback()</i>	Callback function called in the IT subroutine if a peripheral error or a DMA transfer error occurred
<i>Control group</i>	<i>HAL_ADC_ConfigChannel()</i>	This function configures the selected ADC regular channel, the corresponding rank in the sequencer and the sample time
	<i>HAL_ADC_AnalogWDGConfig</i>	This function configures the analog watchdog for the selected ADC
<i>State and Errors group</i>	<i>HAL_ADC_GetState()</i>	This function allows getting in run time the peripheral and the data flow states.
	<i>HAL_ADC_GetError()</i>	This function allows getting in run time the error that occurred during IT routine

3.7 HAL extension APIs

3.7.1 HAL extension model overview

The extension APIs provide specific functions or overwrite modified APIs for a specific family (series) or specific part number within the same family.

The extension model consists of an additional file, *stm32f2xx_hal_ppp_ex.c*, that includes all the specific functions and define statements (*stm32f2xx_hal_ppp_ex.h*) for a given part number.

Below an example based on the ADC peripheral:

Table 10. HAL extension APIs

Function Group	Common API Name
<i>HAL_ADCEx_InjectedStart()</i>	This function starts injected channel ADC conversions when the polling method is used
<i>HAL_ADCEx_InjectedStop()</i>	This function stops injected channel ADC conversions when the polling method is used
<i>HAL_ADCEx_InjectedStart_IT()</i>	This function starts injected channel ADC conversions when the interrupt method is used
<i>HAL_ADCEx_InjectedStop_IT()</i>	This function stops injected channel ADC conversions when the interrupt method is used
<i>HAL_ADCEx_InjectedConfigChannel()</i>	This function configures the selected ADC Injected channel (corresponding rank in the sequencer and sample time)

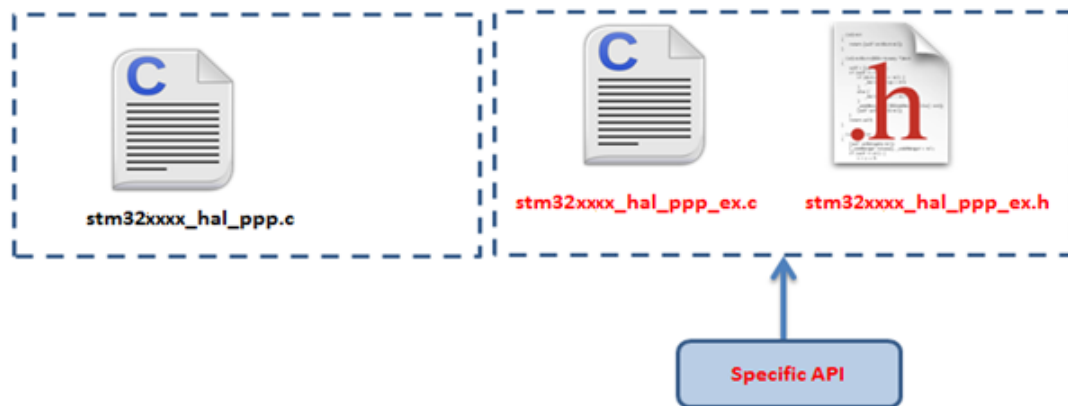
3.7.2 HAL extension model cases

The specific IP features can be handled by the HAL drivers in five different ways. They are described below.

Adding a part number-specific function

When a new feature specific to a given device is required, the new APIs are added in the stm32f2xx_hal_ppp_ex.c extension file. They are named HAL_PPPEX_Function().

Figure 2. Adding device-specific functions



Example: `stm32f2xx_hal_adc_ex.c/h`

```
HAL_StatusTypeDef HAL_ADCEx_InjectedStop(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStop_IT(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStart(ADC_HandleTypeDef* hadc);
HAL_StatusTypeDef HAL_ADCEx_InjectedStart_IT(ADC_HandleTypeDef* hadc);
```

Adding a family-specific function

In this case, the API is added in the extension driver C file and named HAL_PPPEX_Function().

Figure 3. Adding family-specific functions



Adding a new peripheral (specific to a device belonging to a given family)

When a peripheral which is available only in a specific device is required, the APIs corresponding to this new peripheral/module (newPPP) are added in a new `stm32f2xx_hal_newppp.c`. However the inclusion of this file is selected in the `stm32f2xx_hal_conf.h` using the macro:

```
#define HAL_NEWPPP_MODULE_ENABLED
```


Figure 4. Adding new peripherals

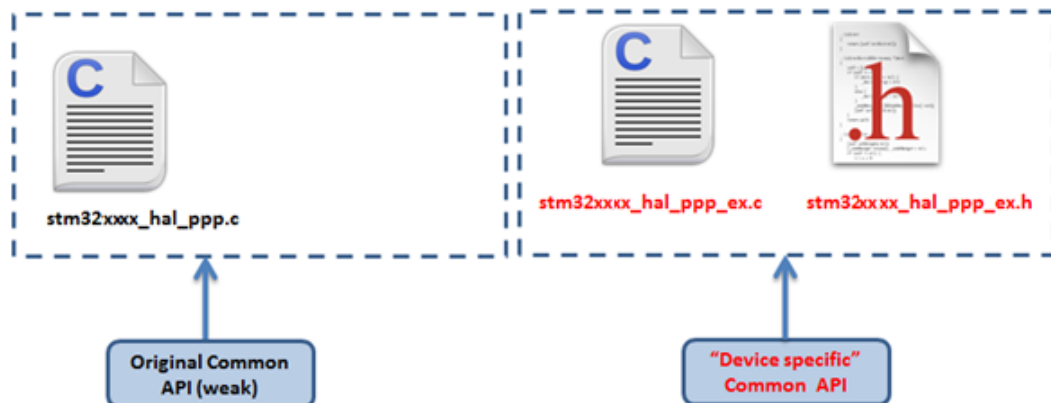


Example: stm32f2xx_hal_adc.c/h

Updating existing common APIs

In this case, the routines are defined with the same names in the stm32f2xx_hal_ppp_ex.c extension file, while the generic API is defined as *weak*, so that the compiler will overwrite the original routine by the new defined function.

Figure 5. Updating existing APIs



Updating existing data structures

The data structure for a specific device part number (e.g. PPP_InitTypeDef) can have different fields. In this case, the data structure is defined in the extension header file and delimited by the specific part number define statement.

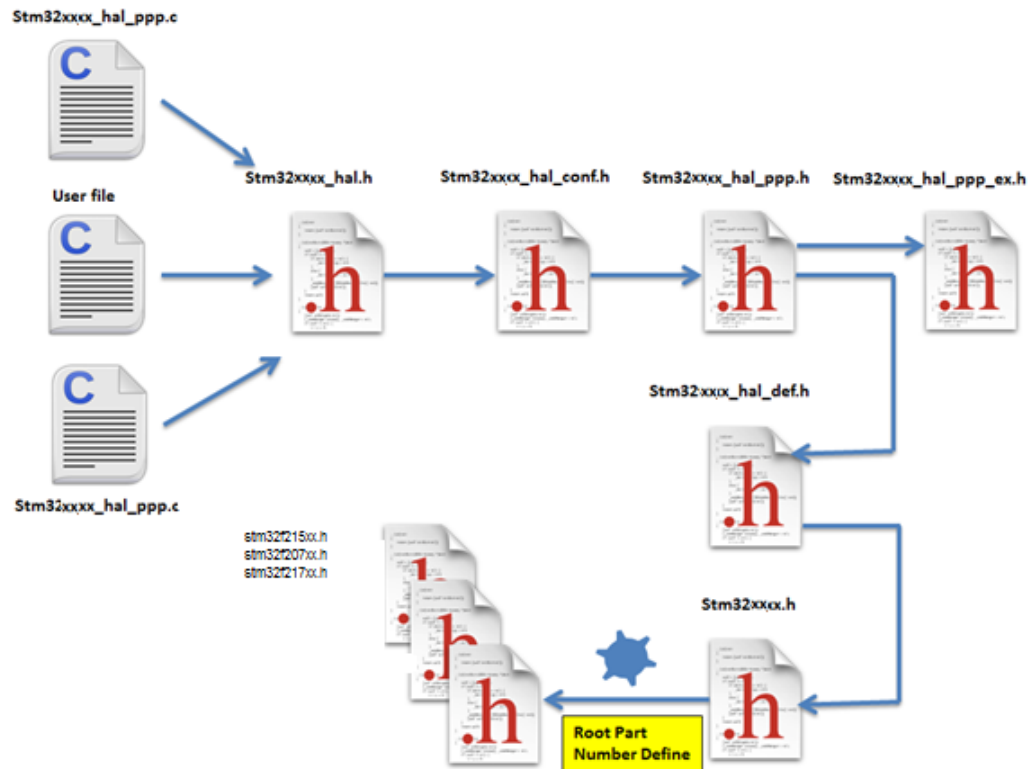
Example:

```
#if defined (STM32F205xx)
typedef struct
{
    (...)
} PPP_InitTypeDef;
#endif /* STM32F205xx */
```

3.8 File inclusion model

The header of the common HAL driver file (*stm32f2xx_hal.h*) includes the common configurations for the whole HAL library. It is the only header file that is included in the user sources and the HAL C sources files to be able to use the HAL resources.

Figure 6. File inclusion model



A PPP driver is a standalone module which is used in a project. The user must enable the corresponding `USE_HAL_PPP_MODULE` define statement in the configuration file.

```

/*****
 * @file stm32f2xx_hal_conf.h
 * @author MCD Application Team
 * @version VX.Y.Z * @date dd-mm-yyyy
 * @brief This file contains the modules to be used
 *****/
(...)
#define USE_HAL_USART_MODULE
#define USE_HAL_IRDA_MODULE
#define USE_HAL_DMA_MODULE
#define USE_HAL_RCC_MODULE
(...)

```

3.9 HAL common resources

The common HAL resources, such as common define enumerations, structures and macros, are defined in *stm32f2xx_hal_def.h*. The main common define enumeration is *HAL_StatusTypeDef*.

- **HAL Status**

The HAL status is used by almost all HAL APIs, except for boolean functions and IRQ handler. It returns the status of the current API operations. It has four possible values as described below:

```
typedef enum
{
  HAL_OK = 0x00,
  HAL_ERROR = 0x01,
  HAL_BUSY = 0x02,
  HAL_TIMEOUT = 0x03
} HAL_StatusTypeDef;
```

- **HAL Locked**

The HAL lock is used by all HAL APIs to prevent accessing by accident shared resources.

```
typedef enum
{
  HAL_UNLOCKED = 0x00, /*!<Resources unlocked */
  HAL_LOCKED = 0x01 /*!< Resources locked */
} HAL_LockTypeDef;
```

In addition to common resources, the stm32f2xx_hal_def.h file calls the stm32f2xx.h file in CMSIS library to get the data structures and the address mapping for all peripherals:

- Declarations of peripheral registers and bits definition.
- Macros to access peripheral registers hardware (Write register, Read register...etc.).

- **Common macros**

- Macro defining HAL_MAX_DELAY

```
#define HAL_MAX_DELAY 0xFFFFFFFF
```

- Macro linking a PPP peripheral to a DMA structure pointer:

```
#define __HAL_LINKDMA(__HANDLE__, __PPP_DMA_FIELD__, __DMA_HANDLE__) \
do{ \
  (__HANDLE__)->__PPP_DMA_FIELD__ = &(__DMA_HANDLE__); \
  (__DMA_HANDLE__).Parent = (__HANDLE__); \
} while(0)
```

3.10 HAL configuration

The configuration file, stm32f2xx_hal_conf.h, allows customizing the drivers for the user application. Modifying this configuration is not mandatory: the application can use the default configuration without any modification.

To configure these parameters, the user should enable, disable or modify some options by uncommenting, commenting or modifying the values of the related define statements as described in the table below:

Table 11. Define statements used for HAL configuration

Configuration item	Description	Default Value
HSE_VALUE	Defines the value of the external oscillator (HSE) expressed in Hz. The user must adjust this define statement when using a different crystal value.	25 000 000 (Hz)
HSE_STARTUP_TIMEOUT	Timeout for HSE start-up, expressed in ms	5000
HSI_VALUE	Defines the value of the internal oscillator (HSI) expressed in Hz.	16 000 000 (Hz)
EXTERNAL_CLOCK_VALUE	This value is used by the I2S HAL module to compute the I2S clock source frequency, this source is inserted directly through I2S_CKIN pad.	12288000 (Hz)
VDD_VALUE	VDD value	3300 (mV)

Configuration item	Description	Default Value
USE_RTOS	Enables the use of RTOS	FALSE (for future use)
PREFETCH_ENABLE	Enables prefetch feature	TRUE
INSTRUCTION_CACHE_ENABLE	Enables instruction cache	TRUE
DATA_CACHE_ENABLE	Enables data cache	TRUE
USE_HAL_PPP_MODULE	Enables module to be used in the HAL driver	
MAC_ADDRx	Ethernet peripheral configuration : MAC address	
ETH_RX_BUF_SIZE	Ethernet buffer size for reception	ETH_MAX_PACKET_SIZE
ETH_TX_BUF_SIZE	Ethernet buffer size for transmit	ETH_MAX_PACKET_SIZE
ETH_RXBUFNB	The number of Rx buffers of size ETH_RX_BUF_SIZE	4
ETH_TXBUFNB	The number of Tx buffers of size ETH_RX_BUF_SIZE	4
DP83848_PHY_ADDRESS	DB83848 Ethernet PHY Address	0x01
PHY_RESET_DELAY	PHY Reset delay these values are based on a 1 ms SysTick interrupt	0x000000FF
PHY_CONFIG_DELAY	PHY Configuration delay	0x00000FFF
PHY_BCR PHY_BSR	Common PHY Registers	
PHY_SR PHY_MICR PHY_MISR	Extended PHY registers	

Note: The `stm32f2xx_hal_conf_template.h` file is located in the HAL drivers Inc folder. It should be copied to the user folder, renamed and modified as described above.

Note: By default, the values defined in the `stm32f2xx_hal_conf_template.h` file are the same as the ones used for the examples and demonstrations. All HAL include files are enabled so that they can be used in the user code without modifications.

3.11 HAL system peripheral handling

This chapter gives an overview of how the system peripherals are handled by the HAL drivers. The full API list is provided within each peripheral driver description section.

3.11.1 Clock

Two main functions can be used to configure the system clock:

- `HAL_RCC_OscConfig` (`RCC_OscInitTypeDef *RCC_OscInitStruct`). This function configures/enables multiple clock sources (HSE, HSI, LSE, LSI, PLL).
- `HAL_RCC_ClockConfig` (`RCC_ClkInitTypeDef *RCC_ClkInitStruct`, `uint32_t FLatency`). This function
 - selects the system clock source
 - configures AHB, APB1 and APB2 clock dividers
 - configures the number of Flash memory wait states
 - updates the SysTick configuration when HCLK clock changes.

Some peripheral clocks are not derived from the system clock (such as RTC, SDIO, I2S, Audio, PLL). In this case, the clock configuration is performed by an extended API defined in `stm32f2xx_hal_rcc_ex.c`:

`HAL_RCCEX_PeriphCLKConfig(RCC_PeriphCLKInitTypeDef *PeriphClkInit).`

Additional RCC HAL driver functions are available:

- `HAL_RCC_DeInit()` Clock de-initialization function that returns clock configuration to reset state
- Get clock functions that allow retrieving various clock configurations (system clock, HCLK, PCLK1, PCLK2, ...)
- MCO and CSS configuration functions

A set of macros are defined in `stm32f2xx_hal_rcc.h` and `stm32f2xx_hal_rcc_ex.h`. They allow executing elementary operations on RCC block registers, such as peripherals clock gating/reset control:

- `__HAL_PPP_CLK_ENABLE/ __HAL_PPP_CLK_DISABLE` to enable/disable the peripheral clock
- `__HAL_PPP_FORCE_RESET/ __HAL_PPP_RELEASE_RESET` to force/release peripheral reset
- `__HAL_PPP_CLK_SLEEP_ENABLE/ __HAL_PPP_CLK_SLEEP_DISABLE` to enable/disable the peripheral clock during Sleep mode.

3.11.2

GPIOs

GPIO HAL APIs are the following:

- `HAL_GPIO_Init()` / `HAL_GPIO_DeInit()`
- `HAL_GPIO_ReadPin()` / `HAL_GPIO_WritePin()`
- `HAL_GPIO_TogglePin()`.

In addition to standard GPIO modes (input, output, analog), the pin mode can be configured as EXTI with interrupt or event generation.

When selecting EXTI mode with interrupt generation, the user must call `HAL_GPIO_EXTI_IRQHandler()` from `stm32f2xx_it.c` and implement `HAL_GPIO_EXTI_Callback()`

The table below describes the `GPIO_InitTypeDef` structure field.

Table 12. Description of `GPIO_InitTypeDef` structure

Structure field	Description
Pin	Specifies the GPIO pins to be configured. Possible values: <code>GPIO_PIN_x</code> or <code>GPIO_PIN_All</code> , where <code>x[0..15]</code>
Mode	Specifies the operating mode for the selected pins: GPIO mode or EXTI mode. Possible values are: • <u>GPIO mode</u> – <code>GPIO_MODE_INPUT</code> : Input floating – <code>GPIO_MODE_OUTPUT_PP</code> : Output push-pull – <code>GPIO_MODE_OUTPUT_OD</code> : Output open drain – <code>GPIO_MODE_AF_PP</code> : Alternate function push-pull – <code>GPIO_MODE_AF_OD</code> : Alternate function open drain – <code>GPIO_MODE_ANALOG</code> : Analog mode • <u>External Interrupt mode</u> – <code>GPIO_MODE_IT_RISING</code> : Rising edge trigger detection – <code>GPIO_MODE_IT_FALLING</code> : Falling edge trigger detection – <code>GPIO_MODE_IT_RISING_FALLING</code> : Rising/Falling edge trigger detection • <u>External Event mode</u> – <code>GPIO_MODE_EVT_RISING</code> : Rising edge trigger detection – <code>GPIO_MODE_EVT_FALLING</code> : Falling edge trigger detection – <code>GPIO_MODE_EVT_RISING_FALLING</code> : Rising/Falling edge trigger detection
Pull	Specifies the Pull-up or Pull-down activation for the selected pins. Possible values are: <code>GPIO_NOPULL</code> <code>GPIO_PULLUP</code> <code>GPIO_PULLDOWN</code>
Speed	Specifies the speed for the selected pins Possible values are: <code>GPIO_SPEED_LOW</code> <code>GPIO_SPEED_MEDIUM</code> <code>GPIO_SPEED_HIGH</code>

Please find below typical GPIO configuration examples:

- Configuring GPIOs as output push-pull to drive external LEDs:

```
GPIO_InitStruct.Pin = GPIO_PIN_12 | GPIO_PIN_13 | GPIO_PIN_14 | GPIO_PIN_15;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_PULLUP;
GPIO_InitStruct.Speed = GPIO_SPEED_MEDIUM;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);
```

- Configuring PA0 as external interrupt with falling edge sensitivity:

```
GPIO_InitStructure.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStructure.Pull = GPIO_NOPULL;
GPIO_InitStructure.Pin = GPIO_PIN_0;
HAL_GPIO_Init(GPIOA, &GPIO_InitStructure);
```

3.11.3 Cortex® NVIC and SysTick timer

The Cortex® HAL driver, `stm32f2xx_hal_cortex.c`, provides APIs to handle NVIC and SysTick. The supported APIs include:

- HAL_NVIC_SetPriority()/ HAL_NVIC_SetPriorityGrouping()
- HAL_NVIC_GetPriority() / HAL_NVIC_GetPriorityGrouping()
- HAL_NVIC_EnableIRQ()/HAL_NVIC_DisableIRQ()
- HAL_NVIC_SystemReset()
- HAL_SYSTICK_IRQHandler()
- HAL_NVIC_GetPendingIRQ() / HAL_NVIC_SetPendingIRQ () / HAL_NVIC_ClearPendingIRQ()
- HAL_NVIC_GetActive(IRQn)
- HAL_SYSTICK_Config()
- HAL_SYSTICK_CLKSourceConfig()
- HAL_SYSTICK_Callback()

3.11.4 PWR

The PWR HAL driver handles power management. The features shared between all STM32 Series are listed below:

- PVD configuration, enabling/disabling and interrupt handling
 - HAL_PWR_ConfigPVD()
 - HAL_PWR_EnablePVD() / HAL_PWR_DisablePVD()
 - HAL_PWR_PVD_IRQHandler()
 - HAL_PWR_PVDCallback()
- Wakeup pin configuration
 - HAL_PWR_EnableWakeUpPin() / HAL_PWR_DisableWakeUpPin()
- Low-power mode entry
 - HAL_PWR_EnterSLEEPMode()
 - HAL_PWR_EnterSTOPMode()
 - HAL_PWR_EnterSTANDBYMode()

Depending on the STM32 Series, extension functions are available in `stm32f2xx_hal_pwr_ex`. Here are a few examples (the list is not exhaustive):

- Backup domain registers enable/disable
 - HAL_PWREx_EnableBkUpReg() / HAL_PWREx_DisableBkUpReg()
- Flash power-down control:
 - HAL_PWREx_EnableFlashPowerDown()
 - HAL_PWREx_DisableFlashPowerDown().

3.11.5

EXTI

The EXTI is not considered as a standalone peripheral but rather as a service used by other peripheral, that are handled through EXTI HAL APIs. In addition, each peripheral HAL driver implements the associated EXTI configuration and function as macros in its header file.

The first 16 EXTI lines connected to the GPIOs are managed within the GPIO driver. The GPIO_InitTypeDef structure allows configuring an I/O as external interrupt or external event.

The EXTI lines connected internally to the PVD, RTC, USB, and Ethernet are configured within the HAL drivers of these peripheral through the macros given in the table below.

The EXTI internal connections depend on the targeted STM32 microcontroller (refer to the product datasheet for more details):

Table 13. Description of EXTI configuration macros

Macros	Description
PPP_EXTI_LINE_FUNCTION	Defines the EXTI line connected to the internal peripheral. Example: <pre>#define PWR_EXTI_LINE_PVD ((uint32_t)0x00010000) /*!<External interrupt line 16 Connected to the PVD EXTI Line */</pre>
__HAL_PPP_EXTI_ENABLE_IT(__EXTI_LINE__)	Enables a given EXTI line Example: <pre>__HAL_PVD_EXTI_ENABLE_IT(PWR_EXTI_LINE_PVD)</pre>
__HAL_PPP_EXTI_DISABLE_IT(__EXTI_LINE__)	Disables a given EXTI line. Example: <pre>__HAL_PVD_EXTI_DISABLE_IT(PWR_EXTI_LINE_PVD)</pre>
__HAL_PPP_EXTI_GET_FLAG(__EXTI_LINE__)	Gets a given EXTI line interrupt flag pending bit status. Example: <pre>__HAL_PVD_EXTI_GET_FLAG(PWR_EXTI_LINE_PVD)</pre>
__HAL_PPP_EXTI_CLEAR_FLAG(__EXTI_LINE__)	Clears a given EXTI line interrupt flag pending bit. Example; <pre>__HAL_PVD_EXTI_CLEAR_FLAG(PWR_EXTI_LINE_PVD)</pre>
__HAL_PPP_EXTI_GENERATE_SWIT (__EXTI_LINE__)	Generates a software interrupt for a given EXTI line. Example: <pre>__HAL_PVD_EXTI_GENERATE_SWIT (PWR_EXTI_LINE_PVD)</pre>

If the EXTI interrupt mode is selected, the user application must call HAL_PPP_FUNCTION_IRQHandler() (for example HAL_PWR_PVD_IRQHandler()), from stm32f2xx_it.c file, and implement HAL_PPP_FUNCTIONCallback() callback function (for example HAL_PWR_PVDCallback()).

3.11.6

DMA

The DMA HAL driver allows enabling and configuring the peripheral to be connected to the DMA Channels (except for internal SRAM/FLASH memory which do not require any initialization). Refer to the product reference manual for details on the DMA request corresponding to each peripheral.

For a given channel, HAL_DMA_Init() API allows programming the required configuration through the following parameters:

- Transfer direction
- Source and destination data formats
- Circular, Normal or peripheral flow control mode
- Channel priority level
- Source and destination Increment mode
- FIFO mode and its threshold (if needed)

- Burst mode for source and/or destination (if needed).

Two operating modes are available:

- Polling mode I/O operation
 1. Use `HAL_DMA_Start()` to start DMA transfer when the source and destination addresses and the Length of data to be transferred have been configured.
 2. Use `HAL_DMA_PollForTransfer()` to poll for the end of current transfer. In this case a fixed timeout can be configured depending on the user application.
- Interrupt mode I/O operation
 1. Configure the DMA interrupt priority using `HAL_NVIC_SetPriority()`
 2. Enable the DMA IRQ handler using `HAL_NVIC_EnableIRQ()`
 3. Use `HAL_DMA_Start_IT()` to start DMA transfer when the source and destination addresses and the length of data to be transferred have been configured. In this case the DMA interrupt is configured.
 4. Use `HAL_DMA_IRQHandler()` called under `DMA_IRQHandler()` Interrupt subroutine
 5. When data transfer is complete, `HAL_DMA_IRQHandler()` function is executed and a user function can be called by customizing `XferCpltCallback` and `XferErrorCallback` function pointer (i.e. a member of DMA handle structure).

Additional functions and macros are available to ensure efficient DMA management:

- Use `HAL_DMA_GetState()` function to return the DMA state and `HAL_DMA_GetError()` in case of error detection.
- Use `HAL_DMA_Abort()` function to abort the current transfer

The most used DMA HAL driver macros are the following:

- `__HAL_DMA_ENABLE`: enables the specified DMA channel.
- `__HAL_DMA_DISABLE`: disables the specified DMA channel.
- `__HAL_DMA_GET_FS`: returns the current DMA Stream FIFO filled level.
- `__HAL_DMA_GET_FLAG`: gets the DMA channel pending flags.
- `__HAL_DMA_CLEAR_FLAG`: clears the DMA channel pending flags.
- `__HAL_DMA_ENABLE_IT`: enables the specified DMA channel interrupts.
- `__HAL_DMA_DISABLE_IT`: disables the specified DMA channel interrupts.
- `__HAL_DMA_GET_IT_SOURCE`: checks whether the specified DMA channel interrupt has been enabled or not.

Note: When a peripheral is used in DMA mode, the DMA initialization should be done in the `HAL_PPP_MspInit()` callback. In addition, the user application should associate the DMA handle to the PPP handle (refer to section "HAL IO operation functions").

Note: DMA double-buffering feature is handled as an extension API.

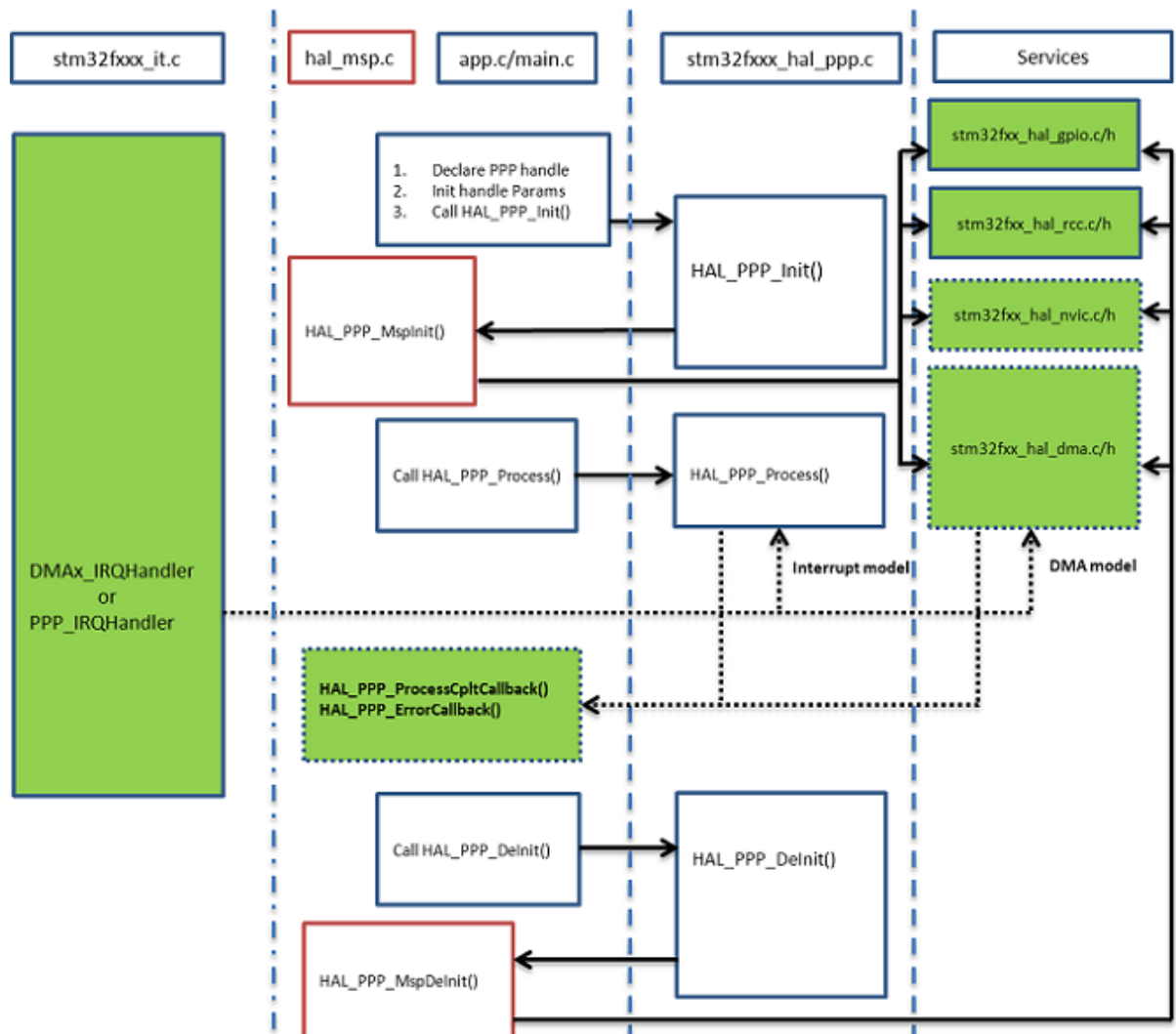
Note: DMA channel callbacks need to be initialized by the user application only in case of memory-to-memory transfer. However when peripheral-to-memory transfers are used, these callbacks are automatically initialized by calling a process API function that uses the DMA.

3.12 How to use HAL drivers

3.12.1 HAL usage models

The following figure shows the typical use of the HAL driver and the interaction between the application user, the HAL driver and the interrupts.

Figure 7. HAL driver model



Note: The functions implemented in the HAL driver are shown in green, the functions called from interrupt handlers in dotted lines, and the msp functions implemented in the user application in red. Non-dotted lines represent the interactions between the user application functions.

Basically, the HAL driver APIs are called from user files and optionally from interrupt handlers file when the APIs based on the DMA or the PPP peripheral dedicated interrupts are used.

When DMA or PPP peripheral interrupts are used, the PPP process complete callbacks are called to inform the user about the process completion in real-time event mode (interrupts). Note that the same process completion callbacks are used for DMA in interrupt mode.

3.12.2 HAL initialization

3.12.2.1 HAL global initialization

In addition to the peripheral initialization and de-initialization functions, a set of APIs are provided to initialize the HAL core implemented in file stm32f2xx_hal.c.

- HAL_Init(): this function must be called at application startup to
 - initialize data/instruction cache and pre-fetch queue
 - set SysTick timer to generate an interrupt each 1ms (based on HSI clock) with the lowest priority
 - call HAL_MspInit() user callback function to perform system level initializations (Clock, GPIOs, DMA, interrupts). HAL_MspInit() is defined as “weak” empty function in the HAL drivers.

- **HAL_DeInit()**
 - resets all peripherals
 - calls function **HAL_MspDeInit()** which is a user callback function to do system level De-Initializations.
- **HAL_GetTick()**: this function gets current SysTick counter value (incremented in SysTick interrupt) used by peripherals drivers to handle timeouts.
- **HAL_Delay()**. this function implements a delay (expressed in milliseconds) using the SysTick timer.
 Care must be taken when using **HAL_Delay()** since this function provides an accurate delay (expressed in milliseconds) based on a variable incremented in SysTick ISR. This means that if **HAL_Delay()** is called from a peripheral ISR, then the SysTick interrupt must have highest priority (numerically lower) than the peripheral interrupt, otherwise the caller ISR will be blocked.

3.12.2.2 System clock initialization

The clock configuration is done at the beginning of the user code. However the user can change the configuration of the clock in his own code.

Please find below the typical Clock configuration sequence:

```
static void SystemClock_Config(void)
{
  RCC_ClkInitTypeDef RCC_ClkInitStruct;
  RCC_OscInitTypeDef RCC_OscInitStruct;
  /* Enable HSE Oscillator and activate PLL with HSE as source */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON; RCC_OscInitStruct.PLL.PLLState =
  RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLM = 25; RCC_OscInitStruct.PLL.PLLN = 240;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
  RCC_OscInitStruct.PLL.PLLQ = 5;
  HAL_RCC_OscConfig(&RCC_OscInitStruct);
  /* Select PLL as system clock source and configure the HCLK, PCLK1 and PCLK2 clocks
  dividers */
  RCC_ClkInitStruct.ClockType = (RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK |
  RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2);
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
  HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_3); }
```

3.12.2.3 HAL MSP initialization process

The peripheral initialization is done through **HAL_PPP_Init()** while the hardware resources initialization used by a peripheral (PPP) is performed during this initialization by calling MSP callback function **HAL_PPP_MspInit()**.

The **MspInit** callback performs the low level initialization related to the different additional hardware resources: RCC, GPIO, NVIC and DMA.

All the HAL drivers with handles include two MSP callbacks for initialization and de-initialization:

```
/**
 * @brief Initializes the PPP MSP.
 * @param hppp: PPP handle
 * @retval None */
void __weak HAL_PPP_MspInit(PPP_HandleTypeDef *hppp) {
  /* NOTE : This function Should not be modified, when the callback is needed,
  the HAL_PPP_MspInit could be implemented in the user file */
}
/**
 * @brief DeInitializes PPP MSP.
 * @param hppp: PPP handle
 * @retval None */
void __weak HAL_PPP_MspDeInit(PPP_HandleTypeDef *hppp) {
  /* NOTE : This function Should not be modified, when the callback is needed,
  the HAL_PPP_MspDeInit could be implemented in the user file */
}
```

The MSP callbacks are declared empty as weak functions in each peripheral driver. The user can use them to set the low level initialization code or omit them and use his own initialization routine.

The HAL MSP callback is implemented inside the *stm32f2xx_hal_msp.c* file in the user folders. An *stm32f2xx_hal_msp.c* file template is located in the HAL folder and should be copied to the user folder. It can be generated automatically by STM32CubeMX tool and further modified. Note that all the routines are declared as weak functions and could be overwritten or removed to use user low level initialization code.

stm32f2xx_hal_msp.c file contains the following functions:

Table 14. MSP functions

Routine	Description
void HAL_MspInit()	Global MSP initialization routine
void HAL_MspDeInit()	Global MSP de-initialization routine
void HAL_PPP_MspInit()	PPP MSP initialization routine
void HAL_PPP_MspDeInit()	PPP MSP de-initialization routine

By default, if no peripheral needs to be de-initialized during the program execution, the whole MSP initialization is done in *Hal_MspInit()* and MSP De-Initialization in the *Hal_MspDeInit()*. In this case the *HAL_PPP_MspInit()* and *HAL_PPP_MspDeInit()* are not implemented.

When one or more peripherals needs to be de-initialized in run time and the low level resources of a given peripheral need to be released and used by another peripheral, *HAL_PPP_MspDeInit()* and *HAL_PPP_MspInit()* are implemented for the concerned peripheral and other peripherals initialization and de-Initialization are kept in the global *HAL_MspInit()* and the *HAL_MspDeInit()*.

If there is nothing to be initialized by the global *HAL_MspInit()* and *HAL_MspDeInit()*, the two routines can simply be omitted.

3.12.3 HAL I/O operation process

The HAL functions with internal data processing like transmit, receive, write and read are generally provided with three data processing modes as follows:

- Polling mode
- Interrupt mode
- DMA mode

3.12.3.1 Polling mode

In Polling mode, the HAL functions return the process status when the data processing in blocking mode is complete. The operation is considered complete when the function returns the HAL_OK status, otherwise an error status is returned. The user can get more information through the *HAL_PPP_GetState()* function. The data processing is handled internally in a loop. A timeout (expressed in ms) is used to prevent process hanging.

The example below shows the typical Polling mode processing sequence :

```

HAL_StatusTypeDef HAL_PPP_Transmit ( PPP_HandleTypeDef * phandle, uint8_t pData,
int16_t Size, uint32_t Timeout)
{
  if((pData == NULL ) || (Size == 0))
  {
    return HAL_ERROR;
  }
  (...) while (data processing is running)
  {
    if( timeout reached )
    {
      return HAL_TIMEOUT;
    }
  }
  (...)
  return HAL_OK; }

```

3.12.3.2 Interrupt mode

In Interrupt mode, the HAL function returns the process status after starting the data processing and enabling the appropriate interruption. The end of the operation is indicated by a callback declared as a weak function. It can be customized by the user to be informed in real-time about the process completion. The user can also get the process status through the *HAL_PPP_GetState()* function.

In Interrupt mode, four functions are declared in the driver:

- *HAL_PPP_Process_IT()*: launch the process
- *HAL_PPP_IRQHandler()*: the global PPP peripheral interruption
- *__weak HAL_PPP_ProcessCpltCallback ()*: the callback relative to the process completion.
- *__weak HAL_PPP_ProcessErrorCallback()*: the callback relative to the process Error.

To use a process in Interrupt mode, *HAL_PPP_Process_IT()* is called in the user file and *HAL_PPP_IRQHandler* in *stm32f2xx_it.c*.

The *HAL_PPP_ProcessCpltCallback()* function is declared as weak function in the driver. This means that the user can declare it again in the application. The function in the driver is not modified.

An example of use is illustrated below:

main.c file:

```

UART_HandleTypeDef UartHandle;
int main(void)
{
  /* Set User Parameters */
  UartHandle.Init.BaudRate = 9600;
  UartHandle.Init.WordLength = UART_DATABITS_8;
  UartHandle.Init.StopBits = UART_STOPBITS_1;
  UartHandle.Init.Parity = UART_PARITY_NONE;
  UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  UartHandle.Init.Mode = UART_MODE_TX_RX;
  UartHandle.Init.Instance = USART1;
  HAL_UART_Init(&UartHandle);
  HAL_UART_SendIT(&UartHandle, TxBuffer, sizeof(TxBuffer));
  while (1);
}
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
{
}
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
{
}

```

stm32f2xx_it.c file:

```
extern UART_HandleTypeDef UartHandle;
void USART1_IRQHandler(void)
{
  HAL_UART_IRQHandler(&UartHandle);
}
```

3.12.3.3 DMA mode

In DMA mode, the HAL function returns the process status after starting the data processing through the DMA and after enabling the appropriate DMA interruption. The end of the operation is indicated by a callback declared as a weak function and can be customized by the user to be informed in real-time about the process completion. The user can also get the process status through the *HAL_PPP_GetState()* function. For the DMA mode, three functions are declared in the driver:

- *HAL_PPP_Process_DMA()*: launch the process
- *HAL_PPP_DMA_IRQHandler()*: the DMA interruption used by the PPP peripheral
- *__weak HAL_PPP_ProcessCpltCallback()*: the callback relative to the process completion.
- *__weak HAL_PPP_ErrorCpltCallback()*: the callback relative to the process Error.

To use a process in DMA mode, *HAL_PPP_Process_DMA()* is called in the user file and the *HAL_PPP_DMA_IRQHandler()* is placed in the *stm32f2xx_it.c*. When DMA mode is used, the DMA initialization is done in the *HAL_PPP_MspInit()* callback. The user should also associate the DMA handle to the PPP handle. For this purpose, the handles of all the peripheral drivers that use the DMA must be declared as follows:

```
typedef struct
{
  PPP_TypeDef *Instance; /* Register base address */
  PPP_InitTypeDef Init; /* PPP communication parameters */
  HAL_StateTypeDef State; /* PPP communication state */
  (...)
  DMA_HandleTypeDef *hdma; /* associated DMA handle */
} PPP_HandleTypeDef;
```

The initialization is done as follows (UART example):

```
int main(void)
{
  /* Set User Parameters */
  UartHandle.Init.BaudRate = 9600;
  UartHandle.Init.WordLength = UART_DATABITS_8;
  UartHandle.Init.StopBits = UART_STOPBITS_1;
  UartHandle.Init.Parity = UART_PARITY_NONE;
  UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  UartHandle.Init.Mode = UART_MODE_TX_RX;
  UartHandle.Init.Instance = UART1;
  HAL_UART_Init(&UartHandle);
  (...)
}

void HAL_USART_MspInit (UART_HandleTypeDef * huart)
{
  static DMA_HandleTypeDef hdma_tx;
  static DMA_HandleTypeDef hdma_rx;
  (...)
  __HAL_LINKDMA(UartHandle, DMA_Handle_tx, hdma_tx);
  __HAL_LINKDMA(UartHandle, DMA_Handle_rx, hdma_rx);
  (...)
}
```

The *HAL_PPP_ProcessCpltCallback()* function is declared as weak function in the driver that means, the user can declare it again in the application code. The function in the driver should not be modified.

An example of use is illustrated below:

main.c file:

```
UART_HandleTypeDef UartHandle;
int main(void)
{
    /* Set User Paramaters */
    UartHandle.Init.BaudRate = 9600;
    UartHandle.Init.WordLength = UART_DATABITS_8;
    UartHandle.Init.StopBits = UART_STOPBITS_1;
    UartHandle.Init.Parity = UART_PARITY_NONE;
    UartHandle.Init.HwFlowCtl = UART_HWCONTROL_NONE;
    UartHandle.Init.Mode = UART_MODE_TX_RX; UartHandle.Init.Instance = USART1;
    HAL_UART_Init(&UartHandle);
    HAL_UART_Send_DMA(&UartHandle, TxBuffer, sizeof(TxBuffer));
    while (1);
}
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *phuart)
{
}
void HAL_UART_TxErrorCallback(UART_HandleTypeDef *phuart)
{
}
```

stm32f2xx_it.c file:

```
extern UART_HandleTypeDef UartHandle;
void DMAx_IRQHandler(void)
{
    HAL_DMA_IRQHandler(&UartHandle.DMA_Handle_tx);
}
```

HAL_USART_TxCpltCallback() and *HAL_USART_ErrorCallback()* should be linked in the *HAL_PPP_Process_DMA()* function to the DMA transfer complete callback and the DMA transfer Error callback by using the following statement:

```
HAL_PPP_Process_DMA (PPP_HandleTypeDef *hPPP, Params...)
{
    (...)
    hPPP->DMA_Handle->XferCpltCallback = HAL_UART_TxCpltCallback ;
    hPPP->DMA_Handle->XferErrorCallback = HAL_UART_ErrorCallback ;
    (...)
}
```

3.12.4 Timeout and error management

3.12.4.1 Timeout management

The timeout is often used for the APIs that operate in Polling mode. It defines the delay during which a blocking process should wait till an error is returned. An example is provided below:

```
HAL_StatusTypeDef HAL_DMA_PollForTransfer(DMA_HandleTypeDef *hdma, uint32_t CompleteLevel, uint32_t Timeout)
```

The timeout possible value are the following:

Table 15. Timeout values

Timeout value	Description
0	No poll : Immediate process check and exit
1 ... (HAL_MAX_DELAY -1) ⁽¹⁾	Timeout in ms
HAL_MAX_DELAY	Infinite poll till process is successful

1. HAL_MAX_DELAY is defined in the *stm32f2xx_hal_def.h* as 0xFFFFFFFF

However, in some cases, a fixed timeout is used for system peripherals or internal HAL driver processes. In these cases, the timeout has the same meaning and is used in the same way, except when it is defined locally in the drivers and cannot be modified or introduced as an argument in the user application.

Example of fixed timeout:

```
#define LOCAL_PROCESS_TIMEOUT 100
HAL_StatusTypeDef HAL_PPP_Process(PPP_HandleTypeDef)
{
  (...)
  timeout = HAL_GetTick() + LOCAL_PROCESS_TIMEOUT;
  (...)
  while(ProcessOngoing)
  {
    (...)
    if(HAL_GetTick() >= timeout)
    {
      /* Process unlocked */
      __HAL_UNLOCK(hppp);
      hppp->State= HAL_PPP_STATE_TIMEOUT;
      return HAL_PPP_STATE_TIMEOUT;
    }
  }
  (...)
}
```

The following example shows how to use the timeout inside the polling functions:

```
HAL_PPP_StateTypeDef HAL_PPP_Poll (PPP_HandleTypeDef *hppp, uint32_t Timeout)
{
  (...)
  timeout = HAL_GetTick() + Timeout;
  (...)
  while(ProcessOngoing)
  {
    (...)
    if(Timeout != HAL_MAX_DELAY)
    {
      if(HAL_GetTick() >= timeout)
      {
        /* Process unlocked */
        __HAL_UNLOCK(hppp);
        hppp->State= HAL_PPP_STATE_TIMEOUT;
        return hppp->State;
      }
    }
  }
  (...)
}
```

3.12.4.2 Error management

The HAL drivers implement a check on the following items:

- Valid parameters: for some process the used parameters should be valid and already defined, otherwise the system may crash or go into an undefined state. These critical parameters are checked before being used (see example below).

```
HAL_StatusTypeDef HAL_PPP_Process(PPP_HandleTypeDef* hppp, uint32_t *pdata, uint32 Size)
{
  if ((pData == NULL ) || (Size == 0))
  {
    return HAL_ERROR;
  }
}
```

- Valid handle: the PPP peripheral handle is the most important argument since it keeps the PPP driver vital parameters. It is always checked in the beginning of the *HAL_PPP_Init()* function.

```
HAL_StatusTypeDef HAL_PPP_Init(PPP_HandleTypeDef* hppp)
{
  if (hppp == NULL) //the handle should be already allocated
  {
    return HAL_ERROR;
  }
}
```

- Timeout error: the following statement is used when a timeout error occurs:

```
while (Process ongoing)
{
  timeout = HAL_GetTick() + Timeout; while (data processing is running)
  {
    if(timeout) { return HAL_TIMEOUT;
  }
}
```

When an error occurs during a peripheral process, *HAL_PPP_Process ()* returns with a *HAL_ERROR* status. The HAL PPP driver implements the *HAL_PPP_GetError ()* to allow retrieving the origin of the error.

```
HAL_PPP_ErrorTypeDef HAL_PPP_GetError (PPP_HandleTypeDef *hppp);
```

In all peripheral handles, a *HAL_PPP_ErrorTypeDef* is defined and used to store the last error code.

```
typedef struct
{
  PPP_TypeDef * Instance; /* PPP registers base address */
  PPP_InitTypeDef Init; /* PPP initialization parameters */
  HAL_LockTypeDef Lock; /* PPP locking object */
  __IO HAL_PPP_StateTypeDef State; /* PPP state */
  __IO HAL_PPP_ErrorTypeDef ErrorCode; /* PPP Error code */
  (...)
  /* PPP specific parameters */
}
PPP_HandleTypeDef;
```

The error state and the peripheral global state are always updated before returning an error:

```
PPP->State = HAL_PPP_READY; /* Set the peripheral ready */
PPP->ErrorCode = HAL_ERRORCODE ; /* Set the error code */
__HAL_UNLOCK(PPP) ; /* Unlock the PPP resources */
return HAL_ERROR; /*return with HAL error */
```

HAL_PPP_GetError () must be used in interrupt mode in the error callback:

```
void HAL_PPP_ProcessCpltCallback(PPP_HandleTypeDef *hspi)
{
  ErrorCode = HAL_PPP_GetError (hppp); /* retrieve error code */
}
```

3.12.4.3 Run-time checking

The HAL implements run-time failure detection by checking the input values of all HAL driver functions. The run-time checking is achieved by using an *assert_param* macro. This macro is used in all the HAL driver functions which have an input parameter. It allows verifying that the input value lies within the parameter allowed values.

To enable the run-time checking, use the *assert_param* macro, and leave the define ***USE_FULL_ASSERT*** uncommented in *stm32f2xx_hal_conf.h* file.


```
void HAL_UART_Init(UART_HandleTypeDef *huart)
{
  (...) /* Check the parameters */
  assert_param(IS_UART_INSTANCE(huart->Instance));
  assert_param(IS_UART_BAUDRATE(huart->Init.BaudRate));
  assert_param(IS_UART_WORD_LENGTH(huart->Init.WordLength));
  assert_param(IS_UART_STOPBITS(huart->Init.StopBits));
  assert_param(IS_UART_PARITY(huart->Init.Parity));
  assert_param(IS_UART_MODE(huart->Init.Mode));
  assert_param(IS_UART_HARDWARE_FLOW_CONTROL(huart->Init.HwFlowCtl));
  (...)
}
```

```
/** @defgroup UART_Word_Length *
 *{
 */
#define UART_WORDLENGTH_8B ((uint32_t)0x00000000)
#define UART_WORDLENGTH_9B ((uint32_t)USART_CR1_M)
#define IS_UART_WORD_LENGTH(LENGTH) (((LENGTH) == UART_WORDLENGTH_8B) || \
  \ ((LENGTH) == UART_WORDLENGTH_9B))
```

If the expression passed to the `assert_param` macro is false, the `assert_failed` function is called and returns the name of the source file and the source line number of the call that failed. If the expression is true, no value is returned.

The `assert_param` macro is implemented in `stm32f2xx_hal_conf.h`:

```
/* Exported macro -----*/
#ifdef USE_FULL_ASSERT
/**
 * @brief The assert_param macro is used for function's parameters check.
 * @param expr: If expr is false, it calls assert_failed function
 * which reports the name of the source file and the source
 * line number of the call that failed.
 * If expr is true, it returns no value.
 * @retval None */
#define assert_param(expr) ((expr)?(void)0:assert_failed((__FILE__, __LINE__))
/* Exported functions -----*/
void assert_failed(uint8_t* file, uint32_t line);
#else
#define assert_param(expr) ((void)0)
#endif /* USE_FULL_ASSERT */
```

The `assert_failed` function is implemented in the `main.c` file or in any other user C file:

```
#ifdef USE_FULL_ASSERT /**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None */
void assert_failed(uint8_t* file, uint32_t line)
{
  /* User can add his own implementation to report the file name and line number,
  ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* Infinite loop */
  while (1)
  {
  }
}
```

Note:

Because of the overhead run-time checking introduces, it is recommended to use it during application code development and debugging, and to remove it from the final application to improve code size and speed.

4 Overview of low-layer drivers

The low-layer (LL) drivers are designed to offer a fast light-weight expert-oriented layer which is closer to the hardware than the HAL. Contrary to the HAL, LL APIs are not provided for peripherals where optimized access is not a key feature, or those requiring heavy software configuration and/or complex upper-level stack (such as USB).

The LL drivers feature:

- A set of functions to initialize peripheral main features according to the parameters specified in data structures
- A set of functions used to fill initialization data structures with the reset values of each field
- Functions to perform peripheral de-initialization (peripheral registers restored to their default values)
- A set of inline functions for direct and atomic register access
- Full independence from HAL since LL drivers can be used either in standalone mode (without HAL drivers) or in mixed mode (with HAL drivers)
- Full coverage of the supported peripheral features.

The low-layer drivers provide hardware services based on the available features of the STM32 peripherals. These services reflect exactly the hardware capabilities and provide one-shot operations that must be called following the programming model described in the microcontroller line reference manual. As a result, the LL services do not implement any processing and do not require any additional memory resources to save their states, counter or data pointers: all the operations are performed by changing the associated peripheral registers content.

4.1 Low-layer files

The low-layer drivers are built around header/C files (one per each supported peripheral) plus five header files for some System and Cortex related features.

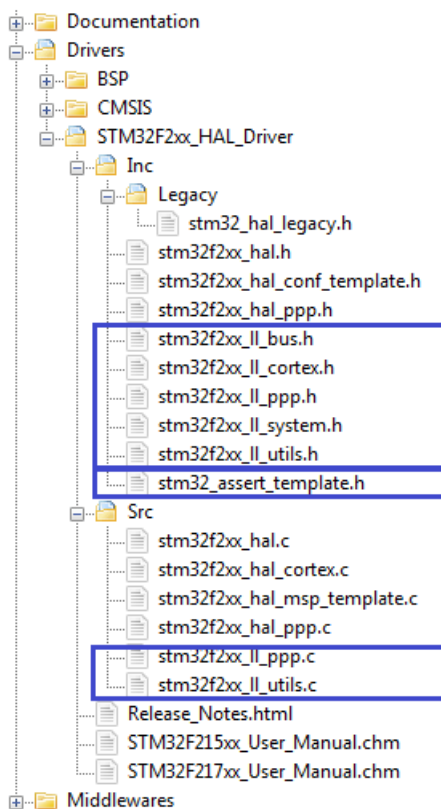
Table 16. LL driver files

File	Description
<i>stm32f2xx_ll_bus.h</i>	This is the h-source file for core bus control and peripheral clock activation and deactivation <i>Example: LL_AHB2_GRP1_EnableClock</i>
<i>stm32f2xx_ll_ppp.h/c</i>	stm32f2xx_ll_ppp.c provides peripheral initialization functions such as LL_PPP_Init(), LL_PPP_StructInit(), LL_PPP_DeInit(). All the other APIs are defined within stm32f2xx_ll_ppp.h file. The low-layer PPP driver is a standalone module. To use it, the application must include it in the stm32f2xx_ll_ppp.h file.
<i>stm32f2xx_ll_cortex.h</i>	Cortex-M related register operation APIs including the SysTick, Low power (LL_SYSTICK_XXXX, LL_LPM_XXXX "Low Power Mode" ...)
<i>stm32f2xx_ll_utils.h/c</i>	This file covers the generic APIs: • Read of device unique ID and electronic signature • Timebase and delay management • System clock configuration.
<i>stm32f2xx_ll_system.h</i>	System related operations. <i>Example: LL_SYSCFG_XXX, LL_DBGMCU_XXX and LL_FLASH_XXX and LL_VREBUF_XXX</i>
<i>stm32_assert_template.h</i>	Template file allowing to define the assert_param macro, that is used when run-time checking is enabled. This file is required only when the LL drivers are used in standalone mode (without calling the HAL APIs). It should be copied to the application folder and renamed to stm32_assert.h.

Note: There is no configuration file for the LL drivers.

The low-layer files are located in the same HAL driver folder.

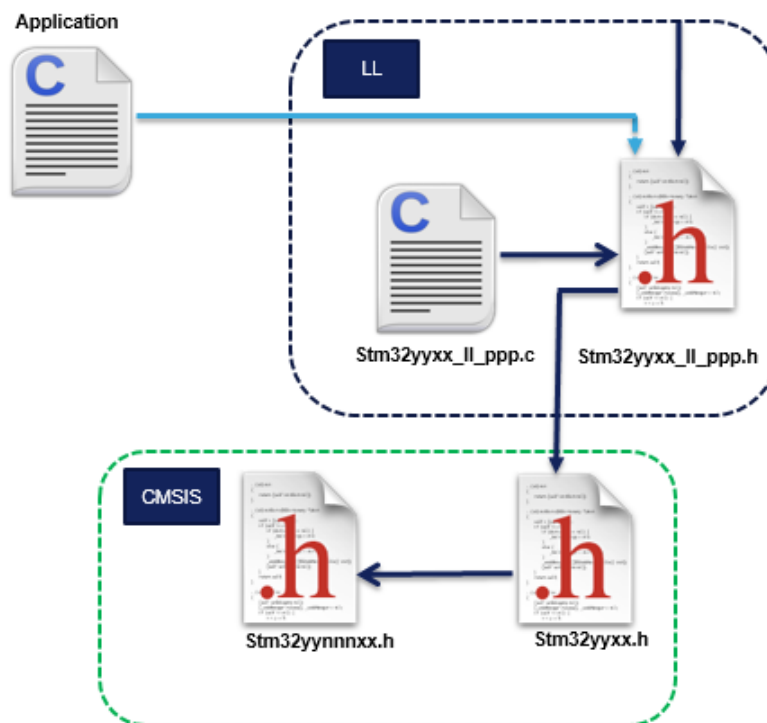
Figure 8. Low-layer driver folders



In general, low-layer drivers include only the STM32 CMSIS device file.

```
#include "stm32yxxx.h"
```

Figure 9. Low-layer driver CMSIS files



Application files have to include only the used low-layer driver header files.

4.2 Overview of low-layer APIs and naming rules

4.2.1 Peripheral initialization functions

The LL drivers offer three sets of initialization functions. They are defined in stm32f2xx_ll_ppp.c file:

- Functions to initialize peripheral main features according to the parameters specified in data structures
- A set of functions used to fill initialization data structures with the reset values of each field
- Function for peripheral de-initialization (peripheral registers restored to their default values)

The definition of these LL initialization functions and associated resources (structure, literals and prototypes) is conditioned by a compilation switch: *USE_FULL_LL_DRIVER*. To use these functions, this switch must be added in the toolchain compiler preprocessor or to any generic header file which is processed before the LL drivers.

The below table shows the list of the common functions provided for all the supported peripherals:

Table 17. Common peripheral initialization functions

Functions	Return Type	Parameters	Description
LL_PPP_Init	ErrorStatus	• <i>PPP_TypeDef* PPPx</i> • <i>LL_PPP_InitTypeDef* PPP_InitStruct</i>	Initializes the peripheral main features according to the parameters specified in PPP_InitStruct. Example: LL_USART_Init(USART_TypeDef *USARTx, LL_USART_InitTypeDef *USART_InitStruct)
LL_PPP_StructInit	void	• <i>LL_PPP_InitTypeDef* PPP_InitStruct</i>	Fills each PPP_InitStruct member with its default value. Example. LL_USART_StructInit(LL_USART_InitTypeDef *USART_InitStruct)

Functions	Return Type	Parameters	Description
LL_PPP_DeInit	ErrorStatus	PPP_TypeDef* PPPx	De-initializes the peripheral registers, that is restore them to their default reset values. Example. LL_USART_DeInit(USART_TypeDef *USARTx)

Additional functions are available for some peripherals (refer to [Table 18. Optional peripheral initialization functions](#)).

Table 18. Optional peripheral initialization functions

Functions	Return Type	Parameters	Examples
LL_PPP{CATEGORY}_Init	ErrorStatus	- PPP_TypeDef* PPPx - LL_PPP{CATEGORY}_InitTypeDef* PPP{CATEGORY}_InitStruct	Initializes peripheral features according to the parameters specified in PPP_InitStruct. Example: LL_ADC_INJ_Init(ADC_TypeDef *ADCx, LL_ADC_INJ_InitTypeDef *ADC_INJ_InitStruct) LL_RTC_TIME_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format, LL_RTC_TimeTypeDef *RTC_TimeStruct) LL_RTC_DATE_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format, LL_RTC_DateTypeDef *RTC_DateStruct) LL_TIM_IC_Init(TIM_TypeDef* TIMx, uint32_t Channel, LL_TIM_IC_InitTypeDef* TIM_IC_InitStruct) LL_TIM_ENCODER_Init(TIM_TypeDef* TIMx, LL_TIM_ENCODER_InitTypeDef* TIM_EncoderInitStruct)
LL_PPP{CATEGORY}_StructInit	void	LL_PPP{CATEGORY}_InitTypeDef* PPP{CATEGORY}_InitStruct	Fills each PPP{CATEGORY}_InitStruct member with its default value. Example: LL_ADC_INJ_StructInit(LL_ADC_INJ_InitTypeDef *ADC_INJ_InitStruct)
LL_PPP_CommonInit	ErrorStatus	- PPP_TypeDef* PPPx - LL_PPP_CommonInitTypeDef* PPP_CommonInitStruct	Initializes the common features shared between different instances of the same peripheral. Example: LL_ADC_CommonInit(ADC_Common_TypeDef *ADCxy_COMMON, LL_ADC_CommonInitTypeDef *ADC_CommonInitStruct)
LL_PPP_CommonStructInit	void	LL_PPP_CommonInitTypeDef* PPP_CommonInitStruct	Fills each PPP_CommonInitStruct member with its default value Example: LL_ADC_CommonStructInit(LL_ADC_CommonInitTypeDef *ADC_CommonInitStruct)
LL_PPP_ClockInit	ErrorStatus	- PPP_TypeDef* PPPx - LL_PPP_ClockInitTypeDef* PPP_ClockInitStruct	Initializes the peripheral clock configuration in synchronous mode. Example: LL_USART_ClockInit(USART_TypeDef *USARTx, LL_USART_ClockInitTypeDef *USART_ClockInitStruct)
LL_PPP_ClockStructInit	void	LL_PPP_ClockInitTypeDef* PPP_ClockInitStruct	Fills each PPP_ClockInitStruct member with its default value Example: LL_USART_ClockStructInit(LL_USART_ClockInitTypeDef *USART_ClockInitStruct)

4.2.1.1 Run-time checking

Like HAL drivers, LL initialization functions implement run-time failure detection by checking the input values of all LL driver functions. For more details please refer to [Section 3.12.4.3 Run-time checking](#).

When using the LL drivers in standalone mode (without calling HAL functions), the following actions are required to use run-time checking:

1. Copy `stm32_assert_template.h` to the application folder and rename it to `stm32_assert.h`. This file defines the `assert_param` macro which is used when run-time checking is enabled.
2. Include `stm32_assert.h` file within the application main header file.
3. Add the `USE_FULL_ASSERT` compilation switch in the toolchain compiler preprocessor or in any generic header file which is processed before the `stm32_assert.h` driver.

Note: Run-time checking is not available for LL inline functions.

4.2.2 Peripheral register-level configuration functions

On top of the peripheral initialization functions, the LL drivers offer a set of inline functions for direct atomic register access. Their format is as follows:

```
__STATIC_INLINE return_type LL_PPP_Function (PPPx_TypeDef *PPPx, args)
```

The “Function” naming is defined depending to the action category:

- **Specific Interrupt, DMA request and status flags management:** Set/Get/Clear/Enable/Disable flags on interrupt and status registers

Table 19. Specific Interrupt, DMA request and status flags management

Name	Examples
<code>LL_PPP_{CATEGORY}_ActionItem_BITNAME</code> <code>LL_PPP{CATEGORY}_IsItem_BITNAME_Action</code>	• <code>LL_RCC_IsActiveFlag_LSIRDY</code> • <code>LL_RCC_IsActiveFlag_FWRST()</code> • <code>LL_ADC_ClearFlag_EOC(ADC1)</code> • <code>LL_DMA_ClearFlag_TCx(DMA_TypeDef* DMAx)</code>

Table 20. Available function formats

Item	Action	Format
Flag	Get	<code>LL_PPP_IsActiveFlag_BITNAME</code>
	Clear	<code>LL_PPP_ClearFlag_BITNAME</code>
Interrupts	Enable	<code>LL_PPP_EnableIT_BITNAME</code>
	Disable	<code>LL_PPP_DisableIT_BITNAME</code>
	Get	<code>LL_PPP_IsEnabledIT_BITNAME</code>
DMA	Enable	<code>LL_PPP_EnableDMAReq_BITNAME</code>
	Disable	<code>LL_PPP_DisableDMAReq_BITNAME</code>
	Get	<code>LL_PPP_IsEnabledDMAReq_BITNAME</code>

Note: BITNAME refers to the peripheral register bit name as described in the product line reference manual.

- **Peripheral clock activation/deactivation management:** Enable/Disable/Reset a peripheral clock

Table 21. Peripheral clock activation/deactivation management

Name	Examples
<code>LL_BUS_GRPx_ActionClock{Mode}</code>	• <code>LL_AHB2_GRP1_EnableClock (LL_AHB2_GRP1_PERIPH_GPIOA LL_AHB2_GRP1_PERIPH_GPIOB)</code>

Name	Examples
	<code>LL_APB1_GRP1_EnableClockSleep (LL_APB1_GRP1_PERIPH_DAC1)</code>

Note: 'x' corresponds to the group index and refers to the index of the modified register on a given bus. 'bus' corresponds to the bus name.

- Peripheral activation/deactivation management** : Enable/disable a peripheral or activate/deactivate specific peripheral features

Table 22. Peripheral activation/deactivation management

Name	Examples
<code>LL_PPP[_CATEGORY]_Action{Item}</code> <code>LL_PPP[_CATEGORY]_IsItemAction</code>	<code>LL_ADC_Enable ()</code> <code>LL_ADC_StartCalibration();</code> <code>LL_ADC_IsCalibrationOnGoing;</code> <code>LL_RCC_HSI_Enable ()</code> <code>LL_RCC_HSI_IsReady()</code>

- Peripheral configuration management** : Set/get a peripheral configuration settings

Table 23. Peripheral configuration management

Name	Examples
<code>LL_PPP[_CATEGORY]_Set{ or Get}ConfigItem</code>	<code>LL_USART_SetBaudRate (USART2, Clock, LL_USART_BAUDRATE_9600)</code>

- Peripheral register management** : Write/read the content of a register/retrun DMA relative register address

Table 24. Peripheral register management

Name
<code>LL_PPP_WriteReg(__INSTANCE__, __REG__, __VALUE__)</code>
<code>LL_PPP_ReadReg(__INSTANCE__, __REG__)</code>
<code>LL_PPP_DMA_GetRegAddr (PPP_TypeDef *PPPx, {Sub Instance if any ex: Channel} , {uint32_t Propriety})</code>

Note: The Propriety is a variable used to identify the DMA transfer direction or the data register type.

5 Cohabiting of HAL and LL

The low-layer APIs are designed to be used in standalone mode or combined with the HAL. They cannot be automatically used with the HAL for the same peripheral instance. If you use the LL APIs for a specific instance, you can still use the HAL APIs for other instances. Be careful that the low-layer APIs might overwrite some registers which content is mirrored in the HAL handles.

5.1 Low-layer driver used in Standalone mode

The low-layer APIs can be used without calling the HAL driver services. This is done by simply including `stm32f2xx_ll_ppp.h` in the application files. The LL APIs for a given peripheral are called by executing the same sequence as the one recommended by the programming model in the corresponding product line reference manual. In this case the HAL drivers associated to the used peripheral can be removed from the workspace. However the [STM32CubeF2](#) framework should be used in the same way as in the HAL drivers case which means that System file, startup file and CMSIS should always be used.

Note: *When the BSP drivers are included, the used HAL drivers associated with the BSP functions drivers should be included in the workspace, even if they are not used by the application layer.*

5.2 Mixed use of low-layer APIs and HAL drivers

In this case the low-layer APIs are used in conjunction with the HAL drivers to achieve direct and register level based operations.

Mixed use is allowed, however some consideration should be taken into account:

- It is recommended to avoid using simultaneously the HAL APIs and the combination of low-layer APIs for a given peripheral instance. If this is the case, one or more private fields in the HAL PPP handle structure should be updated accordingly.
- For operations and processes that do not alter the handle fields including the initialization structure, the HAL driver APIs and the low-layer services can be used together for the same peripheral instance.
- The low-layer drivers can be used without any restriction with all the HAL drivers that are not based on handle objects (RCC, common HAL, flash and GPIO).

Several examples showing how to use HAL and LL in the same application are provided within `stm32f2` firmware package (refer to `Examples_MIX` projects).

- Note:**
1. *When the HAL `Init/DelInit` APIs are not used and are replaced by the low-layer macros, the `InitMsp()` functions are not called and the MSP initialization should be done in the user application.*
 2. *When process APIs are not used and the corresponding function is performed through the low-layer APIs, the callbacks are not called and post processing or error management should be done by the user application.*
 3. *When the LL APIs is used for process operations, the IRQ handler HAL APIs cannot be called and the IRQ should be implemented by the user application. Each LL driver implements the macros needed to read and clear the associated interrupt flags.*

6 HAL System Driver

6.1 HAL Firmware driver API description

The following section lists the various functions of the HAL library.

6.1.1 How to use this driver

The common HAL driver contains a set of generic and common APIs that can be used by the PPP peripheral drivers and the user to start using the HAL.

The HAL contains two APIs' categories:

- Common HAL APIs
- Services HAL APIs

6.1.2 Initialization and de-initialization functions

This section provides functions allowing to:

- Initializes the Flash interface the NVIC allocation and initial clock configuration. It initializes the systick also when timeout is needed and the backup domain when enabled.
- de-Initializes common part of the HAL
- Configure The time base source to have 1ms time base with a dedicated Tick interrupt priority.
 - Systick timer is used by default as source of time base, but user can eventually implement his proper time base source (a general purpose timer for example or other time source), keeping in mind that Time base duration should be kept 1ms since PPP_TIMEOUT_VALUES are defined and handled in milliseconds basis.
 - Time base configuration function (HAL_InitTick ()) is called automatically at the beginning of the program after reset by HAL_Init() or at any time when clock is configured, by HAL_RCC_ClockConfig().
 - Source of time base is configured to generate interrupts at regular time intervals. Care must be taken if HAL_Delay() is called from a peripheral ISR process, the Tick interrupt line must have higher priority (numerically lower) than the peripheral interrupt. Otherwise the caller ISR process will be blocked.
 - functions affecting time base configurations are declared as __weak to make override possible in case of other implementations in user file.

This section contains the following APIs:

- [*HAL_Init*](#)
- [*HAL_DeInit*](#)
- [*HAL_MspInit*](#)
- [*HAL_MspDeInit*](#)
- [*HAL_InitTick*](#)

6.1.3 HAL Control functions

This section provides functions allowing to:

- Provide a tick value in millisecond
- Provide a blocking delay in millisecond
- Suspend the time base source interrupt
- Resume the time base source interrupt
- Get the HAL API driver version
- Get the device identifier
- Get the device revision identifier
- Enable/Disable Debug module during SLEEP mode
- Enable/Disable Debug module during STOP mode
- Enable/Disable Debug module during STANDBY mode

This section contains the following APIs:

- *HAL_IncTick*
- *HAL_GetTick*
- *HAL_GetTickPrio*
- *HAL_SetTickFreq*
- *HAL_GetTickFreq*
- *HAL_Delay*
- *HAL_SuspendTick*
- *HAL_ResumeTick*
- *HAL_GetHalVersion*
- *HAL_GetREVID*
- *HAL_GetDEVID*
- *HAL_DBGMCU_EnableDBGSleepMode*
- *HAL_DBGMCU_DisableDBGSleepMode*
- *HAL_DBGMCU_EnableDBGStopMode*
- *HAL_DBGMCU_DisableDBGStopMode*
- *HAL_DBGMCU_EnableDBGStandbyMode*
- *HAL_DBGMCU_DisableDBGStandbyMode*
- *HAL_EnableCompensationCell*
- *HAL_DisableCompensationCell*
- *HAL_GetUIDw0*
- *HAL_GetUIDw1*
- *HAL_GetUIDw2*

6.1.4 Detailed description of functions

HAL_Init

Function name

HAL_StatusTypeDef HAL_Init (void)

Function description

This function is used to initialize the HAL Library; it must be the first instruction to be executed in the main program (before to call any other HAL function), it performs the following: Configure the Flash prefetch, instruction and Data caches.

Return values

- **HAL:** status

Notes

- SysTick is used as time base for the HAL_Delay() function, the application need to ensure that the SysTick time base is always set to 1 millisecond to have correct HAL operation.

HAL_DeInit

Function name

HAL_StatusTypeDef HAL_DeInit (void)

Function description

This function de-Initializes common part of the HAL and stops the systick.

Return values

- **HAL:** status

HAL_Msplnit

Function name

void HAL_Msplnit (void)

Function description

Initializes the MSP.

Return values

- **None:**

HAL_MspDeInit

Function name

void HAL_MspDeInit (void)

Function description

DeInitializes the MSP.

Return values

- **None:**

HAL_InitTick

Function name

HAL_StatusTypeDef HAL_InitTick (uint32_t TickPriority)

Function description

This function configures the source of the time base.

Parameters

- **TickPriority:** Tick interrupt priority.

Return values

- **HAL:** status

Notes

- This function is called automatically at the beginning of program after reset by HAL_Init() or at any time when clock is reconfigured by HAL_RCC_ClockConfig().
- In the default implementation, SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Care must be taken if HAL_Delay() is called from a peripheral ISR process, The the SysTick interrupt must have higher priority (numerically lower) than the peripheral interrupt. Otherwise the caller ISR process will be blocked. The function is declared as __weak to be overwritten in case of other implementation in user file.

HAL_IncTick

Function name

void HAL_IncTick (void)

Function description

This function is called to increment a global variable "uwTick" used as application time base.

Return values

- **None:**

Notes

- In the default implementation, this variable is incremented each 1ms in SysTick ISR.
- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_Delay

Function name

```
void HAL_Delay (__IO uint32_t Delay)
```

Function description

This function provides minimum delay (in milliseconds) based on variable incremented.

Parameters

- **Delay:** specifies the delay time length, in milliseconds.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals where uwTick is incremented.
- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_GetTick

Function name

```
uint32_t HAL_GetTick (void )
```

Function description

Provides a tick value in millisecond.

Return values

- **tick:** value

Notes

- This function is declared as `__weak` to be overwritten in case of other implementations in user file.

HAL_GetTickPrio

Function name

```
uint32_t HAL_GetTickPrio (void )
```

Function description

This function returns a tick priority.

Return values

- **tick:** priority

HAL_SetTickFreq

Function name

```
HAL_StatusTypeDef HAL_SetTickFreq (HAL_TickFreqTypeDef Freq)
```

Function description

Set new tick Freq.

Return values

- **status:**

HAL_GetTickFreq

Function name

HAL_TickFreqTypeDef HAL_GetTickFreq (void)

Function description

return tick frequency.

Return values

- **tick:** period in Hz

HAL_SuspendTick

Function name

void HAL_SuspendTick (void)

Function description

Suspend Tick increment.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Once HAL_SuspendTick() is called, the SysTick interrupt will be disabled and so Tick increment is suspended.
- This function is declared as __weak to be overwritten in case of other implementations in user file.

HAL_ResumeTick

Function name

void HAL_ResumeTick (void)

Function description

Resume Tick increment.

Return values

- **None:**

Notes

- In the default implementation , SysTick timer is the source of time base. It is used to generate interrupts at regular time intervals. Once HAL_ResumeTick() is called, the SysTick interrupt will be enabled and so Tick increment is resumed.
- This function is declared as __weak to be overwritten in case of other implementations in user file.

HAL_GetHalVersion

Function name

uint32_t HAL_GetHalVersion (void)

Function description

Returns the HAL revision.

Return values

- **version**: : 0xXYZR (8bits for each decimal, R for RC)

HAL_GetREVID

Function name

uint32_t HAL_GetREVID (void)

Function description

Returns the device revision identifier.

Return values

- **Device**: revision identifier

HAL_GetDEVID

Function name

uint32_t HAL_GetDEVID (void)

Function description

Returns the device identifier.

Return values

- **Device**: identifier

HAL_DBGMCU_EnableDBGSleepMode

Function name

void HAL_DBGMCU_EnableDBGSleepMode (void)

Function description

Enable the Debug Module during SLEEP mode.

Return values

- **None**:

HAL_DBGMCU_DisableDBGSleepMode

Function name

void HAL_DBGMCU_DisableDBGSleepMode (void)

Function description

Disable the Debug Module during SLEEP mode.

Return values

- **None**:

HAL_DBGMCU_EnableDBGStopMode

Function name

void HAL_DBGMCU_EnableDBGStopMode (void)

Function description

Enable the Debug Module during STOP mode.

Return values

- **None**:

HAL_DBGMCU_DisableDBGStopMode

Function name

void HAL_DBGMCU_DisableDBGStopMode (void)

Function description

Disable the Debug Module during STOP mode.

Return values

- **None:**

HAL_DBGMCU_EnableDBGStandbyMode

Function name

void HAL_DBGMCU_EnableDBGStandbyMode (void)

Function description

Enable the Debug Module during STANDBY mode.

Return values

- **None:**

HAL_DBGMCU_DisableDBGStandbyMode

Function name

void HAL_DBGMCU_DisableDBGStandbyMode (void)

Function description

Disable the Debug Module during STANDBY mode.

Return values

- **None:**

HAL_EnableCompensationCell

Function name

void HAL_EnableCompensationCell (void)

Function description

Enables the I/O Compensation Cell.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V.

HAL_DisableCompensationCell

Function name

void HAL_DisableCompensationCell (void)

Function description

Power-down the I/O Compensation Cell.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V.

HAL_GetUIDw0

Function name

`uint32_t HAL_GetUIDw0 (void )`

Function description

Returns first word of the unique device identifier (UID based on 96 bits)

Return values

- **Device:** identifier

HAL_GetUIDw1

Function name

`uint32_t HAL_GetUIDw1 (void )`

Function description

Returns second word of the unique device identifier (UID based on 96 bits)

Return values

- **Device:** identifier

HAL_GetUIDw2

Function name

`uint32_t HAL_GetUIDw2 (void )`

Function description

Returns third word of the unique device identifier (UID based on 96 bits)

Return values

- **Device:** identifier

6.2 HAL Firmware driver defines

The following section lists the various define and macros of the module.

6.2.1

HAL

HAL

HAL Exported Macros

`__HAL_DBGMCU_FREEZE_TIM2`

`__HAL_DBGMCU_FREEZE_TIM3`

`__HAL_DBGMCU_FREEZE_TIM4`

`__HAL_DBGMCU_FREEZE_TIM5`

`__HAL_DBGMCU_FREEZE_TIM6`

__HAL_DBGMCU_FREEZE_TIM7
__HAL_DBGMCU_FREEZE_TIM12
__HAL_DBGMCU_FREEZE_TIM13
__HAL_DBGMCU_FREEZE_TIM14
__HAL_DBGMCU_FREEZE_RTC
__HAL_DBGMCU_FREEZE_WWDG
__HAL_DBGMCU_FREEZE_IWDG
__HAL_DBGMCU_FREEZE_I2C1_TIMEOUT
__HAL_DBGMCU_FREEZE_I2C2_TIMEOUT
__HAL_DBGMCU_FREEZE_I2C3_TIMEOUT
__HAL_DBGMCU_FREEZE_CAN1
__HAL_DBGMCU_FREEZE_CAN2
__HAL_DBGMCU_FREEZE_TIM1
__HAL_DBGMCU_FREEZE_TIM8
__HAL_DBGMCU_FREEZE_TIM9
__HAL_DBGMCU_FREEZE_TIM10
__HAL_DBGMCU_FREEZE_TIM11
__HAL_DBGMCU_UNFREEZE_TIM2
__HAL_DBGMCU_UNFREEZE_TIM3
__HAL_DBGMCU_UNFREEZE_TIM4
__HAL_DBGMCU_UNFREEZE_TIM5
__HAL_DBGMCU_UNFREEZE_TIM6
__HAL_DBGMCU_UNFREEZE_TIM7
__HAL_DBGMCU_UNFREEZE_TIM12
__HAL_DBGMCU_UNFREEZE_TIM13
__HAL_DBGMCU_UNFREEZE_TIM14
__HAL_DBGMCU_UNFREEZE_RTC
__HAL_DBGMCU_UNFREEZE_WWDG

```
__HAL_DBGMCU_UNFREEZE_IWDG
__HAL_DBGMCU_UNFREEZE_I2C1_TIMEOUT
__HAL_DBGMCU_UNFREEZE_I2C2_TIMEOUT
__HAL_DBGMCU_UNFREEZE_I2C3_TIMEOUT
__HAL_DBGMCU_UNFREEZE_CAN1
__HAL_DBGMCU_UNFREEZE_CAN2
__HAL_DBGMCU_UNFREEZE_TIM1
__HAL_DBGMCU_UNFREEZE_TIM8
__HAL_DBGMCU_UNFREEZE_TIM9
__HAL_DBGMCU_UNFREEZE_TIM10
__HAL_DBGMCU_UNFREEZE_TIM11
__HAL_SYSCFG_REMAPMEMORY_FLASH
__HAL_SYSCFG_REMAPMEMORY_SYSTEMFLASH
__HAL_SYSCFG_REMAPMEMORY_SRAM
__HAL_SYSCFG_REMAPMEMORY_FSMC
```

7 HAL ADC Generic Driver

7.1 ADC Firmware driver registers structures

7.1.1 ADC_InitTypeDef

ADC_InitTypeDef is defined in the `stm32f2xx_hal_adc.h`

Data Fields

- **`uint32_t ClockPrescaler`**
- **`uint32_t Resolution`**
- **`uint32_t DataAlign`**
- **`uint32_t ScanConvMode`**
- **`uint32_t EOCSelection`**
- **`uint32_t ContinuousConvMode`**
- **`uint32_t NbrOfConversion`**
- **`uint32_t DiscontinuousConvMode`**
- **`uint32_t NbrOfDiscConversion`**
- **`uint32_t ExternalTrigConv`**
- **`uint32_t ExternalTrigConvEdge`**
- **`uint32_t DMAContinuousRequests`**

Field Documentation

- **`uint32_t ADC_InitTypeDef::ClockPrescaler`**
 Select ADC clock prescaler. The clock is common for all the ADCs. This parameter can be a value of [ADC_ClockPrescaler](#)
- **`uint32_t ADC_InitTypeDef::Resolution`**
 Configures the ADC resolution. This parameter can be a value of [ADC_Resolution](#)
- **`uint32_t ADC_InitTypeDef::DataAlign`**
 Specifies ADC data alignment to right (MSB on register bit 11 and LSB on register bit 0) (default setting) or to left (if regular group: MSB on register bit 15 and LSB on register bit 4, if injected group (MSB kept as signed value due to potential negative value after offset application): MSB on register bit 14 and LSB on register bit 3). This parameter can be a value of [ADC_Data_align](#)
- **`uint32_t ADC_InitTypeDef::ScanConvMode`**
 Configures the sequencer of regular and injected groups. This parameter can be associated to parameter 'DiscontinuousConvMode' to have main sequence subdivided in successive parts. If disabled: Conversion is performed in single mode (one channel converted, the one defined in rank 1). Parameters 'NbrOfConversion' and 'InjectedNbrOfConversion' are discarded (equivalent to set to 1). If enabled: Conversions are performed in sequence mode (multiple ranks defined by 'NbrOfConversion'/'InjectedNbrOfConversion' and each channel rank). Scan direction is upward: from rank1 to rank 'n'. This parameter can be set to ENABLE or DISABLE
- **`uint32_t ADC_InitTypeDef::EOCSelection`**
 Specifies what EOC (End Of Conversion) flag is used for conversion by polling and interruption: end of conversion of each rank or complete sequence. This parameter can be a value of [ADC_EOCSelection](#).
 Note: For injected group, end of conversion (flag&IT) is raised only at the end of the sequence. Therefore, if end of conversion is set to end of each conversion, injected group should not be used with interruption (HAL_ADCEX_InjectedStart_IT) or polling (HAL_ADCEX_InjectedStart and HAL_ADCEX_InjectedPollForConversion). By the way, polling is still possible since driver will use an estimated timing for end of injected conversion. Note: If overrun feature is intended to be used, use ADC in mode 'interruption' (function **HAL_ADC_Start_IT()**) with parameter EOCSelection set to end of each conversion or in mode 'transfer by DMA' (function **HAL_ADC_Start_DMA()**). If overrun feature is intended to be bypassed, use ADC in mode 'polling' or 'interruption' with parameter EOCSelection must be set to end of sequence

- `uint32_t ADC_InitTypeDef::ContinuousConvMode`**
Specifies whether the conversion is performed in single mode (one conversion) or continuous mode for regular group, after the selected trigger occurred (software start or external trigger). This parameter can be set to ENABLE or DISABLE.
- `uint32_t ADC_InitTypeDef::NbrOfConversion`**
Specifies the number of ranks that will be converted within the regular group sequencer. To use regular group sequencer and convert several ranks, parameter 'ScanConvMode' must be enabled. This parameter must be a number between Min_Data = 1 and Max_Data = 16.
- `uint32_t ADC_InitTypeDef::DiscontinuousConvMode`**
Specifies whether the conversions sequence of regular group is performed in Complete-sequence/Discontinuous-sequence (main sequence subdivided in successive parts). Discontinuous mode is used only if sequencer is enabled (parameter 'ScanConvMode'). If sequencer is disabled, this parameter is discarded. Discontinuous mode can be enabled only if continuous mode is disabled. If continuous mode is enabled, this parameter setting is discarded. This parameter can be set to ENABLE or DISABLE.
- `uint32_t ADC_InitTypeDef::NbrOfDiscConversion`**
Specifies the number of discontinuous conversions in which the main sequence of regular group (parameter NbrOfConversion) will be subdivided. If parameter 'DiscontinuousConvMode' is disabled, this parameter is discarded. This parameter must be a number between Min_Data = 1 and Max_Data = 8.
- `uint32_t ADC_InitTypeDef::ExternalTrigConv`**
Selects the external event used to trigger the conversion start of regular group. If set to ADC_SOFTWARE_START, external triggers are disabled. If set to external trigger source, triggering is on event rising edge by default. This parameter can be a value of [ADC_External_trigger_Source_Regular](#)
- `uint32_t ADC_InitTypeDef::ExternalTrigConvEdge`**
Selects the external trigger edge of regular group. If trigger is set to ADC_SOFTWARE_START, this parameter is discarded. This parameter can be a value of [ADC_External_trigger_edge_Regular](#)
- `uint32_t ADC_InitTypeDef::DMAContinuousRequests`**
Specifies whether the DMA requests are performed in one shot mode (DMA transfer stop when number of conversions is reached) or in Continuous mode (DMA transfer unlimited, whatever number of conversions). Note: In continuous mode, DMA must be configured in circular mode. Otherwise an overrun will be triggered when DMA buffer maximum pointer is reached. Note: This parameter must be modified when no conversion is on going on both regular and injected groups (ADC disabled, or ADC enabled without continuous mode or external trigger that could launch a conversion). This parameter can be set to ENABLE or DISABLE.

7.1.2

ADC_ChannelConfTypeDef

ADC_ChannelConfTypeDef is defined in the `stm32f2xx_hal_adc.h`

Data Fields

- `uint32_t Channel`**
- `uint32_t Rank`**
- `uint32_t SamplingTime`**
- `uint32_t Offset`**

Field Documentation

- `uint32_t ADC_ChannelConfTypeDef::Channel`**
Specifies the channel to configure into ADC regular group. This parameter can be a value of [ADC_channels](#)
- `uint32_t ADC_ChannelConfTypeDef::Rank`**
Specifies the rank in the regular group sequencer. This parameter must be a number between Min_Data = 1 and Max_Data = 16

- **`uint32_t ADC_ChannelConfTypeDef::SamplingTime`**
Sampling time value to be set for the selected channel. Unit: ADC clock cycles Conversion time is the addition of sampling time and processing time (12 ADC clock cycles at ADC resolution 12 bits, 11 cycles at 10 bits, 9 cycles at 8 bits, 7 cycles at 6 bits). This parameter can be a value of [ADC_sampling_times](#)
Caution: This parameter updates the parameter property of the channel, that can be used into regular and/or injected groups. If this same channel has been previously configured in the other group (regular/injected), it will be updated to last setting. Note: In case of usage of internal measurement channels (VrefInt/Vbat/TempSensor), sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting) Refer to device datasheet for timings values, parameters TS_vrefint, TS_temp (values rough order: 4us min).
- **`uint32_t ADC_ChannelConfTypeDef::Offset`**
Reserved for future use, can be set to 0

7.1.3

ADC_AnalogWDGConfTypeDef

ADC_AnalogWDGConfTypeDef is defined in the `stm32f2xx_hal_adc.h`

Data Fields

- **`uint32_t WatchdogMode`**
- **`uint32_t HighThreshold`**
- **`uint32_t LowThreshold`**
- **`uint32_t Channel`**
- **`uint32_t ITMode`**
- **`uint32_t WatchdogNumber`**

Field Documentation

- **`uint32_t ADC_AnalogWDGConfTypeDef::WatchdogMode`**
Configures the ADC analog watchdog mode. This parameter can be a value of [ADC_analog_watchdog_selection](#)
- **`uint32_t ADC_AnalogWDGConfTypeDef::HighThreshold`**
Configures the ADC analog watchdog High threshold value. This parameter must be a 12-bit value.
- **`uint32_t ADC_AnalogWDGConfTypeDef::LowThreshold`**
Configures the ADC analog watchdog High threshold value. This parameter must be a 12-bit value.
- **`uint32_t ADC_AnalogWDGConfTypeDef::Channel`**
Configures ADC channel for the analog watchdog. This parameter has an effect only if watchdog mode is configured on single channel This parameter can be a value of [ADC_channels](#)
- **`uint32_t ADC_AnalogWDGConfTypeDef::ITMode`**
Specifies whether the analog watchdog is configured is interrupt mode or in polling mode. This parameter can be set to ENABLE or DISABLE
- **`uint32_t ADC_AnalogWDGConfTypeDef::WatchdogNumber`**
Reserved for future use, can be set to 0

7.1.4

ADC_HandleTypeDef

ADC_HandleTypeDef is defined in the `stm32f2xx_hal_adc.h`

Data Fields

- **`ADC_TypeDef * Instance`**
- **`ADC_InitTypeDef Init`**
- **`__IO uint32_t NbrOfCurrentConversionRank`**
- **`DMA_HandleTypeDef * DMA_Handle`**
- **`HAL_LockTypeDef Lock`**
- **`__IO uint32_t State`**
- **`__IO uint32_t ErrorCode`**

Field Documentation

- **`ADC_TypeDef* ADC_HandleTypeDef::Instance`**
Register base address

- ***ADC_InitTypeDef ADC_HandleTypeDef::Init***
ADC required parameters
- ***__IO uint32_t ADC_HandleTypeDef::NbrOfCurrentConversionRank***
ADC number of current conversion rank
- ***DMA_HandleTypeDef* ADC_HandleTypeDef::DMA_Handle***
Pointer DMA Handler
- ***HAL_LockTypeDef ADC_HandleTypeDef::Lock***
ADC locking object
- ***__IO uint32_t ADC_HandleTypeDef::State***
ADC communication state
- ***__IO uint32_t ADC_HandleTypeDef::ErrorCode***
ADC Error code

7.2 ADC Firmware driver API description

The following section lists the various functions of the ADC library.

7.2.1 ADC Peripheral features

1. 12-bit, 10-bit, 8-bit or 6-bit configurable resolution.
2. Interrupt generation at the end of conversion, end of injected conversion, and in case of analog watchdog or overrun events
3. Single and continuous conversion modes.
4. Scan mode for automatic conversion of channel 0 to channel x.
5. Data alignment with in-built data coherency.
6. Channel-wise programmable sampling time.
7. External trigger option with configurable polarity for both regular and injected conversion.
8. Dual/Triple mode (on devices with 2 ADCs or more).
9. Configurable DMA data storage in Dual/Triple ADC mode.
10. Configurable delay between conversions in Dual/Triple interleaved mode.
11. ADC conversion type (refer to the datasheets).
12. ADC supply requirements: 2.4 V to 3.6 V at full speed and down to 1.8 V at slower speed.
13. ADC input range: VREF(minus) = VIN = VREF(plus).
14. DMA request generation during regular channel conversion.

7.2.2 How to use this driver

1. Initialize the ADC low level resources by implementing the HAL_ADC_MspInit():
 - a. Enable the ADC interface clock using __HAL_RCC_ADC_CLK_ENABLE()
 - b. ADC pins configuration
 - Enable the clock for the ADC GPIOs using the following function: __HAL_RCC_GPIOx_CLK_ENABLE()
 - Configure these ADC pins in analog mode using HAL_GPIO_Init()
 - c. In case of using interrupts (e.g. HAL_ADC_Start_IT())
 - Configure the ADC interrupt priority using HAL_NVIC_SetPriority()
 - Enable the ADC IRQ handler using HAL_NVIC_EnableIRQ()
 - In ADC IRQ handler, call HAL_ADC_IRQHandler()
 - d. In case of using DMA to control data transfer (e.g. HAL_ADC_Start_DMA())
 - Enable the DMAx interface clock using __HAL_RCC_DMAx_CLK_ENABLE()
 - Configure and enable two DMA streams stream for managing data transfer from peripheral to memory (output stream)
 - Associate the initialized DMA handle to the CRYp DMA handle using __HAL_LINKDMA()
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the two DMA Streams. The output stream should have higher priority than the input stream.

Configuration of ADC, groups regular/injected, channels parameters

1. Configure the ADC parameters (resolution, data alignment, ...) and regular group parameters (conversion trigger, sequencer, ...) using function HAL_ADC_Init().
2. Configure the channels for regular group parameters (channel number, channel rank into sequencer, ..., into regular group) using function HAL_ADC_ConfigChannel().
3. Optionally, configure the injected group parameters (conversion trigger, sequencer, ..., of injected group) and the channels for injected group parameters (channel number, channel rank into sequencer, ..., into injected group) using function HAL_ADCEX_InjectedConfigChannel().
4. Optionally, configure the analog watchdog parameters (channels monitored, thresholds, ...) using function HAL_ADC_AnalogWDGConfig().
5. Optionally, for devices with several ADC instances: configure the multimode parameters using function HAL_ADCEX_MultiModeConfigChannel().

Execution of ADC conversions

1. ADC driver can be used among three modes: polling, interruption, transfer by DMA.

Polling mode IO operation

- Start the ADC peripheral using HAL_ADC_Start()
- Wait for end of conversion using HAL_ADC_PollForConversion(), at this stage user can specify the value of timeout according to his end application
- To read the ADC converted values, use the HAL_ADC_GetValue() function.
- Stop the ADC peripheral using HAL_ADC_Stop()

Interrupt mode IO operation

- Start the ADC peripheral using HAL_ADC_Start_IT()
- Use HAL_ADC_IRQHandler() called under ADC_IRQHandler() Interrupt subroutine
- At ADC end of conversion HAL_ADC_ConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ConvCpltCallback
- In case of ADC Error, HAL_ADC_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADC_ErrorCallback
- Stop the ADC peripheral using HAL_ADC_Stop_IT()

ADC HAL driver macros list

Below the list of most used macros in ADC HAL driver.

- `__HAL_ADC_ENABLE` : Enable the ADC peripheral
- `__HAL_ADC_DISABLE` : Disable the ADC peripheral
- `__HAL_ADC_ENABLE_IT`: Enable the ADC end of conversion interrupt
- `__HAL_ADC_DISABLE_IT`: Disable the ADC end of conversion interrupt
- `__HAL_ADC_GET_IT_SOURCE`: Check if the specified ADC interrupt source is enabled or disabled
- `__HAL_ADC_CLEAR_FLAG`: Clear the ADC's pending flags
- `__HAL_ADC_GET_FLAG`: Get the selected ADC's flag status
- `ADC_GET_RESOLUTION`: Return resolution bits in CR1 register

Note: You can refer to the ADC HAL driver header file for more useful macros

Deinitialization of ADC

1. Disable the ADC interface
 - ADC clock can be hard reset and disabled at RCC top level.
 - Hard reset of ADC peripherals using macro `__HAL_RCC_ADC_FORCE_RESET()`, `__HAL_RCC_ADC_RELEASE_RESET()`.
 - ADC clock disable using the equivalent macro/functions as configuration step.
 - Example: Into `HAL_ADC_MspDeInit()` (recommended code location) or with other device clock parameters configuration:
 - `HAL_RCC_GetOscConfig(&RCC_OscInitStructure);`
 - `RCC_OscInitStructure.OscillatorType = RCC_OSCILLATORTYPE_HSI;`
 - `RCC_OscInitStructure.HSIState = RCC_HSI_OFF;` (if not used for system clock)
 - `HAL_RCC_OscConfig(&RCC_OscInitStructure);`
2. ADC pins configuration
 - Disable the clock for the ADC GPIOs using macro `__HAL_RCC_GPIOx_CLK_DISABLE()`
3. Optionally, in case of usage of ADC with interruptions:
 - Disable the NVIC for ADC using function `HAL_NVIC_DisableIRQ(ADCx_IRQn)`
4. Optionally, in case of usage of DMA:
 - Deinitialize the DMA using function `HAL_DMA_DeInit()`.
 - Disable the NVIC for DMA using function `HAL_NVIC_DisableIRQ(DMAx_Channelx_IRQn)`

Callback registration

The compilation flag `USE_HAL_ADC_REGISTER_CALLBACKS`, when set to 1, allows the user to configure dynamically the driver callbacks. Use Functions [@ref HAL_ADC_RegisterCallback\(\)](#) to register an interrupt callback.

Function [@ref HAL_ADC_RegisterCallback\(\)](#) allows to register following callbacks:

- `ConvCpltCallback` : ADC conversion complete callback
- `ConvHalfCpltCallback` : ADC conversion DMA half-transfer callback
- `LevelOutOfWindowCallback` : ADC analog watchdog 1 callback
- `ErrorCallback` : ADC error callback
- `InjectedConvCpltCallback` : ADC group injected conversion complete callback
- `InjectedQueueOverflowCallback` : ADC group injected context queue overflow callback
- `LevelOutOfWindow2Callback` : ADC analog watchdog 2 callback
- `LevelOutOfWindow3Callback` : ADC analog watchdog 3 callback
- `EndOfSamplingCallback` : ADC end of sampling callback
- `MspInitCallback` : ADC Msp Init callback
- `MspDeInitCallback` : ADC Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function [@ref HAL_ADC_UnRegisterCallback](#) to reset a callback to the default weak function.

[@ref HAL_ADC_UnRegisterCallback](#) takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- ConvCpltCallback : ADC conversion complete callback
- ConvHalfCpltCallback : ADC conversion DMA half-transfer callback
- LevelOutOfWindowCallback : ADC analog watchdog 1 callback
- ErrorCallback : ADC error callback
- InjectedConvCpltCallback : ADC group injected conversion complete callback
- InjectedQueueOverflowCallback : ADC group injected context queue overflow callback
- LevelOutOfWindow2Callback : ADC analog watchdog 2 callback
- LevelOutOfWindow3Callback : ADC analog watchdog 3 callback
- EndOfSamplingCallback : ADC end of sampling callback
- MspInitCallback : ADC Msp Init callback
- MspDeInitCallback : ADC Msp DeInit callback

By default, after the @ref HAL_ADC_Init() and when the state is @ref HAL_ADC_STATE_RESET all callbacks are set to the corresponding weak functions: examples @ref HAL_ADC_ConvCpltCallback(), @ref HAL_ADC_ErrorCallback(). Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functions in the @ref HAL_ADC_Init()/ @ref HAL_ADC_DeInit() only when these callbacks are null (not registered beforehand).

If MspInit or MspDeInit are not null, the @ref HAL_ADC_Init()/ @ref HAL_ADC_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in @ref HAL_ADC_STATE_READY state only. Exception done MspInit/ MspDeInit functions that can be registered/unregistered in @ref HAL_ADC_STATE_READY or @ref HAL_ADC_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/ DeInit.

Then, the user first registers the MspInit/MspDeInit user callbacks using @ref HAL_ADC_RegisterCallback() before calling @ref HAL_ADC_DeInit() or @ref HAL_ADC_Init() function.

When the compilation flag USE_HAL_ADC_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

7.2.3 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize and configure the ADC.
- De-initialize the ADC.

This section contains the following APIs:

- [HAL_ADC_Init](#)
- [HAL_ADC_DeInit](#)
- [HAL_ADC_MspInit](#)
- [HAL_ADC_MspDeInit](#)

7.2.4 IO operation functions

This section provides functions allowing to:

- Start conversion of regular channel.
- Stop conversion of regular channel.
- Start conversion of regular channel and enable interrupt.
- Stop conversion of regular channel and disable interrupt.
- Start conversion of regular channel and enable DMA transfer.
- Stop conversion of regular channel and disable DMA transfer.
- Handle ADC interrupt request.

This section contains the following APIs:

- [HAL_ADC_Start](#)
- [HAL_ADC_Stop](#)
- [HAL_ADC_PollForConversion](#)
- [HAL_ADC_PollForEvent](#)

- [HAL_ADC_Start_IT](#)
- [HAL_ADC_Stop_IT](#)
- [HAL_ADC_IRQHandler](#)
- [HAL_ADC_Start_DMA](#)
- [HAL_ADC_Stop_DMA](#)
- [HAL_ADC_GetValue](#)
- [HAL_ADC_ConvCpltCallback](#)
- [HAL_ADC_ConvHalfCpltCallback](#)
- [HAL_ADC_LevelOutOfWindowCallback](#)
- [HAL_ADC_ErrorCallback](#)

7.2.5 Peripheral Control functions

This section provides functions allowing to:

- Configure regular channels.
- Configure injected channels.
- Configure multimode.
- Configure the analog watch dog.

This section contains the following APIs:

- [HAL_ADC_ConfigChannel](#)
- [HAL_ADC_AnalogWDGConfig](#)

7.2.6 Peripheral State and errors functions

This subsection provides functions allowing to

- Check the ADC state
- Check the ADC Error

This section contains the following APIs:

- [HAL_ADC_GetState](#)
- [HAL_ADC_GetError](#)

7.2.7 Detailed description of functions

HAL_ADC_Init

Function name

HAL_StatusTypeDef HAL_ADC_Init (ADC_HandleTypeDef * hadc)

Function description

Initializes the ADCx peripheral according to the specified parameters in the ADC_InitStruct and initializes the ADC MSP.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status

Notes

- This function is used to configure the global features of the ADC (ClockPrescaler, Resolution, Data Alignment and number of conversion), however, the rest of the configuration parameters are specific to the regular channels group (scan mode activation, continuous mode activation, External trigger source and edge, DMA continuous request after the last transfer and End of conversion selection).

HAL_ADC_DeInit

Function name

HAL_StatusTypeDef HAL_ADC_DeInit (ADC_HandleTypeDef * hadc)

Function description

Deinitializes the ADCx peripheral registers to their default reset values.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status

HAL_ADC_MspInit

Function name

void HAL_ADC_MspInit (ADC_HandleTypeDef * hadc)

Function description

Initializes the ADC MSP.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

HAL_ADC_MspDeInit

Function name

void HAL_ADC_MspDeInit (ADC_HandleTypeDef * hadc)

Function description

DeInitializes the ADC MSP.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

HAL_ADC_Start

Function name

HAL_StatusTypeDef HAL_ADC_Start (ADC_HandleTypeDef * hadc)

Function description

Enables ADC and starts conversion of the regular channels.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADC_Stop

Function name

HAL_StatusTypeDef HAL_ADC_Stop (ADC_HandleTypeDef * hadc)

Function description

Disables ADC and stop conversion of regular channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

Notes

- Caution: This function will stop also injected channels.

HAL_ADC_PollForConversion

Function name

HAL_StatusTypeDef HAL_ADC_PollForConversion (ADC_HandleTypeDef * hadc, uint32_t Timeout)

Function description

Poll for regular conversion complete.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **Timeout:** Timeout value in millisecond.

Return values

- **HAL:** status

Notes

- ADC conversion flags EOS (end of sequence) and EOC (end of conversion) are cleared by this function.
- This function cannot be used in a particular setup: ADC configured in DMA mode and polling for end of each conversion (ADC init parameter "EOCSelection" set to ADC_EOC_SINGLE_CONV). In this case, DMA resets the flag EOC and polling cannot be performed on each conversion. Nevertheless, polling can still be performed on the complete sequence.

HAL_ADC_PollForEvent

Function name

HAL_StatusTypeDef HAL_ADC_PollForEvent (ADC_HandleTypeDef * hadc, uint32_t EventType, uint32_t Timeout)

Function description

Poll for conversion event.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **EventType**: the ADC event type. This parameter can be one of the following values:
 - ADC_AWD_EVENT: ADC Analog watch Dog event.
 - ADC_OVR_EVENT: ADC Overrun event.
- **Timeout**: Timeout value in millisecond.

Return values

- **HAL**: status

HAL_ADC_Start_IT

Function name

HAL_StatusTypeDef HAL_ADC_Start_IT (ADC_HandleTypeDef * hadc)

Function description

Enables the interrupt and starts ADC conversion of regular channels.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status.

HAL_ADC_Stop_IT

Function name

HAL_StatusTypeDef HAL_ADC_Stop_IT (ADC_HandleTypeDef * hadc)

Function description

Disables the interrupt and stop ADC conversion of regular channels.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status.

Notes

- Caution: This function will stop also injected channels.

HAL_ADC_IRQHandler

Function name

void HAL_ADC_IRQHandler (ADC_HandleTypeDef * hadc)

Function description

Handles ADC interrupt request.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADC_Start_DMA

Function name

HAL_StatusTypeDef HAL_ADC_Start_DMA (ADC_HandleTypeDef * hadc, uint32_t * pData, uint32_t Length)

Function description

Enables ADC DMA request after last transfer (Single-ADC mode) and enables ADC peripheral.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from ADC peripheral to memory.

Return values

- **HAL:** status

HAL_ADC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_ADC_Stop_DMA (ADC_HandleTypeDef * hadc)

Function description

Disables ADC DMA (Single-ADC mode) and disables ADC peripheral.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADC_GetValue

Function name

uint32_t HAL_ADC_GetValue (ADC_HandleTypeDef * hadc)

Function description

Gets the converted value from data register of regular channel.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **Converted:** value

HAL_ADC_ConvCpltCallback

Function name

void HAL_ADC_ConvCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Regular conversion complete callback in non blocking mode.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

HAL_ADC_ConvHalfCpltCallback

Function name

void HAL_ADC_ConvHalfCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Regular conversion half DMA transfer callback in non blocking mode.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

HAL_ADC_LevelOutOfWindowCallback

Function name

void HAL_ADC_LevelOutOfWindowCallback (ADC_HandleTypeDef * hadc)

Function description

Analog watchdog callback in non blocking mode.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

HAL_ADC_ErrorCallback

Function name

void HAL_ADC_ErrorCallback (ADC_HandleTypeDef * hadc)

Function description

Error ADC callback.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None**:

Notes

- In case of error due to overrun when using ADC with DMA transfer (HAL ADC handle parameter "ErrorCode" to state "HAL_ADC_ERROR_OVR"): Reinitialize the DMA using function "HAL_ADC_Stop_DMA()". If needed, restart a new ADC conversion using function "HAL_ADC_Start_DMA()" (this function is also clearing overrun flag)

HAL_ADC_ConfigChannel

Function name

HAL_StatusTypeDef HAL_ADC_ConfigChannel (ADC_HandleTypeDef * hadc, ADC_ChannelConfTypeDef * sConfig)

Function description

Configures for the selected ADC regular channel its corresponding rank in the sequencer and its sample time.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **sConfig**: ADC configuration structure.

Return values

- **HAL**: status

HAL_ADC_AnalogWDGConfig

Function name

HAL_StatusTypeDef HAL_ADC_AnalogWDGConfig (ADC_HandleTypeDef * hadc, ADC_AnalogWDGConfTypeDef * AnalogWDGConfig)

Function description

Configures the analog watchdog.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **AnalogWDGConfig**: pointer to an ADC_AnalogWDGConfTypeDef structure that contains the configuration information of ADC analog watchdog.

Return values

- **HAL**: status

Notes

- Analog watchdog thresholds can be modified while ADC conversion is on going. In this case, some constraints must be taken into account: The programmed threshold values are effective from the next ADC EOC (end of unitary conversion). Considering that registers write delay may happen due to bus activity, this might cause an uncertainty on the effective timing of the new programmed threshold values.

HAL_ADC_GetState

Function name

uint32_t HAL_ADC_GetState (ADC_HandleTypeDef * hadc)

Function description

return the ADC state

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: state

HAL_ADC_GetError

Function name

uint32_t HAL_ADC_GetError (ADC_HandleTypeDef * hadc)

Function description

Return the ADC error code.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **ADC**: Error Code

7.3 ADC Firmware driver defines

The following section lists the various define and macros of the module.

7.3.1 ADC

ADC

ADC Analog Watchdog Selection

ADC_ANALOGWATCHDOG_SINGLE_REG

ADC_ANALOGWATCHDOG_SINGLE_INJEC

ADC_ANALOGWATCHDOG_SINGLE_REGINJEC

ADC_ANALOGWATCHDOG_ALL_REG

ADC_ANALOGWATCHDOG_ALL_INJEC

ADC_ANALOGWATCHDOG_ALL_REGINJEC

ADC_ANALOGWATCHDOG_NONE

ADC Common Channels

ADC_CHANNEL_0

ADC_CHANNEL_1

ADC_CHANNEL_2

ADC_CHANNEL_3

ADC_CHANNEL_4

ADC_CHANNEL_5

ADC_CHANNEL_6

ADC_CHANNEL_7

ADC_CHANNEL_8

ADC_CHANNEL_9

ADC_CHANNEL_10

ADC_CHANNEL_11

ADC_CHANNEL_12

ADC_CHANNEL_13

ADC_CHANNEL_14

ADC_CHANNEL_15

ADC_CHANNEL_16

ADC_CHANNEL_17

ADC_CHANNEL_18

ADC_CHANNEL_VREFINT

ADC_CHANNEL_VBAT

ADC Channels Type

ADC_ALL_CHANNELS

ADC_REGULAR_CHANNELS

reserved for future use

ADC_INJECTED_CHANNELS

reserved for future use

ADC Clock Prescaler

ADC_CLOCK_SYNC_PCLK_DIV2

ADC_CLOCK_SYNC_PCLK_DIV4

ADC_CLOCK_SYNC_PCLK_DIV6

ADC_CLOCK_SYNC_PCLK_DIV8

ADC Data Align

ADC_DATAALIGN_RIGHT

ADC_DATAALIGN_LEFT

ADC Delay Between 2 Sampling Phases

ADC_TWOSAMPLINGDELAY_5CYCLES

ADC_TWOSAMPLINGDELAY_6CYCLES

ADC_TWOSAMPLINGDELAY_7CYCLES

ADC_TWOSAMPLINGDELAY_8CYCLES

ADC_TWOSAMPLINGDELAY_9CYCLES

ADC_TWOSAMPLINGDELAY_10CYCLES

ADC_TWOSAMPLINGDELAY_11CYCLES

ADC_TWOSAMPLINGDELAY_12CYCLES

ADC_TWOSAMPLINGDELAY_13CYCLES

ADC_TWOSAMPLINGDELAY_14CYCLES

ADC_TWOSAMPLINGDELAY_15CYCLES

ADC_TWOSAMPLINGDELAY_16CYCLES

ADC_TWOSAMPLINGDELAY_17CYCLES

ADC_TWOSAMPLINGDELAY_18CYCLES

ADC_TWOSAMPLINGDELAY_19CYCLES

ADC_TWOSAMPLINGDELAY_20CYCLES

ADC EOC Selection

ADC_EOC_SEQ_CONV

ADC_EOC_SINGLE_CONV

ADC_EOC_SINGLE_SEQ_CONV

reserved for future use

ADC Error Code

HAL_ADC_ERROR_NONE

No error

HAL_ADC_ERROR_INTERNAL

ADC IP internal error: if problem of clocking, enable/disable, erroneous state

HAL_ADC_ERROR_OVR

Overrun error

HAL_ADC_ERROR_DMA

DMA transfer error

ADC Event Type

ADC_AWD_EVENT

ADC_OVR_EVENT

ADC Exported Macros

__HAL_ADC_RESET_HANDLE_STATE

Description:

- Reset ADC handle state.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_ENABLE

Description:

- Enable the ADC peripheral.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_DISABLE

Description:

- Disable the ADC peripheral.

Parameters:

- __HANDLE__: ADC handle

Return value:

- None

__HAL_ADC_ENABLE_IT

Description:

- Enable the ADC end of conversion interrupt.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: ADC Interrupt.

Return value:

- None

__HAL_ADC_DISABLE_IT

Description:

- Disable the ADC end of conversion interrupt.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: ADC interrupt.

Return value:

- None

__HAL_ADC_GET_IT_SOURCE

Description:

- Check if the specified ADC interrupt source is enabled or disabled.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __INTERRUPT__: specifies the ADC interrupt source to check.

Return value:

- The: new state of __IT__ (TRUE or FALSE).

__HAL_ADC_CLEAR_FLAG

Description:

- Clear the ADC's pending flags.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __FLAG__: ADC flag.

Return value:

- None

__HAL_ADC_GET_FLAG

Description:

- Get the selected ADC's flag status.

Parameters:

- __HANDLE__: specifies the ADC Handle.
- __FLAG__: ADC flag.

Return value:

- None

ADC Exported Types

HAL_ADC_STATE_RESET

ADC not yet initialized or disabled

HAL_ADC_STATE_READY

ADC peripheral ready for use

HAL_ADC_STATE_BUSY_INTERNAL

ADC is busy to internal process (initialization, calibration)

HAL_ADC_STATE_TIMEOUT

TimeOut occurrence

HAL_ADC_STATE_ERROR_INTERNAL

Internal error occurrence

HAL_ADC_STATE_ERROR_CONFIG

Configuration error occurrence

HAL_ADC_STATE_ERROR_DMA

DMA error occurrence

HAL_ADC_STATE_REG_BUSY

A conversion on group regular is ongoing or can occur (either by continuous mode, external trigger, low power auto power-on (if feature available), multimode ADC master control (if feature available))

HAL_ADC_STATE_REG_EOC

Conversion data available on group regular

HAL_ADC_STATE_REG_OVR

Overrun occurrence

HAL_ADC_STATE_INJ_BUSY

A conversion on group injected is ongoing or can occur (either by auto-injection mode, external trigger, low power auto power-on (if feature available), multimode ADC master control (if feature available))

HAL_ADC_STATE_INJ_EOC

Conversion data available on group injected

HAL_ADC_STATE_AWD1

Out-of-window occurrence of analog watchdog 1

ADC External Trigger Edge Regular

ADC_EXTERNALTRIGCONVEDGE_NONE

ADC_EXTERNALTRIGCONVEDGE_RISING

ADC_EXTERNALTRIGCONVEDGE_FALLING

ADC_EXTERNALTRIGCONVEDGE_RISINGFALLING

ADC External Trigger Source Regular

ADC_EXTERNALTRIGCONV_T1_CC1

ADC_EXTERNALTRIGCONV_T1_CC2

ADC_EXTERNALTRIGCONV_T1_CC3

ADC_EXTERNALTRIGCONV_T2_CC2

ADC_EXTERNALTRIGCONV_T2_CC3

ADC_EXTERNALTRIGCONV_T2_CC4

ADC_EXTERNALTRIGCONV_T2_TRGO

ADC_EXTERNALTRIGCONV_T3_CC1

ADC_EXTERNALTRIGCONV_T3_TRGO

ADC_EXTERNALTRIGCONV_T4_CC4

ADC_EXTERNALTRIGCONV_T5_CC1

ADC_EXTERNALTRIGCONV_T5_CC2

ADC_EXTERNALTRIGCONV_T5_CC3

ADC_EXTERNALTRIGCONV_T8_CC1

ADC_EXTERNALTRIGCONV_T8_TRGO

ADC_EXTERNALTRIGCONV_Ext_IT11

ADC_SOFTWARE_START

ADC Flags Definition

ADC_FLAG_AWD

ADC_FLAG_EOC

ADC_FLAG_JEOC

ADC_FLAG_JSTRT

ADC_FLAG_STRT

ADC_FLAG_OVR

ADC Interrupts Definition

ADC_IT_EOC

ADC_IT_AWD

ADC_IT_JEOC

ADC_IT_OVR

ADC Resolution

ADC_RESOLUTION_12B

ADC_RESOLUTION_10B

ADC_RESOLUTION_8B

ADC_RESOLUTION_6B

ADC Sampling Times

ADC_SAMPLETIME_3CYCLES

ADC_SAMPLETIME_15CYCLES

ADC_SAMPLETIME_28CYCLES

ADC_SAMPLETIME_56CYCLES

ADC_SAMPLETIME_84CYCLES

ADC_SAMPLETIME_112CYCLES

ADC_SAMPLETIME_144CYCLES

ADC_SAMPLETIME_480CYCLES

8 HAL ADC Extension Driver

8.1 ADCEX Firmware driver registers structures

8.1.1 ADC_InjectionConfTypeDef

ADC_InjectionConfTypeDef is defined in the `stm32f2xx_hal_adc_ex.h`

Data Fields

- **`uint32_t InjectedChannel`**
- **`uint32_t InjectedRank`**
- **`uint32_t InjectedSamplingTime`**
- **`uint32_t InjectedOffset`**
- **`uint32_t InjectedNbrOfConversion`**
- **`uint32_t InjectedDiscontinuousConvMode`**
- **`uint32_t AutoInjectedConv`**
- **`uint32_t ExternalTrigInjecConv`**
- **`uint32_t ExternalTrigInjecConvEdge`**

Field Documentation

- **`uint32_t ADC_InjectionConfTypeDef::InjectedChannel`**
Selection of ADC channel to configure This parameter can be a value of [ADC_channels](#) Note: Depending on devices, some channels may not be available on package pins. Refer to device datasheet for channels availability.
- **`uint32_t ADC_InjectionConfTypeDef::InjectedRank`**
Rank in the injected group sequencer This parameter must be a value of [ADCEX_injected_rank](#) Note: In case of need to disable a channel or change order of conversion sequencer, rank containing a previous channel setting can be overwritten by the new channel setting (or parameter number of conversions can be adjusted)
- **`uint32_t ADC_InjectionConfTypeDef::InjectedSamplingTime`**
Sampling time value to be set for the selected channel. Unit: ADC clock cycles Conversion time is the addition of sampling time and processing time (12 ADC clock cycles at ADC resolution 12 bits, 11 cycles at 10 bits, 9 cycles at 8 bits, 7 cycles at 6 bits). This parameter can be a value of [ADC_sampling_times](#) Caution: This parameter updates the parameter property of the channel, that can be used into regular and/or injected groups. If this same channel has been previously configured in the other group (regular/injected), it will be updated to last setting. Note: In case of usage of internal measurement channels (VrefInt/Vbat/TempSensor), sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting) Refer to device datasheet for timings values, parameters TS_vrefint, TS_temp (values rough order: 4us min).
- **`uint32_t ADC_InjectionConfTypeDef::InjectedOffset`**
Defines the offset to be subtracted from the raw converted data (for channels set on injected group only). Offset value must be a positive number. Depending of ADC resolution selected (12, 10, 8 or 6 bits), this parameter must be a number between Min_Data = 0x000 and Max_Data = 0xFFF, 0x3FF, 0xFF or 0x3F respectively.
- **`uint32_t ADC_InjectionConfTypeDef::InjectedNbrOfConversion`**
Specifies the number of ranks that will be converted within the injected group sequencer. To use the injected group sequencer and convert several ranks, parameter 'ScanConvMode' must be enabled. This parameter must be a number between Min_Data = 1 and Max_Data = 4. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEX_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.

- `uint32_t ADC_InjectionConfTypeDef::InjectedDiscontinuousConvMode`**
Specifies whether the conversions sequence of injected group is performed in Complete-sequence/ Discontinuous-sequence (main sequence subdivided in successive parts). Discontinuous mode is used only if sequencer is enabled (parameter 'ScanConvMode'). If sequencer is disabled, this parameter is discarded. Discontinuous mode can be enabled only if continuous mode is disabled. If continuous mode is enabled, this parameter setting is discarded. This parameter can be set to ENABLE or DISABLE. Note: For injected group, number of discontinuous ranks increment is fixed to one-by-one. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- `uint32_t ADC_InjectionConfTypeDef::AutoInjectedConv`**
Enables or disables the selected ADC automatic injected group conversion after regular one. This parameter can be set to ENABLE or DISABLE. Note: To use Automatic injected conversion, discontinuous mode must be disabled ('DiscontinuousConvMode' and 'InjectedDiscontinuousConvMode' set to DISABLE). Note: To use Automatic injected conversion, injected group external triggers must be disabled ('ExternalTrigInjecConv' set to ADC_SOFTWARE_START). Note: In case of DMA used with regular group: if DMA configured in normal mode (single shot) JAUTO will be stopped upon DMA transfer complete. To maintain JAUTO always enabled, DMA must be configured in circular mode. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- `uint32_t ADC_InjectionConfTypeDef::ExternalTrigInjecConv`**
Selects the external event used to trigger the conversion start of injected group. If set to ADC_INJECTED_SOFTWARE_START, external triggers are disabled. If set to external trigger source, triggering is on event rising edge. This parameter can be a value of [ADCEx_External_trigger_Source_Injected](#). Note: This parameter must be modified when ADC is disabled (before ADC start conversion or after ADC stop conversion). If ADC is enabled, this parameter setting is bypassed without error reporting (as it can be the expected behaviour in case of another parameter update on the fly). Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.
- `uint32_t ADC_InjectionConfTypeDef::ExternalTrigInjecConvEdge`**
Selects the external trigger edge of injected group. This parameter can be a value of [ADCEx_External_trigger_edge_Injected](#). If trigger is set to ADC_INJECTED_SOFTWARE_START, this parameter is discarded. Caution: this setting impacts the entire injected group. Therefore, call of **HAL_ADCEx_InjectedConfigChannel()** to configure a channel on injected group can impact the configuration of other channels previously set.

8.1.2

ADC_MultiModeTypeDef

ADC_MultiModeTypeDef is defined in the `stm32f2xx_hal_adc_ex.h`

Data Fields

- `uint32_t Mode`**
- `uint32_t DMAAccessMode`**
- `uint32_t TwoSamplingDelay`**

Field Documentation

- `uint32_t ADC_MultiModeTypeDef::Mode`**
Configures the ADC to operate in independent or multi mode. This parameter can be a value of [ADCEx_Common_mode](#)
- `uint32_t ADC_MultiModeTypeDef::DMAAccessMode`**
Configures the Direct memory access mode for multi ADC mode. This parameter can be a value of [ADCEx_Direct_memory_access_mode_for_multi_mode](#)
- `uint32_t ADC_MultiModeTypeDef::TwoSamplingDelay`**
Configures the Delay between 2 sampling phases. This parameter can be a value of [ADC_delay_between_2_sampling_phases](#)

8.2

ADCEx Firmware driver API description

The following section lists the various functions of the ADCEx library.

8.2.1

How to use this driver

1. Initialize the ADC low level resources by implementing the HAL_ADC_MspInit():
 - a. Enable the ADC interface clock using __HAL_RCC_ADC_CLK_ENABLE()
 - b. ADC pins configuration
 - Enable the clock for the ADC GPIOs using the following function: __HAL_RCC_GPIOx_CLK_ENABLE()
 - Configure these ADC pins in analog mode using HAL_GPIO_Init()
 - c. In case of using interrupts (e.g. HAL_ADC_Start_IT())
 - Configure the ADC interrupt priority using HAL_NVIC_SetPriority()
 - Enable the ADC IRQ handler using HAL_NVIC_EnableIRQ()
 - In ADC IRQ handler, call HAL_ADC_IRQHandler()
 - d. In case of using DMA to control data transfer (e.g. HAL_ADC_Start_DMA())
 - Enable the DMAx interface clock using __HAL_RCC_DMAx_CLK_ENABLE()
 - Configure and enable two DMA streams stream for managing data transfer from peripheral to memory (output stream)
 - Associate the initialized DMA handle to the ADC DMA handle using __HAL_LINKDMA()
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the two DMA Streams. The output stream should have higher priority than the input stream.
2. Configure the ADC Prescaler, conversion resolution and data alignment using the HAL_ADC_Init() function.
3. Configure the ADC Injected channels group features, use HAL_ADC_Init() and HAL_ADC_ConfigChannel() functions.
4. Three operation modes are available within this driver :

Polling mode IO operation

- Start the ADC peripheral using HAL_ADCEx_InjectedStart()
- Wait for end of conversion using HAL_ADC_PollForConversion(), at this stage user can specify the value of timeout according to his end application
- To read the ADC converted values, use the HAL_ADCEx_InjectedGetValue() function.
- Stop the ADC peripheral using HAL_ADCEx_InjectedStop()

Interrupt mode IO operation

- Start the ADC peripheral using HAL_ADCEx_InjectedStart_IT()
- Use HAL_ADC_IRQHandler() called under ADC_IRQHandler() Interrupt subroutine
- At ADC end of conversion HAL_ADCEx_InjectedConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedConvCpltCallback
- In case of ADC Error, HAL_ADCEx_InjectedErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedErrorCallback
- Stop the ADC peripheral using HAL_ADCEx_InjectedStop_IT()

DMA mode IO operation

- Start the ADC peripheral using HAL_ADCEx_InjectedStart_DMA(), at this stage the user specify the length of data to be transferred at each end of conversion
- At The end of data transfer ba HAL_ADCEx_InjectedConvCpltCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedConvCpltCallback
- In case of transfer Error, HAL_ADCEx_InjectedErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_ADCEx_InjectedErrorCallback
- Stop the ADC peripheral using HAL_ADCEx_InjectedStop_DMA()

Multi mode ADCs Regular channels configuration

- Select the Multi mode ADC regular channels features (dual or triple mode) and configure the DMA mode using HAL_ADCEx_MultiModeConfigChannel() functions.

- Start the ADC peripheral using HAL_ADCEX_MultiModeStart_DMA(), at this stage the user specify the length of data to be transferred at each end of conversion
- Read the ADCs converted values using the HAL_ADCEX_MultiModeGetValue() function.

8.2.2 Extended features functions

This section provides functions allowing to:

- Start conversion of injected channel.
- Stop conversion of injected channel.
- Start multimode and enable DMA transfer.
- Stop multimode and disable DMA transfer.
- Get result of injected channel conversion.
- Get result of multimode conversion.
- Configure injected channels.
- Configure multimode.

This section contains the following APIs:

- [*HAL_ADCEX_InjectedStart*](#)
- [*HAL_ADCEX_InjectedStart_IT*](#)
- [*HAL_ADCEX_InjectedStop*](#)
- [*HAL_ADCEX_InjectedPollForConversion*](#)
- [*HAL_ADCEX_InjectedStop_IT*](#)
- [*HAL_ADCEX_InjectedGetValue*](#)
- [*HAL_ADCEX_MultiModeStart_DMA*](#)
- [*HAL_ADCEX_MultiModeStop_DMA*](#)
- [*HAL_ADCEX_MultiModeGetValue*](#)
- [*HAL_ADCEX_InjectedConvCpltCallback*](#)
- [*HAL_ADCEX_InjectedConfigChannel*](#)
- [*HAL_ADCEX_MultiModeConfigChannel*](#)

8.2.3 Detailed description of functions

HAL_ADCEX_InjectedStart

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStart (ADC_HandleTypeDef * hadc)

Function description

Enables the selected ADC software start conversion of the injected channels.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL**: status

HAL_ADCEX_InjectedStop

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStop (ADC_HandleTypeDef * hadc)

Function description

Stop conversion of injected channels.

Parameters

- **hadc:** ADC handle

Return values

- **None:**

Notes

- If ADC must be disabled and if conversion is on going on regular group, function HAL_ADC_Stop must be used to stop both injected and regular groups, and disable the ADC.
- If injected group mode auto-injection is enabled, function HAL_ADC_Stop must be used.
- In case of auto-injection mode, HAL_ADC_Stop must be used.

HAL_ADCEX_InjectedPollForConversion

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedPollForConversion (ADC_HandleTypeDef * hadc, uint32_t Timeout)

Function description

Poll for injected conversion complete.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **Timeout:** Timeout value in millisecond.

Return values

- **HAL:** status

HAL_ADCEX_InjectedStart_IT

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStart_IT (ADC_HandleTypeDef * hadc)

Function description

Enables the interrupt and starts ADC conversion of injected channels.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status.

HAL_ADCEX_InjectedStop_IT

Function name

HAL_StatusTypeDef HAL_ADCEX_InjectedStop_IT (ADC_HandleTypeDef * hadc)

Function description

Stop conversion of injected channels, disable interruption of end-of-conversion.

Parameters

- **hadc:** ADC handle

Return values

- **None:**

Notes

- If ADC must be disabled and if conversion is on going on regular group, function HAL_ADC_Stop must be used to stop both injected and regular groups, and disable the ADC.
- If injected group mode auto-injection is enabled, function HAL_ADC_Stop must be used.

HAL_ADCEx_InjectedGetValue

Function name

```
uint32_t HAL_ADCEx_InjectedGetValue (ADC_HandleTypeDef * hadc, uint32_t InjectedRank)
```

Function description

Gets the converted value from data register of injected channel.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **InjectedRank**: the ADC injected rank. This parameter can be one of the following values:
 - ADC_INJECTED_RANK_1: Injected Channel1 selected
 - ADC_INJECTED_RANK_2: Injected Channel2 selected
 - ADC_INJECTED_RANK_3: Injected Channel3 selected
 - ADC_INJECTED_RANK_4: Injected Channel4 selected

Return values

- **None**:

HAL_ADCEx_MultiModeStart_DMA

Function name

```
HAL_StatusTypeDef HAL_ADCEx_MultiModeStart_DMA (ADC_HandleTypeDef * hadc, uint32_t * pData, uint32_t Length)
```

Function description

Enables ADC DMA request after last transfer (Multi-ADC mode) and enables ADC peripheral.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **pData**: Pointer to buffer in which transferred from ADC peripheral to memory will be stored.
- **Length**: The length of data to be transferred from ADC peripheral to memory.

Return values

- **HAL**: status

Notes

- Caution: This function must be used only with the ADC master.

HAL_ADCEx_MultiModeStop_DMA

Function name

```
HAL_StatusTypeDef HAL_ADCEx_MultiModeStop_DMA (ADC_HandleTypeDef * hadc)
```

Function description

Disables ADC DMA (multi-ADC mode) and disables ADC peripheral.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **HAL:** status

HAL_ADCEx_MultiModeGetValue

Function name

uint32_t HAL_ADCEx_MultiModeGetValue (ADC_HandleTypeDef * hadc)

Function description

Returns the last ADC1, ADC2 and ADC3 regular conversions results data in the selected multi mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **The:** converted data value.

HAL_ADCEx_InjectedConvCpltCallback

Function name

void HAL_ADCEx_InjectedConvCpltCallback (ADC_HandleTypeDef * hadc)

Function description

Injected conversion complete callback in non blocking mode.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.

Return values

- **None:**

HAL_ADCEx_InjectedConfigChannel

Function name

HAL_StatusTypeDef HAL_ADCEx_InjectedConfigChannel (ADC_HandleTypeDef * hadc, ADC_InjectionConfTypeDef * sConfigInjected)

Function description

Configures for the selected ADC injected channel its corresponding rank in the sequencer and its sample time.

Parameters

- **hadc:** pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **sConfigInjected:** ADC configuration structure for injected channel.

Return values

- **None:**

HAL_ADCEX_MultiModeConfigChannel

Function name

HAL_StatusTypeDef HAL_ADCEX_MultiModeConfigChannel (ADC_HandleTypeDef * hadc,
ADC_MultiModeTypeDef * multimode)

Function description

Configures the ADC multi-mode.

Parameters

- **hadc**: pointer to a ADC_HandleTypeDef structure that contains the configuration information for the specified ADC.
- **multimode**: pointer to an ADC_MultiModeTypeDef structure that contains the configuration information for multimode.

Return values

- **HAL**: status

8.3 ADCEX Firmware driver defines

The following section lists the various define and macros of the module.

8.3.1

ADCEX

ADCEX

ADC Specific Channels

ADC_CHANNEL_TEMPSENSOR

ADC Common Mode

ADC_MODE_INDEPENDENT

ADC_DUALMODE_REGSIMULT_INJECSIMULT

ADC_DUALMODE_REGSIMULT_ALTERTRIG

ADC_DUALMODE_INJECSIMULT

ADC_DUALMODE_REGSIMULT

ADC_DUALMODE_INTERL

ADC_DUALMODE_ALTERTRIG

ADC_TRIPLEMODE_REGSIMULT_INJECSIMULT

ADC_TRIPLEMODE_REGSIMULT_AlterTrig

ADC_TRIPLEMODE_INJECSIMULT

ADC_TRIPLEMODE_REGSIMULT

ADC_TRIPLEMODE_INTERL

ADC_TRIPLEMODE_ALTERTRIG

ADC Direct Memory Access Mode For Multi Mode

ADC_DMAACCESSMODE_DISABLED

DMA mode disabled

ADC_DMAACCESSMODE_1

DMA mode 1 enabled (2 / 3 half-words one by one - 1 then 2 then 3)

ADC_DMAACCESSMODE_2

DMA mode 2 enabled (2 / 3 half-words by pairs - 2&1 then 1&3 then 3&2)

ADC_DMAACCESSMODE_3

DMA mode 3 enabled (2 / 3 bytes by pairs - 2&1 then 1&3 then 3&2)

ADC External Trigger Edge Injected**ADC_EXTERNALTRIGINJECCONVEDGE_NONE****ADC_EXTERNALTRIGINJECCONVEDGE_RISING****ADC_EXTERNALTRIGINJECCONVEDGE_FALLING****ADC_EXTERNALTRIGINJECCONVEDGE_RISINGFALLING*****ADC External Trigger Source Injected*****ADC_EXTERNALTRIGINJECCONV_T1_CC4****ADC_EXTERNALTRIGINJECCONV_T1_TRGO****ADC_EXTERNALTRIGINJECCONV_T2_CC1****ADC_EXTERNALTRIGINJECCONV_T2_TRGO****ADC_EXTERNALTRIGINJECCONV_T3_CC2****ADC_EXTERNALTRIGINJECCONV_T3_CC4****ADC_EXTERNALTRIGINJECCONV_T4_CC1****ADC_EXTERNALTRIGINJECCONV_T4_CC2****ADC_EXTERNALTRIGINJECCONV_T4_CC3****ADC_EXTERNALTRIGINJECCONV_T4_TRGO****ADC_EXTERNALTRIGINJECCONV_T5_CC4****ADC_EXTERNALTRIGINJECCONV_T5_TRGO****ADC_EXTERNALTRIGINJECCONV_T8_CC2****ADC_EXTERNALTRIGINJECCONV_T8_CC3****ADC_EXTERNALTRIGINJECCONV_T8_CC4****ADC_EXTERNALTRIGINJECCONV_EXT_IT15****ADC_INJECTED_SOFTWARE_START**

ADC Injected Rank`ADC_INJECTED_RANK_1``ADC_INJECTED_RANK_2``ADC_INJECTED_RANK_3``ADC_INJECTED_RANK_4`

9 HAL CAN Generic Driver

9.1 CAN Firmware driver registers structures

9.1.1 CAN_InitTypeDef

CAN_InitTypeDef is defined in the `stm32f2xx_hal_can.h`

Data Fields

- **uint32_t Prescaler**
- **uint32_t Mode**
- **uint32_t SyncJumpWidth**
- **uint32_t TimeSeg1**
- **uint32_t TimeSeg2**
- **FunctionalState TimeTriggeredMode**
- **FunctionalState AutoBusOff**
- **FunctionalState AutoWakeUp**
- **FunctionalState AutoRetransmission**
- **FunctionalState ReceiveFifoLocked**
- **FunctionalState TransmitFifoPriority**

Field Documentation

- **uint32_t CAN_InitTypeDef::Prescaler**
Specifies the length of a time quantum. This parameter must be a number between Min_Data = 1 and Max_Data = 1024.
- **uint32_t CAN_InitTypeDef::Mode**
Specifies the CAN operating mode. This parameter can be a value of [CAN_operating_mode](#)
- **uint32_t CAN_InitTypeDef::SyncJumpWidth**
Specifies the maximum number of time quanta the CAN hardware is allowed to lengthen or shorten a bit to perform resynchronization. This parameter can be a value of [CAN_synchronisation_jump_width](#)
- **uint32_t CAN_InitTypeDef::TimeSeg1**
Specifies the number of time quanta in Bit Segment 1. This parameter can be a value of [CAN_time_quantum_in_bit_segment_1](#)
- **uint32_t CAN_InitTypeDef::TimeSeg2**
Specifies the number of time quanta in Bit Segment 2. This parameter can be a value of [CAN_time_quantum_in_bit_segment_2](#)
- **FunctionalState CAN_InitTypeDef::TimeTriggeredMode**
Enable or disable the time triggered communication mode. This parameter can be set to ENABLE or DISABLE.
- **FunctionalState CAN_InitTypeDef::AutoBusOff**
Enable or disable the automatic bus-off management. This parameter can be set to ENABLE or DISABLE.
- **FunctionalState CAN_InitTypeDef::AutoWakeUp**
Enable or disable the automatic wake-up mode. This parameter can be set to ENABLE or DISABLE.
- **FunctionalState CAN_InitTypeDef::AutoRetransmission**
Enable or disable the non-automatic retransmission mode. This parameter can be set to ENABLE or DISABLE.
- **FunctionalState CAN_InitTypeDef::ReceiveFifoLocked**
Enable or disable the Receive FIFO Locked mode. This parameter can be set to ENABLE or DISABLE.
- **FunctionalState CAN_InitTypeDef::TransmitFifoPriority**
Enable or disable the transmit FIFO priority. This parameter can be set to ENABLE or DISABLE.

9.1.2

CAN_FilterTypeDef

CAN_FilterTypeDef is defined in the stm32f2xx_hal_can.h

Data Fields

- **uint32_t FilterIdHigh**
- **uint32_t FilterIdLow**
- **uint32_t FilterMaskIdHigh**
- **uint32_t FilterMaskIdLow**
- **uint32_t FilterFIFOAssignment**
- **uint32_t FilterBank**
- **uint32_t FilterMode**
- **uint32_t FilterScale**
- **uint32_t FilterActivation**
- **uint32_t SlaveStartFilterBank**

Field Documentation

- **uint32_t CAN_FilterTypeDef::FilterIdHigh**
Specifies the filter identification number (MSBs for a 32-bit configuration, first one for a 16-bit configuration). This parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t CAN_FilterTypeDef::FilterIdLow**
Specifies the filter identification number (LSBs for a 32-bit configuration, second one for a 16-bit configuration). This parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t CAN_FilterTypeDef::FilterMaskIdHigh**
Specifies the filter mask number or identification number, according to the mode (MSBs for a 32-bit configuration, first one for a 16-bit configuration). This parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t CAN_FilterTypeDef::FilterMaskIdLow**
Specifies the filter mask number or identification number, according to the mode (LSBs for a 32-bit configuration, second one for a 16-bit configuration). This parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t CAN_FilterTypeDef::FilterFIFOAssignment**
Specifies the FIFO (0 or 1U) which will be assigned to the filter. This parameter can be a value of [CAN_filter_FIFO](#)
- **uint32_t CAN_FilterTypeDef::FilterBank**
Specifies the filter bank which will be initialized. For single CAN instance(14 dedicated filter banks), this parameter must be a number between Min_Data = 0 and Max_Data = 13. For dual CAN instances(28 filter banks shared), this parameter must be a number between Min_Data = 0 and Max_Data = 27.
- **uint32_t CAN_FilterTypeDef::FilterMode**
Specifies the filter mode to be initialized. This parameter can be a value of [CAN_filter_mode](#)
- **uint32_t CAN_FilterTypeDef::FilterScale**
Specifies the filter scale. This parameter can be a value of [CAN_filter_scale](#)
- **uint32_t CAN_FilterTypeDef::FilterActivation**
Enable or disable the filter. This parameter can be a value of [CAN_filter_activation](#)
- **uint32_t CAN_FilterTypeDef::SlaveStartFilterBank**
Select the start filter bank for the slave CAN instance. For single CAN instances, this parameter is meaningless. For dual CAN instances, all filter banks with lower index are assigned to master CAN instance, whereas all filter banks with greater index are assigned to slave CAN instance. This parameter must be a number between Min_Data = 0 and Max_Data = 27.

9.1.3

CAN_TxHeaderTypeDef

CAN_TxHeaderTypeDef is defined in the stm32f2xx_hal_can.h

Data Fields

- **uint32_t StdId**
- **uint32_t ExtId**

- `uint32_t IDE`
- `uint32_t RTR`
- `uint32_t DLC`
- ***FunctionalState TransmitGlobalTime***

Field Documentation

- **`uint32_t CAN_TxHeaderTypeDef::StdId`**
Specifies the standard identifier. This parameter must be a number between Min_Data = 0 and Max_Data = 0x7FF.
- **`uint32_t CAN_TxHeaderTypeDef::ExtId`**
Specifies the extended identifier. This parameter must be a number between Min_Data = 0 and Max_Data = 0x1FFFFFFF.
- **`uint32_t CAN_TxHeaderTypeDef::IDE`**
Specifies the type of identifier for the message that will be transmitted. This parameter can be a value of [CAN_identifier_type](#)
- **`uint32_t CAN_TxHeaderTypeDef::RTR`**
Specifies the type of frame for the message that will be transmitted. This parameter can be a value of [CAN_remote_transmission_request](#)
- **`uint32_t CAN_TxHeaderTypeDef::DLC`**
Specifies the length of the frame that will be transmitted. This parameter must be a number between Min_Data = 0 and Max_Data = 8.
- ***FunctionalState CAN_TxHeaderTypeDef::TransmitGlobalTime***
Specifies whether the timestamp counter value captured on start of frame transmission, is sent in DATA6 and DATA7 replacing pData[6] and pData[7].

Note:

- : Time Triggered Communication Mode must be enabled.
- : DLC must be programmed as 8 bytes, in order these 2 bytes are sent. This parameter can be set to ENABLE or DISABLE.

9.1.4

CAN_RxHeaderTypeDef

CAN_RxHeaderTypeDef is defined in the `stm32f2xx_hal_can.h`

Data Fields

- `uint32_t StdId`
- `uint32_t ExtId`
- `uint32_t IDE`
- `uint32_t RTR`
- `uint32_t DLC`
- `uint32_t Timestamp`
- `uint32_t FilterMatchIndex`

Field Documentation

- **`uint32_t CAN_RxHeaderTypeDef::StdId`**
Specifies the standard identifier. This parameter must be a number between Min_Data = 0 and Max_Data = 0x7FF.
- **`uint32_t CAN_RxHeaderTypeDef::ExtId`**
Specifies the extended identifier. This parameter must be a number between Min_Data = 0 and Max_Data = 0x1FFFFFFF.
- **`uint32_t CAN_RxHeaderTypeDef::IDE`**
Specifies the type of identifier for the message that will be transmitted. This parameter can be a value of [CAN_identifier_type](#)
- **`uint32_t CAN_RxHeaderTypeDef::RTR`**
Specifies the type of frame for the message that will be transmitted. This parameter can be a value of [CAN_remote_transmission_request](#)

- **`uint32_t CAN_RxHeaderTypeDef::DLC`**
Specifies the length of the frame that will be transmitted. This parameter must be a number between Min_Data = 0 and Max_Data = 8.
- **`uint32_t CAN_RxHeaderTypeDef::Timestamp`**
Specifies the timestamp counter value captured on start of frame reception.
Note:
 - : Time Triggered Communication Mode must be enabled. This parameter must be a number between Min_Data = 0 and Max_Data = 0xFFFF.
- **`uint32_t CAN_RxHeaderTypeDef::FilterMatchIndex`**
Specifies the index of matching acceptance filter element. This parameter must be a number between Min_Data = 0 and Max_Data = 0xFF.

9.1.5

`__CAN_HandleTypeDef`

`__CAN_HandleTypeDef` is defined in the `stm32f2xx_hal_can.h`

Data Fields

- **`CAN_TypeDef * Instance`**
- **`CAN_InitTypeDef Init`**
- **`__IO HAL_CAN_StateTypeDef State`**
- **`__IO uint32_t ErrorCode`**

Field Documentation

- **`CAN_TypeDef* __CAN_HandleTypeDef::Instance`**
Register base address
- **`CAN_InitTypeDef __CAN_HandleTypeDef::Init`**
CAN required parameters
- **`__IO HAL_CAN_StateTypeDef __CAN_HandleTypeDef::State`**
CAN communication state
- **`__IO uint32_t __CAN_HandleTypeDef::ErrorCode`**
CAN Error code. This parameter can be a value of [CAN_Error_Code](#)

9.2

CAN Firmware driver API description

The following section lists the various functions of the CAN library.

9.2.1

How to use this driver

1. Initialize the CAN low level resources by implementing the `HAL_CAN_MspInit()`:
 - Enable the CAN interface clock using `__HAL_RCC_CANx_CLK_ENABLE()`
 - Configure CAN pins
 - Enable the clock for the CAN GPIOs
 - Configure CAN pins as alternate function open-drain
 - In case of using interrupts (e.g. `HAL_CAN_ActivateNotification()`)
 - Configure the CAN interrupt priority using `HAL_NVIC_SetPriority()`
 - Enable the CAN IRQ handler using `HAL_NVIC_EnableIRQ()`
 - In CAN IRQ handler, call `HAL_CAN_IRQHandler()`
2. Initialize the CAN peripheral using `HAL_CAN_Init()` function. This function resorts to `HAL_CAN_MspInit()` for low-level initialization.
3. Configure the reception filters using the following configuration functions:
 - `HAL_CAN_ConfigFilter()`
4. Start the CAN module using `HAL_CAN_Start()` function. At this level the node is active on the bus: it receive messages, and can send messages.

5. To manage messages transmission, the following Tx control functions can be used:
 - HAL_CAN_AddTxMessage() to request transmission of a new message.
 - HAL_CAN_AbortTxRequest() to abort transmission of a pending message.
 - HAL_CAN_GetTxMailboxesFreeLevel() to get the number of free Tx mailboxes.
 - HAL_CAN_IsTxMessagePending() to check if a message is pending in a Tx mailbox.
 - HAL_CAN_GetTxTimestamp() to get the timestamp of Tx message sent, if time triggered communication mode is enabled.
6. When a message is received into the CAN Rx FIFOs, it can be retrieved using the HAL_CAN_GetRxMessage() function. The function HAL_CAN_GetRxFifoFillLevel() allows to know how many Rx message are stored in the Rx Fifo.
7. Calling the HAL_CAN_Stop() function stops the CAN module.
8. The deinitialization is achieved with HAL_CAN_DeInit() function.

Polling mode operation

1. Reception:
 - Monitor reception of message using HAL_CAN_GetRxFifoFillLevel() until at least one message is received.
 - Then get the message using HAL_CAN_GetRxMessage().
2. Transmission:
 - Monitor the Tx mailboxes availability until at least one Tx mailbox is free, using HAL_CAN_GetTxMailboxesFreeLevel().
 - Then request transmission of a message using HAL_CAN_AddTxMessage().

Interrupt mode operation

1. Notifications are activated using HAL_CAN_ActivateNotification() function. Then, the process can be controlled through the available user callbacks: HAL_CAN_xxxCallback(), using same APIs HAL_CAN_GetRxMessage() and HAL_CAN_AddTxMessage().
2. Notifications can be deactivated using HAL_CAN_DeactivateNotification() function.
3. Special care should be taken for CAN_IT_RX_FIFO0_MSG_PENDING and CAN_IT_RX_FIFO1_MSG_PENDING notifications. These notifications trig the callbacks HAL_CAN_RxFIFO0MsgPendingCallback() and HAL_CAN_RxFIFO1MsgPendingCallback(). User has two possible options here.
 - Directly get the Rx message in the callback, using HAL_CAN_GetRxMessage().
 - Or deactivate the notification in the callback without getting the Rx message. The Rx message can then be got later using HAL_CAN_GetRxMessage(). Once the Rx message have been read, the notification can be activated again.

Sleep mode

1. The CAN peripheral can be put in sleep mode (low power), using HAL_CAN_RequestSleep(). The sleep mode will be entered as soon as the current CAN activity (transmission or reception of a CAN frame) will be completed.
2. A notification can be activated to be informed when the sleep mode will be entered.
3. It can be checked if the sleep mode is entered using HAL_CAN_IsSleepActive(). Note that the CAN state (accessible from the API HAL_CAN_GetState()) is HAL_CAN_STATE_SLEEP_PENDING as soon as the sleep mode request is submitted (the sleep mode is not yet entered), and become HAL_CAN_STATE_SLEEP_ACTIVE when the sleep mode is effective.
4. The wake-up from sleep mode can be triggered by two ways:
 - Using HAL_CAN_WakeUp(). When returning from this function, the sleep mode is exited (if return status is HAL_OK).
 - When a start of Rx CAN frame is detected by the CAN peripheral, if automatic wake up mode is enabled.

Callback registration

9.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

- HAL_CAN_Init : Initialize and configure the CAN.
- HAL_CAN_DeInit : De-initialize the CAN.
- HAL_CAN_MspInit : Initialize the CAN MSP.
- HAL_CAN_MspDeInit : DeInitialize the CAN MSP.

This section contains the following APIs:

- [HAL_CAN_Init](#)
- [HAL_CAN_DeInit](#)
- [HAL_CAN_MspInit](#)
- [HAL_CAN_MspDeInit](#)

9.2.3 Configuration functions

This section provides functions allowing to:

- HAL_CAN_ConfigFilter : Configure the CAN reception filters

This section contains the following APIs:

- [HAL_CAN_ConfigFilter](#)

9.2.4 Control functions

This section provides functions allowing to:

- HAL_CAN_Start : Start the CAN module
- HAL_CAN_Stop : Stop the CAN module
- HAL_CAN_RequestSleep : Request sleep mode entry.
- HAL_CAN_WakeUp : Wake up from sleep mode.
- HAL_CAN_IsSleepActive : Check is sleep mode is active.
- HAL_CAN_AddTxMessage : Add a message to the Tx mailboxes and activate the corresponding transmission request
- HAL_CAN_AbortTxRequest : Abort transmission request
- HAL_CAN_GetTxMailboxesFreeLevel : Return Tx mailboxes free level
- HAL_CAN_IsTxMessagePending : Check if a transmission request is pending on the selected Tx mailbox
- HAL_CAN_GetRxMessage : Get a CAN frame from the Rx FIFO
- HAL_CAN_GetRxFifoFillLevel : Return Rx FIFO fill level

This section contains the following APIs:

- [HAL_CAN_Start](#)
- [HAL_CAN_Stop](#)
- [HAL_CAN_RequestSleep](#)
- [HAL_CAN_WakeUp](#)
- [HAL_CAN_IsSleepActive](#)
- [HAL_CAN_AddTxMessage](#)
- [HAL_CAN_AbortTxRequest](#)
- [HAL_CAN_GetTxMailboxesFreeLevel](#)
- [HAL_CAN_IsTxMessagePending](#)
- [HAL_CAN_GetTxTimestamp](#)
- [HAL_CAN_GetRxMessage](#)
- [HAL_CAN_GetRxFifoFillLevel](#)

9.2.5 Interrupts management

This section provides functions allowing to:

- HAL_CAN_ActivateNotification : Enable interrupts

- HAL_CAN_DeactivateNotification : Disable interrupts
- HAL_CAN_IRQHandler : Handles CAN interrupt request

This section contains the following APIs:

- [HAL_CAN_ActivateNotification](#)
- [HAL_CAN_DeactivateNotification](#)
- [HAL_CAN_IRQHandler](#)

9.2.6 Peripheral State and Error functions

This subsection provides functions allowing to :

- HAL_CAN_GetState() : Return the CAN state.
- HAL_CAN_GetError() : Return the CAN error codes if any.
- HAL_CAN_ResetError(): Reset the CAN error codes if any.

This section contains the following APIs:

- [HAL_CAN_GetState](#)
- [HAL_CAN_GetError](#)
- [HAL_CAN_ResetError](#)

9.2.7 Detailed description of functions

HAL_CAN_Init

Function name

HAL_StatusTypeDef HAL_CAN_Init (CAN_HandleTypeDef * hcan)

Function description

Initializes the CAN peripheral according to the specified parameters in the CAN_InitStruct.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL:** status

HAL_CAN_DeInit

Function name

HAL_StatusTypeDef HAL_CAN_DeInit (CAN_HandleTypeDef * hcan)

Function description

Deinitializes the CAN peripheral registers to their default reset values.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL:** status

HAL_CAN_MspInit

Function name

void HAL_CAN_MspInit (CAN_HandleTypeDef * hcan)

Function description

Initializes the CAN MSP.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_MspDeInit

Function name

void HAL_CAN_MspDeInit (CAN_HandleTypeDef * hcan)

Function description

DeInitializes the CAN MSP.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_ConfigFilter

Function name

HAL_StatusTypeDef HAL_CAN_ConfigFilter (CAN_HandleTypeDef * hcan, CAN_FilterTypeDef * sFilterConfig)

Function description

Configures the CAN reception filter according to the specified parameters in the CAN_FilterInitStruct.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **sFilterConfig:** pointer to a CAN_FilterTypeDef structure that contains the filter configuration information.

Return values

- **None:**

HAL_CAN_Start

Function name

HAL_StatusTypeDef HAL_CAN_Start (CAN_HandleTypeDef * hcan)

Function description

Start the CAN module.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL:** status

HAL_CAN_Stop

Function name

HAL_StatusTypeDef HAL_CAN_Stop (CAN_HandleTypeDef * hcan)

Function description

Stop the CAN module and enable access to configuration registers.

Parameters

- **hcan**: pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL**: status

HAL_CAN_RequestSleep

Function name

HAL_StatusTypeDef HAL_CAN_RequestSleep (CAN_HandleTypeDef * hcan)

Function description

Request the sleep mode (low power) entry.

Parameters

- **hcan**: pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL**: status.

HAL_CAN_WakeUp

Function name

HAL_StatusTypeDef HAL_CAN_WakeUp (CAN_HandleTypeDef * hcan)

Function description

Wake up from sleep mode.

Parameters

- **hcan**: pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL**: status.

HAL_CAN_IsSleepActive

Function name

uint32_t HAL_CAN_IsSleepActive (CAN_HandleTypeDef * hcan)

Function description

Check is sleep mode is active.

Parameters

- **hcan**: pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **Status:**
 - 0 : Sleep mode is not active.
 - 1 : Sleep mode is active.

HAL_CAN_AddTxMessage

Function name

HAL_StatusTypeDef HAL_CAN_AddTxMessage (CAN_HandleTypeDef * hcan, CAN_TxHeaderTypeDef * pHeader, uint8_t aData, uint32_t * pTxMailbox)

Function description

Add a message to the first free Tx mailbox and activate the corresponding transmission request.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **pHeader:** pointer to a CAN_TxHeaderTypeDef structure.
- **aData:** array containing the payload of the Tx frame.
- **pTxMailbox:** pointer to a variable where the function will return the TxMailbox used to store the Tx message. This parameter can be a value of
 - CAN_Tx_Mailboxes.

Return values

- **HAL:** status

HAL_CAN_AbortTxRequest

Function name

HAL_StatusTypeDef HAL_CAN_AbortTxRequest (CAN_HandleTypeDef * hcan, uint32_t TxMailboxes)

Function description

Abort transmission requests.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **TxMailboxes:** List of the Tx Mailboxes to abort. This parameter can be any combination of
 - CAN_Tx_Mailboxes.

Return values

- **HAL:** status

HAL_CAN_GetTxMailboxesFreeLevel

Function name

uint32_t HAL_CAN_GetTxMailboxesFreeLevel (CAN_HandleTypeDef * hcan)

Function description

Return Tx Mailboxes free level: number of free Tx Mailboxes.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **Number:** of free Tx Mailboxes.

HAL_CAN_IsTxMessagePending

Function name

uint32_t HAL_CAN_IsTxMessagePending (CAN_HandleTypeDef * hcan, uint32_t TxMailboxes)

Function description

Check if a transmission request is pending on the selected Tx Mailboxes.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **TxMailboxes:** List of Tx Mailboxes to check. This parameter can be any combination of
 - CAN_Tx_Mailboxes.

Return values

- **Status:**
 - 0 : No pending transmission request on any selected Tx Mailboxes.
 - 1 : Pending transmission request on at least one of the selected Tx Mailbox.

HAL_CAN_GetTxTimestamp

Function name

uint32_t HAL_CAN_GetTxTimestamp (CAN_HandleTypeDef * hcan, uint32_t TxMailbox)

Function description

Return timestamp of Tx message sent, if time triggered communication mode is enabled.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **TxMailbox:** Tx Mailbox where the timestamp of message sent will be read. This parameter can be one value of
 - CAN_Tx_Mailboxes.

Return values

- **Timestamp:** of message sent from Tx Mailbox.

HAL_CAN_GetRxMessage

Function name

HAL_StatusTypeDef HAL_CAN_GetRxMessage (CAN_HandleTypeDef * hcan, uint32_t RxFifo, CAN_RxHeaderTypeDef * pHeader, uint8_t aData)

Function description

Get an CAN frame from the Rx FIFO zone into the message RAM.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **RxFifo:** Fifo number of the received message to be read. This parameter can be a value of
 - CAN_receive_FIFO_number.
- **pHeader:** pointer to a CAN_RxHeaderTypeDef structure where the header of the Rx frame will be stored.
- **aData:** array where the payload of the Rx frame will be stored.

Return values

- **HAL:** status

HAL_CAN_GetRxFifoFillLevel

Function name

uint32_t HAL_CAN_GetRxFifoFillLevel (CAN_HandleTypeDef * hcan, uint32_t RxFifo)

Function description

Return Rx FIFO fill level.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **RxFifo:** Rx FIFO. This parameter can be a value of
 - CAN_receive_FIFO_number.

Return values

- **Number:** of messages available in Rx FIFO.

HAL_CAN_ActivateNotification

Function name

HAL_StatusTypeDef HAL_CAN_ActivateNotification (CAN_HandleTypeDef * hcan, uint32_t ActiveITs)

Function description

Enable interrupts.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **ActiveITs:** indicates which interrupts will be enabled. This parameter can be any combination of
 - CAN_Interrupts.

Return values

- **HAL:** status

HAL_CAN_DeactivateNotification

Function name

HAL_StatusTypeDef HAL_CAN_DeactivateNotification (CAN_HandleTypeDef * hcan, uint32_t InactiveITs)

Function description

Disable interrupts.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.
- **InactiveITs:** indicates which interrupts will be disabled. This parameter can be any combination of
 - CAN_Interrupts.

Return values

- **HAL:** status

HAL_CAN_IRQHandler

Function name

void HAL_CAN_IRQHandler (CAN_HandleTypeDef * hcan)

Function description

Handles CAN interrupt request.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox0CompleteCallback

Function name

void HAL_CAN_TxMailbox0CompleteCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 0 complete callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox1CompleteCallback

Function name

void HAL_CAN_TxMailbox1CompleteCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 1 complete callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox2CompleteCallback

Function name

void HAL_CAN_TxMailbox2CompleteCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 2 complete callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox0AbortCallback

Function name

void HAL_CAN_TxMailbox0AbortCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 0 Cancellation callback.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox1AbortCallback

Function name

void HAL_CAN_TxMailbox1AbortCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 1 Cancellation callback.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_TxMailbox2AbortCallback

Function name

void HAL_CAN_TxMailbox2AbortCallback (CAN_HandleTypeDef * hcan)

Function description

Transmission Mailbox 2 Cancellation callback.

Parameters

- **hcan:** pointer to an CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_RxFifo0MsgPendingCallback

Function name

void HAL_CAN_RxFifo0MsgPendingCallback (CAN_HandleTypeDef * hcan)

Function description

Rx FIFO 0 message pending callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_RxFifo0FullCallback

Function name

void HAL_CAN_RxFifo0FullCallback (CAN_HandleTypeDef * hcan)

Function description

Rx FIFO 0 full callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_RxFifo1MsgPendingCallback

Function name

void HAL_CAN_RxFifo1MsgPendingCallback (CAN_HandleTypeDef * hcan)

Function description

Rx FIFO 1 message pending callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_RxFifo1FullCallback

Function name

void HAL_CAN_RxFifo1FullCallback (CAN_HandleTypeDef * hcan)

Function description

Rx FIFO 1 full callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_SleepCallback

Function name

void HAL_CAN_SleepCallback (CAN_HandleTypeDef * hcan)

Function description

Sleep callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_WakeUpFromRxMsgCallback

Function name

void HAL_CAN_WakeUpFromRxMsgCallback (CAN_HandleTypeDef * hcan)

Function description

WakeUp from Rx message callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_ErrorCallback

Function name

void HAL_CAN_ErrorCallback (CAN_HandleTypeDef * hcan)

Function description

Error CAN callback.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **None:**

HAL_CAN_GetState

Function name

HAL_CAN_StateTypeDef HAL_CAN_GetState (CAN_HandleTypeDef * hcan)

Function description

Return the CAN state.

Parameters

- **hcan:** pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL:** state

HAL_CAN_GetError

Function name

uint32_t HAL_CAN_GetError (CAN_HandleTypeDef * hcan)

Function description

Return the CAN error code.

Parameters

- **hcan**: pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **CAN**: Error Code

HAL_CAN_ResetError

Function name

HAL_StatusTypeDef HAL_CAN_ResetError (CAN_HandleTypeDef * hcan)

Function description

Reset the CAN error code.

Parameters

- **hcan**: pointer to a CAN_HandleTypeDef structure that contains the configuration information for the specified CAN.

Return values

- **HAL**: status

9.3 CAN Firmware driver defines

The following section lists the various define and macros of the module.

9.3.1 CAN

CAN

CAN Error Code

HAL_CAN_ERROR_NONE

No error

HAL_CAN_ERROR_EWG

Protocol Error Warning

HAL_CAN_ERROR_EPV

Error Passive

HAL_CAN_ERROR_BOF

Bus-off error

HAL_CAN_ERROR_STF

Stuff error

HAL_CAN_ERROR_FOR

Form error

HAL_CAN_ERROR_ACK

Acknowledgment error

HAL_CAN_ERROR_BR

Bit recessive error

HAL_CAN_ERROR_BD

Bit dominant error

HAL_CAN_ERROR_CRC

CRC error

HAL_CAN_ERROR_RX_FOV0

Rx FIFO0 overrun error

HAL_CAN_ERROR_RX_FOV1

Rx FIFO1 overrun error

HAL_CAN_ERROR_TX_ALST0

TxMailbox 0 transmit failure due to arbitration lost

HAL_CAN_ERROR_TX_TERR0

TxMailbox 1 transmit failure due to transmit error

HAL_CAN_ERROR_TX_ALST1

TxMailbox 0 transmit failure due to arbitration lost

HAL_CAN_ERROR_TX_TERR1

TxMailbox 1 transmit failure due to transmit error

HAL_CAN_ERROR_TX_ALST2

TxMailbox 0 transmit failure due to arbitration lost

HAL_CAN_ERROR_TX_TERR2

TxMailbox 1 transmit failure due to transmit error

HAL_CAN_ERROR_TIMEOUT

Timeout error

HAL_CAN_ERROR_NOT_INITIALIZED

Peripheral not initialized

HAL_CAN_ERROR_NOT_READY

Peripheral not ready

HAL_CAN_ERROR_NOT_STARTED

Peripheral not started

HAL_CAN_ERROR_PARAM

Parameter error

HAL_CAN_ERROR_INTERNAL

Internal error

CAN Exported Macros

__HAL_CAN_RESET_HANDLE_STATE

Description:

- Reset CAN handle state.

Parameters:

- `__HANDLE__`: CAN handle.

Return value:

- None

__HAL_CAN_ENABLE_IT

Description:

- Enable the specified CAN interrupts.

Parameters:

- `__HANDLE__`: CAN handle.
- `__INTERRUPT__`: CAN Interrupt sources to enable. This parameter can be any combination of
 - `CAN_Interrupts`

Return value:

- None

__HAL_CAN_DISABLE_IT

Description:

- Disable the specified CAN interrupts.

Parameters:

- `__HANDLE__`: CAN handle.
- `__INTERRUPT__`: CAN Interrupt sources to disable. This parameter can be any combination of
 - `CAN_Interrupts`

Return value:

- None

__HAL_CAN_GET_IT_SOURCE

Description:

- Check if the specified CAN interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the CAN Handle.
- `__INTERRUPT__`: specifies the CAN interrupt source to check. This parameter can be a value of
 - `CAN_Interrupts`

Return value:

- The: state of `__IT__` (TRUE or FALSE).

__HAL_CAN_GET_FLAG

Description:

- Check whether the specified CAN flag is set or not.

Parameters:

- `__HANDLE__`: specifies the CAN Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of
 - `CAN_flags`

Return value:

- The: state of `__FLAG__` (TRUE or FALSE).

__HAL_CAN_CLEAR_FLAG

Description:

- Clear the specified CAN pending flag.

Parameters:

- **__HANDLE__**: specifies the CAN Handle.
- **__FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - CAN_FLAG_RQCP0: Request complete MailBox 0 Flag
 - CAN_FLAG_TXOK0: Transmission OK MailBox 0 Flag
 - CAN_FLAG_ALST0: Arbitration Lost MailBox 0 Flag
 - CAN_FLAG_TERR0: Transmission error MailBox 0 Flag
 - CAN_FLAG_RQCP1: Request complete MailBox 1 Flag
 - CAN_FLAG_TXOK1: Transmission OK MailBox 1 Flag
 - CAN_FLAG_ALST1: Arbitration Lost MailBox 1 Flag
 - CAN_FLAG_TERR1: Transmission error MailBox 1 Flag
 - CAN_FLAG_RQCP2: Request complete MailBox 2 Flag
 - CAN_FLAG_TXOK2: Transmission OK MailBox 2 Flag
 - CAN_FLAG_ALST2: Arbitration Lost MailBox 2 Flag
 - CAN_FLAG_TERR2: Transmission error MailBox 2 Flag
 - CAN_FLAG_FF0: RX FIFO 0 Full Flag
 - CAN_FLAG_FOV0: RX FIFO 0 Overrun Flag
 - CAN_FLAG_FF1: RX FIFO 1 Full Flag
 - CAN_FLAG_FOV1: RX FIFO 1 Overrun Flag
 - CAN_FLAG_WKUI: Wake up Interrupt Flag
 - CAN_FLAG_SLAKI: Sleep acknowledge Interrupt Flag

Return value:

- None

CAN Filter Activation

CAN_FILTER_DISABLE

Disable filter

CAN_FILTER_ENABLE

Enable filter

CAN Filter FIFO

CAN_FILTER_FIFO0

Filter FIFO 0 assignment for filter x

CAN_FILTER_FIFO1

Filter FIFO 1 assignment for filter x

CAN Filter Mode

CAN_FILTERMODE_IDMASK

Identifier mask mode

CAN_FILTERMODE_IDLIST

Identifier list mode

CAN Filter Scale

CAN_FILTERSCALE_16BIT

Two 16-bit filters

CAN_FILTERSCALE_32BIT

One 32-bit filter

CAN Flags**CAN_FLAG_RQCP0**

Request complete MailBox 0 flag

CAN_FLAG_TXOK0

Transmission OK MailBox 0 flag

CAN_FLAG_ALST0

Arbitration Lost MailBox 0 flag

CAN_FLAG_TERR0

Transmission error MailBox 0 flag

CAN_FLAG_RQCP1

Request complete MailBox1 flag

CAN_FLAG_TXOK1

Transmission OK MailBox 1 flag

CAN_FLAG_ALST1

Arbitration Lost MailBox 1 flag

CAN_FLAG_TERR1

Transmission error MailBox 1 flag

CAN_FLAG_RQCP2

Request complete MailBox2 flag

CAN_FLAG_TXOK2

Transmission OK MailBox 2 flag

CAN_FLAG_ALST2

Arbitration Lost MailBox 2 flag

CAN_FLAG_TERR2

Transmission error MailBox 2 flag

CAN_FLAG_TME0

Transmit mailbox 0 empty flag

CAN_FLAG_TME1

Transmit mailbox 1 empty flag

CAN_FLAG_TME2

Transmit mailbox 2 empty flag

CAN_FLAG_LOW0

Lowest priority mailbox 0 flag

CAN_FLAG_LOW1

Lowest priority mailbox 1 flag

CAN_FLAG_LOW2

Lowest priority mailbox 2 flag

CAN_FLAG_FF0

RX FIFO 0 Full flag

CAN_FLAG_FOV0

RX FIFO 0 Overrun flag

CAN_FLAG_FF1

RX FIFO 1 Full flag

CAN_FLAG_FOV1

RX FIFO 1 Overrun flag

CAN_FLAG_INAK

Initialization acknowledge flag

CAN_FLAG_SLAKE

Sleep acknowledge flag

CAN_FLAG_ERRI

Error flag

CAN_FLAG_WKU

Wake up interrupt flag

CAN_FLAG_SLAKEI

Sleep acknowledge interrupt flag

CAN_FLAG_EWG

Error warning flag

CAN_FLAG_EPV

Error passive flag

CAN_FLAG_BOFF

Bus-Off flag

CAN Identifier Type

CAN_ID_STD

Standard Id

CAN_ID_EXT

Extended Id

CAN InitStatus

CAN_INITSTATUS_FAILED

CAN initialization failed

CAN_INITSTATUS_SUCCESS

CAN initialization OK

CAN Interrupts

CAN_IT_TX_MAILBOX_EMPTY

Transmit mailbox empty interrupt

CAN_IT_RX_FIFO0_MSG_PENDING

FIFO 0 message pending interrupt

CAN_IT_RX_FIFO0_FULL

FIFO 0 full interrupt

CAN_IT_RX_FIFO0_OVERRUN

FIFO 0 overrun interrupt

CAN_IT_RX_FIFO1_MSG_PENDING

FIFO 1 message pending interrupt

CAN_IT_RX_FIFO1_FULL

FIFO 1 full interrupt

CAN_IT_RX_FIFO1_OVERRUN

FIFO 1 overrun interrupt

CAN_IT_WAKEUP

Wake-up interrupt

CAN_IT_SLEEP_ACK

Sleep acknowledge interrupt

CAN_IT_ERROR_WARNING

Error warning interrupt

CAN_IT_ERROR_PASSIVE

Error passive interrupt

CAN_IT_BUSOFF

Bus-off interrupt

CAN_IT_LAST_ERROR_CODE

Last error code interrupt

CAN_IT_ERROR

Error Interrupt

CAN Operating Mode

CAN_MODE_NORMAL

Normal mode

CAN_MODE_LOOPBACK

Loopback mode

CAN_MODE_SILENT

Silent mode

CAN_MODE_SILENT_LOOPBACK

Loopback combined with silent mode

CAN Receive FIFO Number

CAN_RX_FIFO0

CAN receive FIFO 0

CAN_RX_FIFO1

CAN receive FIFO 1

CAN Remote Transmission Request

CAN_RTR_DATA

Data frame

CAN_RTR_REMOTE

Remote frame

CAN Synchronization Jump Width**CAN_SJW_1TQ**

1 time quantum

CAN_SJW_2TQ

2 time quantum

CAN_SJW_3TQ

3 time quantum

CAN_SJW_4TQ

4 time quantum

CAN Time Quantum in Bit Segment 1**CAN_BS1_1TQ**

1 time quantum

CAN_BS1_2TQ

2 time quantum

CAN_BS1_3TQ

3 time quantum

CAN_BS1_4TQ

4 time quantum

CAN_BS1_5TQ

5 time quantum

CAN_BS1_6TQ

6 time quantum

CAN_BS1_7TQ

7 time quantum

CAN_BS1_8TQ

8 time quantum

CAN_BS1_9TQ

9 time quantum

CAN_BS1_10TQ

10 time quantum

CAN_BS1_11TQ

11 time quantum

CAN_BS1_12TQ

12 time quantum

CAN_BS1_13TQ

13 time quantum

CAN_BS1_14TQ

14 time quantum

CAN_BS1_15TQ

15 time quantum

CAN_BS1_16TQ

16 time quantum

CAN Time Quantum in Bit Segment 2**CAN_BS2_1TQ**

1 time quantum

CAN_BS2_2TQ

2 time quantum

CAN_BS2_3TQ

3 time quantum

CAN_BS2_4TQ

4 time quantum

CAN_BS2_5TQ

5 time quantum

CAN_BS2_6TQ

6 time quantum

CAN_BS2_7TQ

7 time quantum

CAN_BS2_8TQ

8 time quantum

CAN Tx Mailboxes**CAN_TX_MAILBOX0**

Tx Mailbox 0

CAN_TX_MAILBOX1

Tx Mailbox 1

CAN_TX_MAILBOX2

Tx Mailbox 2

10 HAL CORTEX Generic Driver

10.1 CORTEX Firmware driver registers structures

10.1.1 MPU_Region_InitTypeDef

MPU_Region_InitTypeDef is defined in the `stm32f2xx_hal_cortex.h`

Data Fields

- **uint8_t Enable**
- **uint8_t Number**
- **uint32_t BaseAddress**
- **uint8_t Size**
- **uint8_t SubRegionDisable**
- **uint8_t TypeExtField**
- **uint8_t AccessPermission**
- **uint8_t DisableExec**
- **uint8_t IsShareable**
- **uint8_t IsCacheable**
- **uint8_t IsBufferable**

Field Documentation

- **uint8_t MPU_Region_InitTypeDef::Enable**
Specifies the status of the region. This parameter can be a value of [CORTEX_MPU_Region_Enable](#)
- **uint8_t MPU_Region_InitTypeDef::Number**
Specifies the number of the region to protect. This parameter can be a value of [CORTEX_MPU_Region_Number](#)
- **uint32_t MPU_Region_InitTypeDef::BaseAddress**
Specifies the base address of the region to protect.
- **uint8_t MPU_Region_InitTypeDef::Size**
Specifies the size of the region to protect. This parameter can be a value of [CORTEX_MPU_Region_Size](#)
- **uint8_t MPU_Region_InitTypeDef::SubRegionDisable**
Specifies the number of the subregion protection to disable. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0xFF
- **uint8_t MPU_Region_InitTypeDef::TypeExtField**
Specifies the TEX field level. This parameter can be a value of [CORTEX_MPU_TEX_Levels](#)
- **uint8_t MPU_Region_InitTypeDef::AccessPermission**
Specifies the region access permission type. This parameter can be a value of [CORTEX_MPU_Region_Permission_Attributes](#)
- **uint8_t MPU_Region_InitTypeDef::DisableExec**
Specifies the instruction access status. This parameter can be a value of [CORTEX_MPU_Instruction_Access](#)
- **uint8_t MPU_Region_InitTypeDef::IsShareable**
Specifies the shareability status of the protected region. This parameter can be a value of [CORTEX_MPU_Access_Shareable](#)
- **uint8_t MPU_Region_InitTypeDef::IsCacheable**
Specifies the cacheable status of the region protected. This parameter can be a value of [CORTEX_MPU_Access_Cacheable](#)
- **uint8_t MPU_Region_InitTypeDef::IsBufferable**
Specifies the bufferable status of the protected region. This parameter can be a value of [CORTEX_MPU_Access_Bufferable](#)

10.2 CORTEX Firmware driver API description

The following section lists the various functions of the CORTEX library.

10.2.1 How to use this driver

How to configure Interrupts using CORTEX HAL driver

This section provides functions allowing to configure the NVIC interrupts (IRQ). The Cortex-M3 exceptions are managed by CMSIS functions.

1. Configure the NVIC Priority Grouping using HAL_NVIC_SetPriorityGrouping() function according to the following table.
2. Configure the priority of the selected IRQ Channels using HAL_NVIC_SetPriority().
3. Enable the selected IRQ Channels using HAL_NVIC_EnableIRQ().
4. please refer to programming manual for details in how to configure priority.

Note: When the NVIC_PRIORITYGROUP_0 is selected, IRQ preemption is no more possible. The pending IRQ priority will be managed only by the sub priority.

Note: IRQ priority order (sorted by highest to lowest priority):

- Lowest preemption priority
- Lowest sub priority
- Lowest hardware priority (IRQ number)

How to configure SysTick using CORTEX HAL driver

Setup SysTick Timer for time base.

- The HAL_SYSTICK_Config() function calls the SysTick_Config() function which is a CMSIS function that:
 - Configures the SysTick Reload register with value passed as function parameter.
 - Configures the SysTick IRQ priority to the lowest value 0x0F.
 - Resets the SysTick Counter register.
 - Configures the SysTick Counter clock source to be Core Clock Source (HCLK).
 - Enables the SysTick Interrupt.
 - Starts the SysTick Counter.
- You can change the SysTick Clock source to be HCLK_Div8 by calling the macro __HAL_CORTEX_SYSTICKCLK_CONFIG(SYSTICK_CLKSOURCE_HCLK_DIV8) just after the HAL_SYSTICK_Config() function call. The __HAL_CORTEX_SYSTICKCLK_CONFIG() macro is defined inside the stm32f2xx_hal_cortex.h file.
- You can change the SysTick IRQ priority by calling the HAL_NVIC_SetPriority(SysTick_IRQn,...) function just after the HAL_SYSTICK_Config() function call. The HAL_NVIC_SetPriority() call the NVIC_SetPriority() function which is a CMSIS function.
- To adjust the SysTick time base, use the following formula: Reload Value = SysTick Counter Clock (Hz) x Desired Time base (s)
 - Reload Value is the parameter to be passed for HAL_SYSTICK_Config() function
 - Reload Value should not exceed 0xFFFFFF

10.2.2 Initialization and de-initialization functions

This section provides the CORTEX HAL driver functions allowing to configure Interrupts SysTick functionalities

This section contains the following APIs:

- [HAL_NVIC_SetPriorityGrouping](#)
- [HAL_NVIC_SetPriority](#)
- [HAL_NVIC_EnableIRQ](#)
- [HAL_NVIC_DisableIRQ](#)
- [HAL_NVIC_SystemReset](#)
- [HAL_SYSTICK_Config](#)

10.2.3 Peripheral Control functions

This subsection provides a set of functions allowing to control the CORTEX (NVIC, SYSTICK, MPU) functionalities.

This section contains the following APIs:

- [*HAL_MPU_Disable*](#)
- [*HAL_MPU_Enable*](#)
- [*HAL_MPU_ConfigRegion*](#)
- [*HAL_NVIC_GetPriorityGrouping*](#)
- [*HAL_NVIC_GetPriority*](#)
- [*HAL_NVIC_SetPendingIRQ*](#)
- [*HAL_NVIC_GetPendingIRQ*](#)
- [*HAL_NVIC_ClearPendingIRQ*](#)
- [*HAL_NVIC_GetActive*](#)
- [*HAL_SYSTICK_CLKSourceConfig*](#)
- [*HAL_SYSTICK_IRQHandler*](#)
- [*HAL_SYSTICK_Callback*](#)

10.2.4 Detailed description of functions

HAL_NVIC_SetPriorityGrouping

Function name

void HAL_NVIC_SetPriorityGrouping (uint32_t PriorityGroup)

Function description

Sets the priority grouping field (preemption priority and subpriority) using the required unlock sequence.

Parameters

- **PriorityGroup:** The priority grouping bits length. This parameter can be one of the following values:
 - NVIC_PRIORITYGROUP_0: 0 bits for preemption priority 4 bits for subpriority
 - NVIC_PRIORITYGROUP_1: 1 bits for preemption priority 3 bits for subpriority
 - NVIC_PRIORITYGROUP_2: 2 bits for preemption priority 2 bits for subpriority
 - NVIC_PRIORITYGROUP_3: 3 bits for preemption priority 1 bits for subpriority
 - NVIC_PRIORITYGROUP_4: 4 bits for preemption priority 0 bits for subpriority

Return values

- **None:**

Notes

- When the NVIC_PriorityGroup_0 is selected, IRQ preemption is no more possible. The pending IRQ priority will be managed only by the subpriority.

HAL_NVIC_SetPriority

Function name

void HAL_NVIC_SetPriority (IRQn_Type IRQn, uint32_t PreemptPriority, uint32_t SubPriority)

Function description

Sets the priority of an interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxx.h))
- **PreemptPriority:** The preemption priority for the IRQn channel. This parameter can be a value between 0 and 15 A lower priority value indicates a higher priority
- **SubPriority:** the subpriority level for the IRQ channel. This parameter can be a value between 0 and 15 A lower priority value indicates a higher priority.

Return values

- **None:**

HAL_NVIC_EnableIRQ

Function name

```
void HAL_NVIC_EnableIRQ (IRQn_Type IRQn)
```

Function description

Enables a device specific interrupt in the NVIC interrupt controller.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxx.h))

Return values

- **None:**

Notes

- To configure interrupts priority correctly, the NVIC_PriorityGroupConfig() function should be called before.

HAL_NVIC_DisableIRQ

Function name

```
void HAL_NVIC_DisableIRQ (IRQn_Type IRQn)
```

Function description

Disables a device specific interrupt in the NVIC interrupt controller.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxx.h))

Return values

- **None:**

HAL_NVIC_SystemReset

Function name

```
void HAL_NVIC_SystemReset (void )
```

Function description

Initiates a system reset request to reset the MCU.

Return values

- **None:**

HAL_SYSTICK_Config

Function name

uint32_t HAL_SYSTICK_Config (uint32_t TicksNumb)

Function description

Initializes the System Timer and its interrupt, and starts the System Tick Timer.

Parameters

- **TicksNumb:** Specifies the ticks Number of ticks between two interrupts.

Return values

- **status:** - 0 Function succeeded.
– 1 Function failed.

HAL_NVIC_GetPriorityGrouping

Function name

uint32_t HAL_NVIC_GetPriorityGrouping (void)

Function description

Gets the priority grouping field from the NVIC Interrupt Controller.

Return values

- **Priority:** grouping field (SCB->AIRCR [10:8] PRIGROUP field)

HAL_NVIC_GetPriority

Function name

void HAL_NVIC_GetPriority (IRQn_Type IRQn, uint32_t PriorityGroup, uint32_t * pPreemptPriority, uint32_t * pSubPriority)

Function description

Gets the priority of an interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxx.h))
- **PriorityGroup:** the priority grouping bits length. This parameter can be one of the following values:
 - NVIC_PRIORITYGROUP_0: 0 bits for preemption priority 4 bits for subpriority
 - NVIC_PRIORITYGROUP_1: 1 bits for preemption priority 3 bits for subpriority
 - NVIC_PRIORITYGROUP_2: 2 bits for preemption priority 2 bits for subpriority
 - NVIC_PRIORITYGROUP_3: 3 bits for preemption priority 1 bits for subpriority
 - NVIC_PRIORITYGROUP_4: 4 bits for preemption priority 0 bits for subpriority
- **pPreemptPriority:** Pointer on the Preemptive priority value (starting from 0).
- **pSubPriority:** Pointer on the Subpriority value (starting from 0).

Return values

- **None:**

HAL_NVIC_GetPendingIRQ

Function name

uint32_t HAL_NVIC_GetPendingIRQ (IRQn_Type IRQn)

Function description

Gets Pending Interrupt (reads the pending register in the NVIC and returns the pending bit for the specified interrupt).

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxxx.h))

Return values

- **status:** - 0 Interrupt status is not pending.
– 1 Interrupt status is pending.

HAL_NVIC_SetPendingIRQ

Function name

void HAL_NVIC_SetPendingIRQ (IRQn_Type IRQn)

Function description

Sets Pending bit of an external interrupt.

Parameters

- **IRQn:** External interrupt number This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxxx.h))

Return values

- **None:**

HAL_NVIC_ClearPendingIRQ

Function name

void HAL_NVIC_ClearPendingIRQ (IRQn_Type IRQn)

Function description

Clears the pending bit of an external interrupt.

Parameters

- **IRQn:** External interrupt number. This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxxx.h))

Return values

- **None:**

HAL_NVIC_GetActive

Function name

uint32_t HAL_NVIC_GetActive (IRQn_Type IRQn)

Function description

Gets active interrupt (reads the active register in NVIC and returns the active bit).

Parameters

- **IRQn:** External interrupt number This parameter can be an enumerator of IRQn_Type enumeration (For the complete STM32 Devices IRQ Channels list, please refer to the appropriate CMSIS device file (stm32f2xxxx.h))

Return values

- **status:** - 0 Interrupt status is not pending.
– 1 Interrupt status is pending.

HAL_SYSTICK_CLKSourceConfig

Function name

```
void HAL_SYSTICK_CLKSourceConfig (uint32_t CLKSource)
```

Function description

Configures the SysTick clock source.

Parameters

- **CLKSource:** specifies the SysTick clock source. This parameter can be one of the following values:
 - SYSTICK_CLKSOURCE_HCLK_DIV8: AHB clock divided by 8 selected as SysTick clock source.
 - SYSTICK_CLKSOURCE_HCLK: AHB clock selected as SysTick clock source.

Return values

- **None:**

HAL_SYSTICK_IRQHandler

Function name

```
void HAL_SYSTICK_IRQHandler (void )
```

Function description

This function handles SYSTICK interrupt request.

Return values

- **None:**

HAL_SYSTICK_Callback

Function name

```
void HAL_SYSTICK_Callback (void )
```

Function description

SYSTICK callback.

Return values

- **None:**

HAL_MPU_Enable

Function name

```
void HAL_MPU_Enable (uint32_t MPU_Control)
```

Function description

Enable the MPU.

Parameters

- **MPU_Control:** Specifies the control mode of the MPU during hard fault, NMI, FAULTMASK and privileged access to the default memory This parameter can be one of the following values:
 - MPU_HFNMI_PRIVDEF_NONE
 - MPU_HARDFAULT_NMI
 - MPU_PRIVILEGED_DEFAULT
 - MPU_HFNMI_PRIVDEF

Return values

- **None:**

HAL_MPU_Disable

Function name

void HAL_MPU_Disable (void)

Function description

Disables the MPU.

Return values

- **None:**

HAL_MPU_ConfigRegion

Function name

void HAL_MPU_ConfigRegion (MPU_Region_InitTypeDef * MPU_Init)

Function description

Initializes and configures the Region and the memory to be protected.

Parameters

- **MPU_Init:** Pointer to a MPU_Region_InitTypeDef structure that contains the initialization and configuration information.

Return values

- **None:**

10.3 CORTEx Firmware driver defines

The following section lists the various define and macros of the module.

10.3.1 CORTEx

CORTEx

CORTEx MPU Instruction Access Bufferable

MPU_ACCESS_BUFFERABLE

MPU_ACCESS_NOT_BUFFERABLE

CORTEx MPU Instruction Access Cacheable

MPU_ACCESS_CACHEABLE

MPU_ACCESS_NOT_CACHEABLE

CORTEx MPU Instruction Access Shareable

MPU_ACCESS_SHAREABLE

MPU_ACCESS_NOT_SHAREABLE

MPU HFNMI and PRIVILEGED Access control

MPU_HFNMI_PRIVDEF_NONE

MPU_HARDFAULT_NMI

MPU_PRIVILEGED_DEFAULT

MPU_HFNMI_PRIVDEF

CORTEx MPU Instruction Access

MPU_INSTRUCTION_ACCESS_ENABLE

MPU_INSTRUCTION_ACCESS_DISABLE

CORTEx MPU Region Enable

MPU_REGION_ENABLE

MPU_REGION_DISABLE

CORTEx MPU Region Number

MPU_REGION_NUMBER0

MPU_REGION_NUMBER1

MPU_REGION_NUMBER2

MPU_REGION_NUMBER3

MPU_REGION_NUMBER4

MPU_REGION_NUMBER5

MPU_REGION_NUMBER6

MPU_REGION_NUMBER7

CORTEx MPU Region Permission Attributes

MPU_REGION_NO_ACCESS

MPU_REGION_PRIV_RW

MPU_REGION_PRIV_RW_URO

MPU_REGION_FULL_ACCESS

MPU_REGION_PRIV_RO

MPU_REGION_PRIV_RO_URO

CORTEx MPU Region Size

MPU_REGION_SIZE_32B

MPU_REGION_SIZE_64B

MPU_REGION_SIZE_128B

MPU_REGION_SIZE_256B

MPU_REGION_SIZE_512B

MPU_REGION_SIZE_1KB

MPU_REGION_SIZE_2KB

MPU_REGION_SIZE_4KB

MPU_REGION_SIZE_8KB

MPU_REGION_SIZE_16KB

MPU_REGION_SIZE_32KB

MPU_REGION_SIZE_64KB

MPU_REGION_SIZE_128KB

MPU_REGION_SIZE_256KB

MPU_REGION_SIZE_512KB

MPU_REGION_SIZE_1MB

MPU_REGION_SIZE_2MB

MPU_REGION_SIZE_4MB

MPU_REGION_SIZE_8MB

MPU_REGION_SIZE_16MB

MPU_REGION_SIZE_32MB

MPU_REGION_SIZE_64MB

MPU_REGION_SIZE_128MB

MPU_REGION_SIZE_256MB

MPU_REGION_SIZE_512MB

MPU_REGION_SIZE_1GB

MPU_REGION_SIZE_2GB

MPU_REGION_SIZE_4GB

MPU TEX Levels

MPU_TEX_LEVEL0

MPU_TEX_LEVEL1

MPU_TEX_LEVEL2

CORTEx Preemption Priority Group

NVIC_PRIORITYGROUP_0

0 bits for pre-emption priority 4 bits for subpriority

NVIC_PRIORITYGROUP_1

1 bits for pre-emption priority 3 bits for subpriority

NVIC_PRIORITYGROUP_2

2 bits for pre-emption priority 2 bits for subpriority

NVIC_PRIORITYGROUP_3

3 bits for pre-emption priority 1 bits for subpriority

NVIC_PRIORITYGROUP_4

4 bits for pre-emption priority 0 bits for subpriority

CORTEx SysTick clock source

SYSTICK_CLKSOURCE_HCLK_DIV8

SYSTICK_CLKSOURCE_HCLK

11 HAL CRC Generic Driver

11.1 CRC Firmware driver registers structures

11.1.1 CRC_HandleTypeDef

CRC_HandleTypeDef is defined in the stm32f2xx_hal_crc.h

Data Fields

- **CRC_TypeDef * Instance**
- **HAL_LockTypeDef Lock**
- **__IO HAL_CRC_StateTypeDef State**

Field Documentation

- **CRC_TypeDef* CRC_HandleTypeDef::Instance**
Register base address
- **HAL_LockTypeDef CRC_HandleTypeDef::Lock**
CRC Locking object
- **__IO HAL_CRC_StateTypeDef CRC_HandleTypeDef::State**
CRC communication state

11.2 CRC Firmware driver API description

The following section lists the various functions of the CRC library.

11.2.1 How to use this driver

- Enable CRC AHB clock using `__HAL_RCC_CRC_CLK_ENABLE();`
- Initialize CRC calculator
 - specify generating polynomial (peripheral default or non-default one)
 - specify initialization value (peripheral default or non-default one)
 - specify input data format
 - specify input or output data inversion mode if any
- Use `HAL_CRC_Accumulate()` function to compute the CRC value of the input data buffer starting with the previously computed CRC as initialization value
- Use `HAL_CRC_Calculate()` function to compute the CRC value of the input data buffer starting with the defined initialization value (default or non-default) to initiate CRC calculation

11.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize the CRC according to the specified parameters in the `CRC_InitTypeDef` and create the associated handle
- DeInitialize the CRC peripheral
- Initialize the CRC MSP (MCU Specific Package)
- DeInitialize the CRC MSP

This section contains the following APIs:

- [*HAL_CRC_Init*](#)
- [*HAL_CRC_DeInit*](#)
- [*HAL_CRC_MspInit*](#)
- [*HAL_CRC_MspDeInit*](#)

11.2.3 Peripheral Control functions

This section provides functions allowing to:

- compute the 32-bit CRC value of a 32-bit data buffer using combination of the previous CRC value and the new one.

or

- compute the 32-bit CRC value of a 32-bit data buffer independently of the previous CRC value.

This section contains the following APIs:

- [HAL_CRC_Accumulate](#)
- [HAL_CRC_Calculate](#)

11.2.4 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral.

This section contains the following APIs:

- [HAL_CRC_GetState](#)

11.2.5 Detailed description of functions

HAL_CRC_Init

Function name

HAL_StatusTypeDef HAL_CRC_Init (CRC_HandleTypeDef * hcrc)

Function description

Initialize the CRC according to the specified parameters in the CRC_InitTypeDef and create the associated handle.

Parameters

- **hcrc**: CRC handle

Return values

- **HAL**: status

HAL_CRC_DeInit

Function name

HAL_StatusTypeDef HAL_CRC_DeInit (CRC_HandleTypeDef * hcrc)

Function description

DeInitialize the CRC peripheral.

Parameters

- **hcrc**: CRC handle

Return values

- **HAL**: status

HAL_CRC_MspInit

Function name

void HAL_CRC_MspInit (CRC_HandleTypeDef * hcrc)

Function description

Initializes the CRC MSP.

Parameters

- **hcrc**: CRC handle

Return values

- **None:**

HAL_CRC_MspDeInit

Function name

void HAL_CRC_MspDeInit (CRC_HandleTypeDef * hcrc)

Function description

Deinitialize the CRC MSP.

Parameters

- **hcrc:** CRC handle

Return values

- **None:**

HAL_CRC_Accumulate

Function name

uint32_t HAL_CRC_Accumulate (CRC_HandleTypeDef * hcrc, uint32_t pBuffer, uint32_t BufferLength)

Function description

Compute the 32-bit CRC value of a 32-bit data buffer starting with the previously computed CRC as initialization value.

Parameters

- **hcrc:** CRC handle
- **pBuffer:** pointer to the input data buffer.
- **BufferLength:** input data buffer length (number of uint32_t words).

Return values

- **uint32_t:** CRC (returned value LSBs for CRC shorter than 32 bits)

HAL_CRC_Calculate

Function name

uint32_t HAL_CRC_Calculate (CRC_HandleTypeDef * hcrc, uint32_t pBuffer, uint32_t BufferLength)

Function description

Compute the 32-bit CRC value of a 32-bit data buffer starting with hcrc->Instance->INIT as initialization value.

Parameters

- **hcrc:** CRC handle
- **pBuffer:** pointer to the input data buffer.
- **BufferLength:** input data buffer length (number of uint32_t words).

Return values

- **uint32_t:** CRC (returned value LSBs for CRC shorter than 32 bits)

HAL_CRC_GetState

Function name

HAL_CRC_StateTypeDef HAL_CRC_GetState (CRC_HandleTypeDef * hcrc)

Function description

Return the CRC handle state.

Parameters

- **hcrc**: CRC handle

Return values

- **HAL**: state

11.3 CRC Firmware driver defines

The following section lists the various define and macros of the module.

11.3.1 CRC

CRC

CRC Exported Macros

__HAL_CRC_RESET_HANDLE_STATE

Description:

- Reset CRC handle state.

Parameters:

- **__HANDLE__**: CRC handle.

Return value:

- None

__HAL_CRC_DR_RESET

Description:

- Reset CRC Data Register.

Parameters:

- **__HANDLE__**: CRC handle

Return value:

- None

__HAL_CRC_SET_IDR

Description:

- Store data in the Independent Data (ID) register.

Parameters:

- **__HANDLE__**: CRC handle
- **__VALUE__**: Value to be stored in the ID register

Return value:

- None

Notes:

- Refer to the Reference Manual to get the authorized **__VALUE__** length in bits

__HAL_CRC_GET_IDR**Description:**

- Return the data stored in the Independent Data (ID) register.

Parameters:

- `__HANDLE__`: CRC handle

Return value:

- Value: of the ID register

Notes:

- Refer to the Reference Manual to get the authorized `__VALUE__` length in bits

12 HAL DAC Generic Driver

12.1 DAC Firmware driver registers structures

12.1.1 DAC_HandleTypeDef

DAC_HandleTypeDef is defined in the stm32f2xx_hal_dac.h

Data Fields

- **DAC_TypeDef * Instance**
- **__IO HAL_DAC_StateTypeDef State**
- **HAL_LockTypeDef Lock**
- **DMA_HandleTypeDef * DMA_Handle1**
- **DMA_HandleTypeDef * DMA_Handle2**
- **__IO uint32_t ErrorCode**

Field Documentation

- **DAC_TypeDef* DAC_HandleTypeDef::Instance**
Register base address
- **__IO HAL_DAC_StateTypeDef DAC_HandleTypeDef::State**
DAC communication state
- **HAL_LockTypeDef DAC_HandleTypeDef::Lock**
DAC locking object
- **DMA_HandleTypeDef* DAC_HandleTypeDef::DMA_Handle1**
Pointer DMA handler for channel 1
- **DMA_HandleTypeDef* DAC_HandleTypeDef::DMA_Handle2**
Pointer DMA handler for channel 2
- **__IO uint32_t DAC_HandleTypeDef::ErrorCode**
DAC Error code

12.1.2 DAC_ChannelConfTypeDef

DAC_ChannelConfTypeDef is defined in the stm32f2xx_hal_dac.h

Data Fields

- **uint32_t DAC_Trigger**
- **uint32_t DAC_OutputBuffer**

Field Documentation

- **uint32_t DAC_ChannelConfTypeDef::DAC_Trigger**
Specifies the external trigger for the selected DAC channel. This parameter can be a value of [DAC_trigger_selection](#)
- **uint32_t DAC_ChannelConfTypeDef::DAC_OutputBuffer**
Specifies whether the DAC channel output buffer is enabled or disabled. This parameter can be a value of [DAC_output_buffer](#)

12.2 DAC Firmware driver API description

The following section lists the various functions of the DAC library.

12.2.1 DAC Peripheral features

DAC Channels

STM32F2 devices integrate two 12-bit Digital Analog Converters. The 2 converters (i.e. channel1 & channel2) can be used independently or simultaneously (dual mode):

1. DAC channel1 with DAC_OUT1 (PA4) as output or connected to on-chip peripherals (ex. timers).
2. DAC channel2 with DAC_OUT2 (PA5) as output or connected to on-chip peripherals (ex. timers).

DAC Triggers

Digital to Analog conversion can be non-triggered using DAC_TRIGGER_NONE and DAC_OUT1/DAC_OUT2 is available once writing to DHRx register.

Digital to Analog conversion can be triggered by:

1. External event: EXTI Line 9 (any GPIOx_PIN_9) using DAC_TRIGGER_EXT_IT9. The used pin (GPIOx_PIN_9) must be configured in input mode.
2. Timers TRGO: TIM2, TIM4, TIM5, TIM6, TIM7 and TIM8 (DAC_TRIGGER_T2_TRGO, DAC_TRIGGER_T4_TRGO...)
3. Software using DAC_TRIGGER_SOFTWARE

DAC Buffer mode feature

Each DAC channel integrates an output buffer that can be used to reduce the output impedance, and to drive external loads directly without having to add an external operational amplifier. To enable, the output buffer use `sConfig.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE`;

Note: Refer to the device datasheet for more details about output impedance value with and without output buffer.

DAC connect feature

Each DAC channel can be connected internally. To connect, use `sConfig.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_ENABLE`;

GPIO configurations guidelines

When a DAC channel is used (ex channel1 on PA4) and the other is not (ex channel2 on PA5 is configured in Analog and disabled). Channel1 may disturb channel2 as coupling effect. Note that there is no coupling on channel2 as soon as channel2 is turned on. Coupling on adjacent channel could be avoided as follows: when unused PA5 is configured as INPUT PULL-UP or DOWN. PA5 is configured in ANALOG just before it is turned on.

DAC wave generation feature

Both DAC channels can be used to generate

1. Noise wave
2. Triangle wave

DAC data format

The DAC data format can be:

1. 8-bit right alignment using DAC_ALIGN_8B_R
2. 12-bit left alignment using DAC_ALIGN_12B_L
3. 12-bit right alignment using DAC_ALIGN_12B_R

DAC data value to voltage correspondence

The analog output voltage on each DAC channel pin is determined by the following equation:

$$\text{DAC_OUTx} = \text{VREF+} * \text{DOR} / 4095$$

- with DOR is the Data Output Register

VEF+ is the input voltage reference (refer to the device datasheet)

e.g. To set DAC_OUT1 to 0.7V, use

- Assuming that VREF+ = 3.3V, $\text{DAC_OUT1} = (3.3 * 868) / 4095 = 0.7\text{V}$

DMA requests

A DMA1 request can be generated when an external trigger (but not a software trigger) occurs if DMA1 requests are enabled using HAL_DAC_Start_DMA(). DMA1 requests are mapped as following:

1. DAC channel1 mapped on DMA1 Stream5 channel7
2. DAC channel2 mapped on DMA1 Stream6 channel7

Note: *For Dual mode and specific signal (Triangle and noise) generation please refer to Extended Features Driver description*

12.2.2 How to use this driver

- DAC APB clock must be enabled to get write access to DAC registers using HAL_DAC_Init()
- Configure DAC_OUTx (DAC_OUT1: PA4, DAC_OUT2: PA5) in analog mode.
- Configure the DAC channel using HAL_DAC_ConfigChannel() function.
- Enable the DAC channel using HAL_DAC_Start() or HAL_DAC_Start_DMA() functions.

Polling mode IO operation

- Start the DAC peripheral using HAL_DAC_Start()
- To read the DAC last data output value, use the HAL_DAC_GetValue() function.
- Stop the DAC peripheral using HAL_DAC_Stop()

DMA mode IO operation

- Start the DAC peripheral using HAL_DAC_Start_DMA(), at this stage the user specify the length of data to be transferred at each end of conversion First issued trigger will start the conversion of the value previously set by HAL_DAC_SetValue().
- At the middle of data transfer HAL_DAC_ConvHalfCpltCallbackCh1() or HAL_DACEx_ConvHalfCpltCallbackCh2() function is executed and user can add his own code by customization of function pointer HAL_DAC_ConvHalfCpltCallbackCh1() or HAL_DACEx_ConvHalfCpltCallbackCh2()
- At The end of data transfer HAL_DAC_ConvCpltCallbackCh1() or HAL_DACEx_ConvHalfCpltCallbackCh2() function is executed and user can add his own code by customization of function pointer HAL_DAC_ConvCpltCallbackCh1() or HAL_DACEx_ConvHalfCpltCallbackCh2()
- In case of transfer Error, HAL_DAC_ErrorCallbackCh1() function is executed and user can add his own code by customization of function pointer HAL_DAC_ErrorCallbackCh1
- In case of DMA underrun, DAC interruption triggers and execute internal function HAL_DAC_IRQHandler. HAL_DAC_DMAUnderrunCallbackCh1() or HAL_DACEx_DMAUnderrunCallbackCh2() function is executed and user can add his own code by customization of function pointer HAL_DAC_DMAUnderrunCallbackCh1() or HAL_DACEx_DMAUnderrunCallbackCh2() and add his own code by customization of function pointer HAL_DAC_ErrorCallbackCh1()
- Stop the DAC peripheral using HAL_DAC_Stop_DMA()

Callback registration

The compilation define USE_HAL_DAC_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions @ref HAL_DAC_RegisterCallback() to register a user callback, it allows to register following callbacks:

- ConvCpltCallbackCh1 : callback when a half transfer is completed on Ch1.
- ConvHalfCpltCallbackCh1 : callback when a transfer is completed on Ch1.
- ErrorCallbackCh1 : callback when an error occurs on Ch1.
- DMAUnderrunCallbackCh1 : callback when an underrun error occurs on Ch1.
- ConvCpltCallbackCh2 : callback when a half transfer is completed on Ch2.
- ConvHalfCpltCallbackCh2 : callback when a transfer is completed on Ch2.
- ErrorCallbackCh2 : callback when an error occurs on Ch2.
- DMAUnderrunCallbackCh2 : callback when an underrun error occurs on Ch2.

- MspInitCallback : DAC MspInit.
- MspDeInitCallback : DAC MspdeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function @ref HAL_DAC_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- ConvCpltCallbackCh1 : callback when a half transfer is completed on Ch1.
- ConvHalfCpltCallbackCh1 : callback when a transfer is completed on Ch1.
- ErrorCallbackCh1 : callback when an error occurs on Ch1.
- DMAUnderrunCallbackCh1 : callback when an underrun error occurs on Ch1.
- ConvCpltCallbackCh2 : callback when a half transfer is completed on Ch2.
- ConvHalfCpltCallbackCh2 : callback when a transfer is completed on Ch2.
- ErrorCallbackCh2 : callback when an error occurs on Ch2.
- DMAUnderrunCallbackCh2 : callback when an underrun error occurs on Ch2.
- MspInitCallback : DAC MspInit.
- MspDeInitCallback : DAC MspdeInit.
- All Callbacks This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the @ref HAL_DAC_Init and if the state is HAL_DAC_STATE_RESET all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for MspInit and MspDeInit callbacks that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_DAC_Init and @ref HAL_DAC_DeInit only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_DAC_Init and @ref HAL_DAC_DeInit keep and use the user MspInit/ MspDeInit callbacks (registered beforehand) Callbacks can be registered/unregistered in READY state only. Exception done for MspInit/MspDeInit callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_DAC_RegisterCallback before calling @ref HAL_DAC_DeInit or @ref HAL_DAC_Init function. When The compilation define USE_HAL_DAC_REGISTER_CALLBACKS is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

DAC HAL driver macros list

Below the list of most used macros in DAC HAL driver.

- __HAL_DAC_ENABLE : Enable the DAC peripheral
- __HAL_DAC_DISABLE : Disable the DAC peripheral
- __HAL_DAC_CLEAR_FLAG: Clear the DAC's pending flags
- __HAL_DAC_GET_FLAG: Get the selected DAC's flag status

Note: You can refer to the DAC HAL driver header file for more useful macros

12.2.3 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize and configure the DAC.
- De-initialize the DAC.

This section contains the following APIs:

- [HAL_DAC_Init](#)
- [HAL_DAC_DeInit](#)
- [HAL_DAC_MspInit](#)
- [HAL_DAC_MspDeInit](#)

12.2.4 IO operation functions

This section provides functions allowing to:

- Start conversion.
- Stop conversion.
- Start conversion and enable DMA transfer.
- Stop conversion and disable DMA transfer.

- Get result of conversion.

This section contains the following APIs:

- [HAL_DAC_Start](#)
- [HAL_DAC_Stop](#)
- [HAL_DAC_Start_DMA](#)
- [HAL_DAC_Stop_DMA](#)
- [HAL_DAC_IRQHandler](#)
- [HAL_DAC_SetValue](#)
- [HAL_DAC_ConvCpltCallbackCh1](#)
- [HAL_DAC_ConvHalfCpltCallbackCh1](#)
- [HAL_DAC_ErrorCallbackCh1](#)
- [HAL_DAC_DMAUnderrunCallbackCh1](#)

12.2.5 Peripheral Control functions

This section provides functions allowing to:

- Configure channels.
- Set the specified data holding register value for DAC channel.

This section contains the following APIs:

- [HAL_DAC_GetValue](#)
- [HAL_DAC_ConfigChannel](#)

12.2.6 Peripheral State and Errors functions

This subsection provides functions allowing to

- Check the DAC state.
- Check the DAC Errors.

This section contains the following APIs:

- [HAL_DAC_GetState](#)
- [HAL_DAC_GetError](#)

12.2.7 Detailed description of functions

HAL_DAC_Init

Function name

HAL_StatusTypeDef HAL_DAC_Init (DAC_HandleTypeDef * hdac)

Function description

Initialize the DAC peripheral according to the specified parameters in the DAC_InitStruct and initialize the associated handle.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **HAL**: status

HAL_DAC_DeInit

Function name

HAL_StatusTypeDef HAL_DAC_DeInit (DAC_HandleTypeDef * hdac)

Function description

Deinitialize the DAC peripheral registers to their default reset values.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **HAL:** status

HAL_DAC_Msplnit

Function name

void HAL_DAC_Msplnit (DAC_HandleTypeDef * hdac)

Function description

Initialize the DAC MSP.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DAC_MspDeInit

Function name

void HAL_DAC_MspDeInit (DAC_HandleTypeDef * hdac)

Function description

DeInitialize the DAC MSP.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DAC_Start

Function name

HAL_StatusTypeDef HAL_DAC_Start (DAC_HandleTypeDef * hdac, uint32_t Channel)

Function description

Enables DAC and starts conversion of channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected

Return values

- **HAL:** status

HAL_DAC_Stop

Function name

HAL_StatusTypeDef HAL_DAC_Stop (DAC_HandleTypeDef * hdac, uint32_t Channel)

Function description

Disables DAC and stop conversion of channel.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel**: The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected

Return values

- **HAL**: status

HAL_DAC_Start_DMA

Function name

HAL_StatusTypeDef HAL_DAC_Start_DMA (DAC_HandleTypeDef * hdac, uint32_t Channel, uint32_t * pData, uint32_t Length, uint32_t Alignment)

Function description

Enables DAC and starts conversion of channel.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel**: The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected
- **pData**: The destination peripheral Buffer address.
- **Length**: The length of data to be transferred from memory to DAC peripheral
- **Alignment**: Specifies the data alignment for DAC channel. This parameter can be one of the following values:
 - DAC_ALIGN_8B_R: 8bit right data alignment selected
 - DAC_ALIGN_12B_L: 12bit left data alignment selected
 - DAC_ALIGN_12B_R: 12bit right data alignment selected

Return values

- **HAL**: status

HAL_DAC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_DAC_Stop_DMA (DAC_HandleTypeDef * hdac, uint32_t Channel)

Function description

Disables DAC and stop conversion of channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected

Return values

- **HAL:** status

HAL_DAC_IRQHandler

Function name

void HAL_DAC_IRQHandler (DAC_HandleTypeDef * hdac)

Function description

Handles DAC interrupt request This function uses the interruption of DMA underrun.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DAC_SetValue

Function name

HAL_StatusTypeDef HAL_DAC_SetValue (DAC_HandleTypeDef * hdac, uint32_t Channel, uint32_t Alignment, uint32_t Data)

Function description

Set the specified data holding register value for DAC channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected
- **Alignment:** Specifies the data alignment. This parameter can be one of the following values:
 - DAC_ALIGN_8B_R: 8bit right data alignment selected
 - DAC_ALIGN_12B_L: 12bit left data alignment selected
 - DAC_ALIGN_12B_R: 12bit right data alignment selected
- **Data:** Data to be loaded in the selected data holding register.

Return values

- **HAL:** status

HAL_DAC_ConvCpltCallbackCh1

Function name

void HAL_DAC_ConvCpltCallbackCh1 (DAC_HandleTypeDef * hdac)

Function description

Conversion complete callback in non-blocking mode for Channel1.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None**:

HAL_DAC_ConvHalfCpltCallbackCh1

Function name

void HAL_DAC_ConvHalfCpltCallbackCh1 (DAC_HandleTypeDef * hdac)

Function description

Conversion half DMA transfer callback in non-blocking mode for Channel1.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None**:

HAL_DAC_ErrorCallbackCh1

Function name

void HAL_DAC_ErrorCallbackCh1 (DAC_HandleTypeDef * hdac)

Function description

Error DAC callback for Channel1.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None**:

HAL_DAC_DMAUnderrunCallbackCh1

Function name

void HAL_DAC_DMAUnderrunCallbackCh1 (DAC_HandleTypeDef * hdac)

Function description

DMA underrun DAC callback for channel1.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None**:

HAL_DAC_GetValue

Function name

uint32_t HAL_DAC_GetValue (DAC_HandleTypeDef * hdac, uint32_t Channel)

Function description

Returns the last data output value of the selected DAC channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected

Return values

- **The:** selected DAC channel data output value.

HAL_DAC_ConfigChannel

Function name

HAL_StatusTypeDef HAL_DAC_ConfigChannel (DAC_HandleTypeDef * hdac, DAC_ChannelConfTypeDef * sConfig, uint32_t Channel)

Function description

Configures the selected DAC channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **sConfig:** DAC configuration structure.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected

Return values

- **HAL:** status

HAL_DAC_GetState

Function name

HAL_DAC_StateTypeDef HAL_DAC_GetState (DAC_HandleTypeDef * hdac)

Function description

return the DAC handle state

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **HAL:** state

HAL_DAC_GetError

Function name

```
uint32_t HAL_DAC_GetError (DAC_HandleTypeDef * hdac)
```

Function description

Return the DAC error code.

Parameters

- **hdac**: pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **DAC**: Error Code

DAC_DMAConvCpltCh1

Function name

```
void DAC_DMAConvCpltCh1 (DMA_HandleTypeDef * hdma)
```

Function description

DMA conversion complete callback.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None**:

DAC_DMAErrorCh1

Function name

```
void DAC_DMAErrorCh1 (DMA_HandleTypeDef * hdma)
```

Function description

DMA error callback.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None**:

DAC_DMAHalfConvCpltCh1

Function name

```
void DAC_DMAHalfConvCpltCh1 (DMA_HandleTypeDef * hdma)
```

Function description

DMA half transfer complete callback.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None:**

12.3 DAC Firmware driver defines

The following section lists the various define and macros of the module.

12.3.1 DAC

DAC

DAC Channel selection

DAC_CHANNEL_1

DAC_CHANNEL_2

DAC data alignment

DAC_ALIGN_12B_R

DAC_ALIGN_12B_L

DAC_ALIGN_8B_R

DAC Error Code

HAL_DAC_ERROR_NONE

No error

HAL_DAC_ERROR_DMAUNDERRUNCH1

DAC channel1 DMA underrun error

HAL_DAC_ERROR_DMAUNDERRUNCH2

DAC channel2 DMA underrun error

HAL_DAC_ERROR_DMA

DMA error

HAL_DAC_ERROR_TIMEOUT

Timeout error

DAC Exported Macros

__HAL_DAC_RESET_HANDLE_STATE

Description:

- Reset DAC handle state.

Parameters:

- `__HANDLE__`: specifies the DAC handle.

Return value:

- None

__HAL_DAC_ENABLE

Description:

- Enable the DAC channel.

Parameters:

- __HANDLE__: specifies the DAC handle.
- __DAC_Channel__: specifies the DAC channel

Return value:

- None

__HAL_DAC_DISABLE

Description:

- Disable the DAC channel.

Parameters:

- __HANDLE__: specifies the DAC handle
- __DAC_Channel__: specifies the DAC channel.

Return value:

- None

DAC_DHR12R1_ALIGNMENT

Description:

- Set DHR12R1 alignment.

Parameters:

- __ALIGNMENT__: specifies the DAC alignment

Return value:

- None

DAC_DHR12R2_ALIGNMENT

Description:

- Set DHR12R2 alignment.

Parameters:

- __ALIGNMENT__: specifies the DAC alignment

Return value:

- None

DAC_DHR12RD_ALIGNMENT

Description:

- Set DHR12RD alignment.

Parameters:

- __ALIGNMENT__: specifies the DAC alignment

Return value:

- None

__HAL_DAC_ENABLE_IT

Description:

- Enable the DAC interrupt.

Parameters:

- `__HANDLE__`: specifies the DAC handle
- `__INTERRUPT__`: specifies the DAC interrupt. This parameter can be any combination of the following values:
 - `DAC_IT_DMAUDR1`: DAC channel 1 DMA underrun interrupt
 - `DAC_IT_DMAUDR2`: DAC channel 2 DMA underrun interrupt

Return value:

- None

__HAL_DAC_DISABLE_IT

Description:

- Disable the DAC interrupt.

Parameters:

- `__HANDLE__`: specifies the DAC handle
- `__INTERRUPT__`: specifies the DAC interrupt. This parameter can be any combination of the following values:
 - `DAC_IT_DMAUDR1`: DAC channel 1 DMA underrun interrupt
 - `DAC_IT_DMAUDR2`: DAC channel 2 DMA underrun interrupt

Return value:

- None

__HAL_DAC_GET_IT_SOURCE

Description:

- Check whether the specified DAC interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: DAC handle
- `__INTERRUPT__`: DAC interrupt source to check This parameter can be any combination of the following values:
 - `DAC_IT_DMAUDR1`: DAC channel 1 DMA underrun interrupt
 - `DAC_IT_DMAUDR2`: DAC channel 2 DMA underrun interrupt

Return value:

- State: of interruption (SET or RESET)

__HAL_DAC_GET_FLAG

Description:

- Get the selected DAC's flag status.

Parameters:

- `__HANDLE__`: specifies the DAC handle.
- `__FLAG__`: specifies the DAC flag to get. This parameter can be any combination of the following values:
 - `DAC_FLAG_DMAUDR1`: DAC channel 1 DMA underrun flag
 - `DAC_FLAG_DMAUDR2`: DAC channel 2 DMA underrun flag

Return value:

- None

__HAL_DAC_CLEAR_FLAG

Description:

- Clear the DAC's flag.

Parameters:

- __HANDLE__: specifies the DAC handle.
- __FLAG__: specifies the DAC flag to clear. This parameter can be any combination of the following values:
 - DAC_FLAG_DMAUDR1: DAC channel 1 DMA underrun flag
 - DAC_FLAG_DMAUDR2: DAC channel 2 DMA underrun flag

Return value:

- None

DAC flags definition

DAC_FLAG_DMAUDR1

DAC_FLAG_DMAUDR2

DAC IT definition

DAC_IT_DMAUDR1

DAC_IT_DMAUDR2

DAC output buffer

DAC_OUTPUTBUFFER_ENABLE

DAC_OUTPUTBUFFER_DISABLE

DAC trigger selection

DAC_TRIGGER_NONE

Conversion is automatic once the DAC1_DHRxxxx register has been loaded, and not by external trigger

DAC_TRIGGER_T2_TRGO

TIM2 TRGO selected as external conversion trigger for DAC channel

DAC_TRIGGER_T4_TRGO

TIM4 TRGO selected as external conversion trigger for DAC channel

DAC_TRIGGER_T5_TRGO

TIM3 TRGO selected as external conversion trigger for DAC channel

DAC_TRIGGER_T6_TRGO

Conversion started by software trigger for DAC channel

DAC_TRIGGER_T7_TRGO

TIM7 TRGO selected as external conversion trigger for DAC channel

DAC_TRIGGER_T8_TRGO

TIM8 TRGO selected as external conversion trigger for DAC channel

DAC_TRIGGER_EXT_IT9

EXTI Line9 event selected as external conversion trigger for DAC channel

DAC_TRIGGER_SOFTWARE

Conversion started by software trigger for DAC channel

13 HAL DAC Extension Driver

13.1 DACEx Firmware driver API description

The following section lists the various functions of the DACEx library.

13.1.1 How to use this driver

Dual mode IO operation

Signal generation operation

13.1.2 Extended features functions

This section provides functions allowing to:

- Start conversion.
- Stop conversion.
- Start conversion and enable DMA transfer.
- Stop conversion and disable DMA transfer.
- Get result of conversion.
- Get result of dual mode conversion.

This section contains the following APIs:

- [*HAL_DACEx_TriangleWaveGenerate*](#)
- [*HAL_DACEx_NoiseWaveGenerate*](#)
- [*HAL_DACEx_DualSetValue*](#)
- [*HAL_DACEx_ConvCpltCallbackCh2*](#)
- [*HAL_DACEx_ConvHalfCpltCallbackCh2*](#)
- [*HAL_DACEx_ErrorCallbackCh2*](#)
- [*HAL_DACEx_DMAUnderrunCallbackCh2*](#)
- [*HAL_DACEx_DualGetValue*](#)

13.1.3 Peripheral Control functions

This section provides functions allowing to:

- Set the specified data holding register value for DAC channel.

This section contains the following APIs:

- [*HAL_DACEx_DualGetValue*](#)

13.1.4 Detailed description of functions

HAL_DACEx_TriangleWaveGenerate

Function name

HAL_StatusTypeDef HAL_DACEx_TriangleWaveGenerate (DAC_HandleTypeDef * hdac, uint32_t Channel, uint32_t Amplitude)

Function description

Enable or disable the selected DAC channel wave generation.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected
- **Amplitude:** Select max triangle amplitude. This parameter can be one of the following values:
 - DAC_TRIANGLEAMPLITUDE_1: Select max triangle amplitude of 1
 - DAC_TRIANGLEAMPLITUDE_3: Select max triangle amplitude of 3
 - DAC_TRIANGLEAMPLITUDE_7: Select max triangle amplitude of 7
 - DAC_TRIANGLEAMPLITUDE_15: Select max triangle amplitude of 15
 - DAC_TRIANGLEAMPLITUDE_31: Select max triangle amplitude of 31
 - DAC_TRIANGLEAMPLITUDE_63: Select max triangle amplitude of 63
 - DAC_TRIANGLEAMPLITUDE_127: Select max triangle amplitude of 127
 - DAC_TRIANGLEAMPLITUDE_255: Select max triangle amplitude of 255
 - DAC_TRIANGLEAMPLITUDE_511: Select max triangle amplitude of 511
 - DAC_TRIANGLEAMPLITUDE_1023: Select max triangle amplitude of 1023
 - DAC_TRIANGLEAMPLITUDE_2047: Select max triangle amplitude of 2047
 - DAC_TRIANGLEAMPLITUDE_4095: Select max triangle amplitude of 4095

Return values

- **HAL:** status

HAL_DACEx_NoiseWaveGenerate

Function name

HAL_StatusTypeDef HAL_DACEx_NoiseWaveGenerate (DAC_HandleTypeDef * hdac, uint32_t Channel, uint32_t Amplitude)

Function description

Enable or disable the selected DAC channel wave generation.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Channel:** The selected DAC channel. This parameter can be one of the following values:
 - DAC_CHANNEL_1: DAC Channel1 selected
 - DAC_CHANNEL_2: DAC Channel2 selected
- **Amplitude:** Unmask DAC channel LFSR for noise wave generation. This parameter can be one of the following values:
 - DAC_LFSRUNMASK_BIT0: Unmask DAC channel LFSR bit0 for noise wave generation
 - DAC_LFSRUNMASK_BITS1_0: Unmask DAC channel LFSR bit[1:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS2_0: Unmask DAC channel LFSR bit[2:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS3_0: Unmask DAC channel LFSR bit[3:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS4_0: Unmask DAC channel LFSR bit[4:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS5_0: Unmask DAC channel LFSR bit[5:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS6_0: Unmask DAC channel LFSR bit[6:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS7_0: Unmask DAC channel LFSR bit[7:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS8_0: Unmask DAC channel LFSR bit[8:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS9_0: Unmask DAC channel LFSR bit[9:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS10_0: Unmask DAC channel LFSR bit[10:0] for noise wave generation
 - DAC_LFSRUNMASK_BITS11_0: Unmask DAC channel LFSR bit[11:0] for noise wave generation

Return values

- **HAL:** status

HAL_DACEx_DualSetValue

Function name

HAL_StatusTypeDef HAL_DACEx_DualSetValue (DAC_HandleTypeDef * hdac, uint32_t Alignment, uint32_t Data1, uint32_t Data2)

Function description

Set the specified data holding register value for dual DAC channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.
- **Alignment:** Specifies the data alignment for dual channel DAC. This parameter can be one of the following values: DAC_ALIGN_8B_R: 8bit right data alignment selected DAC_ALIGN_12B_L: 12bit left data alignment selected DAC_ALIGN_12B_R: 12bit right data alignment selected
- **Data1:** Data for DAC Channel1 to be loaded in the selected data holding register.
- **Data2:** Data for DAC Channel2 to be loaded in the selected data holding register.

Return values

- **HAL:** status

Notes

- In dual mode, a unique register access is required to write in both DAC channels at the same time.

HAL_DACEx_DualGetValue

Function name

uint32_t HAL_DACEx_DualGetValue (DAC_HandleTypeDef * hdac)

Function description

Return the last data output value of the selected DAC channel.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **The:** selected DAC channel data output value.

HAL_DACEx_ConvCpltCallbackCh2

Function name

void HAL_DACEx_ConvCpltCallbackCh2 (DAC_HandleTypeDef * hdac)

Function description

Conversion complete callback in non-blocking mode for Channel2.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DACEx_ConvHalfCpltCallbackCh2

Function name

void HAL_DACEx_ConvHalfCpltCallbackCh2 (DAC_HandleTypeDef * hdac)

Function description

Conversion half DMA transfer callback in non-blocking mode for Channel2.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DACEx_ErrorCallbackCh2

Function name

void HAL_DACEx_ErrorCallbackCh2 (DAC_HandleTypeDef * hdac)

Function description

Error DAC callback for Channel2.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

HAL_DACEx_DMAUnderrunCallbackCh2

Function name

void HAL_DACEx_DMAUnderrunCallbackCh2 (DAC_HandleTypeDef * hdac)

Function description

DMA underrun DAC callback for Channel2.

Parameters

- **hdac:** pointer to a DAC_HandleTypeDef structure that contains the configuration information for the specified DAC.

Return values

- **None:**

DAC_DMAConvCpltCh2

Function name

void DAC_DMAConvCpltCh2 (DMA_HandleTypeDef * hdma)

Function description

DMA conversion complete callback.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None:**

DAC_DMAErrorCh2

Function name

void DAC_DMAErrorCh2 (DMA_HandleTypeDef * hdma)

Function description

DMA error callback.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None:**

DAC_DMAHalfConvCpltCh2

Function name

void DAC_DMAHalfConvCpltCh2 (DMA_HandleTypeDef * hdma)

Function description

DMA half transfer complete callback.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA module.

Return values

- **None:**

13.2 DACEx Firmware driver defines

The following section lists the various define and macros of the module.

13.2.1

DACEx

DACEx

DACEx lfsrunmask triangle amplitude

DAC_LFSRUNMASK_BIT0

Unmask DAC channel LFSR bit0 for noise wave generation

DAC_LFSRUNMASK_BITS1_0

Unmask DAC channel LFSR bit[1:0] for noise wave generation

DAC_LFSRUNMASK_BITS2_0

Unmask DAC channel LFSR bit[2:0] for noise wave generation

DAC_LFSRUNMASK_BITS3_0

Unmask DAC channel LFSR bit[3:0] for noise wave generation

DAC_LFSRUNMASK_BITS4_0

Unmask DAC channel LFSR bit[4:0] for noise wave generation

DAC_LFSRUNMASK_BITS5_0

Unmask DAC channel LFSR bit[5:0] for noise wave generation

DAC_LFSRUNMASK_BITS6_0

Unmask DAC channel LFSR bit[6:0] for noise wave generation

DAC_LFSRUNMASK_BITS7_0

Unmask DAC channel LFSR bit[7:0] for noise wave generation

DAC_LFSRUNMASK_BITS8_0

Unmask DAC channel LFSR bit[8:0] for noise wave generation

DAC_LFSRUNMASK_BITS9_0

Unmask DAC channel LFSR bit[9:0] for noise wave generation

DAC_LFSRUNMASK_BITS10_0

Unmask DAC channel LFSR bit[10:0] for noise wave generation

DAC_LFSRUNMASK_BITS11_0

Unmask DAC channel LFSR bit[11:0] for noise wave generation

DAC_TRIANGLEAMPLITUDE_1

Select max triangle amplitude of 1

DAC_TRIANGLEAMPLITUDE_3

Select max triangle amplitude of 3

DAC_TRIANGLEAMPLITUDE_7

Select max triangle amplitude of 7

DAC_TRIANGLEAMPLITUDE_15

Select max triangle amplitude of 15

DAC_TRIANGLEAMPLITUDE_31

Select max triangle amplitude of 31

DAC_TRIANGLEAMPLITUDE_63

Select max triangle amplitude of 63

DAC_TRIANGLEAMPLITUDE_127

Select max triangle amplitude of 127

DAC_TRIANGLEAMPLITUDE_255

Select max triangle amplitude of 255

DAC_TRIANGLEAMPLITUDE_511

Select max triangle amplitude of 511

DAC_TRIANGLEAMPLITUDE_1023

Select max triangle amplitude of 1023

DAC_TRIANGLEAMPLITUDE_2047

Select max triangle amplitude of 2047

DAC_TRIANGLEAMPLITUDE_4095

Select max triangle amplitude of 4095

14 HAL DCMI Generic Driver

14.1 DCMI Firmware driver registers structures

14.1.1 DCMI_CodesInitTypeDef

DCMI_CodesInitTypeDef is defined in the stm32f2xx_hal_dcmi.h

Data Fields

- **uint8_t FrameStartCode**
- **uint8_t LineStartCode**
- **uint8_t LineEndCode**
- **uint8_t FrameEndCode**

Field Documentation

- **uint8_t DCMI_CodesInitTypeDef::FrameStartCode**
Specifies the code of the frame start delimiter.
- **uint8_t DCMI_CodesInitTypeDef::LineStartCode**
Specifies the code of the line start delimiter.
- **uint8_t DCMI_CodesInitTypeDef::LineEndCode**
Specifies the code of the line end delimiter.
- **uint8_t DCMI_CodesInitTypeDef::FrameEndCode**
Specifies the code of the frame end delimiter.

14.1.2 DCMI_SyncUnmaskTypeDef

DCMI_SyncUnmaskTypeDef is defined in the stm32f2xx_hal_dcmi.h

Data Fields

- **uint8_t FrameStartUnmask**
- **uint8_t LineStartUnmask**
- **uint8_t LineEndUnmask**
- **uint8_t FrameEndUnmask**

Field Documentation

- **uint8_t DCMI_SyncUnmaskTypeDef::FrameStartUnmask**
Specifies the frame start delimiter unmask.
- **uint8_t DCMI_SyncUnmaskTypeDef::LineStartUnmask**
Specifies the line start delimiter unmask.
- **uint8_t DCMI_SyncUnmaskTypeDef::LineEndUnmask**
Specifies the line end delimiter unmask.
- **uint8_t DCMI_SyncUnmaskTypeDef::FrameEndUnmask**
Specifies the frame end delimiter unmask.

14.1.3 DCMI_InitTypeDef

DCMI_InitTypeDef is defined in the stm32f2xx_hal_dcmi.h

Data Fields

- **uint32_t SynchroMode**
- **uint32_t PCKPolarity**
- **uint32_t VSPolarity**
- **uint32_t HSPolarity**
- **uint32_t CaptureRate**
- **uint32_t ExtendedDataMode**

- **DCMI_CodesInitTypeDef SyncroCode**
- **uint32_t JPEGMode**

Field Documentation

- **uint32_t DCMI_InitTypeDef::SynchroMode**
Specifies the Synchronization Mode: Hardware or Embedded. This parameter can be a value of [DCMI_Synchronization_Mode](#)
- **uint32_t DCMI_InitTypeDef::PCKPolarity**
Specifies the Pixel clock polarity: Falling or Rising. This parameter can be a value of [DCMI_PIXCK_Polarity](#)
- **uint32_t DCMI_InitTypeDef::VSPolarity**
Specifies the Vertical synchronization polarity: High or Low. This parameter can be a value of [DCMI_VSYNC_Polarity](#)
- **uint32_t DCMI_InitTypeDef::HSPolarity**
Specifies the Horizontal synchronization polarity: High or Low. This parameter can be a value of [DCMI_HSYNC_Polarity](#)
- **uint32_t DCMI_InitTypeDef::CaptureRate**
Specifies the frequency of frame capture: All, 1/2 or 1/4. This parameter can be a value of [DCMI_Capture_Rate](#)
- **uint32_t DCMI_InitTypeDef::ExtendedDataMode**
Specifies the data width: 8-bit, 10-bit, 12-bit or 14-bit. This parameter can be a value of [DCMI_Extended_Data_Mode](#)
- **DCMI_CodesInitTypeDef DCMI_InitTypeDef::SyncroCode**
Specifies the code of the line/frame start delimiter and the line/frame end delimiter
- **uint32_t DCMI_InitTypeDef::JPEGMode**
Enable or Disable the JPEG mode. This parameter can be a value of [DCMI_MODE_JPEG](#)

14.1.4

__DCMI_HandleTypeDef

__DCMI_HandleTypeDef is defined in the stm32f2xx_hal_dcmi.h

Data Fields

- **DCMI_TypeDef * Instance**
- **DCMI_InitTypeDef Init**
- **HAL_LockTypeDef Lock**
- **__IO HAL_DCMI_StateTypeDef State**
- **__IO uint32_t XferCount**
- **__IO uint32_t XferSize**
- **uint32_t XferTransferNumber**
- **uint32_t pBuffPtr**
- **DMA_HandleTypeDef * DMA_Handle**
- **__IO uint32_t ErrorCode**

Field Documentation

- **DCMI_TypeDef* __DCMI_HandleTypeDef::Instance**
DCMI Register base address
- **DCMI_InitTypeDef __DCMI_HandleTypeDef::Init**
DCMI parameters
- **HAL_LockTypeDef __DCMI_HandleTypeDef::Lock**
DCMI locking object
- **__IO HAL_DCMI_StateTypeDef __DCMI_HandleTypeDef::State**
DCMI state
- **__IO uint32_t __DCMI_HandleTypeDef::XferCount**
DMA transfer counter

- **__IO uint32_t __DCMI_HandleTypeDef::XferSize**
DMA transfer size
- **uint32_t __DCMI_HandleTypeDef::XferTransferNumber**
DMA transfer number
- **uint32_t __DCMI_HandleTypeDef::pBuffPtr**
Pointer to DMA output buffer
- **DMA_HandleTypeDef* __DCMI_HandleTypeDef::DMA_Handle**
Pointer to the DMA handler
- **__IO uint32_t __DCMI_HandleTypeDef::ErrorCode**
DCMI Error code

14.2 DCMI Firmware driver API description

The following section lists the various functions of the DCMI library.

14.2.1 How to use this driver

The sequence below describes how to use this driver to capture image from a camera module connected to the DCMI Interface. This sequence does not take into account the configuration of the camera module, which should be made before to configure and enable the DCMI to capture images.

1. Program the required configuration through following parameters: horizontal and vertical polarity, pixel clock polarity, Capture Rate, Synchronization Mode, code of the frame delimiter and data width using HAL_DCMI_Init() function.
2. Configure the DMA2_Stream1 channel1 to transfer Data from DCMI DR register to the destination memory buffer.
3. Program the required configuration through following parameters: DCMI mode, destination memory Buffer address and the data length and enable capture using HAL_DCMI_Start_DMA() function.
4. Optionally, configure and Enable the CROP feature to select a rectangular window from the received image using HAL_DCMI_ConfigCrop() and HAL_DCMI_EnableCROP() functions
5. The capture can be stopped using HAL_DCMI_Stop() function.
6. To control DCMI state you can use the function HAL_DCMI_GetState().

DCMI HAL driver macros list

Below the list of most used macros in DCMI HAL driver.

- **__HAL_DCMI_ENABLE**: Enable the DCMI peripheral.
- **__HAL_DCMI_DISABLE**: Disable the DCMI peripheral.
- **__HAL_DCMI_GET_FLAG**: Get the DCMI pending flags.
- **__HAL_DCMI_CLEAR_FLAG**: Clear the DCMI pending flags.
- **__HAL_DCMI_ENABLE_IT**: Enable the specified DCMI interrupts.
- **__HAL_DCMI_DISABLE_IT**: Disable the specified DCMI interrupts.
- **__HAL_DCMI_GET_IT_SOURCE**: Check whether the specified DCMI interrupt has occurred or not.

Note: You can refer to the DCMI HAL driver header file for more useful macros

Callback registration

14.2.2 Initialization and Configuration functions

This section provides functions allowing to:

- Initialize and configure the DCMI
- De-initialize the DCMI

This section contains the following APIs:

- **HAL_DCMI_Init**
- **HAL_DCMI_DeInit**
- **HAL_DCMI_MspInit**

- [HAL_DCMI_MspDeInit](#)

14.2.3 IO operation functions

This section provides functions allowing to:

- Configure destination address and data length and Enables DCMI DMA request and enables DCMI capture
- Stop the DCMI capture.
- Handles DCMI interrupt request.

This section contains the following APIs:

- [HAL_DCMI_Start_DMA](#)
- [HAL_DCMI_Stop](#)
- [HAL_DCMI_Suspend](#)
- [HAL_DCMI_Resume](#)
- [HAL_DCMI_IRQHandler](#)
- [HAL_DCMI_ErrorCallback](#)
- [HAL_DCMI_LineEventCallback](#)
- [HAL_DCMI_VsyncEventCallback](#)
- [HAL_DCMI_FrameEventCallback](#)

14.2.4 Peripheral Control functions

This section provides functions allowing to:

- Configure the CROP feature.
- Enable/Disable the CROP feature.
- Set embedded synchronization delimiters unmask.

This section contains the following APIs:

- [HAL_DCMI_ConfigCrop](#)
- [HAL_DCMI_DisableCrop](#)
- [HAL_DCMI_EnableCrop](#)
- [HAL_DCMI_ConfigSyncUnmask](#)

14.2.5 Peripheral State and Errors functions

This subsection provides functions allowing to

- Check the DCMI state.
- Get the specific DCMI error flag.

This section contains the following APIs:

- [HAL_DCMI_GetState](#)
- [HAL_DCMI_GetError](#)

14.2.6 Detailed description of functions

HAL_DCMI_Init

Function name

HAL_StatusTypeDef HAL_DCMI_Init (DCMI_HandleTypeDef * hdcmi)

Function description

Initializes the DCMI according to the specified parameters in the DCMI_InitTypeDef and create the associated handle.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL:** status

HAL_DCMI_DeInit

Function name

HAL_StatusTypeDef HAL_DCMI_DeInit (DCMI_HandleTypeDef * hdcmi)

Function description

Deinitializes the DCMI peripheral registers to their default reset values.

Parameters

- **hdcmi:** pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL:** status

HAL_DCMI_MspInit

Function name

void HAL_DCMI_MspInit (DCMI_HandleTypeDef * hdcmi)

Function description

Initializes the DCMI MSP.

Parameters

- **hdcmi:** pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None:**

HAL_DCMI_MspDeInit

Function name

void HAL_DCMI_MspDeInit (DCMI_HandleTypeDef * hdcmi)

Function description

Deinitializes the DCMI MSP.

Parameters

- **hdcmi:** pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None:**

HAL_DCMI_Start_DMA

Function name

HAL_StatusTypeDef HAL_DCMI_Start_DMA (DCMI_HandleTypeDef * hdcmi, uint32_t DCMI_Mode, uint32_t pData, uint32_t Length)

Function description

Enables DCMI DMA request and enables DCMI capture.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.
- **DCMI_Mode**: DCMI capture mode snapshot or continuous grab.
- **pData**: The destination memory Buffer address (LCD Frame buffer).
- **Length**: The length of capture to be transferred.

Return values

- **HAL**: status

HAL_DCMI_Stop

Function name

HAL_StatusTypeDef HAL_DCMI_Stop (DCMI_HandleTypeDef * hdcmi)

Function description

Disable DCMI DMA request and Disable DCMI capture.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: status

HAL_DCMI_Suspend

Function name

HAL_StatusTypeDef HAL_DCMI_Suspend (DCMI_HandleTypeDef * hdcmi)

Function description

Suspend DCMI capture.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: status

HAL_DCMI_Resume

Function name

HAL_StatusTypeDef HAL_DCMI_Resume (DCMI_HandleTypeDef * hdcmi)

Function description

Resume DCMI capture.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: status

HAL_DCMI_ErrorCallback

Function name

void HAL_DCMI_ErrorCallback (DCMI_HandleTypeDef * hdcmi)

Function description

Error DCMI callback.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None**:

HAL_DCMI_LineEventCallback

Function name

void HAL_DCMI_LineEventCallback (DCMI_HandleTypeDef * hdcmi)

Function description

Line Event callback.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None**:

HAL_DCMI_FrameEventCallback

Function name

void HAL_DCMI_FrameEventCallback (DCMI_HandleTypeDef * hdcmi)

Function description

Frame Event callback.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None**:

HAL_DCMI_VsyncEventCallback

Function name

void HAL_DCMI_VsyncEventCallback (DCMI_HandleTypeDef * hdcmi)

Function description

VSYNC Event callback.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **None**:

HAL_DCMI_IRQHandler

Function name

void HAL_DCMI_IRQHandler (DCMI_HandleTypeDef * hdcmi)

Function description

Handles DCMI interrupt request.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for the DCMI.

Return values

- **None**:

HAL_DCMI_ConfigCrop

Function name

HAL_StatusTypeDef HAL_DCMI_ConfigCrop (DCMI_HandleTypeDef * hdcmi, uint32_t X0, uint32_t Y0, uint32_t XSize, uint32_t YSize)

Function description

Configure the DCMI CROP coordinate.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.
- **YSize**: DCMI Line number
- **XSize**: DCMI Pixel per line
- **X0**: DCMI window X offset
- **Y0**: DCMI window Y offset

Return values

- **HAL**: status

HAL_DCMI_EnableCrop

Function name

HAL_StatusTypeDef HAL_DCMI_EnableCrop (DCMI_HandleTypeDef * hdcmi)

Function description

Enable the Crop feature.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: status

HAL_DCMI_DisableCrop

Function name

HAL_StatusTypeDef HAL_DCMI_DisableCrop (DCMI_HandleTypeDef * hdcmi)

Function description

Disable the Crop feature.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: status

HAL_DCMI_ConfigSyncUnmask

Function name

```
HAL_StatusTypeDef HAL_DCMI_ConfigSyncUnmask (DCMI_HandleTypeDef * hdcmi,
DCMI_SyncUnmaskTypeDef * SyncUnmask)
```

Function description

Set embedded synchronization delimiters unmasks.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.
- **SyncUnmask**: pointer to a DCMI_SyncUnmaskTypeDef structure that contains the embedded synchronization delimiters unmasks.

Return values

- **HAL**: status

HAL_DCMI_GetState

Function name

```
HAL_DCMI_StateTypeDef HAL_DCMI_GetState (DCMI_HandleTypeDef * hdcmi)
```

Function description

Return the DCMI state.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **HAL**: state

HAL_DCMI_GetError

Function name

```
uint32_t HAL_DCMI_GetError (DCMI_HandleTypeDef * hdcmi)
```

Function description

Return the DCMI error code.

Parameters

- **hdcmi**: pointer to a DCMI_HandleTypeDef structure that contains the configuration information for DCMI.

Return values

- **DCMI**: Error Code

14.3 DCMI Firmware driver defines

The following section lists the various define and macros of the module.

14.3.1

DCMI

DCMI

DCMI Capture Mode

DCMI_MODE_CONTINUOUS

The received data are transferred continuously into the destination memory through the DMA

DCMI_MODE_SNAPSHOT

Once activated, the interface waits for the start of frame and then transfers a single frame through the DMA

DCMI Capture Rate

DCMI_CR_ALL_FRAME

All frames are captured

DCMI_CR_ALTERNATE_2_FRAME

Every alternate frame captured

DCMI_CR_ALTERNATE_4_FRAME

One frame in 4 frames captured

DCMI Error Code

HAL_DCMI_ERROR_NONE

No error

HAL_DCMI_ERROR_OVR

Overrun error

HAL_DCMI_ERROR_SYNC

Synchronization error

HAL_DCMI_ERROR_TIMEOUT

Timeout error

HAL_DCMI_ERROR_DMA

DMA error

DCMI Exported Macros

__HAL_DCMI_RESET_HANDLE_STATE

Description:

- Reset DCMI handle state.

Parameters:

- `__HANDLE__`: specifies the DCMI handle.

Return value:

- None

__HAL_DCMI_ENABLE

Description:

- Enable the DCMI.

Parameters:

- `__HANDLE__`: DCMI handle

Return value:

- None

__HAL_DCMI_DISABLE

Description:

- Disable the DCMI.

Parameters:

- `__HANDLE__`: DCMI handle

Return value:

- None

__HAL_DCMI_GET_FLAG

Description:

- Get the DCMI pending flag.

Parameters:

- `__HANDLE__`: DCMI handle
- `__FLAG__`: Get the specified flag. This parameter can be one of the following values (no combination allowed)
 - `DCMI_FLAG_HSYNC`: HSYNC pin state (active line / synchronization between lines)
 - `DCMI_FLAG_VSYNC`: VSYNC pin state (active frame / synchronization between frames)
 - `DCMI_FLAG_FNE`: FIFO empty flag
 - `DCMI_FLAG_FRAMERI`: Frame capture complete flag mask
 - `DCMI_FLAG_OVRRI`: Overrun flag mask
 - `DCMI_FLAG_ERRRI`: Synchronization error flag mask
 - `DCMI_FLAG_VSYNCR`: VSYNC flag mask
 - `DCMI_FLAG_LINER`: Line flag mask
 - `DCMI_FLAG_FRAMEMI`: DCMI Capture complete masked interrupt status
 - `DCMI_FLAG_OVRMI`: DCMI Overrun masked interrupt status
 - `DCMI_FLAG_ERRMI`: DCMI Synchronization error masked interrupt status
 - `DCMI_FLAG_VSYNCMI`: DCMI VSYNC masked interrupt status
 - `DCMI_FLAG_LINEMI`: DCMI Line masked interrupt status

Return value:

- The: state of FLAG.

__HAL_DCMI_CLEAR_FLAG

Description:

- Clear the DCMI pending flags.

Parameters:

- `__HANDLE__`: DCMI handle
- `__FLAG__`: specifies the flag to clear. This parameter can be any combination of the following values:
 - `DCMI_FLAG_FRAMERI`: Frame capture complete flag mask
 - `DCMI_FLAG_OVRRI`: Overflow flag mask
 - `DCMI_FLAG_ERRRI`: Synchronization error flag mask
 - `DCMI_FLAG_VSYNCR`: VSYNC flag mask
 - `DCMI_FLAG_LINER`: Line flag mask

Return value:

- None

__HAL_DCMI_ENABLE_IT

Description:

- Enable the specified DCMI interrupts.

Parameters:

- `__HANDLE__`: DCMI handle
- `__INTERRUPT__`: specifies the DCMI interrupt sources to be enabled. This parameter can be any combination of the following values:
 - `DCMI_IT_FRAME`: Frame capture complete interrupt mask
 - `DCMI_IT_OVF`: Overflow interrupt mask
 - `DCMI_IT_ERR`: Synchronization error interrupt mask
 - `DCMI_IT_VSYNC`: VSYNC interrupt mask
 - `DCMI_IT_LINE`: Line interrupt mask

Return value:

- None

__HAL_DCMI_DISABLE_IT

Description:

- Disable the specified DCMI interrupts.

Parameters:

- `__HANDLE__`: DCMI handle
- `__INTERRUPT__`: specifies the DCMI interrupt sources to be enabled. This parameter can be any combination of the following values:
 - `DCMI_IT_FRAME`: Frame capture complete interrupt mask
 - `DCMI_IT_OVF`: Overflow interrupt mask
 - `DCMI_IT_ERR`: Synchronization error interrupt mask
 - `DCMI_IT_VSYNC`: VSYNC interrupt mask
 - `DCMI_IT_LINE`: Line interrupt mask

Return value:

- None

__HAL_DCMI_GET_IT_SOURCE

Description:

- Check whether the specified DCMI interrupt has occurred or not.

Parameters:

- `__HANDLE__`: DCMI handle
- `__INTERRUPT__`: specifies the DCMI interrupt source to check. This parameter can be one of the following values:
 - `DCMI_IT_FRAME`: Frame capture complete interrupt mask
 - `DCMI_IT_OVF`: Overflow interrupt mask
 - `DCMI_IT_ERR`: Synchronization error interrupt mask
 - `DCMI_IT_VSYNC`: VSYNC interrupt mask
 - `DCMI_IT_LINE`: Line interrupt mask

Return value:

- The: state of `INTERRUPT`.

DCMI Extended Data Mode

DCMI_EXTEND_DATA_8B

Interface captures 8-bit data on every pixel clock

DCMI_EXTEND_DATA_10B

Interface captures 10-bit data on every pixel clock

DCMI_EXTEND_DATA_12B

Interface captures 12-bit data on every pixel clock

DCMI_EXTEND_DATA_14B

Interface captures 14-bit data on every pixel clock

DCMI Flags

DCMI_FLAG_HSYNC

HSYNC pin state (active line / synchronization between lines)

DCMI_FLAG_VSYNC

VSYSN pin state (active frame / synchronization between frames)

DCMI_FLAG_FNE

FIFO not empty flag

DCMI_FLAG_FRAMERI

Frame capture complete interrupt flag

DCMI_FLAG_OVRRRI

Overrun interrupt flag

DCMI_FLAG_ERRRI

Synchronization error interrupt flag

DCMI_FLAG_VSYNCRRI

VSYSN interrupt flag

DCMI_FLAG_LINERI

Line interrupt flag

DCMI_FLAG_FRAMEMI

DCMI Frame capture complete masked interrupt status

DCMI_FLAG_OVRMI

DCMI Overrun masked interrupt status

DCMI_FLAG_ERRMI

DCMI Synchronization error masked interrupt status

DCMI_FLAG_VSYNCFMI

DCMI VSYSN masked interrupt status

DCMI_FLAG_LINEMI

DCMI Line masked interrupt status

DCMI HSYNC Polarity

DCMI_HSPOLARITY_LOW

Horizontal synchronization active Low

DCMI_HSPOLARITY_HIGH

Horizontal synchronization active High

DCMI interrupt sources

DCMI_IT_FRAME

Capture complete interrupt

DCMI_IT_OVR

Overrun interrupt

DCMI_IT_ERR

Synchronization error interrupt

DCMI_IT_VSYNC

VSYNC interrupt

DCMI_IT_LINE

Line interrupt

DCMI MODE JPEG

DCMI_JPEG_DISABLE

Mode JPEG Disabled

DCMI_JPEG_ENABLE

Mode JPEG Enabled

DCMI PIXCK Polarity

DCMI_PCKPOLARITY_FALLING

Pixel clock active on Falling edge

DCMI_PCKPOLARITY_RISING

Pixel clock active on Rising edge

DCMI Synchronization Mode

DCMI_SYNCHRO_HARDWARE

Hardware synchronization data capture (frame/line start/stop) is synchronized with the HSYNC/VSYNC signals

DCMI_SYNCHRO_EMBEDDED

Embedded synchronization data capture is synchronized with synchronization codes embedded in the data flow

DCMI VSYNC Polarity

DCMI_VSPOLARITY_LOW

Vertical synchronization active Low

DCMI_VSPOLARITY_HIGH

Vertical synchronization active High

DCMI Window Coordinate

DCMI_WINDOW_COORDINATE

Window coordinate

DCMI Window Height

DCMI_WINDOW_HEIGHT

Window Height

15 HAL DMA Generic Driver

15.1 DMA Firmware driver registers structures

15.1.1 DMA_InitTypeDef

DMA_InitTypeDef is defined in the `stm32f2xx_hal_dma.h`

Data Fields

- *uint32_t Channel*
- *uint32_t Direction*
- *uint32_t PeriphInc*
- *uint32_t MemInc*
- *uint32_t PeriphDataAlignment*
- *uint32_t MemDataAlignment*
- *uint32_t Mode*
- *uint32_t Priority*
- *uint32_t FIFOMode*
- *uint32_t FIFOThreshold*
- *uint32_t MemBurst*
- *uint32_t PeriphBurst*

Field Documentation

- *uint32_t DMA_InitTypeDef::Channel*
Specifies the channel used for the specified stream. This parameter can be a value of [DMA_Channel_selection](#)
 - *uint32_t DMA_InitTypeDef::Direction*
Specifies if the data will be transferred from memory to peripheral, from memory to memory or from peripheral to memory. This parameter can be a value of [DMA_Data_transfer_direction](#)
 - *uint32_t DMA_InitTypeDef::PeriphInc*
Specifies whether the Peripheral address register should be incremented or not. This parameter can be a value of [DMA_Peripheral_incremented_mode](#)
 - *uint32_t DMA_InitTypeDef::MemInc*
Specifies whether the memory address register should be incremented or not. This parameter can be a value of [DMA_Memory_incremented_mode](#)
 - *uint32_t DMA_InitTypeDef::PeriphDataAlignment*
Specifies the Peripheral data width. This parameter can be a value of [DMA_Peripheral_data_size](#)
 - *uint32_t DMA_InitTypeDef::MemDataAlignment*
Specifies the Memory data width. This parameter can be a value of [DMA_Memory_data_size](#)
 - *uint32_t DMA_InitTypeDef::Mode*
Specifies the operation mode of the DMAy Streamx. This parameter can be a value of [DMA_mode](#)
- Note:**
- The circular buffer mode cannot be used if the memory-to-memory data transfer is configured on the selected Stream
- *uint32_t DMA_InitTypeDef::Priority*
Specifies the software priority for the DMAy Streamx. This parameter can be a value of [DMA_Priority_level](#)

- **`uint32_t DMA_InitTypeDef::FIFOMode`**
Specifies if the FIFO mode or Direct mode will be used for the specified stream. This parameter can be a value of [DMA_FIFO_direct_mode](#)
Note:
 - The Direct mode (FIFO mode disabled) cannot be used if the memory-to-memory data transfer is configured on the selected stream
- **`uint32_t DMA_InitTypeDef::FIFOThreshold`**
Specifies the FIFO threshold level. This parameter can be a value of [DMA_FIFO_threshold_level](#)
- **`uint32_t DMA_InitTypeDef::MemBurst`**
Specifies the Burst transfer configuration for the memory transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_Memory_burst](#)
Note:
 - The burst mode is possible only if the address Increment mode is enabled.
- **`uint32_t DMA_InitTypeDef::PeriphBurst`**
Specifies the Burst transfer configuration for the peripheral transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_Peripheral_burst](#)
Note:
 - The burst mode is possible only if the address Increment mode is enabled.

15.1.2

`__DMA_HandleTypeDef`

`__DMA_HandleTypeDef` is defined in the `stm32f2xx_hal_dma.h`

Data Fields

- **`DMA_Stream_TypeDef * Instance`**
- **`DMA_InitTypeDef Init`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_DMA_StateTypeDef State`**
- **`void * Parent`**
- **`void(* XferCpltCallback`**
- **`void(* XferHalfCpltCallback`**
- **`void(* XferM1CpltCallback`**
- **`void(* XferM1HalfCpltCallback`**
- **`void(* XferErrorCallback`**
- **`void(* XferAbortCallback`**
- **`__IO uint32_t ErrorCode`**
- **`uint32_t StreamBaseAddress`**
- **`uint32_t StreamIndex`**

Field Documentation

- **`DMA_Stream_TypeDef* __DMA_HandleTypeDef::Instance`**
Register base address
- **`DMA_InitTypeDef __DMA_HandleTypeDef::Init`**
DMA communication parameters
- **`HAL_LockTypeDef __DMA_HandleTypeDef::Lock`**
DMA locking object
- **`__IO HAL_DMA_StateTypeDef __DMA_HandleTypeDef::State`**
DMA transfer state
- **`void* __DMA_HandleTypeDef::Parent`**
Parent object state
- **`void(* __DMA_HandleTypeDef::XferCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer complete callback

- **`void(* __DMA_HandleTypeDef::XferHalfCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA Half transfer complete callback
- **`void(* __DMA_HandleTypeDef::XferM1CpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer complete Memory1 callback
- **`void(* __DMA_HandleTypeDef::XferM1HalfCpltCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer Half complete Memory1 callback
- **`void(* __DMA_HandleTypeDef::XferErrorCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer error callback
- **`void(* __DMA_HandleTypeDef::XferAbortCallback)(struct __DMA_HandleTypeDef *hdma)`**
DMA transfer Abort callback
- **`__IO uint32_t __DMA_HandleTypeDef::ErrorCode`**
DMA Error code
- **`uint32_t __DMA_HandleTypeDef::StreamBaseAddress`**
DMA Stream Base Address
- **`uint32_t __DMA_HandleTypeDef::StreamIndex`**
DMA Stream Index

15.2 DMA Firmware driver API description

The following section lists the various functions of the DMA library.

15.2.1 How to use this driver

1. Enable and configure the peripheral to be connected to the DMA Stream (except for internal SRAM/FLASH memories: no initialization is necessary) please refer to Reference manual for connection between peripherals and DMA requests.
2. For a given Stream, program the required configuration through the following parameters: Transfer Direction, Source and Destination data formats, Circular, Normal or peripheral flow control mode, Stream Priority level, Source and Destination Increment mode, FIFO mode and its Threshold (if needed), Burst mode for Source and/or Destination (if needed) using `HAL_DMA_Init()` function.

Note: *Prior to `HAL_DMA_Init()` the clock must be enabled for DMA through the following macros: `__HAL_RCC_DMA1_CLK_ENABLE()` or `__HAL_RCC_DMA2_CLK_ENABLE()`.*

Polling mode IO operation

- Use `HAL_DMA_Start()` to start DMA transfer after the configuration of Source address and destination address and the Length of data to be transferred.
- Use `HAL_DMA_PollForTransfer()` to poll for the end of current transfer, in this case a fixed Timeout can be configured by User depending from his application.
- Use `HAL_DMA_Abort()` function to abort the current transfer.

Interrupt mode IO operation

- Configure the DMA interrupt priority using `HAL_NVIC_SetPriority()`
 - Enable the DMA IRQ handler using `HAL_NVIC_EnableIRQ()`
 - Use `HAL_DMA_Start_IT()` to start DMA transfer after the configuration of Source address and destination address and the Length of data to be transferred. In this case the DMA interrupt is configured
 - Use `HAL_DMA_IRQHandler()` called under `DMA_IRQHandler()` Interrupt subroutine
 - At the end of data transfer `HAL_DMA_IRQHandler()` function is executed and user can add his own function by customization of function pointer `XferCpltCallback` and `XferErrorCallback` (i.e a member of DMA handle structure).
1. Use `HAL_DMA_GetState()` function to return the DMA state and `HAL_DMA_GetError()` in case of error detection.

2. Use `HAL_DMA_Abort_IT()` function to abort the current transfer

Note: *In Memory-to-Memory transfer mode, Circular mode is not allowed.*

Note: *The FIFO is used mainly to reduce bus usage and to allow data packing/unpacking: it is possible to set different Data Sizes for the Peripheral and the Memory (ie. you can set Half-Word data size for the peripheral to access its data register and set Word data size for the Memory to gain in access time. Each two half words will be packed and written in a single access to a Word in the Memory).*

Note: *When FIFO is disabled, it is not allowed to configure different Data Sizes for Source and Destination. In this case the Peripheral Data Size will be applied to both Source and Destination.*

DMA HAL driver macros list

Below the list of most used macros in DMA HAL driver.

- `__HAL_DMA_ENABLE`: Enable the specified DMA Stream.
- `__HAL_DMA_DISABLE`: Disable the specified DMA Stream.
- `__HAL_DMA_GET_IT_SOURCE`: Check whether the specified DMA Stream interrupt has occurred or not.

Note: *You can refer to the DMA HAL driver header file for more useful macros*

15.2.2 Initialization and de-initialization functions

This section provides functions allowing to initialize the DMA Stream source and destination addresses, incrementation and data sizes, transfer direction, circular/normal mode selection, memory-to-memory mode selection and Stream priority value.

The `HAL_DMA_Init()` function follows the DMA configuration procedures as described in reference manual.

This section contains the following APIs:

- [`HAL\_DMA\_Init`](#)
- [`HAL\_DMA\_DeInit`](#)

15.2.3 IO operation functions

This section provides functions allowing to:

- Configure the source, destination address and data length and Start DMA transfer
- Configure the source, destination address and data length and Start DMA transfer with interrupt
- Abort DMA transfer
- Poll for transfer complete
- Handle DMA interrupt request

This section contains the following APIs:

- [`HAL\_DMA\_Start`](#)
- [`HAL\_DMA\_Start\_IT`](#)
- [`HAL\_DMA\_Abort`](#)
- [`HAL\_DMA\_Abort\_IT`](#)
- [`HAL\_DMA\_PollForTransfer`](#)
- [`HAL\_DMA\_IRQHandler`](#)
- [`HAL\_DMA\_RegisterCallback`](#)
- [`HAL\_DMA\_UnRegisterCallback`](#)
- [`HAL\_DMA\_CleanCallbacks`](#)

15.2.4 State and Errors functions

This subsection provides functions allowing to

- Check the DMA state
- Get error code

This section contains the following APIs:

- [`HAL\_DMA\_GetState`](#)
- [`HAL\_DMA\_GetError`](#)

15.2.5 Detailed description of functions

HAL_DMA_Init

Function name

HAL_StatusTypeDef HAL_DMA_Init (DMA_HandleTypeDef * hdma)

Function description

Initialize the DMA according to the specified parameters in the DMA_InitTypeDef and create the associated handle.

Parameters

- **hdma:** Pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

HAL_DMA_DeInit

Function name

HAL_StatusTypeDef HAL_DMA_DeInit (DMA_HandleTypeDef * hdma)

Function description

DeInitializes the DMA peripheral.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL:** status

HAL_DMA_Start

Function name

HAL_StatusTypeDef HAL_DMA_Start (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)

Function description

Starts the DMA Transfer.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMA_Start_IT

Function name

HAL_StatusTypeDef HAL_DMA_Start_IT (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength)

Function description

Start the DMA Transfer with interrupt enabled.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress**: The source memory Buffer address
- **DstAddress**: The destination memory Buffer address
- **DataLength**: The length of data to be transferred from source to destination

Return values

- **HAL**: status

HAL_DMA_Abort

Function name

HAL_StatusTypeDef HAL_DMA_Abort (DMA_HandleTypeDef * hdma)

Function description

Aborts the DMA Transfer.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL**: status

Notes

- After disabling a DMA Stream, a check for wait until the DMA Stream is effectively disabled is added. If a Stream is disabled while a data transfer is ongoing, the current data will be transferred and the Stream will be effectively disabled only after the transfer of this single data is finished.

HAL_DMA_Abort_IT

Function name

HAL_StatusTypeDef HAL_DMA_Abort_IT (DMA_HandleTypeDef * hdma)

Function description

Aborts the DMA Transfer in Interrupt mode.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL**: status

HAL_DMA_PollForTransfer

Function name

HAL_StatusTypeDef HAL_DMA_PollForTransfer (DMA_HandleTypeDef * hdma, HAL_DMA_LevelCompleteTypeDef CompleteLevel, uint32_t Timeout)

Function description

Polling for transfer complete.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CompleteLevel:** Specifies the DMA level complete.
- **Timeout:** Timeout duration.

Return values

- **HAL:** status

Notes

- The polling mode is kept in this version for legacy. it is recommended to use the IT model instead. This model could be used for debug purpose.
- The HAL_DMA_PollForTransfer API cannot be used in circular and double buffering mode (automatic circular mode).

HAL_DMA_IRQHandler

Function name

```
void HAL_DMA_IRQHandler (DMA_HandleTypeDef * hdma)
```

Function description

Handles DMA interrupt request.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **None:**

HAL_DMA_CleanCallbacks

Function name

```
HAL_StatusTypeDef HAL_DMA_CleanCallbacks (DMA_HandleTypeDef * hdma)
```

Function description

HAL_DMA_RegisterCallback

Function name

```
HAL_StatusTypeDef HAL_DMA_RegisterCallback (DMA_HandleTypeDef * hdma,  
HAL_DMA_CallbackIDTypeDef CallbackID, void(*)(DMA_HandleTypeDef * _hdma) pCallback)
```

Function description

Register callbacks.

Parameters

- **hdma:** pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CallbackID:** User Callback identifier a DMA_HandleTypeDef structure as parameter.
- **pCallback:** pointer to private callback function which has pointer to a DMA_HandleTypeDef structure as parameter.

Return values

- **HAL:** status

HAL_DMA_UnRegisterCallback

Function name

**HAL_StatusTypeDef HAL_DMA_UnRegisterCallback (DMA_HandleTypeDef * hdma,
HAL_DMA_CallbackIDTypeDef CallbackID)**

Function description

UnRegister callbacks.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **CallbackID**: User Callback identifier a HAL_DMA_CallbackIDTypeDef ENUM as parameter.

Return values

- **HAL**: status

HAL_DMA_GetState

Function name

HAL_DMA_StateTypeDef HAL_DMA_GetState (DMA_HandleTypeDef * hdma)

Function description

Returns the DMA state.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **HAL**: state

HAL_DMA_GetError

Function name

uint32_t HAL_DMA_GetError (DMA_HandleTypeDef * hdma)

Function description

Return the DMA error code.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.

Return values

- **DMA**: Error Code

15.3 DMA Firmware driver defines

The following section lists the various define and macros of the module.

15.3.1 DMA

DMA

DMA Channel selection

DMA_CHANNEL_0

DMA Channel 0

DMA_CHANNEL_1

DMA Channel 1

DMA_CHANNEL_2

DMA Channel 2

DMA_CHANNEL_3

DMA Channel 3

DMA_CHANNEL_4

DMA Channel 4

DMA_CHANNEL_5

DMA Channel 5

DMA_CHANNEL_6

DMA Channel 6

DMA_CHANNEL_7

DMA Channel 7

DMA Data transfer direction

DMA_PERIPH_TO_MEMORY

Peripheral to memory direction

DMA_MEMORY_TO_PERIPH

Memory to peripheral direction

DMA_MEMORY_TO_MEMORY

Memory to memory direction

DMA Error Code

HAL_DMA_ERROR_NONE

No error

HAL_DMA_ERROR_TE

Transfer error

HAL_DMA_ERROR_FE

FIFO error

HAL_DMA_ERROR_DME

Direct Mode error

HAL_DMA_ERROR_TIMEOUT

Timeout error

HAL_DMA_ERROR_PARAM

Parameter error

HAL_DMA_ERROR_NO_XFER

Abort requested with no Xfer ongoing

HAL_DMA_ERROR_NOT_SUPPORTED

Not supported mode

DMA FIFO direct mode

DMA_FIFOMODE_DISABLE

FIFO mode disable

DMA_FIFOMODE_ENABLE

FIFO mode enable

DMA FIFO threshold level**DMA_FIFO_THRESHOLD_1QUARTERFULL**

FIFO threshold 1 quart full configuration

DMA_FIFO_THRESHOLD_HALFFULL

FIFO threshold half full configuration

DMA_FIFO_THRESHOLD_3QUARTERSFULL

FIFO threshold 3 quarts full configuration

DMA_FIFO_THRESHOLD_FULL

FIFO threshold full configuration

DMA flag definitions**DMA_FLAG_FEIF0_4****DMA_FLAG_DMEIF0_4****DMA_FLAG_TEIF0_4****DMA_FLAG_HTIF0_4****DMA_FLAG_TCIF0_4****DMA_FLAG_FEIF1_5****DMA_FLAG_DMEIF1_5****DMA_FLAG_TEIF1_5****DMA_FLAG_HTIF1_5****DMA_FLAG_TCIF1_5****DMA_FLAG_FEIF2_6****DMA_FLAG_DMEIF2_6****DMA_FLAG_TEIF2_6****DMA_FLAG_HTIF2_6****DMA_FLAG_TCIF2_6****DMA_FLAG_FEIF3_7****DMA_FLAG_DMEIF3_7****DMA_FLAG_TEIF3_7**

DMA_FLAG_HTIF3_7

DMA_FLAG_TCIF3_7

TIM DMA Handle Index

TIM_DMA_ID_UPDATE

Index of the DMA handle used for Update DMA requests

TIM_DMA_ID_CC1

Index of the DMA handle used for Capture/Compare 1 DMA requests

TIM_DMA_ID_CC2

Index of the DMA handle used for Capture/Compare 2 DMA requests

TIM_DMA_ID_CC3

Index of the DMA handle used for Capture/Compare 3 DMA requests

TIM_DMA_ID_CC4

Index of the DMA handle used for Capture/Compare 4 DMA requests

TIM_DMA_ID_COMMUTATION

Index of the DMA handle used for Commutation DMA requests

TIM_DMA_ID_TRIGGER

Index of the DMA handle used for Trigger DMA requests

DMA interrupt enable definitions

DMA_IT_TC

DMA_IT_HT

DMA_IT_TE

DMA_IT_DME

DMA_IT_FE

DMA Memory burst

DMA_MBURST_SINGLE

DMA_MBURST_INC4

DMA_MBURST_INC8

DMA_MBURST_INC16

DMA Memory data size

DMA_MDATAALIGN_BYTE

Memory data alignment: Byte

DMA_MDATAALIGN_HALFWORD

Memory data alignment: HalfWord

DMA_MDATAALIGN_WORD

Memory data alignment: Word

DMA Memory incremented mode

DMA_MINC_ENABLE

Memory increment mode enable

DMA_MINC_DISABLE

Memory increment mode disable

DMA mode

DMA_NORMAL

Normal mode

DMA_CIRCULAR

Circular mode

DMA_PFCTRL

Peripheral flow control mode

DMA Peripheral burst

DMA_PBURST_SINGLE

DMA_PBURST_INC4

DMA_PBURST_INC8

DMA_PBURST_INC16

DMA Peripheral data size

DMA_PDATAALIGN_BYTE

Peripheral data alignment: Byte

DMA_PDATAALIGN_HALFWORD

Peripheral data alignment: HalfWord

DMA_PDATAALIGN_WORD

Peripheral data alignment: Word

DMA Peripheral incremented mode

DMA_PINC_ENABLE

Peripheral increment mode enable

DMA_PINC_DISABLE

Peripheral increment mode disable

DMA Priority level

DMA_PRIORITY_LOW

Priority level: Low

DMA_PRIORITY_MEDIUM

Priority level: Medium

DMA_PRIORITY_HIGH

Priority level: High

DMA_PRIORITY_VERY_HIGH

Priority level: Very High

16 HAL DMA Extension Driver

16.1 DMAEx Firmware driver API description

The following section lists the various functions of the DMAEx library.

16.1.1 How to use this driver

The DMA Extension HAL driver can be used as follows:

1. Start a multi buffer transfer using the `HAL_DMA_MultiBufferStart()` function for polling mode or `HAL_DMA_MultiBufferStart_IT()` for interrupt mode.

Note: In Memory-to-Memory transfer mode, Multi (Double) Buffer mode is not allowed.

Note: When Multi (Double) Buffer mode is enabled the, transfer is circular by default.

Note: In Multi (Double) buffer mode, it is possible to update the base address for the AHB memory port on the fly (`DMA_SxM0AR` or `DMA_SxM1AR`) when the stream is enabled.

16.1.2 Extended features functions

This section provides functions allowing to:

- Configure the source, destination address and data length and Start MultiBuffer DMA transfer
- Configure the source, destination address and data length and Start MultiBuffer DMA transfer with interrupt
- Change on the fly the memory0 or memory1 address.

This section contains the following APIs:

- [`HAL\_DMAEx\_MultiBufferStart`](#)
- [`HAL\_DMAEx\_MultiBufferStart\_IT`](#)
- [`HAL\_DMAEx\_ChangeMemory`](#)

16.1.3 Detailed description of functions

HAL_DMAEx_MultiBufferStart

Function name

`HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t SecondMemAddress, uint32_t DataLength)`

Function description

Starts the multi_buffer DMA Transfer.

Parameters

- **hdma:** pointer to a `DMA_HandleTypeDef` structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress:** The source memory Buffer address
- **DstAddress:** The destination memory Buffer address
- **SecondMemAddress:** The second memory Buffer address in case of multi buffer Transfer
- **DataLength:** The length of data to be transferred from source to destination

Return values

- **HAL:** status

HAL_DMAEx_MultiBufferStart_IT

Function name

`HAL_StatusTypeDef HAL_DMAEx_MultiBufferStart_IT (DMA_HandleTypeDef * hdma, uint32_t SrcAddress, uint32_t DstAddress, uint32_t SecondMemAddress, uint32_t DataLength)`

Function description

Starts the multi_buffer DMA Transfer with interrupt enabled.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **SrcAddress**: The source memory Buffer address
- **DstAddress**: The destination memory Buffer address
- **SecondMemAddress**: The second memory Buffer address in case of multi buffer Transfer
- **DataLength**: The length of data to be transferred from source to destination

Return values

- **HAL**: status

HAL_DMAEx_ChangeMemory

Function name

HAL_StatusTypeDef HAL_DMAEx_ChangeMemory (DMA_HandleTypeDef * hdma, uint32_t Address, HAL_DMA_MemoryTypeDef memory)

Function description

Change the memory0 or memory1 address on the fly.

Parameters

- **hdma**: pointer to a DMA_HandleTypeDef structure that contains the configuration information for the specified DMA Stream.
- **Address**: The new address
- **memory**: the memory to be changed, This parameter can be one of the following values: MEMORY0 / MEMORY1

Return values

- **HAL**: status

Notes

- The MEMORY0 address can be changed only when the current transfer use MEMORY1 and the MEMORY1 address can be changed only when the current transfer use MEMORY0.

17 HAL ETH Generic Driver

17.1 ETH Firmware driver registers structures

17.1.1 ETH_InitTypeDef

ETH_InitTypeDef is defined in the `stm32f2xx_hal_eth.h`

Data Fields

- **uint32_t AutoNegotiation**
- **uint32_t Speed**
- **uint32_t DuplexMode**
- **uint16_t PhyAddress**
- **uint8_t * MACAddr**
- **uint32_t RxMode**
- **uint32_t ChecksumMode**
- **uint32_t MedialInterface**

Field Documentation

- **uint32_t ETH_InitTypeDef::AutoNegotiation**
Selects or not the AutoNegotiation mode for the external PHY. The AutoNegotiation allows an automatic setting of the Speed (10/100Mbps) and the mode (half/full-duplex). This parameter can be a value of [ETH_AutoNegotiation](#)
- **uint32_t ETH_InitTypeDef::Speed**
Sets the Ethernet speed: 10/100 Mbps. This parameter can be a value of [ETH_Speed](#)
- **uint32_t ETH_InitTypeDef::DuplexMode**
Selects the MAC duplex mode: Half-Duplex or Full-Duplex mode. This parameter can be a value of [ETH_Duplex_Mode](#)
- **uint16_t ETH_InitTypeDef::PhyAddress**
Ethernet PHY address. This parameter must be a number between `Min_Data = 0` and `Max_Data = 32`
- **uint8_t * ETH_InitTypeDef::MACAddr**
MAC Address of used Hardware: must be pointer on an array of 6 bytes
- **uint32_t ETH_InitTypeDef::RxMode**
Selects the Ethernet Rx mode: Polling mode, Interrupt mode. This parameter can be a value of [ETH_Rx_Mode](#)
- **uint32_t ETH_InitTypeDef::ChecksumMode**
Selects if the checksum is checked by hardware or by software. This parameter can be a value of [ETH_Checksum_Mode](#)
- **uint32_t ETH_InitTypeDef::MedialInterface**
Selects the media-independent interface or the reduced media-independent interface. This parameter can be a value of [ETH_Media_Interface](#)

17.1.2 ETH_MACInitTypeDef

ETH_MACInitTypeDef is defined in the `stm32f2xx_hal_eth.h`

Data Fields

- **uint32_t Watchdog**
- **uint32_t Jabber**
- **uint32_t InterFrameGap**
- **uint32_t CarrierSense**
- **uint32_t ReceiveOwn**
- **uint32_t LoopbackMode**

- ***uint32_t ChecksumOffload***
- ***uint32_t RetryTransmission***
- ***uint32_t AutomaticPadCRCStrip***
- ***uint32_t BackOffLimit***
- ***uint32_t DeferralCheck***
- ***uint32_t ReceiveAll***
- ***uint32_t SourceAddrFilter***
- ***uint32_t PassControlFrames***
- ***uint32_t BroadcastFramesReception***
- ***uint32_t DestinationAddrFilter***
- ***uint32_t PromiscuousMode***
- ***uint32_t MulticastFramesFilter***
- ***uint32_t UnicastFramesFilter***
- ***uint32_t HashTableHigh***
- ***uint32_t HashTableLow***
- ***uint32_t PauseTime***
- ***uint32_t ZeroQuantaPause***
- ***uint32_t PauseLowThreshold***
- ***uint32_t UnicastPauseFrameDetect***
- ***uint32_t ReceiveFlowControl***
- ***uint32_t TransmitFlowControl***
- ***uint32_t VLANTagComparison***
- ***uint32_t VLANTagIdentifier***

Field Documentation

- ***uint32_t ETH_MACInitTypeDef::Watchdog***
Selects or not the Watchdog timer When enabled, the MAC allows no more then 2048 bytes to be received. When disabled, the MAC can receive up to 16384 bytes. This parameter can be a value of [ETH_Watchdog](#)
- ***uint32_t ETH_MACInitTypeDef::Jabber***
Selects or not Jabber timer When enabled, the MAC allows no more then 2048 bytes to be sent. When disabled, the MAC can send up to 16384 bytes. This parameter can be a value of [ETH_Jabber](#)
- ***uint32_t ETH_MACInitTypeDef::InterFrameGap***
Selects the minimum IFG between frames during transmission. This parameter can be a value of [ETH_Inter_Frame_Gap](#)
- ***uint32_t ETH_MACInitTypeDef::CarrierSense***
Selects or not the Carrier Sense. This parameter can be a value of [ETH_Carrier_Sense](#)
- ***uint32_t ETH_MACInitTypeDef::ReceiveOwn***
Selects or not the ReceiveOwn, ReceiveOwn allows the reception of frames when the TX_EN signal is asserted in Half-Duplex mode. This parameter can be a value of [ETH_Receive_Own](#)
- ***uint32_t ETH_MACInitTypeDef::LoopbackMode***
Selects or not the internal MAC MII Loopback mode. This parameter can be a value of [ETH_Loop_Back_Mode](#)
- ***uint32_t ETH_MACInitTypeDef::ChecksumOffload***
Selects or not the IPv4 checksum checking for received frame payloads' TCP/UDP/ICMP headers. This parameter can be a value of [ETH_Checksum_Offload](#)
- ***uint32_t ETH_MACInitTypeDef::RetryTransmission***
Selects or not the MAC attempt retries transmission, based on the settings of BL, when a collision occurs (Half-Duplex mode). This parameter can be a value of [ETH_Retry_Transmission](#)
- ***uint32_t ETH_MACInitTypeDef::AutomaticPadCRCStrip***
Selects or not the Automatic MAC Pad/CRC Stripping. This parameter can be a value of [ETH_Automatic_Pad_CRC_Strip](#)

- **`uint32_t ETH_MACInitTypeDef::BackOffLimit`**
Selects the BackOff limit value. This parameter can be a value of [ETH_Back_Off_Limit](#)
- **`uint32_t ETH_MACInitTypeDef::DeferralCheck`**
Selects or not the deferral check function (Half-Duplex mode). This parameter can be a value of [ETH_Deferral_Check](#)
- **`uint32_t ETH_MACInitTypeDef::ReceiveAll`**
Selects or not all frames reception by the MAC (No filtering). This parameter can be a value of [ETH_Receive_All](#)
- **`uint32_t ETH_MACInitTypeDef::SourceAddrFilter`**
Selects the Source Address Filter mode. This parameter can be a value of [ETH_Source_Addr_Filter](#)
- **`uint32_t ETH_MACInitTypeDef::PassControlFrames`**
Sets the forwarding mode of the control frames (including unicast and multicast PAUSE frames) This parameter can be a value of [ETH_Pass_Control_Frames](#)
- **`uint32_t ETH_MACInitTypeDef::BroadcastFramesReception`**
Selects or not the reception of Broadcast Frames. This parameter can be a value of [ETH_Broadcast_Frames_Reception](#)
- **`uint32_t ETH_MACInitTypeDef::DestinationAddrFilter`**
Sets the destination filter mode for both unicast and multicast frames. This parameter can be a value of [ETH_Destination_Addr_Filter](#)
- **`uint32_t ETH_MACInitTypeDef::PromiscuousMode`**
Selects or not the Promiscuous Mode This parameter can be a value of [ETH_Promiscuous_Mode](#)
- **`uint32_t ETH_MACInitTypeDef::MulticastFramesFilter`**
Selects the Multicast Frames filter mode: None/HashTableFilter/PerfectFilter/PerfectHashTableFilter. This parameter can be a value of [ETH_Multicast_Frames_Filter](#)
- **`uint32_t ETH_MACInitTypeDef::UnicastFramesFilter`**
Selects the Unicast Frames filter mode: HashTableFilter/PerfectFilter/PerfectHashTableFilter. This parameter can be a value of [ETH_Unicast_Frames_Filter](#)
- **`uint32_t ETH_MACInitTypeDef::HashTableHigh`**
This field holds the higher 32 bits of Hash table. This parameter must be a number between Min_Data = 0x0 and Max_Data = 0xFFFFFFFF
- **`uint32_t ETH_MACInitTypeDef::HashTableLow`**
This field holds the lower 32 bits of Hash table. This parameter must be a number between Min_Data = 0x0 and Max_Data = 0xFFFFFFFF
- **`uint32_t ETH_MACInitTypeDef::PauseTime`**
This field holds the value to be used in the Pause Time field in the transmit control frame. This parameter must be a number between Min_Data = 0x0 and Max_Data = 0xFFFF
- **`uint32_t ETH_MACInitTypeDef::ZeroQuantaPause`**
Selects or not the automatic generation of Zero-Quanta Pause Control frames. This parameter can be a value of [ETH_Zero_Quanta_Pause](#)
- **`uint32_t ETH_MACInitTypeDef::PauseLowThreshold`**
This field configures the threshold of the PAUSE to be checked for automatic retransmission of PAUSE Frame. This parameter can be a value of [ETH_Pause_Low_Threshold](#)
- **`uint32_t ETH_MACInitTypeDef::UnicastPauseFrameDetect`**
Selects or not the MAC detection of the Pause frames (with MAC Address0 unicast address and unique multicast address). This parameter can be a value of [ETH_Unicast_Pause_Frame_Detect](#)
- **`uint32_t ETH_MACInitTypeDef::ReceiveFlowControl`**
Enables or disables the MAC to decode the received Pause frame and disable its transmitter for a specified time (Pause Time) This parameter can be a value of [ETH_Receive_Flow_Control](#)
- **`uint32_t ETH_MACInitTypeDef::TransmitFlowControl`**
Enables or disables the MAC to transmit Pause frames (Full-Duplex mode) or the MAC back-pressure operation (Half-Duplex mode) This parameter can be a value of [ETH_Transmit_Flow_Control](#)

- **`uint32_t ETH_MACInitTypeDef::VLANTagComparison`**
Selects the 12-bit VLAN identifier or the complete 16-bit VLAN tag for comparison and filtering. This parameter can be a value of [ETH_VLAN_Tag_Comparison](#)
- **`uint32_t ETH_MACInitTypeDef::VLANTagIdentifier`**
Holds the VLAN tag identifier for receive frames

17.1.3

ETH_DMAInitTypeDef

ETH_DMAInitTypeDef is defined in the `stm32f2xx_hal_eth.h`

Data Fields

- **`uint32_t DropTCPIPChecksumErrorFrame`**
- **`uint32_t ReceiveStoreForward`**
- **`uint32_t FlushReceivedFrame`**
- **`uint32_t TransmitStoreForward`**
- **`uint32_t TransmitThresholdControl`**
- **`uint32_t ForwardErrorFrames`**
- **`uint32_t ForwardUndersizedGoodFrames`**
- **`uint32_t ReceiveThresholdControl`**
- **`uint32_t SecondFrameOperate`**
- **`uint32_t AddressAlignedBeats`**
- **`uint32_t FixedBurst`**
- **`uint32_t RxDMA BurstLength`**
- **`uint32_t TxDMA BurstLength`**
- **`uint32_t EnhancedDescriptorFormat`**
- **`uint32_t DescriptorSkipLength`**
- **`uint32_t DMA Arbitration`**

Field Documentation

- **`uint32_t ETH_DMAInitTypeDef::DropTCPIPChecksumErrorFrame`**
Selects or not the Dropping of TCP/IP Checksum Error Frames. This parameter can be a value of [ETH_Drop_TCP_IP_Checksum_Error_Frame](#)
- **`uint32_t ETH_DMAInitTypeDef::ReceiveStoreForward`**
Enables or disables the Receive store and forward mode. This parameter can be a value of [ETH_Receive_Store_Forward](#)
- **`uint32_t ETH_DMAInitTypeDef::FlushReceivedFrame`**
Enables or disables the flushing of received frames. This parameter can be a value of [ETH_Flush_Received_Frame](#)
- **`uint32_t ETH_DMAInitTypeDef::TransmitStoreForward`**
Enables or disables Transmit store and forward mode. This parameter can be a value of [ETH_Transmit_Store_Forward](#)
- **`uint32_t ETH_DMAInitTypeDef::TransmitThresholdControl`**
Selects or not the Transmit Threshold Control. This parameter can be a value of [ETH_Transmit_Threshold_Control](#)
- **`uint32_t ETH_DMAInitTypeDef::ForwardErrorFrames`**
Selects or not the forward to the DMA of erroneous frames. This parameter can be a value of [ETH_Forward_Error_Frames](#)
- **`uint32_t ETH_DMAInitTypeDef::ForwardUndersizedGoodFrames`**
Enables or disables the Rx FIFO to forward Undersized frames (frames with no Error and length less than 64 bytes) including pad-bytes and CRC) This parameter can be a value of [ETH_Forward_Undersized_Good_Frames](#)
- **`uint32_t ETH_DMAInitTypeDef::ReceiveThresholdControl`**
Selects the threshold level of the Receive FIFO. This parameter can be a value of [ETH_Receive_Threshold_Control](#)

- **`uint32_t ETH_DMAInitTypeDef::SecondFrameOperate`**
Selects or not the Operate on second frame mode, which allows the DMA to process a second frame of Transmit data even before obtaining the status for the first frame. This parameter can be a value of [ETH_Second_Frame_Operate](#)
- **`uint32_t ETH_DMAInitTypeDef::AddressAlignedBeats`**
Enables or disables the Address Aligned Beats. This parameter can be a value of [ETH_Address_Aligned_Beats](#)
- **`uint32_t ETH_DMAInitTypeDef::FixedBurst`**
Enables or disables the AHB Master interface fixed burst transfers. This parameter can be a value of [ETH_Fixed_Burst](#)
- **`uint32_t ETH_DMAInitTypeDef::RxDMABurstLength`**
Indicates the maximum number of beats to be transferred in one Rx DMA transaction. This parameter can be a value of [ETH_Rx_DMA_Burst_Length](#)
- **`uint32_t ETH_DMAInitTypeDef::TxDMABurstLength`**
Indicates the maximum number of beats to be transferred in one Tx DMA transaction. This parameter can be a value of [ETH_Tx_DMA_Burst_Length](#)
- **`uint32_t ETH_DMAInitTypeDef::EnhancedDescriptorFormat`**
Enables the enhanced descriptor format. This parameter can be a value of [ETH_DMA_Enhanced_descriptor_format](#)
- **`uint32_t ETH_DMAInitTypeDef::DescriptorSkipLength`**
Specifies the number of word to skip between two unchained descriptors (Ring mode) This parameter must be a number between Min_Data = 0 and Max_Data = 32
- **`uint32_t ETH_DMAInitTypeDef::DMAArbitration`**
Selects the DMA Tx/Rx arbitration. This parameter can be a value of [ETH_DMA_Arbitration](#)

17.1.4

ETH_DMADescTypeDef

ETH_DMADescTypeDef is defined in the `stm32f2xx_hal_eth.h`

Data Fields

- **`__IO uint32_t Status`**
- **`uint32_t ControlBufferSize`**
- **`uint32_t Buffer1Addr`**
- **`uint32_t Buffer2NextDescAddr`**
- **`uint32_t ExtendedStatus`**
- **`uint32_t Reserved1`**
- **`uint32_t TimeStampLow`**
- **`uint32_t TimeStampHigh`**

Field Documentation

- **`__IO uint32_t ETH_DMADescTypeDef::Status`**
Status
- **`uint32_t ETH_DMADescTypeDef::ControlBufferSize`**
Control and Buffer1, Buffer2 lengths
- **`uint32_t ETH_DMADescTypeDef::Buffer1Addr`**
Buffer1 address pointer
- **`uint32_t ETH_DMADescTypeDef::Buffer2NextDescAddr`**
Buffer2 or next descriptor address pointer Enhanced ETHERNET DMA PTP Descriptors
- **`uint32_t ETH_DMADescTypeDef::ExtendedStatus`**
Extended status for PTP receive descriptor
- **`uint32_t ETH_DMADescTypeDef::Reserved1`**
Reserved
- **`uint32_t ETH_DMADescTypeDef::TimeStampLow`**
Time Stamp Low value for transmit and receive

- ***uint32_t ETH_DMADescTypeDef::TimeStampHigh***
Time Stamp High value for transmit and receive

17.1.5

ETH_DMARxFramelInfos

ETH_DMARxFramelInfos is defined in the stm32f2xx_hal_eth.h

Data Fields

- ***ETH_DMADescTypeDef * FSRxDesc***
- ***ETH_DMADescTypeDef * LSRxDesc***
- ***uint32_t SegCount***
- ***uint32_t length***
- ***uint32_t buffer***

Field Documentation

- ***ETH_DMADescTypeDef* ETH_DMARxFramelInfos::FSRxDesc***
First Segment Rx Desc
- ***ETH_DMADescTypeDef* ETH_DMARxFramelInfos::LSRxDesc***
Last Segment Rx Desc
- ***uint32_t ETH_DMARxFramelInfos::SegCount***
Segment count
- ***uint32_t ETH_DMARxFramelInfos::length***
Frame length
- ***uint32_t ETH_DMARxFramelInfos::buffer***
Frame buffer

17.1.6

ETH_HandleTypeDef

ETH_HandleTypeDef is defined in the stm32f2xx_hal_eth.h

Data Fields

- ***ETH_TypeDef * Instance***
- ***ETH_InitTypeDef Init***
- ***uint32_t LinkStatus***
- ***ETH_DMADescTypeDef * RxDesc***
- ***ETH_DMADescTypeDef * TxDesc***
- ***ETH_DMARxFramelInfos RxFrameInfos***
- ***__IO HAL_ETH_StateTypeDef State***
- ***HAL_LockTypeDef Lock***

Field Documentation

- ***ETH_TypeDef* ETH_HandleTypeDef::Instance***
Register base address
- ***ETH_InitTypeDef ETH_HandleTypeDef::Init***
Ethernet Init Configuration
- ***uint32_t ETH_HandleTypeDef::LinkStatus***
Ethernet link status
- ***ETH_DMADescTypeDef* ETH_HandleTypeDef::RxDesc***
Rx descriptor to Get
- ***ETH_DMADescTypeDef* ETH_HandleTypeDef::TxDesc***
Tx descriptor to Set
- ***ETH_DMARxFramelInfos ETH_HandleTypeDef::RxFrameInfos***
last Rx frame infos
- ***__IO HAL_ETH_StateTypeDef ETH_HandleTypeDef::State***
ETH communication state

- ***HAL_LockTypeDef ETH_HandleTypeDef::Lock***
ETH Lock

17.2 ETH Firmware driver API description

The following section lists the various functions of the ETH library.

17.2.1 How to use this driver

1. Declare a `ETH_HandleTypeDef` handle structure, for example: `ETH_HandleTypeDef heth;`
2. Fill parameters of `Init` structure in `heth` handle
3. Call `HAL_ETH_Init()` API to initialize the Ethernet peripheral (MAC, DMA, ...)
4. Initialize the ETH low level resources through the `HAL_ETH_MspInit()` API:
 - a. Enable the Ethernet interface clock using
 - `__HAL_RCC_ETHMAC_CLK_ENABLE();`
 - `__HAL_RCC_ETHMACTX_CLK_ENABLE();`
 - `__HAL_RCC_ETHMACRX_CLK_ENABLE();`
 - b. Initialize the related GPIO clocks
 - c. Configure Ethernet pin-out
 - d. Configure Ethernet NVIC interrupt (IT mode)
5. Initialize Ethernet DMA Descriptors in chain mode and point to allocated buffers:
 - a. `HAL_ETH_DMATxDescListInit();` for Transmission process
 - b. `HAL_ETH_DMARxDescListInit();` for Reception process
6. Enable MAC and DMA transmission and reception:
 - a. `HAL_ETH_Start();`
7. Prepare ETH DMA TX Descriptors and give the hand to ETH DMA to transfer the frame to MAC TX FIFO:
 - a. `HAL_ETH_TransmitFrame();`
8. Poll for a received frame in ETH RX DMA Descriptors and get received frame parameters
 - a. `HAL_ETH_GetReceivedFrame();` (should be called into an infinite loop)
9. Get a received frame when an ETH RX interrupt occurs:
 - a. `HAL_ETH_GetReceivedFrame_IT();` (called in IT mode only)
10. Communicate with external PHY device:
 - a. Read a specific register from the PHY `HAL_ETH_ReadPHYRegister();`
 - b. Write data to a specific RHY register: `HAL_ETH_WritePHYRegister();`
11. Configure the Ethernet MAC after ETH peripheral initialization `HAL_ETH_ConfigMAC();` all MAC parameters should be filled.
12. Configure the Ethernet DMA after ETH peripheral initialization `HAL_ETH_ConfigDMA();` all DMA parameters should be filled.

Callback registration

17.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize and configure the Ethernet peripheral
- De-initialize the Ethernet peripheral

This section contains the following APIs:

- ***HAL_ETH_Init***
- ***HAL_ETH_DeInit***
- ***HAL_ETH_DMATxDescListInit***
- ***HAL_ETH_DMARxDescListInit***
- ***HAL_ETH_MspInit***
- ***HAL_ETH_MspDeInit***

17.2.3 IO operation functions

This section provides functions allowing to:

- Transmit a frame HAL_ETH_TransmitFrame();
- Receive a frame HAL_ETH_GetReceivedFrame(); HAL_ETH_GetReceivedFrame_IT();
- Read from an External PHY register HAL_ETH_ReadPHYRegister();
- Write to an External PHY register HAL_ETH_WritePHYRegister();

This section contains the following APIs:

- [HAL_ETH_TransmitFrame](#)
- [HAL_ETH_GetReceivedFrame](#)
- [HAL_ETH_GetReceivedFrame_IT](#)
- [HAL_ETH_IRQHandler](#)
- [HAL_ETH_TxCpltCallback](#)
- [HAL_ETH_RxCpltCallback](#)
- [HAL_ETH_ErrorCallback](#)
- [HAL_ETH_ReadPHYRegister](#)
- [HAL_ETH_WritePHYRegister](#)

17.2.4 Peripheral Control functions

This section provides functions allowing to:

- Enable MAC and DMA transmission and reception. HAL_ETH_Start();
- Disable MAC and DMA transmission and reception. HAL_ETH_Stop();
- Set the MAC configuration in runtime mode HAL_ETH_ConfigMAC();
- Set the DMA configuration in runtime mode HAL_ETH_ConfigDMA();

This section contains the following APIs:

- [HAL_ETH_Start](#)
- [HAL_ETH_Stop](#)
- [HAL_ETH_ConfigMAC](#)
- [HAL_ETH_ConfigDMA](#)

17.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

- Get the ETH handle state: HAL_ETH_GetState();

This section contains the following APIs:

- [HAL_ETH_GetState](#)

17.2.6 Detailed description of functions

HAL_ETH_Init

Function name

HAL_StatusTypeDef HAL_ETH_Init (ETH_HandleTypeDef * heth)

Function description

Initializes the Ethernet MAC and DMA according to default parameters.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL**: status

HAL_ETH_DeInit

Function name

HAL_StatusTypeDef HAL_ETH_DeInit (ETH_HandleTypeDef * heth)

Function description

De-Initializes the ETH peripheral.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL**: status

HAL_ETH_MspInit

Function name

void HAL_ETH_MspInit (ETH_HandleTypeDef * heth)

Function description

Initializes the ETH MSP.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **None**:

HAL_ETH_MspDeInit

Function name

void HAL_ETH_MspDeInit (ETH_HandleTypeDef * heth)

Function description

DeInitializes ETH MSP.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **None**:

HAL_ETH_DMATxDescListInit

Function name

HAL_StatusTypeDef HAL_ETH_DMATxDescListInit (ETH_HandleTypeDef * heth, ETH_DMADescTypeDef * DMATxDescTab, uint8_t * TxBuff, uint32_t TxBuffCount)

Function description

Initializes the DMA Tx descriptors in chain mode.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **DMATxDescTab**: Pointer to the first Tx desc list
- **TxBuff**: Pointer to the first TxBuffer list
- **TxBuffCount**: Number of the used Tx desc in the list

Return values

- **HAL**: status

HAL_ETH_DMARxDescListInit

Function name

```
HAL_StatusTypeDef HAL_ETH_DMARxDescListInit (ETH_HandleTypeDef * heth, ETH_DMADescTypeDef * DMARxDescTab, uint8_t * RxBuff, uint32_t RxBuffCount)
```

Function description

Initializes the DMA Rx descriptors in chain mode.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **DMARxDescTab**: Pointer to the first Rx desc list
- **RxBuff**: Pointer to the first RxBuffer list
- **RxBuffCount**: Number of the used Rx desc in the list

Return values

- **HAL**: status

HAL_ETH_TransmitFrame

Function name

```
HAL_StatusTypeDef HAL_ETH_TransmitFrame (ETH_HandleTypeDef * heth, uint32_t FrameLength)
```

Function description

Sends an Ethernet frame.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **FrameLength**: Amount of data to be sent

Return values

- **HAL**: status

HAL_ETH_GetReceivedFrame

Function name

```
HAL_StatusTypeDef HAL_ETH_GetReceivedFrame (ETH_HandleTypeDef * heth)
```

Function description

Checks for received frames.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL:** status

HAL_ETH_ReadPHYRegister

Function name

HAL_StatusTypeDef HAL_ETH_ReadPHYRegister (ETH_HandleTypeDef * heth, uint16_t PHYReg, uint32_t * RegValue)

Function description

Reads a PHY register.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **PHYReg:** PHY register address, is the index of one of the 32 PHY register. This parameter can be one of the following values: PHY_BCR: Transceiver Basic Control Register, PHY_BSR: Transceiver Basic Status Register. More PHY register could be read depending on the used PHY
- **RegValue:** PHY register value

Return values

- **HAL:** status

HAL_ETH_WritePHYRegister

Function name

HAL_StatusTypeDef HAL_ETH_WritePHYRegister (ETH_HandleTypeDef * heth, uint16_t PHYReg, uint32_t RegValue)

Function description

Writes to a PHY register.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **PHYReg:** PHY register address, is the index of one of the 32 PHY register. This parameter can be one of the following values: PHY_BCR: Transceiver Control Register. More PHY register could be written depending on the used PHY
- **RegValue:** the value to write

Return values

- **HAL:** status

HAL_ETH_GetReceivedFrame_IT

Function name

HAL_StatusTypeDef HAL_ETH_GetReceivedFrame_IT (ETH_HandleTypeDef * heth)

Function description

Gets the Received frame in interrupt mode.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL:** status

HAL_ETH_IRQHandler

Function name

void HAL_ETH_IRQHandler (ETH_HandleTypeDef * heth)

Function description

This function handles ETH interrupt request.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL**: status

HAL_ETH_TxCpltCallback

Function name

void HAL_ETH_TxCpltCallback (ETH_HandleTypeDef * heth)

Function description

Tx Transfer completed callbacks.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **None**:

HAL_ETH_RxCpltCallback

Function name

void HAL_ETH_RxCpltCallback (ETH_HandleTypeDef * heth)

Function description

Rx Transfer completed callbacks.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **None**:

HAL_ETH_ErrorCallback

Function name

void HAL_ETH_ErrorCallback (ETH_HandleTypeDef * heth)

Function description

Ethernet transfer error callbacks.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **None:**

HAL_ETH_Start

Function name

HAL_StatusTypeDef HAL_ETH_Start (ETH_HandleTypeDef * heth)

Function description

Enables Ethernet MAC and DMA reception/transmission.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL:** status

HAL_ETH_Stop

Function name

HAL_StatusTypeDef HAL_ETH_Stop (ETH_HandleTypeDef * heth)

Function description

Stop Ethernet MAC and DMA reception/transmission.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL:** status

HAL_ETH_ConfigMAC

Function name

HAL_StatusTypeDef HAL_ETH_ConfigMAC (ETH_HandleTypeDef * heth, ETH_MACInitTypeDef * macconf)

Function description

Set ETH MAC Configuration.

Parameters

- **heth:** pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **macconf:** MAC Configuration structure

Return values

- **HAL:** status

HAL_ETH_ConfigDMA

Function name

HAL_StatusTypeDef HAL_ETH_ConfigDMA (ETH_HandleTypeDef * heth, ETH_DMAInitTypeDef * dmaconf)

Function description

Sets ETH DMA Configuration.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module
- **dmaconf**: DMA Configuration structure

Return values

- **HAL**: status

HAL_ETH_GetState

Function name

HAL_ETH_StateTypeDef HAL_ETH_GetState (ETH_HandleTypeDef * heth)

Function description

Return the ETH HAL state.

Parameters

- **heth**: pointer to a ETH_HandleTypeDef structure that contains the configuration information for ETHERNET module

Return values

- **HAL**: state

17.3 ETH Firmware driver defines

The following section lists the various define and macros of the module.

17.3.1 ETH

ETH

ETH Address Aligned Beats

ETH_ADDRESSALIGNEDBEATS_ENABLE

ETH_ADDRESSALIGNEDBEATS_DISABLE

ETH Automatic Pad CRC Strip

ETH_AUTOMATICPADCRCSTRIP_ENABLE

ETH_AUTOMATICPADCRCSTRIP_DISABLE

ETH AutoNegotiation

ETH_AUTONEGOTIATION_ENABLE

ETH_AUTONEGOTIATION_DISABLE

ETH Back Off Limit

ETH_BACKOFFLIMIT_10

ETH_BACKOFFLIMIT_8

ETH_BACKOFFLIMIT_4

ETH_BACKOFFLIMIT_1

ETH Broadcast Frames Reception

ETH_BROADCASTFRAMESRECEPTION_ENABLE

ETH_BROADCASTFRAMESRECEPTION_DISABLE

ETH Buffers setting

ETH_MAX_PACKET_SIZE

ETH_HEADER + ETH_EXTRA + ETH_VLAN_TAG + ETH_MAX_ETH_PAYLOAD + ETH_CRC

ETH_HEADER

6 byte Dest addr, 6 byte Src addr, 2 byte length/type

ETH_CRC

Ethernet CRC

ETH_EXTRA

Extra bytes in some cases

ETH_VLAN_TAG

optional 802.1q VLAN Tag

ETH_MIN_ETH_PAYLOAD

Minimum Ethernet payload size

ETH_MAX_ETH_PAYLOAD

Maximum Ethernet payload size

ETH_JUMBO_FRAME_PAYLOAD

Jumbo frame payload size

ETH_RX_BUF_SIZE

ETH_RXBUFNB

ETH_TX_BUF_SIZE

ETH_TXBUFNB

ETH Carrier Sense

ETH_CARRIERSENCE_ENABLE

ETH_CARRIERSENCE_DISABLE

ETH Checksum Mode

ETH_CHECKSUM_BY_HARDWARE

ETH_CHECKSUM_BY_SOFTWARE

ETH Checksum Offload

ETH_CHECKSUMOFFLOAD_ENABLE

ETH_CHECKSUMOFFLOAD_DISABLE

ETH Deferral Check

ETH_DEFFERRALCHECK_ENABLE

ETH_DEFFERRALCHECK_DISABLE

ETH Destination Addr Filter

ETH_DESTINATIONADDRFILTER_NORMAL

ETH_DESTINATIONADDRFILTER_INVERSE

ETH DMA Arbitration

ETH_DMAARBITRATION_ROUNDROBIN_RXTX_1_1

ETH_DMAARBITRATION_ROUNDROBIN_RXTX_2_1

ETH_DMAARBITRATION_ROUNDROBIN_RXTX_3_1

ETH_DMAARBITRATION_ROUNDROBIN_RXTX_4_1

ETH_DMAARBITRATION_RXPRIORTX

ETH DMA Enhanced descriptor format

ETH_DMAENHANCEDDESCRIPTOR_ENABLE

ETH_DMAENHANCEDDESCRIPTOR_DISABLE

ETH DMA Flags

ETH_DMA_FLAG_TST

Time-stamp trigger interrupt (on DMA)

ETH_DMA_FLAG_PMT

PMT interrupt (on DMA)

ETH_DMA_FLAG_MMC

MMC interrupt (on DMA)

ETH_DMA_FLAG_DATATRANSFERERROR

Error bits 0-Rx DMA, 1-Tx DMA

ETH_DMA_FLAG_READWRITEERROR

Error bits 0-write transfer, 1-read transfer

ETH_DMA_FLAG_ACCESSERROR

Error bits 0-data buffer, 1-desc. access

ETH_DMA_FLAG_NIS

Normal interrupt summary flag

ETH_DMA_FLAG_AIS

Abnormal interrupt summary flag

ETH_DMA_FLAG_ER

Early receive flag

ETH_DMA_FLAG_FBE

Fatal bus error flag

ETH_DMA_FLAG_ET

Early transmit flag

ETH_DMA_FLAG_RWT

Receive watchdog timeout flag

ETH_DMA_FLAG_RPS

Receive process stopped flag

ETH_DMA_FLAG_RBU

Receive buffer unavailable flag

ETH_DMA_FLAG_R

Receive flag

ETH_DMA_FLAG_TU

Underflow flag

ETH_DMA_FLAG_RO

Overflow flag

ETH_DMA_FLAG_TJT

Transmit jabber timeout flag

ETH_DMA_FLAG_TBU

Transmit buffer unavailable flag

ETH_DMA_FLAG_TPS

Transmit process stopped flag

ETH_DMA_FLAG_T

Transmit flag

ETH DMA Interrupts**ETH_DMA_IT_TST**

Time-stamp trigger interrupt (on DMA)

ETH_DMA_IT_PMT

PMT interrupt (on DMA)

ETH_DMA_IT_MMC

MMC interrupt (on DMA)

ETH_DMA_IT_NIS

Normal interrupt summary

ETH_DMA_IT_AIS

Abnormal interrupt summary

ETH_DMA_IT_ER

Early receive interrupt

ETH_DMA_IT_FBE

Fatal bus error interrupt

ETH_DMA_IT_ET

Early transmit interrupt

ETH_DMA_IT_RWT

Receive watchdog timeout interrupt

ETH_DMA_IT_RPS

Receive process stopped interrupt

ETH_DMA_IT_RBU

Receive buffer unavailable interrupt

ETH_DMA_IT_R

Receive interrupt

ETH_DMA_IT_TU

Underflow interrupt

ETH_DMA_IT_RO

Overflow interrupt

ETH_DMA_IT_TJT

Transmit jabber timeout interrupt

ETH_DMA_IT_TBU

Transmit buffer unavailable interrupt

ETH_DMA_IT_TPS

Transmit process stopped interrupt

ETH_DMA_IT_T

Transmit interrupt

ETH DMA overflow**ETH_DMA_OVERFLOW_RXFIFOCOUNTER**

Overflow bit for FIFO overflow counter

ETH_DMA_OVERFLOW_MISSEDFRAMECOUNTER

Overflow bit for missed frame counter

ETH DMA receive process state**ETH_DMA_RECEIVEPROCESS_STOPPED**

Stopped - Reset or Stop Rx Command issued

ETH_DMA_RECEIVEPROCESS_FETCHING

Running - fetching the Rx descriptor

ETH_DMA_RECEIVEPROCESS_WAITING

Running - waiting for packet

ETH_DMA_RECEIVEPROCESS_SUSPENDED

Suspended - Rx Descriptor unavailable

ETH_DMA_RECEIVEPROCESS_CLOSING

Running - closing descriptor

ETH_DMA_RECEIVEPROCESS_QUEUING

Running - queuing the receive frame into host memory

ETH DMA RX Descriptor

ETH_DMARXDESC_OWN

OWN bit: descriptor is owned by DMA engine

ETH_DMARXDESC_AFM

DA Filter Fail for the rx frame

ETH_DMARXDESC_FL

Receive descriptor frame length

ETH_DMARXDESC_ES

Error summary: OR of the following bits: DE || OE || IPC || LC || RWT || RE || CE

ETH_DMARXDESC_DE

Descriptor error: no more descriptors for receive frame

ETH_DMARXDESC_SAF

SA Filter Fail for the received frame

ETH_DMARXDESC_LE

Frame size not matching with length field

ETH_DMARXDESC_OE

Overflow Error: Frame was damaged due to buffer overflow

ETH_DMARXDESC_VLAN

VLAN Tag: received frame is a VLAN frame

ETH_DMARXDESC_FS

First descriptor of the frame

ETH_DMARXDESC_LS

Last descriptor of the frame

ETH_DMARXDESC_IPV4HCE

IPC Checksum Error: Rx Ipv4 header checksum error

ETH_DMARXDESC_LC

Late collision occurred during reception

ETH_DMARXDESC_FT

Frame type - Ethernet, otherwise 802.3

ETH_DMARXDESC_RWT

Receive Watchdog Timeout: watchdog timer expired during reception

ETH_DMARXDESC_RE

Receive error: error reported by MII interface

ETH_DMARXDESC_DBE

Dribble bit error: frame contains non int multiple of 8 bits

ETH_DMARXDESC_CE

CRC error

ETH_DMARXDESC_MAMPCE

Rx MAC Address/Payload Checksum Error: Rx MAC address matched/ Rx Payload Checksum Error

ETH_DMARXDESC_DIC

Disable Interrupt on Completion

ETH_DMARXDESC_RBS2

Receive Buffer2 Size

ETH_DMARXDESC_RER

Receive End of Ring

ETH_DMARXDESC_RCH

Second Address Chained

ETH_DMARXDESC_RBS1

Receive Buffer1 Size

ETH_DMARXDESC_B1AP

Buffer1 Address Pointer

ETH_DMARXDESC_B2AP

Buffer2 Address Pointer

ETH_DMAPTPRXDESC_PTPV**ETH_DMAPTPRXDESC_PTPFT****ETH_DMAPTPRXDESC_PTPMT****ETH_DMAPTPRXDESC_PTPMT_SYNC****ETH_DMAPTPRXDESC_PTPMT_FOLLOWUP****ETH_DMAPTPRXDESC_PTPMT_DELAYREQ****ETH_DMAPTPRXDESC_PTPMT_DELAYRESP****ETH_DMAPTPRXDESC_PTPMT_PDELAYREQ_ANNOUNCE****ETH_DMAPTPRXDESC_PTPMT_PDELAYRESP_MANAG****ETH_DMAPTPRXDESC_PTPMT_PDELAYRESPFOLLOWUP_SIGNAL****ETH_DMAPTPRXDESC_IPV6PR****ETH_DMAPTPRXDESC_IPV4PR****ETH_DMAPTPRXDESC_IPCB****ETH_DMAPTPRXDESC_IPPE****ETH_DMAPTPRXDESC_IPHE****ETH_DMAPTPRXDESC_IPPT****ETH_DMAPTPRXDESC_IPPT_UDP****ETH_DMAPTPRXDESC_IPPT_TCP**

ETH_DMAPTPRXDESC_IPPT_ICMP

ETH_DMAPTPRXDESC_RTSL

ETH_DMAPTPRXDESC_RTSH

ETH DMA Rx descriptor buffers

ETH_DMARXDESC_BUFFER1

DMA Rx Desc Buffer1

ETH_DMARXDESC_BUFFER2

DMA Rx Desc Buffer2

ETH DMA transmit process state

ETH_DMA_TRANSMITPROCESS_STOPPED

Stopped - Reset or Stop Tx Command issued

ETH_DMA_TRANSMITPROCESS_FETCHING

Running - fetching the Tx descriptor

ETH_DMA_TRANSMITPROCESS_WAITING

Running - waiting for status

ETH_DMA_TRANSMITPROCESS_READING

Running - reading the data from host memory

ETH_DMA_TRANSMITPROCESS_SUSPENDED

Suspended - Tx Descriptor unavailable

ETH_DMA_TRANSMITPROCESS_CLOSING

Running - closing Rx descriptor

ETH DMA TX Descriptor

ETH_DMATXDESC_OWN

OWN bit: descriptor is owned by DMA engine

ETH_DMATXDESC_IC

Interrupt on Completion

ETH_DMATXDESC_LS

Last Segment

ETH_DMATXDESC_FS

First Segment

ETH_DMATXDESC_DC

Disable CRC

ETH_DMATXDESC_DP

Disable Padding

ETH_DMATXDESC_TTSE

Transmit Time Stamp Enable

ETH_DMATXDESC_CIC

Checksum Insertion Control: 4 cases

ETH_DMATXDESC_CIC_BYPASS

Do Nothing: Checksum Engine is bypassed

ETH_DMATXDESC_CIC_IPV4HEADER

IPv4 header Checksum Insertion

ETH_DMATXDESC_CIC_TCPUDPICMP_SEGMENT

TCP/UDP/ICMP Checksum Insertion calculated over segment only

ETH_DMATXDESC_CIC_TCPUDPICMP_FULL

TCP/UDP/ICMP Checksum Insertion fully calculated

ETH_DMATXDESC_TER

Transmit End of Ring

ETH_DMATXDESC_TCH

Second Address Chained

ETH_DMATXDESC_TTSS

Tx Time Stamp Status

ETH_DMATXDESC_IHE

IP Header Error

ETH_DMATXDESC_ES

Error summary: OR of the following bits: UE || ED || EC || LCO || NC || LCA || FF || JT

ETH_DMATXDESC_JT

Jabber Timeout

ETH_DMATXDESC_FF

Frame Flushed: DMA/MTL flushed the frame due to SW flush

ETH_DMATXDESC_PCE

Payload Checksum Error

ETH_DMATXDESC_LCA

Loss of Carrier: carrier lost during transmission

ETH_DMATXDESC_NC

No Carrier: no carrier signal from the transceiver

ETH_DMATXDESC_LCO

Late Collision: transmission aborted due to collision

ETH_DMATXDESC_EC

Excessive Collision: transmission aborted after 16 collisions

ETH_DMATXDESC_VF

VLAN Frame

ETH_DMATXDESC_CC

Collision Count

ETH_DMATXDESC_ED

Excessive Deferral

ETH_DMATXDESC_UF

Underflow Error: late data arrival from the memory

ETH_DMATXDESC_DB

Deferred Bit

ETH_DMATXDESC_TBS2

Transmit Buffer2 Size

ETH_DMATXDESC_TBS1

Transmit Buffer1 Size

ETH_DMATXDESC_B1AP

Buffer1 Address Pointer

ETH_DMATXDESC_B2AP

Buffer2 Address Pointer

ETH_DMAPTPTXDESC_TTSL

ETH_DMAPTPTXDESC_TTSH

ETH DMA Tx descriptor Checksum Insertion Control

ETH_DMATXDESC_CHECKSUMBYPASS

Checksum engine bypass

ETH_DMATXDESC_CHECKSUMIPV4HEADER

IPv4 header checksum insertion

ETH_DMATXDESC_CHECKSUMTCPUDPICMPSEGMENT

TCP/UDP/ICMP checksum insertion. Pseudo header checksum is assumed to be present

ETH_DMATXDESC_CHECKSUMTCPUDPICMPFULL

TCP/UDP/ICMP checksum fully in hardware including pseudo header

ETH DMA Tx descriptor segment

ETH_DMATXDESC_LASTSEGMENTS

Last Segment

ETH_DMATXDESC_FIRSTSEGMENT

First Segment

ETH Drop TCP IP Checksum Error Frame

ETH_DROPTCPIPCHECKSUMERRORFRAME_ENABLE

ETH_DROPTCPIPCHECKSUMERRORFRAME_DISABLE

ETH Duplex Mode

ETH_MODE_FULLDUPLEX

ETH_MODE_HALFDUPLEX

ETH Exported Macros

__HAL_ETH_RESET_HANDLE_STATE

Description:

- Reset ETH handle state.

Parameters:

- `__HANDLE__`: specifies the ETH handle.

Return value:

- None

__HAL_ETH_DMATXDESC_GET_FLAG

Description:

- Checks whether the specified ETHERNET DMA Tx Desc flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle
- `__FLAG__`: specifies the flag of TDES0 to check.

Return value:

- the: ETH_DMATxDescFlag (SET or RESET).

__HAL_ETH_DMARXDESC_GET_FLAG

Description:

- Checks whether the specified ETHERNET DMA Rx Desc flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle
- `__FLAG__`: specifies the flag of RDES0 to check.

Return value:

- the: ETH_DMATxDescFlag (SET or RESET).

__HAL_ETH_DMARXDESC_ENABLE_IT

Description:

- Enables the specified DMA Rx Desc receive interrupt.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_DMARXDESC_DISABLE_IT

Description:

- Disables the specified DMA Rx Desc receive interrupt.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

`__HAL_ETH_DMARXDESC_SET_OWN_BIT`

Description:

- Set the specified DMA Rx Desc Own bit.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

`__HAL_ETH_DMATXDESC_GET_COLLISION_COUNT`

Description:

- Returns the specified ETHERNET DMA Tx Desc collision count.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- The: Transmit descriptor collision counter value.

`__HAL_ETH_DMATXDESC_SET_OWN_BIT`

Description:

- Set the specified DMA Tx Desc Own bit.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

`__HAL_ETH_DMATXDESC_ENABLE_IT`

Description:

- Enables the specified DMA Tx Desc Transmit interrupt.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

`__HAL_ETH_DMATXDESC_DISABLE_IT`

Description:

- Disables the specified DMA Tx Desc Transmit interrupt.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_DMATXDESC_CHECKSUM_INSERTION

Description:

- Selects the specified ETHERNET DMA Tx Desc Checksum Insertion.

Parameters:

- `__HANDLE__`: ETH Handle
- `__CHECKSUM__`: specifies is the DMA Tx desc checksum insertion. This parameter can be one of the following values:
 - `ETH_DMATXDESC_CHECKSUMBYPASS`: Checksum bypass
 - `ETH_DMATXDESC_CHECKSUMIPV4HEADER`: IPv4 header checksum
 - `ETH_DMATXDESC_CHECKSUMTCPUDPICMPSEGMENT`: TCP/UDP/ICMP checksum. Pseudo header checksum is assumed to be present
 - `ETH_DMATXDESC_CHECKSUMTCPUDPICMPFULL`: TCP/UDP/ICMP checksum fully in hardware including pseudo header

Return value:

- None

__HAL_ETH_DMATXDESC_CRC_ENABLE

Description:

- Enables the DMA Tx Desc CRC.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_DMATXDESC_CRC_DISABLE

Description:

- Disables the DMA Tx Desc CRC.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_DMATXDESC_SHORT_FRAME_PADDING_ENABLE

Description:

- Enables the DMA Tx Desc padding for frame shorter than 64 bytes.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_DMATXDESC_SHORT_FRAME_PADDING_DISABLE

Description:

- Disables the DMA Tx Desc padding for frame shorter than 64 bytes.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_MAC_ENABLE_IT

Description:

- Enables the specified ETHERNET MAC interrupts.

Parameters:

- __HANDLE__: ETH Handle
- __INTERRUPT__: specifies the ETHERNET MAC interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - ETH_MAC_IT_TST : Time stamp trigger interrupt
 - ETH_MAC_IT_PMT : PMT interrupt

Return value:

- None

__HAL_ETH_MAC_DISABLE_IT

Description:

- Disables the specified ETHERNET MAC interrupts.

Parameters:

- __HANDLE__: ETH Handle
- __INTERRUPT__: specifies the ETHERNET MAC interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - ETH_MAC_IT_TST : Time stamp trigger interrupt
 - ETH_MAC_IT_PMT : PMT interrupt

Return value:

- None

__HAL_ETH_INITIATE_PAUSE_CONTROL_FRAME

Description:

- Initiate a Pause Control Frame (Full-duplex only).

Parameters:

- __HANDLE__: ETH Handle

Return value:

- None

__HAL_ETH_GET_FLOW_CONTROL_BUSY_STATUS

Description:

- Checks whether the ETHERNET flow control busy bit is set or not.

Parameters:

- __HANDLE__: ETH Handle

Return value:

- The: new state of flow control busy status bit (SET or RESET).

__HAL_ETH_BACK_PRESSURE_ACTIVATION_ENABLE

Description:

- Enables the MAC Back Pressure operation activation (Half-duplex only).

Parameters:

- __HANDLE__: ETH Handle

Return value:

- None

__HAL_ETH_BACK_PRESSURE_ACTIVATION_DISABLE

Description:

- Disables the MAC BackPressure operation activation (Half-duplex only).

Parameters:

- __HANDLE__: ETH Handle

Return value:

- None

__HAL_ETH_MAC_GET_FLAG

Description:

- Checks whether the specified ETHERNET MAC flag is set or not.

Parameters:

- __HANDLE__: ETH Handle
- __FLAG__: specifies the flag to check. This parameter can be one of the following values:
 - ETH_MAC_FLAG_TST : Time stamp trigger flag
 - ETH_MAC_FLAG_MMCT : MMC transmit flag
 - ETH_MAC_FLAG_MMCR : MMC receive flag
 - ETH_MAC_FLAG_MMC : MMC flag
 - ETH_MAC_FLAG_PMT : PMT flag

Return value:

- The: state of ETHERNET MAC flag.

__HAL_ETH_DMA_ENABLE_IT

Description:

- Enables the specified ETHERNET DMA interrupts.

Parameters:

- __HANDLE__: ETH Handle
- __INTERRUPT__: specifies the ETHERNET DMA interrupt sources to be enabled

Return value:

- None

__HAL_ETH_DMA_DISABLE_IT

Description:

- Disables the specified ETHERNET DMA interrupts.

Parameters:

- __HANDLE__: ETH Handle
- __INTERRUPT__: specifies the ETHERNET DMA interrupt sources to be disabled.

Return value:

- None

__HAL_ETH_DMA_CLEAR_IT

Description:

- Clears the ETHERNET DMA IT pending bit.

Parameters:

- __HANDLE__: ETH Handle
- __INTERRUPT__: specifies the interrupt pending bit to clear.

Return value:

- None

__HAL_ETH_DMA_GET_FLAG

Description:

- Checks whether the specified ETHERNET DMA flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle
- `__FLAG__`: specifies the flag to check.

Return value:

- The: new state of ETH_DMA_FLAG (SET or RESET).

__HAL_ETH_DMA_CLEAR_FLAG

Description:

- Checks whether the specified ETHERNET DMA flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle
- `__FLAG__`: specifies the flag to clear.

Return value:

- The: new state of ETH_DMA_FLAG (SET or RESET).

__HAL_ETH_GET_DMA_OVERFLOW_STATUS

Description:

- Checks whether the specified ETHERNET DMA overflow flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle
- `__OVERFLOW__`: specifies the DMA overflow flag to check. This parameter can be one of the following values:
 - `ETH_DMA_OVERFLOW_RXFIFOCOUNTER`: Overflow for FIFO Overflows Counter
 - `ETH_DMA_OVERFLOW_MISSEDFRAMECOUNTER`: Overflow for Buffer Unavailable Missed Frame Counter

Return value:

- The: state of ETHERNET DMA overflow Flag (SET or RESET).

__HAL_ETH_SET_RECEIVE_WATCHDOG_TIMER

Description:

- Set the DMA Receive status watchdog timer register value.

Parameters:

- `__HANDLE__`: ETH Handle
- `__VALUE__`: DMA Receive status watchdog timer register value

Return value:

- None

__HAL_ETH_GLOBAL_UNICAST_WAKEUP_ENABLE

Description:

- Enables any unicast packet filtered by the MAC address recognition to be a wake-up frame.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_GLOBAL_UNICAST_WAKEUP_DISABLE`

Description:

- Disables any unicast packet filtered by the MAC address recognition to be a wake-up frame.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_WAKEUP_FRAME_DETECTION_ENABLE`

Description:

- Enables the MAC Wake-Up Frame Detection.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_WAKEUP_FRAME_DETECTION_DISABLE`

Description:

- Disables the MAC Wake-Up Frame Detection.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_MAGIC_PACKET_DETECTION_ENABLE`

Description:

- Enables the MAC Magic Packet Detection.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_MAGIC_PACKET_DETECTION_DISABLE`

Description:

- Disables the MAC Magic Packet Detection.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_POWER_DOWN_ENABLE`

Description:

- Enables the MAC Power Down.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_POWER_DOWN_DISABLE

Description:

- Disables the MAC Power Down.

Parameters:

- `__HANDLE__`: ETH Handle

Return value:

- None

__HAL_ETH_GET_PMT_FLAG_STATUS

Description:

- Checks whether the specified ETHERNET PMT flag is set or not.

Parameters:

- `__HANDLE__`: ETH Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `ETH_PMT_FLAG_WUFFRPR`: Wake-Up Frame Filter Register Pointer Reset
 - `ETH_PMT_FLAG_WUFR`: Wake-Up Frame Received
 - `ETH_PMT_FLAG_MPR`: Magic Packet Received

Return value:

- The: new state of ETHERNET PMT Flag (SET or RESET).

__HAL_ETH_MMC_COUNTER_FULL_PRESET

Description:

- Preset and Initialize the MMC counters to almost-full value: `0xFFFF_FFF0` (full - 16)

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

__HAL_ETH_MMC_COUNTER_HALF_PRESET

Description:

- Preset and Initialize the MMC counters to almost-half value: `0x7FFF_FFF0` (half - 16)

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

__HAL_ETH_MMC_COUNTER_FREEZE_ENABLE

Description:

- Enables the MMC Counter Freeze.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_MMC_COUNTER_FREEZE_DISABLE`

Description:

- Disables the MMC Counter Freeze.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_ETH_MMC_RESET_ONREAD_ENABLE`

Description:

- Enables the MMC Reset On Read.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_ETH_MMC_RESET_ONREAD_DISABLE`

Description:

- Disables the MMC Reset On Read.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_ETH_MMC_COUNTER_ROLLOVER_ENABLE`

Description:

- Enables the MMC Counter Stop Rollover.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_ETH_MMC_COUNTER_ROLLOVER_DISABLE`

Description:

- Disables the MMC Counter Stop Rollover.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_MMC_COUNTERS_RESET`

Description:

- Resets the MMC Counters.

Parameters:

- `__HANDLE__`: ETH Handle.

Return value:

- None

`__HAL_ETH_MMC_RX_IT_ENABLE`

Description:

- Enables the specified ETHERNET MMC Rx interrupts.

Parameters:

- `__HANDLE__`: ETH Handle.
- `__INTERRUPT__`: specifies the ETHERNET MMC interrupt sources to be enabled or disabled. This parameter can be one of the following values:
 - `ETH_MMC_IT_RGUF`: When Rx good unicast frames counter reaches half the maximum value
 - `ETH_MMC_IT_RFAE`: When Rx alignment error counter reaches half the maximum value
 - `ETH_MMC_IT_RFCE`: When Rx crc error counter reaches half the maximum value

Return value:

- None

`__HAL_ETH_MMC_RX_IT_DISABLE`

Description:

- Disables the specified ETHERNET MMC Rx interrupts.

Parameters:

- `__HANDLE__`: ETH Handle.
- `__INTERRUPT__`: specifies the ETHERNET MMC interrupt sources to be enabled or disabled. This parameter can be one of the following values:
 - `ETH_MMC_IT_RGUF`: When Rx good unicast frames counter reaches half the maximum value
 - `ETH_MMC_IT_RFAE`: When Rx alignment error counter reaches half the maximum value
 - `ETH_MMC_IT_RFCE`: When Rx crc error counter reaches half the maximum value

Return value:

- None

`__HAL_ETH_MMC_TX_IT_ENABLE`

Description:

- Enables the specified ETHERNET MMC Tx interrupts.

Parameters:

- `__HANDLE__`: ETH Handle.
- `__INTERRUPT__`: specifies the ETHERNET MMC interrupt sources to be enabled or disabled. This parameter can be one of the following values:
 - `ETH_MMC_IT_TGF`: When Tx good frame counter reaches half the maximum value
 - `ETH_MMC_IT_TGFMSC`: When Tx good multi col counter reaches half the maximum value
 - `ETH_MMC_IT_TGFSC`: When Tx good single col counter reaches half the maximum value

Return value:

- None

__HAL_ETH_MMC_TX_IT_DISABLE

Description:

- Disables the specified ETHERNET MMC Tx interrupts.

Parameters:

- __HANDLE__: ETH Handle.
- __INTERRUPT__: specifies the ETHERNET MMC interrupt sources to be enabled or disabled. This parameter can be one of the following values:
 - ETH_MMC_IT_TGF : When Tx good frame counter reaches half the maximum value
 - ETH_MMC_IT_TGFMSC: When Tx good multi col counter reaches half the maximum value
 - ETH_MMC_IT_TGFSC : When Tx good single col counter reaches half the maximum value

Return value:

- None

__HAL_ETH_WAKEUP_EXTI_ENABLE_IT

Description:

- Enables the ETH External interrupt line.

Return value:

- None

__HAL_ETH_WAKEUP_EXTI_DISABLE_IT

Description:

- Disables the ETH External interrupt line.

Return value:

- None

__HAL_ETH_WAKEUP_EXTI_ENABLE_EVENT

Description:

- Enable event on ETH External event line.

Return value:

- None.

__HAL_ETH_WAKEUP_EXTI_DISABLE_EVENT

Description:

- Disable event on ETH External event line.

Return value:

- None.

__HAL_ETH_WAKEUP_EXTI_GET_FLAG

Description:

- Get flag of the ETH External interrupt line.

Return value:

- None

__HAL_ETH_WAKEUP_EXTI_CLEAR_FLAG

Description:

- Clear flag of the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_ENABLE_RISING_EDGE_TRIGGER`

Description:

- Enables rising edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_DISABLE_RISING_EDGE_TRIGGER`

Description:

- Disables the rising edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_ENABLE_FALLING_EDGE_TRIGGER`

Description:

- Enables falling edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_DISABLE_FALLING_EDGE_TRIGGER`

Description:

- Disables falling edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_ENABLE_FALLINGRISING_TRIGGER`

Description:

- Enables rising/falling edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_DISABLE_FALLINGRISING_TRIGGER`

Description:

- Disables rising/falling edge trigger to the ETH External interrupt line.

Return value:

- None

`__HAL_ETH_WAKEUP_EXTI_GENERATE_SWIT`

Description:

- Generate a Software interrupt on selected EXTI line.

Return value:

- None.

ETH EXTI LINE WAKEUP

`ETH_EXTI_LINE_WAKEUP`

External interrupt line 19 Connected to the ETH EXTI Line

ETH Fixed Burst

`ETH_FIXEDBURST_ENABLE`

`ETH_FIXEDBURST_DISABLE`

ETH Flush Received Frame

ETH_FLUSHRECEIVEDFRAME_ENABLE

ETH_FLUSHRECEIVEDFRAME_DISABLE

ETH Forward Error Frames

ETH_FORWARDERRORFRAMES_ENABLE

ETH_FORWARDERRORFRAMES_DISABLE

ETH Forward Undersized Good Frames

ETH_FORWARDUNDERSIZEDGOODFRAMES_ENABLE

ETH_FORWARDUNDERSIZEDGOODFRAMES_DISABLE

ETH Inter Frame Gap

ETH_INTERFRAMEGAP_96BIT

minimum IFG between frames during transmission is 96Bit

ETH_INTERFRAMEGAP_88BIT

minimum IFG between frames during transmission is 88Bit

ETH_INTERFRAMEGAP_80BIT

minimum IFG between frames during transmission is 80Bit

ETH_INTERFRAMEGAP_72BIT

minimum IFG between frames during transmission is 72Bit

ETH_INTERFRAMEGAP_64BIT

minimum IFG between frames during transmission is 64Bit

ETH_INTERFRAMEGAP_56BIT

minimum IFG between frames during transmission is 56Bit

ETH_INTERFRAMEGAP_48BIT

minimum IFG between frames during transmission is 48Bit

ETH_INTERFRAMEGAP_40BIT

minimum IFG between frames during transmission is 40Bit

ETH Jabber

ETH_JABBER_ENABLE

ETH_JABBER_DISABLE

ETH Loop Back Mode

ETH_LOOPBACKMODE_ENABLE

ETH_LOOPBACKMODE_DISABLE

ETH MAC addresses

ETH_MAC_ADDRESS0

ETH_MAC_ADDRESS1

ETH_MAC_ADDRESS2

ETH_MAC_ADDRESS3

ETH MAC addresses filter Mask bytes

ETH_MAC_ADDRESSMASK_BYTE6

Mask MAC Address high reg bits [15:8]

ETH_MAC_ADDRESSMASK_BYTE5

Mask MAC Address high reg bits [7:0]

ETH_MAC_ADDRESSMASK_BYTE4

Mask MAC Address low reg bits [31:24]

ETH_MAC_ADDRESSMASK_BYTE3

Mask MAC Address low reg bits [23:16]

ETH_MAC_ADDRESSMASK_BYTE2

Mask MAC Address low reg bits [15:8]

ETH_MAC_ADDRESSMASK_BYTE1

Mask MAC Address low reg bits [7:0]

ETH MAC addresses filter SA DA

ETH_MAC_ADDRESSFILTER_SA

ETH_MAC_ADDRESSFILTER_DA

ETH MAC Flags

ETH_MAC_FLAG_TST

Time stamp trigger flag (on MAC)

ETH_MAC_FLAG_MMCT

MMC transmit flag

ETH_MAC_FLAG_MMCR

MMC receive flag

ETH_MAC_FLAG_MMC

MMC flag (on MAC)

ETH_MAC_FLAG_PMT

PMT flag (on MAC)

ETH MAC Interrupts

ETH_MAC_IT_TST

Time stamp trigger interrupt (on MAC)

ETH_MAC_IT_MMCT

MMC transmit interrupt

ETH_MAC_IT_MMCR

MMC receive interrupt

ETH_MAC_IT_MMC

MMC interrupt (on MAC)

ETH_MAC_IT_PMT

PMT interrupt (on MAC)

ETH Media Interface

ETH_MEDIA_INTERFACE_MII

ETH_MEDIA_INTERFACE_RMII

ETH MMC Rx Interrupts

ETH_MMC_IT_RGUF

When Rx good unicast frames counter reaches half the maximum value

ETH_MMC_IT_RFAE

When Rx alignment error counter reaches half the maximum value

ETH_MMC_IT_RFCE

When Rx crc error counter reaches half the maximum value

ETH MMC Tx Interrupts

ETH_MMC_IT_TGF

When Tx good frame counter reaches half the maximum value

ETH_MMC_IT_TGFMSC

When Tx good multi col counter reaches half the maximum value

ETH_MMC_IT_TGFSC

When Tx good single col counter reaches half the maximum value

ETH Multicast Frames Filter

ETH_MULTICASTFRAMESFILTER_PERFECTHASHTABLE

ETH_MULTICASTFRAMESFILTER_HASHTABLE

ETH_MULTICASTFRAMESFILTER_PERFECT

ETH_MULTICASTFRAMESFILTER_NONE

ETH Pass Control Frames

ETH_PASSCONTROLFRAMES_BLOCKALL

MAC filters all control frames from reaching the application

ETH_PASSCONTROLFRAMES_FORWARDALL

MAC forwards all control frames to application even if they fail the Address Filter

ETH_PASSCONTROLFRAMES_FORWARDPASSEDADDRFILTER

MAC forwards control frames that pass the Address Filter.

ETH Pause Low Threshold

ETH_PAUSELOWTHRESHOLD_MINUS4

Pause time minus 4 slot times

ETH_PAUSELOWTHRESHOLD_MINUS28

Pause time minus 28 slot times

ETH_PAUSELOWTHRESHOLD_MINUS144

Pause time minus 144 slot times

ETH_PAUSELOWTHRESHOLD_MINUS256

Pause time minus 256 slot times

ETH PMT Flags

ETH_PMT_FLAG_WUFFRPR

Wake-Up Frame Filter Register Pointer Reset

ETH_PMT_FLAG_WUFR

Wake-Up Frame Received

ETH_PMT_FLAG_MPR

Magic Packet Received

ETH Promiscuous Mode

ETH_PROMISCUOUS_MODE_ENABLE

ETH_PROMISCUOUS_MODE_DISABLE

ETH Receive All

ETH_RECEIVEALL_ENABLE

ETH_RECEIVEALL_DISABLE

ETH Receive Flow Control

ETH_RECEIVEFLOWCONTROL_ENABLE

ETH_RECEIVEFLOWCONTROL_DISABLE

ETH Receive Own

ETH_RECEIVEOWN_ENABLE

ETH_RECEIVEOWN_DISABLE

ETH Receive Store Forward

ETH_RECEIVESTOREFORWARD_ENABLE

ETH_RECEIVESTOREFORWARD_DISABLE

ETH Receive Threshold Control

ETH_RECEIVEDTHRESHOLDCONTROL_64BYTES

threshold level of the MTL Receive FIFO is 64 Bytes

ETH_RECEIVEDTHRESHOLDCONTROL_32BYTES

threshold level of the MTL Receive FIFO is 32 Bytes

ETH_RECEIVEDTHRESHOLDCONTROL_96BYTES

threshold level of the MTL Receive FIFO is 96 Bytes

ETH_RECEIVEDTHRESHOLDCONTROL_128BYTES

threshold level of the MTL Receive FIFO is 128 Bytes

ETH Retry Transmission

ETH_RETRYTRANSMISSION_ENABLE

ETH_RETRYTRANSMISSION_DISABLE

ETH Rx DMA Burst Length

ETH_RXDMABURSTLENGTH_1BEAT

maximum number of beats to be transferred in one RxDMA transaction is 1

ETH_RXDMABURSTLENGTH_2BEAT

maximum number of beats to be transferred in one RxDMA transaction is 2

ETH_RXDMABURSTLENGTH_4BEAT

maximum number of beats to be transferred in one RxDMA transaction is 4

ETH_RXDMABURSTLENGTH_8BEAT

maximum number of beats to be transferred in one RxDMA transaction is 8

ETH_RXDMABURSTLENGTH_16BEAT

maximum number of beats to be transferred in one RxDMA transaction is 16

ETH_RXDMABURSTLENGTH_32BEAT

maximum number of beats to be transferred in one RxDMA transaction is 32

ETH_RXDMABURSTLENGTH_4XPBL_4BEAT

maximum number of beats to be transferred in one RxDMA transaction is 4

ETH_RXDMABURSTLENGTH_4XPBL_8BEAT

maximum number of beats to be transferred in one RxDMA transaction is 8

ETH_RXDMABURSTLENGTH_4XPBL_16BEAT

maximum number of beats to be transferred in one RxDMA transaction is 16

ETH_RXDMABURSTLENGTH_4XPBL_32BEAT

maximum number of beats to be transferred in one RxDMA transaction is 32

ETH_RXDMABURSTLENGTH_4XPBL_64BEAT

maximum number of beats to be transferred in one RxDMA transaction is 64

ETH_RXDMABURSTLENGTH_4XPBL_128BEAT

maximum number of beats to be transferred in one RxDMA transaction is 128

ETH Rx Mode

ETH_RXPOLLING_MODE

ETH_RXINTERRUPT_MODE

ETH Second Frame Operate

ETH_SECONDFRAMEOPERARTE_ENABLE

ETH_SECONDFRAMEOPERARTE_DISABLE

ETH Source Addr Filter

ETH_SOURCEADDRFILTER_NORMAL_ENABLE

ETH_SOURCEADDRFILTER_INVERSE_ENABLE

ETH_SOURCEADDRFILTER_DISABLE***ETH Speed*****ETH_SPEED_10M****ETH_SPEED_100M*****ETH Transmit Flow Control*****ETH_TRANSMITFLOWCONTROL_ENABLE****ETH_TRANSMITFLOWCONTROL_DISABLE*****ETH Transmit Store Forward*****ETH_TRANSMITSTOREFORWARD_ENABLE****ETH_TRANSMITSTOREFORWARD_DISABLE*****ETH Transmit Threshold Control*****ETH_TRANSMITTHRESHOLDCONTROL_64BYTES**

threshold level of the MTL Transmit FIFO is 64 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_128BYTES

threshold level of the MTL Transmit FIFO is 128 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_192BYTES

threshold level of the MTL Transmit FIFO is 192 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_256BYTES

threshold level of the MTL Transmit FIFO is 256 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_40BYTES

threshold level of the MTL Transmit FIFO is 40 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_32BYTES

threshold level of the MTL Transmit FIFO is 32 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_24BYTES

threshold level of the MTL Transmit FIFO is 24 Bytes

ETH_TRANSMITTHRESHOLDCONTROL_16BYTES

threshold level of the MTL Transmit FIFO is 16 Bytes

ETH Tx DMA Burst Length**ETH_TXDMABURSTLENGTH_1BEAT**

maximum number of beats to be transferred in one TxDMA (or both) transaction is 1

ETH_TXDMABURSTLENGTH_2BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 2

ETH_TXDMABURSTLENGTH_4BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 4

ETH_TXDMABURSTLENGTH_8BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 8

ETH_TXDMABURSTLENGTH_16BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 16

ETH_TXDMABURSTLENGTH_32BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 32

ETH_TXDMABURSTLENGTH_4XPBL_4BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 4

ETH_TXDMABURSTLENGTH_4XPBL_8BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 8

ETH_TXDMABURSTLENGTH_4XPBL_16BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 16

ETH_TXDMABURSTLENGTH_4XPBL_32BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 32

ETH_TXDMABURSTLENGTH_4XPBL_64BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 64

ETH_TXDMABURSTLENGTH_4XPBL_128BEAT

maximum number of beats to be transferred in one TxDMA (or both) transaction is 128

ETH Unicast Frames Filter

ETH_UNICASTFRAMESFILTER_PERFECTHASHTABLE

ETH_UNICASTFRAMESFILTER_HASHTABLE

ETH_UNICASTFRAMESFILTER_PERFECT

ETH Unicast Pause Frame Detect

ETH_UNICASTPAUSEFRAMEDETECT_ENABLE

ETH_UNICASTPAUSEFRAMEDETECT_DISABLE

ETH VLAN Tag Comparison

ETH_VLANTAGCOMPARISON_12BIT

ETH_VLANTAGCOMPARISON_16BIT

ETH Watchdog

ETH_WATCHDOG_ENABLE

ETH_WATCHDOG_DISABLE

ETH Zero Quanta Pause

ETH_ZEROQUANTAPAUSE_ENABLE

ETH_ZEROQUANTAPAUSE_DISABLE

18 HAL EXTI Generic Driver

18.1 EXTI Firmware driver registers structures

18.1.1 EXTI_HandleTypeDef

EXTI_HandleTypeDef is defined in the stm32f2xx_hal_exti.h

Data Fields

- **uint32_t Line**
- **void(* PendingCallback)**

Field Documentation

- **uint32_t EXTI_HandleTypeDef::Line**
Exti line number
- **void(* EXTI_HandleTypeDef::PendingCallback)(void)**
Exti pending callback

18.1.2 EXTI_ConfigTypeDef

EXTI_ConfigTypeDef is defined in the stm32f2xx_hal_exti.h

Data Fields

- **uint32_t Line**
- **uint32_t Mode**
- **uint32_t Trigger**
- **uint32_t GPIOSel**

Field Documentation

- **uint32_t EXTI_ConfigTypeDef::Line**
The Exti line to be configured. This parameter can be a value of [EXTI_Line](#)
- **uint32_t EXTI_ConfigTypeDef::Mode**
The Exit Mode to be configured for a core. This parameter can be a combination of [EXTI_Mode](#)
- **uint32_t EXTI_ConfigTypeDef::Trigger**
The Exti Trigger to be configured. This parameter can be a value of [EXTI_Trigger](#)
- **uint32_t EXTI_ConfigTypeDef::GPIOSel**
The Exti GPIO multiplexer selection to be configured. This parameter is only possible for line 0 to 15. It can be a value of [EXTI_GPIOSel](#)

18.2 EXTI Firmware driver API description

The following section lists the various functions of the EXTI library.

18.2.1 EXTI Peripheral features

- Each Exti line can be configured within this driver.
- Exti line can be configured in 3 different modes
 - Interrupt
 - Event
 - Both of them
- Configurable Exti lines can be configured with 3 different triggers
 - Rising
 - Falling
 - Both of them

- When set in interrupt mode, configurable Exti lines have two different interrupts pending registers which allow to distinguish which transition occurs:
 - Rising edge pending interrupt
 - Falling
- Exti lines 0 to 15 are linked to gpio pin number 0 to 15. Gpio port can be selected through multiplexer.

18.2.2 How to use this driver

1. Configure the EXTI line using HAL_EXTI_SetConfigLine().
 - Choose the interrupt line number by setting "Line" member from EXTI_ConfigTypeDef structure.
 - Configure the interrupt and/or event mode using "Mode" member from EXTI_ConfigTypeDef structure.
 - For configurable lines, configure rising and/or falling trigger "Trigger" member from EXTI_ConfigTypeDef structure.
 - For Exti lines linked to gpio, choose gpio port using "GPIOSel" member from GPIO_InitTypeDef structure.
2. Get current Exti configuration of a dedicated line using HAL_EXTI_GetConfigLine().
 - Provide exiting handle as parameter.
 - Provide pointer on EXTI_ConfigTypeDef structure as second parameter.
3. Clear Exti configuration of a dedicated line using HAL_EXTI_GetConfigLine().
 - Provide exiting handle as parameter.
4. Register callback to treat Exti interrupts using HAL_EXTI_RegisterCallback().
 - Provide exiting handle as first parameter.
 - Provide which callback will be registered using one value from EXTI_CallbackIDTypeDef.
 - Provide callback function pointer.
5. Get interrupt pending bit using HAL_EXTI_GetPending().
6. Clear interrupt pending bit using HAL_EXTI_GetPending().
7. Generate software interrupt using HAL_EXTI_GenerateSWI().

18.2.3 Configuration functions

This section contains the following APIs:

- [HAL_EXTI_SetConfigLine](#)
- [HAL_EXTI_GetConfigLine](#)
- [HAL_EXTI_ClearConfigLine](#)
- [HAL_EXTI_RegisterCallback](#)
- [HAL_EXTI_GetHandle](#)

18.2.4 Detailed description of functions

HAL_EXTI_SetConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_SetConfigLine (EXTI_HandleTypeDef * hexti, EXTI_ConfigTypeDef * pExtiConfig)

Function description

Set configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **pExtiConfig**: Pointer on EXTI configuration to be set.

Return values

- **HAL**: Status.

HAL_EXTI_GetConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_GetConfigLine (EXTI_HandleTypeDef * hexti, EXTI_ConfigTypeDef * pExtiConfig)

Function description

Get configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **pExtiConfig**: Pointer on structure to store Exti configuration.

Return values

- **HAL**: Status.

HAL_EXTI_ClearConfigLine

Function name

HAL_StatusTypeDef HAL_EXTI_ClearConfigLine (EXTI_HandleTypeDef * hexti)

Function description

Clear whole configuration of a dedicated Exti line.

Parameters

- **hexti**: Exti handle.

Return values

- **HAL**: Status.

HAL_EXTI_RegisterCallback

Function name

HAL_StatusTypeDef HAL_EXTI_RegisterCallback (EXTI_HandleTypeDef * hexti, EXTI_CallbackIDTypeDef CallbackID, void(*)(void) pPendingCbf)

Function description

Register callback for a dedicated Exti line.

Parameters

- **hexti**: Exti handle.
- **CallbackID**: User callback identifier. This parameter can be one of
– EXTI_CallbackIDTypeDef values.
- **pPendingCbf**: function pointer to be stored as callback.

Return values

- **HAL**: Status.

HAL_EXTI_GetHandle

Function name

HAL_StatusTypeDef HAL_EXTI_GetHandle (EXTI_HandleTypeDef * hexti, uint32_t ExtiLine)

Function description

Store line number as handle private field.

Parameters

- **hexti:** Exti handle.
- **ExtiLine:** Exti line number. This parameter can be from 0 to EXTI_LINE_NB.

Return values

- **HAL:** Status.

HAL_EXTI_IRQHandler

Function name

```
void HAL_EXTI_IRQHandler (EXTI_HandleTypeDef * hexti)
```

Function description

Handle EXTI interrupt request.

Parameters

- **hexti:** Exti handle.

Return values

- **none.:**

HAL_EXTI_GetPending

Function name

```
uint32_t HAL_EXTI_GetPending (EXTI_HandleTypeDef * hexti, uint32_t Edge)
```

Function description

Get interrupt pending bit of a dedicated line.

Parameters

- **hexti:** Exti handle.
- **Edge:** Specify which pending edge as to be checked. This parameter can be one of the following values:
 - EXTI_TRIGGER_RISING_FALLING This parameter is kept for compatibility with other series.

Return values

- **1:** if interrupt is pending else 0.

HAL_EXTI_ClearPending

Function name

```
void HAL_EXTI_ClearPending (EXTI_HandleTypeDef * hexti, uint32_t Edge)
```

Function description

Clear interrupt pending bit of a dedicated line.

Parameters

- **hexti:** Exti handle.
- **Edge:** Specify which pending edge as to be clear. This parameter can be one of the following values:
 - EXTI_TRIGGER_RISING_FALLING This parameter is kept for compatibility with other series.

Return values

- **None.:**

HAL_EXTI_GenerateSWI

Function name

void HAL_EXTI_GenerateSWI (EXTI_HandleTypeDef * hexti)

Function description

Generate a software interrupt for a dedicated line.

Parameters

- **hexti**: Exti handle.

Return values

- **None.**

18.3 EXTI Firmware driver defines

The following section lists the various define and macros of the module.

18.3.1 EXTI

EXTI
EXTI GPIOSeI

EXTI_GPIOA

EXTI_GPIOB

EXTI_GPIOC

EXTI_GPIOD

EXTI_GPIOE

EXTI_GPIOF

EXTI_GPIOG

EXTI_GPIOH

EXTI_GPIOI

EXTI Line

EXTI_LINE_0

External interrupt line 0

EXTI_LINE_1

External interrupt line 1

EXTI_LINE_2

External interrupt line 2

EXTI_LINE_3

External interrupt line 3

EXTI_LINE_4

External interrupt line 4

EXTI_LINE_5

External interrupt line 5

EXTI_LINE_6

External interrupt line 6

EXTI_LINE_7

External interrupt line 7

EXTI_LINE_8

External interrupt line 8

EXTI_LINE_9

External interrupt line 9

EXTI_LINE_10

External interrupt line 10

EXTI_LINE_11

External interrupt line 11

EXTI_LINE_12

External interrupt line 12

EXTI_LINE_13

External interrupt line 13

EXTI_LINE_14

External interrupt line 14

EXTI_LINE_15

External interrupt line 15

EXTI_LINE_16

External interrupt line 16 Connected to the PVD Output

EXTI_LINE_17

External interrupt line 17 Connected to the RTC Alarm event

EXTI_LINE_18

External interrupt line 18 Connected to the USB OTG FS Wakeup from suspend event

EXTI_LINE_19

External interrupt line 19 Connected to the Ethernet Wakeup event

EXTI_LINE_20

External interrupt line 20 Connected to the USB OTG HS (configured in FS) Wakeup event

EXTI_LINE_21

External interrupt line 21 Connected to the RTC Tamper and Time Stamp events

EXTI_LINE_22

External interrupt line 22 Connected to the RTC Wakeup event

EXTI Mode**EXTI_MODE_NONE**

EXTI_MODE_INTERRUPT

EXTI_MODE_EVENT

EXTI Trigger

EXTI_TRIGGER_NONE

EXTI_TRIGGER_RISING

EXTI_TRIGGER_FALLING

EXTI_TRIGGER_RISING_FALLING

19 HAL FLASH Generic Driver

19.1 FLASH Firmware driver registers structures

19.1.1 FLASH_ProcessTypeDef

FLASH_ProcessTypeDef is defined in the stm32f2xx_hal_flash.h

Data Fields

- `__IO FLASH_ProcedureTypeDef ProcedureOnGoing`
- `__IO uint32_t NbSectorsToErase`
- `__IO uint8_t VoltageForErase`
- `__IO uint32_t Sector`
- `__IO uint32_t Bank`
- `__IO uint32_t Address`
- `HAL_LockTypeDef Lock`
- `__IO uint32_t ErrorCode`

Field Documentation

- `__IO FLASH_ProcedureTypeDef FLASH_ProcessTypeDef::ProcedureOnGoing`
- `__IO uint32_t FLASH_ProcessTypeDef::NbSectorsToErase`
- `__IO uint8_t FLASH_ProcessTypeDef::VoltageForErase`
- `__IO uint32_t FLASH_ProcessTypeDef::Sector`
- `__IO uint32_t FLASH_ProcessTypeDef::Bank`
- `__IO uint32_t FLASH_ProcessTypeDef::Address`
- `HAL_LockTypeDef FLASH_ProcessTypeDef::Lock`
- `__IO uint32_t FLASH_ProcessTypeDef::ErrorCode`

19.2 FLASH Firmware driver API description

The following section lists the various functions of the FLASH library.

19.2.1 FLASH peripheral features

The Flash memory interface manages CPU AHB I-Code and D-Code accesses to the Flash memory. It implements the erase and program Flash memory operations and the read and write protection mechanisms. The Flash memory interface accelerates code execution with a system of instruction prefetch and cache lines. The FLASH main features are:

- Flash memory read operations
- Flash memory program/erase operations
- Read / write protections
- Prefetch on I-Code
- 64 cache lines of 128 bits on I-Code
- 8 cache lines of 128 bits on D-Code

19.2.2 How to use this driver

This driver provides functions and macros to configure and program the FLASH memory of all STM32F2xx devices.

1. FLASH Memory IO Programming functions:
 - Lock and Unlock the FLASH interface using HAL_FLASH_Unlock() and HAL_FLASH_Lock() functions
 - Program functions: byte, half word, word and double word
 - There Two modes of programming :
 - Polling mode using HAL_FLASH_Program() function
 - Interrupt mode using HAL_FLASH_Program_IT() function
2. Interrupts and flags management functions :
 - Handle FLASH interrupts by calling HAL_FLASH_IRQHandler()
 - Wait for last FLASH operation according to its status
 - Get error flag status by calling HAL_SetErrorCode()

In addition to these functions, this driver includes a set of macros allowing to handle the following operations:

- Set the latency
- Enable/Disable the prefetch buffer
- Enable/Disable the Instruction cache and the Data cache
- Reset the Instruction cache and the Data cache
- Enable/Disable the FLASH interrupts
- Monitor the FLASH flags status

19.2.3 Programming operation functions

This subsection provides a set of functions allowing to manage the FLASH program operations.

This section contains the following APIs:

- [*HAL_FLASH_Program*](#)
- [*HAL_FLASH_Program_IT*](#)
- [*HAL_FLASH_IRQHandler*](#)
- [*HAL_FLASH_EndOfOperationCallback*](#)
- [*HAL_FLASH_OperationErrorCallback*](#)

19.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the FLASH memory operations.

This section contains the following APIs:

- [*HAL_FLASH_Unlock*](#)
- [*HAL_FLASH_Lock*](#)
- [*HAL_FLASH_OB_Unlock*](#)
- [*HAL_FLASH_OB_Lock*](#)
- [*HAL_FLASH_OB_Launch*](#)

19.2.5 Peripheral Errors functions

This subsection permits to get in run-time Errors of the FLASH peripheral.

This section contains the following APIs:

- [*HAL_FLASH_GetError*](#)
- [*FLASH_WaitForLastOperation*](#)

19.2.6 Detailed description of functions

HAL_FLASH_Program

Function name

HAL_StatusTypeDef HAL_FLASH_Program (uint32_t TypeProgram, uint32_t Address, uint64_t Data)

Function description

Program byte, halfword, word or double word at a specified address.

Parameters

- **TypeProgram:** Indicate the way to program at a specified address. This parameter can be a value of FLASH Type Program
- **Address:** specifies the address to be programmed.
- **Data:** specifies the data to be programmed

Return values

- **HAL_StatusTypeDef:** HAL Status

HAL_FLASH_Program_IT

Function name

HAL_StatusTypeDef HAL_FLASH_Program_IT (uint32_t TypeProgram, uint32_t Address, uint64_t Data)

Function description

Program byte, halfword, word or double word at a specified address with interrupt enabled.

Parameters

- **TypeProgram:** Indicate the way to program at a specified address. This parameter can be a value of FLASH Type Program
- **Address:** specifies the address to be programmed.
- **Data:** specifies the data to be programmed

Return values

- **HAL:** Status

HAL_FLASH_IRQHandler

Function name

void HAL_FLASH_IRQHandler (void)

Function description

This function handles FLASH interrupt request.

Return values

- **None:**

HAL_FLASH_EndOfOperationCallback

Function name

void HAL_FLASH_EndOfOperationCallback (uint32_t ReturnValue)

Function description

FLASH end of operation interrupt callback.

Parameters

- **ReturnValue:** The value saved in this parameter depends on the ongoing procedure Mass Erase: Bank number which has been requested to erase Sectors Erase: Sector which has been erased (if 0xFFFFFFFF, it means that all the selected sectors have been erased) Program: Address which was selected for data program

Return values

- **None:**

HAL_FLASH_OperationErrorCallback

Function name

void HAL_FLASH_OperationErrorCallback (uint32_t ReturnValue)

Function description

FLASH operation error interrupt callback.

Parameters

- **ReturnValue:** The value saved in this parameter depends on the ongoing procedure Mass Erase: Bank number which has been requested to erase Sectors Erase: Sector number which returned an error Program: Address which was selected for data program

Return values

- **None:**

HAL_FLASH_Unlock

Function name

HAL_StatusTypeDef HAL_FLASH_Unlock (void)

Function description

Unlock the FLASH control register access.

Return values

- **HAL:** Status

HAL_FLASH_Lock

Function name

HAL_StatusTypeDef HAL_FLASH_Lock (void)

Function description

Locks the FLASH control register access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Unlock

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Unlock (void)

Function description

Unlock the FLASH Option Control Registers access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Lock

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Lock (void)

Function description

Lock the FLASH Option Control Registers access.

Return values

- **HAL:** Status

HAL_FLASH_OB_Launch

Function name

HAL_StatusTypeDef HAL_FLASH_OB_Launch (void)

Function description

Launch the option byte loading.

Return values

- **HAL:** Status

HAL_FLASH_GetError

Function name

uint32_t HAL_FLASH_GetError (void)

Function description

Get the specific FLASH error flag.

Return values

- **FLASH_ErrorCode:** The returned value can be a combination of:
 - HAL_FLASH_ERROR_NONE: FLASH Programming Sequence error flag
 - HAL_FLASH_ERROR_PGS: FLASH Programming Sequence error flag
 - HAL_FLASH_ERROR_PGP: FLASH Programming Parallelism error flag
 - HAL_FLASH_ERROR_PGA: FLASH Programming Alignment error flag
 - HAL_FLASH_ERROR_WRP: FLASH Write protected error flag
 - HAL_FLASH_ERROR_OPERATION: FLASH operation Error flag

FLASH_WaitForLastOperation

Function name

HAL_StatusTypeDef FLASH_WaitForLastOperation (uint32_t Timeout)

Function description

Wait for a FLASH operation to complete.

Parameters

- **Timeout:** maximum flash operationtimeout

Return values

- **HAL:** Status

19.3 FLASH Firmware driver defines

The following section lists the various define and macros of the module.

19.3.1 FLASH

FLASH

FLASH Error Code

HAL_FLASH_ERROR_NONE

No error

HAL_FLASH_ERROR_PGS

Programming Sequence error

HAL_FLASH_ERROR_PGP

Programming Parallelism error

HAL_FLASH_ERROR_PGA

Programming Alignment error

HAL_FLASH_ERROR_WRP

Write protection error

HAL_FLASH_ERROR_OPERATION

Operation Error

FLASH Exported Macros

__HAL_FLASH_SET_LATENCY

Description:

- Set the FLASH Latency.

Parameters:

- `__LATENCY__`: FLASH Latency The value of this parameter depend on device used within the same series

Return value:

- none

__HAL_FLASH_GET_LATENCY

Description:

- Get the FLASH Latency.

Return value:

- FLASH: Latency The value of this parameter depend on device used within the same series

__HAL_FLASH_PREFETCH_BUFFER_ENABLE

Description:

- Enable the FLASH prefetch buffer.

Return value:

- none

__HAL_FLASH_PREFETCH_BUFFER_DISABLE

Description:

- Disable the FLASH prefetch buffer.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_ENABLE

Description:

- Enable the FLASH instruction cache.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_DISABLE

Description:

- Disable the FLASH instruction cache.

Return value:

- none

__HAL_FLASH_DATA_CACHE_ENABLE

Description:

- Enable the FLASH data cache.

Return value:

- none

__HAL_FLASH_DATA_CACHE_DISABLE

Description:

- Disable the FLASH data cache.

Return value:

- none

__HAL_FLASH_INSTRUCTION_CACHE_RESET

Description:

- Resets the FLASH instruction Cache.

Return value:

- None

Notes:

- This function must be used only when the Instruction Cache is disabled.

__HAL_FLASH_DATA_CACHE_RESET

Description:

- Resets the FLASH data Cache.

Return value:

- None

Notes:

- This function must be used only when the data Cache is disabled.

__HAL_FLASH_ENABLE_IT

Description:

- Enable the specified FLASH interrupt.

Parameters:

- **__INTERRUPT__**: FLASH interrupt This parameter can be any combination of the following values:
 - FLASH_IT_EOP: End of FLASH Operation Interrupt
 - FLASH_IT_ERR: Error Interrupt

Return value:

- none

__HAL_FLASH_DISABLE_IT

Description:

- Disable the specified FLASH interrupt.

Parameters:

- **__INTERRUPT__**: FLASH interrupt This parameter can be any combination of the following values:
 - **FLASH_IT_EOP**: End of FLASH Operation Interrupt
 - **FLASH_IT_ERR**: Error Interrupt

Return value:

- none

__HAL_FLASH_GET_FLAG

Description:

- Get the specified FLASH flag status.

Parameters:

- **__FLAG__**: specifies the FLASH flags to check. This parameter can be any combination of the following values:
 - **FLASH_FLAG_EOP** : FLASH End of Operation flag
 - **FLASH_FLAG_OPERR** : FLASH operation Error flag
 - **FLASH_FLAG_WRPERR**: FLASH Write protected error flag
 - **FLASH_FLAG_PGAERR**: FLASH Programming Alignment error flag
 - **FLASH_FLAG_PGPERR**: FLASH Programming Parallelism error flag
 - **FLASH_FLAG_PGSERR**: FLASH Programming Sequence error flag
 - **FLASH_FLAG_BSY** : FLASH Busy flag

Return value:

- The: new state of **__FLAG__** (SET or RESET).

__HAL_FLASH_CLEAR_FLAG

Description:

- Clear the specified FLASH flags.

Parameters:

- **__FLAG__**: specifies the FLASH flags to clear. This parameter can be any combination of the following values:
 - **FLASH_FLAG_EOP** : FLASH End of Operation flag
 - **FLASH_FLAG_OPERR** : FLASH operation Error flag
 - **FLASH_FLAG_WRPERR**: FLASH Write protected error flag
 - **FLASH_FLAG_PGAERR**: FLASH Programming Alignment error flag
 - **FLASH_FLAG_PGPERR**: FLASH Programming Parallelism error flag
 - **FLASH_FLAG_PGSERR**: FLASH Programming Sequence error flag

Return value:

- none

FLASH Flag definition

FLASH_FLAG_EOP

FLASH End of Operation flag

FLASH_FLAG_OPERR

FLASH operation Error flag

FLASH_FLAG_WRPERR

FLASH Write protected error flag

FLASH_FLAG_PGAERR

FLASH Programming Alignment error flag

FLASH_FLAG_PGPERR

FLASH Programming Parallelism error flag

FLASH_FLAG_PGSERR

FLASH Programming Sequence error flag

FLASH_FLAG_BSY

FLASH Busy flag

FLASH Interrupt definition**FLASH_IT_EOP**

End of FLASH Operation Interrupt source

FLASH_IT_ERR

Error Interrupt source

FLASH Private macros to check input parameters**IS_FLASH_TYPEPROGRAM*****FLASH Keys*****RDP_KEY****FLASH_KEY1****FLASH_KEY2****FLASH_OPT_KEY1****FLASH_OPT_KEY2*****FLASH Latency*****FLASH_LATENCY_0**

FLASH Zero Latency cycle

FLASH_LATENCY_1

FLASH One Latency cycle

FLASH_LATENCY_2

FLASH Two Latency cycles

FLASH_LATENCY_3

FLASH Three Latency cycles

FLASH_LATENCY_4

FLASH Four Latency cycles

FLASH_LATENCY_5

FLASH Five Latency cycles

FLASH_LATENCY_6

FLASH Six Latency cycles

FLASH_LATENCY_7

FLASH Seven Latency cycles

FLASH Program Parallelism**FLASH_PSIZE_BYTE****FLASH_PSIZE_HALF_WORD****FLASH_PSIZE_WORD****FLASH_PSIZE_DOUBLE_WORD****CR_PSIZE_MASK*****FLASH Type Program*****FLASH_TYPEPROGRAM_BYTE**

Program byte (8-bit) at a specified address

FLASH_TYPEPROGRAM_HALFWORD

Program a half-word (16-bit) at a specified address

FLASH_TYPEPROGRAM_WORD

Program a word (32-bit) at a specified address

FLASH_TYPEPROGRAM_DOUBLEWORD

Program a double word (64-bit) at a specified address

20 HAL FLASH Extension Driver

20.1 FLASHEx Firmware driver registers structures

20.1.1 FLASH_EraseInitTypeDef

FLASH_EraseInitTypeDef is defined in the stm32f2xx_hal_flash_ex.h

Data Fields

- **uint32_t TypeErase**
- **uint32_t Banks**
- **uint32_t Sector**
- **uint32_t NbSectors**
- **uint32_t VoltageRange**

Field Documentation

- **uint32_t FLASH_EraseInitTypeDef::TypeErase**
Mass erase or sector Erase. This parameter can be a value of [FLASHEx_Type_Erase](#)
- **uint32_t FLASH_EraseInitTypeDef::Banks**
Select banks to erase when Mass erase is enabled. This parameter must be a value of [FLASHEx_Banks](#)
- **uint32_t FLASH_EraseInitTypeDef::Sector**
Initial FLASH sector to erase when Mass erase is disabled This parameter must be a value of [FLASHEx_Sectors](#)
- **uint32_t FLASH_EraseInitTypeDef::NbSectors**
Number of sectors to be erased. This parameter must be a value between 1 and (max number of sectors - value of Initial sector)
- **uint32_t FLASH_EraseInitTypeDef::VoltageRange**
The device voltage range which defines the erase parallelism This parameter must be a value of [FLASHEx_Voltage_Range](#)

20.1.2 FLASH_OBProgramInitTypeDef

FLASH_OBProgramInitTypeDef is defined in the stm32f2xx_hal_flash_ex.h

Data Fields

- **uint32_t OptionType**
- **uint32_t WRPState**
- **uint32_t WRPSector**
- **uint32_t Banks**
- **uint32_t RDPLLevel**
- **uint32_t BORLevel**
- **uint8_t USERConfig**

Field Documentation

- **uint32_t FLASH_OBProgramInitTypeDef::OptionType**
Option byte to be configured. This parameter can be a value of [FLASHEx_Option_Type](#)
- **uint32_t FLASH_OBProgramInitTypeDef::WRPState**
Write protection activation or deactivation. This parameter can be a value of [FLASHEx_WRP_State](#)
- **uint32_t FLASH_OBProgramInitTypeDef::WRPSector**
Specifies the sector(s) to be write protected. The value of this parameter depend on device used within the same series
- **uint32_t FLASH_OBProgramInitTypeDef::Banks**
Select banks for WRP activation/deactivation of all sectors. This parameter must be a value of [FLASHEx_Banks](#)

- **`uint32_t FLASH_OBProgramInitTypeDef::RDPLLevel`**
Set the read protection level. This parameter can be a value of [FLASHEx_Option_Bytes_Read_Protection](#)
- **`uint32_t FLASH_OBProgramInitTypeDef::BORLevel`**
Set the BOR Level. This parameter can be a value of [FLASHEx_BOR_Reset_Level](#)
- **`uint8_t FLASH_OBProgramInitTypeDef::USERConfig`**
Program the FLASH User Option Byte: IWDG_SW / RST_STOP / RST_STDBY.

20.2 FLASHEx Firmware driver API description

The following section lists the various functions of the FLASHEx library.

20.2.1 Flash Extension features

20.2.2 How to use this driver

This driver provides functions to configure and program the FLASH memory of all STM32F2xx devices. It includes

1. FLASH Memory Erase functions:
 - Lock and Unlock the FLASH interface using `HAL_FLASH_Unlock()` and `HAL_FLASH_Lock()` functions
 - Erase function: Erase sector, erase all sectors
 - There are two modes of erase :
 - Polling Mode using `HAL_FLASHEx_Erase()`
 - Interrupt Mode using `HAL_FLASHEx_Erase_IT()`
2. Option Bytes Programming functions: Use `HAL_FLASHEx_OBProgram()` to :
 - Set/Reset the write protection
 - Set the Read protection Level
 - Set the BOR level
 - Program the user Option Bytes

20.2.3 Extended programming operation functions

This subsection provides a set of functions allowing to manage the Extension FLASH programming operations.

This section contains the following APIs:

- [HAL_FLASHEx_Erase](#)
- [HAL_FLASHEx_Erase_IT](#)
- [HAL_FLASHEx_OBProgram](#)
- [HAL_FLASHEx_OBGetConfig](#)

20.2.4 Detailed description of functions

HAL_FLASHEx_Erase

Function name

`HAL_StatusTypeDef HAL_FLASHEx_Erase (FLASH_EraseInitTypeDef * pEraseInit, uint32_t * SectorError)`

Function description

Perform a mass erase or erase the specified FLASH memory sectors.

Parameters

- **`pEraseInit`**: pointer to an `FLASH_EraseInitTypeDef` structure that contains the configuration information for the erasing.
- **`SectorError`**: pointer to variable that contains the configuration information on faulty sector in case of error (0xFFFFFFFF means that all the sectors have been correctly erased)

Return values

- **`HAL`**: Status

HAL_FLASHEx_Erase_IT

Function name

HAL_StatusTypeDef HAL_FLASHEx_Erase_IT (FLASH_EraseInitTypeDef * pEraseInit)

Function description

Perform a mass erase or erase the specified FLASH memory sectors with interrupt enabled.

Parameters

- **pEraseInit:** pointer to an FLASH_EraseInitTypeDef structure that contains the configuration information for the erasing.

Return values

- **HAL:** Status

HAL_FLASHEx_OBProgram

Function name

HAL_StatusTypeDef HAL_FLASHEx_OBProgram (FLASH_OBProgramInitTypeDef * pOBInit)

Function description

Program option bytes.

Parameters

- **pOBInit:** pointer to an FLASH_OBInitStruct structure that contains the configuration information for the programming.

Return values

- **HAL:** Status

HAL_FLASHEx_OBGetConfig

Function name

void HAL_FLASHEx_OBGetConfig (FLASH_OBProgramInitTypeDef * pOBInit)

Function description

Get the Option byte configuration.

Parameters

- **pOBInit:** pointer to an FLASH_OBInitStruct structure that contains the configuration information for the programming.

Return values

- **None:**

FLASH_Erase_Sector

Function name

void FLASH_Erase_Sector (uint32_t Sector, uint8_t VoltageRange)

Function description

Erase the specified FLASH memory sector.

Parameters

- **Sector:** FLASH sector to erase The value of this parameter depend on device used within the same series
- **VoltageRange:** The device voltage range which defines the erase parallelism. This parameter can be one of the following values:
 - FLASH_VOLTAGE_RANGE_1: when the device voltage range is 1.8V to 2.1V, the operation will be done by byte (8-bit)
 - FLASH_VOLTAGE_RANGE_2: when the device voltage range is 2.1V to 2.7V, the operation will be done by half word (16-bit)
 - FLASH_VOLTAGE_RANGE_3: when the device voltage range is 2.7V to 3.6V, the operation will be done by word (32-bit)
 - FLASH_VOLTAGE_RANGE_4: when the device voltage range is 2.7V to 3.6V + External Vpp, the operation will be done by double word (64-bit)

Return values

- **None:**

FLASH_FlushCaches

Function name

void FLASH_FlushCaches (void)

Function description

Flush the instruction and data caches.

Return values

- **None:**

20.3 FLASHEx Firmware driver defines

The following section lists the various define and macros of the module.

20.3.1 FLASHEx

FLASHEx

FLASH Banks

FLASH_BANK_1

Bank 1

FLASH BOR Reset Level

OB_BOR_LEVEL3

Supply voltage ranges from 2.70 to 3.60 V

OB_BOR_LEVEL2

Supply voltage ranges from 2.40 to 2.70 V

OB_BOR_LEVEL1

Supply voltage ranges from 2.10 to 2.40 V

OB_BOR_OFF

Supply voltage ranges from 1.62 to 2.10 V

FLASH Private macros to check input parameters

IS_FLASH_TYPEERASE

IS_VOLTAGERANGE

IS_WRPSTATE

IS_OPTIONBYTE

IS_OB_RDP_LEVEL

IS_OB_IWDG_SOURCE

IS_OB_STOP_SOURCE

IS_OB_STDBY_SOURCE

IS_OB_BOR_LEVEL

IS_FLASH_LATENCY

IS_FLASH_BANK

IS_FLASH_SECTOR

IS_FLASH_ADDRESS

IS_FLASH_NBSECTORS

IS_OB_WRP_SECTOR

FLASH Mass Erase bit

FLASH_MER_BIT

only 1 MER Bit

FLASH Option Bytes IWatchdog

OB_IWDG_SW

Software IWDG selected

OB_IWDG_HW

Hardware IWDG selected

FLASH Option Bytes nRST_STDBY

OB_STDBY_NO_RST

No reset generated when entering in STANDBY

OB_STDBY_RST

Reset generated when entering in STANDBY

FLASH Option Bytes nRST_STOP

OB_STOP_NO_RST

No reset generated when entering in STOP

OB_STOP_RST

Reset generated when entering in STOP

FLASH Option Bytes Read Protection

OB_RDP_LEVEL_0

OB_RDP_LEVEL_1

OB_RDP_LEVEL_2

Warning: When enabling read protection level 2 it s no more possible to go back to level 1 or 0

FLASH Option Bytes Write Protection

OB_WRP_SECTOR_0

Write protection of Sector0

OB_WRP_SECTOR_1

Write protection of Sector1

OB_WRP_SECTOR_2

Write protection of Sector2

OB_WRP_SECTOR_3

Write protection of Sector3

OB_WRP_SECTOR_4

Write protection of Sector4

OB_WRP_SECTOR_5

Write protection of Sector5

OB_WRP_SECTOR_6

Write protection of Sector6

OB_WRP_SECTOR_7

Write protection of Sector7

OB_WRP_SECTOR_8

Write protection of Sector8

OB_WRP_SECTOR_9

Write protection of Sector9

OB_WRP_SECTOR_10

Write protection of Sector10

OB_WRP_SECTOR_11

Write protection of Sector11

OB_WRP_SECTOR_All

Write protection of all Sectors

FLASH Option Type

OPTIONBYTE_WRP

WRP option byte configuration

OPTIONBYTE_RDP

RDP option byte configuration

OPTIONBYTE_USER

USER option byte configuration

OPTIONBYTE_BOR

BOR option byte configuration

FLASH Sectors

FLASH_SECTOR_0

Sector Number 0

FLASH_SECTOR_1

Sector Number 1

FLASH_SECTOR_2

Sector Number 2

FLASH_SECTOR_3

Sector Number 3

FLASH_SECTOR_4

Sector Number 4

FLASH_SECTOR_5

Sector Number 5

FLASH_SECTOR_6

Sector Number 6

FLASH_SECTOR_7

Sector Number 7

FLASH_SECTOR_8

Sector Number 8

FLASH_SECTOR_9

Sector Number 9

FLASH_SECTOR_10

Sector Number 10

FLASH_SECTOR_11

Sector Number 11

FLASH Type Erase**FLASH_TYPEERASE_SECTORS**

Sectors erase only

FLASH_TYPEERASE_MASSERASE

Flash Mass erase activation

FLASH Voltage Range**FLASH_VOLTAGE_RANGE_1**

Device operating range: 1.8V to 2.1V

FLASH_VOLTAGE_RANGE_2

Device operating range: 2.1V to 2.7V

FLASH_VOLTAGE_RANGE_3

Device operating range: 2.7V to 3.6V

FLASH_VOLTAGE_RANGE_4

Device operating range: 2.7V to 3.6V + External Vpp

FLASH WRP State

OB_WRPSTATE_DISABLE

Disable the write protection of the desired bank 1 sectors

OB_WRPSTATE_ENABLE

Enable the write protection of the desired bank 1 sectors

21 HAL GPIO Generic Driver

21.1 GPIO Firmware driver registers structures

21.1.1 GPIO_InitTypeDef

GPIO_InitTypeDef is defined in the `stm32f2xx_hal_gpio.h`

Data Fields

- **uint32_t Pin**
- **uint32_t Mode**
- **uint32_t Pull**
- **uint32_t Speed**
- **uint32_t Alternate**

Field Documentation

- **uint32_t GPIO_InitTypeDef::Pin**
Specifies the GPIO pins to be configured. This parameter can be any value of [GPIO_pins_define](#)
- **uint32_t GPIO_InitTypeDef::Mode**
Specifies the operating mode for the selected pins. This parameter can be a value of [GPIO_mode_define](#)
- **uint32_t GPIO_InitTypeDef::Pull**
Specifies the Pull-up or Pull-Down activation for the selected pins. This parameter can be a value of [GPIO_pull_define](#)
- **uint32_t GPIO_InitTypeDef::Speed**
Specifies the speed for the selected pins. This parameter can be a value of [GPIO_speed_define](#)
- **uint32_t GPIO_InitTypeDef::Alternate**
Peripheral to be connected to the selected pins. This parameter can be a value of [GPIO_Alternate_function_selection](#)

21.2 GPIO Firmware driver API description

The following section lists the various functions of the GPIO library.

21.2.1 GPIO Peripheral features

Subject to the specific hardware characteristics of each I/O port listed in the datasheet, each port bit of the General Purpose IO (GPIO) Ports, can be individually configured by software in several modes:

- Input mode
- Analog mode
- Output mode
- Alternate function mode
- External interrupt/event lines

During and just after reset, the alternate functions and external interrupt lines are not active and the I/O ports are configured in input floating mode.

All GPIO pins have weak internal pull-up and pull-down resistors, which can be activated or not.

In Output or Alternate mode, each IO can be configured on open-drain or push-pull type and the IO speed can be selected depending on the VDD value.

All ports have external interrupt/event capability. To use external interrupt lines, the port must be configured in input mode. All available GPIO pins are connected to the 16 external interrupt/event lines from EXTI0 to EXTI15.

The external interrupt/event controller consists of up to 23 edge detectors (16 lines are connected to GPIO) for generating event/interrupt requests (each input line can be independently configured to select the type (interrupt or event) and the corresponding trigger event (rising or falling or both). Each line can also be masked independently.

21.2.2 How to use this driver

1. Enable the GPIO AHB clock using the following function: `__HAL_RCC_GPIOx_CLK_ENABLE()`.
2. Configure the GPIO pin(s) using `HAL_GPIO_Init()`.
 - Configure the IO mode using "Mode" member from `GPIO_InitTypeDef` structure
 - Activate Pull-up, Pull-down resistor using "Pull" member from `GPIO_InitTypeDef` structure.
 - In case of Output or alternate function mode selection: the speed is configured through "Speed" member from `GPIO_InitTypeDef` structure.
 - In alternate mode is selection, the alternate function connected to the IO is configured through "Alternate" member from `GPIO_InitTypeDef` structure.
 - Analog mode is required when a pin is to be used as ADC channel or DAC output.
 - In case of external interrupt/event selection the "Mode" member from `GPIO_InitTypeDef` structure select the type (interrupt or event) and the corresponding trigger event (rising or falling or both).
3. In case of external interrupt/event mode selection, configure NVIC IRQ priority mapped to the EXTI line using `HAL_NVIC_SetPriority()` and enable it using `HAL_NVIC_EnableIRQ()`.
4. To get the level of a pin configured in input mode use `HAL_GPIO_ReadPin()`.
5. To set/reset the level of a pin configured in output mode use `HAL_GPIO_WritePin()/HAL_GPIO_TogglePin()`.
6. To lock pin configuration until next reset use `HAL_GPIO_LockPin()`.
7. During and just after reset, the alternate functions are not active and the GPIO pins are configured in input floating mode (except JTAG pins).
8. The LSE oscillator pins `OSC32_IN` and `OSC32_OUT` can be used as general purpose (PC14 and PC15, respectively) when the LSE oscillator is off. The LSE has priority over the GPIO function.
9. The HSE oscillator pins `OSC_IN/OSC_OUT` can be used as general purpose PH0 and PH1, respectively, when the HSE oscillator is off. The HSE has priority over the GPIO function.

21.2.3 Initialization and de-initialization functions

This section provides functions allowing to initialize and de-initialize the GPIOs to be ready for use.

This section contains the following APIs:

- [*HAL_GPIO_Init*](#)
- [*HAL_GPIO_DeInit*](#)

21.2.4 IO operation functions

This section contains the following APIs:

- [*HAL_GPIO_ReadPin*](#)
- [*HAL_GPIO_WritePin*](#)
- [*HAL_GPIO_TogglePin*](#)
- [*HAL_GPIO_LockPin*](#)
- [*HAL_GPIO_EXTI_IRQHandler*](#)
- [*HAL_GPIO_EXTI_Callback*](#)

21.2.5 Detailed description of functions

HAL_GPIO_Init

Function name

void HAL_GPIO_Init (GPIO_TypeDef * GPIOx, GPIO_InitTypeDef * GPIO_Init)

Function description

Initializes the GPIOx peripheral according to the specified parameters in the `GPIO_Init`.

Parameters

- **GPIOx:** where x can be (A..I) to select the GPIO peripheral.
- **GPIO_Init:** pointer to a `GPIO_InitTypeDef` structure that contains the configuration information for the specified GPIO peripheral.

Return values

- **None:**

HAL_GPIO_DeInit

Function name

```
void HAL_GPIO_DeInit (GPIO_TypeDef * GPIOx, uint32_t GPIO_Pin)
```

Function description

De-initializes the GPIOx peripheral registers to their default reset values.

Parameters

- **GPIOx:** where x can be (A..I) to select the GPIO peripheral.
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).

Return values

- **None:**

HAL_GPIO_ReadPin

Function name

```
GPIO_PinState HAL_GPIO_ReadPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Reads the specified input port pin.

Parameters

- **GPIOx:** where x can be (A..I) to select the GPIO peripheral.
- **GPIO_Pin:** specifies the port bit to read. This parameter can be GPIO_PIN_x where x can be (0..15).

Return values

- **The:** input port pin value.

HAL_GPIO_WritePin

Function name

```
void HAL_GPIO_WritePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin, GPIO_PinState PinState)
```

Function description

Sets or clears the selected data port bit.

Parameters

- **GPIOx:** where x can be (A..I) to select the GPIO peripheral for all STM32F2XX devices
- **GPIO_Pin:** specifies the port bit to be written. This parameter can be one of GPIO_PIN_x where x can be (0..15).
- **PinState:** specifies the value to be written to the selected bit. This parameter can be one of the GPIO_PinState enum values:
 - GPIO_PIN_RESET: to clear the port pin
 - GPIO_PIN_SET: to set the port pin

Return values

- **None:**

Notes

- This function uses GPIOx_BSRR register to allow atomic read/modify accesses. In this way, there is no risk of an IRQ occurring between the read and the modify access.

HAL_GPIO_TogglePin

Function name

```
void HAL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Toggles the specified GPIO pins.

Parameters

- **GPIOx**: where x can be (A..I) to select the GPIO peripheral.
- **GPIO_Pin**: Specifies the pins to be toggled.

Return values

- **None**:

HAL_GPIO_LockPin

Function name

```
HAL_StatusTypeDef HAL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint16_t GPIO_Pin)
```

Function description

Locks GPIO Pins configuration registers.

Parameters

- **GPIOx**: where x can be (A..I) to select the GPIO peripheral for STM32F2XX family
- **GPIO_Pin**: specifies the port bit to be locked. This parameter can be any combination of GPIO_PIN_x where x can be (0..15).

Return values

- **None**:

Notes

- The locked registers are GPIOx_MODER, GPIOx_OTYPER, GPIOx_OSPEEDR, GPIOx_PUPDR, GPIOx_AFRL and GPIOx_AFRH.
- The configuration of the locked GPIO pins can no longer be modified until the next reset.

HAL_GPIO_EXTI_IRQHandler

Function name

```
void HAL_GPIO_EXTI_IRQHandler (uint16_t GPIO_Pin)
```

Function description

This function handles EXTI interrupt request.

Parameters

- **GPIO_Pin**: Specifies the pins connected EXTI line

Return values

- **None**:

HAL_GPIO_EXTI_Callback

Function name

void HAL_GPIO_EXTI_Callback (uint16_t GPIO_Pin)

Function description

EXTI line detection callbacks.

Parameters

- **GPIO_Pin:** Specifies the pins connected EXTI line

Return values

- **None:**

21.3 GPIO Firmware driver defines

The following section lists the various define and macros of the module.

21.3.1 GPIO

GPIO

GPIO Alternate function selection

GPIO_AF0_RTC_50Hz

GPIO_AF0_MCO

GPIO_AF0_TAMPER

GPIO_AF0_SWJ

GPIO_AF0_TRACE

GPIO_AF1_TIM1

GPIO_AF1_TIM2

GPIO_AF2_TIM3

GPIO_AF2_TIM4

GPIO_AF2_TIM5

GPIO_AF3_TIM8

GPIO_AF3_TIM9

GPIO_AF3_TIM10

GPIO_AF3_TIM11

GPIO_AF4_I2C1

GPIO_AF4_I2C2

GPIO_AF4_I2C3

GPIO_AF5_SPI1

GPIO_AF5_SPI2

GPIO_AF6_SPI3

GPIO_AF7_USART1

GPIO_AF7_USART2

GPIO_AF7_USART3

GPIO_AF8_UART4

GPIO_AF8_UART5

GPIO_AF8_USART6

GPIO_AF9_CAN1

GPIO_AF9_CAN2

GPIO_AF9_TIM12

GPIO_AF9_TIM13

GPIO_AF9_TIM14

GPIO_AF10_OTG_FS

GPIO_AF10_OTG_HS

GPIO_AF11_ETH

GPIO_AF12_FSMC

GPIO_AF12_OTG_HS_FS

GPIO_AF12_SDIO

GPIO_AF13_DCMI

GPIO_AF15_EVENTOUT

GPIO Exported Macros

__HAL_GPIO_EXTI_GET_FLAG

Description:

- Checks whether the specified EXTI line flag is set or not.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line flag to check. This parameter can be `GPIO_PIN_x` where `x` can be (0..15)

Return value:

- The: new state of `__EXTI_LINE__` (SET or RESET).

__HAL_GPIO_EXTI_CLEAR_FLAG

Description:

- Clears the EXTI's line pending flags.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI lines flags to clear. This parameter can be any combination of `GPIO_PIN_x` where x can be (0..15)

Return value:

- None

__HAL_GPIO_EXTI_GET_IT

Description:

- Checks whether the specified EXTI line is asserted or not.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line to check. This parameter can be `GPIO_PIN_x` where x can be (0..15)

Return value:

- The: new state of `__EXTI_LINE__` (SET or RESET).

__HAL_GPIO_EXTI_CLEAR_IT

Description:

- Clears the EXTI's line pending bits.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI lines to clear. This parameter can be any combination of `GPIO_PIN_x` where x can be (0..15)

Return value:

- None

__HAL_GPIO_EXTI_GENERATE_SWIT

Description:

- Generates a Software interrupt on selected EXTI line.

Parameters:

- `__EXTI_LINE__`: specifies the EXTI line to check. This parameter can be `GPIO_PIN_x` where x can be (0..15)

Return value:

- None

GPIO mode define

GPIO_MODE_INPUT

Input Floating Mode

GPIO_MODE_OUTPUT_PP

Output Push Pull Mode

GPIO_MODE_OUTPUT_OD

Output Open Drain Mode

GPIO_MODE_AF_PP

Alternate Function Push Pull Mode

GPIO_MODE_AF_OD

Alternate Function Open Drain Mode

GPIO_MODE_ANALOG

Analog Mode

GPIO_MODE_IT_RISING

External Interrupt Mode with Rising edge trigger detection

GPIO_MODE_IT_FALLING

External Interrupt Mode with Falling edge trigger detection

GPIO_MODE_IT_RISING_FALLING

External Interrupt Mode with Rising/Falling edge trigger detection

GPIO_MODE_EVT_RISING

External Event Mode with Rising edge trigger detection

GPIO_MODE_EVT_FALLING

External Event Mode with Falling edge trigger detection

GPIO_MODE_EVT_RISING_FALLING

External Event Mode with Rising/Falling edge trigger detection

GPIO pins define**GPIO_PIN_0****GPIO_PIN_1****GPIO_PIN_2****GPIO_PIN_3****GPIO_PIN_4****GPIO_PIN_5****GPIO_PIN_6****GPIO_PIN_7****GPIO_PIN_8****GPIO_PIN_9****GPIO_PIN_10****GPIO_PIN_11****GPIO_PIN_12****GPIO_PIN_13****GPIO_PIN_14****GPIO_PIN_15****GPIO_PIN_All**

GPIO_PIN_MASK

GPIO pull define

GPIO_NOPULL

No Pull-up or Pull-down activation

GPIO_PULLUP

Pull-up activation

GPIO_PULLDOWN

Pull-down activation

GPIO speed define

GPIO_SPEED_FREQ_LOW

IO works at 2 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_MEDIUM

range 12,5 MHz to 50 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_HIGH

range 25 MHz to 100 MHz, please refer to the product datasheet

GPIO_SPEED_FREQ_VERY_HIGH

range 50 MHz to 200 MHz, please refer to the product datasheet

22 HAL GPIO Extension Driver

22.1 GPIOEx Firmware driver defines

The following section lists the various define and macros of the module.

22.1.1 GPIOEx

GPIOEx

GPIO Get Port Index

GPIO_GET_INDEX

GPIO Check Alternate Function

IS_GPIO_AF

23 HAL HASH Generic Driver

23.1 HASH Firmware driver registers structures

23.1.1 HASH_InitTypeDef

HASH_InitTypeDef is defined in the stm32f2xx_hal_hash.h

Data Fields

- *uint32_t* *DataType*
- *uint32_t* *KeySize*
- *uint8_t* * *pKey*

Field Documentation

- *uint32_t* *HASH_InitTypeDef::DataType*
32-bit data, 16-bit data, 8-bit data or 1-bit data. This parameter can be a value of [HASH_Data_Type](#).
- *uint32_t* *HASH_InitTypeDef::KeySize*
The key size is used only in HMAC operation.
- *uint8_t** *HASH_InitTypeDef::pKey*
The key is used only in HMAC operation.

23.1.2 HASH_HandleTypeDef

HASH_HandleTypeDef is defined in the stm32f2xx_hal_hash.h

Data Fields

- *HASH_InitTypeDef* *Init*
- *uint8_t* * *pHashInBuffPtr*
- *uint8_t* * *pHashOutBuffPtr*
- *uint8_t* * *pHashKeyBuffPtr*
- *uint8_t* * *pHashMsgBuffPtr*
- *uint32_t* *HashBuffSize*
- *__IO uint32_t* *HashInCount*
- *__IO uint32_t* *HashITCounter*
- *__IO uint32_t* *HashKeyCount*
- *HAL_StatusTypeDef* *Status*
- *HAL_HASH_PhaseTypeDef* *Phase*
- *DMA_HandleTypeDef* * *hdmain*
- *HAL_LockTypeDef* *Lock*
- *__IO HAL_HASH_StateTypeDef* *State*
- *HAL_HASH_SuspendTypeDef* *SuspendRequest*
- *FlagStatus* *DigestCalculationDisable*
- *__IO uint32_t* *NbWordsAlreadyPushed*
- *__IO uint32_t* *ErrorCode*

Field Documentation

- *HASH_InitTypeDef* *HASH_HandleTypeDef::Init*
HASH required parameters
- *uint8_t** *HASH_HandleTypeDef::pHashInBuffPtr*
Pointer to input buffer
- *uint8_t** *HASH_HandleTypeDef::pHashOutBuffPtr*
Pointer to output buffer (digest)

- ***uint8_t* HASH_HandleTypeDef::pHashKeyBuffPtr***
Pointer to key buffer (HMAC only)
- ***uint8_t* HASH_HandleTypeDef::pHashMsgBuffPtr***
Pointer to message buffer (HMAC only)
- ***uint32_t HASH_HandleTypeDef::HashBuffSize***
Size of buffer to be processed
- ***__IO uint32_t HASH_HandleTypeDef::HashInCount***
Counter of inputted data
- ***__IO uint32_t HASH_HandleTypeDef::HashITCounter***
Counter of issued interrupts
- ***__IO uint32_t HASH_HandleTypeDef::HashKeyCount***
Counter for Key inputted data (HMAC only)
- ***HAL_StatusTypeDef HASH_HandleTypeDef::Status***
HASH peripheral status
- ***HAL_HASH_PhaseTypeDef HASH_HandleTypeDef::Phase***
HASH peripheral phase
- ***DMA_HandleTypeDef* HASH_HandleTypeDef::hdmain***
HASH In DMA Handle parameters
- ***HAL_LockTypeDef HASH_HandleTypeDef::Lock***
Locking object
- ***__IO HAL_HASH_StateTypeDef HASH_HandleTypeDef::State***
HASH peripheral state
- ***HAL_HASH_SuspendTypeDef HASH_HandleTypeDef::SuspendRequest***
HASH peripheral suspension request flag
- ***FlagStatus HASH_HandleTypeDef::DigestCalculationDisable***
Digest calculation phase skip (MDMAT bit control) for multi-buffers DMA-based HMAC computation
- ***__IO uint32_t HASH_HandleTypeDef::NbWordsAlreadyPushed***
Numbers of words already pushed in FIFO before inputting new block
- ***__IO uint32_t HASH_HandleTypeDef::ErrorCode***
HASH Error code

23.2 HASH Firmware driver API description

The following section lists the various functions of the HASH library.

23.2.1 How to use this driver

The HASH HAL driver can be used as follows:

1. Initialize the HASH low level resources by implementing the HAL_HASH_MspInit():
 - a. Enable the HASH interface clock using `__HASH_CLK_ENABLE()`
 - b. When resorting to interrupt-based APIs (e.g. `HAL_HASH_XXX_Start_IT()`)
 - Configure the HASH interrupt priority using `HAL_NVIC_SetPriority()`
 - Enable the HASH IRQ handler using `HAL_NVIC_EnableIRQ()`
 - In HASH IRQ handler, call `HAL_HASH_IRQHandler()` API
 - c. When resorting to DMA-based APIs (e.g. `HAL_HASH_XXX_Start_DMA()`)
 - Enable the DMAx interface clock using `__DMAx_CLK_ENABLE()`
 - Configure and enable one DMA stream to manage data transfer from memory to peripheral (input stream). Managing data transfer from peripheral to memory can be performed only using CPU.
 - Associate the initialized DMA handle to the HASH DMA handle using `__HAL_LINKDMA()`
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Stream: use `HAL_NVIC_SetPriority()` and `HAL_NVIC_EnableIRQ()`

2. Initialize the HASH HAL using HAL_HASH_Init(). This function:
 - a. resorts to HAL_HASH_MspInit() for low-level initialization,
 - b. configures the data type: 1-bit, 8-bit, 16-bit or 32-bit.
3. Three processing schemes are available:
 - a. Polling mode: processing APIs are blocking functions i.e. they process the data and wait till the digest computation is finished, e.g. HAL_HASH_xxx_Start() for HASH or HAL_HMAC_xxx_Start() for HMAC
 - b. Interrupt mode: processing APIs are not blocking functions i.e. they process the data under interrupt, e.g. HAL_HASH_xxx_Start_IT() for HASH or HAL_HMAC_xxx_Start_IT() for HMAC
 - c. DMA mode: processing APIs are not blocking functions and the CPU is not used for data transfer i.e. the data transfer is ensured by DMA, e.g. HAL_HASH_xxx_Start_DMA() for HASH or HAL_HMAC_xxx_Start_DMA() for HMAC. Note that in DMA mode, a call to HAL_HASH_xxx_Finish() is then required to retrieve the digest.
4. When the processing function is called after HAL_HASH_Init(), the HASH peripheral is initialized and processes the buffer fed in input. When the input data have all been fed to the IP, the digest computation can start.
5. Multi-buffer processing is possible in polling and DMA mode.
 - a. In polling mode, only multi-buffer HASH processing is possible. API HAL_HASH_xxx_Accumulate() must be called for each input buffer, except for the last one. User must resort to HAL_HASH_xxx_Start() to enter the last one and retrieve as well the computed digest.
 - b. In DMA mode, multi-buffer HASH and HMAC processing are possible.
 - HASH processing DMA mode: processing APIs are not blocking functions and the CPU is not used for data transfer i.e. the data transfer is ensured by DMA e.g. HAL_HASH_SHA1_Start_DMA()
 - HMAC processing (requires to resort to extended functions): after initialization, the key and the first input buffer are entered in the IP with the API HAL_HMACEx_xxx_Step1_2_DMA(). This carries out HMAC step 1 and starts step 2. The following buffers are next entered with the API HAL_HMACEx_xxx_Step2_DMA(). At this point, the HMAC processing is still carrying out step 2. Then, step 2 for the last input buffer and step 3 are carried out by a single call to HAL_HMACEx_xxx_Step2_3_DMA(). The digest can finally be retrieved with a call to API HAL_HASH_xxx_Finish().
6. Context swapping.
 - a. Two APIs are available to suspend HASH or HMAC processing:
 - HAL_HASH_SwFeed_ProcessSuspend() when data are entered by software (polling or IT mode),
 - HAL_HASH_DMAFeed_ProcessSuspend() when data are entered by DMA.
 - b. When HASH or HMAC processing is suspended, HAL_HASH_ContextSaving() allows to save in memory the IP context. This context can be restored afterwards to resume the HASH processing thanks to HAL_HASH_ContextRestoring().
 - c. Once the HASH IP has been restored to the same configuration as that at suspension time, processing can be restarted with the same API call (same API, same handle, same parameters) as done before the suspension. Relevant parameters to restart at the proper location are internally saved in the HASH handle.
7. Call HAL_HASH_DeInit() to deinitialize the HASH peripheral.

Callback registration

1. The compilation define USE_HAL_HASH_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks. Use function @ref HAL_HASH_RegisterCallback() to register a user callback.
2. Function @ref HAL_HASH_RegisterCallback() allows to register following callbacks: (+) InCpltCallback : callback for input completion. (+) DgstCpltCallback : callback for digest computation completion. (+) ErrorCallback : callback for error. (+) MspInitCallback : HASH MspInit. (+) MspDeInitCallback : HASH MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

3. Use function `@ref HAL_HASH_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `@ref HAL_HASH_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks: (+) `InCpltCallback` : callback for input completion. (+) `DgstCpltCallback` : callback for digest computation completion. (+) `ErrorCallback` : callback for error. (+) `MspInitCallback` : HASH MspInit. (+) `MspDeInitCallback` : HASH MspDeInit.
 4. By default, after the `@ref HAL_HASH_Init` and if the state is `HAL_HASH_STATE_RESET` all callbacks are reset to the corresponding legacy weak (surcharged) functions: examples `@ref HAL_HASH_InCpltCallback()`, `@ref HAL_HASH_DgstCpltCallback()` Exception done for `MspInit` and `MspDeInit` callbacks that are respectively reset to the legacy weak (surcharged) functions in the `@ref HAL_HASH_Init` and `@ref HAL_HASH_DeInit` only when these callbacks are null (not registered beforehand) If not, `MspInit` or `MspDeInit` are not null, the `@ref HAL_HASH_Init` and `@ref HAL_HASH_DeInit` keep and use the user `MspInit/MspDeInit` callbacks (registered beforehand). Callbacks can be registered/unregistered in READY state only. Exception done for `MspInit/MspDeInit` callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) `MspInit/DeInit` callbacks can be used during the `Init/DeInit`. In that case first register the `MspInit/MspDeInit` user callbacks using `@ref HAL_HASH_RegisterCallback` before calling `@ref HAL_HASH_DeInit` or `@ref HAL_HASH_Init` function. When The compilation define `USE_HAL_HASH_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.
- (#) The compilation define `USE_HAL_HASH_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use function `@ref HAL_HASH_RegisterCallback()` to register a user callback. (#) Function `@ref HAL_HASH_RegisterCallback()` allows to register following callbacks:
- `InCpltCallback` : callback for input completion.
 - `DgstCpltCallback` : callback for digest computation completion.
 - `ErrorCallback` : callback for error.
 - `MspInitCallback` : HASH MspInit.
 - `MspDeInitCallback` : HASH MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. (#) Use function `@ref HAL_HASH_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `@ref HAL_HASH_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:
 - `InCpltCallback` : callback for input completion.
 - `DgstCpltCallback` : callback for digest computation completion.
 - `ErrorCallback` : callback for error.
 - `MspInitCallback` : HASH MspInit.
 - `MspDeInitCallback` : HASH MspDeInit. (#) By default, after the `@ref HAL_HASH_Init` and if the state is `HAL_HASH_STATE_RESET` all callbacks are reset to the corresponding legacy weak (surcharged) functions: examples `@ref HAL_HASH_InCpltCallback()`, `@ref HAL_HASH_DgstCpltCallback()` Exception done for `MspInit` and `MspDeInit` callbacks that are respectively reset to the legacy weak (surcharged) functions in the `@ref HAL_HASH_Init` and `@ref HAL_HASH_DeInit` only when these callbacks are null (not registered beforehand) If not, `MspInit` or `MspDeInit` are not null, the `@ref HAL_HASH_Init` and `@ref HAL_HASH_DeInit` keep and use the user `MspInit/MspDeInit` callbacks (registered beforehand). Callbacks can be registered/unregistered in READY state only. Exception done for `MspInit/MspDeInit` callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) `MspInit/DeInit` callbacks can be used during the `Init/DeInit`. In that case first register the `MspInit/MspDeInit` user callbacks using `@ref HAL_HASH_RegisterCallback` before calling `@ref HAL_HASH_DeInit` or `@ref HAL_HASH_Init` function. When The compilation define `USE_HAL_HASH_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

23.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

- Initialize the HASH according to the specified parameters in the `HASH_InitTypeDef` and create the associated handle
- DeInitialize the HASH peripheral
- Initialize the HASH MCU Specific Package (MSP)
- DeInitialize the HASH MSP

This section provides as well call back functions definitions for user code to manage:

- Input data transfer to IP completion
- Calculated digest retrieval completion
- Error management

This section contains the following APIs:

- [*HAL_HASH_Init*](#)
- [*HAL_HASH_DeInit*](#)
- [*HAL_HASH_MspInit*](#)
- [*HAL_HASH_MspDeInit*](#)
- [*HAL_HASH_InCpltCallback*](#)
- [*HAL_HASH_DgstCpltCallback*](#)
- [*HAL_HASH_ErrorCallback*](#)

23.2.3 Polling mode HASH processing functions

This section provides functions allowing to calculate in polling mode the hash value using one of the following algorithms:

- MD5
 - [*HAL_HASH_MD5_Start\(\)*](#)
 - [*HAL_HASH_MD5_Accumulate\(\)*](#)
- SHA1
 - [*HAL_HASH_SHA1_Start\(\)*](#)
 - [*HAL_HASH_SHA1_Accumulate\(\)*](#)

For a single buffer to be hashed, user can resort to [*HAL_HASH_xxx_Start\(\)*](#).

In case of multi-buffer HASH processing (a single digest is computed while several buffers are fed to the IP), the user can resort to successive calls to [*HAL_HASH_xxx_Accumulate\(\)*](#) and wrap-up the digest computation by a call to [*HAL_HASH_xxx_Start\(\)*](#).

This section contains the following APIs:

- [*HAL_HASH_MD5_Start*](#)
- [*HAL_HASH_MD5_Accumulate*](#)
- [*HAL_HASH_SHA1_Start*](#)
- [*HAL_HASH_SHA1_Accumulate*](#)

23.2.4 Interruption mode HASH processing functions

This section provides functions allowing to calculate in interrupt mode the hash value using one of the following algorithms:

- MD5
 - [*HAL_HASH_MD5_Start_IT\(\)*](#)
- SHA1
 - [*HAL_HASH_SHA1_Start_IT\(\)*](#)

API [*HAL_HASH_IRQHandler\(\)*](#) manages each HASH interruption.

Note that [*HAL_HASH_IRQHandler\(\)*](#) manages as well HASH IP interruptions when in HMAC processing mode.

This section contains the following APIs:

- [*HAL_HASH_MD5_Start_IT*](#)
- [*HAL_HASH_SHA1_Start_IT*](#)
- [*HAL_HASH_IRQHandler*](#)

23.2.5 DMA mode HASH processing functions

This section provides functions allowing to calculate in DMA mode the hash value using one of the following algorithms:

- MD5
 - HAL_HASH_MD5_Start_DMA()
 - HAL_HASH_MD5_Finish()
- SHA1
 - HAL_HASH_SHA1_Start_DMA()
 - HAL_HASH_SHA1_Finish()

When resorting to DMA mode to enter the data in the IP, user must resort to HAL_HASH_xxx_Start_DMA() then read the resulting digest with HAL_HASH_xxx_Finish().

This section contains the following APIs:

- [HAL_HASH_MD5_Start_DMA](#)
- [HAL_HASH_MD5_Finish](#)
- [HAL_HASH_SHA1_Start_DMA](#)
- [HAL_HASH_SHA1_Finish](#)

23.2.6 Polling mode HMAC processing functions

This section provides functions allowing to calculate in polling mode the HMAC value using one of the following algorithms:

- MD5
 - HAL_HMAC_MD5_Start()
- SHA1
 - HAL_HMAC_SHA1_Start()

This section contains the following APIs:

- [HAL_HMAC_MD5_Start](#)
- [HAL_HMAC_SHA1_Start](#)

23.2.7 Interrupt mode HMAC processing functions

This section provides functions allowing to calculate in interrupt mode the HMAC value using one of the following algorithms:

- MD5
 - HAL_HMAC_MD5_Start_IT()
- SHA1
 - HAL_HMAC_SHA1_Start_IT()

This section contains the following APIs:

- [HAL_HMAC_MD5_Start_IT](#)
- [HAL_HMAC_SHA1_Start_IT](#)

23.2.8 DMA mode HMAC processing functions

This section provides functions allowing to calculate in DMA mode the HMAC value using one of the following algorithms:

- MD5
 - HAL_HMAC_MD5_Start_DMA()
- SHA1
 - HAL_HMAC_SHA1_Start_DMA()

When resorting to DMA mode to enter the data in the IP for HMAC processing, user must resort to HAL_HMAC_xxx_Start_DMA() then read the resulting digest with HAL_HASH_xxx_Finish().

This section contains the following APIs:

- [HAL_HMAC_MD5_Start_DMA](#)
- [HAL_HMAC_SHA1_Start_DMA](#)

23.2.9 Peripheral State methods

This section permits to get in run-time the state and the peripheral handle status of the peripheral:

- HAL_HASH_GetState()
- HAL_HASH_GetStatus()

Additionally, this subsection provides functions allowing to save and restore the HASH or HMAC processing context in case of calculation suspension:

- HAL_HASH_ContextSaving()
- HAL_HASH_ContextRestoring()

This subsection provides functions allowing to suspend the HASH processing

- when input are fed to the IP by software
 - HAL_HASH_SwFeed_ProcessSuspend()
- when input are fed to the IP by DMA
 - HAL_HASH_DMAFeed_ProcessSuspend()

This section contains the following APIs:

- [HAL_HASH_GetState](#)
- [HAL_HASH_GetStatus](#)
- [HAL_HASH_ContextSaving](#)
- [HAL_HASH_ContextRestoring](#)
- [HAL_HASH_SwFeed_ProcessSuspend](#)
- [HAL_HASH_DMAFeed_ProcessSuspend](#)
- [HAL_HASH_GetError](#)

23.2.10 Detailed description of functions

HAL_HASH_Init

Function name

HAL_StatusTypeDef HAL_HASH_Init (HASH_HandleTypeDef * hhash)

Function description

Initialize the HASH according to the specified parameters in the HASH_HandleTypeDef and create the associated handle.

Parameters

- **hhash:** HASH handle

Return values

- **HAL:** status

Notes

- Only MDMAT and DATATYPE bits of HASH IP are set by HAL_HASH_Init(), other configuration bits are set by HASH or HMAC processing APIs.

HAL_HASH_DeInit

Function name

HAL_StatusTypeDef HAL_HASH_DeInit (HASH_HandleTypeDef * hhash)

Function description

Deinitialize the HASH peripheral.

Parameters

- **hhash:** HASH handle.

Return values

- **HAL:** status

HAL_HASH_MspInit

Function name

```
void HAL_HASH_MspInit (HASH_HandleTypeDef * hhash)
```

Function description

Initialize the HASH MSP.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

HAL_HASH_MspDeInit

Function name

```
void HAL_HASH_MspDeInit (HASH_HandleTypeDef * hhash)
```

Function description

DeInitialize the HASH MSP.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

HAL_HASH_InCpltCallback

Function name

```
void HAL_HASH_InCpltCallback (HASH_HandleTypeDef * hhash)
```

Function description

Input data transfer complete call back.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

Notes

- HAL_HASH_InCpltCallback() is called when the complete input message has been fed to the IP. This API is invoked only when input data are entered under interruption or thru DMA.

HAL_HASH_DgstCpltCallback

Function name

```
void HAL_HASH_DgstCpltCallback (HASH_HandleTypeDef * hhash)
```

Function description

Digest computation complete call back.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

Notes

- HAL_HASH_DgstCpltCallback() is used under interruption, is not relevant with DMA.

HAL_HASH_ErrorCallback

Function name

```
void HAL_HASH_ErrorCallback (HASH_HandleTypeDef * hhash)
```

Function description

Error callback.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

Notes

- Code user can resort to hhash->Status (HAL_ERROR, HAL_TIMEOUT,...) to retrieve the error type.

HAL_HASH_SHA1_Start

Function name

```
HAL_StatusTypeDef HAL_HASH_SHA1_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer,
uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout)
```

Function description

Initialize the HASH peripheral in SHA1 mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest. Digest size is 20 bytes.
- **Timeout:** Timeout value

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HAL_HASH_MD5_Start

Function name

```
HAL_StatusTypeDef HAL_HASH_MD5_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t
Size, uint8_t * pOutBuffer, uint32_t Timeout)
```

Function description

Initialize the HASH peripheral in MD5 mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest. Digest size is 16 bytes.
- **Timeout:** Timeout value

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HAL_HASH_MD5_Accumulate

Function name

HAL_StatusTypeDef HAL_HASH_MD5_Accumulate (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

If not already done, initialize the HASH peripheral in MD5 mode then processes pInBuffer.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes, must be a multiple of 4.

Return values

- **HAL:** status

Notes

- Consecutive calls to HAL_HASH_MD5_Accumulate() can be used to feed several input buffers back-to-back to the IP that will yield a single HASH signature once all buffers have been entered. Wrap-up of input buffers feeding and retrieval of digest is done by a call to HAL_HASH_MD5_Start().
- Field hhash->Phase of HASH handle is tested to check whether or not the IP has already been initialized.
- Digest is not retrieved by this API, user must resort to HAL_HASH_MD5_Start() to read it, feeding at the same time the last input buffer to the IP.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted. Only HAL_HASH_MD5_Start() is able to manage the ending buffer with a length in bytes not a multiple of 4.

HAL_HASH_SHA1_Accumulate

Function name

HAL_StatusTypeDef HAL_HASH_SHA1_Accumulate (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

If not already done, initialize the HASH peripheral in SHA1 mode then processes pInBuffer.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes, must be a multiple of 4.

Return values

- **HAL:** status

Notes

- Consecutive calls to HAL_HASH_SHA1_Accumulate() can be used to feed several input buffers back-to-back to the IP that will yield a single HASH signature once all buffers have been entered. Wrap-up of input buffers feeding and retrieval of digest is done by a call to HAL_HASH_SHA1_Start().
- Field hhash->Phase of HASH handle is tested to check whether or not the IP has already been initialized.
- Digest is not retrieved by this API, user must resort to HAL_HASH_SHA1_Start() to read it, feeding at the same time the last input buffer to the IP.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted. Only HAL_HASH_SHA1_Start() is able to manage the ending buffer with a length in bytes not a multiple of 4.

HAL_HASH_SHA1_Start_IT

Function name

HAL_StatusTypeDef HAL_HASH_SHA1_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in SHA1 mode, next process pInBuffer then read the computed digest in interruption mode.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest. Digest size is 20 bytes.

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HAL_HASH_MD5_Start_IT

Function name

HAL_StatusTypeDef HAL_HASH_MD5_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in MD5 mode, next process pInBuffer then read the computed digest in interruption mode.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest. Digest size is 16 bytes.

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HAL_HASH_IRQHandler

Function name

```
void HAL_HASH_IRQHandler (HASH_HandleTypeDef * hhash)
```

Function description

Handle HASH interrupt request.

Parameters

- **hhash**: HASH handle.

Return values

- **None**:

Notes

- HAL_HASH_IRQHandler() handles interrupts in HMAC processing as well.
- In case of error reported during the HASH interruption processing, HAL_HASH_ErrorCallback() API is called so that user code can manage the error. The error type is available in hhash->Status field.

HAL_HASH_SHA1_Start_DMA

Function name

```
HAL_StatusTypeDef HAL_HASH_SHA1_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)
```

Function description

Initialize the HASH peripheral in SHA1 mode then initiate a DMA transfer to feed the input buffer to the IP.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Once the DMA transfer is finished, HAL_HASH_SHA1_Finish() API must be called to retrieve the computed digest.

HAL_HASH_SHA1_Finish

Function name

```
HAL_StatusTypeDef HAL_HASH_SHA1_Finish (HASH_HandleTypeDef * hhash, uint8_t * pOutBuffer, uint32_t Timeout)
```

Function description

Return the computed digest in SHA1 mode.

Parameters

- **hhash**: HASH handle.
- **pOutBuffer**: pointer to the computed digest. Digest size is 20 bytes.
- **Timeout**: Timeout value.

Return values

- **HAL:** status

Notes

- The API waits for DCIS to be set then reads the computed digest.
- HAL_HASH_SHA1_Finish() can be used as well to retrieve the digest in HMAC SHA1 mode.

HAL_HASH_MD5_Start_DMA

Function name

HAL_StatusTypeDef HAL_HASH_MD5_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

Initialize the HASH peripheral in MD5 mode then initiate a DMA transfer to feed the input buffer to the IP.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.

Return values

- **HAL:** status

Notes

- Once the DMA transfer is finished, HAL_HASH_MD5_Finish() API must be called to retrieve the computed digest.

HAL_HASH_MD5_Finish

Function name

HAL_StatusTypeDef HAL_HASH_MD5_Finish (HASH_HandleTypeDef * hhash, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Return the computed digest in MD5 mode.

Parameters

- **hhash:** HASH handle.
- **pOutBuffer:** pointer to the computed digest. Digest size is 16 bytes.
- **Timeout:** Timeout value.

Return values

- **HAL:** status

Notes

- The API waits for DCIS to be set then reads the computed digest.
- HAL_HASH_MD5_Finish() can be used as well to retrieve the digest in HMAC MD5 mode.

HAL_HMAC_SHA1_Start

Function name

HAL_StatusTypeDef HAL_HMAC_SHA1_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Initialize the HASH peripheral in HMAC SHA1 mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 20 bytes.
- **Timeout**: Timeout value.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMAC_MD5_Start

Function name

HAL_StatusTypeDef HAL_HMAC_MD5_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout)

Function description

Initialize the HASH peripheral in HMAC MD5 mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 16 bytes.
- **Timeout**: Timeout value.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMAC_MD5_Start_IT

Function name

HAL_StatusTypeDef HAL_HMAC_MD5_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in HMAC MD5 mode, next process pInBuffer then read the computed digest in interrupt mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 16 bytes.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMAC_SHA1_Start_IT

Function name

HAL_StatusTypeDef HAL_HMAC_SHA1_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer)

Function description

Initialize the HASH peripheral in HMAC SHA1 mode, next process pInBuffer then read the computed digest in interrupt mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest. Digest size is 20 bytes.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMAC_SHA1_Start_DMA

Function name

HAL_StatusTypeDef HAL_HMAC_SHA1_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

Initialize the HASH peripheral in HMAC SHA1 mode then initiate the required DMA transfers to feed the key and the input buffer to the IP.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.

Return values

- **HAL**: status

Notes

- Once the DMA transfers are finished (indicated by hhash->State set back to HAL_HASH_STATE_READY), HAL_HASH_SHA1_Finish() API must be called to retrieve the computed digest.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HMAC_MD5_Start_DMA

Function name

HAL_StatusTypeDef HAL_HMAC_MD5_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size)

Function description

Initialize the HASH peripheral in HMAC MD5 mode then initiate the required DMA transfers to feed the key and the input buffer to the IP.

Parameters

- hhash**: HASH handle.
- pInBuffer**: pointer to the input buffer (buffer to be hashed).
- Size**: length of the input buffer in bytes.

Return values

- HAL**: status

Notes

- Once the DMA transfers are finished (indicated by hhash->State set back to HAL_HASH_STATE_READY), HAL_HASH_MD5_Finish() API must be called to retrieve the computed digest.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HAL_HASH_GetState

Function name

HAL_HASH_StateTypeDef HAL_HASH_GetState (HASH_HandleTypeDef * hhash)

Function description

Return the HASH handle state.

Parameters

- hhash**: HASH handle.

Return values

- HAL**: HASH state

Notes

- The API yields the current state of the handle (BUSY, READY,...).

HAL_HASH_GetStatus

Function name

HAL_StatusTypeDef HAL_HASH_GetStatus (HASH_HandleTypeDef * hhash)

Function description

Return the HASH HAL status.

Parameters

- hhash**: HASH handle.

Return values

- **HAL:** status

Notes

- The API yields the HAL status of the handle: it is the result of the latest HASH processing and allows to report any issue (e.g. HAL_TIMEOUT).

HAL_HASH_ContextSaving

Function name

```
void HAL_HASH_ContextSaving (HASH_HandleTypeDef * hhash, uint8_t * pMemBuffer)
```

Function description

Save the HASH context in case of processing suspension.

Parameters

- **hhash:** HASH handle.
- **pMemBuffer:** pointer to the memory buffer where the HASH context is saved.

Return values

- **None:**

Notes

- The IMR, STR, CR then all the CSR registers are saved in that order. Only the r/w bits are read to be restored later on.
- By default, all the context swap registers (there are HASH_NUMBER_OF_CSR_REGISTERS of those) are saved.
- pMemBuffer points to a buffer allocated by the user. The buffer size must be at least (HASH_NUMBER_OF_CSR_REGISTERS + 3) * 4 uint8 long.

HAL_HASH_ContextRestoring

Function name

```
void HAL_HASH_ContextRestoring (HASH_HandleTypeDef * hhash, uint8_t * pMemBuffer)
```

Function description

Restore the HASH context in case of processing resumption.

Parameters

- **hhash:** HASH handle.
- **pMemBuffer:** pointer to the memory buffer where the HASH context is stored.

Return values

- **None:**

Notes

- The IMR, STR, CR then all the CSR registers are restored in that order. Only the r/w bits are restored.
- By default, all the context swap registers (HASH_NUMBER_OF_CSR_REGISTERS of those) are restored (all of them have been saved by default beforehand).

HAL_HASH_SwFeed_ProcessSuspend

Function name

```
void HAL_HASH_SwFeed_ProcessSuspend (HASH_HandleTypeDef * hhash)
```

Function description

Initiate HASH processing suspension when in polling or interruption mode.

Parameters

- **hhash:** HASH handle.

Return values

- **None:**

Notes

- Set the handle field SuspendRequest to the appropriate value so that the on-going HASH processing is suspended as soon as the required conditions are met. Note that the actual suspension is carried out by the functions HASH_WriteData() in polling mode and HASH_IT() in interruption mode.

HAL_HASH_DMAFeed_ProcessSuspend

Function name

HAL_StatusTypeDef HAL_HASH_DMAFeed_ProcessSuspend (HASH_HandleTypeDef * hhash)

Function description

Suspend the HASH processing when in DMA mode.

Parameters

- **hhash:** HASH handle.

Return values

- **HAL:** status

Notes

- When suspension attempt occurs at the very end of a DMA transfer and all the data have already been entered in the IP, hhash->State is set to HAL_HASH_STATE_READY and the API returns HAL_ERROR. It is recommended to wrap-up the processing in reading the digest as usual.

HAL_HASH_GetError

Function name

uint32_t HAL_HASH_GetError (HASH_HandleTypeDef * hhash)

Function description

Return the HASH handle error code.

Parameters

- **hhash:** pointer to a HASH_HandleTypeDef structure.

Return values

- **HASH:** Error Code

HASH_Start

Function name

HAL_StatusTypeDef HASH_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Timeout, uint32_t Algorithm)

Function description

Initialize the HASH peripheral, next process pInBuffer then read the computed digest.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest.
- **Timeout:** Timeout value.
- **Algorithm:** HASH algorithm.

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HASH_Accumulate

Function name

HAL_StatusTypeDef HASH_Accumulate (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint32_t Algorithm)

Function description

If not already done, initialize the HASH peripheral then processes pInBuffer.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes, must be a multiple of 4.
- **Algorithm:** HASH algorithm.

Return values

- **HAL:** status

Notes

- Field hhash->Phase of HASH handle is tested to check whether or not the IP has already been initialized.
- The input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted.

HASH_Start_IT

Function name

HAL_StatusTypeDef HASH_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Algorithm)

Function description

Initialize the HASH peripheral, next process pInBuffer then read the computed digest in interruption mode.

Parameters

- **hhash:** HASH handle.
- **pInBuffer:** pointer to the input buffer (buffer to be hashed).
- **Size:** length of the input buffer in bytes.
- **pOutBuffer:** pointer to the computed digest.
- **Algorithm:** HASH algorithm.

Return values

- **HAL:** status

Notes

- Digest is available in pOutBuffer.

HASH_Start_DMA

Function name

```
HAL_StatusTypeDef HASH_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size,
uint32_t Algorithm)
```

Function description

Initialize the HASH peripheral then initiate a DMA transfer to feed the input buffer to the IP.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **Algorithm**: HASH algorithm.

Return values

- **HAL**: status

HASH_Finish

Function name

```
HAL_StatusTypeDef HASH_Finish (HASH_HandleTypeDef * hhash, uint8_t * pOutBuffer, uint32_t
Timeout)
```

Function description

Return the computed digest.

Parameters

- **hhash**: HASH handle.
- **pOutBuffer**: pointer to the computed digest.
- **Timeout**: Timeout value.

Return values

- **HAL**: status

Notes

- The API waits for DCIS to be set then reads the computed digest.

HMAC_Start

Function name

```
HAL_StatusTypeDef HMAC_Start (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size,
uint8_t * pOutBuffer, uint32_t Timeout, uint32_t Algorithm)
```

Function description

Initialize the HASH peripheral in HMAC mode, next process pInBuffer then read the computed digest.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest.
- **Timeout**: Timeout value.
- **Algorithm**: HASH algorithm.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HMAC_Start_IT

Function name

HAL_StatusTypeDef HMAC_Start_IT (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint8_t * pOutBuffer, uint32_t Algorithm)

Function description

Initialize the HASH peripheral in HMAC mode, next process pInBuffer then read the computed digest in interruption mode.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **pOutBuffer**: pointer to the computed digest.
- **Algorithm**: HASH algorithm.

Return values

- **HAL**: status

Notes

- Digest is available in pOutBuffer.
- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.

HMAC_Start_DMA

Function name

HAL_StatusTypeDef HMAC_Start_DMA (HASH_HandleTypeDef * hhash, uint8_t * pInBuffer, uint32_t Size, uint32_t Algorithm)

Function description

Initialize the HASH peripheral in HMAC mode then initiate the required DMA transfers to feed the key and the input buffer to the IP.

Parameters

- **hhash**: HASH handle.
- **pInBuffer**: pointer to the input buffer (buffer to be hashed).
- **Size**: length of the input buffer in bytes.
- **Algorithm**: HASH algorithm.

Return values

- **HAL**: status

Notes

- Same key is used for the inner and the outer hash functions; pointer to key and key size are respectively stored in hhash->Init.pKey and hhash->Init.KeySize.
- In case of multi-buffer HMAC processing, the input buffer size (in bytes) must be a multiple of 4 otherwise, the HASH digest computation is corrupted. Only the length of the last buffer of the thread doesn't have to be a multiple of 4.

23.3 HASH Firmware driver defines

The following section lists the various define and macros of the module.

23.3.1 HASH

HASH

HASH algorithm mode

HASH_ALGOMODE_HASH

Algorithm is HASH

HASH_ALGOMODE_HMAC

Algorithm is HMAC

HASH algorithm selection

HASH_ALGOSELECTION_SHA1

HASH function is SHA1

HASH_ALGOSELECTION_MD5

HASH function is MD5

HASH input data type

HASH_DATATYPE_32B

32-bit data. No swapping

HASH_DATATYPE_16B

16-bit data. Each half word is swapped

HASH_DATATYPE_8B

8-bit data. All bytes are swapped

HASH_DATATYPE_1B

1-bit data. In the word all bits are swapped

HASH Digest Calculation Status

HASH_DIGEST_CALCULATION_NOT_STARTED

DCAL not set after input data written in DIN register

HASH_DIGEST_CALCULATION_STARTED

DCAL set after input data written in DIN register

HASH DMA suspension words limit

HASH_DMA_SUSPENSION_WORDS_LIMIT

Number of words below which DMA suspension is aborted

HASH Error Definition

HAL_HASH_ERROR_NONE

No error

HAL_HASH_ERROR_IT

IT-based process error

HAL_HASH_ERROR_DMA

DMA-based process error

HASH Exported Macros

__HAL_HASH_GET_FLAG

Description:

- Check whether or not the specified HASH flag is set.

Parameters:

- **__FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - **HASH_FLAG_DINIS** A new block can be entered into the input buffer.
 - **HASH_FLAG_DCIS** Digest calculation complete.
 - **HASH_FLAG_DMAS** DMA interface is enabled (DMAE=1) or a transfer is ongoing.
 - **HASH_FLAG_BUSY** The hash core is Busy : processing a block of data.
 - **HASH_FLAG_DINNE** DIN not empty : the input buffer contains at least one word of data.

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

__HAL_HASH_CLEAR_FLAG

Description:

- Clear the specified HASH flag.

Parameters:

- **__FLAG__**: specifies the flag to clear. This parameter can be one of the following values:
 - **HASH_FLAG_DINIS** A new block can be entered into the input buffer.
 - **HASH_FLAG_DCIS** Digest calculation complete

Return value:

- None

__HAL_HASH_ENABLE_IT

Description:

- Enable the specified HASH interrupt.

Parameters:

- **__INTERRUPT__**: specifies the HASH interrupt source to enable. This parameter can be one of the following values:
 - **HASH_IT_DINI** A new block can be entered into the input buffer (DIN)
 - **HASH_IT_DCI** Digest calculation complete

Return value:

- None

__HAL_HASH_DISABLE_IT

Description:

- Disable the specified HASH interrupt.

Parameters:

- `__INTERRUPT__`: specifies the HASH interrupt source to disable. This parameter can be one of the following values:
 - `HASH_IT_DINI` A new block can be entered into the input buffer (DIN)
 - `HASH_IT_DCI` Digest calculation complete

Return value:

- None

__HAL_HASH_RESET_HANDLE_STATE

Description:

- Reset HASH handle state.

Parameters:

- `__HANDLE__`: HASH handle.

Return value:

- None

__HAL_HASH_RESET_HANDLE_STATUS

Description:

- Reset HASH handle status.

Parameters:

- `__HANDLE__`: HASH handle.

Return value:

- None

__HAL_HASH_START_DIGEST

Description:

- Start the digest computation.

Return value:

- None

__HAL_HASH_SET_NBVALIDBITS

Description:

- Set the number of valid bits in the last word written in data register DIN.

Parameters:

- `__SIZE__`: size in bytes of last data written in Data register.

Return value:

- None

__HAL_HASH_INIT

Description:

- Reset the HASH core.

Return value:

- None

HASH flags definitions

HASH_FLAG_DINIS

16 locations are free in the DIN : a new block can be entered in the IP

HASH_FLAG_DCIS

Digest calculation complete

HASH_FLAG_DMAS

DMA interface is enabled (DMAE=1) or a transfer is ongoing

HASH_FLAG_BUSY

The hash core is Busy, processing a block of data

HASH_FLAG_DINNE

DIN not empty : the input buffer contains at least one word of data

HMAC key length type

HASH_HMAC_KEYTYPE_SHORTKEY

HMAC Key size is <= 64 bytes

HASH_HMAC_KEYTYPE_LONGKEY

HMAC Key size is > 64 bytes

HASH interrupts definitions

HASH_IT_DINI

A new block can be entered into the input buffer (DIN)

HASH_IT_DCI

Digest calculation complete

HASH Number of Context Swap Registers

HASH_NUMBER_OF_CSR_REGISTERS

Number of Context Swap Registers

HASH TimeOut Value

HASH_TIMEOUTVALUE

Time-out value

24 HAL HCD Generic Driver

24.1 HCD Firmware driver registers structures

24.1.1 HCD_HandleTypeDef

HCD_HandleTypeDef is defined in the stm32f2xx_hal_hcd.h

Data Fields

- *HCD_TypeDef * Instance*
- *HCD_InitTypeDef Init*
- *HCD_HCTypeDef hc*
- *HAL_LockTypeDef Lock*
- *__IO HCD_StateTypeDef State*
- *__IO uint32_t ErrorCode*
- *void * pData*

Field Documentation

- *HCD_TypeDef* HCD_HandleTypeDef::Instance*
Register base address
- *HCD_InitTypeDef HCD_HandleTypeDef::Init*
HCD required parameters
- *HCD_HCTypeDef HCD_HandleTypeDef::hc[16]*
Host channels parameters
- *HAL_LockTypeDef HCD_HandleTypeDef::Lock*
HCD peripheral status
- *__IO HCD_StateTypeDef HCD_HandleTypeDef::State*
HCD communication state
- *__IO uint32_t HCD_HandleTypeDef::ErrorCode*
HCD Error code
- *void* HCD_HandleTypeDef::pData*
Pointer Stack Handler

24.2 HCD Firmware driver API description

The following section lists the various functions of the HCD library.

24.2.1 How to use this driver

1. Declare a HCD_HandleTypeDef handle structure, for example: HCD_HandleTypeDef hhcd;
2. Fill parameters of Init structure in HCD handle
3. Call HAL_HCD_Init() API to initialize the HCD peripheral (Core, Host core, ...)
4. Initialize the HCD low level resources through the HAL_HCD_MspInit() API:
 - a. Enable the HCD/USB Low Level interface clock using the following macros
 - `__HAL_RCC_USB_OTG_FS_CLK_ENABLE();`
 - `__HAL_RCC_USB_OTG_HS_CLK_ENABLE();` (For High Speed Mode)
 - `__HAL_RCC_USB_OTG_HS_ULPI_CLK_ENABLE();` (For High Speed Mode)
 - b. Initialize the related GPIO clocks
 - c. Configure HCD pin-out
 - d. Configure HCD NVIC interrupt
5. Associate the Upper USB Host stack to the HAL HCD Driver:
 - a. `hhcd.pData = phost;`

6. Enable HCD transmission and reception:
 - a. `HAL_HCD_Start()`;

24.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

This section contains the following APIs:

- `HAL_HCD_Init`
- `HAL_HCD_HC_Init`
- `HAL_HCD_HC_Halt`
- `HAL_HCD_DeInit`
- `HAL_HCD_MspInit`
- `HAL_HCD_MspDeInit`

24.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the USB Host Data Transfer

This section contains the following APIs:

- `HAL_HCD_HC_SubmitRequest`
- `HAL_HCD_IRQHandler`
- `HAL_HCD_SOF_Callback`
- `HAL_HCD_Connect_Callback`
- `HAL_HCD_Disconnect_Callback`
- `HAL_HCD_PortEnabled_Callback`
- `HAL_HCD_PortDisabled_Callback`
- `HAL_HCD_HC_NotifyURBChange_Callback`

24.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the HCD data transfers.

This section contains the following APIs:

- `HAL_HCD_Start`
- `HAL_HCD_Stop`
- `HAL_HCD_ResetPort`

24.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- `HAL_HCD_GetState`
- `HAL_HCD_HC_GetURBState`
- `HAL_HCD_HC_GetXferCount`
- `HAL_HCD_HC_GetState`
- `HAL_HCD_GetCurrentFrame`
- `HAL_HCD_GetCurrentSpeed`

24.2.6 Detailed description of functions

HAL_HCD_Init

Function name

`HAL_StatusTypeDef HAL_HCD_Init (HCD_HandleTypeDef * hhcd)`

Function description

Initialize the host driver.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_DeInit

Function name

HAL_StatusTypeDef HAL_HCD_DeInit (HCD_HandleTypeDef * hhcd)

Function description

DeInitialize the host driver.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL**: status

HAL_HCD_HC_Init

Function name

HAL_StatusTypeDef HAL_HCD_HC_Init (HCD_HandleTypeDef * hhcd, uint8_t ch_num, uint8_t epnum, uint8_t dev_address, uint8_t speed, uint8_t ep_type, uint16_t mps)

Function description

Initialize a host channel.

Parameters

- **hhcd**: HCD handle
- **ch_num**: Channel number. This parameter can be a value from 1 to 15
- **epnum**: Endpoint number. This parameter can be a value from 1 to 15
- **dev_address**: Current device address This parameter can be a value from 0 to 255
- **speed**: Current device speed. This parameter can be one of these values: HCD_SPEED_HIGH: High speed mode, HCD_SPEED_FULL: Full speed mode, HCD_SPEED_LOW: Low speed mode
- **ep_type**: Endpoint Type. This parameter can be one of these values: EP_TYPE_CTRL: Control type, EP_TYPE_ISOC: Isochronous type, EP_TYPE_BULK: Bulk type, EP_TYPE_INTR: Interrupt type
- **mps**: Max Packet Size. This parameter can be a value from 0 to 32K

Return values

- **HAL**: status

HAL_HCD_HC_Halt

Function name

HAL_StatusTypeDef HAL_HCD_HC_Halt (HCD_HandleTypeDef * hhcd, uint8_t ch_num)

Function description

Halt a host channel.

Parameters

- **hhcd**: HCD handle
- **ch_num**: Channel number. This parameter can be a value from 1 to 15

Return values

- **HAL:** status

HAL_HCD_MspInit

Function name

```
void HAL_HCD_MspInit (HCD_HandleTypeDef * hhcd)
```

Function description

Initialize the HCD MSP.

Parameters

- **hhcd:** HCD handle

Return values

- **None:**

HAL_HCD_MspDeInit

Function name

```
void HAL_HCD_MspDeInit (HCD_HandleTypeDef * hhcd)
```

Function description

DeInitialize the HCD MSP.

Parameters

- **hhcd:** HCD handle

Return values

- **None:**

HAL_HCD_HC_SubmitRequest

Function name

```
HAL_StatusTypeDef HAL_HCD_HC_SubmitRequest (HCD_HandleTypeDef * hhcd, uint8_t ch_num,
uint8_t direction, uint8_t ep_type, uint8_t token, uint8_t * pbuff, uint16_t length, uint8_t do_ping)
```

Function description

Submit a new URB for processing.

Parameters

- **hhcd:** HCD handle
- **ch_num:** Channel number. This parameter can be a value from 1 to 15
- **direction:** Channel number. This parameter can be one of these values: 0 : Output / 1 : Input
- **ep_type:** Endpoint Type. This parameter can be one of these values: EP_TYPE_CTRL: Control type/ EP_TYPE_ISOC: Isochronous type/ EP_TYPE_BULK: Bulk type/ EP_TYPE_INTR: Interrupt type/
- **token:** Endpoint Type. This parameter can be one of these values: 0: HC_PID_SETUP / 1: HC_PID_DATA1
- **pbuff:** pointer to URB data
- **length:** Length of URB data
- **do_ping:** activate do ping protocol (for high speed only). This parameter can be one of these values: 0 : do ping inactive / 1 : do ping active

Return values

- **HAL:** status

HAL_HCD_IRQHandler

Function name

void HAL_HCD_IRQHandler (HCD_HandleTypeDef * hhcd)

Function description

Handle HCD interrupt request.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_SOF_Callback

Function name

void HAL_HCD_SOF_Callback (HCD_HandleTypeDef * hhcd)

Function description

SOF callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_Connect_Callback

Function name

void HAL_HCD_Connect_Callback (HCD_HandleTypeDef * hhcd)

Function description

Connection Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_Disconnect_Callback

Function name

void HAL_HCD_Disconnect_Callback (HCD_HandleTypeDef * hhcd)

Function description

Disconnection Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_PortEnabled_Callback

Function name

```
void HAL_HCD_PortEnabled_Callback (HCD_HandleTypeDef * hhcd)
```

Function description

Port Enabled Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_PortDisabled_Callback

Function name

```
void HAL_HCD_PortDisabled_Callback (HCD_HandleTypeDef * hhcd)
```

Function description

Port Disabled Event callback.

Parameters

- **hhcd**: HCD handle

Return values

- **None**:

HAL_HCD_HC_NotifyURBChange_Callback

Function name

```
void HAL_HCD_HC_NotifyURBChange_Callback (HCD_HandleTypeDef * hhcd, uint8_t chnum,  
HCD_URBStateTypeDef urb_state)
```

Function description

Notify URB state change callback.

Parameters

- **hhcd**: HCD handle
- **chnum**: Channel number. This parameter can be a value from 1 to 15
- **urb_state**: This parameter can be one of these values: URB_IDLE/ URB_DONE/ URB_NOTREADY/ URB_NYET/ URB_ERROR/ URB_STALL/

Return values

- **None**:

HAL_HCD_ResetPort

Function name

```
HAL_StatusTypeDef HAL_HCD_ResetPort (HCD_HandleTypeDef * hhcd)
```

Function description

Reset the host port.

Parameters

- **hhcd**: HCD handle

Return values

- **HAL:** status

HAL_HCD_Start

Function name

HAL_StatusTypeDef HAL_HCD_Start (HCD_HandleTypeDef * hhcd)

Function description

Start the host driver.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** status

HAL_HCD_Stop

Function name

HAL_StatusTypeDef HAL_HCD_Stop (HCD_HandleTypeDef * hhcd)

Function description

Stop the host driver.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** status

HAL_HCD_GetState

Function name

HCD_StateTypeDef HAL_HCD_GetState (HCD_HandleTypeDef * hhcd)

Function description

Return the HCD handle state.

Parameters

- **hhcd:** HCD handle

Return values

- **HAL:** state

HAL_HCD_HC_GetURBState

Function name

HCD_URBStateTypeDef HAL_HCD_HC_GetURBState (HCD_HandleTypeDef * hhcd, uint8_t chnum)

Function description

Return URB state for a channel.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **URB:** state. This parameter can be one of these values: URB_IDLE/ URB_DONE/ URB_NOTREADY/ URB_NYET/ URB_ERROR/ URB_STALL

HAL_HCD_HC_GetXferCount

Function name

```
uint32_t HAL_HCD_HC_GetXferCount (HCD_HandleTypeDef * hhcd, uint8_t chnum)
```

Function description

Return the last host transfer size.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **last:** transfer size in byte

HAL_HCD_HC_GetState

Function name

```
HCD_HCStateTypeDef HAL_HCD_HC_GetState (HCD_HandleTypeDef * hhcd, uint8_t chnum)
```

Function description

Return the Host Channel state.

Parameters

- **hhcd:** HCD handle
- **chnum:** Channel number. This parameter can be a value from 1 to 15

Return values

- **Host:** channel state This parameter can be one of these values: HC_IDLE/ HC_XFRC/ HC_HALTED/ HC_NYET/ HC_NAK/ HC_STALL/ HC_XACTERR/ HC_BBLERR/ HC_DATATGLERR

HAL_HCD_GetCurrentFrame

Function name

```
uint32_t HAL_HCD_GetCurrentFrame (HCD_HandleTypeDef * hhcd)
```

Function description

Return the current Host frame number.

Parameters

- **hhcd:** HCD handle

Return values

- **Current:** Host frame number

HAL_HCD_GetCurrentSpeed

Function name

```
uint32_t HAL_HCD_GetCurrentSpeed (HCD_HandleTypeDef * hhcd)
```

Function description

Return the Host enumeration speed.

Parameters

- **hhcd**: HCD handle

Return values

- **Enumeration**: speed

24.3 HCD Firmware driver defines

The following section lists the various define and macros of the module.

24.3.1 HCD

HCD

HCD Exported Macros

`__HAL_HCD_ENABLE`

`__HAL_HCD_DISABLE`

`__HAL_HCD_GET_FLAG`

`__HAL_HCD_CLEAR_FLAG`

`__HAL_HCD_IS_INVALID_INTERRUPT`

`__HAL_HCD_CLEAR_HC_INT`

`__HAL_HCD_MASK_HALT_HC_INT`

`__HAL_HCD_UNMASK_HALT_HC_INT`

`__HAL_HCD_MASK_ACK_HC_INT`

`__HAL_HCD_UNMASK_ACK_HC_INT`

HCD PHY Module

`HCD_PHY_ULPI`

`HCD_PHY_EMBEDDED`

HCD Speed

`HCD_SPEED_HIGH`

`HCD_SPEED_FULL`

`HCD_SPEED_LOW`

25 HAL I2C Generic Driver

25.1 I2C Firmware driver registers structures

25.1.1 I2C_InitTypeDef

I2C_InitTypeDef is defined in the `stm32f2xx_hal_i2c.h`

Data Fields

- *uint32_t* **ClockSpeed**
- *uint32_t* **DutyCycle**
- *uint32_t* **OwnAddress1**
- *uint32_t* **AddressingMode**
- *uint32_t* **DualAddressMode**
- *uint32_t* **OwnAddress2**
- *uint32_t* **GeneralCallMode**
- *uint32_t* **NoStretchMode**

Field Documentation

- *uint32_t* **I2C_InitTypeDef::ClockSpeed**
Specifies the clock frequency. This parameter must be set to a value lower than 400kHz
- *uint32_t* **I2C_InitTypeDef::DutyCycle**
Specifies the I2C fast mode duty cycle. This parameter can be a value of [I2C_duty_cycle_in_fast_mode](#)
- *uint32_t* **I2C_InitTypeDef::OwnAddress1**
Specifies the first device own address. This parameter can be a 7-bit or 10-bit address.
- *uint32_t* **I2C_InitTypeDef::AddressingMode**
Specifies if 7-bit or 10-bit addressing mode is selected. This parameter can be a value of [I2C_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::DualAddressMode**
Specifies if dual addressing mode is selected. This parameter can be a value of [I2C_dual_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::OwnAddress2**
Specifies the second device own address if dual addressing mode is selected. This parameter can be a 7-bit address.
- *uint32_t* **I2C_InitTypeDef::GeneralCallMode**
Specifies if general call mode is selected. This parameter can be a value of [I2C_general_call_addressing_mode](#)
- *uint32_t* **I2C_InitTypeDef::NoStretchMode**
Specifies if nostretch mode is selected. This parameter can be a value of [I2C_nostretch_mode](#)

25.1.2 __I2C_HandleTypeDef

__I2C_HandleTypeDef is defined in the `stm32f2xx_hal_i2c.h`

Data Fields

- *I2C_TypeDef ** **Instance**
- *I2C_InitTypeDef* **Init**
- *uint8_t ** **pBuffPtr**
- *uint16_t* **XferSize**
- *__IO uint16_t* **XferCount**
- *__IO uint32_t* **XferOptions**
- *__IO uint32_t* **PreviousState**
- *DMA_HandleTypeDef ** **hdmatx**

- ***DMA_HandleTypeDef * hdmarx***
- ***HAL_LockTypeDef Lock***
- ***__IO HAL_I2C_StateTypeDef State***
- ***__IO HAL_I2C_ModeTypeDef Mode***
- ***__IO uint32_t ErrorCode***
- ***__IO uint32_t Devaddress***
- ***__IO uint32_t Memaddress***
- ***__IO uint32_t MemaddSize***
- ***__IO uint32_t EventCount***

Field Documentation

- ***I2C_TypeDef* __I2C_HandleTypeDef::Instance***
I2C registers base address
- ***I2C_InitTypeDef __I2C_HandleTypeDef::Init***
I2C communication parameters
- ***uint8_t* __I2C_HandleTypeDef::pBuffPtr***
Pointer to I2C transfer buffer
- ***uint16_t __I2C_HandleTypeDef::XferSize***
I2C transfer size
- ***__IO uint16_t __I2C_HandleTypeDef::XferCount***
I2C transfer counter
- ***__IO uint32_t __I2C_HandleTypeDef::XferOptions***
I2C transfer options
- ***__IO uint32_t __I2C_HandleTypeDef::PreviousState***
I2C communication Previous state and mode context for internal usage
- ***DMA_HandleTypeDef* __I2C_HandleTypeDef::hdmatx***
I2C Tx DMA handle parameters
- ***DMA_HandleTypeDef* __I2C_HandleTypeDef::hdmarx***
I2C Rx DMA handle parameters
- ***HAL_LockTypeDef __I2C_HandleTypeDef::Lock***
I2C locking object
- ***__IO HAL_I2C_StateTypeDef __I2C_HandleTypeDef::State***
I2C communication state
- ***__IO HAL_I2C_ModeTypeDef __I2C_HandleTypeDef::Mode***
I2C communication mode
- ***__IO uint32_t __I2C_HandleTypeDef::ErrorCode***
I2C Error code
- ***__IO uint32_t __I2C_HandleTypeDef::Devaddress***
I2C Target device address
- ***__IO uint32_t __I2C_HandleTypeDef::Memaddress***
I2C Target memory address
- ***__IO uint32_t __I2C_HandleTypeDef::MemaddSize***
I2C Target memory address size
- ***__IO uint32_t __I2C_HandleTypeDef::EventCount***
I2C Event counter

25.2 I2C Firmware driver API description

The following section lists the various functions of the I2C library.

25.2.1 How to use this driver

The I2C HAL driver can be used as follows:

1. Declare a I2C_HandleTypeDef handle structure, for example: I2C_HandleTypeDef hi2c;
2. Initialize the I2C low level resources by implementing the @ref HAL_I2C_MspInit() API:
 - a. Enable the I2Cx interface clock
 - b. I2C pins configuration
 - Enable the clock for the I2C GPIOs
 - Configure I2C pins as alternate function open-drain
 - c. NVIC configuration if you need to use interrupt process
 - Configure the I2Cx interrupt priority
 - Enable the NVIC I2C IRQ Channel
 - d. DMA Configuration if you need to use DMA process
 - Declare a DMA_HandleTypeDef handle structure for the transmit or receive stream
 - Enable the DMAx interface clock using
 - Configure the DMA handle parameters
 - Configure the DMA Tx or Rx stream
 - Associate the initialized DMA handle to the hi2c DMA Tx or Rx handle
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx or Rx stream
3. Configure the Communication Speed, Duty cycle, Addressing mode, Own Address1, Dual Addressing mode, Own Address2, General call and Nostretch mode in the hi2c Init structure.
4. Initialize the I2C registers by calling the @ref HAL_I2C_Init(), configures also the low level Hardware (GPIO, CLOCK, NVIC...etc) by calling the customized @ref HAL_I2C_MspInit() API.
5. To check if target device is ready for communication, use the function @ref HAL_I2C_IsDeviceReady()
6. For I2C IO and IO MEM operations, three operation modes are available within this driver :

Polling mode IO operation

- Transmit in master mode an amount of data in blocking mode using @ref HAL_I2C_Master_Transmit()
- Receive in master mode an amount of data in blocking mode using @ref HAL_I2C_Master_Receive()
- Transmit in slave mode an amount of data in blocking mode using @ref HAL_I2C_Slave_Transmit()
- Receive in slave mode an amount of data in blocking mode using @ref HAL_I2C_Slave_Receive()

Polling mode IO MEM operation

- Write an amount of data in blocking mode to a specific memory address using @ref HAL_I2C_Mem_Write()
- Read an amount of data in blocking mode from a specific memory address using @ref HAL_I2C_Mem_Read()

Interrupt mode IO operation

- Transmit in master mode an amount of data in non-blocking mode using @ref HAL_I2C_Master_Transmit_IT()
- At transmission end of transfer, @ref HAL_I2C_MasterTxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterTxCpltCallback()
- Receive in master mode an amount of data in non-blocking mode using @ref HAL_I2C_Master_Receive_IT()
- At reception end of transfer, @ref HAL_I2C_MasterRxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterRxCpltCallback()
- Transmit in slave mode an amount of data in non-blocking mode using @ref HAL_I2C_Slave_Transmit_IT()
- At transmission end of transfer, @ref HAL_I2C_SlaveTxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveTxCpltCallback()
- Receive in slave mode an amount of data in non-blocking mode using @ref HAL_I2C_Slave_Receive_IT()
- At reception end of transfer, @ref HAL_I2C_SlaveRxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveRxCpltCallback()

- In case of transfer Error, @ref HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ErrorCallback()
- Abort a master I2C process communication with Interrupt using @ref HAL_I2C_Master_Abort_IT()
- End of abort process, @ref HAL_I2C_AbortCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_AbortCpltCallback()

Interrupt mode or DMA mode IO sequential operation

Note: *These interfaces allow to manage a sequential transfer with a repeated start condition when a direction change during transfer*

- A specific option field manage the different steps of a sequential transfer
- Option field values are defined through @ref I2C_XferOptions_definition and are listed below:
 - I2C_FIRST_AND_LAST_FRAME: No sequential usage, functional is same as associated interfaces in no sequential mode
 - I2C_FIRST_FRAME: Sequential usage, this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition
 - I2C_FIRST_AND_NEXT_FRAME: Sequential usage (Master only), this option allow to manage a sequence with start condition, address and data to transfer without a final stop condition, an then permit a call the same master sequential interface several times (like @ref HAL_I2C_Master_Seq_Transmit_IT() then @ref HAL_I2C_Master_Seq_Transmit_IT() or @ref HAL_I2C_Master_Seq_Transmit_DMA() then @ref HAL_I2C_Master_Seq_Transmit_DMA())
 - I2C_NEXT_FRAME: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and without a final stop condition in both cases
 - I2C_LAST_FRAME: Sequential usage, this option allow to manage a sequence with a restart condition, address and with new data to transfer if the direction change or manage only the new data to transfer if no direction change and with a final stop condition in both cases
 - I2C_LAST_FRAME_NO_STOP: Sequential usage (Master only), this option allow to manage a restart condition after several call of the same master sequential interface several times (link with option I2C_FIRST_AND_NEXT_FRAME). Usage can, transfer several bytes one by one using HAL_I2C_Master_Seq_Transmit_IT(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Receive_IT(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Transmit_DMA(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME) or HAL_I2C_Master_Seq_Receive_DMA(option I2C_FIRST_AND_NEXT_FRAME then I2C_NEXT_FRAME). Then usage of this option I2C_LAST_FRAME_NO_STOP at the last Transmit or Receive sequence permit to call the oposite interface Receive or Transmit without stopping the communication and so generate a restart condition.
 - I2C_OTHER_FRAME: Sequential usage (Master only), this option allow to manage a restart condition after each call of the same master sequential interface. Usage can, transfer several bytes one by one with a restart with slave address between each bytes using HAL_I2C_Master_Seq_Transmit_IT(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Receive_IT(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Transmit_DMA(option I2C_FIRST_FRAME then I2C_OTHER_FRAME) or HAL_I2C_Master_Seq_Receive_DMA(option I2C_FIRST_FRAME then I2C_OTHER_FRAME). Then usage of this option I2C_OTHER_AND_LAST_FRAME at the last frame to help automatic generation of STOP condition.

- Differents sequential I2C interfaces are listed below:
 - Sequential transmit in master I2C mode an amount of data in non-blocking mode using @ref HAL_I2C_Master_Seq_Transmit_IT() or using @ref HAL_I2C_Master_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, @ref HAL_I2C_MasterTxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterTxCallback()
 - Sequential receive in master I2C mode an amount of data in non-blocking mode using @ref HAL_I2C_Master_Seq_Receive_IT() or using @ref HAL_I2C_Master_Seq_Receive_DMA()
 - At reception end of current frame transfer, @ref HAL_I2C_MasterRxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterRxCallback()
 - Abort a master IT or DMA I2C process communication with Interrupt using @ref HAL_I2C_Master_Abort_IT()
 - End of abort process, @ref HAL_I2C_AbortCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_AbortCallback()
 - Enable/disable the Address listen mode in slave I2C mode using @ref HAL_I2C_EnableListen_IT() @ref HAL_I2C_DisableListen_IT()
 - When address slave I2C match, @ref HAL_I2C_AddrCallback() is executed and user can add his own code to check the Address Match Code and the transmission direction request by master (Write/Read).
 - At Listen mode end @ref HAL_I2C_ListenCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ListenCallback()
 - Sequential transmit in slave I2C mode an amount of data in non-blocking mode using @ref HAL_I2C_Slave_Seq_Transmit_IT() or using @ref HAL_I2C_Slave_Seq_Transmit_DMA()
 - At transmission end of current frame transfer, @ref HAL_I2C_SlaveTxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveTxCallback()
 - Sequential receive in slave I2C mode an amount of data in non-blocking mode using @ref HAL_I2C_Slave_Seq_Receive_IT() or using @ref HAL_I2C_Slave_Seq_Receive_DMA()
 - At reception end of current frame transfer, @ref HAL_I2C_SlaveRxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveRxCallback()
 - In case of transfer Error, @ref HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ErrorCallback()

Interrupt mode IO MEM operation

- Write an amount of data in non-blocking mode with Interrupt to a specific memory address using @ref HAL_I2C_Mem_Write_IT()
- At Memory end of write transfer, @ref HAL_I2C_MemTxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MemTxCallback()
- Read an amount of data in non-blocking mode with Interrupt from a specific memory address using @ref HAL_I2C_Mem_Read_IT()
- At Memory end of read transfer, @ref HAL_I2C_MemRxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MemRxCallback()
- In case of transfer Error, @ref HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ErrorCallback()

DMA mode IO operation

- Transmit in master mode an amount of data in non-blocking mode (DMA) using @ref HAL_I2C_Master_Transmit_DMA()
- At transmission end of transfer, @ref HAL_I2C_MasterTxCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterTxCallback()
- Receive in master mode an amount of data in non-blocking mode (DMA) using @ref HAL_I2C_Master_Receive_DMA()

- At reception end of transfer, @ref HAL_I2C_MasterRxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MasterRxCpltCallback()
- Transmit in slave mode an amount of data in non-blocking mode (DMA) using @ref HAL_I2C_Slave_Transmit_DMA()
- At transmission end of transfer, @ref HAL_I2C_SlaveTxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveTxCpltCallback()
- Receive in slave mode an amount of data in non-blocking mode (DMA) using @ref HAL_I2C_Slave_Receive_DMA()
- At reception end of transfer, @ref HAL_I2C_SlaveRxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_SlaveRxCpltCallback()
- In case of transfer Error, @ref HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ErrorCallback()
- Abort a master I2C process communication with Interrupt using @ref HAL_I2C_Master_Abort_IT()
- End of abort process, @ref HAL_I2C_AbortCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_AbortCpltCallback()

DMA mode IO MEM operation

- Write an amount of data in non-blocking mode with DMA to a specific memory address using @ref HAL_I2C_Mem_Write_DMA()
- At Memory end of write transfer, @ref HAL_I2C_MemTxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MemTxCpltCallback()
- Read an amount of data in non-blocking mode with DMA from a specific memory address using @ref HAL_I2C_Mem_Read_DMA()
- At Memory end of read transfer, @ref HAL_I2C_MemRxCpltCallback() is executed and user can add his own code by customization of function pointer @ref HAL_I2C_MemRxCpltCallback()
- In case of transfer Error, @ref HAL_I2C_ErrorCallback() function is executed and user can add his own code by customization of function pointer @ref HAL_I2C_ErrorCallback()

I2C HAL driver macros list

Below the list of most used macros in I2C HAL driver.

- @ref __HAL_I2C_ENABLE: Enable the I2C peripheral
- @ref __HAL_I2C_DISABLE: Disable the I2C peripheral
- @ref __HAL_I2C_GET_FLAG: Checks whether the specified I2C flag is set or not
- @ref __HAL_I2C_CLEAR_FLAG: Clear the specified I2C pending flag
- @ref __HAL_I2C_ENABLE_IT: Enable the specified I2C interrupt
- @ref __HAL_I2C_DISABLE_IT: Disable the specified I2C interrupt

Callback registration

The compilation flag `USE_HAL_I2C_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions @ref HAL_I2C_RegisterCallback() or @ref HAL_I2C_RegisterAddrCallback() to register an interrupt callback.

Function @ref HAL_I2C_RegisterCallback() allows to register following callbacks:

- MasterTxCpltCallback : callback for Master transmission end of transfer.
- MasterRxCpltCallback : callback for Master reception end of transfer.
- SlaveTxCpltCallback : callback for Slave transmission end of transfer.
- SlaveRxCpltCallback : callback for Slave reception end of transfer.
- ListenCpltCallback : callback for end of listen mode.
- MemTxCpltCallback : callback for Memory transmission end of transfer.
- MemRxCpltCallback : callback for Memory reception end of transfer.
- ErrorCallback : callback for error detection.
- AbortCpltCallback : callback for abort completion process.
- MspInitCallback : callback for Msp Init.

- MspDeInitCallback : callback for Msp DeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

For specific callback AddrCallback use dedicated register callbacks : @ref HAL_I2C_RegisterAddrCallback().

Use function @ref HAL_I2C_UnRegisterCallback to reset a callback to the default weak function. @ref HAL_I2C_UnRegisterCallback takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- MasterTxCpltCallback : callback for Master transmission end of transfer.
- MasterRxCpltCallback : callback for Master reception end of transfer.
- SlaveTxCpltCallback : callback for Slave transmission end of transfer.
- SlaveRxCpltCallback : callback for Slave reception end of transfer.
- ListenCpltCallback : callback for end of listen mode.
- MemTxCpltCallback : callback for Memory transmission end of transfer.
- MemRxCpltCallback : callback for Memory reception end of transfer.
- ErrorCallback : callback for error detection.
- AbortCpltCallback : callback for abort completion process.
- MspInitCallback : callback for Msp Init.
- MspDeInitCallback : callback for Msp DeInit.

For callback AddrCallback use dedicated register callbacks : @ref HAL_I2C_UnRegisterAddrCallback().

By default, after the @ref HAL_I2C_Init() and when the state is @ref HAL_I2C_STATE_RESET all callbacks are set to the corresponding weak functions: examples @ref HAL_I2C_MasterTxCpltCallback(), @ref HAL_I2C_MasterRxCpltCallback(). Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functions in the @ref HAL_I2C_Init()/ @ref HAL_I2C_DeInit() only when these callbacks are null (not registered beforehand). If MspInit or MspDeInit are not null, the @ref HAL_I2C_Init()/ @ref HAL_I2C_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in @ref HAL_I2C_STATE_READY state only. Exception done MspInit/ MspDeInit functions that can be registered/unregistered in @ref HAL_I2C_STATE_READY or @ref HAL_I2C_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. Then, the user first registers the MspInit/MspDeInit user callbacks using @ref HAL_I2C_RegisterCallback() before calling @ref HAL_I2C_DeInit() or @ref HAL_I2C_Init() function.

When the compilation flag USE_HAL_I2C_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

Note: You can refer to the I2C HAL driver header file for more useful macros

25.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and deinitialize the I2Cx peripheral:

- User must implement HAL_I2C_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function HAL_I2C_Init() to configure the selected device with the selected configuration:
 - Communication Speed
 - Duty cycle
 - Addressing mode
 - Own Address 1
 - Dual Addressing mode
 - Own Address 2
 - General call mode
 - Nostretch mode
- Call the function HAL_I2C_DeInit() to restore the default configuration of the selected I2Cx peripheral.

This section contains the following APIs:

- [HAL_I2C_Init](#)
- [HAL_I2C_DeInit](#)
- [HAL_I2C_MspInit](#)

- [HAL_I2C_MspDeInit](#)

25.2.3

IO operation functions

This subsection provides a set of functions allowing to manage the I2C data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2C IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - HAL_I2C_Master_Transmit()
 - HAL_I2C_Master_Receive()
 - HAL_I2C_Slave_Transmit()
 - HAL_I2C_Slave_Receive()
 - HAL_I2C_Mem_Write()
 - HAL_I2C_Mem_Read()
 - HAL_I2C_IsDeviceReady()
3. No-Blocking mode functions with Interrupt are :
 - HAL_I2C_Master_Transmit_IT()
 - HAL_I2C_Master_Receive_IT()
 - HAL_I2C_Slave_Transmit_IT()
 - HAL_I2C_Slave_Receive_IT()
 - HAL_I2C_Mem_Write_IT()
 - HAL_I2C_Mem_Read_IT()
 - HAL_I2C_Master_Seq_Transmit_IT()
 - HAL_I2C_Master_Seq_Receive_IT()
 - HAL_I2C_Slave_Seq_Transmit_IT()
 - HAL_I2C_Slave_Seq_Receive_IT()
 - HAL_I2C_EnableListen_IT()
 - HAL_I2C_DisableListen_IT()
 - HAL_I2C_Master_Abort_IT()
4. No-Blocking mode functions with DMA are :
 - HAL_I2C_Master_Transmit_DMA()
 - HAL_I2C_Master_Receive_DMA()
 - HAL_I2C_Slave_Transmit_DMA()
 - HAL_I2C_Slave_Receive_DMA()
 - HAL_I2C_Mem_Write_DMA()
 - HAL_I2C_Mem_Read_DMA()
 - HAL_I2C_Master_Seq_Transmit_DMA()
 - HAL_I2C_Master_Seq_Receive_DMA()
 - HAL_I2C_Slave_Seq_Transmit_DMA()
 - HAL_I2C_Slave_Seq_Receive_DMA()

5. A set of Transfer Complete Callbacks are provided in non Blocking mode:

- HAL_I2C_MasterTxCpltCallback()
- HAL_I2C_MasterRxCpltCallback()
- HAL_I2C_SlaveTxCpltCallback()
- HAL_I2C_SlaveRxCpltCallback()
- HAL_I2C_MemTxCpltCallback()
- HAL_I2C_MemRxCpltCallback()
- HAL_I2C_AddrCallback()
- HAL_I2C_ListenCpltCallback()
- HAL_I2C_ErrorCallback()
- HAL_I2C_AbortCpltCallback()

This section contains the following APIs:

- [HAL_I2C_Master_Transmit](#)
- [HAL_I2C_Master_Receive](#)
- [HAL_I2C_Slave_Transmit](#)
- [HAL_I2C_Slave_Receive](#)
- [HAL_I2C_Master_Transmit_IT](#)
- [HAL_I2C_Master_Receive_IT](#)
- [HAL_I2C_Slave_Transmit_IT](#)
- [HAL_I2C_Slave_Receive_IT](#)
- [HAL_I2C_Master_Transmit_DMA](#)
- [HAL_I2C_Master_Receive_DMA](#)
- [HAL_I2C_Slave_Transmit_DMA](#)
- [HAL_I2C_Slave_Receive_DMA](#)
- [HAL_I2C_Mem_Write](#)
- [HAL_I2C_Mem_Read](#)
- [HAL_I2C_Mem_Write_IT](#)
- [HAL_I2C_Mem_Read_IT](#)
- [HAL_I2C_Mem_Write_DMA](#)
- [HAL_I2C_Mem_Read_DMA](#)
- [HAL_I2C_IsDeviceReady](#)
- [HAL_I2C_Master_Seq_Transmit_IT](#)
- [HAL_I2C_Master_Seq_Transmit_DMA](#)
- [HAL_I2C_Master_Seq_Receive_IT](#)
- [HAL_I2C_Master_Seq_Receive_DMA](#)
- [HAL_I2C_Slave_Seq_Transmit_IT](#)
- [HAL_I2C_Slave_Seq_Transmit_DMA](#)
- [HAL_I2C_Slave_Seq_Receive_IT](#)
- [HAL_I2C_Slave_Seq_Receive_DMA](#)
- [HAL_I2C_EnableListen_IT](#)
- [HAL_I2C_DisableListen_IT](#)
- [HAL_I2C_Master_Abort_IT](#)

25.2.4 Peripheral State, Mode and Error functions

This subsection permit to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [HAL_I2C_GetState](#)
- [HAL_I2C_GetMode](#)
- [HAL_I2C_GetError](#)

25.2.5 Detailed description of functions

HAL_I2C_Init

Function name

HAL_StatusTypeDef HAL_I2C_Init (I2C_HandleTypeDef * hi2c)

Function description

Initializes the I2C according to the specified parameters in the I2C_InitTypeDef and initialize the associated handle.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_DeInit

Function name

HAL_StatusTypeDef HAL_I2C_DeInit (I2C_HandleTypeDef * hi2c)

Function description

DeInitialize the I2C peripheral.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_MspInit

Function name

void HAL_I2C_MspInit (I2C_HandleTypeDef * hi2c)

Function description

Initialize the I2C MSP.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MspDeInit

Function name

void HAL_I2C_MspDeInit (I2C_HandleTypeDef * hi2c)

Function description

DeInitialize the I2C MSP.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_Master_Transmit

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in master mode an amount of data in blocking mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_Master_Receive

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receives in master mode an amount of data in blocking mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_Slave_Transmit

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Transmit (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in slave mode an amount of data in blocking mode.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_Slave_Receive

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Receive (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive in slave mode an amount of data in blocking mode.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_Mem_Write

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Write an amount of data in blocking mode to a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_I2C_Mem_Read

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Read an amount of data in blocking mode from a specific memory address.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress:** Internal memory address
- **MemAddSize:** Size of internal memory address
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_I2C_IsDeviceReady

Function name

HAL_StatusTypeDef HAL_I2C_IsDeviceReady (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint32_t Trials, uint32_t Timeout)

Function description

Checks if target device is ready for communication.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **Trials:** Number of trials
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- This function is used with Memory devices

HAL_I2C_Master_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Master_Receive_IT

Function name

```
HAL_StatusTypeDef HAL_I2C_Master_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress,
uint8_t * pData, uint16_t Size)
```

Function description

Receive in master mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Transmit_IT

Function name

```
HAL_StatusTypeDef HAL_I2C_Slave_Transmit_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t
Size)
```

Function description

Transmit in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Receive_IT

Function name

```
HAL_StatusTypeDef HAL_I2C_Slave_Receive_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t
Size)
```


Function description

Receive in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Mem_Write_IT

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with Interrupt to a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Mem_Read_IT

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Read an amount of data in non-blocking mode with Interrupt from a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Master_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Transmit_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master I2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Master_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Receive_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in master I2C mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Transmit_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Receive_IT (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in slave mode an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_EnableListen_IT

Function name

HAL_StatusTypeDef HAL_I2C_EnableListen_IT (I2C_HandleTypeDef * hi2c)

Function description

Enable the Address listen mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_DisableListen_IT

Function name

HAL_StatusTypeDef HAL_I2C_DisableListen_IT (I2C_HandleTypeDef * hi2c)

Function description

Disable the Address listen mode with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** status

HAL_I2C_Master_Abort_IT

Function name

HAL_StatusTypeDef HAL_I2C_Master_Abort_IT (I2C_HandleTypeDef * hi2c, uint16_t DevAddress)

Function description

Abort a master I2C IT or DMA process communication with Interrupt.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface

Return values

- **HAL:** status

HAL_I2C_Master_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Transmit in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Master_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Transmit in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Slave_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Receive_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size)

Function description

Receive in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_I2C_Mem_Write_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Write_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Write an amount of data in non-blocking mode with DMA to a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent

Return values

- **HAL**: status

HAL_I2C_Mem_Read_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Mem_Read_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint16_t MemAddress, uint16_t MemAddSize, uint8_t * pData, uint16_t Size)

Function description

Reads an amount of data in non-blocking mode with DMA from a specific memory address.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **MemAddress**: Internal memory address
- **MemAddSize**: Size of internal memory address
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be read

Return values

- **HAL**: status

HAL_I2C_Master_Seq_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in master I2C mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Master_Seq_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Seq_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Transmit_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential transmit in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_Slave_Seq_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Slave_Seq_Receive_DMA (I2C_HandleTypeDef * hi2c, uint8_t * pData, uint16_t Size, uint32_t XferOptions)

Function description

Sequential receive in slave mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **XferOptions:** Options of Transfer, value of I2C XferOptions definition

Return values

- **HAL:** status

Notes

- This interface allow to manage repeated start condition when a direction change during transfer

HAL_I2C_EV_IRQHandler

Function name

void HAL_I2C_EV_IRQHandler (I2C_HandleTypeDef * hi2c)

Function description

This function handles I2C event interrupt request.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_ER_IRQHandler

Function name

void HAL_I2C_ER_IRQHandler (I2C_HandleTypeDef * hi2c)

Function description

This function handles I2C error interrupt request.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MasterTxCpltCallback

Function name

void HAL_I2C_MasterTxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Master Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MasterRxCpltCallback

Function name

void HAL_I2C_MasterRxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Master Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_SlaveTxCpltCallback

Function name

void HAL_I2C_SlaveTxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Slave Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_SlaveRxCpltCallback

Function name

void HAL_I2C_SlaveRxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Slave Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_AddrCallback

Function name

void HAL_I2C_AddrCallback (I2C_HandleTypeDef * hi2c, uint8_t TransferDirection, uint16_t AddrMatchCode)

Function description

Slave Address Match callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **TransferDirection:** Master request Transfer Direction (Write/Read), value of I2C XferDirection definition
- **AddrMatchCode:** Address Match Code

Return values

- **None:**

HAL_I2C_ListenCpltCallback

Function name

void HAL_I2C_ListenCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Listen Complete callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MemTxCpltCallback

Function name

void HAL_I2C_MemTxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Memory Tx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_MemRxCpltCallback

Function name

void HAL_I2C_MemRxCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

Memory Rx Transfer completed callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_ErrorCallback

Function name

void HAL_I2C_ErrorCallback (I2C_HandleTypeDef * hi2c)

Function description

I2C error callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_AbortCpltCallback

Function name

void HAL_I2C_AbortCpltCallback (I2C_HandleTypeDef * hi2c)

Function description

I2C abort callback.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **None:**

HAL_I2C_GetState

Function name

HAL_I2C_StateTypeDef HAL_I2C_GetState (I2C_HandleTypeDef * hi2c)

Function description

Return the I2C handle state.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **HAL:** state

HAL_I2C_GetMode

Function name

HAL_I2C_ModeTypeDef HAL_I2C_GetMode (I2C_HandleTypeDef * hi2c)

Function description

Returns the I2C Master, Slave, Memory or no mode.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for I2C module

Return values

- **HAL:** mode

HAL_I2C_GetError

Function name

uint32_t HAL_I2C_GetError (I2C_HandleTypeDef * hi2c)

Function description

Return the I2C error code.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.

Return values

- **I2C:** Error Code

25.3 I2C Firmware driver defines

The following section lists the various define and macros of the module.

25.3.1 I2C

I2C

I2C addressing mode

I2C_ADDRESSINGMODE_7BIT

I2C_ADDRESSINGMODE_10BIT

I2C dual addressing mode

I2C_DUALADDRESS_DISABLE

I2C_DUALADDRESS_ENABLE

I2C duty cycle in fast mode

I2C_DUTYCYCLE_2

I2C_DUTYCYCLE_16_9

I2C Error Code definition

HAL_I2C_ERROR_NONE

No error

HAL_I2C_ERROR_BERR

BERR error

HAL_I2C_ERROR_ARLO

ARLO error

HAL_I2C_ERROR_AF

AF error

HAL_I2C_ERROR_OVR

OVR error

HAL_I2C_ERROR_DMA

DMA transfer error

HAL_I2C_ERROR_TIMEOUT

Timeout Error

HAL_I2C_ERROR_SIZE

Size Management error

HAL_I2C_ERROR_DMA_PARAM

DMA Parameter Error

I2C Exported Macros

__HAL_I2C_RESET_HANDLE_STATE

Description:

- Reset I2C handle state.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_ENABLE_IT

Description:

- Enable or disable the specified I2C interrupts.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - `I2C_IT_BUF`: Buffer interrupt enable
 - `I2C_IT_EVT`: Event interrupt enable
 - `I2C_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_I2C_DISABLE_IT

__HAL_I2C_GET_IT_SOURCE

Description:

- Checks if the specified I2C interrupt source is enabled or disabled.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__INTERRUPT__`: specifies the I2C interrupt source to check. This parameter can be one of the following values:
 - `I2C_IT_BUF`: Buffer interrupt enable
 - `I2C_IT_EVT`: Event interrupt enable
 - `I2C_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__INTERRUPT__` (TRUE or FALSE).

__HAL_I2C_GET_FLAG

Description:

- Checks whether the specified I2C flag is set or not.

Parameters:

- `__HANDLE__`: specifies the I2C Handle.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `I2C_FLAG_OVR`: Overrun/Underrun flag
 - `I2C_FLAG_AF`: Acknowledge failure flag
 - `I2C_FLAG_ARLO`: Arbitration lost flag
 - `I2C_FLAG_BERR`: Bus error flag
 - `I2C_FLAG_TXE`: Data register empty flag
 - `I2C_FLAG_RXNE`: Data register not empty flag
 - `I2C_FLAG_STOPF`: Stop detection flag
 - `I2C_FLAG_ADD10`: 10-bit header sent flag
 - `I2C_FLAG_BTF`: Byte transfer finished flag
 - `I2C_FLAG_ADDR`: Address sent flag Address matched flag
 - `I2C_FLAG_SB`: Start bit flag
 - `I2C_FLAG_DUALF`: Dual flag
 - `I2C_FLAG_GENCALL`: General call header flag
 - `I2C_FLAG_TRA`: Transmitter/Receiver flag
 - `I2C_FLAG_BUSY`: Bus busy flag
 - `I2C_FLAG_MSL`: Master/Slave flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_I2C_CLEAR_FLAG

Description:

- Clears the I2C pending flags which are cleared by writing 0 in a specific bit.

Parameters:

- __HANDLE__: specifies the I2C Handle.
- __FLAG__: specifies the flag to clear. This parameter can be any combination of the following values:
 - I2C_FLAG_OVR: Overrun/Underrun flag (Slave mode)
 - I2C_FLAG_AF: Acknowledge failure flag
 - I2C_FLAG_ARLO: Arbitration lost flag (Master mode)
 - I2C_FLAG_BERR: Bus error flag

Return value:

- None

__HAL_I2C_CLEAR_ADDRFLAG

Description:

- Clears the I2C ADDR pending flag.

Parameters:

- __HANDLE__: specifies the I2C Handle. This parameter can be I2C where x: 1, 2, or 3 to select the I2C peripheral.

Return value:

- None

__HAL_I2C_CLEAR_STOPFLAG

Description:

- Clears the I2C STOPF pending flag.

Parameters:

- __HANDLE__: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_ENABLE

Description:

- Enable the specified I2C peripheral.

Parameters:

- __HANDLE__: specifies the I2C Handle.

Return value:

- None

__HAL_I2C_DISABLE

Description:

- Disable the specified I2C peripheral.

Parameters:

- __HANDLE__: specifies the I2C Handle.

Return value:

- None

I2C Flag definition

I2C_FLAG_OVR

I2C_FLAG_AF

I2C_FLAG_ARLO

I2C_FLAG_BERR

I2C_FLAG_TXE

I2C_FLAG_RXNE

I2C_FLAG_STOPF

I2C_FLAG_ADD10

I2C_FLAG_BTF

I2C_FLAG_ADDR

I2C_FLAG_SB

I2C_FLAG_DUALF

I2C_FLAG_GENCALL

I2C_FLAG_TRA

I2C_FLAG_BUSY

I2C_FLAG_MSL

I2C general call addressing mode

I2C_GENERALCALL_DISABLE

I2C_GENERALCALL_ENABLE

I2C Interrupt configuration definition

I2C_IT_BUF

I2C_IT_EVT

I2C_IT_ERR

I2C Private macros to check input parameters

IS_I2C_DUTY_CYCLE

IS_I2C_ADDRESSING_MODE

IS_I2C_DUAL_ADDRESS

IS_I2C_GENERAL_CALL

IS_I2C_NO_STRETCH

IS_I2C_MEMADD_SIZE

IS_I2C_CLOCK_SPEED

IS_I2C_OWN_ADDRESS1

IS_I2C_OWN_ADDRESS2

IS_I2C_TRANSFER_OPTIONS_REQUEST

IS_I2C_TRANSFER_OTHER_OPTIONS_REQUEST

I2C_CHECK_FLAG

I2C_CHECK_IT_SOURCE

I2C Memory Address Size

I2C_MEMADD_SIZE_8BIT

I2C_MEMADD_SIZE_16BIT

I2C nostretch mode

I2C_NOSTRETCH_DISABLE

I2C_NOSTRETCH_ENABLE

I2C XferDirection definition

I2C_DIRECTION_RECEIVE

I2C_DIRECTION_TRANSMIT

I2C XferOptions definition

I2C_FIRST_FRAME

I2C_FIRST_AND_NEXT_FRAME

I2C_NEXT_FRAME

I2C_FIRST_AND_LAST_FRAME

I2C_LAST_FRAME_NO_STOP

I2C_LAST_FRAME

I2C_OTHER_FRAME

I2C_OTHER_AND_LAST_FRAME

26 HAL I2S Generic Driver

26.1 I2S Firmware driver registers structures

26.1.1 I2S_InitTypeDef

I2S_InitTypeDef is defined in the stm32f2xx_hal_i2s.h

Data Fields

- *uint32_t Mode*
- *uint32_t Standard*
- *uint32_t DataFormat*
- *uint32_t MCLKOutput*
- *uint32_t AudioFreq*
- *uint32_t CPOL*
- *uint32_t ClockSource*

Field Documentation

- *uint32_t I2S_InitTypeDef::Mode*
Specifies the I2S operating mode. This parameter can be a value of [I2S_Mode](#)
- *uint32_t I2S_InitTypeDef::Standard*
Specifies the standard used for the I2S communication. This parameter can be a value of [I2S_Standard](#)
- *uint32_t I2S_InitTypeDef::DataFormat*
Specifies the data format for the I2S communication. This parameter can be a value of [I2S_Data_Format](#)
- *uint32_t I2S_InitTypeDef::MCLKOutput*
Specifies whether the I2S MCLK output is enabled or not. This parameter can be a value of [I2S_MCLK_Output](#)
- *uint32_t I2S_InitTypeDef::AudioFreq*
Specifies the frequency selected for the I2S communication. This parameter can be a value of [I2S_Audio_Frequency](#)
- *uint32_t I2S_InitTypeDef::CPOL*
Specifies the idle state of the I2S clock. This parameter can be a value of [I2S_Clock_Polarity](#)
- *uint32_t I2S_InitTypeDef::ClockSource*
Specifies the I2S Clock Source. This parameter can be a value of [I2S_Clock_Source](#)

26.1.2 I2S_HandleTypeDef

I2S_HandleTypeDef is defined in the stm32f2xx_hal_i2s.h

Data Fields

- *SPI_TypeDef * Instance*
- *I2S_InitTypeDef Init*
- *uint16_t * pTxBuffPtr*
- *__IO uint16_t TxXferSize*
- *__IO uint16_t TxXferCount*
- *uint16_t * pRxBuffPtr*
- *__IO uint16_t RxXferSize*
- *__IO uint16_t RxXferCount*
- *DMA_HandleTypeDef * hdmatx*
- *DMA_HandleTypeDef * hdmarx*
- *__IO HAL_LockTypeDef Lock*
- *__IO HAL_I2S_StateTypeDef State*

- **__IO uint32_t ErrorCode**

Field Documentation

- **SPI_TypeDef* I2S_HandleTypeDef::Instance**
I2S registers base address
- **I2S_InitTypeDef I2S_HandleTypeDef::Init**
I2S communication parameters
- **uint16_t* I2S_HandleTypeDef::pTxBuffPtr**
Pointer to I2S Tx transfer buffer
- **__IO uint16_t I2S_HandleTypeDef::TxXferSize**
I2S Tx transfer size
- **__IO uint16_t I2S_HandleTypeDef::TxXferCount**
I2S Tx transfer Counter
- **uint16_t* I2S_HandleTypeDef::pRxBuffPtr**
Pointer to I2S Rx transfer buffer
- **__IO uint16_t I2S_HandleTypeDef::RxXferSize**
I2S Rx transfer size
- **__IO uint16_t I2S_HandleTypeDef::RxXferCount**
I2S Rx transfer counter (This field is initialized at the same value as transfer size at the beginning of the transfer and decremented when a sample is received NbSamplesReceived = RxBufferSize-RxBufferCount)
- **DMA_HandleTypeDef* I2S_HandleTypeDef::hdmatx**
I2S Tx DMA handle parameters
- **DMA_HandleTypeDef* I2S_HandleTypeDef::hdmarx**
I2S Rx DMA handle parameters
- **__IO HAL_LockTypeDef I2S_HandleTypeDef::Lock**
I2S locking object
- **__IO HAL_I2S_StateTypeDef I2S_HandleTypeDef::State**
I2S communication state
- **__IO uint32_t I2S_HandleTypeDef::ErrorCode**
I2S Error code This parameter can be a value of [I2S_Error](#)

26.2 I2S Firmware driver API description

The following section lists the various functions of the I2S library.

26.2.1 How to use this driver

The I2S HAL driver can be used as follow:

1. Declare a I2S_HandleTypeDef handle structure.

2. Initialize the I2S low level resources by implement the HAL_I2S_MspInit() API:
 - a. Enable the SPIx interface clock.
 - b. I2S pins configuration:
 - Enable the clock for the I2S GPIOs.
 - Configure these I2S pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_I2S_Transmit_IT() and HAL_I2S_Receive_IT()) APIs.
 - Configure the I2Sx interrupt priority.
 - Enable the NVIC I2S IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_I2S_Transmit_DMA() and HAL_I2S_Receive_DMA()) APIs:
 - Declare a DMA handle structure for the Tx/Rx Stream/Channel.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx Stream/Channel.
 - Associate the initialized DMA handle to the I2S DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx Stream/Channel.
3. Program the Mode, Standard, Data Format, MCLK Output, Audio frequency and Polarity using HAL_I2S_Init() function.

Note: *The specific I2S interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros `__HAL_I2S_ENABLE_IT()` and `__HAL_I2S_DISABLE_IT()` inside the transmit and receive process.*

Note: *Make sure that either:*

- *I2S PLL clock is configured or*
- *External clock source is configured after setting correctly the define constant `EXTERNAL_CLOCK_VALUE` in the `stm32f2xx_hal_conf.h` file.*

4. Three mode of operations are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_I2S_Transmit()
- Receive an amount of data in blocking mode using HAL_I2S_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_I2S_Transmit_IT()
- At transmission end of half transfer HAL_I2S_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxHalfCpltCallback
- At transmission end of transfer HAL_I2S_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_I2S_Receive_IT()
- At reception end of half transfer HAL_I2S_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxHalfCpltCallback
- At reception end of transfer HAL_I2S_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxCpltCallback
- In case of transfer Error, HAL_I2S_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2S_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_I2S_Transmit_DMA()
- At transmission end of half transfer HAL_I2S_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxHalfCpltCallback

- At transmission end of transfer HAL_I2S_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_I2S_Receive_DMA()
- At reception end of half transfer HAL_I2S_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxHalfCpltCallback
- At reception end of transfer HAL_I2S_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_I2S_RxCpltCallback
- In case of transfer Error, HAL_I2S_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_I2S_ErrorCallback
- Pause the DMA Transfer using HAL_I2S_DMAPause()
- Resume the DMA Transfer using HAL_I2S_DMAResume()
- Stop the DMA Transfer using HAL_I2S_DMAStop()

I2S HAL driver macros list

Below the list of most used macros in I2S HAL driver.

- `__HAL_I2S_ENABLE`: Enable the specified SPI peripheral (in I2S mode)
- `__HAL_I2S_DISABLE`: Disable the specified SPI peripheral (in I2S mode)
- `__HAL_I2S_ENABLE_IT`: Enable the specified I2S interrupts
- `__HAL_I2S_DISABLE_IT`: Disable the specified I2S interrupts
- `__HAL_I2S_GET_FLAG`: Check whether the specified I2S flag is set or not

Note: You can refer to the I2S HAL driver header file for more useful macros

I2S HAL driver macros list

Callback registration:

1. The compilation flag `USE_HAL_I2S_REGISTER_CALLBACKS` when set to 1U allows the user to configure dynamically the driver callbacks. Use Functions `HAL_I2S_RegisterCallback()` to register an interrupt callback. Function `HAL_I2S_RegisterCallback()` allows to register following callbacks:
 - `TxCpltCallback`: I2S Tx Completed callback
 - `RxCpltCallback`: I2S Rx Completed callback
 - `TxHalfCpltCallback`: I2S Tx Half Completed callback
 - `RxHalfCpltCallback`: I2S Rx Half Completed callback
 - `ErrorCallback`: I2S Error callback
 - `MspInitCallback`: I2S Msp Init callback
 - `MspDeInitCallback`: I2S Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.
2. Use function `HAL_I2S_UnRegisterCallback` to reset a callback to the default weak function. `HAL_I2S_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:
 - `TxCpltCallback`: I2S Tx Completed callback
 - `RxCpltCallback`: I2S Rx Completed callback
 - `TxHalfCpltCallback`: I2S Tx Half Completed callback
 - `RxHalfCpltCallback`: I2S Rx Half Completed callback
 - `ErrorCallback`: I2S Error callback
 - `MspInitCallback`: I2S Msp Init callback
 - `MspDeInitCallback`: I2S Msp DeInit callback

By default, after the `HAL_I2S_Init()` and when the state is `HAL_I2S_STATE_RESET` all callbacks are set to the corresponding weak functions: examples `HAL_I2S_MasterTxCpltCallback()`, `HAL_I2S_MasterRxCpltCallback()`. Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functions in the `HAL_I2S_Init()` / `HAL_I2S_DeInit()` only when these callbacks are null (not registered beforehand). If `MspInit` or `MspDeInit` are not null, the `HAL_I2S_Init()` / `HAL_I2S_DeInit()` keep and use the user `MspInit` / `MspDeInit` callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in HAL_I2S_STATE_READY state only. Exception done MspInit/MspDeInit functions that can be registered/unregistered in HAL_I2S_STATE_READY or HAL_I2S_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. Then, the user first registers the MspInit/MspDeInit user callbacks using HAL_I2S_RegisterCallback() before calling HAL_I2S_DeInit() or HAL_I2S_Init() function.

When the compilation define USE_HAL_I2S_REGISTER_CALLBACKS is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

26.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and de-initialize the I2Sx peripheral in simplex mode:

- User must Implement HAL_I2S_MspInit() function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).
- Call the function HAL_I2S_Init() to configure the selected device with the selected configuration:
 - Mode
 - Standard
 - Data Format
 - MCLK Output
 - Audio frequency
 - Polarity
- Call the function HAL_I2S_DeInit() to restore the default configuration of the selected I2Sx peripheral.

This section contains the following APIs:

- [*HAL_I2S_Init*](#)
- [*HAL_I2S_DeInit*](#)
- [*HAL_I2S_MspInit*](#)
- [*HAL_I2S_MspDeInit*](#)

26.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the I2S data transfers.

1. There are two modes of transfer:
 - Blocking mode : The communication is performed in the polling mode. The status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode : The communication is performed using Interrupts or DMA. These functions return the status of the transfer startup. The end of the data processing will be indicated through the dedicated I2S IRQ when using Interrupt mode or the DMA IRQ when using DMA mode.
2. Blocking mode functions are :
 - HAL_I2S_Transmit()
 - HAL_I2S_Receive()
3. No-Blocking mode functions with Interrupt are :
 - HAL_I2S_Transmit_IT()
 - HAL_I2S_Receive_IT()
4. No-Blocking mode functions with DMA are :
 - HAL_I2S_Transmit_DMA()
 - HAL_I2S_Receive_DMA()
5. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_I2S_TxCpltCallback()
 - HAL_I2S_RxCpltCallback()
 - HAL_I2S_ErrorCallback()

This section contains the following APIs:

- [*HAL_I2S_Transmit*](#)
- [*HAL_I2S_Receive*](#)

- [HAL_I2S_Transmit_IT](#)
- [HAL_I2S_Receive_IT](#)
- [HAL_I2S_Transmit_DMA](#)
- [HAL_I2S_Receive_DMA](#)
- [HAL_I2S_DMABPause](#)
- [HAL_I2S_DMABResume](#)
- [HAL_I2S_DMABStop](#)
- [HAL_I2S_IRQHandler](#)
- [HAL_I2S_TxHalfCpltCallback](#)
- [HAL_I2S_TxCpltCallback](#)
- [HAL_I2S_RxHalfCpltCallback](#)
- [HAL_I2S_RxCpltCallback](#)
- [HAL_I2S_ErrorCallback](#)

26.2.4 Peripheral State and Errors functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [HAL_I2S_GetState](#)
- [HAL_I2S_GetError](#)

26.2.5 Detailed description of functions

HAL_I2S_Init

Function name

HAL_StatusTypeDef HAL_I2S_Init (I2S_HandleTypeDef * hi2s)

Function description

Initializes the I2S according to the specified parameters in the I2S_InitTypeDef and create the associated handle.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DeInit

Function name

HAL_StatusTypeDef HAL_I2S_DeInit (I2S_HandleTypeDef * hi2s)

Function description

DeInitializes the I2S peripheral.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_MspltInit

Function name

void HAL_I2S_MspltInit (I2S_HandleTypeDef * hi2s)

Function description

I2S MSP Init.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_MspDeInit

Function name

void HAL_I2S_MspDeInit (I2S_HandleTypeDef * hi2s)

Function description

I2S MSP DeInit.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_Transmit

Function name

HAL_StatusTypeDef HAL_I2S_Transmit (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmit an amount of data in blocking mode.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData:** a 16-bit pointer to data buffer.
- **Size:** number of data sample to be sent:
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive

Function name

HAL_StatusTypeDef HAL_I2S_Receive (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData:** a 16-bit pointer to data buffer.
- **Size:** number of data sample to be sent:
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).
- In I2S Master Receiver mode, just after enabling the peripheral the clock will be generate in continuous way and as the I2S is not disabled at the end of the I2S transaction.

HAL_I2S_Transmit_IT

Function name

HAL_StatusTypeDef HAL_I2S_Transmit_IT (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData:** a 16-bit pointer to data buffer.
- **Size:** number of data sample to be sent:

Return values

- **HAL:** status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive_IT

Function name

HAL_StatusTypeDef HAL_I2S_Receive_IT (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData:** a 16-bit pointer to the Receive data buffer.
- **Size:** number of data sample to be sent:

Return values

- **HAL:** status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).
- It is recommended to use DMA for the I2S receiver to avoid de-synchronization between Master and Slave otherwise the I2S interrupt should be optimized.

HAL_I2S_IRQHandler

Function name

```
void HAL_I2S_IRQHandler (I2S_HandleTypeDef * hi2s)
```

Function description

This function handles I2S interrupt request.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None**:

HAL_I2S_Transmit_DMA

Function name

```
HAL_StatusTypeDef HAL_I2S_Transmit_DMA (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)
```

Function description

Transmit an amount of data in non-blocking mode with DMA.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to the Transmit data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_Receive_DMA

Function name

```
HAL_StatusTypeDef HAL_I2S_Receive_DMA (I2S_HandleTypeDef * hi2s, uint16_t * pData, uint16_t Size)
```

Function description

Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module
- **pData**: a 16-bit pointer to the Receive data buffer.
- **Size**: number of data sample to be sent:

Return values

- **HAL**: status

Notes

- When a 16-bit data frame or a 16-bit data frame extended is selected during the I2S configuration phase, the Size parameter means the number of 16-bit data length in the transaction and when a 24-bit data frame or a 32-bit data frame is selected the Size parameter means the number of 16-bit data length.
- The I2S is kept enabled at the end of transaction to avoid the clock de-synchronization between Master and Slave(example: audio streaming).

HAL_I2S_DMAPause

Function name

HAL_StatusTypeDef HAL_I2S_DMAPause (I2S_HandleTypeDef * hi2s)

Function description

Pauses the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DMAResume

Function name

HAL_StatusTypeDef HAL_I2S_DMAResume (I2S_HandleTypeDef * hi2s)

Function description

Resumes the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_DMAStop

Function name

HAL_StatusTypeDef HAL_I2S_DMAStop (I2S_HandleTypeDef * hi2s)

Function description

Stops the audio DMA Stream/Channel playing from the Media.

Parameters

- **hi2s**: pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL**: status

HAL_I2S_TxHalfCpltCallback

Function name

void HAL_I2S_TxHalfCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx Transfer Half completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_TxCpltCallback

Function name

void HAL_I2S_TxCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Tx Transfer completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_RxHalfCpltCallback

Function name

void HAL_I2S_RxHalfCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Rx Transfer half completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_RxCpltCallback

Function name

void HAL_I2S_RxCpltCallback (I2S_HandleTypeDef * hi2s)

Function description

Rx Transfer completed callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_ErrorCallback

Function name

void HAL_I2S_ErrorCallback (I2S_HandleTypeDef * hi2s)

Function description

I2S error callbacks.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **None:**

HAL_I2S_GetState

Function name

HAL_I2S_StateTypeDef HAL_I2S_GetState (I2S_HandleTypeDef * hi2s)

Function description

Return the I2S state.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **HAL:** state

HAL_I2S_GetError

Function name

uint32_t HAL_I2S_GetError (I2S_HandleTypeDef * hi2s)

Function description

Return the I2S error code.

Parameters

- **hi2s:** pointer to a I2S_HandleTypeDef structure that contains the configuration information for I2S module

Return values

- **I2S:** Error Code

26.3 I2S Firmware driver defines

The following section lists the various define and macros of the module.

26.3.1

I2S

I2S

I2S Audio Frequency

I2S_AUDIOFREQ_192K

I2S_AUDIOFREQ_96K

I2S_AUDIOFREQ_48K

I2S_AUDIOFREQ_44K

I2S_AUDIOFREQ_32K

I2S_AUDIOFREQ_22K

I2S_AUDIOFREQ_16K

I2S_AUDIOFREQ_11K

I2S_AUDIOFREQ_8K

I2S_AUDIOFREQ_DEFAULT

I2S Clock Polarity

I2S_CPOL_LOW

I2S_CPOL_HIGH

I2S Clock Source

I2S_CLOCK_PLL

I2S_CLOCK_EXTERNAL

I2S Data Format

I2S_DATAFORMAT_16B

I2S_DATAFORMAT_16B_EXTENDED

I2S_DATAFORMAT_24B

I2S_DATAFORMAT_32B

I2S Error

HAL_I2S_ERROR_NONE

No error

HAL_I2S_ERROR_TIMEOUT

Timeout error

HAL_I2S_ERROR_OVR

OVR error

HAL_I2S_ERROR_UDR

UDR error

HAL_I2S_ERROR_DMA

DMA transfer error

HAL_I2S_ERROR_PRESCALER

Prescaler Calculation error

I2S Exported Macros

`__HAL_I2S_RESET_HANDLE_STATE`

Description:

- Reset I2S handle state.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

`__HAL_I2S_ENABLE`

Description:

- Enable the specified SPI peripheral (in I2S mode).

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

`__HAL_I2S_DISABLE`

Description:

- Disable the specified SPI peripheral (in I2S mode).

Parameters:

- `__HANDLE__`: specifies the I2S Handle.

Return value:

- None

`__HAL_I2S_ENABLE_IT`

Description:

- Enable the specified I2S interrupts.

Parameters:

- `__HANDLE__`: specifies the I2S Handle.
- `__INTERRUPT__`: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - `I2S_IT_TXE`: Tx buffer empty interrupt enable
 - `I2S_IT_RXNE`: RX buffer not empty interrupt enable
 - `I2S_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_I2S_DISABLE_IT

Description:

- Disable the specified I2S interrupts.

Parameters:

- **__HANDLE__**: specifies the I2S Handle.
- **__INTERRUPT__**: specifies the interrupt source to enable or disable. This parameter can be one of the following values:
 - I2S_IT_TXE: Tx buffer empty interrupt enable
 - I2S_IT_RXNE: RX buffer not empty interrupt enable
 - I2S_IT_ERR: Error interrupt enable

Return value:

- None

__HAL_I2S_GET_IT_SOURCE

Description:

- Checks if the specified I2S interrupt source is enabled or disabled.

Parameters:

- **__HANDLE__**: specifies the I2S Handle. This parameter can be I2S where x: 1, 2, or 3 to select the I2S peripheral.
- **__INTERRUPT__**: specifies the I2S interrupt source to check. This parameter can be one of the following values:
 - I2S_IT_TXE: Tx buffer empty interrupt enable
 - I2S_IT_RXNE: RX buffer not empty interrupt enable
 - I2S_IT_ERR: Error interrupt enable

Return value:

- The: new state of **__IT__** (TRUE or FALSE).

__HAL_I2S_GET_FLAG

Description:

- Checks whether the specified I2S flag is set or not.

Parameters:

- **__HANDLE__**: specifies the I2S Handle.
- **__FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - I2S_FLAG_RXNE: Receive buffer not empty flag
 - I2S_FLAG_TXE: Transmit buffer empty flag
 - I2S_FLAG_UDR: Underrun flag
 - I2S_FLAG_OVR: Overrun flag
 - I2S_FLAG_FRE: Frame error flag
 - I2S_FLAG_CHSIDE: Channel Side flag
 - I2S_FLAG_BSY: Busy flag

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

__HAL_I2S_CLEAR_OVRFLAG

Description:

- Clears the I2S OVR pending flag.

Parameters:

- __HANDLE__: specifies the I2S Handle.

Return value:

- None

__HAL_I2S_CLEAR_UDRFLAG

Description:

- Clears the I2S UDR pending flag.

Parameters:

- __HANDLE__: specifies the I2S Handle.

Return value:

- None

I2S Flags Definition

I2S_FLAG_TXE

I2S_FLAG_RXNE

I2S_FLAG_UDR

I2S_FLAG_OVR

I2S_FLAG_FRE

I2S_FLAG_CHSIDE

I2S_FLAG_BSY

I2S_FLAG_MASK

I2S Interrupts Definition

I2S_IT_TXE

I2S_IT_RXNE

I2S_IT_ERR

I2S MCLK Output

I2S_MCLKOUTPUT_ENABLE

I2S_MCLKOUTPUT_DISABLE

I2S Mode

I2S_MODE_SLAVE_TX

I2S_MODE_SLAVE_RX

I2S_MODE_MASTER_TX

I2S_MODE_MASTER_RX

I2S Standard

I2S_STANDARD_PHILIPS

I2S_STANDARD_MSB

I2S_STANDARD_LSB

I2S_STANDARD_PCM_SHORT

I2S_STANDARD_PCM_LONG

27 HAL IRDA Generic Driver

27.1 IRDA Firmware driver registers structures

27.1.1 IRDA_InitTypeDef

IRDA_InitTypeDef is defined in the `stm32f2xx_hal_irda.h`

Data Fields

- *uint32_t* **BaudRate**
- *uint32_t* **WordLength**
- *uint32_t* **Parity**
- *uint32_t* **Mode**
- *uint8_t* **Prescaler**
- *uint32_t* **IrDAMode**

Field Documentation

- *uint32_t* **IRDA_InitTypeDef::BaudRate**
This member configures the IRDA communication baud rate. The baud rate is computed using the following formula:
 - $\text{IntegerDivider} = (\text{PCLKx}) / (8 * (\text{hirda->Init.BaudRate}))$
 - $\text{FractionalDivider} = ((\text{IntegerDivider} - ((\text{uint32_t}) \text{IntegerDivider})) * 8) + 0.5$
- *uint32_t* **IRDA_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [IRDA_Word_Length](#)
- *uint32_t* **IRDA_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [IRDA_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- *uint32_t* **IRDA_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [IRDA_Mode](#)
- *uint8_t* **IRDA_InitTypeDef::Prescaler**
Specifies the Prescaler value to be programmed in the IrDA low-power Baud Register, for defining pulse width on which burst acceptance/rejection will be decided. This value is used as divisor of system clock to achieve required pulse width.
- *uint32_t* **IRDA_InitTypeDef::IrDAMode**
Specifies the IrDA mode This parameter can be a value of [IRDA_Low_Power](#)

27.1.2 IRDA_HandleTypeDef

IRDA_HandleTypeDef is defined in the `stm32f2xx_hal_irda.h`

Data Fields

- *USART_TypeDef ** **Instance**
- *IRDA_InitTypeDef* **Init**
- *uint8_t ** **pTxBuffPtr**
- *uint16_t* **TxXferSize**
- *__IO uint16_t* **TxXferCount**
- *uint8_t ** **pRxBuffPtr**
- *uint16_t* **RxXferSize**
- *__IO uint16_t* **RxXferCount**

- ***DMA_HandleTypeDef * hdmatrix***
- ***DMA_HandleTypeDef * hdmatrix***
- ***HAL_LockTypeDef Lock***
- ***__IO HAL_IRDA_StateTypeDef gState***
- ***__IO HAL_IRDA_StateTypeDef RxState***
- ***__IO uint32_t ErrorCode***

Field Documentation

- ***USART_TypeDef* IRDA_HandleTypeDef::Instance***
USART registers base address
- ***IRDA_InitTypeDef IRDA_HandleTypeDef::Init***
IRDA communication parameters
- ***uint8_t* IRDA_HandleTypeDef::pTxBuffPtr***
Pointer to IRDA Tx transfer Buffer
- ***uint16_t IRDA_HandleTypeDef::TxXferSize***
IRDA Tx Transfer size
- ***__IO uint16_t IRDA_HandleTypeDef::TxXferCount***
IRDA Tx Transfer Counter
- ***uint8_t* IRDA_HandleTypeDef::pRxBuffPtr***
Pointer to IRDA Rx transfer Buffer
- ***uint16_t IRDA_HandleTypeDef::RxXferSize***
IRDA Rx Transfer size
- ***__IO uint16_t IRDA_HandleTypeDef::RxXferCount***
IRDA Rx Transfer Counter
- ***DMA_HandleTypeDef* IRDA_HandleTypeDef::hdmatrix***
IRDA Tx DMA Handle parameters
- ***DMA_HandleTypeDef* IRDA_HandleTypeDef::hdmatrix***
IRDA Rx DMA Handle parameters
- ***HAL_LockTypeDef IRDA_HandleTypeDef::Lock***
Locking object
- ***__IO HAL_IRDA_StateTypeDef IRDA_HandleTypeDef::gState***
IRDA state information related to global Handle management and also related to Tx operations. This parameter can be a value of **HAL_IRDA_StateTypeDef**
- ***__IO HAL_IRDA_StateTypeDef IRDA_HandleTypeDef::RxState***
IRDA state information related to Rx operations. This parameter can be a value of **HAL_IRDA_StateTypeDef**
- ***__IO uint32_t IRDA_HandleTypeDef::ErrorCode***
IRDA Error code

27.2 IRDA Firmware driver API description

The following section lists the various functions of the IRDA library.

27.2.1 How to use this driver

The IRDA HAL driver can be used as follows:

1. Declare a **IRDA_HandleTypeDef** handle structure (eg. **IRDA_HandleTypeDef hirda**).

2. Initialize the IRDA low level resources by implementing the HAL_IRDA_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. IRDA pins configuration:
 - Enable the clock for the IRDA GPIOs.
 - Configure IRDA pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_IRDA_Transmit_IT() and HAL_IRDA_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_IRDA_Transmit_DMA() and HAL_IRDA_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the IRDA DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the IRDAx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Parity, IrDA Mode, Prescaler and Mode(Receiver/Transmitter) in the hirda Init structure.
4. Initialize the IRDA registers by calling the HAL_IRDA_Init() API:
 - This API configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_IRDA_MspInit() API.

Note: *The specific IRDA interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros `__HAL_IRDA_ENABLE_IT()` and `__HAL_IRDA_DISABLE_IT()` inside the transmit and receive process.*

5. Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_IRDA_Transmit()
- Receive an amount of data in blocking mode using HAL_IRDA_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_IRDA_Transmit_IT()
- At transmission end of transfer HAL_IRDA_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_IRDA_Receive_IT()
- At reception end of transfer HAL_IRDA_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxCpltCallback
- In case of transfer Error, HAL_IRDA_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_IRDA_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_IRDA_Transmit_DMA()
- At transmission end of half transfer HAL_IRDA_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxHalfCpltCallback
- At transmission end of transfer HAL_IRDA_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_IRDA_Receive_DMA()

- At reception end of half transfer HAL_IRDA_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxHalfCpltCallback
- At reception end of transfer HAL_IRDA_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_IRDA_RxCpltCallback
- In case of transfer Error, HAL_IRDA_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_IRDA_ErrorCallback
- Pause the DMA Transfer using HAL_IRDA_DMABasePause()
- Resume the DMA Transfer using HAL_IRDA_DMABaseResume()
- Stop the DMA Transfer using HAL_IRDA_DMABaseStop()

IRDA HAL driver macros list

Below the list of most used macros in IRDA HAL driver.

- `__HAL_IRDA_ENABLE`: Enable the IRDA peripheral
- `__HAL_IRDA_DISABLE`: Disable the IRDA peripheral
- `__HAL_IRDA_GET_FLAG`: Check whether the specified IRDA flag is set or not
- `__HAL_IRDA_CLEAR_FLAG`: Clear the specified IRDA pending flag
- `__HAL_IRDA_ENABLE_IT`: Enable the specified IRDA interrupt
- `__HAL_IRDA_DISABLE_IT`: Disable the specified IRDA interrupt
- `__HAL_IRDA_GET_IT_SOURCE`: Check whether the specified IRDA interrupt has occurred or not

Note: You can refer to the IRDA HAL driver header file for more useful macros

27.2.2

Callback registration

The compilation define `USE_HAL_IRDA_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `@ref HAL_IRDA_RegisterCallback()` to register a user callback. Function `@ref HAL_IRDA_RegisterCallback()` allows to register following callbacks:

- `TxHalfCpltCallback`: Tx Half Complete Callback.
- `TxCpltCallback`: Tx Complete Callback.
- `RxHalfCpltCallback`: Rx Half Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.
- `MspInitCallback`: IRDA MspInit.
- `MspDeInitCallback`: IRDA MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `@ref HAL_IRDA_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `@ref HAL_IRDA_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `TxHalfCpltCallback`: Tx Half Complete Callback.
- `TxCpltCallback`: Tx Complete Callback.
- `RxHalfCpltCallback`: Rx Half Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.
- `MspInitCallback`: IRDA MspInit.
- `MspDeInitCallback`: IRDA MspDeInit.

By default, after the @ref HAL_IRDA_Init() and when the state is HAL_IRDA_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples @ref HAL_IRDA_TxCpltCallback(), @ref HAL_IRDA_RxHalfCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_IRDA_Init() and @ref HAL_IRDA_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_IRDA_Init() and @ref HAL_IRDA_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_IRDA_STATE_READY state only. Exception done MspInit/MspDeInit that can be registered/unregistered in HAL_IRDA_STATE_READY or HAL_IRDA_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_IRDA_RegisterCallback() before calling @ref HAL_IRDA_DeInit() or @ref HAL_IRDA_Init() function.

When The compilation define USE_HAL_IRDA_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

Note: *Additionnal remark: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), the possible IRDA frame formats are as listed in the following table: +-----*

+ M bit PCE bit IRDA frame	-----	0 0	SB 8 bit data 1 STB
	-----	0 1	SB 7 bit data PB 1 STB
-----	-----	1 0	SB 9 bit data 1 STB
-----	-----	1 1	SB 8 bit data PB 1 STB
+-----			

27.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USARTx or the UARTy in asynchronous IrDA mode.

- For the asynchronous mode only these parameters can be configured:
 - BaudRate
 - WordLength
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible IRDA frame formats.
 - Prescaler: A pulse of width less than two and greater than one PSC period(s) may or may not be rejected. The receiver set up time should be managed by software. The IrDA physical layer specification specifies a minimum of 10 ms delay between transmission and reception (IrDA is a half duplex protocol).
 - Mode: Receiver/transmitter modes
 - IrDAMode: the IrDA can operate in the Normal mode or in the Low power mode.

The HAL_IRDA_Init() API follows IRDA configuration procedures (details for the procedures are available in reference manual).

This section contains the following APIs:

- [HAL_IRDA_Init](#)
- [HAL_IRDA_DeInit](#)
- [HAL_IRDA_MspInit](#)
- [HAL_IRDA_MspDeInit](#)

27.2.4 IO operation functions

This subsection provides a set of functions allowing to manage the IRDA data transfers. IrDA is a half duplex communication protocol. If the Transmitter is busy, any data on the IrDA receive line will be ignored by the IrDA decoder and if the Receiver is busy, data on the TX from the USART to IrDA will not be encoded by IrDA. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted.

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - Non-Blocking mode: The communication is performed using Interrupts or DMA, these API's return the HAL status. The end of the data processing will be indicated through the dedicated IRDA IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_IRDA_TxCpltCallback(), HAL_IRDA_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process. The HAL_IRDA_ErrorCallback() user callback will be executed when a communication error is detected.
2. Blocking mode APIs are :
 - HAL_IRDA_Transmit()
 - HAL_IRDA_Receive()
3. Non Blocking mode APIs with Interrupt are :
 - HAL_IRDA_Transmit_IT()
 - HAL_IRDA_Receive_IT()
 - HAL_IRDA_IRQHandler()
4. Non Blocking mode functions with DMA are :
 - HAL_IRDA_Transmit_DMA()
 - HAL_IRDA_Receive_DMA()
 - HAL_IRDA_DMAPause()
 - HAL_IRDA_DMAResume()
 - HAL_IRDA_DMAStop()
5. A set of Transfer Complete Callbacks are provided in Non Blocking mode:
 - HAL_IRDA_TxHalfCpltCallback()
 - HAL_IRDA_TxCpltCallback()
 - HAL_IRDA_RxHalfCpltCallback()
 - HAL_IRDA_RxCpltCallback()
 - HAL_IRDA_ErrorCallback()
6. Non-Blocking mode transfers could be aborted using Abort API's : (+) HAL_IRDA_Abort() (+) HAL_IRDA_AbortTransmit() (+) HAL_IRDA_AbortReceive() (+) HAL_IRDA_Abort_IT() (+) HAL_IRDA_AbortTransmit_IT() (+) HAL_IRDA_AbortReceive_IT()
7. For Abort services based on interrupts (HAL_IRDA_Abortxxx_IT), a set of Abort Complete Callbacks are provided: (+) HAL_IRDA_AbortCpltCallback() (+) HAL_IRDA_AbortTransmitCpltCallback() (+) HAL_IRDA_AbortReceiveCpltCallback()
8. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
 - (+) Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed. Transfer is kept ongoing on IRDA side. If user wants to abort it, Abort services should be called by user.
 - (+) Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed.

This subsection provides a set of functions allowing to manage the IRDA data transfers. IrDA is a half duplex communication protocol. If the Transmitter is busy, any data on the IrDA receive line will be ignored by the IrDA decoder and if the Receiver is busy, data on the TX from the USART to IrDA will not be encoded by IrDA. While receiving data, transmission should be avoided as the data to be transmitted could be corrupted. (#) There are two modes of transfer: (++) Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer. (++) Non-Blocking mode: The communication is performed using Interrupts or DMA, these API's return the HAL status. The end of the data processing will be indicated through the dedicated IRDA IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_IRDA_TxCpltCallback(), HAL_IRDA_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process. The HAL_IRDA_ErrorCallback() user callback will be executed when a communication error is detected. (#) Blocking mode APIs are : (++) HAL_IRDA_Transmit() (++) HAL_IRDA_Receive() (#) Non Blocking mode APIs with Interrupt are : (++) HAL_IRDA_Transmit_IT() (++) HAL_IRDA_Receive_IT() (++) HAL_IRDA_IRQHandler() (#) Non Blocking mode functions with DMA are : (++) HAL_IRDA_Transmit_DMA() (++) HAL_IRDA_Receive_DMA() (++) HAL_IRDA_DMAPause() (++) HAL_IRDA_DMAResume() (++) HAL_IRDA_DMAStop() (#) A set of Transfer Complete Callbacks are provided in Non Blocking mode: (++) HAL_IRDA_TxHalfCpltCallback() (++) HAL_IRDA_TxCpltCallback() (++) HAL_IRDA_RxHalfCpltCallback() (++) HAL_IRDA_RxCpltCallback() (++) HAL_IRDA_ErrorCallback() (#) Non-Blocking mode transfers could be aborted using Abort API's :

- HAL_IRDA_Abort()
- HAL_IRDA_AbortTransmit()
- HAL_IRDA_AbortReceive()
- HAL_IRDA_Abort_IT()
- HAL_IRDA_AbortTransmit_IT()
- HAL_IRDA_AbortReceive_IT() (#) For Abort services based on interrupts (HAL_IRDA_Abortxxx_IT), a set of Abort Complete Callbacks are provided:
- HAL_IRDA_AbortCpltCallback()
- HAL_IRDA_AbortTransmitCpltCallback()
- HAL_IRDA_AbortReceiveCpltCallback() (#) In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
- Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed. Transfer is kept ongoing on IRDA side. If user wants to abort it, Abort services should be called by user.
- Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_IRDA_ErrorCallback() user callback is executed.

This section contains the following APIs:

- [**HAL_IRDA_Transmit**](#)
- [**HAL_IRDA_Receive**](#)
- [**HAL_IRDA_Transmit_IT**](#)
- [**HAL_IRDA_Receive_IT**](#)
- [**HAL_IRDA_Transmit_DMA**](#)
- [**HAL_IRDA_Receive_DMA**](#)
- [**HAL_IRDA_DMAPause**](#)
- [**HAL_IRDA_DMAResume**](#)
- [**HAL_IRDA_DMAStop**](#)
- [**HAL_IRDA_Abort**](#)
- [**HAL_IRDA_AbortTransmit**](#)
- [**HAL_IRDA_AbortReceive**](#)
- [**HAL_IRDA_Abort_IT**](#)
- [**HAL_IRDA_AbortTransmit_IT**](#)
- [**HAL_IRDA_AbortReceive_IT**](#)
- [**HAL_IRDA_IRQHandler**](#)

- [HAL_IRDA_TxCpltCallback](#)
- [HAL_IRDA_TxHalfCpltCallback](#)
- [HAL_IRDA_RxCpltCallback](#)
- [HAL_IRDA_RxHalfCpltCallback](#)
- [HAL_IRDA_ErrorCallback](#)
- [HAL_IRDA_AbortCpltCallback](#)
- [HAL_IRDA_AbortTransmitCpltCallback](#)
- [HAL_IRDA_AbortReceiveCpltCallback](#)

27.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of IrDA communication process and also return Peripheral Errors occurred during communication process

- [HAL_IRDA_GetState\(\)](#) API can be helpful to check in run-time the state of the IrDA peripheral.
- [HAL_IRDA_GetError\(\)](#) check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [HAL_IRDA_GetState](#)
- [HAL_IRDA_GetError](#)

27.2.6 Detailed description of functions

HAL_IRDA_Init

Function name

HAL_StatusTypeDef HAL_IRDA_Init (IRDA_HandleTypeDef * hirda)

Function description

Initializes the IRDA mode according to the specified parameters in the [IRDA_InitTypeDef](#) and create the associated handle.

Parameters

- **hirda**: Pointer to a [IRDA_HandleTypeDef](#) structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DeInit

Function name

HAL_StatusTypeDef HAL_IRDA_DeInit (IRDA_HandleTypeDef * hirda)

Function description

Deinitializes the IRDA peripheral.

Parameters

- **hirda**: Pointer to a [IRDA_HandleTypeDef](#) structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_MspltInit

Function name

void HAL_IRDA_MspltInit (IRDA_HandleTypeDef * hirda)

Function description

IRDA MSP Init.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_MspDeInit

Function name

void HAL_IRDA_MspDeInit (IRDA_HandleTypeDef * hirda)

Function description

IRDA MSP DeInit.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_Transmit

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Sends an amount of data in blocking mode.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData**: Pointer to data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be sent.
- **Timeout**: Specify timeout value.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled ($PCE = 0$), and Word Length is configured to 9 bits ($M1-M0 = 01$), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive

Function name

HAL_StatusTypeDef HAL_IRDA_Receive (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receive an amount of data in blocking mode.

Parameters

- **hirda:** Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.
- **Timeout:** Specify timeout value

Return values

- **HAL:** status

Notes

- When UART parity is not enabled ($PCE = 0$), and Word Length is configured to 9 bits ($M1-M0 = 01$), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Transmit_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit_IT (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hirda:** Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled ($PCE = 0$), and Word Length is configured to 9 bits ($M1-M0 = 01$), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Receive_IT (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hirda:** Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_IRDA_Transmit_DMA (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in DMA mode.

Parameters

- hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.
- pData**: Pointer to data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be sent.

Return values

- HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.

HAL_IRDA_Receive_DMA

Function name

HAL_StatusTypeDef HAL_IRDA_Receive_DMA (IRDA_HandleTypeDef * hirda, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in DMA mode.

Parameters

- hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.
- pData**: Pointer to data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be received.

Return values

- HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must reflect the number of u16 available through pData.
- When the IRDA parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_IRDA_DMAPause

Function name

HAL_StatusTypeDef HAL_IRDA_DMAPause (IRDA_HandleTypeDef * hirda)

Function description

Pauses the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DMAResume

Function name

HAL_StatusTypeDef HAL_IRDA_DMAResume (IRDA_HandleTypeDef * hirda)

Function description

Resumes the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_DMAStop

Function name

HAL_StatusTypeDef HAL_IRDA_DMAStop (IRDA_HandleTypeDef * hirda)

Function description

Stops the DMA Transfer.

Parameters

- **hirda**: Pointer to a `IRDA_HandleTypeDef` structure that contains the configuration information for the specified IRDA module.

Return values

- **HAL**: status

HAL_IRDA_Abort

Function name

HAL_StatusTypeDef HAL_IRDA_Abort (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_AbortTransmit

Function name

HAL_StatusTypeDef HAL_IRDA_AbortTransmit (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_AbortReceive

Function name

HAL_StatusTypeDef HAL_IRDA_AbortReceive (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **hirda**: IRDA handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_IRDA_Abort_IT

Function name

HAL_StatusTypeDef HAL_IRDA_Abort_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_IRDA_AbortTransmit_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable IRDA Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_IRDA_AbortReceive_IT (IRDA_HandleTypeDef * hirda)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **hirda:** IRDA handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_IRDA_IRQHandler

Function name

void HAL_IRDA_IRQHandler (IRDA_HandleTypeDef * hirda)

Function description

This function handles IRDA interrupt request.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_TxCpltCallback

Function name

void HAL_IRDA_TxCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Tx Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_RxCpltCallback

Function name

void HAL_IRDA_RxCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Rx Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_TxHalfCpltCallback

Function name

void HAL_IRDA_TxHalfCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Tx Half Transfer completed callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_IRDA_RxHalfCpltCallback

Function name

void HAL_IRDA_RxHalfCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

Rx Half Transfer complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_ErrorCallback

Function name

void HAL_IRDA_ErrorCallback (IRDA_HandleTypeDef * hirda)

Function description

IRDA error callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_AbortCpltCallback

Function name

void HAL_IRDA_AbortCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

IRDA Abort Complete callback.

Parameters

- **hirda:** Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None:**

HAL_IRDA_AbortTransmitCpltCallback

Function name

void HAL_IRDA_AbortTransmitCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

IRDA Abort Transmit Complete callback.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_AbortReceiveCpltCallback

Function name

void HAL_IRDA_AbortReceiveCpltCallback (IRDA_HandleTypeDef * hirda)

Function description

IRDA Abort Receive Complete callback.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA module.

Return values

- **None**:

HAL_IRDA_GetState

Function name

HAL_IRDA_StateTypeDef HAL_IRDA_GetState (IRDA_HandleTypeDef * hirda)

Function description

Return the IRDA state.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA.

Return values

- **HAL**: state

HAL_IRDA_GetError

Function name

uint32_t HAL_IRDA_GetError (IRDA_HandleTypeDef * hirda)

Function description

Return the IRDA error code.

Parameters

- **hirda**: Pointer to a IRDA_HandleTypeDef structure that contains the configuration information for the specified IRDA.

Return values

- **IRDA**: Error Code

27.3 IRDA Firmware driver defines

The following section lists the various define and macros of the module.

27.3.1 IRDA

IRDA

IRDA Error Code

HAL_IRDA_ERROR_NONE

No error

HAL_IRDA_ERROR_PE

Parity error

HAL_IRDA_ERROR_NE

Noise error

HAL_IRDA_ERROR_FE

Frame error

HAL_IRDA_ERROR_ORE

Overrun error

HAL_IRDA_ERROR_DMA

DMA transfer error

IRDA Exported Macros

__HAL_IRDA_RESET_HANDLE_STATE

Description:

- Reset IRDA handle gstate & RxState.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_FLUSH_DRREGISTER

Description:

- Flush the IRDA DR register.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_GET_FLAG

Description:

- Check whether the specified IRDA flag is set or not.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `IRDA_FLAG_TXE`: Transmit data register empty flag
 - `IRDA_FLAG_TC`: Transmission Complete flag
 - `IRDA_FLAG_RXNE`: Receive data register not empty flag
 - `IRDA_FLAG_IDLE`: Idle Line detection flag
 - `IRDA_FLAG_ORE`: OverRun Error flag
 - `IRDA_FLAG_NE`: Noise Error flag
 - `IRDA_FLAG_FE`: Framing Error flag
 - `IRDA_FLAG_PE`: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_IRDA_CLEAR_FLAG

Description:

- Clear the specified IRDA pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be any combination of the following values:
 - `IRDA_FLAG_TC`: Transmission Complete flag.
 - `IRDA_FLAG_RXNE`: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (OverRun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_IRDA_CLEAR_PFLAG

Description:

- Clear the IRDA PE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_FEFLAG

Description:

- Clear the IRDA FE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_NEFLAG

Description:

- Clear the IRDA NE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_OREFLAG

Description:

- Clear the IRDA ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_CLEAR_IDLEFLAG

Description:

- Clear the IRDA IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_ENABLE_IT

Description:

- Enable the specified IRDA interrupt.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the IRDA interrupt source to enable. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_PE`: Parity Error interrupt
 - `IRDA_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_IRDA_DISABLE_IT

Description:

- Disable the specified IRDA interrupt.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__INTERRUPT__`: specifies the IRDA interrupt source to disable. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_PE`: Parity Error interrupt
 - `IRDA_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_IRDA_GET_IT_SOURCE

Description:

- Check whether the specified IRDA interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).
- `__IT__`: specifies the IRDA interrupt source to check. This parameter can be one of the following values:
 - `IRDA_IT_TXE`: Transmit Data Register empty interrupt
 - `IRDA_IT_TC`: Transmission complete interrupt
 - `IRDA_IT_RXNE`: Receive Data register not empty interrupt
 - `IRDA_IT_IDLE`: Idle line detection interrupt
 - `IRDA_IT_ERR`: Error interrupt
 - `IRDA_IT_PE`: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_IRDA_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the IRDA's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle.

Return value:

- None

__HAL_IRDA_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the IRDA's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle.

Return value:

- None

__HAL_IRDA_ENABLE

Description:

- Enable UART/USART associated to IRDA Handle.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_IRDA_DISABLE

Description:

- Disable UART/USART associated to IRDA Handle.

Parameters:

- `__HANDLE__`: specifies the IRDA Handle. IRDA Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

IRDA Flags

IRDA_FLAG_TXE

IRDA_FLAG_TC

IRDA_FLAG_RXNE

IRDA_FLAG_IDLE

IRDA_FLAG_ORE

IRDA_FLAG_NE

IRDA_FLAG_FE

IRDA_FLAG_PE

IRDA Interrupt Definitions

IRDA_IT_PE

IRDA_IT_TXE

IRDA_IT_TC

IRDA_IT_RXNE

IRDA_IT_IDLE

IRDA_IT_LBD

IRDA_IT_CTS

IRDA_IT_ERR

IRDA Low Power

IRDA_POWERMODE_LOWPOWER

IRDA_POWERMODE_NORMAL

IRDA Transfer Mode

IRDA_MODE_RX

IRDA_MODE_TX

IRDA_MODE_TX_RX

IRDA Parity

IRDA_PARITY_NONE

IRDA_PARITY_EVEN

IRDA_PARITY_ODD

IRDA Word Length

IRDA_WORDLENGTH_8B

IRDA_WORDLENGTH_9B

28 HAL IWDG Generic Driver

28.1 IWDG Firmware driver registers structures

28.1.1 IWDG_InitTypeDef

IWDG_InitTypeDef is defined in the stm32f2xx_hal_iwdg.h

Data Fields

- *uint32_t Prescaler*
- *uint32_t Reload*

Field Documentation

- *uint32_t IWDG_InitTypeDef::Prescaler*
Select the prescaler of the IWDG. This parameter can be a value of *IWDG_Prescaler*
- *uint32_t IWDG_InitTypeDef::Reload*
Specifies the IWDG down-counter reload value. This parameter must be a number between Min_Data = 0 and Max_Data = 0x0FFF

28.1.2 IWDG_HandleTypeDef

IWDG_HandleTypeDef is defined in the stm32f2xx_hal_iwdg.h

Data Fields

- *IWDG_TypeDef * Instance*
- *IWDG_InitTypeDef Init*

Field Documentation

- *IWDG_TypeDef* IWDG_HandleTypeDef::Instance*
Register base address
- *IWDG_InitTypeDef IWDG_HandleTypeDef::Init*
IWDG required parameters

28.2 IWDG Firmware driver API description

The following section lists the various functions of the IWDG library.

28.2.1 IWDG Generic features

- The IWDG can be started by either software or hardware (configurable through option byte).
- The IWDG is clocked by Low-Speed clock (LSI) and thus stays active even if the main clock fails.
- Once the IWDG is started, the LSI is forced ON and both can not be disabled. The counter starts counting down from the reset value (0xFFFF). When it reaches the end of count value (0x000) a reset signal is generated (IWDG reset).
- Whenever the key value 0x0000 AAAA is written in the IWDG_KR register, the IWDG_RLR value is reloaded in the counter and the watchdog reset is prevented.
- The IWDG is implemented in the VDD voltage domain that is still functional in STOP and STANDBY mode (IWDG reset can wake-up from STANDBY). IWDGRST flag in RCC_CSR register can be used to inform when an IWDG reset occurs.
- Debug mode : When the microcontroller enters debug mode (core halted), the IWDG counter either continues to work normally or stops, depending on DBG_IWDG_STOP configuration bit in DBG module, accessible through `__HAL_DBGMCU_FREEZE_IWDG()` and `__HAL_DBGMCU_UNFREEZE_IWDG()` macros

Min-max timeout value @32KHz (LSI): ~125us / ~32.7s The IWDG timeout may vary due to LSI frequency dispersion. STM32F2xx devices provide the capability to measure the LSI frequency (LSI clock connected internally to TIM5 CH4 input capture). The measured value can be used to have an IWDG timeout with an acceptable accuracy.

28.2.2 How to use this driver

1. Use IWDG using HAL_IWDG_Init() function to : (+) Enable instance by writing Start keyword in IWDG_KEY register. LSI clock is forced ON and IWDG counter starts downcounting. (+) Enable write access to configuration register: IWDG_PR & IWDG_RLR. (+) Configure the IWDG prescaler and counter reload value. This reload value will be loaded in the IWDG counter each time the watchdog is reloaded, then the IWDG will start counting down from this value. (+) wait for status flags to be reset"
 2. Then the application program must refresh the IWDG counter at regular intervals during normal operation to prevent an MCU reset, using HAL_IWDG_Refresh() function.
- (#) Use IWDG using HAL_IWDG_Init() function to :
- Enable instance by writing Start keyword in IWDG_KEY register. LSI clock is forced ON and IWDG counter starts downcounting.
 - Enable write access to configuration register: IWDG_PR & IWDG_RLR.
 - Configure the IWDG prescaler and counter reload value. This reload value will be loaded in the IWDG counter each time the watchdog is reloaded, then the IWDG will start counting down from this value.
 - wait for status flags to be reset" (#) Then the application program must refresh the IWDG counter at regular intervals during normal operation to prevent an MCU reset, using HAL_IWDG_Refresh() function.

IWDG HAL driver macros list

Below the list of most used macros in IWDG HAL driver:

- `__HAL_IWDG_START`: Enable the IWDG peripheral
- `__HAL_IWDG_RELOAD_COUNTER`: Reloads IWDG counter with value defined in the reload register

28.2.3 Initialization and Start functions

This section provides functions allowing to:

- Initialize the IWDG according to the specified parameters in the IWDG_InitTypeDef of associated handle.
- Once initialization is performed in HAL_IWDG_Init function, Watchdog is reloaded in order to exit function with correct time base.

This section contains the following APIs:

- [HAL_IWDG_Init](#)

28.2.4 IO operation functions

This section provides functions allowing to:

- Refresh the IWDG.

This section contains the following APIs:

- [HAL_IWDG_Refresh](#)

28.2.5 Detailed description of functions

HAL_IWDG_Init

Function name

HAL_StatusTypeDef HAL_IWDG_Init (IWDG_HandleTypeDef * hiwdg)

Function description

Initialize the IWDG according to the specified parameters in the IWDG_InitTypeDef and start watchdog.

Parameters

- **hiwdg**: pointer to a IWDG_HandleTypeDef structure that contains the configuration information for the specified IWDG module.

Return values

- **HAL**: status

HAL_IWDG_Refresh

Function name

HAL_StatusTypeDef HAL_IWDG_Refresh (IWDG_HandleTypeDef * hiwdg)

Function description

Refresh the IWDG.

Parameters

- **hiwdg**: pointer to a IWDG_HandleTypeDef structure that contains the configuration information for the specified IWDG module.

Return values

- **HAL**: status

28.3 IWDG Firmware driver defines

The following section lists the various define and macros of the module.

28.3.1 IWDG

IWDG

IWDG Exported Macros

__HAL_IWDG_START

Description:

- Enable the IWDG peripheral.

Parameters:

- **__HANDLE__**: IWDG handle

Return value:

- None

__HAL_IWDG_RELOAD_COUNTER

Description:

- Reload IWDG counter with value defined in the reload register (write access to IWDG_PR & IWDG_RLR registers disabled).

Parameters:

- **__HANDLE__**: IWDG handle

Return value:

- None

IWDG Prescaler

IWDG_PRESCALER_4

IWDG prescaler set to 4

IWDG_PRESCALER_8

IWDG prescaler set to 8

IWDG_PRESCALER_16

IWDG prescaler set to 16

IWDG_PRESCALER_32

IWDG prescaler set to 32

IWDG_PRESCALER_64

IWDG prescaler set to 64

IWDG_PRESCALER_128

IWDG prescaler set to 128

IWDG_PRESCALER_256

IWDG prescaler set to 256

29 HAL NAND Generic Driver

29.1 NAND Firmware driver registers structures

29.1.1 NAND_IDTypeDef

NAND_IDTypeDef is defined in the stm32f2xx_hal_nand.h

Data Fields

- *uint8_t Maker_Id*
- *uint8_t Device_Id*
- *uint8_t Third_Id*
- *uint8_t Fourth_Id*

Field Documentation

- *uint8_t NAND_IDTypeDef::Maker_Id*
- *uint8_t NAND_IDTypeDef::Device_Id*
- *uint8_t NAND_IDTypeDef::Third_Id*
- *uint8_t NAND_IDTypeDef::Fourth_Id*

29.1.2 NAND_AddressTypeDef

NAND_AddressTypeDef is defined in the stm32f2xx_hal_nand.h

Data Fields

- *uint16_t Page*
- *uint16_t Plane*
- *uint16_t Block*

Field Documentation

- *uint16_t NAND_AddressTypeDef::Page*
NAND memory Page address
- *uint16_t NAND_AddressTypeDef::Plane*
NAND memory Zone address
- *uint16_t NAND_AddressTypeDef::Block*
NAND memory Block address

29.1.3 NAND_DeviceConfigTypeDef

NAND_DeviceConfigTypeDef is defined in the stm32f2xx_hal_nand.h

Data Fields

- *uint32_t PageSize*
- *uint32_t SpareAreaSize*
- *uint32_t BlockSize*
- *uint32_t BlockNbr*
- *uint32_t PlaneNbr*
- *uint32_t PlaneSize*
- *FunctionalState ExtraCommandEnable*

Field Documentation

- *uint32_t NAND_DeviceConfigTypeDef::PageSize*
NAND memory page (without spare area) size measured in bytes for 8 bits addressing or words for 16 bits addressing

- **`uint32_t NAND_DeviceConfigTypeDef::SpareAreaSize`**
NAND memory spare area size measured in bytes for 8 bits addressing or words for 16 bits addressing
- **`uint32_t NAND_DeviceConfigTypeDef::BlockSize`**
NAND memory block size measured in number of pages
- **`uint32_t NAND_DeviceConfigTypeDef::BlockNbr`**
NAND memory number of total blocks
- **`uint32_t NAND_DeviceConfigTypeDef::PlaneNbr`**
NAND memory number of planes
- **`uint32_t NAND_DeviceConfigTypeDef::PlaneSize`**
NAND memory zone size measured in number of blocks
- **`FunctionalState NAND_DeviceConfigTypeDef::ExtraCommandEnable`**
NAND extra command needed for Page reading mode. This parameter is mandatory for some NAND parts after the read command (NAND_CMD_AREA_TRUE1) and before DATA reading sequence. Example: Toshiba TH58BYG3S0HBAI6. This parameter could be ENABLE or DISABLE Please check the Read Mode sequence in the NAND device datasheet

29.1.4

NAND_HandleTypeDef

NAND_HandleTypeDef is defined in the `stm32f2xx_hal_nand.h`

Data Fields

- **`FSMC_NAND_TypeDef * Instance`**
- **`FSMC_NAND_InitTypeDef Init`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_NAND_StateTypeDef State`**
- **`NAND_DeviceConfigTypeDef Config`**

Field Documentation

- **`FSMC_NAND_TypeDef* NAND_HandleTypeDef::Instance`**
Register base address
- **`FSMC_NAND_InitTypeDef NAND_HandleTypeDef::Init`**
NAND device control configuration parameters
- **`HAL_LockTypeDef NAND_HandleTypeDef::Lock`**
NAND locking object
- **`__IO HAL_NAND_StateTypeDef NAND_HandleTypeDef::State`**
NAND device access state
- **`NAND_DeviceConfigTypeDef NAND_HandleTypeDef::Config`**
NAND physical characteristic information structure

29.2

NAND Firmware driver API description

The following section lists the various functions of the NAND library.

29.2.1

How to use this driver

This driver is a generic layered driver which contains a set of APIs used to control NAND flash memories. It uses the FSMC layer functions to interface with NAND devices. This driver is used as follows:

- NAND flash memory configuration sequence using the function `HAL_NAND_Init()` with control and timing parameters for both common and attribute spaces.
- Read NAND flash memory maker and device IDs using the function `HAL_NAND_Read_ID()`. The read information is stored in the `NAND_ID_TypeDef` structure declared by the function caller.

- Access NAND flash memory by read/write operations using the functions HAL_NAND_Read_Page_8b()/HAL_NAND_Read_SpareArea_8b(), HAL_NAND_Write_Page_8b()/HAL_NAND_Write_SpareArea_8b(), HAL_NAND_Read_Page_16b()/HAL_NAND_Read_SpareArea_16b(), HAL_NAND_Write_Page_16b()/HAL_NAND_Write_SpareArea_16b() to read/write page(s)/spare area(s). These functions use specific device information (Block, page size..) predefined by the user in the NAND_DeviceConfigTypeDef structure. The read/write address information is contained by the Nand_Address_Typedef structure passed as parameter.
- Perform NAND flash Reset chip operation using the function HAL_NAND_Reset().
- Perform NAND flash erase block operation using the function HAL_NAND_Erase_Block(). The erase block address information is contained in the Nand_Address_Typedef structure passed as parameter.
- Read the NAND flash status operation using the function HAL_NAND_Read_Status().
- You can also control the NAND device by calling the control APIs HAL_NAND_ECC_Enable()/HAL_NAND_ECC_Disable() to respectively enable/disable the ECC code correction feature or the function HAL_NAND_GetECC() to get the ECC correction code.
- You can monitor the NAND device HAL state by calling the function HAL_NAND_GetState()

Note: *This driver is a set of generic APIs which handle standard NAND flash operations. If a NAND flash device contains different operations and/or implementations, it should be implemented separately.*

Callback registration

The compilation define USE_HAL_NAND_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions @ref HAL_NAND_RegisterCallback() to register a user callback, it allows to register following callbacks:

- MspInitCallback : NAND MspInit.
- MspDeInitCallback : NAND MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function @ref HAL_NAND_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- MspInitCallback : NAND MspInit.
- MspDeInitCallback : NAND MspDeInit. This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the @ref HAL_NAND_Init and if the state is HAL_NAND_STATE_RESET all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for MspInit and MspDeInit callbacks that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_NAND_Init and @ref HAL_NAND_DeInit only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_NAND_Init and @ref HAL_NAND_DeInit keep and use the user MspInit/MspDeInit callbacks (registered beforehand) Callbacks can be registered/unregistered in READY state only. Exception done for MspInit/MspDeInit callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_NAND_RegisterCallback before calling @ref HAL_NAND_DeInit or @ref HAL_NAND_Init function. When The compilation define USE_HAL_NAND_REGISTER_CALLBACKS is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

29.2.2 NAND Initialization and de-initialization functions

This section provides functions allowing to initialize/de-initialize the NAND memory

This section contains the following APIs:

- [HAL_NAND_Init](#)
- [HAL_NAND_DeInit](#)
- [HAL_NAND_MspInit](#)
- [HAL_NAND_MspDeInit](#)
- [HAL_NAND_IRQHandler](#)
- [HAL_NAND_ITCallback](#)
- [HAL_NAND_ConfigDevice](#)
- [HAL_NAND_Read_ID](#)

29.2.3 NAND Input and Output functions

This section provides functions allowing to use and control the NAND memory

This section contains the following APIs:

- [HAL_NAND_Read_ID](#)
- [HAL_NAND_Reset](#)
- [HAL_NAND_ConfigDevice](#)
- [HAL_NAND_Read_Page_8b](#)
- [HAL_NAND_Read_Page_16b](#)
- [HAL_NAND_Write_Page_8b](#)
- [HAL_NAND_Write_Page_16b](#)
- [HAL_NAND_Read_SpareArea_8b](#)
- [HAL_NAND_Read_SpareArea_16b](#)
- [HAL_NAND_Write_SpareArea_8b](#)
- [HAL_NAND_Write_SpareArea_16b](#)
- [HAL_NAND_Erase_Block](#)
- [HAL_NAND_Address_Inc](#)

29.2.4 NAND Control functions

This subsection provides a set of functions allowing to control dynamically the NAND interface.

This section contains the following APIs:

- [HAL_NAND_ECC_Enable](#)
- [HAL_NAND_ECC_Disable](#)
- [HAL_NAND_GetECC](#)

29.2.5 NAND State functions

This subsection permits to get in run-time the status of the NAND controller and the data flow.

This section contains the following APIs:

- [HAL_NAND_GetState](#)
- [HAL_NAND_Read_Status](#)

29.2.6 Detailed description of functions

HAL_NAND_Init

Function name

```
HAL_StatusTypeDef HAL_NAND_Init (NAND_HandleTypeDef * hnand, FSMC_NAND_PCC_TimingTypeDef
* ComSpace_Timing, FSMC_NAND_PCC_TimingTypeDef * AttSpace_Timing)
```

Function description

Perform NAND memory Initialization sequence.

Parameters

- **hnand**: pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **ComSpace_Timing**: pointer to Common space timing structure
- **AttSpace_Timing**: pointer to Attribute space timing structure

Return values

- **HAL**: status

HAL_NAND_DeInit

Function name

```
HAL_StatusTypeDef HAL_NAND_DeInit (NAND_HandleTypeDef * hnand)
```

Function description

Perform NAND memory De-Initialization sequence.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL:** status

HAL_NAND_ConfigDevice

Function name

HAL_StatusTypeDef HAL_NAND_ConfigDevice (NAND_HandleTypeDef * hnand, NAND_DeviceConfigTypeDef * pDeviceConfig)

Function description

Configure the device: Enter the physical parameters of the device.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pDeviceConfig:** pointer to NAND_DeviceConfigTypeDef structure

Return values

- **HAL:** status

HAL_NAND_Read_ID

Function name

HAL_StatusTypeDef HAL_NAND_Read_ID (NAND_HandleTypeDef * hnand, NAND_IDTypeDef * pNAND_ID)

Function description

Read the NAND memory electronic signature.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pNAND_ID:** NAND ID structure

Return values

- **HAL:** status

HAL_NAND_Msplnit

Function name

void HAL_NAND_Msplnit (NAND_HandleTypeDef * hnand)

Function description

NAND MSP Init.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **None:**

HAL_NAND_MspDeInit

Function name

void HAL_NAND_MspDeInit (NAND_HandleTypeDef * hnand)

Function description

NAND MSP DeInit.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **None:**

HAL_NAND_IRQHandler

Function name

void HAL_NAND_IRQHandler (NAND_HandleTypeDef * hnand)

Function description

This function handles NAND device interrupt request.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL:** status

HAL_NAND_ITCallback

Function name

void HAL_NAND_ITCallback (NAND_HandleTypeDef * hnand)

Function description

NAND interrupt feature callback.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **None:**

HAL_NAND_Reset

Function name

HAL_StatusTypeDef HAL_NAND_Reset (NAND_HandleTypeDef * hnand)

Function description

NAND memory reset.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL:** status

HAL_NAND_Read_Page_8b

Function name

HAL_StatusTypeDef HAL_NAND_Read_Page_8b (NAND_HandleTypeDef * hnand, NAND_AddressTypeDef * pAddress, uint8_t * pBuffer, uint32_t NumPageToRead)

Function description

Read Page(s) from NAND memory block (8-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to destination read buffer
- **NumPageToRead:** number of pages to read from block

Return values

- **HAL:** status

HAL_NAND_Write_Page_8b

Function name

HAL_StatusTypeDef HAL_NAND_Write_Page_8b (NAND_HandleTypeDef * hnand, NAND_AddressTypeDef * pAddress, uint8_t * pBuffer, uint32_t NumPageToWrite)

Function description

Write Page(s) to NAND memory block (8-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write
- **NumPageToWrite:** number of pages to write to block

Return values

- **HAL:** status

HAL_NAND_Read_SpareArea_8b

Function name

HAL_StatusTypeDef HAL_NAND_Read_SpareArea_8b (NAND_HandleTypeDef * hnand, NAND_AddressTypeDef * pAddress, uint8_t * pBuffer, uint32_t NumSpareAreaToRead)

Function description

Read Spare area(s) from NAND memory (8-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write
- **NumSpareAreaToRead:** Number of spare area to read

Return values

- **HAL:** status

HAL_NAND_Write_SpareArea_8b

Function name

```
HAL_StatusTypeDef HAL_NAND_Write_SpareArea_8b (NAND_HandleTypeDef * hnand,
NAND_AddressTypeDef * pAddress, uint8_t * pBuffer, uint32_t NumSpareAreaTowrite)
```

Function description

Write Spare area(s) to NAND memory (8-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write
- **NumSpareAreaTowrite:** number of spare areas to write to block

Return values

- **HAL:** status

HAL_NAND_Read_Page_16b

Function name

```
HAL_StatusTypeDef HAL_NAND_Read_Page_16b (NAND_HandleTypeDef * hnand,
NAND_AddressTypeDef * pAddress, uint16_t * pBuffer, uint32_t NumPageToRead)
```

Function description

Read Page(s) from NAND memory block (16-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to destination read buffer. pBuffer should be 16bits aligned
- **NumPageToRead:** number of pages to read from block

Return values

- **HAL:** status

HAL_NAND_Write_Page_16b

Function name

```
HAL_StatusTypeDef HAL_NAND_Write_Page_16b (NAND_HandleTypeDef * hnand,
NAND_AddressTypeDef * pAddress, uint16_t * pBuffer, uint32_t NumPageToWrite)
```

Function description

Write Page(s) to NAND memory block (16-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write. pBuffer should be 16bits aligned
- **NumPageToWrite:** number of pages to write to block

Return values

- **HAL:** status

HAL_NAND_Read_SpareArea_16b

Function name

```
HAL_StatusTypeDef HAL_NAND_Read_SpareArea_16b (NAND_HandleTypeDef * hnand,
NAND_AddressTypeDef * pAddress, uint16_t * pBuffer, uint32_t NumSpareAreaToRead)
```

Function description

Read Spare area(s) from NAND memory (16-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write. pBuffer should be 16bits aligned.
- **NumSpareAreaToRead:** Number of spare area to read

Return values

- **HAL:** status

HAL_NAND_Write_SpareArea_16b

Function name

```
HAL_StatusTypeDef HAL_NAND_Write_SpareArea_16b (NAND_HandleTypeDef * hnand,
NAND_AddressTypeDef * pAddress, uint16_t * pBuffer, uint32_t NumSpareAreaTowrite)
```

Function description

Write Spare area(s) to NAND memory (16-bits addressing)

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure
- **pBuffer:** pointer to source buffer to write. pBuffer should be 16bits aligned.
- **NumSpareAreaTowrite:** number of spare areas to write to block

Return values

- **HAL:** status

HAL_NAND_Erase_Block

Function name

```
HAL_StatusTypeDef HAL_NAND_Erase_Block (NAND_HandleTypeDef * hnand, NAND_AddressTypeDef *
pAddress)
```

Function description

NAND memory Block erase.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure

Return values

- **HAL:** status

HAL_NAND_Address_Inc

Function name

```
uint32_t HAL_NAND_Address_Inc (NAND_HandleTypeDef * hnand, NAND_AddressTypeDef * pAddress)
```

Function description

Increment the NAND memory address.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **pAddress:** pointer to NAND address structure

Return values

- **The:** new status of the increment address operation. It can be:
 - NAND_VALID_ADDRESS: When the new address is valid address
 - NAND_INVALID_ADDRESS: When the new address is invalid address

HAL_NAND_ECC_Enable

Function name

```
HAL_StatusTypeDef HAL_NAND_ECC_Enable (NAND_HandleTypeDef * hnand)
```

Function description

Enables dynamically NAND ECC feature.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL:** status

HAL_NAND_ECC_Disable

Function name

```
HAL_StatusTypeDef HAL_NAND_ECC_Disable (NAND_HandleTypeDef * hnand)
```

Function description

Disables dynamically FSMC_NAND ECC feature.

Parameters

- **hnand:** pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL:** status

HAL_NAND_GetECC

Function name

HAL_StatusTypeDef HAL_NAND_GetECC (NAND_HandleTypeDef * hnd, uint32_t * ECCval, uint32_t Timeout)

Function description

Disables dynamically NAND ECC feature.

Parameters

- **hnd**: pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.
- **ECCval**: pointer to ECC value
- **Timeout**: maximum timeout to wait

Return values

- **HAL**: status

HAL_NAND_GetState

Function name

HAL_NAND_StateTypeDef HAL_NAND_GetState (NAND_HandleTypeDef * hnd)

Function description

return the NAND state

Parameters

- **hnd**: pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **HAL**: state

HAL_NAND_Read_Status

Function name

uint32_t HAL_NAND_Read_Status (NAND_HandleTypeDef * hnd)

Function description

NAND memory read status.

Parameters

- **hnd**: pointer to a NAND_HandleTypeDef structure that contains the configuration information for NAND module.

Return values

- **NAND**: status

29.3 NAND Firmware driver defines

The following section lists the various define and macros of the module.

29.3.1 NAND

NAND

NAND Exported Macros

`__HAL_NAND_RESET_HANDLE_STATE`

Description:

- Reset NAND handle state.

Parameters:

- `__HANDLE__`: specifies the NAND handle.

Return value:

- None

30 HAL NOR Generic Driver

30.1 NOR Firmware driver registers structures

30.1.1 NOR_IDTypeDef

NOR_IDTypeDef is defined in the stm32f2xx_hal_nor.h

Data Fields

- *uint16_t Manufacturer_Code*
- *uint16_t Device_Code1*
- *uint16_t Device_Code2*
- *uint16_t Device_Code3*

Field Documentation

- *uint16_t NOR_IDTypeDef::Manufacturer_Code*
Defines the device's manufacturer code used to identify the memory
- *uint16_t NOR_IDTypeDef::Device_Code1*
- *uint16_t NOR_IDTypeDef::Device_Code2*
- *uint16_t NOR_IDTypeDef::Device_Code3*
Defines the device's codes used to identify the memory. These codes can be accessed by performing read operations with specific control signals and addresses set. They can also be accessed by issuing an Auto Select command

30.1.2 NOR_CFITypeDef

NOR_CFITypeDef is defined in the stm32f2xx_hal_nor.h

Data Fields

- *uint16_t CFI_1*
- *uint16_t CFI_2*
- *uint16_t CFI_3*
- *uint16_t CFI_4*

Field Documentation

- *uint16_t NOR_CFITypeDef::CFI_1*
< Defines the information stored in the memory's Common flash interface which contains a description of various electrical and timing parameters, density information and functions supported by the memory
- *uint16_t NOR_CFITypeDef::CFI_2*
- *uint16_t NOR_CFITypeDef::CFI_3*
- *uint16_t NOR_CFITypeDef::CFI_4*

30.1.3 NOR_HandleTypeDef

NOR_HandleTypeDef is defined in the stm32f2xx_hal_nor.h

Data Fields

- *FSMC_NORSRAM_TypeDef * Instance*
- *FSMC_NORSRAM_EXTENDED_TypeDef * Extended*
- *FSMC_NORSRAM_InitTypeDef Init*
- *HAL_LockTypeDef Lock*
- *__IO HAL_NOR_StateTypeDef State*

Field Documentation

- *FSMC_NORSRAM_TypeDef* NOR_HandleTypeDef::Instance*
Register base address

- **FSMC_NORSRAM_EXTENDED_TypeDef* NOR_HandleTypeDef::Extended**
Extended mode register base address
- **FSMC_NORSRAM_InitTypeDef NOR_HandleTypeDef::Init**
NOR device control configuration parameters
- **HAL_LockTypeDef NOR_HandleTypeDef::Lock**
NOR locking object
- **__IO HAL_NOR_StateTypeDef NOR_HandleTypeDef::State**
NOR device access state

30.2 NOR Firmware driver API description

The following section lists the various functions of the NOR library.

30.2.1 How to use this driver

This driver is a generic layered driver which contains a set of APIs used to control NOR flash memories. It uses the FSMC layer functions to interface with NOR devices. This driver is used as follows:

- NOR flash memory configuration sequence using the function HAL_NOR_Init() with control and timing parameters for both normal and extended mode.
- Read NOR flash memory manufacturer code and device IDs using the function HAL_NOR_Read_ID(). The read information is stored in the NOR_ID_TypeDef structure declared by the function caller.
- Access NOR flash memory by read/write data unit operations using the functions HAL_NOR_Read(), HAL_NOR_Program().
- Perform NOR flash erase block/chip operations using the functions HAL_NOR_Erase_Block() and HAL_NOR_Erase_Chip().
- Read the NOR flash CFI (common flash interface) IDs using the function HAL_NOR_Read_CFI(). The read information is stored in the NOR_CFI_TypeDef structure declared by the function caller.
- You can also control the NOR device by calling the control APIs HAL_NOR_WriteOperation_Enable()/HAL_NOR_WriteOperation_Disable() to respectively enable/disable the NOR write operation
- You can monitor the NOR device HAL state by calling the function HAL_NOR_GetState()

Note: *This driver is a set of generic APIs which handle standard NOR flash operations. If a NOR flash device contains different operations and/or implementations, it should be implemented separately.*

NOR HAL driver macros list

Below the list of most used macros in NOR HAL driver.

- **NOR_WRITE** : NOR memory write data to specified address

Callback registration

The compilation define USE_HAL_NOR_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions @ref HAL_NOR_RegisterCallback() to register a user callback, it allows to register following callbacks:

- MspInitCallback : NOR MspInit.
- MspDeInitCallback : NOR MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function @ref HAL_NOR_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- MspInitCallback : NOR MspInit.

- **MspDeInitCallback** : NOR MspDeInit. This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the @ref HAL_NOR_Init and if the state is HAL_NOR_STATE_RESET all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for MspInit and MspDeInit callbacks that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_NOR_Init and @ref HAL_NOR_DeInit only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_NOR_Init and @ref HAL_NOR_DeInit keep and use the user MspInit/MspDeInit callbacks (registered beforehand) Callbacks can be registered/unregistered in READY state only. Exception done for MspInit/MspDeInit callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_NOR_RegisterCallback before calling @ref HAL_NOR_DeInit or @ref HAL_NOR_Init function. When The compilation define USE_HAL_NOR_REGISTER_CALLBACKS is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

30.2.2 NOR Initialization and de_initialization functions

This section provides functions allowing to initialize/de-initialize the NOR memory

This section contains the following APIs:

- [HAL_NOR_Init](#)
- [HAL_NOR_DeInit](#)
- [HAL_NOR_MspInit](#)
- [HAL_NOR_MspDeInit](#)
- [HAL_NOR_MspWait](#)

30.2.3 NOR Input and Output functions

This section provides functions allowing to use and control the NOR memory

This section contains the following APIs:

- [HAL_NOR_Read_ID](#)
- [HAL_NOR_ReturnToReadMode](#)
- [HAL_NOR_Read](#)
- [HAL_NOR_Program](#)
- [HAL_NOR_ReadBuffer](#)
- [HAL_NOR_ProgramBuffer](#)
- [HAL_NOR_Erase_Block](#)
- [HAL_NOR_Erase_Chip](#)
- [HAL_NOR_Read_CFI](#)

30.2.4 NOR Control functions

This subsection provides a set of functions allowing to control dynamically the NOR interface.

This section contains the following APIs:

- [HAL_NOR_WriteOperation_Enable](#)
- [HAL_NOR_WriteOperation_Disable](#)

30.2.5 NOR State functions

This subsection permits to get in run-time the status of the NOR controller and the data flow.

This section contains the following APIs:

- [HAL_NOR_GetState](#)
- [HAL_NOR_GetStatus](#)

30.2.6 Detailed description of functions

HAL_NOR_Init

Function name

HAL_StatusTypeDef HAL_NOR_Init (NOR_HandleTypeDef * hnor, FSMC_NORSRAM_TimingTypeDef * Timing, FSMC_NORSRAM_TimingTypeDef * ExtTiming)

Function description

Perform the NOR memory Initialization sequence.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **Timing**: pointer to NOR control timing structure
- **ExtTiming**: pointer to NOR extended mode timing structure

Return values

- **HAL**: status

HAL_NOR_DeInit

Function name

HAL_StatusTypeDef HAL_NOR_DeInit (NOR_HandleTypeDef * hnor)

Function description

Perform NOR memory De-Initialization sequence.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **HAL**: status

HAL_NOR_MspInit

Function name

void HAL_NOR_MspInit (NOR_HandleTypeDef * hnor)

Function description

NOR MSP Init.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **None**:

HAL_NOR_MspDeInit

Function name

void HAL_NOR_MspDeInit (NOR_HandleTypeDef * hnor)

Function description

NOR MSP DeInit.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **None**:

HAL_NOR_MspWait

Function name

void HAL_NOR_MspWait (NOR_HandleTypeDef * hnor, uint32_t Timeout)

Function description

NOR MSP Wait for Ready/Busy signal.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **Timeout**: Maximum timeout value

Return values

- **None**:

HAL_NOR_Read_ID

Function name

HAL_StatusTypeDef HAL_NOR_Read_ID (NOR_HandleTypeDef * hnor, NOR_IDTypeDef * pNOR_ID)

Function description

Read NOR flash IDs.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **pNOR_ID**: pointer to NOR ID structure

Return values

- **HAL**: status

HAL_NOR_ReturnToReadMode

Function name

HAL_StatusTypeDef HAL_NOR_ReturnToReadMode (NOR_HandleTypeDef * hnor)

Function description

Returns the NOR memory to Read mode.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **HAL**: status

HAL_NOR_Read

Function name

HAL_StatusTypeDef HAL_NOR_Read (NOR_HandleTypeDef * hnor, uint32_t * pAddress, uint16_t * pData)

Function description

Read data from NOR memory.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **pAddress**: pointer to Device address
- **pData**: pointer to read data

Return values

- **HAL**: status

HAL_NOR_Program

Function name

HAL_StatusTypeDef HAL_NOR_Program (NOR_HandleTypeDef * hnor, uint32_t * pAddress, uint16_t * pData)

Function description

Program data to NOR memory.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **pAddress**: Device address
- **pData**: pointer to the data to write

Return values

- **HAL**: status

HAL_NOR_ReadBuffer

Function name

HAL_StatusTypeDef HAL_NOR_ReadBuffer (NOR_HandleTypeDef * hnor, uint32_t uwAddress, uint16_t * pData, uint32_t uwBufferSize)

Function description

Reads a half-word buffer from the NOR memory.

Parameters

- **hnor**: pointer to the NOR handle
- **uwAddress**: NOR memory internal address to read from.
- **pData**: pointer to the buffer that receives the data read from the NOR memory.
- **uwBufferSize**: number of Half word to read.

Return values

- **HAL**: status

HAL_NOR_ProgramBuffer

Function name

HAL_StatusTypeDef HAL_NOR_ProgramBuffer (NOR_HandleTypeDef * hnor, uint32_t uwAddress, uint16_t * pData, uint32_t uwBufferSize)

Function description

Writes a half-word buffer to the NOR memory.

Parameters

- **hnor**: pointer to the NOR handle
- **uwAddress**: NOR memory internal start write address
- **pData**: pointer to source data buffer.
- **uwBufferSize**: Size of the buffer to write

Return values

- **HAL**: status

HAL_NOR_Erase_Block

Function name

HAL_StatusTypeDef HAL_NOR_Erase_Block (NOR_HandleTypeDef * hnor, uint32_t BlockAddress, uint32_t Address)

Function description

Erase the specified block of the NOR memory.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **BlockAddress**: Block to erase address
- **Address**: Device address

Return values

- **HAL**: status

HAL_NOR_Erase_Chip

Function name

HAL_StatusTypeDef HAL_NOR_Erase_Chip (NOR_HandleTypeDef * hnor, uint32_t Address)

Function description

Erase the entire NOR chip.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **Address**: Device address

Return values

- **HAL**: status

HAL_NOR_Read_CFI

Function name

HAL_StatusTypeDef HAL_NOR_Read_CFI (NOR_HandleTypeDef * hnor, NOR_CFITypeDef * pNOR_CFI)

Function description

Read NOR flash CFI IDs.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **pNOR_CFI**: pointer to NOR CFI IDs structure

Return values

- **HAL**: status

HAL_NOR_WriteOperation_Enable

Function name

HAL_StatusTypeDef HAL_NOR_WriteOperation_Enable (NOR_HandleTypeDef * hnor)

Function description

Enables dynamically NOR write operation.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **HAL**: status

HAL_NOR_WriteOperation_Disable

Function name

HAL_StatusTypeDef HAL_NOR_WriteOperation_Disable (NOR_HandleTypeDef * hnor)

Function description

Disables dynamically NOR write operation.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **HAL**: status

HAL_NOR_GetState

Function name

HAL_NOR_StateTypeDef HAL_NOR_GetState (NOR_HandleTypeDef * hnor)

Function description

return the NOR controller state

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.

Return values

- **NOR**: controller state

HAL_NOR_GetStatus

Function name

HAL_NOR_StatusTypeDef HAL_NOR_GetStatus (NOR_HandleTypeDef * hnor, uint32_t Address, uint32_t Timeout)

Function description

Returns the NOR operation status.

Parameters

- **hnor**: pointer to a NOR_HandleTypeDef structure that contains the configuration information for NOR module.
- **Address**: Device address
- **Timeout**: NOR programming Timeout

Return values

- **NOR_Status**: The returned value can be: HAL_NOR_STATUS_SUCCESS, HAL_NOR_STATUS_ERROR or HAL_NOR_STATUS_TIMEOUT

30.3 NOR Firmware driver defines

The following section lists the various define and macros of the module.

30.3.1 NOR

NOR

NOR Exported Macros

__HAL_NOR_RESET_HANDLE_STATE

Description:

- Reset NOR handle state.

Parameters:

- **__HANDLE__**: specifies the NOR handle.

Return value:

- None

31 HAL PCCARD Generic Driver

31.1 PCCARD Firmware driver registers structures

31.1.1 PCCARD_HandleTypeDef

PCCARD_HandleTypeDef is defined in the stm32f2xx_hal_pccard.h

Data Fields

- **FSMC_PCCARD_TypeDef * Instance**
- **FSMC_PCCARD_InitTypeDef Init**
- **__IO HAL_PCCARD_StateTypeDef State**
- **HAL_LockTypeDef Lock**

Field Documentation

- **FSMC_PCCARD_TypeDef* PCCARD_HandleTypeDef::Instance**
Register base address for PCCARD device
- **FSMC_PCCARD_InitTypeDef PCCARD_HandleTypeDef::Init**
PCCARD device control configuration parameters
- **__IO HAL_PCCARD_StateTypeDef PCCARD_HandleTypeDef::State**
PCCARD device access state
- **HAL_LockTypeDef PCCARD_HandleTypeDef::Lock**
PCCARD Lock

31.2 PCCARD Firmware driver API description

The following section lists the various functions of the PCCARD library.

31.2.1 How to use this driver

This driver is a generic layered driver which contains a set of APIs used to control PCCARD/compact flash memories. It uses the FSMC layer functions to interface with PCCARD devices. This driver is used for:

- PCCARD/Compact Flash memory configuration sequence using the function HAL_PCCARD_Init()/ HAL_CF_Init() with control and timing parameters for both common and attribute spaces.
- Read PCCARD/Compact Flash memory maker and device IDs using the function HAL_PCCARD_Read_ID()/HAL_CF_Read_ID(). The read information is stored in the CompactFlash_ID structure declared by the function caller.
- Access PCCARD/Compact Flash memory by read/write operations using the functions HAL_PCCARD_Read_Sector()/ HAL_PCCARD_Write_Sector() - HAL_CF_Read_Sector()/ HAL_CF_Write_Sector(), to read/write sector.
- Perform PCCARD/Compact Flash Reset chip operation using the function HAL_PCCARD_Reset()/ HAL_CF_Reset.
- Perform PCCARD/Compact Flash erase sector operation using the function HAL_PCCARD_Erase_Sector()/ HAL_CF_Erase_Sector.
- Read the PCCARD/Compact Flash status operation using the function HAL_PCCARD_ReadStatus()/ HAL_CF_ReadStatus().
- You can monitor the PCCARD/Compact Flash device HAL state by calling the function HAL_PCCARD_GetState()/HAL_CF_GetState()

Note: *This driver is a set of generic APIs which handle standard PCCARD/compact flash operations. If a PCCARD/ Compact Flash device contains different operations and/or implementations, it should be implemented separately.*

Callback registration

The compilation define `USE_HAL_PCCARD_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `@ref HAL_PCCARD_RegisterCallback()` to register a user callback, it allows to register following callbacks:

- `MspInitCallback` : PCCARD `MspInit`.
- `MspDeInitCallback` : PCCARD `MspDeInit`. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function `@ref HAL_PCCARD_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- `MspInitCallback` : PCCARD `MspInit`.
- `MspDeInitCallback` : PCCARD `MspDeInit`. This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the `@ref HAL_PCCARD_Init` and if the state is `HAL_PCCARD_STATE_RESET` all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for `MspInit` and `MspDeInit` callbacks that are respectively reset to the legacy weak (surcharged) functions in the `@ref HAL_PCCARD_Init` and `@ref HAL_PCCARD_DeInit` only when these callbacks are null (not registered beforehand). If not, `MspInit` or `MspDeInit` are not null, the `@ref HAL_PCCARD_Init` and `@ref HAL_PCCARD_DeInit` keep and use the user `MspInit`/`MspDeInit` callbacks (registered beforehand) Callbacks can be registered/unregistered in `READY` state only. Exception done for `MspInit`/`MspDeInit` callbacks that can be registered/unregistered in `READY` or `RESET` state, thus registered (user) `MspInit`/`DeInit` callbacks can be used during the `Init`/`DeInit`. In that case first register the `MspInit`/`MspDeInit` user callbacks using `@ref HAL_PCCARD_RegisterCallback` before calling `@ref HAL_PCCARD_DeInit` or `@ref HAL_PCCARD_Init` function. When The compilation define `USE_HAL_PCCARD_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

31.2.2 PCCARD Initialization and de-initialization functions

This section provides functions allowing to initialize/de-initialize the PCCARD memory

This section contains the following APIs:

- [`HAL\_PCCARD\_Init`](#)
- [`HAL\_PCCARD\_DeInit`](#)
- [`HAL\_PCCARD\_MspInit`](#)
- [`HAL\_PCCARD\_MspDeInit`](#)

31.2.3 PCCARD Input and Output functions

This section provides functions allowing to use and control the PCCARD memory

This section contains the following APIs:

- [`HAL\_PCCARD\_Read\_ID`](#)
- [`HAL\_PCCARD\_Read\_Sector`](#)
- [`HAL\_PCCARD\_Write\_Sector`](#)
- [`HAL\_PCCARD\_Erase\_Sector`](#)
- [`HAL\_PCCARD\_Reset`](#)
- [`HAL\_PCCARD\_IRQHandler`](#)
- [`HAL\_PCCARD\_ITCallback`](#)

31.2.4 PCCARD State functions

This subsection permits to get in run-time the status of the PCCARD controller and the data flow.

This section contains the following APIs:

- [`HAL\_PCCARD\_GetState`](#)
- [`HAL\_PCCARD\_GetStatus`](#)
- [`HAL\_PCCARD\_ReadStatus`](#)

31.2.5 Detailed description of functions

HAL_PCCARD_Init

Function name

```
HAL_StatusTypeDef HAL_PCCARD_Init (PCCARD_HandleTypeDef * hpccard,
FSMC_NAND_PCC_TimingTypeDef * ComSpaceTiming, FSMC_NAND_PCC_TimingTypeDef *
AttSpaceTiming, FSMC_NAND_PCC_TimingTypeDef * IOSpaceTiming)
```

Function description

Perform the PCCARD memory Initialization sequence.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.
- **ComSpaceTiming**: Common space timing structure
- **AttSpaceTiming**: Attribute space timing structure
- **IOSpaceTiming**: IO space timing structure

Return values

- **HAL**: status

HAL_PCCARD_DeInit

Function name

```
HAL_StatusTypeDef HAL_PCCARD_DeInit (PCCARD_HandleTypeDef * hpccard)
```

Function description

Perform the PCCARD memory De-initialization sequence.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **HAL**: status

HAL_PCCARD_MspInit

Function name

```
void HAL_PCCARD_MspInit (PCCARD_HandleTypeDef * hpccard)
```

Function description

PCCARD MSP Init.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **None**:

HAL_PCCARD_MspDeInit

Function name

```
void HAL_PCCARD_MspDeInit (PCCARD_HandleTypeDef * hpccard)
```

Function description

PCCARD MSP DeInit.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **None**:

HAL_PCCARD_Read_ID

Function name

HAL_StatusTypeDef HAL_PCCARD_Read_ID (PCCARD_HandleTypeDef * hpccard, uint8_t CompactFlash_ID, uint8_t * pStatus)

Function description

Read Compact Flash's ID.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.
- **CompactFlash_ID**: Compact flash ID structure.
- **pStatus**: pointer to compact flash status

Return values

- **HAL**: status

HAL_PCCARD_Write_Sector

Function name

HAL_StatusTypeDef HAL_PCCARD_Write_Sector (PCCARD_HandleTypeDef * hpccard, uint16_t * pBuffer, uint16_t SectorAddress, uint8_t * pStatus)

Function description

Write sector to PCCARD memory.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.
- **pBuffer**: pointer to source write buffer
- **SectorAddress**: Sector address to write
- **pStatus**: pointer to PCCARD status

Return values

- **HAL**: status

HAL_PCCARD_Read_Sector

Function name

HAL_StatusTypeDef HAL_PCCARD_Read_Sector (PCCARD_HandleTypeDef * hpccard, uint16_t * pBuffer, uint16_t SectorAddress, uint8_t * pStatus)

Function description

Read sector from PCCARD memory.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.
- **pBuffer**: pointer to destination read buffer
- **SectorAddress**: Sector address to read
- **pStatus**: pointer to PCCARD status

Return values

- **HAL**: status

HAL_PCCARD_Erase_Sector

Function name

HAL_StatusTypeDef HAL_PCCARD_Erase_Sector (PCCARD_HandleTypeDef * hpccard, uint16_t SectorAddress, uint8_t * pStatus)

Function description

Erase sector from PCCARD memory.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.
- **SectorAddress**: Sector address to erase
- **pStatus**: pointer to PCCARD status

Return values

- **HAL**: status

HAL_PCCARD_Reset

Function name

HAL_StatusTypeDef HAL_PCCARD_Reset (PCCARD_HandleTypeDef * hpccard)

Function description

Reset the PCCARD memory.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **HAL**: status

HAL_PCCARD_IRQHandler

Function name

void HAL_PCCARD_IRQHandler (PCCARD_HandleTypeDef * hpccard)

Function description

This function handles PCCARD device interrupt request.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **HAL:** status

HAL_PCCARD_ITCallback

Function name

```
void HAL_PCCARD_ITCallback (PCCARD_HandleTypeDef * hpccard)
```

Function description

PCCARD interrupt feature callback.

Parameters

- **hpccard:** pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **None:**

HAL_PCCARD_GetState

Function name

```
HAL_PCCARD_StateTypeDef HAL_PCCARD_GetState (PCCARD_HandleTypeDef * hpccard)
```

Function description

return the PCCARD controller state

Parameters

- **hpccard:** pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **HAL:** state

HAL_PCCARD_GetStatus

Function name

```
HAL_PCCARD_StatusTypeDef HAL_PCCARD_GetStatus (PCCARD_HandleTypeDef * hpccard)
```

Function description

Get the compact flash memory status.

Parameters

- **hpccard:** pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **New:** status of the PCCARD operation. This parameter can be:
 - CompactFlash_TIMEOUT_ERROR: when the previous operation generate a Timeout error
 - CompactFlash_READY: when memory is ready for the next operation

HAL_PCCARD_ReadStatus

Function name

```
HAL_PCCARD_StatusTypeDef HAL_PCCARD_ReadStatus (PCCARD_HandleTypeDef * hpccard)
```

Function description

Reads the Compact Flash memory status using the Read status command.

Parameters

- **hpccard**: pointer to a PCCARD_HandleTypeDef structure that contains the configuration information for PCCARD module.

Return values

- **The**: status of the Compact Flash memory. This parameter can be:
 - CompactFlash_BUSY: when memory is busy
 - CompactFlash_READY: when memory is ready for the next operation
 - CompactFlash_ERROR: when the previous operation generates error

31.3 PCCARD Firmware driver defines

The following section lists the various define and macros of the module.

31.3.1 PCCARD

PCCARD

PCCARD Exported Macros

__HAL_PCCARD_RESET_HANDLE_STATE

Description:

- Reset PCCARD handle state.

Parameters:

- **__HANDLE__**: specifies the PCCARD handle.

Return value:

- None

32 HAL PCD Generic Driver

32.1 PCD Firmware driver registers structures

32.1.1 PCD_HandleTypeDef

PCD_HandleTypeDef is defined in the `stm32f2xx_hal_pcd.h`

Data Fields

- *PCD_TypeDef * Instance*
- *PCD_InitTypeDef Init*
- *__IO uint8_t USB_Address*
- *PCD_EPTTypeDef IN_ep*
- *PCD_EPTTypeDef OUT_ep*
- *HAL_LockTypeDef Lock*
- *__IO PCD_StateTypeDef State*
- *__IO uint32_t ErrorCode*
- *uint32_t Setup*
- *PCD_LPM_StateTypeDef LPM_State*
- *uint32_t BESL*
- *void * pData*

Field Documentation

- *PCD_TypeDef* PCD_HandleTypeDef::Instance*
Register base address
- *PCD_InitTypeDef PCD_HandleTypeDef::Init*
PCD required parameters
- *__IO uint8_t PCD_HandleTypeDef::USB_Address*
USB Address
- *PCD_EPTTypeDef PCD_HandleTypeDef::IN_ep[16]*
IN endpoint parameters
- *PCD_EPTTypeDef PCD_HandleTypeDef::OUT_ep[16]*
OUT endpoint parameters
- *HAL_LockTypeDef PCD_HandleTypeDef::Lock*
PCD peripheral status
- *__IO PCD_StateTypeDef PCD_HandleTypeDef::State*
PCD communication state
- *__IO uint32_t PCD_HandleTypeDef::ErrorCode*
PCD Error code
- *uint32_t PCD_HandleTypeDef::Setup[12]*
Setup packet buffer
- *PCD_LPM_StateTypeDef PCD_HandleTypeDef::LPM_State*
LPM State
- *uint32_t PCD_HandleTypeDef::BESL*
- *void* PCD_HandleTypeDef::pData*
Pointer to upper stack Handler

32.2 PCD Firmware driver API description

The following section lists the various functions of the PCD library.

32.2.1 How to use this driver

The PCD HAL driver can be used as follows:

1. Declare a PCD_HandleTypeDef handle structure, for example: PCD_HandleTypeDef hpcd;
2. Fill parameters of Init structure in HCD handle
3. Call HAL_PCD_Init() API to initialize the PCD peripheral (Core, Device core, ...)
4. Initialize the PCD low level resources through the HAL_PCD_MspInit() API:
 - a. Enable the PCD/USB Low Level interface clock using
 - __HAL_RCC_USB_OTG_FS_CLK_ENABLE();
 - __HAL_RCC_USB_OTG_HS_CLK_ENABLE(); (For High Speed Mode)
 - b. Initialize the related GPIO clocks
 - c. Configure PCD pin-out
 - d. Configure PCD NVIC interrupt
5. Associate the Upper USB device stack to the HAL PCD Driver:
 - a. hpcd.pData = pdev;
6. Enable PCD transmission and reception:
 - a. HAL_PCD_Start();

32.2.2 Initialization and de-initialization functions

This section provides functions allowing to:

This section contains the following APIs:

- [HAL_PCD_Init](#)
- [HAL_PCD_DeInit](#)
- [HAL_PCD_MspInit](#)
- [HAL_PCD_MspDeInit](#)

32.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the PCD data transfers.

This section contains the following APIs:

- [HAL_PCD_Start](#)
- [HAL_PCD_Stop](#)
- [HAL_PCD_IRQHandler](#)
- [HAL_PCD_DataOutStageCallback](#)
- [HAL_PCD_DataInStageCallback](#)
- [HAL_PCD_SetupStageCallback](#)
- [HAL_PCD_SOFCallback](#)
- [HAL_PCD_ResetCallback](#)
- [HAL_PCD_SuspendCallback](#)
- [HAL_PCD_ResumeCallback](#)
- [HAL_PCD_ISOOUTIncompleteCallback](#)
- [HAL_PCD_ISOINIncompleteCallback](#)
- [HAL_PCD_ConnectCallback](#)
- [HAL_PCD_DisconnectCallback](#)

32.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the PCD data transfers.

This section contains the following APIs:

- [HAL_PCD_DevConnect](#)
- [HAL_PCD_DevDisconnect](#)
- [HAL_PCD_SetAddress](#)
- [HAL_PCD_EP_Open](#)

- [HAL_PCD_EP_Close](#)
- [HAL_PCD_EP_Receive](#)
- [HAL_PCD_EP_GetRxCount](#)
- [HAL_PCD_EP_Transmit](#)
- [HAL_PCD_EP_SetStall](#)
- [HAL_PCD_EP_ClrStall](#)
- [HAL_PCD_EP_Flush](#)
- [HAL_PCD_ActivateRemoteWakeup](#)
- [HAL_PCD_DeActivateRemoteWakeup](#)

32.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [HAL_PCD_GetState](#)

32.2.6 Detailed description of functions

HAL_PCD_Init

Function name

HAL_StatusTypeDef HAL_PCD_Init (PCD_HandleTypeDef * hpcd)

Function description

Initializes the PCD according to the specified parameters in the PCD_InitTypeDef and initialize the associated handle.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_DeInit

Function name

HAL_StatusTypeDef HAL_PCD_DeInit (PCD_HandleTypeDef * hpcd)

Function description

DeInitializes the PCD peripheral.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_MspInit

Function name

void HAL_PCD_MspInit (PCD_HandleTypeDef * hpcd)

Function description

Initializes the PCD MSP.

Parameters

- **hpcd**: PCD handle

Return values

- **None:**

HAL_PCD_MspDeInit

Function name

void HAL_PCD_MspDeInit (PCD_HandleTypeDef * hpcd)

Function description

DeInitializes PCD MSP.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_Start

Function name

HAL_StatusTypeDef HAL_PCD_Start (PCD_HandleTypeDef * hpcd)

Function description

Start the USB device.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_Stop

Function name

HAL_StatusTypeDef HAL_PCD_Stop (PCD_HandleTypeDef * hpcd)

Function description

Stop the USB device.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_IRQHandler

Function name

void HAL_PCD_IRQHandler (PCD_HandleTypeDef * hpcd)

Function description

Handles PCD interrupt request.

Parameters

- **hpcd:** PCD handle

Return values

- **HAL:** status

HAL_PCD_SOFCallback

Function name

void HAL_PCD_SOFCallback (PCD_HandleTypeDef * hpcd)

Function description

USB Start Of Frame callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_SetupStageCallback

Function name

void HAL_PCD_SetupStageCallback (PCD_HandleTypeDef * hpcd)

Function description

Setup stage callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_ResetCallback

Function name

void HAL_PCD_ResetCallback (PCD_HandleTypeDef * hpcd)

Function description

USB Reset callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_SuspendCallback

Function name

void HAL_PCD_SuspendCallback (PCD_HandleTypeDef * hpcd)

Function description

Suspend event callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_ResumeCallback

Function name

void HAL_PCD_ResumeCallback (PCD_HandleTypeDef * hpcd)

Function description

Resume event callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_ConnectCallback

Function name

void HAL_PCD_ConnectCallback (PCD_HandleTypeDef * hpcd)

Function description

Connection event callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_DisconnectCallback

Function name

void HAL_PCD_DisconnectCallback (PCD_HandleTypeDef * hpcd)

Function description

Disconnection event callback.

Parameters

- **hpcd:** PCD handle

Return values

- **None:**

HAL_PCD_DataOutStageCallback

Function name

void HAL_PCD_DataOutStageCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)

Function description

Data OUT stage callback.

Parameters

- **hpcd:** PCD handle
- **epnum:** endpoint number

Return values

- **None:**

HAL_PCD_DataInStageCallback

Function name

void HAL_PCD_DataInStageCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)

Function description

Data IN stage callback.

Parameters

- **hpcd:** PCD handle
- **epnum:** endpoint number

Return values

- **None:**

HAL_PCD_ISOOUTIncompleteCallback

Function name

void HAL_PCD_ISOOUTIncompleteCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)

Function description

Incomplete ISO OUT callback.

Parameters

- **hpcd:** PCD handle
- **epnum:** endpoint number

Return values

- **None:**

HAL_PCD_ISOINIncompleteCallback

Function name

void HAL_PCD_ISOINIncompleteCallback (PCD_HandleTypeDef * hpcd, uint8_t epnum)

Function description

Incomplete ISO IN callback.

Parameters

- **hpcd:** PCD handle
- **epnum:** endpoint number

Return values

- **None:**

HAL_PCD_DevConnect

Function name

HAL_StatusTypeDef HAL_PCD_DevConnect (PCD_HandleTypeDef * hpcd)

Function description

Connect the USB device.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_DevDisconnect

Function name

```
HAL_StatusTypeDef HAL_PCD_DevDisconnect (PCD_HandleTypeDef * hpcd)
```

Function description

Disconnect the USB device.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_SetAddress

Function name

```
HAL_StatusTypeDef HAL_PCD_SetAddress (PCD_HandleTypeDef * hpcd, uint8_t address)
```

Function description

Set the USB Device address.

Parameters

- **hpcd**: PCD handle
- **address**: new device address

Return values

- **HAL**: status

HAL_PCD_EP_Open

Function name

```
HAL_StatusTypeDef HAL_PCD_EP_Open (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint16_t ep_mps, uint8_t ep_type)
```

Function description

Open and configure an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **ep_mps**: endpoint max packet size
- **ep_type**: endpoint type

Return values

- **HAL**: status

HAL_PCD_EP_Close

Function name

```
HAL_StatusTypeDef HAL_PCD_EP_Close (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)
```

Function description

Deactivate an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_EP_Receive

Function name

```
HAL_StatusTypeDef HAL_PCD_EP_Receive (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint8_t * pBuf, uint32_t len)
```

Function description

Receive an amount of data.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **pBuf**: pointer to the reception buffer
- **len**: amount of data to be received

Return values

- **HAL**: status

HAL_PCD_EP_Transmit

Function name

```
HAL_StatusTypeDef HAL_PCD_EP_Transmit (PCD_HandleTypeDef * hpcd, uint8_t ep_addr, uint8_t * pBuf, uint32_t len)
```

Function description

Send an amount of data.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address
- **pBuf**: pointer to the transmission buffer
- **len**: amount of data to be sent

Return values

- **HAL**: status

HAL_PCD_EP_GetRxCount

Function name

```
uint32_t HAL_PCD_EP_GetRxCount (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)
```

Function description

Get Received Data Size.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **Data**: Size

HAL_PCD_EP_SetStall

Function name

HAL_StatusTypeDef HAL_PCD_EP_SetStall (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Set a STALL condition over an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_EP_ClrStall

Function name

HAL_StatusTypeDef HAL_PCD_EP_ClrStall (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Clear a STALL condition over in an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_EP_Flush

Function name

HAL_StatusTypeDef HAL_PCD_EP_Flush (PCD_HandleTypeDef * hpcd, uint8_t ep_addr)

Function description

Flush an endpoint.

Parameters

- **hpcd**: PCD handle
- **ep_addr**: endpoint address

Return values

- **HAL**: status

HAL_PCD_ActivateRemoteWakeup

Function name

HAL_StatusTypeDef HAL_PCD_ActivateRemoteWakeup (PCD_HandleTypeDef * hpcd)

Function description

Activate remote wakeup signalling.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_DeActivateRemoteWakeup

Function name

HAL_StatusTypeDef HAL_PCD_DeActivateRemoteWakeup (PCD_HandleTypeDef * hpcd)

Function description

De-activate remote wakeup signalling.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: status

HAL_PCD_GetState

Function name

PCD_StateTypeDef HAL_PCD_GetState (PCD_HandleTypeDef * hpcd)

Function description

Return the PCD handle state.

Parameters

- **hpcd**: PCD handle

Return values

- **HAL**: state

32.3 PCD Firmware driver defines

The following section lists the various define and macros of the module.

32.3.1 PCD

PCD

PCD Exported Macros

__HAL_PCD_ENABLE

__HAL_PCD_DISABLE

__HAL_PCD_GET_FLAG

__HAL_PCD_CLEAR_FLAG

__HAL_PCD_IS_INVALID_INTERRUPT

__HAL_PCD_UNGATE_PHYCLOCK

__HAL_PCD_GATE_PHYCLOCK

__HAL_PCD_IS_PHY_SUSPENDED

__HAL_USB_OTG_HS_WAKEUP_EXTI_ENABLE_IT

__HAL_USB_OTG_HS_WAKEUP_EXTI_DISABLE_IT

__HAL_USB_OTG_HS_WAKEUP_EXTI_GET_FLAG

__HAL_USB_OTG_HS_WAKEUP_EXTI_CLEAR_FLAG

__HAL_USB_OTG_HS_WAKEUP_EXTI_ENABLE_RISING_EDGE

__HAL_USB_OTG_FS_WAKEUP_EXTI_ENABLE_IT

__HAL_USB_OTG_FS_WAKEUP_EXTI_DISABLE_IT

__HAL_USB_OTG_FS_WAKEUP_EXTI_GET_FLAG

__HAL_USB_OTG_FS_WAKEUP_EXTI_CLEAR_FLAG

__HAL_USB_OTG_FS_WAKEUP_EXTI_ENABLE_RISING_EDGE

PCD PHY Module

PCD_PHY_ULPI

PCD_PHY_EMBEDDED

PCD_PHY_UTMI

PCD Speed

PCD_SPEED_HIGH

PCD_SPEED_HIGH_IN_FULL

PCD_SPEED_FULL

33 HAL PCD Extension Driver

33.1 PCDEx Firmware driver API description

The following section lists the various functions of the PCDEx library.

33.1.1 Extended features functions

This section provides functions allowing to:

- Update FIFO configuration

This section contains the following APIs:

- [HAL_PCDEx_SetTxFiFo](#)
- [HAL_PCDEx_SetRxFiFo](#)
- [HAL_PCDEx_LPM_Callback](#)
- [HAL_PCDEx_BCD_Callback](#)

33.1.2 Detailed description of functions

HAL_PCDEx_SetTxFiFo

Function name

HAL_StatusTypeDef HAL_PCDEx_SetTxFiFo (PCD_HandleTypeDef * hpcd, uint8_t fifo, uint16_t size)

Function description

Set Tx FIFO.

Parameters

- **hpcd**: PCD handle
- **fifo**: The number of Tx fifo
- **size**: Fifo size

Return values

- **HAL**: status

HAL_PCDEx_SetRxFiFo

Function name

HAL_StatusTypeDef HAL_PCDEx_SetRxFiFo (PCD_HandleTypeDef * hpcd, uint16_t size)

Function description

Set Rx FIFO.

Parameters

- **hpcd**: PCD handle
- **size**: Size of Rx fifo

Return values

- **HAL**: status

HAL_PCDEx_LPM_Callback

Function name

void HAL_PCDEx_LPM_Callback (PCD_HandleTypeDef * hpcd, PCD_LPM_MsgTypeDef msg)

Function description

Send LPM message to user layer callback.

Parameters

- **hpcd**: PCD handle
- **msg**: LPM message

Return values

- **HAL**: status

HAL_PCDEx_BCD_Callback

Function name

void HAL_PCDEx_BCD_Callback (PCD_HandleTypeDef * hpcd, PCD_BCD_MsgTypeDef msg)

Function description

Send BatteryCharging message to user layer callback.

Parameters

- **hpcd**: PCD handle
- **msg**: LPM message

Return values

- **HAL**: status

34 HAL PWR Generic Driver

34.1 PWR Firmware driver registers structures

34.1.1 PWR_PVDTypeDef

PWR_PVDTypeDef is defined in the `stm32f2xx_hal_pwr.h`

Data Fields

- *uint32_t PVDLevel*
- *uint32_t Mode*

Field Documentation

- *uint32_t PWR_PVDTypeDef::PVDLevel*
PVDLevel: Specifies the PVD detection level. This parameter can be a value of *PWR_PVD_detection_level*
- *uint32_t PWR_PVDTypeDef::Mode*
Mode: Specifies the operating mode for the selected pins. This parameter can be a value of *PWR_PVD_Mode*

34.2 PWR Firmware driver API description

The following section lists the various functions of the PWR library.

34.2.1 Initialization and de-initialization functions

After reset, the backup domain (RTC registers, RTC backup data registers and backup SRAM) is protected against possible unwanted write accesses. To enable access to the RTC Domain and RTC registers, proceed as follows:

- Enable the Power Controller (PWR) APB1 interface clock using the `__HAL_RCC_PWR_CLK_ENABLE()` macro.
- Enable access to RTC domain using the `HAL_PWR_EnableBkUpAccess()` function.

This section contains the following APIs:

- *HAL_PWR_DeInit*
- *HAL_PWR_EnableBkUpAccess*
- *HAL_PWR_DisableBkUpAccess*

34.2.2 Peripheral Control functions

PVD configuration

- The PVD is used to monitor the VDD power supply by comparing it to a threshold selected by the PVD Level (PLS[2:0] bits in the `PWR_CR`).
- A PVDO flag is available to indicate if VDD/VDDA is higher or lower than the PVD threshold. This event is internally connected to the EXTI line16 and can generate an interrupt if enabled. This is done through `__HAL_PWR_PVD_EXTI_ENABLE_IT()` macro.
- The PVD is stopped in Standby mode.

Wake-up pin configuration

- Wake-up pin is used to wake up the system from Standby mode. This pin is forced in input pull-down configuration and is active on rising edges.
- There is one Wake-up pin: Wake-up Pin 1 on PA.00.

Low Power modes configuration

The devices feature 3 low-power modes:

- Sleep mode: Cortex-M3 core stopped, peripherals kept running.
- Stop mode: all clocks are stopped, regulator running, regulator in low power mode
- Standby mode: 1.2V domain powered off.

Sleep mode

- Entry: The Sleep mode is entered by using the `HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI)` functions with
 - `PWR_SLEEPENTRY_WFI`: enter SLEEP mode with WFI instruction
 - `PWR_SLEEPENTRY_WFE`: enter SLEEP mode with WFE instruction

Note: The Regulator parameter is not used for the STM32F2 family and is kept as parameter just to maintain compatibility with the lower power families (STM32L).

- Exit: Any peripheral interrupt acknowledged by the nested vectored interrupt controller (NVIC) can wake up the device from Sleep mode.

Stop mode

In Stop mode, all clocks in the 1.2V domain are stopped, the PLL, the HSI, and the HSE RC oscillators are disabled. Internal SRAM and register contents are preserved. The voltage regulator can be configured either in normal or low-power mode. To minimize the consumption In Stop mode, FLASH can be powered off before entering the Stop mode using the `HAL_PWREx_EnableFlashPowerDown()` function. It can be switched on again by software after exiting the Stop mode using the `HAL_PWREx_DisableFlashPowerDown()` function.

- Entry: The Stop mode is entered using the `HAL_PWR_EnterSTOPMode(PWR_MAINREGULATOR_ON)` function with:
 - Main regulator ON.
 - Low Power regulator ON.
- Exit: Any EXTI Line (Internal or External) configured in Interrupt/Event mode.

Standby mode

- The Standby mode allows to achieve the lowest power consumption. It is based on the Cortex-M3 deep sleep mode, with the voltage regulator disabled. The 1.2V domain is consequently powered off. The PLL, the HSI oscillator and the HSE oscillator are also switched off. SRAM and register contents are lost except for the RTC registers, RTC backup registers, backup SRAM and Standby circuitry. The voltage regulator is OFF.
 - Entry:
 - The Standby mode is entered using the `HAL_PWR_EnterSTANDBYMode()` function.
 - Exit:
 - WKUP pin rising edge, RTC alarm (Alarm A and Alarm B), RTC wake-up, tamper event, time-stamp event, external reset in NRST pin, IWDG reset.

Auto-wake-up (AWU) from low-power mode

- The MCU can be woken up from low-power mode by an RTC Alarm event, an RTC Wake-up event, a tamper event or a time-stamp event, without depending on an external interrupt (Auto-wake-up mode).
- RTC auto-wake-up (AWU) from the Stop and Standby modes
 - To wake up from the Stop mode with an RTC alarm event, it is necessary to configure the RTC to generate the RTC alarm using the `HAL_RTC_SetAlarm_IT()` function.
 - To wake up from the Stop mode with an RTC Tamper or time stamp event, it is necessary to configure the RTC to detect the tamper or time stamp event using the `HAL_RTCEx_SetTimeStamp_IT()` or `HAL_RTCEx_SetTamper_IT()` functions.
 - To wake up from the Stop mode with an RTC Wake-up event, it is necessary to configure the RTC to generate the RTC Wake-up event using the `HAL_RTCEx_SetWakeUpTimer_IT()` function.

This section contains the following APIs:

- [***HAL_PWR_ConfigPVD***](#)
- [***HAL_PWR_EnablePVD***](#)
- [***HAL_PWR_DisablePVD***](#)

- [HAL_PWR_EnableWakeUpPin](#)
- [HAL_PWR_DisableWakeUpPin](#)
- [HAL_PWR_EnterSLEEPMode](#)
- [HAL_PWR_EnterSTOPMode](#)
- [HAL_PWR_EnterSTANDBYMode](#)
- [HAL_PWR_PVD_IRQHandler](#)
- [HAL_PWR_PVDCallback](#)
- [HAL_PWR_EnableSleepOnExit](#)
- [HAL_PWR_DisableSleepOnExit](#)
- [HAL_PWR_EnableSEVOnPend](#)
- [HAL_PWR_DisableSEVOnPend](#)

34.2.3 Detailed description of functions

HAL_PWR_DeInit

Function name

void HAL_PWR_DeInit (void)

Function description

Deinitializes the HAL PWR peripheral registers to their default reset values.

Return values

- **None:**

HAL_PWR_EnableBkUpAccess

Function name

void HAL_PWR_EnableBkUpAccess (void)

Function description

Enables access to the backup domain (RTC registers, RTC backup data registers and backup SRAM).

Return values

- **None:**

Notes

- If the HSE divided by 2, 3, ..31 is used as the RTC clock, the Backup Domain Access should be kept enabled.

HAL_PWR_DisableBkUpAccess

Function name

void HAL_PWR_DisableBkUpAccess (void)

Function description

Disables access to the backup domain (RTC registers, RTC backup data registers and backup SRAM).

Return values

- **None:**

Notes

- If the HSE divided by 2, 3, ..31 is used as the RTC clock, the Backup Domain Access should be kept enabled.

HAL_PWR_ConfigPVD

Function name

```
void HAL_PWR_ConfigPVD (PWR_PVDTypeDef * sConfigPVD)
```

Function description

Configures the voltage threshold detected by the Power Voltage Detector(PVD).

Parameters

- **sConfigPVD**: pointer to an PWR_PVDTypeDef structure that contains the configuration information for the PVD.

Return values

- **None**:

Notes

- Refer to the electrical characteristics of your device datasheet for more details about the voltage threshold corresponding to each detection level.

HAL_PWR_EnablePVD

Function name

```
void HAL_PWR_EnablePVD (void )
```

Function description

Enables the Power Voltage Detector(PVD).

Return values

- **None**:

HAL_PWR_DisablePVD

Function name

```
void HAL_PWR_DisablePVD (void )
```

Function description

Disables the Power Voltage Detector(PVD).

Return values

- **None**:

HAL_PWR_EnableWakeUpPin

Function name

```
void HAL_PWR_EnableWakeUpPin (uint32_t WakeUpPinx)
```

Function description

Enables the Wake-up PINx functionality.

Parameters

- **WakeUpPinx**: Specifies the Power Wake-Up pin to enable. This parameter can be one of the following values:
 - PWR_WAKEUP_PIN1

Return values

- **None**:

HAL_PWR_DisableWakeUpPin

Function name

```
void HAL_PWR_DisableWakeUpPin (uint32_t WakeUpPinx)
```

Function description

Disables the Wake-up PINx functionality.

Parameters

- **WakeUpPinx:** Specifies the Power Wake-Up pin to disable. This parameter can be one of the following values:
 - PWR_WAKEUP_PIN1

Return values

- **None:**

HAL_PWR_EnterSTOPMode

Function name

```
void HAL_PWR_EnterSTOPMode (uint32_t Regulator, uint8_t STOPEntry)
```

Function description

Enters Stop mode.

Parameters

- **Regulator:** Specifies the regulator state in Stop mode. This parameter can be one of the following values:
 - PWR_MAINREGULATOR_ON: Stop mode with regulator ON
 - PWR_LOWPOWERREGULATOR_ON: Stop mode with low power regulator ON
- **STOPEntry:** Specifies if Stop mode is entered with WFI or WFE instruction. This parameter can be one of the following values:
 - PWR_STOPENTRY_WFI: Enter Stop mode with WFI instruction
 - PWR_STOPENTRY_WFE: Enter Stop mode with WFE instruction

Return values

- **None:**

Notes

- In Stop mode, all I/O pins keep the same state as in Run mode.
- When exiting Stop mode by issuing an interrupt or a wake-up event, the HSI RC oscillator is selected as system clock.
- When the voltage regulator operates in low power mode, an additional startup delay is incurred when waking up from Stop mode. By keeping the internal regulator ON during Stop mode, the consumption is higher although the startup time is reduced.

HAL_PWR_EnterSLEEPMode

Function name

```
void HAL_PWR_EnterSLEEPMode (uint32_t Regulator, uint8_t SLEEPEntry)
```

Function description

Enters Sleep mode.

Parameters

- **Regulator:** Specifies the regulator state in SLEEP mode. This parameter can be one of the following values:
 - PWR_MAINREGULATOR_ON: SLEEP mode with regulator ON
 - PWR_LOWPOWERREGULATOR_ON: SLEEP mode with low power regulator ON
- **SLEEPEntry:** Specifies if SLEEP mode is entered with WFI or WFE instruction. This parameter can be one of the following values:
 - PWR_SLEEPENTRY_WFI: enter SLEEP mode with WFI instruction
 - PWR_SLEEPENTRY_WFE: enter SLEEP mode with WFE instruction

Return values

- **None:**

Notes

- In Sleep mode, all I/O pins keep the same state as in Run mode.
- In Sleep mode, the systick is stopped to avoid exit from this mode with systick interrupt when used as time base for Timeout
- This parameter is not used for the STM32F2 family and is kept as parameter just to maintain compatibility with the lower power families.

HAL_PWR_EnterSTANDBYMode

Function name

void HAL_PWR_EnterSTANDBYMode (void)

Function description

Enters Standby mode.

Return values

- **None:**

Notes

- In Standby mode, all I/O pins are high impedance except for: Reset pad (still available) RTC_AF1 pin (PC13) if configured for tamper, time-stamp, RTC Alarm out, or RTC clock calibration out. RTC_AF2 pin (PI8) if configured for tamper or time-stamp. WKUP pin 1 (PA0) if enabled.

HAL_PWR_PVD_IRQHandler

Function name

void HAL_PWR_PVD_IRQHandler (void)

Function description

This function handles the PWR PVD interrupt request.

Return values

- **None:**

Notes

- This API should be called under the PVD_IRQHandler().

HAL_PWR_PVDCallback

Function name

void HAL_PWR_PVDCallback (void)

Function description

PWR PVD interrupt callback.

Return values

- **None:**

HAL_PWR_EnableSleepOnExit

Function name

void HAL_PWR_EnableSleepOnExit (void)

Function description

Indicates Sleep-On-Exit when returning from Handler mode to Thread mode.

Return values

- **None:**

Notes

- Set SLEEPONEXIT bit of SCR register. When this bit is set, the processor re-enters SLEEP mode when an interruption handling is over. Setting this bit is useful when the processor is expected to run only on interruptions handling.

HAL_PWR_DisableSleepOnExit

Function name

void HAL_PWR_DisableSleepOnExit (void)

Function description

Disables Sleep-On-Exit feature when returning from Handler mode to Thread mode.

Return values

- **None:**

Notes

- Clears SLEEPONEXIT bit of SCR register. When this bit is set, the processor re-enters SLEEP mode when an interruption handling is over.

HAL_PWR_EnableSEVOnPend

Function name

void HAL_PWR_EnableSEVOnPend (void)

Function description

Enables CORTEX M3 SEVONPEND bit.

Return values

- **None:**

Notes

- Sets SEVONPEND bit of SCR register. When this bit is set, this causes WFE to wake up when an interrupt moves from inactive to pended.

HAL_PWR_DisableSEVOnPend

Function name

void HAL_PWR_DisableSEVOnPend (void)

Function description

Disables CORTEX M3 SEVONPEND bit.

Return values

- **None:**

Notes

- Clears SEVONPEND bit of SCR register. When this bit is set, this causes WFE to wake up when an interrupt moves from inactive to pended.

34.3 PWR Firmware driver defines

The following section lists the various define and macros of the module.

34.3.1 PWR

PWR

PWR CR Register alias address

DBP_BIT_NUMBER

CR_DBP_BB

PVDE_BIT_NUMBER

CR_PVDE_BB

PWR CSR Register alias address

EWUP_BIT_NUMBER

CSR_EWUP_BB

PWR Exported Macro

__HAL_PWR_GET_FLAG

Description:

- Check PWR flag is set or not.

Parameters:

- **__FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - **PWR_FLAG_WU**: Wake Up flag. This flag indicates that a wakeup event was received from the WKUP pin or from the RTC alarm (Alarm A or Alarm B), RTC Tamper event, RTC TimeStamp event or RTC Wakeup. An additional wakeup event is detected if the WKUP pin is enabled (by setting the EWUP bit) when the WKUP pin level is already high.
 - **PWR_FLAG_SB**: StandBy flag. This flag indicates that the system was resumed from StandBy mode.
 - **PWR_FLAG_PVDO**: PVD Output. This flag is valid only if PVD is enabled by the **HAL_PWR_EnablePVD()** function. The PVD is stopped by Standby mode For this reason, this bit is equal to 0 after Standby or reset until the PVDE bit is set.
 - **PWR_FLAG_BRR**: Backup regulator ready flag. This bit is not reset when the device wakes up from Standby mode or by a system reset or power reset.

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

`__HAL_PWR_CLEAR_FLAG`

Description:

- Clear the PWR's pending flags.

Parameters:

- `__FLAG__`: specifies the flag to clear. This parameter can be one of the following values:
 - `PWR_FLAG_WU`: Wake Up flag
 - `PWR_FLAG_SB`: StandBy flag

`__HAL_PWR_PVD_EXTI_ENABLE_IT`

Description:

- Enable the PVD Exti Line 16.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_DISABLE_IT`

Description:

- Disable the PVD EXTI Line 16.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_ENABLE_EVENT`

Description:

- Enable event on PVD Exti Line 16.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_DISABLE_EVENT`

Description:

- Disable event on PVD Exti Line 16.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_ENABLE_RISING_EDGE`

Description:

- Enable the PVD Extended Interrupt Rising Trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_DISABLE_RISING_EDGE`

Description:

- Disable the PVD Extended Interrupt Rising Trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_ENABLE_FALLING_EDGE`

Description:

- Enable the PVD Extended Interrupt Falling Trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_DISABLE_FALLING_EDGE`

Description:

- Disable the PVD Extended Interrupt Falling Trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_ENABLE_RISING_FALLING_EDGE`

Description:

- PVD EXTI line configuration: set rising & falling edge trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_DISABLE_RISING_FALLING_EDGE`

Description:

- Disable the PVD Extended Interrupt Rising & Falling Trigger.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_GET_FLAG`

Description:

- checks whether the specified PVD Exti interrupt flag is set or not.

Return value:

- EXTI: PVD Line Status.

`__HAL_PWR_PVD_EXTI_CLEAR_FLAG`

Description:

- Clear the PVD Exti flag.

Return value:

- None.

`__HAL_PWR_PVD_EXTI_GENERATE_SWIT`

Description:

- Generates a Software interrupt on PVD EXTI line.

Return value:

- None

PWR Flag

`PWR_FLAG_WU`

`PWR_FLAG_SB`

`PWR_FLAG_PVDO`

`PWR_FLAG_BRR`

PWR Private macros to check input parameters

`IS_PWR_WAKEUP_PIN`

`IS_PWR_PVD_LEVEL`

`IS_PWR_PVD_MODE`

IS_PWR_REGULATOR

IS_PWR_SLEEP_ENTRY

IS_PWR_STOP_ENTRY

PWR PVD detection level

PWR_PVDLEVEL_0

PWR_PVDLEVEL_1

PWR_PVDLEVEL_2

PWR_PVDLEVEL_3

PWR_PVDLEVEL_4

PWR_PVDLEVEL_5

PWR_PVDLEVEL_6

PWR_PVDLEVEL_7

PWR PVD EXTI Line

PWR_EXTI_LINE_PVD

External interrupt line 16 Connected to the PVD EXTI Line

PWR PVD Mode

PWR_PVD_MODE_NORMAL

basic mode is used

PWR_PVD_MODE_IT_RISING

External Interrupt Mode with Rising edge trigger detection

PWR_PVD_MODE_IT_FALLING

External Interrupt Mode with Falling edge trigger detection

PWR_PVD_MODE_IT_RISING_FALLING

External Interrupt Mode with Rising/Falling edge trigger detection

PWR_PVD_MODE_EVENT_RISING

Event Mode with Rising edge trigger detection

PWR_PVD_MODE_EVENT_FALLING

Event Mode with Falling edge trigger detection

PWR_PVD_MODE_EVENT_RISING_FALLING

Event Mode with Rising/Falling edge trigger detection

PWR PVD Mode Mask

PVD_MODE_IT

PVD_MODE_EVT

PVD_RISING_EDGE

PVD_FALLING_EDGE

PWR Register alias address

PWR_OFFSET

PWR_CR_OFFSET

PWR_CSR_OFFSET

PWR_CR_OFFSET_BB

PWR_CSR_OFFSET_BB

PWR Regulator state in SLEEP/STOP mode

PWR_MAINREGULATOR_ON

PWR_LOWPOWERREGULATOR_ON

PWR SLEEP mode entry

PWR_SLEEPENTRY_WFI

PWR_SLEEPENTRY_WFE

PWR STOP mode entry

PWR_STOPENTRY_WFI

PWR_STOPENTRY_WFE

PWR WakeUp Pins

PWR_WAKEUP_PIN1

35 HAL PWR Extension Driver

35.1 PWREx Firmware driver API description

The following section lists the various functions of the PWREx library.

35.1.1 Peripheral extended features functions

Main and Backup Regulators configuration

- The backup domain includes 4 Kbytes of backup SRAM accessible only from the CPU, and address in 32-bit, 16-bit or 8-bit mode. Its content is retained even in Standby or VBAT mode when the low power backup regulator is enabled. It can be considered as an internal EEPROM when VBAT is always present. You can use the HAL_PWREx_EnableBkUpReg() function to enable the low power backup regulator.
- When the backup domain is supplied by VDD (analog switch connected to VDD) the backup SRAM is powered from VDD which replaces the VBAT power supply to save battery life.
- The backup SRAM is not mass erased by a tamper event. It is read protected to prevent confidential data, such as cryptographic private key, from being accessed. The backup SRAM can be erased only through the Flash interface when a protection level change from level 1 to level 0 is requested.

Note: Refer to the description of Read protection (RDP) in the Flash programming manual. Refer to the product datasheets for more details.

FLASH Power Down configuration

- By setting the FPDS bit in the PWR_CR register by using the HAL_PWREx_EnableFlashPowerDown() function, the Flash memory also enters power down mode when the device enters Stop mode. When the Flash memory is in power down mode, an additional startup delay is incurred when waking up from Stop mode.

This section contains the following APIs:

- [HAL_PWREx_EnableBkUpReg](#)
- [HAL_PWREx_DisableBkUpReg](#)
- [HAL_PWREx_EnableFlashPowerDown](#)
- [HAL_PWREx_DisableFlashPowerDown](#)

35.1.2 Detailed description of functions

HAL_PWREx_EnableFlashPowerDown

Function name

void HAL_PWREx_EnableFlashPowerDown (void)

Function description

Enables the Flash Power Down in Stop mode.

Return values

- **None:**

HAL_PWREx_DisableFlashPowerDown

Function name

void HAL_PWREx_DisableFlashPowerDown (void)

Function description

Disables the Flash Power Down in Stop mode.

Return values

- **None:**

HAL_PWREx_EnableBkUpReg

Function name

HAL_StatusTypeDef HAL_PWREx_EnableBkUpReg (void)

Function description

Enables the Backup Regulator.

Return values

- **HAL:** status

HAL_PWREx_DisableBkUpReg

Function name

HAL_StatusTypeDef HAL_PWREx_DisableBkUpReg (void)

Function description

Disables the Backup Regulator.

Return values

- **HAL:** status

35.2 PWREx Firmware driver defines

The following section lists the various define and macros of the module.

35.2.1 PWREx

PWREx

PWR CSR Register alias address

BRE_BIT_NUMBER

CSR_BRE_BB

PWR Register alias address

FPDS_BIT_NUMBER

CR_FPDS_BB

36 HAL RCC Generic Driver

36.1 RCC Firmware driver registers structures

36.1.1 RCC_PLLInitTypeDef

RCC_PLLInitTypeDef is defined in the stm32f2xx_hal_rcc.h

Data Fields

- **uint32_t PLLState**
- **uint32_t PLLSource**
- **uint32_t PLLM**
- **uint32_t PLLN**
- **uint32_t PLLP**
- **uint32_t PLLQ**

Field Documentation

- **uint32_t RCC_PLLInitTypeDef::PLLState**
The new state of the PLL. This parameter can be a value of [RCC_PLL_Config](#)
- **uint32_t RCC_PLLInitTypeDef::PLLSource**
RCC_PLLSource: PLL entry clock source. This parameter must be a value of [RCC_PLL_Clock_Source](#)
- **uint32_t RCC_PLLInitTypeDef::PLLM**
PLLM: Division factor for PLL VCO input clock. This parameter must be a number between Min_Data = 0 and Max_Data = 63
- **uint32_t RCC_PLLInitTypeDef::PLLN**
PLLN: Multiplication factor for PLL VCO output clock. This parameter must be a number between Min_Data = 192 and Max_Data = 432
- **uint32_t RCC_PLLInitTypeDef::PLLP**
PLLP: Division factor for main system clock (SYSCLK). This parameter must be a value of [RCC_PLLP_Clock_Divider](#)
- **uint32_t RCC_PLLInitTypeDef::PLLQ**
PLLQ: Division factor for OTG FS, SDIO and RNG clocks. This parameter must be a number between Min_Data = 2 and Max_Data = 15

36.1.2 RCC_OscInitTypeDef

RCC_OscInitTypeDef is defined in the stm32f2xx_hal_rcc.h

Data Fields

- **uint32_t OscillatorType**
- **uint32_t HSEState**
- **uint32_t LSEState**
- **uint32_t HSISState**
- **uint32_t HSICalibrationValue**
- **uint32_t LSISState**
- **RCC_PLLInitTypeDef PLL**

Field Documentation

- **uint32_t RCC_OscInitTypeDef::OscillatorType**
The oscillators to be configured. This parameter can be a value of [RCC_Oscillator_Type](#)
- **uint32_t RCC_OscInitTypeDef::HSEState**
The new state of the HSE. This parameter can be a value of [RCC_HSE_Config](#)
- **uint32_t RCC_OscInitTypeDef::LSEState**
The new state of the LSE. This parameter can be a value of [RCC_LSE_Config](#)

- **`uint32_t RCC_OscInitTypeDef::HSIState`**
The new state of the HSI. This parameter can be a value of [RCC_HSI_Config](#)
- **`uint32_t RCC_OscInitTypeDef::HSICalibrationValue`**
The HSI calibration trimming value (default is `RCC_HSICALIBRATION_DEFAULT`). This parameter must be a number between `Min_Data = 0x00` and `Max_Data = 0x1F`
- **`uint32_t RCC_OscInitTypeDef::LSIState`**
The new state of the LSI. This parameter can be a value of [RCC_LSI_Config](#)
- **`RCC_PLLInitTypeDef RCC_OscInitTypeDef::PLL`**
PLL structure parameters

36.1.3

RCC_ClkInitTypeDef

`RCC_ClkInitTypeDef` is defined in the `stm32f2xx_hal_rcc.h`

Data Fields

- **`uint32_t ClockType`**
- **`uint32_t SYSCLKSource`**
- **`uint32_t AHBCLKDivider`**
- **`uint32_t APB1CLKDivider`**
- **`uint32_t APB2CLKDivider`**

Field Documentation

- **`uint32_t RCC_ClkInitTypeDef::ClockType`**
The clock to be configured. This parameter can be a value of [RCC_System_Clock_Type](#)
- **`uint32_t RCC_ClkInitTypeDef::SYSCLKSource`**
The clock source (SYSCLKS) used as system clock. This parameter can be a value of [RCC_System_Clock_Source](#)
- **`uint32_t RCC_ClkInitTypeDef::AHBCLKDivider`**
The AHB clock (HCLK) divider. This clock is derived from the system clock (SYSCLK). This parameter can be a value of [RCC_AHB_Clock_Source](#)
- **`uint32_t RCC_ClkInitTypeDef::APB1CLKDivider`**
The APB1 clock (PCLK1) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of [RCC_APB1_APB2_Clock_Source](#)
- **`uint32_t RCC_ClkInitTypeDef::APB2CLKDivider`**
The APB2 clock (PCLK2) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of [RCC_APB1_APB2_Clock_Source](#)

36.2

RCC Firmware driver API description

The following section lists the various functions of the RCC library.

36.2.1

RCC specific features

After reset the device is running from Internal High Speed oscillator (HSI 16MHz) with Flash 0 wait state, Flash prefetch buffer, D-Cache and I-Cache are disabled, and all peripherals are off except internal SRAM, Flash and JTAG.

- There is no prescaler on High speed (AHB) and Low speed (APB) busses; all peripherals mapped on these busses are running at HSI speed.
- The clock for all peripherals is switched off, except the SRAM and FLASH.
- All GPIOs are in input floating state, except the JTAG pins which are assigned to be used for debug purpose.

Once the device started from reset, the user application has to:

- Configure the clock source to be used to drive the System clock (if the application needs higher frequency/performance)
- Configure the System clock frequency and Flash settings
- Configure the AHB and APB busses prescalers

- Enable the clock for the peripheral(s) to be used
- Configure the clock source(s) for peripherals which clocks are not derived from the System clock (I2S, RTC, ADC, USB OTG FS/SDIO/RNG)

36.2.2 RCC Limitations

A delay between an RCC peripheral clock enable and the effective peripheral enabling should be taken into account in order to manage the peripheral read/write from/to registers.

- This delay depends on the peripheral mapping.
- If peripheral is mapped on AHB: the delay is 2 AHB clock cycle after the clock enable bit is set on the hardware register
- If peripheral is mapped on APB: the delay is 2 APB clock cycle after the clock enable bit is set on the hardware register

Implemented Workaround:

- For AHB & APB peripherals, a dummy read to the peripheral register has been inserted in each `__HAL_RCC_PPP_CLK_ENABLE()` macro.

36.2.3 Initialization and de-initialization functions

This section provides functions allowing to configure the internal/external oscillators (HSE, HSI, LSE, LSI, PLL, CSS and MCO) and the System busses clocks (SYSCLK, AHB, APB1 and APB2).

Internal/external clock and PLL configuration

1. HSI (high-speed internal), 16 MHz factory-trimmed RC used directly or through the PLL as System clock source.
2. LSI (low-speed internal), 32 KHz low consumption RC used as IWDG and/or RTC clock source.
3. HSE (high-speed external), 4 to 26 MHz crystal oscillator used directly or through the PLL as System clock source. Can be used also as RTC clock source.
4. LSE (low-speed external), 32 KHz oscillator used as RTC clock source.
5. PLL (clocked by HSI or HSE), featuring two different output clocks:
 - The first output is used to generate the high speed system clock (up to 120 MHz)
 - The second output is used to generate the clock for the USB OTG FS (48 MHz), the random analog generator (≤ 48 MHz) and the SDIO (≤ 48 MHz).
6. CSS (Clock security system), once enable using the macro `__HAL_RCC_CSS_ENABLE()` and if a HSE clock failure occurs (HSE used directly or through PLL as System clock source), the System clocks automatically switched to HSI and an interrupt is generated if enabled. The interrupt is linked to the Cortex-M3 NMI (Non-Maskable Interrupt) exception vector.
7. MCO1 (microcontroller clock output), used to output HSI, LSE, HSE or PLL clock (through a configurable prescaler) on PA8 pin.
8. MCO2 (microcontroller clock output), used to output HSE, PLL, SYSCLK or PLLI2S clock (through a configurable prescaler) on PC9 pin.

System, AHB and APB busses clocks configuration

1. Several clock sources can be used to drive the System clock (SYSCLK): HSI, HSE and PLL. The AHB clock (HCLK) is derived from System clock through configurable prescaler and used to clock the CPU, memory and peripherals mapped on AHB bus (DMA, GPIO...). APB1 (PCLK1) and APB2 (PCLK2) clocks are derived from AHB clock through configurable prescalers and used to clock the peripherals mapped on these busses. You can use `"HAL_RCC_GetSysClockFreq()"` function to retrieve the frequencies of these clocks.

Note:

All the peripheral clocks are derived from the System clock (SYSCLK) except:

- *I2S: the I2S clock can be derived either from a specific PLL (PLLI2S) or from an external clock mapped on the I2S_CKIN pin. You have to use `__HAL_RCC_PLLI2S_CONFIG()` macro to configure this clock.*
- *RTC: the RTC clock can be derived either from the LSI, LSE or HSE clock divided by 2 to 31. You have to use `__HAL_RCC_RTC_CONFIG()` and `__HAL_RCC_RTC_ENABLE()` macros to configure this clock.*
- *USB OTG FS, SDIO and RTC: USB OTG FS require a frequency equal to 48 MHz to work correctly, while the SDIO require a frequency equal or lower than 48. This clock is derived of the main PLL through PLLQ divider.*
- *IWDG clock which is always the LSI clock.*

2. For the stm32f2xx devices, the maximum frequency of the SYSCLK and HCLK is 120 MHz, PCLK2 60 MHz and PCLK1 30 MHz. Depending on the device voltage range, the maximum frequency should be adapted accordingly: +-----+ | Latency | HCLK clock frequency (MHz) | |-----+ | voltage range | voltage range |
voltage range | voltage range | | 2.7 V - 3.6 V | 2.4 V - 2.7 V | 2.1 V - 2.4 V | 1.8 V - 2.1 V |
|-----+ |0WS(1CPU cycle)| 0 < HCLK <= 30 | 0 < HCLK <= 24 | 0 < HCLK <= 18 | 0 < HCLK <= 16 | | |
|-----+ |1WS(2CPU cycle)| 30 < HCLK <= 60 | 24 < HCLK <= 48 | 18 < HCLK <= 36 | 16 < HCLK <= 32 |
|-----+ |2WS(3CPU cycle)| 60 < HCLK <= 90 | 48 < HCLK <= 72 | 36 < HCLK <= 54 | 32 < HCLK <= 48 |
|-----+ |3WS(4CPU cycle)| 90 < HCLK <= 120 | 72 < HCLK <= 96 | 54 < HCLK <= 72 | 48 < HCLK <= 64 |
|-----+ |4WS(5CPU cycle)| NA | 96 < HCLK <= 120 | 72 < HCLK <= 90 | 64 < HCLK <= 80 | |-----+ |
5WS(6CPU cycle)| NA | NA | 90 < HCLK <= 108 | 80 < HCLK <= 96 |
|-----+ |6WS(7CPU cycle)| NA | NA | 108 < HCLK <= 120 | 96 < HCLK <= 112 | |-----+ |
7WS(8CPU cycle)| NA | NA | NA | 112 < HCLK <= 120 | +-----+
+

This section contains the following APIs:

- [HAL_RCC_DeInit](#)
- [HAL_RCC_OscConfig](#)
- [HAL_RCC_ClockConfig](#)

36.2.4 Peripheral Control functions

This subsection provides a set of functions allowing to control the RCC Clocks frequencies.

This section contains the following APIs:

- [HAL_RCC_MCOConfig](#)
- [HAL_RCC_EnableCSS](#)
- [HAL_RCC_DisableCSS](#)
- [HAL_RCC_GetSysClockFreq](#)
- [HAL_RCC_GetHCLKFreq](#)
- [HAL_RCC_GetPCLK1Freq](#)
- [HAL_RCC_GetPCLK2Freq](#)
- [HAL_RCC_GetOscConfig](#)
- [HAL_RCC_GetClockConfig](#)
- [HAL_RCC_NMI_IRQHandler](#)
- [HAL_RCC_CSSCallback](#)

36.2.5 Detailed description of functions

HAL_RCC_DeInit

Function name

HAL_StatusTypeDef HAL_RCC_DeInit (void)

Function description

Resets the RCC clock configuration to the default reset state.

Return values

- **HAL:** status

Notes

- The default reset state of the clock configuration is given below: HSI ON and used as system clock source HSE, PLL and PLLI2S OFF AHB, APB1 and APB2 prescaler set to 1. CSS, MCO1 and MCO2 OFF All interrupts disabled
- This function doesn't modify the configuration of the Peripheral clocks LSI, LSE and RTC clocks

HAL_RCC_OscConfig

Function name

HAL_StatusTypeDef HAL_RCC_OscConfig (RCC_OscInitTypeDef * RCC_OscInitStruct)

Function description

Initializes the RCC Oscillators according to the specified parameters in the RCC_OscInitTypeDef.

Parameters

- **RCC_OscInitStruct:** pointer to an RCC_OscInitTypeDef structure that contains the configuration information for the RCC Oscillators.

Return values

- **HAL:** status

Notes

- The PLL is not disabled when used as system clock.
- Transitions LSE Bypass to LSE On and LSE On to LSE Bypass are not supported by this API. User should request a transition to LSE Off first and then LSE On or LSE Bypass.
- Transition HSE Bypass to HSE On and HSE On to HSE Bypass are not supported by this API. User should request a transition to HSE Off first and then HSE On or HSE Bypass.

HAL_RCC_ClockConfig

Function name

HAL_StatusTypeDef HAL_RCC_ClockConfig (RCC_ClkInitTypeDef * RCC_ClkInitStruct, uint32_t FLatency)

Function description

Initializes the CPU, AHB and APB busses clocks according to the specified parameters in the RCC_ClkInitStruct.

Parameters

- **RCC_ClkInitStruct:** pointer to an RCC_OscInitTypeDef structure that contains the configuration information for the RCC peripheral.
- **FLatency:** FLASH Latency, this parameter depend on device selected

Return values

- **None:**

Notes

- The SystemCoreClock CMSIS variable is used to store System Clock Frequency and updated by HAL_RCC_GetHCLKFreq() function called within this function
- The HSI is used (enabled by hardware) as system clock source after startup from Reset, wake-up from STOP and STANDBY mode, or in case of failure of the HSE used directly or indirectly as system clock (if the Clock Security System CSS is enabled).
- A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source which is not yet ready is selected, the switch will occur when the clock source will be ready.
- Depending on the device voltage range, the software has to set correctly HPRE[3:0] bits to ensure that HCLK not exceed the maximum allowed frequency (for more details refer to section above "Initialization/de-initialization functions")

HAL_RCC_MCOConfig

Function name

void HAL_RCC_MCOConfig (uint32_t RCC_MCOx, uint32_t RCC_MCOSource, uint32_t RCC_MCODiv)

Function description

Selects the clock source to output on MCO1 pin(PA8) or on MCO2 pin(PC9).

Parameters

- **RCC_MCOx:** specifies the output direction for the clock source. This parameter can be one of the following values:
 - RCC_MCO1: Clock source to output on MCO1 pin(PA8).
 - RCC_MCO2: Clock source to output on MCO2 pin(PC9).
- **RCC_MCOSource:** specifies the clock source to output. This parameter can be one of the following values:
 - RCC_MCO1SOURCE_HSI: HSI clock selected as MCO1 source
 - RCC_MCO1SOURCE_LSE: LSE clock selected as MCO1 source
 - RCC_MCO1SOURCE_HSE: HSE clock selected as MCO1 source
 - RCC_MCO1SOURCE_PLLCLK: main PLL clock selected as MCO1 source
 - RCC_MCO2SOURCE_SYSCLK: System clock (SYSCLK) selected as MCO2 source
 - RCC_MCO2SOURCE_PLLI2SCLK: PLLI2S clock selected as MCO2 source
 - RCC_MCO2SOURCE_HSE: HSE clock selected as MCO2 source
 - RCC_MCO2SOURCE_PLLCLK: main PLL clock selected as MCO2 source
- **RCC_MCODiv:** specifies the MCOx prescaler. This parameter can be one of the following values:
 - RCC_MCODIV_1: no division applied to MCOx clock
 - RCC_MCODIV_2: division by 2 applied to MCOx clock
 - RCC_MCODIV_3: division by 3 applied to MCOx clock
 - RCC_MCODIV_4: division by 4 applied to MCOx clock
 - RCC_MCODIV_5: division by 5 applied to MCOx clock

Return values

- **None:**

Notes

- PA8/PC9 should be configured in alternate function mode.

HAL_RCC_EnableCSS

Function name

void HAL_RCC_EnableCSS (void)

Function description

Enables the Clock Security System.

Return values

- **None:**

Notes

- If a failure is detected on the HSE oscillator clock, this oscillator is automatically disabled and an interrupt is generated to inform the software about the failure (Clock Security System Interrupt, CSSI), allowing the MCU to perform rescue operations. The CSSI is linked to the Cortex-M3 NMI (Non-Maskable Interrupt) exception vector.

HAL_RCC_DisableCSS

Function name

void HAL_RCC_DisableCSS (void)

Function description

Disables the Clock Security System.

Return values

- **None:**

HAL_RCC_GetSysClockFreq

Function name

uint32_t HAL_RCC_GetSysClockFreq (void)

Function description

Returns the SYSCLK frequency.

Return values

- **SYSCLK:** frequency

Notes

- The system frequency computed by this function is not the real frequency in the chip. It is calculated based on the predefined constant and the selected clock source:
- If SYSCLK source is HSI, function returns values based on HSI_VALUE(*)
- If SYSCLK source is HSE, function returns values based on HSE_VALUE(**)
- If SYSCLK source is PLL, function returns values based on HSE_VALUE(**) or HSI_VALUE(*) multiplied/divided by the PLL factors.
- (*) HSI_VALUE is a constant defined in stm32f2xx_hal_conf.h file (default value 16 MHz) but the real value may vary depending on the variations in voltage and temperature.
- (**) HSE_VALUE is a constant defined in stm32f2xx_hal_conf.h file (default value 25 MHz), user has to ensure that HSE_VALUE is same as the real frequency of the crystal used. Otherwise, this function may have wrong result.
- The result of this function could be not correct when using fractional value for HSE crystal.
- This function can be used by the user application to compute the baudrate for the communication peripherals or configure other parameters.
- Each time SYSCLK changes, this function must be called to update the right SYSCLK value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetHCLKFreq

Function name

uint32_t HAL_RCC_GetHCLKFreq (void)

Function description

Returns the HCLK frequency.

Return values

- **HCLK:** frequency

Notes

- Each time HCLK changes, this function must be called to update the right HCLK value. Otherwise, any configuration based on this function will be incorrect.
- The SystemCoreClock CMSIS variable is used to store System Clock Frequency and updated within this function

HAL_RCC_GetPCLK1Freq

Function name

uint32_t HAL_RCC_GetPCLK1Freq (void)

Function description

Returns the PCLK1 frequency.

Return values

- **PCLK1**: frequency

Notes

- Each time PCLK1 changes, this function must be called to update the right PCLK1 value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetPCLK2Freq

Function name

uint32_t HAL_RCC_GetPCLK2Freq (void)

Function description

Returns the PCLK2 frequency.

Return values

- **PCLK2**: frequency

Notes

- Each time PCLK2 changes, this function must be called to update the right PCLK2 value. Otherwise, any configuration based on this function will be incorrect.

HAL_RCC_GetOscConfig

Function name

void HAL_RCC_GetOscConfig (RCC_OscInitTypeDef * RCC_OscInitStruct)

Function description

Configures the RCC_OscInitStruct according to the internal RCC configuration registers.

Parameters

- **RCC_OscInitStruct**: pointer to an RCC_OscInitTypeDef structure that will be configured.

Return values

- **None**:

HAL_RCC_GetClockConfig

Function name

void HAL_RCC_GetClockConfig (RCC_ClkInitTypeDef * RCC_ClkInitStruct, uint32_t * pFLatency)

Function description

Configures the RCC_ClkInitStruct according to the internal RCC configuration registers.

Parameters

- **RCC_ClkInitStruct:** pointer to an RCC_ClkInitTypeDef structure that will be configured.
- **pFLatency:** Pointer on the Flash Latency.

Return values

- **None:**

HAL_RCC_NMI_IRQHandler

Function name

void HAL_RCC_NMI_IRQHandler (void)

Function description

This function handles the RCC CSS interrupt request.

Return values

- **None:**

Notes

- This API should be called under the NMI_Handler().

HAL_RCC_CSSCallback

Function name

void HAL_RCC_CSSCallback (void)

Function description

RCC Clock Security System interrupt callback.

Return values

- **None:**

36.3 RCC Firmware driver defines

The following section lists the various define and macros of the module.

36.3.1 RCC

RCC

AHB1 Peripheral Clock Enable Disable

__HAL_RCC_GPIOA_CLK_ENABLE

__HAL_RCC_GPIOB_CLK_ENABLE

__HAL_RCC_GPIOC_CLK_ENABLE

__HAL_RCC_GPIOD_CLK_ENABLE

__HAL_RCC_GPIOE_CLK_ENABLE

__HAL_RCC_GPIOF_CLK_ENABLE

__HAL_RCC_GPIOG_CLK_ENABLE

__HAL_RCC_GPIOH_CLK_ENABLE

__HAL_RCC_GPIOI_CLK_ENABLE
__HAL_RCC_CRC_CLK_ENABLE
__HAL_RCC_BKPSRAM_CLK_ENABLE
__HAL_RCC_DMA1_CLK_ENABLE
__HAL_RCC_DMA2_CLK_ENABLE
__HAL_RCC_USB_OTG_HS_CLK_ENABLE
__HAL_RCC_USB_OTG_HS_ULPI_CLK_ENABLE
__HAL_RCC_GPIOA_CLK_DISABLE
__HAL_RCC_GPIOB_CLK_DISABLE
__HAL_RCC_GPIOC_CLK_DISABLE
__HAL_RCC_GPIOD_CLK_DISABLE
__HAL_RCC_GPIOE_CLK_DISABLE
__HAL_RCC_GPIOF_CLK_DISABLE
__HAL_RCC_GPIOG_CLK_DISABLE
__HAL_RCC_GPIOH_CLK_DISABLE
__HAL_RCC_GPIOI_CLK_DISABLE
__HAL_RCC_CRC_CLK_DISABLE
__HAL_RCC_BKPSRAM_CLK_DISABLE
__HAL_RCC_DMA1_CLK_DISABLE
__HAL_RCC_DMA2_CLK_DISABLE
__HAL_RCC_USB_OTG_HS_CLK_DISABLE
__HAL_RCC_USB_OTG_HS_ULPI_CLK_DISABLE

AHB1 Force Release Reset

__HAL_RCC_AHB1_FORCE_RESET
__HAL_RCC_GPIOA_FORCE_RESET
__HAL_RCC_GPIOB_FORCE_RESET
__HAL_RCC_GPIOC_FORCE_RESET
__HAL_RCC_GPIOD_FORCE_RESET

__HAL_RCC_GPIOE_FORCE_RESET
__HAL_RCC_GPIOF_FORCE_RESET
__HAL_RCC_GPIOG_FORCE_RESET
__HAL_RCC_GPIOH_FORCE_RESET
__HAL_RCC_GPIOI_FORCE_RESET
__HAL_RCC_CRC_FORCE_RESET
__HAL_RCC_DMA1_FORCE_RESET
__HAL_RCC_DMA2_FORCE_RESET
__HAL_RCC_USB_OTG_HS_FORCE_RESET
__HAL_RCC_OTGHSULPI_FORCE_RESET
__HAL_RCC_AHB1_RELEASE_RESET
__HAL_RCC_GPIOA_RELEASE_RESET
__HAL_RCC_GPIOB_RELEASE_RESET
__HAL_RCC_GPIOC_RELEASE_RESET
__HAL_RCC_GPIOD_RELEASE_RESET
__HAL_RCC_GPIOE_RELEASE_RESET
__HAL_RCC_GPIOF_RELEASE_RESET
__HAL_RCC_GPIOG_RELEASE_RESET
__HAL_RCC_GPIOH_RELEASE_RESET
__HAL_RCC_GPIOI_RELEASE_RESET
__HAL_RCC_CRC_RELEASE_RESET
__HAL_RCC_DMA1_RELEASE_RESET
__HAL_RCC_DMA2_RELEASE_RESET
__HAL_RCC_USB_OTG_HS_RELEASE_RESET
__HAL_RCC_OTGHSULPI_RELEASE_RESET

AHB1 Peripheral Low Power Enable Disable
__HAL_RCC_GPIOA_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOB_CLK_SLEEP_ENABLE

__HAL_RCC_GPIOC_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOD_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOE_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOF_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOG_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOH_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOI_CLK_SLEEP_ENABLE
__HAL_RCC_CRC_CLK_SLEEP_ENABLE
__HAL_RCC_FLITF_CLK_SLEEP_ENABLE
__HAL_RCC_SRAM1_CLK_SLEEP_ENABLE
__HAL_RCC_SRAM2_CLK_SLEEP_ENABLE
__HAL_RCC_BKPSRAM_CLK_SLEEP_ENABLE
__HAL_RCC_DMA1_CLK_SLEEP_ENABLE
__HAL_RCC_DMA2_CLK_SLEEP_ENABLE
__HAL_RCC_USB_OTG_HS_CLK_SLEEP_ENABLE
__HAL_RCC_USB_OTG_HS_ULPI_CLK_SLEEP_ENABLE
__HAL_RCC_GPIOA_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOB_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOC_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOD_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOE_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOF_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOG_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOH_CLK_SLEEP_DISABLE
__HAL_RCC_GPIOI_CLK_SLEEP_DISABLE
__HAL_RCC_CRC_CLK_SLEEP_DISABLE
__HAL_RCC_FLITF_CLK_SLEEP_DISABLE
__HAL_RCC_SRAM1_CLK_SLEEP_DISABLE

`__HAL_RCC_SRAM2_CLK_SLEEP_DISABLE`
`__HAL_RCC_BKPSRAM_CLK_SLEEP_DISABLE`
`__HAL_RCC_DMA1_CLK_SLEEP_DISABLE`
`__HAL_RCC_DMA2_CLK_SLEEP_DISABLE`
`__HAL_RCC_USB_OTG_HS_CLK_SLEEP_DISABLE`
`__HAL_RCC_USB_OTG_HS_ULPI_CLK_SLEEP_DISABLE`

AHB1 Peripheral Clock Enable Disable Status

`__HAL_RCC_GPIOA_IS_CLK_ENABLED`
`__HAL_RCC_GPIOB_IS_CLK_ENABLED`
`__HAL_RCC_GPIOC_IS_CLK_ENABLED`
`__HAL_RCC_GPIOD_IS_CLK_ENABLED`
`__HAL_RCC_GPIOE_IS_CLK_ENABLED`
`__HAL_RCC_GPIOF_IS_CLK_ENABLED`
`__HAL_RCC_GPIOG_IS_CLK_ENABLED`
`__HAL_RCC_GPIOH_IS_CLK_ENABLED`
`__HAL_RCC_GPIOI_IS_CLK_ENABLED`
`__HAL_RCC_CRC_IS_CLK_ENABLED`
`__HAL_RCC_BKPSRAM_IS_CLK_ENABLED`
`__HAL_RCC_DMA1_IS_CLK_ENABLED`
`__HAL_RCC_DMA2_IS_CLK_ENABLED`
`__HAL_RCC_USB_OTG_HS_IS_CLK_ENABLED`
`__HAL_RCC_USB_OTG_HS_ULPI_IS_CLK_ENABLED`
`__HAL_RCC_GPIOA_IS_CLK_DISABLED`
`__HAL_RCC_GPIOB_IS_CLK_DISABLED`
`__HAL_RCC_GPIOC_IS_CLK_DISABLED`
`__HAL_RCC_GPIOD_IS_CLK_DISABLED`
`__HAL_RCC_GPIOE_IS_CLK_DISABLED`
`__HAL_RCC_GPIOF_IS_CLK_DISABLED`

__HAL_RCC_GPIOG_IS_CLK_DISABLED
__HAL_RCC_GPIOH_IS_CLK_DISABLED
__HAL_RCC_GPIOI_IS_CLK_DISABLED
__HAL_RCC_CRC_IS_CLK_DISABLED
__HAL_RCC_BKPSRAM_IS_CLK_DISABLED
__HAL_RCC_DMA1_IS_CLK_DISABLED
__HAL_RCC_DMA2_IS_CLK_DISABLED
__HAL_RCC_USB_OTG_HS_IS_CLK_DISABLED
__HAL_RCC_USB_OTG_HS_ULPI_IS_CLK_DISABLED
__HAL_RCC_ETHMAC_IS_CLK_ENABLED
__HAL_RCC_ETHMACTX_IS_CLK_ENABLED
__HAL_RCC_ETHMACRX_IS_CLK_ENABLED
__HAL_RCC_ETHMACPTP_IS_CLK_ENABLED
__HAL_RCC_ETH_IS_CLK_ENABLED
__HAL_RCC_ETHMAC_IS_CLK_DISABLED
__HAL_RCC_ETHMACTX_IS_CLK_DISABLED
__HAL_RCC_ETHMACRX_IS_CLK_DISABLED
__HAL_RCC_ETHMACPTP_IS_CLK_DISABLED
__HAL_RCC_ETH_IS_CLK_DISABLED
AHB2 Peripheral Clock Enable Disable
__HAL_RCC_USB_OTG_FS_CLK_ENABLE
__HAL_RCC_RNG_CLK_ENABLE
__HAL_RCC_USB_OTG_FS_CLK_DISABLE
__HAL_RCC_RNG_CLK_DISABLE
AHB2 Force Release Reset
__HAL_RCC_AHB2_FORCE_RESET
__HAL_RCC_RNG_FORCE_RESET
__HAL_RCC_USB_OTG_FS_FORCE_RESET
__HAL_RCC_AHB2_RELEASE_RESET

__HAL_RCC_RNG_RELEASE_RESET

__HAL_RCC_USB_OTG_FS_RELEASE_RESET

AHB2 Peripheral Low Power Enable Disable

__HAL_RCC_USB_OTG_FS_CLK_SLEEP_ENABLE

__HAL_RCC_RNG_CLK_SLEEP_ENABLE

__HAL_RCC_USB_OTG_FS_CLK_SLEEP_DISABLE

__HAL_RCC_RNG_CLK_SLEEP_DISABLE

AHB2 Peripheral Clock Enable Disable Status

__HAL_RCC_USB_OTG_FS_IS_CLK_ENABLED

__HAL_RCC_RNG_IS_CLK_ENABLED

__HAL_RCC_USB_OTG_FS_IS_CLK_DISABLED

__HAL_RCC_RNG_IS_CLK_DISABLED

__HAL_RCC_DCMI_IS_CLK_ENABLED

__HAL_RCC_DCMI_IS_CLK_DISABLED

__HAL_RCC_Cryp_IS_CLK_ENABLED

__HAL_RCC_HASH_IS_CLK_ENABLED

__HAL_RCC_Cryp_IS_CLK_DISABLED

__HAL_RCC_HASH_IS_CLK_DISABLED

AHB3 Peripheral Clock Enable Disable

__HAL_RCC_FSMC_CLK_ENABLE

__HAL_RCC_FSMC_CLK_DISABLE

AHB3 Force Release Reset

__HAL_RCC_AHB3_FORCE_RESET

__HAL_RCC_FSMC_FORCE_RESET

__HAL_RCC_AHB3_RELEASE_RESET

__HAL_RCC_FSMC_RELEASE_RESET

AHB3 Peripheral Low Power Enable Disable

__HAL_RCC_FSMC_CLK_SLEEP_ENABLE

__HAL_RCC_FSMC_CLK_SLEEP_DISABLE

AHB3 Peripheral Clock Enable Disable Status

__HAL_RCC_FSMC_IS_CLK_ENABLED

__HAL_RCC_FSMC_IS_CLK_DISABLED

AHB Clock Source

RCC_SYSCLK_DIV1

RCC_SYSCLK_DIV2

RCC_SYSCLK_DIV4

RCC_SYSCLK_DIV8

RCC_SYSCLK_DIV16

RCC_SYSCLK_DIV64

RCC_SYSCLK_DIV128

RCC_SYSCLK_DIV256

RCC_SYSCLK_DIV512

APB1/APB2 Clock Source

RCC_HCLK_DIV1

RCC_HCLK_DIV2

RCC_HCLK_DIV4

RCC_HCLK_DIV8

RCC_HCLK_DIV16

APB1 Peripheral Clock Enable Disable

__HAL_RCC_TIM2_CLK_ENABLE

__HAL_RCC_TIM3_CLK_ENABLE

__HAL_RCC_TIM4_CLK_ENABLE

__HAL_RCC_TIM5_CLK_ENABLE

__HAL_RCC_TIM6_CLK_ENABLE

__HAL_RCC_TIM7_CLK_ENABLE

__HAL_RCC_TIM12_CLK_ENABLE

__HAL_RCC_TIM13_CLK_ENABLE

__HAL_RCC_TIM14_CLK_ENABLE

__HAL_RCC_WWDG_CLK_ENABLE

__HAL_RCC_SPI2_CLK_ENABLE
__HAL_RCC_SPI3_CLK_ENABLE
__HAL_RCC_USART2_CLK_ENABLE
__HAL_RCC_USART3_CLK_ENABLE
__HAL_RCC_UART4_CLK_ENABLE
__HAL_RCC_UART5_CLK_ENABLE
__HAL_RCC_I2C1_CLK_ENABLE
__HAL_RCC_I2C2_CLK_ENABLE
__HAL_RCC_I2C3_CLK_ENABLE
__HAL_RCC_CAN1_CLK_ENABLE
__HAL_RCC_CAN2_CLK_ENABLE
__HAL_RCC_PWR_CLK_ENABLE
__HAL_RCC_DAC_CLK_ENABLE
__HAL_RCC_TIM2_CLK_DISABLE
__HAL_RCC_TIM3_CLK_DISABLE
__HAL_RCC_TIM4_CLK_DISABLE
__HAL_RCC_TIM5_CLK_DISABLE
__HAL_RCC_TIM6_CLK_DISABLE
__HAL_RCC_TIM7_CLK_DISABLE
__HAL_RCC_TIM12_CLK_DISABLE
__HAL_RCC_TIM13_CLK_DISABLE
__HAL_RCC_TIM14_CLK_DISABLE
__HAL_RCC_WWDG_CLK_DISABLE
__HAL_RCC_SPI2_CLK_DISABLE
__HAL_RCC_SPI3_CLK_DISABLE
__HAL_RCC_USART2_CLK_DISABLE
__HAL_RCC_USART3_CLK_DISABLE
__HAL_RCC_UART4_CLK_DISABLE

__HAL_RCC_UART5_CLK_DISABLE

__HAL_RCC_I2C1_CLK_DISABLE

__HAL_RCC_I2C2_CLK_DISABLE

__HAL_RCC_I2C3_CLK_DISABLE

__HAL_RCC_PWR_CLK_DISABLE

__HAL_RCC_CAN1_CLK_DISABLE

__HAL_RCC_CAN2_CLK_DISABLE

__HAL_RCC_DAC_CLK_DISABLE

APB1 Force Release Reset

__HAL_RCC_APB1_FORCE_RESET

__HAL_RCC_TIM2_FORCE_RESET

__HAL_RCC_TIM3_FORCE_RESET

__HAL_RCC_TIM4_FORCE_RESET

__HAL_RCC_TIM5_FORCE_RESET

__HAL_RCC_TIM6_FORCE_RESET

__HAL_RCC_TIM7_FORCE_RESET

__HAL_RCC_TIM12_FORCE_RESET

__HAL_RCC_TIM13_FORCE_RESET

__HAL_RCC_TIM14_FORCE_RESET

__HAL_RCC_WWDG_FORCE_RESET

__HAL_RCC_SPI2_FORCE_RESET

__HAL_RCC_SPI3_FORCE_RESET

__HAL_RCC_USART2_FORCE_RESET

__HAL_RCC_USART3_FORCE_RESET

__HAL_RCC_UART4_FORCE_RESET

__HAL_RCC_UART5_FORCE_RESET

__HAL_RCC_I2C1_FORCE_RESET

__HAL_RCC_I2C2_FORCE_RESET

__HAL_RCC_I2C3_FORCE_RESET
__HAL_RCC_CAN1_FORCE_RESET
__HAL_RCC_CAN2_FORCE_RESET
__HAL_RCC_PWR_FORCE_RESET
__HAL_RCC_DAC_FORCE_RESET
__HAL_RCC_APB1_RELEASE_RESET
__HAL_RCC_TIM2_RELEASE_RESET
__HAL_RCC_TIM3_RELEASE_RESET
__HAL_RCC_TIM4_RELEASE_RESET
__HAL_RCC_TIM5_RELEASE_RESET
__HAL_RCC_TIM6_RELEASE_RESET
__HAL_RCC_TIM7_RELEASE_RESET
__HAL_RCC_TIM12_RELEASE_RESET
__HAL_RCC_TIM13_RELEASE_RESET
__HAL_RCC_TIM14_RELEASE_RESET
__HAL_RCC_WWDG_RELEASE_RESET
__HAL_RCC_SPI2_RELEASE_RESET
__HAL_RCC_SPI3_RELEASE_RESET
__HAL_RCC_USART2_RELEASE_RESET
__HAL_RCC_USART3_RELEASE_RESET
__HAL_RCC_UART4_RELEASE_RESET
__HAL_RCC_UART5_RELEASE_RESET
__HAL_RCC_I2C1_RELEASE_RESET
__HAL_RCC_I2C2_RELEASE_RESET
__HAL_RCC_I2C3_RELEASE_RESET
__HAL_RCC_CAN1_RELEASE_RESET
__HAL_RCC_CAN2_RELEASE_RESET
__HAL_RCC_PWR_RELEASE_RESET

__HAL_RCC_DAC_RELEASE_RESET

APB1 Peripheral Low Power Enable Disable

__HAL_RCC_TIM2_CLK_SLEEP_ENABLE

__HAL_RCC_TIM3_CLK_SLEEP_ENABLE

__HAL_RCC_TIM4_CLK_SLEEP_ENABLE

__HAL_RCC_TIM5_CLK_SLEEP_ENABLE

__HAL_RCC_TIM6_CLK_SLEEP_ENABLE

__HAL_RCC_TIM7_CLK_SLEEP_ENABLE

__HAL_RCC_TIM12_CLK_SLEEP_ENABLE

__HAL_RCC_TIM13_CLK_SLEEP_ENABLE

__HAL_RCC_TIM14_CLK_SLEEP_ENABLE

__HAL_RCC_WWDG_CLK_SLEEP_ENABLE

__HAL_RCC_SPI2_CLK_SLEEP_ENABLE

__HAL_RCC_SPI3_CLK_SLEEP_ENABLE

__HAL_RCC_USART2_CLK_SLEEP_ENABLE

__HAL_RCC_USART3_CLK_SLEEP_ENABLE

__HAL_RCC_UART4_CLK_SLEEP_ENABLE

__HAL_RCC_UART5_CLK_SLEEP_ENABLE

__HAL_RCC_I2C1_CLK_SLEEP_ENABLE

__HAL_RCC_I2C2_CLK_SLEEP_ENABLE

__HAL_RCC_I2C3_CLK_SLEEP_ENABLE

__HAL_RCC_PWR_CLK_SLEEP_ENABLE

__HAL_RCC_CAN1_CLK_SLEEP_ENABLE

__HAL_RCC_CAN2_CLK_SLEEP_ENABLE

__HAL_RCC_DAC_CLK_SLEEP_ENABLE

__HAL_RCC_TIM2_CLK_SLEEP_DISABLE

__HAL_RCC_TIM3_CLK_SLEEP_DISABLE

__HAL_RCC_TIM4_CLK_SLEEP_DISABLE

__HAL_RCC_TIM5_CLK_SLEEP_DISABLE
__HAL_RCC_TIM6_CLK_SLEEP_DISABLE
__HAL_RCC_TIM7_CLK_SLEEP_DISABLE
__HAL_RCC_TIM12_CLK_SLEEP_DISABLE
__HAL_RCC_TIM13_CLK_SLEEP_DISABLE
__HAL_RCC_TIM14_CLK_SLEEP_DISABLE
__HAL_RCC_WWDG_CLK_SLEEP_DISABLE
__HAL_RCC_SPI2_CLK_SLEEP_DISABLE
__HAL_RCC_SPI3_CLK_SLEEP_DISABLE
__HAL_RCC_USART2_CLK_SLEEP_DISABLE
__HAL_RCC_USART3_CLK_SLEEP_DISABLE
__HAL_RCC_UART4_CLK_SLEEP_DISABLE
__HAL_RCC_UART5_CLK_SLEEP_DISABLE
__HAL_RCC_I2C1_CLK_SLEEP_DISABLE
__HAL_RCC_I2C2_CLK_SLEEP_DISABLE
__HAL_RCC_I2C3_CLK_SLEEP_DISABLE
__HAL_RCC_PWR_CLK_SLEEP_DISABLE
__HAL_RCC_CAN1_CLK_SLEEP_DISABLE
__HAL_RCC_CAN2_CLK_SLEEP_DISABLE
__HAL_RCC_DAC_CLK_SLEEP_DISABLE

APB1 Peripheral Clock Enable Disable Status

__HAL_RCC_TIM2_IS_CLK_ENABLED
__HAL_RCC_TIM3_IS_CLK_ENABLED
__HAL_RCC_TIM4_IS_CLK_ENABLED
__HAL_RCC_TIM5_IS_CLK_ENABLED
__HAL_RCC_TIM6_IS_CLK_ENABLED
__HAL_RCC_TIM7_IS_CLK_ENABLED
__HAL_RCC_TIM12_IS_CLK_ENABLED

__HAL_RCC_TIM13_IS_CLK_ENABLED
__HAL_RCC_TIM14_IS_CLK_ENABLED
__HAL_RCC_WWDG_IS_CLK_ENABLED
__HAL_RCC_SPI2_IS_CLK_ENABLED
__HAL_RCC_SPI3_IS_CLK_ENABLED
__HAL_RCC_USART2_IS_CLK_ENABLED
__HAL_RCC_USART3_IS_CLK_ENABLED
__HAL_RCC_UART4_IS_CLK_ENABLED
__HAL_RCC_UART5_IS_CLK_ENABLED
__HAL_RCC_I2C1_IS_CLK_ENABLED
__HAL_RCC_I2C2_IS_CLK_ENABLED
__HAL_RCC_I2C3_IS_CLK_ENABLED
__HAL_RCC_PWR_IS_CLK_ENABLED
__HAL_RCC_CAN1_IS_CLK_ENABLED
__HAL_RCC_CAN2_IS_CLK_ENABLED
__HAL_RCC_DAC_IS_CLK_ENABLED
__HAL_RCC_TIM2_IS_CLK_DISABLED
__HAL_RCC_TIM3_IS_CLK_DISABLED
__HAL_RCC_TIM4_IS_CLK_DISABLED
__HAL_RCC_TIM5_IS_CLK_DISABLED
__HAL_RCC_TIM6_IS_CLK_DISABLED
__HAL_RCC_TIM7_IS_CLK_DISABLED
__HAL_RCC_TIM12_IS_CLK_DISABLED
__HAL_RCC_TIM13_IS_CLK_DISABLED
__HAL_RCC_TIM14_IS_CLK_DISABLED
__HAL_RCC_WWDG_IS_CLK_DISABLED
__HAL_RCC_SPI2_IS_CLK_DISABLED
__HAL_RCC_SPI3_IS_CLK_DISABLED

__HAL_RCC_USART2_IS_CLK_DISABLED

__HAL_RCC_USART3_IS_CLK_DISABLED

__HAL_RCC_UART4_IS_CLK_DISABLED

__HAL_RCC_UART5_IS_CLK_DISABLED

__HAL_RCC_I2C1_IS_CLK_DISABLED

__HAL_RCC_I2C2_IS_CLK_DISABLED

__HAL_RCC_I2C3_IS_CLK_DISABLED

__HAL_RCC_PWR_IS_CLK_DISABLED

__HAL_RCC_CAN1_IS_CLK_DISABLED

__HAL_RCC_CAN2_IS_CLK_DISABLED

__HAL_RCC_DAC_IS_CLK_DISABLED

APB2 Peripheral Clock Enable Disable

__HAL_RCC_TIM1_CLK_ENABLE

__HAL_RCC_TIM8_CLK_ENABLE

__HAL_RCC_USART1_CLK_ENABLE

__HAL_RCC_USART6_CLK_ENABLE

__HAL_RCC_ADC1_CLK_ENABLE

__HAL_RCC_ADC2_CLK_ENABLE

__HAL_RCC_ADC3_CLK_ENABLE

__HAL_RCC_SDIO_CLK_ENABLE

__HAL_RCC_SPI1_CLK_ENABLE

__HAL_RCC_SYSCFG_CLK_ENABLE

__HAL_RCC_TIM9_CLK_ENABLE

__HAL_RCC_TIM10_CLK_ENABLE

__HAL_RCC_TIM11_CLK_ENABLE

__HAL_RCC_TIM1_CLK_DISABLE

__HAL_RCC_TIM8_CLK_DISABLE

__HAL_RCC_USART1_CLK_DISABLE

__HAL_RCC_USART6_CLK_DISABLE

__HAL_RCC_ADC1_CLK_DISABLE

__HAL_RCC_ADC2_CLK_DISABLE

__HAL_RCC_ADC3_CLK_DISABLE

__HAL_RCC_SDIO_CLK_DISABLE

__HAL_RCC_SPI1_CLK_DISABLE

__HAL_RCC_SYSCFG_CLK_DISABLE

__HAL_RCC_TIM9_CLK_DISABLE

__HAL_RCC_TIM10_CLK_DISABLE

__HAL_RCC_TIM11_CLK_DISABLE

APB2 Force Release Reset

__HAL_RCC_APB2_FORCE_RESET

__HAL_RCC_TIM1_FORCE_RESET

__HAL_RCC_TIM8_FORCE_RESET

__HAL_RCC_USART1_FORCE_RESET

__HAL_RCC_USART6_FORCE_RESET

__HAL_RCC_ADC_FORCE_RESET

__HAL_RCC_SDIO_FORCE_RESET

__HAL_RCC_SPI1_FORCE_RESET

__HAL_RCC_SYSCFG_FORCE_RESET

__HAL_RCC_TIM9_FORCE_RESET

__HAL_RCC_TIM10_FORCE_RESET

__HAL_RCC_TIM11_FORCE_RESET

__HAL_RCC_APB2_RELEASE_RESET

__HAL_RCC_TIM1_RELEASE_RESET

__HAL_RCC_TIM8_RELEASE_RESET

__HAL_RCC_USART1_RELEASE_RESET

__HAL_RCC_USART6_RELEASE_RESET

__HAL_RCC_ADC_RELEASE_RESET

__HAL_RCC_SDIO_RELEASE_RESET

__HAL_RCC_SPI1_RELEASE_RESET

__HAL_RCC_SYSCFG_RELEASE_RESET

__HAL_RCC_TIM9_RELEASE_RESET

__HAL_RCC_TIM10_RELEASE_RESET

__HAL_RCC_TIM11_RELEASE_RESET

APB2 Peripheral Low Power Enable Disable

__HAL_RCC_TIM1_CLK_SLEEP_ENABLE

__HAL_RCC_USART1_CLK_SLEEP_ENABLE

__HAL_RCC_USART6_CLK_SLEEP_ENABLE

__HAL_RCC_ADC1_CLK_SLEEP_ENABLE

__HAL_RCC_SDIO_CLK_SLEEP_ENABLE

__HAL_RCC_SPI1_CLK_SLEEP_ENABLE

__HAL_RCC_SYSCFG_CLK_SLEEP_ENABLE

__HAL_RCC_TIM8_CLK_SLEEP_ENABLE

__HAL_RCC_TIM9_CLK_SLEEP_ENABLE

__HAL_RCC_TIM10_CLK_SLEEP_ENABLE

__HAL_RCC_TIM11_CLK_SLEEP_ENABLE

__HAL_RCC_ADC2_CLK_SLEEP_ENABLE

__HAL_RCC_ADC3_CLK_SLEEP_ENABLE

__HAL_RCC_TIM1_CLK_SLEEP_DISABLE

__HAL_RCC_USART1_CLK_SLEEP_DISABLE

__HAL_RCC_USART6_CLK_SLEEP_DISABLE

__HAL_RCC_ADC1_CLK_SLEEP_DISABLE

__HAL_RCC_SDIO_CLK_SLEEP_DISABLE

__HAL_RCC_SPI1_CLK_SLEEP_DISABLE

__HAL_RCC_SYSCFG_CLK_SLEEP_DISABLE

__HAL_RCC_TIM8_CLK_SLEEP_DISABLE

__HAL_RCC_TIM9_CLK_SLEEP_DISABLE

__HAL_RCC_TIM10_CLK_SLEEP_DISABLE

__HAL_RCC_TIM11_CLK_SLEEP_DISABLE

__HAL_RCC_ADC2_CLK_SLEEP_DISABLE

__HAL_RCC_ADC3_CLK_SLEEP_DISABLE

APB2 Peripheral Clock Enable Disable Status

__HAL_RCC_TIM1_IS_CLK_ENABLED

__HAL_RCC_TIM8_IS_CLK_ENABLED

__HAL_RCC_USART1_IS_CLK_ENABLED

__HAL_RCC_USART6_IS_CLK_ENABLED

__HAL_RCC_ADC1_IS_CLK_ENABLED

__HAL_RCC_ADC2_IS_CLK_ENABLED

__HAL_RCC_ADC3_IS_CLK_ENABLED

__HAL_RCC_SDIO_IS_CLK_ENABLED

__HAL_RCC_SPI1_IS_CLK_ENABLED

__HAL_RCC_SYSCFG_IS_CLK_ENABLED

__HAL_RCC_TIM9_IS_CLK_ENABLED

__HAL_RCC_TIM10_IS_CLK_ENABLED

__HAL_RCC_TIM11_IS_CLK_ENABLED

__HAL_RCC_TIM1_IS_CLK_DISABLED

__HAL_RCC_TIM8_IS_CLK_DISABLED

__HAL_RCC_USART1_IS_CLK_DISABLED

__HAL_RCC_USART6_IS_CLK_DISABLED

__HAL_RCC_ADC1_IS_CLK_DISABLED

__HAL_RCC_ADC2_IS_CLK_DISABLED

__HAL_RCC_ADC3_IS_CLK_DISABLED

__HAL_RCC_SDIO_IS_CLK_DISABLED

__HAL_RCC_SPI1_IS_CLK_DISABLED

__HAL_RCC_SYSCFG_IS_CLK_DISABLED

__HAL_RCC_TIM9_IS_CLK_DISABLED

__HAL_RCC_TIM10_IS_CLK_DISABLED

__HAL_RCC_TIM11_IS_CLK_DISABLED

RCC BitAddress AliasRegion

RCC_OFFSET

RCC_CR_OFFSET

RCC_HSION_BIT_NUMBER

RCC_CR_HSION_BB

RCC_CSSON_BIT_NUMBER

RCC_CR_CSSON_BB

RCC_PLLON_BIT_NUMBER

RCC_CR_PLLON_BB

RCC_PLLI2SON_BIT_NUMBER

RCC_CR_PLLI2SON_BB

RCC_CFGR_OFFSET

RCC_I2SSRC_BIT_NUMBER

RCC_CFGR_I2SSRC_BB

RCC_BDCR_OFFSET

RCC_RTCEN_BIT_NUMBER

RCC_BDCR_RTCEN_BB

RCC_BDRST_BIT_NUMBER

RCC_BDCR_BDRST_BB

RCC_CSR_OFFSET

RCC_LSION_BIT_NUMBER

RCC_CSR_LSION_BB

RCC_CR_BYTE2_ADDRESS

RCC_CIR_BYTE1_ADDRESS

RCC_CIR_BYTE2_ADDRESS

RCC_BDCR_BYTE0_ADDRESS

RCC_DBP_TIMEOUT_VALUE

RCC_LSE_TIMEOUT_VALUE

HSE_TIMEOUT_VALUE

HSI_TIMEOUT_VALUE

LSI_TIMEOUT_VALUE

PLLI2S_TIMEOUT_VALUE

RCC Exported Macros

__HAL_RCC_PLL_PLLSOURCE_CONFIG

Description:

- Macro to configure the PLL clock source.

Parameters:

- __PLLSOURCE__**: specifies the PLL entry clock source. This parameter can be one of the following values:
 - RCC_PLLSOURCE_HSI: HSI oscillator clock selected as PLL clock entry
 - RCC_PLLSOURCE_HSE: HSE oscillator clock selected as PLL clock entry

Notes:

- This function must be used only when the main PLL is disabled.

__HAL_RCC_PLL_PLLM_CONFIG

Description:

- Macro to configure the PLL multiplication factor.

Parameters:

- __PLLM__**: specifies the division factor for PLL VCO input clock This parameter must be a number between Min_Data = 2 and Max_Data = 63.

Notes:

- This function must be used only when the main PLL is disabled.
- You have to set the PLLM parameter correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.

Flags

RCC_FLAG_HSIRDY

RCC_FLAG_HSERDY

RCC_FLAG_PLLRDY

RCC_FLAG_PLI2SRDY

RCC_FLAG_LSERDY

RCC_FLAG_LSIRDY

RCC_FLAG_BORRST

RCC_FLAG_PINRST

RCC_FLAG_PORRST

RCC_FLAG_SFTRST

RCC_FLAG_IWDGRST

RCC_FLAG_WWDGRST

RCC_FLAG_LPWRRST

Flags Interrupts Management

__HAL_RCC_ENABLE_IT

Description:

- Enable RCC interrupt (Perform Byte access to RCC_CIR[14:8] bits to enable the selected interrupts).

Parameters:

- __INTERRUPT__: specifies the RCC interrupt sources to be enabled. This parameter can be any combination of the following values:
 - RCC_IT_LSIRDY: LSI ready interrupt.
 - RCC_IT_LSERDY: LSE ready interrupt.
 - RCC_IT_HSIIRDY: HSI ready interrupt.
 - RCC_IT_HSERDY: HSE ready interrupt.
 - RCC_IT_PLLRDY: Main PLL ready interrupt.
 - RCC_IT_PLLI2SRDY: PLLI2S ready interrupt.

__HAL_RCC_DISABLE_IT

Description:

- Disable RCC interrupt (Perform Byte access to RCC_CIR[14:8] bits to disable the selected interrupts).

Parameters:

- __INTERRUPT__: specifies the RCC interrupt sources to be disabled. This parameter can be any combination of the following values:
 - RCC_IT_LSIRDY: LSI ready interrupt.
 - RCC_IT_LSERDY: LSE ready interrupt.
 - RCC_IT_HSIIRDY: HSI ready interrupt.
 - RCC_IT_HSERDY: HSE ready interrupt.
 - RCC_IT_PLLRDY: Main PLL ready interrupt.
 - RCC_IT_PLLI2SRDY: PLLI2S ready interrupt.

__HAL_RCC_CLEAR_IT

Description:

- Clear the RCC's interrupt pending bits (Perform Byte access to RCC_CIR[23:16] bits to clear the selected interrupt pending bits.

Parameters:

- `__INTERRUPT__`: specifies the interrupt pending bit to clear. This parameter can be any combination of the following values:
 - `RCC_IT_LSIRDY`: LSI ready interrupt.
 - `RCC_IT_LSERDY`: LSE ready interrupt.
 - `RCC_IT_HSIRDY`: HSI ready interrupt.
 - `RCC_IT_HSERDY`: HSE ready interrupt.
 - `RCC_IT_PLLRDY`: Main PLL ready interrupt.
 - `RCC_IT_PLLI2SRDY`: PLLI2S ready interrupt.
 - `RCC_IT_CSS`: Clock Security System interrupt

__HAL_RCC_GET_IT

Description:

- Check the RCC's interrupt has occurred or not.

Parameters:

- `__INTERRUPT__`: specifies the RCC interrupt source to check. This parameter can be one of the following values:
 - `RCC_IT_LSIRDY`: LSI ready interrupt.
 - `RCC_IT_LSERDY`: LSE ready interrupt.
 - `RCC_IT_HSIRDY`: HSI ready interrupt.
 - `RCC_IT_HSERDY`: HSE ready interrupt.
 - `RCC_IT_PLLRDY`: Main PLL ready interrupt.
 - `RCC_IT_PLLI2SRDY`: PLLI2S ready interrupt.
 - `RCC_IT_CSS`: Clock Security System interrupt

Return value:

- The: new state of `__INTERRUPT__` (TRUE or FALSE).

__HAL_RCC_CLEAR_RESET_FLAGS

RCC_FLAG_MASK

Description:

- Check RCC flag is set or not.

Parameters:

- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `RCC_FLAG_HSIIRDY`: HSI oscillator clock ready.
 - `RCC_FLAG_HSERDY`: HSE oscillator clock ready.
 - `RCC_FLAG_PLLRDY`: Main PLL clock ready.
 - `RCC_FLAG_PLLI2SRDY`: PLLI2S clock ready.
 - `RCC_FLAG_LSERDY`: LSE oscillator clock ready.
 - `RCC_FLAG_LSIRDY`: LSI oscillator clock ready.
 - `RCC_FLAG BORRST`: POR/PDR or BOR reset.
 - `RCC_FLAG_PINRST`: Pin reset.
 - `RCC_FLAG_PORRST`: POR/PDR reset.
 - `RCC_FLAG_SFTRST`: Software reset.
 - `RCC_FLAG_IWDGRST`: Independent Watchdog reset.
 - `RCC_FLAG_WWDGRST`: Window Watchdog reset.
 - `RCC_FLAG_LPWRST`: Low Power reset.

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_RCC_GET_FLAG

RCC_GET_PLL_OSCSOURCE

Get Clock source

__HAL_RCC_SYSCLK_CONFIG

Description:

- Macro to configure the system clock source.

Parameters:

- `__RCC_SYSCLKSOURCE__`: specifies the system clock source. This parameter can be one of the following values:
 - `RCC_SYSCLKSOURCE_HSI`: HSI oscillator is used as system clock source.
 - `RCC_SYSCLKSOURCE_HSE`: HSE oscillator is used as system clock source.
 - `RCC_SYSCLKSOURCE_PLLCLK`: PLL output is used as system clock source.

__HAL_RCC_GET_SYSCLK_SOURCE

Description:

- Macro to get the clock source used as system clock.

Return value:

- The: clock source used as system clock. The returned value can be one of the following:
 - `RCC_SYSCLKSOURCE_STATUS_HSI`: HSI used as system clock.
 - `RCC_SYSCLKSOURCE_STATUS_HSE`: HSE used as system clock.
 - `RCC_SYSCLKSOURCE_STATUS_PLLCLK`: PLL used as system clock.

__HAL_RCC_GET_PLL_OSCSOURCE

Description:

- Macro to get the oscillator used as PLL clock source.

Return value:

- The oscillator used as PLL clock source. The returned value can be one of the following:
 - RCC_PLLSOURCE_HSI: HSI oscillator is used as PLL clock source.
 - RCC_PLLSOURCE_HSE: HSE oscillator is used as PLL clock source.

HSE Config

RCC_HSE_OFF

RCC_HSE_ON

RCC_HSE_BYPASS

HSE Configuration

__HAL_RCC_HSE_CONFIG

Description:

- Macro to configure the External High Speed oscillator (HSE).

Parameters:

- __STATE__: specifies the new state of the HSE. This parameter can be one of the following values:
 - RCC_HSE_OFF: turn OFF the HSE oscillator, HSERDY flag goes low after 6 HSE oscillator clock cycles.
 - RCC_HSE_ON: turn ON the HSE oscillator.
 - RCC_HSE_BYPASS: HSE oscillator bypassed with external clock.

Notes:

- Transition HSE Bypass to HSE On and HSE On to HSE Bypass are not supported by this macro. User should request a transition to HSE Off first and then HSE On or HSE Bypass. After enabling the HSE (RCC_HSE_ON or RCC_HSE_Bypass), the application software should wait on HSERDY flag to be set indicating that HSE clock is stable and can be used to clock the PLL and/or system clock. HSE state can not be changed if it is used directly or through the PLL as system clock. In this case, you have to select another source of the system clock then change the HSE state (ex. disable it). The HSE is stopped by hardware when entering STOP and STANDBY modes. This function reset the CSSON bit, so if the clock security system (CSS) was previously enabled you have to enable it again after calling this function.

HSI Config

RCC_HSI_OFF

RCC_HSI_ON

RCC_HSICALIBRATION_DEFAULT

HSI Configuration

__HAL_RCC_HSI_ENABLE

Notes:

- The HSI is stopped by hardware when entering STOP and STANDBY modes. It is used (enabled by hardware) as system clock source after startup from Reset, wake-up from STOP and STANDBY mode, or in case of failure of the HSE used directly or indirectly as system clock (if the Clock Security System CSS is enabled). HSI can not be stopped if it is used as system clock source. In this case, you have to select another source of the system clock then stop the HSI. After enabling the HSI, the application software should wait on HSIRDY flag to be set indicating that HSI clock is stable and can be used as system clock source. This parameter can be: ENABLE or DISABLE. When the HSI is stopped, HSIRDY flag goes low after 6 HSI oscillator clock cycles.

__HAL_RCC_HSI_DISABLE

__HAL_RCC_HSI_CALIBRATIONVALUE_ADJUST

Description:

- Macro to adjust the Internal High Speed oscillator (HSI) calibration value.

Parameters:

- `__HSICalibrationValue__`: specifies the calibration trimming value. (default is `RCC_HSICALIBRATION_DEFAULT`). This parameter must be a number between 0 and 0x1F.

Notes:

- The calibration is used to compensate for the variations in voltage and temperature that influence the frequency of the internal HSI RC.

RTC Clock Configuration

__HAL_RCC_RTC_ENABLE

Notes:

- These macros must be used only after the RTC clock source was selected.

__HAL_RCC_RTC_DISABLE

__HAL_RCC_RTC_CLKPRESCALER

Description:

- Macros to configure the RTC clock (RTCCLK).

Parameters:

- `__RTCCLKSource__`: specifies the RTC clock source. This parameter can be one of the following values:
 - `RCC_RTCCLKSOURCE_LSE`: LSE selected as RTC clock.
 - `RCC_RTCCLKSOURCE_LSI`: LSI selected as RTC clock.
 - `RCC_RTCCLKSOURCE_HSE_DIVx`: HSE clock divided by x selected as RTC clock, where x:[2,31]

Notes:

- As the RTC clock configuration bits are in the Backup domain and write access is denied to this domain after reset, you have to enable write access using the Power Backup Access macro before to configure the RTC clock source (to be done once after reset). Once the RTC clock is configured it can't be changed unless the Backup domain is reset using `__HAL_RCC_BackupReset_RELEASE()` macro, or by a Power On Reset (POR).
- If the LSE or LSI is used as RTC clock source, the RTC continues to work in STOP and STANDBY modes, and can be used as wake-up source. However, when the HSE clock is used as RTC clock source, the RTC cannot be used in STOP and STANDBY modes. The maximum input clock frequency for RTC is 1MHz (when using HSE as RTC clock source).

__HAL_RCC_RTC_CONFIG

`__HAL_RCC_BACKUPRESET_FORCE`

Notes:

- This function resets the RTC peripheral (including the backup registers) and the RTC clock source selection in RCC_CSR register. The BKPSRAM is not affected by this reset.

`__HAL_RCC_BACKUPRESET_RELEASE`

Interrupts

`RCC_IT_LSIRDY`

`RCC_IT_LSERDY`

`RCC_IT_HSIRDY`

`RCC_IT_HSERDY`

`RCC_IT_PLLRDY`

`RCC_IT_PLLI2SRDY`

`RCC_IT_CSS`

RCC Private macros to check input parameters

`IS_RCC_OSCILLATORTYPE`

`IS_RCC_HSE`

`IS_RCC_LSE`

`IS_RCC_HSI`

`IS_RCC_LSI`

`IS_RCC_PLL`

`IS_RCC_PLLSOURCE`

`IS_RCC_SYSCLKSOURCE`

`IS_RCC_RTCCLKSOURCE`

`IS_RCC_PLLM_VALUE`

`IS_RCC_PLLN_VALUE`

`IS_RCC_PLLP_VALUE`

`IS_RCC_PLLQ_VALUE`

`IS_RCC_HCLK`

`IS_RCC_CLOCKTYPE`

`IS_RCC_PCLK`

IS_RCC_MCO

IS_RCC_MCO1SOURCE

IS_RCC_MCO2SOURCE

IS_RCC_MCODIV

IS_RCC_CALIBRATION_VALUE

LSE Config

RCC_LSE_OFF

RCC_LSE_ON

RCC_LSE_BYPASS

LSE Configuration

__HAL_RCC_LSE_CONFIG

Description:

- Macro to configure the External Low Speed oscillator (LSE).

Parameters:

- __STATE__: specifies the new state of the LSE. This parameter can be one of the following values:
 - RCC_LSE_OFF: turn OFF the LSE oscillator, LSERDY flag goes low after 6 LSE oscillator clock cycles.
 - RCC_LSE_ON: turn ON the LSE oscillator.
 - RCC_LSE_BYPASS: LSE oscillator bypassed with external clock.

Notes:

- Transition LSE Bypass to LSE On and LSE On to LSE Bypass are not supported by this macro. User should request a transition to LSE Off first and then LSE On or LSE Bypass. As the LSE is in the Backup domain and write access is denied to this domain after reset, you have to enable write access using HAL_PWR_EnableBkUpAccess() function before to configure the LSE (to be done once after reset). After enabling the LSE (RCC_LSE_ON or RCC_LSE_BYPASS), the application software should wait on LSERDY flag to be set indicating that LSE clock is stable and can be used to clock the RTC.

LSI Config

RCC_LSI_OFF

RCC_LSI_ON

LSI Configuration

__HAL_RCC_LSI_ENABLE

Notes:

- After enabling the LSI, the application software should wait on LSIRDY flag to be set indicating that LSI clock is stable and can be used to clock the IWDG and/or the RTC. LSI can not be disabled if the IWDG is running. When the LSI is stopped, LSIRDY flag goes low after 6 LSI oscillator clock cycles.

__HAL_RCC_LSI_DISABLE

MCO1 Clock Source

RCC_MCO1SOURCE_HSI

RCC_MCO1SOURCE_LSE

RCC_MCO1SOURCE_HSE

RCC_MCO1SOURCE_PLLCLK

MCO2 Clock Source

RCC_MCO2SOURCE_SYSCLK

RCC_MCO2SOURCE_PLI2SCLK

RCC_MCO2SOURCE_HSE

RCC_MCO2SOURCE_PLLCLK

MCOx Clock Prescaler

RCC_MCODIV_1

RCC_MCODIV_2

RCC_MCODIV_3

RCC_MCODIV_4

RCC_MCODIV_5

MCO Index

RCC_MCO1

RCC_MCO2

Oscillator Type

RCC_OSCILLATORTYPE_NONE

RCC_OSCILLATORTYPE_HSE

RCC_OSCILLATORTYPE_HSI

RCC_OSCILLATORTYPE_LSE

RCC_OSCILLATORTYPE_LSI

PLL Clock Divider

RCC_PLLP_DIV2

RCC_PLLP_DIV4

RCC_PLLP_DIV6

RCC_PLLP_DIV8

PLL Clock Source

RCC_PLLSOURCE_HSI

RCC_PLLSOURCE_HSE

PLL Config

RCC_PLL_NONE

RCC_PLL_OFF

RCC_PLL_ON

PLL Configuration

__HAL_RCC_PLL_ENABLE

Notes:

- After enabling the main PLL, the application software should wait on PLLRDY flag to be set indicating that PLL clock is stable and can be used as system clock source. The main PLL can not be disabled if it is used as system clock source. The main PLL is disabled by hardware when entering STOP and STANDBY modes.

__HAL_RCC_PLL_DISABLE

__HAL_RCC_PLL_CONFIG

Description:

- Macro to configure the main PLL clock source, multiplication and division factors.

Parameters:

- `__RCC_PLLSource__`: specifies the PLL entry clock source. This parameter can be one of the following values:
 - `RCC_PLLSOURCE_HSI`: HSI oscillator clock selected as PLL clock entry
 - `RCC_PLLSOURCE_HSE`: HSE oscillator clock selected as PLL clock entry
- `__PLLM__`: specifies the division factor for PLL VCO input clock. This parameter must be a number between `Min_Data = 2` and `Max_Data = 63`.
- `__PLLN__`: specifies the multiplication factor for PLL VCO output clock. This parameter must be a number between `Min_Data = 192` and `Max_Data = 432`.
- `__PLLP__`: specifies the division factor for main system clock (SYSCLK). This parameter must be a number in the range {2, 4, 6, or 8}.
- `__PLLQ__`: specifies the division factor for OTG FS, SDIO and RNG clocks. This parameter must be a number between `Min_Data = 2` and `Max_Data = 15`.

Notes:

- This function must be used only when the main PLL is disabled.
- This clock source (`RCC_PLLSource`) is common for the main PLL and PLLI2S.
- You have to set the `PLLM` parameter correctly to ensure that the VCO input frequency ranges from 1 to 2 MHz. It is recommended to select a frequency of 2 MHz to limit PLL jitter.
- You have to set the `PLLN` parameter correctly to ensure that the VCO output frequency is between 192 and 432 MHz.
- If the USB OTG FS is used in your application, you have to set the `PLLQ` parameter correctly to have 48 MHz clock for the USB. However, the SDIO and RNG need a frequency lower than or equal to 48 MHz to work correctly.

PLL I2S Configuration

__HAL_RCC_PLLI2S_ENABLE

Notes:

- The PLLI2S is disabled by hardware when entering STOP and STANDBY modes.

__HAL_RCC_PLLI2S_DISABLE

__HAL_RCC_PLLI2S_CONFIG

Description:

- Macro to configure the PLLI2S clock multiplication and division factors .

Parameters:

- `__PLLI2SN__`: specifies the multiplication factor for PLLI2S VCO output clock This parameter must be a number between Min_Data = 192 and Max_Data = 432.
- `__PLLI2SR__`: specifies the division factor for I2S clock This parameter must be a number between Min_Data = 2 and Max_Data = 7.

Notes:

- This macro must be used only when the PLLI2S is disabled. PLLI2S clock source is common with the main PLL (configured in HAL_RCC_ClockConfig() API).
- You have to set the PLLI2SN parameter correctly to ensure that the VCO output frequency is between Min_Data = 192 and Max_Data = 432 MHz.
- You have to set the PLLI2SR parameter correctly to not exceed 192 MHz on the I2S clock frequency.

__HAL_RCC_I2S_CONFIG

Description:

- Macro to configure the I2S clock source (I2SCLK).

Parameters:

- `__SOURCE__`: specifies the I2S clock source. This parameter can be one of the following values:
 - `RCC_I2SCLKSOURCE_PLLI2S`: PLLI2S clock used as I2S clock source.
 - `RCC_I2SCLKSOURCE_EXT`: External clock mapped on the I2S_CKIN pin used as I2S clock source.

Notes:

- This function must be called before enabling the I2S APB clock.

RTC Clock Source

RCC_RTCCLKSOURCE_LSE

RCC_RTCCLKSOURCE_LSI

RCC_RTCCLKSOURCE_HSE_DIV2

RCC_RTCCLKSOURCE_HSE_DIV3

RCC_RTCCLKSOURCE_HSE_DIV4

RCC_RTCCLKSOURCE_HSE_DIV5

RCC_RTCCLKSOURCE_HSE_DIV6

RCC_RTCCLKSOURCE_HSE_DIV7

RCC_RTCCLKSOURCE_HSE_DIV8

RCC_RTCCLKSOURCE_HSE_DIV9

RCC_RTCCLKSOURCE_HSE_DIV10

RCC_RTCCLKSOURCE_HSE_DIV11

RCC_RTCCLKSOURCE_HSE_DIV12

RCC_RTCCLKSOURCE_HSE_DIV13

RCC_RTCCLKSOURCE_HSE_DIV14

RCC_RTCCLKSOURCE_HSE_DIV15

RCC_RTCCLKSOURCE_HSE_DIV16

RCC_RTCCLKSOURCE_HSE_DIV17

RCC_RTCCLKSOURCE_HSE_DIV18

RCC_RTCCLKSOURCE_HSE_DIV19

RCC_RTCCLKSOURCE_HSE_DIV20

RCC_RTCCLKSOURCE_HSE_DIV21

RCC_RTCCLKSOURCE_HSE_DIV22

RCC_RTCCLKSOURCE_HSE_DIV23

RCC_RTCCLKSOURCE_HSE_DIV24

RCC_RTCCLKSOURCE_HSE_DIV25

RCC_RTCCLKSOURCE_HSE_DIV26

RCC_RTCCLKSOURCE_HSE_DIV27

RCC_RTCCLKSOURCE_HSE_DIV28

RCC_RTCCLKSOURCE_HSE_DIV29

RCC_RTCCLKSOURCE_HSE_DIV30

RCC_RTCCLKSOURCE_HSE_DIV31

System Clock Source

RCC_SYSCLKSOURCE_HSI

RCC_SYSCLKSOURCE_HSE

RCC_SYSCLKSOURCE_PLLCLK

System Clock Source Status

RCC_SYSCLKSOURCE_STATUS_HSI

HSI used as system clock

RCC_SYSCLKSOURCE_STATUS_HSE

HSE used as system clock

RCC_SYSCLKSOURCE_STATUS_PLLCLK

PLL used as system clock

System Clock Type

RCC_CLOCKTYPE_SYSCLK

RCC_CLOCKTYPE_HCLK

RCC_CLOCKTYPE_PCLK1

RCC_CLOCKTYPE_PCLK2

37 HAL RCC Extension Driver

37.1 RCCEX Firmware driver registers structures

37.1.1 RCC_PLLI2SInitTypeDef

RCC_PLLI2SInitTypeDef is defined in the stm32f2xx_hal_rcc_ex.h

Data Fields

- **uint32_t PLLI2SN**
- **uint32_t PLLI2SR**

Field Documentation

- **uint32_t RCC_PLLI2SInitTypeDef::PLLI2SN**
Specifies the multiplication factor for PLLI2S VCO output clock. This parameter must be a number between Min_Data = 192 and Max_Data = 432. This parameter will be used only when PLLI2S is selected as Clock Source I2S
- **uint32_t RCC_PLLI2SInitTypeDef::PLLI2SR**
Specifies the division factor for I2S clock. This parameter must be a number between Min_Data = 2 and Max_Data = 7. This parameter will be used only when PLLI2S is selected as Clock Source I2S

37.1.2 RCC_PeriphCLKInitTypeDef

RCC_PeriphCLKInitTypeDef is defined in the stm32f2xx_hal_rcc_ex.h

Data Fields

- **uint32_t PeriphClockSelection**
- **RCC_PLLI2SInitTypeDef PLLI2S**
- **uint32_t RTCClockSelection**
- **uint8_t TIMPresSelection**

Field Documentation

- **uint32_t RCC_PeriphCLKInitTypeDef::PeriphClockSelection**
The Extended Clock to be configured. This parameter can be a value of [RCCEX_Periph_Clock_Selection](#)
- **RCC_PLLI2SInitTypeDef RCC_PeriphCLKInitTypeDef::PLLI2S**
PLL I2S structure parameters. This parameter will be used only when PLLI2S is selected as Clock Source I2S
- **uint32_t RCC_PeriphCLKInitTypeDef::RTCClockSelection**
Specifies RTC Clock Prescalers Selection. This parameter can be a value of [RCC_RTC_Clock_Source](#)
- **uint8_t RCC_PeriphCLKInitTypeDef::TIMPresSelection**
Specifies TIM Clock Prescalers Selection. This parameter can be a value of [RCCEX_TIM_PRescaler_Selection](#)

37.2 RCCEX Firmware driver API description

The following section lists the various functions of the RCCEX library.

37.2.1 Extended Peripheral Control functions

This subsection provides a set of functions allowing to control the RCC Clocks frequencies.

Note: *Important note: Care must be taken when HAL_RCCEX_PeriphCLKConfig() is used to select the RTC clock source; in this case the Backup domain will be reset in order to modify the RTC Clock source, as consequence RTC registers (including the backup registers) and RCC_BDCR register are set to their reset values.*

This section contains the following APIs:

- [HAL_RCCEX_PeriphCLKConfig](#)
- [HAL_RCCEX_GetPeriphCLKConfig](#)

37.2.2 Detailed description of functions

HAL_RCCEX_PeriphCLKConfig

Function name

HAL_StatusTypeDef HAL_RCCEX_PeriphCLKConfig (RCC_PeriphCLKInitTypeDef * PeriphClkInit)

Function description

Initializes the RCC extended peripherals clocks according to the specified parameters in the RCC_PeriphCLKInitTypeDef.

Parameters

- **PeriphClkInit:** pointer to an RCC_PeriphCLKInitTypeDef structure that contains the configuration information for the Extended Peripherals clocks(I2S and RTC clocks).

Return values

- **HAL:** status

Notes

- A caution to be taken when HAL_RCCEX_PeriphCLKConfig() is used to select RTC clock selection, in this case the Reset of Backup domain will be applied in order to modify the RTC Clock source as consequence all backup domain (RTC and RCC_BDCR register expect BKPSRAM) will be reset

HAL_RCCEX_GetPeriphCLKConfig

Function name

void HAL_RCCEX_GetPeriphCLKConfig (RCC_PeriphCLKInitTypeDef * PeriphClkInit)

Function description

Configures the RCC_OscInitStruct according to the internal RCC configuration registers.

Parameters

- **PeriphClkInit:** pointer to an RCC_PeriphCLKInitTypeDef structure that will be configured.

Return values

- **None:**

37.3 RCCEX Firmware driver defines

The following section lists the various define and macros of the module.

37.3.1

RCCEX

RCCEX

AHB1 Peripheral Clock Enable Disable

__HAL_RCC_ETHMAC_CLK_ENABLE

__HAL_RCC_ETHMACTX_CLK_ENABLE

__HAL_RCC_ETHMACRX_CLK_ENABLE

__HAL_RCC_ETHMACPTP_CLK_ENABLE

__HAL_RCC_ETHMAC_CLK_DISABLE

__HAL_RCC_ETHMACTX_CLK_DISABLE

__HAL_RCC_ETHMACRX_CLK_DISABLE

__HAL_RCC_ETHMACPTP_CLK_DISABLE

__HAL_RCC_ETH_CLK_ENABLE

__HAL_RCC_ETH_CLK_DISABLE

AHB1 Force Release Reset

__HAL_RCC_ETHMAC_FORCE_RESET

__HAL_RCC_ETHMAC_RELEASE_RESET

AHB1 Peripheral Low Power Enable Disable

__HAL_RCC_ETHMAC_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACTX_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACRX_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMACPTP_CLK_SLEEP_ENABLE

__HAL_RCC_ETHMAC_CLK_SLEEP_DISABLE

__HAL_RCC_ETHMACTX_CLK_SLEEP_DISABLE

__HAL_RCC_ETHMACRX_CLK_SLEEP_DISABLE

__HAL_RCC_ETHMACPTP_CLK_SLEEP_DISABLE

AHB2 Peripheral Clock Enable Disable

__HAL_RCC_DCMI_CLK_ENABLE

__HAL_RCC_DCMI_CLK_DISABLE

__HAL_RCC_Cryp_CLK_ENABLE

__HAL_RCC_HASH_CLK_ENABLE

__HAL_RCC_Cryp_CLK_DISABLE

__HAL_RCC_HASH_CLK_DISABLE

AHB2 Force Release Reset

__HAL_RCC_DCMI_FORCE_RESET

__HAL_RCC_DCMI_RELEASE_RESET

__HAL_RCC_Cryp_FORCE_RESET

__HAL_RCC_HASH_FORCE_RESET

__HAL_RCC_Cryp_RELEASE_RESET

`__HAL_RCC_HASH_RELEASE_RESET`

AHB2 Peripheral Low Power Enable Disable

`__HAL_RCC_DCMI_CLK_SLEEP_ENABLE`

`__HAL_RCC_DCMI_CLK_SLEEP_DISABLE`

`__HAL_RCC_CRYPT_CLK_SLEEP_ENABLE`

`__HAL_RCC_HASH_CLK_SLEEP_ENABLE`

`__HAL_RCC_CRYPT_CLK_SLEEP_DISABLE`

`__HAL_RCC_HASH_CLK_SLEEP_DISABLE`

RCC BitAddress AliasRegion

`PLL_TIMEOUT_VALUE`

RCC Private macros to check input parameters

`IS_RCC_PERIPHCLK`

`IS_RCC_PLLI2SN_VALUE`

`IS_RCC_PLLI2SR_VALUE`

RCC Extended MCOx Clock Config

`__HAL_RCC_MCO1_CONFIG`

Description:

- Macro to configure the MCO1 clock.

Parameters:

- `__MCOCLKSOURCE__`: specifies the MCO clock source. This parameter can be one of the following values:
 - `RCC_MCO1SOURCE_HSI`: HSI clock selected as MCO1 source
 - `RCC_MCO1SOURCE_LSE`: LSE clock selected as MCO1 source
 - `RCC_MCO1SOURCE_HSE`: HSE clock selected as MCO1 source
 - `RCC_MCO1SOURCE_PLLCLK`: main PLL clock selected as MCO1 source
- `__MCODIV__`: specifies the MCO clock prescaler. This parameter can be one of the following values:
 - `RCC_MCODIV_1`: no division applied to MCOx clock
 - `RCC_MCODIV_2`: division by 2 applied to MCOx clock
 - `RCC_MCODIV_3`: division by 3 applied to MCOx clock
 - `RCC_MCODIV_4`: division by 4 applied to MCOx clock
 - `RCC_MCODIV_5`: division by 5 applied to MCOx clock

__HAL_RCC_MCO2_CONFIG

Description:

- Macro to configure the MCO2 clock.

Parameters:

- __MCOCLKSOURCE__: specifies the MCO clock source. This parameter can be one of the following values:
 - RCC_MCO2SOURCE_SYSCLK: System clock (SYSCLK) selected as MCO2 source
 - RCC_MCO2SOURCE_PLLI2SCLK: PLLI2S clock selected as MCO2 source
 - RCC_MCO2SOURCE_HSE: HSE clock selected as MCO2 source
 - RCC_MCO2SOURCE_PLLCLK: main PLL clock selected as MCO2 source
- __MCO DIV__: specifies the MCO clock prescaler. This parameter can be one of the following values:
 - RCC_MCO DIV_1: no division applied to MCOx clock
 - RCC_MCO DIV_2: division by 2 applied to MCOx clock
 - RCC_MCO DIV_3: division by 3 applied to MCOx clock
 - RCC_MCO DIV_4: division by 4 applied to MCOx clock
 - RCC_MCO DIV_5: division by 5 applied to MCOx clock

RCC Periph Clock Selection

RCC_PERIPHCLK_I2S

RCC_PERIPHCLK_TIM

RCC_PERIPHCLK_RTC

RCC_PERIPHCLK_PLLI2S

RCC TIM PRescaler Selection

RCC_TIMPRES_DESACTIVATED

RCC_TIMPRES_ACTIVATED

38 HAL RNG Generic Driver

38.1 RNG Firmware driver registers structures

38.1.1 RNG_HandleTypeDef

RNG_HandleTypeDef is defined in the stm32f2xx_hal_rng.h

Data Fields

- **RNG_TypeDef * Instance**
- **HAL_LockTypeDef Lock**
- **__IO HAL_RNG_StateTypeDef State**
- **__IO uint32_t ErrorCode**
- **uint32_t RandomNumber**

Field Documentation

- **RNG_TypeDef* RNG_HandleTypeDef::Instance**
Register base address
- **HAL_LockTypeDef RNG_HandleTypeDef::Lock**
RNG locking object
- **__IO HAL_RNG_StateTypeDef RNG_HandleTypeDef::State**
RNG communication state
- **__IO uint32_t RNG_HandleTypeDef::ErrorCode**
RNG Error code
- **uint32_t RNG_HandleTypeDef::RandomNumber**
Last Generated RNG Data

38.2 RNG Firmware driver API description

The following section lists the various functions of the RNG library.

38.2.1 How to use this driver

The RNG HAL driver can be used as follows:

1. Enable the RNG controller clock using `__HAL_RCC_RNG_CLK_ENABLE()` macro in `HAL_RNG_MspInit()`.
2. Activate the RNG peripheral using `HAL_RNG_Init()` function.
3. Wait until the 32 bit Random Number Generator contains a valid random data using (polling/interrupt) mode.
4. Get the 32 bit random number using `HAL_RNG_GenerateRandomNumber()` function.

38.2.2 Callback registration

The compilation define `USE_HAL_RNG_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `@ref HAL_RNG_RegisterCallback()` to register a user callback. Function `@ref HAL_RNG_RegisterCallback()` allows to register following callbacks:

- `ErrorCallback` : RNG Error Callback.
- `MspInitCallback` : RNG MspInit.
- `MspDeInitCallback` : RNG MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `@ref HAL_RNG_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `@ref HAL_RNG_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `ErrorCallback` : RNG Error Callback.
- `MspInitCallback` : RNG MspInit.

- MspDeInitCallback : RNG MspDeInit.

For specific callback ReadyDataCallback, use dedicated register callbacks: respectively @ref HAL_RNG_RegisterReadyDataCallback() , @ref HAL_RNG_UnRegisterReadyDataCallback().

By default, after the @ref HAL_RNG_Init() and when the state is HAL_RNG_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: example @ref HAL_RNG_ErrorCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_RNG_Init() and @ref HAL_RNG_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_RNG_Init() and @ref HAL_RNG_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_RNG_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_RNG_STATE_READY or HAL_RNG_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_RNG_RegisterCallback() before calling @ref HAL_RNG_DeInit() or @ref HAL_RNG_Init() function.

When The compilation define USE_HAL_RNG_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

38.2.3 Initialization and configuration functions

This section provides functions allowing to:

- Initialize the RNG according to the specified parameters in the RNG_InitTypeDef and create the associated handle
- DeInitialize the RNG peripheral
- Initialize the RNG MSP
- DeInitialize RNG MSP

This section contains the following APIs:

- [HAL_RNG_Init](#)
- [HAL_RNG_DeInit](#)
- [HAL_RNG_MspInit](#)
- [HAL_RNG_MspDeInit](#)

38.2.4 Peripheral Control functions

This section provides functions allowing to:

- Get the 32 bit Random number
- Get the 32 bit Random number with interrupt enabled
- Handle RNG interrupt request

This section contains the following APIs:

- [HAL_RNG_GenerateRandomNumber](#)
- [HAL_RNG_GenerateRandomNumber_IT](#)
- [HAL_RNG_GetRandomNumber](#)
- [HAL_RNG_GetRandomNumber_IT](#)
- [HAL_RNG_IRQHandler](#)
- [HAL_RNG_ReadLastRandomNumber](#)
- [HAL_RNG_ReadyDataCallback](#)
- [HAL_RNG_ErrorCallback](#)

38.2.5 Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [HAL_RNG_GetState](#)
- [HAL_RNG_GetError](#)

38.2.6 Detailed description of functions

HAL_RNG_Init

Function name

HAL_StatusTypeDef HAL_RNG_Init (RNG_HandleTypeDef * hrng)

Function description

Initializes the RNG peripheral and creates the associated handle.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_DeInit

Function name

HAL_StatusTypeDef HAL_RNG_DeInit (RNG_HandleTypeDef * hrng)

Function description

DeInitializes the RNG peripheral.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_MspInit

Function name

void HAL_RNG_MspInit (RNG_HandleTypeDef * hrng)

Function description

Initializes the RNG MSP.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None**:

HAL_RNG_MspDeInit

Function name

void HAL_RNG_MspDeInit (RNG_HandleTypeDef * hrng)

Function description

DeInitializes the RNG MSP.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None**:

HAL_RNG_GetRandomNumber

Function name

```
uint32_t HAL_RNG_GetRandomNumber (RNG_HandleTypeDef * hrng)
```

Function description

Returns generated random number in polling mode (Obsolete) Use HAL_RNG_GenerateRandomNumber() API instead.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **Random**: value

HAL_RNG_GetRandomNumber_IT

Function name

```
uint32_t HAL_RNG_GetRandomNumber_IT (RNG_HandleTypeDef * hrng)
```

Function description

Returns a 32-bit random number with interrupt enabled (Obsolete), Use HAL_RNG_GenerateRandomNumber_IT() API instead.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **32-bit**: random number

HAL_RNG_GenerateRandomNumber

Function name

```
HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber (RNG_HandleTypeDef * hrng, uint32_t * random32bit)
```

Function description

Generates a 32-bit random number.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.
- **random32bit**: pointer to generated random number variable if successful.

Return values

- **HAL**: status

Notes

- Each time the random number data is read the RNG_FLAG_DRDY flag is automatically cleared.

HAL_RNG_GenerateRandomNumber_IT

Function name

```
HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber_IT (RNG_HandleTypeDef * hrng)
```

Function description

Generates a 32-bit random number in interrupt mode.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL**: status

HAL_RNG_ReadLastRandomNumber

Function name

uint32_t HAL_RNG_ReadLastRandomNumber (RNG_HandleTypeDef * hrng)

Function description

Read latest generated random number.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **random**: value

HAL_RNG_IRQHandler

Function name

void HAL_RNG_IRQHandler (RNG_HandleTypeDef * hrng)

Function description

Handles RNG interrupt request.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None**:

Notes

- In the case of a clock error, the RNG is no more able to generate random numbers because the PLL48CLK clock is not correct. User has to check that the clock controller is correctly configured to provide the RNG clock and clear the CEIS bit using `__HAL_RNG_CLEAR_IT()`. The clock error has no impact on the previously generated random numbers, and the RNG_DR register contents can be used.
- In the case of a seed error, the generation of random numbers is interrupted as long as the SECS bit is '1'. If a number is available in the RNG_DR register, it must not be used because it may not have enough entropy. In this case, it is recommended to clear the SEIS bit using `__HAL_RNG_CLEAR_IT()`, then disable and enable the RNG peripheral to reinitialize and restart the RNG.
- User-written `HAL_RNG_ErrorCallback()` API is called once whether SEIS or CEIS are set.

HAL_RNG_ErrorCallback

Function name

void HAL_RNG_ErrorCallback (RNG_HandleTypeDef * hrng)

Function description

RNG error callbacks.

Parameters

- **hrng**: pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **None:**

HAL_RNG_ReadyDataCallback

Function name

void HAL_RNG_ReadyDataCallback (RNG_HandleTypeDef * hrng, uint32_t random32bit)

Function description

Data Ready callback in non-blocking mode.

Parameters

- **hrng:** pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.
- **random32bit:** generated random number.

Return values

- **None:**

HAL_RNG_GetState

Function name

HAL_RNG_StateTypeDef HAL_RNG_GetState (RNG_HandleTypeDef * hrng)

Function description

Returns the RNG state.

Parameters

- **hrng:** pointer to a RNG_HandleTypeDef structure that contains the configuration information for RNG.

Return values

- **HAL:** state

HAL_RNG_GetError

Function name

uint32_t HAL_RNG_GetError (RNG_HandleTypeDef * hrng)

Function description

Return the RNG handle error code.

Parameters

- **hrng:** pointer to a RNG_HandleTypeDef structure.

Return values

- **RNG:** Error Code

38.3 RNG Firmware driver defines

The following section lists the various define and macros of the module.

38.3.1 RNG

RNG

RNG Error Definition

HAL_RNG_ERROR_NONE

No error

HAL_RNG_ERROR_TIMEOUT

Timeout error

HAL_RNG_ERROR_BUSY

Busy error

HAL_RNG_ERROR_SEED

Seed error

HAL_RNG_ERROR_CLOCK

Clock error

RNG Interrupt definition

RNG_IT_DRDY

Data Ready interrupt

RNG_IT_CEI

Clock error interrupt

RNG_IT_SEI

Seed error interrupt

RNG Flag definition

RNG_FLAG_DRDY

Data ready

RNG_FLAG_CECS

Clock error current status

RNG_FLAG_SECS

Seed error current status

RNG Exported Macros

__HAL_RNG_RESET_HANDLE_STATE

Description:

- Reset RNG handle state.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_ENABLE

Description:

- Enables the RNG peripheral.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_DISABLE

Description:

- Disables the RNG peripheral.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_GET_FLAG

Description:

- Check the selected RNG flag status.

Parameters:

- `__HANDLE__`: RNG Handle
- `__FLAG__`: RNG flag This parameter can be one of the following values:
 - `RNG_FLAG_DRDY`: Data ready
 - `RNG_FLAG_CECS`: Clock error current status
 - `RNG_FLAG_SECS`: Seed error current status

Return value:

- The: new state of `__FLAG__` (SET or RESET).

__HAL_RNG_CLEAR_FLAG

Description:

- Clears the selected RNG flag status.

Parameters:

- `__HANDLE__`: RNG handle
- `__FLAG__`: RNG flag to clear

Return value:

- None

Notes:

- WARNING: This is a dummy macro for HAL code alignment, flags `RNG_FLAG_DRDY`, `RNG_FLAG_CECS` and `RNG_FLAG_SECS` are read-only.

__HAL_RNG_ENABLE_IT

Description:

- Enables the RNG interrupts.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_DISABLE_IT

Description:

- Disables the RNG interrupts.

Parameters:

- `__HANDLE__`: RNG Handle

Return value:

- None

__HAL_RNG_GET_IT

Description:

- Checks whether the specified RNG interrupt has occurred or not.

Parameters:

- `__HANDLE__`: RNG Handle
- `__INTERRUPT__`: specifies the RNG interrupt status flag to check. This parameter can be one of the following values:
 - `RNG_IT_DRDY`: Data ready interrupt
 - `RNG_IT_CEI`: Clock error interrupt
 - `RNG_IT_SEI`: Seed error interrupt

Return value:

- The: new state of `__INTERRUPT__` (SET or RESET).

__HAL_RNG_CLEAR_IT

Description:

- Clear the RNG interrupt status flags.

Parameters:

- `__HANDLE__`: RNG Handle
- `__INTERRUPT__`: specifies the RNG interrupt status flag to clear. This parameter can be one of the following values:
 - `RNG_IT_CEI`: Clock error interrupt
 - `RNG_IT_SEI`: Seed error interrupt

Return value:

- None

Notes:

- `RNG_IT_DRDY` flag is read-only, reading `RNG_DR` register automatically clears `RNG_IT_DRDY`.

39 HAL RTC Generic Driver

39.1 RTC Firmware driver registers structures

39.1.1 RTC_InitTypeDef

RTC_InitTypeDef is defined in the stm32f2xx_hal_rtc.h

Data Fields

- **uint32_t HourFormat**
- **uint32_t AsynchPrediv**
- **uint32_t SynchPrediv**
- **uint32_t OutPut**
- **uint32_t OutPutPolarity**
- **uint32_t OutPutType**

Field Documentation

- **uint32_t RTC_InitTypeDef::HourFormat**
Specifies the RTC Hour Format. This parameter can be a value of [RTC_Hour_Formats](#)
- **uint32_t RTC_InitTypeDef::AsynchPrediv**
Specifies the RTC Asynchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x7F
- **uint32_t RTC_InitTypeDef::SynchPrediv**
Specifies the RTC Synchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x1FFF
- **uint32_t RTC_InitTypeDef::OutPut**
Specifies which signal will be routed to the RTC output. This parameter can be a value of [RTC_Output_selection_Definitions](#)
- **uint32_t RTC_InitTypeDef::OutPutPolarity**
Specifies the polarity of the output signal. This parameter can be a value of [RTC_Output_Polarity_Definitions](#)
- **uint32_t RTC_InitTypeDef::OutPutType**
Specifies the RTC Output Pin mode. This parameter can be a value of [RTC_Output_Type_ALARM_OUT](#)

39.1.2 RTC_TimeTypeDef

RTC_TimeTypeDef is defined in the stm32f2xx_hal_rtc.h

Data Fields

- **uint8_t Hours**
- **uint8_t Minutes**
- **uint8_t Seconds**
- **uint8_t TimeFormat**
- **uint32_t DayLightSaving**
- **uint32_t StoreOperation**

Field Documentation

- **uint8_t RTC_TimeTypeDef::Hours**
Specifies the RTC Time Hour. This parameter must be a number between Min_Data = 0 and Max_Data = 12 if the RTC_HourFormat_12 is selected. This parameter must be a number between Min_Data = 0 and Max_Data = 23 if the RTC_HourFormat_24 is selected
- **uint8_t RTC_TimeTypeDef::Minutes**
Specifies the RTC Time Minutes. This parameter must be a number between Min_Data = 0 and Max_Data = 59

- **`uint8_t RTC_TimeTypeDef::Seconds`**
Specifies the RTC Time Seconds. This parameter must be a number between Min_Data = 0 and Max_Data = 59
- **`uint8_t RTC_TimeTypeDef::TimeFormat`**
Specifies the RTC AM/PM Time. This parameter can be a value of [RTC_AM_PM_Definitions](#)
- **`uint32_t RTC_TimeTypeDef::DayLightSaving`**
Specifies DayLight Save Operation. This parameter can be a value of [RTC_DayLightSaving_Definitions](#)
- **`uint32_t RTC_TimeTypeDef::StoreOperation`**
Specifies RTC_StoreOperation value to be written in the BCK bit in CR register to store the operation. This parameter can be a value of [RTC_StoreOperation_Definitions](#)

39.1.3

RTC_DateTypeDef

RTC_DateTypeDef is defined in the `stm32f2xx_hal_rtc.h`

Data Fields

- **`uint8_t WeekDay`**
- **`uint8_t Month`**
- **`uint8_t Date`**
- **`uint8_t Year`**

Field Documentation

- **`uint8_t RTC_DateTypeDef::WeekDay`**
Specifies the RTC Date WeekDay. This parameter can be a value of [RTC_WeekDay_Definitions](#)
- **`uint8_t RTC_DateTypeDef::Month`**
Specifies the RTC Date Month (in BCD format). This parameter can be a value of [RTC_Month_Date_Definitions](#)
- **`uint8_t RTC_DateTypeDef::Date`**
Specifies the RTC Date. This parameter must be a number between Min_Data = 1 and Max_Data = 31
- **`uint8_t RTC_DateTypeDef::Year`**
Specifies the RTC Date Year. This parameter must be a number between Min_Data = 0 and Max_Data = 99

39.1.4

RTC_AlarmTypeDef

RTC_AlarmTypeDef is defined in the `stm32f2xx_hal_rtc.h`

Data Fields

- **`RTC_TimeTypeDef AlarmTime`**
- **`uint32_t AlarmMask`**
- **`uint32_t AlarmDateWeekDaySel`**
- **`uint8_t AlarmDateWeekDay`**
- **`uint32_t Alarm`**

Field Documentation

- **`RTC_TimeTypeDef RTC_AlarmTypeDef::AlarmTime`**
Specifies the RTC Alarm Time members
- **`uint32_t RTC_AlarmTypeDef::AlarmMask`**
Specifies the RTC Alarm Masks. This parameter can be a value of [RTC_AlarmMask_Definitions](#)
- **`uint32_t RTC_AlarmTypeDef::AlarmDateWeekDaySel`**
Specifies the RTC Alarm is on Date or WeekDay. This parameter can be a value of [RTC_AlarmDateWeekDay_Definitions](#)
- **`uint8_t RTC_AlarmTypeDef::AlarmDateWeekDay`**
Specifies the RTC Alarm Date/WeekDay. If the Alarm Date is selected, this parameter must be set to a value in the 1-31 range. If the Alarm WeekDay is selected, this parameter can be a value of [RTC_WeekDay_Definitions](#)
- **`uint32_t RTC_AlarmTypeDef::Alarm`**
Specifies the alarm . This parameter can be a value of [RTC_Alarms_Definitions](#)

39.1.5

RTC_HandleTypeDef

RTC_HandleTypeDef is defined in the stm32f2xx_hal_rtc.h

Data Fields

- **RTC_TypeDef * Instance**
- **RTC_InitTypeDef Init**
- **HAL_LockTypeDef Lock**
- **__IO HAL_RTCStateTypeDef State**

Field Documentation

- **RTC_TypeDef* RTC_HandleTypeDef::Instance**
Register base address
- **RTC_InitTypeDef RTC_HandleTypeDef::Init**
RTC required parameters
- **HAL_LockTypeDef RTC_HandleTypeDef::Lock**
RTC locking object
- **__IO HAL_RTCStateTypeDef RTC_HandleTypeDef::State**
Time communication state

39.2

RTC Firmware driver API description

The following section lists the various functions of the RTC library.

39.2.1

Backup Domain Operating Condition

The real-time clock (RTC), the RTC backup registers, and the backup SRAM (BKP SRAM) can be powered from the VBAT voltage when the main VDD supply is powered off. To retain the content of the RTC backup registers, backup SRAM, and supply the RTC when VDD is turned off, VBAT pin can be connected to an optional standby voltage supplied by a battery or by another source.

To allow the RTC operating even when the main digital supply (VDD) is turned off, the VBAT pin powers the following blocks:

1. The RTC
2. The LSE oscillator
3. The backup SRAM when the low power backup regulator is enabled
4. PC13 to PC15 I/Os, plus PI8 I/O (when available)

When the backup domain is supplied by VDD (analog switch connected to VDD), the following pins are available:

1. PC14 and PC15 can be used as either GPIO or LSE pins
2. PC13 can be used as a GPIO or as the RTC_AF1 pin
3. PI8 can be used as a GPIO or as the RTC_AF2 pin

When the backup domain is supplied by VBAT (analog switch connected to VBAT because VDD is not present), the following pins are available:

1. PC14 and PC15 can be used as LSE pins only
2. PC13 can be used as the RTC_AF1 pin
3. PI8 can be used as the RTC_AF2 pin

39.2.2

Backup Domain Reset

The backup domain reset sets all RTC registers and the RCC_BDCR register to their reset values. The BKPSRAM is not affected by this reset. The only way to reset the BKPSRAM is through the Flash interface by requesting a protection level change from 1 to 0.

A backup domain reset is generated when one of the following events occurs:

1. Software reset, triggered by setting the BDRST bit in the RCC Backup domain control register (RCC_BDCR).
2. VDD or VBAT power on, if both supplies have previously been powered off.

39.2.3

Backup Domain Access

After reset, the backup domain (RTC registers, RTC backup data registers and backup SRAM) is protected against possible unwanted write accesses.

To enable access to the RTC Domain and RTC registers, proceed as follows:

- Enable the Power Controller (PWR) APB1 interface clock using the `__HAL_RCC_PWR_CLK_ENABLE()` function.
- Enable access to RTC domain using the `HAL_PWR_EnableBkUpAccess()` function.
- Select the RTC clock source using the `__HAL_RCC_RTC_CONFIG()` function.
- Enable RTC Clock using the `__HAL_RCC_RTC_ENABLE()` function.

39.2.4

How to use this driver

- Enable the RTC domain access (see description in the section above).
- Configure the RTC Prescaler (Asynchronous and Synchronous) and RTC hour format using the `HAL_RTC_Init()` function.

Time and Date configuration

- To configure the RTC Calendar (Time and Date) use the `HAL_RTC_SetTime()` and `HAL_RTC_SetDate()` functions.
- To read the RTC Calendar, use the `HAL_RTC_GetTime()` and `HAL_RTC_GetDate()` functions.

Alarm configuration

- To configure the RTC Alarm use the `HAL_RTC_SetAlarm()` function. You can also configure the RTC Alarm with interrupt mode using the `HAL_RTC_SetAlarm_IT()` function.
- To read the RTC Alarm, use the `HAL_RTC_GetAlarm()` function.

39.2.5

RTC and low power modes

The MCU can be woken up from a low power mode by an RTC alternate function.

The RTC alternate functions are the RTC alarms (Alarm A and Alarm B), RTC wake-up, RTC tamper event detection and RTC time stamp event detection. These RTC alternate functions can wake up the system from the Stop and Standby low power modes.

The system can also wake up from low power modes without depending on an external interrupt (Auto-wake-up mode), by using the RTC alarm or the RTC wake-up events.

The RTC provides a programmable time base for waking up from the Stop or Standby mode at regular intervals. Wake-up from STOP and STANDBY modes is possible only when the RTC clock source is LSE or LSI.

Callback registration

The compilation define `USE_HAL_RTC_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Function `@ref HAL_RTC_RegisterCallback()` to register an interrupt callback.

Function `@ref HAL_RTC_RegisterCallback()` allows to register following callbacks:

- `AlarmAEventCallback` : RTC Alarm A Event callback.
- `AlarmBEventCallback` : RTC Alarm B Event callback.
- `TimeStampEventCallback` : RTC TimeStamp Event callback.
- `WakeUpTimerEventCallback` : RTC WakeUpTimer Event callback.
- `Tamper1EventCallback` : RTC Tamper 1 Event callback.
- `MspInitCallback` : RTC MspInit callback.
- `MspDeInitCallback` : RTC MspDeInit callback.

This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `@ref HAL_RTC_UnRegisterCallback()` to reset a callback to the default weak function. `@ref HAL_RTC_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `AlarmAEventCallback` : RTC Alarm A Event callback.

- AlarmBEventCallback : RTC Alarm B Event callback.
- TimeStampEventCallback : RTC TimeStamp Event callback.
- WakeUpTimerEventCallback : RTC WakeUpTimer Event callback.
- Tamper1EventCallback : RTC Tamper 1 Event callback.
- MspInitCallback : RTC MspInit callback.
- MspDeInitCallback : RTC MspDeInit callback.

By default, after the @ref HAL_RTC_Init() and when the state is HAL_RTC_STATE_RESET, all callbacks are set to the corresponding weak functions : examples @ref AlarmAEventCallback(), @ref WakeUpTimerEventCallback(). Exception done for MspInit and MspDeInit callbacks that are reset to the legacy weak function in the @ref HAL_RTC_Init()/@ref HAL_RTC_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, @ref HAL_RTC_Init()/@ref HAL_RTC_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand)

Callbacks can be registered/unregistered in HAL_RTC_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_RTC_STATE_READY or HAL_RTC_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_RTC_RegisterCallback() before calling @ref HAL_RTC_DeInit() or @ref HAL_RTC_Init() function.

When The compilation define USE_HAL_RTC_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

39.2.6 Initialization and de-initialization functions

This section provides functions allowing to initialize and configure the RTC Prescaler (Synchronous and Asynchronous), RTC Hour format, disable RTC registers Write protection, enter and exit the RTC initialization mode, RTC registers synchronization check and reference clock detection enable.

1. The RTC Prescaler is programmed to generate the RTC 1Hz time base. It is split into 2 programmable prescalers to minimize power consumption.
 - A 7-bit asynchronous prescaler and a 13-bit synchronous prescaler.
 - When both prescalers are used, it is recommended to configure the asynchronous prescaler to a high value to minimize power consumption.
2. All RTC registers are Write protected. Writing to the RTC registers is enabled by writing a key into the Write Protection register, RTC_WPR.
3. To configure the RTC Calendar, user application should enter initialization mode. In this mode, the calendar counter is stopped and its value can be updated. When the initialization sequence is complete, the calendar restarts counting after 4 RTCCLK cycles.
4. To read the calendar through the shadow registers after Calendar initialization, calendar update or after wake-up from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers. The HAL_RTC_WaitForSynchro() function implements the above software sequence (RSF clear and RSF check).

This section contains the following APIs:

- [HAL_RTC_Init](#)
- [HAL_RTC_DeInit](#)
- [HAL_RTC_MspInit](#)
- [HAL_RTC_MspDeInit](#)

39.2.7 RTC Time and Date functions

This section provides functions allowing to configure Time and Date features

This section contains the following APIs:

- [HAL_RTC_SetTime](#)
- [HAL_RTC_GetTime](#)
- [HAL_RTC_SetDate](#)
- [HAL_RTC_GetDate](#)

39.2.8 RTC Alarm functions

This section provides functions allowing to configure Alarm feature

This section contains the following APIs:

- [HAL_RTC_SetAlarm](#)
- [HAL_RTC_SetAlarm_IT](#)
- [HAL_RTC_DeactivateAlarm](#)
- [HAL_RTC_GetAlarm](#)
- [HAL_RTC_AlarmIRQHandler](#)
- [HAL_RTC_AlarmAEventCallback](#)
- [HAL_RTC_PollForAlarmAEvent](#)

39.2.9 Peripheral Control functions

This subsection provides functions allowing to

- Wait for RTC Time and Date Synchronization

This section contains the following APIs:

- [HAL_RTC_WaitForSynchro](#)

39.2.10 Peripheral State functions

This subsection provides functions allowing to

- Get RTC state

This section contains the following APIs:

- [HAL_RTC_GetState](#)

39.2.11 Detailed description of functions

HAL_RTC_Init

Function name

HAL_StatusTypeDef HAL_RTC_Init (RTC_HandleTypeDef * hrtc)

Function description

Initializes the RTC peripheral.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTC_DeInit

Function name

HAL_StatusTypeDef HAL_RTC_DeInit (RTC_HandleTypeDef * hrtc)

Function description

DeInitializes the RTC peripheral.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

Notes

- This function doesn't reset the RTC Backup Data registers.

HAL_RTC_Msplnit

Function name

void HAL_RTC_Msplnit (RTC_HandleTypeDef * hrtc)

Function description

Initializes the RTC MSP.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_MspDeInit

Function name

void HAL_RTC_MspDeInit (RTC_HandleTypeDef * hrtc)

Function description

DeInitializes the RTC MSP.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_SetTime

Function name

HAL_StatusTypeDef HAL_RTC_SetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)

Function description

Sets RTC current time.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTime**: Pointer to Time structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

HAL_RTC_GetTime

Function name

HAL_StatusTypeDef HAL_RTC_GetTime (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTime, uint32_t Format)

Function description

Gets RTC current time.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTime**: Pointer to Time structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until Current date is read.

HAL_RTC_SetDate

Function name

HAL_StatusTypeDef HAL_RTC_SetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)

Function description

Sets RTC current date.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sDate**: Pointer to date structure
- **Format**: specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

HAL_RTC_GetDate

Function name

HAL_StatusTypeDef HAL_RTC_GetDate (RTC_HandleTypeDef * hrtc, RTC_DateTypeDef * sDate, uint32_t Format)

Function description

Gets RTC current date.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sDate**: Pointer to Date structure
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

Notes

- You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to unlock the values in the higher-order calendar shadow registers to ensure consistency between the time and date values. Reading RTC current time locks the values in calendar shadow registers until Current date is read.

HAL_RTC_SetAlarm

Function name

HAL_StatusTypeDef HAL_RTC_SetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)

Function description

Sets the specified RTC Alarm.

Parameters

- hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- sAlarm**: Pointer to Alarm structure
- Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- HAL**: status

HAL_RTC_SetAlarm_IT

Function name

HAL_StatusTypeDef HAL_RTC_SetAlarm_IT (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Format)

Function description

Sets the specified RTC Alarm with Interrupt.

Parameters

- hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- sAlarm**: Pointer to Alarm structure
- Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- HAL**: status

HAL_RTC_DeactivateAlarm

Function name

HAL_StatusTypeDef HAL_RTC_DeactivateAlarm (RTC_HandleTypeDef * hrtc, uint32_t Alarm)

Function description

Deactivate the specified RTC Alarm.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Alarm**: Specifies the Alarm. This parameter can be one of the following values:
 - RTC_ALARM_A: AlarmA
 - RTC_ALARM_B: AlarmB

Return values

- **HAL**: status

HAL_RTC_GetAlarm

Function name

HAL_StatusTypeDef HAL_RTC_GetAlarm (RTC_HandleTypeDef * hrtc, RTC_AlarmTypeDef * sAlarm, uint32_t Alarm, uint32_t Format)

Function description

Gets the RTC Alarm value and masks.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sAlarm**: Pointer to Date structure
- **Alarm**: Specifies the Alarm. This parameter can be one of the following values:
 - RTC_ALARM_A: AlarmA
 - RTC_ALARM_B: AlarmB
- **Format**: Specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

HAL_RTC_AlarmIRQHandler

Function name

void HAL_RTC_AlarmIRQHandler (RTC_HandleTypeDef * hrtc)

Function description

This function handles Alarm interrupt request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTC_PollForAlarmAEvent

Function name

HAL_StatusTypeDef HAL_RTC_PollForAlarmAEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

This function handles AlarmA Polling request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTC_AlarmAEventCallback

Function name

void HAL_RTC_AlarmAEventCallback (RTC_HandleTypeDef * hrtc)

Function description

Alarm A callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTC_WaitForSynchro

Function name

HAL_StatusTypeDef HAL_RTC_WaitForSynchro (RTC_HandleTypeDef * hrtc)

Function description

Waits until the RTC Time and Date registers (RTC_TR and RTC_DR) are synchronized with RTC APB clock.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

Notes

- The RTC Resynchronization mode is write protected, use the `__HAL_RTC_WRITEPROTECTION_DISABLE()` before calling this function.
- To read the calendar through the shadow registers after Calendar initialization, calendar update or after wake-up from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers.

HAL_RTC_GetState

Function name

HAL_RTCStateTypeDef HAL_RTC_GetState (RTC_HandleTypeDef * hrtc)

Function description

Returns the RTC state.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** state

RTC_EnterInitMode

Function name

HAL_StatusTypeDef RTC_EnterInitMode (RTC_HandleTypeDef * hrtc)

Function description

Enters the RTC Initialization mode.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

Notes

- The RTC Initialization mode is write protected, use the `__HAL_RTC_WRITEPROTECTION_DISABLE()` before calling this function.

RTC_ByteToBcd2

Function name

uint8_t RTC_ByteToBcd2 (uint8_t Value)

Function description

Converts a 2 digit decimal to BCD format.

Parameters

- **Value**: Byte to be converted

Return values

- **Converted**: byte

RTC_Bcd2ToByte

Function name

uint8_t RTC_Bcd2ToByte (uint8_t Value)

Function description

Converts from 2 digit BCD to Binary.

Parameters

- **Value**: BCD value to be converted

Return values

- **Converted**: word

39.3 RTC Firmware driver defines

The following section lists the various define and macros of the module.

39.3.1 RTC

RTC

RTC Alarm Date WeekDay Definitions

RTC_ALARMDATEWEEKDAYSEL_DATE

RTC_ALARMDATEWEEKDAYSEL_WEEKDAY

RTC Alarm Mask Definitions

RTC_ALARMMASK_NONE

RTC_ALARMMASK_DATEWEEKDAY

RTC_ALARMMASK_HOURS

RTC_ALARMMASK_MINUTES

RTC_ALARMMASK_SECONDS

RTC_ALARMMASK_ALL

RTC Alarms Definitions

RTC_ALARM_A

RTC_ALARM_B

RTC AM PM Definitions

RTC_HOURFORMAT12_AM

RTC_HOURFORMAT12_PM

RTC DayLight Saving Definitions

RTC_DAYLIGHTSAVING_SUB1H

RTC_DAYLIGHTSAVING_ADD1H

RTC_DAYLIGHTSAVING_NONE

RTC Exported Macros

__HAL_RTC_RESET_HANDLE_STATE

Description:

- Reset RTC handle state.

Parameters:

- __HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WRITEPROTECTION_DISABLE

Description:

- Disable the write protection for RTC registers.

Parameters:

- __HANDLE__**: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_WRITEPROTECTION_ENABLE`

Description:

- Enable the write protection for RTC registers.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_ALARMA_ENABLE`

Description:

- Enable the RTC ALARMA peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_ALARMA_DISABLE`

Description:

- Disable the RTC ALARMA peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_ALARMB_ENABLE`

Description:

- Enable the RTC ALARMB peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_ALARMB_DISABLE`

Description:

- Disable the RTC ALARMB peripheral.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

`__HAL_RTC_ALARM_ENABLE_IT`

Description:

- Enable the RTC Alarm interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

`__HAL_RTC_ALARM_DISABLE_IT`

Description:

- Disable the RTC Alarm interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt sources to be enabled or disabled. This parameter can be any combination of the following values:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

`__HAL_RTC_ALARM_GET_IT`

Description:

- Check whether the specified RTC Alarm interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Alarm interrupt to check. This parameter can be:
 - `RTC_IT_ALRA`: Alarm A interrupt
 - `RTC_IT_ALRB`: Alarm B interrupt

Return value:

- None

`__HAL_RTC_ALARM_GET_FLAG`

Description:

- Get the selected RTC Alarm's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Alarm Flag to check. This parameter can be:
 - `RTC_FLAG_ALRAF`
 - `RTC_FLAG_ALRBF`
 - `RTC_FLAG_ALRAWF`
 - `RTC_FLAG_ALRBWF`

Return value:

- None

__HAL_RTC_ALARM_CLEAR_FLAG

Description:

- Clear the RTC Alarm's pending flags.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__FLAG__**: specifies the RTC Alarm Flag sources to be enabled or disabled. This parameter can be:
 - **RTC_FLAG_ALRAF**
 - **RTC_FLAG_ALRBF**

Return value:

- None

__HAL_RTC_ALARM_GET_IT_SOURCE

Description:

- Check whether the specified RTC Alarm interrupt has been enabled or not.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__INTERRUPT__**: specifies the RTC Alarm interrupt sources to check. This parameter can be:
 - **RTC_IT_ALRA**: Alarm A interrupt
 - **RTC_IT_ALRB**: Alarm B interrupt

Return value:

- None

__HAL_RTC_ALARM_EXTI_ENABLE_IT

Description:

- Enable interrupt on the RTC Alarm associated Exti line.

Return value:

- None

__HAL_RTC_ALARM_EXTI_DISABLE_IT

Description:

- Disable interrupt on the RTC Alarm associated Exti line.

Return value:

- None

__HAL_RTC_ALARM_EXTI_ENABLE_EVENT

Description:

- Enable event on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_EVENT

Description:

- Disable event on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_FALLING_EDGE

Description:

- Enable falling edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_FALLING_EDGE

Description:

- Disable falling edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_RISING_EDGE

Description:

- Enable rising edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_RISING_EDGE

Description:

- Disable rising edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_ENABLE_RISING_FALLING_EDGE

Description:

- Enable rising & falling edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_DISABLE_RISING_FALLING_EDGE

Description:

- Disable rising & falling edge trigger on the RTC Alarm associated Exti line.

Return value:

- None.

__HAL_RTC_ALARM_EXTI_GET_FLAG

Description:

- Check whether the RTC Alarm associated Exti line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_ALARM_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Alarm associated Exti line flag.

Return value:

- None.

`__HAL_RTC_ALARM_EXTI_GENERATE_SWIT`

Description:

- Generate a Software interrupt on RTC Alarm associated Exti line.

Return value:

- None.

RTC Flags Definitions

`RTC_FLAG_TAMP1F`

`RTC_FLAG_TSOVF`

`RTC_FLAG_TSF`

`RTC_FLAG_WUTF`

`RTC_FLAG_ALRBF`

`RTC_FLAG_ALRAF`

`RTC_FLAG_INITF`

`RTC_FLAG_RSF`

`RTC_FLAG_INITS`

`RTC_FLAG_WUTWF`

`RTC_FLAG_ALRBWF`

`RTC_FLAG_ALRAWF`

RTC Hour Formats

`RTC_HOURFORMAT_24`

`RTC_HOURFORMAT_12`

RTC Input Parameter Format Definitions

`RTC_FORMAT_BIN`

`RTC_FORMAT_BCD`

RTC Interrupts Definitions

`RTC_IT_TS`

`RTC_IT_WUT`

`RTC_IT_ALRB`

`RTC_IT_ALRA`

`RTC_IT_TAMP`

`RTC_IT_TAMP1`

RTC Private macros to check input parameters

IS_RTC_HOUR_FORMAT

IS_RTC_OUTPUT

IS_RTC_OUTPUT_POL

IS_RTC_OUTPUT_TYPE

IS_RTC_HOUR12

IS_RTC_HOUR24

IS_RTC_ASYNC_PREDIV

IS_RTC_SYNC_PREDIV

IS_RTC_MINUTES

IS_RTC_SECONDS

IS_RTC_HOURFORMAT12

IS_RTC_DAYLIGHT_SAVING

IS_RTC_STORE_OPERATION

IS_RTC_FORMAT

IS_RTC_YEAR

IS_RTC_MONTH

IS_RTC_DATE

IS_RTC_WEEKDAY

IS_RTC_ALARM_DATE_WEEKDAY_DATE

IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY

IS_RTC_ALARM_DATE_WEEKDAY_SEL

IS_RTC_ALARM_MASK

IS_RTC_ALARM

RTC Month Date Definitions

RTC_MONTH_JANUARY

RTC_MONTH_FEBRUARY

RTC_MONTH_MARCH

RTC_MONTH_APRIL

RTC_MONTH_MAY

RTC_MONTH_JUNE

RTC_MONTH_JULY

RTC_MONTH_AUGUST

RTC_MONTH_SEPTEMBER

RTC_MONTH_OCTOBER

RTC_MONTH_NOVEMBER

RTC_MONTH_DECEMBER

RTC Output Polarity Definitions

RTC_OUTPUT_POLARITY_HIGH

RTC_OUTPUT_POLARITY_LOW

RTC Output Selection Definitions

RTC_OUTPUT_DISABLE

RTC_OUTPUT_ALARMA

RTC_OUTPUT_ALARM_B

RTC_OUTPUT_WAKEUP

RTC Output Type ALARM OUT

RTC_OUTPUT_TYPE_OPENDRAIN

RTC_OUTPUT_TYPE_PUSH_PULL

RTC Store Operation Definitions

RTC_STORE_OPERATION_RESET

RTC_STORE_OPERATION_SET

RTC WeekDay Definitions

RTC_WEEKDAY_MONDAY

RTC_WEEKDAY_TUESDAY

RTC_WEEKDAY_WEDNESDAY

RTC_WEEKDAY_THURSDAY

RTC_WEEKDAY_FRIDAY

RTC_WEEKDAY_SATURDAY

RTC_WEEKDAY_SUNDAY

40 HAL RTC Extension Driver

40.1 RTCEX Firmware driver registers structures

40.1.1 RTC_TamperTypeDef

RTC_TamperTypeDef is defined in the `stm32f2xx_hal_rtc_ex.h`

Data Fields

- `uint32_t Tamper`
- `uint32_t PinSelection`
- `uint32_t Trigger`

Field Documentation

- `uint32_t RTC_TamperTypeDef::Tamper`
Specifies the Tamper Pin. This parameter can be a value of [RTCEX_Tamper_Pins_Definitions](#)
- `uint32_t RTC_TamperTypeDef::PinSelection`
Specifies the Tamper Pin. This parameter can be a value of [RTCEX_Tamper_Pins_Selection](#)
- `uint32_t RTC_TamperTypeDef::Trigger`
Specifies the Tamper Trigger. This parameter can be a value of [RTCEX_Tamper_Trigger_Definitions](#)

40.2 RTCEX Firmware driver API description

The following section lists the various functions of the RTCEX library.

40.2.1 How to use this driver

- Enable the RTC domain access.
- Configure the RTC Prescaler (Asynchronous and Synchronous) and RTC hour format using the `HAL_RTC_Init()` function.

RTC Wake-up configuration

- To configure the RTC Wake-up Clock source and Counter use the `HAL_RTCEX_SetWakeUpTimer()` function. You can also configure the RTC Wake-up timer in interrupt mode using the `HAL_RTCEX_SetWakeUpTimer_IT()` function.
- To read the RTC Wake-up Counter register, use the `HAL_RTCEX_GetWakeUpTimer()` function.

TimeStamp configuration

- Configure the RTC_AFx trigger and enable the RTC TimeStamp using the `HAL_RTCEX_SetTimeStamp()` function. You can also configure the RTC TimeStamp with interrupt mode using the `HAL_RTCEX_SetTimeStamp_IT()` function.
- To read the RTC TimeStamp Time and Date register, use the `HAL_RTCEX_GetTimeStamp()` function.
- The TIMESTAMP alternate function can be mapped either to RTC_AF1 (PC13) or RTC_AF2 (PI8) depending on the value of TSINSEL bit in RTC_TAFCR register. The corresponding pin is also selected by `HAL_RTCEX_SetTimeStamp()` or `HAL_RTCEX_SetTimeStamp_IT()` function.

Tamper configuration

- Enable the RTC Tamper and configure the trigger using the `HAL_RTCEX_SetTamper()` function. You can configure RTC Tamper in interrupt mode using `HAL_RTCEX_SetTamper_IT()` function.
- The TAMPER1 alternate function can be mapped either to RTC_AF1 (PC13) or RTC_AF2 (PI8) depending on the value of TAMP1INSEL bit in RTC_TAFCR register. The corresponding pin is also selected by `HAL_RTCEX_SetTamper()` or `HAL_RTCEX_SetTamper_IT()` function.

Backup Data Registers configuration

- To write to the RTC Backup Data registers, use the HAL_RTCEX_BKUPWrite() function.
- To read the RTC Backup Data registers, use the HAL_RTCEX_BKUPRead() function.

40.2.2 RTC TimeStamp and Tamper functions

This section provides functions allowing to configure TimeStamp feature

This section contains the following APIs:

- [*HAL_RTCEX_SetTimeStamp*](#)
- [*HAL_RTCEX_SetTimeStamp_IT*](#)
- [*HAL_RTCEX_DeactivateTimeStamp*](#)
- [*HAL_RTCEX_GetTimeStamp*](#)
- [*HAL_RTCEX_SetTamper*](#)
- [*HAL_RTCEX_SetTamper_IT*](#)
- [*HAL_RTCEX_DeactivateTamper*](#)
- [*HAL_RTCEX_TamperTimeStampIRQHandler*](#)
- [*HAL_RTCEX_TimeStampEventCallback*](#)
- [*HAL_RTCEX_Tamper1EventCallback*](#)
- [*HAL_RTCEX_PollForTimeStampEvent*](#)
- [*HAL_RTCEX_PollForTamper1Event*](#)

40.2.3 RTC Wake-up functions

This section provides functions allowing to configure Wake-up feature

This section contains the following APIs:

- [*HAL_RTCEX_SetWakeUpTimer*](#)
- [*HAL_RTCEX_SetWakeUpTimer_IT*](#)
- [*HAL_RTCEX_DeactivateWakeUpTimer*](#)
- [*HAL_RTCEX_GetWakeUpTimer*](#)
- [*HAL_RTCEX_WakeUpTimerIRQHandler*](#)
- [*HAL_RTCEX_WakeUpTimerEventCallback*](#)
- [*HAL_RTCEX_PollForWakeUpTimerEvent*](#)

40.2.4 Extension Peripheral Control functions

This subsection provides functions allowing to

- Write a data in a specified RTC Backup data register
- Read a data in a specified RTC Backup data register
- Set the Coarse calibration parameters.
- Deactivate the Coarse calibration parameters
- Configure the Synchronization Shift Control Settings.
- Configure the Calibration Pinout (RTC_CALIB).
- Deactivate the Calibration Pinout (RTC_CALIB).
- Enable the RTC reference clock detection.
- Disable the RTC reference clock detection.

This section contains the following APIs:

- [*HAL_RTCEX_BKUPWrite*](#)
- [*HAL_RTCEX_BKUPRead*](#)
- [*HAL_RTCEX_SetCoarseCalib*](#)
- [*HAL_RTCEX_DeactivateCoarseCalib*](#)
- [*HAL_RTCEX_SetCalibrationOutPut*](#)
- [*HAL_RTCEX_DeactivateCalibrationOutPut*](#)

- [HAL_RTCEX_SetRefClock](#)
- [HAL_RTCEX_DeactivateRefClock](#)

40.2.5 Extended features functions

This section provides functions allowing to:

- RTC Alarm B callback
- RTC Poll for Alarm B request

This section contains the following APIs:

- [HAL_RTCEX_AlarmBEventCallback](#)
- [HAL_RTCEX_PollForAlarmBEvent](#)

40.2.6 Detailed description of functions

HAL_RTCEX_SetTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEX_SetTimeStamp (RTC_HandleTypeDef * hrtc, uint32_t TimeStampEdge, uint32_t RTC_TimeStampPin)

Function description

Sets TimeStamp.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **TimeStampEdge**: Specifies the pin edge on which the TimeStamp is activated. This parameter can be one of the following values:
 - RTC_TIMESTAMPEDGE_RISING: the Time stamp event occurs on the rising edge of the related pin.
 - RTC_TIMESTAMPEDGE_FALLING: the Time stamp event occurs on the falling edge of the related pin.
- **RTC_TimeStampPin**: specifies the RTC TimeStamp Pin. This parameter can be one of the following values:
 - RTC_TIMESTAMPPIN_DEFAULT: PC13 is selected as RTC TimeStamp Pin.
 - RTC_TIMESTAMPPIN_POS1: PI8 is selected as RTC TimeStamp Pin.

Return values

- **HAL**: status

Notes

- This API must be called before enabling the TimeStamp feature.

HAL_RTCEX_SetTimeStamp_IT

Function name

HAL_StatusTypeDef HAL_RTCEX_SetTimeStamp_IT (RTC_HandleTypeDef * hrtc, uint32_t TimeStampEdge, uint32_t RTC_TimeStampPin)

Function description

Sets TimeStamp with Interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **TimeStampEdge**: Specifies the pin edge on which the TimeStamp is activated. This parameter can be one of the following values:
 - RTC_TIMESTAMPEDGE_RISING: the Time stamp event occurs on the rising edge of the related pin.
 - RTC_TIMESTAMPEDGE_FALLING: the Time stamp event occurs on the falling edge of the related pin.
- **RTC_TimeStampPin**: Specifies the RTC TimeStamp Pin. This parameter can be one of the following values:
 - RTC_TIMESTAMPPIN_DEFAULT: PC13 is selected as RTC TimeStamp Pin.
 - RTC_TIMESTAMPPIN_POS1: PI8 is selected as RTC TimeStamp Pin.

Return values

- **HAL**: status

Notes

- This API must be called before enabling the TimeStamp feature.

HAL_RTCEX_DeactivateTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateTimeStamp (RTC_HandleTypeDef * hrtc)

Function description

Deactivates TimeStamp.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_GetTimeStamp

Function name

HAL_StatusTypeDef HAL_RTCEX_GetTimeStamp (RTC_HandleTypeDef * hrtc, RTC_TimeTypeDef * sTimeStamp, RTC_DateTypeDef * sTimeStampDate, uint32_t Format)

Function description

Gets the RTC TimeStamp value.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTimeStamp**: Pointer to Time structure
- **sTimeStampDate**: Pointer to Date structure
- **Format**: specifies the format of the entered parameters. This parameter can be one of the following values:
 - RTC_FORMAT_BIN: Binary data format
 - RTC_FORMAT_BCD: BCD data format

Return values

- **HAL**: status

HAL_RTCEX_SetTamper

Function name

HAL_StatusTypeDef HAL_RTCEX_SetTamper (RTC_HandleTypeDef * hrtc, RTC_TamperTypeDef * sTamper)

Function description

Sets Tamper.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTamper**: Pointer to Tamper Structure.

Return values

- **HAL**: status

Notes

- By calling this API we disable the tamper interrupt for all tampers.

HAL_RTCEx_SetTamper_IT

Function name

HAL_StatusTypeDef HAL_RTCEx_SetTamper_IT (RTC_HandleTypeDef * hrtc, RTC_TamperTypeDef * sTamper)

Function description

Sets Tamper with interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **sTamper**: Pointer to RTC Tamper.

Return values

- **HAL**: status

Notes

- By calling this API we force the tamper interrupt for all tampers.

HAL_RTCEx_DeactivateTamper

Function name

HAL_StatusTypeDef HAL_RTCEx_DeactivateTamper (RTC_HandleTypeDef * hrtc, uint32_t Tamper)

Function description

Deactivates Tamper.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Tamper**: Selected tamper pin. This parameter can be RTC_Tamper_1.

Return values

- **HAL**: status

HAL_RTCEx_TamperTimeStampIRQHandler

Function name

void HAL_RTCEx_TamperTimeStampIRQHandler (RTC_HandleTypeDef * hrtc)

Function description

This function handles TimeStamp interrupt request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEX_Tamper1EventCallback

Function name

void HAL_RTCEX_Tamper1EventCallback (RTC_HandleTypeDef * hrtc)

Function description

Tamper 1 callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEX_TimeStampEventCallback

Function name

void HAL_RTCEX_TimeStampEventCallback (RTC_HandleTypeDef * hrtc)

Function description

TimeStamp callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEX_PollForTimeStampEvent

Function name

HAL_StatusTypeDef HAL_RTCEX_PollForTimeStampEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

This function handles TimeStamp polling request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_RTCEX_PollForTamper1Event

Function name

HAL_StatusTypeDef HAL_RTCEX_PollForTamper1Event (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

This function handles Tamper1 Polling.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_RTCEX_SetWakeUpTimer

Function name

```
HAL_StatusTypeDef HAL_RTCEX_SetWakeUpTimer (RTC_HandleTypeDef * hrtc, uint32_t
WakeUpCounter, uint32_t WakeUpClock)
```

Function description

Sets wake up timer.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **WakeUpCounter**: Wake up counter
- **WakeUpClock**: Wake up clock

Return values

- **HAL**: status

HAL_RTCEX_SetWakeUpTimer_IT

Function name

```
HAL_StatusTypeDef HAL_RTCEX_SetWakeUpTimer_IT (RTC_HandleTypeDef * hrtc, uint32_t
WakeUpCounter, uint32_t WakeUpClock)
```

Function description

Sets wake up timer with interrupt.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **WakeUpCounter**: Wake up counter
- **WakeUpClock**: Wake up clock

Return values

- **HAL**: status

HAL_RTCEX_DeactivateWakeUpTimer

Function name

```
uint32_t HAL_RTCEX_DeactivateWakeUpTimer (RTC_HandleTypeDef * hrtc)
```

Function description

Deactivates wake up timer counter.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEx_GetWakeUpTimer

Function name

uint32_t HAL_RTCEx_GetWakeUpTimer (RTC_HandleTypeDef * hrtc)

Function description

Gets wake up timer counter.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **Counter**: value

HAL_RTCEx_WakeUpTimerIRQHandler

Function name

void HAL_RTCEx_WakeUpTimerIRQHandler (RTC_HandleTypeDef * hrtc)

Function description

This function handles Wake Up Timer interrupt request.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEx_WakeUpTimerEventCallback

Function name

void HAL_RTCEx_WakeUpTimerEventCallback (RTC_HandleTypeDef * hrtc)

Function description

Wake Up Timer callback.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None**:

HAL_RTCEx_PollForWakeUpTimerEvent

Function name

HAL_StatusTypeDef HAL_RTCEx_PollForWakeUpTimerEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

This function handles Wake Up Timer Polling.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout**: Timeout duration

Return values

- **HAL:** status

HAL_RTCEX_BKUPWrite

Function name

```
void HAL_RTCEX_BKUPWrite (RTC_HandleTypeDef * hrtc, uint32_t BackupRegister, uint32_t Data)
```

Function description

Writes a data in a specified RTC Backup data register.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **BackupRegister:** RTC Backup data Register number. This parameter can be: RTC_BKP_DRx where x can be from 0 to 19 to specify the register.
- **Data:** Data to be written in the specified RTC Backup data register.

Return values

- **None:**

HAL_RTCEX_BKUPRead

Function name

```
uint32_t HAL_RTCEX_BKUPRead (RTC_HandleTypeDef * hrtc, uint32_t BackupRegister)
```

Function description

Reads data from the specified RTC Backup data Register.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **BackupRegister:** RTC Backup data Register number. This parameter can be: RTC_BKP_DRx where x can be from 0 to 19 to specify the register.

Return values

- **Read:** value

HAL_RTCEX_SetCoarseCalib

Function name

```
HAL_StatusTypeDef HAL_RTCEX_SetCoarseCalib (RTC_HandleTypeDef * hrtc, uint32_t CalibSign, uint32_t Value)
```

Function description

Sets the Coarse calibration parameters.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **CalibSign:** Specifies the sign of the coarse calibration value. This parameter can be one of the following values :
 - RTC_CALIBSIGN_POSITIVE: The value sign is positive
 - RTC_CALIBSIGN_NEGATIVE: The value sign is negative
- **Value:** value of coarse calibration expressed in ppm (coded on 5 bits).

Return values

- **HAL:** status

Notes

- This Calibration value should be between 0 and 63 when using negative sign with a 2-ppm step.
- This Calibration value should be between 0 and 126 when using positive sign with a 4-ppm step.

HAL_RTCEX_DeactivateCoarseCalib

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateCoarseCalib (RTC_HandleTypeDef * hrtc)

Function description

Deactivates the Coarse calibration parameters.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_SetCalibrationOutPut

Function name

HAL_StatusTypeDef HAL_RTCEX_SetCalibrationOutPut (RTC_HandleTypeDef * hrtc)

Function description

Configure the Calibration Pinout (RTC_CALIB).

Parameters

- **hrtc**: RTC handle

Return values

- **HAL**: status

HAL_RTCEX_DeactivateCalibrationOutPut

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateCalibrationOutPut (RTC_HandleTypeDef * hrtc)

Function description

Deactivates the Calibration Pinout (RTC_CALIB).

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL**: status

HAL_RTCEX_SetRefClock

Function name

HAL_StatusTypeDef HAL_RTCEX_SetRefClock (RTC_HandleTypeDef * hrtc)

Function description

Enables the RTC reference clock detection.

Parameters

- **hrtc**: pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

HAL_RTCEX_DeactivateRefClock

Function name

HAL_StatusTypeDef HAL_RTCEX_DeactivateRefClock (RTC_HandleTypeDef * hrtc)

Function description

Disable the RTC reference clock detection.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **HAL:** status

HAL_RTCEX_AlarmBEventCallback

Function name

void HAL_RTCEX_AlarmBEventCallback (RTC_HandleTypeDef * hrtc)

Function description

Alarm B callback.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.

Return values

- **None:**

HAL_RTCEX_PollForAlarmBEvent

Function name

HAL_StatusTypeDef HAL_RTCEX_PollForAlarmBEvent (RTC_HandleTypeDef * hrtc, uint32_t Timeout)

Function description

This function handles AlarmB Polling request.

Parameters

- **hrtc:** pointer to a RTC_HandleTypeDef structure that contains the configuration information for RTC.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

40.3 RTCEX Firmware driver defines

The following section lists the various define and macros of the module.

40.3.1 RTCEX

RTCEX

RTC Backup Registers Definitions

RTC_BKP_DR0

RTC_BKP_DR1

RTC_BKP_DR2

RTC_BKP_DR3

RTC_BKP_DR4

RTC_BKP_DR5

RTC_BKP_DR6

RTC_BKP_DR7

RTC_BKP_DR8

RTC_BKP_DR9

RTC_BKP_DR10

RTC_BKP_DR11

RTC_BKP_DR12

RTC_BKP_DR13

RTC_BKP_DR14

RTC_BKP_DR15

RTC_BKP_DR16

RTC_BKP_DR17

RTC_BKP_DR18

RTC_BKP_DR19

RTC Calibration

__HAL_RTC_COARSE_CALIB_ENABLE

Description:

- Enable the Coarse calibration process.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_COARSE_CALIB_DISABLE

Description:

- Disable the Coarse calibration process.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CALIBRATION_OUTPUT_ENABLE

Description:

- Enable the RTC calibration output.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CALIBRATION_OUTPUT_DISABLE

Description:

- Disable the calibration output.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CLOCKREF_DETECTION_ENABLE

Description:

- Enable the clock reference detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

__HAL_RTC_CLOCKREF_DETECTION_DISABLE

Description:

- Disable the clock reference detection.

Parameters:

- `__HANDLE__`: specifies the RTC handle.

Return value:

- None

RTC Digital Calib Definitions

RTC_CALIBSIGN_POSITIVE

RTC_CALIBSIGN_NEGATIVE

Private macros to check input parameters

IS_RTC_BKP

IS_TIMESTAMP_EDGE

IS_RTC_TAMPER_PIN

IS_RTC_TIMESTAMP_PIN

IS_RTC_TAMPER_TRIGGER

IS_RTC_WAKEUP_CLOCK

IS_RTC_WAKEUP_COUNTER

IS_RTC_CALIB_SIGN

IS_RTC_CALIB_VALUE

IS_RTC_TAMPER

RTC Tamper

__HAL_RTC_TAMPER1_ENABLE

Description:

- Enable the RTC Tamper1 input detection.

Parameters:

- __HANDLE__: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER1_DISABLE

Description:

- Disable the RTC Tamper1 input detection.

Parameters:

- __HANDLE__: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TAMPER_GET_IT

Description:

- Check whether the specified RTC Tamper interrupt has occurred or not.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __INTERRUPT__: specifies the RTC Tamper interrupt to check. This parameter can be:
 - RTC_IT_TAMP1

Return value:

- None

__HAL_RTC_TAMPER_GET_IT_SOURCE

Description:

- Check whether the specified RTC Tamper interrupt has been enabled or not.

Parameters:

- __HANDLE__: specifies the RTC handle.
- __INTERRUPT__: specifies the RTC Tamper interrupt source to check. This parameter can be:
 - RTC_IT_TAMP: Tamper interrupt

Return value:

- None

`__HAL_RTC_TAMPER_GET_FLAG`

Description:

- Get the selected RTC Tamper's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Tamper Flag sources to be enabled or disabled. This parameter can be:
 - `RTC_FLAG_TAMP1F`

Return value:

- None

`__HAL_RTC_TAMPER_CLEAR_FLAG`

Description:

- Clear the RTC Tamper's pending flags.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Tamper Flag to clear. This parameter can be:
 - `RTC_FLAG_TAMP1F`

Return value:

- None

RTC Tamper Pins Definitions

`RTC_TAMPER_1`

RTC tamper Pins Selection

`RTC_TAMPERPIN_DEFAULT`

`RTC_TAMPERPIN_POS1`

EXTI RTC Tamper Timestamp EXTI

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_IT`

Description:

- Enable interrupt on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_IT`

Description:

- Disable interrupt on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_EVENT`

Description:

- Enable event on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_EVENT`

Description:

- Disable event on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_FALLING_EDGE`

Description:

- Enable falling edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_FALLING_EDGE`

Description:

- Disable falling edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE`

Description:

- Enable rising edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_EDGE`

Description:

- Disable rising edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_FALLING_EDGE`

Description:

- Enable rising & falling edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_FALLING_EDGE`

Description:

- Disable rising & falling edge trigger on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

`__HAL_RTC_TAMPER_TIMESTAMP_EXTI_GET_FLAG`

Description:

- Check whether the RTC Tamper and Timestamp associated Exti line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Tamper and Timestamp associated Exti line flag.

Return value:

- None.

__HAL_RTC_TAMPER_TIMESTAMP_EXTI_GENERATE_SWIT

Description:

- Generate a Software interrupt on the RTC Tamper and Timestamp associated Exti line.

Return value:

- None.

RTC Tamper Triggers Definitions

RTC_TAMPERTRIGGER_RISINGEDGE

RTC_TAMPERTRIGGER_FALLINGEDGE

RTC Timestamp

__HAL_RTC_TIMESTAMP_ENABLE

Description:

- Enable the RTC TimeStamp peripheral.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TIMESTAMP_DISABLE

Description:

- Disable the RTC TimeStamp peripheral.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_TIMESTAMP_ENABLE_IT

Description:

- Enable the RTC TimeStamp interrupt.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__INTERRUPT__**: specifies the RTC TimeStamp interrupt sources to be enabled or disabled. This parameter can be:
 - **RTC_IT_TS**: TimeStamp interrupt

Return value:

- None

`__HAL_RTC_TIMESTAMP_DISABLE_IT`

Description:

- Disable the RTC TimeStamp interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC TimeStamp interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

`__HAL_RTC_TIMESTAMP_GET_IT`

Description:

- Check whether the specified RTC TimeStamp interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC TimeStamp interrupt to check. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

`__HAL_RTC_TIMESTAMP_GET_IT_SOURCE`

Description:

- Check whether the specified RTC Time Stamp interrupt has been enabled or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Time Stamp interrupt source to check. This parameter can be:
 - `RTC_IT_TS`: TimeStamp interrupt

Return value:

- None

`__HAL_RTC_TIMESTAMP_GET_FLAG`

Description:

- Get the selected RTC TimeStamp's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC TimeStamp flag to check. This parameter can be:
 - `RTC_FLAG_TSF`
 - `RTC_FLAG_TSOVF`

Return value:

- None

__HAL_RTC_TIMESTAMP_CLEAR_FLAG

Description:

- Clear the RTC Time Stamp's pending flags.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__FLAG__**: specifies the RTC Alarm Flag sources to be enabled or disabled. This parameter can be:
 - RTC_FLAG_TSF

Return value:

- None

RTC TimeStamp Pins Selection

RTC_TIMESTAMPPIN_DEFAULT

RTC_TIMESTAMPPIN_POS1

RTC TimeStamp Edges Definitions

RTC_TIMESTAMPEDGE_RISING

RTC_TIMESTAMPEDGE_FALLING

RTC WakeUp Timer

__HAL_RTC_WAKEUPTIMER_ENABLE

Description:

- Enable the RTC WakeUp Timer peripheral.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_DISABLE

Description:

- Disable the RTC Wake-up Timer peripheral.

Parameters:

- **__HANDLE__**: specifies the RTC handle.

Return value:

- None

__HAL_RTC_WAKEUPTIMER_ENABLE_IT

Description:

- Enable the RTC WakeUpTimer interrupt.

Parameters:

- **__HANDLE__**: specifies the RTC handle.
- **__INTERRUPT__**: specifies the RTC WakeUpTimer interrupt sources to be enabled or disabled. This parameter can be:
 - RTC_IT_WUT: WakeUpTimer A interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_DISABLE_IT

Description:

- Disable the RTC WakeUpTimer interrupt.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC WakeUpTimer interrupt sources to be enabled or disabled. This parameter can be:
 - `RTC_IT_WUT`: WakeUpTimer A interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_IT

Description:

- Check whether the specified RTC WakeUpTimer interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC WakeUpTimer interrupt to check. This parameter can be:
 - `RTC_IT_WUT`: WakeUpTimer A interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_IT_SOURCE

Description:

- Check whether the specified RTC Wake Up timer interrupt has been enabled or not.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__INTERRUPT__`: specifies the RTC Wake Up timer interrupt sources to check. This parameter can be:
 - `RTC_IT_WUT`: WakeUpTimer interrupt

Return value:

- None

__HAL_RTC_WAKEUPTIMER_GET_FLAG

Description:

- Get the selected RTC WakeUpTimer's flag status.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC WakeUpTimer Flag to check. This parameter can be:
 - `RTC_FLAG_WUTF`
 - `RTC_FLAG_WUTWF`

Return value:

- None

`__HAL_RTC_WAKEUPTIMER_CLEAR_FLAG`

Description:

- Clear the RTC Wake Up timer's pending flags.

Parameters:

- `__HANDLE__`: specifies the RTC handle.
- `__FLAG__`: specifies the RTC Tamper Flag sources to be enabled or disabled. This parameter can be:
 - `RTC_FLAG_WUTF`

Return value:

- None

`__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_IT`

Description:

- Enable interrupt on the RTC Wake-up Timer associated Exti line.

Return value:

- None

`__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_IT`

Description:

- Disable interrupt on the RTC Wake-up Timer associated Exti line.

Return value:

- None

`__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_EVENT`

Description:

- Enable event on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

`__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_EVENT`

Description:

- Disable event on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

`__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_FALLING_EDGE`

Description:

- Enable falling edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

`__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_FALLING_EDGE`

Description:

- Disable falling edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_EDGE

Description:

- Enable rising edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_EDGE

Description:

- Disable rising edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_FALLING_EDGE

Description:

- Enable rising & falling edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_FALLING_EDGE

Description:

- Disable rising & falling edge trigger on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_GET_FLAG

Description:

- Check whether the RTC Wake-up Timer associated Exti line interrupt flag is set or not.

Return value:

- Line: Status.

__HAL_RTC_WAKEUPTIMER_EXTI_CLEAR_FLAG

Description:

- Clear the RTC Wake-up Timer associated Exti line flag.

Return value:

- None.

__HAL_RTC_WAKEUPTIMER_EXTI_GENERATE_SWIT

Description:

- Generate a Software interrupt on the RTC Wake-up Timer associated Exti line.

Return value:

- None.

RTC Wake-up Timer Definitions

RTC_WAKEUPCLOCK_RTCCLK_DIV16

RTC_WAKEUPCLOCK_RTCCLK_DIV8

RTC_WAKEUPCLOCK_RTCCLK_DIV4

RTC_WAKEUPCLOCK_RTCCLK_DIV2

RTC_WAKEUPCLOCK_CK_SPRE_16BITS

RTC_WAKEUPCLOCK_CK_SPRE_17BITS

41 HAL SMARTCARD Generic Driver

41.1 SMARTCARD Firmware driver registers structures

41.1.1 SMARTCARD_InitTypeDef

SMARTCARD_InitTypeDef is defined in the `stm32f2xx_hal_smartcard.h`

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t CLKLastBit**
- **uint32_t Prescaler**
- **uint32_t GuardTime**
- **uint32_t NACKState**

Field Documentation

- **uint32_t SMARTCARD_InitTypeDef::BaudRate**
This member configures the SmartCard communication baud rate. The baud rate is computed using the following formula:
 - $\text{IntegerDivider} = ((\text{PCLKx}) / (16 * (\text{hsc->Init.BaudRate})))$
 - $\text{FractionalDivider} = ((\text{IntegerDivider} - ((\text{uint32_t}) \text{IntegerDivider})) * 16) + 0.5$
- **uint32_t SMARTCARD_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [SMARTCARD_Word_Length](#)
- **uint32_t SMARTCARD_InitTypeDef::StopBits**
Specifies the number of stop bits transmitted. This parameter can be a value of [SMARTCARD_Stop_Bits](#)
- **uint32_t SMARTCARD_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [SMARTCARD_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t SMARTCARD_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [SMARTCARD_Mode](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKPolarity**
Specifies the steady state of the serial clock. This parameter can be a value of [SMARTCARD_Clock_Polarity](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKPhase**
Specifies the clock transition on which the bit capture is made. This parameter can be a value of [SMARTCARD_Clock_Phase](#)
- **uint32_t SMARTCARD_InitTypeDef::CLKLastBit**
Specifies whether the clock pulse corresponding to the last transmitted data bit (MSB) has to be output on the SCLK pin in synchronous mode. This parameter can be a value of [SMARTCARD_Last_Bit](#)

- **`uint32_t SMARTCARD_InitTypeDef::Prescaler`**
Specifies the SmartCard Prescaler value used for dividing the system clock to provide the smartcard clock. The value given in the register (5 significant bits) is multiplied by 2 to give the division factor of the source clock frequency. This parameter can be a value of [SMARTCARD_Prescaler](#)
- **`uint32_t SMARTCARD_InitTypeDef::GuardTime`**
Specifies the SmartCard Guard Time value in terms of number of baud clocks
- **`uint32_t SMARTCARD_InitTypeDef::NACKState`**
Specifies the SmartCard NACK Transmission state. This parameter can be a value of [SMARTCARD_NACK_State](#)

41.1.2

`__SMARTCARD_HandleTypeDef`

`__SMARTCARD_HandleTypeDef` is defined in the `stm32f2xx_hal_smartcard.h`

Data Fields

- **`USART_TypeDef * Instance`**
- **`SMARTCARD_InitTypeDef Init`**
- **`uint8_t * pTxBuffPtr`**
- **`uint16_t TxXferSize`**
- **`__IO uint16_t TxXferCount`**
- **`uint8_t * pRxBuffPtr`**
- **`uint16_t RxXferSize`**
- **`__IO uint16_t RxXferCount`**
- **`DMA_HandleTypeDef * hdmatx`**
- **`DMA_HandleTypeDef * hdmarx`**
- **`HAL_LockTypeDef Lock`**
- **`__IO HAL_SMARTCARD_StateTypeDef gState`**
- **`__IO HAL_SMARTCARD_StateTypeDef RxState`**
- **`__IO uint32_t ErrorCode`**

Field Documentation

- **`USART_TypeDef* __SMARTCARD_HandleTypeDef::Instance`**
USART registers base address
- **`SMARTCARD_InitTypeDef __SMARTCARD_HandleTypeDef::Init`**
SmartCard communication parameters
- **`uint8_t* __SMARTCARD_HandleTypeDef::pTxBuffPtr`**
Pointer to SmartCard Tx transfer Buffer
- **`uint16_t __SMARTCARD_HandleTypeDef::TxXferSize`**
SmartCard Tx Transfer size
- **`__IO uint16_t __SMARTCARD_HandleTypeDef::TxXferCount`**
SmartCard Tx Transfer Counter
- **`uint8_t* __SMARTCARD_HandleTypeDef::pRxBuffPtr`**
Pointer to SmartCard Rx transfer Buffer
- **`uint16_t __SMARTCARD_HandleTypeDef::RxXferSize`**
SmartCard Rx Transfer size
- **`__IO uint16_t __SMARTCARD_HandleTypeDef::RxXferCount`**
SmartCard Rx Transfer Counter
- **`DMA_HandleTypeDef* __SMARTCARD_HandleTypeDef::hdmatx`**
SmartCard Tx DMA Handle parameters
- **`DMA_HandleTypeDef* __SMARTCARD_HandleTypeDef::hdmarx`**
SmartCard Rx DMA Handle parameters
- **`HAL_LockTypeDef __SMARTCARD_HandleTypeDef::Lock`**
Locking object

- **__IO HAL_SMARTCARD_StateTypeDef __SMARTCARD_HandleTypeDef::gState**
SmartCard state information related to global Handle management and also related to Tx operations. This parameter can be a value of **HAL_SMARTCARD_StateTypeDef**
- **__IO HAL_SMARTCARD_StateTypeDef __SMARTCARD_HandleTypeDef::RxState**
SmartCard state information related to Rx operations. This parameter can be a value of **HAL_SMARTCARD_StateTypeDef**
- **__IO uint32_t __SMARTCARD_HandleTypeDef::ErrorCode**
SmartCard Error code

41.2 SMARTCARD Firmware driver API description

The following section lists the various functions of the SMARTCARD library.

41.2.1 How to use this driver

The SMARTCARD HAL driver can be used as follows:

1. Declare a SMARTCARD_HandleTypeDef handle structure.
2. Initialize the SMARTCARD low level resources by implementing the HAL_SMARTCARD_MspInit() API:
 - a. Enable the interface clock of the USARTx associated to the SMARTCARD.
 - b. SMARTCARD pins configuration:
 - Enable the clock for the SMARTCARD GPIOs.
 - Configure SMARTCARD pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_SMARTCARD_Transmit_IT() and HAL_SMARTCARD_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_SMARTCARD_Transmit_DMA() and HAL_SMARTCARD_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the SMARTCARD DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the SMARTCARD_Init structure.
4. Initialize the SMARTCARD registers by calling the HAL_SMARTCARD_Init() API:
 - These APIs configure also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_SMARTCARD_MspInit() API.

Note: *The specific SMARTCARD interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_SMARTCARD_ENABLE_IT() and __HAL_SMARTCARD_DISABLE_IT() inside the transmit and receive process.*

Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_SMARTCARD_Transmit()
- Receive an amount of data in blocking mode using HAL_SMARTCARD_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_SMARTCARD_Transmit_IT()

- At transmission end of transfer HAL_SMARTCARD_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_SMARTCARD_Receive_IT()
- At reception end of transfer HAL_SMARTCARD_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_RxCpltCallback
- In case of transfer Error, HAL_SMARTCARD_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_SMARTCARD_Transmit_DMA()
- At transmission end of transfer HAL_SMARTCARD_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_SMARTCARD_Receive_DMA()
- At reception end of transfer HAL_SMARTCARD_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_RxCpltCallback
- In case of transfer Error, HAL_SMARTCARD_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_SMARTCARD_ErrorCallback

SMARTCARD HAL driver macros list

Below the list of most used macros in SMARTCARD HAL driver.

- `__HAL_SMARTCARD_ENABLE`: Enable the SMARTCARD peripheral
- `__HAL_SMARTCARD_DISABLE`: Disable the SMARTCARD peripheral
- `__HAL_SMARTCARD_GET_FLAG`: Check whether the specified SMARTCARD flag is set or not
- `__HAL_SMARTCARD_CLEAR_FLAG`: Clear the specified SMARTCARD pending flag
- `__HAL_SMARTCARD_ENABLE_IT`: Enable the specified SMARTCARD interrupt
- `__HAL_SMARTCARD_DISABLE_IT`: Disable the specified SMARTCARD interrupt

Note: You can refer to the SMARTCARD HAL driver header file for more useful macros

41.2.2 Callback registration

The compilation define `USE_HAL_SMARTCARD_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function `@ref HAL_SMARTCARD_RegisterCallback()` to register a user callback. Function `@ref HAL_SMARTCARD_RegisterCallback()` allows to register following callbacks:

- `TxCpltCallback`: Tx Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.
- `MspInitCallback`: SMARTCARD MspInit.
- `MspDeInitCallback`: SMARTCARD MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function `@ref HAL_SMARTCARD_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. `@ref HAL_SMARTCARD_UnRegisterCallback()` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- `TxCpltCallback`: Tx Complete Callback.
- `RxCpltCallback`: Rx Complete Callback.
- `ErrorCallback`: Error Callback.
- `AbortCpltCallback`: Abort Complete Callback.
- `AbortTransmitCpltCallback`: Abort Transmit Complete Callback.
- `AbortReceiveCpltCallback`: Abort Receive Complete Callback.

- MspInitCallback : SMARTCARD MspInit.
- MspDeInitCallback : SMARTCARD MspDeInit.

By default, after the @ref HAL_SMARTCARD_Init() and when the state is HAL_SMARTCARD_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples @ref HAL_SMARTCARD_TxCpltCallback(), @ref HAL_SMARTCARD_RxCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_SMARTCARD_Init() and @ref HAL_SMARTCARD_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_SMARTCARD_Init() and @ref HAL_SMARTCARD_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_SMARTCARD_STATE_READY state only. Exception done MspInit/MspDeInit that can be registered/unregistered in HAL_SMARTCARD_STATE_READY or HAL_SMARTCARD_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_SMARTCARD_RegisterCallback() before calling @ref HAL_SMARTCARD_DeInit() or @ref HAL_SMARTCARD_Init() function.

When The compilation define USE_HAL_SMARTCARD_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

41.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USART in Smartcard mode.

The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard.

The USART can provide a clock to the smartcard through the SCLK output. In smartcard mode, SCLK is not associated to the communication but is simply derived from the internal peripheral input clock through a 5-bit prescaler.

- For the Smartcard mode only these parameters can be configured:
 - Baud Rate
 - Word Length => Should be 9 bits (8 bits + parity)
 - Stop Bit
 - Parity: => Should be enabled
 - USART polarity
 - USART phase
 - USART LastBit
 - Receiver/transmitter modes
 - Prescaler
 - GuardTime
 - NACKState: The Smartcard NACK state
- Recommended SmartCard interface configuration to get the Answer to Reset from the Card:
 - Word Length = 9 Bits
 - 1.5 Stop Bit
 - Even parity
 - BaudRate = 12096 baud
 - Tx and Rx enabled

Please refer to the ISO 7816-3 specification for more details.

Note: *It is also possible to choose 0.5 stop bit for receiving but it is recommended to use 1.5 stop bits for both transmitting and receiving to avoid switching between the two configurations.*

The HAL_SMARTCARD_Init() function follows the USART SmartCard configuration procedures (details for the procedures are available in reference manual (RM0033)).

This section contains the following APIs:

- [HAL_SMARTCARD_Init](#)
- [HAL_SMARTCARD_DeInit](#)
- [HAL_SMARTCARD_MspInit](#)

- [HAL_SMARTCARD_MspDeInit](#)
- [HAL_SMARTCARD_Relnit](#)

41.2.4 IO operation functions

This subsection provides a set of functions allowing to manage the SMARTCARD data transfers.

1. Smartcard is a single wire half duplex communication protocol. The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard.
2. The USART should be configured as:
 - 8 bits plus parity: where M=1 and PCE=1 in the USART_CR1 register
 - 1.5 stop bits when transmitting and receiving: where STOP=11 in the USART_CR2 register.
3. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - Non Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SMARTCARD IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SMARTCARD_TxCpltCallback(), HAL_SMARTCARD_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process The HAL_SMARTCARD_ErrorCallback() user callback will be executed when a communication error is detected
4. Blocking mode APIs are :
 - HAL_SMARTCARD_Transmit()
 - HAL_SMARTCARD_Receive()
5. Non Blocking mode APIs with Interrupt are :
 - HAL_SMARTCARD_Transmit_IT()
 - HAL_SMARTCARD_Receive_IT()
 - HAL_SMARTCARD_IRQHandler()
6. Non Blocking mode functions with DMA are :
 - HAL_SMARTCARD_Transmit_DMA()
 - HAL_SMARTCARD_Receive_DMA()
7. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_SMARTCARD_TxCpltCallback()
 - HAL_SMARTCARD_RxCpltCallback()
 - HAL_SMARTCARD_ErrorCallback()
8. Non-Blocking mode transfers could be aborted using Abort API's : (+) HAL_SMARTCARD_Abort() (+) HAL_SMARTCARD_AbortTransmit() (+) HAL_SMARTCARD_AbortReceive() (+) HAL_SMARTCARD_Abort_IT() (+) HAL_SMARTCARD_AbortTransmit_IT() (+) HAL_SMARTCARD_AbortReceive_IT()
9. For Abort services based on interrupts (HAL_SMARTCARD_Abortxxx_IT), a set of Abort Complete Callbacks are provided: (+) HAL_SMARTCARD_AbortCpltCallback() (+) HAL_SMARTCARD_AbortTransmitCpltCallback() (+) HAL_SMARTCARD_AbortReceiveCpltCallback()
10. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
 - (+) Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed. Transfer is kept ongoing on SMARTCARD side. If user wants to abort it, Abort services should be called by user. (+) Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Frame Error in Interrupt mode transmission, Overrun Error in Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed.

(#) Smartcard is a single wire half duplex communication protocol. The Smartcard interface is designed to support asynchronous protocol Smartcards as defined in the ISO 7816-3 standard. (#) The USART should be configured as: (++) 8 bits plus parity: where M=1 and PCE=1 in the USART_CR1 register (++) 1.5 stop bits when transmitting and receiving: where STOP=11 in the USART_CR2 register. (#) There are two modes of transfer: (++) Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer. (++) Non Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SMARTCARD IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SMARTCARD_TxCpltCallback(), HAL_SMARTCARD_RxCpltCallback() user callbacks will be executed respectively at the end of the Transmit or Receive process The HAL_SMARTCARD_ErrorCallback() user callback will be executed when a communication error is detected (#) Blocking mode APIs are : (++) HAL_SMARTCARD_Transmit() (++) HAL_SMARTCARD_Receive() (#) Non Blocking mode APIs with Interrupt are : (++) HAL_SMARTCARD_Transmit_IT() (++) HAL_SMARTCARD_Receive_IT() (++) HAL_SMARTCARD_IRQHandler() (#) Non Blocking mode functions with DMA are : (++) HAL_SMARTCARD_Transmit_DMA() (++) HAL_SMARTCARD_Receive_DMA() (#) A set of Transfer Complete Callbacks are provided in non Blocking mode: (++) HAL_SMARTCARD_TxCpltCallback() (++) HAL_SMARTCARD_RxCpltCallback() (++) HAL_SMARTCARD_ErrorCallback() (#) Non-Blocking mode transfers could be aborted using Abort API's :

- HAL_SMARTCARD_Abort()
- HAL_SMARTCARD_AbortTransmit()
- HAL_SMARTCARD_AbortReceive()
- HAL_SMARTCARD_Abort_IT()
- HAL_SMARTCARD_AbortTransmit_IT()
- HAL_SMARTCARD_AbortReceive_IT() (#) For Abort services based on interrupts (HAL_SMARTCARD_Abortxxx_IT), a set of Abort Complete Callbacks are provided:
- HAL_SMARTCARD_AbortCpltCallback()
- HAL_SMARTCARD_AbortTransmitCpltCallback()
- HAL_SMARTCARD_AbortReceiveCpltCallback() (#) In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
- Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed. Transfer is kept ongoing on SMARTCARD side. If user wants to abort it, Abort services should be called by user.
- Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Frame Error in Interrupt mode transmission, Overrun Error in Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_SMARTCARD_ErrorCallback() user callback is executed.

This section contains the following APIs:

- [HAL_SMARTCARD_Transmit](#)
- [HAL_SMARTCARD_Receive](#)
- [HAL_SMARTCARD_Transmit_IT](#)
- [HAL_SMARTCARD_Receive_IT](#)
- [HAL_SMARTCARD_Transmit_DMA](#)
- [HAL_SMARTCARD_Receive_DMA](#)
- [HAL_SMARTCARD_Abort](#)
- [HAL_SMARTCARD_AbortTransmit](#)
- [HAL_SMARTCARD_AbortReceive](#)
- [HAL_SMARTCARD_Abort_IT](#)
- [HAL_SMARTCARD_AbortTransmit_IT](#)
- [HAL_SMARTCARD_AbortReceive_IT](#)
- [HAL_SMARTCARD_IRQHandler](#)
- [HAL_SMARTCARD_TxCpltCallback](#)
- [HAL_SMARTCARD_RxCpltCallback](#)

- [HAL_SMARTCARD_ErrorCallback](#)
- [HAL_SMARTCARD_AbortCpltCallback](#)
- [HAL_SMARTCARD_AbortTransmitCpltCallback](#)
- [HAL_SMARTCARD_AbortReceiveCpltCallback](#)

41.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to control the SmartCard.

- `HAL_SMARTCARD_GetState()` API can be helpful to check in run-time the state of the SmartCard peripheral.
- `HAL_SMARTCARD_GetError()` check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [HAL_SMARTCARD_GetState](#)
- [HAL_SMARTCARD_GetError](#)

41.2.6 Detailed description of functions

HAL_SMARTCARD_Init

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Init (SMARTCARD_HandleTypeDef * hsc)

Function description

Initializes the SmartCard mode according to the specified parameters in the SMARTCARD_InitTypeDef and create the associated handle.

Parameters

- **hsc**: Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL**: status

HAL_SMARTCARD_Relnit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Relnit (SMARTCARD_HandleTypeDef * hsc)

Function description

HAL_SMARTCARD_DeInit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_DeInit (SMARTCARD_HandleTypeDef * hsc)

Function description

DeInitializes the USART SmartCard peripheral.

Parameters

- **hsc**: Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL**: status

HAL_SMARTCARD_MspInit

Function name

```
void HAL_SMARTCARD_MspInit (SMARTCARD_HandleTypeDef * hsc)
```

Function description

SMARTCARD MSP Init.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_MspDeInit

Function name

```
void HAL_SMARTCARD_MspDeInit (SMARTCARD_HandleTypeDef * hsc)
```

Function description

SMARTCARD MSP DeInit.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_Transmit

Function name

```
HAL_StatusTypeDef HAL_SMARTCARD_Transmit (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData,
uint16_t Size, uint32_t Timeout)
```

Function description

Send an amount of data in blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SMARTCARD_Receive

Function name

```
HAL_StatusTypeDef HAL_SMARTCARD_Receive (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData,
uint16_t Size, uint32_t Timeout)
```


Function description

Receive an amount of data in blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be received
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SMARTCARD_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Transmit_IT (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_SMARTCARD_Receive_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Receive_IT (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be received

Return values

- **HAL:** status

HAL_SMARTCARD_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Transmit_DMA (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Send an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

Return values

- **HAL:** status

HAL_SMARTCARD_Receive_DMA

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Receive_DMA (SMARTCARD_HandleTypeDef * hsc, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non blocking mode.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be received

Return values

- **HAL:** status

Notes

- When the SMARTCARD parity is enabled (PCE = 1) the data received contain the parity bit.s

HAL_SMARTCARD_Abort

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Abort (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_AbortTransmit

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortTransmit (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_AbortReceive

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortReceive (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SMARTCARD_Abort_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_Abort_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable PPP InterruptsDisable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortTransmit_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_SMARTCARD_AbortReceive_IT (SMARTCARD_HandleTypeDef * hsc)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable SMARTCARD Interrupts (Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SMARTCARD_IRQHandler

Function name

void HAL_SMARTCARD_IRQHandler (SMARTCARD_HandleTypeDef * hsc)

Function description

This function handles SMARTCARD interrupt request.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_TxCpltCallback

Function name

void HAL_SMARTCARD_TxCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

Tx Transfer completed callbacks.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_RxCpltCallback

Function name

void HAL_SMARTCARD_RxCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

Rx Transfer completed callback.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_ErrorCallback

Function name

void HAL_SMARTCARD_ErrorCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD error callback.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **None:**

HAL_SMARTCARD_AbortCpltCallback

Function name

void HAL_SMARTCARD_AbortCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_AbortTransmitCpltCallback

Function name

void HAL_SMARTCARD_AbortTransmitCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Transmit Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_AbortReceiveCpltCallback

Function name

void HAL_SMARTCARD_AbortReceiveCpltCallback (SMARTCARD_HandleTypeDef * hsc)

Function description

SMARTCARD Abort Receive Complete callback.

Parameters

- **hsc:** SMARTCARD handle.

Return values

- **None:**

HAL_SMARTCARD_GetState

Function name

HAL_SMARTCARD_StateTypeDef HAL_SMARTCARD_GetState (SMARTCARD_HandleTypeDef * hsc)

Function description

Return the SMARTCARD handle state.

Parameters

- **hsc:** Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for SMARTCARD module.

Return values

- **HAL:** state

HAL_SMARTCARD_GetError

Function name

uint32_t HAL_SMARTCARD_GetError (SMARTCARD_HandleTypeDef * hsc)

Function description

Return the SMARTCARD error code.

Parameters

- **hsc**: Pointer to a SMARTCARD_HandleTypeDef structure that contains the configuration information for the specified SMARTCARD.

Return values

- **SMARTCARD**: Error Code

41.3 SMARTCARD Firmware driver defines

The following section lists the various define and macros of the module.

41.3.1 SMARTCARD

SMARTCARD

SMARTCARD Clock Phase

SMARTCARD_PHASE_1EDGE

SMARTCARD_PHASE_2EDGE

SMARTCARD Clock Polarity

SMARTCARD_POLARITY_LOW

SMARTCARD_POLARITY_HIGH

SMARTCARD DMA requests

SMARTCARD_DMAREQ_TX

SMARTCARD_DMAREQ_RX

SMARTCARD Error Code

HAL_SMARTCARD_ERROR_NONE

No error

HAL_SMARTCARD_ERROR_PE

Parity error

HAL_SMARTCARD_ERROR_NE

Noise error

HAL_SMARTCARD_ERROR_FE

Frame error

HAL_SMARTCARD_ERROR_ORE

Overrun error

HAL_SMARTCARD_ERROR_DMA

DMA transfer error

SMARTCARD Exported Macros

__HAL_SMARTCARD_RESET_HANDLE_STATE

Description:

- Reset SMARTCARD handle gstate & RxState.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_FLUSH_DRREGISTER

Description:

- Flush the Smartcard DR register.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_GET_FLAG

Description:

- Check whether the specified Smartcard flag is set or not.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - SMARTCARD_FLAG_TXE: Transmit data register empty flag
 - SMARTCARD_FLAG_TC: Transmission Complete flag
 - SMARTCARD_FLAG_RXNE: Receive data register not empty flag
 - SMARTCARD_FLAG_IDLE: Idle Line detection flag
 - SMARTCARD_FLAG_ORE: Overrun Error flag
 - SMARTCARD_FLAG_NE: Noise Error flag
 - SMARTCARD_FLAG_FE: Framing Error flag
 - SMARTCARD_FLAG_PE: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_SMARTCARD_CLEAR_FLAG

Description:

- Clear the specified Smartcard pending flags.

Parameters:

- **__HANDLE__**: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- **__FLAG__**: specifies the flag to check. This parameter can be any combination of the following values:
 - SMARTCARD_FLAG_TC: Transmission Complete flag.
 - SMARTCARD_FLAG_RXNE: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error) and ORE (Overrun error) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_SMARTCARD_CLEAR_PEFLLAG

Description:

- Clear the SMARTCARD PE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_FEFLAG

Description:

- Clear the SMARTCARD FE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_NEFLAG

Description:

- Clear the SMARTCARD NE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_OREFLAG

Description:

- Clear the SMARTCARD ORE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_CLEAR_IDLEFLAG

Description:

- Clear the SMARTCARD IDLE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_ENABLE_IT

Description:

- Enable the specified SmartCard interrupt.

Parameters:

- **__HANDLE__**: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- **__INTERRUPT__**: specifies the SMARTCARD interrupt to enable. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt
 - SMARTCARD_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_SMARTCARD_DISABLE_IT

Description:

- Disable the specified SmartCard interrupt.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__INTERRUPT__`: specifies the SMARTCARD interrupt to disable. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt
 - SMARTCARD_IT_ERR: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_SMARTCARD_GET_IT_SOURCE

Description:

- Checks whether the specified SmartCard interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the SmartCard Handle.
- `__IT__`: specifies the SMARTCARD interrupt source to check. This parameter can be one of the following values:
 - SMARTCARD_IT_TXE: Transmit Data Register empty interrupt
 - SMARTCARD_IT_TC: Transmission complete interrupt
 - SMARTCARD_IT_RXNE: Receive Data register not empty interrupt
 - SMARTCARD_IT_IDLE: Idle line detection interrupt
 - SMARTCARD_IT_ERR: Error interrupt
 - SMARTCARD_IT_PE: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_SMARTCARD_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the SMARTCARD's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle.

Return value:

- None

__HAL_SMARTCARD_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the SMARTCARD's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the SMARTCARD Handle.

Return value:

- None

__HAL_SMARTCARD_ENABLE

Description:

- Enable the USART associated to the SMARTCARD Handle.

Parameters:

- **__HANDLE__**: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_DISABLE

Description:

- Disable the USART associated to the SMARTCARD Handle.

Parameters:

- **__HANDLE__**: specifies the SMARTCARD Handle. SMARTCARD Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_SMARTCARD_DMA_REQUEST_ENABLE

Description:

- Macros to enable the SmartCard DMA request.

Parameters:

- **__HANDLE__**: specifies the SmartCard Handle.
- **__REQUEST__**: specifies the SmartCard DMA request. This parameter can be one of the following values:
 - SMARTCARD_DMAREQ_TX: SmartCard DMA transmit request
 - SMARTCARD_DMAREQ_RX: SmartCard DMA receive request

Return value:

- None

__HAL_SMARTCARD_DMA_REQUEST_DISABLE

Description:

- Macros to disable the SmartCard DMA request.

Parameters:

- **__HANDLE__**: specifies the SmartCard Handle.
- **__REQUEST__**: specifies the SmartCard DMA request. This parameter can be one of the following values:
 - SMARTCARD_DMAREQ_TX: SmartCard DMA transmit request
 - SMARTCARD_DMAREQ_RX: SmartCard DMA receive request

Return value:

- None

SMARTCARD Last Bit

SMARTCARD_LASTBIT_DISABLE

SMARTCARD_LASTBIT_ENABLE

SMARTCARD Mode

SMARTCARD_MODE_RX

SMARTCARD_MODE_TX

SMARTCARD_MODE_TX_RX

SMARTCARD NACK State

SMARTCARD_NACK_ENABLE

SMARTCARD_NACK_DISABLE

SMARTCARD Parity

SMARTCARD_PARITY_EVEN

SMARTCARD_PARITY_ODD

SMARTCARD Prescaler

SMARTCARD_PRESCALER_SYSCLK_DIV2

SYSCLK divided by 2

SMARTCARD_PRESCALER_SYSCLK_DIV4

SYSCLK divided by 4

SMARTCARD_PRESCALER_SYSCLK_DIV6

SYSCLK divided by 6

SMARTCARD_PRESCALER_SYSCLK_DIV8

SYSCLK divided by 8

SMARTCARD_PRESCALER_SYSCLK_DIV10

SYSCLK divided by 10

SMARTCARD_PRESCALER_SYSCLK_DIV12

SYSCLK divided by 12

SMARTCARD_PRESCALER_SYSCLK_DIV14

SYSCLK divided by 14

SMARTCARD_PRESCALER_SYSCLK_DIV16

SYSCLK divided by 16

SMARTCARD_PRESCALER_SYSCLK_DIV18

SYSCLK divided by 18

SMARTCARD_PRESCALER_SYSCLK_DIV20

SYSCLK divided by 20

SMARTCARD_PRESCALER_SYSCLK_DIV22

SYSCLK divided by 22

SMARTCARD_PRESCALER_SYSCLK_DIV24

SYSCLK divided by 24

SMARTCARD_PRESCALER_SYSCLK_DIV26

SYSCLK divided by 26

SMARTCARD_PRESCALER_SYSCLK_DIV28

SYSCLK divided by 28

SMARTCARD_PRESCALER_SYSCLK_DIV30

SYSCLK divided by 30

SMARTCARD_PRESCALER_SYSCLK_DIV32

SYSCLK divided by 32

SMARTCARD_PRESCALER_SYSCLK_DIV34

SYSCLK divided by 34

SMARTCARD_PRESCALER_SYSCLK_DIV36

SYSCLK divided by 36

SMARTCARD_PRESCALER_SYSCLK_DIV38

SYSCLK divided by 38

SMARTCARD_PRESCALER_SYSCLK_DIV40

SYSCLK divided by 40

SMARTCARD_PRESCALER_SYSCLK_DIV42

SYSCLK divided by 42

SMARTCARD_PRESCALER_SYSCLK_DIV44

SYSCLK divided by 44

SMARTCARD_PRESCALER_SYSCLK_DIV46

SYSCLK divided by 46

SMARTCARD_PRESCALER_SYSCLK_DIV48

SYSCLK divided by 48

SMARTCARD_PRESCALER_SYSCLK_DIV50

SYSCLK divided by 50

SMARTCARD_PRESCALER_SYSCLK_DIV52

SYSCLK divided by 52

SMARTCARD_PRESCALER_SYSCLK_DIV54

SYSCLK divided by 54

SMARTCARD_PRESCALER_SYSCLK_DIV56

SYSCLK divided by 56

SMARTCARD_PRESCALER_SYSCLK_DIV58

SYSCLK divided by 58

SMARTCARD_PRESCALER_SYSCLK_DIV60

SYSCLK divided by 60

SMARTCARD_PRESCALER_SYSCLK_DIV62

SYSCLK divided by 62

SMARTCARD Number of Stop Bits**SMARTCARD_STOPBITS_0_5****SMARTCARD_STOPBITS_1_5*****SMARTCARD Word Length***

SMARTCARD_WORDLENGTH_9B

42 HAL SPI Generic Driver

42.1 SPI Firmware driver registers structures

42.1.1 SPI_InitTypeDef

SPI_InitTypeDef is defined in the `stm32f2xx_hal_spi.h`

Data Fields

- **uint32_t Mode**
- **uint32_t Direction**
- **uint32_t DataSize**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t NSS**
- **uint32_t BaudRatePrescaler**
- **uint32_t FirstBit**
- **uint32_t TIMode**
- **uint32_t CRCCalculation**
- **uint32_t CRCPolynomial**

Field Documentation

- **uint32_t SPI_InitTypeDef::Mode**
Specifies the SPI operating mode. This parameter can be a value of [SPI_Mode](#)
- **uint32_t SPI_InitTypeDef::Direction**
Specifies the SPI bidirectional mode state. This parameter can be a value of [SPI_Direction](#)
- **uint32_t SPI_InitTypeDef::DataSize**
Specifies the SPI data size. This parameter can be a value of [SPI_Data_Size](#)
- **uint32_t SPI_InitTypeDef::CLKPolarity**
Specifies the serial clock steady state. This parameter can be a value of [SPI_Clock_Polarity](#)
- **uint32_t SPI_InitTypeDef::CLKPhase**
Specifies the clock active edge for the bit capture. This parameter can be a value of [SPI_Clock_Phase](#)
- **uint32_t SPI_InitTypeDef::NSS**
Specifies whether the NSS signal is managed by hardware (NSS pin) or by software using the SSI bit. This parameter can be a value of [SPI_Slave_Select_management](#)
- **uint32_t SPI_InitTypeDef::BaudRatePrescaler**
Specifies the Baud Rate prescaler value which will be used to configure the transmit and receive SCK clock. This parameter can be a value of [SPI_BaudRate_Prescaler](#)
Note:
– The communication clock is derived from the master clock. The slave clock does not need to be set.
- **uint32_t SPI_InitTypeDef::FirstBit**
Specifies whether data transfers start from MSB or LSB bit. This parameter can be a value of [SPI_MSB_LSB_transmission](#)
- **uint32_t SPI_InitTypeDef::TIMode**
Specifies if the TI mode is enabled or not. This parameter can be a value of [SPI_TI_mode](#)
- **uint32_t SPI_InitTypeDef::CRCCalculation**
Specifies if the CRC calculation is enabled or not. This parameter can be a value of [SPI_CRC_Calculation](#)
- **uint32_t SPI_InitTypeDef::CRCPolynomial**
Specifies the polynomial used for the CRC calculation. This parameter must be an odd number between Min_Data = 1 and Max_Data = 65535

42.1.2

__SPI_HandleTypeDef

__SPI_HandleTypeDef is defined in the stm32f2xx_hal_spi.h

Data Fields

- **SPI_TypeDef * Instance**
- **SPI_InitTypeDef Init**
- **uint8_t * pTxBuffPtr**
- **uint16_t TxXferSize**
- **__IO uint16_t TxXferCount**
- **uint8_t * pRxBuffPtr**
- **uint16_t RxXferSize**
- **__IO uint16_t RxXferCount**
- **void(* RxISR**
- **void(* TxISR**
- **DMA_HandleTypeDef * hdmatx**
- **DMA_HandleTypeDef * hdmarx**
- **HAL_LockTypeDef Lock**
- **__IO HAL_SPI_StateTypeDef State**
- **__IO uint32_t ErrorCode**

Field Documentation

- **SPI_TypeDef* __SPI_HandleTypeDef::Instance**
SPI registers base address
- **SPI_InitTypeDef __SPI_HandleTypeDef::Init**
SPI communication parameters
- **uint8_t* __SPI_HandleTypeDef::pTxBuffPtr**
Pointer to SPI Tx transfer Buffer
- **uint16_t __SPI_HandleTypeDef::TxXferSize**
SPI Tx Transfer size
- **__IO uint16_t __SPI_HandleTypeDef::TxXferCount**
SPI Tx Transfer Counter
- **uint8_t* __SPI_HandleTypeDef::pRxBuffPtr**
Pointer to SPI Rx transfer Buffer
- **uint16_t __SPI_HandleTypeDef::RxXferSize**
SPI Rx Transfer size
- **__IO uint16_t __SPI_HandleTypeDef::RxXferCount**
SPI Rx Transfer Counter
- **void(* __SPI_HandleTypeDef::RxISR)(struct __SPI_HandleTypeDef *hspi)**
function pointer on Rx ISR
- **void(* __SPI_HandleTypeDef::TxISR)(struct __SPI_HandleTypeDef *hspi)**
function pointer on Tx ISR
- **DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmatx**
SPI Tx DMA Handle parameters
- **DMA_HandleTypeDef* __SPI_HandleTypeDef::hdmarx**
SPI Rx DMA Handle parameters
- **HAL_LockTypeDef __SPI_HandleTypeDef::Lock**
Locking object
- **__IO HAL_SPI_StateTypeDef __SPI_HandleTypeDef::State**
SPI communication state
- **__IO uint32_t __SPI_HandleTypeDef::ErrorCode**
SPI Error code

42.2 SPI Firmware driver API description

The following section lists the various functions of the SPI library.

42.2.1 How to use this driver

The SPI HAL driver can be used as follows:

1. Declare a SPI_HandleTypeDef handle structure, for example: SPI_HandleTypeDef hspi;
2. Initialize the SPI low level resources by implementing the HAL_SPI_MspInit() API:
 - a. Enable the SPIx interface clock
 - b. SPI pins configuration
 - Enable the clock for the SPI GPIOs
 - Configure these SPI pins as alternate function push-pull
 - c. NVIC configuration if you need to use interrupt process
 - Configure the SPIx interrupt priority
 - Enable the NVIC SPI IRQ handle
 - d. DMA Configuration if you need to use DMA process
 - Declare a DMA_HandleTypeDef handle structure for the transmit or receive Stream/Channel
 - Enable the DMAx clock
 - Configure the DMA handle parameters
 - Configure the DMA Tx or Rx Stream/Channel
 - Associate the initialized hdma_tx(or _rx) handle to the hspi DMA Tx or Rx handle
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx or Rx Stream/Channel
3. Program the Mode, BidirectionalMode , Data size, Baudrate Prescaler, NSS management, Clock polarity and phase, FirstBit and CRC configuration in the hspi Init structure.
4. Initialize the SPI registers by calling the HAL_SPI_Init() API:
 - This API configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_SPI_MspInit() API.

Circular mode restriction:

1. The DMA circular mode cannot be used when the SPI is configured in these modes:
 - a. Master 2Lines RxOnly
 - b. Master 1Line Rx
2. The CRC feature is not managed when the DMA circular mode is enabled
3. When the SPI DMA Pause/Stop features are used, we must use the following APIs the HAL_SPI_DMAPause()/ HAL_SPI_DMAStop() only under the SPI callbacks

Master Receive mode restriction:

1. In Master unidirectional receive-only mode (MSTR =1, BIDIMODE=0, RXONLY=1) or bidirectional receive mode (MSTR=1, BIDIMODE=1, BIDIOE=0), to ensure that the SPI does not initiate a new transfer the following procedure has to be respected:
 - a. HAL_SPI_DeInit()
 - b. HAL_SPI_Init()

Callback registration:

1. The compilation flag `USE_HAL_SPI_REGISTER_CALLBACKS` when set to 1U allows the user to configure dynamically the driver callbacks. Use Functions `HAL_SPI_RegisterCallback()` to register an interrupt callback. Function `HAL_SPI_RegisterCallback()` allows to register following callbacks:
 - `TxCpltCallback` : SPI Tx Completed callback
 - `RxCpltCallback` : SPI Rx Completed callback
 - `TxRxCpltCallback` : SPI TxRx Completed callback
 - `TxHalfCpltCallback` : SPI Tx Half Completed callback
 - `RxHalfCpltCallback` : SPI Rx Half Completed callback
 - `TxRxHalfCpltCallback` : SPI TxRx Half Completed callback
 - `ErrorCallback` : SPI Error callback
 - `AbortCpltCallback` : SPI Abort callback
 - `MspInitCallback` : SPI Msp Init callback
 - `MspDeInitCallback` : SPI Msp DeInit callback This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.
2. Use function `HAL_SPI_UnRegisterCallback` to reset a callback to the default weak function. `HAL_SPI_UnRegisterCallback` takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:
 - `TxCpltCallback` : SPI Tx Completed callback
 - `RxCpltCallback` : SPI Rx Completed callback
 - `TxRxCpltCallback` : SPI TxRx Completed callback
 - `TxHalfCpltCallback` : SPI Tx Half Completed callback
 - `RxHalfCpltCallback` : SPI Rx Half Completed callback
 - `TxRxHalfCpltCallback` : SPI TxRx Half Completed callback
 - `ErrorCallback` : SPI Error callback
 - `AbortCpltCallback` : SPI Abort callback
 - `MspInitCallback` : SPI Msp Init callback
 - `MspDeInitCallback` : SPI Msp DeInit callback

By default, after the `HAL_SPI_Init()` and when the state is `HAL_SPI_STATE_RESET` all callbacks are set to the corresponding weak functions: examples `HAL_SPI_MasterTxCpltCallback()`, `HAL_SPI_MasterRxTxCpltCallback()`. Exception done for `MspInit` and `MspDeInit` functions that are reset to the legacy weak functions in the `HAL_SPI_Init()` / `HAL_SPI_DeInit()` only when these callbacks are null (not registered beforehand). If `MspInit` or `MspDeInit` are not null, the `HAL_SPI_Init()` / `HAL_SPI_DeInit()` keep and use the user `MspInit` / `MspDeInit` callbacks (registered beforehand) whatever the state.

Callbacks can be registered/unregistered in `HAL_SPI_STATE_READY` state only. Exception done `MspInit` / `MspDeInit` functions that can be registered/unregistered in `HAL_SPI_STATE_READY` or `HAL_SPI_STATE_RESET` state, thus registered (user) `MspInit` / `DeInit` callbacks can be used during the `Init` / `DeInit`. Then, the user first registers the `MspInit` / `MspDeInit` user callbacks using `HAL_SPI_RegisterCallback()` before calling `HAL_SPI_DeInit()` or `HAL_SPI_Init()` function.

When the compilation define `USE_HAL_PPP_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

Using the HAL it is not possible to reach all supported SPI frequency with the different SPI Modes, the following table resume the max SPI frequency reached with data size 8bits/16bits, according to frequency of the APBx Peripheral Clock (fPCLK) used by the SPI instance.

42.2.2 Initialization and de-initialization functions

This subsection provides a set of functions allowing to initialize and de-initialize the SPIx peripheral:

- User must implement `HAL_SPI_MspInit()` function in which he configures all related peripherals resources (CLOCK, GPIO, DMA, IT and NVIC).

- Call the function HAL_SPI_Init() to configure the selected device with the selected configuration:
 - Mode
 - Direction
 - Data Size
 - Clock Polarity and Phase
 - NSS Management
 - BaudRate Prescaler
 - FirstBit
 - TIMode
 - CRC Calculation
 - CRC Polynomial if CRC enabled
- Call the function HAL_SPI_DeInit() to restore the default configuration of the selected SPIx peripheral.

This section contains the following APIs:

- [*HAL_SPI_Init*](#)
- [*HAL_SPI_DeInit*](#)
- [*HAL_SPI_MspInit*](#)
- [*HAL_SPI_MspDeInit*](#)

42.2.3 IO operation functions

This subsection provides a set of functions allowing to manage the SPI data transfers.

The SPI supports master and slave mode :

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode: The communication is performed using Interrupts or DMA, These APIs return the HAL status. The end of the data processing will be indicated through the dedicated SPI IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_SPI_TxCpltCallback(), HAL_SPI_RxCpltCallback() and HAL_SPI_TxRxCpltCallback() user callbacks will be executed respectively at the end of the transmit or Receive process The HAL_SPI_ErrorCallback() user callback will be executed when a communication error is detected
2. APIs provided for these 2 transfer modes (Blocking mode or Non blocking mode using either Interrupt or DMA) exist for 1Line (simplex) and 2Lines (full duplex) modes.

This section contains the following APIs:

- [*HAL_SPI_Transmit*](#)
- [*HAL_SPI_Receive*](#)
- [*HAL_SPI_TransmitReceive*](#)
- [*HAL_SPI_Transmit_IT*](#)
- [*HAL_SPI_Receive_IT*](#)
- [*HAL_SPI_TransmitReceive_IT*](#)
- [*HAL_SPI_Transmit_DMA*](#)
- [*HAL_SPI_Receive_DMA*](#)
- [*HAL_SPI_TransmitReceive_DMA*](#)
- [*HAL_SPI_Abort*](#)
- [*HAL_SPI_Abort_IT*](#)
- [*HAL_SPI_DMABase*](#)
- [*HAL_SPI_DMAResume*](#)
- [*HAL_SPI_DMABase*](#)
- [*HAL_SPI_DMAStop*](#)
- [*HAL_SPI_IRQHandler*](#)
- [*HAL_SPI_TxCpltCallback*](#)
- [*HAL_SPI_RxCpltCallback*](#)

- [HAL_SPI_TxRxCpltCallback](#)
- [HAL_SPI_TxHalfCpltCallback](#)
- [HAL_SPI_RxHalfCpltCallback](#)
- [HAL_SPI_TxRxHalfCpltCallback](#)
- [HAL_SPI_ErrorCallback](#)
- [HAL_SPI_AbortCpltCallback](#)

42.2.4 Peripheral State and Errors functions

This subsection provides a set of functions allowing to control the SPI.

- [HAL_SPI_GetState\(\)](#) API can be helpful to check in run-time the state of the SPI peripheral
- [HAL_SPI_GetError\(\)](#) check in run-time Errors occurring during communication

This section contains the following APIs:

- [HAL_SPI_GetState](#)
- [HAL_SPI_GetError](#)

42.2.5 Detailed description of functions

HAL_SPI_Init

Function name

HAL_StatusTypeDef HAL_SPI_Init (SPI_HandleTypeDef * hspi)

Function description

Initialize the SPI according to the specified parameters in the SPI_InitTypeDef and initialize the associated handle.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **HAL**: status

HAL_SPI_DeInit

Function name

HAL_StatusTypeDef HAL_SPI_DeInit (SPI_HandleTypeDef * hspi)

Function description

De-Initialize the SPI peripheral.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **HAL**: status

HAL_SPI_MspInit

Function name

void HAL_SPI_MspInit (SPI_HandleTypeDef * hspi)

Function description

Initialize the SPI MSP.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_MspDeInit

Function name

```
void HAL_SPI_MspDeInit (SPI_HandleTypeDef * hspi)
```

Function description

De-Initialize the SPI MSP.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None:**

HAL_SPI_Transmit

Function name

```
HAL_StatusTypeDef HAL_SPI_Transmit (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size,
uint32_t Timeout)
```

Function description

Transmit an amount of data in blocking mode.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData:** pointer to data buffer
- **Size:** amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SPI_Receive

Function name

```
HAL_StatusTypeDef HAL_SPI_Receive (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size,
uint32_t Timeout)
```

Function description

Receive an amount of data in blocking mode.

Parameters

- **hspi:** pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData:** pointer to data buffer
- **Size:** amount of data to be received
- **Timeout:** Timeout duration

Return values

- **HAL:** status

HAL_SPI_TransmitReceive

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Transmit and Receive an amount of data in blocking mode.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent and received
- **Timeout**: Timeout duration

Return values

- **HAL**: status

HAL_SPI_Transmit_IT

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_IT (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_Receive_IT

Function name

HAL_StatusTypeDef HAL_SPI_Receive_IT (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_TransmitReceive_IT

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive_IT (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Transmit and Receive an amount of data in non-blocking mode with Interrupt.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent and received

Return values

- **HAL**: status

HAL_SPI_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Transmit_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Transmit an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

HAL_SPI_Receive_DMA

Function name

HAL_StatusTypeDef HAL_SPI_Receive_DMA (SPI_HandleTypeDef * hspi, uint8_t * pData, uint16_t Size)

Function description

Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pData**: pointer to data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

Notes

- In case of MASTER mode and SPI_DIRECTION_2LINES direction, hdmatx shall be defined.
- When the CRC feature is enabled the pData Length must be Size + 1.

HAL_SPI_TransmitReceive_DMA

Function name

HAL_StatusTypeDef HAL_SPI_TransmitReceive_DMA (SPI_HandleTypeDef * hspi, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Transmit and Receive an amount of data in non-blocking mode with DMA.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.
- **pTxData**: pointer to transmission data buffer
- **pRxData**: pointer to reception data buffer
- **Size**: amount of data to be sent

Return values

- **HAL**: status

Notes

- When the CRC feature is enabled the pRxData Length must be Size + 1

HAL_SPI_DMAPause

Function name

HAL_StatusTypeDef HAL_SPI_DMAPause (SPI_HandleTypeDef * hspi)

Function description

Pause the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_DMAResume

Function name

HAL_StatusTypeDef HAL_SPI_DMAResume (SPI_HandleTypeDef * hspi)

Function description

Resume the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_DMAStop

Function name

HAL_StatusTypeDef HAL_SPI_DMAStop (SPI_HandleTypeDef * hspi)

Function description

Stop the DMA Transfer.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **HAL**: status

HAL_SPI_Abort

Function name

HAL_StatusTypeDef HAL_SPI_Abort (SPI_HandleTypeDef * hspi)

Function description

Abort ongoing transfer (blocking mode).

Parameters

- **hspi**: SPI handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer (Tx and Rx), started in Interrupt or DMA mode. This procedure performs following operations : Disable SPI Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_SPI_Abort_IT

Function name

HAL_StatusTypeDef HAL_SPI_Abort_IT (SPI_HandleTypeDef * hspi)

Function description

Abort ongoing transfer (Interrupt mode).

Parameters

- **hspi**: SPI handle.

Return values

- **HAL**: status

Notes

- This procedure could be used for aborting any ongoing transfer (Tx and Rx), started in Interrupt or DMA mode. This procedure performs following operations : Disable SPI Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_SPI_IRQHandler

Function name

void HAL_SPI_IRQHandler (SPI_HandleTypeDef * hspi)

Function description

Handle SPI interrupt request.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for the specified SPI module.

Return values

- **None**:

HAL_SPI_TxCpltCallback

Function name

void HAL_SPI_TxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_RxCpltCallback

Function name

void HAL_SPI_RxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Rx Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_TxRxCpltCallback

Function name

void HAL_SPI_TxRxCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx and Rx Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_TxHalfCpltCallback

Function name

void HAL_SPI_TxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx Half Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_RxHalfCpltCallback

Function name

void HAL_SPI_RxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Rx Half Transfer completed callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_TxRxHalfCpltCallback

Function name

void HAL_SPI_TxRxHalfCpltCallback (SPI_HandleTypeDef * hspi)

Function description

Tx and Rx Half Transfer callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_ErrorCallback

Function name

void HAL_SPI_ErrorCallback (SPI_HandleTypeDef * hspi)

Function description

SPI error callback.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **None**:

HAL_SPI_AbortCpltCallback

Function name

void HAL_SPI_AbortCpltCallback (SPI_HandleTypeDef * hspi)

Function description

SPI Abort Complete callback.

Parameters

- **hspi**: SPI handle.

Return values

- **None**:

HAL_SPI_GetState

Function name

HAL_SPI_StateTypeDef HAL_SPI_GetState (SPI_HandleTypeDef * hspi)

Function description

Return the SPI handle state.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **SPI**: state

HAL_SPI_GetError

Function name

uint32_t HAL_SPI_GetError (SPI_HandleTypeDef * hspi)

Function description

Return the SPI error code.

Parameters

- **hspi**: pointer to a SPI_HandleTypeDef structure that contains the configuration information for SPI module.

Return values

- **SPI**: error code in bitmap format

42.3 SPI Firmware driver defines

The following section lists the various define and macros of the module.

42.3.1 SPI

SPI

SPI BaudRate Prescaler

SPI_BAUDRATEPRESCALER_2

SPI_BAUDRATEPRESCALER_4

SPI_BAUDRATEPRESCALER_8

SPI_BAUDRATEPRESCALER_16

SPI_BAUDRATEPRESCALER_32

SPI_BAUDRATEPRESCALER_64

SPI_BAUDRATEPRESCALER_128

SPI_BAUDRATEPRESCALER_256

SPI Clock Phase

SPI_PHASE_1EDGE

SPI_PHASE_2EDGE

SPI Clock Polarity

SPI_POLARITY_LOW

SPI_POLARITY_HIGH

SPI CRC Calculation

SPI_CRCCALCULATION_DISABLE

SPI_CRCCALCULATION_ENABLE

SPI Data Size

SPI_DATASIZE_8BIT

SPI_DATASIZE_16BIT

SPI Direction Mode

SPI_DIRECTION_2LINES

SPI_DIRECTION_2LINES_RXONLY

SPI_DIRECTION_1LINE

SPI Error Code

HAL_SPI_ERROR_NONE

No error

HAL_SPI_ERROR_MODF

MODF error

HAL_SPI_ERROR_CRC

CRC error

HAL_SPI_ERROR_OVR

OVR error

HAL_SPI_ERROR_FRE

FRE error

HAL_SPI_ERROR_DMA

DMA transfer error

HAL_SPI_ERROR_FLAG

Error on RXNE/TXE/BSY Flag

HAL_SPI_ERROR_ABORT

Error during SPI Abort procedure

SPI Exported Macros

__HAL_SPI_RESET_HANDLE_STATE

Description:

- Reset SPI handle state.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_ENABLE_IT

Description:

- Enable the specified SPI interrupts.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the interrupt source to enable. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_SPI_DISABLE_IT

Description:

- Disable the specified SPI interrupts.

Parameters:

- `__HANDLE__`: specifies the SPI handle. This parameter can be SPIx where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the interrupt source to disable. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- None

__HAL_SPI_GET_IT_SOURCE

Description:

- Check whether the specified SPI interrupt source is enabled or not.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__INTERRUPT__`: specifies the SPI interrupt source to check. This parameter can be one of the following values:
 - `SPI_IT_TXE`: Tx buffer empty interrupt enable
 - `SPI_IT_RXNE`: RX buffer not empty interrupt enable
 - `SPI_IT_ERR`: Error interrupt enable

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_SPI_GET_FLAG

Description:

- Check whether the specified SPI flag is set or not.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `SPI_FLAG_RXNE`: Receive buffer not empty flag
 - `SPI_FLAG_TXE`: Transmit buffer empty flag
 - `SPI_FLAG_CRCERR`: CRC error flag
 - `SPI_FLAG_MODF`: Mode fault flag
 - `SPI_FLAG_OVR`: Overrun flag
 - `SPI_FLAG_BSY`: Busy flag
 - `SPI_FLAG_FRE`: Frame format error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_SPI_CLEAR_CRCERRFLAG

Description:

- Clear the SPI CRCERR pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_MODFFLAG

Description:

- Clear the SPI MODF pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_OVRFLAG

Description:

- Clear the SPI OVR pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_CLEAR_FREFLAG

Description:

- Clear the SPI FRE pending flag.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_ENABLE

Description:

- Enable the SPI peripheral.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

__HAL_SPI_DISABLE

Description:

- Disable the SPI peripheral.

Parameters:

- `__HANDLE__`: specifies the SPI Handle. This parameter can be SPI where x: 1, 2, or 3 to select the SPI peripheral.

Return value:

- None

SPI Flags Definition

SPI_FLAG_RXNE

SPI_FLAG_TXE

SPI_FLAG_BSY

SPI_FLAG_CRCERR

SPI_FLAG_MODF

SPI_FLAG_OVR

SPI_FLAG_FRE

SPI_FLAG_MASK

SPI Interrupt Definition

SPI_IT_TXE

SPI_IT_RXNE

SPI_IT_ERR

SPI Mode

SPI_MODE_SLAVE

SPI_MODE_MASTER

SPI MSB LSB Transmission

SPI_FIRSTBIT_MSB

SPI_FIRSTBIT_LSB

SPI Slave Select Management

SPI_NSS_SOFT

SPI_NSS_HARD_INPUT

SPI_NSS_HARD_OUTPUT

SPI TI Mode

SPI_TIMODE_DISABLE

SPI_TIMODE_ENABLE

43 HAL SRAM Generic Driver

43.1 SRAM Firmware driver registers structures

43.1.1 SRAM_HandleTypeDef

SRAM_HandleTypeDef is defined in the stm32f2xx_hal_sram.h

Data Fields

- **FSMC_NORSRAM_TypeDef * Instance**
- **FSMC_NORSRAM_EXTENDED_TypeDef * Extended**
- **FSMC_NORSRAM_InitTypeDef Init**
- **HAL_LockTypeDef Lock**
- **__IO HAL_SRAM_StateTypeDef State**
- **DMA_HandleTypeDef * hdma**

Field Documentation

- **FSMC_NORSRAM_TypeDef* SRAM_HandleTypeDef::Instance**
Register base address
- **FSMC_NORSRAM_EXTENDED_TypeDef* SRAM_HandleTypeDef::Extended**
Extended mode register base address
- **FSMC_NORSRAM_InitTypeDef SRAM_HandleTypeDef::Init**
SRAM device control configuration parameters
- **HAL_LockTypeDef SRAM_HandleTypeDef::Lock**
SRAM locking object
- **__IO HAL_SRAM_StateTypeDef SRAM_HandleTypeDef::State**
SRAM device access state
- **DMA_HandleTypeDef* SRAM_HandleTypeDef::hdma**
Pointer DMA handler

43.2 SRAM Firmware driver API description

The following section lists the various functions of the SRAM library.

43.2.1 How to use this driver

This driver is a generic layered driver which contains a set of APIs used to control SRAM memories. It uses the FSMC layer functions to interface with SRAM devices. The following sequence should be followed to configure the FSMC to interface with SRAM/PSRAM memories:

1. Declare a SRAM_HandleTypeDef handle structure, for example: SRAM_HandleTypeDef hsram; and:
 - Fill the SRAM_HandleTypeDef handle "Init" field with the allowed values of the structure member.
 - Fill the SRAM_HandleTypeDef handle "Instance" field with a predefined base register instance for NOR or SRAM device
 - Fill the SRAM_HandleTypeDef handle "Extended" field with a predefined base register instance for NOR or SRAM extended mode
2. Declare two FSMC_NORSRAM_TimingTypeDef structures, for both normal and extended mode timings; for example: FSMC_NORSRAM_TimingTypeDef Timing and FSMC_NORSRAM_TimingTypeDef ExTiming; and fill its fields with the allowed values of the structure member.

3. Initialize the SRAM Controller by calling the function `HAL_SRAM_Init()`. This function performs the following sequence:
 - a. MSP hardware layer configuration using the function `HAL_SRAM_MspInit()`
 - b. Control register configuration using the FSMC NORSRAM interface function `FSMC_NORSRAM_Init()`
 - c. Timing register configuration using the FSMC NORSRAM interface function `FSMC_NORSRAM_Timing_Init()`
 - d. Extended mode Timing register configuration using the FSMC NORSRAM interface function `FSMC_NORSRAM_Extended_Timing_Init()`
 - e. Enable the SRAM device using the macro `__FSMC_NORSRAM_ENABLE()`
4. At this stage you can perform read/write accesses from/to the memory connected to the NOR/SRAM Bank. You can perform either polling or DMA transfer using the following APIs:
 - `HAL_SRAM_Read()/HAL_SRAM_Write()` for polling read/write access
 - `HAL_SRAM_Read_DMA()/HAL_SRAM_Write_DMA()` for DMA read/write transfer
5. You can also control the SRAM device by calling the control APIs `HAL_SRAM_WriteOperation_Enable()/HAL_SRAM_WriteOperation_Disable()` to respectively enable/disable the SRAM write operation
6. You can continuously monitor the SRAM device HAL state by calling the function `HAL_SRAM_GetState()`

Callback registration

The compilation define `USE_HAL_SRAM_REGISTER_CALLBACKS` when set to 1 allows the user to configure dynamically the driver callbacks. Use Functions `@ref HAL_SRAM_RegisterCallback()` to register a user callback, it allows to register following callbacks:

- `MspInitCallback` : SRAM `MspInit`.
- `MspDeInitCallback` : SRAM `MspDeInit`. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function. Use function `@ref HAL_SRAM_UnRegisterCallback()` to reset a callback to the default weak (surcharged) function. It allows to reset following callbacks:
- `MspInitCallback` : SRAM `MspInit`.
- `MspDeInitCallback` : SRAM `MspDeInit`. This function) takes as parameters the HAL peripheral handle and the Callback ID. By default, after the `@ref HAL_SRAM_Init` and if the state is `HAL_SRAM_STATE_RESET` all callbacks are reset to the corresponding legacy weak (surcharged) functions. Exception done for `MspInit` and `MspDeInit` callbacks that are respectively reset to the legacy weak (surcharged) functions in the `@ref HAL_SRAM_Init` and `@ref HAL_SRAM_DeInit` only when these callbacks are null (not registered beforehand). If not, `MspInit` or `MspDeInit` are not null, the `@ref HAL_SRAM_Init` and `@ref HAL_SRAM_DeInit` keep and use the user `MspInit/MspDeInit` callbacks (registered beforehand) Callbacks can be registered/unregistered in READY state only. Exception done for `MspInit/MspDeInit` callbacks that can be registered/unregistered in READY or RESET state, thus registered (user) `MspInit/DeInit` callbacks can be used during the `Init/DeInit`. In that case first register the `MspInit/MspDeInit` user callbacks using `@ref HAL_SRAM_RegisterCallback` before calling `@ref HAL_SRAM_DeInit` or `@ref HAL_SRAM_Init` function. When The compilation define `USE_HAL_SRAM_REGISTER_CALLBACKS` is set to 0 or not defined, the callback registering feature is not available and weak (surcharged) callbacks are used.

43.2.2 SRAM Initialization and de_initialization functions

This section provides functions allowing to initialize/de-initialize the SRAM memory

This section contains the following APIs:

- [*HAL_SRAM_Init*](#)
- [*HAL_SRAM_DeInit*](#)
- [*HAL_SRAM_MspInit*](#)
- [*HAL_SRAM_MspDeInit*](#)
- [*HAL_SRAM_DMA_XferCpltCallback*](#)
- [*HAL_SRAM_DMA_XferErrorCallback*](#)

43.2.3 SRAM Input and Output functions

This section provides functions allowing to use and control the SRAM memory

This section contains the following APIs:

- [HAL_SRAM_Read_8b](#)
- [HAL_SRAM_Write_8b](#)
- [HAL_SRAM_Read_16b](#)
- [HAL_SRAM_Write_16b](#)
- [HAL_SRAM_Read_32b](#)
- [HAL_SRAM_Write_32b](#)
- [HAL_SRAM_Read_DMA](#)
- [HAL_SRAM_Write_DMA](#)
- [HAL_SRAM_DMA_XferCpltCallback](#)
- [HAL_SRAM_DMA_XferErrorCallback](#)

43.2.4 SRAM Control functions

This subsection provides a set of functions allowing to control dynamically the SRAM interface.

This section contains the following APIs:

- [HAL_SRAM_WriteOperation_Enable](#)
- [HAL_SRAM_WriteOperation_Disable](#)

43.2.5 SRAM State functions

This subsection permits to get in run-time the status of the SRAM controller and the data flow.

This section contains the following APIs:

- [HAL_SRAM_GetState](#)

43.2.6 Detailed description of functions

HAL_SRAM_Init

Function name

HAL_StatusTypeDef HAL_SRAM_Init (SRAM_HandleTypeDef * hsram, FSMC_NORSRAM_TimingTypeDef * Timing, FSMC_NORSRAM_TimingTypeDef * ExtTiming)

Function description

Performs the SRAM device initialization sequence.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **Timing:** Pointer to SRAM control timing structure
- **ExtTiming:** Pointer to SRAM extended mode timing structure

Return values

- **HAL:** status

HAL_SRAM_DeInit

Function name

HAL_StatusTypeDef HAL_SRAM_DeInit (SRAM_HandleTypeDef * hsram)

Function description

Performs the SRAM device De-initialization sequence.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL:** status

HAL_SRAM_MspInit

Function name

void HAL_SRAM_MspInit (SRAM_HandleTypeDef * hsram)

Function description

SRAM MSP Init.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None:**

HAL_SRAM_MspDeInit

Function name

void HAL_SRAM_MspDeInit (SRAM_HandleTypeDef * hsram)

Function description

SRAM MSP DeInit.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None:**

HAL_SRAM_Read_8b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_8b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint8_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 8-bit buffer from SRAM memory.

Parameters

- **hsram:** pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress:** Pointer to read start address
- **pDstBuffer:** Pointer to destination buffer
- **BufferSize:** Size of the buffer to read from memory

Return values

- **HAL:** status

HAL_SRAM_Write_8b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_8b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint8_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 8-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_16b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_16b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint16_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 16-bit buffer from SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_16b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_16b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint16_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 16-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_32b

Function name

HAL_StatusTypeDef HAL_SRAM_Read_32b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads 32-bit buffer from SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_32b

Function name

HAL_StatusTypeDef HAL_SRAM_Write_32b (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes 32-bit buffer to SRAM memory.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_Read_DMA

Function name

HAL_StatusTypeDef HAL_SRAM_Read_DMA (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pDstBuffer, uint32_t BufferSize)

Function description

Reads a Words data from the SRAM memory using DMA transfer.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to read start address
- **pDstBuffer**: Pointer to destination buffer
- **BufferSize**: Size of the buffer to read from memory

Return values

- **HAL**: status

HAL_SRAM_Write_DMA

Function name

HAL_StatusTypeDef HAL_SRAM_Write_DMA (SRAM_HandleTypeDef * hsram, uint32_t * pAddress, uint32_t * pSrcBuffer, uint32_t BufferSize)

Function description

Writes a Words data buffer to SRAM memory using DMA transfer.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.
- **pAddress**: Pointer to write start address
- **pSrcBuffer**: Pointer to source buffer to write
- **BufferSize**: Size of the buffer to write to memory

Return values

- **HAL**: status

HAL_SRAM_DMA_XferCpltCallback

Function name

void HAL_SRAM_DMA_XferCpltCallback (DMA_HandleTypeDef * hdma)

Function description

DMA transfer complete callback.

Parameters

- **hdma**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None**:

HAL_SRAM_DMA_XferErrorCallback

Function name

void HAL_SRAM_DMA_XferErrorCallback (DMA_HandleTypeDef * hdma)

Function description

DMA transfer complete error callback.

Parameters

- **hdma**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **None**:

HAL_SRAM_WriteOperation_Enable

Function name

HAL_StatusTypeDef HAL_SRAM_WriteOperation_Enable (SRAM_HandleTypeDef * hsram)

Function description

Enables dynamically SRAM write operation.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: status

HAL_SRAM_WriteOperation_Disable

Function name

HAL_StatusTypeDef HAL_SRAM_WriteOperation_Disable (SRAM_HandleTypeDef * hsram)

Function description

Disables dynamically SRAM write operation.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: status

HAL_SRAM_GetState

Function name

HAL_SRAM_StateTypeDef HAL_SRAM_GetState (SRAM_HandleTypeDef * hsram)

Function description

Returns the SRAM controller state.

Parameters

- **hsram**: pointer to a SRAM_HandleTypeDef structure that contains the configuration information for SRAM module.

Return values

- **HAL**: state

43.3 SRAM Firmware driver defines

The following section lists the various define and macros of the module.

43.3.1 SRAM

SRAM

SRAM Exported Macros

__HAL_SRAM_RESET_HANDLE_STATE

Description:

- Reset SRAM handle state.

Parameters:

- **__HANDLE__**: SRAM handle

Return value:

- None

44 HAL TIM Generic Driver

44.1 TIM Firmware driver registers structures

44.1.1 TIM_Base_InitTypeDef

TIM_Base_InitTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **uint32_t Prescaler**
- **uint32_t CounterMode**
- **uint32_t Period**
- **uint32_t ClockDivision**
- **uint32_t RepetitionCounter**
- **uint32_t AutoReloadPreload**

Field Documentation

- **uint32_t TIM_Base_InitTypeDef::Prescaler**
Specifies the prescaler value used to divide the TIM clock. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF
- **uint32_t TIM_Base_InitTypeDef::CounterMode**
Specifies the counter mode. This parameter can be a value of [TIM_Counter_Mode](#)
- **uint32_t TIM_Base_InitTypeDef::Period**
Specifies the period value to be loaded into the active Auto-Reload Register at the next update event. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t TIM_Base_InitTypeDef::ClockDivision**
Specifies the clock division. This parameter can be a value of [TIM_ClockDivision](#)
- **uint32_t TIM_Base_InitTypeDef::RepetitionCounter**
Specifies the repetition counter value. Each time the RCR downcounter reaches zero, an update event is generated and counting restarts from the RCR value (N). This means in PWM mode that (N+1) corresponds to:
 - the number of PWM periods in edge-aligned mode
 - the number of half PWM period in center-aligned mode GP timers: this parameter must be a number between Min_Data = 0x00 and Max_Data = 0xFF. Advanced timers: this parameter must be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF.
- **uint32_t TIM_Base_InitTypeDef::AutoReloadPreload**
Specifies the auto-reload preload. This parameter can be a value of [TIM_AutoReloadPreload](#)

44.1.2 TIM_OC_InitTypeDef

TIM_OC_InitTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **uint32_t OCMode**
- **uint32_t Pulse**
- **uint32_t OCPolarity**
- **uint32_t OCNPolarity**
- **uint32_t OCFastMode**
- **uint32_t OCIdleState**
- **uint32_t OCNIdleState**

Field Documentation

- **uint32_t TIM_OC_InitTypeDef::OCMode**
Specifies the TIM mode. This parameter can be a value of [TIM_Output_Compare_and_PWM_modes](#)

- **`uint32_t TIM_OC_InitTypeDef::Pulse`**
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between `Min_Data = 0x0000` and `Max_Data = 0xFFFF`
- **`uint32_t TIM_OC_InitTypeDef::OCpolarity`**
Specifies the output polarity. This parameter can be a value of [TIM_Output_Compare_Polarity](#)
- **`uint32_t TIM_OC_InitTypeDef::OCNPolarity`**
Specifies the complementary output polarity. This parameter can be a value of [TIM_Output_Compare_N_Polarity](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OC_InitTypeDef::OCFastMode`**
Specifies the Fast mode state. This parameter can be a value of [TIM_Output_Fast_State](#)
Note:
 - This parameter is valid only in PWM1 and PWM2 mode.
- **`uint32_t TIM_OC_InitTypeDef::OCIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OC_InitTypeDef::OCNIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_N_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.

44.1.3

TIM_OnePulse_InitTypeDef

`TIM_OnePulse_InitTypeDef` is defined in the `stm32f2xx_hal_tim.h`

Data Fields

- **`uint32_t OCMODE`**
- **`uint32_t Pulse`**
- **`uint32_t OCPolarity`**
- **`uint32_t OCNPolarity`**
- **`uint32_t OCIdleState`**
- **`uint32_t OCNIdleState`**
- **`uint32_t ICPolarity`**
- **`uint32_t ICSelection`**
- **`uint32_t ICFILTER`**

Field Documentation

- **`uint32_t TIM_OnePulse_InitTypeDef::OCMODE`**
Specifies the TIM mode. This parameter can be a value of [TIM_Output_Compare_and_PWM_modes](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::Pulse`**
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between `Min_Data = 0x0000` and `Max_Data = 0xFFFF`
- **`uint32_t TIM_OnePulse_InitTypeDef::OCPolarity`**
Specifies the output polarity. This parameter can be a value of [TIM_Output_Compare_Polarity](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::OCNPolarity`**
Specifies the complementary output polarity. This parameter can be a value of [TIM_Output_Compare_N_Polarity](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.

- **`uint32_t TIM_OnePulse_InitTypeDef::OCIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OnePulse_InitTypeDef::OCNIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of [TIM_Output_Compare_N_Idle_State](#)
Note:
 - This parameter is valid only for timer instances supporting break feature.
- **`uint32_t TIM_OnePulse_InitTypeDef::ICPolarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::ICSelection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_OnePulse_InitTypeDef::ICFilter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.4

TIM_IC_InitTypeDef

TIM_IC_InitTypeDef is defined in the `stm32f2xx_hal_tim.h`

Data Fields

- **`uint32_t ICPolarity`**
- **`uint32_t ICSelection`**
- **`uint32_t ICPrescaler`**
- **`uint32_t ICFilter`**

Field Documentation

- **`uint32_t TIM_IC_InitTypeDef::ICPolarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_IC_InitTypeDef::ICSelection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_IC_InitTypeDef::ICPrescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_IC_InitTypeDef::ICFilter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.5

TIM_Encoder_InitTypeDef

TIM_Encoder_InitTypeDef is defined in the `stm32f2xx_hal_tim.h`

Data Fields

- **`uint32_t EncoderMode`**
- **`uint32_t IC1Polarity`**
- **`uint32_t IC1Selection`**
- **`uint32_t IC1Prescaler`**
- **`uint32_t IC1Filter`**
- **`uint32_t IC2Polarity`**
- **`uint32_t IC2Selection`**
- **`uint32_t IC2Prescaler`**
- **`uint32_t IC2Filter`**

Field Documentation

- **`uint32_t TIM_Encoder_InitTypeDef::EncoderMode`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Encoder_Mode](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Polarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Selection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Prescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC1Filter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Polarity`**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Selection`**
Specifies the input. This parameter can be a value of [TIM_Input_Capture_Selection](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Prescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **`uint32_t TIM_Encoder_InitTypeDef::IC2Filter`**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.6

TIM_ClockConfigTypeDef

TIM_ClockConfigTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **`uint32_t ClockSource`**
- **`uint32_t ClockPolarity`**
- **`uint32_t ClockPrescaler`**
- **`uint32_t ClockFilter`**

Field Documentation

- **`uint32_t TIM_ClockConfigTypeDef::ClockSource`**
TIM clock sources This parameter can be a value of [TIM_Clock_Source](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockPolarity`**
TIM clock polarity This parameter can be a value of [TIM_Clock_Polarity](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockPrescaler`**
TIM clock prescaler This parameter can be a value of [TIM_Clock_Prescaler](#)
- **`uint32_t TIM_ClockConfigTypeDef::ClockFilter`**
TIM clock filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.7

TIM_ClearInputConfigTypeDef

TIM_ClearInputConfigTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **`uint32_t ClearInputState`**
- **`uint32_t ClearInputSource`**
- **`uint32_t ClearInputPolarity`**
- **`uint32_t ClearInputPrescaler`**
- **`uint32_t ClearInputFilter`**

Field Documentation

- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputState`**
TIM clear Input state This parameter can be ENABLE or DISABLE

- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputSource`**
TIM clear Input sources This parameter can be a value of [TIM_ClearInput_Source](#)
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputPolarity`**
TIM Clear Input polarity This parameter can be a value of [TIM_ClearInput_Polarity](#)
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputPrescaler`**
TIM Clear Input prescaler This parameter must be 0: When OCRef clear feature is used with ETR source, ETR prescaler must be off
- **`uint32_t TIM_ClearInputConfigTypeDef::ClearInputFilter`**
TIM Clear Input filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.8

TIM_MasterConfigTypeDef

TIM_MasterConfigTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **`uint32_t MasterOutputTrigger`**
- **`uint32_t MasterSlaveMode`**

Field Documentation

- **`uint32_t TIM_MasterConfigTypeDef::MasterOutputTrigger`**
Trigger output (TRGO) selection This parameter can be a value of [TIM_Master_Mode_Selection](#)
- **`uint32_t TIM_MasterConfigTypeDef::MasterSlaveMode`**
Master/slave mode selection This parameter can be a value of [TIM_Master_Slave_Mode](#)

Note:

- When the Master/slave mode is enabled, the effect of an event on the trigger input (TRGI) is delayed to allow a perfect synchronization between the current timer and its slaves (through TRGO). It is not mandatory in case of timer synchronization mode.

44.1.9

TIM_SlaveConfigTypeDef

TIM_SlaveConfigTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **`uint32_t SlaveMode`**
- **`uint32_t InputTrigger`**
- **`uint32_t TriggerPolarity`**
- **`uint32_t TriggerPrescaler`**
- **`uint32_t TriggerFilter`**

Field Documentation

- **`uint32_t TIM_SlaveConfigTypeDef::SlaveMode`**
Slave mode selection This parameter can be a value of [TIM_Slave_Mode](#)
- **`uint32_t TIM_SlaveConfigTypeDef::InputTrigger`**
Input Trigger source This parameter can be a value of [TIM_Trigger_Selection](#)
- **`uint32_t TIM_SlaveConfigTypeDef::TriggerPolarity`**
Input Trigger polarity This parameter can be a value of [TIM_Trigger_Polarity](#)
- **`uint32_t TIM_SlaveConfigTypeDef::TriggerPrescaler`**
Input trigger prescaler This parameter can be a value of [TIM_Trigger_Prescaler](#)
- **`uint32_t TIM_SlaveConfigTypeDef::TriggerFilter`**
Input trigger filter This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF

44.1.10

TIM_BreakDeadTimeConfigTypeDef

TIM_BreakDeadTimeConfigTypeDef is defined in the stm32f2xx_hal_tim.h

Data Fields

- **`uint32_t OffStateRunMode`**
- **`uint32_t OffStateIDLEMode`**
- **`uint32_t LockLevel`**

- `uint32_t DeadTime`
- `uint32_t BreakState`
- `uint32_t BreakPolarity`
- `uint32_t BreakFilter`
- `uint32_t AutomaticOutput`

Field Documentation

- `uint32_t TIM_BreakDeadTimeConfigTypeDef::OffStateRunMode`
TIM off state in run mode This parameter can be a value of [TIM_OSSR_Off_State_Selection_for_Run_mode_state](#)
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::OffStateIDLEMode`
TIM off state in IDLE mode This parameter can be a value of [TIM_OSSI_Off_State_Selection_for_Idle_mode_state](#)
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::LockLevel`
TIM Lock level This parameter can be a value of [TIM_Lock_level](#)
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::DeadTime`
TIM dead Time This parameter can be a number between Min_Data = 0x00 and Max_Data = 0xFF
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakState`
TIM Break State This parameter can be a value of [TIM_Break_Input_enable_disable](#)
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakPolarity`
TIM Break input polarity This parameter can be a value of [TIM_Break_Polarity](#)
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::BreakFilter`
Specifies the break input filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- `uint32_t TIM_BreakDeadTimeConfigTypeDef::AutomaticOutput`
TIM Automatic Output Enable state This parameter can be a value of [TIM_AOE_Bit_Set_Reset](#)

44.1.11

TIM_HandleTypeDef

`TIM_HandleTypeDef` is defined in the `stm32f2xx_hal_tim.h`

Data Fields

- `TIM_TypeDef * Instance`
- `TIM_Base_InitTypeDef Init`
- `HAL_TIM_ActiveChannel Channel`
- `DMA_HandleTypeDef * hdma`
- `HAL_LockTypeDef Lock`
- `__IO HAL_TIM_StateTypeDef State`

Field Documentation

- `TIM_TypeDef* TIM_HandleTypeDef::Instance`
Register base address
- `TIM_Base_InitTypeDef TIM_HandleTypeDef::Init`
TIM Time Base required parameters
- `HAL_TIM_ActiveChannel TIM_HandleTypeDef::Channel`
Active channel
- `DMA_HandleTypeDef* TIM_HandleTypeDef::hdma[7]`
DMA Handlers array This array is accessed by a [DMA_Handle_index](#)
- `HAL_LockTypeDef TIM_HandleTypeDef::Lock`
Locking object
- `__IO HAL_TIM_StateTypeDef TIM_HandleTypeDef::State`
TIM operation state

44.2 TIM Firmware driver API description

The following section lists the various functions of the TIM library.

44.2.1 TIMER Generic features

The Timer features include:

1. 16-bit up, down, up/down auto-reload counter.
2. 16-bit programmable prescaler allowing dividing (also on the fly) the counter clock frequency either by any factor between 1 and 65536.
3. Up to 4 independent channels for:
 - Input Capture
 - Output Compare
 - PWM generation (Edge and Center-aligned Mode)
 - One-pulse mode output
4. Synchronization circuit to control the timer with external signals and to interconnect several timers together.
5. Supports incremental encoder for positioning purposes

44.2.2 How to use this driver

1. Initialize the TIM low level resources by implementing the following functions depending on the selected feature:
 - Time Base : `HAL_TIM_Base_MspInit()`
 - Input Capture : `HAL_TIM_IC_MspInit()`
 - Output Compare : `HAL_TIM_OC_MspInit()`
 - PWM generation : `HAL_TIM_PWM_MspInit()`
 - One-pulse mode output : `HAL_TIM_OnePulse_MspInit()`
 - Encoder mode output : `HAL_TIM_Encoder_MspInit()`
2. Initialize the TIM low level resources :
 - a. Enable the TIM interface clock using `__HAL_RCC_TIMx_CLK_ENABLE()`;
 - b. TIM pins configuration
 - Enable the clock for the TIM GPIOs using the following function:
`__HAL_RCC_GPIOx_CLK_ENABLE()`;
 - Configure these TIM pins in Alternate function mode using `HAL_GPIO_Init()`;
3. The external Clock can be configured, if needed (the default clock is the internal clock from the APBx), using the following function: `HAL_TIM_ConfigClockSource`, the clock configuration should be done before any start function.
4. Configure the TIM in the desired functioning mode using one of the Initialization function of this driver:
 - `HAL_TIM_Base_Init`: to use the Timer to generate a simple time base
 - `HAL_TIM_OC_Init` and `HAL_TIM_OC_ConfigChannel`: to use the Timer to generate an Output Compare signal.
 - `HAL_TIM_PWM_Init` and `HAL_TIM_PWM_ConfigChannel`: to use the Timer to generate a PWM signal.
 - `HAL_TIM_IC_Init` and `HAL_TIM_IC_ConfigChannel`: to use the Timer to measure an external signal.
 - `HAL_TIM_OnePulse_Init` and `HAL_TIM_OnePulse_ConfigChannel`: to use the Timer in One Pulse Mode.
 - `HAL_TIM_Encoder_Init`: to use the Timer Encoder Interface.

5. Activate the TIM peripheral using one of the start functions depending from the feature used:
 - Time Base : HAL_TIM_Base_Start(), HAL_TIM_Base_Start_DMA(), HAL_TIM_Base_Start_IT()
 - Input Capture : HAL_TIM_IC_Start(), HAL_TIM_IC_Start_DMA(), HAL_TIM_IC_Start_IT()
 - Output Compare : HAL_TIM_OC_Start(), HAL_TIM_OC_Start_DMA(), HAL_TIM_OC_Start_IT()
 - PWM generation : HAL_TIM_PWM_Start(), HAL_TIM_PWM_Start_DMA(), HAL_TIM_PWM_Start_IT()
 - One-pulse mode output : HAL_TIM_OnePulse_Start(), HAL_TIM_OnePulse_Start_IT()
 - Encoder mode output : HAL_TIM_Encoder_Start(), HAL_TIM_Encoder_Start_DMA(), HAL_TIM_Encoder_Start_IT().
6. The DMA Burst is managed with the two following functions: HAL_TIM_DMABurst_WriteStart()
HAL_TIM_DMABurst_ReadStart()

Callback registration

The compilation define USE_HAL_TIM_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function @ref HAL_TIM_RegisterCallback() to register a callback. @ref HAL_TIM_RegisterCallback() takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function @ref HAL_TIM_UnRegisterCallback() to reset a callback to the default weak function. @ref HAL_TIM_UnRegisterCallback takes as parameters the HAL peripheral handle, and the Callback ID.

These functions allow to register/unregister following callbacks:

- Base_MspInitCallback : TIM Base Msp Init Callback.
- Base_MspDeInitCallback : TIM Base Msp DeInit Callback.
- IC_MspInitCallback : TIM IC Msp Init Callback.
- IC_MspDeInitCallback : TIM IC Msp DeInit Callback.
- OC_MspInitCallback : TIM OC Msp Init Callback.
- OC_MspDeInitCallback : TIM OC Msp DeInit Callback.
- PWM_MspInitCallback : TIM PWM Msp Init Callback.
- PWM_MspDeInitCallback : TIM PWM Msp DeInit Callback.
- OnePulse_MspInitCallback : TIM One Pulse Msp Init Callback.
- OnePulse_MspDeInitCallback : TIM One Pulse Msp DeInit Callback.
- Encoder_MspInitCallback : TIM Encoder Msp Init Callback.
- Encoder_MspDeInitCallback : TIM Encoder Msp DeInit Callback.
- HallSensor_MspInitCallback : TIM Hall Sensor Msp Init Callback.
- HallSensor_MspDeInitCallback : TIM Hall Sensor Msp DeInit Callback.
- PeriodElapsedCallback : TIM Period Elapsed Callback.
- PeriodElapsedHalfCpltCallback : TIM Period Elapsed half complete Callback.
- TriggerCallback : TIM Trigger Callback.
- TriggerHalfCpltCallback : TIM Trigger half complete Callback.
- IC_CaptureCallback : TIM Input Capture Callback.
- IC_CaptureHalfCpltCallback : TIM Input Capture half complete Callback.
- OC_DelayElapsedCallback : TIM Output Compare Delay Elapsed Callback.
- PWM_PulseFinishedCallback : TIM PWM Pulse Finished Callback.
- PWM_PulseFinishedHalfCpltCallback : TIM PWM Pulse Finished half complete Callback.
- ErrorCallback : TIM Error Callback.
- CommutationCallback : TIM Commutation Callback.
- CommutationHalfCpltCallback : TIM Commutation half complete Callback.
- BreakCallback : TIM Break Callback.

By default, after the Init and when the state is HAL_TIM_STATE_RESET all interrupt callbacks are set to the corresponding weak functions: examples @ref HAL_TIM_TriggerCallback(), @ref HAL_TIM_ErrorCallback().

Exception done for MspInit and MspDeInit functions that are reset to the legacy weak functionalities in the Init / DeInit only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the Init / DeInit keep and use the user MspInit / MspDeInit callbacks(registered beforehand)

Callbacks can be registered / unregistered in HAL_TIM_STATE_READY state only. Exception done MspInit / MspDeInit that can be registered / unregistered in HAL_TIM_STATE_READY or HAL_TIM_STATE_RESET state, thus registered(user) MspInit / DeInit callbacks can be used during the Init / DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_TIM_RegisterCallback() before calling DeInit or Init function.

When The compilation define USE_HAL_TIM_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and all callbacks are set to the corresponding weak functions.

44.2.3 Time Base functions

This section provides functions allowing to:

- Initialize and configure the TIM base.
- De-initialize the TIM base.
- Start the Time Base.
- Stop the Time Base.
- Start the Time Base and enable interrupt.
- Stop the Time Base and disable interrupt.
- Start the Time Base and enable DMA transfer.
- Stop the Time Base and disable DMA transfer.

This section contains the following APIs:

- [*HAL_TIM_Base_Init*](#)
- [*HAL_TIM_Base_DeInit*](#)
- [*HAL_TIM_Base_MspInit*](#)
- [*HAL_TIM_Base_MspDeInit*](#)
- [*HAL_TIM_Base_Start*](#)
- [*HAL_TIM_Base_Stop*](#)
- [*HAL_TIM_Base_Start_IT*](#)
- [*HAL_TIM_Base_Stop_IT*](#)
- [*HAL_TIM_Base_Start_DMA*](#)
- [*HAL_TIM_Base_Stop_DMA*](#)

44.2.4 TIM Output Compare functions

This section provides functions allowing to:

- Initialize and configure the TIM Output Compare.
- De-initialize the TIM Output Compare.
- Start the TIM Output Compare.
- Stop the TIM Output Compare.
- Start the TIM Output Compare and enable interrupt.
- Stop the TIM Output Compare and disable interrupt.
- Start the TIM Output Compare and enable DMA transfer.
- Stop the TIM Output Compare and disable DMA transfer.

This section contains the following APIs:

- [*HAL_TIM_OC_Init*](#)
- [*HAL_TIM_OC_DeInit*](#)
- [*HAL_TIM_OC_MspInit*](#)
- [*HAL_TIM_OC_MspDeInit*](#)
- [*HAL_TIM_OC_Start*](#)
- [*HAL_TIM_OC_Stop*](#)
- [*HAL_TIM_OC_Start_IT*](#)

- [HAL_TIM_OC_Stop_IT](#)
- [HAL_TIM_OC_Start_DMA](#)
- [HAL_TIM_OC_Stop_DMA](#)

44.2.5

TIM PWM functions

This section provides functions allowing to:

- Initialize and configure the TIM PWM.
- De-initialize the TIM PWM.
- Start the TIM PWM.
- Stop the TIM PWM.
- Start the TIM PWM and enable interrupt.
- Stop the TIM PWM and disable interrupt.
- Start the TIM PWM and enable DMA transfer.
- Stop the TIM PWM and disable DMA transfer.

This section contains the following APIs:

- [HAL_TIM_PWM_Init](#)
- [HAL_TIM_PWM_DeInit](#)
- [HAL_TIM_PWM_MspInit](#)
- [HAL_TIM_PWM_MspDeInit](#)
- [HAL_TIM_PWM_Start](#)
- [HAL_TIM_PWM_Stop](#)
- [HAL_TIM_PWM_Start_IT](#)
- [HAL_TIM_PWM_Stop_IT](#)
- [HAL_TIM_PWM_Start_DMA](#)
- [HAL_TIM_PWM_Stop_DMA](#)

44.2.6

TIM Input Capture functions

This section provides functions allowing to:

- Initialize and configure the TIM Input Capture.
- De-initialize the TIM Input Capture.
- Start the TIM Input Capture.
- Stop the TIM Input Capture.
- Start the TIM Input Capture and enable interrupt.
- Stop the TIM Input Capture and disable interrupt.
- Start the TIM Input Capture and enable DMA transfer.
- Stop the TIM Input Capture and disable DMA transfer.

This section contains the following APIs:

- [HAL_TIM_IC_Init](#)
- [HAL_TIM_IC_DeInit](#)
- [HAL_TIM_IC_MspInit](#)
- [HAL_TIM_IC_MspDeInit](#)
- [HAL_TIM_IC_Start](#)
- [HAL_TIM_IC_Stop](#)
- [HAL_TIM_IC_Start_IT](#)
- [HAL_TIM_IC_Stop_IT](#)
- [HAL_TIM_IC_Start_DMA](#)
- [HAL_TIM_IC_Stop_DMA](#)

44.2.7

TIM One Pulse functions

This section provides functions allowing to:

- Initialize and configure the TIM One Pulse.
- De-initialize the TIM One Pulse.
- Start the TIM One Pulse.
- Stop the TIM One Pulse.
- Start the TIM One Pulse and enable interrupt.
- Stop the TIM One Pulse and disable interrupt.
- Start the TIM One Pulse and enable DMA transfer.
- Stop the TIM One Pulse and disable DMA transfer.

This section contains the following APIs:

- [*HAL_TIM_OnePulse_Init*](#)
- [*HAL_TIM_OnePulse_DeInit*](#)
- [*HAL_TIM_OnePulse_MspInit*](#)
- [*HAL_TIM_OnePulse_MspDeInit*](#)
- [*HAL_TIM_OnePulse_Start*](#)
- [*HAL_TIM_OnePulse_Stop*](#)
- [*HAL_TIM_OnePulse_Start_IT*](#)
- [*HAL_TIM_OnePulse_Stop_IT*](#)

44.2.8

TIM Encoder functions

This section provides functions allowing to:

- Initialize and configure the TIM Encoder.
- De-initialize the TIM Encoder.
- Start the TIM Encoder.
- Stop the TIM Encoder.
- Start the TIM Encoder and enable interrupt.
- Stop the TIM Encoder and disable interrupt.
- Start the TIM Encoder and enable DMA transfer.
- Stop the TIM Encoder and disable DMA transfer.

This section contains the following APIs:

- [*HAL_TIM_Encoder_Init*](#)
- [*HAL_TIM_Encoder_DeInit*](#)
- [*HAL_TIM_Encoder_MspInit*](#)
- [*HAL_TIM_Encoder_MspDeInit*](#)
- [*HAL_TIM_Encoder_Start*](#)
- [*HAL_TIM_Encoder_Stop*](#)
- [*HAL_TIM_Encoder_Start_IT*](#)
- [*HAL_TIM_Encoder_Stop_IT*](#)
- [*HAL_TIM_Encoder_Start_DMA*](#)
- [*HAL_TIM_Encoder_Stop_DMA*](#)

44.2.9

TIM Callbacks functions

This section provides TIM callback functions:

- TIM Period elapsed callback
- TIM Output Compare callback
- TIM Input capture callback
- TIM Trigger callback
- TIM Error callback

This section contains the following APIs:

- [HAL_TIM_PeriodElapsedCallback](#)
- [HAL_TIM_PeriodElapsedHalfCpltCallback](#)
- [HAL_TIM_OC_DelayElapsedCallback](#)
- [HAL_TIM_IC_CaptureCallback](#)
- [HAL_TIM_IC_CaptureHalfCpltCallback](#)
- [HAL_TIM_PWM_PulseFinishedCallback](#)
- [HAL_TIM_PWM_PulseFinishedHalfCpltCallback](#)
- [HAL_TIM_TriggerCallback](#)
- [HAL_TIM_TriggerHalfCpltCallback](#)
- [HAL_TIM_ErrorCallback](#)

44.2.10 Detailed description of functions

HAL_TIM_Base_Init

Function name

HAL_StatusTypeDef HAL_TIM_Base_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Time base Unit according to the specified parameters in the TIM_HandleTypeDef and initialize the associated handle.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_Base_DeInit() before HAL_TIM_Base_Init()

HAL_TIM_Base_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_Base_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM Base peripheral.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_MspInit

Function name

void HAL_TIM_Base_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Base MSP.

Parameters

- **htim**: TIM Base handle

Return values

- **None**:

HAL_TIM_Base_MspDeInit

Function name

void HAL_TIM_Base_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Base MSP.

Parameters

- **htim**: TIM Base handle

Return values

- **None**:

HAL_TIM_Base_Start

Function name

HAL_StatusTypeDef HAL_TIM_Base_Start (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Base generation.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Stop

Function name

HAL_StatusTypeDef HAL_TIM_Base_Stop (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Base generation.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_Base_Start_IT (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Base generation in interrupt mode.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_Base_Stop_IT (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Base generation in interrupt mode.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_Base_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Base_Start_DMA (TIM_HandleTypeDef * htim, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Base generation in DMA mode.

Parameters

- **htim**: TIM Base handle
- **pData**: The source Buffer address.
- **Length**: The length of data to be transferred from memory to peripheral.

Return values

- **HAL**: status

HAL_TIM_Base_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Base_Stop_DMA (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Base generation in DMA mode.

Parameters

- **htim**: TIM Base handle

Return values

- **HAL**: status

HAL_TIM_OC_Init

Function name

HAL_StatusTypeDef HAL_TIM_OC_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Output Compare according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_OC_DeInit() before HAL_TIM_OC_Init()

HAL_TIM_OC_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_OC_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM peripheral.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **HAL:** status

HAL_TIM_OC_MspInit

Function name

void HAL_TIM_OC_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Output Compare MSP.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **None:**

HAL_TIM_OC_MspDeInit

Function name

void HAL_TIM_OC_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Output Compare MSP.

Parameters

- **htim:** TIM Output Compare handle

Return values

- **None:**

HAL_TIM_OC_Start

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Stop

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation in interrupt mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in interrupt mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OC_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_OC_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Output Compare signal generation in DMA mode.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIM_OC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_OC_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in DMA mode.

Parameters

- **htim**: TIM Output Compare handle
- **Channel**: TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL**: status

HAL_TIM_PWM_Init

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM PWM Time Base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim**: TIM PWM handle

Return values

- **HAL**: status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_PWM_DeInit() before HAL_TIM_PWM_Init()

HAL_TIM_PWM_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_PWM_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM peripheral.

Parameters

- **htim**: TIM PWM handle

Return values

- **HAL**: status

HAL_TIM_PWM_MspInit

Function name

void HAL_TIM_PWM_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM PWM MSP.

Parameters

- **htim**: TIM PWM handle

Return values

- **None:**

HAL_TIM_PWM_MspDeInit

Function name

void HAL_TIM_PWM_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM PWM MSP.

Parameters

- **htim:** TIM PWM handle

Return values

- **None:**

HAL_TIM_PWM_Start

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Stop

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation in interrupt mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation in interrupt mode.

Parameters

- **htim:** TIM PWM handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_PWM_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM PWM signal generation in DMA mode.

Parameters

- **htim**: TIM PWM handle
- **Channel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData**: The source Buffer address.
- **Length**: The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL**: status

HAL_TIM_PWM_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_PWM_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM PWM signal generation in DMA mode.

Parameters

- **htim**: TIM PWM handle
- **Channel**: TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL**: status

HAL_TIM_IC_Init

Function name

HAL_StatusTypeDef HAL_TIM_IC_Init (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Input Capture Time base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim**: TIM Input Capture handle

Return values

- **HAL**: status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_IC_DeInit() before HAL_TIM_IC_Init()

HAL_TIM_IC_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_IC_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM peripheral.

Parameters

- **htim**: TIM Input Capture handle

Return values

- **HAL**: status

HAL_TIM_IC_MspInit

Function name

void HAL_TIM_IC_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Input Capture MSP.

Parameters

- **htim**: TIM Input Capture handle

Return values

- **None**:

HAL_TIM_IC_MspDeInit

Function name

void HAL_TIM_IC_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Input Capture MSP.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_IC_Start

Function name

HAL_StatusTypeDef HAL_TIM_IC_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Input Capture measurement.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Stop

Function name

HAL_StatusTypeDef HAL_TIM_IC_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_IC_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Input Capture measurement in interrupt mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_IC_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement in interrupt mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_IC_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_IC_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Input Capture measurement in DMA mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL:** status

HAL_TIM_IC_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_IC_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Input Capture measurement in DMA mode.

Parameters

- **htim:** TIM Input Capture handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL:** status

HAL_TIM_OnePulse_Init

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Init (TIM_HandleTypeDef * htim, uint32_t OnePulseMode)

Function description

Initializes the TIM One Pulse Time Base according to the specified parameters in the TIM_HandleTypeDef and initializes the associated handle.

Parameters

- **htim:** TIM One Pulse handle
- **OnePulseMode:** Select the One pulse mode. This parameter can be one of the following values:
 - TIM_OPMode_SINGLE: Only one pulse will be generated.
 - TIM_OPMode_REPETITIVE: Repetitive pulses will be generated.

Return values

- **HAL:** status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_OnePulse_DeInit() before HAL_TIM_OnePulse_Init()

HAL_TIM_OnePulse_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_DeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes the TIM One Pulse.

Parameters

- **htim:** TIM One Pulse handle

Return values

- **HAL:** status

HAL_TIM_OnePulse_MspInit

Function name

void HAL_TIM_OnePulse_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM One Pulse MSP.

Parameters

- **htim:** TIM One Pulse handle

Return values

- **None:**

HAL_TIM_OnePulse_MspDeInit

Function name

void HAL_TIM_OnePulse_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM One Pulse MSP.

Parameters

- **htim**: TIM One Pulse handle

Return values

- **None**:

HAL_TIM_OnePulse_Start

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Start (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIM_OnePulse_Stop

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Stop (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channels to be disable This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIM_OnePulse_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Start_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation in interrupt mode.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIM_OnePulse_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_Stop_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation in interrupt mode.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIM_Encoder_Init

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Init (TIM_HandleTypeDef * htim, TIM_Encoder_InitTypeDef * sConfig)

Function description

Initializes the TIM Encoder Interface and initialize the associated handle.

Parameters

- **htim**: TIM Encoder Interface handle
- **sConfig**: TIM Encoder Interface configuration structure

Return values

- **HAL**: status

Notes

- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode. Ex: call HAL_TIM_Encoder_DeInit() before HAL_TIM_Encoder_Init()
- Encoder mode and External clock mode 2 are not compatible and must not be selected together Ex: A call for HAL_TIM_Encoder_Init will erase the settings of HAL_TIM_ConfigClockSource using TIM_CLOCKSOURCE_ETRMODE2 and vice versa

HAL_TIM_Encoder_DeInit

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_DeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes the TIM Encoder interface.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **HAL:** status

HAL_TIM_Encoder_MspInit

Function name

void HAL_TIM_Encoder_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Encoder Interface MSP.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **None:**

HAL_TIM_Encoder_MspDeInit

Function name

void HAL_TIM_Encoder_MspDeInit (TIM_HandleTypeDef * htim)

Function description

DeInitializes TIM Encoder Interface MSP.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **None:**

HAL_TIM_Encoder_Start

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Encoder Interface.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - **TIM_CHANNEL_1:** TIM Channel 1 selected
 - **TIM_CHANNEL_2:** TIM Channel 2 selected
 - **TIM_CHANNEL_ALL:** TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Stop

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Start_IT

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Encoder Interface in interrupt mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface in interrupt mode.

Parameters

- **htim:** TIM Encoder Interface handle
- **Channel:** TIM Channels to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL:** status

HAL_TIM_Encoder_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData1, uint32_t * pData2, uint16_t Length)

Function description

Starts the TIM Encoder Interface in DMA mode.

Parameters

- **htim**: TIM Encoder Interface handle
- **Channel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected
- **pData1**: The destination Buffer address for IC1.
- **pData2**: The destination Buffer address for IC2.
- **Length**: The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL**: status

HAL_TIM_Encoder_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIM_Encoder_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Encoder Interface in DMA mode.

Parameters

- **htim**: TIM Encoder Interface handle
- **Channel**: TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_ALL: TIM Channel 1 and TIM Channel 2 are selected

Return values

- **HAL**: status

HAL_TIM_IRQHandler

Function name

void HAL_TIM_IRQHandler (TIM_HandleTypeDef * htim)

Function description

This function handles TIM interrupts requests.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_OC_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_OC_ConfigChannel (TIM_HandleTypeDef * htim, TIM_OC_InitTypeDef *
sConfig, uint32_t Channel)
```

Function description

Initializes the TIM Output Compare Channels according to the specified parameters in the TIM_OC_InitTypeDef.

Parameters

- **htim**: TIM Output Compare handle
- **sConfig**: TIM Output Compare configuration structure
- **Channel**: TIM Channels to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL**: status

HAL_TIM_PWM_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel (TIM_HandleTypeDef * htim, TIM_OC_InitTypeDef *
sConfig, uint32_t Channel)
```

Function description

Initializes the TIM PWM channels according to the specified parameters in the TIM_OC_InitTypeDef.

Parameters

- **htim**: TIM PWM handle
- **sConfig**: TIM PWM configuration structure
- **Channel**: TIM Channels to be configured This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL**: status

HAL_TIM_IC_ConfigChannel

Function name

```
HAL_StatusTypeDef HAL_TIM_IC_ConfigChannel (TIM_HandleTypeDef * htim, TIM_IC_InitTypeDef *
sConfig, uint32_t Channel)
```

Function description

Initializes the TIM Input Capture Channels according to the specified parameters in the TIM_IC_InitTypeDef.

Parameters

- **htim**: TIM IC handle
- **sConfig**: TIM Input Capture configuration structure
- **Channel**: TIM Channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **HAL**: status

HAL_TIM_OnePulse_ConfigChannel

Function name

HAL_StatusTypeDef HAL_TIM_OnePulse_ConfigChannel (TIM_HandleTypeDef * htim, TIM_OnePulse_InitTypeDef * sConfig, uint32_t OutputChannel, uint32_t InputChannel)

Function description

Initializes the TIM One Pulse Channels according to the specified parameters in the TIM_OnePulse_InitTypeDef.

Parameters

- **htim**: TIM One Pulse handle
- **sConfig**: TIM One Pulse configuration structure
- **OutputChannel**: TIM output channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
- **InputChannel**: TIM input Channel to configure This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

Notes

- To output a waveform with a minimum delay user can enable the fast mode by calling the `__HAL_TIM_ENABLE_OCxFAST` macro. Then CCx output is forced in response to the edge detection on Tlx input, without taking in account the comparison.

HAL_TIM_ConfigOCrefClear

Function name

HAL_StatusTypeDef HAL_TIM_ConfigOCrefClear (TIM_HandleTypeDef * htim, TIM_ClearInputConfigTypeDef * sClearInputConfig, uint32_t Channel)

Function description

Configures the OCRef clear feature.

Parameters

- **htim**: TIM handle
- **sClearInputConfig**: pointer to a TIM_ClearInputConfigTypeDef structure that contains the OCREF clear feature and parameters for the TIM peripheral.
- **Channel**: specifies the TIM Channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3
 - TIM_CHANNEL_4: TIM Channel 4

Return values

- **HAL**: status

HAL_TIM_ConfigClockSource

Function name

HAL_StatusTypeDef HAL_TIM_ConfigClockSource (TIM_HandleTypeDef * htim, TIM_ClockConfigTypeDef * sClockSourceConfig)

Function description

Configures the clock source to be used.

Parameters

- **htim**: TIM handle
- **sClockSourceConfig**: pointer to a TIM_ClockConfigTypeDef structure that contains the clock source information for the TIM peripheral.

Return values

- **HAL**: status

HAL_TIM_ConfigTI1Input

Function name

HAL_StatusTypeDef HAL_TIM_ConfigTI1Input (TIM_HandleTypeDef * htim, uint32_t TI1_Selection)

Function description

Selects the signal connected to the TI1 input: direct from CH1_input or a XOR combination between CH1_input, CH2_input & CH3_input.

Parameters

- **htim**: TIM handle.
- **TI1_Selection**: Indicate whether or not channel 1 is connected to the output of a XOR gate. This parameter can be one of the following values:
 - TIM_TI1SELECTION_CH1: The TIMx_CH1 pin is connected to TI1 input
 - TIM_TI1SELECTION_XORCOMBINATION: The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)

Return values

- **HAL**: status

HAL_TIM_SlaveConfigSynchro

Function name

HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro (TIM_HandleTypeDef * htim, TIM_SlaveConfigTypeDef * sSlaveConfig)

Function description

Configures the TIM in Slave mode.

Parameters

- **htim**: TIM handle.
- **sSlaveConfig**: pointer to a TIM_SlaveConfigTypeDef structure that contains the selected trigger (internal trigger input, filtered timer input or external trigger input) and the Slave mode (Disable, Reset, Gated, Trigger, External clock mode 1).

Return values

- **HAL**: status

HAL_TIM_SlaveConfigSynchro_IT

Function name

**HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro_IT (TIM_HandleTypeDef * htim,
TIM_SlaveConfigTypeDef * sSlaveConfig)**

Function description

Configures the TIM in Slave mode in interrupt mode.

Parameters

- **htim**: TIM handle.
- **sSlaveConfig**: pointer to a TIM_SlaveConfigTypeDef structure that contains the selected trigger (internal trigger input, filtered timer input or external trigger input) and the Slave mode (Disable, Reset, Gated, Trigger, External clock mode 1).

Return values

- **HAL**: status

HAL_TIM_DMABurst_WriteStart

Function name

**HAL_StatusTypeDef HAL_TIM_DMABurst_WriteStart (TIM_HandleTypeDef * htim, uint32_t
BurstBaseAddress, uint32_t BurstRequestSrc, uint32_t * BurstBuffer, uint32_t BurstLength)**

Function description

Configure the DMA Burst to transfer Data from the memory to the TIM peripheral.

Parameters

- **htim**: TIM handle
- **BurstBaseAddress**: TIM Base address from where the DMA will start the Data write This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc**: TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer**: The Buffer address.
- **BurstLength**: DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.

Return values

- **HAL**: status

Notes

- This function should be used only when BurstLength is equal to DMA data transfer length.

HAL_TIM_DMABurst_WriteStop

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_WriteStop (TIM_HandleTypeDef * htim, uint32_t BurstRequestSrc)

Function description

Stops the TIM DMA Burst mode.

Parameters

- **htim**: TIM handle
- **BurstRequestSrc**: TIM DMA Request sources to disable

Return values

- **HAL:** status

HAL_TIM_DMABurst_ReadStart

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_ReadStart (TIM_HandleTypeDef * htim, uint32_t BurstBaseAddress, uint32_t BurstRequestSrc, uint32_t * BurstBuffer, uint32_t BurstLength)

Function description

Configure the DMA Burst to transfer Data from the TIM peripheral to the memory.

Parameters

- **htim:** TIM handle
- **BurstBaseAddress:** TIM Base address from where the DMA will start the Data read This parameter can be one of the following values:
 - TIM_DMABASE_CR1
 - TIM_DMABASE_CR2
 - TIM_DMABASE_SMCR
 - TIM_DMABASE_DIER
 - TIM_DMABASE_SR
 - TIM_DMABASE_EGR
 - TIM_DMABASE_CCMR1
 - TIM_DMABASE_CCMR2
 - TIM_DMABASE_CCER
 - TIM_DMABASE_CNT
 - TIM_DMABASE_PSC
 - TIM_DMABASE_ARR
 - TIM_DMABASE_RCR
 - TIM_DMABASE_CCR1
 - TIM_DMABASE_CCR2
 - TIM_DMABASE_CCR3
 - TIM_DMABASE_CCR4
 - TIM_DMABASE_BDTR
- **BurstRequestSrc:** TIM DMA Request sources This parameter can be one of the following values:
 - TIM_DMA_UPDATE: TIM update Interrupt source
 - TIM_DMA_CC1: TIM Capture Compare 1 DMA source
 - TIM_DMA_CC2: TIM Capture Compare 2 DMA source
 - TIM_DMA_CC3: TIM Capture Compare 3 DMA source
 - TIM_DMA_CC4: TIM Capture Compare 4 DMA source
 - TIM_DMA_COM: TIM Commutation DMA source
 - TIM_DMA_TRIGGER: TIM Trigger DMA source
- **BurstBuffer:** The Buffer address.
- **BurstLength:** DMA Burst length. This parameter can be one value between: TIM_DMABURSTLENGTH_1TRANSFER and TIM_DMABURSTLENGTH_18TRANSFERS.

Return values

- **HAL:** status

Notes

- This function should be used only when BurstLength is equal to DMA data transfer length.

HAL_TIM_DMABurst_ReadStop

Function name

HAL_StatusTypeDef HAL_TIM_DMABurst_ReadStop (TIM_HandleTypeDef * htim, uint32_t BurstRequestSrc)

Function description

Stop the DMA burst reading.

Parameters

- **htim:** TIM handle
- **BurstRequestSrc:** TIM DMA Request sources to disable.

Return values

- **HAL:** status

HAL_TIM_GenerateEvent

Function name

HAL_StatusTypeDef HAL_TIM_GenerateEvent (TIM_HandleTypeDef * htim, uint32_t EventSource)

Function description

Generate a software event.

Parameters

- **htim:** TIM handle
- **EventSource:** specifies the event source. This parameter can be one of the following values:
 - TIM_EVENTSOURCE_UPDATE: Timer update Event source
 - TIM_EVENTSOURCE_CC1: Timer Capture Compare 1 Event source
 - TIM_EVENTSOURCE_CC2: Timer Capture Compare 2 Event source
 - TIM_EVENTSOURCE_CC3: Timer Capture Compare 3 Event source
 - TIM_EVENTSOURCE_CC4: Timer Capture Compare 4 Event source
 - TIM_EVENTSOURCE_COM: Timer COM event source
 - TIM_EVENTSOURCE_TRIGGER: Timer Trigger Event source
 - TIM_EVENTSOURCE_BREAK: Timer Break event source

Return values

- **HAL:** status

Notes

- Basic timers can only generate an update event.
- TIM_EVENTSOURCE_COM is relevant only with advanced timer instances.
- TIM_EVENTSOURCE_BREAK are relevant only for timer instances supporting a break input.

HAL_TIM_ReadCapturedValue

Function name

uint32_t HAL_TIM_ReadCapturedValue (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Read the captured value from Capture Compare unit.

Parameters

- **htim:** TIM handle.
- **Channel:** TIM Channels to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return values

- **Captured:** value

HAL_TIM_PeriodElapsedCallback

Function name

void HAL_TIM_PeriodElapsedCallback (TIM_HandleTypeDef * htim)

Function description

Period elapsed callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_PeriodElapsedHalfCpltCallback

Function name

void HAL_TIM_PeriodElapsedHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Period elapsed half complete callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_OC_DelayElapsedCallback

Function name

void HAL_TIM_OC_DelayElapsedCallback (TIM_HandleTypeDef * htim)

Function description

Output Compare callback in non-blocking mode.

Parameters

- **htim:** TIM OC handle

Return values

- **None:**

HAL_TIM_IC_CaptureCallback

Function name

```
void HAL_TIM_IC_CaptureCallback (TIM_HandleTypeDef * htim)
```

Function description

Input Capture callback in non-blocking mode.

Parameters

- **htim**: TIM IC handle

Return values

- **None**:

HAL_TIM_IC_CaptureHalfCpltCallback

Function name

```
void HAL_TIM_IC_CaptureHalfCpltCallback (TIM_HandleTypeDef * htim)
```

Function description

Input Capture half complete callback in non-blocking mode.

Parameters

- **htim**: TIM IC handle

Return values

- **None**:

HAL_TIM_PWM_PulseFinishedCallback

Function name

```
void HAL_TIM_PWM_PulseFinishedCallback (TIM_HandleTypeDef * htim)
```

Function description

PWM Pulse finished callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_PWM_PulseFinishedHalfCpltCallback

Function name

```
void HAL_TIM_PWM_PulseFinishedHalfCpltCallback (TIM_HandleTypeDef * htim)
```

Function description

PWM Pulse finished half complete callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIM_TriggerCallback

Function name

void HAL_TIM_TriggerCallback (TIM_HandleTypeDef * htim)

Function description

Hall Trigger detection callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_TriggerHalfCpltCallback

Function name

void HAL_TIM_TriggerHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Hall Trigger detection half complete callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_ErrorCallback

Function name

void HAL_TIM_ErrorCallback (TIM_HandleTypeDef * htim)

Function description

Timer error callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIM_Base_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_Base_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM Base handle state.

Parameters

- **htim:** TIM Base handle

Return values

- **HAL:** state

HAL_TIM_OC_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_OC_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM OC handle state.

Parameters

- **htim**: TIM Output Compare handle

Return values

- **HAL**: state

HAL_TIM_PWM_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_PWM_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM PWM handle state.

Parameters

- **htim**: TIM handle

Return values

- **HAL**: state

HAL_TIM_IC_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_IC_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM Input Capture handle state.

Parameters

- **htim**: TIM IC handle

Return values

- **HAL**: state

HAL_TIM_OnePulse_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_OnePulse_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM One Pulse Mode handle state.

Parameters

- **htim**: TIM OPM handle

Return values

- **HAL**: state

HAL_TIM_Encoder_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIM_Encoder_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM Encoder Mode handle state.

Parameters

- **htim:** TIM Encoder Interface handle

Return values

- **HAL:** state

TIM_Base_SetConfig

Function name

void TIM_Base_SetConfig (TIM_TypeDef * TIMx, TIM_Base_InitTypeDef * Structure)

Function description

Time Base configuration.

Parameters

- **TIMx:** TIM peripheral
- **Structure:** TIM Base configuration structure

Return values

- **None:**

TIM_TI1_SetConfig

Function name

void TIM_TI1_SetConfig (TIM_TypeDef * TIMx, uint32_t TIM_ICPolarity, uint32_t TIM_ICSelection, uint32_t TIM_ICFilter)

Function description

Configure the TI1 as Input.

Parameters

- **TIMx:** to select the TIM peripheral.
- **TIM_ICPolarity:** The Input Polarity. This parameter can be one of the following values:
 - TIM_ICPOLARITY_RISING
 - TIM_ICPOLARITY_FALLING
 - TIM_ICPOLARITY_BOTHEDGE
- **TIM_ICSelection:** specifies the input to be used. This parameter can be one of the following values:
 - TIM_ICSELECTION_DIRECTTI: TIM Input 1 is selected to be connected to IC1.
 - TIM_ICSELECTION_INDIRECTTI: TIM Input 1 is selected to be connected to IC2.
 - TIM_ICSELECTION_TRC: TIM Input 1 is selected to be connected to TRC.
- **TIM_ICFilter:** Specifies the Input Capture Filter. This parameter must be a value between 0x00 and 0x0F.

Return values

- **None:**

Notes

- TIM_ICFilter and TIM_ICPolarity are not used in INDIRECT mode as TI2FP1 (on channel2 path) is used as the input signal. Therefore CCMR1 must be protected against un-initialized filter and polarity values.

TIM_OC2_SetConfig

Function name

```
void TIM_OC2_SetConfig (TIM_TypeDef * TIMx, TIM_OC_InitTypeDef * OC_Config)
```

Function description

Timer Output Compare 2 configuration.

Parameters

- TIMx**: to select the TIM peripheral
- OC_Config**: The output configuration structure

Return values

- None**:

TIM_ETR_SetConfig

Function name

```
void TIM_ETR_SetConfig (TIM_TypeDef * TIMx, uint32_t TIM_ExtTRGPrescaler, uint32_t TIM_ExtTRGPolarity, uint32_t ExtTRGFilter)
```

Function description

Configures the TIMx External Trigger (ETR).

Parameters

- TIMx**: to select the TIM peripheral
- TIM_ExtTRGPrescaler**: The external Trigger Prescaler. This parameter can be one of the following values:
 - TIM_ETRPRESCALER_DIV1: ETRP Prescaler OFF.
 - TIM_ETRPRESCALER_DIV2: ETRP frequency divided by 2.
 - TIM_ETRPRESCALER_DIV4: ETRP frequency divided by 4.
 - TIM_ETRPRESCALER_DIV8: ETRP frequency divided by 8.
- TIM_ExtTRGPolarity**: The external Trigger Polarity. This parameter can be one of the following values:
 - TIM_ETRPOLARITY_INVERTED: active low or falling edge active.
 - TIM_ETRPOLARITY_NONINVERTED: active high or rising edge active.
- ExtTRGFilter**: External Trigger Filter. This parameter must be a value between 0x00 and 0x0F

Return values

- None**:

TIM_DMADelayPulseCplt

Function name

```
void TIM_DMADelayPulseCplt (DMA_HandleTypeDef * hdma)
```

Function description

TIM DMA Delay Pulse complete callback.

Parameters

- hdma**: pointer to DMA handle.

Return values

- **None:**

TIM_DMADelayPulseHalfCplt

Function name

void TIM_DMADelayPulseHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Delay Pulse half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMAError

Function name

void TIM_DMAError (DMA_HandleTypeDef * hdma)

Function description

TIM DMA error callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMACaptureCplt

Function name

void TIM_DMACaptureCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Capture complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_DMACaptureHalfCplt

Function name

void TIM_DMACaptureHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Capture half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

TIM_CCxChannelCmd

Function name

void TIM_CCxChannelCmd (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ChannelState)

Function description

Enables or disables the TIM Capture Compare Channel x.

Parameters

- **TIMx:** to select the TIM peripheral
- **Channel:** specifies the TIM Channel This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1
 - TIM_CHANNEL_2: TIM Channel 2
 - TIM_CHANNEL_3: TIM Channel 3
 - TIM_CHANNEL_4: TIM Channel 4
- **ChannelState:** specifies the TIM Channel CCxE bit new state. This parameter can be: TIM_CCx_ENABLE or TIM_CCx_DISABLE.

Return values

- **None:**

44.3 TIM Firmware driver defines

The following section lists the various define and macros of the module.

44.3.1

TIM

TIM

TIM Automatic Output Enable

TIM_AUTOMATICOUTPUT_DISABLE

MOE can be set only by software

TIM_AUTOMATICOUTPUT_ENABLE

MOE can be set by software or automatically at the next update event (if none of the break inputs BRK and BRK2 is active)

TIM Auto-Reload Preload

TIM_AUTORELOAD_PRELOAD_DISABLE

TIMx_ARR register is not buffered

TIM_AUTORELOAD_PRELOAD_ENABLE

TIMx_ARR register is buffered

TIM Break Input Enable

TIM_BREAK_ENABLE

Break input BRK is enabled

TIM_BREAK_DISABLE

Break input BRK is disabled

TIM Break Input Polarity

TIM_BREAKPOLARITY_LOW

Break input BRK is active low

TIM_BREAKPOLARITY_HIGH

Break input BRK is active high

TIM Channel

TIM_CHANNEL_1

Capture/compare channel 1 identifier

TIM_CHANNEL_2

Capture/compare channel 2 identifier

TIM_CHANNEL_3

Capture/compare channel 3 identifier

TIM_CHANNEL_4

Capture/compare channel 4 identifier

TIM_CHANNEL_ALL

Global Capture/compare channel identifier

TIM Clear Input Polarity

TIM_CLEARINPUTPOLARITY_INVERTED

Polarity for ETRx pin

TIM_CLEARINPUTPOLARITY_NONINVERTED

Polarity for ETRx pin

TIM Clear Input Prescaler

TIM_CLEARINPUTPRESCALER_DIV1

No prescaler is used

TIM_CLEARINPUTPRESCALER_DIV2

Prescaler for External ETR pin: Capture performed once every 2 events.

TIM_CLEARINPUTPRESCALER_DIV4

Prescaler for External ETR pin: Capture performed once every 4 events.

TIM_CLEARINPUTPRESCALER_DIV8

Prescaler for External ETR pin: Capture performed once every 8 events.

TIM Clear Input Source

TIM_CLEARINPUTSOURCE_NONE

OCREF_CLR is disabled

TIM_CLEARINPUTSOURCE_ETR

OCREF_CLR is connected to ETRF input

TIM Clock Division

TIM_CLOCKDIVISION_DIV1

Clock division: $tDTS=tCK_INT$

TIM_CLOCKDIVISION_DIV2

Clock division: $tDTS=2*tCK_INT$

TIM_CLOCKDIVISION_DIV4

Clock division: $tDTS=4*tCK_INT$

TIM Clock Polarity

TIM_CLOCKPOLARITY_INVERTED

Polarity for ETRx clock sources

TIM_CLOCKPOLARITY_NONINVERTED

Polarity for ETRx clock sources

TIM_CLOCKPOLARITY_RISING

Polarity for Tlx clock sources

TIM_CLOCKPOLARITY_FALLING

Polarity for Tlx clock sources

TIM_CLOCKPOLARITY_BOTHEDGE

Polarity for Tlx clock sources

TIM Clock Prescaler**TIM_CLOCKPRESCALER_DIV1**

No prescaler is used

TIM_CLOCKPRESCALER_DIV2

Prescaler for External ETR Clock: Capture performed once every 2 events.

TIM_CLOCKPRESCALER_DIV4

Prescaler for External ETR Clock: Capture performed once every 4 events.

TIM_CLOCKPRESCALER_DIV8

Prescaler for External ETR Clock: Capture performed once every 8 events.

TIM Clock Source**TIM_CLOCKSOURCE_ETRMODE2**

External clock source mode 2

TIM_CLOCKSOURCE_INTERNAL

Internal clock source

TIM_CLOCKSOURCE_ITR0

External clock source mode 1 (ITR0)

TIM_CLOCKSOURCE_ITR1

External clock source mode 1 (ITR1)

TIM_CLOCKSOURCE_ITR2

External clock source mode 1 (ITR2)

TIM_CLOCKSOURCE_ITR3

External clock source mode 1 (ITR3)

TIM_CLOCKSOURCE_TI1ED

External clock source mode 1 (TTI1FP1 + edge detect.)

TIM_CLOCKSOURCE_TI1

External clock source mode 1 (TTI1FP1)

TIM_CLOCKSOURCE_TI2

External clock source mode 1 (TTI2FP2)

TIM_CLOCKSOURCE_ETRMODE1

External clock source mode 1 (ETRF)

TIM Commutation Source**TIM_COMMUTATION_TRGI**

When Capture/compare control bits are preloaded, they are updated by setting the COMG bit or when an rising edge occurs on trigger input

TIM_COMMUTATION_SOFTWARE

When Capture/compare control bits are preloaded, they are updated by setting the COMG bit

TIM Counter Mode**TIM_COUNTERMODE_UP**

Counter used as up-counter

TIM_COUNTERMODE_DOWN

Counter used as down-counter

TIM_COUNTERMODE_CENTERALIGNED1

Center-aligned mode 1

TIM_COUNTERMODE_CENTERALIGNED2

Center-aligned mode 2

TIM_COUNTERMODE_CENTERALIGNED3

Center-aligned mode 3

TIM DMA Base Address**TIM_DMABASE_CR1****TIM_DMABASE_CR2****TIM_DMABASE_SMCR****TIM_DMABASE_DIER****TIM_DMABASE_SR****TIM_DMABASE_EGR****TIM_DMABASE_CCMR1****TIM_DMABASE_CCMR2****TIM_DMABASE_CCER****TIM_DMABASE_CNT****TIM_DMABASE_PSC****TIM_DMABASE_ARR****TIM_DMABASE_RCR****TIM_DMABASE_CCR1**

TIM_DMABASE_CCR2

TIM_DMABASE_CCR3

TIM_DMABASE_CCR4

TIM_DMABASE_BDTR

TIM_DMABASE_DCR

TIM_DMABASE_DMAR

TIM DMA Burst Length

TIM_DMABURSTLENGTH_1TRANSFER

The transfer is done to 1 register starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_2TRANSFERS

The transfer is done to 2 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_3TRANSFERS

The transfer is done to 3 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_4TRANSFERS

The transfer is done to 4 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_5TRANSFERS

The transfer is done to 5 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_6TRANSFERS

The transfer is done to 6 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_7TRANSFERS

The transfer is done to 7 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_8TRANSFERS

The transfer is done to 8 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_9TRANSFERS

The transfer is done to 9 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_10TRANSFERS

The transfer is done to 10 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_11TRANSFERS

The transfer is done to 11 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_12TRANSFERS

The transfer is done to 12 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_13TRANSFERS

The transfer is done to 13 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_14TRANSFERS

The transfer is done to 14 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_15TRANSFERS

The transfer is done to 15 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_16TRANSFERS

The transfer is done to 16 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_17TRANSFERS

The transfer is done to 17 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM_DMABURSTLENGTH_18TRANSFERS

The transfer is done to 18 registers starting from TIMx_CR1 + TIMx_DCR.DBA

TIM DMA Sources

TIM_DMA_UPDATE

DMA request is triggered by the update event

TIM_DMA_CC1

DMA request is triggered by the capture/compare match 1 event

TIM_DMA_CC2

DMA request is triggered by the capture/compare match 2 event event

TIM_DMA_CC3

DMA request is triggered by the capture/compare match 3 event event

TIM_DMA_CC4

DMA request is triggered by the capture/compare match 4 event event

TIM_DMA_COM

DMA request is triggered by the commutation event

TIM_DMA_TRIGGER

DMA request is triggered by the trigger event

TIM Encoder Mode

TIM_ENCODERMODE_TI1

Quadrature encoder mode 1, x2 mode, counts up/down on TI1FP1 edge depending on TI2FP2 level

TIM_ENCODERMODE_TI2

Quadrature encoder mode 2, x2 mode, counts up/down on TI2FP2 edge depending on TI1FP1 level.

TIM_ENCODERMODE_TI12

Quadrature encoder mode 3, x4 mode, counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input.

TIM ETR Polarity

TIM_ETRPOLARITY_INVERTED

Polarity for ETR source

TIM_ETRPOLARITY_NONINVERTED

Polarity for ETR source

TIM ETR Prescaler

TIM_ETRPRESCALER_DIV1

No prescaler is used

TIM_ETRPRESCALER_DIV2

ETR input source is divided by 2

TIM_ETRPRESCALER_DIV4

ETR input source is divided by 4

TIM_ETRPRESCALER_DIV8

ETR input source is divided by 8

TIM Event Source

TIM_EVENTSOURCE_UPDATE

Reinitialize the counter and generates an update of the registers

TIM_EVENTSOURCE_CC1

A capture/compare event is generated on channel 1

TIM_EVENTSOURCE_CC2

A capture/compare event is generated on channel 2

TIM_EVENTSOURCE_CC3

A capture/compare event is generated on channel 3

TIM_EVENTSOURCE_CC4

A capture/compare event is generated on channel 4

TIM_EVENTSOURCE_COM

A commutation event is generated

TIM_EVENTSOURCE_TRIGGER

A trigger event is generated

TIM_EVENTSOURCE_BREAK

A break event is generated

TIM Exported Macros

__HAL_TIM_RESET_HANDLE_STATE

Description:

- Reset TIM handle state.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- None

__HAL_TIM_ENABLE

Description:

- Enable the TIM peripheral.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

`__HAL_TIM_MOE_ENABLE`

Description:

- Enable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

`__HAL_TIM_DISABLE`

Description:

- Disable the TIM peripheral.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

`__HAL_TIM_MOE_DISABLE`

Description:

- Disable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

Notes:

- The Main Output Enable of a timer instance is disabled only if all the CCx and CCxN channels have been disabled

`__HAL_TIM_MOE_DISABLE_UNCONDITIONALLY`

Description:

- Disable the TIM main Output.

Parameters:

- `__HANDLE__`: TIM handle

Return value:

- None

Notes:

- The Main Output Enable of a timer instance is disabled unconditionally

`__HAL_TIM_ENABLE_IT`

Description:

- Enable the specified TIM interrupt.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__INTERRUPT__`: specifies the TIM interrupt source to enable. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- None

`__HAL_TIM_DISABLE_IT`

Description:

- Disable the specified TIM interrupt.

Parameters:

- `__HANDLE__`: specifies the TIM Handle.
- `__INTERRUPT__`: specifies the TIM interrupt source to disable. This parameter can be one of the following values:
 - `TIM_IT_UPDATE`: Update interrupt
 - `TIM_IT_CC1`: Capture/Compare 1 interrupt
 - `TIM_IT_CC2`: Capture/Compare 2 interrupt
 - `TIM_IT_CC3`: Capture/Compare 3 interrupt
 - `TIM_IT_CC4`: Capture/Compare 4 interrupt
 - `TIM_IT_COM`: Commutation interrupt
 - `TIM_IT_TRIGGER`: Trigger interrupt
 - `TIM_IT_BREAK`: Break interrupt

Return value:

- None

__HAL_TIM_ENABLE_DMA

Description:

- Enable the specified DMA request.

Parameters:

- __HANDLE__: specifies the TIM Handle.
- __DMA__: specifies the TIM DMA request to enable. This parameter can be one of the following values:
 - TIM_DMA_UPDATE: Update DMA request
 - TIM_DMA_CC1: Capture/Compare 1 DMA request
 - TIM_DMA_CC2: Capture/Compare 2 DMA request
 - TIM_DMA_CC3: Capture/Compare 3 DMA request
 - TIM_DMA_CC4: Capture/Compare 4 DMA request
 - TIM_DMA_COM: Commutation DMA request
 - TIM_DMA_TRIGGER: Trigger DMA request

Return value:

- None

__HAL_TIM_DISABLE_DMA

Description:

- Disable the specified DMA request.

Parameters:

- __HANDLE__: specifies the TIM Handle.
- __DMA__: specifies the TIM DMA request to disable. This parameter can be one of the following values:
 - TIM_DMA_UPDATE: Update DMA request
 - TIM_DMA_CC1: Capture/Compare 1 DMA request
 - TIM_DMA_CC2: Capture/Compare 2 DMA request
 - TIM_DMA_CC3: Capture/Compare 3 DMA request
 - TIM_DMA_CC4: Capture/Compare 4 DMA request
 - TIM_DMA_COM: Commutation DMA request
 - TIM_DMA_TRIGGER: Trigger DMA request

Return value:

- None

__HAL_TIM_GET_FLAG

Description:

- Check whether the specified TIM interrupt flag is set or not.

Parameters:

- **__HANDLE__**: specifies the TIM Handle.
- **__FLAG__**: specifies the TIM interrupt flag to check. This parameter can be one of the following values:
 - TIM_FLAG_UPDATE: Update interrupt flag
 - TIM_FLAG_CC1: Capture/Compare 1 interrupt flag
 - TIM_FLAG_CC2: Capture/Compare 2 interrupt flag
 - TIM_FLAG_CC3: Capture/Compare 3 interrupt flag
 - TIM_FLAG_CC4: Capture/Compare 4 interrupt flag
 - TIM_FLAG_COM: Commutation interrupt flag
 - TIM_FLAG_TRIGGER: Trigger interrupt flag
 - TIM_FLAG_BREAK: Break interrupt flag
 - TIM_FLAG_CC1OF: Capture/Compare 1 overcapture flag
 - TIM_FLAG_CC2OF: Capture/Compare 2 overcapture flag
 - TIM_FLAG_CC3OF: Capture/Compare 3 overcapture flag
 - TIM_FLAG_CC4OF: Capture/Compare 4 overcapture flag

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

__HAL_TIM_CLEAR_FLAG

Description:

- Clear the specified TIM interrupt flag.

Parameters:

- **__HANDLE__**: specifies the TIM Handle.
- **__FLAG__**: specifies the TIM interrupt flag to clear. This parameter can be one of the following values:
 - TIM_FLAG_UPDATE: Update interrupt flag
 - TIM_FLAG_CC1: Capture/Compare 1 interrupt flag
 - TIM_FLAG_CC2: Capture/Compare 2 interrupt flag
 - TIM_FLAG_CC3: Capture/Compare 3 interrupt flag
 - TIM_FLAG_CC4: Capture/Compare 4 interrupt flag
 - TIM_FLAG_COM: Commutation interrupt flag
 - TIM_FLAG_TRIGGER: Trigger interrupt flag
 - TIM_FLAG_BREAK: Break interrupt flag
 - TIM_FLAG_CC1OF: Capture/Compare 1 overcapture flag
 - TIM_FLAG_CC2OF: Capture/Compare 2 overcapture flag
 - TIM_FLAG_CC3OF: Capture/Compare 3 overcapture flag
 - TIM_FLAG_CC4OF: Capture/Compare 4 overcapture flag

Return value:

- The: new state of **__FLAG__** (TRUE or FALSE).

__HAL_TIM_GET_IT_SOURCE

Description:

- Check whether the specified TIM interrupt source is enabled or not.

Parameters:

- **__HANDLE__**: TIM handle
- **__INTERRUPT__**: specifies the TIM interrupt source to check. This parameter can be one of the following values:
 - TIM_IT_UPDATE: Update interrupt
 - TIM_IT_CC1: Capture/Compare 1 interrupt
 - TIM_IT_CC2: Capture/Compare 2 interrupt
 - TIM_IT_CC3: Capture/Compare 3 interrupt
 - TIM_IT_CC4: Capture/Compare 4 interrupt
 - TIM_IT_COM: Commutation interrupt
 - TIM_IT_TRIGGER: Trigger interrupt
 - TIM_IT_BREAK: Break interrupt

Return value:

- The: state of TIM_IT (SET or RESET).

__HAL_TIM_CLEAR_IT

Description:

- Clear the TIM interrupt pending bits.

Parameters:

- **__HANDLE__**: TIM handle
- **__INTERRUPT__**: specifies the interrupt pending bit to clear. This parameter can be one of the following values:
 - TIM_IT_UPDATE: Update interrupt
 - TIM_IT_CC1: Capture/Compare 1 interrupt
 - TIM_IT_CC2: Capture/Compare 2 interrupt
 - TIM_IT_CC3: Capture/Compare 3 interrupt
 - TIM_IT_CC4: Capture/Compare 4 interrupt
 - TIM_IT_COM: Commutation interrupt
 - TIM_IT_TRIGGER: Trigger interrupt
 - TIM_IT_BREAK: Break interrupt

Return value:

- None

__HAL_TIM_IS_TIM_COUNTING_DOWN

Description:

- Indicates whether or not the TIM Counter is used as downcounter.

Parameters:

- **__HANDLE__**: TIM handle.

Return value:

- False: (Counter used as upcounter) or True (Counter used as downcounter)

Notes:

- This macro is particularly useful to get the counting mode when the timer operates in Center-aligned mode or Encoder mode.

__HAL_TIM_SET_PRESCALER

Description:

- Set the TIM Prescaler on runtime.

Parameters:

- __HANDLE__: TIM handle.
- __PRESC__: specifies the Prescaler new value.

Return value:

- None

__HAL_TIM_SET_COUNTER

Description:

- Set the TIM Counter Register value on runtime.

Parameters:

- __HANDLE__: TIM handle.
- __COUNTER__: specifies the Counter register new value.

Return value:

- None

__HAL_TIM_GET_COUNTER

Description:

- Get the TIM Counter Register value on runtime.

Parameters:

- __HANDLE__: TIM handle.

Return value:

- 16-bit: or 32-bit value of the timer counter register (TIMx_CNT)

__HAL_TIM_SET_AUTORELOAD

Description:

- Set the TIM Autoreload Register value on runtime without calling another time any Init function.

Parameters:

- __HANDLE__: TIM handle.
- __AUTORELOAD__: specifies the Counter register new value.

Return value:

- None

__HAL_TIM_GET_AUTORELOAD

Description:

- Get the TIM Autoreload Register value on runtime.

Parameters:

- __HANDLE__: TIM handle.

Return value:

- 16-bit: or 32-bit value of the timer auto-reload register(TIMx_ARR)

__HAL_TIM_SET_CLOCKDIVISION

Description:

- Set the TIM Clock Division value on runtime without calling another time any Init function.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CKD__`: specifies the clock division value. This parameter can be one of the following value:
 - `TIM_CLOCKDIVISION_DIV1`: $tDTS=tCK_INT$
 - `TIM_CLOCKDIVISION_DIV2`: $tDTS=2*tCK_INT$
 - `TIM_CLOCKDIVISION_DIV4`: $tDTS=4*tCK_INT$

Return value:

- None

__HAL_TIM_GET_CLOCKDIVISION

Description:

- Get the TIM Clock Division value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- The clock division can be one of the following values:
 - `TIM_CLOCKDIVISION_DIV1`: $tDTS=tCK_INT$
 - `TIM_CLOCKDIVISION_DIV2`: $tDTS=2*tCK_INT$
 - `TIM_CLOCKDIVISION_DIV4`: $tDTS=4*tCK_INT$

__HAL_TIM_SET_ICPRESCALER

Description:

- Set the TIM Input Capture prescaler on runtime without calling another time

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: TIM Channel 1 selected
 - `TIM_CHANNEL_2`: TIM Channel 2 selected
 - `TIM_CHANNEL_3`: TIM Channel 3 selected
 - `TIM_CHANNEL_4`: TIM Channel 4 selected
- `__ICPSC__`: specifies the Input Capture4 prescaler new value. This parameter can be one of the following values:
 - `TIM_ICPSC_DIV1`: no prescaler
 - `TIM_ICPSC_DIV2`: capture is done once every 2 events
 - `TIM_ICPSC_DIV4`: capture is done once every 4 events
 - `TIM_ICPSC_DIV8`: capture is done once every 8 events

Return value:

- None

__HAL_TIM_GET_ICPRESCALER

Description:

- Get the TIM Input Capture prescaler on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: get input capture 1 prescaler value
 - `TIM_CHANNEL_2`: get input capture 2 prescaler value
 - `TIM_CHANNEL_3`: get input capture 3 prescaler value
 - `TIM_CHANNEL_4`: get input capture 4 prescaler value

Return value:

- The: input capture prescaler can be one of the following values:
 - `TIM_ICPSC_DIV1`: no prescaler
 - `TIM_ICPSC_DIV2`: capture is done once every 2 events
 - `TIM_ICPSC_DIV4`: capture is done once every 4 events
 - `TIM_ICPSC_DIV8`: capture is done once every 8 events

__HAL_TIM_SET_COMPARE

Description:

- Set the TIM Capture Compare Register value on runtime without calling another time ConfigChannel function.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: TIM Channel 1 selected
 - `TIM_CHANNEL_2`: TIM Channel 2 selected
 - `TIM_CHANNEL_3`: TIM Channel 3 selected
 - `TIM_CHANNEL_4`: TIM Channel 4 selected
- `__COMPARE__`: specifies the Capture Compare register new value.

Return value:

- None

__HAL_TIM_GET_COMPARE

Description:

- Get the TIM Capture Compare Register value on runtime.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channel associated with the capture compare register This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: get capture/compare 1 register value
 - `TIM_CHANNEL_2`: get capture/compare 2 register value
 - `TIM_CHANNEL_3`: get capture/compare 3 register value
 - `TIM_CHANNEL_4`: get capture/compare 4 register value

Return value:

- 16-bit: or 32-bit value of the capture/compare register (`TIMx_CCRy`)

__HAL_TIM_ENABLE_OCxPRELOAD

Description:

- Set the TIM Output compare preload.

Parameters:

- **__HANDLE__**: TIM handle.
- **__CHANNEL__**: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

__HAL_TIM_DISABLE_OCxPRELOAD

Description:

- Reset the TIM Output compare preload.

Parameters:

- **__HANDLE__**: TIM handle.
- **__CHANNEL__**: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

__HAL_TIM_ENABLE_OCxFAST

Description:

- Enable fast mode for a given channel.

Parameters:

- **__HANDLE__**: TIM handle.
- **__CHANNEL__**: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected

Return value:

- None

Notes:

- When fast mode is enabled an active edge on the trigger input acts like a compare match on CCx output. Delay to sample the trigger input and to activate CCx output is reduced to 3 clock cycles. Fast mode acts only if the channel is configured in PWM1 or PWM2 mode.

__HAL_TIM_DISABLE_OCxFAST

Description:

- Disable fast mode for a given channel.

Parameters:

- `__HANDLE__`: TIM handle.
- `__CHANNEL__`: TIM Channels to be configured. This parameter can be one of the following values:
 - `TIM_CHANNEL_1`: TIM Channel 1 selected
 - `TIM_CHANNEL_2`: TIM Channel 2 selected
 - `TIM_CHANNEL_3`: TIM Channel 3 selected
 - `TIM_CHANNEL_4`: TIM Channel 4 selected

Return value:

- None

Notes:

- When fast mode is disabled CCx output behaves normally depending on counter and CCRx values even when the trigger is ON. The minimum delay to activate CCx output when an active edge occurs on the trigger input is 5 clock cycles.

__HAL_TIM_URS_ENABLE

Description:

- Set the Update Request Source (URS) bit of the TIMx_CR1 register.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- None

Notes:

- When the URS bit of the TIMx_CR1 register is set, only counter overflow/underflow generates an update interrupt or DMA request (if enabled)

__HAL_TIM_URS_DISABLE

Description:

- Reset the Update Request Source (URS) bit of the TIMx_CR1 register.

Parameters:

- `__HANDLE__`: TIM handle.

Return value:

- None

Notes:

- When the URS bit of the TIMx_CR1 register is reset, any of the following events generate an update interrupt or DMA request (if enabled): `_ Counter overflow underflow` `_ Setting the UG bit` `_ Update generation through the slave mode controller`

__HAL_TIM_SET_CAPTUREPOLARITY

Description:

- Set the TIM Capture x input polarity on runtime.

Parameters:

- __HANDLE__: TIM handle.
- __CHANNEL__: TIM Channels to be configured. This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
 - TIM_CHANNEL_4: TIM Channel 4 selected
- __POLARITY__: Polarity for Tlx source
 - TIM_INPUTCHANNELPOLARITY_RISING: Rising Edge
 - TIM_INPUTCHANNELPOLARITY_FALLING: Falling Edge
 - TIM_INPUTCHANNELPOLARITY_BOTHEDGE: Rising and Falling Edge

Return value:

- None

TIM Flag Definition

TIM_FLAG_UPDATE

Update interrupt flag

TIM_FLAG_CC1

Capture/Compare 1 interrupt flag

TIM_FLAG_CC2

Capture/Compare 2 interrupt flag

TIM_FLAG_CC3

Capture/Compare 3 interrupt flag

TIM_FLAG_CC4

Capture/Compare 4 interrupt flag

TIM_FLAG_COM

Commutation interrupt flag

TIM_FLAG_TRIGGER

Trigger interrupt flag

TIM_FLAG_BREAK

Break interrupt flag

TIM_FLAG_CC1OF

Capture 1 overcapture flag

TIM_FLAG_CC2OF

Capture 2 overcapture flag

TIM_FLAG_CC3OF

Capture 3 overcapture flag

TIM_FLAG_CC4OF

Capture 4 overcapture flag

TIM Input Capture Polarity

TIM_ICPOLARITY_RISING

Capture triggered by rising edge on timer input

TIM_ICPOLARITY_FALLING

Capture triggered by falling edge on timer input

TIM_ICPOLARITY_BOTHEDGE

Capture triggered by both rising and falling edges on timer input

TIM Input Capture Prescaler

TIM_ICPSC_DIV1

Capture performed each time an edge is detected on the capture input

TIM_ICPSC_DIV2

Capture performed once every 2 events

TIM_ICPSC_DIV4

Capture performed once every 4 events

TIM_ICPSC_DIV8

Capture performed once every 8 events

TIM Input Capture Selection

TIM_ICSELECTION_DIRECTTI

TIM Input 1, 2, 3 or 4 is selected to be connected to IC1, IC2, IC3 or IC4, respectively

TIM_ICSELECTION_INDIRECTTI

TIM Input 1, 2, 3 or 4 is selected to be connected to IC2, IC1, IC4 or IC3, respectively

TIM_ICSELECTION_TRC

TIM Input 1, 2, 3 or 4 is selected to be connected to TRC

TIM Input Channel polarity

TIM_INPUTCHANNELPOLARITY_RISING

Polarity for Tlx source

TIM_INPUTCHANNELPOLARITY_FALLING

Polarity for Tlx source

TIM_INPUTCHANNELPOLARITY_BOTHEDGE

Polarity for Tlx source

TIM interrupt Definition

TIM_IT_UPDATE

Update interrupt

TIM_IT_CC1

Capture/Compare 1 interrupt

TIM_IT_CC2

Capture/Compare 2 interrupt

TIM_IT_CC3

Capture/Compare 3 interrupt

TIM_IT_CC4

Capture/Compare 4 interrupt

TIM_IT_COM

Commutation interrupt

TIM_IT_TRIGGER

Trigger interrupt

TIM_IT_BREAK

Break interrupt

TIM Lock level

TIM_LOCKLEVEL_OFF

LOCK OFF

TIM_LOCKLEVEL_1

LOCK Level 1

TIM_LOCKLEVEL_2

LOCK Level 2

TIM_LOCKLEVEL_3

LOCK Level 3

TIM Master Mode Selection

TIM_TRGO_RESET

TIMx_EGR.UG bit is used as trigger output (TRGO)

TIM_TRGO_ENABLE

TIMx_CR1.CEN bit is used as trigger output (TRGO)

TIM_TRGO_UPDATE

Update event is used as trigger output (TRGO)

TIM_TRGO_OC1

Capture or a compare match 1 is used as trigger output (TRGO)

TIM_TRGO_OC1REF

OC1REF signal is used as trigger output (TRGO)

TIM_TRGO_OC2REF

OC2REF signal is used as trigger output (TRGO)

TIM_TRGO_OC3REF

OC3REF signal is used as trigger output (TRGO)

TIM_TRGO_OC4REF

OC4REF signal is used as trigger output (TRGO)

TIM Master/Slave Mode

TIM_MASTERSLAVEMODE_ENABLE

No action

TIM_MASTERSLAVEMODE_DISABLE

Master/slave mode is selected

TIM One Pulse Mode

TIM_OPMODE_SINGLE

Counter stops counting at the next update event

TIM_OPMODE_REPETITIVE

Counter is not stopped at update event

TIM OSSI OffState Selection for Idle mode state

TIM_OSSI_ENABLE

When inactive, OC/OCN outputs are enabled (still controlled by the timer)

TIM_OSSI_DISABLE

When inactive, OC/OCN outputs are disabled (not controlled any longer by the timer)

TIM OSSR OffState Selection for Run mode state

TIM_OSSR_ENABLE

When inactive, OC/OCN outputs are enabled (still controlled by the timer)

TIM_OSSR_DISABLE

When inactive, OC/OCN outputs are disabled (not controlled any longer by the timer)

TIM Output Compare and PWM Modes

TIM_OCMODE_TIMING

Frozen

TIM_OCMODE_ACTIVE

Set channel to active level on match

TIM_OCMODE_INACTIVE

Set channel to inactive level on match

TIM_OCMODE_TOGGLE

Toggle

TIM_OCMODE_PWM1

PWM mode 1

TIM_OCMODE_PWM2

PWM mode 2

TIM_OCMODE_FORCED_ACTIVE

Force active level

TIM_OCMODE_FORCED_INACTIVE

Force inactive level

TIM Output Compare Idle State

TIM_OCIDLESTATE_SET

Output Idle state: OCx=1 when MOE=0

TIM_OCIDLESTATE_RESET

Output Idle state: OCx=0 when MOE=0

TIM Complementary Output Compare Idle State

TIM_OCNIDLESTATE_SET

Complementary output Idle state: OCxN=1 when MOE=0

TIM_OCNIDLESTATE_RESET

Complementary output Idle state: OCxN=0 when MOE=0

TIM Complementary Output Compare Polarity

TIM_OCNPOLARITY_HIGH

Capture/Compare complementary output polarity

TIM_OCNPOLARITY_LOW

Capture/Compare complementary output polarity

TIM Complementary Output Compare State

TIM_OUTPUTNSTATE_DISABLE

OCxN is disabled

TIM_OUTPUTNSTATE_ENABLE

OCxN is enabled

TIM Output Compare Polarity

TIM_OCPOLARITY_HIGH

Capture/Compare output polarity

TIM_OCPOLARITY_LOW

Capture/Compare output polarity

TIM Output Compare State

TIM_OUTPUTSTATE_DISABLE

Capture/Compare 1 output disabled

TIM_OUTPUTSTATE_ENABLE

Capture/Compare 1 output enabled

TIM Output Fast State

TIM_OCFAST_DISABLE

Output Compare fast disable

TIM_OCFAST_ENABLE

Output Compare fast enable

TIM Slave mode

TIM_SLAVEMODE_DISABLE

Slave mode disabled

TIM_SLAVEMODE_RESET

Reset Mode

TIM_SLAVEMODE_GATED

Gated Mode

TIM_SLAVEMODE_TRIGGER

Trigger Mode

TIM_SLAVEMODE_EXTERNAL1

External Clock Mode 1

TIM T11 Input Selection

TIM_TI1SELECTION_CH1

The TIMx_CH1 pin is connected to TI1 input

TIM_TI1SELECTION_XORCOMBINATION

The TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input (XOR combination)

TIM Trigger Polarity

TIM_TRIGGERPOLARITY_INVERTED

Polarity for ETRx trigger sources

TIM_TRIGGERPOLARITY_NONINVERTED

Polarity for ETRx trigger sources

TIM_TRIGGERPOLARITY_RISING

Polarity for TIxFPx or TI1_ED trigger sources

TIM_TRIGGERPOLARITY_FALLING

Polarity for TIxFPx or TI1_ED trigger sources

TIM_TRIGGERPOLARITY_BOTHEDGE

Polarity for TIxFPx or TI1_ED trigger sources

TIM Trigger Prescaler

TIM_TRIGGERPRESCALER_DIV1

No prescaler is used

TIM_TRIGGERPRESCALER_DIV2

Prescaler for External ETR Trigger: Capture performed once every 2 events.

TIM_TRIGGERPRESCALER_DIV4

Prescaler for External ETR Trigger: Capture performed once every 4 events.

TIM_TRIGGERPRESCALER_DIV8

Prescaler for External ETR Trigger: Capture performed once every 8 events.

TIM Trigger Selection

TIM_TS_ITR0

Internal Trigger 0 (ITR0)

TIM_TS_ITR1

Internal Trigger 1 (ITR1)

TIM_TS_ITR2

Internal Trigger 2 (ITR2)

TIM_TS_ITR3

Internal Trigger 3 (ITR3)

TIM_TS_TI1F_ED

TI1 Edge Detector (TI1F_ED)

TIM_TS_TI1FP1

Filtered Timer Input 1 (TI1FP1)

TIM_TS_TI2FP2

Filtered Timer Input 2 (TI2FP2)

TIM_TS_ETRF

Filtered External Trigger input (ETRF)

TIM_TS_NONE

No trigger selected

45 HAL TIM Extension Driver

45.1 TIMEx Firmware driver registers structures

45.1.1 TIM_HallSensor_InitTypeDef

TIM_HallSensor_InitTypeDef is defined in the stm32f2xx_hal_tim_ex.h

Data Fields

- **uint32_t IC1Polarity**
- **uint32_t IC1Prescaler**
- **uint32_t IC1Filter**
- **uint32_t Commutation_Delay**

Field Documentation

- **uint32_t TIM_HallSensor_InitTypeDef::IC1Polarity**
Specifies the active edge of the input signal. This parameter can be a value of [TIM_Input_Capture_Polarity](#)
- **uint32_t TIM_HallSensor_InitTypeDef::IC1Prescaler**
Specifies the Input Capture Prescaler. This parameter can be a value of [TIM_Input_Capture_Prescaler](#)
- **uint32_t TIM_HallSensor_InitTypeDef::IC1Filter**
Specifies the input capture filter. This parameter can be a number between Min_Data = 0x0 and Max_Data = 0xF
- **uint32_t TIM_HallSensor_InitTypeDef::Commutation_Delay**
Specifies the pulse value to be loaded into the Capture Compare Register. This parameter can be a number between Min_Data = 0x0000 and Max_Data = 0xFFFF

45.2 TIMEx Firmware driver API description

The following section lists the various functions of the TIMEx library.

45.2.1 TIMER Extended features

The Timer Extended features include:

1. Complementary outputs with programmable dead-time for :
 - Output Compare
 - PWM generation (Edge and Center-aligned Mode)
 - One-pulse mode output
2. Synchronization circuit to control the timer with external signals and to interconnect several timers together.
3. Break input to put the timer output signals in reset state or in a known state.
4. Supports incremental (quadrature) encoder and hall-sensor circuitry for positioning purposes

45.2.2 How to use this driver

1. Initialize the TIM low level resources by implementing the following functions depending on the selected feature:
 - Hall Sensor output : `HAL_TIMEx_HallSensor_MspInit()`
2. Initialize the TIM low level resources :
 - a. Enable the TIM interface clock using `__HAL_RCC_TIMx_CLK_ENABLE();`
 - b. TIM pins configuration
 - Enable the clock for the TIM GPIOs using the following function:
`__HAL_RCC_GPIOx_CLK_ENABLE();`
 - Configure these TIM pins in Alternate function mode using `HAL_GPIO_Init();`

3. The external Clock can be configured, if needed (the default clock is the internal clock from the APBx), using the following function: `HAL_TIM_ConfigClockSource`, the clock configuration should be done before any start function.
4. Configure the TIM in the desired functioning mode using one of the initialization function of this driver:
 - `HAL_TIMEx_HallSensor_Init()` and `HAL_TIMEx_ConfigCommutEvent()`: to use the Timer Hall Sensor Interface and the commutation event with the corresponding Interrupt and DMA request if needed (Note that One Timer is used to interface with the Hall sensor Interface and another Timer should be used to use the commutation event).
5. Activate the TIM peripheral using one of the start functions:
 - Complementary Output Compare : `HAL_TIMEx_OC_N_Start()`, `HAL_TIMEx_OC_N_Start_DMA()`, `HAL_TIMEx_OC_Start_IT()`
 - Complementary PWM generation : `HAL_TIMEx_PWMN_Start()`, `HAL_TIMEx_PWMN_Start_DMA()`, `HAL_TIMEx_PWMN_Start_IT()`
 - Complementary One-pulse mode output : `HAL_TIMEx_OnePulseN_Start()`, `HAL_TIMEx_OnePulseN_Start_IT()`
 - Hall Sensor output : `HAL_TIMEx_HallSensor_Start()`, `HAL_TIMEx_HallSensor_Start_DMA()`, `HAL_TIMEx_HallSensor_Start_IT()`.

45.2.3 Timer Hall Sensor functions

This section provides functions allowing to:

- Initialize and configure TIM HAL Sensor.
- De-initialize TIM HAL Sensor.
- Start the Hall Sensor Interface.
- Stop the Hall Sensor Interface.
- Start the Hall Sensor Interface and enable interrupts.
- Stop the Hall Sensor Interface and disable interrupts.
- Start the Hall Sensor Interface and enable DMA transfers.
- Stop the Hall Sensor Interface and disable DMA transfers.

This section contains the following APIs:

- [*HAL_TIMEx_HallSensor_Init*](#)
- [*HAL_TIMEx_HallSensor_DeInit*](#)
- [*HAL_TIMEx_HallSensor_MspInit*](#)
- [*HAL_TIMEx_HallSensor_MspDeInit*](#)
- [*HAL_TIMEx_HallSensor_Start*](#)
- [*HAL_TIMEx_HallSensor_Stop*](#)
- [*HAL_TIMEx_HallSensor_Start_IT*](#)
- [*HAL_TIMEx_HallSensor_Stop_IT*](#)
- [*HAL_TIMEx_HallSensor_Start_DMA*](#)
- [*HAL_TIMEx_HallSensor_Stop_DMA*](#)

45.2.4 Timer Complementary Output Compare functions

This section provides functions allowing to:

- Start the Complementary Output Compare/PWM.
- Stop the Complementary Output Compare/PWM.
- Start the Complementary Output Compare/PWM and enable interrupts.
- Stop the Complementary Output Compare/PWM and disable interrupts.
- Start the Complementary Output Compare/PWM and enable DMA transfers.
- Stop the Complementary Output Compare/PWM and disable DMA transfers.

This section contains the following APIs:

- [*HAL_TIMEx_OC_N_Start*](#)
- [*HAL_TIMEx_OC_N_Stop*](#)

- [HAL_TIMEx_OCN_Start_IT](#)
- [HAL_TIMEx_OCN_Stop_IT](#)
- [HAL_TIMEx_OCN_Start_DMA](#)
- [HAL_TIMEx_OCN_Stop_DMA](#)

45.2.5 Timer Complementary PWM functions

This section provides functions allowing to:

- Start the Complementary PWM.
- Stop the Complementary PWM.
- Start the Complementary PWM and enable interrupts.
- Stop the Complementary PWM and disable interrupts.
- Start the Complementary PWM and enable DMA transfers.
- Stop the Complementary PWM and disable DMA transfers.
- Start the Complementary Input Capture measurement.
- Stop the Complementary Input Capture.
- Start the Complementary Input Capture and enable interrupts.
- Stop the Complementary Input Capture and disable interrupts.
- Start the Complementary Input Capture and enable DMA transfers.
- Stop the Complementary Input Capture and disable DMA transfers.
- Start the Complementary One Pulse generation.
- Stop the Complementary One Pulse.
- Start the Complementary One Pulse and enable interrupts.
- Stop the Complementary One Pulse and disable interrupts.

This section contains the following APIs:

- [HAL_TIMEx_PWMN_Start](#)
- [HAL_TIMEx_PWMN_Stop](#)
- [HAL_TIMEx_PWMN_Start_IT](#)
- [HAL_TIMEx_PWMN_Stop_IT](#)
- [HAL_TIMEx_PWMN_Start_DMA](#)
- [HAL_TIMEx_PWMN_Stop_DMA](#)

45.2.6 Timer Complementary One Pulse functions

This section provides functions allowing to:

- Start the Complementary One Pulse generation.
- Stop the Complementary One Pulse.
- Start the Complementary One Pulse and enable interrupts.
- Stop the Complementary One Pulse and disable interrupts.

This section contains the following APIs:

- [HAL_TIMEx_OnePulseN_Start](#)
- [HAL_TIMEx_OnePulseN_Stop](#)
- [HAL_TIMEx_OnePulseN_Start_IT](#)
- [HAL_TIMEx_OnePulseN_Stop_IT](#)

45.2.7 Peripheral Control functions

This section provides functions allowing to:

- Configure the commutation event in case of use of the Hall sensor interface.
- Configure Output channels for OC and PWM mode.
- Configure Complementary channels, break features and dead time.
- Configure Master synchronization.

- Configure timer remapping capabilities.

This section contains the following APIs:

- [HAL_TIMEx_ConfigCommutEvent](#)
- [HAL_TIMEx_ConfigCommutEvent_IT](#)
- [HAL_TIMEx_ConfigCommutEvent_DMA](#)
- [HAL_TIMEx_MasterConfigSynchronization](#)
- [HAL_TIMEx_ConfigBreakDeadTime](#)
- [HAL_TIMEx_RemapConfig](#)

45.2.8 Extended Callbacks functions

This section provides Extended TIM callback functions:

- Timer Commutation callback
- Timer Break callback

This section contains the following APIs:

- [HAL_TIMEx_CommutCallback](#)
- [HAL_TIMEx_CommutHalfCpltCallback](#)
- [HAL_TIMEx_BreakCallback](#)

45.2.9 Extended Peripheral State functions

This subsection permits to get in run-time the status of the peripheral and the data flow.

This section contains the following APIs:

- [HAL_TIMEx_HallSensor_GetState](#)

45.2.10 Detailed description of functions

HAL_TIMEx_HallSensor_Init

Function name

```
HAL_StatusTypeDef HAL_TIMEx_HallSensor_Init(TIM_HandleTypeDef * htim,
TIM_HallSensor_InitTypeDef * sConfig)
```

Function description

Initializes the TIM Hall Sensor Interface and initialize the associated handle.

Parameters

- **htim**: TIM Hall Sensor Interface handle
- **sConfig**: TIM Hall Sensor configuration structure

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_DeInit

Function name

```
HAL_StatusTypeDef HAL_TIMEx_HallSensor_DeInit(TIM_HandleTypeDef * htim)
```

Function description

Deinitializes the TIM Hall Sensor interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_MspInit

Function name

void HAL_TIMEx_HallSensor_MspInit (TIM_HandleTypeDef * htim)

Function description

Initializes the TIM Hall Sensor MSP.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **None**:

HAL_TIMEx_HallSensor_MspDeInit

Function name

void HAL_TIMEx_HallSensor_MspDeInit (TIM_HandleTypeDef * htim)

Function description

Deinitializes TIM Hall Sensor MSP.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **None**:

HAL_TIMEx_HallSensor_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Hall Sensor Interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall sensor Interface.

Parameters

- **htim**: TIM Hall Sensor Interface handle

Return values

- **HAL**: status

HAL_TIMEx_HallSensor_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start_IT (TIM_HandleTypeDef * htim)

Function description

Starts the TIM Hall Sensor Interface in interrupt mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop_IT (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall Sensor Interface in interrupt mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Start_DMA (TIM_HandleTypeDef * htim, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Hall Sensor Interface in DMA mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle
- **pData:** The destination Buffer address.
- **Length:** The length of data to be transferred from TIM peripheral to memory.

Return values

- **HAL:** status

HAL_TIMEx_HallSensor_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_HallSensor_Stop_DMA (TIM_HandleTypeDef * htim)

Function description

Stops the TIM Hall Sensor Interface in DMA mode.

Parameters

- **htim:** TIM Hall Sensor Interface handle

Return values

- **HAL:** status

HAL_TIMEx_OCN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the TIM Output Compare signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM OC handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OCN_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM Output Compare signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIMEx_OCN_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_OCN_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM Output Compare signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM Output Compare handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Starts the PWM signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the PWM signal generation in interrupt mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Start_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Start_DMA (TIM_HandleTypeDef * htim, uint32_t Channel, uint32_t * pData, uint16_t Length)

Function description

Starts the TIM PWM signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected
- **pData:** The source Buffer address.
- **Length:** The length of data to be transferred from memory to TIM peripheral

Return values

- **HAL:** status

HAL_TIMEx_PWMN_Stop_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_PWMN_Stop_DMA (TIM_HandleTypeDef * htim, uint32_t Channel)

Function description

Stops the TIM PWM signal generation in DMA mode on the complementary output.

Parameters

- **htim:** TIM handle
- **Channel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected
 - TIM_CHANNEL_3: TIM Channel 3 selected

Return values

- **HAL:** status

HAL_TIMEx_OnePulseN_Start

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Start (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation on the complementary output.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

HAL_TIMEx_OnePulseN_Stop

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Stop (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation on the complementary output.

Parameters

- **htim:** TIM One Pulse handle
- **OutputChannel:** TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL:** status

HAL_TIMEx_OnePulseN_Start_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Start_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Starts the TIM One Pulse signal generation in interrupt mode on the complementary channel.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channel to be enabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIMEx_OnePulseN_Stop_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_OnePulseN_Stop_IT (TIM_HandleTypeDef * htim, uint32_t OutputChannel)

Function description

Stops the TIM One Pulse signal generation in interrupt mode on the complementary channel.

Parameters

- **htim**: TIM One Pulse handle
- **OutputChannel**: TIM Channel to be disabled This parameter can be one of the following values:
 - TIM_CHANNEL_1: TIM Channel 1 selected
 - TIM_CHANNEL_2: TIM Channel 2 selected

Return values

- **HAL**: status

HAL_TIMEx_ConfigCommutEvent

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence.

Parameters

- **htim**: TIM handle
- **InputTrigger**: the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- **CommutationSource**: the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- **HAL**: status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the T11 of the Interface Timer detect a commutation at its input T11.

HAL_TIMEx_ConfigCommutEvent_IT

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent_IT (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence with interrupt.

Parameters

- htim**: TIM handle
- InputTrigger**: the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- CommutationSource**: the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- HAL**: status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the T11 of the Interface Timer detect a commutation at its input T11.

HAL_TIMEx_ConfigCommutEvent_DMA

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigCommutEvent_DMA (TIM_HandleTypeDef * htim, uint32_t InputTrigger, uint32_t CommutationSource)

Function description

Configure the TIM commutation event sequence with DMA.

Parameters

- **htim**: TIM handle
- **InputTrigger**: the Internal trigger corresponding to the Timer Interfacing with the Hall sensor This parameter can be one of the following values:
 - TIM_TS_ITR0: Internal trigger 0 selected
 - TIM_TS_ITR1: Internal trigger 1 selected
 - TIM_TS_ITR2: Internal trigger 2 selected
 - TIM_TS_ITR3: Internal trigger 3 selected
 - TIM_TS_NONE: No trigger is needed
- **CommutationSource**: the Commutation Event source This parameter can be one of the following values:
 - TIM_COMMUTATION_TRGI: Commutation source is the TRGI of the Interface Timer
 - TIM_COMMUTATION_SOFTWARE: Commutation source is set by software using the COMG bit

Return values

- **HAL**: status

Notes

- This function is mandatory to use the commutation event in order to update the configuration at each commutation detection on the TRGI input of the Timer, the typical use of this feature is with the use of another Timer(interface Timer) configured in Hall sensor interface, this interface Timer will generate the commutation at its TRGO output (connected to Timer used in this function) each time the T11 of the Interface Timer detect a commutation at its input TI1.
- The user should configure the DMA in his own software, in This function only the COMDE bit is set

HAL_TIMEx_MasterConfigSynchronization

Function name

HAL_StatusTypeDef HAL_TIMEx_MasterConfigSynchronization (TIM_HandleTypeDef * htim, TIM_MasterConfigTypeDef * sMasterConfig)

Function description

Configures the TIM in master mode.

Parameters

- **htim**: TIM handle.
- **sMasterConfig**: pointer to a TIM_MasterConfigTypeDef structure that contains the selected trigger output (TRGO) and the Master/Slave mode.

Return values

- **HAL**: status

HAL_TIMEx_ConfigBreakDeadTime

Function name

HAL_StatusTypeDef HAL_TIMEx_ConfigBreakDeadTime (TIM_HandleTypeDef * htim, TIM_BreakDeadTimeConfigTypeDef * sBreakDeadTimeConfig)

Function description

Configures the Break feature, dead time, Lock level, OSSR/OSSR State and the AOE(automatic output enable).

Parameters

- **htim**: TIM handle
- **sBreakDeadTimeConfig**: pointer to a TIM_ConfigBreakDeadConfigTypeDef structure that contains the BDTR Register configuration information for the TIM peripheral.

Return values

- **HAL:** status

Notes

- Interrupts can be generated when an active level is detected on the break input, the break 2 input or the system break input. Break interrupt can be enabled by calling the `__HAL_TIM_ENABLE_IT` macro.

HAL_TIMEx_RemapConfig

Function name

HAL_StatusTypeDef HAL_TIMEx_RemapConfig (TIM_HandleTypeDef * htim, uint32_t Remap)

Function description

Configures the TIMx Remapping input capabilities.

Parameters

- **htim:** TIM handle.
- **Remap:** specifies the TIM remapping source. For TIM2, the parameter can have the following values:
 - `TIM_TIM2_TIM8_TRGO`: TIM2 ITR1 is connected to TIM8 TRGO
 - `TIM_TIM2_ETH_PTP`: TIM2 ITR1 is connected to PTP trigger output
 - `TIM_TIM2_USBFS_SOF`: TIM2 ITR1 is connected to OTG FS SOF
 - `TIM_TIM2_USBHS_SOF`: TIM2 ITR1 is connected to OTG FS SOF
 For TIM5, the parameter can have the following values:
 - `TIM_TIM5_GPIO`: TIM5 TI4 is connected to GPIO
 - `TIM_TIM5_LSI`: TIM5 TI4 is connected to LSI
 - `TIM_TIM5_LSE`: TIM5 TI4 is connected to LSE
 - `TIM_TIM5_RTC`: TIM5 TI4 is connected to the RTC wakeup interrupt
 For TIM11, the parameter can have the following values:
 - `TIM_TIM11_GPIO`: TIM11 TI1 is connected to GPIO
 - `TIM_TIM11_HSE`: TIM11 TI1 is connected to HSE_RTC clock

Return values

- **HAL:** status

HAL_TIMEx_CommutCallback

Function name

void HAL_TIMEx_CommutCallback (TIM_HandleTypeDef * htim)

Function description

Hall commutation changed callback in non-blocking mode.

Parameters

- **htim:** TIM handle

Return values

- **None:**

HAL_TIMEx_CommutHalfCpltCallback

Function name

void HAL_TIMEx_CommutHalfCpltCallback (TIM_HandleTypeDef * htim)

Function description

Hall commutation changed half complete callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIMEx_BreakCallback

Function name

void HAL_TIMEx_BreakCallback (TIM_HandleTypeDef * htim)

Function description

Hall Break detection callback in non-blocking mode.

Parameters

- **htim**: TIM handle

Return values

- **None**:

HAL_TIMEx_HallSensor_GetState

Function name

HAL_TIM_StateTypeDef HAL_TIMEx_HallSensor_GetState (TIM_HandleTypeDef * htim)

Function description

Return the TIM Hall Sensor interface handle state.

Parameters

- **htim**: TIM Hall Sensor handle

Return values

- **HAL**: state

TIMEx_DMACommutationCplt

Function name

void TIMEx_DMACommutationCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Commutation callback.

Parameters

- **hdma**: pointer to DMA handle.

Return values

- **None**:

TIMEx_DMACommutationHalfCplt

Function name

void TIMEx_DMACommutationHalfCplt (DMA_HandleTypeDef * hdma)

Function description

TIM DMA Commutation half complete callback.

Parameters

- **hdma:** pointer to DMA handle.

Return values

- **None:**

45.3 TIMEx Firmware driver defines

The following section lists the various define and macros of the module.

45.3.1 TIMEx

TIMEx

TIM Extended Remapping

TIM_TIM2_TIM8_TRGO

TIM2 ITR1 is connected to TIM8 TRGO

TIM_TIM2_ETH_PTP

TIM2 ITR1 is connected to PTP trigger output

TIM_TIM2_USBFS_SOF

TIM2 ITR1 is connected to OTG FS SOF

TIM_TIM2_USBHS_SOF

TIM2 ITR1 is connected to OTG HS SOF

TIM_TIM5_GPIO

TIM5 TI4 is connected to GPIO

TIM_TIM5_LSI

TIM5 TI4 is connected to LSI

TIM_TIM5_LSE

TIM5 TI4 is connected to LSE

TIM_TIM5_RTC

TIM5 TI4 is connected to the RTC wakeup interrupt

TIM_TIM11_GPIO

TIM11 TI1 is connected to GPIO

TIM_TIM11_HSE

TIM11 TI1 is connected to HSE_RTC clock

46 HAL UART Generic Driver

46.1 UART Firmware driver registers structures

46.1.1 UART_InitTypeDef

UART_InitTypeDef is defined in the stm32f2xx_hal_uart.h

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t HwFlowCtl**
- **uint32_t OverSampling**

Field Documentation

- **uint32_t UART_InitTypeDef::BaudRate**
This member configures the UART communication baud rate. The baud rate is computed using the following formula:
 - IntegerDivider = ((PCLKx) / (8 * (OVR8+1) * (huart->Init.BaudRate)))
 - FractionalDivider = ((IntegerDivider - ((uint32_t) IntegerDivider)) * 8 * (OVR8+1)) + 0.5 Where OVR8 is the "oversampling by 8 mode" configuration bit in the CR1 register.
- **uint32_t UART_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [UART_Word_Length](#)
- **uint32_t UART_InitTypeDef::StopBits**
Specifies the number of stop bits transmitted. This parameter can be a value of [UART_Stop_Bits](#)
- **uint32_t UART_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [UART_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t UART_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [UART_Mode](#)
- **uint32_t UART_InitTypeDef::HwFlowCtl**
Specifies whether the hardware flow control mode is enabled or disabled. This parameter can be a value of [UART_Hardware_Flow_Control](#)
- **uint32_t UART_InitTypeDef::OverSampling**
Specifies whether the Over sampling 8 is enabled or disabled, to achieve higher speed (up to fPCLK/8). This parameter can be a value of [UART_Over_Sampling](#)

46.1.2 __UART_HandleTypeDef

__UART_HandleTypeDef is defined in the stm32f2xx_hal_uart.h

Data Fields

- **USART_TypeDef * Instance**
- **UART_InitTypeDef Init**
- **uint8_t * pTxBuffPtr**
- **uint16_t TxXferSize**

- `__IO uint16_t TxXferCount`
- `uint8_t * pRxBuffPtr`
- `uint16_t RxXferSize`
- `__IO uint16_t RxXferCount`
- `DMA_HandleTypeDef * hdmatrix`
- `DMA_HandleTypeDef * hdmatrix`
- `HAL_LockTypeDef Lock`
- `__IO HAL_UART_StateTypeDef gState`
- `__IO HAL_UART_StateTypeDef RxState`
- `__IO uint32_t ErrorCode`

Field Documentation

- **`USART_TypeDef* __UART_HandleTypeDef::Instance`**
UART registers base address
- **`UART_InitTypeDef __UART_HandleTypeDef::Init`**
UART communication parameters
- **`uint8_t* __UART_HandleTypeDef::pTxBuffPtr`**
Pointer to UART Tx transfer Buffer
- **`uint16_t __UART_HandleTypeDef::TxXferSize`**
UART Tx Transfer size
- **`__IO uint16_t __UART_HandleTypeDef::TxXferCount`**
UART Tx Transfer Counter
- **`uint8_t* __UART_HandleTypeDef::pRxBuffPtr`**
Pointer to UART Rx transfer Buffer
- **`uint16_t __UART_HandleTypeDef::RxXferSize`**
UART Rx Transfer size
- **`__IO uint16_t __UART_HandleTypeDef::RxXferCount`**
UART Rx Transfer Counter
- **`DMA_HandleTypeDef* __UART_HandleTypeDef::hdmatrix`**
UART Tx DMA Handle parameters
- **`DMA_HandleTypeDef* __UART_HandleTypeDef::hdmatrix`**
UART Rx DMA Handle parameters
- **`HAL_LockTypeDef __UART_HandleTypeDef::Lock`**
Locking object
- **`__IO HAL_UART_StateTypeDef __UART_HandleTypeDef::gState`**
UART state information related to global Handle management and also related to Tx operations. This parameter can be a value of **`HAL_UART_StateTypeDef`**
- **`__IO HAL_UART_StateTypeDef __UART_HandleTypeDef::RxState`**
UART state information related to Rx operations. This parameter can be a value of **`HAL_UART_StateTypeDef`**
- **`__IO uint32_t __UART_HandleTypeDef::ErrorCode`**
UART Error code

46.2 UART Firmware driver API description

The following section lists the various functions of the UART library.

46.2.1 How to use this driver

The UART HAL driver can be used as follows:

1. Declare a `UART_HandleTypeDef` handle structure (eg. `UART_HandleTypeDef huart`).

2. Initialize the UART low level resources by implementing the HAL_UART_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. UART pins configuration:
 - Enable the clock for the UART GPIOs.
 - Configure these UART pins (TX as alternate function pull-up, RX as alternate function Input).
 - c. NVIC configuration if you need to use interrupt process (HAL_UART_Transmit_IT() and HAL_UART_Receive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_UART_Transmit_DMA() and HAL_UART_Receive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the UART DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the huart Init structure.
4. For the UART asynchronous mode, initialize the UART registers by calling the HAL_UART_Init() API.
5. For the UART Half duplex mode, initialize the UART registers by calling the HAL_HalfDuplex_Init() API.
6. For the LIN mode, initialize the UART registers by calling the HAL_LIN_Init() API.
7. For the Multi-Processor mode, initialize the UART registers by calling the HAL_MultiProcessor_Init() API.

Note: *The specific UART interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_UART_ENABLE_IT() and __HAL_UART_DISABLE_IT() inside the transmit and receive process.*

Note: *These APIs (HAL_UART_Init() and HAL_HalfDuplex_Init()) configure also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_UART_MspInit() API.*

46.2.2 Callback registration

The compilation define USE_HAL_UART_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function @ref HAL_UART_RegisterCallback() to register a user callback. Function @ref HAL_UART_RegisterCallback() allows to register following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : UART MspInit.
- MspDeInitCallback : UART MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function @ref HAL_UART_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. @ref HAL_UART_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- AbortTransmitCpltCallback : Abort Transmit Complete Callback.
- AbortReceiveCpltCallback : Abort Receive Complete Callback.
- MspInitCallback : UART MspInit.
- MspDeInitCallback : UART MspDeInit.

By default, after the @ref HAL_UART_Init() and when the state is HAL_UART_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples @ref HAL_UART_TxCpltCallback(), @ref HAL_UART_RxHalfCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_UART_Init() and @ref HAL_UART_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_UART_Init() and @ref HAL_UART_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_UART_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_UART_STATE_READY or HAL_UART_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_UART_RegisterCallback() before calling @ref HAL_UART_DeInit() or @ref HAL_UART_Init() function.

When The compilation define USE_HAL_UART_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_UART_Transmit()
- Receive an amount of data in blocking mode using HAL_UART_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_UART_Transmit_IT()
- At transmission end of transfer HAL_UART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_UART_Receive_IT()
- At reception end of transfer HAL_UART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxCpltCallback
- In case of transfer Error, HAL_UART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_UART_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_UART_Transmit_DMA()
- At transmission end of half transfer HAL_UART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxHalfCpltCallback
- At transmission end of transfer HAL_UART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_UART_Receive_DMA()
- At reception end of half transfer HAL_UART_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxHalfCpltCallback
- At reception end of transfer HAL_UART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_UART_RxCpltCallback
- In case of transfer Error, HAL_UART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_UART_ErrorCallback

- Pause the DMA Transfer using HAL_UART_DMABase()
- Resume the DMA Transfer using HAL_UART_DMABaseResume()
- Stop the DMA Transfer using HAL_UART_DMABaseStop()

UART HAL driver macros list

Below the list of most used macros in UART HAL driver.

- `__HAL_UART_ENABLE`: Enable the UART peripheral
- `__HAL_UART_DISABLE`: Disable the UART peripheral
- `__HAL_UART_GET_FLAG`: Check whether the specified UART flag is set or not
- `__HAL_UART_CLEAR_FLAG`: Clear the specified UART pending flag
- `__HAL_UART_ENABLE_IT`: Enable the specified UART interrupt
- `__HAL_UART_DISABLE_IT`: Disable the specified UART interrupt
- `__HAL_UART_GET_IT_SOURCE`: Check whether the specified UART interrupt has occurred or not

Note: You can refer to the UART HAL driver header file for more useful macros

Note: Additionnal remark: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), the possible UART frame formats are as listed in the following table: +-----

+ M bit PCE bit UART frame ----- ----- 0 0 SB 8 bit data STB
----- ----- ----- 0 1 SB 7 bit data PB STB
----- ----- ----- 1 0 SB 9 bit data STB
----- ----- ----- 1 1 SB 8 bit data PB STB
+-----+

46.2.3 Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USARTx or the UARTy in asynchronous mode.

- For the asynchronous mode only these parameters can be configured:
 - Baud Rate
 - Word Length
 - Stop Bit
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible UART frame formats.
 - Hardware flow control
 - Receiver/transmitter modes
 - Over Sampling Method

The HAL_UART_Init(), HAL_HalfDuplex_Init(), HAL_LIN_Init() and HAL_MultiProcessor_Init() APIs follow respectively the UART asynchronous, UART Half duplex, LIN and Multi-Processor configuration procedures (details for the procedures are available in reference manual (RM0033)).

This section contains the following APIs:

- [HAL_UART_Init](#)
- [HAL_HalfDuplex_Init](#)
- [HAL_LIN_Init](#)
- [HAL_MultiProcessor_Init](#)
- [HAL_UART_DeInit](#)
- [HAL_UART_MspInit](#)
- [HAL_UART_MspDeInit](#)

46.2.4 IO operation functions

This section contains the following APIs:

- [HAL_UART_Transmit](#)
- [HAL_UART_Receive](#)

- [*HAL_UART_Transmit_IT*](#)
- [*HAL_UART_Receive_IT*](#)
- [*HAL_UART_Transmit_DMA*](#)
- [*HAL_UART_Receive_DMA*](#)
- [*HAL_UART_DMAPause*](#)
- [*HAL_UART_DMAResume*](#)
- [*HAL_UART_DMAStop*](#)
- [*HAL_UART_Abort*](#)
- [*HAL_UART_AbortTransmit*](#)
- [*HAL_UART_AbortReceive*](#)
- [*HAL_UART_Abort_IT*](#)
- [*HAL_UART_AbortTransmit_IT*](#)
- [*HAL_UART_AbortReceive_IT*](#)
- [*HAL_UART_IRQHandler*](#)
- [*HAL_UART_TxCpltCallback*](#)
- [*HAL_UART_TxHalfCpltCallback*](#)
- [*HAL_UART_RxCpltCallback*](#)
- [*HAL_UART_RxHalfCpltCallback*](#)
- [*HAL_UART_ErrorCallback*](#)
- [*HAL_UART_AbortCpltCallback*](#)
- [*HAL_UART_AbortTransmitCpltCallback*](#)
- [*HAL_UART_AbortReceiveCpltCallback*](#)

46.2.5

Peripheral Control functions

This subsection provides a set of functions allowing to control the UART:

- [*HAL_LIN_SendBreak\(\)*](#) API can be helpful to transmit the break character.
- [*HAL_MultiProcessor_EnterMuteMode\(\)*](#) API can be helpful to enter the UART in mute mode.
- [*HAL_MultiProcessor_ExitMuteMode\(\)*](#) API can be helpful to exit the UART mute mode by software.
- [*HAL_HalfDuplex_EnableTransmitter\(\)*](#) API to enable the UART transmitter and disables the UART receiver in Half Duplex mode
- [*HAL_HalfDuplex_EnableReceiver\(\)*](#) API to enable the UART receiver and disables the UART transmitter in Half Duplex mode

This section contains the following APIs:

- [*HAL_LIN_SendBreak*](#)
- [*HAL_MultiProcessor_EnterMuteMode*](#)
- [*HAL_MultiProcessor_ExitMuteMode*](#)
- [*HAL_HalfDuplex_EnableTransmitter*](#)
- [*HAL_HalfDuplex_EnableReceiver*](#)

46.2.6

Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of UART communication process, return Peripheral Errors occurred during communication process

- [*HAL_UART_GetState\(\)*](#) API can be helpful to check in run-time the state of the UART peripheral.
- [*HAL_UART_GetError\(\)*](#) check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [*HAL_UART_GetState*](#)
- [*HAL_UART_GetError*](#)

46.2.7 Detailed description of functions

HAL_UART_Init

Function name

HAL_StatusTypeDef HAL_UART_Init (UART_HandleTypeDef * huart)

Function description

Initializes the UART mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_Init

Function name

HAL_StatusTypeDef HAL_HalfDuplex_Init (UART_HandleTypeDef * huart)

Function description

Initializes the half-duplex mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_LIN_Init

Function name

HAL_StatusTypeDef HAL_LIN_Init (UART_HandleTypeDef * huart, uint32_t BreakDetectLength)

Function description

Initializes the LIN mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **BreakDetectLength:** Specifies the LIN break detection length. This parameter can be one of the following values:
 - **UART_LINBREAKDETECTLENGTH_10B:** 10-bit break detection
 - **UART_LINBREAKDETECTLENGTH_11B:** 11-bit break detection

Return values

- **HAL:** status

HAL_MultiProcessor_Init

Function name

HAL_StatusTypeDef HAL_MultiProcessor_Init (UART_HandleTypeDef * huart, uint8_t Address, uint32_t WakeUpMethod)

Function description

Initializes the Multi-Processor mode according to the specified parameters in the UART_InitTypeDef and create the associated handle.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **Address:** USART address
- **WakeUpMethod:** specifies the USART wake-up method. This parameter can be one of the following values:
 - UART_WAKEUPMETHOD_IDLELINE: Wake-up by an idle line detection
 - UART_WAKEUPMETHOD_ADDRESSMARK: Wake-up by an address mark

Return values

- **HAL:** status

HAL_UART_DeInit

Function name

HAL_StatusTypeDef HAL_UART_DeInit (UART_HandleTypeDef * huart)

Function description

Deinitializes the UART peripheral.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_MspInit

Function name

void HAL_UART_MspInit (UART_HandleTypeDef * huart)

Function description

UART MSP Init.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_MspDeInit

Function name

void HAL_UART_MspDeInit (UART_HandleTypeDef * huart)

Function description

UART MSP DeInit.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_Transmit

Function name

HAL_StatusTypeDef HAL_UART_Transmit (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Sends an amount of data in blocking mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive

Function name

HAL_StatusTypeDef HAL_UART_Receive (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Receives an amount of data in blocking mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.
- **Timeout:** Timeout duration

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.

HAL_UART_Transmit_IT

Function name

HAL_StatusTypeDef HAL_UART_Transmit_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Sends an amount of data in non blocking mode.

Parameters

- huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- pData:** Pointer to data buffer (u8 or u16 data elements).
- Size:** Amount of data elements (u8 or u16) to be sent

Return values

- HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive_IT

Function name

HAL_StatusTypeDef HAL_UART_Receive_IT (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in non blocking mode.

Parameters

- huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- pData:** Pointer to data buffer (u8 or u16 data elements).
- Size:** Amount of data elements (u8 or u16) to be received.

Return values

- HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.

HAL_UART_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_UART_Transmit_DMA (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Sends an amount of data in DMA mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pData.

HAL_UART_Receive_DMA

Function name

HAL_StatusTypeDef HAL_UART_Receive_DMA (UART_HandleTypeDef * huart, uint8_t * pData, uint16_t Size)

Function description

Receives an amount of data in DMA mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.
- **pData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pData.
- When the UART parity is enabled (PCE = 1) the received data contains the parity bit.

HAL_UART_DMABase

Function name

HAL_StatusTypeDef HAL_UART_DMABase (UART_HandleTypeDef * huart)

Function description

Pauses the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_DMAResume

Function name

HAL_StatusTypeDef HAL_UART_DMAResume (UART_HandleTypeDef * huart)

Function description

Resumes the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_DMAStop

Function name

HAL_StatusTypeDef HAL_UART_DMAStop (UART_HandleTypeDef * huart)

Function description

Stops the DMA Transfer.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_Abort

Function name

HAL_StatusTypeDef HAL_UART_Abort (UART_HandleTypeDef * huart)

Function description

Abort ongoing transfers (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx and Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_AbortTransmit

Function name

HAL_StatusTypeDef HAL_UART_AbortTransmit (UART_HandleTypeDef * huart)

Function description

Abort ongoing Transmit transfer (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Tx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_AbortReceive

Function name

HAL_StatusTypeDef HAL_UART_AbortReceive (UART_HandleTypeDef * huart)

Function description

Abort ongoing Receive transfer (blocking mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Rx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_UART_Abort_IT

Function name

HAL_StatusTypeDef HAL_UART_Abort_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing transfers (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx and Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_AbortTransmit_IT

Function name

HAL_StatusTypeDef HAL_UART_AbortTransmit_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing Transmit transfer (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Tx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Tx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_AbortReceive_IT

Function name

HAL_StatusTypeDef HAL_UART_AbortReceive_IT (UART_HandleTypeDef * huart)

Function description

Abort ongoing Receive transfer (Interrupt mode).

Parameters

- **huart:** UART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing Rx transfer started in Interrupt or DMA mode. This procedure performs following operations : Disable UART Interrupts (Rx)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_UART_IRQHandler

Function name

void HAL_UART_IRQHandler (UART_HandleTypeDef * huart)

Function description

This function handles UART interrupt request.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_TxCpltCallback

Function name

void HAL_UART_TxCpltCallback (UART_HandleTypeDef * huart)

Function description

Tx Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_TxHalfCpltCallback

Function name

void HAL_UART_TxHalfCpltCallback (UART_HandleTypeDef * huart)

Function description

Tx Half Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_RxCpltCallback

Function name

void HAL_UART_RxCpltCallback (UART_HandleTypeDef * huart)

Function description

Rx Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_RxHalfCpltCallback

Function name

```
void HAL_UART_RxHalfCpltCallback (UART_HandleTypeDef * huart)
```

Function description

Rx Half Transfer completed callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_ErrorCallback

Function name

```
void HAL_UART_ErrorCallback (UART_HandleTypeDef * huart)
```

Function description

UART error callbacks.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **None:**

HAL_UART_AbortCpltCallback

Function name

```
void HAL_UART_AbortCpltCallback (UART_HandleTypeDef * huart)
```

Function description

UART Abort Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_UART_AbortTransmitCpltCallback

Function name

```
void HAL_UART_AbortTransmitCpltCallback (UART_HandleTypeDef * huart)
```

Function description

UART Abort Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_UART_AbortReceiveCpltCallback

Function name

void HAL_UART_AbortReceiveCpltCallback (UART_HandleTypeDef * huart)

Function description

UART Abort Receive Complete callback.

Parameters

- **huart:** UART handle.

Return values

- **None:**

HAL_LIN_SendBreak

Function name

HAL_StatusTypeDef HAL_LIN_SendBreak (UART_HandleTypeDef * huart)

Function description

Transmits break characters.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_MultiProcessor_EnterMuteMode

Function name

HAL_StatusTypeDef HAL_MultiProcessor_EnterMuteMode (UART_HandleTypeDef * huart)

Function description

Enters the UART in mute mode.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_MultiProcessor_ExitMuteMode

Function name

HAL_StatusTypeDef HAL_MultiProcessor_ExitMuteMode (UART_HandleTypeDef * huart)

Function description

Exits the UART mute mode: wake up software.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_EnableTransmitter

Function name

HAL_StatusTypeDef HAL_HalfDuplex_EnableTransmitter (UART_HandleTypeDef * huart)

Function description

Enables the UART transmitter and disables the UART receiver.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_HalfDuplex_EnableReceiver

Function name

HAL_StatusTypeDef HAL_HalfDuplex_EnableReceiver (UART_HandleTypeDef * huart)

Function description

Enables the UART receiver and disables the UART transmitter.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** status

HAL_UART_GetState

Function name

HAL_UART_StateTypeDef HAL_UART_GetState (UART_HandleTypeDef * huart)

Function description

Returns the UART state.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART module.

Return values

- **HAL:** state

HAL_UART_GetError

Function name

uint32_t HAL_UART_GetError (UART_HandleTypeDef * huart)

Function description

Return the UART error code.

Parameters

- **huart:** Pointer to a UART_HandleTypeDef structure that contains the configuration information for the specified UART.

Return values

- **UART:** Error Code

46.3 UART Firmware driver defines

The following section lists the various define and macros of the module.

46.3.1 UART

UART

UART Error Code

HAL_UART_ERROR_NONE

No error

HAL_UART_ERROR_PE

Parity error

HAL_UART_ERROR_NE

Noise error

HAL_UART_ERROR_FE

Frame error

HAL_UART_ERROR_ORE

Overrun error

HAL_UART_ERROR_DMA

DMA transfer error

UART Exported Macros

__HAL_UART_RESET_HANDLE_STATE

Description:

- Reset UART handle gstate & RxState.

Parameters:

- **__HANDLE__:** specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_FLUSH_DRREGISTER

Description:

- Flushes the UART DR register.

Parameters:

- **__HANDLE__:** specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

__HAL_UART_GET_FLAG

Description:

- Checks whether the specified UART flag is set or not.

Parameters:

- __HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- __FLAG__**: specifies the flag to check. This parameter can be one of the following values:
 - UART_FLAG_CTS: CTS Change flag (not available for UART4 and UART5)
 - UART_FLAG_LBD: LIN Break detection flag
 - UART_FLAG_TXE: Transmit data register empty flag
 - UART_FLAG_TC: Transmission Complete flag
 - UART_FLAG_RXNE: Receive data register not empty flag
 - UART_FLAG_IDLE: Idle Line detection flag
 - UART_FLAG_ORE: Overrun Error flag
 - UART_FLAG_NE: Noise Error flag
 - UART_FLAG_FE: Framing Error flag
 - UART_FLAG_PE: Parity Error flag

Return value:

- The new state of **__FLAG__** (TRUE or FALSE).

__HAL_UART_CLEAR_FLAG

Description:

- Clears the specified UART pending flag.

Parameters:

- __HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- __FLAG__**: specifies the flag to check. This parameter can be any combination of the following values:
 - UART_FLAG_CTS: CTS Change flag (not available for UART4 and UART5).
 - UART_FLAG_LBD: LIN Break detection flag.
 - UART_FLAG_TC: Transmission Complete flag.
 - UART_FLAG_RXNE: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (Overrun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to USART_SR register followed by a read operation to USART_DR register. RXNE flag can be also cleared by a read to the USART_DR register. TC flag can be also cleared by software sequence: a read operation to USART_SR register followed by a write operation to USART_DR register. TXE flag is cleared only by a write to the USART_DR register.

__HAL_UART_CLEAR_PFLAG

Description:

- Clears the UART PE pending flag.

Parameters:

- __HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_FEFLAG

Description:

- Clears the UART FE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_NEFLAG

Description:

- Clears the UART NE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_OREFLAG

Description:

- Clears the UART ORE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_CLEAR_IDLEFLAG

Description:

- Clears the UART IDLE pending flag.

Parameters:

- `__HANDLE__`: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART,UART availability and x,y values depending on device).

Return value:

- None

__HAL_UART_ENABLE_IT

Description:

- Enable the specified UART interrupt.

Parameters:

- **__HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- **__INTERRUPT__**: specifies the UART interrupt source to enable. This parameter can be one of the following values:
 - **UART_IT_CTS**: CTS change interrupt
 - **UART_IT_LBD**: LIN Break detection interrupt
 - **UART_IT_TXE**: Transmit Data Register empty interrupt
 - **UART_IT_TC**: Transmission complete interrupt
 - **UART_IT_RXNE**: Receive Data register not empty interrupt
 - **UART_IT_IDLE**: Idle line detection interrupt
 - **UART_IT_PE**: Parity Error interrupt
 - **UART_IT_ERR**: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_UART_DISABLE_IT

Description:

- Disable the specified UART interrupt.

Parameters:

- **__HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- **__INTERRUPT__**: specifies the UART interrupt source to disable. This parameter can be one of the following values:
 - **UART_IT_CTS**: CTS change interrupt
 - **UART_IT_LBD**: LIN Break detection interrupt
 - **UART_IT_TXE**: Transmit Data Register empty interrupt
 - **UART_IT_TC**: Transmission complete interrupt
 - **UART_IT_RXNE**: Receive Data register not empty interrupt
 - **UART_IT_IDLE**: Idle line detection interrupt
 - **UART_IT_PE**: Parity Error interrupt
 - **UART_IT_ERR**: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_UART_GET_IT_SOURCE

Description:

- Checks whether the specified UART interrupt has occurred or not.

Parameters:

- **__HANDLE__**: specifies the UART Handle. UART Handle selects the USARTx or UARTy peripheral (USART, UART availability and x,y values depending on device).
- **__IT__**: specifies the UART interrupt source to check. This parameter can be one of the following values:
 - **UART_IT_CTS**: CTS change interrupt (not available for UART4 and UART5)
 - **UART_IT_LBD**: LIN Break detection interrupt
 - **UART_IT_TXE**: Transmit Data Register empty interrupt
 - **UART_IT_TC**: Transmission complete interrupt
 - **UART_IT_RXNE**: Receive Data register not empty interrupt
 - **UART_IT_IDLE**: Idle line detection interrupt
 - **UART_IT_ERR**: Error interrupt

Return value:

- The: new state of **__IT__** (TRUE or FALSE).

__HAL_UART_HWCONTROL_CTS_ENABLE

Description:

- Enable CTS flow control.

Parameters:

- **__HANDLE__**: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- This macro allows to enable CTS hardware flow control for a given UART instance, without need to call **HAL_UART_Init()** function. As involving direct access to UART registers, usage of this macro should be fully endorsed by user. As macro is expected to be used for modifying CTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : UART instance should have already been initialised (through call of **HAL_UART_Init()**)macro could only be called when corresponding UART instance is disabled (i.e **__HAL_UART_DISABLE(__HANDLE__)**) and should be followed by an Enable macro (i.e **__HAL_UART_ENABLE(__HANDLE__)**).

__HAL_UART_HWCONTROL_CTS_DISABLE

Description:

- Disable CTS flow control.

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- This macro allows to disable CTS hardware flow control for a given UART instance, without need to call `HAL_UART_Init()` function. As involving direct access to UART registers, usage of this macro should be fully endorsed by user. As macro is expected to be used for modifying CTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : USART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_HWCONTROL_RTS_ENABLE

Description:

- Enable RTS flow control This macro allows to enable RTS hardware flow control for a given UART instance, without need to call

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- As macro is expected to be used for modifying RTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : USART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_HWCONTROL_RTS_DISABLE

Description:

- Disable RTS flow control This macro allows to disable RTS hardware flow control for a given UART instance, without need to call

Parameters:

- `__HANDLE__`: specifies the UART Handle. The Handle Instance can be any USARTx (supporting the HW Flow control feature). It is used to select the USART peripheral (USART availability and x value depending on device).

Return value:

- None

Notes:

- As macro is expected to be used for modifying RTS Hw flow control feature activation, without need for USART instance Deinit/Init, following conditions for macro call should be fulfilled : USART instance should have already been initialised (through call of `HAL_UART_Init()`) macro could only be called when corresponding UART instance is disabled (i.e `__HAL_UART_DISABLE(__HANDLE__)`) and should be followed by an Enable macro (i.e `__HAL_UART_ENABLE(__HANDLE__)`).

__HAL_UART_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the UART's one bit sample method.

Parameters:

- __HANDLE__: specifies the UART Handle.

Return value:

- None

__HAL_UART_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the UART's one bit sample method.

Parameters:

- __HANDLE__: specifies the UART Handle.

Return value:

- None

__HAL_UART_ENABLE

Description:

- Enable UART.

Parameters:

- __HANDLE__: specifies the UART Handle.

Return value:

- None

__HAL_UART_DISABLE

Description:

- Disable UART.

Parameters:

- __HANDLE__: specifies the UART Handle.

Return value:

- None

UART Flags

UART_FLAG_CTS

UART_FLAG_LBD

UART_FLAG_TXE

UART_FLAG_TC

UART_FLAG_RXNE

UART_FLAG_IDLE

UART_FLAG_ORE

UART_FLAG_NE

UART_FLAG_FE

UART_FLAG_PE

UART Hardware Flow Control

UART_HWCONTROL_NONE

UART_HWCONTROL_RTS

UART_HWCONTROL_CTS

UART_HWCONTROL_RTS_CTS

UART Interrupt Definitions

UART_IT_PE

UART_IT_TXE

UART_IT_TC

UART_IT_RXNE

UART_IT_IDLE

UART_IT_LBD

UART_IT_CTS

UART_IT_ERR

UART LIN Break Detection Length

UART_LINBREAKDETECTLENGTH_10B

UART_LINBREAKDETECTLENGTH_11B

UART Transfer Mode

UART_MODE_RX

UART_MODE_TX

UART_MODE_TX_RX

UART Over Sampling

UART_OVERSAMPLING_16

UART_OVERSAMPLING_8

UART Parity

UART_PARITY_NONE

UART_PARITY_EVEN

UART_PARITY_ODD

UART State

UART_STATE_DISABLE

UART_STATE_ENABLE

UART Number of Stop Bits

UART_STOPBITS_1

UART_STOPBITS_2

UART Wakeup Functions

UART_WAKEUPMETHOD_IDLELINE

UART_WAKEUPMETHOD_ADDRESSMARK

UART Word Length

UART_WORDLENGTH_8B

UART_WORDLENGTH_9B

47 HAL USART Generic Driver

47.1 USART Firmware driver registers structures

47.1.1 USART_InitTypeDef

USART_InitTypeDef is defined in the stm32f2xx_hal_usart.h

Data Fields

- **uint32_t BaudRate**
- **uint32_t WordLength**
- **uint32_t StopBits**
- **uint32_t Parity**
- **uint32_t Mode**
- **uint32_t CLKPolarity**
- **uint32_t CLKPhase**
- **uint32_t CLKLastBit**

Field Documentation

- **uint32_t USART_InitTypeDef::BaudRate**
This member configures the Usart communication baud rate. The baud rate is computed using the following formula:
 - IntegerDivider = ((PCLKx) / (8 * (husart->Init.BaudRate)))
 - FractionalDivider = ((IntegerDivider - ((uint32_t) IntegerDivider)) * 8) + 0.5
- **uint32_t USART_InitTypeDef::WordLength**
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of [USART_Word_Length](#)
- **uint32_t USART_InitTypeDef::StopBits**
Specifies the number of stop bits transmitted. This parameter can be a value of [USART_Stop_Bits](#)
- **uint32_t USART_InitTypeDef::Parity**
Specifies the parity mode. This parameter can be a value of [USART_Parity](#)
Note:
 - When parity is enabled, the computed parity is inserted at the MSB position of the transmitted data (9th bit when the word length is set to 9 data bits; 8th bit when the word length is set to 8 data bits).
- **uint32_t USART_InitTypeDef::Mode**
Specifies whether the Receive or Transmit mode is enabled or disabled. This parameter can be a value of [USART_Mode](#)
- **uint32_t USART_InitTypeDef::CLKPolarity**
Specifies the steady state of the serial clock. This parameter can be a value of [USART_Clock_Polarity](#)
- **uint32_t USART_InitTypeDef::CLKPhase**
Specifies the clock transition on which the bit capture is made. This parameter can be a value of [USART_Clock_Phase](#)
- **uint32_t USART_InitTypeDef::CLKLastBit**
Specifies whether the clock pulse corresponding to the last transmitted data bit (MSB) has to be output on the SCLK pin in synchronous mode. This parameter can be a value of [USART_Last_Bit](#)

47.1.2 __USART_HandleTypeDef

__USART_HandleTypeDef is defined in the stm32f2xx_hal_usart.h

Data Fields

- **USART_TypeDef * Instance**
- **USART_InitTypeDef Init**

- `uint8_t *pTxBuffPtr`
- `uint16_t TxXferSize`
- `__IO uint16_t TxXferCount`
- `uint8_t *pRxBuffPtr`
- `uint16_t RxXferSize`
- `__IO uint16_t RxXferCount`
- `DMA_HandleTypeDef *hdmatx`
- `DMA_HandleTypeDef *hdmarx`
- `HAL_LockTypeDef Lock`
- `__IO HAL_USART_StateTypeDef State`
- `__IO uint32_t ErrorCode`

Field Documentation

- **`USART_TypeDef* __USART_HandleTypeDef::Instance`**
USART registers base address
- **`USART_InitTypeDef __USART_HandleTypeDef::Init`**
Usart communication parameters
- **`uint8_t* __USART_HandleTypeDef::pTxBuffPtr`**
Pointer to Usart Tx transfer Buffer
- **`uint16_t __USART_HandleTypeDef::TxXferSize`**
Usart Tx Transfer size
- **`__IO uint16_t __USART_HandleTypeDef::TxXferCount`**
Usart Tx Transfer Counter
- **`uint8_t* __USART_HandleTypeDef::pRxBuffPtr`**
Pointer to Usart Rx transfer Buffer
- **`uint16_t __USART_HandleTypeDef::RxXferSize`**
Usart Rx Transfer size
- **`__IO uint16_t __USART_HandleTypeDef::RxXferCount`**
Usart Rx Transfer Counter
- **`DMA_HandleTypeDef* __USART_HandleTypeDef::hdmatx`**
Usart Tx DMA Handle parameters
- **`DMA_HandleTypeDef* __USART_HandleTypeDef::hdmarx`**
Usart Rx DMA Handle parameters
- **`HAL_LockTypeDef __USART_HandleTypeDef::Lock`**
Locking object
- **`__IO HAL_USART_StateTypeDef __USART_HandleTypeDef::State`**
Usart communication state
- **`__IO uint32_t __USART_HandleTypeDef::ErrorCode`**
USART Error code

47.2 USART Firmware driver API description

The following section lists the various functions of the USART library.

47.2.1 How to use this driver

The USART HAL driver can be used as follows:

1. Declare a `USART_HandleTypeDef` handle structure (eg. `USART_HandleTypeDef husart`).

2. Initialize the USART low level resources by implementing the HAL_USART_MspInit() API:
 - a. Enable the USARTx interface clock.
 - b. USART pins configuration:
 - Enable the clock for the USART GPIOs.
 - Configure the USART pins as alternate function pull-up.
 - c. NVIC configuration if you need to use interrupt process (HAL_USART_Transmit_IT(), HAL_USART_Receive_IT() and HAL_USART_TransmitReceive_IT() APIs):
 - Configure the USARTx interrupt priority.
 - Enable the NVIC USART IRQ handle.
 - d. DMA Configuration if you need to use DMA process (HAL_USART_Transmit_DMA(), HAL_USART_Receive_DMA() and HAL_USART_TransmitReceive_DMA() APIs):
 - Declare a DMA handle structure for the Tx/Rx stream.
 - Enable the DMAx interface clock.
 - Configure the declared DMA handle structure with the required Tx/Rx parameters.
 - Configure the DMA Tx/Rx stream.
 - Associate the initialized DMA handle to the USART DMA Tx/Rx handle.
 - Configure the priority and enable the NVIC for the transfer complete interrupt on the DMA Tx/Rx stream.
 - Configure the USARTx interrupt priority and enable the NVIC USART IRQ handle (used for last byte sending completion detection in DMA non circular mode)
3. Program the Baud Rate, Word Length, Stop Bit, Parity, Hardware flow control and Mode(Receiver/Transmitter) in the husart Init structure.
4. Initialize the USART registers by calling the HAL_USART_Init() API:
 - These APIs configures also the low level Hardware GPIO, CLOCK, CORTEX...etc) by calling the customized HAL_USART_MspInit(&husart) API.

Note: *The specific USART interrupts (Transmission complete interrupt, RXNE interrupt and Error Interrupts) will be managed using the macros __HAL_USART_ENABLE_IT() and __HAL_USART_DISABLE_IT() inside the transmit and receive process.*

5. Three operation modes are available within this driver :

Polling mode IO operation

- Send an amount of data in blocking mode using HAL_USART_Transmit()
- Receive an amount of data in blocking mode using HAL_USART_Receive()

Interrupt mode IO operation

- Send an amount of data in non blocking mode using HAL_USART_Transmit_IT()
- At transmission end of transfer HAL_USART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxCpltCallback
- Receive an amount of data in non blocking mode using HAL_USART_Receive_IT()
- At reception end of transfer HAL_USART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxCpltCallback
- In case of transfer Error, HAL_USART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_USART_ErrorCallback

DMA mode IO operation

- Send an amount of data in non blocking mode (DMA) using HAL_USART_Transmit_DMA()
- At transmission end of half transfer HAL_USART_TxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxHalfCpltCallback
- At transmission end of transfer HAL_USART_TxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_TxCpltCallback
- Receive an amount of data in non blocking mode (DMA) using HAL_USART_Receive_DMA()

- At reception end of half transfer HAL_USART_RxHalfCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxHalfCpltCallback
- At reception end of transfer HAL_USART_RxCpltCallback is executed and user can add his own code by customization of function pointer HAL_USART_RxCpltCallback
- In case of transfer Error, HAL_USART_ErrorCallback() function is executed and user can add his own code by customization of function pointer HAL_USART_ErrorCallback
- Pause the DMA Transfer using HAL_USART_DMABasePause()
- Resume the DMA Transfer using HAL_USART_DMABaseResume()
- Stop the DMA Transfer using HAL_USART_DMABaseStop()

USART HAL driver macros list

Below the list of most used macros in USART HAL driver.

- __HAL_USART_ENABLE: Enable the USART peripheral
- __HAL_USART_DISABLE: Disable the USART peripheral
- __HAL_USART_GET_FLAG : Check whether the specified USART flag is set or not
- __HAL_USART_CLEAR_FLAG : Clear the specified USART pending flag
- __HAL_USART_ENABLE_IT: Enable the specified USART interrupt
- __HAL_USART_DISABLE_IT: Disable the specified USART interrupt

Note: You can refer to the USART HAL driver header file for more useful macros

47.2.2 Callback registration

The compilation define USE_HAL_USART_REGISTER_CALLBACKS when set to 1 allows the user to configure dynamically the driver callbacks.

Use Function @ref HAL_USART_RegisterCallback() to register a user callback. Function @ref HAL_USART_RegisterCallback() allows to register following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- TxRxCpltCallback : Tx Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- MspInitCallback : USART MspInit.
- MspDeInitCallback : USART MspDeInit. This function takes as parameters the HAL peripheral handle, the Callback ID and a pointer to the user callback function.

Use function @ref HAL_USART_UnRegisterCallback() to reset a callback to the default weak (surcharged) function. @ref HAL_USART_UnRegisterCallback() takes as parameters the HAL peripheral handle, and the Callback ID. This function allows to reset following callbacks:

- TxHalfCpltCallback : Tx Half Complete Callback.
- TxCpltCallback : Tx Complete Callback.
- RxHalfCpltCallback : Rx Half Complete Callback.
- RxCpltCallback : Rx Complete Callback.
- TxRxCpltCallback : Tx Rx Complete Callback.
- ErrorCallback : Error Callback.
- AbortCpltCallback : Abort Complete Callback.
- MspInitCallback : USART MspInit.
- MspDeInitCallback : USART MspDeInit.

By default, after the @ref HAL_USART_Init() and when the state is HAL_USART_STATE_RESET all callbacks are set to the corresponding weak (surcharged) functions: examples @ref HAL_USART_TxCpltCallback(), @ref HAL_USART_RxHalfCpltCallback(). Exception done for MspInit and MspDeInit functions that are respectively reset to the legacy weak (surcharged) functions in the @ref HAL_USART_Init() and @ref HAL_USART_DeInit() only when these callbacks are null (not registered beforehand). If not, MspInit or MspDeInit are not null, the @ref HAL_USART_Init() and @ref HAL_USART_DeInit() keep and use the user MspInit/MspDeInit callbacks (registered beforehand).

Callbacks can be registered/unregistered in HAL_USART_STATE_READY state only. Exception done MspInit/ MspDeInit that can be registered/unregistered in HAL_USART_STATE_READY or HAL_USART_STATE_RESET state, thus registered (user) MspInit/DeInit callbacks can be used during the Init/DeInit. In that case first register the MspInit/MspDeInit user callbacks using @ref HAL_USART_RegisterCallback() before calling @ref HAL_USART_DeInit() or @ref HAL_USART_Init() function.

When The compilation define USE_HAL_USART_REGISTER_CALLBACKS is set to 0 or not defined, the callback registration feature is not available and weak (surcharged) callbacks are used.

Note:

Additional remark: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), the possible USART frame formats are as listed in the following table:

+-----+ M bit PCE bit USART frame			
-----	0 0	SB 8 bit data	STB
-----	0 1	SB 7 bit data	PB STB
-----	1 0	SB 9 bit data	STB
-----	1 1	SB 8 bit data	PB STB
+-----+			

47.2.3

Initialization and Configuration functions

This subsection provides a set of functions allowing to initialize the USART in asynchronous and in synchronous modes.

- For the asynchronous mode only these parameters can be configured:
 - Baud Rate
 - Word Length
 - Stop Bit
 - Parity: If the parity is enabled, then the MSB bit of the data written in the data register is transmitted but is changed by the parity bit. Depending on the frame length defined by the M bit (8-bits or 9-bits), please refer to Reference manual for possible USART frame formats.
 - USART polarity
 - USART phase
 - USART LastBit
 - Receiver/transmitter modes

The HAL_USART_Init() function follows the USART synchronous configuration procedures (details for the procedures are available in reference manual (RM0033)).

This section contains the following APIs:

- [HAL_USART_Init](#)
- [HAL_USART_DeInit](#)
- [HAL_USART_MspInit](#)
- [HAL_USART_MspDeInit](#)

47.2.4

IO operation functions

This subsection provides a set of functions allowing to manage the USART synchronous data transfers.

The USART supports master mode only: it cannot receive or send data related to an input clock (SCLK is always an output).

1. There are two modes of transfer:
 - Blocking mode: The communication is performed in polling mode. The HAL status of all data processing is returned by the same function after finishing transfer.
 - No-Blocking mode: The communication is performed using Interrupts or DMA. These APIs return the HAL status. The end of the data processing will be indicated through the dedicated USART IRQ when using Interrupt mode or the DMA IRQ when using DMA mode. The HAL_USART_TxCpltCallback(), HAL_USART_RxCpltCallback() and HAL_USART_TxRxCpltCallback() user callbacks will be executed respectively at the end of the transmit or Receive process. The HAL_USART_ErrorCallback() user callback will be executed when a communication error is detected.
2. Blocking mode APIs are :
 - HAL_USART_Transmit() in simplex mode
 - HAL_USART_Receive() in full duplex receive only
 - HAL_USART_TransmitReceive() in full duplex mode
3. Non Blocking mode APIs with Interrupt are :
 - HAL_USART_Transmit_IT() in simplex mode
 - HAL_USART_Receive_IT() in full duplex receive only
 - HAL_USART_TransmitReceive_IT() in full duplex mode
 - HAL_USART_IRQHandler()
4. Non Blocking mode functions with DMA are :
 - HAL_USART_Transmit_DMA() in simplex mode
 - HAL_USART_Receive_DMA() in full duplex receive only
 - HAL_USART_TransmitReceive_DMA() in full duplex mode
 - HAL_USART_DMAPause()
 - HAL_USART_DMAResume()
 - HAL_USART_DMAStop()
5. A set of Transfer Complete Callbacks are provided in non Blocking mode:
 - HAL_USART_TxHalfCpltCallback()
 - HAL_USART_TxCpltCallback()
 - HAL_USART_RxHalfCpltCallback()
 - HAL_USART_RxCpltCallback()
 - HAL_USART_ErrorCallback()
 - HAL_USART_TxRxCpltCallback()
6. Non-Blocking mode transfers could be aborted using Abort API's :
 - HAL_USART_Abort()
 - HAL_USART_Abort_IT()
7. For Abort services based on interrupts (HAL_USART_Abort_IT), a Abort Complete Callbacks is provided:
 - HAL_USART_AbortCpltCallback()
8. In Non-Blocking mode transfers, possible errors are split into 2 categories. Errors are handled as follows :
 - Error is considered as Recoverable and non blocking : Transfer could go till end, but error severity is to be evaluated by user : this concerns Frame Error, Parity Error or Noise Error in Interrupt mode reception . Received character is then retrieved and stored in Rx buffer, Error code is set to allow user to identify error type, and HAL_USART_ErrorCallback() user callback is executed. Transfer is kept ongoing on USART side. If user wants to abort it, Abort services should be called by user.
 - Error is considered as Blocking : Transfer could not be completed properly and is aborted. This concerns Overrun Error In Interrupt mode reception and all errors in DMA mode. Error code is set to allow user to identify error type, and HAL_USART_ErrorCallback() user callback is executed.

This section contains the following APIs:

- [HAL_USART_Transmit](#)
- [HAL_USART_Receive](#)
- [HAL_USART_TransmitReceive](#)
- [HAL_USART_Transmit_IT](#)

- [HAL_USART_Receive_IT](#)
- [HAL_USART_TransmitReceive_IT](#)
- [HAL_USART_Transmit_DMA](#)
- [HAL_USART_Receive_DMA](#)
- [HAL_USART_TransmitReceive_DMA](#)
- [HAL_USART_DMAPause](#)
- [HAL_USART_DMAResume](#)
- [HAL_USART_DMAStop](#)
- [HAL_USART_Abort](#)
- [HAL_USART_Abort_IT](#)
- [HAL_USART_IRQHandler](#)
- [HAL_USART_TxCpltCallback](#)
- [HAL_USART_TxHalfCpltCallback](#)
- [HAL_USART_RxCpltCallback](#)
- [HAL_USART_RxHalfCpltCallback](#)
- [HAL_USART_TxRxCpltCallback](#)
- [HAL_USART_ErrorCallback](#)
- [HAL_USART_AbortCpltCallback](#)

47.2.5 Peripheral State and Errors functions

This subsection provides a set of functions allowing to return the State of USART communication process, return Peripheral Errors occurred during communication process

- [HAL_USART_GetState\(\)](#) API can be helpful to check in run-time the state of the USART peripheral.
- [HAL_USART_GetError\(\)](#) check in run-time errors that could be occurred during communication.

This section contains the following APIs:

- [HAL_USART_GetState](#)
- [HAL_USART_GetError](#)

47.2.6 Detailed description of functions

HAL_USART_Init

Function name

HAL_StatusTypeDef HAL_USART_Init (USART_HandleTypeDef * husart)

Function description

Initialize the USART mode according to the specified parameters in the USART_InitTypeDef and initialize the associated handle.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL**: status

HAL_USART_DeInit

Function name

HAL_StatusTypeDef HAL_USART_DeInit (USART_HandleTypeDef * husart)

Function description

DeInitializes the USART peripheral.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_MspInit

Function name

void HAL_USART_MspInit (USART_HandleTypeDef * husart)

Function description

USART MSP Init.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_MspDeInit

Function name

void HAL_USART_MspDeInit (USART_HandleTypeDef * husart)

Function description

USART MSP DeInit.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_Transmit

Function name

HAL_StatusTypeDef HAL_USART_Transmit (USART_HandleTypeDef * husart, uint8_t * pTxData, uint16_t Size, uint32_t Timeout)

Function description

Simplex Send an amount of data in blocking mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent.
- **Timeout:** Timeout duration.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.

HAL_USART_Receive

Function name

HAL_StatusTypeDef HAL_USART_Receive (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Full-Duplex Receive an amount of data in blocking mode.

Parameters

- husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- pRxData**: Pointer to data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be received.
- Timeout**: Timeout duration.

Return values

- HAL**: status

Notes

- To receive synchronous data, dummy data are simultaneously transmitted.
- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.

HAL_USART_TransmitReceive

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive (USART_HandleTypeDef * husart, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size, uint32_t Timeout)

Function description

Full-Duplex Send and Receive an amount of data in full-duplex mode (blocking mode).

Parameters

- husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- pTxData**: Pointer to TX data buffer (u8 or u16 data elements).
- pRxData**: Pointer to RX data buffer (u8 or u16 data elements).
- Size**: Amount of data elements (u8 or u16) to be sent (same amount to be received).
- Timeout**: Timeout duration

Return values

- HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.

HAL_USART_Transmit_IT

Function name

HAL_StatusTypeDef HAL_USART_Transmit_IT (USART_HandleTypeDef * husart, uint8_t * pTxData, uint16_t Size)

Function description

Simplex Send an amount of data in non-blocking mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.
- The USART errors are not managed to avoid the overrun error.

HAL_USART_Receive_IT

Function name

HAL_StatusTypeDef HAL_USART_Receive_IT (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size)

Function description

Simplex Receive an amount of data in non-blocking mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pRxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- To receive synchronous data, dummy data are simultaneously transmitted.
- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.

HAL_USART_TransmitReceive_IT

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive_IT (USART_HandleTypeDef * husart, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Send and Receive an amount of data in full-duplex mode (non-blocking).

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData:** Pointer to TX data buffer (u8 or u16 data elements).
- **pRxData:** Pointer to RX data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent (same amount to be received).

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.

HAL_USART_Transmit_DMA

Function name

HAL_StatusTypeDef HAL_USART_Transmit_DMA (USART_HandleTypeDef * husart, uint8_t * pTxData, uint16_t Size)

Function description

Simplex Send an amount of data in DMA mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be sent.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data is handled as a set of u16. In this case, Size must indicate the number of u16 provided through pTxData.

HAL_USART_Receive_DMA

Function name

HAL_StatusTypeDef HAL_USART_Receive_DMA (USART_HandleTypeDef * husart, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Receive an amount of data in DMA mode.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pRxData:** Pointer to data buffer (u8 or u16 data elements).
- **Size:** Amount of data elements (u8 or u16) to be received.

Return values

- **HAL:** status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the received data is handled as a set of u16. In this case, Size must indicate the number of u16 available through pRxData.
- The USART DMA transmit stream must be configured in order to generate the clock for the slave.
- When the USART parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_USART_TransmitReceive_DMA

Function name

HAL_StatusTypeDef HAL_USART_TransmitReceive_DMA (USART_HandleTypeDef * husart, uint8_t * pTxData, uint8_t * pRxData, uint16_t Size)

Function description

Full-Duplex Transmit Receive an amount of data in DMA mode.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.
- **pTxData**: Pointer to TX data buffer (u8 or u16 data elements).
- **pRxData**: Pointer to RX data buffer (u8 or u16 data elements).
- **Size**: Amount of data elements (u8 or u16) to be received/sent.

Return values

- **HAL**: status

Notes

- When UART parity is not enabled (PCE = 0), and Word Length is configured to 9 bits (M1-M0 = 01), the sent data and the received data are handled as sets of u16. In this case, Size must indicate the number of u16 available through pTxData and through pRxData.
- When the USART parity is enabled (PCE = 1) the data received contain the parity bit.

HAL_USART_DMAPause

Function name

HAL_StatusTypeDef HAL_USART_DMAPause (USART_HandleTypeDef * husart)

Function description

Pauses the DMA Transfer.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL**: status

HAL_USART_DMAResume

Function name

HAL_StatusTypeDef HAL_USART_DMAResume (USART_HandleTypeDef * husart)

Function description

Resumes the DMA Transfer.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_DMASStop

Function name

HAL_StatusTypeDef HAL_USART_DMASStop (USART_HandleTypeDef * husart)

Function description

Stops the DMA Transfer.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** status

HAL_USART_Abort

Function name

HAL_StatusTypeDef HAL_USART_Abort (USART_HandleTypeDef * husart)

Function description

Abort ongoing transfer (blocking mode).

Parameters

- **husart:** USART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer (either Tx or Rx, as described by TransferType parameter) started in Interrupt or DMA mode. This procedure performs following operations :
Disable PPP Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort (in case of transfer in DMA mode)Set handle State to READY
- This procedure is executed in blocking mode : when exiting function, Abort is considered as completed.

HAL_USART_Abort_IT

Function name

HAL_StatusTypeDef HAL_USART_Abort_IT (USART_HandleTypeDef * husart)

Function description

Abort ongoing transfer (Interrupt mode).

Parameters

- **husart:** USART handle.

Return values

- **HAL:** status

Notes

- This procedure could be used for aborting any ongoing transfer (either Tx or Rx, as described by TransferType parameter) started in Interrupt or DMA mode. This procedure performs following operations :
Disable PPP Interrupts (depending of transfer direction)Disable the DMA transfer in the peripheral register (if enabled)Abort DMA transfer by calling HAL_DMA_Abort_IT (in case of transfer in DMA mode)Set handle State to READYAt abort completion, call user abort complete callback
- This procedure is executed in Interrupt mode, meaning that abort procedure could be considered as completed only when user abort complete callback is executed (not when exiting function).

HAL_USART_IRQHandler

Function name

```
void HAL_USART_IRQHandler (USART_HandleTypeDef * husart)
```

Function description

This function handles USART interrupt request.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None**:

HAL_USART_TxCpltCallback

Function name

```
void HAL_USART_TxCpltCallback (USART_HandleTypeDef * husart)
```

Function description

Tx Transfer completed callbacks.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None**:

HAL_USART_TxHalfCpltCallback

Function name

```
void HAL_USART_TxHalfCpltCallback (USART_HandleTypeDef * husart)
```

Function description

Tx Half Transfer completed callbacks.

Parameters

- **husart**: Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None**:

HAL_USART_RxCpltCallback

Function name

void HAL_USART_RxCpltCallback (USART_HandleTypeDef * husart)

Function description

Rx Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_RxHalfCpltCallback

Function name

void HAL_USART_RxHalfCpltCallback (USART_HandleTypeDef * husart)

Function description

Rx Half Transfer completed callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_TxRxCpltCallback

Function name

void HAL_USART_TxRxCpltCallback (USART_HandleTypeDef * husart)

Function description

Tx/Rx Transfers completed callback for the non-blocking process.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_ErrorCallback

Function name

void HAL_USART_ErrorCallback (USART_HandleTypeDef * husart)

Function description

USART error callbacks.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **None:**

HAL_USART_AbortCpltCallback

Function name

void HAL_USART_AbortCpltCallback (USART_HandleTypeDef * husart)

Function description

USART Abort Complete callback.

Parameters

- **husart:** USART handle.

Return values

- **None:**

HAL_USART_GetState

Function name

HAL_USART_StateTypeDef HAL_USART_GetState (USART_HandleTypeDef * husart)

Function description

Returns the USART state.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART module.

Return values

- **HAL:** state

HAL_USART_GetError

Function name

uint32_t HAL_USART_GetError (USART_HandleTypeDef * husart)

Function description

Return the USART error code.

Parameters

- **husart:** Pointer to a USART_HandleTypeDef structure that contains the configuration information for the specified USART.

Return values

- **USART:** Error Code

47.3 USART Firmware driver defines

The following section lists the various define and macros of the module.

47.3.1 USART

USART

USART Clock

USART_CLOCK_DISABLE

USART_CLOCK_ENABLE

USART Clock Phase

USART_PHASE_1EDGE

USART_PHASE_2EDGE

USART Clock Polarity

USART_POLARITY_LOW

USART_POLARITY_HIGH

USART Error Code

HAL_USART_ERROR_NONE

No error

HAL_USART_ERROR_PE

Parity error

HAL_USART_ERROR_NE

Noise error

HAL_USART_ERROR_FE

Frame error

HAL_USART_ERROR_ORE

Overrun error

HAL_USART_ERROR_DMA

DMA transfer error

USART Exported Macros

__HAL_USART_RESET_HANDLE_STATE

Description:

- Reset USART handle state.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_GET_FLAG

Description:

- Check whether the specified USART flag is set or not.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `USART_FLAG_TXE`: Transmit data register empty flag
 - `USART_FLAG_TC`: Transmission Complete flag
 - `USART_FLAG_RXNE`: Receive data register not empty flag
 - `USART_FLAG_IDLE`: Idle Line detection flag
 - `USART_FLAG_ORE`: Overrun Error flag
 - `USART_FLAG_NE`: Noise Error flag
 - `USART_FLAG_FE`: Framing Error flag
 - `USART_FLAG_PE`: Parity Error flag

Return value:

- The: new state of `__FLAG__` (TRUE or FALSE).

__HAL_USART_CLEAR_FLAG

Description:

- Clear the specified USART pending flags.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__FLAG__`: specifies the flag to check. This parameter can be any combination of the following values:
 - `USART_FLAG_TC`: Transmission Complete flag.
 - `USART_FLAG_RXNE`: Receive data register not empty flag.

Return value:

- None

Notes:

- PE (Parity error), FE (Framing error), NE (Noise error), ORE (Overrun error) and IDLE (Idle line detected) flags are cleared by software sequence: a read operation to `USART_SR` register followed by a read operation to `USART_DR` register. RXNE flag can be also cleared by a read to the `USART_DR` register. TC flag can be also cleared by software sequence: a read operation to `USART_SR` register followed by a write operation to `USART_DR` register. TXE flag is cleared only by a write to the `USART_DR` register.

__HAL_USART_CLEAR_PFLAG

Description:

- Clear the USART PE pending flag.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_FEFLAG

Description:

- Clear the USART FE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_NEFLAG

Description:

- Clear the USART NE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_OREFLAG

Description:

- Clear the USART ORE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_CLEAR_IDLEFLAG

Description:

- Clear the USART IDLE pending flag.

Parameters:

- **__HANDLE__**: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_ENABLE_IT

Description:

- Enables or disables the specified USART interrupts.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__INTERRUPT__`: specifies the USART interrupt source to check. This parameter can be one of the following values:
 - `USART_IT_TXE`: Transmit Data Register empty interrupt
 - `USART_IT_TC`: Transmission complete interrupt
 - `USART_IT_RXNE`: Receive Data register not empty interrupt
 - `USART_IT_IDLE`: Idle line detection interrupt
 - `USART_IT_PE`: Parity Error interrupt
 - `USART_IT_ERR`: Error interrupt(Frame error, noise error, overrun error)

Return value:

- None

__HAL_USART_DISABLE_IT

__HAL_USART_GET_IT_SOURCE

Description:

- Checks whether the specified USART interrupt has occurred or not.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).
- `__IT__`: specifies the USART interrupt source to check. This parameter can be one of the following values:
 - `USART_IT_TXE`: Transmit Data Register empty interrupt
 - `USART_IT_TC`: Transmission complete interrupt
 - `USART_IT_RXNE`: Receive Data register not empty interrupt
 - `USART_IT_IDLE`: Idle line detection interrupt
 - `USART_IT_ERR`: Error interrupt
 - `USART_IT_PE`: Parity Error interrupt

Return value:

- The: new state of `__IT__` (TRUE or FALSE).

__HAL_USART_ONE_BIT_SAMPLE_ENABLE

Description:

- Macro to enable the USART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the USART Handle.

Return value:

- None

__HAL_USART_ONE_BIT_SAMPLE_DISABLE

Description:

- Macro to disable the USART's one bit sample method.

Parameters:

- `__HANDLE__`: specifies the USART Handle.

Return value:

- None

__HAL_USART_ENABLE

Description:

- Enable USART.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

__HAL_USART_DISABLE

Description:

- Disable USART.

Parameters:

- `__HANDLE__`: specifies the USART Handle. USART Handle selects the USARTx peripheral (USART availability and x value depending on device).

Return value:

- None

USART Flags

USART_FLAG_TXE

USART_FLAG_TC

USART_FLAG_RXNE

USART_FLAG_IDLE

USART_FLAG_ORE

USART_FLAG_NE

USART_FLAG_FE

USART_FLAG_PE

USART Interrupts Definition

USART_IT_PE

USART_IT_TXE

USART_IT_TC

USART_IT_RXNE

USART_IT_IDLE

USART_IT_ERR

USART Last Bit

USART_LASTBIT_DISABLE

USART_LASTBIT_ENABLE

USART Mode

USART_MODE_RX

USART_MODE_TX

USART_MODE_TX_RX

USART NACK State

USART_NACK_ENABLE

USART_NACK_DISABLE

USART Parity

USART_PARITY_NONE

USART_PARITY_EVEN

USART_PARITY_ODD

USART Number of Stop Bits

USART_STOPBITS_1

USART_STOPBITS_0_5

USART_STOPBITS_2

USART_STOPBITS_1_5

USART Word Length

USART_WORDLENGTH_8B

USART_WORDLENGTH_9B

48 HAL WWDG Generic Driver

48.1 WWDG Firmware driver registers structures

48.1.1 WWDG_InitTypeDef

WWDG_InitTypeDef is defined in the stm32f2xx_hal_wwdg.h

Data Fields

- **uint32_t Prescaler**
- **uint32_t Window**
- **uint32_t Counter**
- **uint32_t EWIMode**

Field Documentation

- **uint32_t WWDG_InitTypeDef::Prescaler**
Specifies the prescaler value of the WWDG. This parameter can be a value of [WWDG_Prescaler](#)
- **uint32_t WWDG_InitTypeDef::Window**
Specifies the WWDG window value to be compared to the downcounter. This parameter must be a number Min_Data = 0x40 and Max_Data = 0x7F
- **uint32_t WWDG_InitTypeDef::Counter**
Specifies the WWDG free-running downcounter value. This parameter must be a number between Min_Data = 0x40 and Max_Data = 0x7F
- **uint32_t WWDG_InitTypeDef::EWIMode**
Specifies if WWDG Early Wakeup Interrupt is enable or not. This parameter can be a value of [WWDG_EWI_Mode](#)

48.1.2 WWDG_HandleTypeDef

WWDG_HandleTypeDef is defined in the stm32f2xx_hal_wwdg.h

Data Fields

- **WWDG_TypeDef * Instance**
- **WWDG_InitTypeDef Init**

Field Documentation

- **WWDG_TypeDef* WWDG_HandleTypeDef::Instance**
Register base address
- **WWDG_InitTypeDef WWDG_HandleTypeDef::Init**
WWDG required parameters

48.2 WWDG Firmware driver API description

The following section lists the various functions of the WWDG library.

48.2.1 Initialization and Configuration functions

This section provides functions allowing to:

- Initialize and start the WWDG according to the specified parameters in the WWDG_InitTypeDef of associated handle.
- Initialize the WWDG MSP.

This section contains the following APIs:

- [HAL_WWDG_Init](#)
- [HAL_WWDG_MspInit](#)

48.2.2 IO operation functions

This section provides functions allowing to:

- Refresh the WWDG.
- Handle WWDG interrupt request and associated function callback.

This section contains the following APIs:

- [HAL_WWDG_Refresh](#)
- [HAL_WWDG_IRQHandler](#)
- [HAL_WWDG_EarlyWakeupCallback](#)

48.2.3 Detailed description of functions

HAL_WWDG_Init

Function name

HAL_StatusTypeDef HAL_WWDG_Init (WWDG_HandleTypeDef * hwwdg)

Function description

Initialize the WWDG according to the specified.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **HAL**: status

HAL_WWDG_MspInit

Function name

void HAL_WWDG_MspInit (WWDG_HandleTypeDef * hwwdg)

Function description

Initialize the WWDG MSP.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None**:

Notes

- When rewriting this function in user file, mechanism may be added to avoid multiple initialize when HAL_WWDG_Init function is called again to change parameters.

HAL_WWDG_Refresh

Function name

HAL_StatusTypeDef HAL_WWDG_Refresh (WWDG_HandleTypeDef * hwwdg)

Function description

Refresh the WWDG.

Parameters

- **hwwdg**: pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **HAL:** status

HAL_WWDG_IRQHandler

Function name

void HAL_WWDG_IRQHandler (WWDG_HandleTypeDef * hwwdg)

Function description

Handle WWDG interrupt request.

Parameters

- **hwwdg:** pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None:**

Notes

- The Early Wakeup Interrupt (EWI) can be used if specific safety operations or data logging must be performed before the actual reset is generated. The EWI interrupt is enabled by calling HAL_WWDG_Init function with EWIMode set to WWDG_EWI_ENABLE. When the downcounter reaches the value 0x40, and EWI interrupt is generated and the corresponding Interrupt Service Routine (ISR) can be used to trigger specific actions (such as communications or data logging), before resetting the device.

HAL_WWDG_EarlyWakeupCallback

Function name

void HAL_WWDG_EarlyWakeupCallback (WWDG_HandleTypeDef * hwwdg)

Function description

WWDG Early Wakeup callback.

Parameters

- **hwwdg:** pointer to a WWDG_HandleTypeDef structure that contains the configuration information for the specified WWDG module.

Return values

- **None:**

48.3 WWDG Firmware driver defines

The following section lists the various define and macros of the module.

48.3.1 WWDG

WWDG

WWDG Early Wakeup Interrupt Mode

WWDG_EWI_DISABLE

EWI Disable

WWDG_EWI_ENABLE

EWI Enable

WWDG Exported Macros

__HAL_WWDG_ENABLE

Description:

- Enable the WWDG peripheral.

Parameters:

- `__HANDLE__`: WWDG handle

Return value:

- None

__HAL_WWDG_ENABLE_IT

Description:

- Enable the WWDG early wakeup interrupt.

Parameters:

- `__HANDLE__`: WWDG handle
- `__INTERRUPT__`: specifies the interrupt to enable. This parameter can be one of the following values:
 - `WWDG_IT_EWI`: Early wakeup interrupt

Return value:

- None

Notes:

- Once enabled this interrupt cannot be disabled except by a system reset.

__HAL_WWDG_GET_IT

Description:

- Check whether the selected WWDG interrupt has occurred or not.

Parameters:

- `__HANDLE__`: WWDG handle
- `__INTERRUPT__`: specifies the it to check. This parameter can be one of the following values:
 - `WWDG_FLAG_EWIF`: Early wakeup interrupt IT

Return value:

- The: new state of `WWDG_FLAG` (SET or RESET).

__HAL_WWDG_CLEAR_IT

Description:

- Clear the WWDG interrupt pending bits.

Parameters:

- `__HANDLE__`: WWDG handle
- `__INTERRUPT__`: specifies the interrupt pending bit to clear. This parameter can be one of the following values:
 - `WWDG_FLAG_EWIF`: Early wakeup interrupt flag

__HAL_WWDG_GET_FLAG

Description:

- Check whether the specified WWDG flag is set or not.

Parameters:

- `__HANDLE__`: WWDG handle
- `__FLAG__`: specifies the flag to check. This parameter can be one of the following values:
 - `WWDG_FLAG_EWIF`: Early wakeup interrupt flag

Return value:

- The: new state of `WWDG_FLAG` (SET or RESET).

__HAL_WWDG_CLEAR_FLAG

Description:

- Clear the WWDG's pending flags.

Parameters:

- __HANDLE__: WWDG handle
- __FLAG__: specifies the flag to clear. This parameter can be one of the following values:
 - WWDG_FLAG_EWIF: Early wakeup interrupt flag

Return value:

- None

__HAL_WWDG_GET_IT_SOURCE

Description:

- Check whether the specified WWDG interrupt source is enabled or not.

Parameters:

- __HANDLE__: WWDG Handle.
- __INTERRUPT__: specifies the WWDG interrupt source to check. This parameter can be one of the following values:
 - WWDG_IT_EWI: Early Wakeup Interrupt

Return value:

- state: of __INTERRUPT__ (TRUE or FALSE).

WWDG Flag definition

WWDG_FLAG_EWIF

Early wakeup interrupt flag

WWDG Interrupt definition

WWDG_IT_EWI

Early wakeup interrupt

WWDG Prescaler

WWDG_PRESCALER_1

WWDG counter clock = (PCLK1/4096)/1

WWDG_PRESCALER_2

WWDG counter clock = (PCLK1/4096)/2

WWDG_PRESCALER_4

WWDG counter clock = (PCLK1/4096)/4

WWDG_PRESCALER_8

WWDG counter clock = (PCLK1/4096)/8

49 LL ADC Generic Driver

49.1 ADC Firmware driver registers structures

49.1.1 LL_ADC_CommonInitTypeDef

LL_ADC_CommonInitTypeDef is defined in the stm32f2xx_ll_adc.h

Data Fields

- **uint32_t CommonClock**
- **uint32_t Multimode**
- **uint32_t MultiDMATransfer**
- **uint32_t MultiTwoSamplingDelay**

Field Documentation

- **uint32_t LL_ADC_CommonInitTypeDef::CommonClock**
Set parameter common to several ADC: Clock source and prescaler. This parameter can be a value of [ADC_LL_EC_COMMON_CLOCK_SOURCE](#). This feature can be modified afterwards using unitary function [LL_ADC_SetCommonClock\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::Multimode**
Set ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances). This parameter can be a value of [ADC_LL_EC_MULTI_MODE](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultimode\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::MultiDMATransfer**
Set ADC multimode conversion data transfer: no transfer or transfer by DMA. This parameter can be a value of [ADC_LL_EC_MULTI_DMA_TRANSFER](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultiDMATransfer\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::MultiTwoSamplingDelay**
Set ADC multimode delay between 2 sampling phases. This parameter can be a value of [ADC_LL_EC_MULTI_TWOSMP_DELAY](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultiTwoSamplingDelay\(\)](#).

49.1.2 LL_ADC_InitTypeDef

LL_ADC_InitTypeDef is defined in the stm32f2xx_ll_adc.h

Data Fields

- **uint32_t Resolution**
- **uint32_t DataAlignment**
- **uint32_t SequencersScanMode**

Field Documentation

- **uint32_t LL_ADC_InitTypeDef::Resolution**
Set ADC resolution. This parameter can be a value of [ADC_LL_EC_RESOLUTION](#). This feature can be modified afterwards using unitary function [LL_ADC_SetResolution\(\)](#).
- **uint32_t LL_ADC_InitTypeDef::DataAlignment**
Set ADC conversion data alignment. This parameter can be a value of [ADC_LL_EC_DATA_ALIGN](#). This feature can be modified afterwards using unitary function [LL_ADC_SetDataAlignment\(\)](#).
- **uint32_t LL_ADC_InitTypeDef::SequencersScanMode**
Set ADC scan selection. This parameter can be a value of [ADC_LL_EC_SCAN_SELECTION](#). This feature can be modified afterwards using unitary function [LL_ADC_SetSequencersScanMode\(\)](#).

49.1.3 LL_ADC_REG_InitTypeDef

LL_ADC_REG_InitTypeDef is defined in the stm32f2xx_ll_adc.h

Data Fields

- **uint32_t TriggerSource**

- `uint32_t SequencerLength`
- `uint32_t SequencerDiscont`
- `uint32_t ContinuousMode`
- `uint32_t DMATransfer`

Field Documentation

- `uint32_t LL_ADC_REG_InitTypeDef::TriggerSource`

Set ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line). This parameter can be a value of [ADC_LL_EC_REG_TRIGGER_SOURCE](#)

Note:

- On this STM32 serie, setting of external trigger edge is performed using function `LL_ADC_REG_StartConversionExtTrig()`.

This feature can be modified afterwards using unitary function `LL_ADC_REG_SetTriggerSource()`.

- `uint32_t LL_ADC_REG_InitTypeDef::SequencerLength`

Set ADC group regular sequencer length. This parameter can be a value of [ADC_LL_EC_REG_SEQ_SCAN_LENGTH](#)

Note:

- This parameter is discarded if scan mode is disabled (refer to parameter 'ADC_SequencersScanMode').

This feature can be modified afterwards using unitary function `LL_ADC_REG_SetSequencerLength()`.

- `uint32_t LL_ADC_REG_InitTypeDef::SequencerDiscont`

Set ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks. This parameter can be a value of [ADC_LL_EC_REG_SEQ_DISCONT_MODE](#)

Note:

- This parameter has an effect only if group regular sequencer is enabled (scan length of 2 ranks or more).

This feature can be modified afterwards using unitary function `LL_ADC_REG_SetSequencerDiscont()`.

- `uint32_t LL_ADC_REG_InitTypeDef::ContinuousMode`

Set ADC continuous conversion mode on ADC group regular, whether ADC conversions are performed in single mode (one conversion per trigger) or in continuous mode (after the first trigger, following conversions launched successively automatically). This parameter can be a value of [ADC_LL_EC_REG_CONTINUOUS_MODE](#) Note: It is not possible to enable both ADC group regular continuous mode and discontinuous mode. This feature can be modified afterwards using unitary function `LL_ADC_REG_SetContinuousMode()`.

- `uint32_t LL_ADC_REG_InitTypeDef::DMATransfer`

Set ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode. This parameter can be a value of [ADC_LL_EC_REG_DMA_TRANSFER](#) This feature can be modified afterwards using unitary function `LL_ADC_REG_SetDMATransfer()`.

49.1.4

LL_ADC_INJ_InitTypeDef

`LL_ADC_INJ_InitTypeDef` is defined in the `stm32f2xx_ll_adc.h`

Data Fields

- `uint32_t TriggerSource`
- `uint32_t SequencerLength`
- `uint32_t SequencerDiscont`
- `uint32_t TrigAuto`

Field Documentation

- **`uint32_t LL_ADC_INJ_InitTypeDef::TriggerSource`**
Set ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line). This parameter can be a value of [ADC_LL_EC_INJ_TRIGGER_SOURCE](#)
Note:
 - On this STM32 serie, setting of external trigger edge is performed using function `LL_ADC_INJ_StartConversionExtTrig()`.
This feature can be modified afterwards using unitary function `LL_ADC_INJ_SetTriggerSource()`.
- **`uint32_t LL_ADC_INJ_InitTypeDef::SequencerLength`**
Set ADC group injected sequencer length. This parameter can be a value of [ADC_LL_EC_INJ_SEQ_SCAN_LENGTH](#)
Note:
 - This parameter is discarded if scan mode is disabled (refer to parameter 'ADC_SequencersScanMode').
This feature can be modified afterwards using unitary function `LL_ADC_INJ_SetSequencerLength()`.
- **`uint32_t LL_ADC_INJ_InitTypeDef::SequencerDiscont`**
Set ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks. This parameter can be a value of [ADC_LL_EC_INJ_SEQ_DISCONT_MODE](#)
Note:
 - This parameter has an effect only if group injected sequencer is enabled (scan length of 2 ranks or more).
This feature can be modified afterwards using unitary function `LL_ADC_INJ_SetSequencerDiscont()`.
- **`uint32_t LL_ADC_INJ_InitTypeDef::TrigAuto`**
Set ADC group injected conversion trigger: independent or from ADC group regular. This parameter can be a value of [ADC_LL_EC_INJ_TRIG_AUTO](#) **Note:** This parameter must be set to independent trigger if injected trigger source is set to an external trigger. This feature can be modified afterwards using unitary function `LL_ADC_INJ_SetTrigAuto()`.

49.2 ADC Firmware driver API description

The following section lists the various functions of the ADC library.

49.2.1 Detailed description of functions

`LL_ADC_DMA_GetRegAddr`

Function name

`__STATIC_INLINE uint32_t LL_ADC_DMA_GetRegAddr (ADC_TypeDef * ADCx, uint32_t Register)`

Function description

Function to help to configure DMA transfer from ADC: retrieve the ADC register address from ADC instance and a list of ADC registers intended to be used (most commonly) with DMA transfer.

Parameters

- **ADCx:** ADC instance
- **Register:** This parameter can be one of the following values:
 - `LL_ADC_DMA_REG_REGULAR_DATA`
 - `LL_ADC_DMA_REG_REGULAR_DATA_MULTI` (1)
(1) Available on devices with several ADC instances.

Return values

- **ADC:** register address

Notes

- These ADC registers are data registers: when ADC conversion data is available in ADC data registers, ADC generates a DMA transfer request.
- This macro is intended to be used with LL DMA driver, refer to function "LL_DMA_ConfigAddresses()". Example: LL_DMA_ConfigAddresses(DMA1, LL_DMA_CHANNEL_1, LL_ADC_DMA_GetRegAddr(ADC1, LL_ADC_DMA_REG_REGULAR_DATA), (uint32_t)< array or variable >, LL_DMA_DIRECTION_PERIPH_TO_MEMORY);
- For devices with several ADC: in multimode, some devices use a different data register outside of ADC instance scope (common data register). This macro manages this register difference, only ADC instance has to be set as parameter.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_DMA_GetRegAddr
- CDR RDATA_MST LL_ADC_DMA_GetRegAddr
- CDR RDATA_SLV LL_ADC_DMA_GetRegAddr

LL_ADC_SetCommonClock

Function name

__STATIC_INLINE void LL_ADC_SetCommonClock (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t CommonClock)

Function description

Set parameter common to several ADC: Clock source and prescaler.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **CommonClock:** This parameter can be one of the following values:
 - LL_ADC_CLOCK_SYNC_PCLK_DIV2
 - LL_ADC_CLOCK_SYNC_PCLK_DIV4
 - LL_ADC_CLOCK_SYNC_PCLK_DIV6
 - LL_ADC_CLOCK_SYNC_PCLK_DIV8

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCR ADCPRE LL_ADC_SetCommonClock

LL_ADC_GetCommonClock

Function name

__STATIC_INLINE uint32_t LL_ADC_GetCommonClock (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get parameter common to several ADC: Clock source and prescaler.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CLOCK_SYNC_PCLK_DIV2
 - LL_ADC_CLOCK_SYNC_PCLK_DIV4
 - LL_ADC_CLOCK_SYNC_PCLK_DIV6
 - LL_ADC_CLOCK_SYNC_PCLK_DIV8

Reference Manual to LL API cross reference:

- CCR ADCPRE LL_ADC_GetCommonClock

LL_ADC_SetCommonPathInternalCh

Function name

**__STATIC_INLINE void LL_ADC_SetCommonPathInternalCh (ADC_Common_TypeDef *
ADCxy_COMMON, uint32_t PathInternal)**

Function description

Set parameter common to several ADC: measurement path to internal channels (VrefInt, temperature sensor, ...).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **PathInternal:** This parameter can be a combination of the following values:
 - LL_ADC_PATH_INTERNAL_NONE
 - LL_ADC_PATH_INTERNAL_VREFINT
 - LL_ADC_PATH_INTERNAL_TEMPSENSOR
 - LL_ADC_PATH_INTERNAL_VBAT

Return values

- **None:**

Notes

- One or several values can be selected. Example: (LL_ADC_PATH_INTERNAL_VREFINT | LL_ADC_PATH_INTERNAL_TEMPSENSOR)
- Stabilization time of measurement path to internal channel: After enabling internal paths, before starting ADC conversion, a delay is required for internal voltage reference and temperature sensor stabilization time. Refer to device datasheet. Refer to literal LL_ADC_DELAY_VREFINT_STAB_US. Refer to literal LL_ADC_DELAY_TEMPSENSOR_STAB_US.
- ADC internal channel sampling time constraint: For ADC conversion of internal channels, a sampling time minimum value is required. Refer to device datasheet.

Reference Manual to LL API cross reference:

- CCR TSVREFE LL_ADC_SetCommonPathInternalCh
- CCR VBATE LL_ADC_SetCommonPathInternalCh

LL_ADC_GetCommonPathInternalCh

Function name

**__STATIC_INLINE uint32_t LL_ADC_GetCommonPathInternalCh (ADC_Common_TypeDef *
ADCxy_COMMON)**

Function description

Get parameter common to several ADC: measurement path to internal channels (VrefInt, temperature sensor, ...).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be a combination of the following values:
 - `LL_ADC_PATH_INTERNAL_NONE`
 - `LL_ADC_PATH_INTERNAL_VREFINT`
 - `LL_ADC_PATH_INTERNAL_TEMPSENSOR`
 - `LL_ADC_PATH_INTERNAL_VBAT`

Notes

- One or several values can be selected. Example: (`LL_ADC_PATH_INTERNAL_VREFINT | LL_ADC_PATH_INTERNAL_TEMPSENSOR`)

Reference Manual to LL API cross reference:

- CCR TSVREFE `LL_ADC_GetCommonPathInternalCh`
- CCR VBATE `LL_ADC_GetCommonPathInternalCh`

`LL_ADC_SetResolution`

Function name

`__STATIC_INLINE void LL_ADC_SetResolution (ADC_TypeDef * ADCx, uint32_t Resolution)`

Function description

Set ADC resolution.

Parameters

- **ADCx:** ADC instance
- **Resolution:** This parameter can be one of the following values:
 - `LL_ADC_RESOLUTION_12B`
 - `LL_ADC_RESOLUTION_10B`
 - `LL_ADC_RESOLUTION_8B`
 - `LL_ADC_RESOLUTION_6B`

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RES `LL_ADC_SetResolution`

`LL_ADC_GetResolution`

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetResolution (ADC_TypeDef * ADCx)`

Function description

Get ADC resolution.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Reference Manual to LL API cross reference:

- CR1 RES LL_ADC_GetResolution
LL_ADC_SetDataAlignment

Function name

__STATIC_INLINE void LL_ADC_SetDataAlignment (ADC_TypeDef * ADCx, uint32_t DataAlignment)

Function description

Set ADC conversion data alignment.

Parameters

- **ADCx:** ADC instance
- **DataAlignment:** This parameter can be one of the following values:
 - LL_ADC_DATA_ALIGN_RIGHT
 - LL_ADC_DATA_ALIGN_LEFT

Return values

- **None:**

Notes

- Refer to reference manual for alignments formats dependencies to ADC resolutions.

Reference Manual to LL API cross reference:

- CR2 ALIGN LL_ADC_SetDataAlignment
LL_ADC_GetDataAlignment

Function name

__STATIC_INLINE uint32_t LL_ADC_GetDataAlignment (ADC_TypeDef * ADCx)

Function description

Get ADC conversion data alignment.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_DATA_ALIGN_RIGHT
 - LL_ADC_DATA_ALIGN_LEFT

Notes

- Refer to reference manual for alignments formats dependencies to ADC resolutions.

Reference Manual to LL API cross reference:

- CR2 ALIGN LL_ADC_SetDataAlignment
LL_ADC_SetSequencersScanMode

Function name

```
__STATIC_INLINE void LL_ADC_SetSequencersScanMode (ADC_TypeDef * ADCx, uint32_t ScanMode)
```

Function description

Set ADC sequencers scan mode, for all ADC groups (group regular, group injected).

Parameters

- **ADCx:** ADC instance
- **ScanMode:** This parameter can be one of the following values:
 - LL_ADC_SEQ_SCAN_DISABLE
 - LL_ADC_SEQ_SCAN_ENABLE

Return values

- **None:**

Notes

- According to sequencers scan mode : If disabled: ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of sequencers of all ADC groups (sequencer scan length, ...) is discarded: equivalent to scan length of 1 rank. If enabled: ADC conversions are performed in sequence conversions mode, according to configuration of sequencers of each ADC group (sequencer scan length, ...). Refer to function LL_ADC_REG_SetSequencerLength() and to function LL_ADC_INJ_SetSequencerLength().

Reference Manual to LL API cross reference:

- CR1 SCAN LL_ADC_SetSequencersScanMode

LL_ADC_GetSequencersScanMode

Function name

```
__STATIC_INLINE uint32_t LL_ADC_GetSequencersScanMode (ADC_TypeDef * ADCx)
```

Function description

Get ADC sequencers scan mode, for all ADC groups (group regular, group injected).

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_SEQ_SCAN_DISABLE
 - LL_ADC_SEQ_SCAN_ENABLE

Notes

- According to sequencers scan mode : If disabled: ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of sequencers of all ADC groups (sequencer scan length, ...) is discarded: equivalent to scan length of 1 rank. If enabled: ADC conversions are performed in sequence conversions mode, according to configuration of sequencers of each ADC group (sequencer scan length, ...). Refer to function LL_ADC_REG_SetSequencerLength() and to function LL_ADC_INJ_SetSequencerLength().

Reference Manual to LL API cross reference:

- CR1 SCAN LL_ADC_GetSequencersScanMode

LL_ADC_REG_SetTriggerSource

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetTriggerSource (ADC_TypeDef * ADCx, uint32_t TriggerSource)
```

Function description

Set ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance
- **TriggerSource:** This parameter can be one of the following values:
 - LL_ADC_REG_TRIG_SOFTWARE
 - LL_ADC_REG_TRIG_EXT_TIM1_CH1
 - LL_ADC_REG_TRIG_EXT_TIM1_CH2
 - LL_ADC_REG_TRIG_EXT_TIM1_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH2
 - LL_ADC_REG_TRIG_EXT_TIM2_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH4
 - LL_ADC_REG_TRIG_EXT_TIM2_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM3_CH1
 - LL_ADC_REG_TRIG_EXT_TIM3_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM4_CH4
 - LL_ADC_REG_TRIG_EXT_TIM5_CH1
 - LL_ADC_REG_TRIG_EXT_TIM5_CH2
 - LL_ADC_REG_TRIG_EXT_TIM5_CH3
 - LL_ADC_REG_TRIG_EXT_TIM8_CH1
 - LL_ADC_REG_TRIG_EXT_TIM8_TRGO
 - LL_ADC_REG_TRIG_EXT_EXTI_LINE11

Return values

- **None:**

Notes

- On this STM32 serie, setting of external trigger edge is performed using function LL_ADC_REG_StartConversionExtTrig().
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 EXTSEL LL_ADC_REG_SetTriggerSource
- CR2 EXTEN LL_ADC_REG_SetTriggerSource

LL_ADC_REG_GetTriggerSource

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetTriggerSource (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_TRIG_SOFTWARE
 - LL_ADC_REG_TRIG_EXT_TIM1_CH1
 - LL_ADC_REG_TRIG_EXT_TIM1_CH2
 - LL_ADC_REG_TRIG_EXT_TIM1_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH2
 - LL_ADC_REG_TRIG_EXT_TIM2_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH4
 - LL_ADC_REG_TRIG_EXT_TIM2_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM3_CH1
 - LL_ADC_REG_TRIG_EXT_TIM3_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM4_CH4
 - LL_ADC_REG_TRIG_EXT_TIM5_CH1
 - LL_ADC_REG_TRIG_EXT_TIM5_CH2
 - LL_ADC_REG_TRIG_EXT_TIM5_CH3
 - LL_ADC_REG_TRIG_EXT_TIM8_CH1
 - LL_ADC_REG_TRIG_EXT_TIM8_TRGO
 - LL_ADC_REG_TRIG_EXT_EXTI_LINE11

Notes

- To determine whether group regular trigger source is internal (SW start) or external, without detail of which peripheral is selected as external trigger, (equivalent to "if(LL_ADC_REG_GetTriggerSource(ADC1) == LL_ADC_REG_TRIG_SOFTWARE)") use function LL_ADC_REG_IsTriggerSourceSWStart.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 EXTSEL LL_ADC_REG_GetTriggerSource
- CR2 EXTEN LL_ADC_REG_GetTriggerSource

LL_ADC_REG_IsTriggerSourceSWStart

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_IsTriggerSourceSWStart (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion trigger source internal (SW start) or external.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** "0" if trigger source external trigger Value "1" if trigger source SW start.

Notes

- In case of group regular trigger source set to external trigger, to determine which peripheral is selected as external trigger, use function LL_ADC_REG_GetTriggerSource().

Reference Manual to LL API cross reference:

- CR2 EXTEN LL_ADC_REG_IsTriggerSourceSWStart

LL_ADC_REG_GetTriggerEdge

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetTriggerEdge (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion trigger polarity.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_TRIG_EXT_RISING
 - LL_ADC_REG_TRIG_EXT_FALLING
 - LL_ADC_REG_TRIG_EXT_RISINGFALLING

Notes

- Applicable only for trigger source set to external trigger.
- On this STM32 series, setting of external trigger edge is performed using function LL_ADC_REG_StartConversionExtTrig().

Reference Manual to LL API cross reference:

- CR2 EXTEN LL_ADC_REG_GetTriggerEdge

LL_ADC_REG_SetSequencerLength

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetSequencerLength (ADC_TypeDef * ADCx, uint32_t SequencerNbRanks)
```

Function description

Set ADC group regular sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance
- **SequencerNbRanks:** This parameter can be one of the following values:
 - LL_ADC_REG_SEQ_SCAN_DISABLE
 - LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

Return values

- **None:**

Notes

- Description of ADC group regular sequencer features: For devices with sequencer fully configurable (function "LL_ADC_REG_SetSequencerRanks()" available): sequencer length and each rank affectation to a channel are configurable. This function performs configuration of: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerRanks()". For devices with sequencer not fully configurable (function "LL_ADC_REG_SetSequencerChannels()" available): sequencer length and each rank affectation to a channel are defined by channel number. This function performs configuration of: Sequence length: Number of ranks in the scan sequence is defined by number of channels set in the sequence, rank of each channel is fixed by channel HW number. (channel 0 fixed on rank 0, channel 1 fixed on rank1, ...). Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from lowest channel number to highest channel number). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerChannels()".
- On this STM32 serie, group regular sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- SQR1 L LL_ADC_REG_SetSequencerLength

LL_ADC_REG_GetSequencerLength

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerLength (ADC_TypeDef * ADCx)

Function description

Get ADC group regular sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_SEQ_SCAN_DISABLE
 - LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

Notes

- Description of ADC group regular sequencer features: For devices with sequencer fully configurable (function "LL_ADC_REG_SetSequencerRanks()" available): sequencer length and each rank affectation to a channel are configurable. This function retrieves: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerRanks()". For devices with sequencer not fully configurable (function "LL_ADC_REG_SetSequencerChannels()" available): sequencer length and each rank affectation to a channel are defined by channel number. This function retrieves: Sequence length: Number of ranks in the scan sequence is defined by number of channels set in the sequence, rank of each channel is fixed by channel HW number. (channel 0 fixed on rank 0, channel 1 fixed on rank1, ...). Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from lowest channel number to highest channel number). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerChannels()".
- On this STM32 serie, group regular sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- SQR1 L LL_ADC_REG_SetSequencerLength

LL_ADC_REG_SetSequencerDiscont

Function name

__STATIC_INLINE void LL_ADC_REG_SetSequencerDiscont (ADC_TypeDef * ADCx, uint32_t SeqDiscont)

Function description

Set ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance
- **SeqDiscont:** This parameter can be one of the following values:
 - LL_ADC_REG_SEQ_DISCONT_DISABLE
 - LL_ADC_REG_SEQ_DISCONT_1RANK
 - LL_ADC_REG_SEQ_DISCONT_2RANKS
 - LL_ADC_REG_SEQ_DISCONT_3RANKS
 - LL_ADC_REG_SEQ_DISCONT_4RANKS
 - LL_ADC_REG_SEQ_DISCONT_5RANKS
 - LL_ADC_REG_SEQ_DISCONT_6RANKS
 - LL_ADC_REG_SEQ_DISCONT_7RANKS
 - LL_ADC_REG_SEQ_DISCONT_8RANKS

Return values

- **None:**

Notes

- It is not possible to enable both ADC group regular continuous mode and sequencer discontinuous mode.
- It is not possible to enable both ADC auto-injected mode and ADC group regular sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_SetSequencerDiscont
- CR1 DISCNUM LL_ADC_REG_SetSequencerDiscont

```
LL_ADC_REG_GetSequencerDiscont
```

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerDiscont (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_SEQ_DISCONT_DISABLE
 - LL_ADC_REG_SEQ_DISCONT_1RANK
 - LL_ADC_REG_SEQ_DISCONT_2RANKS
 - LL_ADC_REG_SEQ_DISCONT_3RANKS
 - LL_ADC_REG_SEQ_DISCONT_4RANKS
 - LL_ADC_REG_SEQ_DISCONT_5RANKS
 - LL_ADC_REG_SEQ_DISCONT_6RANKS
 - LL_ADC_REG_SEQ_DISCONT_7RANKS
 - LL_ADC_REG_SEQ_DISCONT_8RANKS

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_GetSequencerDiscont
- CR1 DISCNUM LL_ADC_REG_GetSequencerDiscont

```
LL_ADC_REG_SetSequencerRanks
```

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t Channel)
```

Function description

Set ADC group regular sequence: channel on the selected scan sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_REG_RANK_1
 - LL_ADC_REG_RANK_2
 - LL_ADC_REG_RANK_3
 - LL_ADC_REG_RANK_4
 - LL_ADC_REG_RANK_5
 - LL_ADC_REG_RANK_6
 - LL_ADC_REG_RANK_7
 - LL_ADC_REG_RANK_8
 - LL_ADC_REG_RANK_9
 - LL_ADC_REG_RANK_10
 - LL_ADC_REG_RANK_11
 - LL_ADC_REG_RANK_12
 - LL_ADC_REG_RANK_13
 - LL_ADC_REG_RANK_14
 - LL_ADC_REG_RANK_15
 - LL_ADC_REG_RANK_16
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F2, parameter available only on ADC instance: ADC1.
-

Return values

- **None:**

Notes

- This function performs configuration of: Channels ordering into each rank of scan sequence: whatever channel can be placed into whatever rank.
- On this STM32 serie, ADC group regular sequencer is fully configurable: sequencer length and each rank affectation to a channel are configurable. Refer to description of function `LL_ADC_REG_SetSequencerLength()`.
- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- On this STM32 serie, to measure internal channels (VrefInt, TempSensor, ...), measurement paths to internal channels must be enabled separately. This can be done using function `LL_ADC_SetCommonPathInternalCh()`.

Reference Manual to LL API cross reference:

- SQR3 SQ1 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ2 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ3 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ4 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ5 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ6 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ7 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ8 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ9 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ10 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ11 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ12 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ13 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ14 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ15 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ16 `LL_ADC_REG_SetSequencerRanks`

`LL_ADC_REG_GetSequencerRanks`

Function name

`__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank)`

Function description

Get ADC group regular sequence: channel on the selected scan sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_REG_RANK_1
 - LL_ADC_REG_RANK_2
 - LL_ADC_REG_RANK_3
 - LL_ADC_REG_RANK_4
 - LL_ADC_REG_RANK_5
 - LL_ADC_REG_RANK_6
 - LL_ADC_REG_RANK_7
 - LL_ADC_REG_RANK_8
 - LL_ADC_REG_RANK_9
 - LL_ADC_REG_RANK_10
 - LL_ADC_REG_RANK_11
 - LL_ADC_REG_RANK_12
 - LL_ADC_REG_RANK_13
 - LL_ADC_REG_RANK_14
 - LL_ADC_REG_RANK_15
 - LL_ADC_REG_RANK_16

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F2, parameter available only on ADC instance: ADC1.
- (1) For ADC channel read back from ADC register, comparison with internal channel parameter to be done using helper macro `__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL()`.

Notes

- On this STM32 serie, ADC group regular sequencer is fully configurable: sequencer length and each rank affectation to a channel are configurable. Refer to description of function `LL_ADC_REG_SetSequencerLength()`.
- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- Usage of the returned channel number: To reinject this channel into another function `LL_ADC_xxx`: the returned channel number is only partly formatted on definition of literals `LL_ADC_CHANNEL_x`. Therefore, it has to be compared with parts of literals `LL_ADC_CHANNEL_x` or using helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`. Then the selected literal `LL_ADC_CHANNEL_x` can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`.

Reference Manual to LL API cross reference:

- SQR3 SQ1 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ2 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ3 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ4 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ5 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ6 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ7 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ8 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ9 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ10 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ11 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ12 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ13 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ14 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ15 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ16 `LL_ADC_REG_GetSequencerRanks`

`LL_ADC_REG_SetContinuousMode`

Function name

`__STATIC_INLINE void LL_ADC_REG_SetContinuousMode (ADC_TypeDef * ADCx, uint32_t Continuous)`

Function description

Set ADC continuous conversion mode on ADC group regular.

Parameters

- ADCx:** ADC instance
- Continuous:** This parameter can be one of the following values:
 - `LL_ADC_REG_CONV_SINGLE`
 - `LL_ADC_REG_CONV_CONTINUOUS`

Return values

- None:**

Notes

- Description of ADC continuous conversion mode: single mode: one conversion per triggercontinuous mode: after the first trigger, following conversions launched successively automatically.
- It is not possible to enable both ADC group regular continuous mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR2 CONT LL_ADC_REG_SetContinuousMode
LL_ADC_REG_GetContinuousMode

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetContinuousMode (ADC_TypeDef * ADCx)

Function description

Get ADC continuous conversion mode on ADC group regular.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_CONV_SINGLE
 - LL_ADC_REG_CONV_CONTINUOUS

Notes

- Description of ADC continuous conversion mode: single mode: one conversion per triggercontinuous mode: after the first trigger, following conversions launched successively automatically.

Reference Manual to LL API cross reference:

- CR2 CONT LL_ADC_REG_GetContinuousMode
LL_ADC_REG_SetDMATransfer

Function name

__STATIC_INLINE void LL_ADC_REG_SetDMATransfer (ADC_TypeDef * ADCx, uint32_t DMATransfer)

Function description

Set ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode.

Parameters

- **ADCx:** ADC instance
- **DMATransfer:** This parameter can be one of the following values:
 - LL_ADC_REG_DMA_TRANSFER_NONE
 - LL_ADC_REG_DMA_TRANSFER_LIMITED
 - LL_ADC_REG_DMA_TRANSFER_UNLIMITED

Return values

- **None:**

Notes

- If transfer by DMA selected, specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular.Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- For devices with several ADC instances: ADC multimode DMA settings are available using function LL_ADC_SetMultiDMATransfer().
- To configure DMA source address (peripheral address), use function LL_ADC_DMA_GetRegAddr().

Reference Manual to LL API cross reference:

- CR2 DMA LL_ADC_REG_SetDMATransfer
- CR2 DDS LL_ADC_REG_SetDMATransfer

LL_ADC_REG_GetDMATransfer

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetDMATransfer (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_DMA_TRANSFER_NONE
 - LL_ADC_REG_DMA_TRANSFER_LIMITED
 - LL_ADC_REG_DMA_TRANSFER_UNLIMITED

Notes

- If transfer by DMA selected, specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- For devices with several ADC instances: ADC multimode DMA settings are available using function LL_ADC_GetMultiDMATransfer().
- To configure DMA source address (peripheral address), use function LL_ADC_DMA_GetRegAddr().

Reference Manual to LL API cross reference:

- CR2 DMA LL_ADC_REG_GetDMATransfer
- CR2 DDS LL_ADC_REG_GetDMATransfer

LL_ADC_REG_SetFlagEndOfConversion

Function name

__STATIC_INLINE void LL_ADC_REG_SetFlagEndOfConversion (ADC_TypeDef * ADCx, uint32_t EocSelection)

Function description

Specify which ADC flag between EOC (end of unitary conversion) or EOS (end of sequence conversions) is used to indicate the end of conversion.

Parameters

- **ADCx:** ADC instance
- **EocSelection:** This parameter can be one of the following values:
 - LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV
 - LL_ADC_REG_FLAG_EOC_UNITARY_CONV

Return values

- **None:**

Notes

- This feature is aimed to be set when using ADC with programming model by polling or interruption (programming model by DMA usually uses DMA interruptions to indicate end of conversion and data transfer).
- For ADC group injected, end of conversion (flag&IT) is raised only at the end of the sequence.

Reference Manual to LL API cross reference:

- CR2 EOCS LL_ADC_REG_SetFlagEndOfConversion
LL_ADC_REG_GetFlagEndOfConversion

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetFlagEndOfConversion (ADC_TypeDef * ADCx)

Function description

Get which ADC flag between EOC (end of unitary conversion) or EOS (end of sequence conversions) is used to indicate the end of conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV
 - LL_ADC_REG_FLAG_EOC_UNITARY_CONV

Reference Manual to LL API cross reference:

- CR2 EOCS LL_ADC_REG_GetFlagEndOfConversion
LL_ADC_INJ_SetTriggerSource

Function name

__STATIC_INLINE void LL_ADC_INJ_SetTriggerSource (ADC_TypeDef * ADCx, uint32_t TriggerSource)

Function description

Set ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance
- **TriggerSource:** This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_SOFTWARE
 - LL_ADC_INJ_TRIG_EXT_TIM1_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM1_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM2_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM2_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM4_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM5_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM5_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH4
 - LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

Return values

- **None:**

Notes

- On this STM32 serie, setting of external trigger edge is performed using function LL_ADC_INJ_StartConversionExtTrig().
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 JEXTSEL LL_ADC_INJ_SetTriggerSource
- CR2 JEXTEN LL_ADC_INJ_SetTriggerSource

LL_ADC_INJ_GetTriggerSource

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetTriggerSource (ADC_TypeDef * ADCx)

Function description

Get ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_SOFTWARE
 - LL_ADC_INJ_TRIG_EXT_TIM1_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM1_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM2_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM2_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM4_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM5_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM5_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH4
 - LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

Notes

- To determine whether group injected trigger source is internal (SW start) or external, without detail of which peripheral is selected as external trigger, (equivalent to "if(LL_ADC_INJ_GetTriggerSource(ADC1) == LL_ADC_INJ_TRIG_SOFTWARE)") use function LL_ADC_INJ_IsTriggerSourceSWStart.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 JEXTSEL LL_ADC_INJ_GetTriggerSource
- CR2 JEXTEN LL_ADC_INJ_GetTriggerSource

LL_ADC_INJ_IsTriggerSourceSWStart

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_IsTriggerSourceSWStart (ADC_TypeDef * ADCx)

Function description

Get ADC group injected conversion trigger source internal (SW start) or external.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** "0" if trigger source external trigger Value "1" if trigger source SW start.

Notes

- In case of group injected trigger source set to external trigger, to determine which peripheral is selected as external trigger, use function LL_ADC_INJ_GetTriggerSource.

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_IsTriggerSourceSWStart

LL_ADC_INJ_GetTriggerEdge

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetTriggerEdge (ADC_TypeDef * ADCx)
```

Function description

Get ADC group injected conversion trigger polarity.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_EXT_RISING
 - LL_ADC_INJ_TRIG_EXT_FALLING
 - LL_ADC_INJ_TRIG_EXT_RISINGFALLING

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_GetTriggerEdge

LL_ADC_INJ_SetSequencerLength

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetSequencerLength (ADC_TypeDef * ADCx, uint32_t SequencerNbRanks)
```

Function description

Set ADC group injected sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance
- **SequencerNbRanks:** This parameter can be one of the following values:
 - LL_ADC_INJ_SEQ_SCAN_DISABLE
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

Return values

- **None:**

Notes

- This function performs configuration of: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n).
- On this STM32 serie, group injected sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- JSQR JL LL_ADC_INJ_SetSequencerLength

LL_ADC_INJ_GetSequencerLength

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerLength (ADC_TypeDef * ADCx)
```

Function description

Get ADC group injected sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_SEQ_SCAN_DISABLE
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

Notes

- This function retrieves: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n).
- On this STM32 serie, group injected sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- JSQR JL LL_ADC_INJ_GetSequencerLength
- LL_ADC_INJ_SetSequencerDiscont

Function name

__STATIC_INLINE void LL_ADC_INJ_SetSequencerDiscont (ADC_TypeDef * ADCx, uint32_t SeqDiscont)

Function description

Set ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance
- **SeqDiscont:** This parameter can be one of the following values:
 - LL_ADC_INJ_SEQ_DISCONT_DISABLE
 - LL_ADC_INJ_SEQ_DISCONT_1RANK

Return values

- **None:**

Notes

- It is not possible to enable both ADC group injected auto-injected mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_INJ_SetSequencerDiscont
- LL_ADC_INJ_GetSequencerDiscont

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerDiscont (ADC_TypeDef * ADCx)

Function description

Get ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_SEQ_DISCONT_DISABLE
 - LL_ADC_INJ_SEQ_DISCONT_1RANK

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_GetSequencerDiscont

LL_ADC_INJ_SetSequencerRanks

Function name

__STATIC_INLINE void LL_ADC_INJ_SetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t Channel)

Function description

Set ADC group injected sequence: channel on the selected sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F2, parameter available only on ADC instance: ADC1.
-

Return values

- **None:**

Notes

- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- On this STM32 serie, to measure internal channels (VrefInt, TempSensor, ...), measurement paths to internal channels must be enabled separately. This can be done using function LL_ADC_SetCommonPathInternalCh().

Reference Manual to LL API cross reference:

- JSQR JSQ1 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ2 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ3 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ4 LL_ADC_INJ_SetSequencerRanks

LL_ADC_INJ_GetSequencerRanks

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected sequence: channel on the selected sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F2, parameter available only on ADC instance: ADC1.
- (1) For ADC channel read back from ADC register, comparison with internal channel parameter to be done using helper macro `__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL()`.

Notes

- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- Usage of the returned channel number: To reinject this channel into another function `LL_ADC_xxx`: the returned channel number is only partly formatted on definition of literals `LL_ADC_CHANNEL_x`. Therefore, it has to be compared with parts of literals `LL_ADC_CHANNEL_x` or using helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`. Then the selected literal `LL_ADC_CHANNEL_x` can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`.

Reference Manual to LL API cross reference:

- JSQR JSQ1 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ2 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ3 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ4 LL_ADC_INJ_SetSequencerRanks

LL_ADC_INJ_SetTrigAuto

Function name

__STATIC_INLINE void LL_ADC_INJ_SetTrigAuto (ADC_TypeDef * ADCx, uint32_t TrigAuto)

Function description

Set ADC group injected conversion trigger: independent or from ADC group regular.

Parameters

- **ADCx:** ADC instance
- **TrigAuto:** This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_INDEPENDENT
 - LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

Return values

- **None:**

Notes

- This mode can be used to extend number of data registers updated after one ADC conversion trigger and with data permanently kept (not erased by successive conversions of scan of ADC sequencer ranks), up to 5 data registers: 1 data register on ADC group regular, 4 data registers on ADC group injected.
- If ADC group injected injected trigger source is set to an external trigger, this feature must be must be set to independent trigger. ADC group injected automatic trigger is compliant only with group injected trigger source set to SW start, without any further action on ADC group injected conversion start or stop: in this case, ADC group injected is controlled only from ADC group regular.
- It is not possible to enable both ADC group injected auto-injected mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 JAUTO LL_ADC_INJ_SetTrigAuto

LL_ADC_INJ_GetTrigAuto

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetTrigAuto (ADC_TypeDef * ADCx)

Function description

Get ADC group injected conversion trigger: independent or from ADC group regular.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_INDEPENDENT
 - LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

Reference Manual to LL API cross reference:

- CR1 JAUTO LL_ADC_INJ_GetTrigAuto

LL_ADC_INJ_SetOffset

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetOffset (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t OffsetLevel)
```

Function description

Set ADC group injected offset.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4
- **OffsetLevel:** Value between Min_Data=0x000 and Max_Data=0xFFFF

Return values

- **None:**

Notes

- It sets: ADC group injected rank to which the offset programmed will be appliedOffset level (offset to be subtracted from the raw converted data). Caution: Offset format is dependent to ADC resolution: offset has to be left-aligned on bit 11, the LSB (right bits) are set to 0.
- Offset cannot be enabled or disabled. To emulate offset disabled, set an offset value equal to 0.

Reference Manual to LL API cross reference:

- JOFR1 JOFFSET1 LL_ADC_INJ_SetOffset
- JOFR2 JOFFSET2 LL_ADC_INJ_SetOffset
- JOFR3 JOFFSET3 LL_ADC_INJ_SetOffset
- JOFR4 JOFFSET4 LL_ADC_INJ_SetOffset

LL_ADC_INJ_GetOffset

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetOffset (ADC_TypeDef * ADCx, uint32_t Rank)
```

Function description

Get ADC group injected offset.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFFF

Notes

- It gives offset level (offset to be subtracted from the raw converted data). Caution: Offset format is dependent to ADC resolution: offset has to be left-aligned on bit 11, the LSB (right bits) are set to 0.

Reference Manual to LL API cross reference:

- JOFR1 JOFFSET1 LL_ADC_INJ_GetOffset
- JOFR2 JOFFSET2 LL_ADC_INJ_GetOffset
- JOFR3 JOFFSET3 LL_ADC_INJ_GetOffset
- JOFR4 JOFFSET4 LL_ADC_INJ_GetOffset

LL_ADC_SetChannelSamplingTime

Function name

__STATIC_INLINE void LL_ADC_SetChannelSamplingTime (ADC_TypeDef * ADCx, uint32_t Channel, uint32_t SamplingTime)

Function description

Set sampling time of the selected ADC channel Unit: ADC clock cycles.

Parameters

- **ADCx:** ADC instance
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F2, parameter available only on ADC instance: ADC1.
- **SamplingTime:** This parameter can be one of the following values:
 - LL_ADC_SAMPLINGTIME_3CYCLES
 - LL_ADC_SAMPLINGTIME_15CYCLES
 - LL_ADC_SAMPLINGTIME_28CYCLES
 - LL_ADC_SAMPLINGTIME_56CYCLES
 - LL_ADC_SAMPLINGTIME_84CYCLES
 - LL_ADC_SAMPLINGTIME_112CYCLES
 - LL_ADC_SAMPLINGTIME_144CYCLES
 - LL_ADC_SAMPLINGTIME_480CYCLES

Return values

- **None:**

Notes

- On this device, sampling time is on channel scope: independently of channel mapped on ADC group regular or injected.
- In case of internal channel (VrefInt, TempSensor, ...) to be converted: sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting). Refer to device datasheet for timings values (parameters TS_vrefint, TS_temp, ...).
- Conversion time is the addition of sampling time and processing time. Refer to reference manual for ADC processing time of this STM32 serie.
- In case of ADC conversion of internal channel (VrefInt, temperature sensor, ...), a sampling time minimum value is required. Refer to device datasheet.

Reference Manual to LL API cross reference:

- SMPR1 SMP18 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP17 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP16 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP15 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP14 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP13 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP12 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP11 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP10 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP9 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP8 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP7 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP6 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP5 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP4 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP3 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP2 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP1 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP0 LL_ADC_SetChannelSamplingTime

LL_ADC_GetChannelSamplingTime

Function name

__STATIC_INLINE uint32_t LL_ADC_GetChannelSamplingTime (ADC_TypeDef * ADCx, uint32_t Channel)

Function description

Get sampling time of the selected ADC channel Unit: ADC clock cycles.

Parameters

- **ADCx:** ADC instance
 - **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F2, parameter available only on ADC instance: ADC1.

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_SAMPLINGTIME_3CYCLES
 - LL_ADC_SAMPLINGTIME_15CYCLES
 - LL_ADC_SAMPLINGTIME_28CYCLES
 - LL_ADC_SAMPLINGTIME_56CYCLES
 - LL_ADC_SAMPLINGTIME_84CYCLES
 - LL_ADC_SAMPLINGTIME_112CYCLES
 - LL_ADC_SAMPLINGTIME_144CYCLES
 - LL_ADC_SAMPLINGTIME_480CYCLES

Notes

- On this device, sampling time is on channel scope: independently of channel mapped on ADC group regular or injected.
- Conversion time is the addition of sampling time and processing time. Refer to reference manual for ADC processing time of this STM32 serie.

Reference Manual to LL API cross reference:

- SMPR1 SMP18 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP17 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP16 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP15 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP14 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP13 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP12 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP11 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP10 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP9 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP8 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP7 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP6 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP5 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP4 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP3 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP2 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP1 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP0 LL_ADC_GetChannelSamplingTime

LL_ADC_SetAnalogWDMonitChannels

Function name

__STATIC_INLINE void LL_ADC_SetAnalogWDMonitChannels (ADC_TypeDef * ADCx, uint32_t AWDCChannelGroup)

Function description

Set ADC analog watchdog monitored channels: a single channel or all channels, on ADC groups regular and-or injected.

Parameters

- **ADCx:** ADC instance
- **AWDChannelGroup:** This parameter can be one of the following values:
 - LL_ADC_AWD_DISABLE
 - LL_ADC_AWD_ALL_CHANNELS_REG
 - LL_ADC_AWD_ALL_CHANNELS_INJ
 - LL_ADC_AWD_ALL_CHANNELS_REG_INJ
 - LL_ADC_AWD_CHANNEL_0_REG
 - LL_ADC_AWD_CHANNEL_0_INJ
 - LL_ADC_AWD_CHANNEL_0_REG_INJ
 - LL_ADC_AWD_CHANNEL_1_REG
 - LL_ADC_AWD_CHANNEL_1_INJ
 - LL_ADC_AWD_CHANNEL_1_REG_INJ
 - LL_ADC_AWD_CHANNEL_2_REG
 - LL_ADC_AWD_CHANNEL_2_INJ
 - LL_ADC_AWD_CHANNEL_2_REG_INJ
 - LL_ADC_AWD_CHANNEL_3_REG
 - LL_ADC_AWD_CHANNEL_3_INJ
 - LL_ADC_AWD_CHANNEL_3_REG_INJ
 - LL_ADC_AWD_CHANNEL_4_REG
 - LL_ADC_AWD_CHANNEL_4_INJ
 - LL_ADC_AWD_CHANNEL_4_REG_INJ
 - LL_ADC_AWD_CHANNEL_5_REG
 - LL_ADC_AWD_CHANNEL_5_INJ
 - LL_ADC_AWD_CHANNEL_5_REG_INJ
 - LL_ADC_AWD_CHANNEL_6_REG
 - LL_ADC_AWD_CHANNEL_6_INJ
 - LL_ADC_AWD_CHANNEL_6_REG_INJ
 - LL_ADC_AWD_CHANNEL_7_REG
 - LL_ADC_AWD_CHANNEL_7_INJ
 - LL_ADC_AWD_CHANNEL_7_REG_INJ
 - LL_ADC_AWD_CHANNEL_8_REG
 - LL_ADC_AWD_CHANNEL_8_INJ
 - LL_ADC_AWD_CHANNEL_8_REG_INJ
 - LL_ADC_AWD_CHANNEL_9_REG
 - LL_ADC_AWD_CHANNEL_9_INJ
 - LL_ADC_AWD_CHANNEL_9_REG_INJ
 - LL_ADC_AWD_CHANNEL_10_REG
 - LL_ADC_AWD_CHANNEL_10_INJ
 - LL_ADC_AWD_CHANNEL_10_REG_INJ
 - LL_ADC_AWD_CHANNEL_11_REG
 - LL_ADC_AWD_CHANNEL_11_INJ
 - LL_ADC_AWD_CHANNEL_11_REG_INJ
 - LL_ADC_AWD_CHANNEL_12_REG
 - LL_ADC_AWD_CHANNEL_12_INJ
 - LL_ADC_AWD_CHANNEL_12_REG_INJ
 - LL_ADC_AWD_CHANNEL_13_REG
 - LL_ADC_AWD_CHANNEL_13_INJ
 - LL_ADC_AWD_CHANNEL_13_REG_INJ

- LL_ADC_AWD_CHANNEL_14_REG
- LL_ADC_AWD_CHANNEL_14_INJ
- LL_ADC_AWD_CHANNEL_14_REG_INJ
- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ
- LL_ADC_AWD_CH_VREFINT_REG (1)
- LL_ADC_AWD_CH_VREFINT_INJ (1)
- LL_ADC_AWD_CH_VREFINT_REG_INJ (1)
- LL_ADC_AWD_CH_TEMPSENSOR_REG (1)
- LL_ADC_AWD_CH_TEMPSENSOR_INJ (1)
- LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG (1)
- LL_ADC_AWD_CH_VBAT_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG_INJ (1)

(1) On STM32F2, parameter available only on ADC instance: ADC1.

•

Return values

- **None:**

Notes

- Once monitored channels are selected, analog watchdog is enabled.
- In case of need to define a single channel to monitor with analog watchdog from sequencer channel definition, use helper macro `__LL_ADC_ANALOGWD_CHANNEL_GROUP()`.
- On this STM32 series, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels.groups monitored: ADC groups regular and-or injected.resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- CR1 AWD1CH LL_ADC_SetAnalogWDMonitChannels
- CR1 AWD1SGL LL_ADC_SetAnalogWDMonitChannels
- CR1 AWD1EN LL_ADC_SetAnalogWDMonitChannels

LL_ADC_GetAnalogWDMonitChannels

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetAnalogWDMonitChannels(ADC_TypeDef * ADCx)`

Function description

Get ADC analog watchdog monitored channel.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_AWD_DISABLE
 - LL_ADC_AWD_ALL_CHANNELS_REG
 - LL_ADC_AWD_ALL_CHANNELS_INJ
 - LL_ADC_AWD_ALL_CHANNELS_REG_INJ
 - LL_ADC_AWD_CHANNEL_0_REG
 - LL_ADC_AWD_CHANNEL_0_INJ
 - LL_ADC_AWD_CHANNEL_0_REG_INJ
 - LL_ADC_AWD_CHANNEL_1_REG
 - LL_ADC_AWD_CHANNEL_1_INJ
 - LL_ADC_AWD_CHANNEL_1_REG_INJ
 - LL_ADC_AWD_CHANNEL_2_REG
 - LL_ADC_AWD_CHANNEL_2_INJ
 - LL_ADC_AWD_CHANNEL_2_REG_INJ
 - LL_ADC_AWD_CHANNEL_3_REG
 - LL_ADC_AWD_CHANNEL_3_INJ
 - LL_ADC_AWD_CHANNEL_3_REG_INJ
 - LL_ADC_AWD_CHANNEL_4_REG
 - LL_ADC_AWD_CHANNEL_4_INJ
 - LL_ADC_AWD_CHANNEL_4_REG_INJ
 - LL_ADC_AWD_CHANNEL_5_REG
 - LL_ADC_AWD_CHANNEL_5_INJ
 - LL_ADC_AWD_CHANNEL_5_REG_INJ
 - LL_ADC_AWD_CHANNEL_6_REG
 - LL_ADC_AWD_CHANNEL_6_INJ
 - LL_ADC_AWD_CHANNEL_6_REG_INJ
 - LL_ADC_AWD_CHANNEL_7_REG
 - LL_ADC_AWD_CHANNEL_7_INJ
 - LL_ADC_AWD_CHANNEL_7_REG_INJ
 - LL_ADC_AWD_CHANNEL_8_REG
 - LL_ADC_AWD_CHANNEL_8_INJ
 - LL_ADC_AWD_CHANNEL_8_REG_INJ
 - LL_ADC_AWD_CHANNEL_9_REG
 - LL_ADC_AWD_CHANNEL_9_INJ
 - LL_ADC_AWD_CHANNEL_9_REG_INJ
 - LL_ADC_AWD_CHANNEL_10_REG
 - LL_ADC_AWD_CHANNEL_10_INJ
 - LL_ADC_AWD_CHANNEL_10_REG_INJ
 - LL_ADC_AWD_CHANNEL_11_REG
 - LL_ADC_AWD_CHANNEL_11_INJ
 - LL_ADC_AWD_CHANNEL_11_REG_INJ
 - LL_ADC_AWD_CHANNEL_12_REG
 - LL_ADC_AWD_CHANNEL_12_INJ
 - LL_ADC_AWD_CHANNEL_12_REG_INJ
 - LL_ADC_AWD_CHANNEL_13_REG
 - LL_ADC_AWD_CHANNEL_13_INJ
 - LL_ADC_AWD_CHANNEL_13_REG_INJ
 - LL_ADC_AWD_CHANNEL_14_REG

- LL_ADC_AWD_CHANNEL_14_INJ
- LL_ADC_AWD_CHANNEL_14_REG_INJ
- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ

Notes

- Usage of the returned channel number: To reinject this channel into another function LL_ADC_xxx: the returned channel number is only partly formatted on definition of literals LL_ADC_CHANNEL_x. Therefore, it has to be compared with parts of literals LL_ADC_CHANNEL_x or using helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`. Then the selected literal LL_ADC_CHANNEL_x can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`. Applicable only when the analog watchdog is set to monitor one channel.
- On this STM32 serie, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels. groups monitored: ADC groups regular and-or injected. resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- CR1 AWD1CH LL_ADC_GetAnalogWDMonitChannels
- CR1 AWD1SGL LL_ADC_GetAnalogWDMonitChannels
- CR1 AWD1EN LL_ADC_GetAnalogWDMonitChannels

LL_ADC_SetAnalogWDT thresholds

Function name

__STATIC_INLINE void LL_ADC_SetAnalogWDT thresholds (ADC_TypeDef * ADCx, uint32_t AWDThresholdsHighLow, uint32_t AWDThresholdValue)

Function description

Set ADC analog watchdog threshold value of threshold high or low.

Parameters

- **ADCx:** ADC instance
- **AWDThresholdsHighLow:** This parameter can be one of the following values:
 - LL_ADC_AWD_THRESHOLD_HIGH
 - LL_ADC_AWD_THRESHOLD_LOW
- **AWDThresholdValue:** Value between Min_Data=0x000 and Max_Data=0xFFFF

Return values

- **None:**

Notes

- In case of ADC resolution different of 12 bits, analog watchdog thresholds data require a specific shift. Use helper macro `__LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION()`.
- On this STM32 serie, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels.groups monitored: ADC groups regular and-or injected.resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- HTR HT LL_ADC_SetAnalogWDThresholds
- LTR LT LL_ADC_SetAnalogWDThresholds

LL_ADC_GetAnalogWDThresholds

Function name

__STATIC_INLINE uint32_t LL_ADC_GetAnalogWDThresholds (ADC_TypeDef * ADCx, uint32_t AWDThresholdsHighLow)

Function description

Get ADC analog watchdog threshold value of threshold high or threshold low.

Parameters

- **ADCx:** ADC instance
- **AWDThresholdsHighLow:** This parameter can be one of the following values:
 - LL_ADC_AWD_THRESHOLD_HIGH
 - LL_ADC_AWD_THRESHOLD_LOW

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFFF

Notes

- In case of ADC resolution different of 12 bits, analog watchdog thresholds data require a specific shift. Use helper macro `__LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION()`.

Reference Manual to LL API cross reference:

- HTR HT LL_ADC_GetAnalogWDThresholds
- LTR LT LL_ADC_GetAnalogWDThresholds

LL_ADC_SetMultimode

Function name

__STATIC_INLINE void LL_ADC_SetMultimode (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t Multimode)

Function description

Set ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **Multimode:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_INDEPENDENT`
 - `LL_ADC_MULTI_DUAL_REG_SIMULT`
 - `LL_ADC_MULTI_DUAL_REG_INTERL`
 - `LL_ADC_MULTI_DUAL_INJ_SIMULT`
 - `LL_ADC_MULTI_DUAL_INJ_ALTERN`
 - `LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM`
 - `LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT`
 - `LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM`
 - `LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM`
 - `LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT`
 - `LL_ADC_MULTI_TRIPLE_INJ_SIMULT`
 - `LL_ADC_MULTI_TRIPLE_REG_SIMULT`
 - `LL_ADC_MULTI_TRIPLE_REG_INTERL`
 - `LL_ADC_MULTI_TRIPLE_INJ_ALTERN`

Return values

- **None:**

Notes

- If multimode configuration: the selected ADC instance is either master or slave depending on hardware. Refer to reference manual.

Reference Manual to LL API cross reference:

- CCR MULTI `LL_ADC_SetMultimode`

`LL_ADC_GetMultimode`

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetMultimode (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_MULTI_INDEPENDENT
 - LL_ADC_MULTI_DUAL_REG_SIMULT
 - LL_ADC_MULTI_DUAL_REG_INTERL
 - LL_ADC_MULTI_DUAL_INJ_SIMULT
 - LL_ADC_MULTI_DUAL_INJ_ALTERN
 - LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM
 - LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT
 - LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM
 - LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM
 - LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT
 - LL_ADC_MULTI_TRIPLE_INJ_SIMULT
 - LL_ADC_MULTI_TRIPLE_REG_SIMULT
 - LL_ADC_MULTI_TRIPLE_REG_INTERL
 - LL_ADC_MULTI_TRIPLE_INJ_ALTERN

Notes

- If multimode configuration: the selected ADC instance is either master or slave depending on hardware. Refer to reference manual.

Reference Manual to LL API cross reference:

- CCR MULTI LL_ADC_GetMultimode

LL_ADC_SetMultiDMATransfer

Function name

__STATIC_INLINE void LL_ADC_SetMultiDMATransfer (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t MultiDMATransfer)

Function description

Set ADC multimode conversion data transfer: no transfer or transfer by DMA.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **MultiDMATransfer:** This parameter can be one of the following values:
 - LL_ADC_MULTI_REG_DMA_EACH_ADC
 - LL_ADC_MULTI_REG_DMA_LIMIT_1
 - LL_ADC_MULTI_REG_DMA_LIMIT_2
 - LL_ADC_MULTI_REG_DMA_LIMIT_3
 - LL_ADC_MULTI_REG_DMA_UNLMT_1
 - LL_ADC_MULTI_REG_DMA_UNLMT_2
 - LL_ADC_MULTI_REG_DMA_UNLMT_3

Return values

- **None:**

Notes

- If ADC multimode transfer by DMA is not selected: each ADC uses its own DMA channel, with its individual DMA transfer settings. If ADC multimode transfer by DMA is selected: One DMA channel is used for both ADC (DMA of ADC master) Specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- How to retrieve multimode conversion data: Whatever multimode transfer by DMA setting: using function `LL_ADC_REG_ReadMultiConversionData32()`. If ADC multimode transfer by DMA is selected: conversion data is a raw data with ADC master and slave concatenated. A macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`.

Reference Manual to LL API cross reference:

- CCR MDMA `LL_ADC_SetMultiDMATransfer`
- CCR DDS `LL_ADC_SetMultiDMATransfer`

`LL_ADC_GetMultiDMATransfer`

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetMultiDMATransfer (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get ADC multimode conversion data transfer: no transfer or transfer by DMA.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - `LL_ADC_MULTI_REG_DMA_EACH_ADC`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_1`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_2`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_3`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_1`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_2`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_3`

Notes

- If ADC multimode transfer by DMA is not selected: each ADC uses its own DMA channel, with its individual DMA transfer settings. If ADC multimode transfer by DMA is selected: One DMA channel is used for both ADC (DMA of ADC master) Specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- How to retrieve multimode conversion data: Whatever multimode transfer by DMA setting: using function `LL_ADC_REG_ReadMultiConversionData32()`. If ADC multimode transfer by DMA is selected: conversion data is a raw data with ADC master and slave concatenated. A macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`.

Reference Manual to LL API cross reference:

- CCR MDMA `LL_ADC_GetMultiDMATransfer`
- CCR DDS `LL_ADC_GetMultiDMATransfer`

`LL_ADC_SetMultiTwoSamplingDelay`

Function name

`__STATIC_INLINE void LL_ADC_SetMultiTwoSamplingDelay (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t MultiTwoSamplingDelay)`

Function description

Set ADC multimode delay between 2 sampling phases.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **MultiTwoSamplingDelay:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES`

Return values

- **None:**

Notes

- The sampling delay range depends on ADC resolution: ADC resolution 12 bits can have maximum delay of 12 cycles. ADC resolution 10 bits can have maximum delay of 10 cycles. ADC resolution 8 bits can have maximum delay of 8 cycles. ADC resolution 6 bits can have maximum delay of 6 cycles.

Reference Manual to LL API cross reference:

- CCR DELAY LL_ADC_SetMultiTwoSamplingDelay
LL_ADC_GetMultiTwoSamplingDelay

Function name

__STATIC_INLINE uint32_t LL_ADC_GetMultiTwoSamplingDelay (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get ADC multimode delay between 2 sampling phases.

Parameters

- ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- Returned:** value can be one of the following values:
 - LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES

Reference Manual to LL API cross reference:

- CCR DELAY LL_ADC_GetMultiTwoSamplingDelay
LL_ADC_Enable

Function name

__STATIC_INLINE void LL_ADC_Enable (ADC_TypeDef * ADCx)

Function description

Enable the selected ADC instance.

Parameters

- ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 serie, after ADC enable, a delay for ADC internal analog stabilization is required before performing a ADC conversion start. Refer to device datasheet, parameter tSTAB.

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_Enable
LL_ADC_Disable

Function name

__STATIC_INLINE void LL_ADC_Disable (ADC_TypeDef * ADCx)

Function description

Disable the selected ADC instance.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_Disable
LL_ADC_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_ADC_IsEnabled (ADC_TypeDef * ADCx)

Function description

Get the selected ADC instance enable state.

Parameters

- **ADCx:** ADC instance

Return values

- **0:** ADC is disabled, **1:** ADC is enabled.

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_IsEnabled
LL_ADC_REG_StartConversionSWStart

Function name

__STATIC_INLINE void LL_ADC_REG_StartConversionSWStart (ADC_TypeDef * ADCx)

Function description

Start ADC group regular conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 serie, this function is relevant only for internal trigger (SW start), not for external trigger: If ADC trigger has been set to software start, ADC conversion starts immediately. If ADC trigger has been set to external trigger, ADC conversion start must be performed using function `LL_ADC_REG_StartConversionExtTrig()`. (if external trigger edge would have been set during ADC other settings, ADC conversion would start at trigger event as soon as ADC is enabled).

Reference Manual to LL API cross reference:

- CR2 SWSTART `LL_ADC_REG_StartConversionSWStart`

`LL_ADC_REG_StartConversionExtTrig`

Function name

`__STATIC_INLINE void LL_ADC_REG_StartConversionExtTrig (ADC_TypeDef * ADCx, uint32_t ExternalTriggerEdge)`

Function description

Start ADC group regular conversion from external trigger.

Parameters

- ExternalTriggerEdge:** This parameter can be one of the following values:
 - `LL_ADC_REG_TRIG_EXT_RISING`
 - `LL_ADC_REG_TRIG_EXT_FALLING`
 - `LL_ADC_REG_TRIG_EXT_RISINGFALLING`
- ADCx:** ADC instance

Return values

- None:**

Notes

- ADC conversion will start at next trigger event (on the selected trigger edge) following the ADC start conversion command.
- On this STM32 serie, this function is relevant for ADC conversion start from external trigger. If internal trigger (SW start) is needed, perform ADC conversion start using function `LL_ADC_REG_StartConversionSWStart()`.

Reference Manual to LL API cross reference:

- CR2 EXTEN `LL_ADC_REG_StartConversionExtTrig`

`LL_ADC_REG_StopConversionExtTrig`

Function name

`__STATIC_INLINE void LL_ADC_REG_StopConversionExtTrig (ADC_TypeDef * ADCx)`

Function description

Stop ADC group regular conversion from external trigger.

Parameters

- ADCx:** ADC instance

Return values

- None:**

Notes

- No more ADC conversion will start at next trigger event following the ADC stop conversion command. If a conversion is on-going, it will be completed.
- On this STM32 serie, there is no specific command to stop a conversion on-going or to stop ADC converting in continuous mode. These actions can be performed using function LL_ADC_Disable().

Reference Manual to LL API cross reference:

- CR2_EXTEN LL_ADC_REG_StopConversionExtTrig
LL_ADC_REG_ReadConversionData32

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_ReadConversionData32 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for all ADC configurations: all ADC resolutions and all oversampling increased data width (for devices with feature oversampling).

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x00000000 and Max_Data=0xFFFFFFFF

Reference Manual to LL API cross reference:

- DR_RDATA LL_ADC_REG_ReadConversionData32
LL_ADC_REG_ReadConversionData12

Function name

__STATIC_INLINE uint16_t LL_ADC_REG_ReadConversionData12 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 12 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR_RDATA LL_ADC_REG_ReadConversionData12
LL_ADC_REG_ReadConversionData10

Function name

__STATIC_INLINE uint16_t LL_ADC_REG_ReadConversionData10 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 10 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0x3FF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData10
LL_ADC_REG_ReadConversionData8

Function name

__STATIC_INLINE uint8_t LL_ADC_REG_ReadConversionData8 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 8 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData8
LL_ADC_REG_ReadConversionData6

Function name

__STATIC_INLINE uint8_t LL_ADC_REG_ReadConversionData6 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 6 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x3F

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData6
LL_ADC_REG_ReadMultiConversionData32

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_ReadMultiConversionData32 (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t ConversionData)

Function description

Get ADC multimode conversion data of ADC master, ADC slave or raw data with ADC master and slave concatenated.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **ConversionData:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_MASTER`
 - `LL_ADC_MULTI_SLAVE`
 - `LL_ADC_MULTI_MASTER_SLAVE`

Return values

- **Value:** between `Min_Data=0x00000000` and `Max_Data=0xFFFFFFFF`

Notes

- If raw data with ADC master and slave concatenated is retrieved, a macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`. (however this macro is mainly intended for multimode transfer by DMA, because this function can do the same by getting multimode conversion data of ADC master or ADC slave separately).

Reference Manual to LL API cross reference:

- CDR DATA1 `LL_ADC_REG_ReadMultiConversionData32`
- CDR DATA2 `LL_ADC_REG_ReadMultiConversionData32`

`LL_ADC_INJ_StartConversionSWStart`

Function name

`__STATIC_INLINE void LL_ADC_INJ_StartConversionSWStart (ADC_TypeDef * ADCx)`

Function description

Start ADC group injected conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 serie, this function is relevant only for internal trigger (SW start), not for external trigger: If ADC trigger has been set to software start, ADC conversion starts immediately. If ADC trigger has been set to external trigger, ADC conversion start must be performed using function `LL_ADC_INJ_StartConversionExtTrig()`. (if external trigger edge would have been set during ADC other settings, ADC conversion would start at trigger event as soon as ADC is enabled).

Reference Manual to LL API cross reference:

- CR2 JSWSTART `LL_ADC_INJ_StartConversionSWStart`

`LL_ADC_INJ_StartConversionExtTrig`

Function name

`__STATIC_INLINE void LL_ADC_INJ_StartConversionExtTrig (ADC_TypeDef * ADCx, uint32_t ExternalTriggerEdge)`

Function description

Start ADC group injected conversion from external trigger.

Parameters

- **ExternalTriggerEdge:** This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_EXT_RISING
 - LL_ADC_INJ_TRIG_EXT_FALLING
 - LL_ADC_INJ_TRIG_EXT_RISINGFALLING
- **ADCx:** ADC instance

Return values

- **None:**

Notes

- ADC conversion will start at next trigger event (on the selected trigger edge) following the ADC start conversion command.
- On this STM32 serie, this function is relevant for ADC conversion start from external trigger. If internal trigger (SW start) is needed, perform ADC conversion start using function LL_ADC_INJ_StartConversionSWStart().

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_StartConversionExtTrig

LL_ADC_INJ_StopConversionExtTrig

Function name

__STATIC_INLINE void LL_ADC_INJ_StopConversionExtTrig (ADC_TypeDef * ADCx)

Function description

Stop ADC group injected conversion from external trigger.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- No more ADC conversion will start at next trigger event following the ADC stop conversion command. If a conversion is on-going, it will be completed.
- On this STM32 serie, there is no specific command to stop a conversion on-going or to stop ADC converting in continuous mode. These actions can be performed using function LL_ADC_Disable().

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_StopConversionExtTrig

LL_ADC_INJ_ReadConversionData32

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_ReadConversionData32 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group regular conversion data, range fit for all ADC configurations: all ADC resolutions and all oversampling increased data width (for devices with feature oversampling).

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00000000 and Max_Data=0xFFFFFFFF

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData32
- JDR2 JDATA LL_ADC_INJ_ReadConversionData32
- JDR3 JDATA LL_ADC_INJ_ReadConversionData32
- JDR4 JDATA LL_ADC_INJ_ReadConversionData32

LL_ADC_INJ_ReadConversionData12

Function name

__STATIC_INLINE uint16_t LL_ADC_INJ_ReadConversionData12 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 12 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData12
- JDR2 JDATA LL_ADC_INJ_ReadConversionData12
- JDR3 JDATA LL_ADC_INJ_ReadConversionData12
- JDR4 JDATA LL_ADC_INJ_ReadConversionData12

LL_ADC_INJ_ReadConversionData10

Function name

__STATIC_INLINE uint16_t LL_ADC_INJ_ReadConversionData10 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 10 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0x3FF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData10
- JDR2 JDATA LL_ADC_INJ_ReadConversionData10
- JDR3 JDATA LL_ADC_INJ_ReadConversionData10
- JDR4 JDATA LL_ADC_INJ_ReadConversionData10

LL_ADC_INJ_ReadConversionData8

Function name

__STATIC_INLINE uint8_t LL_ADC_INJ_ReadConversionData8 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 8 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData8
- JDR2 JDATA LL_ADC_INJ_ReadConversionData8
- JDR3 JDATA LL_ADC_INJ_ReadConversionData8
- JDR4 JDATA LL_ADC_INJ_ReadConversionData8

LL_ADC_INJ_ReadConversionData6

Function name

__STATIC_INLINE uint8_t LL_ADC_INJ_ReadConversionData6 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 6 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x3F

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData6
- JDR2 JDATA LL_ADC_INJ_ReadConversionData6
- JDR3 JDATA LL_ADC_INJ_ReadConversionData6
- JDR4 JDATA LL_ADC_INJ_ReadConversionData6

LL_ADC_IsActiveFlag_EOCS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_EOCS (ADC_TypeDef * ADCx)

Function description

Get flag ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- SR EOC LL_ADC_IsActiveFlag_EOCS

LL_ADC_IsActiveFlag_OVR

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_OVR (ADC_TypeDef * ADCx)

Function description

Get flag ADC group regular overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR OVR LL_ADC_IsActiveFlag_OVR
LL_ADC_IsActiveFlag_JEOS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_JEOS (ADC_TypeDef * ADCx)

Function description

Get flag ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR JEOC LL_ADC_IsActiveFlag_JEOS
LL_ADC_IsActiveFlag_AWD1

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_AWD1 (ADC_TypeDef * ADCx)

Function description

Get flag ADC analog watchdog 1 flag.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR AWD LL_ADC_IsActiveFlag_AWD1
LL_ADC_ClearFlag_EOCS

Function name

__STATIC_INLINE void LL_ADC_ClearFlag_EOCS (ADC_TypeDef * ADCx)

Function description

Clear flag ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- SR EOC LL_ADC_ClearFlag_EOCS
LL_ADC_ClearFlag_OVR

Function name

```
__STATIC_INLINE void LL_ADC_ClearFlag_OVR (ADC_TypeDef * ADCx)
```

Function description

Clear flag ADC group regular overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR OVR LL_ADC_ClearFlag_OVR
LL_ADC_ClearFlag_JEOS

Function name

```
__STATIC_INLINE void LL_ADC_ClearFlag_JEOS (ADC_TypeDef * ADCx)
```

Function description

Clear flag ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR JEOP LL_ADC_ClearFlag_JEOS
LL_ADC_ClearFlag_AWD1

Function name

```
__STATIC_INLINE void LL_ADC_ClearFlag_AWD1 (ADC_TypeDef * ADCx)
```

Function description

Clear flag ADC analog watchdog 1.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR AWD LL_ADC_ClearFlag_AWD1
LL_ADC_IsActiveFlag_MST_EOCS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_EOCS (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC1 `LL_ADC_IsActiveFlag_MST_EOCS`

`LL_ADC_IsActiveFlag_SLV1_EOCS`

Function name

`__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_EOCS (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC2 `LL_ADC_IsActiveFlag_SLV1_EOCS`

`LL_ADC_IsActiveFlag_SLV2_EOCS`

Function name

`__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_EOCS (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC3 `LL_ADC_IsActiveFlag_SLV2_EOCS`
`LL_ADC_IsActiveFlag_MST_OVR`

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_OVR (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group regular overrun of the ADC master.

Parameters

- ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR1 `LL_ADC_IsActiveFlag_MST_OVR`
`LL_ADC_IsActiveFlag_SLV1_OVR`

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_OVR (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group regular overrun of the ADC slave 1.

Parameters

- ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR2 `LL_ADC_IsActiveFlag_SLV1_OVR`
`LL_ADC_IsActiveFlag_SLV2_OVR`

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_OVR (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group regular overrun of the ADC slave 2.

Parameters

- ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR3 LL_ADC_IsActiveFlag_SLV2_OVR
LL_ADC_IsActiveFlag_MST_JEOS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_JEOS (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JEOC LL_ADC_IsActiveFlag_MST_EOCS
LL_ADC_IsActiveFlag_SLV1_JEOS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_JEOS (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JEOC2 LL_ADC_IsActiveFlag_SLV1_JEOS
LL_ADC_IsActiveFlag_SLV2_JEOS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_JEOS (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JEOC3 LL_ADC_IsActiveFlag_SLV2_JEOS

```
LL_ADC_IsActiveFlag_MST_AWD1
```

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_AWD1 (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode ADC analog watchdog 1 of the ADC master.

Parameters

- **ADCxy_COMMON**: ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD1 LL_ADC_IsActiveFlag_MST_AWD1

```
LL_ADC_IsActiveFlag_SLV1_AWD1
```

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_AWD1 (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode analog watchdog 1 of the ADC slave 1.

Parameters

- **ADCxy_COMMON**: ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD2 LL_ADC_IsActiveFlag_SLV1_AWD1

```
LL_ADC_IsActiveFlag_SLV2_AWD1
```

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_AWD1 (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode analog watchdog 1 of the ADC slave 2.

Parameters

- **ADCxy_COMMON**: ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD3 LL_ADC_IsActiveFlag_SLV2_AWD1

```
LL_ADC_EnableIT_EOCS
```


Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_EOCS (ADC_TypeDef * ADCx)
```

Function description

Enable interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- CR1 EOCIE LL_ADC_EnableIT_EOCS

```
LL_ADC_EnableIT_OVR
```

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_OVR (ADC_TypeDef * ADCx)
```

Function description

Enable ADC group regular interruption overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OVRIE LL_ADC_EnableIT_OVR

```
LL_ADC_EnableIT_JEOS
```

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_JEOS (ADC_TypeDef * ADCx)
```

Function description

Enable interruption ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 JEOCIE LL_ADC_EnableIT_JEOS

```
LL_ADC_EnableIT_AWD1
```

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_AWD1 (ADC_TypeDef * ADCx)
```

Function description

Enable interruption ADC analog watchdog 1.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 AWDIE LL_ADC_EnableIT_AWD1
LL_ADC_DisableIT_EOCS

Function name

__STATIC_INLINE void LL_ADC_DisableIT_EOCS (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- CR1 EOCIE LL_ADC_DisableIT_EOCS
LL_ADC_DisableIT_OVR

Function name

__STATIC_INLINE void LL_ADC_DisableIT_OVR (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC group regular overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OVRIE LL_ADC_DisableIT_OVR
LL_ADC_DisableIT_JEOS

Function name

__STATIC_INLINE void LL_ADC_DisableIT_JEOS (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 JEOCIE LL_ADC_EnableIT_JEOS
LL_ADC_DisableIT_AWD1

Function name

__STATIC_INLINE void LL_ADC_DisableIT_AWD1 (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC analog watchdog 1.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 AWDIE LL_ADC_EnableIT_AWD1
LL_ADC_IsEnabledIT_EOCS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_EOCS (ADC_TypeDef * ADCx)

Function description

Get state of interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion(). (0: interrupt disabled, 1: interrupt enabled)

Reference Manual to LL API cross reference:

- CR1 EOCIE LL_ADC_IsEnabledIT_EOCS
LL_ADC_IsEnabledIT_OVR

Function name

__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_OVR (ADC_TypeDef * ADCx)

Function description

Get state of interruption ADC group regular overrun (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 OVR1E LL_ADC_IsEnabledIT_OVR
LL_ADC_IsEnabledIT_JEOS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_JEOS (ADC_TypeDef * ADCx)

Function description

Get state of interruption ADC group injected end of sequence conversions (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 JEO1E LL_ADC_EnableIT_JEOS
LL_ADC_IsEnabledIT_AWD1

Function name

__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_AWD1 (ADC_TypeDef * ADCx)

Function description

Get state of interruption ADC analog watchdog 1 (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 AWD1E LL_ADC_EnableIT_AWD1
LL_ADC_CommonDeInit

Function name

ErrorStatus LL_ADC_CommonDeInit (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

De-initialize registers of all ADC instances belonging to the same ADC common instance to their default reset values.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC common registers are de-initialized
 - ERROR: not applicable

```
LL_ADC_CommonInit
```

Function name

```
ErrorStatus LL_ADC_CommonInit (ADC_Common_TypeDef * ADCxy_COMMON,
LL_ADC_CommonInitTypeDef * ADC_CommonInitStruct)
```

Function description

Initialize some features of ADC common parameters (all ADC instances belonging to the same ADC common instance) and multimode (for devices with several ADC instances available).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **ADC_CommonInitStruct:** Pointer to a `LL_ADC_CommonInitTypeDef` structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC common registers are initialized
 - ERROR: ADC common registers are not initialized

Notes

- The setting of ADC common parameters is conditioned to ADC instances state: All ADC instances belonging to the same ADC common instance must be disabled.

```
LL_ADC_CommonStructInit
```

Function name

```
void LL_ADC_CommonStructInit (LL_ADC_CommonInitTypeDef * ADC_CommonInitStruct)
```

Function description

Set each `LL_ADC_CommonInitTypeDef` field to default value.

Parameters

- **ADC_CommonInitStruct:** Pointer to a `LL_ADC_CommonInitTypeDef` structure whose fields will be set to default values.

Return values

- **None:**

```
LL_ADC_DeInit
```

Function name

```
ErrorStatus LL_ADC_DeInit (ADC_TypeDef * ADCx)
```

Function description

De-initialize registers of the selected ADC instance to their default reset values.

Parameters

- **ADCx:** ADC instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are de-initialized
 - ERROR: ADC registers are not de-initialized

Notes

- To reset all ADC instances quickly (perform a hard reset), use function `LL_ADC_CommonDeInit()`.

LL_ADC_Init

Function name

ErrorStatus LL_ADC_Init (ADC_TypeDef * ADCx, LL_ADC_InitTypeDef * ADC_InitStruct)

Function description

Initialize some features of ADC instance.

Parameters

- **ADCx:** ADC instance
- **ADC_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC instance. Affects both group regular and group injected (availability of ADC group injected depends on STM32 families). Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: ADC instance .
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, some other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group regular or group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_REG_SetSequencerRanks().Set ADC channel sampling time Refer to function LL_ADC_SetChannelSamplingTime();

LL_ADC_StructInit

Function name

void LL_ADC_StructInit (LL_ADC_InitTypeDef * ADC_InitStruct)

Function description

Set each LL_ADC_InitTypeDef field to default value.

Parameters

- **ADC_InitStruct:** Pointer to a LL_ADC_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_ADC_REG_Init

Function name

ErrorStatus LL_ADC_REG_Init (ADC_TypeDef * ADCx, LL_ADC_REG_InitTypeDef * ADC_REG_InitStruct)

Function description

Initialize some features of ADC group regular.

Parameters

- **ADCx:** ADC instance
- **ADC_REG_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC group regular. Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: group regular (functions with prefix "REG").
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group regular or group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_REG_SetSequencerRanks(). Set ADC channel sampling time Refer to function LL_ADC_SetChannelSamplingTime();

```
LL_ADC_REG_StructInit
```

Function name

```
void LL_ADC_REG_StructInit (LL_ADC_REG_InitTypeDef * ADC_REG_InitStruct)
```

Function description

Set each LL_ADC_REG_InitTypeDef field to default value.

Parameters

- **ADC_REG_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

```
LL_ADC_INJ_Init
```

Function name

```
ErrorStatus LL_ADC_INJ_Init (ADC_TypeDef * ADCx, LL_ADC_INJ_InitTypeDef * ADC_INJ_InitStruct)
```

Function description

Initialize some features of ADC group injected.

Parameters

- **ADCx:** ADC instance
- **ADC_INJ_InitStruct:** Pointer to a LL_ADC_INJ_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC group injected. Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: group regular (functions with prefix "INJ").
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_INJ_SetSequencerRanks(). Set ADC channel sampling time. Refer to function LL_ADC_SetChannelSamplingTime();

```
LL_ADC_INJ_StructInit
```

Function name

```
void LL_ADC_INJ_StructInit (LL_ADC_INJ_InitTypeDef * ADC_INJ_InitStruct)
```

Function description

Set each LL_ADC_INJ_InitTypeDef field to default value.

Parameters

- **ADC_INJ_InitStruct:** Pointer to a LL_ADC_INJ_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

49.3 ADC Firmware driver defines

The following section lists the various define and macros of the module.

49.3.1 ADC

ADC

Analog watchdog - Monitored channels

LL_ADC_AWD_DISABLE

ADC analog watchdog monitoring disabled

LL_ADC_AWD_ALL_CHANNELS_REG

ADC analog watchdog monitoring of all channels, converted by group regular only

LL_ADC_AWD_ALL_CHANNELS_INJ

ADC analog watchdog monitoring of all channels, converted by group injected only

LL_ADC_AWD_ALL_CHANNELS_REG_INJ

ADC analog watchdog monitoring of all channels, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_0_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by group regular only

LL_ADC_AWD_CHANNEL_0_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by group injected only

LL_ADC_AWD_CHANNEL_0_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_1_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by group regular only

LL_ADC_AWD_CHANNEL_1_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by group injected only

LL_ADC_AWD_CHANNEL_1_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_2_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by group regular only

LL_ADC_AWD_CHANNEL_2_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by group injected only

LL_ADC_AWD_CHANNEL_2_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_3_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by group regular only

LL_ADC_AWD_CHANNEL_3_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by group injected only

LL_ADC_AWD_CHANNEL_3_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_4_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by group regular only

LL_ADC_AWD_CHANNEL_4_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by group injected only

LL_ADC_AWD_CHANNEL_4_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_5_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by group regular only

LL_ADC_AWD_CHANNEL_5_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by group injected only

LL_ADC_AWD_CHANNEL_5_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_6_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by group regular only

LL_ADC_AWD_CHANNEL_6_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by group injected only

LL_ADC_AWD_CHANNEL_6_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_7_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by group regular only

LL_ADC_AWD_CHANNEL_7_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by group injected only

LL_ADC_AWD_CHANNEL_7_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_8_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by group regular only

LL_ADC_AWD_CHANNEL_8_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by group injected only

LL_ADC_AWD_CHANNEL_8_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_9_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by group regular only

LL_ADC_AWD_CHANNEL_9_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by group injected only

LL_ADC_AWD_CHANNEL_9_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_10_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by group regular only

LL_ADC_AWD_CHANNEL_10_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by group injected only

LL_ADC_AWD_CHANNEL_10_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_11_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by group regular only

LL_ADC_AWD_CHANNEL_11_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by group injected only

LL_ADC_AWD_CHANNEL_11_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_12_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by group regular only

LL_ADC_AWD_CHANNEL_12_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by group injected only

LL_ADC_AWD_CHANNEL_12_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_13_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by group regular only

LL_ADC_AWD_CHANNEL_13_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by group injected only

LL_ADC_AWD_CHANNEL_13_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_14_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by group regular only

LL_ADC_AWD_CHANNEL_14_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by group injected only

LL_ADC_AWD_CHANNEL_14_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_15_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by group regular only

LL_ADC_AWD_CHANNEL_15_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by group injected only

LL_ADC_AWD_CHANNEL_15_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_16_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by group regular only

LL_ADC_AWD_CHANNEL_16_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by group injected only

LL_ADC_AWD_CHANNEL_16_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_17_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by group regular only

LL_ADC_AWD_CHANNEL_17_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by group injected only

LL_ADC_AWD_CHANNEL_17_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_18_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by group regular only

LL_ADC_AWD_CHANNEL_18_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by group injected only

LL_ADC_AWD_CHANNEL_18_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by either group regular or injected

LL_ADC_AWD_CH_VREFINT_REG

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by group regular only

LL_ADC_AWD_CH_VREFINT_INJ

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by group injected only

LL_ADC_AWD_CH_VREFINT_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by either group regular or injected

LL_ADC_AWD_CH_VBAT_REG

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda, converted by group regular only

LL_ADC_AWD_CH_VBAT_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda, converted by group injected only

LL_ADC_AWD_CH_VBAT_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda

LL_ADC_AWD_CH_TEMPSENSOR_REG

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by group regular only

LL_ADC_AWD_CH_TEMPSENSOR_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by group injected only

LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by either group regular or injected

Analog watchdog - Analog watchdog number

LL_ADC_AWD1

ADC analog watchdog number 1

Analog watchdog - Thresholds

LL_ADC_AWD_THRESHOLD_HIGH

ADC analog watchdog threshold high

LL_ADC_AWD_THRESHOLD_LOW

ADC analog watchdog threshold low

ADC instance - Channel number

LL_ADC_CHANNEL_0

ADC external channel (channel connected to GPIO pin) ADCx_IN0

LL_ADC_CHANNEL_1

ADC external channel (channel connected to GPIO pin) ADCx_IN1

LL_ADC_CHANNEL_2

ADC external channel (channel connected to GPIO pin) ADCx_IN2

LL_ADC_CHANNEL_3

ADC external channel (channel connected to GPIO pin) ADCx_IN3

LL_ADC_CHANNEL_4

ADC external channel (channel connected to GPIO pin) ADCx_IN4

LL_ADC_CHANNEL_5

ADC external channel (channel connected to GPIO pin) ADCx_IN5

LL_ADC_CHANNEL_6

ADC external channel (channel connected to GPIO pin) ADCx_IN6

LL_ADC_CHANNEL_7

ADC external channel (channel connected to GPIO pin) ADCx_IN7

LL_ADC_CHANNEL_8

ADC external channel (channel connected to GPIO pin) ADCx_IN8

LL_ADC_CHANNEL_9

ADC external channel (channel connected to GPIO pin) ADCx_IN9

LL_ADC_CHANNEL_10

ADC external channel (channel connected to GPIO pin) ADCx_IN10

LL_ADC_CHANNEL_11

ADC external channel (channel connected to GPIO pin) ADCx_IN11

LL_ADC_CHANNEL_12

ADC external channel (channel connected to GPIO pin) ADCx_IN12

LL_ADC_CHANNEL_13

ADC external channel (channel connected to GPIO pin) ADCx_IN13

LL_ADC_CHANNEL_14

ADC external channel (channel connected to GPIO pin) ADCx_IN14

LL_ADC_CHANNEL_15

ADC external channel (channel connected to GPIO pin) ADCx_IN15

LL_ADC_CHANNEL_16

ADC external channel (channel connected to GPIO pin) ADCx_IN16

LL_ADC_CHANNEL_17

ADC external channel (channel connected to GPIO pin) ADCx_IN17

LL_ADC_CHANNEL_18

ADC external channel (channel connected to GPIO pin) ADCx_IN18

LL_ADC_CHANNEL_VREFINT

ADC internal channel connected to VrefInt: Internal voltage reference. On STM32F2, ADC channel available only on ADC instance: ADC1.

LL_ADC_CHANNEL_VBAT

ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda. On STM32F2, ADC channel available only on ADC instance: ADC1.

LL_ADC_CHANNEL_TEMPSENSOR

ADC internal channel connected to Temperature sensor. On STM32F2, ADC channel available only on ADC instance: ADC1.

Channel - Sampling time

LL_ADC_SAMPLINGTIME_3CYCLES

Sampling time 3 ADC clock cycles

LL_ADC_SAMPLINGTIME_15CYCLES

Sampling time 15 ADC clock cycles

LL_ADC_SAMPLINGTIME_28CYCLES

Sampling time 28 ADC clock cycles

LL_ADC_SAMPLINGTIME_56CYCLES

Sampling time 56 ADC clock cycles

LL_ADC_SAMPLINGTIME_84CYCLES

Sampling time 84 ADC clock cycles

LL_ADC_SAMPLINGTIME_112CYCLES

Sampling time 112 ADC clock cycles

LL_ADC_SAMPLINGTIME_144CYCLES

Sampling time 144 ADC clock cycles

LL_ADC_SAMPLINGTIME_480CYCLES

Sampling time 480 ADC clock cycles

ADC common - Clock source

LL_ADC_CLOCK_SYNC_PCLK_DIV2

ADC synchronous clock derived from AHB clock with prescaler division by 2

LL_ADC_CLOCK_SYNC_PCLK_DIV4

ADC synchronous clock derived from AHB clock with prescaler division by 4

LL_ADC_CLOCK_SYNC_PCLK_DIV6

ADC synchronous clock derived from AHB clock with prescaler division by 6

LL_ADC_CLOCK_SYNC_PCLK_DIV8

ADC synchronous clock derived from AHB clock with prescaler division by 8

ADC common - Measurement path to internal channels

LL_ADC_PATH_INTERNAL_NONE

ADC measurement pathes all disabled

LL_ADC_PATH_INTERNAL_VREFINT

ADC measurement path to internal channel VrefInt

LL_ADC_PATH_INTERNAL_TEMPSENSOR

ADC measurement path to internal channel temperature sensor

LL_ADC_PATH_INTERNAL_VBAT

ADC measurement path to internal channel Vbat

ADC instance - Data alignment

LL_ADC_DATA_ALIGN_RIGHT

ADC conversion data alignment: right aligned (alignment on data register LSB bit 0)

LL_ADC_DATA_ALIGN_LEFT

ADC conversion data alignment: left aligned (alignment on data register MSB bit 15)

ADC flags

LL_ADC_FLAG_STRT

ADC flag ADC group regular conversion start

LL_ADC_FLAG_EOCS

ADC flag ADC group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_OVR

ADC flag ADC group regular overrun

LL_ADC_FLAG_JSTRT

ADC flag ADC group injected conversion start

LL_ADC_FLAG_JEOS

ADC flag ADC group injected end of sequence conversions (Note: on this STM32 serie, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_AWD1

ADC flag ADC analog watchdog 1

LL_ADC_FLAG_EOCS_MST

ADC flag ADC multimode master group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_EOCS_SLV1

ADC flag ADC multimode slave 1 group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_EOCS_SLV2

ADC flag ADC multimode slave 2 group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_OVR_MST

ADC flag ADC multimode master group regular overrun

LL_ADC_FLAG_OVR_SLV1

ADC flag ADC multimode slave 1 group regular overrun

LL_ADC_FLAG_OVR_SLV2

ADC flag ADC multimode slave 2 group regular overrun

LL_ADC_FLAG_JEOS_MST

ADC flag ADC multimode master group injected end of sequence conversions (Note: on this STM32 serie, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_JEOS_SLV1

ADC flag ADC multimode slave 1 group injected end of sequence conversions (Note: on this STM32 serie, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_JEOS_SLV2

ADC flag ADC multimode slave 2 group injected end of sequence conversions (Note: on this STM32 serie, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_AWD1_MST

ADC flag ADC multimode master analog watchdog 1 of the ADC master

LL_ADC_FLAG_AWD1_SLV1

ADC flag ADC multimode slave 1 analog watchdog 1

LL_ADC_FLAG_AWD1_SLV2

ADC flag ADC multimode slave 2 analog watchdog 1

ADC instance - Groups

LL_ADC_GROUP_REGULAR

ADC group regular (available on all STM32 devices)

LL_ADC_GROUP_INJECTED

ADC group injected (not available on all STM32 devices)

LL_ADC_GROUP_REGULAR_INJECTED

ADC both groups regular and injected

Definitions of ADC hardware constraints delays

LL_ADC_DELAY_VREFINT_STAB_US

Delay for internal voltage reference stabilization time

LL_ADC_DELAY_TEMPSENSOR_STAB_US

Delay for internal voltage reference stabilization time

ADC group injected - Sequencer discontinuous mode

LL_ADC_INJ_SEQ_DISCONT_DISABLE

ADC group injected sequencer discontinuous mode disable

LL_ADC_INJ_SEQ_DISCONT_1RANK

ADC group injected sequencer discontinuous mode enable with sequence interruption every rank

ADC group injected - Sequencer ranks

LL_ADC_INJ_RANK_1

ADC group injected sequencer rank 1

LL_ADC_INJ_RANK_2

ADC group injected sequencer rank 2

LL_ADC_INJ_RANK_3

ADC group injected sequencer rank 3

LL_ADC_INJ_RANK_4

ADC group injected sequencer rank 4

ADC group injected - Sequencer scan length

LL_ADC_INJ_SEQ_SCAN_DISABLE

ADC group injected sequencer disable (equivalent to sequencer of 1 rank: ADC conversion on only 1 channel)

LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS

ADC group injected sequencer enable with 2 ranks in the sequence

LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS

ADC group injected sequencer enable with 3 ranks in the sequence

LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

ADC group injected sequencer enable with 4 ranks in the sequence

ADC group injected - Trigger edge

LL_ADC_INJ_TRIG_EXT_RISING

ADC group injected conversion trigger polarity set to rising edge

LL_ADC_INJ_TRIG_EXT_FALLING

ADC group injected conversion trigger polarity set to falling edge

LL_ADC_INJ_TRIG_EXT_RISINGFALLING

ADC group injected conversion trigger polarity set to both rising and falling edges

ADC group injected - Trigger source

LL_ADC_INJ_TRIG_SOFTWARE

ADC group injected conversion trigger internal: SW start.

LL_ADC_INJ_TRIG_EXT_TIM1_CH4

ADC group injected conversion trigger from external IP: TIM1 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM1_TRGO

ADC group injected conversion trigger from external IP: TIM1 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM2_CH1

ADC group injected conversion trigger from external IP: TIM2 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM2_TRGO

ADC group injected conversion trigger from external IP: TIM2 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM3_CH2

ADC group injected conversion trigger from external IP: TIM3 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM3_CH4

ADC group injected conversion trigger from external IP: TIM3 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH1

ADC group injected conversion trigger from external IP: TIM4 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH2

ADC group injected conversion trigger from external IP: TIM4 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH3

ADC group injected conversion trigger from external IP: TIM4 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_TRGO

ADC group injected conversion trigger from external IP: TIM4 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM5_CH4

ADC group injected conversion trigger from external IP: TIM5 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM5_TRGO

ADC group injected conversion trigger from external IP: TIM5 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH2

ADC group injected conversion trigger from external IP: TIM8 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH3

ADC group injected conversion trigger from external IP: TIM8 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH4

ADC group injected conversion trigger from external IP: TIM8 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

ADC group injected conversion trigger from external IP: external interrupt line 15. Trigger edge set to rising edge (default setting).

ADC group injected - Automatic trigger mode

LL_ADC_INJ_TRIG_INDEPENDENT

ADC group injected conversion trigger independent. Setting mandatory if ADC group injected injected trigger source is set to an external trigger.

LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

ADC group injected conversion trigger from ADC group regular. Setting compliant only with group injected trigger source set to SW start, without any further action on ADC group injected conversion start or stop: in this case, ADC group injected is controlled only from ADC group regular.

ADC interruptions for configuration (interruption enable or disable)

LL_ADC_IT_EOCS

ADC interruption ADC group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_IT_OVR

ADC interruption ADC group regular overrun

LL_ADC_IT_JEOS

ADC interruption ADC group injected end of sequence conversions (Note: on this STM32 serie, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_IT_AWD1

ADC interruption ADC analog watchdog 1

Multimode - DMA transfer

LL_ADC_MULTI_REG_DMA_EACH_ADC

ADC multimode group regular conversions are transferred by DMA: each ADC uses its own DMA channel, with its individual DMA transfer settings

LL_ADC_MULTI_REG_DMA_LIMIT_1

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 1: 2 or 3 (dual or triple mode) half-words one by one, ADC1 then ADC2 then ADC3.

LL_ADC_MULTI_REG_DMA_LIMIT_2

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 2: 2 or 3 (dual or triple mode) half-words one by one, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_LIMIT_3

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 3: 2 or 3 (dual or triple mode) bytes one by one, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_UNLMT_1

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 1: 2 or 3 (dual or triple mode) half-words one by one, ADC1 then ADC2 then ADC3.

LL_ADC_MULTI_REG_DMA_UNLMT_2

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 2: 2 or 3 (dual or triple mode) half-words by pairs, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_UNLMT_3

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 3: 2 or 3 (dual or triple mode) bytes one by one, ADC2&1 then ADC1&3 then ADC3&2.

Multimode - ADC master or slave

LL_ADC_MULTI_MASTER

In multimode, selection among several ADC instances: ADC master

LL_ADC_MULTI_SLAVE

In multimode, selection among several ADC instances: ADC slave

LL_ADC_MULTI_MASTER_SLAVE

In multimode, selection among several ADC instances: both ADC master and ADC slave

Multimode - Mode

LL_ADC_MULTI_INDEPENDENT

ADC dual mode disabled (ADC independent mode)

LL_ADC_MULTI_DUAL_REG_SIMULT

ADC dual mode enabled: group regular simultaneous

LL_ADC_MULTI_DUAL_REG_INTERL

ADC dual mode enabled: Combined group regular interleaved

LL_ADC_MULTI_DUAL_INJ_SIMULT

ADC dual mode enabled: group injected simultaneous

LL_ADC_MULTI_DUAL_INJ_ALTERN

ADC dual mode enabled: group injected alternate trigger. Works only with external triggers (not internal SW start)

LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM

ADC dual mode enabled: Combined group regular simultaneous + group injected simultaneous

LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT

ADC dual mode enabled: Combined group regular simultaneous + group injected alternate trigger

LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM

ADC dual mode enabled: Combined group regular interleaved + group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM

ADC triple mode enabled: Combined group regular simultaneous + group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT

ADC triple mode enabled: Combined group regular simultaneous + group injected alternate trigger

LL_ADC_MULTI_TRIPLE_INJ_SIMULT

ADC triple mode enabled: group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIMULT

ADC triple mode enabled: group regular simultaneous

LL_ADC_MULTI_TRIPLE_REG_INTERL

ADC triple mode enabled: Combined group regular interleaved

LL_ADC_MULTI_TRIPLE_INJ_ALTERN

ADC triple mode enabled: group injected alternate trigger. Works only with external triggers (not internal SW start)

Multimode - Delay between two sampling phases

LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES

ADC multimode delay between two sampling phases: 5 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES

ADC multimode delay between two sampling phases: 6 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES

ADC multimode delay between two sampling phases: 7 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES

ADC multimode delay between two sampling phases: 8 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES

ADC multimode delay between two sampling phases: 9 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES

ADC multimode delay between two sampling phases: 10 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES

ADC multimode delay between two sampling phases: 11 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES

ADC multimode delay between two sampling phases: 12 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES

ADC multimode delay between two sampling phases: 13 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES

ADC multimode delay between two sampling phases: 14 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES

ADC multimode delay between two sampling phases: 15 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES

ADC multimode delay between two sampling phases: 16 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES

ADC multimode delay between two sampling phases: 17 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES

ADC multimode delay between two sampling phases: 18 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES

ADC multimode delay between two sampling phases: 19 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES

ADC multimode delay between two sampling phases: 20 ADC clock cycles

ADC registers compliant with specific purpose

LL_ADC_DMA_REG_REGULAR_DATA

LL_ADC_DMA_REG_REGULAR_DATA_MULTI

ADC group regular - Continuous mode

LL_ADC_REG_CONV_SINGLE

ADC conversions are performed in single mode: one conversion per trigger

LL_ADC_REG_CONV_CONTINUOUS

ADC conversions are performed in continuous mode: after the first trigger, following conversions launched successively automatically

ADC group regular - DMA transfer of ADC conversion data

LL_ADC_REG_DMA_TRANSFER_NONE

ADC conversions are not transferred by DMA

LL_ADC_REG_DMA_TRANSFER_LIMITED

ADC conversion data are transferred by DMA, in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular.

LL_ADC_REG_DMA_TRANSFER_UNLIMITED

ADC conversion data are transferred by DMA, in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.

ADC group regular - Flag EOC selection (unitary or sequence conversions)**LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV**

ADC flag EOC (end of unitary conversion) selected

LL_ADC_REG_FLAG_EOC_UNITARY_CONV

ADC flag EOS (end of sequence conversions) selected

ADC group regular - Sequencer discontinuous mode**LL_ADC_REG_SEQ_DISCONT_DISABLE**

ADC group regular sequencer discontinuous mode disable

LL_ADC_REG_SEQ_DISCONT_1RANK

ADC group regular sequencer discontinuous mode enable with sequence interruption every rank

LL_ADC_REG_SEQ_DISCONT_2RANKS

ADC group regular sequencer discontinuous mode enabled with sequence interruption every 2 ranks

LL_ADC_REG_SEQ_DISCONT_3RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 3 ranks

LL_ADC_REG_SEQ_DISCONT_4RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 4 ranks

LL_ADC_REG_SEQ_DISCONT_5RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 5 ranks

LL_ADC_REG_SEQ_DISCONT_6RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 6 ranks

LL_ADC_REG_SEQ_DISCONT_7RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 7 ranks

LL_ADC_REG_SEQ_DISCONT_8RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 8 ranks

ADC group regular - Sequencer ranks**LL_ADC_REG_RANK_1**

ADC group regular sequencer rank 1

LL_ADC_REG_RANK_2

ADC group regular sequencer rank 2

LL_ADC_REG_RANK_3

ADC group regular sequencer rank 3

LL_ADC_REG_RANK_4

ADC group regular sequencer rank 4

LL_ADC_REG_RANK_5

ADC group regular sequencer rank 5

LL_ADC_REG_RANK_6

ADC group regular sequencer rank 6

LL_ADC_REG_RANK_7

ADC group regular sequencer rank 7

LL_ADC_REG_RANK_8

ADC group regular sequencer rank 8

LL_ADC_REG_RANK_9

ADC group regular sequencer rank 9

LL_ADC_REG_RANK_10

ADC group regular sequencer rank 10

LL_ADC_REG_RANK_11

ADC group regular sequencer rank 11

LL_ADC_REG_RANK_12

ADC group regular sequencer rank 12

LL_ADC_REG_RANK_13

ADC group regular sequencer rank 13

LL_ADC_REG_RANK_14

ADC group regular sequencer rank 14

LL_ADC_REG_RANK_15

ADC group regular sequencer rank 15

LL_ADC_REG_RANK_16

ADC group regular sequencer rank 16

ADC group regular - Sequencer scan length**LL_ADC_REG_SEQ_SCAN_DISABLE**

ADC group regular sequencer disable (equivalent to sequencer of 1 rank: ADC conversion on only 1 channel)

LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS

ADC group regular sequencer enable with 2 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS

ADC group regular sequencer enable with 3 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS

ADC group regular sequencer enable with 4 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS

ADC group regular sequencer enable with 5 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS

ADC group regular sequencer enable with 6 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS

ADC group regular sequencer enable with 7 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS

ADC group regular sequencer enable with 8 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS

ADC group regular sequencer enable with 9 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS

ADC group regular sequencer enable with 10 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS

ADC group regular sequencer enable with 11 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS

ADC group regular sequencer enable with 12 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS

ADC group regular sequencer enable with 13 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS

ADC group regular sequencer enable with 14 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS

ADC group regular sequencer enable with 15 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

ADC group regular sequencer enable with 16 ranks in the sequence

ADC group regular - Trigger edge

LL_ADC_REG_TRIG_EXT_RISING

ADC group regular conversion trigger polarity set to rising edge

LL_ADC_REG_TRIG_EXT_FALLING

ADC group regular conversion trigger polarity set to falling edge

LL_ADC_REG_TRIG_EXT_RISINGFALLING

ADC group regular conversion trigger polarity set to both rising and falling edges

ADC group regular - Trigger source

LL_ADC_REG_TRIG_SOFTWARE

ADC group regular conversion trigger internal: SW start.

LL_ADC_REG_TRIG_EXT_TIM1_CH1

ADC group regular conversion trigger from external IP: TIM1 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM1_CH2

ADC group regular conversion trigger from external IP: TIM1 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM1_CH3

ADC group regular conversion trigger from external IP: TIM1 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH2

ADC group regular conversion trigger from external IP: TIM2 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH3

ADC group regular conversion trigger from external IP: TIM2 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH4

ADC group regular conversion trigger from external IP: TIM2 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_TRGO

ADC group regular conversion trigger from external IP: TIM2 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM3_CH1

ADC group regular conversion trigger from external IP: TIM3 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM3_TRGO

ADC group regular conversion trigger from external IP: TIM3 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM4_CH4

ADC group regular conversion trigger from external IP: TIM4 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH1

ADC group regular conversion trigger from external IP: TIM5 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH2

ADC group regular conversion trigger from external IP: TIM5 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH3

ADC group regular conversion trigger from external IP: TIM5 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM8_CH1

ADC group regular conversion trigger from external IP: TIM8 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM8_TRGO

ADC group regular conversion trigger from external IP: TIM8 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_EXTI_LINE11

ADC group regular conversion trigger from external IP: external interrupt line 11. Trigger edge set to rising edge (default setting).

ADC instance - Resolution

LL_ADC_RESOLUTION_12B

ADC resolution 12 bits

LL_ADC_RESOLUTION_10B

ADC resolution 10 bits

LL_ADC_RESOLUTION_8B

ADC resolution 8 bits

LL_ADC_RESOLUTION_6B

ADC resolution 6 bits

ADC instance - Scan selection

LL_ADC_SEQ_SCAN_DISABLE

ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of both groups regular and injected sequencers (sequence length, ...) is discarded: equivalent to length of 1 rank.

LL_ADC_SEQ_SCAN_ENABLE

ADC conversions are performed in sequence conversions mode, according to configuration of both groups regular and injected sequencers (sequence length, ...).

ADC helper macro

__LL_ADC_CHANNEL_TO_DECIMAL_NB

Description:

- Helper macro to get ADC channel number in decimal format from literals LL_ADC_CHANNEL_x.

Parameters:

- **__CHANNEL__**: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Value: between Min_Data=0 and Max_Data=18

Notes:

- Example: __LL_ADC_CHANNEL_TO_DECIMAL_NB(LL_ADC_CHANNEL_4) will return decimal number "4". The input can be a value from functions where a channel number is returned, either defined with number or with bitfield (only one bit must be set).

__LL_ADC_DECIMAL_NB_TO_CHANNEL

Description:

- Helper macro to get ADC channel in literal format LL_ADC_CHANNEL_x from number in decimal format.

Parameters:

- __DECIMAL_NB__: Value between Min_Data=0 and Max_Data=18

Return value:

- Returned: value can be one of the following values:

- LL_ADC_CHANNEL_0
- LL_ADC_CHANNEL_1
- LL_ADC_CHANNEL_2
- LL_ADC_CHANNEL_3
- LL_ADC_CHANNEL_4
- LL_ADC_CHANNEL_5
- LL_ADC_CHANNEL_6
- LL_ADC_CHANNEL_7
- LL_ADC_CHANNEL_8
- LL_ADC_CHANNEL_9
- LL_ADC_CHANNEL_10
- LL_ADC_CHANNEL_11
- LL_ADC_CHANNEL_12
- LL_ADC_CHANNEL_13
- LL_ADC_CHANNEL_14
- LL_ADC_CHANNEL_15
- LL_ADC_CHANNEL_16
- LL_ADC_CHANNEL_17
- LL_ADC_CHANNEL_18
- LL_ADC_CHANNEL_VREFINT (1)
- LL_ADC_CHANNEL_TEMPSENSOR (1)
- LL_ADC_CHANNEL_VBAT (1)

Notes:

- Example: __LL_ADC_DECIMAL_NB_TO_CHANNEL(4) will return a data equivalent to "LL_ADC_CHANNEL_4".

__LL_ADC_IS_CHANNEL_INTERNAL

Description:

- Helper macro to determine whether the selected channel corresponds to literal definitions of driver.

Parameters:

- `__CHANNEL__`: This parameter can be one of the following values:
 - `LL_ADC_CHANNEL_0`
 - `LL_ADC_CHANNEL_1`
 - `LL_ADC_CHANNEL_2`
 - `LL_ADC_CHANNEL_3`
 - `LL_ADC_CHANNEL_4`
 - `LL_ADC_CHANNEL_5`
 - `LL_ADC_CHANNEL_6`
 - `LL_ADC_CHANNEL_7`
 - `LL_ADC_CHANNEL_8`
 - `LL_ADC_CHANNEL_9`
 - `LL_ADC_CHANNEL_10`
 - `LL_ADC_CHANNEL_11`
 - `LL_ADC_CHANNEL_12`
 - `LL_ADC_CHANNEL_13`
 - `LL_ADC_CHANNEL_14`
 - `LL_ADC_CHANNEL_15`
 - `LL_ADC_CHANNEL_16`
 - `LL_ADC_CHANNEL_17`
 - `LL_ADC_CHANNEL_18`
 - `LL_ADC_CHANNEL_VREFINT` (1)
 - `LL_ADC_CHANNEL_TEMPSENSOR` (1)
 - `LL_ADC_CHANNEL_VBAT` (1)

Return value:

- Value: "0" if the channel corresponds to a parameter definition of a ADC external channel (channel connected to a GPIO pin). Value "1" if the channel corresponds to a parameter definition of a ADC internal channel.

Notes:

- The different literal definitions of ADC channels are: ADC internal channel: `LL_ADC_CHANNEL_VREFINT`, `LL_ADC_CHANNEL_TEMPSENSOR`, ... ADC external channel (channel connected to a GPIO pin): `LL_ADC_CHANNEL_1`, `LL_ADC_CHANNEL_2`, ... The channel parameter must be a value defined from literal definition of a ADC internal channel (`LL_ADC_CHANNEL_VREFINT`, `LL_ADC_CHANNEL_TEMPSENSOR`, ...), ADC external channel (`LL_ADC_CHANNEL_1`, `LL_ADC_CHANNEL_2`, ...), must not be a value from functions where a channel number is returned from ADC registers, because internal and external channels share the same channel number in ADC registers. The differentiation is made only with parameters definitions of driver.

__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL

Description:

- Helper macro to convert a channel defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), to its equivalent parameter definition of a ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...).

Parameters:

- __CHANNEL__: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Returned: value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17

- LL_ADC_CHANNEL_18

Notes:

- The channel parameter can be, additionally to a value defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), a value defined from parameter definition of ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...) or a value from functions where a channel number is returned from ADC registers.

__LL_ADC_IS_CHANNEL_INTERNAL_AVAILABLE

Description:

- Helper macro to determine whether the internal channel selected is available on the ADC instance selected.

Parameters:

- __ADC_INSTANCE__: ADC instance
- __CHANNEL__: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Value: "0" if the internal channel selected is not available on the ADC instance selected. Value "1" if the internal channel selected is available on the ADC instance selected.

Notes:

- The channel parameter must be a value defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), must not be a value defined from parameter definition of ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...) or a value from functions where a channel number is returned from ADC registers, because internal and external channels share the same channel number in ADC registers. The differentiation is made only with parameters definitions of driver.

__LL_ADC_ANALOGWD_CHANNEL_GROUP

Description:

- Helper macro to define ADC analog watchdog parameter: define a single channel to monitor with analog watchdog from sequencer channel and groups definition.

Parameters:

- `__CHANNEL__`: This parameter can be one of the following values:
 - `LL_ADC_CHANNEL_0`
 - `LL_ADC_CHANNEL_1`
 - `LL_ADC_CHANNEL_2`
 - `LL_ADC_CHANNEL_3`
 - `LL_ADC_CHANNEL_4`
 - `LL_ADC_CHANNEL_5`
 - `LL_ADC_CHANNEL_6`
 - `LL_ADC_CHANNEL_7`
 - `LL_ADC_CHANNEL_8`
 - `LL_ADC_CHANNEL_9`
 - `LL_ADC_CHANNEL_10`
 - `LL_ADC_CHANNEL_11`
 - `LL_ADC_CHANNEL_12`
 - `LL_ADC_CHANNEL_13`
 - `LL_ADC_CHANNEL_14`
 - `LL_ADC_CHANNEL_15`
 - `LL_ADC_CHANNEL_16`
 - `LL_ADC_CHANNEL_17`
 - `LL_ADC_CHANNEL_18`
 - `LL_ADC_CHANNEL_VREFINT` (1)
 - `LL_ADC_CHANNEL_TEMPSENSOR` (1)
 - `LL_ADC_CHANNEL_VBAT` (1)
- `__GROUP__`: This parameter can be one of the following values:
 - `LL_ADC_GROUP_REGULAR`
 - `LL_ADC_GROUP_INJECTED`
 - `LL_ADC_GROUP_REGULAR_INJECTED`

Return value:

- Returned: value can be one of the following values:
 - `LL_ADC_AWD_DISABLE`
 - `LL_ADC_AWD_ALL_CHANNELS_REG`
 - `LL_ADC_AWD_ALL_CHANNELS_INJ`
 - `LL_ADC_AWD_ALL_CHANNELS_REG_INJ`
 - `LL_ADC_AWD_CHANNEL_0_REG`
 - `LL_ADC_AWD_CHANNEL_0_INJ`
 - `LL_ADC_AWD_CHANNEL_0_REG_INJ`
 - `LL_ADC_AWD_CHANNEL_1_REG`
 - `LL_ADC_AWD_CHANNEL_1_INJ`
 - `LL_ADC_AWD_CHANNEL_1_REG_INJ`
 - `LL_ADC_AWD_CHANNEL_2_REG`
 - `LL_ADC_AWD_CHANNEL_2_INJ`
 - `LL_ADC_AWD_CHANNEL_2_REG_INJ`
 - `LL_ADC_AWD_CHANNEL_3_REG`
 - `LL_ADC_AWD_CHANNEL_3_INJ`

- LL_ADC_AWD_CHANNEL_3_REG_INJ
- LL_ADC_AWD_CHANNEL_4_REG
- LL_ADC_AWD_CHANNEL_4_INJ
- LL_ADC_AWD_CHANNEL_4_REG_INJ
- LL_ADC_AWD_CHANNEL_5_REG
- LL_ADC_AWD_CHANNEL_5_INJ
- LL_ADC_AWD_CHANNEL_5_REG_INJ
- LL_ADC_AWD_CHANNEL_6_REG
- LL_ADC_AWD_CHANNEL_6_INJ
- LL_ADC_AWD_CHANNEL_6_REG_INJ
- LL_ADC_AWD_CHANNEL_7_REG
- LL_ADC_AWD_CHANNEL_7_INJ
- LL_ADC_AWD_CHANNEL_7_REG_INJ
- LL_ADC_AWD_CHANNEL_8_REG
- LL_ADC_AWD_CHANNEL_8_INJ
- LL_ADC_AWD_CHANNEL_8_REG_INJ
- LL_ADC_AWD_CHANNEL_9_REG
- LL_ADC_AWD_CHANNEL_9_INJ
- LL_ADC_AWD_CHANNEL_9_REG_INJ
- LL_ADC_AWD_CHANNEL_10_REG
- LL_ADC_AWD_CHANNEL_10_INJ
- LL_ADC_AWD_CHANNEL_10_REG_INJ
- LL_ADC_AWD_CHANNEL_11_REG
- LL_ADC_AWD_CHANNEL_11_INJ
- LL_ADC_AWD_CHANNEL_11_REG_INJ
- LL_ADC_AWD_CHANNEL_12_REG
- LL_ADC_AWD_CHANNEL_12_INJ
- LL_ADC_AWD_CHANNEL_12_REG_INJ
- LL_ADC_AWD_CHANNEL_13_REG
- LL_ADC_AWD_CHANNEL_13_INJ
- LL_ADC_AWD_CHANNEL_13_REG_INJ
- LL_ADC_AWD_CHANNEL_14_REG
- LL_ADC_AWD_CHANNEL_14_INJ
- LL_ADC_AWD_CHANNEL_14_REG_INJ
- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ
- LL_ADC_AWD_CH_VREFINT_REG (1)
- LL_ADC_AWD_CH_VREFINT_INJ (1)
- LL_ADC_AWD_CH_VREFINT_REG_INJ (1)

- LL_ADC_AWD_CH_TEMPSENSOR_REG (1)
- LL_ADC_AWD_CH_TEMPSENSOR_INJ (1)
- LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG (1)
- LL_ADC_AWD_CH_VBAT_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG_INJ (1)

Notes:

- To be used with function LL_ADC_SetAnalogWDMonitChannels(). Example:
LL_ADC_SetAnalogWDMonitChannels(ADC1, LL_ADC_AWD1,
__LL_ADC_ANALOGWD_CHANNEL_GROUP(LL_ADC_CHANNEL4, LL_ADC_GROUP_REGULAR))

__LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION

Description:

- Helper macro to set the value of ADC analog watchdog threshold high or low in function of ADC resolution, when ADC resolution is different of 12 bits.

Parameters:

- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B
- __AWD_THRESHOLD__: Value between Min_Data=0x000 and Max_Data=0xFFFF

Return value:

- Value: between Min_Data=0x000 and Max_Data=0xFFFF

Notes:

- To be used with function LL_ADC_SetAnalogWDThresholds(). Example, with a ADC resolution of 8 bits, to set the value of analog watchdog threshold high (on 8 bits): LL_ADC_SetAnalogWDThresholds (< ADCx param >, __LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION(LL_ADC_RESOLUTION_8B, <threshold_value_8_bits>));

__LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION

Description:

- Helper macro to get the value of ADC analog watchdog threshold high or low in function of ADC resolution, when ADC resolution is different of 12 bits.

Parameters:

- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B
- __AWD_THRESHOLD_12_BITS__: Value between Min_Data=0x000 and Max_Data=0xFFFF

Return value:

- Value: between Min_Data=0x000 and Max_Data=0xFFFF

Notes:

- To be used with function LL_ADC_GetAnalogWDThresholds(). Example, with a ADC resolution of 8 bits, to get the value of analog watchdog threshold high (on 8 bits): < threshold_value_6_bits > = __LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION (LL_ADC_RESOLUTION_8B, LL_ADC_GetAnalogWDThresholds(<ADCx param>, LL_ADC_AWD_THRESHOLD_HIGH));

__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE

Description:

- Helper macro to get the ADC multimode conversion data of ADC master or ADC slave from raw value with both ADC conversion data concatenated.

Parameters:

- `__ADC_MULTI_MASTER_SLAVE__`: This parameter can be one of the following values:
 - `LL_ADC_MULTI_MASTER`
 - `LL_ADC_MULTI_SLAVE`
- `__ADC_MULTI_CONV_DATA__`: Value between `Min_Data=0x000` and `Max_Data=0xFFFF`

Return value:

- Value: between `Min_Data=0x000` and `Max_Data=0xFFFF`

Notes:

- This macro is intended to be used when multimode transfer by DMA is enabled: refer to function `LL_ADC_SetMultiDMATransfer()`. In this case the transferred data need to be processed with this macro to separate the conversion data of ADC master and ADC slave.

__LL_ADC_COMMON_INSTANCE

Description:

- Helper macro to select the ADC common instance to which is belonging the selected ADC instance.

Parameters:

- `__ADCx__`: ADC instance

Return value:

- ADC: common register instance

Notes:

- ADC common register instance can be used for: Set parameters common to several ADC instancesMultimode (for devices with several ADC instances) Refer to functions having argument "ADCxy_COMMON" as parameter.

__LL_ADC_IS_ENABLED_ALL_COMMON_INSTANCE

Description:

- Helper macro to check if all ADC instances sharing the same ADC common instance are disabled.

Parameters:

- `__ADCXY_COMMON__`: ADC common instance (can be set directly from CMSIS definition or by using helper macro)

Return value:

- Value: "0" if all ADC instances sharing the same ADC common instance are disabled. Value "1" if at least one ADC instance sharing the same ADC common instance is enabled.

Notes:

- This check is required by functions with setting conditioned to ADC state: All ADC instances of the ADC common group must be disabled. Refer to functions having argument "ADCxy_COMMON" as parameter. On devices with only 1 ADC common instance, parameter of this macro is useless and can be ignored (parameter kept for compatibility with devices featuring several ADC common instances).

__LL_ADC_DIGITAL_SCALE

Description:

- Helper macro to define the ADC conversion data full-scale digital value corresponding to the selected ADC resolution.

Parameters:

- `__ADC_RESOLUTION__`: This parameter can be one of the following values:
 - `LL_ADC_RESOLUTION_12B`
 - `LL_ADC_RESOLUTION_10B`
 - `LL_ADC_RESOLUTION_8B`
 - `LL_ADC_RESOLUTION_6B`

Return value:

- ADC: conversion data equivalent voltage value (unit: mVolt)

Notes:

- ADC conversion data full-scale corresponds to voltage range determined by analog voltage references V_{ref+} and V_{ref-} (refer to reference manual).

__LL_ADC_CONVERT_DATA_RESOLUTION

Description:

- Helper macro to convert the ADC conversion data from a resolution to another resolution.

Parameters:

- `__DATA__`: ADC conversion data to be converted
- `__ADC_RESOLUTION_CURRENT__`: Resolution of the data to be converted This parameter can be one of the following values:
 - `LL_ADC_RESOLUTION_12B`
 - `LL_ADC_RESOLUTION_10B`
 - `LL_ADC_RESOLUTION_8B`
 - `LL_ADC_RESOLUTION_6B`
- `__ADC_RESOLUTION_TARGET__`: Resolution of the data after conversion This parameter can be one of the following values:
 - `LL_ADC_RESOLUTION_12B`
 - `LL_ADC_RESOLUTION_10B`
 - `LL_ADC_RESOLUTION_8B`
 - `LL_ADC_RESOLUTION_6B`

Return value:

- ADC: conversion data to the requested resolution

__LL_ADC_CALC_DATA_TO_VOLTAGE

Description:

- Helper macro to calculate the voltage (unit: mVolt) corresponding to a ADC conversion data (unit: digital value).

Parameters:

- __VREFANALOG_VOLTAGE__: Analog reference voltage (unit: mV)
- __ADC_DATA__: ADC conversion data (resolution 12 bits) (unit: digital value).
- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return value:

- ADC: conversion data equivalent voltage value (unit: mVolt)

Notes:

- Analog reference voltage (Vref+) must be known from user board environment or can be calculated using ADC measurement.

__LL_ADC_CALC_TEMPERATURE_TYP_PARAMS

Description:

- Helper macro to calculate the temperature (unit: degree Celsius) from ADC conversion data of internal temperature sensor.

Parameters:

- **__TEMPSENSOR_TYP_AVGSLOPE__**: Device datasheet data: Temperature sensor slope typical value (unit: uV/DegCelsius). On STM32F2, refer to device datasheet parameter "Avg_Slope".
- **__TEMPSENSOR_TYP_CALX_V__**: Device datasheet data: Temperature sensor voltage typical value (at temperature and Vref+ defined in parameters below) (unit: mV). On STM32F2, refer to device datasheet parameter "V25".
- **__TEMPSENSOR_CALX_TEMP__**: Device datasheet data: Temperature at which temperature sensor voltage (see parameter above) is corresponding (unit: mV)
- **__VREFANALOG_VOLTAGE__**: Analog voltage reference (Vref+) voltage (unit: mV)
- **__TEMPSENSOR_ADC_DATA__**: ADC conversion data of internal temperature sensor (unit: digital value).
- **__ADC_RESOLUTION__**: ADC resolution at which internal temperature sensor voltage has been measured. This parameter can be one of the following values:
 - **LL_ADC_RESOLUTION_12B**
 - **LL_ADC_RESOLUTION_10B**
 - **LL_ADC_RESOLUTION_8B**
 - **LL_ADC_RESOLUTION_6B**

Return value:

- Temperature: (unit: degree Celsius)

Notes:

- Computation is using temperature sensor typical values (refer to device datasheet). Calculation formula: $\text{Temperature} = (\text{TS_TYP_CALx_VOLT}(\mu\text{V}) - \text{TS_ADC_DATA} * \text{Conversion_uV}) / \text{Avg_Slope} + \text{CALx_TEMP}$ with **TS_ADC_DATA** = temperature sensor raw data measured by ADC (unit: digital value) **Avg_Slope** = temperature sensor slope (unit: uV/Degree Celsius) **TS_TYP_CALx_VOLT** = temperature sensor digital value at temperature **CALx_TEMP** (unit: mV) Caution: Calculation relevancy under reserve the temperature sensor of the current device has characteristics in line with datasheet typical values. If temperature sensor calibration values are available on on this device (presence of macro **__LL_ADC_CALC_TEMPERATURE__**()), temperature calculation will be more accurate using helper macro **__LL_ADC_CALC_TEMPERATURE__**(). As calculation input, the analog reference voltage (Vref+) must be defined as it impacts the ADC LSB equivalent voltage. Analog reference voltage (Vref+) must be known from user board environment or can be calculated using ADC measurement. ADC measurement data must correspond to a resolution of 12bits (full scale digital value 4095). If not the case, the data must be preliminarily rescaled to an equivalent resolution of 12 bits.

Common write and read registers Macros

LL_ADC_WriteReg

Description:

- Write a value in ADC register.

Parameters:

- **__INSTANCE__**: ADC Instance
- **__REG__**: Register to be written
- **__VALUE__**: Value to be written in the register

Return value:

- None

LL_ADC_ReadReg

Description:

- Read a value in ADC register.

Parameters:

- `__INSTANCE__`: ADC Instance
- `__REG__`: Register to be read

Return value:

- Register: value

50 LL BUS Generic Driver

50.1 BUS Firmware driver API description

The following section lists the various functions of the BUS library.

50.1.1 Detailed description of functions

`LL_AHB1_GRP1_EnableClock`

Function name

`__STATIC_INLINE void LL_AHB1_GRP1_EnableClock (uint32_t Periphs)`

Function description

Enable AHB1 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:

- `LL_AHB1_GRP1_PERIPH_GPIOA`
- `LL_AHB1_GRP1_PERIPH_GPIOB`
- `LL_AHB1_GRP1_PERIPH_GPIOC`
- `LL_AHB1_GRP1_PERIPH_GPIOD`
- `LL_AHB1_GRP1_PERIPH_GPIOE`
- `LL_AHB1_GRP1_PERIPH_GPIOF`
- `LL_AHB1_GRP1_PERIPH_GPIOG`
- `LL_AHB1_GRP1_PERIPH_GPIOH`
- `LL_AHB1_GRP1_PERIPH_GPIOI`
- `LL_AHB1_GRP1_PERIPH_CRC`
- `LL_AHB1_GRP1_PERIPH_BKPSRAM`
- `LL_AHB1_GRP1_PERIPH_DMA1`
- `LL_AHB1_GRP1_PERIPH_DMA2`
- `LL_AHB1_GRP1_PERIPH_ETHMAC (*)`
- `LL_AHB1_GRP1_PERIPH_ETHMACTX (*)`
- `LL_AHB1_GRP1_PERIPH_ETHMACRX (*)`
- `LL_AHB1_GRP1_PERIPH_ETHMACPTP (*)`
- `LL_AHB1_GRP1_PERIPH_OTGHS`
- `LL_AHB1_GRP1_PERIPH_OTGHSULPI`

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1ENR GPIOAEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOBEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOCEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIODEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOEEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOFEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOGEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOHEN LL_AHB1_GRP1_EnableClock
- AHB1ENR GPIOIEN LL_AHB1_GRP1_EnableClock
- AHB1ENR CRCEN LL_AHB1_GRP1_EnableClock
- AHB1ENR BKPSRAMEN LL_AHB1_GRP1_EnableClock
- AHB1ENR DMA1EN LL_AHB1_GRP1_EnableClock
- AHB1ENR DMA2EN LL_AHB1_GRP1_EnableClock
- AHB1ENR ETHMACEN LL_AHB1_GRP1_EnableClock
- AHB1ENR ETHMACTXEN LL_AHB1_GRP1_EnableClock
- AHB1ENR ETHMACRXEN LL_AHB1_GRP1_EnableClock
- AHB1ENR ETHMACPTPEN LL_AHB1_GRP1_EnableClock
- AHB1ENR OTGHSEN LL_AHB1_GRP1_EnableClock
- AHB1ENR OTGHSULPIEN LL_AHB1_GRP1_EnableClock

LL_AHB1_GRP1_IsEnabledClock

Function name

__STATIC_INLINE uint32_t LL_AHB1_GRP1_IsEnabledClock (uint32_t Periphs)

Function description

Check if AHB1 peripheral clock is enabled or not.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB1_GRP1_PERIPH_GPIOA
 - LL_AHB1_GRP1_PERIPH_GPIOB
 - LL_AHB1_GRP1_PERIPH_GPIOC
 - LL_AHB1_GRP1_PERIPH_GPIOD
 - LL_AHB1_GRP1_PERIPH_GPIOE
 - LL_AHB1_GRP1_PERIPH_GPIOF
 - LL_AHB1_GRP1_PERIPH_GPIOG
 - LL_AHB1_GRP1_PERIPH_GPIOH
 - LL_AHB1_GRP1_PERIPH_GPIOI
 - LL_AHB1_GRP1_PERIPH_CRC
 - LL_AHB1_GRP1_PERIPH_BKPSRAM
 - LL_AHB1_GRP1_PERIPH_DMA1
 - LL_AHB1_GRP1_PERIPH_DMA2
 - LL_AHB1_GRP1_PERIPH_ETHMAC (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACTX (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACRX (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACPTP (*)
 - LL_AHB1_GRP1_PERIPH_OTGHS
 - LL_AHB1_GRP1_PERIPH_OTGHSULPI

(*) value not defined in all devices.

Return values

- **State:** of Periphs (1 or 0).

Reference Manual to LL API cross reference:

- AHB1ENR GPIOAEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOBEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOCEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIODEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOEEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOFEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOGEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOHEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR GPIOIEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR CRCEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR BKPSRAMEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR DMA1EN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR DMA2EN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR ETHMACEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR ETHMACTXEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR ETHMACRXEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR ETHMACPTPEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR OTGHSSEN LL_AHB1_GRP1_IsEnabledClock
- AHB1ENR OTGHSULPIEN LL_AHB1_GRP1_IsEnabledClock

LL_AHB1_GRP1_DisableClock

Function name

__STATIC_INLINE void LL_AHB1_GRP1_DisableClock (uint32_t Periphs)

Function description

Disable AHB1 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB1_GRP1_PERIPH_GPIOA
 - LL_AHB1_GRP1_PERIPH_GPIOB
 - LL_AHB1_GRP1_PERIPH_GPIOC
 - LL_AHB1_GRP1_PERIPH_GPIOD
 - LL_AHB1_GRP1_PERIPH_GPIOE
 - LL_AHB1_GRP1_PERIPH_GPIOF
 - LL_AHB1_GRP1_PERIPH_GPIOG
 - LL_AHB1_GRP1_PERIPH_GPIOH
 - LL_AHB1_GRP1_PERIPH_GPIOI
 - LL_AHB1_GRP1_PERIPH_CRC
 - LL_AHB1_GRP1_PERIPH_BKPSRAM
 - LL_AHB1_GRP1_PERIPH_DMA1
 - LL_AHB1_GRP1_PERIPH_DMA2
 - LL_AHB1_GRP1_PERIPH_ETHMAC (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACTX (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACRX (*)
 - LL_AHB1_GRP1_PERIPH_ETHMACPTP (*)
 - LL_AHB1_GRP1_PERIPH_OTGHS
 - LL_AHB1_GRP1_PERIPH_OTGHSULPI

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1ENR GPIOAEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOBEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOCEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIODEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOEEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOFEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOGEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOHEN LL_AHB1_GRP1_DisableClock
- AHB1ENR GPIOIEN LL_AHB1_GRP1_DisableClock
- AHB1ENR CRCEN LL_AHB1_GRP1_DisableClock
- AHB1ENR BKPSRAMEN LL_AHB1_GRP1_DisableClock
- AHB1ENR DMA1EN LL_AHB1_GRP1_DisableClock
- AHB1ENR DMA2EN LL_AHB1_GRP1_DisableClock
- AHB1ENR ETHMACEN LL_AHB1_GRP1_DisableClock
- AHB1ENR ETHMACTXEN LL_AHB1_GRP1_DisableClock
- AHB1ENR ETHMACRXEN LL_AHB1_GRP1_DisableClock
- AHB1ENR ETHMACPTPEN LL_AHB1_GRP1_DisableClock
- AHB1ENR OTGHSEN LL_AHB1_GRP1_DisableClock
- AHB1ENR OTGHSULPIEN LL_AHB1_GRP1_DisableClock

LL_AHB1_GRP1_ForceReset

Function name

__STATIC_INLINE void LL_AHB1_GRP1_ForceReset (uint32_t Periphs)

Function description

Force AHB1 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB1_GRP1_PERIPH_ALL
 - LL_AHB1_GRP1_PERIPH_GPIOA
 - LL_AHB1_GRP1_PERIPH_GPIOB
 - LL_AHB1_GRP1_PERIPH_GPIOC
 - LL_AHB1_GRP1_PERIPH_GPIOD
 - LL_AHB1_GRP1_PERIPH_GPIOE
 - LL_AHB1_GRP1_PERIPH_GPIOF
 - LL_AHB1_GRP1_PERIPH_GPIOG
 - LL_AHB1_GRP1_PERIPH_GPIOH
 - LL_AHB1_GRP1_PERIPH_GPIOI
 - LL_AHB1_GRP1_PERIPH_CRC
 - LL_AHB1_GRP1_PERIPH_DMA1
 - LL_AHB1_GRP1_PERIPH_DMA2
 - LL_AHB1_GRP1_PERIPH_ETHMAC (*)
 - LL_AHB1_GRP1_PERIPH_OTGHS
- (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1RSTR GPIOARST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOBRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOCRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIODRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOERST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOFRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOGRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOHRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR GPIOIRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR CRCRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR DMA1RST LL_AHB1_GRP1_ForceReset
- AHB1RSTR DMA2RST LL_AHB1_GRP1_ForceReset
- AHB1RSTR ETHMACRST LL_AHB1_GRP1_ForceReset
- AHB1RSTR OTGHSRST LL_AHB1_GRP1_ForceReset

LL_AHB1_GRP1_ReleaseReset

Function name

__STATIC_INLINE void LL_AHB1_GRP1_ReleaseReset (uint32_t Periphs)

Function description

Release AHB1 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB1_GRP1_PERIPH_ALL
 - LL_AHB1_GRP1_PERIPH_GPIOA
 - LL_AHB1_GRP1_PERIPH_GPIOB
 - LL_AHB1_GRP1_PERIPH_GPIOC
 - LL_AHB1_GRP1_PERIPH_GPIOD
 - LL_AHB1_GRP1_PERIPH_GPIOE
 - LL_AHB1_GRP1_PERIPH_GPIOF
 - LL_AHB1_GRP1_PERIPH_GPIOG
 - LL_AHB1_GRP1_PERIPH_GPIOH
 - LL_AHB1_GRP1_PERIPH_GPIOI
 - LL_AHB1_GRP1_PERIPH_CRC
 - LL_AHB1_GRP1_PERIPH_DMA1
 - LL_AHB1_GRP1_PERIPH_DMA2
 - LL_AHB1_GRP1_PERIPH_ETHMAC (*)
 - LL_AHB1_GRP1_PERIPH_OTGHS
- (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1RSTR GPIOARST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOBRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOCRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIODRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOERST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOFRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOGRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOHRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR GPIOIRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR CRCRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR DMA1RST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR DMA2RST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR ETHMACRST LL_AHB1_GRP1_ReleaseReset
- AHB1RSTR OTGHSRST LL_AHB1_GRP1_ReleaseReset

LL_AHB1_GRP1_EnableClockLowPower

Function name

__STATIC_INLINE void LL_AHB1_GRP1_EnableClockLowPower (uint32_t Periphs)

Function description

Enable AHB1 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:

- LL_AHB1_GRP1_PERIPH_GPIOA
- LL_AHB1_GRP1_PERIPH_GPIOB
- LL_AHB1_GRP1_PERIPH_GPIOC
- LL_AHB1_GRP1_PERIPH_GPIOD
- LL_AHB1_GRP1_PERIPH_GPIOE
- LL_AHB1_GRP1_PERIPH_GPIOF
- LL_AHB1_GRP1_PERIPH_GPIOG
- LL_AHB1_GRP1_PERIPH_GPIOH
- LL_AHB1_GRP1_PERIPH_GPIOI
- LL_AHB1_GRP1_PERIPH_CRC
- LL_AHB1_GRP1_PERIPH_BKPSRAM
- LL_AHB1_GRP1_PERIPH_FLITF
- LL_AHB1_GRP1_PERIPH_SRAM1
- LL_AHB1_GRP1_PERIPH_SRAM2
- LL_AHB1_GRP1_PERIPH_DMA1
- LL_AHB1_GRP1_PERIPH_DMA2
- LL_AHB1_GRP1_PERIPH_ETHMAC (*)
- LL_AHB1_GRP1_PERIPH_ETHMACTX (*)
- LL_AHB1_GRP1_PERIPH_ETHMACRX (*)
- LL_AHB1_GRP1_PERIPH_ETHMACPTP (*)
- LL_AHB1_GRP1_PERIPH_OTGHS
- LL_AHB1_GRP1_PERIPH_OTGHSULPI

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1LPENR GPIOALPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOBLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOCLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIODLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOELPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOFLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOGLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOHLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR GPIOILPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR CRCLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR BKPSRAMPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR FLITFLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR SRAM1LPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR SRAM2LPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR BKPSRAMPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR DMA1LPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR DMA2LPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR ETHMACLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR ETHMACTXLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR ETHMACRXLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR ETHMACPTLPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR OTGHS LPEN LL_AHB1_GRP1_EnableClockLowPower
- AHB1LPENR OTGHSULPILPEN LL_AHB1_GRP1_EnableClockLowPower

LL_AHB1_GRP1_DisableClockLowPower

Function name

__STATIC_INLINE void LL_AHB1_GRP1_DisableClockLowPower (uint32_t Periphs)

Function description

Disable AHB1 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:

- LL_AHB1_GRP1_PERIPH_GPIOA
- LL_AHB1_GRP1_PERIPH_GPIOB
- LL_AHB1_GRP1_PERIPH_GPIOC
- LL_AHB1_GRP1_PERIPH_GPIOD
- LL_AHB1_GRP1_PERIPH_GPIOE
- LL_AHB1_GRP1_PERIPH_GPIOF
- LL_AHB1_GRP1_PERIPH_GPIOG
- LL_AHB1_GRP1_PERIPH_GPIOH
- LL_AHB1_GRP1_PERIPH_GPIOI
- LL_AHB1_GRP1_PERIPH_CRC
- LL_AHB1_GRP1_PERIPH_BKPSRAM
- LL_AHB1_GRP1_PERIPH_FLITF
- LL_AHB1_GRP1_PERIPH_SRAM1
- LL_AHB1_GRP1_PERIPH_SRAM2
- LL_AHB1_GRP1_PERIPH_DMA1
- LL_AHB1_GRP1_PERIPH_DMA2
- LL_AHB1_GRP1_PERIPH_ETHMAC (*)
- LL_AHB1_GRP1_PERIPH_ETHMACTX (*)
- LL_AHB1_GRP1_PERIPH_ETHMACRX (*)
- LL_AHB1_GRP1_PERIPH_ETHMACPTP (*)
- LL_AHB1_GRP1_PERIPH_OTGHS
- LL_AHB1_GRP1_PERIPH_OTGHSULPI

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB1LPENR GPIOALPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOBLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOCLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIODLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOELPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOFLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOGLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOHLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR GPIOILPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR CRCLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR BKPSRMLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR FLITFLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR SRAM1LPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR SRAM2LPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR BKPSRMLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR DMA1LPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR DMA2LPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR ETHMACLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR ETHMACTXPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR ETHMACRXLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR ETHMACPTLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR OTGHSLPEN LL_AHB1_GRP1_DisableClockLowPower
- AHB1LPENR OTGHSULPILPEN LL_AHB1_GRP1_DisableClockLowPower

LL_AHB2_GRP1_EnableClock

Function name

__STATIC_INLINE void LL_AHB2_GRP1_EnableClock (uint32_t Periphs)

Function description

Enable AHB2 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_DCM1 (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
- (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2ENR DCM1EN LL_AHB2_GRP1_EnableClock
- AHB2ENR CRYPEN LL_AHB2_GRP1_EnableClock
- AHB2ENR HASHEN LL_AHB2_GRP1_EnableClock
- AHB2ENR RNGEN LL_AHB2_GRP1_EnableClock
- AHB2ENR OTGFSEN LL_AHB2_GRP1_EnableClock

LL_AHB2_GRP1_IsEnabledClock

Function name

__STATIC_INLINE uint32_t LL_AHB2_GRP1_IsEnabledClock (uint32_t Periphs)

Function description

Check if AHB2 peripheral clock is enabled or not.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_DCMI (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **State:** of Periphs (1 or 0).

Reference Manual to LL API cross reference:

- AHB2ENR DCMIEN LL_AHB2_GRP1_IsEnabledClock
- AHB2ENR CRYPEN LL_AHB2_GRP1_IsEnabledClock
- AHB2ENR HASHEN LL_AHB2_GRP1_IsEnabledClock
- AHB2ENR RNGEN LL_AHB2_GRP1_IsEnabledClock
- AHB2ENR OTGFSEN LL_AHB2_GRP1_IsEnabledClock

LL_AHB2_GRP1_DisableClock

Function name

__STATIC_INLINE void LL_AHB2_GRP1_DisableClock (uint32_t Periphs)

Function description

Disable AHB2 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_DCMI (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2ENR DCMIEN LL_AHB2_GRP1_DisableClock
- AHB2ENR CRYPEN LL_AHB2_GRP1_DisableClock
- AHB2ENR HASHEN LL_AHB2_GRP1_DisableClock
- AHB2ENR RNGEN LL_AHB2_GRP1_DisableClock
- AHB2ENR OTGFSEN LL_AHB2_GRP1_DisableClock

```
LL_AHB2_GRP1_ForceReset
```

Function name

```
__STATIC_INLINE void LL_AHB2_GRP1_ForceReset (uint32_t Periphs)
```

Function description

Force AHB2 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_ALL
 - LL_AHB2_GRP1_PERIPH_DCM1 (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2RSTR DCMIRST LL_AHB2_GRP1_ForceReset
- AHB2RSTR CRYPYST LL_AHB2_GRP1_ForceReset
- AHB2RSTR HASHYST LL_AHB2_GRP1_ForceReset
- AHB2RSTR RNGYST LL_AHB2_GRP1_ForceReset
- AHB2RSTR OTGFSYST LL_AHB2_GRP1_ForceReset

```
LL_AHB2_GRP1_ReleaseReset
```

Function name

```
__STATIC_INLINE void LL_AHB2_GRP1_ReleaseReset (uint32_t Periphs)
```

Function description

Release AHB2 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_ALL
 - LL_AHB2_GRP1_PERIPH_DCM1 (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2RSTR DCMIRST LL_AHB2_GRP1_ReleaseReset
- AHB2RSTR CRYPRST LL_AHB2_GRP1_ReleaseReset
- AHB2RSTR HASHRST LL_AHB2_GRP1_ReleaseReset
- AHB2RSTR RNGRST LL_AHB2_GRP1_ReleaseReset
- AHB2RSTR OTGFSRST LL_AHB2_GRP1_ReleaseReset

LL_AHB2_GRP1_EnableClockLowPower

Function name

__STATIC_INLINE void LL_AHB2_GRP1_EnableClockLowPower (uint32_t Periphs)

Function description

Enable AHB2 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_DCMI (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2LPENR DCMILPEN LL_AHB2_GRP1_EnableClockLowPower
- AHB2LPENR CRYLPEN LL_AHB2_GRP1_EnableClockLowPower
- AHB2LPENR HASHLPEN LL_AHB2_GRP1_EnableClockLowPower
- AHB2LPENR RNGLPEN LL_AHB2_GRP1_EnableClockLowPower
- AHB2LPENR OTGFSLPEN LL_AHB2_GRP1_EnableClockLowPower

LL_AHB2_GRP1_DisableClockLowPower

Function name

__STATIC_INLINE void LL_AHB2_GRP1_DisableClockLowPower (uint32_t Periphs)

Function description

Disable AHB2 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_DCMI (*)
 - LL_AHB2_GRP1_PERIPH_Cryp (*)
 - LL_AHB2_GRP1_PERIPH_HASH (*)
 - LL_AHB2_GRP1_PERIPH_RNG
 - LL_AHB2_GRP1_PERIPH_OTGFS
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB2LPENR DCMILPEN LL_AHB2_GRP1_DisableClockLowPower
- AHB2LPENR CRYLPEN LL_AHB2_GRP1_DisableClockLowPower
- AHB2LPENR HASHLPEN LL_AHB2_GRP1_DisableClockLowPower
- AHB2LPENR RNGLPEN LL_AHB2_GRP1_DisableClockLowPower
- AHB2LPENR OTGFSLPEN LL_AHB2_GRP1_DisableClockLowPower

LL_AHB3_GRP1_EnableClock

Function name

__STATIC_INLINE void LL_AHB3_GRP1_EnableClock (uint32_t Periphs)

Function description

Enable AHB3 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3ENR FSMCEN LL_AHB3_GRP1_EnableClock

LL_AHB3_GRP1_IsEnabledClock

Function name

__STATIC_INLINE uint32_t LL_AHB3_GRP1_IsEnabledClock (uint32_t Periphs)

Function description

Check if AHB3 peripheral clock is enabled or not.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **State:** of Periphs (1 or 0).

Reference Manual to LL API cross reference:

- AHB3ENR FSMCEN LL_AHB3_GRP1_IsEnabledClock

LL_AHB3_GRP1_DisableClock

Function name

__STATIC_INLINE void LL_AHB3_GRP1_DisableClock (uint32_t Periphs)

Function description

Disable AHB3 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3ENR FSMCEN LL_AHB3_GRP1_DisableClock

LL_AHB3_GRP1_ForceReset

Function name

__STATIC_INLINE void LL_AHB3_GRP1_ForceReset (uint32_t Periphs)

Function description

Force AHB3 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_ALL
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3RSTR FSMCRST LL_AHB3_GRP1_ForceReset

LL_AHB3_GRP1_ReleaseReset

Function name

__STATIC_INLINE void LL_AHB3_GRP1_ReleaseReset (uint32_t Periphs)

Function description

Release AHB3 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB2_GRP1_PERIPH_ALL
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3RSTR FSMCRST LL_AHB3_GRP1_ReleaseReset

LL_AHB3_GRP1_EnableClockLowPower

Function name

__STATIC_INLINE void LL_AHB3_GRP1_EnableClockLowPower (uint32_t Periphs)

Function description

Enable AHB3 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3LPENR FSMCLPEN LL_AHB3_GRP1_EnableClockLowPower

```
LL_AHB3_GRP1_DisableClockLowPower
```

Function name

```
__STATIC_INLINE void LL_AHB3_GRP1_DisableClockLowPower (uint32_t Periphs)
```

Function description

Disable AHB3 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_AHB3_GRP1_PERIPH_FSMC

Return values

- **None:**

Reference Manual to LL API cross reference:

- AHB3LPENR FSMCLPEN LL_AHB3_GRP1_DisableClockLowPower

```
LL_APB1_GRP1_EnableClock
```

Function name

```
__STATIC_INLINE void LL_APB1_GRP1_EnableClock (uint32_t Periphs)
```

Function description

Enable APB1 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1ENR TIM2EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM3EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM4EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM5EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM6EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM7EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM12EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM13EN LL_APB1_GRP1_EnableClock
- APB1ENR TIM14EN LL_APB1_GRP1_EnableClock
- APB1ENR WWDGEN LL_APB1_GRP1_EnableClock
- APB1ENR SPI2EN LL_APB1_GRP1_EnableClock
- APB1ENR SPI3EN LL_APB1_GRP1_EnableClock
- APB1ENR USART2EN LL_APB1_GRP1_EnableClock
- APB1ENR USART3EN LL_APB1_GRP1_EnableClock
- APB1ENR UART4EN LL_APB1_GRP1_EnableClock
- APB1ENR UART5EN LL_APB1_GRP1_EnableClock
- APB1ENR I2C1EN LL_APB1_GRP1_EnableClock
- APB1ENR I2C2EN LL_APB1_GRP1_EnableClock
- APB1ENR I2C3EN LL_APB1_GRP1_EnableClock
- APB1ENR CAN1EN LL_APB1_GRP1_EnableClock
- APB1ENR CAN2EN LL_APB1_GRP1_EnableClock
- APB1ENR PWREN LL_APB1_GRP1_EnableClock
- APB1ENR DACEN LL_APB1_GRP1_EnableClock

LL_APB1_GRP1_IsEnabledClock

Function name

__STATIC_INLINE uint32_t LL_APB1_GRP1_IsEnabledClock (uint32_t Periphs)

Function description

Check if APB1 peripheral clock is enabled or not.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **State:** of Periphs (1 or 0).

Reference Manual to LL API cross reference:

- APB1ENR TIM2EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM3EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM4EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM5EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM6EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM7EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM12EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM13EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR TIM14EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR WWDGEN LL_APB1_GRP1_IsEnabledClock
- APB1ENR SPI2EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR SPI3EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR USART2EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR USART3EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR UART4EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR UART5EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR I2C1EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR I2C2EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR I2C3EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR CAN1EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR CAN2EN LL_APB1_GRP1_IsEnabledClock
- APB1ENR PWREN LL_APB1_GRP1_IsEnabledClock
- APB1ENR DACEN LL_APB1_GRP1_IsEnabledClock

LL_APB1_GRP1_DisableClock

Function name

__STATIC_INLINE void LL_APB1_GRP1_DisableClock (uint32_t Periphs)

Function description

Disable APB1 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1ENR TIM2EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM3EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM4EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM5EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM6EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM7EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM12EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM13EN LL_APB1_GRP1_DisableClock
- APB1ENR TIM14EN LL_APB1_GRP1_DisableClock
- APB1ENR WWDGEN LL_APB1_GRP1_DisableClock
- APB1ENR SPI2EN LL_APB1_GRP1_DisableClock
- APB1ENR SPI3EN LL_APB1_GRP1_DisableClock
- APB1ENR USART2EN LL_APB1_GRP1_DisableClock
- APB1ENR USART3EN LL_APB1_GRP1_DisableClock
- APB1ENR UART4EN LL_APB1_GRP1_DisableClock
- APB1ENR UART5EN LL_APB1_GRP1_DisableClock
- APB1ENR I2C1EN LL_APB1_GRP1_DisableClock
- APB1ENR I2C2EN LL_APB1_GRP1_DisableClock
- APB1ENR I2C3EN LL_APB1_GRP1_DisableClock
- APB1ENR CAN1EN LL_APB1_GRP1_DisableClock
- APB1ENR CAN2EN LL_APB1_GRP1_DisableClock
- APB1ENR PWREN LL_APB1_GRP1_DisableClock
- APB1ENR DACEN LL_APB1_GRP1_DisableClock

LL_APB1_GRP1_ForceReset

Function name

__STATIC_INLINE void LL_APB1_GRP1_ForceReset (uint32_t Periphs)

Function description

Force APB1 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_ALL
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1RSTR TIM2RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM3RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM4RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM5RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM6RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM7RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM12RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM13RST LL_APB1_GRP1_ForceReset
- APB1RSTR TIM14RST LL_APB1_GRP1_ForceReset
- APB1RSTR WWDGRST LL_APB1_GRP1_ForceReset
- APB1RSTR SPI2RST LL_APB1_GRP1_ForceReset
- APB1RSTR SPI3RST LL_APB1_GRP1_ForceReset
- APB1RSTR USART2RST LL_APB1_GRP1_ForceReset
- APB1RSTR USART3RST LL_APB1_GRP1_ForceReset
- APB1RSTR UART4RST LL_APB1_GRP1_ForceReset
- APB1RSTR UART5RST LL_APB1_GRP1_ForceReset
- APB1RSTR I2C1RST LL_APB1_GRP1_ForceReset
- APB1RSTR I2C2RST LL_APB1_GRP1_ForceReset
- APB1RSTR I2C3RST LL_APB1_GRP1_ForceReset
- APB1RSTR CAN1RST LL_APB1_GRP1_ForceReset
- APB1RSTR CAN2RST LL_APB1_GRP1_ForceReset
- APB1RSTR PWRRST LL_APB1_GRP1_ForceReset
- APB1RSTR DACRST LL_APB1_GRP1_ForceReset

LL_APB1_GRP1_ReleaseReset

Function name

__STATIC_INLINE void LL_APB1_GRP1_ReleaseReset (uint32_t Periphs)

Function description

Release APB1 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_ALL
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1RSTR TIM2RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM3RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM4RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM5RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM6RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM7RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM12RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM13RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR TIM14RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR WWDGRST LL_APB1_GRP1_ReleaseReset
- APB1RSTR SPI2RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR SPI3RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR USART2RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR USART3RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR UART4RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR UART5RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR I2C1RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR I2C2RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR I2C3RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR CAN1RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR CAN2RST LL_APB1_GRP1_ReleaseReset
- APB1RSTR PWRRST LL_APB1_GRP1_ReleaseReset
- APB1RSTR DACRST LL_APB1_GRP1_ReleaseReset

LL_APB1_GRP1_EnableClockLowPower

Function name

__STATIC_INLINE void LL_APB1_GRP1_EnableClockLowPower (uint32_t Periphs)

Function description

Enable APB1 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1LPENR TIM2LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM3LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM4LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM5LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM6LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM7LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM12LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM13LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR TIM14LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR WWDGLPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR SPI2LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR SPI3LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR USART2LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR USART3LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR UART4LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR UART5LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR I2C1LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR I2C2LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR I2C3LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR CAN1LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR CAN2LPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR PWRLPEN LL_APB1_GRP1_EnableClockLowPower
- APB1LPENR DACLPEN LL_APB1_GRP1_EnableClockLowPower

LL_APB1_GRP1_DisableClockLowPower

Function name

__STATIC_INLINE void LL_APB1_GRP1_DisableClockLowPower (uint32_t Periphs)

Function description

Disable APB1 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB1_GRP1_PERIPH_TIM2
 - LL_APB1_GRP1_PERIPH_TIM3
 - LL_APB1_GRP1_PERIPH_TIM4
 - LL_APB1_GRP1_PERIPH_TIM5
 - LL_APB1_GRP1_PERIPH_TIM6
 - LL_APB1_GRP1_PERIPH_TIM7
 - LL_APB1_GRP1_PERIPH_TIM12
 - LL_APB1_GRP1_PERIPH_TIM13
 - LL_APB1_GRP1_PERIPH_TIM14
 - LL_APB1_GRP1_PERIPH_WWDG
 - LL_APB1_GRP1_PERIPH_SPI2
 - LL_APB1_GRP1_PERIPH_SPI3
 - LL_APB1_GRP1_PERIPH_USART2
 - LL_APB1_GRP1_PERIPH_USART3
 - LL_APB1_GRP1_PERIPH_UART4
 - LL_APB1_GRP1_PERIPH_UART5
 - LL_APB1_GRP1_PERIPH_I2C1
 - LL_APB1_GRP1_PERIPH_I2C2
 - LL_APB1_GRP1_PERIPH_I2C3
 - LL_APB1_GRP1_PERIPH_CAN1
 - LL_APB1_GRP1_PERIPH_CAN2
 - LL_APB1_GRP1_PERIPH_PWR
 - LL_APB1_GRP1_PERIPH_DAC1

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB1LPENR TIM2LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM3LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM4LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM5LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM6LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM7LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM12LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM13LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR TIM14LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR WWDGLPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR SPI2LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR SPI3LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR USART2LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR USART3LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR UART4LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR UART5LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR I2C1LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR I2C2LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR I2C3LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR CAN1LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR CAN2LPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR PWRLPEN LL_APB1_GRP1_DisableClockLowPower
- APB1LPENR DACLPEN LL_APB1_GRP1_DisableClockLowPower

LL_APB2_GRP1_EnableClock

Function name

__STATIC_INLINE void LL_APB2_GRP1_EnableClock (uint32_t Periphs)

Function description

Enable APB2 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC1
 - LL_APB2_GRP1_PERIPH_ADC2
 - LL_APB2_GRP1_PERIPH_ADC3
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2ENR TIM1EN LL_APB2_GRP1_EnableClock
- APB2ENR TIM8EN LL_APB2_GRP1_EnableClock
- APB2ENR USART1EN LL_APB2_GRP1_EnableClock
- APB2ENR USART6EN LL_APB2_GRP1_EnableClock
- APB2ENR ADC1EN LL_APB2_GRP1_EnableClock
- APB2ENR ADC2EN LL_APB2_GRP1_EnableClock
- APB2ENR ADC3EN LL_APB2_GRP1_EnableClock
- APB2ENR SDIOEN LL_APB2_GRP1_EnableClock
- APB2ENR SPI1EN LL_APB2_GRP1_EnableClock
- APB2ENR SYSCFGEN LL_APB2_GRP1_EnableClock
- APB2ENR TIM9EN LL_APB2_GRP1_EnableClock
- APB2ENR TIM10EN LL_APB2_GRP1_EnableClock
- APB2ENR TIM11EN LL_APB2_GRP1_EnableClock

LL_APB2_GRP1_IsEnabledClock

Function name

__STATIC_INLINE uint32_t LL_APB2_GRP1_IsEnabledClock (uint32_t Periphs)

Function description

Check if APB2 peripheral clock is enabled or not.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC1
 - LL_APB2_GRP1_PERIPH_ADC2
 - LL_APB2_GRP1_PERIPH_ADC3
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **State:** of Periphs (1 or 0).

Reference Manual to LL API cross reference:

- APB2ENR TIM1EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR TIM8EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR USART1EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR USART6EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR ADC1EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR ADC2EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR ADC3EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR SDIOEN LL_APB2_GRP1_IsEnabledClock
- APB2ENR SPI1EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR SYSCFGEN LL_APB2_GRP1_IsEnabledClock
- APB2ENR TIM9EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR TIM10EN LL_APB2_GRP1_IsEnabledClock
- APB2ENR TIM11EN LL_APB2_GRP1_IsEnabledClock

LL_APB2_GRP1_DisableClock

Function name

__STATIC_INLINE void LL_APB2_GRP1_DisableClock (uint32_t Periphs)

Function description

Disable APB2 peripherals clock.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC1
 - LL_APB2_GRP1_PERIPH_ADC2
 - LL_APB2_GRP1_PERIPH_ADC3
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2ENR TIM1EN LL_APB2_GRP1_DisableClock
- APB2ENR TIM8EN LL_APB2_GRP1_DisableClock
- APB2ENR USART1EN LL_APB2_GRP1_DisableClock
- APB2ENR USART6EN LL_APB2_GRP1_DisableClock
- APB2ENR ADC1EN LL_APB2_GRP1_DisableClock
- APB2ENR ADC2EN LL_APB2_GRP1_DisableClock
- APB2ENR ADC3EN LL_APB2_GRP1_DisableClock
- APB2ENR SDIOEN LL_APB2_GRP1_DisableClock
- APB2ENR SPI1EN LL_APB2_GRP1_DisableClock
- APB2ENR SYSCFGEN LL_APB2_GRP1_DisableClock
- APB2ENR TIM9EN LL_APB2_GRP1_DisableClock
- APB2ENR TIM10EN LL_APB2_GRP1_DisableClock
- APB2ENR TIM11EN LL_APB2_GRP1_DisableClock

LL_APB2_GRP1_ForceReset

Function name

__STATIC_INLINE void LL_APB2_GRP1_ForceReset (uint32_t Periphs)

Function description

Force APB2 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_ALL
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2RSTR TIM1RST LL_APB2_GRP1_ForceReset
- APB2RSTR TIM8RST LL_APB2_GRP1_ForceReset
- APB2RSTR USART1RST LL_APB2_GRP1_ForceReset
- APB2RSTR USART6RST LL_APB2_GRP1_ForceReset
- APB2RSTR ADCRST LL_APB2_GRP1_ForceReset
- APB2RSTR SDIORST LL_APB2_GRP1_ForceReset
- APB2RSTR SPI1RST LL_APB2_GRP1_ForceReset
- APB2RSTR SYSCFGRST LL_APB2_GRP1_ForceReset
- APB2RSTR TIM9RST LL_APB2_GRP1_ForceReset
- APB2RSTR TIM10RST LL_APB2_GRP1_ForceReset
- APB2RSTR TIM11RST LL_APB2_GRP1_ForceReset

LL_APB2_GRP1_ReleaseReset

Function name

__STATIC_INLINE void LL_APB2_GRP1_ReleaseReset (uint32_t Periphs)

Function description

Release APB2 peripherals reset.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_ALL
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2RSTR TIM1RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR TIM8RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR USART1RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR USART6RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR ADCRST LL_APB2_GRP1_ReleaseReset
- APB2RSTR SDIORST LL_APB2_GRP1_ReleaseReset
- APB2RSTR SPI1RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR SYSCFGRST LL_APB2_GRP1_ReleaseReset
- APB2RSTR TIM9RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR TIM10RST LL_APB2_GRP1_ReleaseReset
- APB2RSTR TIM11RST LL_APB2_GRP1_ReleaseReset


```
LL_APB2_GRP1_EnableClockLowPower
```

Function name

```
__STATIC_INLINE void LL_APB2_GRP1_EnableClockLowPower (uint32_t Periphs)
```

Function description

Enable APB2 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC1
 - LL_APB2_GRP1_PERIPH_ADC2
 - LL_APB2_GRP1_PERIPH_ADC3
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2LPENR TIM1LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR TIM8LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR USART1LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR USART6LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR ADC1LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR ADC2LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR ADC3LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR SDIOLPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR SPI1LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR SYSCFGLPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR TIM9LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR TIM10LPEN LL_APB2_GRP1_EnableClockLowPower
- APB2LPENR TIM11LPEN LL_APB2_GRP1_EnableClockLowPower

```
LL_APB2_GRP1_DisableClockLowPower
```

Function name

```
__STATIC_INLINE void LL_APB2_GRP1_DisableClockLowPower (uint32_t Periphs)
```

Function description

Disable APB2 peripheral clocks in low-power mode.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_APB2_GRP1_PERIPH_TIM1
 - LL_APB2_GRP1_PERIPH_TIM8
 - LL_APB2_GRP1_PERIPH_USART1
 - LL_APB2_GRP1_PERIPH_USART6
 - LL_APB2_GRP1_PERIPH_ADC1
 - LL_APB2_GRP1_PERIPH_ADC2
 - LL_APB2_GRP1_PERIPH_ADC3
 - LL_APB2_GRP1_PERIPH_SDIO
 - LL_APB2_GRP1_PERIPH_SPI1
 - LL_APB2_GRP1_PERIPH_SYSCFG
 - LL_APB2_GRP1_PERIPH_TIM9
 - LL_APB2_GRP1_PERIPH_TIM10
 - LL_APB2_GRP1_PERIPH_TIM11

Return values

- **None:**

Reference Manual to LL API cross reference:

- APB2LPENR TIM1LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR TIM8LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR USART1LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR USART6LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR ADC1LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR ADC2LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR ADC3LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR SDIOLPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR SPI1LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR SYSCFGLPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR TIM9LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR TIM10LPEN LL_APB2_GRP1_DisableClockLowPower
- APB2LPENR TIM11LPEN LL_APB2_GRP1_DisableClockLowPower

50.2 BUS Firmware driver defines

The following section lists the various define and macros of the module.

50.2.1

BUS

BUS

AHB1_GRP1_PERIPH

LL_AHB1_GRP1_PERIPH_ALL

LL_AHB1_GRP1_PERIPH_GPIOA

LL_AHB1_GRP1_PERIPH_GPIOB

LL_AHB1_GRP1_PERIPH_GPIOC

LL_AHB1_GRP1_PERIPH_GPIOD

LL_AHB1_GRP1_PERIPH_GPIOE

LL_AHB1_GRP1_PERIPH_GPIOF

LL_AHB1_GRP1_PERIPH_GPIOG

LL_AHB1_GRP1_PERIPH_GPIOH

LL_AHB1_GRP1_PERIPH_GPIOI

LL_AHB1_GRP1_PERIPH_CRC

LL_AHB1_GRP1_PERIPH_BKPSRAM

LL_AHB1_GRP1_PERIPH_DMA1

LL_AHB1_GRP1_PERIPH_DMA2

LL_AHB1_GRP1_PERIPH_ETHMAC

LL_AHB1_GRP1_PERIPH_ETHMACTX

LL_AHB1_GRP1_PERIPH_ETHMACRX

LL_AHB1_GRP1_PERIPH_ETHMACPTP

LL_AHB1_GRP1_PERIPH_OTGHS

LL_AHB1_GRP1_PERIPH_OTGHSULPI

LL_AHB1_GRP1_PERIPH_FLITF

LL_AHB1_GRP1_PERIPH_SRAM1

LL_AHB1_GRP1_PERIPH_SRAM2

AHB2 GRP1 PERIPH

LL_AHB2_GRP1_PERIPH_ALL

LL_AHB2_GRP1_PERIPH_DCMI

LL_AHB2_GRP1_PERIPH_Cryp

LL_AHB2_GRP1_PERIPH_HASH

LL_AHB2_GRP1_PERIPH_RNG

LL_AHB2_GRP1_PERIPH_OTGFS

AHB3 GRP1 PERIPH

LL_AHB3_GRP1_PERIPH_ALL

LL_AHB3_GRP1_PERIPH_FSMC

APB1 GRP1 PERIPH

LL_APB1_GRP1_PERIPH_ALL
LL_APB1_GRP1_PERIPH_TIM2
LL_APB1_GRP1_PERIPH_TIM3
LL_APB1_GRP1_PERIPH_TIM4
LL_APB1_GRP1_PERIPH_TIM5
LL_APB1_GRP1_PERIPH_TIM6
LL_APB1_GRP1_PERIPH_TIM7
LL_APB1_GRP1_PERIPH_TIM12
LL_APB1_GRP1_PERIPH_TIM13
LL_APB1_GRP1_PERIPH_TIM14
LL_APB1_GRP1_PERIPH_WWDG
LL_APB1_GRP1_PERIPH_SPI2
LL_APB1_GRP1_PERIPH_SPI3
LL_APB1_GRP1_PERIPH_USART2
LL_APB1_GRP1_PERIPH_USART3
LL_APB1_GRP1_PERIPH_UART4
LL_APB1_GRP1_PERIPH_UART5
LL_APB1_GRP1_PERIPH_I2C1
LL_APB1_GRP1_PERIPH_I2C2
LL_APB1_GRP1_PERIPH_I2C3
LL_APB1_GRP1_PERIPH_CAN1
LL_APB1_GRP1_PERIPH_CAN2
LL_APB1_GRP1_PERIPH_PWR
LL_APB1_GRP1_PERIPH_DAC1

APB2 GRP1 PERIPH

LL_APB2_GRP1_PERIPH_ALL
LL_APB2_GRP1_PERIPH_TIM1
LL_APB2_GRP1_PERIPH_TIM8

LL_APB2_GRP1_PERIPH_USART1

LL_APB2_GRP1_PERIPH_USART6

LL_APB2_GRP1_PERIPH_ADC1

LL_APB2_GRP1_PERIPH_ADC2

LL_APB2_GRP1_PERIPH_ADC3

LL_APB2_GRP1_PERIPH_SDIO

LL_APB2_GRP1_PERIPH_SPI1

LL_APB2_GRP1_PERIPH_SYSCFG

LL_APB2_GRP1_PERIPH_TIM9

LL_APB2_GRP1_PERIPH_TIM10

LL_APB2_GRP1_PERIPH_TIM11

LL_APB2_GRP1_PERIPH_ADC

51 LL CORTEX Generic Driver

51.1 CORTEX Firmware driver API description

The following section lists the various functions of the CORTEX library.

51.1.1 Detailed description of functions

`LL_SYSTICK_IsActiveCounterFlag`

Function name

`__STATIC_INLINE uint32_t LL_SYSTICK_IsActiveCounterFlag (void )`

Function description

This function checks if the SysTick counter flag is active or not.

Return values

- **State:** of bit (1 or 0).

Notes

- It can be used in timeout function on application side.

Reference Manual to LL API cross reference:

- STK_CTRL COUNTFLAG `LL_SYSTICK_IsActiveCounterFlag`

`LL_SYSTICK_SetClkSource`

Function name

`__STATIC_INLINE void LL_SYSTICK_SetClkSource (uint32_t Source)`

Function description

Configures the SysTick clock source.

Parameters

- **Source:** This parameter can be one of the following values:
 - `LL_SYSTICK_CLKSOURCE_HCLK_DIV8`
 - `LL_SYSTICK_CLKSOURCE_HCLK`

Return values

- **None:**

Reference Manual to LL API cross reference:

- STK_CTRL CLKSOURCE `LL_SYSTICK_SetClkSource`

`LL_SYSTICK_GetClkSource`

Function name

`__STATIC_INLINE uint32_t LL_SYSTICK_GetClkSource (void )`

Function description

Get the SysTick clock source.

Return values

- **Returned:** value can be one of the following values:
 - `LL_SYSTICK_CLKSOURCE_HCLK_DIV8`
 - `LL_SYSTICK_CLKSOURCE_HCLK`

Reference Manual to LL API cross reference:

- STK_CTRL CLKSOURCE LL_SYSTICK_GetClkSource
LL_SYSTICK_EnableIT

Function name

__STATIC_INLINE void LL_SYSTICK_EnableIT (void)

Function description

Enable SysTick exception request.

Return values

- **None:**

Reference Manual to LL API cross reference:

- STK_CTRL TICKINT LL_SYSTICK_EnableIT
LL_SYSTICK_DisableIT

Function name

__STATIC_INLINE void LL_SYSTICK_DisableIT (void)

Function description

Disable SysTick exception request.

Return values

- **None:**

Reference Manual to LL API cross reference:

- STK_CTRL TICKINT LL_SYSTICK_DisableIT
LL_SYSTICK_IsEnabledIT

Function name

__STATIC_INLINE uint32_t LL_SYSTICK_IsEnabledIT (void)

Function description

Checks if the SYSTICK interrupt is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- STK_CTRL TICKINT LL_SYSTICK_IsEnabledIT
LL_LPM_EnableSleep

Function name

__STATIC_INLINE void LL_LPM_EnableSleep (void)

Function description

Processor uses sleep as its low power mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SCR SLEEPDEEP LL_LPM_EnableSleep

```
LL_LPM_EnableDeepSleep
```

Function name

```
__STATIC_INLINE void LL_LPM_EnableDeepSleep (void )
```

Function description

Processor uses deep sleep as its low power mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SCR SLEEPDEEP LL_LPM_EnableDeepSleep

```
LL_LPM_EnableSleepOnExit
```

Function name

```
__STATIC_INLINE void LL_LPM_EnableSleepOnExit (void )
```

Function description

Configures sleep-on-exit when returning from Handler mode to Thread mode.

Return values

- **None:**

Notes

- Setting this bit to 1 enables an interrupt-driven application to avoid returning to an empty main application.

Reference Manual to LL API cross reference:

- SCB_SCR SLEEPONEXIT LL_LPM_EnableSleepOnExit

```
LL_LPM_DisableSleepOnExit
```

Function name

```
__STATIC_INLINE void LL_LPM_DisableSleepOnExit (void )
```

Function description

Do not sleep when returning to Thread mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SCR SLEEPONEXIT LL_LPM_DisableSleepOnExit

```
LL_LPM_EnableEventOnPend
```

Function name

```
__STATIC_INLINE void LL_LPM_EnableEventOnPend (void )
```

Function description

Enabled events and all interrupts, including disabled interrupts, can wakeup the processor.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SCR SEVEONPEND LL_LPM_EnableEventOnPend

LL_LPM_DisableEventOnPend

Function name

__STATIC_INLINE void LL_LPM_DisableEventOnPend (void)

Function description

Only enabled interrupts or events can wakeup the processor, disabled interrupts are excluded.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SCR SEVEONPEND LL_LPM_DisableEventOnPend

LL_HANDLER_EnableFault

Function name

__STATIC_INLINE void LL_HANDLER_EnableFault (uint32_t Fault)

Function description

Enable a fault in System handler control register (SHCSR)

Parameters

- **Fault:** This parameter can be a combination of the following values:
 - LL_HANDLER_FAULT_USG
 - LL_HANDLER_FAULT_BUS
 - LL_HANDLER_FAULT_MEM

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SHCSR MEMFAULTENA LL_HANDLER_EnableFault

LL_HANDLER_DisableFault

Function name

__STATIC_INLINE void LL_HANDLER_DisableFault (uint32_t Fault)

Function description

Disable a fault in System handler control register (SHCSR)

Parameters

- **Fault:** This parameter can be a combination of the following values:
 - LL_HANDLER_FAULT_USG
 - LL_HANDLER_FAULT_BUS
 - LL_HANDLER_FAULT_MEM

Return values

- **None:**

Reference Manual to LL API cross reference:

- SCB_SHCSR MEMFAULTENA LL_HANDLER_DisableFault

LL_CPUID_GetImplementer

Function name

__STATIC_INLINE uint32_t LL_CPUID_GetImplementer (void)

Function description

Get Implementer code.

Return values

- **Value:** should be equal to 0x41 for ARM

Reference Manual to LL API cross reference:

- SCB_CPUID IMPLEMENTER LL_CPUID_GetImplementer
LL_CPUID_GetVariant

Function name

__STATIC_INLINE uint32_t LL_CPUID_GetVariant (void)

Function description

Get Variant number (The r value in the mpm product revision identifier)

Return values

- **Value:** between 0 and 255 (0x1: revision 1, 0x2: revision 2)

Reference Manual to LL API cross reference:

- SCB_CPUID VARIANT LL_CPUID_GetVariant
LL_CPUID_GetConstant

Function name

__STATIC_INLINE uint32_t LL_CPUID_GetConstant (void)

Function description

Get Constant number.

Return values

- **Value:** should be equal to 0xF for Cortex-M3 devices

Reference Manual to LL API cross reference:

- SCB_CPUID ARCHITECTURE LL_CPUID_GetConstant
LL_CPUID_GetParNo

Function name

__STATIC_INLINE uint32_t LL_CPUID_GetParNo (void)

Function description

Get Part number.

Return values

- **Value:** should be equal to 0xC23 for Cortex-M3

Reference Manual to LL API cross reference:

- SCB_CPUID PARTNO LL_CPUID_GetParNo
LL_CPUID_GetRevision

Function name

__STATIC_INLINE uint32_t LL_CPUID_GetRevision (void)

Function description

Get Revision number (The p value in the rmpn product revision identifier, indicates patch release)

Return values

- **Value:** between 0 and 255 (0x0: patch 0, 0x1: patch 1)

Reference Manual to LL API cross reference:

- SCB_CPUID REVISION LL_CPUID_GetRevision
LL_MPU_Enable

Function name

__STATIC_INLINE void LL_MPU_Enable (uint32_t Options)

Function description

Enable MPU with input options.

Parameters

- **Options:** This parameter can be one of the following values:
 - LL_MPU_CTRL_HFNMI_PRIVDEF_NONE
 - LL_MPU_CTRL_HARDFAULT_NMI
 - LL_MPU_CTRL_PRIVILEGED_DEFAULT
 - LL_MPU_CTRL_HFNMI_PRIVDEF

Return values

- **None:**

Reference Manual to LL API cross reference:

- MPU_CTRL ENABLE LL_MPU_Enable
LL_MPU_Disable

Function name

__STATIC_INLINE void LL_MPU_Disable (void)

Function description

Disable MPU.

Return values

- **None:**

Reference Manual to LL API cross reference:

- MPU_CTRL ENABLE LL_MPU_Disable
LL_MPU_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_MPU_IsEnabled (void)

Function description

Check if MPU is enabled or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- MPU_CTRL ENABLE LL_MPU_IsEnabled
LL_MPU_EnableRegion

Function name

__STATIC_INLINE void LL_MPU_EnableRegion (uint32_t Region)

Function description

Enable a MPU region.

Parameters

- **Region:** This parameter can be one of the following values:
 - LL_MPU_REGION_NUMBER0
 - LL_MPU_REGION_NUMBER1
 - LL_MPU_REGION_NUMBER2
 - LL_MPU_REGION_NUMBER3
 - LL_MPU_REGION_NUMBER4
 - LL_MPU_REGION_NUMBER5
 - LL_MPU_REGION_NUMBER6
 - LL_MPU_REGION_NUMBER7

Return values

- **None:**

Reference Manual to LL API cross reference:

- MPU_RASR ENABLE LL_MPU_EnableRegion

LL_MPU_ConfigRegion

Function name

__STATIC_INLINE void LL_MPU_ConfigRegion (uint32_t Region, uint32_t SubRegionDisable, uint32_t Address, uint32_t Attributes)

Function description

Configure and enable a region.

Parameters

- **Region:** This parameter can be one of the following values:
 - LL_MPU_REGION_NUMBER0
 - LL_MPU_REGION_NUMBER1
 - LL_MPU_REGION_NUMBER2
 - LL_MPU_REGION_NUMBER3
 - LL_MPU_REGION_NUMBER4
 - LL_MPU_REGION_NUMBER5
 - LL_MPU_REGION_NUMBER6
 - LL_MPU_REGION_NUMBER7
- **Address:** Value of region base address
- **SubRegionDisable:** Sub-region disable value between Min_Data = 0x00 and Max_Data = 0xFF
- **Attributes:** This parameter can be a combination of the following values:
 - LL_MPU_REGION_SIZE_32B or LL_MPU_REGION_SIZE_64B or LL_MPU_REGION_SIZE_128B or LL_MPU_REGION_SIZE_256B or LL_MPU_REGION_SIZE_512B or LL_MPU_REGION_SIZE_1KB or LL_MPU_REGION_SIZE_2KB or LL_MPU_REGION_SIZE_4KB or LL_MPU_REGION_SIZE_8KB or LL_MPU_REGION_SIZE_16KB or LL_MPU_REGION_SIZE_32KB or LL_MPU_REGION_SIZE_64KB or LL_MPU_REGION_SIZE_128KB or LL_MPU_REGION_SIZE_256KB or LL_MPU_REGION_SIZE_512KB or LL_MPU_REGION_SIZE_1MB or LL_MPU_REGION_SIZE_2MB or LL_MPU_REGION_SIZE_4MB or LL_MPU_REGION_SIZE_8MB or LL_MPU_REGION_SIZE_16MB or LL_MPU_REGION_SIZE_32MB or LL_MPU_REGION_SIZE_64MB or LL_MPU_REGION_SIZE_128MB or LL_MPU_REGION_SIZE_256MB or LL_MPU_REGION_SIZE_512MB or LL_MPU_REGION_SIZE_1GB or LL_MPU_REGION_SIZE_2GB or LL_MPU_REGION_SIZE_4GB
 - LL_MPU_REGION_NO_ACCESS or LL_MPU_REGION_PRIV_RW or LL_MPU_REGION_PRIV_RW_URO or LL_MPU_REGION_FULL_ACCESS or LL_MPU_REGION_PRIV_RO or LL_MPU_REGION_PRIV_RO_URO
 - LL_MPU_TEX_LEVEL0 or LL_MPU_TEX_LEVEL1 or LL_MPU_TEX_LEVEL2 or LL_MPU_TEX_LEVEL4
 - LL_MPU_INSTRUCTION_ACCESS_ENABLE or LL_MPU_INSTRUCTION_ACCESS_DISABLE
 - LL_MPU_ACCESS_SHAREABLE or LL_MPU_ACCESS_NOT_SHAREABLE
 - LL_MPU_ACCESS_CACHEABLE or LL_MPU_ACCESS_NOT_CACHEABLE
 - LL_MPU_ACCESS_BUFFERABLE or LL_MPU_ACCESS_NOT_BUFFERABLE

Return values

- **None:**

Reference Manual to LL API cross reference:

- MPU_RNR REGION LL_MPU_ConfigRegion
- MPU_RBAR REGION LL_MPU_ConfigRegion
- MPU_RBAR ADDR LL_MPU_ConfigRegion
- MPU_RASR XN LL_MPU_ConfigRegion
- MPU_RASR AP LL_MPU_ConfigRegion
- MPU_RASR S LL_MPU_ConfigRegion
- MPU_RASR C LL_MPU_ConfigRegion
- MPU_RASR B LL_MPU_ConfigRegion
- MPU_RASR SIZE LL_MPU_ConfigRegion

LL_MPU_DisableRegion

Function name

__STATIC_INLINE void LL_MPU_DisableRegion (uint32_t Region)

Function description

Disable a region.

Parameters

- **Region:** This parameter can be one of the following values:
 - LL_MPU_REGION_NUMBER0
 - LL_MPU_REGION_NUMBER1
 - LL_MPU_REGION_NUMBER2
 - LL_MPU_REGION_NUMBER3
 - LL_MPU_REGION_NUMBER4
 - LL_MPU_REGION_NUMBER5
 - LL_MPU_REGION_NUMBER6
 - LL_MPU_REGION_NUMBER7

Return values

- **None:**

Reference Manual to LL API cross reference:

- MPU_RNR REGION LL_MPU_DisableRegion
- MPU_RASR ENABLE LL_MPU_DisableRegion

51.2 CORTEx Firmware driver defines

The following section lists the various define and macros of the module.

51.2.1 CORTEx

CORTEx

MPU Bufferable Access

LL_MPU_ACCESS_BUFFERABLE

Bufferable memory attribute

LL_MPU_ACCESS_NOT_BUFFERABLE

Not Bufferable memory attribute

MPU Cacheable Access

LL_MPU_ACCESS_CACHEABLE

Cacheable memory attribute

LL_MPU_ACCESS_NOT_CACHEABLE

Not Cacheable memory attribute

SYSTICK Clock Source

LL_SYSTICK_CLKSOURCE_HCLK_DIV8

AHB clock divided by 8 selected as SysTick clock source.

LL_SYSTICK_CLKSOURCE_HCLK

AHB clock selected as SysTick clock source.

MPU Control

LL_MPU_CTRL_HFNMI_PRIVDEF_NONE

Disable NMI and privileged SW access

LL_MPU_CTRL_HARDFAULT_NMI

Enables the operation of MPU during hard fault, NMI, and FAULTMASK handlers

LL_MPU_CTRL_PRIVILEGED_DEFAULT

Enable privileged software access to default memory map

LL_MPU_CTRL_HFNMI_PRIVDEF

Enable NMI and privileged SW access

Handler Fault type

LL_HANDLER_FAULT_USG

Usage fault

LL_HANDLER_FAULT_BUS

Bus fault

LL_HANDLER_FAULT_MEM

Memory management fault

MPU Instruction Access

LL_MPU_INSTRUCTION_ACCESS_ENABLE

Instruction fetches enabled

LL_MPU_INSTRUCTION_ACCESS_DISABLE

Instruction fetches disabled

MPU Region Number

LL_MPU_REGION_NUMBER0

REGION Number 0

LL_MPU_REGION_NUMBER1

REGION Number 1

LL_MPU_REGION_NUMBER2

REGION Number 2

LL_MPU_REGION_NUMBER3

REGION Number 3

LL_MPU_REGION_NUMBER4

REGION Number 4

LL_MPU_REGION_NUMBER5

REGION Number 5

LL_MPU_REGION_NUMBER6

REGION Number 6

LL_MPU_REGION_NUMBER7

REGION Number 7

MPU Region Privileges

LL_MPU_REGION_NO_ACCESS

No access

LL_MPU_REGION_PRIV_RW

RW privileged (privileged access only)

LL_MPU_REGION_PRIV_RW_URO

RW privileged - RO user (Write in a user program generates a fault)

LL_MPU_REGION_FULL_ACCESS

RW privileged & user (Full access)

LL_MPU_REGION_PRIV_RO

RO privileged (privileged read only)

LL_MPU_REGION_PRIV_RO_URO

RO privileged & user (read only)

MPU Region Size

LL_MPU_REGION_SIZE_32B

32B Size of the MPU protection region

LL_MPU_REGION_SIZE_64B

64B Size of the MPU protection region

LL_MPU_REGION_SIZE_128B

128B Size of the MPU protection region

LL_MPU_REGION_SIZE_256B

256B Size of the MPU protection region

LL_MPU_REGION_SIZE_512B

512B Size of the MPU protection region

LL_MPU_REGION_SIZE_1KB

1KB Size of the MPU protection region

LL_MPU_REGION_SIZE_2KB

2KB Size of the MPU protection region

LL_MPU_REGION_SIZE_4KB

4KB Size of the MPU protection region

LL_MPU_REGION_SIZE_8KB

8KB Size of the MPU protection region

LL_MPU_REGION_SIZE_16KB

16KB Size of the MPU protection region

LL_MPU_REGION_SIZE_32KB

32KB Size of the MPU protection region

LL_MPU_REGION_SIZE_64KB

64KB Size of the MPU protection region

LL_MPU_REGION_SIZE_128KB

128KB Size of the MPU protection region

LL_MPU_REGION_SIZE_256KB

256KB Size of the MPU protection region

LL_MPU_REGION_SIZE_512KB

512KB Size of the MPU protection region

LL_MPU_REGION_SIZE_1MB

1MB Size of the MPU protection region

LL_MPU_REGION_SIZE_2MB

2MB Size of the MPU protection region

LL_MPU_REGION_SIZE_4MB

4MB Size of the MPU protection region

LL_MPU_REGION_SIZE_8MB

8MB Size of the MPU protection region

LL_MPU_REGION_SIZE_16MB

16MB Size of the MPU protection region

LL_MPU_REGION_SIZE_32MB

32MB Size of the MPU protection region

LL_MPU_REGION_SIZE_64MB

64MB Size of the MPU protection region

LL_MPU_REGION_SIZE_128MB

128MB Size of the MPU protection region

LL_MPU_REGION_SIZE_256MB

256MB Size of the MPU protection region

LL_MPU_REGION_SIZE_512MB

512MB Size of the MPU protection region

LL_MPU_REGION_SIZE_1GB

1GB Size of the MPU protection region

LL_MPU_REGION_SIZE_2GB

2GB Size of the MPU protection region

LL_MPU_REGION_SIZE_4GB

4GB Size of the MPU protection region

MPU Shareable Access

LL_MPU_ACCESS_SHAREABLE

Shareable memory attribute

LL_MPU_ACCESS_NOT_SHAREABLE

Not Shareable memory attribute

MPU TEX Level

LL_MPU_TEX_LEVEL0

b000 for TEX bits

LL_MPU_TEX_LEVEL1

b001 for TEX bits

LL_MPU_TEX_LEVEL2

b010 for TEX bits

LL_MPU_TEX_LEVEL4

b100 for TEX bits

52 LL CRC Generic Driver

52.1 CRC Firmware driver API description

The following section lists the various functions of the CRC library.

52.1.1 Detailed description of functions

`LL_CRC_ResetCRCCalculationUnit`

Function name

`__STATIC_INLINE void LL_CRC_ResetCRCCalculationUnit (CRC_TypeDef * CRCx)`

Function description

Reset the CRC calculation unit.

Parameters

- **CRCx:** CRC Instance

Return values

- **None:**

Notes

- If Programmable Initial CRC value feature is available, also set the Data Register to the value stored in the CRC_INIT register, otherwise, reset Data Register to its default value.

Reference Manual to LL API cross reference:

- CR RESET `LL_CRC_ResetCRCCalculationUnit`

`LL_CRC_FeedData32`

Function name

`__STATIC_INLINE void LL_CRC_FeedData32 (CRC_TypeDef * CRCx, uint32_t InData)`

Function description

Write given 32-bit data to the CRC calculator.

Parameters

- **CRCx:** CRC Instance
- **InData:** value to be provided to CRC calculator between between Min_Data=0 and Max_Data=0xFFFFFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR `LL_CRC_FeedData32`

`LL_CRC_ReadData32`

Function name

`__STATIC_INLINE uint32_t LL_CRC_ReadData32 (CRC_TypeDef * CRCx)`

Function description

Return current CRC calculation result.

Parameters

- **CRCx:** CRC Instance

Return values

- **Current:** CRC calculation result as stored in CRC_DR register (32 bits).

Reference Manual to LL API cross reference:

- DR DR LL_CRC_ReadData32
LL_CRC_Read_IDR

Function name

__STATIC_INLINE uint32_t LL_CRC_Read_IDR (CRC_TypeDef * CRCx)

Function description

Return data stored in the Independent Data(IDR) register.

Parameters

- **CRCx:** CRC Instance

Return values

- **Value:** stored in CRC_IDR register (General-purpose 8-bit data register).

Notes

- This register can be used as a temporary storage location for one byte.

Reference Manual to LL API cross reference:

- IDR IDR LL_CRC_Read_IDR
LL_CRC_Write_IDR

Function name

__STATIC_INLINE void LL_CRC_Write_IDR (CRC_TypeDef * CRCx, uint32_t InData)

Function description

Store data in the Independent Data(IDR) register.

Parameters

- **CRCx:** CRC Instance
- **InData:** value to be stored in CRC_IDR register (8-bit) between Min_Data=0 and Max_Data=0xFF

Return values

- **None:**

Notes

- This register can be used as a temporary storage location for one byte.

Reference Manual to LL API cross reference:

- IDR IDR LL_CRC_Write_IDR
LL_CRC_DeInit

Function name

ErrorStatus LL_CRC_DeInit (CRC_TypeDef * CRCx)

Function description

De-initialize CRC registers (Registers restored to their default values).

Parameters

- **CRCx:** CRC Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: CRC registers are de-initialized
 - ERROR: CRC registers are not de-initialized

52.2 CRC Firmware driver defines

The following section lists the various define and macros of the module.

52.2.1 CRC

CRC

Common Write and read registers Macros

LL_CRC_WriteReg

Description:

- Write a value in CRC register.

Parameters:

- **__INSTANCE__:** CRC Instance
- **__REG__:** Register to be written
- **__VALUE__:** Value to be written in the register

Return value:

- None

LL_CRC_ReadReg

Description:

- Read a value in CRC register.

Parameters:

- **__INSTANCE__:** CRC Instance
- **__REG__:** Register to be read

Return value:

- Register: value

53 LL DAC Generic Driver

53.1 DAC Firmware driver registers structures

53.1.1 LL_DAC_InitTypeDef

LL_DAC_InitTypeDef is defined in the stm32f2xx_ll_dac.h

Data Fields

- **uint32_t TriggerSource**
- **uint32_t WaveAutoGeneration**
- **uint32_t WaveAutoGenerationConfig**
- **uint32_t OutputBuffer**

Field Documentation

- **uint32_t LL_DAC_InitTypeDef::TriggerSource**
Set the conversion trigger source for the selected DAC channel: internal (SW start) or from external peripheral (timer event, external interrupt line). This parameter can be a value of [DAC_LL_EC_TRIGGER_SOURCE](#). This feature can be modified afterwards using unitary function [LL_DAC_SetTriggerSource\(\)](#).
- **uint32_t LL_DAC_InitTypeDef::WaveAutoGeneration**
Set the waveform automatic generation mode for the selected DAC channel. This parameter can be a value of [DAC_LL_EC_WAVE_AUTO_GENERATION_MODE](#). This feature can be modified afterwards using unitary function [LL_DAC_SetWaveAutoGeneration\(\)](#).
- **uint32_t LL_DAC_InitTypeDef::WaveAutoGenerationConfig**
Set the waveform automatic generation mode for the selected DAC channel. If waveform automatic generation mode is set to noise, this parameter can be a value of [DAC_LL_EC_WAVE_NOISE_LFSR_UNMASK_BITS](#). If waveform automatic generation mode is set to triangle, this parameter can be a value of [DAC_LL_EC_WAVE_TRIANGLE_AMPLITUDE](#).
Note:
– If waveform automatic generation mode is disabled, this parameter is discarded.
This feature can be modified afterwards using unitary function [LL_DAC_SetWaveNoiseLFSR\(\)](#), [LL_DAC_SetWaveTriangleAmplitude\(\)](#) depending on the wave automatic generation selected.
- **uint32_t LL_DAC_InitTypeDef::OutputBuffer**
Set the output buffer for the selected DAC channel. This parameter can be a value of [DAC_LL_EC_OUTPUT_BUFFER](#). This feature can be modified afterwards using unitary function [LL_DAC_SetOutputBuffer\(\)](#).

53.2 DAC Firmware driver API description

The following section lists the various functions of the DAC library.

53.2.1 Detailed description of functions

[LL_DAC_SetTriggerSource](#)

Function name

```
__STATIC_INLINE void LL_DAC_SetTriggerSource (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t TriggerSource)
```

Function description

Set the conversion trigger source for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **TriggerSource:** This parameter can be one of the following values:
 - LL_DAC_TRIG_SOFTWARE
 - LL_DAC_TRIG_EXT_TIM8_TRGO
 - LL_DAC_TRIG_EXT_TIM7_TRGO
 - LL_DAC_TRIG_EXT_TIM6_TRGO
 - LL_DAC_TRIG_EXT_TIM5_TRGO
 - LL_DAC_TRIG_EXT_TIM4_TRGO
 - LL_DAC_TRIG_EXT_TIM2_TRGO
 - LL_DAC_TRIG_EXT_EXTI_LINE9

Return values

- **None:**

Notes

- For conversion trigger source to be effective, DAC trigger must be enabled using function LL_DAC_EnableTrigger().
- To set conversion trigger source, DAC channel must be disabled. Otherwise, the setting is discarded.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR TSEL1 LL_DAC_SetTriggerSource
- CR TSEL2 LL_DAC_SetTriggerSource

LL_DAC_GetTriggerSource

Function name

__STATIC_INLINE uint32_t LL_DAC_GetTriggerSource (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Get the conversion trigger source for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **Returned:** value can be one of the following values:
 - LL_DAC_TRIG_SOFTWARE
 - LL_DAC_TRIG_EXT_TIM8_TRGO
 - LL_DAC_TRIG_EXT_TIM7_TRGO
 - LL_DAC_TRIG_EXT_TIM6_TRGO
 - LL_DAC_TRIG_EXT_TIM5_TRGO
 - LL_DAC_TRIG_EXT_TIM4_TRGO
 - LL_DAC_TRIG_EXT_TIM2_TRGO
 - LL_DAC_TRIG_EXT_EXTI_LINE9

Notes

- For conversion trigger source to be effective, DAC trigger must be enabled using function `LL_DAC_EnableTrigger()`.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR TSEL1 `LL_DAC_GetTriggerSource`
- CR TSEL2 `LL_DAC_GetTriggerSource`

`LL_DAC_SetWaveAutoGeneration`

Function name

`__STATIC_INLINE void LL_DAC_SetWaveAutoGeneration (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t WaveAutoGeneration)`

Function description

Set the waveform automatic generation mode for the selected DAC channel.

Parameters

- DACx:** DAC instance
- DAC_Channel:** This parameter can be one of the following values:
 - `LL_DAC_CHANNEL_1`
 - `LL_DAC_CHANNEL_2`
- WaveAutoGeneration:** This parameter can be one of the following values:
 - `LL_DAC_WAVE_AUTO_GENERATION_NONE`
 - `LL_DAC_WAVE_AUTO_GENERATION_NOISE`
 - `LL_DAC_WAVE_AUTO_GENERATION_TRIANGLE`

Return values

- None:**

Reference Manual to LL API cross reference:

- CR WAVE1 `LL_DAC_SetWaveAutoGeneration`
- CR WAVE2 `LL_DAC_SetWaveAutoGeneration`

`LL_DAC_GetWaveAutoGeneration`

Function name

`__STATIC_INLINE uint32_t LL_DAC_GetWaveAutoGeneration (DAC_TypeDef * DACx, uint32_t DAC_Channel)`

Function description

Get the waveform automatic generation mode for the selected DAC channel.

Parameters

- DACx:** DAC instance
- DAC_Channel:** This parameter can be one of the following values:
 - `LL_DAC_CHANNEL_1`
 - `LL_DAC_CHANNEL_2`

Return values

- **Returned:** value can be one of the following values:
 - LL_DAC_WAVE_AUTO_GENERATION_NONE
 - LL_DAC_WAVE_AUTO_GENERATION_NOISE
 - LL_DAC_WAVE_AUTO_GENERATION_TRIANGLE

Reference Manual to LL API cross reference:

- CR WAVE1 LL_DAC_GetWaveAutoGeneration
- CR WAVE2 LL_DAC_GetWaveAutoGeneration

LL_DAC_SetWaveNoiseLFSR

Function name

__STATIC_INLINE void LL_DAC_SetWaveNoiseLFSR (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t NoiseLFSRMask)

Function description

Set the noise waveform generation for the selected DAC channel: Noise mode and parameters LFSR (linear feedback shift register).

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **NoiseLFSRMask:** This parameter can be one of the following values:
 - LL_DAC_NOISE_LFSR_UNMASK_BIT0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS1_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS2_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS3_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS4_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS5_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS6_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS7_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS8_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS9_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS10_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS11_0

Return values

- **None:**

Notes

- For wave generation to be effective, DAC channel wave generation mode must be enabled using function LL_DAC_SetWaveAutoGeneration().
- This setting can be set when the selected DAC channel is disabled (otherwise, the setting operation is ignored).

Reference Manual to LL API cross reference:

- CR MAMP1 LL_DAC_SetWaveNoiseLFSR
- CR MAMP2 LL_DAC_SetWaveNoiseLFSR

LL_DAC_GetWaveNoiseLFSR

Function name

```
__STATIC_INLINE uint32_t LL_DAC_GetWaveNoiseLFSR (DAC_TypeDef * DACx, uint32_t DAC_Channel)
```

Function description

Get the noise waveform generation for the selected DAC channel: Noise mode and parameters LFSR (linear feedback shift register).

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **Returned:** value can be one of the following values:
 - LL_DAC_NOISE_LFSR_UNMASK_BIT0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS1_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS2_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS3_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS4_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS5_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS6_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS7_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS8_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS9_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS10_0
 - LL_DAC_NOISE_LFSR_UNMASK_BITS11_0

Reference Manual to LL API cross reference:

- CR MAMP1 LL_DAC_GetWaveNoiseLFSR
- CR MAMP2 LL_DAC_GetWaveNoiseLFSR

```
LL_DAC_SetWaveTriangleAmplitude
```

Function name

```
__STATIC_INLINE void LL_DAC_SetWaveTriangleAmplitude (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t TriangleAmplitude)
```

Function description

Set the triangle waveform generation for the selected DAC channel: triangle mode and amplitude.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **TriangleAmplitude:** This parameter can be one of the following values:
 - LL_DAC_TRIANGLE_AMPLITUDE_1
 - LL_DAC_TRIANGLE_AMPLITUDE_3
 - LL_DAC_TRIANGLE_AMPLITUDE_7
 - LL_DAC_TRIANGLE_AMPLITUDE_15
 - LL_DAC_TRIANGLE_AMPLITUDE_31
 - LL_DAC_TRIANGLE_AMPLITUDE_63
 - LL_DAC_TRIANGLE_AMPLITUDE_127
 - LL_DAC_TRIANGLE_AMPLITUDE_255
 - LL_DAC_TRIANGLE_AMPLITUDE_511
 - LL_DAC_TRIANGLE_AMPLITUDE_1023
 - LL_DAC_TRIANGLE_AMPLITUDE_2047
 - LL_DAC_TRIANGLE_AMPLITUDE_4095

Return values

- **None:**

Notes

- For wave generation to be effective, DAC channel wave generation mode must be enabled using function LL_DAC_SetWaveAutoGeneration().
- This setting can be set when the selected DAC channel is disabled (otherwise, the setting operation is ignored).

Reference Manual to LL API cross reference:

- CR MAMP1 LL_DAC_SetWaveTriangleAmplitude
- CR MAMP2 LL_DAC_SetWaveTriangleAmplitude

LL_DAC_GetWaveTriangleAmplitude

Function name

__STATIC_INLINE uint32_t LL_DAC_GetWaveTriangleAmplitude (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Get the triangle waveform generation for the selected DAC channel: triangle mode and amplitude.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **Returned:** value can be one of the following values:
 - LL_DAC_TRIANGLE_AMPLITUDE_1
 - LL_DAC_TRIANGLE_AMPLITUDE_3
 - LL_DAC_TRIANGLE_AMPLITUDE_7
 - LL_DAC_TRIANGLE_AMPLITUDE_15
 - LL_DAC_TRIANGLE_AMPLITUDE_31
 - LL_DAC_TRIANGLE_AMPLITUDE_63
 - LL_DAC_TRIANGLE_AMPLITUDE_127
 - LL_DAC_TRIANGLE_AMPLITUDE_255
 - LL_DAC_TRIANGLE_AMPLITUDE_511
 - LL_DAC_TRIANGLE_AMPLITUDE_1023
 - LL_DAC_TRIANGLE_AMPLITUDE_2047
 - LL_DAC_TRIANGLE_AMPLITUDE_4095

Reference Manual to LL API cross reference:

- CR MAMP1 LL_DAC_GetWaveTriangleAmplitude
- CR MAMP2 LL_DAC_GetWaveTriangleAmplitude

LL_DAC_SetOutputBuffer

Function name

__STATIC_INLINE void LL_DAC_SetOutputBuffer (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t OutputBuffer)

Function description

Set the output buffer for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **OutputBuffer:** This parameter can be one of the following values:
 - LL_DAC_OUTPUT_BUFFER_ENABLE
 - LL_DAC_OUTPUT_BUFFER_DISABLE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR BOFF1 LL_DAC_SetOutputBuffer
- CR BOFF2 LL_DAC_SetOutputBuffer

LL_DAC_GetOutputBuffer

Function name

__STATIC_INLINE uint32_t LL_DAC_GetOutputBuffer (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Get the output buffer state for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **Returned:** value can be one of the following values:
 - LL_DAC_OUTPUT_BUFFER_ENABLE
 - LL_DAC_OUTPUT_BUFFER_DISABLE

Reference Manual to LL API cross reference:

- CR BOFF1 LL_DAC_GetOutputBuffer
- CR BOFF2 LL_DAC_GetOutputBuffer

LL_DAC_EnableDMAReq

Function name

__STATIC_INLINE void LL_DAC_EnableDMAReq (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Enable DAC DMA transfer request of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Notes

- To configure DMA source address (peripheral address), use function LL_DAC_DMA_GetRegAddr().

Reference Manual to LL API cross reference:

- CR DMAEN1 LL_DAC_EnableDMAReq
- CR DMAEN2 LL_DAC_EnableDMAReq

LL_DAC_DisableDMAReq

Function name

__STATIC_INLINE void LL_DAC_DisableDMAReq (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Disable DAC DMA transfer request of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Notes

- To configure DMA source address (peripheral address), use function `LL_DAC_DMA_GetRegAddr()`.

Reference Manual to LL API cross reference:

- CR DMAEN1 `LL_DAC_DisableDMAReq`
- CR DMAEN2 `LL_DAC_DisableDMAReq`

`LL_DAC_IsDMAReqEnabled`

Function name

__STATIC_INLINE uint32_t LL_DAC_IsDMAReqEnabled (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Get DAC DMA transfer request state of the selected channel.

Parameters

- DACx:** DAC instance
- DAC_Channel:** This parameter can be one of the following values:
 - `LL_DAC_CHANNEL_1`
 - `LL_DAC_CHANNEL_2`

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR DMAEN1 `LL_DAC_IsDMAReqEnabled`
- CR DMAEN2 `LL_DAC_IsDMAReqEnabled`

`LL_DAC_DMA_GetRegAddr`

Function name

__STATIC_INLINE uint32_t LL_DAC_DMA_GetRegAddr (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t Register)

Function description

Function to help to configure DMA transfer to DAC: retrieve the DAC register address from DAC instance and a list of DAC registers intended to be used (most commonly) with DMA transfer.

Parameters

- DACx:** DAC instance
- DAC_Channel:** This parameter can be one of the following values:
 - `LL_DAC_CHANNEL_1`
 - `LL_DAC_CHANNEL_2`
- Register:** This parameter can be one of the following values:
 - `LL_DAC_DMA_REG_DATA_12BITS_RIGHT_ALIGNED`
 - `LL_DAC_DMA_REG_DATA_12BITS_LEFT_ALIGNED`
 - `LL_DAC_DMA_REG_DATA_8BITS_RIGHT_ALIGNED`

Return values

- DAC:** register address

Notes

- These DAC registers are data holding registers: when DAC conversion is requested, DAC generates a DMA transfer request to have data available in DAC data holding registers.
- This macro is intended to be used with LL DMA driver, refer to function "LL_DMA_ConfigAddresses()". Example: LL_DMA_ConfigAddresses(DMA1, LL_DMA_CHANNEL_1, (uint32_t)< array or variable >, LL_DAC_DMA_GetRegAddr(DAC1, LL_DAC_CHANNEL_1, LL_DAC_DMA_REG_DATA_12BITS_RIGHT_ALIGNED), LL_DMA_DIRECTION_MEMORY_TO_PERIPH);

Reference Manual to LL API cross reference:

- DHR12R1 DACC1DHR LL_DAC_DMA_GetRegAddr
- DHR12L1 DACC1DHR LL_DAC_DMA_GetRegAddr
- DHR8R1 DACC1DHR LL_DAC_DMA_GetRegAddr
- DHR12R2 DACC2DHR LL_DAC_DMA_GetRegAddr
- DHR12L2 DACC2DHR LL_DAC_DMA_GetRegAddr
- DHR8R2 DACC2DHR LL_DAC_DMA_GetRegAddr

LL_DAC_Enable

Function name

__STATIC_INLINE void LL_DAC_Enable (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Enable DAC selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Notes

- After enable from off state, DAC channel requires a delay for output voltage to reach accuracy +/- 1 LSB. Refer to device datasheet, parameter "tWAKEUP".

Reference Manual to LL API cross reference:

- CR EN1 LL_DAC_Enable
- CR EN2 LL_DAC_Enable

LL_DAC_Disable

Function name

__STATIC_INLINE void LL_DAC_Disable (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Disable DAC selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR EN1 LL_DAC_Disable
- CR EN2 LL_DAC_Disable

LL_DAC_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_DAC_IsEnabled (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Get DAC enable state of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR EN1 LL_DAC_IsEnabled
- CR EN2 LL_DAC_IsEnabled

LL_DAC_EnableTrigger

Function name

__STATIC_INLINE void LL_DAC_EnableTrigger (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Enable DAC trigger of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Notes

- - If DAC trigger is disabled, DAC conversion is performed automatically once the data holding register is updated, using functions "LL_DAC_ConvertData{8; 12}{Right; Left} Aligned()": LL_DAC_ConvertData12RightAligned(), ... If DAC trigger is enabled, DAC conversion is performed only when a hardware or software trigger event is occurring. Select trigger source using function LL_DAC_SetTriggerSource().

Reference Manual to LL API cross reference:

- CR TEN1 LL_DAC_EnableTrigger
- CR TEN2 LL_DAC_EnableTrigger

LL_DAC_DisableTrigger

Function name

```
__STATIC_INLINE void LL_DAC_DisableTrigger (DAC_TypeDef * DACx, uint32_t DAC_Channel)
```

Function description

Disable DAC trigger of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR TEN1 LL_DAC_DisableTrigger
- CR TEN2 LL_DAC_DisableTrigger

LL_DAC_IsTriggerEnabled

Function name

```
__STATIC_INLINE uint32_t LL_DAC_IsTriggerEnabled (DAC_TypeDef * DACx, uint32_t DAC_Channel)
```

Function description

Get DAC trigger state of the selected channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR TEN1 LL_DAC_IsTriggerEnabled
- CR TEN2 LL_DAC_IsTriggerEnabled

LL_DAC_TrigSWConversion

Function name

```
__STATIC_INLINE void LL_DAC_TrigSWConversion (DAC_TypeDef * DACx, uint32_t DAC_Channel)
```

Function description

Trig DAC conversion by software for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can a combination of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **None:**

Notes

- Preliminarily, DAC trigger must be set to software trigger using function LL_DAC_Init() LL_DAC_SetTriggerSource() with parameter "LL_DAC_TRIGGER_SOFTWARE". and DAC trigger must be enabled using function LL_DAC_EnableTrigger().
- For devices featuring DAC with 2 channels: this function can perform a SW start of both DAC channels simultaneously. Two channels can be selected as parameter. Example: (LL_DAC_CHANNEL_1 | LL_DAC_CHANNEL_2)

Reference Manual to LL API cross reference:

- SWTRIGR SWTRIG1 LL_DAC_TrigSWConversion
- SWTRIGR SWTRIG2 LL_DAC_TrigSWConversion

LL_DAC_ConvertData12RightAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertData12RightAligned (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t Data)

Function description

Set the data to be loaded in the data holding register in format 12 bits left alignment (LSB aligned on bit 0), for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **Data:** Value between Min_Data=0x000 and Max_Data=0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR12R1 DACC1DHR LL_DAC_ConvertData12RightAligned
- DHR12R2 DACC2DHR LL_DAC_ConvertData12RightAligned

LL_DAC_ConvertData12LeftAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertData12LeftAligned (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t Data)

Function description

Set the data to be loaded in the data holding register in format 12 bits left alignment (MSB aligned on bit 15), for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **Data:** Value between Min_Data=0x000 and Max_Data=0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR12L1 DACC1DHR LL_DAC_ConvertData12LeftAligned
- DHR12L2 DACC2DHR LL_DAC_ConvertData12LeftAligned

LL_DAC_ConvertData8RightAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertData8RightAligned (DAC_TypeDef * DACx, uint32_t DAC_Channel, uint32_t Data)

Function description

Set the data to be loaded in the data holding register in format 8 bits left alignment (LSB aligned on bit 0), for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **Data:** Value between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR8R1 DACC1DHR LL_DAC_ConvertData8RightAligned
- DHR8R2 DACC2DHR LL_DAC_ConvertData8RightAligned

LL_DAC_ConvertDualData12RightAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertDualData12RightAligned (DAC_TypeDef * DACx, uint32_t DataChannel1, uint32_t DataChannel2)

Function description

Set the data to be loaded in the data holding register in format 12 bits left alignment (LSB aligned on bit 0), for both DAC channels.

Parameters

- **DACx:** DAC instance
- **DataChannel1:** Value between Min_Data=0x000 and Max_Data=0xFFF
- **DataChannel2:** Value between Min_Data=0x000 and Max_Data=0xFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR12RD DACC1DHR LL_DAC_ConvertDualData12RightAligned
- DHR12RD DACC2DHR LL_DAC_ConvertDualData12RightAligned

LL_DAC_ConvertDualData12LeftAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertDualData12LeftAligned (DAC_TypeDef * DACx, uint32_t DataChannel1, uint32_t DataChannel2)

Function description

Set the data to be loaded in the data holding register in format 12 bits left alignment (MSB aligned on bit 15), for both DAC channels.

Parameters

- **DACx:** DAC instance
- **DataChannel1:** Value between Min_Data=0x000 and Max_Data=0xFFFF
- **DataChannel2:** Value between Min_Data=0x000 and Max_Data=0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR12LD DAC1DHR LL_DAC_ConvertDualData12LeftAligned
- DHR12LD DAC2DHR LL_DAC_ConvertDualData12LeftAligned

LL_DAC_ConvertDualData8RightAligned

Function name

__STATIC_INLINE void LL_DAC_ConvertDualData8RightAligned (DAC_TypeDef * DACx, uint32_t DataChannel1, uint32_t DataChannel2)

Function description

Set the data to be loaded in the data holding register in format 8 bits left alignment (LSB aligned on bit 0), for both DAC channels.

Parameters

- **DACx:** DAC instance
- **DataChannel1:** Value between Min_Data=0x00 and Max_Data=0xFF
- **DataChannel2:** Value between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DHR8RD DAC1DHR LL_DAC_ConvertDualData8RightAligned
- DHR8RD DAC2DHR LL_DAC_ConvertDualData8RightAligned

LL_DAC_RetrieveOutputData

Function name

__STATIC_INLINE uint32_t LL_DAC_RetrieveOutputData (DAC_TypeDef * DACx, uint32_t DAC_Channel)

Function description

Retrieve output data currently generated for the selected DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFFF

Notes

- Whatever alignment and resolution settings (using functions "LL_DAC_ConvertData{8; 12}{Right; Left} Aligned()": LL_DAC_ConvertData12RightAligned(), ...), output data format is 12 bits right aligned (LSB aligned on bit 0).

Reference Manual to LL API cross reference:

- DOR1 DACC1DOR LL_DAC_RetrieveOutputData
- DOR2 DACC2DOR LL_DAC_RetrieveOutputData

LL_DAC_IsActiveFlag_DMAUDR1

Function name

__STATIC_INLINE uint32_t LL_DAC_IsActiveFlag_DMAUDR1 (DAC_TypeDef * DACx)

Function description

Get DAC underrun flag for DAC channel 1.

Parameters

- DACx:** DAC instance

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR DMAUDR1 LL_DAC_IsActiveFlag_DMAUDR1

LL_DAC_IsActiveFlag_DMAUDR2

Function name

__STATIC_INLINE uint32_t LL_DAC_IsActiveFlag_DMAUDR2 (DAC_TypeDef * DACx)

Function description

Get DAC underrun flag for DAC channel 2.

Parameters

- DACx:** DAC instance

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR DMAUDR2 LL_DAC_IsActiveFlag_DMAUDR2

LL_DAC_ClearFlag_DMAUDR1

Function name

__STATIC_INLINE void LL_DAC_ClearFlag_DMAUDR1 (DAC_TypeDef * DACx)

Function description

Clear DAC underrun flag for DAC channel 1.

Parameters

- DACx:** DAC instance

Return values

- None:**

Reference Manual to LL API cross reference:

- SR DMAUDR1 LL_DAC_ClearFlag_DMAUDR1
LL_DAC_ClearFlag_DMAUDR2

Function name

__STATIC_INLINE void LL_DAC_ClearFlag_DMAUDR2 (DAC_TypeDef * DACx)

Function description

Clear DAC underrun flag for DAC channel 2.

Parameters

- **DACx:** DAC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR DMAUDR2 LL_DAC_ClearFlag_DMAUDR2
LL_DAC_EnableIT_DMAUDR1

Function name

__STATIC_INLINE void LL_DAC_EnableIT_DMAUDR1 (DAC_TypeDef * DACx)

Function description

Enable DMA underrun interrupt for DAC channel 1.

Parameters

- **DACx:** DAC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMAUDRIE1 LL_DAC_EnableIT_DMAUDR1
LL_DAC_EnableIT_DMAUDR2

Function name

__STATIC_INLINE void LL_DAC_EnableIT_DMAUDR2 (DAC_TypeDef * DACx)

Function description

Enable DMA underrun interrupt for DAC channel 2.

Parameters

- **DACx:** DAC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMAUDRIE2 LL_DAC_EnableIT_DMAUDR2
LL_DAC_DisableIT_DMAUDR1

Function name

__STATIC_INLINE void LL_DAC_DisableIT_DMAUDR1 (DAC_TypeDef * DACx)

Function description

Disable DMA underrun interrupt for DAC channel 1.

Parameters

- **DACx:** DAC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMAUDRIE1 LL_DAC_DisableIT_DMAUDR1
LL_DAC_DisableIT_DMAUDR2

Function name

__STATIC_INLINE void LL_DAC_DisableIT_DMAUDR2 (DAC_TypeDef * DACx)

Function description

Disable DMA underrun interrupt for DAC channel 2.

Parameters

- **DACx:** DAC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMAUDRIE2 LL_DAC_DisableIT_DMAUDR2
LL_DAC_IsEnabledIT_DMAUDR1

Function name

__STATIC_INLINE uint32_t LL_DAC_IsEnabledIT_DMAUDR1 (DAC_TypeDef * DACx)

Function description

Get DMA underrun interrupt for DAC channel 1.

Parameters

- **DACx:** DAC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR DMAUDRIE1 LL_DAC_IsEnabledIT_DMAUDR1
LL_DAC_IsEnabledIT_DMAUDR2

Function name

__STATIC_INLINE uint32_t LL_DAC_IsEnabledIT_DMAUDR2 (DAC_TypeDef * DACx)

Function description

Get DMA underrun interrupt for DAC channel 2.

Parameters

- **DACx:** DAC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR DMAUDRIE2 LL_DAC_IsEnabledIT_DMAUDR2
LL_DAC_DeInit

Function name

ErrorStatus LL_DAC_DeInit (DAC_TypeDef * DACx)

Function description

De-initialize registers of the selected DAC instance to their default reset values.

Parameters

- **DACx:** DAC instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: DAC registers are de-initialized
 - ERROR: not applicable

LL_DAC_Init

Function name

ErrorStatus LL_DAC_Init (DAC_TypeDef * DACx, uint32_t DAC_Channel, LL_DAC_InitTypeDef * DAC_InitStruct)

Function description

Initialize some features of DAC channel.

Parameters

- **DACx:** DAC instance
- **DAC_Channel:** This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2
- **DAC_InitStruct:** Pointer to a LL_DAC_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: DAC registers are initialized
 - ERROR: DAC registers are not initialized

Notes

- LL_DAC_Init() aims to ease basic configuration of a DAC channel. Leaving it ready to be enabled and output: a level by calling one of LL_DAC_ConvertData12RightAligned LL_DAC_ConvertData12LeftAligned LL_DAC_ConvertData8RightAligned or one of the supported autogenerated wave.
- This function allows configuration of: Output modeTriggerWave generation
- The setting of these parameters by function LL_DAC_Init() is conditioned to DAC state: DAC channel must be disabled.

LL_DAC_StructInit

Function name

void LL_DAC_StructInit (LL_DAC_InitTypeDef * DAC_InitStruct)

Function description

Set each LL_DAC_InitTypeDef field to default value.

Parameters

- **DAC_InitStruct:** pointer to a LL_DAC_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

53.3 DAC Firmware driver defines

The following section lists the various define and macros of the module.

53.3.1 DAC

DAC

DAC channels

LL_DAC_CHANNEL_1

DAC channel 1

LL_DAC_CHANNEL_2

DAC channel 2

DAC flags

LL_DAC_FLAG_DMAUDR1

DAC channel 1 flag DMA underrun

LL_DAC_FLAG_DMAUDR2

DAC channel 2 flag DMA underrun

Definitions of DAC hardware constraints delays

LL_DAC_DELAY_STARTUP_VOLTAGE_SETTLING_US

Delay for DAC channel voltage settling time from DAC channel startup (transition from disable to enable)

LL_DAC_DELAY_VOLTAGE_SETTLING_US

Delay for DAC channel voltage settling time

DAC interruptions

LL_DAC_IT_DMAUDRIE1

DAC channel 1 interruption DMA underrun

LL_DAC_IT_DMAUDRIE2

DAC channel 2 interruption DMA underrun

DAC channel output buffer

LL_DAC_OUTPUT_BUFFER_ENABLE

The selected DAC channel output is buffered: higher drive current capability, but also higher current consumption

LL_DAC_OUTPUT_BUFFER_DISABLE

The selected DAC channel output is not buffered: lower drive current capability, but also lower current consumption

DAC registers compliant with specific purpose

LL_DAC_DMA_REG_DATA_12BITS_RIGHT_ALIGNED

DAC channel data holding register 12 bits right aligned

LL_DAC_DMA_REG_DATA_12BITS_LEFT_ALIGNED

DAC channel data holding register 12 bits left aligned

LL_DAC_DMA_REG_DATA_8BITS_RIGHT_ALIGNED

DAC channel data holding register 8 bits right aligned

DAC channel output resolution

LL_DAC_RESOLUTION_12B

DAC channel resolution 12 bits

LL_DAC_RESOLUTION_8B

DAC channel resolution 8 bits

DAC trigger source

LL_DAC_TRIG_SOFTWARE

DAC channel conversion trigger internal (SW start)

LL_DAC_TRIG_EXT_TIM2_TRGO

DAC channel conversion trigger from external peripheral: TIM2 TRGO.

LL_DAC_TRIG_EXT_TIM8_TRGO

DAC channel conversion trigger from external peripheral: TIM8 TRGO.

LL_DAC_TRIG_EXT_TIM4_TRGO

DAC channel conversion trigger from external peripheral: TIM4 TRGO.

LL_DAC_TRIG_EXT_TIM6_TRGO

DAC channel conversion trigger from external peripheral: TIM6 TRGO.

LL_DAC_TRIG_EXT_TIM7_TRGO

DAC channel conversion trigger from external peripheral: TIM7 TRGO.

LL_DAC_TRIG_EXT_TIM5_TRGO

DAC channel conversion trigger from external peripheral: TIM5 TRGO.

LL_DAC_TRIG_EXT_EXTI_LINE9

DAC channel conversion trigger from external peripheral: external interrupt line 9.

DAC waveform automatic generation mode

LL_DAC_WAVE_AUTO_GENERATION_NONE

DAC channel wave auto generation mode disabled.

LL_DAC_WAVE_AUTO_GENERATION_NOISE

DAC channel wave auto generation mode enabled, set generated noise waveform.

LL_DAC_WAVE_AUTO_GENERATION_TRIANGLE

DAC channel wave auto generation mode enabled, set generated triangle waveform.

DAC wave generation - Noise LFSR unmask bits

LL_DAC_NOISE_LFSR_UNMASK_BIT0

Noise wave generation, unmask LFSR bit0, for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS1_0

Noise wave generation, unmask LFSR bits[1:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS2_0

Noise wave generation, unmask LFSR bits[2:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS3_0

Noise wave generation, unmask LFSR bits[3:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS4_0

Noise wave generation, unmask LFSR bits[4:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS5_0

Noise wave generation, unmask LFSR bits[5:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS6_0

Noise wave generation, unmask LFSR bits[6:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS7_0

Noise wave generation, unmask LFSR bits[7:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS8_0

Noise wave generation, unmask LFSR bits[8:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS9_0

Noise wave generation, unmask LFSR bits[9:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS10_0

Noise wave generation, unmask LFSR bits[10:0], for the selected DAC channel

LL_DAC_NOISE_LFSR_UNMASK_BITS11_0

Noise wave generation, unmask LFSR bits[11:0], for the selected DAC channel

DAC wave generation - Triangle amplitude

LL_DAC_TRIANGLE_AMPLITUDE_1

Triangle wave generation, amplitude of 1 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_3

Triangle wave generation, amplitude of 3 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_7

Triangle wave generation, amplitude of 7 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_15

Triangle wave generation, amplitude of 15 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_31

Triangle wave generation, amplitude of 31 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_63

Triangle wave generation, amplitude of 63 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_127

Triangle wave generation, amplitude of 127 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_255

Triangle wave generation, amplitude of 255 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_511

Triangle wave generation, amplitude of 512 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_1023

Triangle wave generation, amplitude of 1023 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_2047

Triangle wave generation, amplitude of 2047 LSB of DAC output range, for the selected DAC channel

LL_DAC_TRIANGLE_AMPLITUDE_4095

Triangle wave generation, amplitude of 4095 LSB of DAC output range, for the selected DAC channel

DAC helper macro

__LL_DAC_CHANNEL_TO_DECIMAL_NB

Description:

- Helper macro to get DAC channel number in decimal format from literals LL_DAC_CHANNEL_x.

Parameters:

- __CHANNEL__: This parameter can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Return value:

- 1...2

Notes:

- The input can be a value from functions where a channel number is returned.

__LL_DAC_DECIMAL_NB_TO_CHANNEL

Description:

- Helper macro to get DAC channel in literal format LL_DAC_CHANNEL_x from number in decimal format.

Parameters:

- __DECIMAL_NB__: 1...2

Return value:

- Returned: value can be one of the following values:
 - LL_DAC_CHANNEL_1
 - LL_DAC_CHANNEL_2

Notes:

- If the input parameter does not correspond to a DAC channel, this macro returns value '0'.

__LL_DAC_DIGITAL_SCALE

Description:

- Helper macro to define the DAC conversion data full-scale digital value corresponding to the selected DAC resolution.

Parameters:

- __DAC_RESOLUTION__: This parameter can be one of the following values:
 - LL_DAC_RESOLUTION_12B
 - LL_DAC_RESOLUTION_8B

Return value:

- ADC: conversion data equivalent voltage value (unit: mVolt)

Notes:

- DAC conversion data full-scale corresponds to voltage range determined by analog voltage references Vref+ and Vref- (refer to reference manual).

__LL_DAC_CALC_VOLTAGE_TO_DATA

Description:

- Helper macro to calculate the DAC conversion data (unit: digital value) corresponding to a voltage (unit: mVolt).

Parameters:

- __VREFANALOG_VOLTAGE__: Analog reference voltage (unit: mV)
- __DAC_VOLTAGE__: Voltage to be generated by DAC channel (unit: mVolt).
- __DAC_RESOLUTION__: This parameter can be one of the following values:
 - LL_DAC_RESOLUTION_12B
 - LL_DAC_RESOLUTION_8B

Return value:

- DAC: conversion data (unit: digital value)

Notes:

- This helper macro is intended to provide input data in voltage rather than digital value, to be used with LL DAC functions such as LL_DAC_ConvertData12RightAligned(). Analog reference voltage (Vref+) must be either known from user board environment or can be calculated using ADC measurement and ADC helper macro __LL_ADC_CALC_VREFANALOG_VOLTAGE().

Common write and read registers macros

LL_DAC_WriteReg

Description:

- Write a value in DAC register.

Parameters:

- __INSTANCE__: DAC Instance
- __REG__: Register to be written
- __VALUE__: Value to be written in the register

Return value:

- None

LL_DAC_ReadReg

Description:

- Read a value in DAC register.

Parameters:

- __INSTANCE__: DAC Instance
- __REG__: Register to be read

Return value:

- Register: value

54 LL DMA Generic Driver

54.1 DMA Firmware driver registers structures

54.1.1 LL_DMA_InitTypeDef

LL_DMA_InitTypeDef is defined in the stm32f2xx_ll_dma.h

Data Fields

- **uint32_t** *PeriphOrM2MSrcAddress*
- **uint32_t** *MemoryOrM2MDstAddress*
- **uint32_t** *Direction*
- **uint32_t** *Mode*
- **uint32_t** *PeriphOrM2MSrcIncMode*
- **uint32_t** *MemoryOrM2MDstIncMode*
- **uint32_t** *PeriphOrM2MSrcDataSize*
- **uint32_t** *MemoryOrM2MDstDataSize*
- **uint32_t** *NbData*
- **uint32_t** *Channel*
- **uint32_t** *Priority*
- **uint32_t** *FIFOmode*
- **uint32_t** *FIFOThreshold*
- **uint32_t** *MemBurst*
- **uint32_t** *PeriphBurst*

Field Documentation

- **uint32_t** **LL_DMA_InitTypeDef::PeriphOrM2MSrcAddress**
Specifies the peripheral base address for DMA transfer or as Source base address in case of memory to memory transfer direction. This parameter must be a value between Min_Data = 0 and Max_Data = 0xFFFFFFFF.
- **uint32_t** **LL_DMA_InitTypeDef::MemoryOrM2MDstAddress**
Specifies the memory base address for DMA transfer or as Destination base address in case of memory to memory transfer direction. This parameter must be a value between Min_Data = 0 and Max_Data = 0xFFFFFFFF.
- **uint32_t** **LL_DMA_InitTypeDef::Direction**
Specifies if the data will be transferred from memory to peripheral, from memory to memory or from peripheral to memory. This parameter can be a value of [DMA_LL_EC_DIRECTION](#). This feature can be modified afterwards using unitary function **LL_DMA_SetDataTransferDirection()**.
- **uint32_t** **LL_DMA_InitTypeDef::Mode**
Specifies the normal or circular operation mode. This parameter can be a value of [DMA_LL_EC_MODE](#)

Note:

- The circular buffer mode cannot be used if the memory to memory data transfer direction is configured on the selected Stream

This feature can be modified afterwards using unitary function **LL_DMA_SetMode()**.

- **uint32_t** **LL_DMA_InitTypeDef::PeriphOrM2MSrcIncMode**
Specifies whether the Peripheral address or Source address in case of memory to memory transfer direction is incremented or not. This parameter can be a value of [DMA_LL_EC_PERIPH](#). This feature can be modified afterwards using unitary function **LL_DMA_SetPeriphIncMode()**.

- **`uint32_t LL_DMA_InitTypeDef::MemoryOrM2MDstIncMode`**
Specifies whether the Memory address or Destination address in case of memory to memory transfer direction is incremented or not. This parameter can be a value of [DMA_LL_EC_MEMORY](#). This feature can be modified afterwards using unitary function `LL_DMA_SetMemoryIncMode()`.
- **`uint32_t LL_DMA_InitTypeDef::PeriphOrM2MSrcDataSize`**
Specifies the Peripheral data size alignment or Source data size alignment (byte, half word, word) in case of memory to memory transfer direction. This parameter can be a value of [DMA_LL_EC_PDATAALIGN](#). This feature can be modified afterwards using unitary function `LL_DMA_SetPeriphSize()`.
- **`uint32_t LL_DMA_InitTypeDef::MemoryOrM2MDstDataSize`**
Specifies the Memory data size alignment or Destination data size alignment (byte, half word, word) in case of memory to memory transfer direction. This parameter can be a value of [DMA_LL_EC_MDATAALIGN](#). This feature can be modified afterwards using unitary function `LL_DMA_SetMemorySize()`.
- **`uint32_t LL_DMA_InitTypeDef::NbData`**
Specifies the number of data to transfer, in data unit. The data unit is equal to the source buffer configuration set in `PeriphSize` or `MemorySize` parameters depending in the transfer direction. This parameter must be a value between `Min_Data = 0` and `Max_Data = 0x0000FFFF`. This feature can be modified afterwards using unitary function `LL_DMA_SetDataLength()`.
- **`uint32_t LL_DMA_InitTypeDef::Channel`**
Specifies the peripheral channel. This parameter can be a value of [DMA_LL_EC_CHANNEL](#). This feature can be modified afterwards using unitary function `LL_DMA_SetChannelSelection()`.
- **`uint32_t LL_DMA_InitTypeDef::Priority`**
Specifies the channel priority level. This parameter can be a value of [DMA_LL_EC_PRIORITY](#). This feature can be modified afterwards using unitary function `LL_DMA_SetStreamPriorityLevel()`.
- **`uint32_t LL_DMA_InitTypeDef::FIFOMode`**
Specifies if the FIFO mode or Direct mode will be used for the specified stream. This parameter can be a value of [DMA_LL_FIFOMODE](#).
Note:
 - The Direct mode (FIFO mode disabled) cannot be used if the memory-to-memory data transfer is configured on the selected stream.This feature can be modified afterwards using unitary functions `LL_DMA_EnableFifoMode()` or `LL_DMA_EnableFifoMode()`.
- **`uint32_t LL_DMA_InitTypeDef::FIFOThreshold`**
Specifies the FIFO threshold level. This parameter can be a value of [DMA_LL_EC_FIFOTHRESHOLD](#). This feature can be modified afterwards using unitary function `LL_DMA_SetFIFOThreshold()`.
- **`uint32_t LL_DMA_InitTypeDef::MemBurst`**
Specifies the Burst transfer configuration for the memory transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_LL_EC_MBURST](#).
Note:
 - The burst mode is possible only if the address Increment mode is enabled.This feature can be modified afterwards using unitary function `LL_DMA_SetMemoryBurstxfer()`.
- **`uint32_t LL_DMA_InitTypeDef::PeriphBurst`**
Specifies the Burst transfer configuration for the peripheral transfers. It specifies the amount of data to be transferred in a single non interruptible transaction. This parameter can be a value of [DMA_LL_EC_PBURST](#).
Note:
 - The burst mode is possible only if the address Increment mode is enabled.This feature can be modified afterwards using unitary function `LL_DMA_SetPeriphBurstxfer()`.

54.2 DMA Firmware driver API description

The following section lists the various functions of the DMA library.

54.2.1 Detailed description of functions

LL_DMA_EnableStream

Function name

__STATIC_INLINE void LL_DMA_EnableStream (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable DMA stream.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR EN LL_DMA_EnableStream

LL_DMA_DisableStream

Function name

__STATIC_INLINE void LL_DMA_DisableStream (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable DMA stream.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR EN LL_DMA_DisableStream

LL_DMA_IsEnabledStream

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsEnabledStream (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Check if DMA stream is enabled or disabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR EN LL_DMA_IsEnabledStream

LL_DMA_ConfigTransfer

Function name

```
__STATIC_INLINE void LL_DMA_ConfigTransfer (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Configuration)
```

Function description

Configure all parameters linked to DMA transfer.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Configuration:** This parameter must be a combination of all the following values:
 - LL_DMA_DIRECTION_PERIPH_TO_MEMORY or LL_DMA_DIRECTION_MEMORY_TO_PERIPH or LL_DMA_DIRECTION_MEMORY_TO_MEMORY
 - LL_DMA_MODE_NORMAL or LL_DMA_MODE_CIRCULAR or LL_DMA_MODE_PFCTRL
 - LL_DMA_PERIPH_INCREMENT or LL_DMA_PERIPH_NOINCREMENT
 - LL_DMA_MEMORY_INCREMENT or LL_DMA_MEMORY_NOINCREMENT
 - LL_DMA_PDATAALIGN_BYTE or LL_DMA_PDATAALIGN_HALFWORD or LL_DMA_PDATAALIGN_WORD
 - LL_DMA_MDATAALIGN_BYTE or LL_DMA_MDATAALIGN_HALFWORD or LL_DMA_MDATAALIGN_WORD
 - LL_DMA_PRIORITY_LOW or LL_DMA_PRIORITY_MEDIUM or LL_DMA_PRIORITY_HIGH or LL_DMA_PRIORITY_VERYHIGH

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DIR LL_DMA_ConfigTransfer
- CR CIRC LL_DMA_ConfigTransfer
- CR PINC LL_DMA_ConfigTransfer
- CR MINC LL_DMA_ConfigTransfer
- CR PSIZE LL_DMA_ConfigTransfer
- CR MSIZE LL_DMA_ConfigTransfer
- CR PL LL_DMA_ConfigTransfer
- CR PFCTRL LL_DMA_ConfigTransfer

LL_DMA_SetDataTransferDirection

Function name

__STATIC_INLINE void LL_DMA_SetDataTransferDirection (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Direction)

Function description

Set Data transfer direction (read from peripheral or from memory).

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Direction:** This parameter can be one of the following values:
 - LL_DMA_DIRECTION_PERIPH_TO_MEMORY
 - LL_DMA_DIRECTION_MEMORY_TO_PERIPH
 - LL_DMA_DIRECTION_MEMORY_TO_MEMORY

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DIR LL_DMA_SetDataTransferDirection

LL_DMA_GetDataTransferDirection

Function name

__STATIC_INLINE uint32_t LL_DMA_GetDataTransferDirection (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Data transfer direction (read from peripheral or from memory).

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_DIRECTION_PERIPH_TO_MEMORY
 - LL_DMA_DIRECTION_MEMORY_TO_PERIPH
 - LL_DMA_DIRECTION_MEMORY_TO_MEMORY

Reference Manual to LL API cross reference:

- CR DIR LL_DMA_GetDataTransferDirection

LL_DMA_SetMode

Function name

```
__STATIC_INLINE void LL_DMA_SetMode (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Mode)
```

Function description

Set DMA mode normal, circular or peripheral flow control.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Mode:** This parameter can be one of the following values:
 - LL_DMA_MODE_NORMAL
 - LL_DMA_MODE_CIRCULAR
 - LL_DMA_MODE_PFCTRL

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CIRC LL_DMA_SetMode
- CR PFCTRL LL_DMA_SetMode

LL_DMA_GetMode

Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Get DMA mode normal, circular or peripheral flow control.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_MODE_NORMAL
 - LL_DMA_MODE_CIRCULAR
 - LL_DMA_MODE_PFCTRL

Reference Manual to LL API cross reference:

- CR CIRC LL_DMA_GetMode
- CR PFCTRL LL_DMA_GetMode

LL_DMA_SetPeriphIncMode

Function name

__STATIC_INLINE void LL_DMA_SetPeriphIncMode (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t IncrementMode)

Function description

Set Peripheral increment mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **IncrementMode:** This parameter can be one of the following values:
 - LL_DMA_PERIPH_NOINCREMENT
 - LL_DMA_PERIPH_INCREMENT

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PINC LL_DMA_SetPeriphIncMode

LL_DMA_GetPeriphIncMode

Function name

__STATIC_INLINE uint32_t LL_DMA_GetPeriphIncMode (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Peripheral increment mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_PERIPH_NOINCREMENT
 - LL_DMA_PERIPH_INCREMENT

Reference Manual to LL API cross reference:

- CR PINC LL_DMA_GetPeriphIncMode

LL_DMA_SetMemoryIncMode

Function name

__STATIC_INLINE void LL_DMA_SetMemoryIncMode (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t IncrementMode)

Function description

Set Memory increment mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **IncrementMode:** This parameter can be one of the following values:
 - LL_DMA_MEMORY_NOINCREMENT
 - LL_DMA_MEMORY_INCREMENT

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR MINC LL_DMA_SetMemoryIncMode

LL_DMA_GetMemoryIncMode

Function name

__STATIC_INLINE uint32_t LL_DMA_GetMemoryIncMode (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Memory increment mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_MEMORY_NOINCREMENT
 - LL_DMA_MEMORY_INCREMENT

Reference Manual to LL API cross reference:

- CR MINC LL_DMA_GetMemoryIncMode

LL_DMA_SetPeriphSize

Function name

__STATIC_INLINE void LL_DMA_SetPeriphSize (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Size)

Function description

Set Peripheral size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Size:** This parameter can be one of the following values:
 - LL_DMA_PDATAALIGN_BYTE
 - LL_DMA_PDATAALIGN_HALFWORD
 - LL_DMA_PDATAALIGN_WORD

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PSIZE LL_DMA_SetPeriphSize

LL_DMA_GetPeriphSize

Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetPeriphSize (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Get Peripheral size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_PDATAALIGN_BYTE
 - LL_DMA_PDATAALIGN_HALFWORD
 - LL_DMA_PDATAALIGN_WORD

Reference Manual to LL API cross reference:

- CR PSIZE LL_DMA_GetPeriphSize

LL_DMA_SetMemorySize

Function name

```
__STATIC_INLINE void LL_DMA_SetMemorySize (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Size)
```

Function description

Set Memory size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Size:** This parameter can be one of the following values:
 - LL_DMA_MDATAALIGN_BYTE
 - LL_DMA_MDATAALIGN_HALFWORD
 - LL_DMA_MDATAALIGN_WORD

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR MSIZE LL_DMA_SetMemorySize

LL_DMA_GetMemorySize

Function name

__STATIC_INLINE uint32_t LL_DMA_GetMemorySize (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Memory size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_MDATAALIGN_BYTE
 - LL_DMA_MDATAALIGN_HALFWORD
 - LL_DMA_MDATAALIGN_WORD

Reference Manual to LL API cross reference:

- CR MSIZE LL_DMA_GetMemorySize

LL_DMA_SetIncOffsetSize

Function name

__STATIC_INLINE void LL_DMA_SetIncOffsetSize (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t OffsetSize)

Function description

Set Peripheral increment offset size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **OffsetSize:** This parameter can be one of the following values:
 - LL_DMA_OFFSETSIZE_PSIZE
 - LL_DMA_OFFSETSIZE_FIXEDTO4

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PINCOS LL_DMA_SetIncOffsetSize

LL_DMA_GetIncOffsetSize

Function name

__STATIC_INLINE uint32_t LL_DMA_GetIncOffsetSize (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Peripheral increment offset size.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_OFFSETSIZE_PSIZE
 - LL_DMA_OFFSETSIZE_FIXEDTO4

Reference Manual to LL API cross reference:

- CR PINCOS LL_DMA_GetIncOffsetSize

LL_DMA_SetStreamPriorityLevel

Function name

__STATIC_INLINE void LL_DMA_SetStreamPriorityLevel (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Priority)

Function description

Set Stream priority level.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Priority:** This parameter can be one of the following values:
 - LL_DMA_PRIORITY_LOW
 - LL_DMA_PRIORITY_MEDIUM
 - LL_DMA_PRIORITY_HIGH
 - LL_DMA_PRIORITY_VERYHIGH

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PL LL_DMA_SetStreamPriorityLevel

LL_DMA_GetStreamPriorityLevel

Function name

__STATIC_INLINE uint32_t LL_DMA_GetStreamPriorityLevel (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Stream priority level.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_PRIORITY_LOW
 - LL_DMA_PRIORITY_MEDIUM
 - LL_DMA_PRIORITY_HIGH
 - LL_DMA_PRIORITY_VERYHIGH

Reference Manual to LL API cross reference:

- CR PL LL_DMA_GetStreamPriorityLevel

LL_DMA_SetDataLength

Function name

__STATIC_INLINE void LL_DMA_SetDataLength (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t NbData)

Function description

Set Number of data to transfer.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **NbData:** Between 0 to 0xFFFFFFFF

Return values

- **None:**

Notes

- This action has no effect if stream is enabled.

Reference Manual to LL API cross reference:

- NDTR NDT LL_DMA_SetDataLength

LL_DMA_GetDataLength

Function name

__STATIC_INLINE uint32_t LL_DMA_GetDataLength (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Number of data to transfer.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Between:** 0 to 0xFFFFFFFF

Notes

- Once the stream is enabled, the return value indicate the remaining bytes to be transmitted.

Reference Manual to LL API cross reference:

- NDTR NDT LL_DMA_GetDataLength
LL_DMA_SetChannelSelection

Function name

```
__STATIC_INLINE void LL_DMA_SetChannelSelection (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Channel)
```

Function description

Select Channel number associated to the Stream.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Channel:** This parameter can be one of the following values:
 - LL_DMA_CHANNEL_0
 - LL_DMA_CHANNEL_1
 - LL_DMA_CHANNEL_2
 - LL_DMA_CHANNEL_3
 - LL_DMA_CHANNEL_4
 - LL_DMA_CHANNEL_5
 - LL_DMA_CHANNEL_6
 - LL_DMA_CHANNEL_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CHSEL LL_DMA_SetChannelSelection
LL_DMA_GetChannelSelection

Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetChannelSelection (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Get the Channel number associated to the Stream.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_CHANNEL_0
 - LL_DMA_CHANNEL_1
 - LL_DMA_CHANNEL_2
 - LL_DMA_CHANNEL_3
 - LL_DMA_CHANNEL_4
 - LL_DMA_CHANNEL_5
 - LL_DMA_CHANNEL_6
 - LL_DMA_CHANNEL_7

Reference Manual to LL API cross reference:

- CR CHSEL LL_DMA_GetChannelSelection
- LL_DMA_SetMemoryBurstxfer

Function name

__STATIC_INLINE void LL_DMA_SetMemoryBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Mburst)

Function description

Set Memory burst transfer configuration.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Mburst:** This parameter can be one of the following values:
 - LL_DMA_MBURST_SINGLE
 - LL_DMA_MBURST_INC4
 - LL_DMA_MBURST_INC8
 - LL_DMA_MBURST_INC16

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR MBURST LL_DMA_SetMemoryBurstxfer

LL_DMA_GetMemoryBurstxfer

Function name

__STATIC_INLINE uint32_t LL_DMA_GetMemoryBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Memory burst transfer configuration.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_MBURST_SINGLE
 - LL_DMA_MBURST_INC4
 - LL_DMA_MBURST_INC8
 - LL_DMA_MBURST_INC16

Reference Manual to LL API cross reference:

- CR MBURST LL_DMA_GetMemoryBurstxfer

LL_DMA_SetPeriphBurstxfer

Function name

__STATIC_INLINE void LL_DMA_SetPeriphBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Pburst)

Function description

Set Peripheral burst transfer configuration.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Pburst:** This parameter can be one of the following values:
 - LL_DMA_PBURST_SINGLE
 - LL_DMA_PBURST_INC4
 - LL_DMA_PBURST_INC8
 - LL_DMA_PBURST_INC16

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PBURST LL_DMA_SetPeriphBurstxfer

LL_DMA_GetPeriphBurstxfer

Function name

__STATIC_INLINE uint32_t LL_DMA_GetPeriphBurstxfer (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Peripheral burst transfer configuration.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_PBURST_SINGLE
 - LL_DMA_PBURST_INC4
 - LL_DMA_PBURST_INC8
 - LL_DMA_PBURST_INC16

Reference Manual to LL API cross reference:

- CR PBURST LL_DMA_GetPeriphBurstxfer


```
LL_DMA_SetCurrentTargetMem
```

Function name

```
__STATIC_INLINE void LL_DMA_SetCurrentTargetMem (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t CurrentMemory)
```

Function description

Set Current target (only in double buffer mode) to Memory 1 or Memory 0.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **CurrentMemory:** This parameter can be one of the following values:
 - LL_DMA_CURRENTTARGETMEM0
 - LL_DMA_CURRENTTARGETMEM1

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CT LL_DMA_SetCurrentTargetMem

```
LL_DMA_GetCurrentTargetMem
```

Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetCurrentTargetMem (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Set Current target (only in double buffer mode) to Memory 1 or Memory 0.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_CURRENTTARGETMEM0
 - LL_DMA_CURRENTTARGETMEM1

Reference Manual to LL API cross reference:

- CR CT LL_DMA_GetCurrentTargetMem

LL_DMA_EnableDoubleBufferMode

Function name

__STATIC_INLINE void LL_DMA_EnableDoubleBufferMode (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable the double buffer mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DBM LL_DMA_EnableDoubleBufferMode

LL_DMA_DisableDoubleBufferMode

Function name

__STATIC_INLINE void LL_DMA_DisableDoubleBufferMode (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable the double buffer mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DBM LL_DMA_DisableDoubleBufferMode

```
LL_DMA_GetFIFOStatus
```

Function name

```
__STATIC_INLINE uint32_t LL_DMA_GetFIFOStatus (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Get FIFO status.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_FIFOSTATUS_0_25
 - LL_DMA_FIFOSTATUS_25_50
 - LL_DMA_FIFOSTATUS_50_75
 - LL_DMA_FIFOSTATUS_75_100
 - LL_DMA_FIFOSTATUS_EMPTY
 - LL_DMA_FIFOSTATUS_FULL

Reference Manual to LL API cross reference:

- FCR FS LL_DMA_GetFIFOStatus

```
LL_DMA_DisableFifoMode
```

Function name

```
__STATIC_INLINE void LL_DMA_DisableFifoMode (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Disable Fifo mode.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- FCR DMDIS LL_DMA_DisableFifoMode

LL_DMA_EnableFifoMode

Function name

__STATIC_INLINE void LL_DMA_EnableFifoMode (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable Fifo mode.

Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- None:**

Reference Manual to LL API cross reference:

- FCR DMDIS LL_DMA_EnableFifoMode

LL_DMA_SetFIFOThreshold

Function name

__STATIC_INLINE void LL_DMA_SetFIFOThreshold (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Threshold)

Function description

Select FIFO threshold.

Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- Threshold:** This parameter can be one of the following values:
 - LL_DMA_FIFOTHRESHOLD_1_4
 - LL_DMA_FIFOTHRESHOLD_1_2
 - LL_DMA_FIFOTHRESHOLD_3_4
 - LL_DMA_FIFOTHRESHOLD_FULL

Return values

- **None:**

Reference Manual to LL API cross reference:

- FCR FTH LL_DMA_SetFIFOThreshold

LL_DMA_GetFIFOThreshold

Function name

__STATIC_INLINE uint32_t LL_DMA_GetFIFOThreshold (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get FIFO threshold.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Returned:** value can be one of the following values:
 - LL_DMA_FIFOTHRESHOLD_1_4
 - LL_DMA_FIFOTHRESHOLD_1_2
 - LL_DMA_FIFOTHRESHOLD_3_4
 - LL_DMA_FIFOTHRESHOLD_FULL

Reference Manual to LL API cross reference:

- FCR FTH LL_DMA_GetFIFOThreshold

LL_DMA_ConfigFifo

Function name

__STATIC_INLINE void LL_DMA_ConfigFifo (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t FifoMode, uint32_t FifoThreshold)

Function description

Configure the FIFO .

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **FifoMode:** This parameter can be one of the following values:
 - LL_DMA_FIFOMODE_ENABLE
 - LL_DMA_FIFOMODE_DISABLE
- **FifoThreshold:** This parameter can be one of the following values:
 - LL_DMA_FIFOTHRESHOLD_1_4
 - LL_DMA_FIFOTHRESHOLD_1_2
 - LL_DMA_FIFOTHRESHOLD_3_4
 - LL_DMA_FIFOTHRESHOLD_FULL

Return values

- **None:**

Reference Manual to LL API cross reference:

- FCR FTH LL_DMA_ConfigFifo
- FCR DMDIS LL_DMA_ConfigFifo

LL_DMA_ConfigAddresses

Function name

__STATIC_INLINE void LL_DMA_ConfigAddresses (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t SrcAddress, uint32_t DstAddress, uint32_t Direction)

Function description

Configure the Source and Destination addresses.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **SrcAddress:** Between 0 to 0xFFFFFFFF
- **DstAddress:** Between 0 to 0xFFFFFFFF
- **Direction:** This parameter can be one of the following values:
 - LL_DMA_DIRECTION_PERIPH_TO_MEMORY
 - LL_DMA_DIRECTION_MEMORY_TO_PERIPH
 - LL_DMA_DIRECTION_MEMORY_TO_MEMORY

Return values

- **None:**

Notes

- This API must not be called when the DMA stream is enabled.

Reference Manual to LL API cross reference:

- M0AR M0A LL_DMA_ConfigAddresses
- PAR PA LL_DMA_ConfigAddresses

LL_DMA_SetMemoryAddress

Function name

__STATIC_INLINE void LL_DMA_SetMemoryAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)

Function description

Set the Memory address.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **MemoryAddress:** Between 0 to 0xFFFFFFFF

Return values

- **None:**

Notes

- Interface used for direction LL_DMA_DIRECTION_PERIPH_TO_MEMORY or LL_DMA_DIRECTION_MEMORY_TO_PERIPH only.
- This API must not be called when the DMA channel is enabled.

Reference Manual to LL API cross reference:

- M0AR M0A LL_DMA_SetMemoryAddress

LL_DMA_SetPeriphAddress

Function name

__STATIC_INLINE void LL_DMA_SetPeriphAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t PeriphAddress)

Function description

Set the Peripheral address.

Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- PeriphAddress:** Between 0 to 0xFFFFFFFF

Return values

- None:**

Notes

- Interface used for direction LL_DMA_DIRECTION_PERIPH_TO_MEMORY or LL_DMA_DIRECTION_MEMORY_TO_PERIPH only.
- This API must not be called when the DMA channel is enabled.

Reference Manual to LL API cross reference:

- PAR PA LL_DMA_SetPeriphAddress

LL_DMA_GetMemoryAddress

Function name

__STATIC_INLINE uint32_t LL_DMA_GetMemoryAddress (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get the Memory address.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Between:** 0 to 0xFFFFFFFF

Notes

- Interface used for direction LL_DMA_DIRECTION_PERIPH_TO_MEMORY or LL_DMA_DIRECTION_MEMORY_TO_PERIPH only.

Reference Manual to LL API cross reference:

- M0AR M0A LL_DMA_GetMemoryAddress

LL_DMA_GetPeriphAddress

Function name

__STATIC_INLINE uint32_t LL_DMA_GetPeriphAddress (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get the Peripheral address.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Between:** 0 to 0xFFFFFFFF

Notes

- Interface used for direction LL_DMA_DIRECTION_PERIPH_TO_MEMORY or LL_DMA_DIRECTION_MEMORY_TO_PERIPH only.

Reference Manual to LL API cross reference:

- PAR PA LL_DMA_GetPeriphAddress

LL_DMA_SetM2MSrcAddress

Function name

```
__STATIC_INLINE void LL_DMA_SetM2MSrcAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)
```

Function description

Set the Memory to Memory Source address.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **MemoryAddress:** Between 0 to 0xFFFFFFFF

Return values

- **None:**

Notes

- Interface used for direction LL_DMA_DIRECTION_MEMORY_TO_MEMORY only.
- This API must not be called when the DMA channel is enabled.

Reference Manual to LL API cross reference:

- PAR PA LL_DMA_SetM2MSrcAddress

LL_DMA_SetM2MDstAddress

Function name

```
__STATIC_INLINE void LL_DMA_SetM2MDstAddress (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t MemoryAddress)
```

Function description

Set the Memory to Memory Destination address.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **MemoryAddress:** Between 0 to 0xFFFFFFFF

Return values

- **None:**

Notes

- Interface used for direction LL_DMA_DIRECTION_MEMORY_TO_MEMORY only.
- This API must not be called when the DMA channel is enabled.

Reference Manual to LL API cross reference:

- M0AR M0A LL_DMA_SetM2MDstAddress

LL_DMA_GetM2MSrcAddress

Function name

__STATIC_INLINE uint32_t LL_DMA_GetM2MSrcAddress (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get the Memory to Memory Source address.

Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- Between:** 0 to 0xFFFFFFFF

Notes

- Interface used for direction LL_DMA_DIRECTION_MEMORY_TO_MEMORY only.

Reference Manual to LL API cross reference:

- PAR PA LL_DMA_GetM2MSrcAddress

LL_DMA_GetM2MDstAddress

Function name

__STATIC_INLINE uint32_t LL_DMA_GetM2MDstAddress (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get the Memory to Memory Destination address.

Parameters

- DMAx:** DMAx Instance
- Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Between:** 0 to 0xFFFFFFFF

Notes

- Interface used for direction LL_DMA_DIRECTION_MEMORY_TO_MEMORY only.

Reference Manual to LL API cross reference:

- M0AR M0A LL_DMA_GetM2MDstAddress
LL_DMA_SetMemory1Address

Function name

__STATIC_INLINE void LL_DMA_SetMemory1Address (DMA_TypeDef * DMAx, uint32_t Stream, uint32_t Address)

Function description

Set Memory 1 address (used in case of Double buffer mode).

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **Address:** Between 0 to 0xFFFFFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- M1AR M1A LL_DMA_SetMemory1Address
LL_DMA_GetMemory1Address

Function name

__STATIC_INLINE uint32_t LL_DMA_GetMemory1Address (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Get Memory 1 address (used in case of Double buffer mode).

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **Between:** 0 to 0xFFFFFFFF

Reference Manual to LL API cross reference:

- M1AR M1A LL_DMA_GetMemory1Address
LL_DMA_IsActiveFlag_HT0

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT0 (DMA_TypeDef * DMAx)

Function description

Get Stream 0 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR HTIF0 LL_DMA_IsActiveFlag_HT0
LL_DMA_IsActiveFlag_HT1

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT1 (DMA_TypeDef * DMAx)

Function description

Get Stream 1 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR HTIF1 LL_DMA_IsActiveFlag_HT1
LL_DMA_IsActiveFlag_HT2

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT2 (DMA_TypeDef * DMAx)

Function description

Get Stream 2 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR HTIF2 LL_DMA_IsActiveFlag_HT2
LL_DMA_IsActiveFlag_HT3

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT3 (DMA_TypeDef * DMAx)

Function description

Get Stream 3 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR HTIF3 LL_DMA_IsActiveFlag_HT3
LL_DMA_IsActiveFlag_HT4

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT4 (DMA_TypeDef * DMAx)

Function description

Get Stream 4 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR HTIF4 LL_DMA_IsActiveFlag_HT4
LL_DMA_IsActiveFlag_HT5

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT5 (DMA_TypeDef * DMAx)

Function description

Get Stream 5 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR HTIF0 LL_DMA_IsActiveFlag_HT5
LL_DMA_IsActiveFlag_HT6

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT6 (DMA_TypeDef * DMAx)

Function description

Get Stream 6 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR HTIF6 LL_DMA_IsActiveFlag_HT6
LL_DMA_IsActiveFlag_HT7

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_HT7 (DMA_TypeDef * DMAx)

Function description

Get Stream 7 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR HTIF7 LL_DMA_IsActiveFlag_HT7
LL_DMA_IsActiveFlag_TC0

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC0 (DMA_TypeDef * DMAx)

Function description

Get Stream 0 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TCIF0 LL_DMA_IsActiveFlag_TC0
LL_DMA_IsActiveFlag_TC1

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC1 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 1 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TCIF1 LL_DMA_IsActiveFlag_TC1

LL_DMA_IsActiveFlag_TC2

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC2 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 2 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TCIF2 LL_DMA_IsActiveFlag_TC2

LL_DMA_IsActiveFlag_TC3

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC3 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 3 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TCIF3 LL_DMA_IsActiveFlag_TC3

LL_DMA_IsActiveFlag_TC4

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC4 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 4 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TCIF4 LL_DMA_IsActiveFlag_TC4
LL_DMA_IsActiveFlag_TC5

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC5 (DMA_TypeDef * DMAx)

Function description

Get Stream 5 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TCIF0 LL_DMA_IsActiveFlag_TC5
LL_DMA_IsActiveFlag_TC6

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC6 (DMA_TypeDef * DMAx)

Function description

Get Stream 6 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TCIF6 LL_DMA_IsActiveFlag_TC6
LL_DMA_IsActiveFlag_TC7

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TC7 (DMA_TypeDef * DMAx)

Function description

Get Stream 7 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TCIF7 LL_DMA_IsActiveFlag_TC7
LL_DMA_IsActiveFlag_TE0

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE0 (DMA_TypeDef * DMAx)

Function description

Get Stream 0 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TEIF0 LL_DMA_IsActiveFlag_TE0
LL_DMA_IsActiveFlag_TE1

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE1 (DMA_TypeDef * DMAx)

Function description

Get Stream 1 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TEIF1 LL_DMA_IsActiveFlag_TE1
LL_DMA_IsActiveFlag_TE2

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE2 (DMA_TypeDef * DMAx)

Function description

Get Stream 2 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TEIF2 LL_DMA_IsActiveFlag_TE2
LL_DMA_IsActiveFlag_TE3

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE3 (DMA_TypeDef * DMAx)

Function description

Get Stream 3 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR TEIF3 LL_DMA_IsActiveFlag_TE3
LL_DMA_IsActiveFlag_TE4

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE4 (DMA_TypeDef * DMAx)

Function description

Get Stream 4 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TEIF4 LL_DMA_IsActiveFlag_TE4
LL_DMA_IsActiveFlag_TE5

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE5 (DMA_TypeDef * DMAx)

Function description

Get Stream 5 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TEIF0 LL_DMA_IsActiveFlag_TE5
LL_DMA_IsActiveFlag_TE6

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE6 (DMA_TypeDef * DMAx)

Function description

Get Stream 6 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TEIF6 LL_DMA_IsActiveFlag_TE6
LL_DMA_IsActiveFlag_TE7

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_TE7 (DMA_TypeDef * DMAx)

Function description

Get Stream 7 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR TEIF7 LL_DMA_IsActiveFlag_TE7
LL_DMA_IsActiveFlag_DME0

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME0 (DMA_TypeDef * DMAx)

Function description

Get Stream 0 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR DMEIF0 LL_DMA_IsActiveFlag_DME0
LL_DMA_IsActiveFlag_DME1

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME1 (DMA_TypeDef * DMAx)

Function description

Get Stream 1 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR DMEIF1 LL_DMA_IsActiveFlag_DME1
LL_DMA_IsActiveFlag_DME2

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME2 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 2 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR DMEIF2 LL_DMA_IsActiveFlag_DME2

LL_DMA_IsActiveFlag_DME3

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME3 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 3 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR DMEIF3 LL_DMA_IsActiveFlag_DME3

LL_DMA_IsActiveFlag_DME4

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME4 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 4 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR DMEIF4 LL_DMA_IsActiveFlag_DME4

LL_DMA_IsActiveFlag_DME5

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME5 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 5 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR DMEIF0 LL_DMA_IsActiveFlag_DME5
LL_DMA_IsActiveFlag_DME6

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME6 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 6 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR DMEIF6 LL_DMA_IsActiveFlag_DME6
LL_DMA_IsActiveFlag_DME7

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_DME7 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 7 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR DMEIF7 LL_DMA_IsActiveFlag_DME7
LL_DMA_IsActiveFlag_FE0

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE0 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 0 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR FEIF0 LL_DMA_IsActiveFlag_FE0
LL_DMA_IsActiveFlag_FE1

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE1 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 1 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR FEIF1 LL_DMA_IsActiveFlag_FE1
LL_DMA_IsActiveFlag_FE2

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE2 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 2 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR FEIF2 LL_DMA_IsActiveFlag_FE2
LL_DMA_IsActiveFlag_FE3

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE3 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 3 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LISR FEIF3 LL_DMA_IsActiveFlag_FE3
LL_DMA_IsActiveFlag_FE4

Function name

```
__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE4 (DMA_TypeDef * DMAx)
```

Function description

Get Stream 4 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR FEIF4 LL_DMA_IsActiveFlag_FE4
LL_DMA_IsActiveFlag_FE5

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE5 (DMA_TypeDef * DMAx)

Function description

Get Stream 5 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR FEIF0 LL_DMA_IsActiveFlag_FE5
LL_DMA_IsActiveFlag_FE6

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE6 (DMA_TypeDef * DMAx)

Function description

Get Stream 6 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR FEIF6 LL_DMA_IsActiveFlag_FE6
LL_DMA_IsActiveFlag_FE7

Function name

__STATIC_INLINE uint32_t LL_DMA_IsActiveFlag_FE7 (DMA_TypeDef * DMAx)

Function description

Get Stream 7 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- HISR FEIF7 LL_DMA_IsActiveFlag_FE7
LL_DMA_ClearFlag_HT0

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_HT0 (DMA_TypeDef * DMAx)

Function description

Clear Stream 0 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CHTIF0 LL_DMA_ClearFlag_HT0
LL_DMA_ClearFlag_HT1

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_HT1 (DMA_TypeDef * DMAx)

Function description

Clear Stream 1 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CHTIF1 LL_DMA_ClearFlag_HT1
LL_DMA_ClearFlag_HT2

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_HT2 (DMA_TypeDef * DMAx)

Function description

Clear Stream 2 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CHTIF2 LL_DMA_ClearFlag_HT2
LL_DMA_ClearFlag_HT3

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT3 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 3 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CHTIF3 LL_DMA_ClearFlag_HT3

LL_DMA_ClearFlag_HT4

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT4 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 4 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CHTIF4 LL_DMA_ClearFlag_HT4

LL_DMA_ClearFlag_HT5

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT5 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 5 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CHTIF5 LL_DMA_ClearFlag_HT5

LL_DMA_ClearFlag_HT6

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_HT6 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 6 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CHTIF6 LL_DMA_ClearFlag_HT6
LL_DMA_ClearFlag_HT7

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_HT7 (DMA_TypeDef * DMAx)

Function description

Clear Stream 7 half transfer flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CHTIF7 LL_DMA_ClearFlag_HT7
LL_DMA_ClearFlag_TC0

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC0 (DMA_TypeDef * DMAx)

Function description

Clear Stream 0 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTCIF0 LL_DMA_ClearFlag_TC0
LL_DMA_ClearFlag_TC1

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC1 (DMA_TypeDef * DMAx)

Function description

Clear Stream 1 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTCIF1 LL_DMA_ClearFlag_TC1
LL_DMA_ClearFlag_TC2

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC2 (DMA_TypeDef * DMAx)

Function description

Clear Stream 2 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTCIF2 LL_DMA_ClearFlag_TC2
LL_DMA_ClearFlag_TC3

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC3 (DMA_TypeDef * DMAx)

Function description

Clear Stream 3 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTCIF3 LL_DMA_ClearFlag_TC3
LL_DMA_ClearFlag_TC4

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC4 (DMA_TypeDef * DMAx)

Function description

Clear Stream 4 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTCIF4 LL_DMA_ClearFlag_TC4
LL_DMA_ClearFlag_TC5

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC5 (DMA_TypeDef * DMAx)

Function description

Clear Stream 5 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTCIF5 LL_DMA_ClearFlag_TC5
LL_DMA_ClearFlag_TC6

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC6 (DMA_TypeDef * DMAx)

Function description

Clear Stream 6 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTCIF6 LL_DMA_ClearFlag_TC6
LL_DMA_ClearFlag_TC7

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TC7 (DMA_TypeDef * DMAx)

Function description

Clear Stream 7 transfer complete flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTCIF7 LL_DMA_ClearFlag_TC7
LL_DMA_ClearFlag_TE0

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TE0 (DMA_TypeDef * DMAx)

Function description

Clear Stream 0 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTEIF0 LL_DMA_ClearFlag_TE0
LL_DMA_ClearFlag_TE1

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TE1 (DMA_TypeDef * DMAx)

Function description

Clear Stream 1 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTEIF1 LL_DMA_ClearFlag_TE1
LL_DMA_ClearFlag_TE2

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TE2 (DMA_TypeDef * DMAx)

Function description

Clear Stream 2 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTEIF2 LL_DMA_ClearFlag_TE2
LL_DMA_ClearFlag_TE3

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_TE3 (DMA_TypeDef * DMAx)

Function description

Clear Stream 3 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CTEIF3 LL_DMA_ClearFlag_TE3
LL_DMA_ClearFlag_TE4

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE4 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 4 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTEIF4 LL_DMA_ClearFlag_TE4
LL_DMA_ClearFlag_TE5

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE5 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 5 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTEIF5 LL_DMA_ClearFlag_TE5
LL_DMA_ClearFlag_TE6

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE6 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 6 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTEIF6 LL_DMA_ClearFlag_TE6
LL_DMA_ClearFlag_TE7

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_TE7 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 7 transfer error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CTEIF7 LL_DMA_ClearFlag_TE7
LL_DMA_ClearFlag_DME0

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_DME0 (DMA_TypeDef * DMAx)

Function description

Clear Stream 0 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CDMEIF0 LL_DMA_ClearFlag_DME0
LL_DMA_ClearFlag_DME1

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_DME1 (DMA_TypeDef * DMAx)

Function description

Clear Stream 1 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CDMEIF1 LL_DMA_ClearFlag_DME1
LL_DMA_ClearFlag_DME2

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_DME2 (DMA_TypeDef * DMAx)

Function description

Clear Stream 2 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CDMEIF2 LL_DMA_ClearFlag_DME2
LL_DMA_ClearFlag_DME3

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME3 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 3 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CDMEIF3 LL_DMA_ClearFlag_DME3
LL_DMA_ClearFlag_DME4

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME4 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 4 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CDMEIF4 LL_DMA_ClearFlag_DME4
LL_DMA_ClearFlag_DME5

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME5 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 5 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CDMEIF5 LL_DMA_ClearFlag_DME5
LL_DMA_ClearFlag_DME6

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_DME6 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 6 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CDMEIF6 LL_DMA_ClearFlag_DME6

LL_DMA_ClearFlag_DME7

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_DME7 (DMA_TypeDef * DMAx)

Function description

Clear Stream 7 direct mode error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CDMEIF7 LL_DMA_ClearFlag_DME7

LL_DMA_ClearFlag_FE0

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_FE0 (DMA_TypeDef * DMAx)

Function description

Clear Stream 0 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CFEIF0 LL_DMA_ClearFlag_FE0

LL_DMA_ClearFlag_FE1

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_FE1 (DMA_TypeDef * DMAx)

Function description

Clear Stream 1 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CFEIF1 LL_DMA_ClearFlag_FE1
LL_DMA_ClearFlag_FE2

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_FE2 (DMA_TypeDef * DMAx)

Function description

Clear Stream 2 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CFEIF2 LL_DMA_ClearFlag_FE2
LL_DMA_ClearFlag_FE3

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_FE3 (DMA_TypeDef * DMAx)

Function description

Clear Stream 3 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- LIFCR CFEIF3 LL_DMA_ClearFlag_FE3
LL_DMA_ClearFlag_FE4

Function name

__STATIC_INLINE void LL_DMA_ClearFlag_FE4 (DMA_TypeDef * DMAx)

Function description

Clear Stream 4 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CFEIF4 LL_DMA_ClearFlag_FE4
LL_DMA_ClearFlag_FE5

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE5 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 5 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CFEIF5 LL_DMA_ClearFlag_FE5
LL_DMA_ClearFlag_FE6

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE6 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 6 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CFEIF6 LL_DMA_ClearFlag_FE6
LL_DMA_ClearFlag_FE7

Function name

```
__STATIC_INLINE void LL_DMA_ClearFlag_FE7 (DMA_TypeDef * DMAx)
```

Function description

Clear Stream 7 FIFO error flag.

Parameters

- **DMAx:** DMAx Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- HIFCR CFEIF7 LL_DMA_ClearFlag_FE7
LL_DMA_EnableIT_HT

Function name

```
__STATIC_INLINE void LL_DMA_EnableIT_HT (DMA_TypeDef * DMAx, uint32_t Stream)
```

Function description

Enable Half transfer interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HTIE LL_DMA_EnableIT_HT
LL_DMA_EnableIT_TE

Function name

__STATIC_INLINE void LL_DMA_EnableIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable Transfer error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR TEIE LL_DMA_EnableIT_TE
LL_DMA_EnableIT_TC

Function name

__STATIC_INLINE void LL_DMA_EnableIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable Transfer complete interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR TCIE LL_DMA_EnableIT_TC
LL_DMA_EnableIT_DME

Function name

__STATIC_INLINE void LL_DMA_EnableIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable Direct mode error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMEIE LL_DMA_EnableIT_DME
LL_DMA_EnableIT_FE

Function name

__STATIC_INLINE void LL_DMA_EnableIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Enable FIFO error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- FCR FEIE LL_DMA_EnableIT_FE
LL_DMA_DisableIT_HT

Function name

__STATIC_INLINE void LL_DMA_DisableIT_HT (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable Half transfer interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HTIE LL_DMA_DisableIT_HT
LL_DMA_DisableIT_TE

Function name

__STATIC_INLINE void LL_DMA_DisableIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable Transfer error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR TEIE LL_DMA_DisableIT_TE
LL_DMA_DisableIT_TC

Function name

__STATIC_INLINE void LL_DMA_DisableIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable Transfer complete interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR TCIE LL_DMA_DisableIT_TC
LL_DMA_DisableIT_DME

Function name

__STATIC_INLINE void LL_DMA_DisableIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable Direct mode error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DMEIE LL_DMA_DisableIT_DME
LL_DMA_DisableIT_FE

Function name

__STATIC_INLINE void LL_DMA_DisableIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Disable FIFO error interrupt.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- FCR FEIE LL_DMA_DisableIT_FE
LL_DMA_IsEnabledIT_HT

Function name

__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_HT (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Check if Half transfer interrupt is enabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR HTIE LL_DMA_IsEnabledIT_HT

LL_DMA_IsEnabledIT_TE

Function name

__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_TE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Check if Transfer error ntterup is enabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR TEIE LL_DMA_IsEnabledIT_TE

LL_DMA_IsEnabledIT_TC

Function name

__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_TC (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Check if Transfer complete interrupt is enabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR TCIE LL_DMA_IsEnabledIT_TC
LL_DMA_IsEnabledIT_DME

Function name

__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_DME (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Check if Direct mode error interrupt is enabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR DMEIE LL_DMA_IsEnabledIT_DME
LL_DMA_IsEnabledIT_FE

Function name

__STATIC_INLINE uint32_t LL_DMA_IsEnabledIT_FE (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

Check if FIFO error interrupt is enabled.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- FCR FEIE LL_DMA_IsEnabledIT_FE

LL_DMA_Init

Function name

uint32_t LL_DMA_Init (DMA_TypeDef * DMAx, uint32_t Stream, LL_DMA_InitTypeDef * DMA_InitStruct)

Function description

Initialize the DMA registers according to the specified parameters in DMA_InitStruct.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
- **DMA_InitStruct:** pointer to a LL_DMA_InitTypeDef structure.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: DMA registers are initialized
 - ERROR: Not applicable

Notes

- To convert DMAx_Streamy Instance to DMAx Instance and Streamy, use helper macros :
__LL_DMA_GET_INSTANCE __LL_DMA_GET_STREAM

LL_DMA_DeInit

Function name

uint32_t LL_DMA_DeInit (DMA_TypeDef * DMAx, uint32_t Stream)

Function description

De-initialize the DMA registers to their default reset values.

Parameters

- **DMAx:** DMAx Instance
- **Stream:** This parameter can be one of the following values:
 - LL_DMA_STREAM_0
 - LL_DMA_STREAM_1
 - LL_DMA_STREAM_2
 - LL_DMA_STREAM_3
 - LL_DMA_STREAM_4
 - LL_DMA_STREAM_5
 - LL_DMA_STREAM_6
 - LL_DMA_STREAM_7
 - LL_DMA_STREAM_ALL

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: DMA registers are de-initialized
 - ERROR: DMA registers are not de-initialized

LL_DMA_StructInit

Function name

void LL_DMA_StructInit (LL_DMA_InitTypeDef * DMA_InitStruct)

Function description

Set each LL_DMA_InitTypeDef field to default value.

Parameters

- **DMA_InitStruct:** Pointer to a LL_DMA_InitTypeDef structure.

Return values

- **None:**

54.3 DMA Firmware driver defines

The following section lists the various define and macros of the module.

54.3.1 DMA DMA CHANNEL

LL_DMA_CHANNEL_0

LL_DMA_CHANNEL_1

LL_DMA_CHANNEL_2

LL_DMA_CHANNEL_3

LL_DMA_CHANNEL_4

LL_DMA_CHANNEL_5

LL_DMA_CHANNEL_6

LL_DMA_CHANNEL_7

CURRENTTARGETMEM

LL_DMA_CURRENTTARGETMEM0

Set CurrentTarget Memory to Memory 0

LL_DMA_CURRENTTARGETMEM1

Set CurrentTarget Memory to Memory 1

DIRECTION

LL_DMA_DIRECTION_PERIPH_TO_MEMORY

Peripheral to memory direction

LL_DMA_DIRECTION_MEMORY_TO_PERIPH

Memory to peripheral direction

LL_DMA_DIRECTION_MEMORY_TO_MEMORY

Memory to memory direction

DOUBLE BUFFER MODE

LL_DMA_DOUBLEBUFFER_MODE_DISABLE

Disable double buffering mode

LL_DMA_DOUBLEBUFFER_MODE_ENABLE

Enable double buffering mode

FIFOSTATUS 0

LL_DMA_FIFOSTATUS_0_25

$0 < \text{fifo_level} < 1/4$

LL_DMA_FIFOSTATUS_25_50

$1/4 < \text{fifo_level} < 1/2$

LL_DMA_FIFOSTATUS_50_75

$1/2 < \text{fifo_level} < 3/4$

LL_DMA_FIFOSTATUS_75_100

$3/4 < \text{fifo_level} < \text{full}$

LL_DMA_FIFOSTATUS_EMPTY

FIFO is empty

LL_DMA_FIFOSTATUS_FULL

FIFO is full

FIFOTHRESHOLD

LL_DMA_FIFOTHRESHOLD_1_4

FIFO threshold 1 quart full configuration

LL_DMA_FIFOTHRESHOLD_1_2

FIFO threshold half full configuration

LL_DMA_FIFOTHRESHOLD_3_4

FIFO threshold 3 quarts full configuration

LL_DMA_FIFOTHRESHOLD_FULL

FIFO threshold full configuration

MBURST

LL_DMA_MBURST_SINGLE

Memory burst single transfer configuration

LL_DMA_MBURST_INC4

Memory burst of 4 beats transfer configuration

LL_DMA_MBURST_INC8

Memory burst of 8 beats transfer configuration

LL_DMA_MBURST_INC16

Memory burst of 16 beats transfer configuration

MDATAALIGN

LL_DMA_MDATAALIGN_BYTE

Memory data alignment : Byte

LL_DMA_MDATAALIGN_HALFWORD

Memory data alignment : HalfWord

LL_DMA_MDATAALIGN_WORD

Memory data alignment : Word

MEMORY

LL_DMA_MEMORY_NOINCREMENT

Memory increment mode Disable

LL_DMA_MEMORY_INCREMENT

Memory increment mode Enable

MODE

LL_DMA_MODE_NORMAL

Normal Mode

LL_DMA_MODE_CIRCULAR

Circular Mode

LL_DMA_MODE_PFCTRL

Peripheral flow control mode

OFFSETSIZE

LL_DMA_OFFSETSIZE_PSIZE

Peripheral increment offset size is linked to the PSIZE

LL_DMA_OFFSETSIZE_FIXEDTO4

Peripheral increment offset size is fixed to 4 (32-bit alignment)

PBURST

LL_DMA_PBURST_SINGLE

Peripheral burst single transfer configuration

LL_DMA_PBURST_INC4

Peripheral burst of 4 beats transfer configuration

LL_DMA_PBURST_INC8

Peripheral burst of 8 beats transfer configuration

LL_DMA_PBURST_INC16

Peripheral burst of 16 beats transfer configuration

PDATAALIGN

LL_DMA_PDATAALIGN_BYTE

Peripheral data alignment : Byte

LL_DMA_PDATAALIGN_HALFWORD

Peripheral data alignment : HalfWord

LL_DMA_PDATAALIGN_WORD

Peripheral data alignment : Word

PERIPH

LL_DMA_PERIPH_NOINCREMENT

Peripheral increment mode Disable

LL_DMA_PERIPH_INCREMENT

Peripheral increment mode Enable

PRIORITY

LL_DMA_PRIORITY_LOW

Priority level : Low

LL_DMA_PRIORITY_MEDIUM

Priority level : Medium

LL_DMA_PRIORITY_HIGH

Priority level : High

LL_DMA_PRIORITY_VERYHIGH

Priority level : Very_High

STREAM

LL_DMA_STREAM_0

LL_DMA_STREAM_1

LL_DMA_STREAM_2

LL_DMA_STREAM_3

LL_DMA_STREAM_4

LL_DMA_STREAM_5

LL_DMA_STREAM_6

LL_DMA_STREAM_7

LL_DMA_STREAM_ALL

Convert DMAxStreamy

__LL_DMA_GET_INSTANCE

Description:

- Convert DMAx_Streamy into DMAx.

Parameters:

- __STREAM_INSTANCE__: DMAx_Streamy

Return value:

- DMAx

__LL_DMA_GET_STREAM

Description:

- Convert DMAx_Streamy into LL_DMA_STREAM_y.

Parameters:

- __STREAM_INSTANCE__: DMAx_Streamy

Return value:

- LL_DMA_CHANNEL_y

__LL_DMA_GET_STREAM_INSTANCE

Description:

- Convert DMA Instance DMAx and LL_DMA_STREAM_y into DMAx_Streamy.

Parameters:

- __DMA_INSTANCE__: DMAx
- __STREAM__: LL_DMA_STREAM_y

Return value:

- DMAx_Streamy

Common Write and read registers macros

LL_DMA_WriteReg

Description:

- Write a value in DMA register.

Parameters:

- __INSTANCE__: DMA Instance
- __REG__: Register to be written
- __VALUE__: Value to be written in the register

Return value:

- None

LL_DMA_ReadReg

Description:

- Read a value in DMA register.

Parameters:

- __INSTANCE__: DMA Instance
- __REG__: Register to be read

Return value:

- Register: value

DMA_LL_FIFOMODE

LL_DMA_FIFOMODE_DISABLE

FIFO mode disable (direct mode is enabled)

LL_DMA_FIFOMODE_ENABLE

FIFO mode enable

55 LL EXTI Generic Driver

55.1 EXTI Firmware driver registers structures

55.1.1 LL_EXTI_InitTypeDef

LL_EXTI_InitTypeDef is defined in the stm32f2xx_ll_exti.h

Data Fields

- **uint32_t Line_0_31**
- **FunctionalState LineCommand**
- **uint8_t Mode**
- **uint8_t Trigger**

Field Documentation

- **uint32_t LL_EXTI_InitTypeDef::Line_0_31**
Specifies the EXTI lines to be enabled or disabled for Lines in range 0 to 31 This parameter can be any combination of [EXTI_LL_EC_LINE](#)
- **FunctionalState LL_EXTI_InitTypeDef::LineCommand**
Specifies the new state of the selected EXTI lines. This parameter can be set either to ENABLE or DISABLE
- **uint8_t LL_EXTI_InitTypeDef::Mode**
Specifies the mode for the EXTI lines. This parameter can be a value of [EXTI_LL_EC_MODE](#).
- **uint8_t LL_EXTI_InitTypeDef::Trigger**
Specifies the trigger signal active edge for the EXTI lines. This parameter can be a value of [EXTI_LL_EC_TRIGGER](#).

55.2 EXTI Firmware driver API description

The following section lists the various functions of the EXTI library.

55.2.1 Detailed description of functions

`LL_EXTI_EnableIT_0_31`

Function name

`__STATIC_INLINE void LL_EXTI_EnableIT_0_31 (uint32_t ExtiLine)`

Function description

Enable ExtiLine Interrupt request for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **None:**

Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- IMR IMx LL_EXTI_EnableIT_0_31
LL_EXTI_DisableIT_0_31

Function name

__STATIC_INLINE void LL_EXTI_DisableIT_0_31 (uint32_t ExtiLine)

Function description

Disable ExtiLine Interrupt request for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **None:**

Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- IMR IMx LL_EXTI_DisableIT_0_31
LL_EXTI_IsEnabledIT_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_IsEnabledIT_0_31 (uint32_t ExtiLine)

Function description

Indicate if ExtiLine Interrupt request is enabled for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **State:** of bit (1 or 0).

Notes

- The reset value for the direct or internal lines (see RM) is set to 1 in order to enable the interrupt by default. Bits are set automatically at Power on.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- IMR IMx LL_EXTI_IsEnabledIT_0_31

LL_EXTI_EnableEvent_0_31

Function name

__STATIC_INLINE void LL_EXTI_EnableEvent_0_31 (uint32_t ExtiLine)

Function description

Enable ExtiLine Event request for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **None:**

Notes

- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- EMR EMx LL_EXTI_EnableEvent_0_31

LL_EXTI_DisableEvent_0_31

Function name

__STATIC_INLINE void LL_EXTI_DisableEvent_0_31 (uint32_t ExtiLine)

Function description

Disable ExtiLine Event request for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **None:**

Notes

- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- EMR EMx LL_EXTI_DisableEvent_0_31

LL_EXTI_IsEnabledEvent_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_IsEnabledEvent_0_31 (uint32_t ExtiLine)

Function description

Indicate if ExtiLine Event request is enabled for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be one of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_17
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22
 - LL_EXTI_LINE_ALL_0_31

Return values

- **State:** of bit (1 or 0).

Notes

- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- EMR EMx LL_EXTI_IsEnabledEvent_0_31
LL_EXTI_EnableRisingTrig_0_31

Function name

__STATIC_INLINE void LL_EXTI_EnableRisingTrig_0_31 (uint32_t ExtiLine)

Function description

Enable ExtiLine Rising Edge Trigger for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a rising edge on a configurable interrupt line occurs during a write operation in the EXTI_RTISR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- RTISR RTx LL_EXTI_EnableRisingTrig_0_31
LL_EXTI_DisableRisingTrig_0_31

Function name

__STATIC_INLINE void LL_EXTI_DisableRisingTrig_0_31 (uint32_t ExtiLine)

Function description

Disable ExtiLine Rising Edge Trigger for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a rising edge on a configurable interrupt line occurs during a write operation in the EXTI_RTISR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- RTISR RTx LL_EXTI_DisableRisingTrig_0_31
LL_EXTI_IsEnabledRisingTrig_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_IsEnabledRisingTrig_0_31 (uint32_t ExtiLine)

Function description

Check if rising edge trigger is enabled for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **State:** of bit (1 or 0).

Notes

- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- RTSR RTx LL_EXTI_IsEnabledRisingTrig_0_31
LL_EXTI_EnableFallingTrig_0_31

Function name

__STATIC_INLINE void LL_EXTI_EnableFallingTrig_0_31 (uint32_t ExtiLine)

Function description

Enable ExtiLine Falling Edge Trigger for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a falling edge on a configurable interrupt line occurs during a write operation in the EXTI_FTSR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- FTSR FTx LL_EXTI_EnableFallingTrig_0_31
LL_EXTI_DisableFallingTrig_0_31

Function name

__STATIC_INLINE void LL_EXTI_DisableFallingTrig_0_31 (uint32_t ExtiLine)

Function description

Disable ExtiLine Falling Edge Trigger for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- The configurable wakeup lines are edge-triggered. No glitch must be generated on these lines. If a Falling edge on a configurable interrupt line occurs during a write operation in the EXTI_FTSR register, the pending bit is not set. Rising and falling edge triggers can be set for the same interrupt line. In this case, both generate a trigger condition.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- FTSR FTx LL_EXTI_DisableFallingTrig_0_31
LL_EXTI_IsEnabledFallingTrig_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_IsEnabledFallingTrig_0_31 (uint32_t ExtiLine)

Function description

Check if falling edge trigger is enabled for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **State:** of bit (1 or 0).

Notes

- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- FTSR FTx LL_EXTI_IsEnabledFallingTrig_0_31
LL_EXTI_GenerateSWI_0_31

Function name

__STATIC_INLINE void LL_EXTI_GenerateSWI_0_31 (uint32_t ExtiLine)

Function description

Generate a software Interrupt Event for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- If the interrupt is enabled on this line in the EXTI_IMR, writing a 1 to this bit when it is at '0' sets the corresponding pending bit in EXTI_PR resulting in an interrupt request generation. This bit is cleared by clearing the corresponding bit in the EXTI_PR register (by writing a 1 into the bit)
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- SWIER SWIx LL_EXTI_GenerateSWI_0_31

LL_EXTI_IsActiveFlag_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_IsActiveFlag_0_31 (uint32_t ExtiLine)

Function description

Check if the ExtLine Flag is set or not for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **State:** of bit (1 or 0).

Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- PR PIFx LL_EXTI_IsActiveFlag_0_31
LL_EXTI_ReadFlag_0_31

Function name

__STATIC_INLINE uint32_t LL_EXTI_ReadFlag_0_31 (uint32_t ExtiLine)

Function description

Read ExtLine Combination Flag for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **@note:** This bit is set when the selected edge event arrives on the interrupt

Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- PR PIFx LL_EXTI_ReadFlag_0_31
LL_EXTI_ClearFlag_0_31

Function name

__STATIC_INLINE void LL_EXTI_ClearFlag_0_31 (uint32_t ExtiLine)

Function description

Clear ExtLine Flags for Lines in range 0 to 31.

Parameters

- **ExtiLine:** This parameter can be a combination of the following values:
 - LL_EXTI_LINE_0
 - LL_EXTI_LINE_1
 - LL_EXTI_LINE_2
 - LL_EXTI_LINE_3
 - LL_EXTI_LINE_4
 - LL_EXTI_LINE_5
 - LL_EXTI_LINE_6
 - LL_EXTI_LINE_7
 - LL_EXTI_LINE_8
 - LL_EXTI_LINE_9
 - LL_EXTI_LINE_10
 - LL_EXTI_LINE_11
 - LL_EXTI_LINE_12
 - LL_EXTI_LINE_13
 - LL_EXTI_LINE_14
 - LL_EXTI_LINE_15
 - LL_EXTI_LINE_16
 - LL_EXTI_LINE_18
 - LL_EXTI_LINE_19
 - LL_EXTI_LINE_20
 - LL_EXTI_LINE_21
 - LL_EXTI_LINE_22

Return values

- **None:**

Notes

- This bit is set when the selected edge event arrives on the interrupt line. This bit is cleared by writing a 1 to the bit.
- Please check each device line mapping for EXTI Line availability

Reference Manual to LL API cross reference:

- PR PIFx LL_EXTI_ClearFlag_0_31

LL_EXTI_Init

Function name

uint32_t LL_EXTI_Init (LL_EXTI_InitTypeDef * EXTI_InitStruct)

Function description

Initialize the EXTI registers according to the specified parameters in EXTI_InitStruct.

Parameters

- **EXTI_InitStruct:** pointer to a LL_EXTI_InitTypeDef structure.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: EXTI registers are initialized
 - ERROR: not applicable

LL_EXTI_DeInit

Function name

uint32_t LL_EXTI_DeInit (void)

Function description

De-initialize the EXTI registers to their default reset values.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: EXTI registers are de-initialized
 - ERROR: not applicable

LL_EXTI_StructInit

Function name

void LL_EXTI_StructInit (LL_EXTI_InitTypeDef * EXTI_InitStruct)

Function description

Set each LL_EXTI_InitTypeDef field to default value.

Parameters

- **EXTI_InitStruct:** Pointer to a LL_EXTI_InitTypeDef structure.

Return values

- **None:**

55.3 EXTI Firmware driver defines

The following section lists the various define and macros of the module.

55.3.1 EXTI

EXTI

LINE

LL_EXTI_LINE_0

Extended line 0

LL_EXTI_LINE_1

Extended line 1

LL_EXTI_LINE_2

Extended line 2

LL_EXTI_LINE_3

Extended line 3

LL_EXTI_LINE_4

Extended line 4

LL_EXTI_LINE_5

Extended line 5

LL_EXTI_LINE_6

Extended line 6

LL_EXTI_LINE_7

Extended line 7

LL_EXTI_LINE_8

Extended line 8

LL_EXTI_LINE_9

Extended line 9

LL_EXTI_LINE_10

Extended line 10

LL_EXTI_LINE_11

Extended line 11

LL_EXTI_LINE_12

Extended line 12

LL_EXTI_LINE_13

Extended line 13

LL_EXTI_LINE_14

Extended line 14

LL_EXTI_LINE_15

Extended line 15

LL_EXTI_LINE_16

Extended line 16

LL_EXTI_LINE_17

Extended line 17

LL_EXTI_LINE_18

Extended line 18

LL_EXTI_LINE_19

Extended line 19

LL_EXTI_LINE_20

Extended line 20

LL_EXTI_LINE_21

Extended line 21

LL_EXTI_LINE_22

Extended line 22

LL_EXTI_LINE_ALL_0_31

All Extended line not reserved

LL_EXTI_LINE_ALL

All Extended line

LL_EXTI_LINE_NONE

None Extended line

Mode**LL_EXTI_MODE_IT**

Interrupt Mode

LL_EXTI_MODE_EVENT

Event Mode

LL_EXTI_MODE_IT_EVENT

Interrupt & Event Mode

Edge Trigger

LL_EXTI_TRIGGER_NONE

No Trigger Mode

LL_EXTI_TRIGGER_RISING

Trigger Rising Mode

LL_EXTI_TRIGGER_FALLING

Trigger Falling Mode

LL_EXTI_TRIGGER_RISING_FALLING

Trigger Rising & Falling Mode

Common Write and read registers Macros

LL_EXTI_WriteReg

Description:

- Write a value in EXTI register.

Parameters:

- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_EXTI_ReadReg

Description:

- Read a value in EXTI register.

Parameters:

- `__REG__`: Register to be read

Return value:

- Register: value

56 LL GPIO Generic Driver

56.1 GPIO Firmware driver registers structures

56.1.1 LL_GPIO_InitTypeDef

LL_GPIO_InitTypeDef is defined in the `stm32f2xx_ll_gpio.h`

Data Fields

- `uint32_t Pin`
- `uint32_t Mode`
- `uint32_t Speed`
- `uint32_t OutputType`
- `uint32_t Pull`
- `uint32_t Alternate`

Field Documentation

- `uint32_t LL_GPIO_InitTypeDef::Pin`
Specifies the GPIO pins to be configured. This parameter can be any value of [GPIO_LL_EC_PIN](#)
- `uint32_t LL_GPIO_InitTypeDef::Mode`
Specifies the operating mode for the selected pins. This parameter can be a value of [GPIO_LL_EC_MODE](#). GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinMode()`.
- `uint32_t LL_GPIO_InitTypeDef::Speed`
Specifies the speed for the selected pins. This parameter can be a value of [GPIO_LL_EC_SPEED](#). GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinSpeed()`.
- `uint32_t LL_GPIO_InitTypeDef::OutputType`
Specifies the operating output type for the selected pins. This parameter can be a value of [GPIO_LL_EC_OUTPUT](#). GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinOutputType()`.
- `uint32_t LL_GPIO_InitTypeDef::Pull`
Specifies the operating Pull-up/Pull down for the selected pins. This parameter can be a value of [GPIO_LL_EC_PULL](#). GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetPinPull()`.
- `uint32_t LL_GPIO_InitTypeDef::Alternate`
Specifies the Peripheral to be connected to the selected pins. This parameter can be a value of [GPIO_LL_EC_AF](#). GPIO HW configuration can be modified afterwards using unitary function `LL_GPIO_SetAFPin_0_7()` and `LL_GPIO_SetAFPin_8_15()`.

56.2 GPIO Firmware driver API description

The following section lists the various functions of the GPIO library.

56.2.1 Detailed description of functions

`LL_GPIO_SetPinMode`

Function name

```
__STATIC_INLINE void LL_GPIO_SetPinMode (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Mode)
```

Function description

Configure gpio mode for a dedicated pin on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
- **Mode:** This parameter can be one of the following values:
 - LL_GPIO_MODE_INPUT
 - LL_GPIO_MODE_OUTPUT
 - LL_GPIO_MODE_ALTERNATE
 - LL_GPIO_MODE_ANALOG

Return values

- **None:**

Notes

- I/O mode can be Input mode, General purpose output, Alternate function mode or Analog.
- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- MODER MODEy LL_GPIO_SetPinMode

LL_GPIO_GetPinMode

Function name

__STATIC_INLINE uint32_t LL_GPIO_GetPinMode (GPIO_TypeDef * GPIOx, uint32_t Pin)

Function description

Return gpio mode for a dedicated pin on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_MODE_INPUT
 - LL_GPIO_MODE_OUTPUT
 - LL_GPIO_MODE_ALTERNATE
 - LL_GPIO_MODE_ANALOG

Notes

- I/O mode can be Input mode, General purpose output, Alternate function mode or Analog.
- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- MODER MODEy LL_GPIO_GetPinMode

LL_GPIO_SetPinOutputType

Function name

__STATIC_INLINE void LL_GPIO_SetPinOutputType (GPIO_TypeDef * GPIOx, uint32_t PinMask, uint32_t OutputType)

Function description

Configure gpio output type for several pins on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL
- **OutputType:** This parameter can be one of the following values:
 - LL_GPIO_OUTPUT_PUSH_PULL
 - LL_GPIO_OUTPUT_OPENDRAIN

Return values

- **None:**

Notes

- Output type as to be set when gpio pin is in output or alternate modes. Possible type are Push-pull or Open-drain.

Reference Manual to LL API cross reference:

- OTYPER OTy LL_GPIO_SetPinOutputType
LL_GPIO_GetPinOutputType

Function name

__STATIC_INLINE uint32_t LL_GPIO_GetPinOutputType (GPIO_TypeDef * GPIOx, uint32_t Pin)

Function description

Return gpio output type for several pins on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_OUTPUT_PUSHPULL
 - LL_GPIO_OUTPUT_OPENDRAIN

Notes

- Output type as to be set when gpio pin is in output or alternate modes. Possible type are Push-pull or Open-drain.
- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- OTYPER OTy LL_GPIO_GetPinOutputType
- LL_GPIO_SetPinSpeed

Function name

__STATIC_INLINE void LL_GPIO_SetPinSpeed (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Speed)

Function description

Configure gpio speed for a dedicated pin on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
- **Speed:** This parameter can be one of the following values:
 - LL_GPIO_SPEED_FREQ_LOW
 - LL_GPIO_SPEED_FREQ_MEDIUM
 - LL_GPIO_SPEED_FREQ_HIGH
 - LL_GPIO_SPEED_FREQ_VERY_HIGH

Return values

- **None:**

Notes

- I/O speed can be Low, Medium, Fast or High speed.
- Warning: only one pin can be passed as parameter.
- Refer to datasheet for frequency specifications and the power supply and load conditions for each speed.

Reference Manual to LL API cross reference:

- OSPEEDR OSPEEDy LL_GPIO_SetPinSpeed

LL_GPIO_GetPinSpeed

Function name

__STATIC_INLINE uint32_t LL_GPIO_GetPinSpeed (GPIO_TypeDef * GPIOx, uint32_t Pin)

Function description

Return gpio speed for a dedicated pin on dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_SPEED_FREQ_LOW
 - LL_GPIO_SPEED_FREQ_MEDIUM
 - LL_GPIO_SPEED_FREQ_HIGH
 - LL_GPIO_SPEED_FREQ_VERY_HIGH

Notes

- I/O speed can be Low, Medium, Fast or High speed.
- Warning: only one pin can be passed as parameter.
- Refer to datasheet for frequency specifications and the power supply and load conditions for each speed.

Reference Manual to LL API cross reference:

- OSPEEDR OSPEEDy LL_GPIO_GetPinSpeed

LL_GPIO_SetPinPull

Function name

__STATIC_INLINE void LL_GPIO_SetPinPull (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Pull)

Function description

Configure gpio pull-up or pull-down for a dedicated pin on a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
- **Pull:** This parameter can be one of the following values:
 - LL_GPIO_PULL_NO
 - LL_GPIO_PULL_UP
 - LL_GPIO_PULL_DOWN

Return values

- **None:**

Notes

- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- PUPDR PUPDy LL_GPIO_SetPinPull

LL_GPIO_GetPinPull

Function name

__STATIC_INLINE uint32_t LL_GPIO_GetPinPull (GPIO_TypeDef * GPIOx, uint32_t Pin)

Function description

Return gpio pull-up or pull-down for a dedicated pin on a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_PULL_NO
 - LL_GPIO_PULL_UP
 - LL_GPIO_PULL_DOWN

Notes

- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- PUPDR PUPDy LL_GPIO_GetPinPull

LL_GPIO_SetAFPin_0_7

Function name

__STATIC_INLINE void LL_GPIO_SetAFPin_0_7 (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Alternate)

Function description

Configure gpio alternate function of a dedicated pin from 0 to 7 for a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
- **Alternate:** This parameter can be one of the following values:
 - LL_GPIO_AF_0
 - LL_GPIO_AF_1
 - LL_GPIO_AF_2
 - LL_GPIO_AF_3
 - LL_GPIO_AF_4
 - LL_GPIO_AF_5
 - LL_GPIO_AF_6
 - LL_GPIO_AF_7
 - LL_GPIO_AF_8
 - LL_GPIO_AF_9
 - LL_GPIO_AF_10
 - LL_GPIO_AF_11
 - LL_GPIO_AF_12
 - LL_GPIO_AF_13
 - LL_GPIO_AF_14
 - LL_GPIO_AF_15

Return values

- **None:**

Notes

- Possible values are from AF0 to AF15 depending on target.
- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- AFRL AFSELy LL_GPIO_SetAFPin_0_7
LL_GPIO_GetAFPin_0_7

Function name

`__STATIC_INLINE uint32_t LL_GPIO_GetAFPin_0_7 (GPIO_TypeDef * GPIOx, uint32_t Pin)`

Function description

Return gpio alternate function of a dedicated pin from 0 to 7 for a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_AF_0
 - LL_GPIO_AF_1
 - LL_GPIO_AF_2
 - LL_GPIO_AF_3
 - LL_GPIO_AF_4
 - LL_GPIO_AF_5
 - LL_GPIO_AF_6
 - LL_GPIO_AF_7
 - LL_GPIO_AF_8
 - LL_GPIO_AF_9
 - LL_GPIO_AF_10
 - LL_GPIO_AF_11
 - LL_GPIO_AF_12
 - LL_GPIO_AF_13
 - LL_GPIO_AF_14
 - LL_GPIO_AF_15

Reference Manual to LL API cross reference:

- AFRL AFSELy LL_GPIO_GetAFPin_0_7
LL_GPIO_SetAFPin_8_15

Function name

__STATIC_INLINE void LL_GPIO_SetAFPin_8_15 (GPIO_TypeDef * GPIOx, uint32_t Pin, uint32_t Alternate)

Function description

Configure gpio alternate function of a dedicated pin from 8 to 15 for a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
- **Alternate:** This parameter can be one of the following values:
 - LL_GPIO_AF_0
 - LL_GPIO_AF_1
 - LL_GPIO_AF_2
 - LL_GPIO_AF_3
 - LL_GPIO_AF_4
 - LL_GPIO_AF_5
 - LL_GPIO_AF_6
 - LL_GPIO_AF_7
 - LL_GPIO_AF_8
 - LL_GPIO_AF_9
 - LL_GPIO_AF_10
 - LL_GPIO_AF_11
 - LL_GPIO_AF_12
 - LL_GPIO_AF_13
 - LL_GPIO_AF_14
 - LL_GPIO_AF_15

Return values

- **None:**

Notes

- Possible values are from AF0 to AF15 depending on target.
- Warning: only one pin can be passed as parameter.

Reference Manual to LL API cross reference:

- AFRH AFSELy LL_GPIO_SetAFPin_8_15
LL_GPIO_GetAFPin_8_15

Function name

__STATIC_INLINE uint32_t LL_GPIO_GetAFPin_8_15 (GPIO_TypeDef * GPIOx, uint32_t Pin)

Function description

Return gpio alternate function of a dedicated pin from 8 to 15 for a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **Pin:** This parameter can be one of the following values:
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15

Return values

- **Returned:** value can be one of the following values:
 - LL_GPIO_AF_0
 - LL_GPIO_AF_1
 - LL_GPIO_AF_2
 - LL_GPIO_AF_3
 - LL_GPIO_AF_4
 - LL_GPIO_AF_5
 - LL_GPIO_AF_6
 - LL_GPIO_AF_7
 - LL_GPIO_AF_8
 - LL_GPIO_AF_9
 - LL_GPIO_AF_10
 - LL_GPIO_AF_11
 - LL_GPIO_AF_12
 - LL_GPIO_AF_13
 - LL_GPIO_AF_14
 - LL_GPIO_AF_15

Notes

- Possible values are from AF0 to AF15 depending on target.

Reference Manual to LL API cross reference:

- AFRH AFSELY LL_GPIO_GetAFPin_8_15
LL_GPIO_LockPin

Function name

__STATIC_INLINE void LL_GPIO_LockPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)

Function description

Lock configuration of several pins for a dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **None:**

Notes

- When the lock sequence has been applied on a port bit, the value of this port bit can no longer be modified until the next reset.
- Each lock bit freezes a specific configuration register (control and alternate function registers).

Reference Manual to LL API cross reference:

- LCKR LCKK LL_GPIO_LockPin
LL_GPIO_IsPinLocked

Function name

__STATIC_INLINE uint32_t LL_GPIO_IsPinLocked (GPIO_TypeDef * GPIOx, uint32_t PinMask)

Function description

Return 1 if all pins passed as parameter, of a dedicated port, are locked.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LCKR LCKy LL_GPIO_IsPinLocked
LL_GPIO_IsAnyPinLocked

Function name

__STATIC_INLINE uint32_t LL_GPIO_IsAnyPinLocked (GPIO_TypeDef * GPIOx)

Function description

Return 1 if one of the pin of a dedicated port is locked.

Parameters

- **GPIOx:** GPIO Port

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- LCKR LCKK LL_GPIO_IsAnyPinLocked
LL_GPIO_ReadInputPort

Function name

__STATIC_INLINE uint32_t LL_GPIO_ReadInputPort (GPIO_TypeDef * GPIOx)

Function description

Return full input data register value for a dedicated port.

Parameters

- **GPIOx:** GPIO Port

Return values

- **Input:** data register value of port

Reference Manual to LL API cross reference:

- IDR IDy LL_GPIO_ReadInputPort
- LL_GPIO_IsInputPinSet

Function name

__STATIC_INLINE uint32_t LL_GPIO_IsInputPinSet (GPIO_TypeDef * GPIOx, uint32_t PinMask)

Function description

Return if input data level for several pins of dedicated port is high or low.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- IDR IDy LL_GPIO_IsInputPinSet
- LL_GPIO_WriteOutputPort

Function name

__STATIC_INLINE void LL_GPIO_WriteOutputPort (GPIO_TypeDef * GPIOx, uint32_t PortValue)

Function description

Write output data register for the port.

Parameters

- **GPIOx:** GPIO Port
- **PortValue:** Level value for each pin of the port

Return values

- **None:**

Reference Manual to LL API cross reference:

- ODR ODy LL_GPIO_WriteOutputPort
LL_GPIO_ReadOutputPort

Function name

__STATIC_INLINE uint32_t LL_GPIO_ReadOutputPort (GPIO_TypeDef * GPIOx)

Function description

Return full output data register value for a dedicated port.

Parameters

- **GPIOx:** GPIO Port

Return values

- **Output:** data register value of port

Reference Manual to LL API cross reference:

- ODR ODy LL_GPIO_ReadOutputPort
LL_GPIO_IsOutputPinSet

Function name

__STATIC_INLINE uint32_t LL_GPIO_IsOutputPinSet (GPIO_TypeDef * GPIOx, uint32_t PinMask)

Function description

Return if input data level for several pins of dedicated port is high or low.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ODR ODy LL_GPIO_IsOutputPinSet

```
LL_GPIO_SetOutputPin
```

Function name

```
__STATIC_INLINE void LL_GPIO_SetOutputPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

Function description

Set several pins to high level on dedicated gpio port.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **None:**

Reference Manual to LL API cross reference:

- BSRR BSy LL_GPIO_SetOutputPin

```
LL_GPIO_ResetOutputPin
```

Function name

```
__STATIC_INLINE void LL_GPIO_ResetOutputPin (GPIO_TypeDef * GPIOx, uint32_t PinMask)
```

Function description

Set several pins to low level on dedicated gpio port.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **None:**

Reference Manual to LL API cross reference:

- BSRR BRy LL_GPIO_ResetOutputPin

LL_GPIO_TogglePin

Function name

__STATIC_INLINE void LL_GPIO_TogglePin (GPIO_TypeDef * GPIOx, uint32_t PinMask)

Function description

Toggle data value for several pin of dedicated port.

Parameters

- **GPIOx:** GPIO Port
- **PinMask:** This parameter can be a combination of the following values:
 - LL_GPIO_PIN_0
 - LL_GPIO_PIN_1
 - LL_GPIO_PIN_2
 - LL_GPIO_PIN_3
 - LL_GPIO_PIN_4
 - LL_GPIO_PIN_5
 - LL_GPIO_PIN_6
 - LL_GPIO_PIN_7
 - LL_GPIO_PIN_8
 - LL_GPIO_PIN_9
 - LL_GPIO_PIN_10
 - LL_GPIO_PIN_11
 - LL_GPIO_PIN_12
 - LL_GPIO_PIN_13
 - LL_GPIO_PIN_14
 - LL_GPIO_PIN_15
 - LL_GPIO_PIN_ALL

Return values

- **None:**

Reference Manual to LL API cross reference:

- ODR ODy LL_GPIO_TogglePin

LL_GPIO_DeInit

Function name

ErrorStatus LL_GPIO_DeInit (GPIO_TypeDef * GPIOx)

Function description

De-initialize GPIO registers (Registers restored to their default values).

Parameters

- **GPIOx:** GPIO Port

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: GPIO registers are de-initialized
 - ERROR: Wrong GPIO Port

LL_GPIO_Init

Function name

ErrorStatus LL_GPIO_Init (GPIO_TypeDef * GPIOx, LL_GPIO_InitTypeDef * GPIO_InitStruct)

Function description

Initialize GPIO registers according to the specified parameters in GPIO_InitStruct.

Parameters

- **GPIOx:** GPIO Port
- **GPIO_InitStruct:** pointer to a LL_GPIO_InitTypeDef structure that contains the configuration information for the specified GPIO peripheral.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: GPIO registers are initialized according to GPIO_InitStruct content
 - ERROR: Not applicable

LL_GPIO_StructInit

Function name

void LL_GPIO_StructInit (LL_GPIO_InitTypeDef * GPIO_InitStruct)

Function description

Set each LL_GPIO_InitTypeDef field to default value.

Parameters

- **GPIO_InitStruct:** pointer to a LL_GPIO_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

56.3 GPIO Firmware driver defines

The following section lists the various define and macros of the module.

56.3.1

GPIO

GPIO

Alternate Function

LL_GPIO_AF_0

Select alternate function 0

LL_GPIO_AF_1

Select alternate function 1

LL_GPIO_AF_2

Select alternate function 2

LL_GPIO_AF_3

Select alternate function 3

LL_GPIO_AF_4

Select alternate function 4

LL_GPIO_AF_5

Select alternate function 5

LL_GPIO_AF_6

Select alternate function 6

LL_GPIO_AF_7

Select alternate function 7

LL_GPIO_AF_8

Select alternate function 8

LL_GPIO_AF_9

Select alternate function 9

LL_GPIO_AF_10

Select alternate function 10

LL_GPIO_AF_11

Select alternate function 11

LL_GPIO_AF_12

Select alternate function 12

LL_GPIO_AF_13

Select alternate function 13

LL_GPIO_AF_14

Select alternate function 14

LL_GPIO_AF_15

Select alternate function 15

Mode**LL_GPIO_MODE_INPUT**

Select input mode

LL_GPIO_MODE_OUTPUT

Select output mode

LL_GPIO_MODE_ALTERNATE

Select alternate function mode

LL_GPIO_MODE_ANALOG

Select analog mode

Output Type**LL_GPIO_OUTPUT_PUSHPULL**

Select push-pull as output type

LL_GPIO_OUTPUT_OPENDRAIN

Select open-drain as output type

PIN**LL_GPIO_PIN_0**

Select pin 0

LL_GPIO_PIN_1

Select pin 1

LL_GPIO_PIN_2

Select pin 2

LL_GPIO_PIN_3

Select pin 3

LL_GPIO_PIN_4

Select pin 4

LL_GPIO_PIN_5

Select pin 5

LL_GPIO_PIN_6

Select pin 6

LL_GPIO_PIN_7

Select pin 7

LL_GPIO_PIN_8

Select pin 8

LL_GPIO_PIN_9

Select pin 9

LL_GPIO_PIN_10

Select pin 10

LL_GPIO_PIN_11

Select pin 11

LL_GPIO_PIN_12

Select pin 12

LL_GPIO_PIN_13

Select pin 13

LL_GPIO_PIN_14

Select pin 14

LL_GPIO_PIN_15

Select pin 15

LL_GPIO_PIN_ALL

Select all pins

Pull Up Pull Down**LL_GPIO_PULL_NO**

Select I/O no pull

LL_GPIO_PULL_UP

Select I/O pull up

LL_GPIO_PULL_DOWN

Select I/O pull down

Output Speed**LL_GPIO_SPEED_FREQ_LOW**

Select I/O low output speed

LL_GPIO_SPEED_FREQ_MEDIUM

Select I/O medium output speed

LL_GPIO_SPEED_FREQ_HIGH

Select I/O fast output speed

LL_GPIO_SPEED_FREQ_VERY_HIGH

Select I/O high output speed

Common Write and read registers Macros**LL_GPIO_WriteReg****Description:**

- Write a value in GPIO register.

Parameters:

- `__INSTANCE__`: GPIO Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_GPIO_ReadReg**Description:**

- Read a value in GPIO register.

Parameters:

- `__INSTANCE__`: GPIO Instance
- `__REG__`: Register to be read

Return value:

- Register: value

57 LL I2C Generic Driver

57.1 I2C Firmware driver registers structures

57.1.1 LL_I2C_InitTypeDef

LL_I2C_InitTypeDef is defined in the stm32f2xx_ll_i2c.h

Data Fields

- **uint32_t PeripheralMode**
- **uint32_t ClockSpeed**
- **uint32_t DutyCycle**
- **uint32_t OwnAddress1**
- **uint32_t TypeAcknowledge**
- **uint32_t OwnAddrSize**

Field Documentation

- **uint32_t LL_I2C_InitTypeDef::PeripheralMode**
Specifies the peripheral mode. This parameter can be a value of [I2C_LL_EC_PERIPHERAL_MODE](#) This feature can be modified afterwards using unitary function **LL_I2C_SetMode()**.
- **uint32_t LL_I2C_InitTypeDef::ClockSpeed**
Specifies the clock frequency. This parameter must be set to a value lower than 400kHz (in Hz) This feature can be modified afterwards using unitary function **LL_I2C_SetClockPeriod()** or **LL_I2C_SetDutyCycle()** or **LL_I2C_SetClockSpeedMode()** or **LL_I2C_ConfigSpeed()**.
- **uint32_t LL_I2C_InitTypeDef::DutyCycle**
Specifies the I2C fast mode duty cycle. This parameter can be a value of [I2C_LL_EC_DUTYCYCLE](#) This feature can be modified afterwards using unitary function **LL_I2C_SetDutyCycle()**.
- **uint32_t LL_I2C_InitTypeDef::OwnAddress1**
Specifies the device own address 1. This parameter must be a value between Min_Data = 0x00 and Max_Data = 0x3FF This feature can be modified afterwards using unitary function **LL_I2C_SetOwnAddress1()**.
- **uint32_t LL_I2C_InitTypeDef::TypeAcknowledge**
Specifies the ACKnowledge or Non ACKnowledge condition after the address receive match code or next received byte. This parameter can be a value of [I2C_LL_EC_I2C_ACKNOWLEDGE](#) This feature can be modified afterwards using unitary function **LL_I2C_AcknowledgeNextData()**.
- **uint32_t LL_I2C_InitTypeDef::OwnAddrSize**
Specifies the device own address 1 size (7-bit or 10-bit). This parameter can be a value of [I2C_LL_EC_OWNAADDRESS1](#) This feature can be modified afterwards using unitary function **LL_I2C_SetOwnAddress1()**.

57.2 I2C Firmware driver API description

The following section lists the various functions of the I2C library.

57.2.1 Detailed description of functions

LL_I2C_Enable

Function name

```
__STATIC_INLINE void LL_I2C_Enable (I2C_TypeDef * I2Cx)
```

Function description

Enable I2C peripheral (PE = 1).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 PE LL_I2C_Enable
LL_I2C_Disable

Function name

__STATIC_INLINE void LL_I2C_Disable (I2C_TypeDef * I2Cx)

Function description

Disable I2C peripheral (PE = 0).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 PE LL_I2C_Disable
LL_I2C_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabled (I2C_TypeDef * I2Cx)

Function description

Check if the I2C peripheral is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 PE LL_I2C_IsEnabled
LL_I2C_EnableDMAReq_TX

Function name

__STATIC_INLINE void LL_I2C_EnableDMAReq_TX (I2C_TypeDef * I2Cx)

Function description

Enable DMA transmission requests.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_EnableDMAReq_TX
LL_I2C_DisableDMAReq_TX

Function name

__STATIC_INLINE void LL_I2C_DisableDMAReq_TX (I2C_TypeDef * I2Cx)

Function description

Disable DMA transmission requests.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_DisableDMAReq_TX
LL_I2C_IsEnabledDMAReq_TX

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledDMAReq_TX (I2C_TypeDef * I2Cx)

Function description

Check if DMA transmission requests are enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_IsEnabledDMAReq_TX
LL_I2C_EnableDMAReq_RX

Function name

__STATIC_INLINE void LL_I2C_EnableDMAReq_RX (I2C_TypeDef * I2Cx)

Function description

Enable DMA reception requests.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_EnableDMAReq_RX
LL_I2C_DisableDMAReq_RX

Function name

__STATIC_INLINE void LL_I2C_DisableDMAReq_RX (I2C_TypeDef * I2Cx)

Function description

Disable DMA reception requests.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_DisableDMAReq_RX
LL_I2C_IsEnabledDMAReq_RX

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledDMAReq_RX (I2C_TypeDef * I2Cx)

Function description

Check if DMA reception requests are enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 DMAEN LL_I2C_IsEnabledDMAReq_RX
LL_I2C_DMA_GetRegAddr

Function name

__STATIC_INLINE uint32_t LL_I2C_DMA_GetRegAddr (I2C_TypeDef * I2Cx)

Function description

Get the data register address used for DMA transfer.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Address:** of data register

Reference Manual to LL API cross reference:

- DR DR LL_I2C_DMA_GetRegAddr
LL_I2C_EnableClockStretching

Function name

__STATIC_INLINE void LL_I2C_EnableClockStretching (I2C_TypeDef * I2Cx)

Function description

Enable Clock stretching.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- This bit can only be programmed when the I2C is disabled (PE = 0).

Reference Manual to LL API cross reference:

- CR1 NOSTRETCH LL_I2C_EnableClockStretching
LL_I2C_DisableClockStretching

Function name

```
__STATIC_INLINE void LL_I2C_DisableClockStretching (I2C_TypeDef * I2Cx)
```

Function description

Disable Clock stretching.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- This bit can only be programmed when the I2C is disabled (PE = 0).

Reference Manual to LL API cross reference:

- CR1 NOSTRETCH LL_I2C_DisableClockStretching
LL_I2C_IsEnabledClockStretching

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledClockStretching (I2C_TypeDef * I2Cx)
```

Function description

Check if Clock stretching is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 NOSTRETCH LL_I2C_IsEnabledClockStretching
LL_I2C_EnableGeneralCall

Function name

```
__STATIC_INLINE void LL_I2C_EnableGeneralCall (I2C_TypeDef * I2Cx)
```

Function description

Enable General Call.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- When enabled the Address 0x00 is ACKed.

Reference Manual to LL API cross reference:

- CR1 ENGCG LL_I2C_EnableGeneralCall
LL_I2C_DisableGeneralCall

Function name

```
__STATIC_INLINE void LL_I2C_DisableGeneralCall (I2C_TypeDef * I2Cx)
```

Function description

Disable General Call.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- When disabled the Address 0x00 is NACKed.

Reference Manual to LL API cross reference:

- CR1 ENGCG LL_I2C_DisableGeneralCall
LL_I2C_IsEnabledGeneralCall

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledGeneralCall (I2C_TypeDef * I2Cx)
```

Function description

Check if General Call is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 ENGCG LL_I2C_IsEnabledGeneralCall
LL_I2C_SetOwnAddress1

Function name

```
__STATIC_INLINE void LL_I2C_SetOwnAddress1 (I2C_TypeDef * I2Cx, uint32_t OwnAddress1, uint32_t OwnAddrSize)
```

Function description

Set the Own Address1.

Parameters

- **I2Cx**: I2C Instance.
- **OwnAddress1**: This parameter must be a value between Min_Data=0 and Max_Data=0x3FF.
- **OwnAddrSize**: This parameter can be one of the following values:
 - LL_I2C_OWNADDRESS1_7BIT
 - LL_I2C_OWNADDRESS1_10BIT

Return values

- **None**:

Reference Manual to LL API cross reference:

- OAR1 ADD0 LL_I2C_SetOwnAddress1
- OAR1 ADD1_7 LL_I2C_SetOwnAddress1
- OAR1 ADD8_9 LL_I2C_SetOwnAddress1
- OAR1 ADDMODE LL_I2C_SetOwnAddress1

LL_I2C_SetOwnAddress2

Function name

__STATIC_INLINE void LL_I2C_SetOwnAddress2 (I2C_TypeDef * I2Cx, uint32_t OwnAddress2)

Function description

Set the 7bits Own Address2.

Parameters

- **I2Cx**: I2C Instance.
- **OwnAddress2**: This parameter must be a value between Min_Data=0 and Max_Data=0x7F.

Return values

- **None**:

Notes

- This action has no effect if own address2 is enabled.

Reference Manual to LL API cross reference:

- OAR2 ADD2 LL_I2C_SetOwnAddress2

LL_I2C_EnableOwnAddress2

Function name

__STATIC_INLINE void LL_I2C_EnableOwnAddress2 (I2C_TypeDef * I2Cx)

Function description

Enable acknowledge on Own Address2 match address.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Reference Manual to LL API cross reference:

- OAR2 ENDUAL LL_I2C_EnableOwnAddress2

LL_I2C_DisableOwnAddress2

Function name

```
__STATIC_INLINE void LL_I2C_DisableOwnAddress2 (I2C_TypeDef * I2Cx)
```

Function description

Disable acknowledge on Own Address2 match address.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- OAR2 ENDUAL LL_I2C_DisableOwnAddress2

```
LL_I2C_IsEnabledOwnAddress2
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledOwnAddress2 (I2C_TypeDef * I2Cx)
```

Function description

Check if Own Address1 acknowledge is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- OAR2 ENDUAL LL_I2C_IsEnabledOwnAddress2

```
LL_I2C_SetPeriphClock
```

Function name

```
__STATIC_INLINE void LL_I2C_SetPeriphClock (I2C_TypeDef * I2Cx, uint32_t PeriphClock)
```

Function description

Configure the Peripheral clock frequency.

Parameters

- **I2Cx:** I2C Instance.
- **PeriphClock:** Peripheral Clock (in Hz)

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 FREQ LL_I2C_SetPeriphClock

```
LL_I2C_GetPeriphClock
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_GetPeriphClock (I2C_TypeDef * I2Cx)
```

Function description

Get the Peripheral clock frequency.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Value:** of Peripheral Clock (in Hz)

Reference Manual to LL API cross reference:

- CR2_FREQ LL_I2C_GetPeriphClock
LL_I2C_SetDutyCycle

Function name

```
__STATIC_INLINE void LL_I2C_SetDutyCycle (I2C_TypeDef * I2Cx, uint32_t DutyCycle)
```

Function description

Configure the Duty cycle (Fast mode only).

Parameters

- **I2Cx:** I2C Instance.
- **DutyCycle:** This parameter can be one of the following values:
 - LL_I2C_DUTYCYCLE_2
 - LL_I2C_DUTYCYCLE_16_9

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCR_DUTY LL_I2C_SetDutyCycle
LL_I2C_GetDutyCycle

Function name

```
__STATIC_INLINE uint32_t LL_I2C_GetDutyCycle (I2C_TypeDef * I2Cx)
```

Function description

Get the Duty cycle (Fast mode only).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Returned:** value can be one of the following values:
 - LL_I2C_DUTYCYCLE_2
 - LL_I2C_DUTYCYCLE_16_9

Reference Manual to LL API cross reference:

- CCR_DUTY LL_I2C_GetDutyCycle
LL_I2C_SetClockSpeedMode

Function name

```
__STATIC_INLINE void LL_I2C_SetClockSpeedMode (I2C_TypeDef * I2Cx, uint32_t ClockSpeedMode)
```

Function description

Configure the I2C master clock speed mode.

Parameters

- **I2Cx:** I2C Instance.
- **ClockSpeedMode:** This parameter can be one of the following values:
 - LL_I2C_CLOCK_SPEED_STANDARD_MODE
 - LL_I2C_CLOCK_SPEED_FAST_MODE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCR FS LL_I2C_SetClockSpeedMode

LL_I2C_GetClockSpeedMode

Function name

__STATIC_INLINE uint32_t LL_I2C_GetClockSpeedMode (I2C_TypeDef * I2Cx)

Function description

Get the the I2C master speed mode.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Returned:** value can be one of the following values:
 - LL_I2C_CLOCK_SPEED_STANDARD_MODE
 - LL_I2C_CLOCK_SPEED_FAST_MODE

Reference Manual to LL API cross reference:

- CCR FS LL_I2C_GetClockSpeedMode

LL_I2C_SetRiseTime

Function name

__STATIC_INLINE void LL_I2C_SetRiseTime (I2C_TypeDef * I2Cx, uint32_t RiseTime)

Function description

Configure the SCL, SDA rising time.

Parameters

- **I2Cx:** I2C Instance.
- **RiseTime:** This parameter must be a value between Min_Data=0x02 and Max_Data=0x3F.

Return values

- **None:**

Notes

- This bit can only be programmed when the I2C is disabled (PE = 0).

Reference Manual to LL API cross reference:

- TRISE TRISE LL_I2C_SetRiseTime

LL_I2C_GetRiseTime

Function name

__STATIC_INLINE uint32_t LL_I2C_GetRiseTime (I2C_TypeDef * I2Cx)

Function description

Get the SCL, SDA rising time.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Value:** between Min_Data=0x02 and Max_Data=0x3F

Reference Manual to LL API cross reference:

- TRISE TRISE LL_I2C_GetRiseTime
LL_I2C_SetClockPeriod

Function name

__STATIC_INLINE void LL_I2C_SetClockPeriod (I2C_TypeDef * I2Cx, uint32_t ClockPeriod)

Function description

Configure the SCL high and low period.

Parameters

- **I2Cx:** I2C Instance.
- **ClockPeriod:** This parameter must be a value between Min_Data=0x004 and Max_Data=0xFFF, except in FAST DUTY mode where Min_Data=0x001.

Return values

- **None:**

Notes

- This bit can only be programmed when the I2C is disabled (PE = 0).

Reference Manual to LL API cross reference:

- CCR CCR LL_I2C_SetClockPeriod
LL_I2C_GetClockPeriod

Function name

__STATIC_INLINE uint32_t LL_I2C_GetClockPeriod (I2C_TypeDef * I2Cx)

Function description

Get the SCL high and low period.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Value:** between Min_Data=0x004 and Max_Data=0xFFF, except in FAST DUTY mode where Min_Data=0x001.

Reference Manual to LL API cross reference:

- CCR CCR LL_I2C_GetClockPeriod
LL_I2C_ConfigSpeed

Function name

__STATIC_INLINE void LL_I2C_ConfigSpeed (I2C_TypeDef * I2Cx, uint32_t PeriphClock, uint32_t ClockSpeed, uint32_t DutyCycle)

Function description

Configure the SCL speed.

Parameters

- **I2Cx:** I2C Instance.
- **PeriphClock:** Peripheral Clock (in Hz)
- **ClockSpeed:** This parameter must be a value lower than 400kHz (in Hz).
- **DutyCycle:** This parameter can be one of the following values:
 - LL_I2C_DUTYCYCLE_2
 - LL_I2C_DUTYCYCLE_16_9

Return values

- **None:**

Notes

- This bit can only be programmed when the I2C is disabled (PE = 0).

Reference Manual to LL API cross reference:

- CR2_FREQ LL_I2C_ConfigSpeed
- TRISE_TRISE LL_I2C_ConfigSpeed
- CCR_FS LL_I2C_ConfigSpeed
- CCR_DUTY LL_I2C_ConfigSpeed
- CCR_CCR LL_I2C_ConfigSpeed

LL_I2C_SetMode

Function name

__STATIC_INLINE void LL_I2C_SetMode (I2C_TypeDef * I2Cx, uint32_t PeripheralMode)

Function description

Configure peripheral mode.

Parameters

- **I2Cx:** I2C Instance.
- **PeripheralMode:** This parameter can be one of the following values:
 - LL_I2C_MODE_I2C
 - LL_I2C_MODE_SMBUS_HOST
 - LL_I2C_MODE_SMBUS_DEVICE
 - LL_I2C_MODE_SMBUS_DEVICE_ARP

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1_SMBUS LL_I2C_SetMode
- CR1_SMBTYPE LL_I2C_SetMode
- CR1_ENARP LL_I2C_SetMode

LL_I2C_GetMode

Function name

```
__STATIC_INLINE uint32_t LL_I2C_GetMode (I2C_TypeDef * I2Cx)
```

Function description

Get peripheral mode.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Returned:** value can be one of the following values:
 - LL_I2C_MODE_I2C
 - LL_I2C_MODE_SMBUS_HOST
 - LL_I2C_MODE_SMBUS_DEVICE
 - LL_I2C_MODE_SMBUS_DEVICE_ARP

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 SMBUS LL_I2C_GetMode
- CR1 SMBTYPE LL_I2C_GetMode
- CR1 ENARP LL_I2C_GetMode

LL_I2C_EnableSMBusAlert

Function name

```
__STATIC_INLINE void LL_I2C_EnableSMBusAlert (I2C_TypeDef * I2Cx)
```

Function description

Enable SMBus alert (Host or Device mode)

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- SMBus Device mode: SMBus Alert pin is driven low and Alert Response Address Header acknowledge is enabled. SMBus Host mode: SMBus Alert pin management is supported.

Reference Manual to LL API cross reference:

- CR1 ALERT LL_I2C_EnableSMBusAlert

LL_I2C_DisableSMBusAlert

Function name

```
__STATIC_INLINE void LL_I2C_DisableSMBusAlert (I2C_TypeDef * I2Cx)
```

Function description

Disable SMBus alert (Host or Device mode)

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- SMBus Device mode: SMBus Alert pin is not driven (can be used as a standard GPIO) and Alert Response Address Header acknowledge is disabled. SMBus Host mode: SMBus Alert pin management is not supported.

Reference Manual to LL API cross reference:

- CR1 ALERT LL_I2C_DisableSMBusAlert

LL_I2C_IsEnabledSMBusAlert

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledSMBusAlert (I2C_TypeDef * I2Cx)

Function description

Check if SMBus alert (Host or Device mode) is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 ALERT LL_I2C_IsEnabledSMBusAlert

LL_I2C_EnableSMBusPEC

Function name

__STATIC_INLINE void LL_I2C_EnableSMBusPEC (I2C_TypeDef * I2Cx)

Function description

Enable SMBus Packet Error Calculation (PEC).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 ENPEC LL_I2C_EnableSMBusPEC

```
LL_I2C_DisableSMBusPEC
```

Function name

```
__STATIC_INLINE void LL_I2C_DisableSMBusPEC (I2C_TypeDef * I2Cx)
```

Function description

Disable SMBus Packet Error Calculation (PEC).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 ENPEC LL_I2C_DisableSMBusPEC

```
LL_I2C_IsEnabledSMBusPEC
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledSMBusPEC (I2C_TypeDef * I2Cx)
```

Function description

Check if SMBus Packet Error Calculation (PEC) is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 ENPEC LL_I2C_IsEnabledSMBusPEC

```
LL_I2C_EnableIT_TX
```

Function name

```
__STATIC_INLINE void LL_I2C_EnableIT_TX (I2C_TypeDef * I2Cx)
```

Function description

Enable TXE interrupt.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_EnableIT_TX
- CR2 ITBUFEN LL_I2C_EnableIT_TX

LL_I2C_DisableIT_TX

Function name

__STATIC_INLINE void LL_I2C_DisableIT_TX (I2C_TypeDef * I2Cx)

Function description

Disable TXE interrupt.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_DisableIT_TX
- CR2 ITBUFEN LL_I2C_DisableIT_TX

LL_I2C_IsEnabledIT_TX

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledIT_TX (I2C_TypeDef * I2Cx)

Function description

Check if the TXE Interrupt is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_IsEnabledIT_TX
- CR2 ITBUFEN LL_I2C_IsEnabledIT_TX

LL_I2C_EnableIT_RX

Function name

__STATIC_INLINE void LL_I2C_EnableIT_RX (I2C_TypeDef * I2Cx)

Function description

Enable RXNE interrupt.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_EnableIT_RX
- CR2 ITBUFEN LL_I2C_EnableIT_RX

```
LL_I2C_DisableIT_RX
```

Function name

```
__STATIC_INLINE void LL_I2C_DisableIT_RX (I2C_TypeDef * I2Cx)
```

Function description

Disable RXNE interrupt.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_DisableIT_RX
- CR2 ITBUFEN LL_I2C_DisableIT_RX

```
LL_I2C_IsEnabledIT_RX
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledIT_RX (I2C_TypeDef * I2Cx)
```

Function description

Check if the RXNE Interrupt is enabled or disabled.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_IsEnabledIT_RX
- CR2 ITBUFEN LL_I2C_IsEnabledIT_RX

```
LL_I2C_EnableIT_EVT
```

Function name

```
__STATIC_INLINE void LL_I2C_EnableIT_EVT (I2C_TypeDef * I2Cx)
```

Function description

Enable Events interrupts.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Notes

- Any of these events will generate interrupt : Start Bit (SB) Address sent, Address matched (ADDR) 10-bit header sent (ADD10) Stop detection (STOPF) Byte transfer finished (BTF)
- Any of these events will generate interrupt if Buffer interrupts are enabled too(using unitary function LL_I2C_EnableIT_BUF()) : Receive buffer not empty (RXNE) Transmit buffer empty (TXE)

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_EnableIT_EVT

```
LL_I2C_DisableIT_EVT
```

Function name

```
__STATIC_INLINE void LL_I2C_DisableIT_EVT (I2C_TypeDef * I2Cx)
```

Function description

Disable Events interrupts.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Notes

- Any of these events will generate interrupt : Start Bit (SB) Address sent, Address matched (ADDR) 10-bit header sent (ADD10) Stop detection (STOPF) Byte transfer finished (BTF) Receive buffer not empty (RXNE) Transmit buffer empty (TXE)

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_DisableIT_EVT

```
LL_I2C_IsEnabledIT_EVT
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledIT_EVT (I2C_TypeDef * I2Cx)
```

Function description

Check if Events interrupts are enabled or disabled.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ITEVTEN LL_I2C_IsEnabledIT_EVT

```
LL_I2C_EnableIT_BUF
```

Function name

```
__STATIC_INLINE void LL_I2C_EnableIT_BUF (I2C_TypeDef * I2Cx)
```

Function description

Enable Buffer interrupts.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Notes

- Any of these Buffer events will generate interrupt if Events interrupts are enabled too(using unitary function LL_I2C_EnableIT_EVT()) : Receive buffer not empty (RXNE) Transmit buffer empty (TXE)

Reference Manual to LL API cross reference:

- CR2 ITBUFEN LL_I2C_EnableIT_BUF
LL_I2C_DisableIT_BUF

Function name

```
__STATIC_INLINE void LL_I2C_DisableIT_BUF (I2C_TypeDef * I2Cx)
```

Function description

Disable Buffer interrupts.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Notes

- Any of these Buffer events will generate interrupt : Receive buffer not empty (RXNE) Transmit buffer empty (TXE)

Reference Manual to LL API cross reference:

- CR2 ITBUFEN LL_I2C_DisableIT_BUF
LL_I2C_IsEnabledIT_BUF

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledIT_BUF (I2C_TypeDef * I2Cx)
```

Function description

Check if Buffer interrupts are enabled or disabled.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ITBUFEN LL_I2C_IsEnabledIT_BUF
LL_I2C_EnableIT_ERR

Function name

```
__STATIC_INLINE void LL_I2C_EnableIT_ERR (I2C_TypeDef * I2Cx)
```

Function description

Enable Error interrupts.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **None**:

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- Any of these errors will generate interrupt : Bus Error detection (BERR) Arbitration Loss (ARLO) Acknowledge Failure(AF) Overrun/Underrun (OVR) SMBus Timeout detection (TIMEOUT) SMBus PEC error detection (PECERR) SMBus Alert pin event detection (SMBALERT)

Reference Manual to LL API cross reference:

- CR2 ITERREN `LL_I2C_EnableIT_ERR`
`LL_I2C_DisableIT_ERR`

Function name

__STATIC_INLINE void LL_I2C_DisableIT_ERR (I2C_TypeDef * I2Cx)

Function description

Disable Error interrupts.

Parameters

- I2Cx:** I2C Instance.

Return values

- None:**

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- Any of these errors will generate interrupt : Bus Error detection (BERR) Arbitration Loss (ARLO) Acknowledge Failure(AF) Overrun/Underrun (OVR) SMBus Timeout detection (TIMEOUT) SMBus PEC error detection (PECERR) SMBus Alert pin event detection (SMBALERT)

Reference Manual to LL API cross reference:

- CR2 ITERREN `LL_I2C_DisableIT_ERR`
`LL_I2C_IsEnabledIT_ERR`

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledIT_ERR (I2C_TypeDef * I2Cx)

Function description

Check if Error interrupts are enabled or disabled.

Parameters

- I2Cx:** I2C Instance.

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ITERREN `LL_I2C_IsEnabledIT_ERR`
`LL_I2C_IsActiveFlag_TXE`

Function name

__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_TXE (I2C_TypeDef * I2Cx)

Function description

Indicate the status of Transmit data register empty flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- RESET: When next data is written in Transmit data register. SET: When Transmit data register is empty.

Reference Manual to LL API cross reference:

- SR1 TXE LL_I2C_IsActiveFlag_TXE
LL_I2C_IsActiveFlag_BTf

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_BTf (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Byte Transfer Finished flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR1 BTf LL_I2C_IsActiveFlag_BTf
LL_I2C_IsActiveFlag_RXNE

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_RXNE (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Receive data register not empty flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- RESET: When Receive data register is read. SET: When the received data is copied in Receive data register.

Reference Manual to LL API cross reference:

- SR1 RXNE LL_I2C_IsActiveFlag_RXNE
LL_I2C_IsActiveFlag_SB

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_SB (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Start Bit (master mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: When No Start condition. SET: When Start condition is generated.

Reference Manual to LL API cross reference:

- SR1 SB LL_I2C_IsActiveFlag_SB
LL_I2C_IsActiveFlag_ADDR

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_ADDR (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Address sent (master mode) or Address matched flag (slave mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When the address is fully sent (master mode) or when the received slave address matched with one of the enabled slave address (slave mode).

Reference Manual to LL API cross reference:

- SR1 ADDR LL_I2C_IsActiveFlag_ADDR
LL_I2C_IsActiveFlag_ADD10

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_ADD10 (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of 10-bit header sent (master mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: When no ADD10 event occurred. SET: When the master has sent the first address byte (header).

Reference Manual to LL API cross reference:

- SR1 ADD10 LL_I2C_IsActiveFlag_ADD10
LL_I2C_IsActiveFlag_AF

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_AF (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Acknowledge failure flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: No acknowledge failure. SET: When an acknowledge failure is received after a byte transmission.

Reference Manual to LL API cross reference:

- SR1 AF LL_I2C_IsActiveFlag_AF

LL_I2C_IsActiveFlag_STOP

Function name

__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_STOP (I2C_TypeDef * I2Cx)

Function description

Indicate the status of Stop detection flag (slave mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When a Stop condition is detected.

Reference Manual to LL API cross reference:

- SR1 STOPF LL_I2C_IsActiveFlag_STOP

LL_I2C_IsActiveFlag_BERR

Function name

__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_BERR (I2C_TypeDef * I2Cx)

Function description

Indicate the status of Bus error flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When a misplaced Start or Stop condition is detected.

Reference Manual to LL API cross reference:

- SR1 BERR LL_I2C_IsActiveFlag_BERR

LL_I2C_IsActiveFlag_ARLO

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_ARLO (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Arbitration lost flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When arbitration lost.

Reference Manual to LL API cross reference:

- SR1 ARLO LL_I2C_IsActiveFlag_ARLO

LL_I2C_IsActiveFlag_OVR

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_OVR (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Overrun/Underrun flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When an overrun/underrun error occurs (Clock Stretching Disabled).

Reference Manual to LL API cross reference:

- SR1 OVR LL_I2C_IsActiveFlag_OVR

LL_I2C_IsActiveSMBusFlag_PECERR

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveSMBusFlag_PECERR (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of SMBus PEC error flag in reception.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR1 PECERR LL_I2C_IsActiveSMBusFlag_PECERR

```
LL_I2C_IsActiveSMBusFlag_TIMEOUT
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveSMBusFlag_TIMEOUT (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of SMBus Timeout detection flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR1 TIMEOUT LL_I2C_IsActiveSMBusFlag_TIMEOUT

```
LL_I2C_IsActiveSMBusFlag_ALERT
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveSMBusFlag_ALERT (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of SMBus alert flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR1 SMBALERT LL_I2C_IsActiveSMBusFlag_ALERT

```
LL_I2C_IsActiveFlag_BUSY
```

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_BUSY (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Bus Busy flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Clear default value. SET: When a Start condition is detected.

Reference Manual to LL API cross reference:

- SR2 BUSY LL_I2C_IsActiveFlag_BUSY
LL_I2C_IsActiveFlag_DUAL

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_DUAL (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of Dual flag.

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- RESET: Received address matched with OAR1. SET: Received address matched with OAR2.

Reference Manual to LL API cross reference:

- SR2 DUALF LL_I2C_IsActiveFlag_DUAL
LL_I2C_IsActiveSMBusFlag_SMBHOST

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveSMBusFlag_SMBHOST (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of SMBus Host address reception (Slave mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State**: of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- RESET: No SMBus Host address SET: SMBus Host address received.
- This status is cleared by hardware after a STOP condition or repeated START condition.

Reference Manual to LL API cross reference:

- SR2 SMBHOST LL_I2C_IsActiveSMBusFlag_SMBHOST
LL_I2C_IsActiveSMBusFlag_SMBDEFAULT

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsActiveSMBusFlag_SMBDEFAULT (I2C_TypeDef * I2Cx)
```

Function description

Indicate the status of SMBus Device default address reception (Slave mode).

Parameters

- **I2Cx**: I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_SMBUS_ALL_INSTANCE(I2Cx)` can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- RESET: No SMBus Device default address SET: SMBus Device default address received.
- This status is cleared by hardware after a STOP condition or repeated START condition.

Reference Manual to LL API cross reference:

- SR2 SMBDEFAULT LL_I2C_IsActiveSMBusFlag_SMBDEFAULT
LL_I2C_IsActiveFlag_GENCALL

Function name

__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_GENCALL (I2C_TypeDef * I2Cx)

Function description

Indicate the status of General call address reception (Slave mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- RESET: No General call address SET: General call address received.
- This status is cleared by hardware after a STOP condition or repeated START condition.

Reference Manual to LL API cross reference:

- SR2 GENCALL LL_I2C_IsActiveFlag_GENCALL
LL_I2C_IsActiveFlag_MSL

Function name

__STATIC_INLINE uint32_t LL_I2C_IsActiveFlag_MSL (I2C_TypeDef * I2Cx)

Function description

Indicate the status of Master/Slave flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- RESET: Slave Mode. SET: Master Mode.

Reference Manual to LL API cross reference:

- SR2 MSL LL_I2C_IsActiveFlag_MSL
LL_I2C_ClearFlag_ADDR

Function name

__STATIC_INLINE void LL_I2C_ClearFlag_ADDR (I2C_TypeDef * I2Cx)

Function description

Clear Address Matched flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the I2Cx_SR1 register followed by a read access to the I2Cx_SR2 register.

Reference Manual to LL API cross reference:

- SR1 ADDR LL_I2C_ClearFlag_ADDR

LL_I2C_ClearFlag_AF

Function name

__STATIC_INLINE void LL_I2C_ClearFlag_AF (I2C_TypeDef * I2Cx)

Function description

Clear Acknowledge failure flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR1 AF LL_I2C_ClearFlag_AF

LL_I2C_ClearFlag_STOP

Function name

__STATIC_INLINE void LL_I2C_ClearFlag_STOP (I2C_TypeDef * I2Cx)

Function description

Clear Stop detection flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the I2Cx_SR1 register followed by a write access to I2Cx_CR1 register.

Reference Manual to LL API cross reference:

- SR1 STOPF LL_I2C_ClearFlag_STOP
- CR1 PE LL_I2C_ClearFlag_STOP

LL_I2C_ClearFlag_BERR

Function name

```
__STATIC_INLINE void LL_I2C_ClearFlag_BERR (I2C_TypeDef * I2Cx)
```

Function description

Clear Bus error flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR1 BERR LL_I2C_ClearFlag_BERR

LL_I2C_ClearFlag_ARLO

Function name

```
__STATIC_INLINE void LL_I2C_ClearFlag_ARLO (I2C_TypeDef * I2Cx)
```

Function description

Clear Arbitration lost flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR1 ARLO LL_I2C_ClearFlag_ARLO

LL_I2C_ClearFlag_OVR

Function name

```
__STATIC_INLINE void LL_I2C_ClearFlag_OVR (I2C_TypeDef * I2Cx)
```

Function description

Clear Overrun/Underrun flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR1 OVR LL_I2C_ClearFlag_OVR

LL_I2C_ClearSMBusFlag_PECERR

Function name

```
__STATIC_INLINE void LL_I2C_ClearSMBusFlag_PECERR (I2C_TypeDef * I2Cx)
```

Function description

Clear SMBus PEC error flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR1 PECERR LL_I2C_ClearSMBusFlag_PECERR
LL_I2C_ClearSMBusFlag_TIMEOUT

Function name

__STATIC_INLINE void LL_I2C_ClearSMBusFlag_TIMEOUT (I2C_TypeDef * I2Cx)

Function description

Clear SMBus Timeout detection flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR1 TIMEOUT LL_I2C_ClearSMBusFlag_TIMEOUT
LL_I2C_ClearSMBusFlag_ALERT

Function name

__STATIC_INLINE void LL_I2C_ClearSMBusFlag_ALERT (I2C_TypeDef * I2Cx)

Function description

Clear SMBus Alert flag.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR1 SMBALERT LL_I2C_ClearSMBusFlag_ALERT
LL_I2C_EnableReset

Function name

__STATIC_INLINE void LL_I2C_EnableReset (I2C_TypeDef * I2Cx)

Function description

Enable Reset of I2C peripheral.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 SWRST LL_I2C_EnableReset
LL_I2C_DisableReset

Function name

```
__STATIC_INLINE void LL_I2C_DisableReset (I2C_TypeDef * I2Cx)
```

Function description

Disable Reset of I2C peripheral.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 SWRST LL_I2C_DisableReset
LL_I2C_IsResetEnabled

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsResetEnabled (I2C_TypeDef * I2Cx)
```

Function description

Check if the I2C peripheral is under reset state or not.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 SWRST LL_I2C_IsResetEnabled
LL_I2C_AcknowledgeNextData

Function name

```
__STATIC_INLINE void LL_I2C_AcknowledgeNextData (I2C_TypeDef * I2Cx, uint32_t TypeAcknowledge)
```

Function description

Prepare the generation of a ACKnowledge or Non ACKnowledge condition after the address receive match code or next received byte.

Parameters

- **I2Cx:** I2C Instance.
- **TypeAcknowledge:** This parameter can be one of the following values:
 - LL_I2C_ACK
 - LL_I2C_NACK

Return values

- **None:**

Notes

- Usage in Slave or Master mode.

Reference Manual to LL API cross reference:

- CR1 ACK LL_I2C_AcknowledgeNextData
LL_I2C_GenerateStartCondition

Function name

```
__STATIC_INLINE void LL_I2C_GenerateStartCondition (I2C_TypeDef * I2Cx)
```

Function description

Generate a START or RESTART condition.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- The START bit can be set even if bus is BUSY or I2C is in slave mode. This action has no effect when RELOAD is set.

Reference Manual to LL API cross reference:

- CR1 START LL_I2C_GenerateStartCondition
LL_I2C_GenerateStopCondition

Function name

```
__STATIC_INLINE void LL_I2C_GenerateStopCondition (I2C_TypeDef * I2Cx)
```

Function description

Generate a STOP condition after the current byte transfer (master mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 STOP LL_I2C_GenerateStopCondition
LL_I2C_EnableBitPOS

Function name

```
__STATIC_INLINE void LL_I2C_EnableBitPOS (I2C_TypeDef * I2Cx)
```

Function description

Enable bit POS (master/host mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- In that case, the ACK bit controls the (N)ACK of the next byte received or the PEC bit indicates that the next byte in shift register is a PEC.

Reference Manual to LL API cross reference:

- CR1 POS LL_I2C_EnableBitPOS
LL_I2C_DisableBitPOS

Function name

```
__STATIC_INLINE void LL_I2C_DisableBitPOS (I2C_TypeDef * I2Cx)
```

Function description

Disable bit POS (master/host mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- In that case, the ACK bit controls the (N)ACK of the current byte received or the PEC bit indicates that the current byte in shift register is a PEC.

Reference Manual to LL API cross reference:

- CR1 POS LL_I2C_DisableBitPOS
LL_I2C_IsEnabledBitPOS

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledBitPOS (I2C_TypeDef * I2Cx)
```

Function description

Check if bit POS is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 POS LL_I2C_IsEnabledBitPOS
LL_I2C_GetTransferDirection

Function name

```
__STATIC_INLINE uint32_t LL_I2C_GetTransferDirection (I2C_TypeDef * I2Cx)
```

Function description

Indicate the value of transfer direction.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Returned:** value can be one of the following values:
 - LL_I2C_DIRECTION_WRITE
 - LL_I2C_DIRECTION_READ

Notes

- RESET: Bus is in read transfer (peripheral point of view). SET: Bus is in write transfer (peripheral point of view).

Reference Manual to LL API cross reference:

- SR2 TRA LL_I2C_GetTransferDirection
- LL_I2C_EnableLastDMA

Function name

__STATIC_INLINE void LL_I2C_EnableLastDMA (I2C_TypeDef * I2Cx)

Function description

Enable DMA last transfer.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- This action mean that next DMA EOT is the last transfer.

Reference Manual to LL API cross reference:

- CR2 LAST LL_I2C_EnableLastDMA
- LL_I2C_DisableLastDMA

Function name

__STATIC_INLINE void LL_I2C_DisableLastDMA (I2C_TypeDef * I2Cx)

Function description

Disable DMA last transfer.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- This action mean that next DMA EOT is not the last transfer.

Reference Manual to LL API cross reference:

- CR2 LAST LL_I2C_DisableLastDMA
- LL_I2C_IsEnabledLastDMA

Function name

__STATIC_INLINE uint32_t LL_I2C_IsEnabledLastDMA (I2C_TypeDef * I2Cx)

Function description

Check if DMA last transfer is enabled or disabled.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 LAST LL_I2C_IsEnabledLastDMA

LL_I2C_EnableSMBusPECCompare

Function name

__STATIC_INLINE void LL_I2C_EnableSMBusPECCompare (I2C_TypeDef * I2Cx)

Function description

Enable transfer or internal comparison of the SMBus Packet Error byte (transmission or reception mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.
- This feature is cleared by hardware when the PEC byte is transferred or compared, or by a START or STOP condition, it is also cleared by software.

Reference Manual to LL API cross reference:

- CR1 PEC LL_I2C_EnableSMBusPECCompare

LL_I2C_DisableSMBusPECCompare

Function name

__STATIC_INLINE void LL_I2C_DisableSMBusPECCompare (I2C_TypeDef * I2Cx)

Function description

Disable transfer or internal comparison of the SMBus Packet Error byte (transmission or reception mode).

Parameters

- **I2Cx:** I2C Instance.

Return values

- **None:**

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 PEC LL_I2C_DisableSMBusPECCompare

LL_I2C_IsEnabledSMBusPECCompare

Function name

```
__STATIC_INLINE uint32_t LL_I2C_IsEnabledSMBusPECCompare (I2C_TypeDef * I2Cx)
```

Function description

Check if the SMBus Packet Error byte transfer or internal comparison is requested or not.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- CR1 PEC LL_I2C_IsEnabledSMBusPECCompare

LL_I2C_GetSMBusPEC

Function name

```
__STATIC_INLINE uint32_t LL_I2C_GetSMBusPEC (I2C_TypeDef * I2Cx)
```

Function description

Get the SMBus Packet Error byte calculated.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- Macro IS_SMBUS_ALL_INSTANCE(I2Cx) can be used to check whether or not SMBus feature is supported by the I2Cx Instance.

Reference Manual to LL API cross reference:

- SR2 PEC LL_I2C_GetSMBusPEC

LL_I2C_ReceiveData8

Function name

```
__STATIC_INLINE uint8_t LL_I2C_ReceiveData8 (I2C_TypeDef * I2Cx)
```

Function description

Read Receive Data register.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **Value:** between Min_Data=0x0 and Max_Data=0xFF

Reference Manual to LL API cross reference:

- DR DR LL_I2C_ReceiveData8

LL_I2C_TransmitData8

Function name

```
__STATIC_INLINE void LL_I2C_TransmitData8 (I2C_TypeDef * I2Cx, uint8_t Data)
```

Function description

Write in Transmit Data Register .

Parameters

- **I2Cx:** I2C Instance.
- **Data:** Value between Min_Data=0x0 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_I2C_TransmitData8
- LL_I2C_Init

Function name

```
uint32_t LL_I2C_Init (I2C_TypeDef * I2Cx, LL_I2C_InitTypeDef * I2C_InitStruct)
```

Function description

Initialize the I2C registers according to the specified parameters in I2C_InitStruct.

Parameters

- **I2Cx:** I2C Instance.
- **I2C_InitStruct:** pointer to a LL_I2C_InitTypeDef structure.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS I2C registers are initialized
 - ERROR Not applicable

```
LL_I2C_DeInit
```

Function name

```
uint32_t LL_I2C_DeInit (I2C_TypeDef * I2Cx)
```

Function description

De-initialize the I2C registers to their default reset values.

Parameters

- **I2Cx:** I2C Instance.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS I2C registers are de-initialized
 - ERROR I2C registers are not de-initialized

```
LL_I2C_StructInit
```

Function name

```
void LL_I2C_StructInit (LL_I2C_InitTypeDef * I2C_InitStruct)
```

Function description

Set each LL_I2C_InitTypeDef field to default value.

Parameters

- **I2C_InitStruct:** Pointer to a LL_I2C_InitTypeDef structure.

Return values

- **None:**

57.3 I2C Firmware driver defines

The following section lists the various define and macros of the module.

57.3.1 I2C

I2C

Master Clock Speed Mode

LL_I2C_CLOCK_SPEED_STANDARD_MODE

Master clock speed range is standard mode

LL_I2C_CLOCK_SPEED_FAST_MODE

Master clock speed range is fast mode

Read Write Direction

LL_I2C_DIRECTION_WRITE

Bus is in write transfer

LL_I2C_DIRECTION_READ

Bus is in read transfer

Fast Mode Duty Cycle

LL_I2C_DUTYCYCLE_2

I2C fast mode Tlow/Thigh = 2

LL_I2C_DUTYCYCLE_16_9

I2C fast mode Tlow/Thigh = 16/9

Get Flags Defines

LL_I2C_SR1_SB

Start Bit (master mode)

LL_I2C_SR1_ADDR

Address sent (master mode) or Address matched flag (slave mode)

LL_I2C_SR1_BTF

Byte Transfer Finished flag

LL_I2C_SR1_ADD10

10-bit header sent (master mode)

LL_I2C_SR1_STOPF

Stop detection flag (slave mode)

LL_I2C_SR1_RXNE

Data register not empty (receivers)

LL_I2C_SR1_TXE

Data register empty (transmitters)

LL_I2C_SR1_BERR

Bus error

LL_I2C_SR1_ARLO

Arbitration lost

LL_I2C_SR1_AF

Acknowledge failure flag

LL_I2C_SR1_OVR

Overrun/Underrun

LL_I2C_SR1_PECERR

PEC Error in reception (SMBus mode)

LL_I2C_SR1_TIMEOUT

Timeout detection flag (SMBus mode)

LL_I2C_SR1_SMALERT

SMBus alert (SMBus mode)

LL_I2C_SR2_MSL

Master/Slave flag

LL_I2C_SR2_BUSY

Bus busy flag

LL_I2C_SR2_TRA

Transmitter/receiver direction

LL_I2C_SR2_GENCALL

General call address (Slave mode)

LL_I2C_SR2_SMBDEFAULT

SMBus Device default address (Slave mode)

LL_I2C_SR2_SMBHOST

SMBus Host address (Slave mode)

LL_I2C_SR2_DUALF

Dual flag (Slave mode)

Acknowledge Generation

LL_I2C_ACK

ACK is sent after current received byte.

LL_I2C_NACK

NACK is sent after current received byte.

IT Defines

LL_I2C_CR2_ITEVTEN

Events interrupts enable

LL_I2C_CR2_ITBUFEN

Buffer interrupts enable

LL_I2C_CR2_ITERREN

Error interrupts enable

Own Address 1 Length

LL_I2C_OWNADDRESS1_7BIT

Own address 1 is a 7-bit address.

LL_I2C_OWNADDRESS1_10BIT

Own address 1 is a 10-bit address.

Peripheral Mode

LL_I2C_MODE_I2C

I2C Master or Slave mode

LL_I2C_MODE_SMBUS_HOST

SMBus Host address acknowledge

LL_I2C_MODE_SMBUS_DEVICE

SMBus Device default mode (Default address not acknowledge)

LL_I2C_MODE_SMBUS_DEVICE_ARP

SMBus Device Default address acknowledge

Exported_Macros_Helper

__LL_I2C_FREQ_HZ_TO_MHZ

Description:

- Convert Peripheral Clock Frequency in Mhz.

Parameters:

- `__PCLK__`: This parameter must be a value of peripheral clock (in Hz).

Return value:

- Value: of peripheral clock (in Mhz)

__LL_I2C_FREQ_MHZ_TO_HZ

Description:

- Convert Peripheral Clock Frequency in Hz.

Parameters:

- `__PCLK__`: This parameter must be a value of peripheral clock (in Mhz).

Return value:

- Value: of peripheral clock (in Hz)

__LL_I2C_RISE_TIME

Description:

- Compute I2C Clock rising time.

Parameters:

- `__FREQRANGE__`: This parameter must be a value of peripheral clock (in Mhz).
- `__SPEED__`: This parameter must be a value lower than 400kHz (in Hz).

Return value:

- Value: between Min_Data=0x02 and Max_Data=0x3F

__LL_I2C_SPEED_TO_CCR

Description:

- Compute Speed clock range to a Clock Control Register (I2C_CCR_CCR) value.

Parameters:

- __PCLK__: This parameter must be a value of peripheral clock (in Hz).
- __SPEED__: This parameter must be a value lower than 400kHz (in Hz).
- __DUTYCYCLE__: This parameter can be one of the following values:
 - LL_I2C_DUTYCYCLE_2
 - LL_I2C_DUTYCYCLE_16_9

Return value:

- Value: between Min_Data=0x004 and Max_Data=0xFFFF, except in FAST DUTY mode where Min_Data=0x001.

__LL_I2C_SPEED_STANDARD_TO_CCR

Description:

- Compute Speed Standard clock range to a Clock Control Register (I2C_CCR_CCR) value.

Parameters:

- __PCLK__: This parameter must be a value of peripheral clock (in Hz).
- __SPEED__: This parameter must be a value lower than 100kHz (in Hz).

Return value:

- Value: between Min_Data=0x004 and Max_Data=0xFFFF.

__LL_I2C_SPEED_FAST_TO_CCR

Description:

- Compute Speed Fast clock range to a Clock Control Register (I2C_CCR_CCR) value.

Parameters:

- __PCLK__: This parameter must be a value of peripheral clock (in Hz).
- __SPEED__: This parameter must be a value between Min_Data=100Khz and Max_Data=400Khz (in Hz).
- __DUTYCYCLE__: This parameter can be one of the following values:
 - LL_I2C_DUTYCYCLE_2
 - LL_I2C_DUTYCYCLE_16_9

Return value:

- Value: between Min_Data=0x001 and Max_Data=0xFFFF

__LL_I2C_10BIT_ADDRESS

Description:

- Get the Least significant bits of a 10-Bits address.

Parameters:

- __ADDRESS__: This parameter must be a value of a 10-Bits slave address.

Return value:

- Value: between Min_Data=0x00 and Max_Data=0xFF

__LL_I2C_10BIT_HEADER_WRITE

Description:

- Convert a 10-Bits address to a 10-Bits header with Write direction.

Parameters:

- `__ADDRESS__`: This parameter must be a value of a 10-Bits slave address.

Return value:

- Value: between `Min_Data=0xF0` and `Max_Data=0xF6`

__LL_I2C_10BIT_HEADER_READ

Description:

- Convert a 10-Bits address to a 10-Bits header with Read direction.

Parameters:

- `__ADDRESS__`: This parameter must be a value of a 10-Bits slave address.

Return value:

- Value: between `Min_Data=0xF1` and `Max_Data=0xF7`

Common Write and read registers Macros

LL_I2C_WriteReg

Description:

- Write a value in I2C register.

Parameters:

- `__INSTANCE__`: I2C Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_I2C_ReadReg

Description:

- Read a value in I2C register.

Parameters:

- `__INSTANCE__`: I2C Instance
- `__REG__`: Register to be read

Return value:

- Register: value

58 LL IWDG Generic Driver

58.1 IWDG Firmware driver API description

The following section lists the various functions of the IWDG library.

58.1.1 Detailed description of functions

`LL_IWDG_Enable`

Function name

`__STATIC_INLINE void LL_IWDG_Enable (IWDG_TypeDef * IWDGx)`

Function description

Start the Independent Watchdog.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **None:**

Notes

- Except if the hardware watchdog option is selected

Reference Manual to LL API cross reference:

- KR KEY LL_IWDG_Enable

`LL_IWDG_ReloadCounter`

Function name

`__STATIC_INLINE void LL_IWDG_ReloadCounter (IWDG_TypeDef * IWDGx)`

Function description

Reloads IWDG counter with value defined in the reload register.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- KR KEY LL_IWDG_ReloadCounter

`LL_IWDG_EnableWriteAccess`

Function name

`__STATIC_INLINE void LL_IWDG_EnableWriteAccess (IWDG_TypeDef * IWDGx)`

Function description

Enable write access to IWDG_PR, IWDG_RLR and IWDG_WINR registers.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- KR KEY LL_IWDG_EnableWriteAccess
LL_IWDG_DisableWriteAccess

Function name

__STATIC_INLINE void LL_IWDG_DisableWriteAccess (IWDG_TypeDef * IWDGx)

Function description

Disable write access to IWDG_PR, IWDG_RLR and IWDG_WINR registers.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- KR KEY LL_IWDG_DisableWriteAccess
LL_IWDG_SetPrescaler

Function name

__STATIC_INLINE void LL_IWDG_SetPrescaler (IWDG_TypeDef * IWDGx, uint32_t Prescaler)

Function description

Select the prescaler of the IWDG.

Parameters

- **IWDGx:** IWDG Instance
- **Prescaler:** This parameter can be one of the following values:
 - LL_IWDG_PRESCALER_4
 - LL_IWDG_PRESCALER_8
 - LL_IWDG_PRESCALER_16
 - LL_IWDG_PRESCALER_32
 - LL_IWDG_PRESCALER_64
 - LL_IWDG_PRESCALER_128
 - LL_IWDG_PRESCALER_256

Return values

- **None:**

Reference Manual to LL API cross reference:

- PR PR LL_IWDG_SetPrescaler
LL_IWDG_GetPrescaler

Function name

__STATIC_INLINE uint32_t LL_IWDG_GetPrescaler (IWDG_TypeDef * IWDGx)

Function description

Get the selected prescaler of the IWDG.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_IWDG_PRESCALER_4
 - LL_IWDG_PRESCALER_8
 - LL_IWDG_PRESCALER_16
 - LL_IWDG_PRESCALER_32
 - LL_IWDG_PRESCALER_64
 - LL_IWDG_PRESCALER_128
 - LL_IWDG_PRESCALER_256

Reference Manual to LL API cross reference:

- PR PR LL_IWDG_GetPrescaler

LL_IWDG_SetReloadCounter

Function name

__STATIC_INLINE void LL_IWDG_SetReloadCounter (IWDG_TypeDef * IWDGx, uint32_t Counter)

Function description

Specify the IWDG down-counter reload value.

Parameters

- **IWDGx:** IWDG Instance
- **Counter:** Value between Min_Data=0 and Max_Data=0x0FFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- RLR RL LL_IWDG_SetReloadCounter

LL_IWDG_GetReloadCounter

Function name

__STATIC_INLINE uint32_t LL_IWDG_GetReloadCounter (IWDG_TypeDef * IWDGx)

Function description

Get the specified IWDG down-counter reload value.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **Value:** between Min_Data=0 and Max_Data=0x0FFF

Reference Manual to LL API cross reference:

- RLR RL LL_IWDG_GetReloadCounter

LL_IWDG_IsActiveFlag_PVU

Function name

__STATIC_INLINE uint32_t LL_IWDG_IsActiveFlag_PVU (IWDG_TypeDef * IWDGx)

Function description

Check if flag Prescaler Value Update is set or not.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR PVU LL_IWDG_IsActiveFlag_PVU

LL_IWDG_IsActiveFlag_RVU

Function name

__STATIC_INLINE uint32_t LL_IWDG_IsActiveFlag_RVU (IWDG_TypeDef * IWDGx)

Function description

Check if flag Reload Value Update is set or not.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR RVU LL_IWDG_IsActiveFlag_RVU

LL_IWDG_IsReady

Function name

__STATIC_INLINE uint32_t LL_IWDG_IsReady (IWDG_TypeDef * IWDGx)

Function description

Check if all flags Prescaler, Reload & Window Value Update are reset or not.

Parameters

- **IWDGx:** IWDG Instance

Return values

- **State:** of bits (1 or 0).

Reference Manual to LL API cross reference:

- SR PVU LL_IWDG_IsReady
- SR RVU LL_IWDG_IsReady

58.2 IWDG Firmware driver defines

The following section lists the various define and macros of the module.

58.2.1 IWDG

IWDG

Get Flags Defines

LL_IWDG_SR_PVU

Watchdog prescaler value update

LL_IWDG_SR_RVU

Watchdog counter reload value update

Prescaler Divider

LL_IWDG_PRESCALER_4

Divider by 4

LL_IWDG_PRESCALER_8

Divider by 8

LL_IWDG_PRESCALER_16

Divider by 16

LL_IWDG_PRESCALER_32

Divider by 32

LL_IWDG_PRESCALER_64

Divider by 64

LL_IWDG_PRESCALER_128

Divider by 128

LL_IWDG_PRESCALER_256

Divider by 256

Common Write and read registers Macros

LL_IWDG_WriteReg

Description:

- Write a value in IWDG register.

Parameters:

- `__INSTANCE__`: IWDG Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_IWDG_ReadReg

Description:

- Read a value in IWDG register.

Parameters:

- `__INSTANCE__`: IWDG Instance
- `__REG__`: Register to be read

Return value:

- Register: value

59 LL PWR Generic Driver

59.1 PWR Firmware driver API description

The following section lists the various functions of the PWR library.

59.1.1 Detailed description of functions

`LL_PWR_EnableFlashPowerDown`

Function name

`__STATIC_INLINE void LL_PWR_EnableFlashPowerDown (void )`

Function description

Enable the Flash Power Down in Stop Mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR FPDS `LL_PWR_EnableFlashPowerDown`

`LL_PWR_DisableFlashPowerDown`

Function name

`__STATIC_INLINE void LL_PWR_DisableFlashPowerDown (void )`

Function description

Disable the Flash Power Down in Stop Mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR FPDS `LL_PWR_DisableFlashPowerDown`

`LL_PWR_IsEnabledFlashPowerDown`

Function name

`__STATIC_INLINE uint32_t LL_PWR_IsEnabledFlashPowerDown (void )`

Function description

Check if the Flash Power Down in Stop Mode is enabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR FPDS `LL_PWR_IsEnabledFlashPowerDown`

`LL_PWR_EnableBkUpAccess`

Function name

`__STATIC_INLINE void LL_PWR_EnableBkUpAccess (void )`

Function description

Enable access to the backup domain.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DBP LL_PWR_EnableBkUpAccess
LL_PWR_DisableBkUpAccess

Function name

__STATIC_INLINE void LL_PWR_DisableBkUpAccess (void)

Function description

Disable access to the backup domain.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR DBP LL_PWR_DisableBkUpAccess
LL_PWR_IsEnabledBkUpAccess

Function name

__STATIC_INLINE uint32_t LL_PWR_IsEnabledBkUpAccess (void)

Function description

Check if the backup domain is enabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR DBP LL_PWR_IsEnabledBkUpAccess
LL_PWR_EnableBkUpRegulator

Function name

__STATIC_INLINE void LL_PWR_EnableBkUpRegulator (void)

Function description

Enable the backup regulator.

Return values

- **None:**

Notes

- The BRE bit of the PWR_CSR register is protected against parasitic write access. The LL_PWR_EnableBkUpAccess() must be called before using this API.

Reference Manual to LL API cross reference:

- CSR BRE LL_PWR_EnableBkUpRegulator
LL_PWR_DisableBkUpRegulator

Function name

__STATIC_INLINE void LL_PWR_DisableBkUpRegulator (void)

Function description

Disable the backup Regulator.

Return values

- **None:**

Notes

- The BRE bit of the PWR_CSR register is protected against parasitic write access. The LL_PWR_EnableBkUpAccess() must be called before using this API.

Reference Manual to LL API cross reference:

- CSR BRE LL_PWR_DisableBkUpRegulator
LL_PWR_IsEnabledBkUpRegulator

Function name

__STATIC_INLINE uint32_t LL_PWR_IsEnabledBkUpRegulator (void)

Function description

Check if the backup Regulator is enabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR BRE LL_PWR_IsEnabledBkUpRegulator
LL_PWR_SetRegulModeDS

Function name

__STATIC_INLINE void LL_PWR_SetRegulModeDS (uint32_t RegulMode)

Function description

Set voltage regulator mode during deep sleep mode.

Parameters

- **RegulMode:** This parameter can be one of the following values:
 - LL_PWR_REGU_DSMODE_MAIN
 - LL_PWR_REGU_DSMODE_LOW_POWER

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR LPDS LL_PWR_SetRegulModeDS
LL_PWR_GetRegulModeDS

Function name

__STATIC_INLINE uint32_t LL_PWR_GetRegulModeDS (void)

Function description

Get voltage regulator mode during deep sleep mode.

Return values

- **Returned:** value can be one of the following values:
 - LL_PWR_REGU_DSMODE_MAIN
 - LL_PWR_REGU_DSMODE_LOW_POWER

Reference Manual to LL API cross reference:

- CR LPDS LL_PWR_GetRegulModeDS


```
LL_PWR_SetPowerMode
```

Function name

```
__STATIC_INLINE void LL_PWR_SetPowerMode (uint32_t PDMode)
```

Function description

Set power down mode when CPU enters deepsleep.

Parameters

- **PDMode:** This parameter can be one of the following values:
 - LL_PWR_MODE_STOP_MAINREGU
 - LL_PWR_MODE_STOP_LPREGU
 - LL_PWR_MODE_STANDBY

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PDDS LL_PWR_SetPowerMode
-
- CR LPDS LL_PWR_SetPowerMode

```
LL_PWR_GetPowerMode
```

Function name

```
__STATIC_INLINE uint32_t LL_PWR_GetPowerMode (void )
```

Function description

Get power down mode when CPU enters deepsleep.

Return values

- **Returned:** value can be one of the following values:
 - LL_PWR_MODE_STOP_MAINREGU
 - LL_PWR_MODE_STOP_LPREGU
 - LL_PWR_MODE_STANDBY

Reference Manual to LL API cross reference:

- CR PDDS LL_PWR_GetPowerMode
-
- CR LPDS LL_PWR_GetPowerMode

```
LL_PWR_SetPVDLevel
```

Function name

```
__STATIC_INLINE void LL_PWR_SetPVDLevel (uint32_t PVDLevel)
```

Function description

Configure the voltage threshold detected by the Power Voltage Detector.

Parameters

- **PVDLevel:** This parameter can be one of the following values:
 - LL_PWR_PVDLEVEL_0
 - LL_PWR_PVDLEVEL_1
 - LL_PWR_PVDLEVEL_2
 - LL_PWR_PVDLEVEL_3
 - LL_PWR_PVDLEVEL_4
 - LL_PWR_PVDLEVEL_5
 - LL_PWR_PVDLEVEL_6
 - LL_PWR_PVDLEVEL_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PLS LL_PWR_SetPVDLevel
LL_PWR_GetPVDLevel

Function name

__STATIC_INLINE uint32_t LL_PWR_GetPVDLevel (void)

Function description

Get the voltage threshold detection.

Return values

- **Returned:** value can be one of the following values:
 - LL_PWR_PVDLEVEL_0
 - LL_PWR_PVDLEVEL_1
 - LL_PWR_PVDLEVEL_2
 - LL_PWR_PVDLEVEL_3
 - LL_PWR_PVDLEVEL_4
 - LL_PWR_PVDLEVEL_5
 - LL_PWR_PVDLEVEL_6
 - LL_PWR_PVDLEVEL_7

Reference Manual to LL API cross reference:

- CR PLS LL_PWR_GetPVDLevel
LL_PWR_EnablePVD

Function name

__STATIC_INLINE void LL_PWR_EnablePVD (void)

Function description

Enable Power Voltage Detector.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PVDE LL_PWR_EnablePVD
LL_PWR_DisablePVD

Function name

```
__STATIC_INLINE void LL_PWR_DisablePVD (void )
```

Function description

Disable Power Voltage Detector.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PVDE LL_PWR_DisablePVD

LL_PWR_IsEnabledPVD

Function name

```
__STATIC_INLINE uint32_t LL_PWR_IsEnabledPVD (void )
```

Function description

Check if Power Voltage Detector is enabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR PVDE LL_PWR_IsEnabledPVD

LL_PWR_EnableWakeUpPin

Function name

```
__STATIC_INLINE void LL_PWR_EnableWakeUpPin (uint32_t WakeUpPin)
```

Function description

Enable the WakeUp PINx functionality.

Parameters

- **WakeUpPin:** This parameter can be one of the following values:
 - LL_PWR_WAKEUP_PIN1

Return values

- **None:**

Reference Manual to LL API cross reference:

- CSR EWUP LL_PWR_EnableWakeUpPin

LL_PWR_DisableWakeUpPin

Function name

```
__STATIC_INLINE void LL_PWR_DisableWakeUpPin (uint32_t WakeUpPin)
```

Function description

Disable the WakeUp PINx functionality.

Parameters

- **WakeUpPin:** This parameter can be one of the following values:
 - LL_PWR_WAKEUP_PIN1

Return values

- **None:**

Reference Manual to LL API cross reference:

- CSR EWUP LL_PWR_DisableWakeUpPin
LL_PWR_IsEnabledWakeUpPin

Function name

```
__STATIC_INLINE uint32_t LL_PWR_IsEnabledWakeUpPin (uint32_t WakeUpPin)
```

Function description

Check if the WakeUp PINx functionality is enabled.

Parameters

- **WakeUpPin:** This parameter can be one of the following values:
 - LL_PWR_WAKEUP_PIN1

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR EWUP LL_PWR_IsEnabledWakeUpPin
LL_PWR_IsActiveFlag_WU

Function name

```
__STATIC_INLINE uint32_t LL_PWR_IsActiveFlag_WU (void )
```

Function description

Get Wake-up Flag.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR WUF LL_PWR_IsActiveFlag_WU
LL_PWR_IsActiveFlag_SB

Function name

```
__STATIC_INLINE uint32_t LL_PWR_IsActiveFlag_SB (void )
```

Function description

Get Standby Flag.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR SBF LL_PWR_IsActiveFlag_SB
LL_PWR_IsActiveFlag_BRR

Function name

```
__STATIC_INLINE uint32_t LL_PWR_IsActiveFlag_BRR (void )
```

Function description

Get Backup regulator ready Flag.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR BRR LL_PWR_IsActiveFlag_BRR
LL_PWR_IsActiveFlag_PVDO

Function name

__STATIC_INLINE uint32_t LL_PWR_IsActiveFlag_PVDO (void)

Function description

Indicate whether VDD voltage is below the selected PVD threshold.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR PVDO LL_PWR_IsActiveFlag_PVDO
LL_PWR_ClearFlag_SB

Function name

__STATIC_INLINE void LL_PWR_ClearFlag_SB (void)

Function description

Clear Standby Flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CSBF LL_PWR_ClearFlag_SB
LL_PWR_ClearFlag_WU

Function name

__STATIC_INLINE void LL_PWR_ClearFlag_WU (void)

Function description

Clear Wake-up Flags.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CWUF LL_PWR_ClearFlag_WU
LL_PWR_DeInit

Function name

ErrorStatus LL_PWR_DeInit (void)

Function description

De-initialize the PWR registers to their default reset values.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: PWR registers are de-initialized
 - ERROR: not applicable

59.2 PWR Firmware driver defines

The following section lists the various define and macros of the module.

59.2.1 PWR

PWR

Clear Flags Defines

LL_PWR_CR_CSBF

Clear standby flag

LL_PWR_CR_CWUF

Clear wakeup flag

Get Flags Defines

LL_PWR_CSR_WUF

Wakeup flag

LL_PWR_CSR_SBF

Standby flag

LL_PWR_CSR_PVDO

Power voltage detector output flag

LL_PWR_CSR_EWUP1

Enable WKUP pin 1

Mode Power

LL_PWR_MODE_STOP_MAINREGU

Enter Stop mode when the CPU enters deepsleep

LL_PWR_MODE_STOP_LPREGU

Enter Stop mode (with low power regulator ON) when the CPU enters deepsleep

LL_PWR_MODE_STANDBY

Enter Standby mode when the CPU enters deepsleep

Power Voltage Detector Level

LL_PWR_PVDLEVEL_0

Voltage threshold detected by PVD 2.2 V

LL_PWR_PVDLEVEL_1

Voltage threshold detected by PVD 2.3 V

LL_PWR_PVDLEVEL_2

Voltage threshold detected by PVD 2.4 V

LL_PWR_PVDLEVEL_3

Voltage threshold detected by PVD 2.5 V

LL_PWR_PVDLEVEL_4

Voltage threshold detected by PVD 2.6 V

LL_PWR_PVDLEVEL_5

Voltage threshold detected by PVD 2.7 V

LL_PWR_PVDLEVEL_6

Voltage threshold detected by PVD 2.8 V

LL_PWR_PVDLEVEL_7

Voltage threshold detected by PVD 2.9 V

Regulator Mode In Deep Sleep Mode

LL_PWR_REGU_DSMODE_MAIN

Voltage regulator in main mode during deepsleep mode

LL_PWR_REGU_DSMODE_LOW_POWER

Voltage regulator in low-power mode during deepsleep mode

Wakeup Pins

LL_PWR_WAKEUP_PIN1

WKUP pin 1 : PA0

Common write and read registers Macros

LL_PWR_WriteReg

Description:

- Write a value in PWR register.

Parameters:

- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_PWR_ReadReg

Description:

- Read a value in PWR register.

Parameters:

- `__REG__`: Register to be read

Return value:

- Register: value

60 LL RCC Generic Driver

60.1 RCC Firmware driver registers structures

60.1.1 LL_RCC_ClocksTypeDef

LL_RCC_ClocksTypeDef is defined in the stm32f2xx_ll_rcc.h

Data Fields

- *uint32_t* **SYSCLK_Frequency**
- *uint32_t* **HCLK_Frequency**
- *uint32_t* **PCLK1_Frequency**
- *uint32_t* **PCLK2_Frequency**

Field Documentation

- *uint32_t* **LL_RCC_ClocksTypeDef::SYSCLK_Frequency**
SYSCLK clock frequency
- *uint32_t* **LL_RCC_ClocksTypeDef::HCLK_Frequency**
HCLK clock frequency
- *uint32_t* **LL_RCC_ClocksTypeDef::PCLK1_Frequency**
PCLK1 clock frequency
- *uint32_t* **LL_RCC_ClocksTypeDef::PCLK2_Frequency**
PCLK2 clock frequency

60.2 RCC Firmware driver API description

The following section lists the various functions of the RCC library.

60.2.1 Detailed description of functions

LL_RCC_HSE_EnableCSS

Function name

__STATIC_INLINE void LL_RCC_HSE_EnableCSS (void)

Function description

Enable the Clock Security System.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR CSSON **LL_RCC_HSE_EnableCSS**

LL_RCC_HSE_EnableBypass

Function name

__STATIC_INLINE void LL_RCC_HSE_EnableBypass (void)

Function description

Enable HSE external oscillator (HSE Bypass)

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSEBYP LL_RCC_HSE_EnableBypass
LL_RCC_HSE_DisableBypass

Function name

__STATIC_INLINE void LL_RCC_HSE_DisableBypass (void)

Function description

Disable HSE external oscillator (HSE Bypass)

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSEBYP LL_RCC_HSE_DisableBypass
LL_RCC_HSE_Enable

Function name

__STATIC_INLINE void LL_RCC_HSE_Enable (void)

Function description

Enable HSE crystal oscillator (HSE ON)

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSEON LL_RCC_HSE_Enable
LL_RCC_HSE_Disable

Function name

__STATIC_INLINE void LL_RCC_HSE_Disable (void)

Function description

Disable HSE crystal oscillator (HSE ON)

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSEON LL_RCC_HSE_Disable
LL_RCC_HSE_IsReady

Function name

__STATIC_INLINE uint32_t LL_RCC_HSE_IsReady (void)

Function description

Check if HSE oscillator Ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR HSERDY LL_RCC_HSE_IsReady

```
LL_RCC_HSI_Enable
```

Function name

```
__STATIC_INLINE void LL_RCC_HSI_Enable (void )
```

Function description

Enable HSI oscillator.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSION LL_RCC_HSI_Enable

```
LL_RCC_HSI_Disable
```

Function name

```
__STATIC_INLINE void LL_RCC_HSI_Disable (void )
```

Function description

Disable HSI oscillator.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR HSION LL_RCC_HSI_Disable

```
LL_RCC_HSI_IsReady
```

Function name

```
__STATIC_INLINE uint32_t LL_RCC_HSI_IsReady (void )
```

Function description

Check if HSI clock is ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR HSIRDY LL_RCC_HSI_IsReady

```
LL_RCC_HSI_GetCalibration
```

Function name

```
__STATIC_INLINE uint32_t LL_RCC_HSI_GetCalibration (void )
```

Function description

Get HSI Calibration value.

Return values

- **Between:** Min_Data = 0x00 and Max_Data = 0xFF

Notes

- When HSITRIM is written, HSICAL is updated with the sum of HSITRIM and the factory trim value

Reference Manual to LL API cross reference:

- CR HSICAL LL_RCC_HSI_GetCalibration

LL_RCC_HSI_SetCalibTrimming

Function name

__STATIC_INLINE void LL_RCC_HSI_SetCalibTrimming (uint32_t Value)

Function description

Set HSI Calibration trimming.

Parameters

- **Value:** Between Min_Data = 0 and Max_Data = 31

Return values

- **None:**

Notes

- user-programmable trimming value that is added to the HSICAL
- Default value is 16, which, when added to the HSICAL value, should trim the HSI to 16 MHz +/- 1 %

Reference Manual to LL API cross reference:

- CR HSITRIM LL_RCC_HSI_SetCalibTrimming

LL_RCC_HSI_GetCalibTrimming

Function name

__STATIC_INLINE uint32_t LL_RCC_HSI_GetCalibTrimming (void)

Function description

Get HSI Calibration trimming.

Return values

- **Between:** Min_Data = 0 and Max_Data = 31

Reference Manual to LL API cross reference:

- CR HSITRIM LL_RCC_HSI_GetCalibTrimming

LL_RCC_LSE_Enable

Function name

__STATIC_INLINE void LL_RCC_LSE_Enable (void)

Function description

Enable Low Speed External (LSE) crystal.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR LSEON LL_RCC_LSE_Enable

LL_RCC_LSE_Disable

Function name

__STATIC_INLINE void LL_RCC_LSE_Disable (void)

Function description

Disable Low Speed External (LSE) crystal.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR LSEON LL_RCC_LSE_Disable
LL_RCC_LSE_EnableBypass

Function name

__STATIC_INLINE void LL_RCC_LSE_EnableBypass (void)

Function description

Enable external clock source (LSE bypass).

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR LSEBYP LL_RCC_LSE_EnableBypass
LL_RCC_LSE_DisableBypass

Function name

__STATIC_INLINE void LL_RCC_LSE_DisableBypass (void)

Function description

Disable external clock source (LSE bypass).

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR LSEBYP LL_RCC_LSE_DisableBypass
LL_RCC_LSE_IsReady

Function name

__STATIC_INLINE uint32_t LL_RCC_LSE_IsReady (void)

Function description

Check if LSE oscillator Ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- BDCR LSEBYP LL_RCC_LSE_IsReady
LL_RCC_LSI_Enable

Function name

__STATIC_INLINE void LL_RCC_LSI_Enable (void)

Function description

Enable LSI Oscillator.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CSR LSION LL_RCC_LSI_Enable

LL_RCC_LSI_Disable

Function name

__STATIC_INLINE void LL_RCC_LSI_Disable (void)

Function description

Disable LSI Oscillator.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CSR LSION LL_RCC_LSI_Disable

LL_RCC_LSI_IsReady

Function name

__STATIC_INLINE uint32_t LL_RCC_LSI_IsReady (void)

Function description

Check if LSI is Ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR LSIRDY LL_RCC_LSI_IsReady

LL_RCC_SetSysClkSource

Function name

__STATIC_INLINE void LL_RCC_SetSysClkSource (uint32_t Source)

Function description

Configure the system clock source.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_SYS_CLKSOURCE_HSI
 - LL_RCC_SYS_CLKSOURCE_HSE
 - LL_RCC_SYS_CLKSOURCE_PLL

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR SW LL_RCC_SetSysClkSource

LL_RCC_GetSysClkSource

Function name

__STATIC_INLINE uint32_t LL_RCC_GetSysClkSource (void)

Function description

Get the system clock source.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_SYS_CLKSOURCE_STATUS_HSI
 - LL_RCC_SYS_CLKSOURCE_STATUS_HSE
 - LL_RCC_SYS_CLKSOURCE_STATUS_PLL

Reference Manual to LL API cross reference:

- CFGR SWS LL_RCC_GetSysClkSource

LL_RCC_SetAHBPrescaler

Function name

__STATIC_INLINE void LL_RCC_SetAHBPrescaler (uint32_t Prescaler)

Function description

Set AHB prescaler.

Parameters

- **Prescaler:** This parameter can be one of the following values:
 - LL_RCC_SYSCLK_DIV_1
 - LL_RCC_SYSCLK_DIV_2
 - LL_RCC_SYSCLK_DIV_4
 - LL_RCC_SYSCLK_DIV_8
 - LL_RCC_SYSCLK_DIV_16
 - LL_RCC_SYSCLK_DIV_64
 - LL_RCC_SYSCLK_DIV_128
 - LL_RCC_SYSCLK_DIV_256
 - LL_RCC_SYSCLK_DIV_512

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR HPRE LL_RCC_SetAHBPrescaler

LL_RCC_SetAPB1Prescaler

Function name

__STATIC_INLINE void LL_RCC_SetAPB1Prescaler (uint32_t Prescaler)

Function description

Set APB1 prescaler.

Parameters

- **Prescaler:** This parameter can be one of the following values:
 - LL_RCC_APB1_DIV_1
 - LL_RCC_APB1_DIV_2
 - LL_RCC_APB1_DIV_4
 - LL_RCC_APB1_DIV_8
 - LL_RCC_APB1_DIV_16

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR PPRE1 LL_RCC_SetAPB1Prescaler
- LL_RCC_SetAPB2Prescaler

Function name

__STATIC_INLINE void LL_RCC_SetAPB2Prescaler (uint32_t Prescaler)

Function description

Set APB2 prescaler.

Parameters

- **Prescaler:** This parameter can be one of the following values:
 - LL_RCC_APB2_DIV_1
 - LL_RCC_APB2_DIV_2
 - LL_RCC_APB2_DIV_4
 - LL_RCC_APB2_DIV_8
 - LL_RCC_APB2_DIV_16

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR PPRE2 LL_RCC_SetAPB2Prescaler
- LL_RCC_GetAHBPrescaler

Function name

__STATIC_INLINE uint32_t LL_RCC_GetAHBPrescaler (void)

Function description

Get AHB prescaler.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_SYSCLK_DIV_1
 - LL_RCC_SYSCLK_DIV_2
 - LL_RCC_SYSCLK_DIV_4
 - LL_RCC_SYSCLK_DIV_8
 - LL_RCC_SYSCLK_DIV_16
 - LL_RCC_SYSCLK_DIV_64
 - LL_RCC_SYSCLK_DIV_128
 - LL_RCC_SYSCLK_DIV_256
 - LL_RCC_SYSCLK_DIV_512

Reference Manual to LL API cross reference:

- CFGR HPRE LL_RCC_GetAHBPrescaler
- LL_RCC_GetAPB1Prescaler

Function name

__STATIC_INLINE uint32_t LL_RCC_GetAPB1Prescaler (void)

Function description

Get APB1 prescaler.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_APB1_DIV_1
 - LL_RCC_APB1_DIV_2
 - LL_RCC_APB1_DIV_4
 - LL_RCC_APB1_DIV_8
 - LL_RCC_APB1_DIV_16

Reference Manual to LL API cross reference:

- CFGR PPRE1 LL_RCC_GetAPB1Prescaler

LL_RCC_GetAPB2Prescaler

Function name

__STATIC_INLINE uint32_t LL_RCC_GetAPB2Prescaler (void)

Function description

Get APB2 prescaler.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_APB2_DIV_1
 - LL_RCC_APB2_DIV_2
 - LL_RCC_APB2_DIV_4
 - LL_RCC_APB2_DIV_8
 - LL_RCC_APB2_DIV_16

Reference Manual to LL API cross reference:

- CFGR PPRE2 LL_RCC_GetAPB2Prescaler

LL_RCC_ConfigMCO

Function name

__STATIC_INLINE void LL_RCC_ConfigMCO (uint32_t MCOxSource, uint32_t MCOxPrescaler)

Function description

Configure MCOx.

Parameters

- **MCOxSource:** This parameter can be one of the following values:
 - LL_RCC_MCO1SOURCE_HSI
 - LL_RCC_MCO1SOURCE_LSE
 - LL_RCC_MCO1SOURCE_HSE
 - LL_RCC_MCO1SOURCE_PLLCLK
 - LL_RCC_MCO2SOURCE_SYSCCLK
 - LL_RCC_MCO2SOURCE_PLLI2S
 - LL_RCC_MCO2SOURCE_HSE
 - LL_RCC_MCO2SOURCE_PLLCLK
- **MCOxPrescaler:** This parameter can be one of the following values:
 - LL_RCC_MCO1_DIV_1
 - LL_RCC_MCO1_DIV_2
 - LL_RCC_MCO1_DIV_3
 - LL_RCC_MCO1_DIV_4
 - LL_RCC_MCO1_DIV_5
 - LL_RCC_MCO2_DIV_1
 - LL_RCC_MCO2_DIV_2
 - LL_RCC_MCO2_DIV_3
 - LL_RCC_MCO2_DIV_4
 - LL_RCC_MCO2_DIV_5

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR MCO1 LL_RCC_ConfigMCO
- CFGR MCO1PRE LL_RCC_ConfigMCO
- CFGR MCO2 LL_RCC_ConfigMCO
- CFGR MCO2PRE LL_RCC_ConfigMCO

LL_RCC_SetI2SClockSource

Function name

__STATIC_INLINE void LL_RCC_SetI2SClockSource (uint32_t Source)

Function description

Configure I2S clock source.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_I2S1_CLKSOURCE_PLLI2S
 - LL_RCC_I2S1_CLKSOURCE_PIN

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR I2SSRC LL_RCC_SetI2SClockSource

LL_RCC_GetI2SClockSource

Function name

__STATIC_INLINE uint32_t LL_RCC_GetI2SClockSource (uint32_t I2Sx)

Function description

Get I2S Clock Source.

Parameters

- **I2Sx:** This parameter can be one of the following values:
 - LL_RCC_I2S1_CLKSOURCE

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_I2S1_CLKSOURCE_PLLI2S
 - LL_RCC_I2S1_CLKSOURCE_PIN

Reference Manual to LL API cross reference:

- CFGR I2SSRC LL_RCC_GetI2SClockSource
LL_RCC_SetRTCClockSource

Function name

__STATIC_INLINE void LL_RCC_SetRTCClockSource (uint32_t Source)

Function description

Set RTC Clock Source.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_RTC_CLKSOURCE_NONE
 - LL_RCC_RTC_CLKSOURCE_LSE
 - LL_RCC_RTC_CLKSOURCE_LSI
 - LL_RCC_RTC_CLKSOURCE_HSE

Return values

- **None:**

Notes

- Once the RTC clock source has been selected, it cannot be changed anymore unless the Backup domain is reset, or unless a failure is detected on LSE (LSECSSD is set). The BDRST bit can be used to reset them.

Reference Manual to LL API cross reference:

- BDCR RTCSEL LL_RCC_SetRTCClockSource
LL_RCC_GetRTCClockSource

Function name

__STATIC_INLINE uint32_t LL_RCC_GetRTCClockSource (void)

Function description

Get RTC Clock Source.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_RTC_CLKSOURCE_NONE
 - LL_RCC_RTC_CLKSOURCE_LSE
 - LL_RCC_RTC_CLKSOURCE_LSI
 - LL_RCC_RTC_CLKSOURCE_HSE

Reference Manual to LL API cross reference:

- BDCR RTCSEL LL_RCC_GetRTCClockSource
LL_RCC_EnableRTC

Function name

__STATIC_INLINE void LL_RCC_EnableRTC (void)

Function description

Enable RTC.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR RTCEN LL_RCC_EnableRTC
LL_RCC_DisableRTC

Function name

__STATIC_INLINE void LL_RCC_DisableRTC (void)

Function description

Disable RTC.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR RTCEN LL_RCC_DisableRTC
LL_RCC_IsEnabledRTC

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledRTC (void)

Function description

Check if RTC has been enabled or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- BDCR RTCEN LL_RCC_IsEnabledRTC
LL_RCC_ForceBackupDomainReset

Function name

__STATIC_INLINE void LL_RCC_ForceBackupDomainReset (void)

Function description

Force the Backup domain reset.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR BDRST LL_RCC_ForceBackupDomainReset

LL_RCC_ReleaseBackupDomainReset

Function name

__STATIC_INLINE void LL_RCC_ReleaseBackupDomainReset (void)

Function description

Release the Backup domain reset.

Return values

- **None:**

Reference Manual to LL API cross reference:

- BDCR BDRST LL_RCC_ReleaseBackupDomainReset

LL_RCC_SetRTC_HSEPrescaler

Function name

__STATIC_INLINE void LL_RCC_SetRTC_HSEPrescaler (uint32_t Prescaler)

Function description

Set HSE Prescalers for RTC Clock.

Parameters

- **Prescaler:** This parameter can be one of the following values:
 - LL_RCC_RTC_NOCLOCK
 - LL_RCC_RTC_HSE_DIV_2
 - LL_RCC_RTC_HSE_DIV_3
 - LL_RCC_RTC_HSE_DIV_4
 - LL_RCC_RTC_HSE_DIV_5
 - LL_RCC_RTC_HSE_DIV_6
 - LL_RCC_RTC_HSE_DIV_7
 - LL_RCC_RTC_HSE_DIV_8
 - LL_RCC_RTC_HSE_DIV_9
 - LL_RCC_RTC_HSE_DIV_10
 - LL_RCC_RTC_HSE_DIV_11
 - LL_RCC_RTC_HSE_DIV_12
 - LL_RCC_RTC_HSE_DIV_13
 - LL_RCC_RTC_HSE_DIV_14
 - LL_RCC_RTC_HSE_DIV_15
 - LL_RCC_RTC_HSE_DIV_16
 - LL_RCC_RTC_HSE_DIV_17
 - LL_RCC_RTC_HSE_DIV_18
 - LL_RCC_RTC_HSE_DIV_19
 - LL_RCC_RTC_HSE_DIV_20
 - LL_RCC_RTC_HSE_DIV_21
 - LL_RCC_RTC_HSE_DIV_22
 - LL_RCC_RTC_HSE_DIV_23
 - LL_RCC_RTC_HSE_DIV_24
 - LL_RCC_RTC_HSE_DIV_25
 - LL_RCC_RTC_HSE_DIV_26
 - LL_RCC_RTC_HSE_DIV_27
 - LL_RCC_RTC_HSE_DIV_28
 - LL_RCC_RTC_HSE_DIV_29
 - LL_RCC_RTC_HSE_DIV_30
 - LL_RCC_RTC_HSE_DIV_31

Return values

- **None:**

Reference Manual to LL API cross reference:

- CFGR RTCPRE LL_RCC_SetRTC_HSEPrescaler

LL_RCC_GetRTC_HSEPrescaler

Function name

__STATIC_INLINE uint32_t LL_RCC_GetRTC_HSEPrescaler (void)

Function description

Get HSE Prescalers for RTC Clock.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_RTC_NOCLOCK
 - LL_RCC_RTC_HSE_DIV_2
 - LL_RCC_RTC_HSE_DIV_3
 - LL_RCC_RTC_HSE_DIV_4
 - LL_RCC_RTC_HSE_DIV_5
 - LL_RCC_RTC_HSE_DIV_6
 - LL_RCC_RTC_HSE_DIV_7
 - LL_RCC_RTC_HSE_DIV_8
 - LL_RCC_RTC_HSE_DIV_9
 - LL_RCC_RTC_HSE_DIV_10
 - LL_RCC_RTC_HSE_DIV_11
 - LL_RCC_RTC_HSE_DIV_12
 - LL_RCC_RTC_HSE_DIV_13
 - LL_RCC_RTC_HSE_DIV_14
 - LL_RCC_RTC_HSE_DIV_15
 - LL_RCC_RTC_HSE_DIV_16
 - LL_RCC_RTC_HSE_DIV_17
 - LL_RCC_RTC_HSE_DIV_18
 - LL_RCC_RTC_HSE_DIV_19
 - LL_RCC_RTC_HSE_DIV_20
 - LL_RCC_RTC_HSE_DIV_21
 - LL_RCC_RTC_HSE_DIV_22
 - LL_RCC_RTC_HSE_DIV_23
 - LL_RCC_RTC_HSE_DIV_24
 - LL_RCC_RTC_HSE_DIV_25
 - LL_RCC_RTC_HSE_DIV_26
 - LL_RCC_RTC_HSE_DIV_27
 - LL_RCC_RTC_HSE_DIV_28
 - LL_RCC_RTC_HSE_DIV_29
 - LL_RCC_RTC_HSE_DIV_30
 - LL_RCC_RTC_HSE_DIV_31

Reference Manual to LL API cross reference:

- CFGR RTCPRE LL_RCC_GetRTC_HSEPrescaler
- LL_RCC_PLL_Enable

Function name

__STATIC_INLINE void LL_RCC_PLL_Enable (void)

Function description

Enable PLL.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR PLLON LL_RCC_PLL_Enable
- LL_RCC_PLL_Disable

Function name

```
__STATIC_INLINE void LL_RCC_PLL_Disable (void )
```

Function description

Disable PLL.

Return values

- **None:**

Notes

- Cannot be disabled if the PLL clock is used as the system clock

Reference Manual to LL API cross reference:

- CR PLLON LL_RCC_PLL_Disable

LL_RCC_PLL_IsReady

Function name

```
__STATIC_INLINE uint32_t LL_RCC_PLL_IsReady (void )
```

Function description

Check if PLL Ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR PLLRDY LL_RCC_PLL_IsReady

LL_RCC_PLL_ConfigDomain_SYS

Function name

```
__STATIC_INLINE void LL_RCC_PLL_ConfigDomain_SYS (uint32_t Source, uint32_t PLLM, uint32_t PLLN, uint32_t PLLP)
```

Function description

Configure PLL used for SYSCLK Domain.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_PLLSOURCE_HSI
 - LL_RCC_PLLSOURCE_HSE
- **PLLM:** This parameter can be one of the following values:
 - LL_RCC_PLLM_DIV_2
 - LL_RCC_PLLM_DIV_3
 - LL_RCC_PLLM_DIV_4
 - LL_RCC_PLLM_DIV_5
 - LL_RCC_PLLM_DIV_6
 - LL_RCC_PLLM_DIV_7
 - LL_RCC_PLLM_DIV_8
 - LL_RCC_PLLM_DIV_9
 - LL_RCC_PLLM_DIV_10
 - LL_RCC_PLLM_DIV_11
 - LL_RCC_PLLM_DIV_12
 - LL_RCC_PLLM_DIV_13
 - LL_RCC_PLLM_DIV_14
 - LL_RCC_PLLM_DIV_15
 - LL_RCC_PLLM_DIV_16
 - LL_RCC_PLLM_DIV_17
 - LL_RCC_PLLM_DIV_18
 - LL_RCC_PLLM_DIV_19
 - LL_RCC_PLLM_DIV_20
 - LL_RCC_PLLM_DIV_21
 - LL_RCC_PLLM_DIV_22
 - LL_RCC_PLLM_DIV_23
 - LL_RCC_PLLM_DIV_24
 - LL_RCC_PLLM_DIV_25
 - LL_RCC_PLLM_DIV_26
 - LL_RCC_PLLM_DIV_27
 - LL_RCC_PLLM_DIV_28
 - LL_RCC_PLLM_DIV_29
 - LL_RCC_PLLM_DIV_30
 - LL_RCC_PLLM_DIV_31
 - LL_RCC_PLLM_DIV_32
 - LL_RCC_PLLM_DIV_33
 - LL_RCC_PLLM_DIV_34
 - LL_RCC_PLLM_DIV_35
 - LL_RCC_PLLM_DIV_36
 - LL_RCC_PLLM_DIV_37
 - LL_RCC_PLLM_DIV_38
 - LL_RCC_PLLM_DIV_39
 - LL_RCC_PLLM_DIV_40
 - LL_RCC_PLLM_DIV_41
 - LL_RCC_PLLM_DIV_42
 - LL_RCC_PLLM_DIV_43
 - LL_RCC_PLLM_DIV_44
 - LL_RCC_PLLM_DIV_45

- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48
- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63
- **PLLN:** Between 192 and 432
- **PLLQ:** This parameter can be one of the following values:
 - LL_RCC_PLLQ_DIV_2
 - LL_RCC_PLLQ_DIV_4
 - LL_RCC_PLLQ_DIV_6
 - LL_RCC_PLLQ_DIV_8

Return values

- **None:**

Notes

- PLL Source and PLLM Divider can be written only when PLL, PLLI2S are disabled
- PLLN/PLLQ can be written only when PLL is disabled

Reference Manual to LL API cross reference:

- PLLCFGR PLLSRC LL_RCC_PLL_ConfigDomain_SYS
- PLLCFGR PLLM LL_RCC_PLL_ConfigDomain_SYS
- PLLCFGR PLLN LL_RCC_PLL_ConfigDomain_SYS
- PLLCFGR PLLQ LL_RCC_PLL_ConfigDomain_SYS

LL_RCC_PLL_ConfigDomain_48M

Function name

__STATIC_INLINE void LL_RCC_PLL_ConfigDomain_48M (uint32_t Source, uint32_t PLLM, uint32_t PLLN, uint32_t PLLQ)

Function description

Configure PLL used for 48Mhz domain clock.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_PLLSOURCE_HSI
 - LL_RCC_PLLSOURCE_HSE
- **PLLM:** This parameter can be one of the following values:
 - LL_RCC_PLLM_DIV_2
 - LL_RCC_PLLM_DIV_3
 - LL_RCC_PLLM_DIV_4
 - LL_RCC_PLLM_DIV_5
 - LL_RCC_PLLM_DIV_6
 - LL_RCC_PLLM_DIV_7
 - LL_RCC_PLLM_DIV_8
 - LL_RCC_PLLM_DIV_9
 - LL_RCC_PLLM_DIV_10
 - LL_RCC_PLLM_DIV_11
 - LL_RCC_PLLM_DIV_12
 - LL_RCC_PLLM_DIV_13
 - LL_RCC_PLLM_DIV_14
 - LL_RCC_PLLM_DIV_15
 - LL_RCC_PLLM_DIV_16
 - LL_RCC_PLLM_DIV_17
 - LL_RCC_PLLM_DIV_18
 - LL_RCC_PLLM_DIV_19
 - LL_RCC_PLLM_DIV_20
 - LL_RCC_PLLM_DIV_21
 - LL_RCC_PLLM_DIV_22
 - LL_RCC_PLLM_DIV_23
 - LL_RCC_PLLM_DIV_24
 - LL_RCC_PLLM_DIV_25
 - LL_RCC_PLLM_DIV_26
 - LL_RCC_PLLM_DIV_27
 - LL_RCC_PLLM_DIV_28
 - LL_RCC_PLLM_DIV_29
 - LL_RCC_PLLM_DIV_30
 - LL_RCC_PLLM_DIV_31
 - LL_RCC_PLLM_DIV_32
 - LL_RCC_PLLM_DIV_33
 - LL_RCC_PLLM_DIV_34
 - LL_RCC_PLLM_DIV_35
 - LL_RCC_PLLM_DIV_36
 - LL_RCC_PLLM_DIV_37
 - LL_RCC_PLLM_DIV_38
 - LL_RCC_PLLM_DIV_39
 - LL_RCC_PLLM_DIV_40
 - LL_RCC_PLLM_DIV_41
 - LL_RCC_PLLM_DIV_42
 - LL_RCC_PLLM_DIV_43
 - LL_RCC_PLLM_DIV_44
 - LL_RCC_PLLM_DIV_45

- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48
- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63
- **PLLN:** Between 192 and 432
- **PLLQ:** This parameter can be one of the following values:
 - LL_RCC_PLLQ_DIV_2
 - LL_RCC_PLLQ_DIV_3
 - LL_RCC_PLLQ_DIV_4
 - LL_RCC_PLLQ_DIV_5
 - LL_RCC_PLLQ_DIV_6
 - LL_RCC_PLLQ_DIV_7
 - LL_RCC_PLLQ_DIV_8
 - LL_RCC_PLLQ_DIV_9
 - LL_RCC_PLLQ_DIV_10
 - LL_RCC_PLLQ_DIV_11
 - LL_RCC_PLLQ_DIV_12
 - LL_RCC_PLLQ_DIV_13
 - LL_RCC_PLLQ_DIV_14
 - LL_RCC_PLLQ_DIV_15

Return values

- **None:**

Notes

- PLL Source and PLLM Divider can be written only when PLL, PLLI2S are disabled
- PLLN/PLLQ can be written only when PLL is disabled
- This can be selected for USB, RNG, SDIO

Reference Manual to LL API cross reference:

- PLLCFGR PLLSRC LL_RCC_PLL_ConfigDomain_48M
- PLLCFGR PLLM LL_RCC_PLL_ConfigDomain_48M
- PLLCFGR PLLN LL_RCC_PLL_ConfigDomain_48M
- PLLCFGR PLLQ LL_RCC_PLL_ConfigDomain_48M

LL_RCC_PLL_GetN

Function name

`__STATIC_INLINE uint32_t LL_RCC_PLL_GetN (void )`

Function description

Get Main PLL multiplication factor for VCO.

Return values

- **Between:** 192 and 432

Reference Manual to LL API cross reference:

- PLLCFGR PLLN LL_RCC_PLL_GetN

LL_RCC_PLL_GetP

Function name

`__STATIC_INLINE uint32_t LL_RCC_PLL_GetP (void )`

Function description

Get Main PLL division factor for PLLP.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_PLLP_DIV_2
 - LL_RCC_PLLP_DIV_4
 - LL_RCC_PLLP_DIV_6
 - LL_RCC_PLLP_DIV_8

Reference Manual to LL API cross reference:

- PLLCFGR PLLP LL_RCC_PLL_GetP

LL_RCC_PLL_GetQ

Function name

`__STATIC_INLINE uint32_t LL_RCC_PLL_GetQ (void )`

Function description

Get Main PLL division factor for PLLQ.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_PLLQ_DIV_2
 - LL_RCC_PLLQ_DIV_3
 - LL_RCC_PLLQ_DIV_4
 - LL_RCC_PLLQ_DIV_5
 - LL_RCC_PLLQ_DIV_6
 - LL_RCC_PLLQ_DIV_7
 - LL_RCC_PLLQ_DIV_8
 - LL_RCC_PLLQ_DIV_9
 - LL_RCC_PLLQ_DIV_10
 - LL_RCC_PLLQ_DIV_11
 - LL_RCC_PLLQ_DIV_12
 - LL_RCC_PLLQ_DIV_13
 - LL_RCC_PLLQ_DIV_14
 - LL_RCC_PLLQ_DIV_15

Notes

- used for PLL48MCLK selected for USB, RNG, SDIO (48 MHz clock)

Reference Manual to LL API cross reference:

- PLLCFGR PLLQ LL_RCC_PLL_GetQ
LL_RCC_PLL_GetMainSource

Function name

__STATIC_INLINE uint32_t LL_RCC_PLL_GetMainSource (void)

Function description

Get the oscillator used as PLL clock source.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_PLLSOURCE_HSI
 - LL_RCC_PLLSOURCE_HSE

Reference Manual to LL API cross reference:

- PLLCFGR PLLSRC LL_RCC_PLL_GetMainSource
LL_RCC_PLL_GetDivider

Function name

__STATIC_INLINE uint32_t LL_RCC_PLL_GetDivider (void)

Function description

Get Division factor for the main PLL and other PLL.

Return values

- **Returned:** value can be one of the following values:

- LL_RCC_PLLM_DIV_2
- LL_RCC_PLLM_DIV_3
- LL_RCC_PLLM_DIV_4
- LL_RCC_PLLM_DIV_5
- LL_RCC_PLLM_DIV_6
- LL_RCC_PLLM_DIV_7
- LL_RCC_PLLM_DIV_8
- LL_RCC_PLLM_DIV_9
- LL_RCC_PLLM_DIV_10
- LL_RCC_PLLM_DIV_11
- LL_RCC_PLLM_DIV_12
- LL_RCC_PLLM_DIV_13
- LL_RCC_PLLM_DIV_14
- LL_RCC_PLLM_DIV_15
- LL_RCC_PLLM_DIV_16
- LL_RCC_PLLM_DIV_17
- LL_RCC_PLLM_DIV_18
- LL_RCC_PLLM_DIV_19
- LL_RCC_PLLM_DIV_20
- LL_RCC_PLLM_DIV_21
- LL_RCC_PLLM_DIV_22
- LL_RCC_PLLM_DIV_23
- LL_RCC_PLLM_DIV_24
- LL_RCC_PLLM_DIV_25
- LL_RCC_PLLM_DIV_26
- LL_RCC_PLLM_DIV_27
- LL_RCC_PLLM_DIV_28
- LL_RCC_PLLM_DIV_29
- LL_RCC_PLLM_DIV_30
- LL_RCC_PLLM_DIV_31
- LL_RCC_PLLM_DIV_32
- LL_RCC_PLLM_DIV_33
- LL_RCC_PLLM_DIV_34
- LL_RCC_PLLM_DIV_35
- LL_RCC_PLLM_DIV_36
- LL_RCC_PLLM_DIV_37
- LL_RCC_PLLM_DIV_38
- LL_RCC_PLLM_DIV_39
- LL_RCC_PLLM_DIV_40
- LL_RCC_PLLM_DIV_41
- LL_RCC_PLLM_DIV_42
- LL_RCC_PLLM_DIV_43
- LL_RCC_PLLM_DIV_44
- LL_RCC_PLLM_DIV_45
- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48

- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63

Reference Manual to LL API cross reference:

- PLLCFGR PLLM LL_RCC_PLL_GetDivider
- LL_RCC_PLL_ConfigSpreadSpectrum

Function name

__STATIC_INLINE void LL_RCC_PLL_ConfigSpreadSpectrum (uint32_t Mod, uint32_t Inc, uint32_t Sel)

Function description

Configure Spread Spectrum used for PLL.

Parameters

- **Mod:** Between Min_Data=0 and Max_Data=8191
- **Inc:** Between Min_Data=0 and Max_Data=32767
- **Sel:** This parameter can be one of the following values:
 - LL_RCC_SPREAD_SELECT_CENTER
 - LL_RCC_SPREAD_SELECT_DOWN

Return values

- **None:**

Notes

- These bits must be written before enabling PLL

Reference Manual to LL API cross reference:

- SSCGR MODPER LL_RCC_PLL_ConfigSpreadSpectrum
- SSCGR INCSTEP LL_RCC_PLL_ConfigSpreadSpectrum
- SSCGR SPREADSEL LL_RCC_PLL_ConfigSpreadSpectrum
- LL_RCC_PLL_GetPeriodModulation

Function name

__STATIC_INLINE uint32_t LL_RCC_PLL_GetPeriodModulation (void)

Function description

Get Spread Spectrum Modulation Period for PLL.

Return values

- **Between:** Min_Data=0 and Max_Data=8191

Reference Manual to LL API cross reference:

- SSCGR MODPER LL_RCC_PLL_GetPeriodModulation
- LL_RCC_PLL_GetStepIncrementation

Function name

__STATIC_INLINE uint32_t LL_RCC_PLL_GetStepIncrementation (void)

Function description

Get Spread Spectrum Incrementation Step for PLL.

Return values

- Between:** Min_Data=0 and Max_Data=32767

Notes

- Must be written before enabling PLL

Reference Manual to LL API cross reference:

- SSCGR INCSTEP LL_RCC_PLL_GetStepIncrementation
- LL_RCC_PLL_GetSpreadSelection

Function name

__STATIC_INLINE uint32_t LL_RCC_PLL_GetSpreadSelection (void)

Function description

Get Spread Spectrum Selection for PLL.

Return values

- Returned:** value can be one of the following values:
 - LL_RCC_SPREAD_SELECT_CENTER
 - LL_RCC_SPREAD_SELECT_DOWN

Notes

- Must be written before enabling PLL

Reference Manual to LL API cross reference:

- SSCGR SPREADSEL LL_RCC_PLL_GetSpreadSelection
- LL_RCC_PLL_SpreadSpectrum_Enable

Function name

__STATIC_INLINE void LL_RCC_PLL_SpreadSpectrum_Enable (void)

Function description

Enable Spread Spectrum for PLL.

Return values

- None:**

Reference Manual to LL API cross reference:

- SSCGR SSCGEN LL_RCC_PLL_SpreadSpectrum_Enable
- LL_RCC_PLL_SpreadSpectrum_Disable

Function name

__STATIC_INLINE void LL_RCC_PLL_SpreadSpectrum_Disable (void)

Function description

Disable Spread Spectrum for PLL.

Return values

- **None:**

Reference Manual to LL API cross reference:

- SSCGR SSCGEN LL_RCC_PLL_SpreadSpectrum_Disable
LL_RCC_PLLI2S_Enable

Function name

__STATIC_INLINE void LL_RCC_PLLI2S_Enable (void)

Function description

Enable PLLI2S.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR_PLLI2SON LL_RCC_PLLI2S_Enable
LL_RCC_PLLI2S_Disable

Function name

__STATIC_INLINE void LL_RCC_PLLI2S_Disable (void)

Function description

Disable PLLI2S.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR_PLLI2SON LL_RCC_PLLI2S_Disable
LL_RCC_PLLI2S_IsReady

Function name

__STATIC_INLINE uint32_t LL_RCC_PLLI2S_IsReady (void)

Function description

Check if PLLI2S Ready.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR_PLLI2SRDY LL_RCC_PLLI2S_IsReady
LL_RCC_PLLI2S_ConfigDomain_I2S

Function name

__STATIC_INLINE void LL_RCC_PLLI2S_ConfigDomain_I2S (uint32_t Source, uint32_t PLLM, uint32_t PLLN, uint32_t PLLR)

Function description

Configure PLLI2S used for I2S1 domain clock.

Parameters

- **Source:** This parameter can be one of the following values:
 - LL_RCC_PLLSOURCE_HSI
 - LL_RCC_PLLSOURCE_HSE
- **PLLM:** This parameter can be one of the following values:
 - LL_RCC_PLLM_DIV_2
 - LL_RCC_PLLM_DIV_3
 - LL_RCC_PLLM_DIV_4
 - LL_RCC_PLLM_DIV_5
 - LL_RCC_PLLM_DIV_6
 - LL_RCC_PLLM_DIV_7
 - LL_RCC_PLLM_DIV_8
 - LL_RCC_PLLM_DIV_9
 - LL_RCC_PLLM_DIV_10
 - LL_RCC_PLLM_DIV_11
 - LL_RCC_PLLM_DIV_12
 - LL_RCC_PLLM_DIV_13
 - LL_RCC_PLLM_DIV_14
 - LL_RCC_PLLM_DIV_15
 - LL_RCC_PLLM_DIV_16
 - LL_RCC_PLLM_DIV_17
 - LL_RCC_PLLM_DIV_18
 - LL_RCC_PLLM_DIV_19
 - LL_RCC_PLLM_DIV_20
 - LL_RCC_PLLM_DIV_21
 - LL_RCC_PLLM_DIV_22
 - LL_RCC_PLLM_DIV_23
 - LL_RCC_PLLM_DIV_24
 - LL_RCC_PLLM_DIV_25
 - LL_RCC_PLLM_DIV_26
 - LL_RCC_PLLM_DIV_27
 - LL_RCC_PLLM_DIV_28
 - LL_RCC_PLLM_DIV_29
 - LL_RCC_PLLM_DIV_30
 - LL_RCC_PLLM_DIV_31
 - LL_RCC_PLLM_DIV_32
 - LL_RCC_PLLM_DIV_33
 - LL_RCC_PLLM_DIV_34
 - LL_RCC_PLLM_DIV_35
 - LL_RCC_PLLM_DIV_36
 - LL_RCC_PLLM_DIV_37
 - LL_RCC_PLLM_DIV_38
 - LL_RCC_PLLM_DIV_39
 - LL_RCC_PLLM_DIV_40
 - LL_RCC_PLLM_DIV_41
 - LL_RCC_PLLM_DIV_42
 - LL_RCC_PLLM_DIV_43
 - LL_RCC_PLLM_DIV_44
 - LL_RCC_PLLM_DIV_45

- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48
- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63
- **PLLN:** Between 192 and 432
- **PLLR:** This parameter can be one of the following values:
 - LL_RCC_PLLI2SR_DIV_2
 - LL_RCC_PLLI2SR_DIV_3
 - LL_RCC_PLLI2SR_DIV_4
 - LL_RCC_PLLI2SR_DIV_5
 - LL_RCC_PLLI2SR_DIV_6
 - LL_RCC_PLLI2SR_DIV_7

Return values

- **None:**

Notes

- PLL Source and PLLM Divider can be written only when PLL, PLLI2S are disabled
- PLLN/PLLR can be written only when PLLI2S is disabled
- This can be selected for I2S

Reference Manual to LL API cross reference:

- PLLCFGR PLLSRC LL_RCC_PLLI2S_ConfigDomain_I2S
- PLLCFGR PLLM LL_RCC_PLLI2S_ConfigDomain_I2S
- PLLI2SCFGR PLLI2SN LL_RCC_PLLI2S_ConfigDomain_I2S
- PLLI2SCFGR PLLI2SR LL_RCC_PLLI2S_ConfigDomain_I2S

LL_RCC_PLLI2S_GetN

Function name

__STATIC_INLINE uint32_t LL_RCC_PLLI2S_GetN (void)

Function description

Get I2SPLL multiplication factor for VCO.

Return values

- **Between:** 192 and 432

Reference Manual to LL API cross reference:

- PLLI2SCFGR PLLI2SN LL_RCC_PLLI2S_GetN

LL_RCC_PLLI2S_GetR

Function name

__STATIC_INLINE uint32_t LL_RCC_PLLI2S_GetR (void)

Function description

Get I2SPLL division factor for PLLI2SR.

Return values

- **Returned:** value can be one of the following values:
 - LL_RCC_PLLI2SR_DIV_2
 - LL_RCC_PLLI2SR_DIV_3
 - LL_RCC_PLLI2SR_DIV_4
 - LL_RCC_PLLI2SR_DIV_5
 - LL_RCC_PLLI2SR_DIV_6
 - LL_RCC_PLLI2SR_DIV_7

Notes

- used for PLLI2SCLK (I2S clock)

Reference Manual to LL API cross reference:

- PLLI2SCFGR PLLI2SR LL_RCC_PLLI2S_GetR

LL_RCC_ClearFlag_LSIRDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_LSIRDY (void)

Function description

Clear LSI ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSIRDYC LL_RCC_ClearFlag_LSIRDY

LL_RCC_ClearFlag_LSERDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_LSERDY (void)

Function description

Clear LSE ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSERDYC LL_RCC_ClearFlag_LSERDY

LL_RCC_ClearFlag_HSIRDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_HSIRDY (void)

Function description

Clear HSI ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSIRDYC LL_RCC_ClearFlag_HSIRDY
LL_RCC_ClearFlag_HSERDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_HSERDY (void)

Function description

Clear HSE ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSERDYC LL_RCC_ClearFlag_HSERDY
LL_RCC_ClearFlag_PLLRDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_PLLRDY (void)

Function description

Clear PLL ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLLRDYC LL_RCC_ClearFlag_PLLRDY
LL_RCC_ClearFlag_PLI2SRDY

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_PLI2SRDY (void)

Function description

Clear PLI2S ready interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLI2SRDYC LL_RCC_ClearFlag_PLI2SRDY
LL_RCC_ClearFlag_HSECSS

Function name

__STATIC_INLINE void LL_RCC_ClearFlag_HSECSS (void)

Function description

Clear Clock security system interrupt flag.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR CSSC LL_RCC_ClearFlag_HSECSS
LL_RCC_IsActiveFlag_LSIRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_LSIRDY (void)

Function description

Check if LSI ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR LSIRDYF LL_RCC_IsActiveFlag_LSIRDY
LL_RCC_IsActiveFlag_LSERDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_LSERDY (void)

Function description

Check if LSE ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR LSERDYF LL_RCC_IsActiveFlag_LSERDY
LL_RCC_IsActiveFlag_HSIRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_HSIRDY (void)

Function description

Check if HSI ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR HSIRDYF LL_RCC_IsActiveFlag_HSIRDY
LL_RCC_IsActiveFlag_HSERDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_HSERDY (void)

Function description

Check if HSE ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR HSERDYF LL_RCC_IsActiveFlag_HSERDY
LL_RCC_IsActiveFlag_PLLRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_PLLRDY (void)

Function description

Check if PLL ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR PLLRDYF LL_RCC_IsActiveFlag_PLLRDY
LL_RCC_IsActiveFlag_PLLI2SRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_PLLI2SRDY (void)

Function description

Check if PLLI2S ready interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR PLLI2SRDYF LL_RCC_IsActiveFlag_PLLI2SRDY
LL_RCC_IsActiveFlag_HSECSS

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_HSECSS (void)

Function description

Check if Clock security system interrupt occurred or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR CSSF LL_RCC_IsActiveFlag_HSECSS
LL_RCC_IsActiveFlag_IWDGRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_IWDGRST (void)

Function description

Check if RCC flag Independent Watchdog reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR IWDGRSTF LL_RCC_IsActiveFlag_IWDGRST

LL_RCC_IsActiveFlag_LPWRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_LPWRST (void)

Function description

Check if RCC flag Low Power reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR LPWRRSTF LL_RCC_IsActiveFlag_LPWRST

LL_RCC_IsActiveFlag_PINRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_PINRST (void)

Function description

Check if RCC flag Pin reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR PINRSTF LL_RCC_IsActiveFlag_PINRST

LL_RCC_IsActiveFlag_PORRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_PORRST (void)

Function description

Check if RCC flag POR/PDR reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR PORRSTF LL_RCC_IsActiveFlag_PORRST

LL_RCC_IsActiveFlag_SFTRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_SFTRST (void)

Function description

Check if RCC flag Software reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR SFTRSTF LL_RCC_IsActiveFlag_SFTRST

LL_RCC_IsActiveFlag_WWDGRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_WWDGRST (void)

Function description

Check if RCC flag Window Watchdog reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR WWDGRSTF LL_RCC_IsActiveFlag_WWDGRST
LL_RCC_IsActiveFlag_BORRST

Function name

__STATIC_INLINE uint32_t LL_RCC_IsActiveFlag_BORRST (void)

Function description

Check if RCC flag BOR reset is set or not.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR BORRSTF LL_RCC_IsActiveFlag_BORRST
LL_RCC_ClearResetFlags

Function name

__STATIC_INLINE void LL_RCC_ClearResetFlags (void)

Function description

Set RMVF bit to clear the reset flags.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CSR RMVF LL_RCC_ClearResetFlags
LL_RCC_EnableIT_LSIRDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_LSIRDY (void)

Function description

Enable LSI ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSIRDYIE LL_RCC_EnableIT_LSIRDY
LL_RCC_EnableIT_LSERDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_LSERDY (void)

Function description

Enable LSE ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSE RDY IE LL_RCC_EnableIT_LSERDY
LL_RCC_EnableIT_HSIRDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_HSIRDY (void)

Function description

Enable HSI ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSI RDY IE LL_RCC_EnableIT_HSIRDY
LL_RCC_EnableIT_HSERDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_HSERDY (void)

Function description

Enable HSE ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSE RDY IE LL_RCC_EnableIT_HSERDY
LL_RCC_EnableIT_PLLRDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_PLLRDY (void)

Function description

Enable PLL ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLL RDY IE LL_RCC_EnableIT_PLLRDY
LL_RCC_EnableIT_PLI2SRDY

Function name

__STATIC_INLINE void LL_RCC_EnableIT_PLI2SRDY (void)

Function description

Enable PLLI2S ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLLI2SRDYIE LL_RCC_EnableIT_PLLI2SRDY
LL_RCC_DisableIT_LSIRDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_LSIRDY (void)

Function description

Disable LSI ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSIRDYIE LL_RCC_DisableIT_LSIRDY
LL_RCC_DisableIT_LSERDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_LSERDY (void)

Function description

Disable LSE ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR LSERDYIE LL_RCC_DisableIT_LSERDY
LL_RCC_DisableIT_HSIRDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_HSIRDY (void)

Function description

Disable HSI ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSIRDYIE LL_RCC_DisableIT_HSIRDY
LL_RCC_DisableIT_HSERDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_HSERDY (void)

Function description

Disable HSE ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR HSERDYIE LL_RCC_DisableIT_HSERDY
LL_RCC_DisableIT_PLLRDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_PLLRDY (void)

Function description

Disable PLL ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLLRDYIE LL_RCC_DisableIT_PLLRDY
LL_RCC_DisableIT_PLLI2SRDY

Function name

__STATIC_INLINE void LL_RCC_DisableIT_PLLI2SRDY (void)

Function description

Disable PLLI2S ready interrupt.

Return values

- **None:**

Reference Manual to LL API cross reference:

- CIR PLLI2SRDYIE LL_RCC_DisableIT_PLLI2SRDY
LL_RCC_IsEnabledIT_LSIRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_LSIRDY (void)

Function description

Checks if LSI ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR LSIRDYIE LL_RCC_IsEnabledIT_LSIRDY
LL_RCC_IsEnabledIT_LSERDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_LSERDY (void)

Function description

Checks if LSE ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR LSERDYIE LL_RCC_IsEnabledIT_LSERDY

LL_RCC_IsEnabledIT_HSIRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_HSIRDY (void)

Function description

Checks if HSI ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR HSIRDYIE LL_RCC_IsEnabledIT_HSIRDY

LL_RCC_IsEnabledIT_HSERDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_HSERDY (void)

Function description

Checks if HSE ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR HSERDYIE LL_RCC_IsEnabledIT_HSERDY

LL_RCC_IsEnabledIT_PLLRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_PLLRDY (void)

Function description

Checks if PLL ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR PLLRDYIE LL_RCC_IsEnabledIT_PLLRDY

LL_RCC_IsEnabledIT_PLLI2SRDY

Function name

__STATIC_INLINE uint32_t LL_RCC_IsEnabledIT_PLLI2SRDY (void)

Function description

Checks if PLLI2S ready interrupt source is enabled or disabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CIR PLLI2SRDYIE LL_RCC_IsEnabledIT_PLLI2SRDY

LL_RCC_DeInit

Function name

ErrorStatus LL_RCC_DeInit (void)

Function description

Reset the RCC clock configuration to the default reset state.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RCC registers are de-initialized
 - ERROR: not applicable

Notes

- The default reset state of the clock configuration is given below: HSI ON and used as system clock sourceHSE, PLL, PLLI2S OFFAHB, APB1 and APB2 prescaler set to 1.CSS, MCO OFFAll interrupts disabled
- This function doesn't modify the configuration of the Peripheral clocksLSI, LSE and RTC clocks

LL_RCC_GetSystemClocksFreq

Function name

void LL_RCC_GetSystemClocksFreq (LL_RCC_ClocksTypeDef * RCC_Clocks)

Function description

Return the frequencies of different on chip clocks; System, AHB, APB1 and APB2 buses clocks.

Parameters

- **RCC_Clocks:** pointer to a LL_RCC_ClocksTypeDef structure which will hold the clocks frequencies

Return values

- **None:**

Notes

- Each time SYSClk, HCLK, PCLK1 and/or PCLK2 clock changes, this function must be called to update structure fields. Otherwise, any configuration based on this function will be incorrect.

LL_RCC_GetI2SClockFreq

Function name

uint32_t LL_RCC_GetI2SClockFreq (uint32_t I2SxSource)

Function description

Return I2Sx clock frequency.

Parameters

- **I2SxSource:** This parameter can be one of the following values:
 - LL_RCC_I2S1_CLKSOURCE

Return values

- **I2S:** clock frequency (in Hz)
 - LL_RCC_PERIPH_FREQUENCY_NO indicates that oscillator is not ready

60.3 RCC Firmware driver defines

The following section lists the various define and macros of the module.

60.3.1 RCC

APB low-speed prescaler (APB1)

LL_RCC_APB1_DIV_1

HCLK not divided

LL_RCC_APB1_DIV_2

HCLK divided by 2

LL_RCC_APB1_DIV_4

HCLK divided by 4

LL_RCC_APB1_DIV_8

HCLK divided by 8

LL_RCC_APB1_DIV_16

HCLK divided by 16

APB high-speed prescaler (APB2)

LL_RCC_APB2_DIV_1

HCLK not divided

LL_RCC_APB2_DIV_2

HCLK divided by 2

LL_RCC_APB2_DIV_4

HCLK divided by 4

LL_RCC_APB2_DIV_8

HCLK divided by 8

LL_RCC_APB2_DIV_16

HCLK divided by 16

Clear Flags Defines

LL_RCC_CIR_LSIRDYC

LSI Ready Interrupt Clear

LL_RCC_CIR_LSERDYC

LSE Ready Interrupt Clear

LL_RCC_CIR_HSIRDYC

HSI Ready Interrupt Clear

LL_RCC_CIR_HSERDYC

HSE Ready Interrupt Clear

LL_RCC_CIR_PLLRDYC

PLL Ready Interrupt Clear

LL_RCC_CIR_PLLI2SRDYC

PLLI2S Ready Interrupt Clear

LL_RCC_CIR_CSSC

Clock Security System Interrupt Clear

Get Flags Defines

LL_RCC_CIR_LSIRDYF

LSI Ready Interrupt flag

LL_RCC_CIR_LSERDYF

LSE Ready Interrupt flag

LL_RCC_CIR_HSIRDYF

HSI Ready Interrupt flag

LL_RCC_CIR_HSERDYF

HSE Ready Interrupt flag

LL_RCC_CIR_PLLRDYF

PLL Ready Interrupt flag

LL_RCC_CIR_PLLI2SRDYF

PLLI2S Ready Interrupt flag

LL_RCC_CIR_CSSF

Clock Security System Interrupt flag

LL_RCC_CSR_LPWRSTF

Low-Power reset flag

LL_RCC_CSR_PINRSTF

PIN reset flag

LL_RCC_CSR_PORRSTF

POR/PDR reset flag

LL_RCC_CSR_SFTRSTF

Software Reset flag

LL_RCC_CSR_IWDGRSTF

Independent Watchdog reset flag

LL_RCC_CSR_WWDGRSTF

Window watchdog reset flag

LL_RCC_CSR_BORRSTF

BOR reset flag

Peripheral I2S get clock source

LL_RCC_I2S1_CLKSOURCE

I2S1 Clock source selection

Peripheral I2S clock source selection

LL_RCC_I2S1_CLKSOURCE_PLLI2S

I2S oscillator clock used as I2S1 clock

LL_RCC_I2S1_CLKSOURCE_PIN

External pin clock used as I2S1 clock

IT Defines

LL_RCC_CIR_LSIRDYIE

LSI Ready Interrupt Enable

LL_RCC_CIR_LSERDYIE

LSE Ready Interrupt Enable

LL_RCC_CIR_HSIIRDYIE

HSI Ready Interrupt Enable

LL_RCC_CIR_HSERDYIE

HSE Ready Interrupt Enable

LL_RCC_CIR_PLLRDYIE

PLL Ready Interrupt Enable

LL_RCC_CIR_PLLI2SRDYIE

PLLI2S Ready Interrupt Enable

MCO source selection**LL_RCC_MCO1SOURCE_HSI**

HSI selection as MCO1 source

LL_RCC_MCO1SOURCE_LSE

LSE selection as MCO1 source

LL_RCC_MCO1SOURCE_HSE

HSE selection as MCO1 source

LL_RCC_MCO1SOURCE_PLLCLK

PLLCLK selection as MCO1 source

LL_RCC_MCO2SOURCE_SYSCLK

SYSCLK selection as MCO2 source

LL_RCC_MCO2SOURCE_PLLI2S

PLLI2S selection as MCO2 source

LL_RCC_MCO2SOURCE_HSE

HSE selection as MCO2 source

LL_RCC_MCO2SOURCE_PLLCLK

PLLCLK selection as MCO2 source

MCO prescaler**LL_RCC_MCO1_DIV_1**

MCO1 not divided

LL_RCC_MCO1_DIV_2

MCO1 divided by 2

LL_RCC_MCO1_DIV_3

MCO1 divided by 3

LL_RCC_MCO1_DIV_4

MCO1 divided by 4

LL_RCC_MCO1_DIV_5

MCO1 divided by 5

LL_RCC_MCO2_DIV_1

MCO2 not divided

LL_RCC_MCO2_DIV_2

MCO2 divided by 2

LL_RCC_MCO2_DIV_3

MCO2 divided by 3

LL_RCC_MCO2_DIV_4

MCO2 divided by 4

LL_RCC_MCO2_DIV_5

MCO2 divided by 5

Oscillator Values adaptation

HSE_VALUE

Value of the HSE oscillator in Hz

HSI_VALUE

Value of the HSI oscillator in Hz

LSE_VALUE

Value of the LSE oscillator in Hz

LSI_VALUE

Value of the LSI oscillator in Hz

EXTERNAL_CLOCK_VALUE

Value of the I2S_CKIN external oscillator in Hz

Peripheral clock frequency

LL_RCC_PERIPH_FREQUENCY_NO

No clock enabled for the peripheral

LL_RCC_PERIPH_FREQUENCY_NA

Frequency cannot be provided as external clock

PLLI2SR division factor (PLLI2SR)

LL_RCC_PLLI2SR_DIV_2

PLLI2S division factor for PLLI2SR output by 2

LL_RCC_PLLI2SR_DIV_3

PLLI2S division factor for PLLI2SR output by 3

LL_RCC_PLLI2SR_DIV_4

PLLI2S division factor for PLLI2SR output by 4

LL_RCC_PLLI2SR_DIV_5

PLLI2S division factor for PLLI2SR output by 5

LL_RCC_PLLI2SR_DIV_6

PLLI2S division factor for PLLI2SR output by 6

LL_RCC_PLLI2SR_DIV_7

PLLI2S division factor for PLLI2SR output by 7

PLL and PLLI2S division factor**LL_RCC_PLLM_DIV_2**

PLL and PLLI2S division factor by 2

LL_RCC_PLLM_DIV_3

PLL and PLLI2S division factor by 3

LL_RCC_PLLM_DIV_4

PLL and PLLI2S division factor by 4

LL_RCC_PLLM_DIV_5

PLL and PLLI2S division factor by 5

LL_RCC_PLLM_DIV_6

PLL and PLLI2S division factor by 6

LL_RCC_PLLM_DIV_7

PLL and PLLI2S division factor by 7

LL_RCC_PLLM_DIV_8

PLL and PLLI2S division factor by 8

LL_RCC_PLLM_DIV_9

PLL and PLLI2S division factor by 9

LL_RCC_PLLM_DIV_10

PLL and PLLI2S division factor by 10

LL_RCC_PLLM_DIV_11

PLL and PLLI2S division factor by 11

LL_RCC_PLLM_DIV_12

PLL and PLLI2S division factor by 12

LL_RCC_PLLM_DIV_13

PLL and PLLI2S division factor by 13

LL_RCC_PLLM_DIV_14

PLL and PLLI2S division factor by 14

LL_RCC_PLLM_DIV_15

PLL and PLLI2S division factor by 15

LL_RCC_PLLM_DIV_16

PLL and PLLI2S division factor by 16

LL_RCC_PLLM_DIV_17

PLL and PLLI2S division factor by 17

LL_RCC_PLLM_DIV_18

PLL and PLLI2S division factor by 18

LL_RCC_PLLM_DIV_19

PLL and PLLI2S division factor by 19

LL_RCC_PLLM_DIV_20

PLL and PLLI2S division factor by 20

LL_RCC_PLLM_DIV_21

PLL and PLLI2S division factor by 21

LL_RCC_PLLM_DIV_22

PLL and PLLI2S division factor by 22

LL_RCC_PLLM_DIV_23

PLL and PLLI2S division factor by 23

LL_RCC_PLLM_DIV_24

PLL and PLLI2S division factor by 24

LL_RCC_PLLM_DIV_25

PLL and PLLI2S division factor by 25

LL_RCC_PLLM_DIV_26

PLL and PLLI2S division factor by 26

LL_RCC_PLLM_DIV_27

PLL and PLLI2S division factor by 27

LL_RCC_PLLM_DIV_28

PLL and PLLI2S division factor by 28

LL_RCC_PLLM_DIV_29

PLL and PLLI2S division factor by 29

LL_RCC_PLLM_DIV_30

PLL and PLLI2S division factor by 30

LL_RCC_PLLM_DIV_31

PLL and PLLI2S division factor by 31

LL_RCC_PLLM_DIV_32

PLL and PLLI2S division factor by 32

LL_RCC_PLLM_DIV_33

PLL and PLLI2S division factor by 33

LL_RCC_PLLM_DIV_34

PLL and PLLI2S division factor by 34

LL_RCC_PLLM_DIV_35

PLL and PLLI2S division factor by 35

LL_RCC_PLLM_DIV_36

PLL and PLLI2S division factor by 36

LL_RCC_PLLM_DIV_37

PLL and PLLI2S division factor by 37

LL_RCC_PLLM_DIV_38

PLL and PLLI2S division factor by 38

LL_RCC_PLLM_DIV_39

PLL and PLLI2S division factor by 39

LL_RCC_PLLM_DIV_40

PLL and PLLI2S division factor by 40

LL_RCC_PLLM_DIV_41

PLL and PLLI2S division factor by 41

LL_RCC_PLLM_DIV_42

PLL and PLLI2S division factor by 42

LL_RCC_PLLM_DIV_43

PLL and PLLI2S division factor by 43

LL_RCC_PLLM_DIV_44

PLL and PLLI2S division factor by 44

LL_RCC_PLLM_DIV_45

PLL and PLLI2S division factor by 45

LL_RCC_PLLM_DIV_46

PLL and PLLI2S division factor by 46

LL_RCC_PLLM_DIV_47

PLL and PLLI2S division factor by 47

LL_RCC_PLLM_DIV_48

PLL and PLLI2S division factor by 48

LL_RCC_PLLM_DIV_49

PLL and PLLI2S division factor by 49

LL_RCC_PLLM_DIV_50

PLL and PLLI2S division factor by 50

LL_RCC_PLLM_DIV_51

PLL and PLLI2S division factor by 51

LL_RCC_PLLM_DIV_52

PLL and PLLI2S division factor by 52

LL_RCC_PLLM_DIV_53

PLL and PLLI2S division factor by 53

LL_RCC_PLLM_DIV_54

PLL and PLLI2S division factor by 54

LL_RCC_PLLM_DIV_55

PLL and PLLI2S division factor by 55

LL_RCC_PLLM_DIV_56

PLL and PLLI2S division factor by 56

LL_RCC_PLLM_DIV_57

PLL and PLLI2S division factor by 57

LL_RCC_PLLM_DIV_58

PLL and PLLI2S division factor by 58

LL_RCC_PLLM_DIV_59

PLL and PLLI2S division factor by 59

LL_RCC_PLLM_DIV_60

PLL and PLLI2S division factor by 60

LL_RCC_PLLM_DIV_61

PLL and PLLI2S division factor by 61

LL_RCC_PLLM_DIV_62

PLL and PLLI2S division factor by 62

LL_RCC_PLLM_DIV_63

PLL and PLLI2S division factor by 63

PLL division factor (PLLP)**LL_RCC_PLLP_DIV_2**

Main PLL division factor for PLLP output by 2

LL_RCC_PLLP_DIV_4

Main PLL division factor for PLLP output by 4

LL_RCC_PLLP_DIV_6

Main PLL division factor for PLLP output by 6

LL_RCC_PLLP_DIV_8

Main PLL division factor for PLLP output by 8

PLL division factor (PLLQ)**LL_RCC_PLLQ_DIV_2**

Main PLL division factor for PLLQ output by 2

LL_RCC_PLLQ_DIV_3

Main PLL division factor for PLLQ output by 3

LL_RCC_PLLQ_DIV_4

Main PLL division factor for PLLQ output by 4

LL_RCC_PLLQ_DIV_5

Main PLL division factor for PLLQ output by 5

LL_RCC_PLLQ_DIV_6

Main PLL division factor for PLLQ output by 6

LL_RCC_PLLQ_DIV_7

Main PLL division factor for PLLQ output by 7

LL_RCC_PLLQ_DIV_8

Main PLL division factor for PLLQ output by 8

LL_RCC_PLLQ_DIV_9

Main PLL division factor for PLLQ output by 9

LL_RCC_PLLQ_DIV_10

Main PLL division factor for PLLQ output by 10

LL_RCC_PLLQ_DIV_11

Main PLL division factor for PLLQ output by 11

LL_RCC_PLLQ_DIV_12

Main PLL division factor for PLLQ output by 12

LL_RCC_PLLQ_DIV_13

Main PLL division factor for PLLQ output by 13

LL_RCC_PLLQ_DIV_14

Main PLL division factor for PLLQ output by 14

LL_RCC_PLLQ_DIV_15

Main PLL division factor for PLLQ output by 15

PLL and PLLI2S entry clock source

LL_RCC_PLLSOURCE_HSI

HSI16 clock selected as PLL entry clock source

LL_RCC_PLLSOURCE_HSE

HSE clock selected as PLL entry clock source

PLL Spread Spectrum Selection

LL_RCC_SPREAD_SELECT_CENTER

PLL center spread spectrum selection

LL_RCC_SPREAD_SELECT_DOWN

PLL down spread spectrum selection

RTC clock source selection

LL_RCC_RTC_CLKSOURCE_NONE

No clock used as RTC clock

LL_RCC_RTC_CLKSOURCE_LSE

LSE oscillator clock used as RTC clock

LL_RCC_RTC_CLKSOURCE_LSI

LSI oscillator clock used as RTC clock

LL_RCC_RTC_CLKSOURCE_HSE

HSE oscillator clock divided by HSE prescaler used as RTC clock

HSE prescaler for RTC clock

LL_RCC_RTC_NOCLOCK

HSE not divided

LL_RCC_RTC_HSE_DIV_2

HSE clock divided by 2

LL_RCC_RTC_HSE_DIV_3

HSE clock divided by 3

LL_RCC_RTC_HSE_DIV_4

HSE clock divided by 4

LL_RCC_RTC_HSE_DIV_5

HSE clock divided by 5

LL_RCC_RTC_HSE_DIV_6

HSE clock divided by 6

LL_RCC_RTC_HSE_DIV_7

HSE clock divided by 7

LL_RCC_RTC_HSE_DIV_8

HSE clock divided by 8

LL_RCC_RTC_HSE_DIV_9

HSE clock divided by 9

LL_RCC_RTC_HSE_DIV_10

HSE clock divided by 10

LL_RCC_RTC_HSE_DIV_11

HSE clock divided by 11

LL_RCC_RTC_HSE_DIV_12

HSE clock divided by 12

LL_RCC_RTC_HSE_DIV_13

HSE clock divided by 13

LL_RCC_RTC_HSE_DIV_14

HSE clock divided by 14

LL_RCC_RTC_HSE_DIV_15

HSE clock divided by 15

LL_RCC_RTC_HSE_DIV_16

HSE clock divided by 16

LL_RCC_RTC_HSE_DIV_17

HSE clock divided by 17

LL_RCC_RTC_HSE_DIV_18

HSE clock divided by 18

LL_RCC_RTC_HSE_DIV_19

HSE clock divided by 19

LL_RCC_RTC_HSE_DIV_20

HSE clock divided by 20

LL_RCC_RTC_HSE_DIV_21

HSE clock divided by 21

LL_RCC_RTC_HSE_DIV_22

HSE clock divided by 22

LL_RCC_RTC_HSE_DIV_23

HSE clock divided by 23

LL_RCC_RTC_HSE_DIV_24

HSE clock divided by 24

LL_RCC_RTC_HSE_DIV_25

HSE clock divided by 25

LL_RCC_RTC_HSE_DIV_26

HSE clock divided by 26

LL_RCC_RTC_HSE_DIV_27

HSE clock divided by 27

LL_RCC_RTC_HSE_DIV_28

HSE clock divided by 28

LL_RCC_RTC_HSE_DIV_29

HSE clock divided by 29

LL_RCC_RTC_HSE_DIV_30

HSE clock divided by 30

LL_RCC_RTC_HSE_DIV_31

HSE clock divided by 31

AHB prescaler**LL_RCC_SYSCLK_DIV_1**

SYSCLK not divided

LL_RCC_SYSCLK_DIV_2

SYSCLK divided by 2

LL_RCC_SYSCLK_DIV_4

SYSCLK divided by 4

LL_RCC_SYSCLK_DIV_8

SYSCLK divided by 8

LL_RCC_SYSCLK_DIV_16

SYSCLK divided by 16

LL_RCC_SYSCLK_DIV_64

SYSCLK divided by 64

LL_RCC_SYSCLK_DIV_128

SYSCLK divided by 128

LL_RCC_SYSCLK_DIV_256

SYSCLK divided by 256

LL_RCC_SYSCLK_DIV_512

SYSCLK divided by 512

System clock switch

LL_RCC_SYS_CLKSOURCE_HSI

HSI selection as system clock

LL_RCC_SYS_CLKSOURCE_HSE

HSE selection as system clock

LL_RCC_SYS_CLKSOURCE_PLL

PLL selection as system clock

System clock switch status**LL_RCC_SYS_CLKSOURCE_STATUS_HSI**

HSI used as system clock

LL_RCC_SYS_CLKSOURCE_STATUS_HSE

HSE used as system clock

LL_RCC_SYS_CLKSOURCE_STATUS_PLL

PLL used as system clock

Calculate frequencies

__LL_RCC_CALC_PLLCLK_FREQ

Description:

- Helper macro to calculate the PLLCLK frequency on system domain.

Parameters:

- __INPUTFREQ__: PLL Input frequency (based on HSE/HSI)
- __PLLM__: This parameter can be one of the following values:
 - LL_RCC_PLLM_DIV_2
 - LL_RCC_PLLM_DIV_3
 - LL_RCC_PLLM_DIV_4
 - LL_RCC_PLLM_DIV_5
 - LL_RCC_PLLM_DIV_6
 - LL_RCC_PLLM_DIV_7
 - LL_RCC_PLLM_DIV_8
 - LL_RCC_PLLM_DIV_9
 - LL_RCC_PLLM_DIV_10
 - LL_RCC_PLLM_DIV_11
 - LL_RCC_PLLM_DIV_12
 - LL_RCC_PLLM_DIV_13
 - LL_RCC_PLLM_DIV_14
 - LL_RCC_PLLM_DIV_15
 - LL_RCC_PLLM_DIV_16
 - LL_RCC_PLLM_DIV_17
 - LL_RCC_PLLM_DIV_18
 - LL_RCC_PLLM_DIV_19
 - LL_RCC_PLLM_DIV_20
 - LL_RCC_PLLM_DIV_21
 - LL_RCC_PLLM_DIV_22
 - LL_RCC_PLLM_DIV_23
 - LL_RCC_PLLM_DIV_24
 - LL_RCC_PLLM_DIV_25
 - LL_RCC_PLLM_DIV_26
 - LL_RCC_PLLM_DIV_27
 - LL_RCC_PLLM_DIV_28
 - LL_RCC_PLLM_DIV_29
 - LL_RCC_PLLM_DIV_30
 - LL_RCC_PLLM_DIV_31
 - LL_RCC_PLLM_DIV_32
 - LL_RCC_PLLM_DIV_33
 - LL_RCC_PLLM_DIV_34
 - LL_RCC_PLLM_DIV_35
 - LL_RCC_PLLM_DIV_36
 - LL_RCC_PLLM_DIV_37
 - LL_RCC_PLLM_DIV_38
 - LL_RCC_PLLM_DIV_39
 - LL_RCC_PLLM_DIV_40
 - LL_RCC_PLLM_DIV_41
 - LL_RCC_PLLM_DIV_42
 - LL_RCC_PLLM_DIV_43
 - LL_RCC_PLLM_DIV_44

- LL_RCC_PLLM_DIV_45
- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48
- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63
- __PLLN__: Between 192 and 432
- __PLL__P__: This parameter can be one of the following values:
 - LL_RCC_PLLP_DIV_2
 - LL_RCC_PLLP_DIV_4
 - LL_RCC_PLLP_DIV_6
 - LL_RCC_PLLP_DIV_8

Return value:

- PLL: clock frequency (in Hz)

Notes:

- ex: __LL_RCC_CALC_PLLCLK_FREQ (HSE_VALUE,LL_RCC_PLL_GetDivider (), LL_RCC_PLL_GetN (), LL_RCC_PLL_GetP ());

__LL_RCC_CALC_PLLCLK_48M_FREQ

Description:

- Helper macro to calculate the PLLCLK frequency used on 48M domain.

Parameters:

- __INPUTFREQ__: PLL Input frequency (based on HSE/HSI)
- __PLLM__: This parameter can be one of the following values:
 - LL_RCC_PLLM_DIV_2
 - LL_RCC_PLLM_DIV_3
 - LL_RCC_PLLM_DIV_4
 - LL_RCC_PLLM_DIV_5
 - LL_RCC_PLLM_DIV_6
 - LL_RCC_PLLM_DIV_7
 - LL_RCC_PLLM_DIV_8
 - LL_RCC_PLLM_DIV_9
 - LL_RCC_PLLM_DIV_10
 - LL_RCC_PLLM_DIV_11
 - LL_RCC_PLLM_DIV_12
 - LL_RCC_PLLM_DIV_13
 - LL_RCC_PLLM_DIV_14
 - LL_RCC_PLLM_DIV_15
 - LL_RCC_PLLM_DIV_16
 - LL_RCC_PLLM_DIV_17
 - LL_RCC_PLLM_DIV_18
 - LL_RCC_PLLM_DIV_19
 - LL_RCC_PLLM_DIV_20
 - LL_RCC_PLLM_DIV_21
 - LL_RCC_PLLM_DIV_22
 - LL_RCC_PLLM_DIV_23
 - LL_RCC_PLLM_DIV_24
 - LL_RCC_PLLM_DIV_25
 - LL_RCC_PLLM_DIV_26
 - LL_RCC_PLLM_DIV_27
 - LL_RCC_PLLM_DIV_28
 - LL_RCC_PLLM_DIV_29
 - LL_RCC_PLLM_DIV_30
 - LL_RCC_PLLM_DIV_31
 - LL_RCC_PLLM_DIV_32
 - LL_RCC_PLLM_DIV_33
 - LL_RCC_PLLM_DIV_34
 - LL_RCC_PLLM_DIV_35
 - LL_RCC_PLLM_DIV_36
 - LL_RCC_PLLM_DIV_37
 - LL_RCC_PLLM_DIV_38
 - LL_RCC_PLLM_DIV_39
 - LL_RCC_PLLM_DIV_40
 - LL_RCC_PLLM_DIV_41
 - LL_RCC_PLLM_DIV_42
 - LL_RCC_PLLM_DIV_43
 - LL_RCC_PLLM_DIV_44

- LL_RCC_PLLM_DIV_45
- LL_RCC_PLLM_DIV_46
- LL_RCC_PLLM_DIV_47
- LL_RCC_PLLM_DIV_48
- LL_RCC_PLLM_DIV_49
- LL_RCC_PLLM_DIV_50
- LL_RCC_PLLM_DIV_51
- LL_RCC_PLLM_DIV_52
- LL_RCC_PLLM_DIV_53
- LL_RCC_PLLM_DIV_54
- LL_RCC_PLLM_DIV_55
- LL_RCC_PLLM_DIV_56
- LL_RCC_PLLM_DIV_57
- LL_RCC_PLLM_DIV_58
- LL_RCC_PLLM_DIV_59
- LL_RCC_PLLM_DIV_60
- LL_RCC_PLLM_DIV_61
- LL_RCC_PLLM_DIV_62
- LL_RCC_PLLM_DIV_63
- __PLLN__: Between 192 and 432
- __PLLQ__: This parameter can be one of the following values:
 - LL_RCC_PLLQ_DIV_2
 - LL_RCC_PLLQ_DIV_3
 - LL_RCC_PLLQ_DIV_4
 - LL_RCC_PLLQ_DIV_5
 - LL_RCC_PLLQ_DIV_6
 - LL_RCC_PLLQ_DIV_7
 - LL_RCC_PLLQ_DIV_8
 - LL_RCC_PLLQ_DIV_9
 - LL_RCC_PLLQ_DIV_10
 - LL_RCC_PLLQ_DIV_11
 - LL_RCC_PLLQ_DIV_12
 - LL_RCC_PLLQ_DIV_13
 - LL_RCC_PLLQ_DIV_14
 - LL_RCC_PLLQ_DIV_15

Return value:

- PLL: clock frequency (in Hz)

Notes:

- ex: __LL_RCC_CALC_PLLCLK_48M_FREQ (HSE_VALUE,LL_RCC_PLL_GetDivider (),
LL_RCC_PLL_GetN (), LL_RCC_PLL_GetQ ());

__LL_RCC_CALC_PLLI2S_I2S_FREQ

Description:

- Helper macro to calculate the PLLI2S frequency used for I2S domain.

Parameters:

- **__INPUTFREQ__**: PLL Input frequency (based on HSE/HSI)
- **__PLLM__**: This parameter can be one of the following values:
- **__PLLI2SN__**: Between 192 and 432
- **__PLLI2SR__**: This parameter can be one of the following values:
 - **LL_RCC_PLLI2SR_DIV_2**
 - **LL_RCC_PLLI2SR_DIV_3**
 - **LL_RCC_PLLI2SR_DIV_4**
 - **LL_RCC_PLLI2SR_DIV_5**
 - **LL_RCC_PLLI2SR_DIV_6**
 - **LL_RCC_PLLI2SR_DIV_7**

Return value:

- PLLI2S: clock frequency (in Hz)
- PLLI2S: clock frequency (in Hz)

Notes:

- ex: **__LL_RCC_CALC_PLLI2S_I2S_FREQ** (HSE_VALUE,LL_RCC_PLL_GetDivider (),
LL_RCC_PLLI2S_GetN (), LL_RCC_PLLI2S_GetR ());

__LL_RCC_CALC_HCLK_FREQ

Description:

- Helper macro to calculate the HCLK frequency.

Parameters:

- **__SYSCLKFREQ__**: SYSCLK frequency (based on HSE/HSI/PLLCLK)
- **__AHBPRESCALER__**: This parameter can be one of the following values:
 - **LL_RCC_SYSCLK_DIV_1**
 - **LL_RCC_SYSCLK_DIV_2**
 - **LL_RCC_SYSCLK_DIV_4**
 - **LL_RCC_SYSCLK_DIV_8**
 - **LL_RCC_SYSCLK_DIV_16**
 - **LL_RCC_SYSCLK_DIV_64**
 - **LL_RCC_SYSCLK_DIV_128**
 - **LL_RCC_SYSCLK_DIV_256**
 - **LL_RCC_SYSCLK_DIV_512**

Return value:

- HCLK: clock frequency (in Hz)

__LL_RCC_CALC_PCLK1_FREQ

Description:

- Helper macro to calculate the PCLK1 frequency (ABP1)

Parameters:

- **__HCLKFREQ__**: HCLK frequency
- **__APB1PRESCALER__**: This parameter can be one of the following values:
 - LL_RCC_APB1_DIV_1
 - LL_RCC_APB1_DIV_2
 - LL_RCC_APB1_DIV_4
 - LL_RCC_APB1_DIV_8
 - LL_RCC_APB1_DIV_16

Return value:

- PCLK1: clock frequency (in Hz)

__LL_RCC_CALC_PCLK2_FREQ

Description:

- Helper macro to calculate the PCLK2 frequency (ABP2)

Parameters:

- **__HCLKFREQ__**: HCLK frequency
- **__APB2PRESCALER__**: This parameter can be one of the following values:
 - LL_RCC_APB2_DIV_1
 - LL_RCC_APB2_DIV_2
 - LL_RCC_APB2_DIV_4
 - LL_RCC_APB2_DIV_8
 - LL_RCC_APB2_DIV_16

Return value:

- PCLK2: clock frequency (in Hz)

Common Write and read registers Macros

LL_RCC_WriteReg

Description:

- Write a value in RCC register.

Parameters:

- **__REG__**: Register to be written
- **__VALUE__**: Value to be written in the register

Return value:

- None

LL_RCC_ReadReg

Description:

- Read a value in RCC register.

Parameters:

- **__REG__**: Register to be read

Return value:

- Register: value

61 LL RNG Generic Driver

61.1 RNG Firmware driver API description

The following section lists the various functions of the RNG library.

61.1.1 Detailed description of functions

`LL_RNG_Enable`

Function name

`__STATIC_INLINE void LL_RNG_Enable (RNG_TypeDef * RNGx)`

Function description

Enable Random Number Generation.

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR RNGEN `LL_RNG_Enable`

`LL_RNG_Disable`

Function name

`__STATIC_INLINE void LL_RNG_Disable (RNG_TypeDef * RNGx)`

Function description

Disable Random Number Generation.

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR RNGEN `LL_RNG_Disable`

`LL_RNG_IsEnabled`

Function name

`__STATIC_INLINE uint32_t LL_RNG_IsEnabled (RNG_TypeDef * RNGx)`

Function description

Check if Random Number Generator is enabled.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR RNGEN LL_RNG_IsEnabled
LL_RNG_IsActiveFlag_DRDY

Function name

__STATIC_INLINE uint32_t LL_RNG_IsActiveFlag_DRDY (RNG_TypeDef * RNGx)

Function description

Indicate if the RNG Data ready Flag is set or not.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR DRDY LL_RNG_IsActiveFlag_DRDY
LL_RNG_IsActiveFlag_CECS

Function name

__STATIC_INLINE uint32_t LL_RNG_IsActiveFlag_CECS (RNG_TypeDef * RNGx)

Function description

Indicate if the Clock Error Current Status Flag is set or not.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CECS LL_RNG_IsActiveFlag_CECS
LL_RNG_IsActiveFlag_SECS

Function name

__STATIC_INLINE uint32_t LL_RNG_IsActiveFlag_SECS (RNG_TypeDef * RNGx)

Function description

Indicate if the Seed Error Current Status Flag is set or not.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR SECS LL_RNG_IsActiveFlag_SECS
LL_RNG_IsActiveFlag_CEIS

Function name

__STATIC_INLINE uint32_t LL_RNG_IsActiveFlag_CEIS (RNG_TypeDef * RNGx)

Function description

Indicate if the Clock Error Interrupt Status Flag is set or not.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CEIS LL_RNG_IsActiveFlag_CEIS

LL_RNG_IsActiveFlag_SEIS

Function name

__STATIC_INLINE uint32_t LL_RNG_IsActiveFlag_SEIS (RNG_TypeDef * RNGx)

Function description

Indicate if the Seed Error Interrupt Status Flag is set or not.

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR SEIS LL_RNG_IsActiveFlag_SEIS

LL_RNG_ClearFlag_CEIS

Function name

__STATIC_INLINE void LL_RNG_ClearFlag_CEIS (RNG_TypeDef * RNGx)

Function description

Clear Clock Error interrupt Status (CEIS) Flag.

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CEIS LL_RNG_ClearFlag_CEIS

LL_RNG_ClearFlag_SEIS

Function name

__STATIC_INLINE void LL_RNG_ClearFlag_SEIS (RNG_TypeDef * RNGx)

Function description

Clear Seed Error interrupt Status (SEIS) Flag.

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR SEIS LL_RNG_ClearFlag_SEIS
LL_RNG_EnableIT

Function name

__STATIC_INLINE void LL_RNG_EnableIT (RNG_TypeDef * RNGx)

Function description

Enable Random Number Generator Interrupt (applies for either Seed error, Clock Error or Data ready interrupts)

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR IE LL_RNG_EnableIT
LL_RNG_DisableIT

Function name

__STATIC_INLINE void LL_RNG_DisableIT (RNG_TypeDef * RNGx)

Function description

Disable Random Number Generator Interrupt (applies for either Seed error, Clock Error or Data ready interrupts)

Parameters

- **RNGx:** RNG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR IE LL_RNG_DisableIT
LL_RNG_IsEnabledIT

Function name

__STATIC_INLINE uint32_t LL_RNG_IsEnabledIT (RNG_TypeDef * RNGx)

Function description

Check if Random Number Generator Interrupt is enabled (applies for either Seed error, Clock Error or Data ready interrupts)

Parameters

- **RNGx:** RNG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR IE LL_RNG_IsEnabledIT
LL_RNG_ReadRandData32

Function name

__STATIC_INLINE uint32_t LL_RNG_ReadRandData32 (RNG_TypeDef * RNGx)

Function description

Return 32-bit Random Number value.

Parameters

- **RNGx:** RNG Instance

Return values

- **Generated:** 32-bit random value

Reference Manual to LL API cross reference:

- DR RNDATA LL_RNG_ReadRandData32
- LL_RNG_DeInit

Function name

ErrorStatus LL_RNG_DeInit (RNG_TypeDef * RNGx)

Function description

De-initialize RNG registers (Registers restored to their default values).

Parameters

- **RNGx:** RNG Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RNG registers are de-initialized
 - ERROR: not applicable

61.2 RNG Firmware driver defines

The following section lists the various define and macros of the module.

61.2.1 RNG

RNG

Get Flags Defines

LL_RNG_SR_DRDY

Register contains valid random data

LL_RNG_SR_CECS

Clock error current status

LL_RNG_SR_SECS

Seed error current status

LL_RNG_SR_CEIS

Clock error interrupt status

LL_RNG_SR_SEIS

Seed error interrupt status

IT Defines

LL_RNG_CR_IE

RNG Interrupt enable

Common Write and read registers Macros

LL_RNG_WriteReg

Description:

- Write a value in RNG register.

Parameters:

- `__INSTANCE__`: RNG Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_RNG_ReadReg

Description:

- Read a value in RNG register.

Parameters:

- `__INSTANCE__`: RNG Instance
- `__REG__`: Register to be read

Return value:

- Register: value

62 LL RTC Generic Driver

62.1 RTC Firmware driver registers structures

62.1.1 LL_RTC_InitTypeDef

LL_RTC_InitTypeDef is defined in the stm32f2xx_ll_rtc.h

Data Fields

- **uint32_t HourFormat**
- **uint32_t AsynchPrescaler**
- **uint32_t SynchPrescaler**

Field Documentation

- **uint32_t LL_RTC_InitTypeDef::HourFormat**
Specifies the RTC Hours Format. This parameter can be a value of [RTC_LL_EC_HOURFORMAT](#). This feature can be modified afterwards using unitary function **LL_RTC_SetHourFormat()**.
- **uint32_t LL_RTC_InitTypeDef::AsynchPrescaler**
Specifies the RTC Asynchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x7F. This feature can be modified afterwards using unitary function **LL_RTC_SetAsynchPrescaler()**.
- **uint32_t LL_RTC_InitTypeDef::SynchPrescaler**
Specifies the RTC Synchronous Predivider value. This parameter must be a number between Min_Data = 0x00 and Max_Data = 0x7FFF. This feature can be modified afterwards using unitary function **LL_RTC_SetSynchPrescaler()**.

62.1.2 LL_RTC_TimeTypeDef

LL_RTC_TimeTypeDef is defined in the stm32f2xx_ll_rtc.h

Data Fields

- **uint32_t TimeFormat**
- **uint8_t Hours**
- **uint8_t Minutes**
- **uint8_t Seconds**

Field Documentation

- **uint32_t LL_RTC_TimeTypeDef::TimeFormat**
Specifies the RTC AM/PM Time. This parameter can be a value of [RTC_LL_EC_TIME_FORMAT](#). This feature can be modified afterwards using unitary function **LL_RTC_TIME_SetFormat()**.
- **uint8_t LL_RTC_TimeTypeDef::Hours**
Specifies the RTC Time Hours. This parameter must be a number between Min_Data = 0 and Max_Data = 12 if the **LL_RTC_TIME_FORMAT_PM** is selected. This parameter must be a number between Min_Data = 0 and Max_Data = 23 if the **LL_RTC_TIME_FORMAT_AM_OR_24** is selected. This feature can be modified afterwards using unitary function **LL_RTC_TIME_SetHour()**.
- **uint8_t LL_RTC_TimeTypeDef::Minutes**
Specifies the RTC Time Minutes. This parameter must be a number between Min_Data = 0 and Max_Data = 59. This feature can be modified afterwards using unitary function **LL_RTC_TIME_SetMinute()**.
- **uint8_t LL_RTC_TimeTypeDef::Seconds**
Specifies the RTC Time Seconds. This parameter must be a number between Min_Data = 0 and Max_Data = 59. This feature can be modified afterwards using unitary function **LL_RTC_TIME_SetSecond()**.

62.1.3 LL_RTC_DateTypeDef

LL_RTC_DateTypeDef is defined in the stm32f2xx_ll_rtc.h

Data Fields

- **uint8_t WeekDay**

- `uint8_t Month`
- `uint8_t Day`
- `uint8_t Year`

Field Documentation

- `uint8_t LL_RTC_DateTypeDef::WeekDay`
Specifies the RTC Date WeekDay. This parameter can be a value of `RTC_LL_EC_WEEKDAY`. This feature can be modified afterwards using unitary function `LL_RTC_DATE_SetWeekDay()`.
- `uint8_t LL_RTC_DateTypeDef::Month`
Specifies the RTC Date Month. This parameter can be a value of `RTC_LL_EC_MONTH`. This feature can be modified afterwards using unitary function `LL_RTC_DATE_SetMonth()`.
- `uint8_t LL_RTC_DateTypeDef::Day`
Specifies the RTC Date Day. This parameter must be a number between Min_Data = 1 and Max_Data = 31. This feature can be modified afterwards using unitary function `LL_RTC_DATE_SetDay()`.
- `uint8_t LL_RTC_DateTypeDef::Year`
Specifies the RTC Date Year. This parameter must be a number between Min_Data = 0 and Max_Data = 99. This feature can be modified afterwards using unitary function `LL_RTC_DATE_SetYear()`.

62.1.4

LL_RTC_AlarmTypeDef

`LL_RTC_AlarmTypeDef` is defined in the `stm32f2xx_ll_rtc.h`

Data Fields

- `LL_RTC_TimeTypeDef AlarmTime`
- `uint32_t AlarmMask`
- `uint32_t AlarmDateWeekDaySel`
- `uint8_t AlarmDateWeekDay`

Field Documentation

- `LL_RTC_TimeTypeDef LL_RTC_AlarmTypeDef::AlarmTime`
Specifies the RTC Alarm Time members.
- `uint32_t LL_RTC_AlarmTypeDef::AlarmMask`
Specifies the RTC Alarm Masks. This parameter can be a value of `RTC_LL_EC_ALMA_MASK` for ALARM A or `RTC_LL_EC_ALMB_MASK` for ALARM B. This feature can be modified afterwards using unitary function `LL_RTC_ALMA_SetMask()` for ALARM A or `LL_RTC_ALMB_SetMask()` for ALARM B.
- `uint32_t LL_RTC_AlarmTypeDef::AlarmDateWeekDaySel`
Specifies the RTC Alarm is on day or WeekDay. This parameter can be a value of `RTC_LL_EC_ALMA_WEEKDAY_SELECTION` for ALARM A or `RTC_LL_EC_ALMB_WEEKDAY_SELECTION` for ALARM B. This feature can be modified afterwards using unitary function `LL_RTC_ALMA_EnableWeekday()` or `LL_RTC_ALMA_DisableWeekday()` for ALARM A or `LL_RTC_ALMB_EnableWeekday()` or `LL_RTC_ALMB_DisableWeekday()` for ALARM B.
- `uint8_t LL_RTC_AlarmTypeDef::AlarmDateWeekDay`
Specifies the RTC Alarm Day/WeekDay. If `AlarmDateWeekDaySel` set to day, this parameter must be a number between Min_Data = 1 and Max_Data = 31. This feature can be modified afterwards using unitary function `LL_RTC_ALMA_SetDay()` for ALARM A or `LL_RTC_ALMB_SetDay()` for ALARM B. If `AlarmDateWeekDaySel` set to Weekday, this parameter can be a value of `RTC_LL_EC_WEEKDAY`. This feature can be modified afterwards using unitary function `LL_RTC_ALMA_SetWeekDay()` for ALARM A or `LL_RTC_ALMB_SetWeekDay()` for ALARM B.

62.2

RTC Firmware driver API description

The following section lists the various functions of the RTC library.

62.2.1

Detailed description of functions

`LL_RTC_SetHourFormat`

Function name

```
__STATIC_INLINE void LL_RTC_SetHourFormat (RTC_TypeDef * RTCx, uint32_t HourFormat)
```

Function description

Set Hours format (24 hour/day or AM/PM hour format)

Parameters

- **RTCx:** RTC Instance
- **HourFormat:** This parameter can be one of the following values:
 - LL_RTC_HOURFORMAT_24HOUR
 - LL_RTC_HOURFORMAT_AMPM

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- CR FMT LL_RTC_SetHourFormat

LL_RTC_GetHourFormat

Function name

```
__STATIC_INLINE uint32_t LL_RTC_GetHourFormat (RTC_TypeDef * RTCx)
```

Function description

Get Hours format (24 hour/day or AM/PM hour format)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_HOURFORMAT_24HOUR
 - LL_RTC_HOURFORMAT_AMPM

Reference Manual to LL API cross reference:

- CR FMT LL_RTC_GetHourFormat

LL_RTC_SetAlarmOutEvent

Function name

```
__STATIC_INLINE void LL_RTC_SetAlarmOutEvent (RTC_TypeDef * RTCx, uint32_t AlarmOutput)
```

Function description

Select the flag to be routed to RTC_ALARM output.

Parameters

- **RTCx:** RTC Instance
- **AlarmOutput:** This parameter can be one of the following values:
 - LL_RTC_ALARMOUT_DISABLE
 - LL_RTC_ALARMOUT_ALMA
 - LL_RTC_ALARMOUT_ALMB
 - LL_RTC_ALARMOUT_WAKEUP

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR OSEL LL_RTC_SetAlarmOutEvent
LL_RTC_GetAlarmOutEvent

Function name

__STATIC_INLINE uint32_t LL_RTC_GetAlarmOutEvent (RTC_TypeDef * RTCx)

Function description

Get the flag to be routed to RTC_ALARM output.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_ALARMOUT_DISABLE
 - LL_RTC_ALARMOUT_ALMA
 - LL_RTC_ALARMOUT_ALMB
 - LL_RTC_ALARMOUT_WAKEUP

Reference Manual to LL API cross reference:

- CR OSEL LL_RTC_GetAlarmOutEvent
LL_RTC_SetAlarmOutputType

Function name

__STATIC_INLINE void LL_RTC_SetAlarmOutputType (RTC_TypeDef * RTCx, uint32_t Output)

Function description

Set RTC_ALARM output type (ALARM in push-pull or open-drain output)

Parameters

- **RTCx:** RTC Instance
- **Output:** This parameter can be one of the following values:
 - LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN
 - LL_RTC_ALARM_OUTPUTTYPE_PUSHPULL

Return values

- **None:**

Notes

- Used only when RTC_ALARM is mapped on PC13

Reference Manual to LL API cross reference:

- TAFCR_ALARMOUTTYPE LL_RTC_SetAlarmOutputType
LL_RTC_GetAlarmOutputType

Function name

__STATIC_INLINE uint32_t LL_RTC_GetAlarmOutputType (RTC_TypeDef * RTCx)

Function description

Get RTC_ALARM output type (ALARM in push-pull or open-drain output)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN
 - LL_RTC_ALARM_OUTPUTTYPE_PUSHPULL

Notes

- used only when RTC_ALARM is mapped on PC13

Reference Manual to LL API cross reference:

- TAFCR ALARMOUTTYPE LL_RTC_GetAlarmOutputType
LL_RTC_EnableInitMode

Function name

__STATIC_INLINE void LL_RTC_EnableInitMode (RTC_TypeDef * RTCx)

Function description

Enable initialization mode.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Initialization mode is used to program time and date register (RTC_TR and RTC_DR) and prescaler register (RTC_PRER). Counters are stopped and start counting from the new value when INIT is reset.

Reference Manual to LL API cross reference:

- ISR INIT LL_RTC_EnableInitMode
LL_RTC_DisableInitMode

Function name

__STATIC_INLINE void LL_RTC_DisableInitMode (RTC_TypeDef * RTCx)

Function description

Disable initialization mode (Free running mode)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ISR INIT LL_RTC_DisableInitMode
LL_RTC_SetOutputPolarity

Function name

```
__STATIC_INLINE void LL_RTC_SetOutputPolarity (RTC_TypeDef * RTCx, uint32_t Polarity)
```

Function description

Set Output polarity (pin is low when ALRAF/ALRBF/WUTF is asserted)

Parameters

- **RTCx:** RTC Instance
- **Polarity:** This parameter can be one of the following values:
 - LL_RTC_OUTPUTPOLARITY_PIN_HIGH
 - LL_RTC_OUTPUTPOLARITY_PIN_LOW

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR POL LL_RTC_SetOutputPolarity

LL_RTC_GetOutputPolarity

Function name

```
__STATIC_INLINE uint32_t LL_RTC_GetOutputPolarity (RTC_TypeDef * RTCx)
```

Function description

Get Output polarity.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_OUTPUTPOLARITY_PIN_HIGH
 - LL_RTC_OUTPUTPOLARITY_PIN_LOW

Reference Manual to LL API cross reference:

- CR POL LL_RTC_GetOutputPolarity

LL_RTC_EnableRefClock

Function name

```
__STATIC_INLINE void LL_RTC_EnableRefClock (RTC_TypeDef * RTCx)
```

Function description

Enable RTC_REFIN reference clock detection (50 or 60 Hz)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- CR REFCKON LL_RTC_EnableRefClock

LL_RTC_DisableRefClock

Function name

__STATIC_INLINE void LL_RTC_DisableRefClock (RTC_TypeDef * RTCx)

Function description

Disable RTC_REFIN reference clock detection (50 or 60 Hz)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- CR REFCKON LL_RTC_DisableRefClock

LL_RTC_SetAsynchPrescaler

Function name

__STATIC_INLINE void LL_RTC_SetAsynchPrescaler (RTC_TypeDef * RTCx, uint32_t AsynchPrescaler)

Function description

Set Asynchronous prescaler factor.

Parameters

- **RTCx:** RTC Instance
- **AsynchPrescaler:** Value between Min_Data = 0 and Max_Data = 0x7F

Return values

- **None:**

Reference Manual to LL API cross reference:

- PRER PREDIV_A LL_RTC_SetAsynchPrescaler

LL_RTC_SetSynchPrescaler

Function name

__STATIC_INLINE void LL_RTC_SetSynchPrescaler (RTC_TypeDef * RTCx, uint32_t SynchPrescaler)

Function description

Set Synchronous prescaler factor.

Parameters

- **RTCx:** RTC Instance
- **SynchPrescaler:** Value between Min_Data = 0 and Max_Data = 0x7FFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- PRER PREDIV_S LL_RTC_SetSynchPrescaler
LL_RTC_GetAsynchPrescaler

Function name

__STATIC_INLINE uint32_t LL_RTC_GetAsynchPrescaler (RTC_TypeDef * RTCx)

Function description

Get Asynchronous prescaler factor.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data = 0 and Max_Data = 0x7F

Reference Manual to LL API cross reference:

- PRER PREDIV_A LL_RTC_GetAsynchPrescaler
LL_RTC_GetSynchPrescaler

Function name

__STATIC_INLINE uint32_t LL_RTC_GetSynchPrescaler (RTC_TypeDef * RTCx)

Function description

Get Synchronous prescaler factor.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data = 0 and Max_Data = 0x7FFF

Reference Manual to LL API cross reference:

- PRER PREDIV_S LL_RTC_GetSynchPrescaler
LL_RTC_EnableWriteProtection

Function name

__STATIC_INLINE void LL_RTC_EnableWriteProtection (RTC_TypeDef * RTCx)

Function description

Enable the write protection for RTC registers.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- WPR KEY LL_RTC_EnableWriteProtection
LL_RTC_DisableWriteProtection

Function name

__STATIC_INLINE void LL_RTC_DisableWriteProtection (RTC_TypeDef * RTCx)

Function description

Disable the write protection for RTC registers.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- WPR KEY LL_RTC_DisableWriteProtection

LL_RTC_TIME_SetFormat

Function name

__STATIC_INLINE void LL_RTC_TIME_SetFormat (RTC_TypeDef * RTCx, uint32_t TimeFormat)

Function description

Set time format (AM/24-hour or PM notation)

Parameters

- **RTCx:** RTC Instance
- **TimeFormat:** This parameter can be one of the following values:
 - LL_RTC_TIME_FORMAT_AM_OR_24
 - LL_RTC_TIME_FORMAT_PM

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- TR PM LL_RTC_TIME_SetFormat

LL_RTC_TIME_GetFormat

Function name

__STATIC_INLINE uint32_t LL_RTC_TIME_GetFormat (RTC_TypeDef * RTCx)

Function description

Get time format (AM or PM notation)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_TIME_FORMAT_AM_OR_24
 - LL_RTC_TIME_FORMAT_PM

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- Read either RTC_SSR or RTC_TR locks the values in the higher-order calendar shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC, DR)).

Reference Manual to LL API cross reference:

- TR PM LL_RTC_TIME_GetFormat
- LL_RTC_TIME_SetHour

Function name

```
__STATIC_INLINE void LL_RTC_TIME_SetHour (RTC_TypeDef * RTCx, uint32_t Hours)
```

Function description

Set Hours in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)
- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert hour from binary to BCD format

Reference Manual to LL API cross reference:

- TR HT LL_RTC_TIME_SetHour
- TR HU LL_RTC_TIME_SetHour
- LL_RTC_TIME_GetHour

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TIME_GetHour (RTC_TypeDef * RTCx)
```

Function description

Get Hours in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- Read either RTC_SSR or RTC_TR locks the values in the higher-order calendar shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC, DR)).
- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert hour from BCD to Binary format

Reference Manual to LL API cross reference:

- TR HT LL_RTC_TIME_GetHour
- TR HU LL_RTC_TIME_GetHour
- LL_RTC_TIME_SetMinute

Function name

```
__STATIC_INLINE void LL_RTC_TIME_SetMinute (RTC_TypeDef * RTCx, uint32_t Minutes)
```


Function description

Set Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Minutes:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)
- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Minutes from binary to BCD format

Reference Manual to LL API cross reference:

- TR MNT LL_RTC_TIME_SetMinute
- TR MNU LL_RTC_TIME_SetMinute

LL_RTC_TIME_GetMinute

Function name

__STATIC_INLINE uint32_t LL_RTC_TIME_GetMinute (RTC_TypeDef * RTCx)

Function description

Get Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- Read either RTC_SSR or RTC_TR locks the values in the higher-order calendar shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC, DR)).
- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert minute from BCD to Binary format

Reference Manual to LL API cross reference:

- TR MNT LL_RTC_TIME_GetMinute
- TR MNU LL_RTC_TIME_GetMinute

LL_RTC_TIME_SetSecond

Function name

__STATIC_INLINE void LL_RTC_TIME_SetSecond (RTC_TypeDef * RTCx, uint32_t Seconds)

Function description

Set Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)
- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Seconds from binary to BCD format

Reference Manual to LL API cross reference:

- TR ST LL_RTC_TIME_SetSecond
- TR SU LL_RTC_TIME_SetSecond

LL_RTC_TIME_GetSecond

Function name

__STATIC_INLINE uint32_t LL_RTC_TIME_GetSecond (RTC_TypeDef * RTCx)

Function description

Get Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- Read either RTC_SSR or RTC_TR locks the values in the higher-order calendar shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC, DR)).
- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert Seconds from BCD to Binary format

Reference Manual to LL API cross reference:

- TR ST LL_RTC_TIME_GetSecond
- TR SU LL_RTC_TIME_GetSecond

LL_RTC_TIME_Config

Function name

__STATIC_INLINE void LL_RTC_TIME_Config (RTC_TypeDef * RTCx, uint32_t Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)

Function description

Set time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Format12_24:** This parameter can be one of the following values:
 - LL_RTC_TIME_FORMAT_AM_OR_24
 - LL_RTC_TIME_FORMAT_PM
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23
- **Minutes:** Value between Min_Data=0x00 and Max_Data=0x59
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)
- TimeFormat and Hours should follow the same format

Reference Manual to LL API cross reference:

- TR PM LL_RTC_TIME_Config
- TR HT LL_RTC_TIME_Config
- TR HU LL_RTC_TIME_Config
- TR MNT LL_RTC_TIME_Config
- TR MNU LL_RTC_TIME_Config
- TR ST LL_RTC_TIME_Config
- TR SU LL_RTC_TIME_Config

LL_RTC_TIME_Get

Function name

__STATIC_INLINE uint32_t LL_RTC_TIME_Get (RTC_TypeDef * RTCx)

Function description

Get time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of hours, minutes and seconds (Format: 0x00HHMMSS).

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- Read either RTC_SSR or RTC_TR locks the values in the higher-order calendar shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC, DR)).
- helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and __LL_RTC_GET_SECOND are available to get independently each parameter.

Reference Manual to LL API cross reference:

- TR HT LL_RTC_TIME_Get
- TR HU LL_RTC_TIME_Get
- TR MNT LL_RTC_TIME_Get
- TR MNU LL_RTC_TIME_Get
- TR ST LL_RTC_TIME_Get
- TR SU LL_RTC_TIME_Get

LL_RTC_TIME_EnableDayLightStore

Function name

__STATIC_INLINE void LL_RTC_TIME_EnableDayLightStore (RTC_TypeDef * RTCx)

Function description

Memorize whether the daylight saving time change has been performed.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR BKP LL_RTC_TIME_EnableDayLightStore
LL_RTC_TIME_DisableDayLightStore

Function name

__STATIC_INLINE void LL_RTC_TIME_DisableDayLightStore (RTC_TypeDef * RTCx)

Function description

Disable memorization whether the daylight saving time change has been performed.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR BKP LL_RTC_TIME_DisableDayLightStore
LL_RTC_TIME_IsDayLightStoreEnabled

Function name

__STATIC_INLINE uint32_t LL_RTC_TIME_IsDayLightStoreEnabled (RTC_TypeDef * RTCx)

Function description

Check if RTC Day Light Saving stored operation has been enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR BKP LL_RTC_TIME_IsDayLightStoreEnabled
LL_RTC_TIME_DecHour

Function name

__STATIC_INLINE void LL_RTC_TIME_DecHour (RTC_TypeDef * RTCx)

Function description

Subtract 1 hour (winter time change)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR SUB1H LL_RTC_TIME_DecHour
LL_RTC_TIME_IncHour

Function name

__STATIC_INLINE void LL_RTC_TIME_IncHour (RTC_TypeDef * RTCx)

Function description

Add 1 hour (summer time change)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ADD1H LL_RTC_TIME_IncHour
LL_RTC_DATE_SetYear

Function name

__STATIC_INLINE void LL_RTC_DATE_SetYear (RTC_TypeDef * RTCx, uint32_t Year)

Function description

Set Year in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Year:** Value between Min_Data=0x00 and Max_Data=0x99

Return values

- **None:**

Notes

- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Year from binary to BCD format

Reference Manual to LL API cross reference:

- DR YT LL_RTC_DATE_SetYear
- DR YU LL_RTC_DATE_SetYear
LL_RTC_DATE_GetYear

Function name

__STATIC_INLINE uint32_t LL_RTC_DATE_GetYear (RTC_TypeDef * RTCx)

Function description

Get Year in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x99

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Year from BCD to Binary format

Reference Manual to LL API cross reference:

- DR YT `LL_RTC_DATE_GetYear`
- DR YU `LL_RTC_DATE_GetYear`

`LL_RTC_DATE_SetWeekDay`

Function name

`__STATIC_INLINE void LL_RTC_DATE_SetWeekDay (RTC_TypeDef * RTCx, uint32_t WeekDay)`

Function description

Set Week day.

Parameters

- **RTCx:** RTC Instance
- **WeekDay:** This parameter can be one of the following values:
 - `LL_RTC_WEEKDAY_MONDAY`
 - `LL_RTC_WEEKDAY_TUESDAY`
 - `LL_RTC_WEEKDAY_WEDNESDAY`
 - `LL_RTC_WEEKDAY_THURSDAY`
 - `LL_RTC_WEEKDAY_FRIDAY`
 - `LL_RTC_WEEKDAY_SATURDAY`
 - `LL_RTC_WEEKDAY_SUNDAY`

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR WDU `LL_RTC_DATE_SetWeekDay`

`LL_RTC_DATE_GetWeekDay`

Function name

`__STATIC_INLINE uint32_t LL_RTC_DATE_GetWeekDay (RTC_TypeDef * RTCx)`

Function description

Get Week day.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit

Reference Manual to LL API cross reference:

- DR WDU LL_RTC_DATE_GetWeekDay

LL_RTC_DATE_SetMonth

Function name

__STATIC_INLINE void LL_RTC_DATE_SetMonth (RTC_TypeDef * RTCx, uint32_t Month)

Function description

Set Month in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Month:** This parameter can be one of the following values:
 - LL_RTC_MONTH_JANUARY
 - LL_RTC_MONTH_FEBRUARY
 - LL_RTC_MONTH_MARCH
 - LL_RTC_MONTH_APRIL
 - LL_RTC_MONTH_MAY
 - LL_RTC_MONTH_JUNE
 - LL_RTC_MONTH_JULY
 - LL_RTC_MONTH_AUGUST
 - LL_RTC_MONTH_SEPTEMBER
 - LL_RTC_MONTH_OCTOBER
 - LL_RTC_MONTH_NOVEMBER
 - LL_RTC_MONTH_DECEMBER

Return values

- **None:**

Notes

- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Month from binary to BCD format

Reference Manual to LL API cross reference:

- DR MT LL_RTC_DATE_SetMonth
- DR MU LL_RTC_DATE_SetMonth

LL_RTC_DATE_GetMonth

Function name

__STATIC_INLINE uint32_t LL_RTC_DATE_GetMonth (RTC_TypeDef * RTCx)

Function description

Get Month in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_MONTH_JANUARY
 - LL_RTC_MONTH_FEBRUARY
 - LL_RTC_MONTH_MARCH
 - LL_RTC_MONTH_APRIL
 - LL_RTC_MONTH_MAY
 - LL_RTC_MONTH_JUNE
 - LL_RTC_MONTH_JULY
 - LL_RTC_MONTH_AUGUST
 - LL_RTC_MONTH_SEPTEMBER
 - LL_RTC_MONTH_OCTOBER
 - LL_RTC_MONTH_NOVEMBER
 - LL_RTC_MONTH_DECEMBER

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Month from BCD to Binary format

Reference Manual to LL API cross reference:

- DR MT LL_RTC_DATE_GetMonth
- DR MU LL_RTC_DATE_GetMonth

`LL_RTC_DATE_SetDay`

Function name

`__STATIC_INLINE void LL_RTC_DATE_SetDay (RTC_TypeDef * RTCx, uint32_t Day)`

Function description

Set Day in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Day:** Value between Min_Data=0x01 and Max_Data=0x31

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Day from binary to BCD format

Reference Manual to LL API cross reference:

- DR DT LL_RTC_DATE_SetDay
- DR DU LL_RTC_DATE_SetDay

`LL_RTC_DATE_GetDay`

Function name

`__STATIC_INLINE uint32_t LL_RTC_DATE_GetDay (RTC_TypeDef * RTCx)`

Function description

Get Day in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x31

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Day from BCD to Binary format

Reference Manual to LL API cross reference:

- DR DT LL_RTC_DATE_GetDay
- DR DU LL_RTC_DATE_GetDay

`LL_RTC_DATE_Config`

Function name

`__STATIC_INLINE void LL_RTC_DATE_Config (RTC_TypeDef * RTCx, uint32_t WeekDay, uint32_t Day, uint32_t Month, uint32_t Year)`

Function description

Set date (WeekDay, Day, Month and Year) in BCD format.

Parameters

- **RTCx:** RTC Instance
- **WeekDay:** This parameter can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY
- **Day:** Value between Min_Data=0x01 and Max_Data=0x31
- **Month:** This parameter can be one of the following values:
 - LL_RTC_MONTH_JANUARY
 - LL_RTC_MONTH_FEBRUARY
 - LL_RTC_MONTH_MARCH
 - LL_RTC_MONTH_APRIL
 - LL_RTC_MONTH_MAY
 - LL_RTC_MONTH_JUNE
 - LL_RTC_MONTH_JULY
 - LL_RTC_MONTH_AUGUST
 - LL_RTC_MONTH_SEPTMBER
 - LL_RTC_MONTH_OCTOBER
 - LL_RTC_MONTH_NOVEMBER
 - LL_RTC_MONTH_DECEMBER
- **Year:** Value between Min_Data=0x00 and Max_Data=0x99

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR WDU LL_RTC_DATE_Config
- DR MT LL_RTC_DATE_Config
- DR MU LL_RTC_DATE_Config
- DR DT LL_RTC_DATE_Config
- DR DU LL_RTC_DATE_Config
- DR YT LL_RTC_DATE_Config
- DR YU LL_RTC_DATE_Config

LL_RTC_DATE_Get

Function name

__STATIC_INLINE uint32_t LL_RTC_DATE_Get (RTC_TypeDef * RTCx)

Function description

Get date (WeekDay, Day, Month and Year) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of WeekDay, Day, Month and Year (Format: 0xWWDDMMYY).

Notes

- if shadow mode is disabled (BYPHAD=0), need to check if RSF flag is set before reading this bit
- helper macros __LL_RTC_GET_WEEKDAY, __LL_RTC_GET_YEAR, __LL_RTC_GET_MONTH, and __LL_RTC_GET_DAY are available to get independently each parameter.

Reference Manual to LL API cross reference:

- DR WDU LL_RTC_DATE_Get
- DR MT LL_RTC_DATE_Get
- DR MU LL_RTC_DATE_Get
- DR DT LL_RTC_DATE_Get
- DR DU LL_RTC_DATE_Get
- DR YT LL_RTC_DATE_Get
- DR YU LL_RTC_DATE_Get

LL_RTC_ALMA_Enable

Function name

__STATIC_INLINE void LL_RTC_ALMA_Enable (RTC_TypeDef * RTCx)

Function description

Enable Alarm A.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRAE LL_RTC_ALMA_Enable
LL_RTC_ALMA_Disable

Function name

```
__STATIC_INLINE void LL_RTC_ALMA_Disable (RTC_TypeDef * RTCx)
```

Function description

Disable Alarm A.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRAE LL_RTC_ALMA_Disable
LL_RTC_ALMA_SetMask

Function name

```
__STATIC_INLINE void LL_RTC_ALMA_SetMask (RTC_TypeDef * RTCx, uint32_t Mask)
```

Function description

Specify the Alarm A masks.

Parameters

- **RTCx:** RTC Instance
- **Mask:** This parameter can be a combination of the following values:
 - LL_RTC_ALMA_MASK_NONE
 - LL_RTC_ALMA_MASK_DATEWEEKDAY
 - LL_RTC_ALMA_MASK_HOURS
 - LL_RTC_ALMA_MASK_MINUTES
 - LL_RTC_ALMA_MASK_SECONDS
 - LL_RTC_ALMA_MASK_ALL

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR MSK4 LL_RTC_ALMA_SetMask
- ALRMAR MSK3 LL_RTC_ALMA_SetMask
- ALRMAR MSK2 LL_RTC_ALMA_SetMask
- ALRMAR MSK1 LL_RTC_ALMA_SetMask
LL_RTC_ALMA_GetMask

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMA_GetMask (RTC_TypeDef * RTCx)
```

Function description

Get the Alarm A masks.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be can be a combination of the following values:
 - LL_RTC_ALMA_MASK_NONE
 - LL_RTC_ALMA_MASK_DATEWEEKDAY
 - LL_RTC_ALMA_MASK_HOURS
 - LL_RTC_ALMA_MASK_MINUTES
 - LL_RTC_ALMA_MASK_SECONDS
 - LL_RTC_ALMA_MASK_ALL

Reference Manual to LL API cross reference:

- ALRMAR MSK4 LL_RTC_ALMA_GetMask
- ALRMAR MSK3 LL_RTC_ALMA_GetMask
- ALRMAR MSK2 LL_RTC_ALMA_GetMask
- ALRMAR MSK1 LL_RTC_ALMA_GetMask

LL_RTC_ALMA_EnableWeekday

Function name

__STATIC_INLINE void LL_RTC_ALMA_EnableWeekday (RTC_TypeDef * RTCx)

Function description

Enable AlarmA Week day selection (DU[3:0] represents the week day.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR WDSSEL LL_RTC_ALMA_EnableWeekday

LL_RTC_ALMA_DisableWeekday

Function name

__STATIC_INLINE void LL_RTC_ALMA_DisableWeekday (RTC_TypeDef * RTCx)

Function description

Disable AlarmA Week day selection (DU[3:0] represents the date)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR WDSSEL LL_RTC_ALMA_DisableWeekday

LL_RTC_ALMA_SetDay

Function name

__STATIC_INLINE void LL_RTC_ALMA_SetDay (RTC_TypeDef * RTCx, uint32_t Day)

Function description

Set ALARM A Day in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Day:** Value between Min_Data=0x01 and Max_Data=0x31

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Day from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMAR DT `LL_RTC_ALMA_SetDay`
- ALRMAR DU `LL_RTC_ALMA_SetDay`

`LL_RTC_ALMA_GetDay`

Function name

`__STATIC_INLINE uint32_t LL_RTC_ALMA_GetDay (RTC_TypeDef * RTCx)`

Function description

Get ALARM A Day in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x31

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Day from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMAR DT `LL_RTC_ALMA_GetDay`
- ALRMAR DU `LL_RTC_ALMA_GetDay`

`LL_RTC_ALMA_SetWeekDay`

Function name

`__STATIC_INLINE void LL_RTC_ALMA_SetWeekDay (RTC_TypeDef * RTCx, uint32_t WeekDay)`

Function description

Set ALARM A Weekday.

Parameters

- **RTCx:** RTC Instance
- **WeekDay:** This parameter can be one of the following values:
 - `LL_RTC_WEEKDAY_MONDAY`
 - `LL_RTC_WEEKDAY_TUESDAY`
 - `LL_RTC_WEEKDAY_WEDNESDAY`
 - `LL_RTC_WEEKDAY_THURSDAY`
 - `LL_RTC_WEEKDAY_FRIDAY`
 - `LL_RTC_WEEKDAY_SATURDAY`
 - `LL_RTC_WEEKDAY_SUNDAY`

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR DU LL_RTC_ALMA_SetWeekDay
LL_RTC_ALMA_GetWeekDay

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMA_GetWeekDay (RTC_TypeDef * RTCx)

Function description

Get ALARM A Weekday.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

Reference Manual to LL API cross reference:

- ALRMAR DU LL_RTC_ALMA_GetWeekDay
LL_RTC_ALMA_SetTimeFormat

Function name

__STATIC_INLINE void LL_RTC_ALMA_SetTimeFormat (RTC_TypeDef * RTCx, uint32_t TimeFormat)

Function description

Set Alarm A time format (AM/24-hour or PM notation)

Parameters

- **RTCx:** RTC Instance
- **TimeFormat:** This parameter can be one of the following values:
 - LL_RTC_ALMA_TIME_FORMAT_AM
 - LL_RTC_ALMA_TIME_FORMAT_PM

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR PM LL_RTC_ALMA_SetTimeFormat
LL_RTC_ALMA_GetTimeFormat

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMA_GetTimeFormat (RTC_TypeDef * RTCx)

Function description

Get Alarm A time format (AM or PM notation)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_ALMA_TIME_FORMAT_AM
 - LL_RTC_ALMA_TIME_FORMAT_PM

Reference Manual to LL API cross reference:

- ALRMAR PM LL_RTC_ALMA_GetTimeFormat
LL_RTC_ALMA_SetHour

Function name

```
__STATIC_INLINE void LL_RTC_ALMA_SetHour (RTC_TypeDef * RTCx, uint32_t Hours)
```

Function description

Set ALARM A Hours in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Return values

- **None:**

Notes

- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Hours from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMAR HT LL_RTC_ALMA_SetHour
- ALRMAR HU LL_RTC_ALMA_SetHour
LL_RTC_ALMA_GetHour

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMA_GetHour (RTC_TypeDef * RTCx)
```

Function description

Get ALARM A Hours in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Notes

- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert Hours from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMAR HT LL_RTC_ALMA_GetHour
- ALRMAR HU LL_RTC_ALMA_GetHour
LL_RTC_ALMA_SetMinute

Function name

```
__STATIC_INLINE void LL_RTC_ALMA_SetMinute (RTC_TypeDef * RTCx, uint32_t Minutes)
```

Function description

Set ALARM A Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Minutes:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Minutes from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMAR MNT `LL_RTC_ALMA_SetMinute`
- ALRMAR MNU `LL_RTC_ALMA_SetMinute`

`LL_RTC_ALMA_GetMinute`

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMA_GetMinute (RTC_TypeDef * RTCx)
```

Function description

Get ALARM A Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Minutes from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMAR MNT `LL_RTC_ALMA_GetMinute`
- ALRMAR MNU `LL_RTC_ALMA_GetMinute`

`LL_RTC_ALMA_SetSecond`

Function name

```
__STATIC_INLINE void LL_RTC_ALMA_SetSecond (RTC_TypeDef * RTCx, uint32_t Seconds)
```

Function description

Set ALARM A Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Seconds from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMAR ST LL_RTC_ALMA_SetSecond
- ALRMAR SU LL_RTC_ALMA_SetSecond

LL_RTC_ALMA_GetSecond

Function name

`__STATIC_INLINE uint32_t LL_RTC_ALMA_GetSecond (RTC_TypeDef * RTCx)`

Function description

Get ALARM A Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Seconds from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMAR ST LL_RTC_ALMA_GetSecond
- ALRMAR SU LL_RTC_ALMA_GetSecond

LL_RTC_ALMA_ConfigTime

Function name

`__STATIC_INLINE void LL_RTC_ALMA_ConfigTime (RTC_TypeDef * RTCx, uint32_t Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)`

Function description

Set Alarm A Time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Format12_24:** This parameter can be one of the following values:
 - LL_RTC_ALMA_TIME_FORMAT_AM
 - LL_RTC_ALMA_TIME_FORMAT_PM
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23
- **Minutes:** Value between Min_Data=0x00 and Max_Data=0x59
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMAR PM LL_RTC_ALMA_ConfigTime
- ALRMAR HT LL_RTC_ALMA_ConfigTime
- ALRMAR HU LL_RTC_ALMA_ConfigTime
- ALRMAR MNT LL_RTC_ALMA_ConfigTime
- ALRMAR MNU LL_RTC_ALMA_ConfigTime
- ALRMAR ST LL_RTC_ALMA_ConfigTime
- ALRMAR SU LL_RTC_ALMA_ConfigTime

LL_RTC_ALMA_GetTime

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMA_GetTime (RTC_TypeDef * RTCx)

Function description

Get Alarm B Time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of hours, minutes and seconds.

Notes

- helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and __LL_RTC_GET_SECOND are available to get independently each parameter.

Reference Manual to LL API cross reference:

- ALRMAR HT LL_RTC_ALMA_GetTime
- ALRMAR HU LL_RTC_ALMA_GetTime
- ALRMAR MNT LL_RTC_ALMA_GetTime
- ALRMAR MNU LL_RTC_ALMA_GetTime
- ALRMAR ST LL_RTC_ALMA_GetTime
- ALRMAR SU LL_RTC_ALMA_GetTime

LL_RTC_ALMB_Enable

Function name

__STATIC_INLINE void LL_RTC_ALMB_Enable (RTC_TypeDef * RTCx)

Function description

Enable Alarm B.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRBE LL_RTC_ALMB_Enable

LL_RTC_ALMB_Disable

Function name

```
__STATIC_INLINE void LL_RTC_ALMB_Disable (RTC_TypeDef * RTCx)
```

Function description

Disable Alarm B.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRBE LL_RTC_ALMB_Disable

LL_RTC_ALMB_SetMask

Function name

```
__STATIC_INLINE void LL_RTC_ALMB_SetMask (RTC_TypeDef * RTCx, uint32_t Mask)
```

Function description

Specify the Alarm B masks.

Parameters

- **RTCx:** RTC Instance
- **Mask:** This parameter can be a combination of the following values:
 - LL_RTC_ALMB_MASK_NONE
 - LL_RTC_ALMB_MASK_DATEWEEKDAY
 - LL_RTC_ALMB_MASK_HOURS
 - LL_RTC_ALMB_MASK_MINUTES
 - LL_RTC_ALMB_MASK_SECONDS
 - LL_RTC_ALMB_MASK_ALL

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR MSK4 LL_RTC_ALMB_SetMask
- ALRMBR MSK3 LL_RTC_ALMB_SetMask
- ALRMBR MSK2 LL_RTC_ALMB_SetMask
- ALRMBR MSK1 LL_RTC_ALMB_SetMask

LL_RTC_ALMB_GetMask

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMB_GetMask (RTC_TypeDef * RTCx)
```

Function description

Get the Alarm B masks.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be can be a combination of the following values:
 - LL_RTC_ALMB_MASK_NONE
 - LL_RTC_ALMB_MASK_DATEWEEKDAY
 - LL_RTC_ALMB_MASK_HOURS
 - LL_RTC_ALMB_MASK_MINUTES
 - LL_RTC_ALMB_MASK_SECONDS
 - LL_RTC_ALMB_MASK_ALL

Reference Manual to LL API cross reference:

- ALRMBR MSK4 LL_RTC_ALMB_GetMask
- ALRMBR MSK3 LL_RTC_ALMB_GetMask
- ALRMBR MSK2 LL_RTC_ALMB_GetMask
- ALRMBR MSK1 LL_RTC_ALMB_GetMask

LL_RTC_ALMB_EnableWeekday

Function name

__STATIC_INLINE void LL_RTC_ALMB_EnableWeekday (RTC_TypeDef * RTCx)

Function description

Enable AlarmB Week day selection (DU[3:0] represents the week day.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR WDSEL LL_RTC_ALMB_EnableWeekday

LL_RTC_ALMB_DisableWeekday

Function name

__STATIC_INLINE void LL_RTC_ALMB_DisableWeekday (RTC_TypeDef * RTCx)

Function description

Disable AlarmB Week day selection (DU[3:0] represents the date)

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR WDSEL LL_RTC_ALMB_DisableWeekday

LL_RTC_ALMB_SetDay

Function name

__STATIC_INLINE void LL_RTC_ALMB_SetDay (RTC_TypeDef * RTCx, uint32_t Day)

Function description

Set ALARM B Day in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Day:** Value between Min_Data=0x01 and Max_Data=0x31

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Day from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMBR DT LL_RTC_ALMB_SetDay
- ALRMBR DU LL_RTC_ALMB_SetDay

LL_RTC_ALMB_GetDay

Function name

`__STATIC_INLINE uint32_t LL_RTC_ALMB_GetDay (RTC_TypeDef * RTCx)`

Function description

Get ALARM B Day in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x31

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Day from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMBR DT LL_RTC_ALMB_GetDay
- ALRMBR DU LL_RTC_ALMB_GetDay

LL_RTC_ALMB_SetWeekDay

Function name

`__STATIC_INLINE void LL_RTC_ALMB_SetWeekDay (RTC_TypeDef * RTCx, uint32_t WeekDay)`

Function description

Set ALARM B Weekday.

Parameters

- **RTCx:** RTC Instance
- **WeekDay:** This parameter can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR DU LL_RTC_ALMB_SetWeekDay
LL_RTC_ALMB_GetWeekDay

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMB_GetWeekDay (RTC_TypeDef * RTCx)

Function description

Get ALARM B Weekday.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

Reference Manual to LL API cross reference:

- ALRMBR DU LL_RTC_ALMB_GetWeekDay
LL_RTC_ALMB_SetTimeFormat

Function name

__STATIC_INLINE void LL_RTC_ALMB_SetTimeFormat (RTC_TypeDef * RTCx, uint32_t TimeFormat)

Function description

Set ALARM B time format (AM/24-hour or PM notation)

Parameters

- **RTCx:** RTC Instance
- **TimeFormat:** This parameter can be one of the following values:
 - LL_RTC_ALMB_TIME_FORMAT_AM
 - LL_RTC_ALMB_TIME_FORMAT_PM

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR PM LL_RTC_ALMB_SetTimeFormat
LL_RTC_ALMB_GetTimeFormat

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMB_GetTimeFormat (RTC_TypeDef * RTCx)

Function description

Get ALARM B time format (AM or PM notation)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_ALMB_TIME_FORMAT_AM
 - LL_RTC_ALMB_TIME_FORMAT_PM

Reference Manual to LL API cross reference:

- ALRMBR PM LL_RTC_ALMB_GetTimeFormat
- LL_RTC_ALMB_SetHour

Function name

```
__STATIC_INLINE void LL_RTC_ALMB_SetHour (RTC_TypeDef * RTCx, uint32_t Hours)
```

Function description

Set ALARM B Hours in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Return values

- **None:**

Notes

- helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert Hours from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMBR HT LL_RTC_ALMB_SetHour
- ALRMBR HU LL_RTC_ALMB_SetHour
- LL_RTC_ALMB_GetHour

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMB_GetHour (RTC_TypeDef * RTCx)
```

Function description

Get ALARM B Hours in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Notes

- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert Hours from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMBR HT LL_RTC_ALMB_GetHour
- ALRMBR HU LL_RTC_ALMB_GetHour
- LL_RTC_ALMB_SetMinute

Function name

```
__STATIC_INLINE void LL_RTC_ALMB_SetMinute (RTC_TypeDef * RTCx, uint32_t Minutes)
```

Function description

Set ALARM B Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Minutes:** between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Minutes from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMBR MNT LL_RTC_ALMB_SetMinute
- ALRMBR MNU LL_RTC_ALMB_SetMinute

LL_RTC_ALMB_GetMinute

Function name

`__STATIC_INLINE uint32_t LL_RTC_ALMB_GetMinute (RTC_TypeDef * RTCx)`

Function description

Get ALARM B Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Minutes from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMBR MNT LL_RTC_ALMB_GetMinute
- ALRMBR MNU LL_RTC_ALMB_GetMinute

LL_RTC_ALMB_SetSecond

Function name

`__STATIC_INLINE void LL_RTC_ALMB_SetSecond (RTC_TypeDef * RTCx, uint32_t Seconds)`

Function description

Set ALARM B Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Notes

- helper macro `__LL_RTC_CONVERT_BIN2BCD` is available to convert Seconds from binary to BCD format

Reference Manual to LL API cross reference:

- ALRMBR ST LL_RTC_ALMB_SetSecond
- ALRMBR SU LL_RTC_ALMB_SetSecond

LL_RTC_ALMB_GetSecond

Function name

__STATIC_INLINE uint32_t LL_RTC_ALMB_GetSecond (RTC_TypeDef * RTCx)

Function description

Get ALARM B Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Seconds from BCD to Binary format

Reference Manual to LL API cross reference:

- ALRMBR ST LL_RTC_ALMB_GetSecond
- ALRMBR SU LL_RTC_ALMB_GetSecond

LL_RTC_ALMB_ConfigTime

Function name

__STATIC_INLINE void LL_RTC_ALMB_ConfigTime (RTC_TypeDef * RTCx, uint32_t Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)

Function description

Set Alarm B Time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance
- **Format12_24:** This parameter can be one of the following values:
 - LL_RTC_ALMB_TIME_FORMAT_AM
 - LL_RTC_ALMB_TIME_FORMAT_PM
- **Hours:** Value between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23
- **Minutes:** Value between Min_Data=0x00 and Max_Data=0x59
- **Seconds:** Value between Min_Data=0x00 and Max_Data=0x59

Return values

- **None:**

Reference Manual to LL API cross reference:

- ALRMBR PM LL_RTC_ALMB_ConfigTime
- ALRMBR HT LL_RTC_ALMB_ConfigTime
- ALRMBR HU LL_RTC_ALMB_ConfigTime
- ALRMBR MNT LL_RTC_ALMB_ConfigTime
- ALRMBR MNU LL_RTC_ALMB_ConfigTime
- ALRMBR ST LL_RTC_ALMB_ConfigTime
- ALRMBR SU LL_RTC_ALMB_ConfigTime

```
LL_RTC_ALMB_GetTime
```

Function name

```
__STATIC_INLINE uint32_t LL_RTC_ALMB_GetTime (RTC_TypeDef * RTCx)
```

Function description

Get Alarm B Time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of hours, minutes and seconds.

Notes

- helper macros `__LL_RTC_GET_HOUR`, `__LL_RTC_GET_MINUTE` and `__LL_RTC_GET_SECOND` are available to get independently each parameter.

Reference Manual to LL API cross reference:

- ALRMBR HT LL_RTC_ALMB_GetTime
- ALRMBR HU LL_RTC_ALMB_GetTime
- ALRMBR MNT LL_RTC_ALMB_GetTime
- ALRMBR MNU LL_RTC_ALMB_GetTime
- ALRMBR ST LL_RTC_ALMB_GetTime
- ALRMBR SU LL_RTC_ALMB_GetTime

```
LL_RTC_TS_Enable
```

Function name

```
__STATIC_INLINE void LL_RTC_TS_Enable (RTC_TypeDef * RTCx)
```

Function description

Enable Timestamp.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. `LL_RTC_DisableWriteProtection` function should be called before.

Reference Manual to LL API cross reference:

- CR TSE LL_RTC_TS_Enable

```
LL_RTC_TS_Disable
```

Function name

```
__STATIC_INLINE void LL_RTC_TS_Disable (RTC_TypeDef * RTCx)
```

Function description

Disable Timestamp.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR TSE LL_RTC_TS_Disable
LL_RTC_TS_SetActiveEdge

Function name

__STATIC_INLINE void LL_RTC_TS_SetActiveEdge (RTC_TypeDef * RTCx, uint32_t Edge)

Function description

Set Time-stamp event active edge.

Parameters

- **RTCx:** RTC Instance
- **Edge:** This parameter can be one of the following values:
 - LL_RTC_TIMESTAMP_EDGE_RISING
 - LL_RTC_TIMESTAMP_EDGE_FALLING

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- TSE must be reset when TSEDGE is changed to avoid unwanted TSF setting

Reference Manual to LL API cross reference:

- CR TSEDGE LL_RTC_TS_SetActiveEdge
LL_RTC_TS_GetActiveEdge

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetActiveEdge (RTC_TypeDef * RTCx)

Function description

Get Time-stamp event active edge.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_TIMESTAMP_EDGE_RISING
 - LL_RTC_TIMESTAMP_EDGE_FALLING

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR TSEDGE LL_RTC_TS_GetActiveEdge
LL_RTC_TS_GetTimeFormat

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TS_GetTimeFormat (RTC_TypeDef * RTCx)
```

Function description

Get Timestamp AM/PM notation (AM or 24-hour format)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_TS_TIME_FORMAT_AM
 - LL_RTC_TS_TIME_FORMAT_PM

Reference Manual to LL API cross reference:

- TSTR PM LL_RTC_TS_GetTimeFormat
LL_RTC_TS_GetHour

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TS_GetHour (RTC_TypeDef * RTCx)
```

Function description

Get Timestamp Hours in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x12 or between Min_Data=0x00 and Max_Data=0x23

Notes

- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert Hours from BCD to Binary format

Reference Manual to LL API cross reference:

- TSTR HT LL_RTC_TS_GetHour
- TSTR HU LL_RTC_TS_GetHour
LL_RTC_TS_GetMinute

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TS_GetMinute (RTC_TypeDef * RTCx)
```

Function description

Get Timestamp Minutes in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert Minutes from BCD to Binary format

Reference Manual to LL API cross reference:

- TSTR MNT LL_RTC_TS_GetMinute
- TSTR MNU LL_RTC_TS_GetMinute

LL_RTC_TS_GetSecond

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetSecond (RTC_TypeDef * RTCx)

Function description

Get Timestamp Seconds in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x59

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Seconds from BCD to Binary format

Reference Manual to LL API cross reference:

- TSTR ST LL_RTC_TS_GetSecond
- TSTR SU LL_RTC_TS_GetSecond

LL_RTC_TS_GetTime

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetTime (RTC_TypeDef * RTCx)

Function description

Get Timestamp time (hour, minute and second) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of hours, minutes and seconds.

Notes

- helper macros `__LL_RTC_GET_HOUR`, `__LL_RTC_GET_MINUTE` and `__LL_RTC_GET_SECOND` are available to get independently each parameter.

Reference Manual to LL API cross reference:

- TSTR HT LL_RTC_TS_GetTime
- TSTR HU LL_RTC_TS_GetTime
- TSTR MNT LL_RTC_TS_GetTime
- TSTR MNU LL_RTC_TS_GetTime
- TSTR ST LL_RTC_TS_GetTime
- TSTR SU LL_RTC_TS_GetTime

LL_RTC_TS_GetWeekDay

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetWeekDay (RTC_TypeDef * RTCx)

Function description

Get Timestamp Week day.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

Reference Manual to LL API cross reference:

- TSDR WDU LL_RTC_TS_GetWeekDay

LL_RTC_TS_GetMonth

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetMonth (RTC_TypeDef * RTCx)

Function description

Get Timestamp Month in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_MONTH_JANUARY
 - LL_RTC_MONTH_FEBRUARY
 - LL_RTC_MONTH_MARCH
 - LL_RTC_MONTH_APRIL
 - LL_RTC_MONTH_MAY
 - LL_RTC_MONTH_JUNE
 - LL_RTC_MONTH_JULY
 - LL_RTC_MONTH_AUGUST
 - LL_RTC_MONTH_SEPTMBER
 - LL_RTC_MONTH_OCTOBER
 - LL_RTC_MONTH_NOVEMBER
 - LL_RTC_MONTH_DECEMBER

Notes

- helper macro **__LL_RTC_CONVERT_BCD2BIN** is available to convert Month from BCD to Binary format

Reference Manual to LL API cross reference:

- TSDR MT LL_RTC_TS_GetMonth
- TSDR MU LL_RTC_TS_GetMonth

LL_RTC_TS_GetDay

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TS_GetDay (RTC_TypeDef * RTCx)
```

Function description

Get Timestamp Day in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x01 and Max_Data=0x31

Notes

- helper macro `__LL_RTC_CONVERT_BCD2BIN` is available to convert Day from BCD to Binary format

Reference Manual to LL API cross reference:

- TSDR DT LL_RTC_TS_GetDay
- TSDR DU LL_RTC_TS_GetDay

```
LL_RTC_TS_GetDate
```

Function name

```
__STATIC_INLINE uint32_t LL_RTC_TS_GetDate (RTC_TypeDef * RTCx)
```

Function description

Get Timestamp date (WeekDay, Day and Month) in BCD format.

Parameters

- **RTCx:** RTC Instance

Return values

- **Combination:** of Weekday, Day and Month

Notes

- helper macros `__LL_RTC_GET_WEEKDAY`, `__LL_RTC_GET_MONTH`, and `__LL_RTC_GET_DAY` are available to get independently each parameter.

Reference Manual to LL API cross reference:

- TSDR WDU LL_RTC_TS_GetDate
- TSDR MT LL_RTC_TS_GetDate
- TSDR MU LL_RTC_TS_GetDate
- TSDR DT LL_RTC_TS_GetDate
- TSDR DU LL_RTC_TS_GetDate

```
LL_RTC_TS_SetPin
```

Function name

```
__STATIC_INLINE void LL_RTC_TS_SetPin (RTC_TypeDef * RTCx, uint32_t TSPin)
```

Function description

Set timestamp Pin.

Parameters

- **RTCx:** RTC Instance
- **TSPin:** specifies the RTC TimeStamp Pin. This parameter can be one of the following values:
 - `LL_RTC_TimeStampPin_Default`: RTC_AF1 is used as RTC TimeStamp.
 - `LL_RTC_TimeStampPin_Pos1`: RTC_AF2 is selected as RTC TimeStamp.

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TSINSEL LL_RTC_TS_SetPin
LL_RTC_TS_GetPin

Function name

__STATIC_INLINE uint32_t LL_RTC_TS_GetPin (RTC_TypeDef * RTCx)

Function description

Get timestamp Pin.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_TimeStampPin_Default: RTC_AF1 is used as RTC TimeStamp Pin.
 - LL_RTC_TimeStampPin_Pos1: RTC_AF2 is selected as RTC TimeStamp Pin.
- **None:**

Reference Manual to LL API cross reference:

- TAFCR TSINSEL LL_RTC_TS_GetPin
LL_RTC_TAMPER_Enable

Function name

__STATIC_INLINE void LL_RTC_TAMPER_Enable (RTC_TypeDef * RTCx, uint32_t Tamper)

Function description

Enable RTC_TAMPx input detection.

Parameters

- **RTCx:** RTC Instance
- **Tamper:** This parameter can be a combination of the following values:
 - LL_RTC_TAMPER_1

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1E LL_RTC_TAMPER_Enable
- LL_RTC_TAMPER_Disable

Function name

__STATIC_INLINE void LL_RTC_TAMPER_Disable (RTC_TypeDef * RTCx, uint32_t Tamper)

Function description

Clear RTC_TAMPx input detection.

Parameters

- **RTCx:** RTC Instance
- **Tamper:** This parameter can be a combination of the following values:
 - LL_RTC_TAMPER_1

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1E LL_RTC_TAMPER_Disable
-

LL_RTC_TAMPER_EnableActiveLevel

Function name

__STATIC_INLINE void LL_RTC_TAMPER_EnableActiveLevel (RTC_TypeDef * RTCx, uint32_t Tamper)

Function description

Enable Active level for Tamper input.

Parameters

- **RTCx:** RTC Instance
- **Tamper:** This parameter can be a combination of the following values:
 - LL_RTC_TAMPER_ACTIVELEVEL_TAMP1

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1TRG LL_RTC_TAMPER_EnableActiveLevel
-

LL_RTC_TAMPER_DisableActiveLevel

Function name

__STATIC_INLINE void LL_RTC_TAMPER_DisableActiveLevel (RTC_TypeDef * RTCx, uint32_t Tamper)

Function description

Disable Active level for Tamper input.

Parameters

- **RTCx:** RTC Instance
- **Tamper:** This parameter can be a combination of the following values:
 - LL_RTC_TAMPER_ACTIVELEVEL_TAMP1

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1TRG LL_RTC_TAMPER_DisableActiveLevel
-

LL_RTC_TAMPER_SetPin

Function name

__STATIC_INLINE void LL_RTC_TAMPER_SetPin (RTC_TypeDef * RTCx, uint32_t TamperPin)

Function description

Set Tamper Pin.

Parameters

- **RTCx:** RTC Instance
- **TamperPin:** specifies the RTC Tamper Pin. This parameter can be one of the following values:
 - LL_RTC_TamperPin_Default: RTC_AF1 is used as RTC Tamper.
 - LL_RTC_TamperPin_Pos1: RTC_AF2 is selected as RTC Tamper.

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1INSEL LL_RTC_TAMPER_SetPin

LL_RTC_TAMPER_GetPin

Function name

__STATIC_INLINE uint32_t LL_RTC_TAMPER_GetPin (RTC_TypeDef * RTCx)

Function description

Get Tamper Pin.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_TamperPin_Default: RTC_AF1 is used as RTC Tamper Pin.
 - LL_RTC_TamperPin_Pos1: RTC_AF2 is selected as RTC Tamper Pin.
- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMP1INSEL LL_RTC_TAMPER_GetPin

LL_RTC_WAKEUP_Enable

Function name

__STATIC_INLINE void LL_RTC_WAKEUP_Enable (RTC_TypeDef * RTCx)

Function description

Enable Wakeup timer.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR WUTE LL_RTC_WAKEUP_Enable

LL_RTC_WAKEUP_Disable

Function name

```
__STATIC_INLINE void LL_RTC_WAKEUP_Disable (RTC_TypeDef * RTCx)
```

Function description

Disable Wakeup timer.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR WUTE LL_RTC_WAKEUP_Disable

LL_RTC_WAKEUP_IsEnabled

Function name

```
__STATIC_INLINE uint32_t LL_RTC_WAKEUP_IsEnabled (RTC_TypeDef * RTCx)
```

Function description

Check if Wakeup timer is enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR WUTE LL_RTC_WAKEUP_IsEnabled

LL_RTC_WAKEUP_SetClock

Function name

```
__STATIC_INLINE void LL_RTC_WAKEUP_SetClock (RTC_TypeDef * RTCx, uint32_t WakeupClock)
```

Function description

Select Wakeup clock.

Parameters

- **RTCx:** RTC Instance
- **WakeupClock:** This parameter can be one of the following values:
 - LL_RTC_WAKEUPCLOCK_DIV_16
 - LL_RTC_WAKEUPCLOCK_DIV_8
 - LL_RTC_WAKEUPCLOCK_DIV_4
 - LL_RTC_WAKEUPCLOCK_DIV_2
 - LL_RTC_WAKEUPCLOCK_CKSPRE
 - LL_RTC_WAKEUPCLOCK_CKSPRE_WUT

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- Bit can be written only when RTC_CR WUTE bit = 0 and RTC_ISR WUTWF bit = 1

Reference Manual to LL API cross reference:

- CR WUCKSEL LL_RTC_WAKEUP_SetClock

LL_RTC_WAKEUP_GetClock

Function name

__STATIC_INLINE uint32_t LL_RTC_WAKEUP_GetClock (RTC_TypeDef * RTCx)

Function description

Get Wakeup clock.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_WAKEUPCLOCK_DIV_16
 - LL_RTC_WAKEUPCLOCK_DIV_8
 - LL_RTC_WAKEUPCLOCK_DIV_4
 - LL_RTC_WAKEUPCLOCK_DIV_2
 - LL_RTC_WAKEUPCLOCK_CKSPRE
 - LL_RTC_WAKEUPCLOCK_CKSPRE_WUT

Reference Manual to LL API cross reference:

- CR WUCKSEL LL_RTC_WAKEUP_GetClock

LL_RTC_WAKEUP_SetAutoReload

Function name

__STATIC_INLINE void LL_RTC_WAKEUP_SetAutoReload (RTC_TypeDef * RTCx, uint32_t Value)

Function description

Set Wakeup auto-reload value.

Parameters

- **RTCx:** RTC Instance
- **Value:** Value between Min_Data=0x00 and Max_Data=0xFFFF

Return values

- **None:**

Notes

- Bit can be written only when WUTWF is set to 1 in RTC_ISR

Reference Manual to LL API cross reference:

- WUTR WUT LL_RTC_WAKEUP_SetAutoReload

LL_RTC_WAKEUP_GetAutoReload

Function name

__STATIC_INLINE uint32_t LL_RTC_WAKEUP_GetAutoReload (RTC_TypeDef * RTCx)

Function description

Get Wakeup auto-reload value.

Parameters

- **RTCx:** RTC Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFFFF

Reference Manual to LL API cross reference:

- WUTR WUT LL_RTC_WAKEUP_GetAutoReload
LL_RTC_BAK_SetRegister

Function name

```
__STATIC_INLINE void LL_RTC_BAK_SetRegister (RTC_TypeDef * RTCx, uint32_t BackupRegister, uint32_t Data)
```

Function description

Writes a data in a specified RTC Backup data register.

Parameters

- **RTCx:** RTC Instance
- **BackupRegister:** This parameter can be one of the following values:
 - LL_RTC_BKP_DR0
 - LL_RTC_BKP_DR1
 - LL_RTC_BKP_DR2
 - LL_RTC_BKP_DR3
 - LL_RTC_BKP_DR4
 - LL_RTC_BKP_DR5
 - LL_RTC_BKP_DR6
 - LL_RTC_BKP_DR7
 - LL_RTC_BKP_DR8
 - LL_RTC_BKP_DR9
 - LL_RTC_BKP_DR10
 - LL_RTC_BKP_DR11
 - LL_RTC_BKP_DR12
 - LL_RTC_BKP_DR13
 - LL_RTC_BKP_DR14
 - LL_RTC_BKP_DR15
 - LL_RTC_BKP_DR16
 - LL_RTC_BKP_DR17
 - LL_RTC_BKP_DR18
 - LL_RTC_BKP_DR19
- **Data:** Value between Min_Data=0x00 and Max_Data=0xFFFFFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- BKPxR BKP LL_RTC_BAK_SetRegister
LL_RTC_BAK_GetRegister

Function name

```
__STATIC_INLINE uint32_t LL_RTC_BAK_GetRegister (RTC_TypeDef * RTCx, uint32_t BackupRegister)
```

Function description

Reads data from the specified RTC Backup data Register.

Parameters

- **RTCx:** RTC Instance
- **BackupRegister:** This parameter can be one of the following values:
 - LL_RTC_BKP_DR0
 - LL_RTC_BKP_DR1
 - LL_RTC_BKP_DR2
 - LL_RTC_BKP_DR3
 - LL_RTC_BKP_DR4
 - LL_RTC_BKP_DR5
 - LL_RTC_BKP_DR6
 - LL_RTC_BKP_DR7
 - LL_RTC_BKP_DR8
 - LL_RTC_BKP_DR9
 - LL_RTC_BKP_DR10
 - LL_RTC_BKP_DR11
 - LL_RTC_BKP_DR12
 - LL_RTC_BKP_DR13
 - LL_RTC_BKP_DR14
 - LL_RTC_BKP_DR15
 - LL_RTC_BKP_DR16
 - LL_RTC_BKP_DR17
 - LL_RTC_BKP_DR18
 - LL_RTC_BKP_DR19

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFFFFFFFF

Reference Manual to LL API cross reference:

- BKPxR BKP LL_RTC_BAK_GetRegister

LL_RTC_CAL_SetOutputFreq

Function name

```
__STATIC_INLINE void LL_RTC_CAL_SetOutputFreq (RTC_TypeDef * RTCx, uint32_t Frequency)
```

Function description

Set Calibration output frequency (1 Hz or 512 Hz)

Parameters

- **RTCx:** RTC Instance
- **Frequency:** This parameter can be one of the following values:
 - LL_RTC_CALIB_OUTPUT_NONE
 - LL_RTC_CALIB_OUTPUT_512HZ

Return values

- **None:**

Notes

- Bits are write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR COE LL_RTC_CAL_SetOutputFreq
- CR COSEL LL_RTC_CAL_SetOutputFreq

LL_RTC_CAL_GetOutputFreq

Function name

__STATIC_INLINE uint32_t LL_RTC_CAL_GetOutputFreq (RTC_TypeDef * RTCx)

Function description

Get Calibration output frequency (1 Hz or 512 Hz)

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_CALIB_OUTPUT_NONE
 - LL_RTC_CALIB_OUTPUT_512HZ

Reference Manual to LL API cross reference:

- CR COE LL_RTC_CAL_GetOutputFreq
- CR COSEL LL_RTC_CAL_GetOutputFreq

LL_RTC_CAL_EnableCoarseDigital

Function name

__STATIC_INLINE void LL_RTC_CAL_EnableCoarseDigital (RTC_TypeDef * RTCx)

Function description

Enable Coarse digital calibration.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- CR DCE LL_RTC_CAL_EnableCoarseDigital

LL_RTC_CAL_DisableCoarseDigital

Function name

__STATIC_INLINE void LL_RTC_CAL_DisableCoarseDigital (RTC_TypeDef * RTCx)

Function description

Disable Coarse digital calibration.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)

Reference Manual to LL API cross reference:

- CR DCE LL_RTC_CAL_DisableCoarseDigital
LL_RTC_CAL_ConfigCoarseDigital

Function name

__STATIC_INLINE void LL_RTC_CAL_ConfigCoarseDigital (RTC_TypeDef * RTCx, uint32_t Sign, uint32_t Value)

Function description

Set the coarse digital calibration.

Parameters

- **RTCx:** RTC Instance
- **Sign:** This parameter can be one of the following values:
 - LL_RTC_CALIB_SIGN_POSITIVE
 - LL_RTC_CALIB_SIGN_NEGATIVE
- **Value:** value of coarse calibration expressed in ppm (coded on 5 bits)

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.
- It can be written in initialization mode only (LL_RTC_EnableInitMode function)
- This Calibration value should be between 0 and 63 when using negative sign with a 2-ppm step.
- This Calibration value should be between 0 and 126 when using positive sign with a 4-ppm step.

Reference Manual to LL API cross reference:

- CALIBR DCS LL_RTC_CAL_ConfigCoarseDigital
- CALIBR DC LL_RTC_CAL_ConfigCoarseDigital
LL_RTC_CAL_GetCoarseDigitalValue

Function name

__STATIC_INLINE uint32_t LL_RTC_CAL_GetCoarseDigitalValue (RTC_TypeDef * RTCx)

Function description

Get the coarse digital calibration value.

Parameters

- **RTCx:** RTC Instance

Return values

- **value:** of coarse calibration expressed in ppm (coded on 5 bits)

Reference Manual to LL API cross reference:

- CALIBR DC LL_RTC_CAL_GetCoarseDigitalValue
LL_RTC_CAL_GetCoarseDigitalSign

Function name

```
__STATIC_INLINE uint32_t LL_RTC_CAL_GetCoarseDigitalSign (RTC_TypeDef * RTCx)
```

Function description

Get the coarse digital calibration sign.

Parameters

- **RTCx:** RTC Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_RTC_CALIB_SIGN_POSITIVE
 - LL_RTC_CALIB_SIGN_NEGATIVE

Reference Manual to LL API cross reference:

- CALIBR DCS LL_RTC_CAL_GetCoarseDigitalSign
LL_RTC_IsActiveFlag_TAMP1

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TAMP1 (RTC_TypeDef * RTCx)
```

Function description

Get RTC_TAMP1 detection flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR TAMP1F LL_RTC_IsActiveFlag_TAMP1
LL_RTC_IsActiveFlag_TSOV

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TSOV (RTC_TypeDef * RTCx)
```

Function description

Get Time-stamp overflow flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR TSOVF LL_RTC_IsActiveFlag_TSOV
LL_RTC_IsActiveFlag_TS

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TS (RTC_TypeDef * RTCx)
```

Function description

Get Time-stamp flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR TSF LL_RTC_IsActiveFlag_TS
LL_RTC_IsActiveFlag_WUT

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_WUT (RTC_TypeDef * RTCx)
```

Function description

Get Wakeup timer flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR WUTF LL_RTC_IsActiveFlag_WUT
LL_RTC_IsActiveFlag_ALRB

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRB (RTC_TypeDef * RTCx)
```

Function description

Get Alarm B flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR ALRBF LL_RTC_IsActiveFlag_ALRB
LL_RTC_IsActiveFlag_ALRA

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRA (RTC_TypeDef * RTCx)
```

Function description

Get Alarm A flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR ALRAF LL_RTC_IsActiveFlag_ALRA
- LL_RTC_ClearFlag_TAMP1

Function name

```
__STATIC_INLINE void LL_RTC_ClearFlag_TAMP1 (RTC_TypeDef * RTCx)
```

Function description

Clear RTC_TAMP1 detection flag.

Parameters

- RTCx:** RTC Instance

Return values

- None:**

Reference Manual to LL API cross reference:

- ISR TAMP1F LL_RTC_ClearFlag_TAMP1
- LL_RTC_ClearFlag_TSOV

Function name

```
__STATIC_INLINE void LL_RTC_ClearFlag_TSOV (RTC_TypeDef * RTCx)
```

Function description

Clear Time-stamp overflow flag.

Parameters

- RTCx:** RTC Instance

Return values

- None:**

Reference Manual to LL API cross reference:

- ISR TSOVF LL_RTC_ClearFlag_TSOV
- LL_RTC_ClearFlag_TS

Function name

```
__STATIC_INLINE void LL_RTC_ClearFlag_TS (RTC_TypeDef * RTCx)
```

Function description

Clear Time-stamp flag.

Parameters

- RTCx:** RTC Instance

Return values

- None:**

Reference Manual to LL API cross reference:

- ISR TSF LL_RTC_ClearFlag_TS
- LL_RTC_ClearFlag_WUT

Function name

```
__STATIC_INLINE void LL_RTC_ClearFlag_WUT (RTC_TypeDef * RTCx)
```

Function description

Clear Wakeup timer flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ISR WUTF LL_RTC_ClearFlag_WUT
LL_RTC_ClearFlag_ALRB

Function name

__STATIC_INLINE void LL_RTC_ClearFlag_ALRB (RTC_TypeDef * RTCx)

Function description

Clear Alarm B flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ISR ALRBF LL_RTC_ClearFlag_ALRB
LL_RTC_ClearFlag_ALRA

Function name

__STATIC_INLINE void LL_RTC_ClearFlag_ALRA (RTC_TypeDef * RTCx)

Function description

Clear Alarm A flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ISR ALRAF LL_RTC_ClearFlag_ALRA
LL_RTC_IsActiveFlag_INIT

Function name

__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_INIT (RTC_TypeDef * RTCx)

Function description

Get Initialization flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR INITF LL_RTC_IsActiveFlag_INIT
LL_RTC_IsActiveFlag_RS

Function name

__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_RS (RTC_TypeDef * RTCx)

Function description

Get Registers synchronization flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR RSF LL_RTC_IsActiveFlag_RS
LL_RTC_ClearFlag_RS

Function name

__STATIC_INLINE void LL_RTC_ClearFlag_RS (RTC_TypeDef * RTCx)

Function description

Clear Registers synchronization flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- ISR RSF LL_RTC_ClearFlag_RS
LL_RTC_IsActiveFlag_INITS

Function name

__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_INITS (RTC_TypeDef * RTCx)

Function description

Get Initialization status flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR INITS LL_RTC_IsActiveFlag_INITS
LL_RTC_IsActiveFlag_WUTW

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_WUTW (RTC_TypeDef * RTCx)
```

Function description

Get Wakeup timer write flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR WUTWF LL_RTC_IsActiveFlag_WUTW

LL_RTC_IsActiveFlag_ALRBW

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRBW (RTC_TypeDef * RTCx)
```

Function description

Get Alarm B write flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR ALRBWF LL_RTC_IsActiveFlag_ALRBW

LL_RTC_IsActiveFlag_ALRAW

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRAW (RTC_TypeDef * RTCx)
```

Function description

Get Alarm A write flag.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- ISR ALRAWF LL_RTC_IsActiveFlag_ALRAW

LL_RTC_EnableIT_TS

Function name

```
__STATIC_INLINE void LL_RTC_EnableIT_TS (RTC_TypeDef * RTCx)
```

Function description

Enable Time-stamp interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR TSIE LL_RTC_EnableIT_TS

LL_RTC_DisableIT_TS

Function name

__STATIC_INLINE void LL_RTC_DisableIT_TS (RTC_TypeDef * RTCx)

Function description

Disable Time-stamp interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR TSIE LL_RTC_DisableIT_TS

LL_RTC_EnableIT_WUT

Function name

__STATIC_INLINE void LL_RTC_EnableIT_WUT (RTC_TypeDef * RTCx)

Function description

Enable Wakeup timer interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR WUTIE LL_RTC_EnableIT_WUT

LL_RTC_DisableIT_WUT

Function name

__STATIC_INLINE void LL_RTC_DisableIT_WUT (RTC_TypeDef * RTCx)

Function description

Disable Wakeup timer interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR WUTIE LL_RTC_DisableIT_WUT

LL_RTC_EnableIT_ALRB

Function name

__STATIC_INLINE void LL_RTC_EnableIT_ALRB (RTC_TypeDef * RTCx)

Function description

Enable Alarm B interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRBIE LL_RTC_EnableIT_ALRB

LL_RTC_DisableIT_ALRB

Function name

__STATIC_INLINE void LL_RTC_DisableIT_ALRB (RTC_TypeDef * RTCx)

Function description

Disable Alarm B interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRBIE LL_RTC_DisableIT_ALRB

LL_RTC_EnableIT_ALRA

Function name

```
__STATIC_INLINE void LL_RTC_EnableIT_ALRA (RTC_TypeDef * RTCx)
```

Function description

Enable Alarm A interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRAIE LL_RTC_EnableIT_ALRA

```
LL_RTC_DisableIT_ALRA
```

Function name

```
__STATIC_INLINE void LL_RTC_DisableIT_ALRA (RTC_TypeDef * RTCx)
```

Function description

Disable Alarm A interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Notes

- Bit is write-protected. LL_RTC_DisableWriteProtection function should be called before.

Reference Manual to LL API cross reference:

- CR ALRAIE LL_RTC_DisableIT_ALRA

```
LL_RTC_EnableIT_TAMP
```

Function name

```
__STATIC_INLINE void LL_RTC_EnableIT_TAMP (RTC_TypeDef * RTCx)
```

Function description

Enable all Tamper Interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMPIE LL_RTC_EnableIT_TAMP

```
LL_RTC_DisableIT_TAMP
```

Function name

```
__STATIC_INLINE void LL_RTC_DisableIT_TAMP (RTC_TypeDef * RTCx)
```

Function description

Disable all Tamper Interrupt.

Parameters

- **RTCx:** RTC Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- TAFCR TAMPIE LL_RTC_DisableIT_TAMP

```
LL_RTC_IsEnabledIT_TS
```

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_TS (RTC_TypeDef * RTCx)
```

Function description

Check if Time-stamp interrupt is enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR TSIE LL_RTC_IsEnabledIT_TS

```
LL_RTC_IsEnabledIT_WUT
```

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_WUT (RTC_TypeDef * RTCx)
```

Function description

Check if Wakeup timer interrupt is enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR WUTIE LL_RTC_IsEnabledIT_WUT

```
LL_RTC_IsEnabledIT_ALRB
```

Function name

```
__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_ALRB (RTC_TypeDef * RTCx)
```

Function description

Check if Alarm B interrupt is enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR ALRBIE LL_RTC_IsEnabledIT_ALRB
LL_RTC_IsEnabledIT_ALRA

Function name

__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_ALRA (RTC_TypeDef * RTCx)

Function description

Check if Alarm A interrupt is enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR ALRAIE LL_RTC_IsEnabledIT_ALRA
LL_RTC_IsEnabledIT_TAMP

Function name

__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_TAMP (RTC_TypeDef * RTCx)

Function description

Check if all the TAMPER interrupts are enabled or not.

Parameters

- **RTCx:** RTC Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- TAFCR TAMPIE LL_RTC_IsEnabledIT_TAMP
LL_RTC_DeInit

Function name

ErrorStatus LL_RTC_DeInit (RTC_TypeDef * RTCx)

Function description

De-Initializes the RTC registers to their default reset values.

Parameters

- **RTCx:** RTC Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC registers are de-initialized
 - ERROR: RTC registers are not de-initialized

Notes

- This function doesn't reset the RTC Clock source and RTC Backup Data registers.

LL_RTC_Init

Function name

ErrorStatus LL_RTC_Init (RTC_TypeDef * RTCx, LL_RTC_InitTypeDef * RTC_InitStruct)

Function description

Initializes the RTC registers according to the specified parameters in RTC_InitStruct.

Parameters

- **RTCx:** RTC Instance
- **RTC_InitStruct:** pointer to a LL_RTC_InitTypeDef structure that contains the configuration information for the RTC peripheral.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC registers are initialized
 - ERROR: RTC registers are not initialized

Notes

- The RTC Prescaler register is write protected and can be written in initialization mode only.

LL_RTC_StructInit

Function name

void LL_RTC_StructInit (LL_RTC_InitTypeDef * RTC_InitStruct)

Function description

Set each LL_RTC_InitTypeDef field to default value.

Parameters

- **RTC_InitStruct:** pointer to a LL_RTC_InitTypeDef structure which will be initialized.

Return values

- **None:**

LL_RTC_TIME_Init

Function name

ErrorStatus LL_RTC_TIME_Init (RTC_TypeDef * RTCx, uint32_t RTC_Format, LL_RTC_TimeTypeDef * RTC_TimeStruct)

Function description

Set the RTC current time.

Parameters

- **RTCx:** RTC Instance
- **RTC_Format:** This parameter can be one of the following values:
 - LL_RTC_FORMAT_BIN
 - LL_RTC_FORMAT_BCD
- **RTC_TimeStruct:** pointer to a RTC_TimeTypeDef structure that contains the time configuration information for the RTC.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC Time register is configured
 - ERROR: RTC Time register is not configured

LL_RTC_TIME_StructInit

Function name

void LL_RTC_TIME_StructInit (LL_RTC_TimeTypeDef * RTC_TimeStruct)

Function description

Set each LL_RTC_TimeTypeDef field to default value (Time = 00h:00min:00sec).

Parameters

- **RTC_TimeStruct:** pointer to a LL_RTC_TimeTypeDef structure which will be initialized.

Return values

- **None:**

LL_RTC_DATE_Init

Function name

ErrorStatus LL_RTC_DATE_Init (RTC_TypeDef * RTCx, uint32_t RTC_Format, LL_RTC_DateTypeDef * RTC_DateStruct)

Function description

Set the RTC current date.

Parameters

- **RTCx:** RTC Instance
- **RTC_Format:** This parameter can be one of the following values:
 - LL_RTC_FORMAT_BIN
 - LL_RTC_FORMAT_BCD
- **RTC_DateStruct:** pointer to a RTC_DateTypeDef structure that contains the date configuration information for the RTC.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC Day register is configured
 - ERROR: RTC Day register is not configured

LL_RTC_DATE_StructInit

Function name

void LL_RTC_DATE_StructInit (LL_RTC_DateTypeDef * RTC_DateStruct)

Function description

Set each LL_RTC_DateTypeDef field to default value (date = Monday, January 01 xx00)

Parameters

- **RTC_DateStruct:** pointer to a LL_RTC_DateTypeDef structure which will be initialized.

Return values

- **None:**

LL_RTC_ALMA_Init

Function name

ErrorStatus LL_RTC_ALMA_Init (RTC_TypeDef * RTCx, uint32_t RTC_Format, LL_RTC_AlarmTypeDef * RTC_AlarmStruct)

Function description

Set the RTC Alarm A.

Parameters

- **RTCx**: RTC Instance
- **RTC_Format**: This parameter can be one of the following values:
 - LL_RTC_FORMAT_BIN
 - LL_RTC_FORMAT_BCD
- **RTC_AlarmStruct**: pointer to a LL_RTC_AlarmTypeDef structure that contains the alarm configuration parameters.

Return values

- **An**: ErrorStatus enumeration value:
 - SUCCESS: ALARMA registers are configured
 - ERROR: ALARMA registers are not configured

Notes

- The Alarm register can only be written when the corresponding Alarm is disabled (Use LL_RTC_ALMA_Disable function).

LL_RTC_ALMB_Init

Function name

ErrorStatus LL_RTC_ALMB_Init (RTC_TypeDef * RTCx, uint32_t RTC_Format, LL_RTC_AlarmTypeDef * RTC_AlarmStruct)

Function description

Set the RTC Alarm B.

Parameters

- **RTCx**: RTC Instance
- **RTC_Format**: This parameter can be one of the following values:
 - LL_RTC_FORMAT_BIN
 - LL_RTC_FORMAT_BCD
- **RTC_AlarmStruct**: pointer to a LL_RTC_AlarmTypeDef structure that contains the alarm configuration parameters.

Return values

- **An**: ErrorStatus enumeration value:
 - SUCCESS: ALARMB registers are configured
 - ERROR: ALARMB registers are not configured

Notes

- The Alarm register can only be written when the corresponding Alarm is disabled (LL_RTC_ALMB_Disable function).

LL_RTC_ALMA_StructInit

Function name

void LL_RTC_ALMA_StructInit (LL_RTC_AlarmTypeDef * RTC_AlarmStruct)

Function description

Set each LL_RTC_AlarmTypeDef of ALARMA field to default value (Time = 00h:00mn:00sec / Day = 1st day of the month/Mask = all fields are masked).

Parameters

- **RTC_AlarmStruct:** pointer to a LL_RTC_AlarmTypeDef structure which will be initialized.

Return values

- **None:**

```
LL_RTC_ALMB_StructInit
```

Function name

```
void LL_RTC_ALMB_StructInit (LL_RTC_AlarmTypeDef * RTC_AlarmStruct)
```

Function description

Set each LL_RTC_AlarmTypeDef of ALARMA field to default value (Time = 00h:00mn:00sec / Day = 1st day of the month/Mask = all fields are masked).

Parameters

- **RTC_AlarmStruct:** pointer to a LL_RTC_AlarmTypeDef structure which will be initialized.

Return values

- **None:**

```
LL_RTC_EnterInitMode
```

Function name

```
ErrorStatus LL_RTC_EnterInitMode (RTC_TypeDef * RTCx)
```

Function description

Enters the RTC Initialization mode.

Parameters

- **RTCx:** RTC Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC is in Init mode
 - ERROR: RTC is not in Init mode

Notes

- The RTC Initialization mode is write protected, use the LL_RTC_DisableWriteProtection before calling this function.

```
LL_RTC_ExitInitMode
```

Function name

```
ErrorStatus LL_RTC_ExitInitMode (RTC_TypeDef * RTCx)
```

Function description

Exit the RTC Initialization mode.

Parameters

- **RTCx:** RTC Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC exited from in Init mode
 - ERROR: Not applicable

Notes

- When the initialization sequence is complete, the calendar restarts counting after 4 RTCCLK cycles.
- The RTC Initialization mode is write protected, use the LL_RTC_DisableWriteProtection before calling this function.

LL_RTC_WaitForSynchro

Function name

ErrorStatus LL_RTC_WaitForSynchro (RTC_TypeDef * RTCx)

Function description

Waits until the RTC Time and Day registers (RTC_TR and RTC_DR) are synchronized with RTC APB clock.

Parameters

- **RTCx:** RTC Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: RTC registers are synchronised
 - ERROR: RTC registers are not synchronised

Notes

- The RTC Resynchronization mode is write protected, use the LL_RTC_DisableWriteProtection before calling this function.
- To read the calendar through the shadow registers after Calendar initialization, calendar update or after wakeup from low power modes the software must first clear the RSF flag. The software must then wait until it is set again before reading the calendar, which means that the calendar registers have been correctly copied into the RTC_TR and RTC_DR shadow registers.

62.3 RTC Firmware driver defines

The following section lists the various define and macros of the module.

62.3.1 RTC

RTC

ALARM OUTPUT

LL_RTC_ALARMOUT_DISABLE

Output disabled

LL_RTC_ALARMOUT_ALMA

Alarm A output enabled

LL_RTC_ALARMOUT_ALMB

Alarm B output enabled

LL_RTC_ALARMOUT_WAKEUP

Wakeup output enabled

ALARM OUTPUT TYPE

LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN

RTC_ALARM, when mapped on PC13, is open-drain output

LL_RTC_ALARM_OUTPUTTYPE_PUSH_PULL

RTC_ALARM, when mapped on PC13, is push-pull output

ALARMA_MASK

LL_RTC_ALMA_MASK_NONE

No masks applied on Alarm A

LL_RTC_ALMA_MASK_DATEWEEKDAY

Date/day do not care in Alarm A comparison

LL_RTC_ALMA_MASK_HOURS

Hours do not care in Alarm A comparison

LL_RTC_ALMA_MASK_MINUTES

Minutes do not care in Alarm A comparison

LL_RTC_ALMA_MASK_SECONDS

Seconds do not care in Alarm A comparison

LL_RTC_ALMA_MASK_ALL

Masks all

ALARMA_TIME_FORMAT

LL_RTC_ALMA_TIME_FORMAT_AM

AM or 24-hour format

LL_RTC_ALMA_TIME_FORMAT_PM

PM

RTC Alarm A Date WeekDay

LL_RTC_ALMA_DATEWEEKDAYSEL_DATE

Alarm A Date is selected

LL_RTC_ALMA_DATEWEEKDAYSEL_WEEKDAY

Alarm A WeekDay is selected

ALARMB_MASK

LL_RTC_ALMB_MASK_NONE

No masks applied on Alarm B

LL_RTC_ALMB_MASK_DATEWEEKDAY

Date/day do not care in Alarm B comparison

LL_RTC_ALMB_MASK_HOURS

Hours do not care in Alarm B comparison

LL_RTC_ALMB_MASK_MINUTES

Minutes do not care in Alarm B comparison

LL_RTC_ALMB_MASK_SECONDS

Seconds do not care in Alarm B comparison

LL_RTC_ALMB_MASK_ALL

Masks all

ALARMB_TIME_FORMAT

LL_RTC_ALMB_TIME_FORMAT_AM

AM or 24-hour format

LL_RTC_ALMB_TIME_FORMAT_PM

PM

RTC Alarm B Date WeekDay

LL_RTC_ALMB_DATEWEEKDAYSEL_DATE

Alarm B Date is selected

LL_RTC_ALMB_DATEWEEKDAYSEL_WEEKDAY

Alarm B WeekDay is selected

BACKUP

LL_RTC_BKP_DR0

LL_RTC_BKP_DR1

LL_RTC_BKP_DR2

LL_RTC_BKP_DR3

LL_RTC_BKP_DR4

LL_RTC_BKP_DR5

LL_RTC_BKP_DR6

LL_RTC_BKP_DR7

LL_RTC_BKP_DR8

LL_RTC_BKP_DR9

LL_RTC_BKP_DR10

LL_RTC_BKP_DR11

LL_RTC_BKP_DR12

LL_RTC_BKP_DR13

LL_RTC_BKP_DR14

LL_RTC_BKP_DR15

LL_RTC_BKP_DR16

LL_RTC_BKP_DR17

LL_RTC_BKP_DR18

LL_RTC_BKP_DR19

Calibration pulse insertion

LL_RTC_CALIB_INSERTPULSE_NONE

No RTCCLK pulses are added

LL_RTC_CALIB_INSERTPULSE_SET

One RTCCLK pulse is effectively inserted every 2exp11 pulses (frequency increased by 488.5 ppm)

Calibration output

LL_RTC_CALIB_OUTPUT_NONE

Calibration output disabled

LL_RTC_CALIB_OUTPUT_512HZ

Calibration output is 512 Hz

Calibration period

LL_RTC_CALIB_PERIOD_32SEC

Use a 32-second calibration cycle period

LL_RTC_CALIB_PERIOD_16SEC

Use a 16-second calibration cycle period

LL_RTC_CALIB_PERIOD_8SEC

Use a 8-second calibration cycle period

Coarse digital calibration sign

LL_RTC_CALIB_SIGN_POSITIVE

Positive calibration: calendar update frequency is increased

LL_RTC_CALIB_SIGN_NEGATIVE

Negative calibration: calendar update frequency is decreased

FORMAT

LL_RTC_FORMAT_BIN

Binary data format

LL_RTC_FORMAT_BCD

BCD data format

Get Flags Defines

LL_RTC_ISR_TAMP3F

LL_RTC_ISR_TAMP2F

LL_RTC_ISR_TAMP1F

LL_RTC_ISR_TSOVF

LL_RTC_ISR_TSF

LL_RTC_ISR_WUTF

LL_RTC_ISR_ALRBF

LL_RTC_ISR_ALRAF

LL_RTC_ISR_INITF

LL_RTC_ISR_RSF

LL_RTC_ISR_INITS

LL_RTC_ISR_SHPF

LL_RTC_ISR_WUTWF

LL_RTC_ISR_ALRBWF

LL_RTC_ISR_ALRAWF

HOUR FORMAT

LL_RTC_HOURFORMAT_24HOUR

24 hour/day format

LL_RTC_HOURFORMAT_AMPM

AM/PM hour format

IT Defines

LL_RTC_CR_TSIE

LL_RTC_CR_WUTIE

LL_RTC_CR_ALRBIE

LL_RTC_CR_ALRAIE

LL_RTC_TAFCR_TAMPIE

MONTH

LL_RTC_MONTH_JANUARY

January

LL_RTC_MONTH_FEBRUARY

February

LL_RTC_MONTH_MARCH

March

LL_RTC_MONTH_APRIL

April

LL_RTC_MONTH_MAY

May

LL_RTC_MONTH_JUNE

June

LL_RTC_MONTH_JULY

July

LL_RTC_MONTH_AUGUST

August

LL_RTC_MONTH_SEPTMBER

September

LL_RTC_MONTH_OCTOBER

October

LL_RTC_MONTH_NOVEMBER

November

LL_RTC_MONTH_DECEMBER

December

OUTPUT POLARITY PIN

LL_RTC_OUTPUTPOLARITY_PIN_HIGH

Pin is high when ALRAF/ALRBF/WUTF is asserted (depending on OSEL)

LL_RTC_OUTPUTPOLARITY_PIN_LOW

Pin is low when ALRAF/ALRBF/WUTF is asserted (depending on OSEL)

TAMPER1 mapping

LL_RTC_TamperPin_Default

Use RTC_AF1 as TAMPER1

LL_RTC_TamperPin_Pos1

Use RTC_AF2 as TAMPER1

TAMPER

LL_RTC_TAMPER_1

RTC_TAMP1 input detection

TAMPER ACTIVE LEVEL

LL_RTC_TAMPER_ACTIVELEVEL_TAMP1

RTC_TAMP1 input falling edge (if TAMPFLT = 00) or staying high (if TAMPFLT != 00) triggers a tamper detection event

TAMPER MASK

LL_RTC_TAMPER_MASK_TAMPER1

Tamper 1 event generates a trigger event. TAMP1F is masked and internally cleared by hardware. The backup registers are not erased

TAMPER NO ERASE

LL_RTC_TAMPER_NOERASE_TAMPER1

Tamper 1 event does not erase the backup registers.

TIMESTAMP EDGE

LL_RTC_TIMESTAMP_EDGE_RISING

RTC_TS input rising edge generates a time-stamp event

LL_RTC_TIMESTAMP_EDGE_FALLING

RTC_TS input falling edge generates a time-stamp even

TIME FORMAT

LL_RTC_TIME_FORMAT_AM_OR_24

AM or 24-hour format

LL_RTC_TIME_FORMAT_PM

PM

TIMESTAMP mapping**LL_RTC_TimeStampPin_Default**

Use RTC_AF1 as TIMESTAMP

LL_RTC_TimeStampPin_Pos1

Use RTC_AF2 as TIMESTAMP

TIMESTAMP TIME FORMAT**LL_RTC_TS_TIME_FORMAT_AM**

AM or 24-hour format

LL_RTC_TS_TIME_FORMAT_PM

PM

WAKEUP CLOCK DIV**LL_RTC_WAKEUPCLOCK_DIV_16**

RTC/16 clock is selected

LL_RTC_WAKEUPCLOCK_DIV_8

RTC/8 clock is selected

LL_RTC_WAKEUPCLOCK_DIV_4

RTC/4 clock is selected

LL_RTC_WAKEUPCLOCK_DIV_2

RTC/2 clock is selected

LL_RTC_WAKEUPCLOCK_CKSPRE

ck_spre (usually 1 Hz) clock is selected

LL_RTC_WAKEUPCLOCK_CKSPRE_WUT

ck_spre (usually 1 Hz) clock is selected and 2exp16 is added to the WUT counter value

WEEK DAY**LL_RTC_WEEKDAY_MONDAY**

Monday

LL_RTC_WEEKDAY_TUESDAY

Tuesday

LL_RTC_WEEKDAY_WEDNESDAY

Wednesday

LL_RTC_WEEKDAY_THURSDAY

Thrusday

LL_RTC_WEEKDAY_FRIDAY

Friday

LL_RTC_WEEKDAY_SATURDAY

Saturday

LL_RTC_WEEKDAY_SUNDAY

Sunday

Convert helper Macros

__LL_RTC_CONVERT_BIN2BCD

Description:

- Helper macro to convert a value from 2 digit decimal format to BCD format.

Parameters:

- `__VALUE__`: Byte to be converted

Return value:

- Converted: byte

__LL_RTC_CONVERT_BCD2BIN

Description:

- Helper macro to convert a value from BCD format to 2 digit decimal format.

Parameters:

- `__VALUE__`: BCD value to be converted

Return value:

- Converted: byte

Date helper Macros

__LL_RTC_GET_WEEKDAY

Description:

- Helper macro to retrieve weekday.

Parameters:

- `__RTC_DATE__`: Date returned by

Return value:

- Returned: value can be one of the following values:
 - LL_RTC_WEEKDAY_MONDAY
 - LL_RTC_WEEKDAY_TUESDAY
 - LL_RTC_WEEKDAY_WEDNESDAY
 - LL_RTC_WEEKDAY_THURSDAY
 - LL_RTC_WEEKDAY_FRIDAY
 - LL_RTC_WEEKDAY_SATURDAY
 - LL_RTC_WEEKDAY_SUNDAY

__LL_RTC_GET_YEAR

Description:

- Helper macro to retrieve Year in BCD format.

Parameters:

- `__RTC_DATE__`: Value returned by

Return value:

- Year: in BCD format (0x00 . . . 0x99)

__LL_RTC_GET_MONTH

Description:

- Helper macro to retrieve Month in BCD format.

Parameters:

- `__RTC_DATE__`: Value returned by

Return value:

- Returned: value can be one of the following values:
 - `LL_RTC_MONTH_JANUARY`
 - `LL_RTC_MONTH_FEBRUARY`
 - `LL_RTC_MONTH_MARCH`
 - `LL_RTC_MONTH_APRIL`
 - `LL_RTC_MONTH_MAY`
 - `LL_RTC_MONTH_JUNE`
 - `LL_RTC_MONTH_JULY`
 - `LL_RTC_MONTH_AUGUST`
 - `LL_RTC_MONTH_SEPTEMBER`
 - `LL_RTC_MONTH_OCTOBER`
 - `LL_RTC_MONTH_NOVEMBER`
 - `LL_RTC_MONTH_DECEMBER`

__LL_RTC_GET_DAY

Description:

- Helper macro to retrieve Day in BCD format.

Parameters:

- `__RTC_DATE__`: Value returned by

Return value:

- Day: in BCD format (0x01 . . . 0x31)

Time helper Macros

__LL_RTC_GET_HOUR

Description:

- Helper macro to retrieve hour in BCD format.

Parameters:

- `__RTC_TIME__`: RTC time returned by

Return value:

- Hours: in BCD format (0x01. . . 0x12 or between `Min_Data=0x00` and `Max_Data=0x23`)

__LL_RTC_GET_MINUTE

Description:

- Helper macro to retrieve minute in BCD format.

Parameters:

- `__RTC_TIME__`: RTC time returned by

Return value:

- Minutes: in BCD format (0x00. . . 0x59)

__LL_RTC_GET_SECOND

Description:

- Helper macro to retrieve second in BCD format.

Parameters:

- `__RTC_TIME__`: RTC time returned by

Return value:

- Seconds: in format (0x00. . .0x59)

Common Write and read registers Macros

LL_RTC_WriteReg

Description:

- Write a value in RTC register.

Parameters:

- `__INSTANCE__`: RTC Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_RTC_ReadReg

Description:

- Read a value in RTC register.

Parameters:

- `__INSTANCE__`: RTC Instance
- `__REG__`: Register to be read

Return value:

- Register: value

63 LL SPI Generic Driver

63.1 SPI Firmware driver registers structures

63.1.1 LL_SPI_InitTypeDef

LL_SPI_InitTypeDef is defined in the `stm32f2xx_ll_spi.h`

Data Fields

- **uint32_t TransferDirection**
- **uint32_t Mode**
- **uint32_t DataWidth**
- **uint32_t ClockPolarity**
- **uint32_t ClockPhase**
- **uint32_t NSS**
- **uint32_t BaudRate**
- **uint32_t BitOrder**
- **uint32_t CRCCalculation**
- **uint32_t CRCPoly**

Field Documentation

- **uint32_t LL_SPI_InitTypeDef::TransferDirection**
Specifies the SPI unidirectional or bidirectional data mode. This parameter can be a value of [SPI_LL_EC_TRANSFER_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetTransferDirection()`.
 - **uint32_t LL_SPI_InitTypeDef::Mode**
Specifies the SPI mode (Master/Slave). This parameter can be a value of [SPI_LL_EC_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetMode()`.
 - **uint32_t LL_SPI_InitTypeDef::DataWidth**
Specifies the SPI data width. This parameter can be a value of [SPI_LL_EC_DATAWIDTH](#). This feature can be modified afterwards using unitary function `LL_SPI_SetDataWidth()`.
 - **uint32_t LL_SPI_InitTypeDef::ClockPolarity**
Specifies the serial clock steady state. This parameter can be a value of [SPI_LL_EC_POLARITY](#). This feature can be modified afterwards using unitary function `LL_SPI_SetClockPolarity()`.
 - **uint32_t LL_SPI_InitTypeDef::ClockPhase**
Specifies the clock active edge for the bit capture. This parameter can be a value of [SPI_LL_EC_PHASE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetClockPhase()`.
 - **uint32_t LL_SPI_InitTypeDef::NSS**
Specifies whether the NSS signal is managed by hardware (NSS pin) or by software using the SSI bit. This parameter can be a value of [SPI_LL_EC_NSS_MODE](#). This feature can be modified afterwards using unitary function `LL_SPI_SetNSSMode()`.
 - **uint32_t LL_SPI_InitTypeDef::BaudRate**
Specifies the BaudRate prescaler value which will be used to configure the transmit and receive SCK clock. This parameter can be a value of [SPI_LL_EC_BAUDRATEPRESCALER](#).
- Note:**
- The communication clock is derived from the master clock. The slave clock does not need to be set. This feature can be modified afterwards using unitary function `LL_SPI_SetBaudRatePrescaler()`.
- **uint32_t LL_SPI_InitTypeDef::BitOrder**
Specifies whether data transfers start from MSB or LSB bit. This parameter can be a value of [SPI_LL_EC_BIT_ORDER](#). This feature can be modified afterwards using unitary function `LL_SPI_SetTransferBitOrder()`.

- **`uint32_t LL_SPI_InitTypeDef::CRCCalculation`**
Specifies if the CRC calculation is enabled or not. This parameter can be a value of **`SPI_LL_EC_CRC_CALCULATION`**. This feature can be modified afterwards using unitary functions **`LL_SPI_EnableCRC()`** and **`LL_SPI_DisableCRC()`**.
- **`uint32_t LL_SPI_InitTypeDef::CRCPoly`**
Specifies the polynomial used for the CRC calculation. This parameter must be a number between `Min_Data = 0x00` and `Max_Data = 0xFFFF`. This feature can be modified afterwards using unitary function **`LL_SPI_SetCRCPolynomial()`**.

63.1.2

LL_I2S_InitTypeDef

`LL_I2S_InitTypeDef` is defined in the `stm32f2xx_ll_spi.h`

Data Fields

- **`uint32_t Mode`**
- **`uint32_t Standard`**
- **`uint32_t DataFormat`**
- **`uint32_t MCLKOutput`**
- **`uint32_t AudioFreq`**
- **`uint32_t ClockPolarity`**

Field Documentation

- **`uint32_t LL_I2S_InitTypeDef::Mode`**
Specifies the I2S operating mode. This parameter can be a value of **`I2S_LL_EC_MODE`**. This feature can be modified afterwards using unitary function **`LL_I2S_SetTransferMode()`**.
- **`uint32_t LL_I2S_InitTypeDef::Standard`**
Specifies the standard used for the I2S communication. This parameter can be a value of **`I2S_LL_EC_STANDARD`**. This feature can be modified afterwards using unitary function **`LL_I2S_SetStandard()`**.
- **`uint32_t LL_I2S_InitTypeDef::DataFormat`**
Specifies the data format for the I2S communication. This parameter can be a value of **`I2S_LL_EC_DATA_FORMAT`**. This feature can be modified afterwards using unitary function **`LL_I2S_SetDataFormat()`**.
- **`uint32_t LL_I2S_InitTypeDef::MCLKOutput`**
Specifies whether the I2S MCLK output is enabled or not. This parameter can be a value of **`I2S_LL_EC_MCLK_OUTPUT`**. This feature can be modified afterwards using unitary functions **`LL_I2S_EnableMasterClock()`** or **`LL_I2S_DisableMasterClock()`**.
- **`uint32_t LL_I2S_InitTypeDef::AudioFreq`**
Specifies the frequency selected for the I2S communication. This parameter can be a value of **`I2S_LL_EC_AUDIO_FREQ`**. Audio Frequency can be modified afterwards using Reference manual formulas to calculate Prescaler Linear, Parity and unitary functions **`LL_I2S_SetPrescalerLinear()`** and **`LL_I2S_SetPrescalerParity()`** to set it.
- **`uint32_t LL_I2S_InitTypeDef::ClockPolarity`**
Specifies the idle state of the I2S clock. This parameter can be a value of **`I2S_LL_EC_POLARITY`**. This feature can be modified afterwards using unitary function **`LL_I2S_SetClockPolarity()`**.

63.2

SPI Firmware driver API description

The following section lists the various functions of the SPI library.

63.2.1

Detailed description of functions

`LL_SPI_Enable`

Function name

`__STATIC_INLINE void LL_SPI_Enable (SPI_TypeDef * SPIx)`

Function description

Enable SPI peripheral.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 SPE LL_SPI_Enable

LL_SPI_Disable

Function name

__STATIC_INLINE void LL_SPI_Disable (SPI_TypeDef * SPIx)

Function description

Disable SPI peripheral.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- When disabling the SPI, follow the procedure described in the Reference Manual.

Reference Manual to LL API cross reference:

- CR1 SPE LL_SPI_Disable

LL_SPI_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_SPI_IsEnabled (SPI_TypeDef * SPIx)

Function description

Check if SPI peripheral is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 SPE LL_SPI_IsEnabled

LL_SPI_SetMode

Function name

__STATIC_INLINE void LL_SPI_SetMode (SPI_TypeDef * SPIx, uint32_t Mode)

Function description

Set SPI operation mode to Master or Slave.

Parameters

- **SPIx:** SPI Instance
- **Mode:** This parameter can be one of the following values:
 - LL_SPI_MODE_MASTER
 - LL_SPI_MODE_SLAVE

Return values

- **None:**

Notes

- This bit should not be changed when communication is ongoing.

Reference Manual to LL API cross reference:

- CR1 MSTR LL_SPI_SetMode
- CR1 SSI LL_SPI_SetMode

LL_SPI_GetMode

Function name

__STATIC_INLINE uint32_t LL_SPI_GetMode (SPI_TypeDef * SPIx)

Function description

Get SPI operation mode (Master or Slave)

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_MODE_MASTER
 - LL_SPI_MODE_SLAVE

Reference Manual to LL API cross reference:

- CR1 MSTR LL_SPI_GetMode
- CR1 SSI LL_SPI_GetMode

LL_SPI_SetStandard

Function name

__STATIC_INLINE void LL_SPI_SetStandard (SPI_TypeDef * SPIx, uint32_t Standard)

Function description

Set serial protocol used.

Parameters

- **SPIx:** SPI Instance
- **Standard:** This parameter can be one of the following values:
 - LL_SPI_PROTOCOL_MOTOROLA
 - LL_SPI_PROTOCOL_TI

Return values

- **None:**

Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

Reference Manual to LL API cross reference:

- CR2 FRF LL_SPI_SetStandard
LL_SPI_GetStandard

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetStandard (SPI_TypeDef * SPIx)
```

Function description

Get serial protocol used.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_PROTOCOL_MOTOROLA
 - LL_SPI_PROTOCOL_TI

Reference Manual to LL API cross reference:

- CR2 FRF LL_SPI_GetStandard
LL_SPI_SetClockPhase

Function name

```
__STATIC_INLINE void LL_SPI_SetClockPhase (SPI_TypeDef * SPIx, uint32_t ClockPhase)
```

Function description

Set clock phase.

Parameters

- **SPIx:** SPI Instance
- **ClockPhase:** This parameter can be one of the following values:
 - LL_SPI_PHASE_1EDGE
 - LL_SPI_PHASE_2EDGE

Return values

- **None:**

Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

Reference Manual to LL API cross reference:

- CR1 CPHA LL_SPI_SetClockPhase
LL_SPI_GetClockPhase

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetClockPhase (SPI_TypeDef * SPIx)
```

Function description

Get clock phase.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_PHASE_1EDGE
 - LL_SPI_PHASE_2EDGE

Reference Manual to LL API cross reference:

- CR1 CPHA LL_SPI_GetClockPhase
LL_SPI_SetClockPolarity

Function name

```
__STATIC_INLINE void LL_SPI_SetClockPolarity (SPI_TypeDef * SPIx, uint32_t ClockPolarity)
```

Function description

Set clock polarity.

Parameters

- **SPIx:** SPI Instance
- **ClockPolarity:** This parameter can be one of the following values:
 - LL_SPI_POLARITY_LOW
 - LL_SPI_POLARITY_HIGH

Return values

- **None:**

Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

Reference Manual to LL API cross reference:

- CR1 CPOL LL_SPI_SetClockPolarity
LL_SPI_GetClockPolarity

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetClockPolarity (SPI_TypeDef * SPIx)
```

Function description

Get clock polarity.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_POLARITY_LOW
 - LL_SPI_POLARITY_HIGH

Reference Manual to LL API cross reference:

- CR1 CPOL LL_SPI_GetClockPolarity
LL_SPI_SetBaudRatePrescaler

Function name

```
__STATIC_INLINE void LL_SPI_SetBaudRatePrescaler (SPI_TypeDef * SPIx, uint32_t BaudRate)
```

Function description

Set baud rate prescaler.

Parameters

- **SPIx:** SPI Instance
- **BaudRate:** This parameter can be one of the following values:
 - LL_SPI_BAUDRATEPRESCALER_DIV2
 - LL_SPI_BAUDRATEPRESCALER_DIV4
 - LL_SPI_BAUDRATEPRESCALER_DIV8
 - LL_SPI_BAUDRATEPRESCALER_DIV16
 - LL_SPI_BAUDRATEPRESCALER_DIV32
 - LL_SPI_BAUDRATEPRESCALER_DIV64
 - LL_SPI_BAUDRATEPRESCALER_DIV128
 - LL_SPI_BAUDRATEPRESCALER_DIV256

Return values

- **None:**

Notes

- These bits should not be changed when communication is ongoing. SPI BaudRate = fPCLK/Prescaler.

Reference Manual to LL API cross reference:

- CR1 BR LL_SPI_SetBaudRatePrescaler
LL_SPI_GetBaudRatePrescaler

Function name

__STATIC_INLINE uint32_t LL_SPI_GetBaudRatePrescaler (SPI_TypeDef * SPIx)

Function description

Get baud rate prescaler.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_BAUDRATEPRESCALER_DIV2
 - LL_SPI_BAUDRATEPRESCALER_DIV4
 - LL_SPI_BAUDRATEPRESCALER_DIV8
 - LL_SPI_BAUDRATEPRESCALER_DIV16
 - LL_SPI_BAUDRATEPRESCALER_DIV32
 - LL_SPI_BAUDRATEPRESCALER_DIV64
 - LL_SPI_BAUDRATEPRESCALER_DIV128
 - LL_SPI_BAUDRATEPRESCALER_DIV256

Reference Manual to LL API cross reference:

- CR1 BR LL_SPI_GetBaudRatePrescaler
LL_SPI_SetTransferBitOrder

Function name

__STATIC_INLINE void LL_SPI_SetTransferBitOrder (SPI_TypeDef * SPIx, uint32_t BitOrder)

Function description

Set transfer bit order.

Parameters

- **SPIx:** SPI Instance
- **BitOrder:** This parameter can be one of the following values:
 - LL_SPI_LSB_FIRST
 - LL_SPI_MSB_FIRST

Return values

- **None:**

Notes

- This bit should not be changed when communication is ongoing. This bit is not used in SPI TI mode.

Reference Manual to LL API cross reference:

- CR1 LSBFIRST LL_SPI_SetTransferBitOrder

LL_SPI_GetTransferBitOrder

Function name

__STATIC_INLINE uint32_t LL_SPI_GetTransferBitOrder (SPI_TypeDef * SPIx)

Function description

Get transfer bit order.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_LSB_FIRST
 - LL_SPI_MSB_FIRST

Reference Manual to LL API cross reference:

- CR1 LSBFIRST LL_SPI_GetTransferBitOrder

LL_SPI_SetTransferDirection

Function name

__STATIC_INLINE void LL_SPI_SetTransferDirection (SPI_TypeDef * SPIx, uint32_t TransferDirection)

Function description

Set transfer direction mode.

Parameters

- **SPIx:** SPI Instance
- **TransferDirection:** This parameter can be one of the following values:
 - LL_SPI_FULL_DUPLEX
 - LL_SPI_SIMPLEX_RX
 - LL_SPI_HALF_DUPLEX_RX
 - LL_SPI_HALF_DUPLEX_TX

Return values

- **None:**

Notes

- For Half-Duplex mode, Rx Direction is set by default. In master mode, the MOSI pin is used and in slave mode, the MISO pin is used for Half-Duplex.

Reference Manual to LL API cross reference:

- CR1 RXONLY LL_SPI_SetTransferDirection
- CR1 BIDIMODE LL_SPI_SetTransferDirection
- CR1 BIDIOE LL_SPI_SetTransferDirection

LL_SPI_GetTransferDirection

Function name

__STATIC_INLINE uint32_t LL_SPI_GetTransferDirection (SPI_TypeDef * SPIx)

Function description

Get transfer direction mode.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_FULL_DUPLEX
 - LL_SPI_SIMPLEX_RX
 - LL_SPI_HALF_DUPLEX_RX
 - LL_SPI_HALF_DUPLEX_TX

Reference Manual to LL API cross reference:

- CR1 RXONLY LL_SPI_GetTransferDirection
- CR1 BIDIMODE LL_SPI_GetTransferDirection
- CR1 BIDIOE LL_SPI_GetTransferDirection

LL_SPI_SetDataWidth

Function name

__STATIC_INLINE void LL_SPI_SetDataWidth (SPI_TypeDef * SPIx, uint32_t DataWidth)

Function description

Set frame data width.

Parameters

- **SPIx:** SPI Instance
- **DataWidth:** This parameter can be one of the following values:
 - LL_SPI_DATAWIDTH_8BIT
 - LL_SPI_DATAWIDTH_16BIT

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 DFF LL_SPI_SetDataWidth

LL_SPI_GetDataWidth

Function name

__STATIC_INLINE uint32_t LL_SPI_GetDataWidth (SPI_TypeDef * SPIx)

Function description

Get frame data width.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_DATAWIDTH_8BIT
 - LL_SPI_DATAWIDTH_16BIT

Reference Manual to LL API cross reference:

- CR1 DFF LL_SPI_GetDataWidth
- LL_SPI_EnableCRC

Function name

```
__STATIC_INLINE void LL_SPI_EnableCRC (SPI_TypeDef * SPIx)
```

Function description

Enable CRC.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

Reference Manual to LL API cross reference:

- CR1 CRCEN LL_SPI_EnableCRC
- LL_SPI_DisableCRC

Function name

```
__STATIC_INLINE void LL_SPI_DisableCRC (SPI_TypeDef * SPIx)
```

Function description

Disable CRC.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

Reference Manual to LL API cross reference:

- CR1 CRCEN LL_SPI_DisableCRC
- LL_SPI_IsEnabledCRC

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledCRC (SPI_TypeDef * SPIx)
```

Function description

Check if CRC is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Notes

- This bit should be written only when SPI is disabled (SPE = 0) for correct operation.

Reference Manual to LL API cross reference:

- CR1 CRCEN LL_SPI_IsEnabledCRC
LL_SPI_SetCRCNext

Function name

```
__STATIC_INLINE void LL_SPI_SetCRCNext (SPI_TypeDef * SPIx)
```

Function description

Set CRCNext to transfer CRC on the line.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit has to be written as soon as the last data is written in the SPIx_DR register.

Reference Manual to LL API cross reference:

- CR1 CRCNEXT LL_SPI_SetCRCNext
LL_SPI_SetCRCPolynomial

Function name

```
__STATIC_INLINE void LL_SPI_SetCRCPolynomial (SPI_TypeDef * SPIx, uint32_t CRCPoly)
```

Function description

Set polynomial for CRC calculation.

Parameters

- **SPIx:** SPI Instance
- **CRCPoly:** This parameter must be a number between Min_Data = 0x00 and Max_Data = 0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- CRCPR CRCPOLY LL_SPI_SetCRCPolynomial
LL_SPI_GetCRCPolynomial

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetCRCPolynomial (SPI_TypeDef * SPIx)
```

Function description

Get polynomial for CRC calculation.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value is a number between Min_Data = 0x00 and Max_Data = 0xFFFF

Reference Manual to LL API cross reference:

- CRCPR CRCPOLY LL_SPI_GetCRCPolynomial
LL_SPI_GetRxCRC

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetRxCRC (SPI_TypeDef * SPIx)
```

Function description

Get Rx CRC.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value is a number between Min_Data = 0x00 and Max_Data = 0xFFFF

Reference Manual to LL API cross reference:

- RXCR CR RXCRC LL_SPI_GetRxCRC
LL_SPI_GetTxCRC

Function name

```
__STATIC_INLINE uint32_t LL_SPI_GetTxCRC (SPI_TypeDef * SPIx)
```

Function description

Get Tx CRC.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value is a number between Min_Data = 0x00 and Max_Data = 0xFFFF

Reference Manual to LL API cross reference:

- TXCR CR TXCRC LL_SPI_GetTxCRC
LL_SPI_SetNSSMode

Function name

```
__STATIC_INLINE void LL_SPI_SetNSSMode (SPI_TypeDef * SPIx, uint32_t NSS)
```

Function description

Set NSS mode.

Parameters

- **SPIx:** SPI Instance
- **NSS:** This parameter can be one of the following values:
 - LL_SPI_NSS_SOFT
 - LL_SPI_NSS_HARD_INPUT
 - LL_SPI_NSS_HARD_OUTPUT

Return values

- **None:**

Notes

- LL_SPI_NSS_SOFT Mode is not used in SPI TI mode.

Reference Manual to LL API cross reference:

- CR1 SSM LL_SPI_SetNSSMode
-
- CR2 SSOE LL_SPI_SetNSSMode

LL_SPI_GetNSSMode

Function name

__STATIC_INLINE uint32_t LL_SPI_GetNSSMode (SPI_TypeDef * SPIx)

Function description

Get NSS mode.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_SPI_NSS_SOFT
 - LL_SPI_NSS_HARD_INPUT
 - LL_SPI_NSS_HARD_OUTPUT

Reference Manual to LL API cross reference:

- CR1 SSM LL_SPI_GetNSSMode
-
- CR2 SSOE LL_SPI_GetNSSMode

LL_SPI_IsActiveFlag_RXNE

Function name

__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_RXNE (SPI_TypeDef * SPIx)

Function description

Check if Rx buffer is not empty.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR RXNE LL_SPI_IsActiveFlag_RXNE

LL_SPI_IsActiveFlag_TXE

Function name

__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_TXE (SPI_TypeDef * SPIx)

Function description

Check if Tx buffer is empty.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR TXE LL_SPI_IsActiveFlag_TXE
LL_SPI_IsActiveFlag_CRCERR

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_CRCERR (SPI_TypeDef * SPIx)
```

Function description

Get CRC error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CRCERR LL_SPI_IsActiveFlag_CRCERR
LL_SPI_IsActiveFlag_MODF

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_MODF (SPI_TypeDef * SPIx)
```

Function description

Get mode fault error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR MODF LL_SPI_IsActiveFlag_MODF
LL_SPI_IsActiveFlag_OVR

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_OVR (SPI_TypeDef * SPIx)
```

Function description

Get overrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR OVR LL_SPI_IsActiveFlag_OVR
LL_SPI_IsActiveFlag_BSY

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_BSY (SPI_TypeDef * SPIx)
```

Function description

Get busy flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Notes

- The BSY flag is cleared under any one of the following conditions: -When the SPI is correctly disabled - When a fault is detected in Master mode (MODF bit set to 1) -In Master mode, when it finishes a data transmission and no new data is ready to be sent -In Slave mode, when the BSY flag is set to '0' for at least one SPI clock cycle between each data transfer.

Reference Manual to LL API cross reference:

- SR BSY LL_SPI_IsActiveFlag_BSY
LL_SPI_IsActiveFlag_FRE

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsActiveFlag_FRE (SPI_TypeDef * SPIx)
```

Function description

Get frame format error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR FRE LL_SPI_IsActiveFlag_FRE
LL_SPI_ClearFlag_CRCERR

Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_CRCERR (SPI_TypeDef * SPIx)
```

Function description

Clear CRC error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CRCERR LL_SPI_ClearFlag_CRCERR


```
LL_SPI_ClearFlag_MODF
```

Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_MODF (SPI_TypeDef * SPIx)
```

Function description

Clear mode fault error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the SPIx_SR register followed by a write access to the SPIx_CR1 register

Reference Manual to LL API cross reference:

- SR MODF LL_SPI_ClearFlag_MODF

```
LL_SPI_ClearFlag_OVR
```

Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_OVR (SPI_TypeDef * SPIx)
```

Function description

Clear overrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the SPIx_DR register followed by a read access to the SPIx_SR register

Reference Manual to LL API cross reference:

- SR OVR LL_SPI_ClearFlag_OVR

```
LL_SPI_ClearFlag_FRE
```

Function name

```
__STATIC_INLINE void LL_SPI_ClearFlag_FRE (SPI_TypeDef * SPIx)
```

Function description

Clear frame format error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- Clearing this flag is done by reading SPIx_SR register

Reference Manual to LL API cross reference:

- SR FRE LL_SPI_ClearFlag_FRE
LL_SPI_EnableIT_ERR

Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_ERR (SPI_TypeDef * SPIx)
```

Function description

Enable error interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit controls the generation of an interrupt when an error condition occurs (CRCERR, OVR, MODF in SPI mode, FRE at TI mode).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_SPI_EnableIT_ERR
LL_SPI_EnableIT_RXNE

Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_RXNE (SPI_TypeDef * SPIx)
```

Function description

Enable Rx buffer not empty interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_SPI_EnableIT_RXNE
LL_SPI_EnableIT_TXE

Function name

```
__STATIC_INLINE void LL_SPI_EnableIT_TXE (SPI_TypeDef * SPIx)
```

Function description

Enable Tx buffer empty interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_SPI_EnableIT_TXE
LL_SPI_DisableIT_ERR

Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_ERR (SPI_TypeDef * SPIx)
```

Function description

Disable error interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit controls the generation of an interrupt when an error condition occurs (CRCERR, OVR, MODF in SPI mode, FRE at TI mode).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_SPI_DisableIT_ERR

```
LL_SPI_DisableIT_RXNE
```

Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_RXNE (SPI_TypeDef * SPIx)
```

Function description

Disable Rx buffer not empty interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_SPI_DisableIT_RXNE

```
LL_SPI_DisableIT_TXE
```

Function name

```
__STATIC_INLINE void LL_SPI_DisableIT_TXE (SPI_TypeDef * SPIx)
```

Function description

Disable Tx buffer empty interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_SPI_DisableIT_TXE

```
LL_SPI_IsEnabledIT_ERR
```

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_ERR (SPI_TypeDef * SPIx)
```

Function description

Check if error interrupt is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_SPI_IsEnabledIT_ERR

LL_SPI_IsEnabledIT_RXNE

Function name

__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_RXNE (SPI_TypeDef * SPIx)

Function description

Check if Rx buffer not empty interrupt is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_SPI_IsEnabledIT_RXNE

LL_SPI_IsEnabledIT_TXE

Function name

__STATIC_INLINE uint32_t LL_SPI_IsEnabledIT_TXE (SPI_TypeDef * SPIx)

Function description

Check if Tx buffer empty interrupt.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_SPI_IsEnabledIT_TXE

LL_SPI_EnableDMAReq_RX

Function name

__STATIC_INLINE void LL_SPI_EnableDMAReq_RX (SPI_TypeDef * SPIx)

Function description

Enable DMA Rx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_SPI_EnableDMAReq_RX
LL_SPI_DisableDMAReq_RX

Function name

__STATIC_INLINE void LL_SPI_DisableDMAReq_RX (SPI_TypeDef * SPIx)

Function description

Disable DMA Rx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_SPI_DisableDMAReq_RX
LL_SPI_IsEnabledDMAReq_RX

Function name

__STATIC_INLINE uint32_t LL_SPI_IsEnabledDMAReq_RX (SPI_TypeDef * SPIx)

Function description

Check if DMA Rx is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_SPI_IsEnabledDMAReq_RX
LL_SPI_EnableDMAReq_TX

Function name

__STATIC_INLINE void LL_SPI_EnableDMAReq_TX (SPI_TypeDef * SPIx)

Function description

Enable DMA Tx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_SPI_EnableDMAReq_TX
LL_SPI_DisableDMAReq_TX

Function name

```
__STATIC_INLINE void LL_SPI_DisableDMAReq_TX (SPI_TypeDef * SPIx)
```

Function description

Disable DMA Tx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_SPI_DisableDMAReq_TX

LL_SPI_IsEnabledDMAReq_TX

Function name

```
__STATIC_INLINE uint32_t LL_SPI_IsEnabledDMAReq_TX (SPI_TypeDef * SPIx)
```

Function description

Check if DMA Tx is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_SPI_IsEnabledDMAReq_TX

LL_SPI_DMA_GetRegAddr

Function name

```
__STATIC_INLINE uint32_t LL_SPI_DMA_GetRegAddr (SPI_TypeDef * SPIx)
```

Function description

Get the data register address used for DMA transfer.

Parameters

- **SPIx:** SPI Instance

Return values

- **Address:** of data register

Reference Manual to LL API cross reference:

- DR DR LL_SPI_DMA_GetRegAddr

LL_SPI_ReceiveData8

Function name

```
__STATIC_INLINE uint8_t LL_SPI_ReceiveData8 (SPI_TypeDef * SPIx)
```

Function description

Read 8-Bits in the data register.

Parameters

- **SPIx:** SPI Instance

Return values

- **RxData:** Value between Min_Data=0x00 and Max_Data=0xFF

Reference Manual to LL API cross reference:

- DR DR LL_SPI_ReceiveData8
LL_SPI_ReceiveData16

Function name

```
__STATIC_INLINE uint16_t LL_SPI_ReceiveData16 (SPI_TypeDef * SPIx)
```

Function description

Read 16-Bits in the data register.

Parameters

- **SPIx:** SPI Instance

Return values

- **RxData:** Value between Min_Data=0x00 and Max_Data=0xFFFF

Reference Manual to LL API cross reference:

- DR DR LL_SPI_ReceiveData16
LL_SPI_TransmitData8

Function name

```
__STATIC_INLINE void LL_SPI_TransmitData8 (SPI_TypeDef * SPIx, uint8_t TxData)
```

Function description

Write 8-Bits in the data register.

Parameters

- **SPIx:** SPI Instance
- **TxData:** Value between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_SPI_TransmitData8
LL_SPI_TransmitData16

Function name

```
__STATIC_INLINE void LL_SPI_TransmitData16 (SPI_TypeDef * SPIx, uint16_t TxData)
```

Function description

Write 16-Bits in the data register.

Parameters

- **SPIx:** SPI Instance
- **TxData:** Value between Min_Data=0x00 and Max_Data=0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_SPI_TransmitData16

LL_SPI_DeInit

Function name

ErrorStatus LL_SPI_DeInit (SPI_TypeDef * SPIx)

Function description

De-initialize the SPI registers to their default reset values.

Parameters

- **SPIx:** SPI Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: SPI registers are de-initialized
 - ERROR: SPI registers are not de-initialized

LL_SPI_Init

Function name

ErrorStatus LL_SPI_Init (SPI_TypeDef * SPIx, LL_SPI_InitTypeDef * SPI_InitStruct)

Function description

Initialize the SPI registers according to the specified parameters in SPI_InitStruct.

Parameters

- **SPIx:** SPI Instance
- **SPI_InitStruct:** pointer to a LL_SPI_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value. (Return always SUCCESS)

Notes

- As some bits in SPI configuration registers can only be written when the SPI is disabled (SPI_CR1_SPE bit =0), SPI peripheral should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.

LL_SPI_StructInit

Function name

void LL_SPI_StructInit (LL_SPI_InitTypeDef * SPI_InitStruct)

Function description

Set each LL_SPI_InitTypeDef field to default value.

Parameters

- **SPI_InitStruct:** pointer to a LL_SPI_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_I2S_Enable

Function name

__STATIC_INLINE void LL_I2S_Enable (SPI_TypeDef * SPIx)

Function description

Select I2S mode and Enable I2S peripheral.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR I2SMOD LL_I2S_Enable
- I2SCFGR I2SE LL_I2S_Enable

LL_I2S_Disable

Function name

__STATIC_INLINE void LL_I2S_Disable (SPI_TypeDef * SPIx)

Function description

Disable I2S peripheral.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR I2SE LL_I2S_Disable

LL_I2S_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_I2S_IsEnabled (SPI_TypeDef * SPIx)

Function description

Check if I2S peripheral is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- I2SCFGR I2SE LL_I2S_IsEnabled

LL_I2S_SetDataFormat

Function name

__STATIC_INLINE void LL_I2S_SetDataFormat (SPI_TypeDef * SPIx, uint32_t DataFormat)

Function description

Set I2S data frame length.

Parameters

- **SPIx:** SPI Instance
- **DataFormat:** This parameter can be one of the following values:
 - LL_I2S_DATAFORMAT_16B
 - LL_I2S_DATAFORMAT_16B_EXTENDED
 - LL_I2S_DATAFORMAT_24B
 - LL_I2S_DATAFORMAT_32B

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR DATLEN LL_I2S_SetDataFormat
- I2SCFGR CHLEN LL_I2S_SetDataFormat

LL_I2S_GetDataFormat

Function name

__STATIC_INLINE uint32_t LL_I2S_GetDataFormat (SPI_TypeDef * SPIx)

Function description

Get I2S data frame length.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_I2S_DATAFORMAT_16B
 - LL_I2S_DATAFORMAT_16B_EXTENDED
 - LL_I2S_DATAFORMAT_24B
 - LL_I2S_DATAFORMAT_32B

Reference Manual to LL API cross reference:

- I2SCFGR DATLEN LL_I2S_GetDataFormat
- I2SCFGR CHLEN LL_I2S_GetDataFormat

LL_I2S_SetClockPolarity

Function name

__STATIC_INLINE void LL_I2S_SetClockPolarity (SPI_TypeDef * SPIx, uint32_t ClockPolarity)

Function description

Set I2S clock polarity.

Parameters

- **SPIx:** SPI Instance
- **ClockPolarity:** This parameter can be one of the following values:
 - LL_I2S_POLARITY_LOW
 - LL_I2S_POLARITY_HIGH

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR CKPOL LL_I2S_SetClockPolarity
LL_I2S_GetClockPolarity

Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetClockPolarity (SPI_TypeDef * SPIx)
```

Function description

Get I2S clock polarity.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_I2S_POLARITY_LOW
 - LL_I2S_POLARITY_HIGH

Reference Manual to LL API cross reference:

- I2SCFGR CKPOL LL_I2S_GetClockPolarity
LL_I2S_SetStandard

Function name

```
__STATIC_INLINE void LL_I2S_SetStandard (SPI_TypeDef * SPIx, uint32_t Standard)
```

Function description

Set I2S standard protocol.

Parameters

- **SPIx:** SPI Instance
- **Standard:** This parameter can be one of the following values:
 - LL_I2S_STANDARD_PHILIPS
 - LL_I2S_STANDARD_MSB
 - LL_I2S_STANDARD_LSB
 - LL_I2S_STANDARD_PCM_SHORT
 - LL_I2S_STANDARD_PCM_LONG

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR I2SSTD LL_I2S_SetStandard
- I2SCFGR PCMSYNC LL_I2S_SetStandard
LL_I2S_GetStandard

Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetStandard (SPI_TypeDef * SPIx)
```

Function description

Get I2S standard protocol.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_I2S_STANDARD_PHILIPS
 - LL_I2S_STANDARD_MSB
 - LL_I2S_STANDARD_LSB
 - LL_I2S_STANDARD_PCM_SHORT
 - LL_I2S_STANDARD_PCM_LONG

Reference Manual to LL API cross reference:

- I2SCFGR I2SSTD LL_I2S_GetStandard
- I2SCFGR PCMSYNC LL_I2S_GetStandard

LL_I2S_SetTransferMode

Function name

__STATIC_INLINE void LL_I2S_SetTransferMode (SPI_TypeDef * SPIx, uint32_t Mode)

Function description

Set I2S transfer mode.

Parameters

- **SPIx:** SPI Instance
- **Mode:** This parameter can be one of the following values:
 - LL_I2S_MODE_SLAVE_TX
 - LL_I2S_MODE_SLAVE_RX
 - LL_I2S_MODE_MASTER_TX
 - LL_I2S_MODE_MASTER_RX

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SCFGR I2SCFG LL_I2S_SetTransferMode

LL_I2S_GetTransferMode

Function name

__STATIC_INLINE uint32_t LL_I2S_GetTransferMode (SPI_TypeDef * SPIx)

Function description

Get I2S transfer mode.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_I2S_MODE_SLAVE_TX
 - LL_I2S_MODE_SLAVE_RX
 - LL_I2S_MODE_MASTER_TX
 - LL_I2S_MODE_MASTER_RX

Reference Manual to LL API cross reference:

- I2SCFGR I2SCFG LL_I2S_GetTransferMode

LL_I2S_SetPrescalerLinear

Function name

```
__STATIC_INLINE void LL_I2S_SetPrescalerLinear (SPI_TypeDef * SPIx, uint8_t PrescalerLinear)
```

Function description

Set I2S linear prescaler.

Parameters

- **SPIx:** SPI Instance
- **PrescalerLinear:** Value between Min_Data=0x02 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SPR I2SDIV LL_I2S_SetPrescalerLinear
LL_I2S_GetPrescalerLinear

Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetPrescalerLinear (SPI_TypeDef * SPIx)
```

Function description

Get I2S linear prescaler.

Parameters

- **SPIx:** SPI Instance

Return values

- **PrescalerLinear:** Value between Min_Data=0x02 and Max_Data=0xFF

Reference Manual to LL API cross reference:

- I2SPR I2SDIV LL_I2S_GetPrescalerLinear
LL_I2S_SetPrescalerParity

Function name

```
__STATIC_INLINE void LL_I2S_SetPrescalerParity (SPI_TypeDef * SPIx, uint32_t PrescalerParity)
```

Function description

Set I2S parity prescaler.

Parameters

- **SPIx:** SPI Instance
- **PrescalerParity:** This parameter can be one of the following values:
 - LL_I2S_PRESCALER_PARITY_EVEN
 - LL_I2S_PRESCALER_PARITY_ODD

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SPR ODD LL_I2S_SetPrescalerParity
LL_I2S_GetPrescalerParity

Function name

```
__STATIC_INLINE uint32_t LL_I2S_GetPrescalerParity (SPI_TypeDef * SPIx)
```

Function description

Get I2S parity prescaler.

Parameters

- **SPIx:** SPI Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_I2S_PRESCALER_PARITY_EVEN
 - LL_I2S_PRESCALER_PARITY_ODD

Reference Manual to LL API cross reference:

- I2SPR ODD LL_I2S_GetPrescalerParity
LL_I2S_EnableMasterClock

Function name

__STATIC_INLINE void LL_I2S_EnableMasterClock (SPI_TypeDef * SPIx)

Function description

Enable the master clock output (Pin MCK)

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SPR MCKOE LL_I2S_EnableMasterClock
LL_I2S_DisableMasterClock

Function name

__STATIC_INLINE void LL_I2S_DisableMasterClock (SPI_TypeDef * SPIx)

Function description

Disable the master clock output (Pin MCK)

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- I2SPR MCKOE LL_I2S_DisableMasterClock
LL_I2S_IsEnabledMasterClock

Function name

__STATIC_INLINE uint32_t LL_I2S_IsEnabledMasterClock (SPI_TypeDef * SPIx)

Function description

Check if the master clock output (Pin MCK) is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- I2SPR MCKOE LL_I2S_IsEnabledMasterClock
LL_I2S_IsActiveFlag_RXNE

Function name

__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_RXNE (SPI_TypeDef * SPIx)

Function description

Check if Rx buffer is not empty.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR RXNE LL_I2S_IsActiveFlag_RXNE
LL_I2S_IsActiveFlag_TXE

Function name

__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_TXE (SPI_TypeDef * SPIx)

Function description

Check if Tx buffer is empty.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR TXE LL_I2S_IsActiveFlag_TXE
LL_I2S_IsActiveFlag_BSY

Function name

__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_BSY (SPI_TypeDef * SPIx)

Function description

Get busy flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR BSY LL_I2S_IsActiveFlag_BSY
LL_I2S_IsActiveFlag_OVR

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_OVR (SPI_TypeDef * SPIx)
```

Function description

Get overrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR OVR LL_I2S_IsActiveFlag_OVR

```
LL_I2S_IsActiveFlag_UDR
```

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_UDR (SPI_TypeDef * SPIx)
```

Function description

Get underrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR UDR LL_I2S_IsActiveFlag_UDR

```
LL_I2S_IsActiveFlag_FRE
```

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_FRE (SPI_TypeDef * SPIx)
```

Function description

Get frame format error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR FRE LL_I2S_IsActiveFlag_FRE

```
LL_I2S_IsActiveFlag_CHSIDE
```

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsActiveFlag_CHSIDE (SPI_TypeDef * SPIx)
```

Function description

Get channel side flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Notes

- 0: Channel Left has to be transmitted or has been received 1: Channel Right has to be transmitted or has been received It has no significance in PCM mode.

Reference Manual to LL API cross reference:

- SR CHSIDE LL_I2S_IsActiveFlag_CHSIDE
LL_I2S_ClearFlag_OVR

Function name

```
__STATIC_INLINE void LL_I2S_ClearFlag_OVR (SPI_TypeDef * SPIx)
```

Function description

Clear overrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR OVR LL_I2S_ClearFlag_OVR
LL_I2S_ClearFlag_UDR

Function name

```
__STATIC_INLINE void LL_I2S_ClearFlag_UDR (SPI_TypeDef * SPIx)
```

Function description

Clear underrun error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR UDR LL_I2S_ClearFlag_UDR
LL_I2S_ClearFlag_FRE

Function name

```
__STATIC_INLINE void LL_I2S_ClearFlag_FRE (SPI_TypeDef * SPIx)
```

Function description

Clear frame format error flag.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR FRE LL_I2S_ClearFlag_FRE
LL_I2S_EnableIT_ERR

Function name

__STATIC_INLINE void LL_I2S_EnableIT_ERR (SPI_TypeDef * SPIx)

Function description

Enable error IT.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Notes

- This bit controls the generation of an interrupt when an error condition occurs (OVR, UDR and FRE in I2S mode).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_I2S_EnableIT_ERR
LL_I2S_EnableIT_RXNE

Function name

__STATIC_INLINE void LL_I2S_EnableIT_RXNE (SPI_TypeDef * SPIx)

Function description

Enable Rx buffer not empty IT.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_I2S_EnableIT_RXNE
LL_I2S_EnableIT_TXE

Function name

__STATIC_INLINE void LL_I2S_EnableIT_TXE (SPI_TypeDef * SPIx)

Function description

Enable Tx buffer empty IT.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_I2S_EnableIT_TXE
LL_I2S_DisableIT_ERR

Function name

```
__STATIC_INLINE void LL_I2S_DisableIT_ERR (SPI_TypeDef * SPIx)
```

Function description

Disable error IT.

Parameters

- **SPIx**: SPI Instance

Return values

- **None:**

Notes

- This bit controls the generation of an interrupt when an error condition occurs (OVR, UDR and FRE in I2S mode).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_I2S_DisableIT_ERR
LL_I2S_DisableIT_RXNE

Function name

```
__STATIC_INLINE void LL_I2S_DisableIT_RXNE (SPI_TypeDef * SPIx)
```

Function description

Disable Rx buffer not empty IT.

Parameters

- **SPIx**: SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_I2S_DisableIT_RXNE
LL_I2S_DisableIT_TXE

Function name

```
__STATIC_INLINE void LL_I2S_DisableIT_TXE (SPI_TypeDef * SPIx)
```

Function description

Disable Tx buffer empty IT.

Parameters

- **SPIx**: SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_I2S_DisableIT_TXE
LL_I2S_IsEnabledIT_ERR

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_ERR (SPI_TypeDef * SPIx)
```

Function description

Check if ERR IT is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 ERRIE LL_I2S_IsEnabledIT_ERR

LL_I2S_IsEnabledIT_RXNE

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_RXNE (SPI_TypeDef * SPIx)
```

Function description

Check if RXNE IT is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 RXNEIE LL_I2S_IsEnabledIT_RXNE

LL_I2S_IsEnabledIT_TXE

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledIT_TXE (SPI_TypeDef * SPIx)
```

Function description

Check if TXE IT is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 TXEIE LL_I2S_IsEnabledIT_TXE

LL_I2S_EnableDMAReq_RX

Function name

```
__STATIC_INLINE void LL_I2S_EnableDMAReq_RX (SPI_TypeDef * SPIx)
```

Function description

Enable DMA Rx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_I2S_EnableDMAReq_RX
LL_I2S_DisableDMAReq_RX

Function name

```
__STATIC_INLINE void LL_I2S_DisableDMAReq_RX (SPI_TypeDef * SPIx)
```

Function description

Disable DMA Rx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_I2S_DisableDMAReq_RX
LL_I2S_IsEnabledDMAReq_RX

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledDMAReq_RX (SPI_TypeDef * SPIx)
```

Function description

Check if DMA Rx is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 RXDMAEN LL_I2S_IsEnabledDMAReq_RX
LL_I2S_EnableDMAReq_TX

Function name

```
__STATIC_INLINE void LL_I2S_EnableDMAReq_TX (SPI_TypeDef * SPIx)
```

Function description

Enable DMA Tx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_I2S_EnableDMAReq_TX
LL_I2S_DisableDMAReq_TX

Function name

```
__STATIC_INLINE void LL_I2S_DisableDMAReq_TX (SPI_TypeDef * SPIx)
```

Function description

Disable DMA Tx.

Parameters

- **SPIx:** SPI Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_I2S_DisableDMAReq_TX
LL_I2S_IsEnabledDMAReq_TX

Function name

```
__STATIC_INLINE uint32_t LL_I2S_IsEnabledDMAReq_TX (SPI_TypeDef * SPIx)
```

Function description

Check if DMA Tx is enabled.

Parameters

- **SPIx:** SPI Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR2 TXDMAEN LL_I2S_IsEnabledDMAReq_TX
LL_I2S_ReceiveData16

Function name

```
__STATIC_INLINE uint16_t LL_I2S_ReceiveData16 (SPI_TypeDef * SPIx)
```

Function description

Read 16-Bits in data register.

Parameters

- **SPIx:** SPI Instance

Return values

- **RxData:** Value between Min_Data=0x0000 and Max_Data=0xFFFF

Reference Manual to LL API cross reference:

- DR DR LL_I2S_ReceiveData16
LL_I2S_TransmitData16

Function name

```
__STATIC_INLINE void LL_I2S_TransmitData16 (SPI_TypeDef * SPIx, uint16_t TxData)
```

Function description

Write 16-Bits in data register.

Parameters

- **SPIx:** SPI Instance
- **TxData:** Value between Min_Data=0x0000 and Max_Data=0xFFFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_I2S_TransmitData16

LL_I2S_DeInit

Function name

ErrorStatus LL_I2S_DeInit (SPI_TypeDef * SPIx)

Function description

De-initialize the SPI/I2S registers to their default reset values.

Parameters

- **SPIx:** SPI Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: SPI registers are de-initialized
 - ERROR: SPI registers are not de-initialized

LL_I2S_Init

Function name

ErrorStatus LL_I2S_Init (SPI_TypeDef * SPIx, LL_I2S_InitTypeDef * I2S_InitStruct)

Function description

Initializes the SPI/I2S registers according to the specified parameters in I2S_InitStruct.

Parameters

- **SPIx:** SPI Instance
- **I2S_InitStruct:** pointer to a LL_I2S_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: SPI registers are Initialized
 - ERROR: SPI registers are not Initialized

Notes

- As some bits in SPI configuration registers can only be written when the SPI is disabled (SPI_CR1_SPE bit =0), SPI peripheral should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.

LL_I2S_StructInit

Function name

void LL_I2S_StructInit (LL_I2S_InitTypeDef * I2S_InitStruct)

Function description

Set each LL_I2S_InitTypeDef field to default value.

Parameters

- **I2S_InitStruct:** pointer to a LL_I2S_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_I2S_ConfigPrescaler

Function name

void LL_I2S_ConfigPrescaler (SPI_TypeDef * SPIx, uint32_t PrescalerLinear, uint32_t PrescalerParity)

Function description

Set linear and parity prescaler.

Parameters

- **SPIx:** SPI Instance
- **PrescalerLinear:** value Min_Data=0x02 and Max_Data=0xFF.
- **PrescalerParity:** This parameter can be one of the following values:
 - LL_I2S_PRESCALER_PARITY_EVEN
 - LL_I2S_PRESCALER_PARITY_ODD

Return values

- **None:**

Notes

- To calculate value of PrescalerLinear(I2SDIV[7:0] bits) and PrescalerParity(ODD bit) Check Audio frequency table and formulas inside Reference Manual (SPI/I2S).

63.3 SPI Firmware driver defines

The following section lists the various define and macros of the module.

63.3.1 SPI

SPI

Baud Rate Prescaler

LL_SPI_BAUDRATEPRESCALER_DIV2

BaudRate control equal to fPCLK/2

LL_SPI_BAUDRATEPRESCALER_DIV4

BaudRate control equal to fPCLK/4

LL_SPI_BAUDRATEPRESCALER_DIV8

BaudRate control equal to fPCLK/8

LL_SPI_BAUDRATEPRESCALER_DIV16

BaudRate control equal to fPCLK/16

LL_SPI_BAUDRATEPRESCALER_DIV32

BaudRate control equal to fPCLK/32

LL_SPI_BAUDRATEPRESCALER_DIV64

BaudRate control equal to fPCLK/64

LL_SPI_BAUDRATEPRESCALER_DIV128

BaudRate control equal to fPCLK/128

LL_SPI_BAUDRATEPRESCALER_DIV256

BaudRate control equal to fPCLK/256

Transmission Bit Order

LL_SPI_LSB_FIRST

Data is transmitted/received with the LSB first

LL_SPI_MSB_FIRST

Data is transmitted/received with the MSB first

CRC Calculation

LL_SPI_CRCCALCULATION_DISABLE

CRC calculation disabled

LL_SPI_CRCCALCULATION_ENABLE

CRC calculation enabled

Datawidth

LL_SPI_DATAWIDTH_8BIT

Data length for SPI transfer: 8 bits

LL_SPI_DATAWIDTH_16BIT

Data length for SPI transfer: 16 bits

Get Flags Defines

LL_SPI_SR_RXNE

Rx buffer not empty flag

LL_SPI_SR_TXE

Tx buffer empty flag

LL_SPI_SR_BSY

Busy flag

LL_SPI_SR_CRCERR

CRC error flag

LL_SPI_SR_MODF

Mode fault flag

LL_SPI_SR_OVR

Overrun flag

LL_SPI_SR_FRE

TI mode frame format error flag

IT Defines

LL_SPI_CR2_RXNEIE

Rx buffer not empty interrupt enable

LL_SPI_CR2_TXEIE

Tx buffer empty interrupt enable

LL_SPI_CR2_ERRIE

Error interrupt enable

LL_I2S_CR2_RXNEIE

Rx buffer not empty interrupt enable

LL_I2S_CR2_TXEIE

Tx buffer empty interrupt enable

LL_I2S_CR2_ERRIE

Error interrupt enable

Operation Mode

LL_SPI_MODE_MASTER

Master configuration

LL_SPI_MODE_SLAVE

Slave configuration

Slave Select Pin Mode

LL_SPI_NSS_SOFT

NSS managed internally. NSS pin not used and free

LL_SPI_NSS_HARD_INPUT

NSS pin used in Input. Only used in Master mode

LL_SPI_NSS_HARD_OUTPUT

NSS pin used in Output. Only used in Slave mode as chip select

Clock Phase

LL_SPI_PHASE_1EDGE

First clock transition is the first data capture edge

LL_SPI_PHASE_2EDGE

Second clock transition is the first data capture edge

Clock Polarity

LL_SPI_POLARITY_LOW

Clock to 0 when idle

LL_SPI_POLARITY_HIGH

Clock to 1 when idle

Serial Protocol

LL_SPI_PROTOCOL_MOTOROLA

Motorola mode. Used as default value

LL_SPI_PROTOCOL_TI

TI mode

Transfer Mode

LL_SPI_FULL_DUPLEX

Full-Duplex mode. Rx and Tx transfer on 2 lines

LL_SPI_SIMPLEX_RX

Simplex Rx mode. Rx transfer only on 1 line

LL_SPI_HALF_DUPLEX_RX

Half-Duplex Rx mode. Rx transfer on 1 line

LL_SPI_HALF_DUPLEX_TX

Half-Duplex Tx mode. Tx transfer on 1 line

Common Write and read registers Macros

LL_SPI_WriteReg

Description:

- Write a value in SPI register.

Parameters:

- `__INSTANCE__`: SPI Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_SPI_ReadReg

Description:

- Read a value in SPI register.

Parameters:

- `__INSTANCE__`: SPI Instance
- `__REG__`: Register to be read

Return value:

- Register: value

64 LL SYSTEM Generic Driver

64.1 SYSTEM Firmware driver API description

The following section lists the various functions of the SYSTEM library.

64.1.1 Detailed description of functions

`LL_SYSCFG_SetRemapMemory`

Function name

`__STATIC_INLINE void LL_SYSCFG_SetRemapMemory (uint32_t Memory)`

Function description

Set memory mapping at address 0x00000000.

Parameters

- **Memory:** This parameter can be one of the following values:
 - `LL_SYSCFG_REMAP_FLASH`
 - `LL_SYSCFG_REMAP_SYSTEMFLASH`
 - `LL_SYSCFG_REMAP_FSMC`
 - `LL_SYSCFG_REMAP_SRAM`

Return values

- **None:**

Reference Manual to LL API cross reference:

- `SYSCFG_MEMRMP MEM_MODE LL_SYSCFG_SetRemapMemory`
`LL_SYSCFG_GetRemapMemory`

Function name

`__STATIC_INLINE uint32_t LL_SYSCFG_GetRemapMemory (void )`

Function description

Get memory mapping at address 0x00000000.

Return values

- **Returned:** value can be one of the following values:
 - `LL_SYSCFG_REMAP_FLASH`
 - `LL_SYSCFG_REMAP_SYSTEMFLASH`
 - `LL_SYSCFG_REMAP_SRAM`
 - `LL_SYSCFG_REMAP_FSMC`

Reference Manual to LL API cross reference:

- `SYSCFG_MEMRMP MEM_MODE LL_SYSCFG_GetRemapMemory`
`LL_SYSCFG_EnableCompensationCell`

Function name

`__STATIC_INLINE void LL_SYSCFG_EnableCompensationCell (void )`

Function description

Enables the Compensation cell Power Down.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V

Reference Manual to LL API cross reference:

- SYSCFG_CMPCR CMP_PD LL_SYSCFG_EnableCompensationCell
LL_SYSCFG_DisableCompensationCell

Function name

__STATIC_INLINE void LL_SYSCFG_DisableCompensationCell (void)

Function description

Disables the Compensation cell Power Down.

Return values

- **None:**

Notes

- The I/O compensation cell can be used only when the device supply voltage ranges from 2.4 to 3.6 V

Reference Manual to LL API cross reference:

- SYSCFG_CMPCR CMP_PD LL_SYSCFG_DisableCompensationCell
LL_SYSCFG_IsActiveFlag_CMPCR

Function name

__STATIC_INLINE uint32_t LL_SYSCFG_IsActiveFlag_CMPCR (void)

Function description

Get Compensation Cell ready Flag.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SYSCFG_CMPCR READY LL_SYSCFG_IsActiveFlag_CMPCR
LL_SYSCFG_SetPHYInterface

Function name

__STATIC_INLINE void LL_SYSCFG_SetPHYInterface (uint32_t Interface)

Function description

Select Ethernet PHY interface.

Parameters

- **Interface:** This parameter can be one of the following values:
 - LL_SYSCFG_PMC_ETHMII
 - LL_SYSCFG_PMC_ETHRMII

Return values

- **None:**

Reference Manual to LL API cross reference:

- SYSCFG_PMC MII_RMII_SEL LL_SYSCFG_SetPHYInterface

LL_SYSCFG_GetPHYInterface

Function name

__STATIC_INLINE uint32_t LL_SYSCFG_GetPHYInterface (void)

Function description

Get Ethernet PHY interface.

Return values

- **Returned:** value can be one of the following values:
 - LL_SYSCFG_PMC_ETHMII
 - LL_SYSCFG_PMC_ETHRMII
- **None:**

Reference Manual to LL API cross reference:

- SYSCFG_PMC MII_RMII_SEL LL_SYSCFG_GetPHYInterface

LL_SYSCFG_SetEXTISource

Function name

__STATIC_INLINE void LL_SYSCFG_SetEXTISource (uint32_t Port, uint32_t Line)

Function description

Configure source input for the EXTI external interrupt.

Parameters

- **Port:** This parameter can be one of the following values:
 - LL_SYSCFG_EXTI_PORTA
 - LL_SYSCFG_EXTI_PORTB
 - LL_SYSCFG_EXTI_PORTC
 - LL_SYSCFG_EXTI_PORTD
 - LL_SYSCFG_EXTI_PORTE
 - LL_SYSCFG_EXTI_PORTF (*)
 - LL_SYSCFG_EXTI_PORTG (*)
 - LL_SYSCFG_EXTI_PORTH
 (*) value not defined in all devices
- **Line:** This parameter can be one of the following values:
 - LL_SYSCFG_EXTI_LINE0
 - LL_SYSCFG_EXTI_LINE1
 - LL_SYSCFG_EXTI_LINE2
 - LL_SYSCFG_EXTI_LINE3
 - LL_SYSCFG_EXTI_LINE4
 - LL_SYSCFG_EXTI_LINE5
 - LL_SYSCFG_EXTI_LINE6
 - LL_SYSCFG_EXTI_LINE7
 - LL_SYSCFG_EXTI_LINE8
 - LL_SYSCFG_EXTI_LINE9
 - LL_SYSCFG_EXTI_LINE10
 - LL_SYSCFG_EXTI_LINE11
 - LL_SYSCFG_EXTI_LINE12
 - LL_SYSCFG_EXTI_LINE13
 - LL_SYSCFG_EXTI_LINE14
 - LL_SYSCFG_EXTI_LINE15

Return values

- **None:**

Reference Manual to LL API cross reference:

- SYSCFG_EXTICR1 EXTIx LL_SYSCFG_SetEXTISource
- SYSCFG_EXTICR2 EXTIx LL_SYSCFG_SetEXTISource
- SYSCFG_EXTICR3 EXTIx LL_SYSCFG_SetEXTISource
- SYSCFG_EXTICR4 EXTIx LL_SYSCFG_SetEXTISource

LL_SYSCFG_GetEXTISource

Function name

__STATIC_INLINE uint32_t LL_SYSCFG_GetEXTISource (uint32_t Line)

Function description

Get the configured defined for specific EXTI Line.

Parameters

- **Line:** This parameter can be one of the following values:
 - LL_SYSCFG_EXTI_LINE0
 - LL_SYSCFG_EXTI_LINE1
 - LL_SYSCFG_EXTI_LINE2
 - LL_SYSCFG_EXTI_LINE3
 - LL_SYSCFG_EXTI_LINE4
 - LL_SYSCFG_EXTI_LINE5
 - LL_SYSCFG_EXTI_LINE6
 - LL_SYSCFG_EXTI_LINE7
 - LL_SYSCFG_EXTI_LINE8
 - LL_SYSCFG_EXTI_LINE9
 - LL_SYSCFG_EXTI_LINE10
 - LL_SYSCFG_EXTI_LINE11
 - LL_SYSCFG_EXTI_LINE12
 - LL_SYSCFG_EXTI_LINE13
 - LL_SYSCFG_EXTI_LINE14
 - LL_SYSCFG_EXTI_LINE15

Return values

- **Returned:** value can be one of the following values:
 - LL_SYSCFG_EXTI_PORTA
 - LL_SYSCFG_EXTI_PORTB
 - LL_SYSCFG_EXTI_PORTC
 - LL_SYSCFG_EXTI_PORTD
 - LL_SYSCFG_EXTI_PORTE
 - LL_SYSCFG_EXTI_PORTF (*)
 - LL_SYSCFG_EXTI_PORTG (*)
 - LL_SYSCFG_EXTI_PORTH (*) value not defined in all devices

Reference Manual to LL API cross reference:

- SYSCFG_EXTICR1 EXTIx LL_SYSCFG_GetEXTISource
- SYSCFG_EXTICR2 EXTIx LL_SYSCFG_GetEXTISource
- SYSCFG_EXTICR3 EXTIx LL_SYSCFG_GetEXTISource
- SYSCFG_EXTICR4 EXTIx LL_SYSCFG_GetEXTISource

LL_DBGMCU_GetDeviceID

Function name

__STATIC_INLINE uint32_t LL_DBGMCU_GetDeviceID (void)

Function description

Return the device identifier.

Return values

- **Values:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- For STM32F2xxx ,the device ID is 0x411

Reference Manual to LL API cross reference:

- DBGMCU_IDCODE DEV_ID LL_DBGMCU_GetDeviceID

LL_DBGMCU_GetRevisionID

Function name

__STATIC_INLINE uint32_t LL_DBGMCU_GetRevisionID (void)

Function description

Return the device revision identifier.

Return values

- **Values:** between Min_Data=0x00 and Max_Data=0xFFFF

Notes

- This field indicates the revision of the device. For example, it is read as revA -> 0x1000, revZ -> 0x1001, revB -> 0x2000, revY -> 0x2001, revX -> 0x2003, rev1 -> 0x2007, revV -> 0x200F, rev2 -> 0x201F

Reference Manual to LL API cross reference:

- DBGMCU_IDCODE REV_ID LL_DBGMCU_GetRevisionID

LL_DBGMCU_EnableDBGSleepMode

Function name

__STATIC_INLINE void LL_DBGMCU_EnableDBGSleepMode (void)

Function description

Enable the Debug Module during SLEEP mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_SLEEP LL_DBGMCU_EnableDBGSleepMode

LL_DBGMCU_DisableDBGSleepMode

Function name

__STATIC_INLINE void LL_DBGMCU_DisableDBGSleepMode (void)

Function description

Disable the Debug Module during SLEEP mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_SLEEP LL_DBGMCU_DisableDBGSleepMode
LL_DBGMCU_EnableDBGStopMode

Function name

__STATIC_INLINE void LL_DBGMCU_EnableDBGStopMode (void)

Function description

Enable the Debug Module during STOP mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_STOP LL_DBGMCU_EnableDBGStopMode
LL_DBGMCU_DisableDBGStopMode

Function name

__STATIC_INLINE void LL_DBGMCU_DisableDBGStopMode (void)

Function description

Disable the Debug Module during STOP mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_STOP LL_DBGMCU_DisableDBGStopMode
LL_DBGMCU_EnableDBGStandbyMode

Function name

__STATIC_INLINE void LL_DBGMCU_EnableDBGStandbyMode (void)

Function description

Enable the Debug Module during STANDBY mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_STANDBY LL_DBGMCU_EnableDBGStandbyMode
LL_DBGMCU_DisableDBGStandbyMode

Function name

__STATIC_INLINE void LL_DBGMCU_DisableDBGStandbyMode (void)

Function description

Disable the Debug Module during STANDBY mode.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR DBG_STANDBY LL_DBGMCU_DisableDBGStandbyMode

LL_DBGMCU_SetTracePinAssignment

Function name

__STATIC_INLINE void LL_DBGMCU_SetTracePinAssignment (uint32_t PinAssignment)

Function description

Set Trace pin assignment control.

Parameters

- **PinAssignment:** This parameter can be one of the following values:
 - LL_DBGMCU_TRACE_NONE
 - LL_DBGMCU_TRACE_ASYNC
 - LL_DBGMCU_TRACE_SYNC_SIZE1
 - LL_DBGMCU_TRACE_SYNC_SIZE2
 - LL_DBGMCU_TRACE_SYNC_SIZE4

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_CR TRACE_IOEN LL_DBGMCU_SetTracePinAssignment
- DBGMCU_CR TRACE_MODE LL_DBGMCU_SetTracePinAssignment

LL_DBGMCU_GetTracePinAssignment

Function name

__STATIC_INLINE uint32_t LL_DBGMCU_GetTracePinAssignment (void)

Function description

Get Trace pin assignment control.

Return values

- **Returned:** value can be one of the following values:
 - LL_DBGMCU_TRACE_NONE
 - LL_DBGMCU_TRACE_ASYNC
 - LL_DBGMCU_TRACE_SYNC_SIZE1
 - LL_DBGMCU_TRACE_SYNC_SIZE2
 - LL_DBGMCU_TRACE_SYNC_SIZE4

Reference Manual to LL API cross reference:

- DBGMCU_CR TRACE_IOEN LL_DBGMCU_GetTracePinAssignment
- DBGMCU_CR TRACE_MODE LL_DBGMCU_GetTracePinAssignment

LL_DBGMCU_APB1_GRP1_FreezePeriph

Function name

__STATIC_INLINE void LL_DBGMCU_APB1_GRP1_FreezePeriph (uint32_t Periphs)

Function description

Freeze APB1 peripherals (group1 peripherals)

Parameters

- **Periphs:** This parameter can be a combination of the following values:

- LL_DBGMCU_APB1_GRP1_TIM2_STOP
- LL_DBGMCU_APB1_GRP1_TIM3_STOP
- LL_DBGMCU_APB1_GRP1_TIM4_STOP
- LL_DBGMCU_APB1_GRP1_TIM5_STOP
- LL_DBGMCU_APB1_GRP1_TIM6_STOP
- LL_DBGMCU_APB1_GRP1_TIM7_STOP
- LL_DBGMCU_APB1_GRP1_TIM12_STOP
- LL_DBGMCU_APB1_GRP1_TIM13_STOP
- LL_DBGMCU_APB1_GRP1_TIM14_STOP
- LL_DBGMCU_APB1_GRP1_RTC_STOP
- LL_DBGMCU_APB1_GRP1_WWDG_STOP
- LL_DBGMCU_APB1_GRP1_IWDG_STOP
- LL_DBGMCU_APB1_GRP1_I2C1_STOP
- LL_DBGMCU_APB1_GRP1_I2C2_STOP
- LL_DBGMCU_APB1_GRP1_I2C3_STOP
- LL_DBGMCU_APB1_GRP1_CAN1_STOP
- LL_DBGMCU_APB1_GRP1_CAN2_STOP

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_APB1_FZ_DBG_TIM2_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM3_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM4_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM5_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM6_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM7_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM12_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM13_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM14_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_RTC_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_WWDG_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_IWDG_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C1_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C2_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C3_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_CAN1_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph
- DBGMCU_APB1_FZ_DBG_CAN2_STOP LL_DBGMCU_APB1_GRP1_FreezePeriph

LL_DBGMCU_APB1_GRP1_UnFreezePeriph

Function name

__STATIC_INLINE void LL_DBGMCU_APB1_GRP1_UnFreezePeriph (uint32_t Periphs)

Function description

Unfreeze APB1 peripherals (group1 peripherals)

Parameters

- **Periphs:** This parameter can be a combination of the following values:

- LL_DBGMCU_APB1_GRP1_TIM2_STOP
- LL_DBGMCU_APB1_GRP1_TIM3_STOP
- LL_DBGMCU_APB1_GRP1_TIM4_STOP
- LL_DBGMCU_APB1_GRP1_TIM5_STOP
- LL_DBGMCU_APB1_GRP1_TIM6_STOP
- LL_DBGMCU_APB1_GRP1_TIM7_STOP
- LL_DBGMCU_APB1_GRP1_TIM12_STOP
- LL_DBGMCU_APB1_GRP1_TIM13_STOP
- LL_DBGMCU_APB1_GRP1_TIM14_STOP
- LL_DBGMCU_APB1_GRP1_RTC_STOP
- LL_DBGMCU_APB1_GRP1_WWDG_STOP
- LL_DBGMCU_APB1_GRP1_IWDG_STOP
- LL_DBGMCU_APB1_GRP1_I2C1_STOP
- LL_DBGMCU_APB1_GRP1_I2C2_STOP
- LL_DBGMCU_APB1_GRP1_I2C3_STOP
- LL_DBGMCU_APB1_GRP1_CAN1_STOP
- LL_DBGMCU_APB1_GRP1_CAN2_STOP

(*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_APB1_FZ_DBG_TIM2_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM3_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM4_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM5_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM6_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM7_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM12_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM13_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_TIM14_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_RTC_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_WWDG_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_IWDG_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C1_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C2_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_I2C3_SMBUS_TIMEOUT LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_CAN1_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph
- DBGMCU_APB1_FZ_DBG_CAN2_STOP LL_DBGMCU_APB1_GRP1_UnFreezePeriph

LL_DBGMCU_APB2_GRP1_FreezePeriph

Function name

__STATIC_INLINE void LL_DBGMCU_APB2_GRP1_FreezePeriph (uint32_t Periphs)

Function description

Freeze APB2 peripherals.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_DBGMCU_APB2_GRP1_TIM1_STOP
 - LL_DBGMCU_APB2_GRP1_TIM8_STOP
 - LL_DBGMCU_APB2_GRP1_TIM9_STOP
 - LL_DBGMCU_APB2_GRP1_TIM10_STOP
 - LL_DBGMCU_APB2_GRP1_TIM11_STOP
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_APB2_FZ_DBG_TIM1_STOP LL_DBGMCU_APB2_GRP1_FreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM8_STOP LL_DBGMCU_APB2_GRP1_FreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM9_STOP LL_DBGMCU_APB2_GRP1_FreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM10_STOP LL_DBGMCU_APB2_GRP1_FreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM11_STOP LL_DBGMCU_APB2_GRP1_FreezePeriph

LL_DBGMCU_APB2_GRP1_UnFreezePeriph

Function name

__STATIC_INLINE void LL_DBGMCU_APB2_GRP1_UnFreezePeriph (uint32_t Periphs)

Function description

Unfreeze APB2 peripherals.

Parameters

- **Periphs:** This parameter can be a combination of the following values:
 - LL_DBGMCU_APB2_GRP1_TIM1_STOP
 - LL_DBGMCU_APB2_GRP1_TIM8_STOP
 - LL_DBGMCU_APB2_GRP1_TIM9_STOP
 - LL_DBGMCU_APB2_GRP1_TIM10_STOP
 - LL_DBGMCU_APB2_GRP1_TIM11_STOP
 (*) value not defined in all devices.

Return values

- **None:**

Reference Manual to LL API cross reference:

- DBGMCU_APB2_FZ_DBG_TIM1_STOP LL_DBGMCU_APB2_GRP1_UnFreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM8_STOP LL_DBGMCU_APB2_GRP1_UnFreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM9_STOP LL_DBGMCU_APB2_GRP1_UnFreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM10_STOP LL_DBGMCU_APB2_GRP1_UnFreezePeriph
- DBGMCU_APB2_FZ_DBG_TIM11_STOP LL_DBGMCU_APB2_GRP1_UnFreezePeriph

LL_FLASH_SetLatency

Function name

__STATIC_INLINE void LL_FLASH_SetLatency (uint32_t Latency)

Function description

Set FLASH Latency.

Parameters

- **Latency:** This parameter can be one of the following values:
 - LL_FLASH_LATENCY_0
 - LL_FLASH_LATENCY_1
 - LL_FLASH_LATENCY_2
 - LL_FLASH_LATENCY_3
 - LL_FLASH_LATENCY_4
 - LL_FLASH_LATENCY_5
 - LL_FLASH_LATENCY_6
 - LL_FLASH_LATENCY_7

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR LATENCY LL_FLASH_SetLatency
LL_FLASH_GetLatency

Function name

__STATIC_INLINE uint32_t LL_FLASH_GetLatency (void)

Function description

Get FLASH Latency.

Return values

- **Returned:** value can be one of the following values:
 - LL_FLASH_LATENCY_0
 - LL_FLASH_LATENCY_1
 - LL_FLASH_LATENCY_2
 - LL_FLASH_LATENCY_3
 - LL_FLASH_LATENCY_4
 - LL_FLASH_LATENCY_5
 - LL_FLASH_LATENCY_6
 - LL_FLASH_LATENCY_7

Reference Manual to LL API cross reference:

- FLASH_ACR LATENCY LL_FLASH_GetLatency
LL_FLASH_EnablePrefetch

Function name

__STATIC_INLINE void LL_FLASH_EnablePrefetch (void)

Function description

Enable Prefetch.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR PRFTEN LL_FLASH_EnablePrefetch
LL_FLASH_DisablePrefetch

Function name

__STATIC_INLINE void LL_FLASH_DisablePrefetch (void)

Function description

Disable Prefetch.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR PRFTEN LL_FLASH_DisablePrefetch
LL_FLASH_IsPrefetchEnabled

Function name

__STATIC_INLINE uint32_t LL_FLASH_IsPrefetchEnabled (void)

Function description

Check if Prefetch buffer is enabled.

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- FLASH_ACR PRFTEN LL_FLASH_IsPrefetchEnabled
LL_FLASH_EnableInstCache

Function name

__STATIC_INLINE void LL_FLASH_EnableInstCache (void)

Function description

Enable Instruction cache.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR ICEN LL_FLASH_EnableInstCache
LL_FLASH_DisableInstCache

Function name

__STATIC_INLINE void LL_FLASH_DisableInstCache (void)

Function description

Disable Instruction cache.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR ICEN LL_FLASH_DisableInstCache
LL_FLASH_EnableDataCache

Function name

__STATIC_INLINE void LL_FLASH_EnableDataCache (void)

Function description

Enable Data cache.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR DCEN LL_FLASH_EnableDataCache
LL_FLASH_DisableDataCache

Function name

__STATIC_INLINE void LL_FLASH_DisableDataCache (void)

Function description

Disable Data cache.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR DCEN LL_FLASH_DisableDataCache
LL_FLASH_EnableInstCacheReset

Function name

__STATIC_INLINE void LL_FLASH_EnableInstCacheReset (void)

Function description

Enable Instruction cache reset.

Return values

- **None:**

Notes

- bit can be written only when the instruction cache is disabled

Reference Manual to LL API cross reference:

- FLASH_ACR ICRST LL_FLASH_EnableInstCacheReset
LL_FLASH_DisableInstCacheReset

Function name

__STATIC_INLINE void LL_FLASH_DisableInstCacheReset (void)

Function description

Disable Instruction cache reset.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR ICRST LL_FLASH_DisableInstCacheReset
LL_FLASH_EnableDataCacheReset

Function name

__STATIC_INLINE void LL_FLASH_EnableDataCacheReset (void)

Function description

Enable Data cache reset.

Return values

- **None:**

Notes

- bit can be written only when the data cache is disabled

Reference Manual to LL API cross reference:

- FLASH_ACR DCRST LL_FLASH_EnableDataCacheReset
LL_FLASH_DisableDataCacheReset

Function name

__STATIC_INLINE void LL_FLASH_DisableDataCacheReset (void)

Function description

Disable Data cache reset.

Return values

- **None:**

Reference Manual to LL API cross reference:

- FLASH_ACR DCRST LL_FLASH_DisableDataCacheReset

64.2 SYSTEM Firmware driver defines

The following section lists the various define and macros of the module.

64.2.1 SYSTEM

SYSTEM

DBGMCU APB1 GRP1 STOP IP

LL_DBGMCU_APB1_GRP1_TIM2_STOP

TIM2 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM3_STOP

TIM3 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM4_STOP

TIM4 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM5_STOP

TIM5 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM6_STOP

TIM6 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM7_STOP

TIM7 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM12_STOP

TIM12 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM13_STOP

TIM13 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_TIM14_STOP

TIM14 counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_RTC_STOP

RTC counter stopped when core is halted

LL_DBGMCU_APB1_GRP1_WWDG_STOP

Debug Window Watchdog stopped when Core is halted

LL_DBGMCU_APB1_GRP1_IWDG_STOP

Debug Independent Watchdog stopped when Core is halted

LL_DBGMCU_APB1_GRP1_I2C1_STOP

I2C1 SMBUS timeout mode stopped when Core is halted

LL_DBGMCU_APB1_GRP1_I2C2_STOP

I2C2 SMBUS timeout mode stopped when Core is halted

LL_DBGMCU_APB1_GRP1_I2C3_STOP

I2C3 SMBUS timeout mode stopped when Core is halted

LL_DBGMCU_APB1_GRP1_CAN1_STOP

CAN1 debug stopped when Core is halted

LL_DBGMCU_APB1_GRP1_CAN2_STOP

CAN2 debug stopped when Core is halted

DBGMCU APB2 GRP1 STOP IP

LL_DBGMCU_APB2_GRP1_TIM1_STOP

TIM1 counter stopped when core is halted

LL_DBGMCU_APB2_GRP1_TIM8_STOP

TIM8 counter stopped when core is halted

LL_DBGMCU_APB2_GRP1_TIM9_STOP

TIM9 counter stopped when core is halted

LL_DBGMCU_APB2_GRP1_TIM10_STOP

TIM10 counter stopped when core is halted

LL_DBGMCU_APB2_GRP1_TIM11_STOP

TIM11 counter stopped when core is halted

SYSCFG EXTI LINE

LL_SYSCFG_EXTI_LINE0

EXTI_POSITION_0 | EXTICR[0]

LL_SYSCFG_EXTI_LINE1

EXTI_POSITION_4 | EXTICR[0]

LL_SYSCFG_EXTI_LINE2

EXTI_POSITION_8 | EXTICR[0]

LL_SYSCFG_EXTI_LINE3

EXTI_POSITION_12 | EXTICR[0]

LL_SYSCFG_EXTI_LINE4

EXTI_POSITION_0 | EXTICR[1]

LL_SYSCFG_EXTI_LINE5

EXTI_POSITION_4 | EXTICR[1]

LL_SYSCFG_EXTI_LINE6

EXTI_POSITION_8 | EXTICR[1]

LL_SYSCFG_EXTI_LINE7

EXTI_POSITION_12 | EXTICR[1]

LL_SYSCFG_EXTI_LINE8

EXTI_POSITION_0 | EXTICR[2]

LL_SYSCFG_EXTI_LINE9

EXTI_POSITION_4 | EXTICR[2]

LL_SYSCFG_EXTI_LINE10

EXTI_POSITION_8 | EXTICR[2]

LL_SYSCFG_EXTI_LINE11

EXTI_POSITION_12 | EXTICR[2]

LL_SYSCFG_EXTI_LINE12

EXTI_POSITION_0 | EXTICR[3]

LL_SYSCFG_EXTI_LINE13

EXTI_POSITION_4 | EXTICR[3]

LL_SYSCFG_EXTI_LINE14

EXTI_POSITION_8 | EXTICR[3]

LL_SYSCFG_EXTI_LINE15

EXTI_POSITION_12 | EXTICR[3]

SYSCFG EXTI PORT

LL_SYSCFG_EXTI_PORTA

EXTI PORT A

LL_SYSCFG_EXTI_PORTB

EXTI PORT B

LL_SYSCFG_EXTI_PORTC

EXTI PORT C

LL_SYSCFG_EXTI_PORTD

EXTI PORT D

LL_SYSCFG_EXTI_PORTE

EXTI PORT E

LL_SYSCFG_EXTI_PORTF

EXTI PORT F

LL_SYSCFG_EXTI_PORTG

EXTI PORT G

LL_SYSCFG_EXTI_PORTH

EXTI PORT H

LL_SYSCFG_EXTI_PORTI

EXTI PORT I

FLASH LATENCY

LL_FLASH_LATENCY_0

FLASH Zero wait state

LL_FLASH_LATENCY_1

FLASH One wait state

LL_FLASH_LATENCY_2

FLASH Two wait states

LL_FLASH_LATENCY_3

FLASH Three wait states

LL_FLASH_LATENCY_4

FLASH Four wait states

LL_FLASH_LATENCY_5

FLASH five wait state

LL_FLASH_LATENCY_6

FLASH six wait state

LL_FLASH_LATENCY_7

FLASH seven wait states

SYSCFG PMC

LL_SYSCFG_PMC_ETHMII

ETH Media MII interface

LL_SYSCFG_PMC_ETHRMII

ETH Media RMII interface

SYSCFG REMAP

LL_SYSCFG_REMAP_FLASH

Main Flash memory mapped at 0x00000000

LL_SYSCFG_REMAP_SYSTEMFLASH

System Flash memory mapped at 0x00000000

LL_SYSCFG_REMAP_FSMC

FSMC(NOR/PSRAM 1 and 2) mapped at 0x00000000

LL_SYSCFG_REMAP_SRAM

SRAM1 mapped at 0x00000000

DBGMCU TRACE Pin Assignment

LL_DBGMCU_TRACE_NONE

TRACE pins not assigned (default state)

LL_DBGMCU_TRACE_ASYNC

TRACE pin assignment for Asynchronous Mode

LL_DBGMCU_TRACE_SYNC_SIZE1

TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 1

LL_DBGMCU_TRACE_SYNC_SIZE2

TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 2

LL_DBGMCU_TRACE_SYNC_SIZE4

TRACE pin assignment for Synchronous Mode with a TRACEDATA size of 4

65 LL TIM Generic Driver

65.1 TIM Firmware driver registers structures

65.1.1 LL_TIM_InitTypeDef

LL_TIM_InitTypeDef is defined in the stm32f2xx_ll_tim.h

Data Fields

- **uint16_t Prescaler**
- **uint32_t CounterMode**
- **uint32_t Autoreload**
- **uint32_t ClockDivision**
- **uint8_t RepetitionCounter**

Field Documentation

- **uint16_t LL_TIM_InitTypeDef::Prescaler**
Specifies the prescaler value used to divide the TIM clock. This parameter can be a number between Min_Data=0x0000 and Max_Data=0xFFFF. This feature can be modified afterwards using unitary function **LL_TIM_SetPrescaler()**.
- **uint32_t LL_TIM_InitTypeDef::CounterMode**
Specifies the counter mode. This parameter can be a value of **TIM_LL_EC_COUNTERMODE**. This feature can be modified afterwards using unitary function **LL_TIM_SetCounterMode()**.
- **uint32_t LL_TIM_InitTypeDef::Autoreload**
Specifies the auto reload value to be loaded into the active Auto-Reload Register at the next update event. This parameter must be a number between Min_Data=0x0000 and Max_Data=0xFFFF. Some timer instances may support 32 bits counters. In that case this parameter must be a number between 0x0000 and 0xFFFFFFFF. This feature can be modified afterwards using unitary function **LL_TIM_SetAutoReload()**.
- **uint32_t LL_TIM_InitTypeDef::ClockDivision**
Specifies the clock division. This parameter can be a value of **TIM_LL_EC_CLOCKDIVISION**. This feature can be modified afterwards using unitary function **LL_TIM_SetClockDivision()**.
- **uint8_t LL_TIM_InitTypeDef::RepetitionCounter**
Specifies the repetition counter value. Each time the RCR downcounter reaches zero, an update event is generated and counting restarts from the RCR value (N). This means in PWM mode that (N+1) corresponds to:
 - the number of PWM periods in edge-aligned mode
 - the number of half PWM period in center-aligned mode This parameter must be a number between 0x00 and 0xFF.
 This feature can be modified afterwards using unitary function **LL_TIM_SetRepetitionCounter()**.

65.1.2 LL_TIM_OC_InitTypeDef

LL_TIM_OC_InitTypeDef is defined in the stm32f2xx_ll_tim.h

Data Fields

- **uint32_t OCMODE**
- **uint32_t OCState**
- **uint32_t OCNState**
- **uint32_t CompareValue**
- **uint32_t OCPolarity**
- **uint32_t OCNPolarity**
- **uint32_t OCIdleState**
- **uint32_t OCNIdleState**

Field Documentation

- **`uint32_t LL_TIM_OC_InitTypeDef::OCMode`**
Specifies the output mode. This parameter can be a value of `TIM_LL_EC_OCMode`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetMode()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCState`**
Specifies the TIM Output Compare state. This parameter can be a value of `TIM_LL_EC_OCSTATE`. This feature can be modified afterwards using unitary functions `LL_TIM_CC_EnableChannel()` or `LL_TIM_CC_DisableChannel()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCNState`**
Specifies the TIM complementary Output Compare state. This parameter can be a value of `TIM_LL_EC_OCSTATE`. This feature can be modified afterwards using unitary functions `LL_TIM_CC_EnableChannel()` or `LL_TIM_CC_DisableChannel()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::CompareValue`**
Specifies the Compare value to be loaded into the Capture Compare Register. This parameter can be a number between `Min_Data=0x0000` and `Max_Data=0xFFFF`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetCompareCHx (x=1..6)`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCPolarity`**
Specifies the output polarity. This parameter can be a value of `TIM_LL_EC_OCPOLARITY`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetPolarity()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCNPolarity`**
Specifies the complementary output polarity. This parameter can be a value of `TIM_LL_EC_OCPOLARITY`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetPolarity()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of `TIM_LL_EC_OCIDLESTATE`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetIdleState()`.
- **`uint32_t LL_TIM_OC_InitTypeDef::OCNIdleState`**
Specifies the TIM Output Compare pin state during Idle state. This parameter can be a value of `TIM_LL_EC_OCIDLESTATE`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetIdleState()`.

65.1.3

LL_TIM_IC_InitTypeDef

`LL_TIM_IC_InitTypeDef` is defined in the `stm32f2xx_ll_tim.h`

Data Fields

- **`uint32_t ICPolarity`**
- **`uint32_t ICActiveInput`**
- **`uint32_t ICPrescaler`**
- **`uint32_t ICFILTER`**

Field Documentation

- **`uint32_t LL_TIM_IC_InitTypeDef::ICPolarity`**
Specifies the active edge of the input signal. This parameter can be a value of `TIM_LL_EC_IC_POLARITY`. This feature can be modified afterwards using unitary function `LL_TIM_IC_SetPolarity()`.
- **`uint32_t LL_TIM_IC_InitTypeDef::ICActiveInput`**
Specifies the input. This parameter can be a value of `TIM_LL_EC_ACTIVEINPUT`. This feature can be modified afterwards using unitary function `LL_TIM_IC_SetActiveInput()`.
- **`uint32_t LL_TIM_IC_InitTypeDef::ICPrescaler`**
Specifies the Input Capture Prescaler. This parameter can be a value of `TIM_LL_EC_ICPSC`. This feature can be modified afterwards using unitary function `LL_TIM_IC_SetPrescaler()`.
- **`uint32_t LL_TIM_IC_InitTypeDef::ICFilter`**
Specifies the input capture filter. This parameter can be a value of `TIM_LL_EC_IC_FILTER`. This feature can be modified afterwards using unitary function `LL_TIM_IC_SetFilter()`.

65.1.4

LL_TIM_ENCODER_InitTypeDef

LL_TIM_ENCODER_InitTypeDef is defined in the stm32f2xx_ll_tim.h

Data Fields

- *uint32_t EncoderMode*
- *uint32_t IC1Polarity*
- *uint32_t IC1ActiveInput*
- *uint32_t IC1Prescaler*
- *uint32_t IC1Filter*
- *uint32_t IC2Polarity*
- *uint32_t IC2ActiveInput*
- *uint32_t IC2Prescaler*
- *uint32_t IC2Filter*

Field Documentation

- *uint32_t LL_TIM_ENCODER_InitTypeDef::EncoderMode*
Specifies the encoder resolution (x2 or x4). This parameter can be a value of [TIM_LL_EC_ENCODERMODE](#). This feature can be modified afterwards using unitary function [LL_TIM_SetEncoderMode\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC1Polarity*
Specifies the active edge of TI1 input. This parameter can be a value of [TIM_LL_EC_IC_POLARITY](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetPolarity\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC1ActiveInput*
Specifies the TI1 input source. This parameter can be a value of [TIM_LL_EC_ACTIVEINPUT](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetActiveInput\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC1Prescaler*
Specifies the TI1 input prescaler value. This parameter can be a value of [TIM_LL_EC_ICPSC](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetPrescaler\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC1Filter*
Specifies the TI1 input filter. This parameter can be a value of [TIM_LL_EC_IC_FILTER](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetFilter\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC2Polarity*
Specifies the active edge of TI2 input. This parameter can be a value of [TIM_LL_EC_IC_POLARITY](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetPolarity\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC2ActiveInput*
Specifies the TI2 input source. This parameter can be a value of [TIM_LL_EC_ACTIVEINPUT](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetActiveInput\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC2Prescaler*
Specifies the TI2 input prescaler value. This parameter can be a value of [TIM_LL_EC_ICPSC](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetPrescaler\(\)](#).
- *uint32_t LL_TIM_ENCODER_InitTypeDef::IC2Filter*
Specifies the TI2 input filter. This parameter can be a value of [TIM_LL_EC_IC_FILTER](#). This feature can be modified afterwards using unitary function [LL_TIM_IC_SetFilter\(\)](#).

65.1.5

LL_TIM_HALLSENSOR_InitTypeDef

LL_TIM_HALLSENSOR_InitTypeDef is defined in the stm32f2xx_ll_tim.h

Data Fields

- *uint32_t IC1Polarity*
- *uint32_t IC1Prescaler*
- *uint32_t IC1Filter*
- *uint32_t CommutationDelay*

Field Documentation

- **`uint32_t LL_TIM_HALLSENSOR_InitTypeDef::IC1Polarity`**
Specifies the active edge of TI1 input. This parameter can be a value of [TIM_LL_EC_IC_POLARITY](#). This feature can be modified afterwards using unitary function `LL_TIM_IC_SetPolarity()`.
- **`uint32_t LL_TIM_HALLSENSOR_InitTypeDef::IC1Prescaler`**
Specifies the TI1 input prescaler value. Prescaler must be set to get a maximum counter period longer than the time interval between 2 consecutive changes on the Hall inputs. This parameter can be a value of [TIM_LL_EC_ICPSC](#). This feature can be modified afterwards using unitary function `LL_TIM_IC_SetPrescaler()`.
- **`uint32_t LL_TIM_HALLSENSOR_InitTypeDef::IC1Filter`**
Specifies the TI1 input filter. This parameter can be a value of [TIM_LL_EC_IC_FILTER](#). This feature can be modified afterwards using unitary function `LL_TIM_IC_SetFilter()`.
- **`uint32_t LL_TIM_HALLSENSOR_InitTypeDef::CommutationDelay`**
Specifies the compare value to be loaded into the Capture Compare Register. A positive pulse (TRGO event) is generated with a programmable delay every time a change occurs on the Hall inputs. This parameter can be a number between `Min_Data = 0x0000` and `Max_Data = 0xFFFF`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetCompareCH2()`.

65.1.6

LL_TIM_BDTR_InitTypeDef

`LL_TIM_BDTR_InitTypeDef` is defined in the `stm32f2xx_ll_tim.h`

Data Fields

- **`uint32_t OSSRState`**
- **`uint32_t OSSISate`**
- **`uint32_t LockLevel`**
- **`uint8_t DeadTime`**
- **`uint16_t BreakState`**
- **`uint32_t BreakPolarity`**
- **`uint32_t AutomaticOutput`**

Field Documentation

- **`uint32_t LL_TIM_BDTR_InitTypeDef::OSSRState`**
Specifies the Off-State selection used in Run mode. This parameter can be a value of [TIM_LL_EC_OSSR](#). This feature can be modified afterwards using unitary function `LL_TIM_SetOffStates()`
Note:
 - This bit-field cannot be modified as long as LOCK level 2 has been programmed.
- **`uint32_t LL_TIM_BDTR_InitTypeDef::OSSISate`**
Specifies the Off-State used in Idle state. This parameter can be a value of [TIM_LL_EC_OSSI](#). This feature can be modified afterwards using unitary function `LL_TIM_SetOffStates()`
Note:
 - This bit-field cannot be modified as long as LOCK level 2 has been programmed.
- **`uint32_t LL_TIM_BDTR_InitTypeDef::LockLevel`**
Specifies the LOCK level parameters. This parameter can be a value of [TIM_LL_EC_LOCKLEVEL](#)
Note:
 - The LOCK bits can be written only once after the reset. Once the TIMx_BDTR register has been written, their content is frozen until the next reset.
- **`uint8_t LL_TIM_BDTR_InitTypeDef::DeadTime`**
Specifies the delay time between the switching-off and the switching-on of the outputs. This parameter can be a number between `Min_Data = 0x00` and `Max_Data = 0xFF`. This feature can be modified afterwards using unitary function `LL_TIM_OC_SetDeadTime()`
Note:
 - This bit-field can not be modified as long as LOCK level 1, 2 or 3 has been programmed.

- **`uint16_t LL_TIM_BDTR_InitTypeDef::BreakState`**
Specifies whether the TIM Break input is enabled or not. This parameter can be a value of [TIM_LL_EC_BREAK_ENABLE](#) This feature can be modified afterwards using unitary functions `LL_TIM_EnableBRK()` or `LL_TIM_DisableBRK()`
Note:
 - This bit-field can not be modified as long as LOCK level 1 has been programmed.
- **`uint32_t LL_TIM_BDTR_InitTypeDef::BreakPolarity`**
Specifies the TIM Break Input pin polarity. This parameter can be a value of [TIM_LL_EC_BREAK_POLARITY](#) This feature can be modified afterwards using unitary function `LL_TIM_ConfigBRK()`
Note:
 - This bit-field can not be modified as long as LOCK level 1 has been programmed.
- **`uint32_t LL_TIM_BDTR_InitTypeDef::AutomaticOutput`**
Specifies whether the TIM Automatic Output feature is enabled or not. This parameter can be a value of [TIM_LL_EC_AUTOMATICOUTPUT_ENABLE](#) This feature can be modified afterwards using unitary functions `LL_TIM_EnableAutomaticOutput()` or `LL_TIM_DisableAutomaticOutput()`
Note:
 - This bit-field can not be modified as long as LOCK level 1 has been programmed.

65.2 TIM Firmware driver API description

The following section lists the various functions of the TIM library.

65.2.1 Detailed description of functions

`LL_TIM_EnableCounter`

Function name

`__STATIC_INLINE void LL_TIM_EnableCounter (TIM_TypeDef * TIMx)`

Function description

Enable timer counter.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 CEN `LL_TIM_EnableCounter`

`LL_TIM_DisableCounter`

Function name

`__STATIC_INLINE void LL_TIM_DisableCounter (TIM_TypeDef * TIMx)`

Function description

Disable timer counter.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 CEN LL_TIM_DisableCounter
LL_TIM_IsEnabledCounter

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledCounter (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the timer counter is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 CEN LL_TIM_IsEnabledCounter
LL_TIM_EnableUpdateEvent

Function name

```
__STATIC_INLINE void LL_TIM_EnableUpdateEvent (TIM_TypeDef * TIMx)
```

Function description

Enable update event generation.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 UDIS LL_TIM_EnableUpdateEvent
LL_TIM_DisableUpdateEvent

Function name

```
__STATIC_INLINE void LL_TIM_DisableUpdateEvent (TIM_TypeDef * TIMx)
```

Function description

Disable update event generation.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 UDIS LL_TIM_DisableUpdateEvent
LL_TIM_IsEnabledUpdateEvent

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledUpdateEvent (TIM_TypeDef * TIMx)
```

Function description

Indicates whether update event generation is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **Inverted:** state of bit (0 or 1).

Reference Manual to LL API cross reference:

- CR1 UDIS LL_TIM_IsEnabledUpdateEvent

LL_TIM_SetUpdateSource

Function name

__STATIC_INLINE void LL_TIM_SetUpdateSource (TIM_TypeDef * TIMx, uint32_t UpdateSource)

Function description

Set update event source.

Parameters

- **TIMx:** Timer instance
- **UpdateSource:** This parameter can be one of the following values:
 - LL_TIM_UPDATESOURCE_REGULAR
 - LL_TIM_UPDATESOURCE_COUNTER

Return values

- **None:**

Notes

- Update event source set to LL_TIM_UPDATESOURCE_REGULAR: any of the following events generate an update interrupt or DMA request if enabled: Counter overflow/underflowSetting the UG bitUpdate generation through the slave mode controller
- Update event source set to LL_TIM_UPDATESOURCE_COUNTER: only counter overflow/underflow generates an update interrupt or DMA request if enabled.

Reference Manual to LL API cross reference:

- CR1 URS LL_TIM_SetUpdateSource

LL_TIM_GetUpdateSource

Function name

__STATIC_INLINE uint32_t LL_TIM_GetUpdateSource (TIM_TypeDef * TIMx)

Function description

Get actual event update source.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_UPDATESOURCE_REGULAR
 - LL_TIM_UPDATESOURCE_COUNTER

Reference Manual to LL API cross reference:

- CR1 URS LL_TIM_GetUpdateSource

```
LL_TIM_SetOnePulseMode
```

Function name

```
__STATIC_INLINE void LL_TIM_SetOnePulseMode (TIM_TypeDef * TIMx, uint32_t OnePulseMode)
```

Function description

Set one pulse mode (one shot v.s.

Parameters

- **TIMx:** Timer instance
- **OnePulseMode:** This parameter can be one of the following values:
 - LL_TIM_ONEPULSEMODE_SINGLE
 - LL_TIM_ONEPULSEMODE_REPETITIVE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OPM LL_TIM_SetOnePulseMode

```
LL_TIM_GetOnePulseMode
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_GetOnePulseMode (TIM_TypeDef * TIMx)
```

Function description

Get actual one pulse mode.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_ONEPULSEMODE_SINGLE
 - LL_TIM_ONEPULSEMODE_REPETITIVE

Reference Manual to LL API cross reference:

- CR1 OPM LL_TIM_GetOnePulseMode

```
LL_TIM_SetCounterMode
```

Function name

```
__STATIC_INLINE void LL_TIM_SetCounterMode (TIM_TypeDef * TIMx, uint32_t CounterMode)
```

Function description

Set the timer counter counting mode.

Parameters

- **TIMx:** Timer instance
- **CounterMode:** This parameter can be one of the following values:
 - LL_TIM_COUNTERMODE_UP
 - LL_TIM_COUNTERMODE_DOWN
 - LL_TIM_COUNTERMODE_CENTER_UP
 - LL_TIM_COUNTERMODE_CENTER_DOWN
 - LL_TIM_COUNTERMODE_CENTER_UP_DOWN

Return values

- **None:**

Notes

- Macro `IS_TIM_COUNTER_MODE_SELECT_INSTANCE(TIMx)` can be used to check whether or not the counter mode selection feature is supported by a timer instance.
- Switching from Center Aligned counter mode to Edge counter mode (or reverse) requires a timer reset to avoid unexpected direction due to DIR bit readonly in center aligned mode.

Reference Manual to LL API cross reference:

- CR1 DIR LL_TIM_SetCounterMode
- CR1 CMS LL_TIM_SetCounterMode

LL_TIM_GetCounterMode

Function name

__STATIC_INLINE uint32_t LL_TIM_GetCounterMode (TIM_TypeDef * TIMx)

Function description

Get actual counter mode.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_COUNTERMODE_UP
 - LL_TIM_COUNTERMODE_DOWN
 - LL_TIM_COUNTERMODE_CENTER_UP
 - LL_TIM_COUNTERMODE_CENTER_DOWN
 - LL_TIM_COUNTERMODE_CENTER_UP_DOWN

Notes

- Macro `IS_TIM_COUNTER_MODE_SELECT_INSTANCE(TIMx)` can be used to check whether or not the counter mode selection feature is supported by a timer instance.

Reference Manual to LL API cross reference:

- CR1 DIR LL_TIM_GetCounterMode
- CR1 CMS LL_TIM_GetCounterMode

LL_TIM_EnableARRPreload

Function name

__STATIC_INLINE void LL_TIM_EnableARRPreload (TIM_TypeDef * TIMx)

Function description

Enable auto-reload (ARR) preload.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 ARPE LL_TIM_EnableARRPreload

```
LL_TIM_DisableARRPreload
```

Function name

```
__STATIC_INLINE void LL_TIM_DisableARRPreload (TIM_TypeDef * TIMx)
```

Function description

Disable auto-reload (ARR) preload.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 ARPE LL_TIM_DisableARRPreload

```
LL_TIM_IsEnabledARRPreload
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledARRPreload (TIM_TypeDef * TIMx)
```

Function description

Indicates whether auto-reload (ARR) preload is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 ARPE LL_TIM_IsEnabledARRPreload

```
LL_TIM_SetClockDivision
```

Function name

```
__STATIC_INLINE void LL_TIM_SetClockDivision (TIM_TypeDef * TIMx, uint32_t ClockDivision)
```

Function description

Set the division ratio between the timer clock and the sampling clock used by the dead-time generators (when supported) and the digital filters.

Parameters

- **TIMx:** Timer instance
- **ClockDivision:** This parameter can be one of the following values:
 - LL_TIM_CLOCKDIVISION_DIV1
 - LL_TIM_CLOCKDIVISION_DIV2
 - LL_TIM_CLOCKDIVISION_DIV4

Return values

- **None:**

Notes

- Macro IS_TIM_CLOCK_DIVISION_INSTANCE(TIMx) can be used to check whether or not the clock division feature is supported by the timer instance.

Reference Manual to LL API cross reference:

- CR1 CKD LL_TIM_SetClockDivision

LL_TIM_GetClockDivision

Function name

__STATIC_INLINE uint32_t LL_TIM_GetClockDivision (TIM_TypeDef * TIMx)

Function description

Get the actual division ratio between the timer clock and the sampling clock used by the dead-time generators (when supported) and the digital filters.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_CLOCKDIVISION_DIV1
 - LL_TIM_CLOCKDIVISION_DIV2
 - LL_TIM_CLOCKDIVISION_DIV4

Notes

- Macro IS_TIM_CLOCK_DIVISION_INSTANCE(TIMx) can be used to check whether or not the clock division feature is supported by the timer instance.

Reference Manual to LL API cross reference:

- CR1 CKD LL_TIM_GetClockDivision

LL_TIM_SetCounter

Function name

__STATIC_INLINE void LL_TIM_SetCounter (TIM_TypeDef * TIMx, uint32_t Counter)

Function description

Set the counter value.

Parameters

- **TIMx:** Timer instance
- **Counter:** Counter value (between Min_Data=0 and Max_Data=0xFFFF or 0xFFFFFFFF)

Return values

- **None:**

Notes

- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.

Reference Manual to LL API cross reference:

- CNT CNT LL_TIM_SetCounter

LL_TIM_GetCounter

Function name

__STATIC_INLINE uint32_t LL_TIM_GetCounter (TIM_TypeDef * TIMx)

Function description

Get the counter value.

Parameters

- **TIMx:** Timer instance

Return values

- **Counter:** value (between Min_Data=0 and Max_Data=0xFFFF or 0xFFFFFFFF)

Notes

- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.

Reference Manual to LL API cross reference:

- CNT CNT LL_TIM_GetCounter

LL_TIM_GetDirection

Function name

__STATIC_INLINE uint32_t LL_TIM_GetDirection (TIM_TypeDef * TIMx)

Function description

Get the current direction of the counter.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_COUNTERDIRECTION_UP
 - LL_TIM_COUNTERDIRECTION_DOWN

Reference Manual to LL API cross reference:

- CR1 DIR LL_TIM_GetDirection

LL_TIM_SetPrescaler

Function name

__STATIC_INLINE void LL_TIM_SetPrescaler (TIM_TypeDef * TIMx, uint32_t Prescaler)

Function description

Set the prescaler value.

Parameters

- **TIMx:** Timer instance
- **Prescaler:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- The counter clock frequency CK_CNT is equal to fCK_PSC / (PSC[15:0] + 1).
- The prescaler can be changed on the fly as this control register is buffered. The new prescaler ratio is taken into account at the next update event.
- Helper macro __LL_TIM_CALC_PSC can be used to calculate the Prescaler parameter

Reference Manual to LL API cross reference:

- PSC PSC LL_TIM_SetPrescaler

LL_TIM_GetPrescaler

Function name

```
__STATIC_INLINE uint32_t LL_TIM_GetPrescaler (TIM_TypeDef * TIMx)
```

Function description

Get the prescaler value.

Parameters

- **TIMx:** Timer instance

Return values

- **Prescaler:** value between Min_Data=0 and Max_Data=65535

Reference Manual to LL API cross reference:

- PSC PSC LL_TIM_GetPrescaler

```
LL_TIM_SetAutoReload
```

Function name

```
__STATIC_INLINE void LL_TIM_SetAutoReload (TIM_TypeDef * TIMx, uint32_t AutoReload)
```

Function description

Set the auto-reload value.

Parameters

- **TIMx:** Timer instance
- **AutoReload:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- The counter is blocked while the auto-reload value is null.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Helper macro __LL_TIM_CALC_ARR can be used to calculate the AutoReload parameter

Reference Manual to LL API cross reference:

- ARR ARR LL_TIM_SetAutoReload

```
LL_TIM_GetAutoReload
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_GetAutoReload (TIM_TypeDef * TIMx)
```

Function description

Get the auto-reload value.

Parameters

- **TIMx:** Timer instance

Return values

- **Auto-reload:** value

Notes

- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.

Reference Manual to LL API cross reference:

- ARR ARR LL_TIM_GetAutoReload
- LL_TIM_SetRepetitionCounter

Function name

__STATIC_INLINE void LL_TIM_SetRepetitionCounter (TIM_TypeDef * TIMx, uint32_t RepetitionCounter)

Function description

Set the repetition counter value.

Parameters

- **TIMx:** Timer instance
- **RepetitionCounter:** between Min_Data=0 and Max_Data=255

Return values

- **None:**

Notes

- Macro IS_TIM_REPETITION_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a repetition counter.

Reference Manual to LL API cross reference:

- RCR REP LL_TIM_SetRepetitionCounter
- LL_TIM_GetRepetitionCounter

Function name

__STATIC_INLINE uint32_t LL_TIM_GetRepetitionCounter (TIM_TypeDef * TIMx)

Function description

Get the repetition counter value.

Parameters

- **TIMx:** Timer instance

Return values

- **Repetition:** counter value

Notes

- Macro IS_TIM_REPETITION_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a repetition counter.

Reference Manual to LL API cross reference:

- RCR REP LL_TIM_GetRepetitionCounter
- LL_TIM_CC_EnablePreload

Function name

__STATIC_INLINE void LL_TIM_CC_EnablePreload (TIM_TypeDef * TIMx)

Function description

Enable the capture/compare control bits (CCxE, CCxNE and OCxM) preload.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- CCxE, CCxNE and OCxM bits are preloaded, after having been written, they are updated only when a commutation event (COM) occurs.
- Only on channels that have a complementary output.
- Macro IS_TIM_COMMUTATION_EVENT_INSTANCE(TIMx) can be used to check whether or not a timer instance is able to generate a commutation event.

Reference Manual to LL API cross reference:

- CR2 CCPC LL_TIM_CC_EnablePreload
LL_TIM_CC_DisablePreload

Function name

__STATIC_INLINE void LL_TIM_CC_DisablePreload (TIM_TypeDef * TIMx)

Function description

Disable the capture/compare control bits (CCxE, CCxNE and OCxM) preload.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro IS_TIM_COMMUTATION_EVENT_INSTANCE(TIMx) can be used to check whether or not a timer instance is able to generate a commutation event.

Reference Manual to LL API cross reference:

- CR2 CCPC LL_TIM_CC_DisablePreload
LL_TIM_CC_SetUpdate

Function name

__STATIC_INLINE void LL_TIM_CC_SetUpdate (TIM_TypeDef * TIMx, uint32_t CCUpdateSource)

Function description

Set the updated source of the capture/compare control bits (CCxE, CCxNE and OCxM).

Parameters

- **TIMx:** Timer instance
- **CCUpdateSource:** This parameter can be one of the following values:
 - LL_TIM_CCUPDATESOURCE_COMG_ONLY
 - LL_TIM_CCUPDATESOURCE_COMG_AND_TRGI

Return values

- **None:**

Notes

- Macro IS_TIM_COMMUTATION_EVENT_INSTANCE(TIMx) can be used to check whether or not a timer instance is able to generate a commutation event.

Reference Manual to LL API cross reference:

- CR2 CCUS LL_TIM_CC_SetUpdate

```
LL_TIM_CC_SetDMAReqTrigger
```

Function name

```
__STATIC_INLINE void LL_TIM_CC_SetDMAReqTrigger (TIM_TypeDef * TIMx, uint32_t DMAReqTrigger)
```

Function description

Set the trigger of the capture/compare DMA request.

Parameters

- **TIMx:** Timer instance
- **DMAReqTrigger:** This parameter can be one of the following values:
 - LL_TIM_CCDMAREQUEST_CC
 - LL_TIM_CCDMAREQUEST_UPDATE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 CCDS LL_TIM_CC_SetDMAReqTrigger

```
LL_TIM_CC_GetDMAReqTrigger
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_CC_GetDMAReqTrigger (TIM_TypeDef * TIMx)
```

Function description

Get actual trigger of the capture/compare DMA request.

Parameters

- **TIMx:** Timer instance

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_CCDMAREQUEST_CC
 - LL_TIM_CCDMAREQUEST_UPDATE

Reference Manual to LL API cross reference:

- CR2 CCDS LL_TIM_CC_GetDMAReqTrigger

```
LL_TIM_CC_SetLockLevel
```

Function name

```
__STATIC_INLINE void LL_TIM_CC_SetLockLevel (TIM_TypeDef * TIMx, uint32_t LockLevel)
```

Function description

Set the lock level to freeze the configuration of several capture/compare parameters.

Parameters

- **TIMx:** Timer instance
- **LockLevel:** This parameter can be one of the following values:
 - LL_TIM_LOCKLEVEL_OFF
 - LL_TIM_LOCKLEVEL_1
 - LL_TIM_LOCKLEVEL_2
 - LL_TIM_LOCKLEVEL_3

Return values

- **None:**

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not the lock mechanism is supported by a timer instance.

Reference Manual to LL API cross reference:

- BDTR LOCK LL_TIM_CC_SetLockLevel
- LL_TIM_CC_EnableChannel

Function name

__STATIC_INLINE void LL_TIM_CC_EnableChannel (TIM_TypeDef * TIMx, uint32_t Channels)

Function description

Enable capture/compare channels.

Parameters

- TIMx:** Timer instance
- Channels:** This parameter can be a combination of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4

Return values

- None:**

Reference Manual to LL API cross reference:

- CCER CC1E LL_TIM_CC_EnableChannel
- CCER CC1NE LL_TIM_CC_EnableChannel
- CCER CC2E LL_TIM_CC_EnableChannel
- CCER CC2NE LL_TIM_CC_EnableChannel
- CCER CC3E LL_TIM_CC_EnableChannel
- CCER CC3NE LL_TIM_CC_EnableChannel
- CCER CC4E LL_TIM_CC_EnableChannel
- LL_TIM_CC_DisableChannel

Function name

__STATIC_INLINE void LL_TIM_CC_DisableChannel (TIM_TypeDef * TIMx, uint32_t Channels)

Function description

Disable capture/compare channels.

Parameters

- **TIMx:** Timer instance
- **Channels:** This parameter can be a combination of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCER CC1E LL_TIM_CC_DisableChannel
- CCER CC1NE LL_TIM_CC_DisableChannel
- CCER CC2E LL_TIM_CC_DisableChannel
- CCER CC2NE LL_TIM_CC_DisableChannel
- CCER CC3E LL_TIM_CC_DisableChannel
- CCER CC3NE LL_TIM_CC_DisableChannel
- CCER CC4E LL_TIM_CC_DisableChannel

LL_TIM_CC_IsEnabledChannel

Function name

__STATIC_INLINE uint32_t LL_TIM_CC_IsEnabledChannel (TIM_TypeDef * TIMx, uint32_t Channels)

Function description

Indicate whether channel(s) is(are) enabled.

Parameters

- **TIMx:** Timer instance
- **Channels:** This parameter can be a combination of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CCER CC1E LL_TIM_CC_IsEnabledChannel
- CCER CC1NE LL_TIM_CC_IsEnabledChannel
- CCER CC2E LL_TIM_CC_IsEnabledChannel
- CCER CC2NE LL_TIM_CC_IsEnabledChannel
- CCER CC3E LL_TIM_CC_IsEnabledChannel
- CCER CC3NE LL_TIM_CC_IsEnabledChannel
- CCER CC4E LL_TIM_CC_IsEnabledChannel

LL_TIM_OC_ConfigOutput

Function name

```
__STATIC_INLINE void LL_TIM_OC_ConfigOutput (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t Configuration)
```

Function description

Configure an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **Configuration:** This parameter must be a combination of all the following values:
 - LL_TIM_OCPOLARITY_HIGH or LL_TIM_OCPOLARITY_LOW
 - LL_TIM_OCIDLESTATE_LOW or LL_TIM_OCIDLESTATE_HIGH

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 CC1S LL_TIM_OC_ConfigOutput
- CCMR1 CC2S LL_TIM_OC_ConfigOutput
- CCMR2 CC3S LL_TIM_OC_ConfigOutput
- CCMR2 CC4S LL_TIM_OC_ConfigOutput
- CCER CC1P LL_TIM_OC_ConfigOutput
- CCER CC2P LL_TIM_OC_ConfigOutput
- CCER CC3P LL_TIM_OC_ConfigOutput
- CCER CC4P LL_TIM_OC_ConfigOutput
- CR2 OIS1 LL_TIM_OC_ConfigOutput
- CR2 OIS2 LL_TIM_OC_ConfigOutput
- CR2 OIS3 LL_TIM_OC_ConfigOutput
- CR2 OIS4 LL_TIM_OC_ConfigOutput

LL_TIM_OC_SetMode

Function name

```
__STATIC_INLINE void LL_TIM_OC_SetMode (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t Mode)
```

Function description

Define the behavior of the output reference signal OCxREF from which OCx and OCxN (when relevant) are derived.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **Mode:** This parameter can be one of the following values:
 - LL_TIM_OC_MODE_FROZEN
 - LL_TIM_OC_MODE_ACTIVE
 - LL_TIM_OC_MODE_INACTIVE
 - LL_TIM_OC_MODE_TOGGLE
 - LL_TIM_OC_MODE_FORCED_INACTIVE
 - LL_TIM_OC_MODE_FORCED_ACTIVE
 - LL_TIM_OC_MODE_PWM1
 - LL_TIM_OC_MODE_PWM2

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1_OC1M LL_TIM_OC_SetMode
- CCMR1_OC2M LL_TIM_OC_SetMode
- CCMR2_OC3M LL_TIM_OC_SetMode
- CCMR2_OC4M LL_TIM_OC_SetMode

LL_TIM_OC_GetMode

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetMode (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the output compare mode of an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_OC_MODE_FROZEN
 - LL_TIM_OC_MODE_ACTIVE
 - LL_TIM_OC_MODE_INACTIVE
 - LL_TIM_OC_MODE_TOGGLE
 - LL_TIM_OC_MODE_FORCED_INACTIVE
 - LL_TIM_OC_MODE_FORCED_ACTIVE
 - LL_TIM_OC_MODE_PWM1
 - LL_TIM_OC_MODE_PWM2

Reference Manual to LL API cross reference:

- CCMR1 OC1M LL_TIM_OC_GetMode
- CCMR1 OC2M LL_TIM_OC_GetMode
- CCMR2 OC3M LL_TIM_OC_GetMode
- CCMR2 OC4M LL_TIM_OC_GetMode

LL_TIM_OC_SetPolarity

Function name

__STATIC_INLINE void LL_TIM_OC_SetPolarity (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t Polarity)

Function description

Set the polarity of an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4
- **Polarity:** This parameter can be one of the following values:
 - LL_TIM_OCPOLARITY_HIGH
 - LL_TIM_OCPOLARITY_LOW

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCER CC1P LL_TIM_OC_SetPolarity
- CCER CC1NP LL_TIM_OC_SetPolarity
- CCER CC2P LL_TIM_OC_SetPolarity
- CCER CC2NP LL_TIM_OC_SetPolarity
- CCER CC3P LL_TIM_OC_SetPolarity
- CCER CC3NP LL_TIM_OC_SetPolarity
- CCER CC4P LL_TIM_OC_SetPolarity

LL_TIM_OC_GetPolarity

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetPolarity (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the polarity of an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_OCPOLARITY_HIGH
 - LL_TIM_OCPOLARITY_LOW

Reference Manual to LL API cross reference:

- CCER CC1P LL_TIM_OC_GetPolarity
- CCER CC1NP LL_TIM_OC_GetPolarity
- CCER CC2P LL_TIM_OC_GetPolarity
- CCER CC2NP LL_TIM_OC_GetPolarity
- CCER CC3P LL_TIM_OC_GetPolarity
- CCER CC3NP LL_TIM_OC_GetPolarity
- CCER CC4P LL_TIM_OC_GetPolarity

LL_TIM_OC_SetIdleState

Function name

__STATIC_INLINE void LL_TIM_OC_SetIdleState (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t IdleState)

Function description

Set the IDLE state of an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH1N
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH2N
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH3N
 - LL_TIM_CHANNEL_CH4
- **IdleState:** This parameter can be one of the following values:
 - LL_TIM_OCIDLESTATE_LOW
 - LL_TIM_OCIDLESTATE_HIGH

Return values

- **None:**

Notes

- This function is significant only for the timer instances supporting the break feature. Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- CR2_OIS1_LL_TIM_OC_SetIdleState
- CR2_OIS1N_LL_TIM_OC_SetIdleState
- CR2_OIS2_LL_TIM_OC_SetIdleState
- CR2_OIS2N_LL_TIM_OC_SetIdleState
- CR2_OIS3_LL_TIM_OC_SetIdleState
- CR2_OIS3N_LL_TIM_OC_SetIdleState
- CR2_OIS4_LL_TIM_OC_SetIdleState

`LL_TIM_OC_GetIdleState`

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetIdleState (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the IDLE state of an output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - `LL_TIM_CHANNEL_CH1`
 - `LL_TIM_CHANNEL_CH1N`
 - `LL_TIM_CHANNEL_CH2`
 - `LL_TIM_CHANNEL_CH2N`
 - `LL_TIM_CHANNEL_CH3`
 - `LL_TIM_CHANNEL_CH3N`
 - `LL_TIM_CHANNEL_CH4`

Return values

- **Returned:** value can be one of the following values:
 - `LL_TIM_OCIDLESTATE_LOW`
 - `LL_TIM_OCIDLESTATE_HIGH`

Reference Manual to LL API cross reference:

- CR2_OIS1_LL_TIM_OC_GetIdleState
- CR2_OIS1N_LL_TIM_OC_GetIdleState
- CR2_OIS2_LL_TIM_OC_GetIdleState
- CR2_OIS2N_LL_TIM_OC_GetIdleState
- CR2_OIS3_LL_TIM_OC_GetIdleState
- CR2_OIS3N_LL_TIM_OC_GetIdleState
- CR2_OIS4_LL_TIM_OC_GetIdleState

`LL_TIM_OC_EnableFast`

Function name

__STATIC_INLINE void LL_TIM_OC_EnableFast (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Enable fast mode for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Notes

- Acts only if the channel is configured in PWM1 or PWM2 mode.

Reference Manual to LL API cross reference:

- CCMR1 OC1FE LL_TIM_OC_EnableFast
- CCMR1 OC2FE LL_TIM_OC_EnableFast
- CCMR2 OC3FE LL_TIM_OC_EnableFast
- CCMR2 OC4FE LL_TIM_OC_EnableFast

LL_TIM_OC_DisableFast

Function name

__STATIC_INLINE void LL_TIM_OC_DisableFast (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Disable fast mode for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 OC1FE LL_TIM_OC_DisableFast
- CCMR1 OC2FE LL_TIM_OC_DisableFast
- CCMR2 OC3FE LL_TIM_OC_DisableFast
- CCMR2 OC4FE LL_TIM_OC_DisableFast

LL_TIM_OC_IsEnabledFast

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_IsEnabledFast (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Indicates whether fast mode is enabled for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CCMR1 OC1FE LL_TIM_OC_IsEnabledFast
- CCMR1 OC2FE LL_TIM_OC_IsEnabledFast
- CCMR2 OC3FE LL_TIM_OC_IsEnabledFast
- CCMR2 OC4FE LL_TIM_OC_IsEnabledFast
-

LL_TIM_OC_EnablePreload

Function name

__STATIC_INLINE void LL_TIM_OC_EnablePreload (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Enable compare register (TIMx_CCRx) preload for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 OC1PE LL_TIM_OC_EnablePreload
- CCMR1 OC2PE LL_TIM_OC_EnablePreload
- CCMR2 OC3PE LL_TIM_OC_EnablePreload
- CCMR2 OC4PE LL_TIM_OC_EnablePreload

LL_TIM_OC_DisablePreload

Function name

__STATIC_INLINE void LL_TIM_OC_DisablePreload (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Disable compare register (TIMx_CCRx) preload for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 OC1PE LL_TIM_OC_DisablePreload
- CCMR1 OC2PE LL_TIM_OC_DisablePreload
- CCMR2 OC3PE LL_TIM_OC_DisablePreload
- CCMR2 OC4PE LL_TIM_OC_DisablePreload

LL_TIM_OC_IsEnabledPreload

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_IsEnabledPreload (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Indicates whether compare register (TIMx_CCRx) preload is enabled for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CCMR1 OC1PE LL_TIM_OC_IsEnabledPreload
- CCMR1 OC2PE LL_TIM_OC_IsEnabledPreload
- CCMR2 OC3PE LL_TIM_OC_IsEnabledPreload
- CCMR2 OC4PE LL_TIM_OC_IsEnabledPreload
-

LL_TIM_OC_EnableClear

Function name

__STATIC_INLINE void LL_TIM_OC_EnableClear (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Enable clearing the output channel on an external event.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Notes

- This function can only be used in Output compare and PWM modes. It does not work in Forced mode.
- Macro IS_TIM_OCXREF_CLEAR_INSTANCE(TIMx) can be used to check whether or not a timer instance can clear the OCxREF signal on an external event.

Reference Manual to LL API cross reference:

- CCMR1 OC1CE LL_TIM_OC_EnableClear
- CCMR1 OC2CE LL_TIM_OC_EnableClear
- CCMR2 OC3CE LL_TIM_OC_EnableClear
- CCMR2 OC4CE LL_TIM_OC_EnableClear

LL_TIM_OC_DisableClear

Function name

__STATIC_INLINE void LL_TIM_OC_DisableClear (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Disable clearing the output channel on an external event.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **None:**

Notes

- Macro IS_TIM_OCXREF_CLEAR_INSTANCE(TIMx) can be used to check whether or not a timer instance can clear the OCxREF signal on an external event.

Reference Manual to LL API cross reference:

- CCMR1 OC1CE LL_TIM_OC_DisableClear
- CCMR1 OC2CE LL_TIM_OC_DisableClear
- CCMR2 OC3CE LL_TIM_OC_DisableClear
- CCMR2 OC4CE LL_TIM_OC_DisableClear

LL_TIM_OC_IsEnabledClear

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_IsEnabledClear (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Indicates clearing the output channel on an external event is enabled for the output channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **State:** of bit (1 or 0).

Notes

- This function enables clearing the output channel on an external event.
- This function can only be used in Output compare and PWM modes. It does not work in Forced mode.
- Macro IS_TIM_OCXREF_CLEAR_INSTANCE(TIMx) can be used to check whether or not a timer instance can clear the OCxREF signal on an external event.

Reference Manual to LL API cross reference:

- CCMR1 OC1CE LL_TIM_OC_IsEnabledClear
- CCMR1 OC2CE LL_TIM_OC_IsEnabledClear
- CCMR2 OC3CE LL_TIM_OC_IsEnabledClear
- CCMR2 OC4CE LL_TIM_OC_IsEnabledClear
-

LL_TIM_OC_SetDeadTime

Function name

__STATIC_INLINE void LL_TIM_OC_SetDeadTime (TIM_TypeDef * TIMx, uint32_t DeadTime)

Function description

Set the dead-time delay (delay inserted between the rising edge of the OCxREF signal and the rising edge of the Ocx and OCxN signals).

Parameters

- **TIMx:** Timer instance
- **DeadTime:** between Min_Data=0 and Max_Data=255

Return values

- **None:**

Notes

- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not dead-time insertion feature is supported by a timer instance.
- Helper macro __LL_TIM_CALC_DEADTIME can be used to calculate the DeadTime parameter

Reference Manual to LL API cross reference:

- BDTR DTG LL_TIM_OC_SetDeadTime

LL_TIM_OC_SetCompareCH1

Function name

__STATIC_INLINE void LL_TIM_OC_SetCompareCH1 (TIM_TypeDef * TIMx, uint32_t CompareValue)

Function description

Set compare value for output channel 1 (TIMx_CCR1).

Parameters

- **TIMx:** Timer instance
- **CompareValue:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- In 32-bit timer implementations compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC1_INSTANCE(TIMx) can be used to check whether or not output channel 1 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR1 CCR1 LL_TIM_OC_SetCompareCH1
LL_TIM_OC_SetCompareCH2

Function name

__STATIC_INLINE void LL_TIM_OC_SetCompareCH2 (TIM_TypeDef * TIMx, uint32_t CompareValue)

Function description

Set compare value for output channel 2 (TIMx_CCR2).

Parameters

- **TIMx:** Timer instance
- **CompareValue:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- In 32-bit timer implementations compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC2_INSTANCE(TIMx) can be used to check whether or not output channel 2 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR2 CCR2 LL_TIM_OC_SetCompareCH2
LL_TIM_OC_SetCompareCH3

Function name

__STATIC_INLINE void LL_TIM_OC_SetCompareCH3 (TIM_TypeDef * TIMx, uint32_t CompareValue)

Function description

Set compare value for output channel 3 (TIMx_CCR3).

Parameters

- **TIMx:** Timer instance
- **CompareValue:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- In 32-bit timer implementations compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro `IS_TIM_32B_COUNTER_INSTANCE(TIMx)` can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro `IS_TIM_CC3_INSTANCE(TIMx)` can be used to check whether or not output channel is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR3 CCR3 LL_TIM_OC_SetCompareCH3
LL_TIM_OC_SetCompareCH4

Function name

__STATIC_INLINE void LL_TIM_OC_SetCompareCH4 (TIM_TypeDef * TIMx, uint32_t CompareValue)

Function description

Set compare value for output channel 4 (TIMx_CCR4).

Parameters

- **TIMx:** Timer instance
- **CompareValue:** between Min_Data=0 and Max_Data=65535

Return values

- **None:**

Notes

- In 32-bit timer implementations compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro `IS_TIM_32B_COUNTER_INSTANCE(TIMx)` can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro `IS_TIM_CC4_INSTANCE(TIMx)` can be used to check whether or not output channel 4 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR4 CCR4 LL_TIM_OC_SetCompareCH4
LL_TIM_OC_GetCompareCH1

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetCompareCH1 (TIM_TypeDef * TIMx)

Function description

Get compare value (TIMx_CCR1) set for output channel 1.

Parameters

- **TIMx:** Timer instance

Return values

- **CompareValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC1_INSTANCE(TIMx) can be used to check whether or not output channel 1 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR1 CCR1 LL_TIM_OC_GetCompareCH1
LL_TIM_OC_GetCompareCH2

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetCompareCH2 (TIM_TypeDef * TIMx)

Function description

Get compare value (TIMx_CCR2) set for output channel 2.

Parameters

- **TIMx:** Timer instance

Return values

- **CompareValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC2_INSTANCE(TIMx) can be used to check whether or not output channel 2 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR2 CCR2 LL_TIM_OC_GetCompareCH2
LL_TIM_OC_GetCompareCH3

Function name

__STATIC_INLINE uint32_t LL_TIM_OC_GetCompareCH3 (TIM_TypeDef * TIMx)

Function description

Get compare value (TIMx_CCR3) set for output channel 3.

Parameters

- **TIMx:** Timer instance

Return values

- **CompareValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC3_INSTANCE(TIMx) can be used to check whether or not output channel 3 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR3 CCR3 LL_TIM_OC_GetCompareCH3

```
LL_TIM_OC_GetCompareCH4
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_OC_GetCompareCH4 (TIM_TypeDef * TIMx)
```

Function description

Get compare value (TIMx_CCR4) set for output channel 4.

Parameters

- **TIMx:** Timer instance

Return values

- **CompareValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned compare value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC4_INSTANCE(TIMx) can be used to check whether or not output channel 4 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR4 CCR4 LL_TIM_OC_GetCompareCH4

```
LL_TIM_IC_Config
```

Function name

```
__STATIC_INLINE void LL_TIM_IC_Config (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t Configuration)
```

Function description

Configure input channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **Configuration:** This parameter must be a combination of all the following values:
 - LL_TIM_ACTIVEINPUT_DIRECTTI or LL_TIM_ACTIVEINPUT_INDIRECTTI or LL_TIM_ACTIVEINPUT_TRC
 - LL_TIM_ICPSC_DIV1 or ... or LL_TIM_ICPSC_DIV8
 - LL_TIM_IC_FILTER_FDIV1 or ... or LL_TIM_IC_FILTER_FDIV32_N8
 - LL_TIM_IC_POLARITY_RISING or LL_TIM_IC_POLARITY_FALLING or LL_TIM_IC_POLARITY_BOTHEDGE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 CC1S LL_TIM_IC_Config
- CCMR1 IC1PSC LL_TIM_IC_Config
- CCMR1 IC1F LL_TIM_IC_Config
- CCMR1 CC2S LL_TIM_IC_Config
- CCMR1 IC2PSC LL_TIM_IC_Config
- CCMR1 IC2F LL_TIM_IC_Config
- CCMR2 CC3S LL_TIM_IC_Config
- CCMR2 IC3PSC LL_TIM_IC_Config
- CCMR2 IC3F LL_TIM_IC_Config
- CCMR2 CC4S LL_TIM_IC_Config
- CCMR2 IC4PSC LL_TIM_IC_Config
- CCMR2 IC4F LL_TIM_IC_Config
- CCER CC1P LL_TIM_IC_Config
- CCER CC1NP LL_TIM_IC_Config
- CCER CC2P LL_TIM_IC_Config
- CCER CC2NP LL_TIM_IC_Config
- CCER CC3P LL_TIM_IC_Config
- CCER CC3NP LL_TIM_IC_Config
- CCER CC4P LL_TIM_IC_Config
- CCER CC4NP LL_TIM_IC_Config

LL_TIM_IC_SetActiveInput

Function name

__STATIC_INLINE void LL_TIM_IC_SetActiveInput (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ICActiveInput)

Function description

Set the active input.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **ICActiveInput:** This parameter can be one of the following values:
 - LL_TIM_ACTIVEINPUT_DIRECTTI
 - LL_TIM_ACTIVEINPUT_INDIRECTTI
 - LL_TIM_ACTIVEINPUT_TRC

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 CC1S LL_TIM_IC_SetActiveInput
- CCMR1 CC2S LL_TIM_IC_SetActiveInput
- CCMR2 CC3S LL_TIM_IC_SetActiveInput
- CCMR2 CC4S LL_TIM_IC_SetActiveInput

LL_TIM_IC_GetActiveInput

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IC_GetActiveInput (TIM_TypeDef * TIMx, uint32_t Channel)
```

Function description

Get the current active input.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_ACTIVEINPUT_DIRECTTI
 - LL_TIM_ACTIVEINPUT_INDIRECTTI
 - LL_TIM_ACTIVEINPUT_TRC

Reference Manual to LL API cross reference:

- CCMR1 CC1S LL_TIM_IC_GetActiveInput
- CCMR1 CC2S LL_TIM_IC_GetActiveInput
- CCMR2 CC3S LL_TIM_IC_GetActiveInput
- CCMR2 CC4S LL_TIM_IC_GetActiveInput

LL_TIM_IC_SetPrescaler

Function name

```
__STATIC_INLINE void LL_TIM_IC_SetPrescaler (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ICPrescaler)
```

Function description

Set the prescaler of input channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **ICPrescaler:** This parameter can be one of the following values:
 - LL_TIM_ICPSC_DIV1
 - LL_TIM_ICPSC_DIV2
 - LL_TIM_ICPSC_DIV4
 - LL_TIM_ICPSC_DIV8

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 IC1PSC LL_TIM_IC_SetPrescaler
- CCMR1 IC2PSC LL_TIM_IC_SetPrescaler
- CCMR2 IC3PSC LL_TIM_IC_SetPrescaler
- CCMR2 IC4PSC LL_TIM_IC_SetPrescaler

LL_TIM_IC_GetPrescaler

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetPrescaler (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the current prescaler value acting on an input channel.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_ICPSC_DIV1
 - LL_TIM_ICPSC_DIV2
 - LL_TIM_ICPSC_DIV4
 - LL_TIM_ICPSC_DIV8

Reference Manual to LL API cross reference:

- CCMR1 IC1PSC LL_TIM_IC_GetPrescaler
- CCMR1 IC2PSC LL_TIM_IC_GetPrescaler
- CCMR2 IC3PSC LL_TIM_IC_GetPrescaler
- CCMR2 IC4PSC LL_TIM_IC_GetPrescaler

LL_TIM_IC_SetFilter

Function name

__STATIC_INLINE void LL_TIM_IC_SetFilter (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ICFILTER)

Function description

Set the input filter duration.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **ICFilter:** This parameter can be one of the following values:
 - LL_TIM_IC_FILTER_FDIV1
 - LL_TIM_IC_FILTER_FDIV1_N2
 - LL_TIM_IC_FILTER_FDIV1_N4
 - LL_TIM_IC_FILTER_FDIV1_N8
 - LL_TIM_IC_FILTER_FDIV2_N6
 - LL_TIM_IC_FILTER_FDIV2_N8
 - LL_TIM_IC_FILTER_FDIV4_N6
 - LL_TIM_IC_FILTER_FDIV4_N8
 - LL_TIM_IC_FILTER_FDIV8_N6
 - LL_TIM_IC_FILTER_FDIV8_N8
 - LL_TIM_IC_FILTER_FDIV16_N5
 - LL_TIM_IC_FILTER_FDIV16_N6
 - LL_TIM_IC_FILTER_FDIV16_N8
 - LL_TIM_IC_FILTER_FDIV32_N5
 - LL_TIM_IC_FILTER_FDIV32_N6
 - LL_TIM_IC_FILTER_FDIV32_N8

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCMR1 IC1F LL_TIM_IC_SetFilter
- CCMR1 IC2F LL_TIM_IC_SetFilter
- CCMR2 IC3F LL_TIM_IC_SetFilter
- CCMR2 IC4F LL_TIM_IC_SetFilter

LL_TIM_IC_GetFilter

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetFilter (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the input filter duration.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_IC_FILTER_FDIV1
 - LL_TIM_IC_FILTER_FDIV1_N2
 - LL_TIM_IC_FILTER_FDIV1_N4
 - LL_TIM_IC_FILTER_FDIV1_N8
 - LL_TIM_IC_FILTER_FDIV2_N6
 - LL_TIM_IC_FILTER_FDIV2_N8
 - LL_TIM_IC_FILTER_FDIV4_N6
 - LL_TIM_IC_FILTER_FDIV4_N8
 - LL_TIM_IC_FILTER_FDIV8_N6
 - LL_TIM_IC_FILTER_FDIV8_N8
 - LL_TIM_IC_FILTER_FDIV16_N5
 - LL_TIM_IC_FILTER_FDIV16_N6
 - LL_TIM_IC_FILTER_FDIV16_N8
 - LL_TIM_IC_FILTER_FDIV32_N5
 - LL_TIM_IC_FILTER_FDIV32_N6
 - LL_TIM_IC_FILTER_FDIV32_N8

Reference Manual to LL API cross reference:

- CCMR1 IC1F LL_TIM_IC_GetFilter
- CCMR1 IC2F LL_TIM_IC_GetFilter
- CCMR2 IC3F LL_TIM_IC_GetFilter
- CCMR2 IC4F LL_TIM_IC_GetFilter

LL_TIM_IC_SetPolarity

Function name

__STATIC_INLINE void LL_TIM_IC_SetPolarity (TIM_TypeDef * TIMx, uint32_t Channel, uint32_t ICPolarity)

Function description

Set the input channel polarity.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **ICPolarity:** This parameter can be one of the following values:
 - LL_TIM_IC_POLARITY_RISING
 - LL_TIM_IC_POLARITY_FALLING
 - LL_TIM_IC_POLARITY_BOTHEDGE

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCER CC1P LL_TIM_IC_SetPolarity
- CCER CC1NP LL_TIM_IC_SetPolarity
- CCER CC2P LL_TIM_IC_SetPolarity
- CCER CC2NP LL_TIM_IC_SetPolarity
- CCER CC3P LL_TIM_IC_SetPolarity
- CCER CC3NP LL_TIM_IC_SetPolarity
- CCER CC4P LL_TIM_IC_SetPolarity
- CCER CC4NP LL_TIM_IC_SetPolarity

LL_TIM_IC_GetPolarity

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetPolarity (TIM_TypeDef * TIMx, uint32_t Channel)

Function description

Get the current input channel polarity.

Parameters

- **TIMx:** Timer instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4

Return values

- **Returned:** value can be one of the following values:
 - LL_TIM_IC_POLARITY_RISING
 - LL_TIM_IC_POLARITY_FALLING
 - LL_TIM_IC_POLARITY_BOTHEDGE

Reference Manual to LL API cross reference:

- CCER CC1P LL_TIM_IC_GetPolarity
- CCER CC1NP LL_TIM_IC_GetPolarity
- CCER CC2P LL_TIM_IC_GetPolarity
- CCER CC2NP LL_TIM_IC_GetPolarity
- CCER CC3P LL_TIM_IC_GetPolarity
- CCER CC3NP LL_TIM_IC_GetPolarity
- CCER CC4P LL_TIM_IC_GetPolarity
- CCER CC4NP LL_TIM_IC_GetPolarity

LL_TIM_IC_EnableXORCombination

Function name

__STATIC_INLINE void LL_TIM_IC_EnableXORCombination (TIM_TypeDef * TIMx)

Function description

Connect the TIMx_CH1, CH2 and CH3 pins to the TI1 input (XOR combination).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro `IS_TIM_XOR_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides an XOR input.

Reference Manual to LL API cross reference:

- CR2 TI1S `LL_TIM_IC_EnableXORCombination`
`LL_TIM_IC_DisableXORCombination`

Function name

__STATIC_INLINE void LL_TIM_IC_DisableXORCombination (TIM_TypeDef * TIMx)

Function description

Disconnect the TIMx_CH1, CH2 and CH3 pins from the TI1 input.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro `IS_TIM_XOR_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides an XOR input.

Reference Manual to LL API cross reference:

- CR2 TI1S `LL_TIM_IC_DisableXORCombination`
`LL_TIM_IC_IsEnabledXORCombination`

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_IsEnabledXORCombination (TIM_TypeDef * TIMx)

Function description

Indicates whether the TIMx_CH1, CH2 and CH3 pins are connected to the TI1 input.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_TIM_XOR_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides an XOR input.

Reference Manual to LL API cross reference:

- CR2 TI1S `LL_TIM_IC_IsEnabledXORCombination`
`LL_TIM_IC_GetCaptureCH1`

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetCaptureCH1 (TIM_TypeDef * TIMx)

Function description

Get captured value for input channel 1.

Parameters

- **TIMx:** Timer instance

Return values

- **CapturedValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned captured value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC1_INSTANCE(TIMx) can be used to check whether or not input channel 1 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR1 CCR1 LL_TIM_IC_GetCaptureCH1
LL_TIM_IC_GetCaptureCH2

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetCaptureCH2 (TIM_TypeDef * TIMx)

Function description

Get captured value for input channel 2.

Parameters

- **TIMx:** Timer instance

Return values

- **CapturedValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned captured value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC2_INSTANCE(TIMx) can be used to check whether or not input channel 2 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR2 CCR2 LL_TIM_IC_GetCaptureCH2
LL_TIM_IC_GetCaptureCH3

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetCaptureCH3 (TIM_TypeDef * TIMx)

Function description

Get captured value for input channel 3.

Parameters

- **TIMx:** Timer instance

Return values

- **CapturedValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned captured value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC3_INSTANCE(TIMx) can be used to check whether or not input channel 3 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR3 CCR3 LL_TIM_IC_GetCaptureCH3

LL_TIM_IC_GetCaptureCH4

Function name

__STATIC_INLINE uint32_t LL_TIM_IC_GetCaptureCH4 (TIM_TypeDef * TIMx)

Function description

Get captured value for input channel 4.

Parameters

- **TIMx:** Timer instance

Return values

- **CapturedValue:** (between Min_Data=0 and Max_Data=65535)

Notes

- In 32-bit timer implementations returned captured value can be between 0x00000000 and 0xFFFFFFFF.
- Macro IS_TIM_32B_COUNTER_INSTANCE(TIMx) can be used to check whether or not a timer instance supports a 32 bits counter.
- Macro IS_TIM_CC4_INSTANCE(TIMx) can be used to check whether or not input channel 4 is supported by a timer instance.

Reference Manual to LL API cross reference:

- CCR4 CCR4 LL_TIM_IC_GetCaptureCH4

LL_TIM_EnableExternalClock

Function name

__STATIC_INLINE void LL_TIM_EnableExternalClock (TIM_TypeDef * TIMx)

Function description

Enable external clock mode 2.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- When external clock mode 2 is enabled the counter is clocked by any active edge on the ETRF signal.
- Macro IS_TIM_CLOCKSOURCE_ETRMODE2_INSTANCE(TIMx) can be used to check whether or not a timer instance supports external clock mode2.

Reference Manual to LL API cross reference:

- SMCR ECE LL_TIM_EnableExternalClock

LL_TIM_DisableExternalClock

Function name

```
__STATIC_INLINE void LL_TIM_DisableExternalClock (TIM_TypeDef * TIMx)
```

Function description

Disable external clock mode 2.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro IS_TIM_CLOCKSOURCE_ETRMODE2_INSTANCE(TIMx) can be used to check whether or not a timer instance supports external clock mode2.

Reference Manual to LL API cross reference:

- SMCR ECE LL_TIM_DisableExternalClock

LL_TIM_IsEnabledExternalClock

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledExternalClock (TIM_TypeDef * TIMx)
```

Function description

Indicate whether external clock mode 2 is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_TIM_CLOCKSOURCE_ETRMODE2_INSTANCE(TIMx) can be used to check whether or not a timer instance supports external clock mode2.

Reference Manual to LL API cross reference:

- SMCR ECE LL_TIM_IsEnabledExternalClock

LL_TIM_SetClockSource

Function name

```
__STATIC_INLINE void LL_TIM_SetClockSource (TIM_TypeDef * TIMx, uint32_t ClockSource)
```

Function description

Set the clock source of the counter clock.

Parameters

- **TIMx:** Timer instance
- **ClockSource:** This parameter can be one of the following values:
 - LL_TIM_CLOCKSOURCE_INTERNAL
 - LL_TIM_CLOCKSOURCE_EXT_MODE1
 - LL_TIM_CLOCKSOURCE_EXT_MODE2

Return values

- **None:**

Notes

- when selected clock source is external clock mode 1, the timer input the external clock is applied is selected by calling the LL_TIM_SetTriggerInput() function. This timer input must be configured by calling the LL_TIM_IC_Config() function.
- Macro IS_TIM_CLOCKSOURCE_ETRMODE1_INSTANCE(TIMx) can be used to check whether or not a timer instance supports external clock mode1.
- Macro IS_TIM_CLOCKSOURCE_ETRMODE2_INSTANCE(TIMx) can be used to check whether or not a timer instance supports external clock mode2.

Reference Manual to LL API cross reference:

- SMCR SMS LL_TIM_SetClockSource
- SMCR ECE LL_TIM_SetClockSource

LL_TIM_SetEncoderMode

Function name

__STATIC_INLINE void LL_TIM_SetEncoderMode (TIM_TypeDef * TIMx, uint32_t EncoderMode)

Function description

Set the encoder interface mode.

Parameters

- **TIMx:** Timer instance
- **EncoderMode:** This parameter can be one of the following values:
 - LL_TIM_ENCODERMODE_X2_TI1
 - LL_TIM_ENCODERMODE_X2_TI2
 - LL_TIM_ENCODERMODE_X4_TI12

Return values

- **None:**

Notes

- Macro IS_TIM_ENCODER_INTERFACE_INSTANCE(TIMx) can be used to check whether or not a timer instance supports the encoder mode.

Reference Manual to LL API cross reference:

- SMCR SMS LL_TIM_SetEncoderMode

LL_TIM_SetTriggerOutput

Function name

__STATIC_INLINE void LL_TIM_SetTriggerOutput (TIM_TypeDef * TIMx, uint32_t TimerSynchronization)

Function description

Set the trigger output (TRGO) used for timer synchronization .

Parameters

- **TIMx:** Timer instance
- **TimerSynchronization:** This parameter can be one of the following values:
 - LL_TIM_TRGO_RESET
 - LL_TIM_TRGO_ENABLE
 - LL_TIM_TRGO_UPDATE
 - LL_TIM_TRGO_CC1IF
 - LL_TIM_TRGO_OC1REF
 - LL_TIM_TRGO_OC2REF
 - LL_TIM_TRGO_OC3REF
 - LL_TIM_TRGO_OC4REF

Return values

- **None:**

Notes

- Macro IS_TIM_MASTER_INSTANCE(TIMx) can be used to check whether or not a timer instance can operate as a master timer.

Reference Manual to LL API cross reference:

- CR2 MMS LL_TIM_SetTriggerOutput

LL_TIM_SetSlaveMode

Function name

__STATIC_INLINE void LL_TIM_SetSlaveMode (TIM_TypeDef * TIMx, uint32_t SlaveMode)

Function description

Set the synchronization mode of a slave timer.

Parameters

- **TIMx:** Timer instance
- **SlaveMode:** This parameter can be one of the following values:
 - LL_TIM_SLAVEMODE_DISABLED
 - LL_TIM_SLAVEMODE_RESET
 - LL_TIM_SLAVEMODE_GATED
 - LL_TIM_SLAVEMODE_TRIGGER

Return values

- **None:**

Notes

- Macro IS_TIM_SLAVE_INSTANCE(TIMx) can be used to check whether or not a timer instance can operate as a slave timer.

Reference Manual to LL API cross reference:

- SMCR SMS LL_TIM_SetSlaveMode

LL_TIM_SetTriggerInput

Function name

__STATIC_INLINE void LL_TIM_SetTriggerInput (TIM_TypeDef * TIMx, uint32_t TriggerInput)

Function description

Set the selects the trigger input to be used to synchronize the counter.

Parameters

- **TIMx:** Timer instance
- **TriggerInput:** This parameter can be one of the following values:
 - LL_TIM_TS_ITR0
 - LL_TIM_TS_ITR1
 - LL_TIM_TS_ITR2
 - LL_TIM_TS_ITR3
 - LL_TIM_TS_TI1F_ED
 - LL_TIM_TS_TI1FP1
 - LL_TIM_TS_TI2FP2
 - LL_TIM_TS_ETRF

Return values

- **None:**

Notes

- Macro IS_TIM_SLAVE_INSTANCE(TIMx) can be used to check whether or not a timer instance can operate as a slave timer.

Reference Manual to LL API cross reference:

- SMCR TS LL_TIM_SetTriggerInput
- LL_TIM_EnableMasterSlaveMode

Function name

__STATIC_INLINE void LL_TIM_EnableMasterSlaveMode (TIM_TypeDef * TIMx)

Function description

Enable the Master/Slave mode.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro IS_TIM_SLAVE_INSTANCE(TIMx) can be used to check whether or not a timer instance can operate as a slave timer.

Reference Manual to LL API cross reference:

- SMCR MSM LL_TIM_EnableMasterSlaveMode
- LL_TIM_DisableMasterSlaveMode

Function name

__STATIC_INLINE void LL_TIM_DisableMasterSlaveMode (TIM_TypeDef * TIMx)

Function description

Disable the Master/Slave mode.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro `IS_TIM_SLAVE_INSTANCE(TIMx)` can be used to check whether or not a timer instance can operate as a slave timer.

Reference Manual to LL API cross reference:

- SMCR MSM `LL_TIM_DisableMasterSlaveMode`
`LL_TIM_IsEnabledMasterSlaveMode`

Function name

`__STATIC_INLINE uint32_t LL_TIM_IsEnabledMasterSlaveMode (TIM_TypeDef * TIMx)`

Function description

Indicates whether the Master/Slave mode is enabled.

Parameters

- TIMx:** Timer instance

Return values

- State:** of bit (1 or 0).

Notes

- Macro `IS_TIM_SLAVE_INSTANCE(TIMx)` can be used to check whether or not a timer instance can operate as a slave timer.

Reference Manual to LL API cross reference:

- SMCR MSM `LL_TIM_IsEnabledMasterSlaveMode`
`LL_TIM_ConfigETR`

Function name

`__STATIC_INLINE void LL_TIM_ConfigETR (TIM_TypeDef * TIMx, uint32_t ETRPolarity, uint32_t ETRPrescaler, uint32_t ETRFilter)`

Function description

Configure the external trigger (ETR) input.

Parameters

- **TIMx:** Timer instance
- **ETRPolarity:** This parameter can be one of the following values:
 - LL_TIM_ETR_POLARITY_NONINVERTED
 - LL_TIM_ETR_POLARITY_INVERTED
- **ETRPrescaler:** This parameter can be one of the following values:
 - LL_TIM_ETR_PRESCALER_DIV1
 - LL_TIM_ETR_PRESCALER_DIV2
 - LL_TIM_ETR_PRESCALER_DIV4
 - LL_TIM_ETR_PRESCALER_DIV8
- **ETRFilter:** This parameter can be one of the following values:
 - LL_TIM_ETR_FILTER_FDIV1
 - LL_TIM_ETR_FILTER_FDIV1_N2
 - LL_TIM_ETR_FILTER_FDIV1_N4
 - LL_TIM_ETR_FILTER_FDIV1_N8
 - LL_TIM_ETR_FILTER_FDIV2_N6
 - LL_TIM_ETR_FILTER_FDIV2_N8
 - LL_TIM_ETR_FILTER_FDIV4_N6
 - LL_TIM_ETR_FILTER_FDIV4_N8
 - LL_TIM_ETR_FILTER_FDIV8_N6
 - LL_TIM_ETR_FILTER_FDIV8_N8
 - LL_TIM_ETR_FILTER_FDIV16_N5
 - LL_TIM_ETR_FILTER_FDIV16_N6
 - LL_TIM_ETR_FILTER_FDIV16_N8
 - LL_TIM_ETR_FILTER_FDIV32_N5
 - LL_TIM_ETR_FILTER_FDIV32_N6
 - LL_TIM_ETR_FILTER_FDIV32_N8

Return values

- **None:**

Notes

- Macro IS_TIM_ETR_INSTANCE(TIMx) can be used to check whether or not a timer instance provides an external trigger input.

Reference Manual to LL API cross reference:

- SMCR ETP LL_TIM_ConfigETR
- SMCR ETPS LL_TIM_ConfigETR
- SMCR ETF LL_TIM_ConfigETR

LL_TIM_EnableBRK

Function name

__STATIC_INLINE void LL_TIM_EnableBRK (TIM_TypeDef * TIMx)

Function description

Enable the break function.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR BKE `LL_TIM_EnableBRK`
`LL_TIM_DisableBRK`

Function name

__STATIC_INLINE void LL_TIM_DisableBRK (TIM_TypeDef * TIMx)

Function description

Disable the break function.

Parameters

- TIMx:** Timer instance

Return values

- None:**

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR BKE `LL_TIM_DisableBRK`
`LL_TIM_ConfigBRK`

Function name

__STATIC_INLINE void LL_TIM_ConfigBRK (TIM_TypeDef * TIMx, uint32_t BreakPolarity)

Function description

Configure the break input.

Parameters

- TIMx:** Timer instance
- BreakPolarity:** This parameter can be one of the following values:
 - `LL_TIM_BREAK_POLARITY_LOW`
 - `LL_TIM_BREAK_POLARITY_HIGH`

Return values

- None:**

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR BKP `LL_TIM_ConfigBRK`
`LL_TIM_SetOffStates`

Function name

__STATIC_INLINE void LL_TIM_SetOffStates (TIM_TypeDef * TIMx, uint32_t OffStateIdle, uint32_t OffStateRun)

Function description

Select the outputs off state (enabled v.s.

Parameters

- **TIMx:** Timer instance
- **OffStateIdle:** This parameter can be one of the following values:
 - LL_TIM_OSSI_DISABLE
 - LL_TIM_OSSI_ENABLE
- **OffStateRun:** This parameter can be one of the following values:
 - LL_TIM_OSSR_DISABLE
 - LL_TIM_OSSR_ENABLE

Return values

- **None:**

Notes

- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR OSSI LL_TIM_SetOffStates
- BDTR OSSR LL_TIM_SetOffStates

LL_TIM_EnableAutomaticOutput

Function name

__STATIC_INLINE void LL_TIM_EnableAutomaticOutput (TIM_TypeDef * TIMx)

Function description

Enable automatic output (MOE can be set by software or automatically when a break input is active).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR AOE LL_TIM_EnableAutomaticOutput

LL_TIM_DisableAutomaticOutput

Function name

__STATIC_INLINE void LL_TIM_DisableAutomaticOutput (TIM_TypeDef * TIMx)

Function description

Disable automatic output (MOE can be set only by software).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR AOE `LL_TIM_DisableAutomaticOutput`
`LL_TIM_IsEnabledAutomaticOutput`

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledAutomaticOutput (TIM_TypeDef * TIMx)

Function description

Indicate whether automatic output is enabled.

Parameters

- TIMx:** Timer instance

Return values

- State:** of bit (1 or 0).

Notes

- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR AOE `LL_TIM_IsEnabledAutomaticOutput`
`LL_TIM_EnableAllOutputs`

Function name

__STATIC_INLINE void LL_TIM_EnableAllOutputs (TIM_TypeDef * TIMx)

Function description

Enable the outputs (set the MOE bit in TIMx_BDTR register).

Parameters

- TIMx:** Timer instance

Return values

- None:**

Notes

- The MOE bit in TIMx_BDTR register allows to enable /disable the outputs by software and is reset in case of break or break2 event
- Macro `IS_TIM_BREAK_INSTANCE(TIMx)` can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR MOE `LL_TIM_EnableAllOutputs`
`LL_TIM_DisableAllOutputs`

Function name

__STATIC_INLINE void LL_TIM_DisableAllOutputs (TIM_TypeDef * TIMx)

Function description

Disable the outputs (reset the MOE bit in TIMx_BDTR register).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Notes

- The MOE bit in TIMx_BDTR register allows to enable /disable the outputs by software and is reset in case of break or break2 event.
- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR MOE LL_TIM_DisableAllOutputs

LL_TIM_IsEnabledAllOutputs

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledAllOutputs (TIM_TypeDef * TIMx)

Function description

Indicates whether outputs are enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not a timer instance provides a break input.

Reference Manual to LL API cross reference:

- BDTR MOE LL_TIM_IsEnabledAllOutputs

LL_TIM_ConfigDMABurst

Function name

__STATIC_INLINE void LL_TIM_ConfigDMABurst (TIM_TypeDef * TIMx, uint32_t DMABurstBaseAddress, uint32_t DMABurstLength)

Function description

Configures the timer DMA burst feature.

Parameters

- **TIMx:** Timer instance
- **DMABurstBaseAddress:** This parameter can be one of the following values:
 - LL_TIM_DMABURST_BASEADDR_CR1
 - LL_TIM_DMABURST_BASEADDR_CR2
 - LL_TIM_DMABURST_BASEADDR_SMCR
 - LL_TIM_DMABURST_BASEADDR_DIER
 - LL_TIM_DMABURST_BASEADDR_SR
 - LL_TIM_DMABURST_BASEADDR_EGR
 - LL_TIM_DMABURST_BASEADDR_CCMR1
 - LL_TIM_DMABURST_BASEADDR_CCMR2
 - LL_TIM_DMABURST_BASEADDR_CCER
 - LL_TIM_DMABURST_BASEADDR_CNT
 - LL_TIM_DMABURST_BASEADDR_PSC
 - LL_TIM_DMABURST_BASEADDR_ARR
 - LL_TIM_DMABURST_BASEADDR_RCR
 - LL_TIM_DMABURST_BASEADDR_CCR1
 - LL_TIM_DMABURST_BASEADDR_CCR2
 - LL_TIM_DMABURST_BASEADDR_CCR3
 - LL_TIM_DMABURST_BASEADDR_CCR4
 - LL_TIM_DMABURST_BASEADDR_BDTR
- **DMABurstLength:** This parameter can be one of the following values:
 - LL_TIM_DMABURST_LENGTH_1TRANSFER
 - LL_TIM_DMABURST_LENGTH_2TRANSFERS
 - LL_TIM_DMABURST_LENGTH_3TRANSFERS
 - LL_TIM_DMABURST_LENGTH_4TRANSFERS
 - LL_TIM_DMABURST_LENGTH_5TRANSFERS
 - LL_TIM_DMABURST_LENGTH_6TRANSFERS
 - LL_TIM_DMABURST_LENGTH_7TRANSFERS
 - LL_TIM_DMABURST_LENGTH_8TRANSFERS
 - LL_TIM_DMABURST_LENGTH_9TRANSFERS
 - LL_TIM_DMABURST_LENGTH_10TRANSFERS
 - LL_TIM_DMABURST_LENGTH_11TRANSFERS
 - LL_TIM_DMABURST_LENGTH_12TRANSFERS
 - LL_TIM_DMABURST_LENGTH_13TRANSFERS
 - LL_TIM_DMABURST_LENGTH_14TRANSFERS
 - LL_TIM_DMABURST_LENGTH_15TRANSFERS
 - LL_TIM_DMABURST_LENGTH_16TRANSFERS
 - LL_TIM_DMABURST_LENGTH_17TRANSFERS
 - LL_TIM_DMABURST_LENGTH_18TRANSFERS

Return values

- **None:**

Notes

- Macro `IS_TIM_DMABURST_INSTANCE(TIMx)` can be used to check whether or not a timer instance supports the DMA burst mode.

Reference Manual to LL API cross reference:

- DCR DBL LL_TIM_ConfigDMABurst
- DCR DBA LL_TIM_ConfigDMABurst

LL_TIM_SetRemap

Function name

__STATIC_INLINE void LL_TIM_SetRemap (TIM_TypeDef * TIMx, uint32_t Remap)

Function description

Remap TIM inputs (input channel, internal/external triggers).

Parameters

- **TIMx:** Timer instance
- **Remap:** Remap param depends on the TIMx. Description available only in CHM version of the User Manual (not in .pdf). Otherwise see Reference Manual description of OR registers.

Return values

- **None:**

Notes

- Macro IS_TIM_REMAP_INSTANCE(TIMx) can be used to check whether or not a some timer inputs can be remapped.

Reference Manual to LL API cross reference:

- TIM2_OR ITR1_RMP LL_TIM_SetRemap
- TIM5_OR TI4_RMP LL_TIM_SetRemap
- TIM11_OR TI1_RMP LL_TIM_SetRemap

LL_TIM_ClearFlag_UPDATE

Function name

__STATIC_INLINE void LL_TIM_ClearFlag_UPDATE (TIM_TypeDef * TIMx)

Function description

Clear the update interrupt flag (UIF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR UIF LL_TIM_ClearFlag_UPDATE

LL_TIM_IsActiveFlag_UPDATE

Function name

__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_UPDATE (TIM_TypeDef * TIMx)

Function description

Indicate whether update interrupt flag (UIF) is set (update interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR UIF LL_TIM_IsActiveFlag_UPDATE
LL_TIM_ClearFlag_CC1

Function name

__STATIC_INLINE void LL_TIM_ClearFlag_CC1 (TIM_TypeDef * TIMx)

Function description

Clear the Capture/Compare 1 interrupt flag (CC1F).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC1IF LL_TIM_ClearFlag_CC1
LL_TIM_IsActiveFlag_CC1

Function name

__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC1 (TIM_TypeDef * TIMx)

Function description

Indicate whether Capture/Compare 1 interrupt flag (CC1F) is set (Capture/Compare 1 interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC1IF LL_TIM_IsActiveFlag_CC1
LL_TIM_ClearFlag_CC2

Function name

__STATIC_INLINE void LL_TIM_ClearFlag_CC2 (TIM_TypeDef * TIMx)

Function description

Clear the Capture/Compare 2 interrupt flag (CC2F).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC2IF LL_TIM_ClearFlag_CC2
LL_TIM_IsActiveFlag_CC2

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC2 (TIM_TypeDef * TIMx)
```

Function description

Indicate whether Capture/Compare 2 interrupt flag (CC2F) is set (Capture/Compare 2 interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC2IF LL_TIM_IsActiveFlag_CC2

```
LL_TIM_ClearFlag_CC3
```

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_CC3 (TIM_TypeDef * TIMx)
```

Function description

Clear the Capture/Compare 3 interrupt flag (CC3F).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC3IF LL_TIM_ClearFlag_CC3

```
LL_TIM_IsActiveFlag_CC3
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC3 (TIM_TypeDef * TIMx)
```

Function description

Indicate whether Capture/Compare 3 interrupt flag (CC3F) is set (Capture/Compare 3 interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC3IF LL_TIM_IsActiveFlag_CC3

```
LL_TIM_ClearFlag_CC4
```

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_CC4 (TIM_TypeDef * TIMx)
```

Function description

Clear the Capture/Compare 4 interrupt flag (CC4F).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC4IF LL_TIM_ClearFlag_CC4
LL_TIM_IsActiveFlag_CC4

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC4 (TIM_TypeDef * TIMx)
```

Function description

Indicate whether Capture/Compare 4 interrupt flag (CC4F) is set (Capture/Compare 4 interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC4IF LL_TIM_IsActiveFlag_CC4
LL_TIM_ClearFlag_COM

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_COM (TIM_TypeDef * TIMx)
```

Function description

Clear the commutation interrupt flag (COMIF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR COMIF LL_TIM_ClearFlag_COM
LL_TIM_IsActiveFlag_COM

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_COM (TIM_TypeDef * TIMx)
```

Function description

Indicate whether commutation interrupt flag (COMIF) is set (commutation interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR COMIF LL_TIM_IsActiveFlag_COM
- LL_TIM_ClearFlag_TRIG

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_TRIG (TIM_TypeDef * TIMx)
```

Function description

Clear the trigger interrupt flag (TIF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR TIF LL_TIM_ClearFlag_TRIG
- LL_TIM_IsActiveFlag_TRIG

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_TRIG (TIM_TypeDef * TIMx)
```

Function description

Indicate whether trigger interrupt flag (TIF) is set (trigger interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR TIF LL_TIM_IsActiveFlag_TRIG
- LL_TIM_ClearFlag_BRK

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_BRK (TIM_TypeDef * TIMx)
```

Function description

Clear the break interrupt flag (BIF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR BIF LL_TIM_ClearFlag_BRK
- LL_TIM_IsActiveFlag_BRK

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_BRK (TIM_TypeDef * TIMx)
```

Function description

Indicate whether break interrupt flag (BIF) is set (break interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR BIF LL_TIM_IsActiveFlag_BRK
LL_TIM_ClearFlag_CC1OVR

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_CC1OVR (TIM_TypeDef * TIMx)
```

Function description

Clear the Capture/Compare 1 over-capture interrupt flag (CC1OF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC1OF LL_TIM_ClearFlag_CC1OVR
LL_TIM_IsActiveFlag_CC1OVR

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC1OVR (TIM_TypeDef * TIMx)
```

Function description

Indicate whether Capture/Compare 1 over-capture interrupt flag (CC1OF) is set (Capture/Compare 1 interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC1OF LL_TIM_IsActiveFlag_CC1OVR
LL_TIM_ClearFlag_CC2OVR

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_CC2OVR (TIM_TypeDef * TIMx)
```

Function description

Clear the Capture/Compare 2 over-capture interrupt flag (CC2OF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC2OF LL_TIM_ClearFlag_CC2OVR
LL_TIM_IsActiveFlag_CC2OVR

Function name

__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC2OVR (TIM_TypeDef * TIMx)

Function description

Indicate whether Capture/Compare 2 over-capture interrupt flag (CC2OF) is set (Capture/Compare 2 over-capture interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC2OF LL_TIM_IsActiveFlag_CC2OVR
LL_TIM_ClearFlag_CC3OVR

Function name

__STATIC_INLINE void LL_TIM_ClearFlag_CC3OVR (TIM_TypeDef * TIMx)

Function description

Clear the Capture/Compare 3 over-capture interrupt flag (CC3OF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC3OF LL_TIM_ClearFlag_CC3OVR
LL_TIM_IsActiveFlag_CC3OVR

Function name

__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC3OVR (TIM_TypeDef * TIMx)

Function description

Indicate whether Capture/Compare 3 over-capture interrupt flag (CC3OF) is set (Capture/Compare 3 over-capture interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC3OF LL_TIM_IsActiveFlag_CC3OVR


```
LL_TIM_ClearFlag_CC4OVR
```

Function name

```
__STATIC_INLINE void LL_TIM_ClearFlag_CC4OVR (TIM_TypeDef * TIMx)
```

Function description

Clear the Capture/Compare 4 over-capture interrupt flag (CC4OF).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR CC4OF LL_TIM_ClearFlag_CC4OVR

```
LL_TIM_IsActiveFlag_CC4OVR
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsActiveFlag_CC4OVR (TIM_TypeDef * TIMx)
```

Function description

Indicate whether Capture/Compare 4 over-capture interrupt flag (CC4OF) is set (Capture/Compare 4 over-capture interrupt is pending).

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR CC4OF LL_TIM_IsActiveFlag_CC4OVR

```
LL_TIM_EnableIT_UPDATE
```

Function name

```
__STATIC_INLINE void LL_TIM_EnableIT_UPDATE (TIM_TypeDef * TIMx)
```

Function description

Enable update interrupt (UIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER UIE LL_TIM_EnableIT_UPDATE

```
LL_TIM_DisableIT_UPDATE
```

Function name

```
__STATIC_INLINE void LL_TIM_DisableIT_UPDATE (TIM_TypeDef * TIMx)
```

Function description

Disable update interrupt (UIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER UIE LL_TIM_DisableIT_UPDATE
LL_TIM_IsEnabledIT_UPDATE

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_UPDATE(TIM_TypeDef * TIMx)
```

Function description

Indicates whether the update interrupt (UIE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER UIE LL_TIM_IsEnabledIT_UPDATE
LL_TIM_EnableIT_CC1

Function name

```
__STATIC_INLINE void LL_TIM_EnableIT_CC1(TIM_TypeDef * TIMx)
```

Function description

Enable capture/compare 1 interrupt (CC1IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC1IE LL_TIM_EnableIT_CC1
LL_TIM_DisableIT_CC1

Function name

```
__STATIC_INLINE void LL_TIM_DisableIT_CC1(TIM_TypeDef * TIMx)
```

Function description

Disable capture/compare 1 interrupt (CC1IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC1IE LL_TIM_DisableIT_CC1
LL_TIM_IsEnabledIT_CC1

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_CC1 (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the capture/compare 1 interrupt (CC1IE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC1IE LL_TIM_IsEnabledIT_CC1
LL_TIM_EnableIT_CC2

Function name

```
__STATIC_INLINE void LL_TIM_EnableIT_CC2 (TIM_TypeDef * TIMx)
```

Function description

Enable capture/compare 2 interrupt (CC2IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC2IE LL_TIM_EnableIT_CC2
LL_TIM_DisableIT_CC2

Function name

```
__STATIC_INLINE void LL_TIM_DisableIT_CC2 (TIM_TypeDef * TIMx)
```

Function description

Disable capture/compare 2 interrupt (CC2IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC2IE LL_TIM_DisableIT_CC2
LL_TIM_IsEnabledIT_CC2

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_CC2 (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the capture/compare 2 interrupt (CC2IE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC2IE LL_TIM_IsEnabledIT_CC2
LL_TIM_EnableIT_CC3

Function name

__STATIC_INLINE void LL_TIM_EnableIT_CC3 (TIM_TypeDef * TIMx)

Function description

Enable capture/compare 3 interrupt (CC3IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC3IE LL_TIM_EnableIT_CC3
LL_TIM_DisableIT_CC3

Function name

__STATIC_INLINE void LL_TIM_DisableIT_CC3 (TIM_TypeDef * TIMx)

Function description

Disable capture/compare 3 interrupt (CC3IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC3IE LL_TIM_DisableIT_CC3
LL_TIM_IsEnabledIT_CC3

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_CC3 (TIM_TypeDef * TIMx)

Function description

Indicates whether the capture/compare 3 interrupt (CC3IE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC3IE LL_TIM_IsEnabledIT_CC3
LL_TIM_EnableIT_CC4

Function name

__STATIC_INLINE void LL_TIM_EnableIT_CC4 (TIM_TypeDef * TIMx)

Function description

Enable capture/compare 4 interrupt (CC4IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC4IE LL_TIM_EnableIT_CC4
LL_TIM_DisableIT_CC4

Function name

__STATIC_INLINE void LL_TIM_DisableIT_CC4 (TIM_TypeDef * TIMx)

Function description

Disable capture/compare 4 interrupt (CC4IE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC4IE LL_TIM_DisableIT_CC4
LL_TIM_IsEnabledIT_CC4

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_CC4 (TIM_TypeDef * TIMx)

Function description

Indicates whether the capture/compare 4 interrupt (CC4IE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC4IE LL_TIM_IsEnabledIT_CC4
LL_TIM_EnableIT_COM

Function name

```
__STATIC_INLINE void LL_TIM_EnableIT_COM (TIM_TypeDef * TIMx)
```

Function description

Enable commutation interrupt (COMIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER COMIE LL_TIM_EnableIT_COM

LL_TIM_DisableIT_COM

Function name

```
__STATIC_INLINE void LL_TIM_DisableIT_COM (TIM_TypeDef * TIMx)
```

Function description

Disable commutation interrupt (COMIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER COMIE LL_TIM_DisableIT_COM

LL_TIM_IsEnabledIT_COM

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_COM (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the commutation interrupt (COMIE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER COMIE LL_TIM_IsEnabledIT_COM

LL_TIM_EnableIT_TRIG

Function name

```
__STATIC_INLINE void LL_TIM_EnableIT_TRIG (TIM_TypeDef * TIMx)
```

Function description

Enable trigger interrupt (TIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER TIE LL_TIM_EnableIT_TRIG
LL_TIM_DisableIT_TRIG

Function name

__STATIC_INLINE void LL_TIM_DisableIT_TRIG (TIM_TypeDef * TIMx)

Function description

Disable trigger interrupt (TIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER TIE LL_TIM_DisableIT_TRIG
LL_TIM_IsEnabledIT_TRIG

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_TRIG (TIM_TypeDef * TIMx)

Function description

Indicates whether the trigger interrupt (TIE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER TIE LL_TIM_IsEnabledIT_TRIG
LL_TIM_EnableIT_BRK

Function name

__STATIC_INLINE void LL_TIM_EnableIT_BRK (TIM_TypeDef * TIMx)

Function description

Enable break interrupt (BIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER BIE LL_TIM_EnableIT_BRK
LL_TIM_DisableIT_BRK

Function name

```
__STATIC_INLINE void LL_TIM_DisableIT_BRK (TIM_TypeDef * TIMx)
```

Function description

Disable break interrupt (BIE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER BIE LL_TIM_DisableIT_BRK
LL_TIM_IsEnabledIT_BRK

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledIT_BRK (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the break interrupt (BIE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER BIE LL_TIM_IsEnabledIT_BRK
LL_TIM_EnableDMAReq_UPDATE

Function name

```
__STATIC_INLINE void LL_TIM_EnableDMAReq_UPDATE (TIM_TypeDef * TIMx)
```

Function description

Enable update DMA request (UDE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER UDE LL_TIM_EnableDMAReq_UPDATE
LL_TIM_DisableDMAReq_UPDATE

Function name

```
__STATIC_INLINE void LL_TIM_DisableDMAReq_UPDATE (TIM_TypeDef * TIMx)
```


Function description

Disable update DMA request (UDE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER UDE LL_TIM_DisableDMAReq_UPDATE

LL_TIM_IsEnabledDMAReq_UPDATE

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_UPDATE (TIM_TypeDef * TIMx)

Function description

Indicates whether the update DMA request (UDE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER UDE LL_TIM_IsEnabledDMAReq_UPDATE

LL_TIM_EnableDMAReq_CC1

Function name

__STATIC_INLINE void LL_TIM_EnableDMAReq_CC1 (TIM_TypeDef * TIMx)

Function description

Enable capture/compare 1 DMA request (CC1DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC1DE LL_TIM_EnableDMAReq_CC1

LL_TIM_DisableDMAReq_CC1

Function name

__STATIC_INLINE void LL_TIM_DisableDMAReq_CC1 (TIM_TypeDef * TIMx)

Function description

Disable capture/compare 1 DMA request (CC1DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC1DE LL_TIM_DisableDMAReq_CC1
LL_TIM_IsEnabledDMAReq_CC1

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_CC1 (TIM_TypeDef * TIMx)

Function description

Indicates whether the capture/compare 1 DMA request (CC1DE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC1DE LL_TIM_IsEnabledDMAReq_CC1
LL_TIM_EnableDMAReq_CC2

Function name

__STATIC_INLINE void LL_TIM_EnableDMAReq_CC2 (TIM_TypeDef * TIMx)

Function description

Enable capture/compare 2 DMA request (CC2DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC2DE LL_TIM_EnableDMAReq_CC2
LL_TIM_DisableDMAReq_CC2

Function name

__STATIC_INLINE void LL_TIM_DisableDMAReq_CC2 (TIM_TypeDef * TIMx)

Function description

Disable capture/compare 2 DMA request (CC2DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC2DE LL_TIM_DisableDMAReq_CC2
LL_TIM_IsEnabledDMAReq_CC2

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_CC2 (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the capture/compare 2 DMA request (CC2DE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC2DE LL_TIM_IsEnabledDMAReq_CC2

```
LL_TIM_EnableDMAReq_CC3
```

Function name

```
__STATIC_INLINE void LL_TIM_EnableDMAReq_CC3 (TIM_TypeDef * TIMx)
```

Function description

Enable capture/compare 3 DMA request (CC3DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC3DE LL_TIM_EnableDMAReq_CC3

```
LL_TIM_DisableDMAReq_CC3
```

Function name

```
__STATIC_INLINE void LL_TIM_DisableDMAReq_CC3 (TIM_TypeDef * TIMx)
```

Function description

Disable capture/compare 3 DMA request (CC3DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC3DE LL_TIM_DisableDMAReq_CC3

```
LL_TIM_IsEnabledDMAReq_CC3
```

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_CC3 (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the capture/compare 3 DMA request (CC3DE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC3DE LL_TIM_IsEnabledDMAReq_CC3
LL_TIM_EnableDMAReq_CC4

Function name

```
__STATIC_INLINE void LL_TIM_EnableDMAReq_CC4 (TIM_TypeDef * TIMx)
```

Function description

Enable capture/compare 4 DMA request (CC4DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC4DE LL_TIM_EnableDMAReq_CC4
LL_TIM_DisableDMAReq_CC4

Function name

```
__STATIC_INLINE void LL_TIM_DisableDMAReq_CC4 (TIM_TypeDef * TIMx)
```

Function description

Disable capture/compare 4 DMA request (CC4DE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER CC4DE LL_TIM_DisableDMAReq_CC4
LL_TIM_IsEnabledDMAReq_CC4

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_CC4 (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the capture/compare 4 DMA request (CC4DE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER CC4DE LL_TIM_IsEnabledDMAReq_CC4
LL_TIM_EnableDMAReq_COM

Function name

```
__STATIC_INLINE void LL_TIM_EnableDMAReq_COM (TIM_TypeDef * TIMx)
```

Function description

Enable commutation DMA request (COMDE).

Parameters

- **TIMx**: Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER COMDE LL_TIM_EnableDMAReq_COM
LL_TIM_DisableDMAReq_COM

Function name

```
__STATIC_INLINE void LL_TIM_DisableDMAReq_COM (TIM_TypeDef * TIMx)
```

Function description

Disable commutation DMA request (COMDE).

Parameters

- **TIMx**: Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER COMDE LL_TIM_DisableDMAReq_COM
LL_TIM_IsEnabledDMAReq_COM

Function name

```
__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_COM (TIM_TypeDef * TIMx)
```

Function description

Indicates whether the commutation DMA request (COMDE) is enabled.

Parameters

- **TIMx**: Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER COMDE LL_TIM_IsEnabledDMAReq_COM
LL_TIM_EnableDMAReq_TRIG

Function name

```
__STATIC_INLINE void LL_TIM_EnableDMAReq_TRIG (TIM_TypeDef * TIMx)
```

Function description

Enable trigger interrupt (TDE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER TDE LL_TIM_EnableDMAReq_TRIG

LL_TIM_DisableDMAReq_TRIG

Function name

__STATIC_INLINE void LL_TIM_DisableDMAReq_TRIG (TIM_TypeDef * TIMx)

Function description

Disable trigger interrupt (TDE).

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- DIER TDE LL_TIM_DisableDMAReq_TRIG

LL_TIM_IsEnabledDMAReq_TRIG

Function name

__STATIC_INLINE uint32_t LL_TIM_IsEnabledDMAReq_TRIG (TIM_TypeDef * TIMx)

Function description

Indicates whether the trigger interrupt (TDE) is enabled.

Parameters

- **TIMx:** Timer instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- DIER TDE LL_TIM_IsEnabledDMAReq_TRIG

LL_TIM_GenerateEvent_UPDATE

Function name

__STATIC_INLINE void LL_TIM_GenerateEvent_UPDATE (TIM_TypeDef * TIMx)

Function description

Generate an update event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR UG LL_TIM_GenerateEvent_UPDATE
LL_TIM_GenerateEvent_CC1

Function name

__STATIC_INLINE void LL_TIM_GenerateEvent_CC1 (TIM_TypeDef * TIMx)

Function description

Generate Capture/Compare 1 event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR CC1G LL_TIM_GenerateEvent_CC1
LL_TIM_GenerateEvent_CC2

Function name

__STATIC_INLINE void LL_TIM_GenerateEvent_CC2 (TIM_TypeDef * TIMx)

Function description

Generate Capture/Compare 2 event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR CC2G LL_TIM_GenerateEvent_CC2
LL_TIM_GenerateEvent_CC3

Function name

__STATIC_INLINE void LL_TIM_GenerateEvent_CC3 (TIM_TypeDef * TIMx)

Function description

Generate Capture/Compare 3 event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR CC3G LL_TIM_GenerateEvent_CC3
LL_TIM_GenerateEvent_CC4

Function name

```
__STATIC_INLINE void LL_TIM_GenerateEvent_CC4 (TIM_TypeDef * TIMx)
```

Function description

Generate Capture/Compare 4 event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR CC4G LL_TIM_GenerateEvent_CC4

LL_TIM_GenerateEvent_COM

Function name

```
__STATIC_INLINE void LL_TIM_GenerateEvent_COM (TIM_TypeDef * TIMx)
```

Function description

Generate commutation event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR COMG LL_TIM_GenerateEvent_COM

LL_TIM_GenerateEvent_TRIG

Function name

```
__STATIC_INLINE void LL_TIM_GenerateEvent_TRIG (TIM_TypeDef * TIMx)
```

Function description

Generate trigger event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR TG LL_TIM_GenerateEvent_TRIG

LL_TIM_GenerateEvent_BRK

Function name

```
__STATIC_INLINE void LL_TIM_GenerateEvent_BRK (TIM_TypeDef * TIMx)
```

Function description

Generate break event.

Parameters

- **TIMx:** Timer instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- EGR BG LL_TIM_GenerateEvent_BRK
LL_TIM_DeInit

Function name

ErrorStatus LL_TIM_DeInit (TIM_TypeDef * TIMx)

Function description

Set TIMx registers to their reset values.

Parameters

- **TIMx:** Timer instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx registers are de-initialized
 - ERROR: invalid TIMx instance

LL_TIM_StructInit

Function name

void LL_TIM_StructInit (LL_TIM_InitTypeDef * TIM_InitStruct)

Function description

Set the fields of the time base unit configuration data structure to their default values.

Parameters

- **TIM_InitStruct:** pointer to a LL_TIM_InitTypeDef structure (time base unit configuration data structure)

Return values

- **None:**

LL_TIM_Init

Function name

ErrorStatus LL_TIM_Init (TIM_TypeDef * TIMx, LL_TIM_InitTypeDef * TIM_InitStruct)

Function description

Configure the TIMx time base unit.

Parameters

- **TIMx:** Timer Instance
- **TIM_InitStruct:** pointer to a LL_TIM_InitTypeDef structure (TIMx time base unit configuration data structure)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx registers are de-initialized
 - ERROR: not applicable

LL_TIM_OC_StructInit

Function name

```
void LL_TIM_OC_StructInit (LL_TIM_OC_InitTypeDef * TIM_OC_InitStruct)
```

Function description

Set the fields of the TIMx output channel configuration data structure to their default values.

Parameters

- **TIM_OC_InitStruct:** pointer to a LL_TIM_OC_InitTypeDef structure (the output channel configuration data structure)

Return values

- **None:**

```
LL_TIM_OC_Init
```

Function name

```
ErrorStatus LL_TIM_OC_Init (TIM_TypeDef * TIMx, uint32_t Channel, LL_TIM_OC_InitTypeDef * TIM_OC_InitStruct)
```

Function description

Configure the TIMx output channel.

Parameters

- **TIMx:** Timer Instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **TIM_OC_InitStruct:** pointer to a LL_TIM_OC_InitTypeDef structure (TIMx output channel configuration data structure)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx output channel is initialized
 - ERROR: TIMx output channel is not initialized

```
LL_TIM_IC_StructInit
```

Function name

```
void LL_TIM_IC_StructInit (LL_TIM_IC_InitTypeDef * TIM_ICInitStruct)
```

Function description

Set the fields of the TIMx input channel configuration data structure to their default values.

Parameters

- **TIM_ICInitStruct:** pointer to a LL_TIM_IC_InitTypeDef structure (the input channel configuration data structure)

Return values

- **None:**

```
LL_TIM_IC_Init
```

Function name

```
ErrorStatus LL_TIM_IC_Init (TIM_TypeDef * TIMx, uint32_t Channel, LL_TIM_IC_InitTypeDef * TIM_IC_InitStruct)
```

Function description

Configure the TIMx input channel.

Parameters

- **TIMx:** Timer Instance
- **Channel:** This parameter can be one of the following values:
 - LL_TIM_CHANNEL_CH1
 - LL_TIM_CHANNEL_CH2
 - LL_TIM_CHANNEL_CH3
 - LL_TIM_CHANNEL_CH4
- **TIM_IC_InitStruct:** pointer to a LL_TIM_IC_InitTypeDef structure (TIMx input channel configuration data structure)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx output channel is initialized
 - ERROR: TIMx output channel is not initialized

LL_TIM_ENCODER_StructInit

Function name

void LL_TIM_ENCODER_StructInit (LL_TIM_ENCODER_InitTypeDef * TIM_EncoderInitStruct)

Function description

Fills each TIM_EncoderInitStruct field with its default value.

Parameters

- **TIM_EncoderInitStruct:** pointer to a LL_TIM_ENCODER_InitTypeDef structure (encoder interface configuration data structure)

Return values

- **None:**

LL_TIM_ENCODER_Init

Function name

ErrorStatus LL_TIM_ENCODER_Init (TIM_TypeDef * TIMx, LL_TIM_ENCODER_InitTypeDef * TIM_EncoderInitStruct)

Function description

Configure the encoder interface of the timer instance.

Parameters

- **TIMx:** Timer Instance
- **TIM_EncoderInitStruct:** pointer to a LL_TIM_ENCODER_InitTypeDef structure (TIMx encoder interface configuration data structure)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx registers are de-initialized
 - ERROR: not applicable

LL_TIM_HALLSENSOR_StructInit

Function name

void LL_TIM_HALLSENSOR_StructInit (LL_TIM_HALLSENSOR_InitTypeDef * TIM_HallSensorInitStruct)

Function description

Set the fields of the TIMx Hall sensor interface configuration data structure to their default values.

Parameters

- **TIM_HallSensorInitStruct:** pointer to a LL_TIM_HALLSENSOR_InitTypeDef structure (HALL sensor interface configuration data structure)

Return values

- **None:**

LL_TIM_HALLSENSOR_Init

Function name

ErrorStatus LL_TIM_HALLSENSOR_Init (TIM_TypeDef * TIMx, LL_TIM_HALLSENSOR_InitTypeDef * TIM_HallSensorInitStruct)

Function description

Configure the Hall sensor interface of the timer instance.

Parameters

- **TIMx:** Timer Instance
- **TIM_HallSensorInitStruct:** pointer to a LL_TIM_HALLSENSOR_InitTypeDef structure (TIMx HALL sensor interface configuration data structure)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: TIMx registers are de-initialized
 - ERROR: not applicable

Notes

- TIMx CH1, CH2 and CH3 inputs connected through a XOR to the TI1 input channel
- TIMx slave mode controller is configured in reset mode. Selected internal trigger is TI1F_ED.
- Channel 1 is configured as input, IC1 is mapped on TRC.
- Captured value stored in TIMx_CCR1 correspond to the time elapsed between 2 changes on the inputs. It gives information about motor speed.
- Channel 2 is configured in output PWM 2 mode.
- Compare value stored in TIMx_CCR2 corresponds to the commutation delay.
- OC2REF is selected as trigger output on TRGO.
- LL_TIM_IC_POLARITY_BOTHEDGE must not be used for TI1 when it is used when TIMx operates in Hall sensor interface mode.

LL_TIM_BDTR_StructInit

Function name

void LL_TIM_BDTR_StructInit (LL_TIM_BDTR_InitTypeDef * TIM_BDTRInitStruct)

Function description

Set the fields of the Break and Dead Time configuration data structure to their default values.

Parameters

- **TIM_BDTRInitStruct:** pointer to a LL_TIM_BDTR_InitTypeDef structure (Break and Dead Time configuration data structure)

Return values

- **None:**

LL_TIM_BDTR_Init

Function name

ErrorStatus LL_TIM_BDTR_Init (TIM_TypeDef * TIMx, LL_TIM_BDTR_InitTypeDef * TIM_BDTRInitStruct)

Function description

Configure the Break and Dead Time feature of the timer instance.

Parameters

- **TIMx**: Timer Instance
- **TIM_BDTRInitStruct**: pointer to a LL_TIM_BDTR_InitTypeDef structure (Break and Dead Time configuration data structure)

Return values

- **An**: ErrorStatus enumeration value:
 - SUCCESS: Break and Dead Time is initialized
 - ERROR: not applicable

Notes

- As the bits AOE, BKP, BKE, OSSR, OSSI and DTG[7:0] can be write-locked depending on the LOCK configuration, it can be necessary to configure all of them during the first write access to the TIMx_BDTR register.
- Macro IS_TIM_BREAK_INSTANCE(TIMx) can be used to check whether or not a timer instance provides a break input.

65.3 TIM Firmware driver defines

The following section lists the various define and macros of the module.

65.3.1 TIM

TIM

Active Input Selection

LL_TIM_ACTIVEINPUT_DIRECTTI

ICx is mapped on TIx

LL_TIM_ACTIVEINPUT_INDIRECTTI

ICx is mapped on TIy

LL_TIM_ACTIVEINPUT_TRC

ICx is mapped on TRC

Automatic output enable

LL_TIM_AUTOMATICOUTPUT_DISABLE

MOE can be set only by software

LL_TIM_AUTOMATICOUTPUT_ENABLE

MOE can be set by software or automatically at the next update event

Break Enable

LL_TIM_BREAK_DISABLE

Break function disabled

LL_TIM_BREAK_ENABLE

Break function enabled

break polarity

LL_TIM_BREAK_POLARITY_LOW

Break input BRK is active low

LL_TIM_BREAK_POLARITY_HIGH

Break input BRK is active high

Capture Compare DMA Request

LL_TIM_CCDMAREQUEST_CC

CCx DMA request sent when CCx event occurs

LL_TIM_CCDMAREQUEST_UPDATE

CCx DMA requests sent when update event occurs

Capture Compare Update Source

LL_TIM_CCUPDATESOURCE_COMG_ONLY

Capture/compare control bits are updated by setting the COMG bit only

LL_TIM_CCUPDATESOURCE_COMG_AND_TRGI

Capture/compare control bits are updated by setting the COMG bit or when a rising edge occurs on trigger input (TRGI)

Channel

LL_TIM_CHANNEL_CH1

Timer input/output channel 1

LL_TIM_CHANNEL_CH1N

Timer complementary output channel 1

LL_TIM_CHANNEL_CH2

Timer input/output channel 2

LL_TIM_CHANNEL_CH2N

Timer complementary output channel 2

LL_TIM_CHANNEL_CH3

Timer input/output channel 3

LL_TIM_CHANNEL_CH3N

Timer complementary output channel 3

LL_TIM_CHANNEL_CH4

Timer input/output channel 4

Clock Division

LL_TIM_CLOCKDIVISION_DIV1

$tDTS=tCK_INT$

LL_TIM_CLOCKDIVISION_DIV2

$tDTS=2*tCK_INT$

LL_TIM_CLOCKDIVISION_DIV4

$tDTS=4*tCK_INT$

Clock Source

LL_TIM_CLOCKSOURCE_INTERNAL

The timer is clocked by the internal clock provided from the RCC

LL_TIM_CLOCKSOURCE_EXT_MODE1

Counter counts at each rising or falling edge on a selected input

LL_TIM_CLOCKSOURCE_EXT_MODE2

Counter counts at each rising or falling edge on the external trigger input ETR

Counter Direction

LL_TIM_COUNTERDIRECTION_UP

Timer counter counts up

LL_TIM_COUNTERDIRECTION_DOWN

Timer counter counts down

Counter Mode

LL_TIM_COUNTERMODE_UP

Counter used as upcounter

LL_TIM_COUNTERMODE_DOWN

Counter used as downcounter

LL_TIM_COUNTERMODE_CENTER_UP

The counter counts up and down alternatively. Output compare interrupt flags of output channels are set only when the counter is counting down.

LL_TIM_COUNTERMODE_CENTER_DOWN

The counter counts up and down alternatively. Output compare interrupt flags of output channels are set only when the counter is counting up

LL_TIM_COUNTERMODE_CENTER_UP_DOWN

The counter counts up and down alternatively. Output compare interrupt flags of output channels are set only when the counter is counting up or down.

DMA Burst Base Address

LL_TIM_DMABURST_BASEADDR_CR1

TIMx_CR1 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CR2

TIMx_CR2 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_SMCR

TIMx_SMCR register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_DIER

TIMx_DIER register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_SR

TIMx_SR register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_EGR

TIMx_EGR register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCMR1

TIMx_CCMR1 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCMR2

TIMx_CCMR2 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCER

TIMx_CCER register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CNT

TIMx_CNT register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_PSC

TIMx_PSC register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_ARR

TIMx_ARR register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_RCR

TIMx_RCR register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCR1

TIMx_CCR1 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCR2

TIMx_CCR2 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCR3

TIMx_CCR3 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_CCR4

TIMx_CCR4 register is the DMA base address for DMA burst

LL_TIM_DMABURST_BASEADDR_BDTR

TIMx_BDTR register is the DMA base address for DMA burst

DMA Burst Length

LL_TIM_DMABURST_LENGTH_1TRANSFER

Transfer is done to 1 register starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_2TRANSFERS

Transfer is done to 2 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_3TRANSFERS

Transfer is done to 3 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_4TRANSFERS

Transfer is done to 4 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_5TRANSFERS

Transfer is done to 5 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_6TRANSFERS

Transfer is done to 6 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_7TRANSFERS

Transfer is done to 7 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_8TRANSFERS

Transfer is done to 1 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_9TRANSFERS

Transfer is done to 9 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_10TRANSFERS

Transfer is done to 10 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_11TRANSFERS

Transfer is done to 11 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_12TRANSFERS

Transfer is done to 12 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_13TRANSFERS

Transfer is done to 13 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_14TRANSFERS

Transfer is done to 14 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_15TRANSFERS

Transfer is done to 15 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_16TRANSFERS

Transfer is done to 16 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_17TRANSFERS

Transfer is done to 17 registers starting from the DMA burst base address

LL_TIM_DMABURST_LENGTH_18TRANSFERS

Transfer is done to 18 registers starting from the DMA burst base address

Encoder Mode

LL_TIM_ENCODERMODE_X2_TI1

Quadrature encoder mode 1, x2 mode - Counter counts up/down on TI1FP1 edge depending on TI2FP2 level

LL_TIM_ENCODERMODE_X2_TI2

Quadrature encoder mode 2, x2 mode - Counter counts up/down on TI2FP2 edge depending on TI1FP1 level

LL_TIM_ENCODERMODE_X4_TI12

Quadrature encoder mode 3, x4 mode - Counter counts up/down on both TI1FP1 and TI2FP2 edges depending on the level of the other input

External Trigger Filter

LL_TIM_ETR_FILTER_FDIV1

No filter, sampling is done at fDTS

LL_TIM_ETR_FILTER_FDIV1_N2

fSAMPLING=fCK_INT, N=2

LL_TIM_ETR_FILTER_FDIV1_N4

fSAMPLING=fCK_INT, N=4

LL_TIM_ETR_FILTER_FDIV1_N8

fSAMPLING=fCK_INT, N=8

LL_TIM_ETR_FILTER_FDIV2_N6

fSAMPLING=fDTS/2, N=6

LL_TIM_ETR_FILTER_FDIV2_N8

fSAMPLING=fDTS/2, N=8

LL_TIM_ETR_FILTER_FDIV4_N6

fSAMPLING=fDTS/4, N=6

LL_TIM_ETR_FILTER_FDIV4_N8

fSAMPLING=fDTS/4, N=8

LL_TIM_ETR_FILTER_FDIV8_N6

fSAMPLING=fDTS/8, N=8

LL_TIM_ETR_FILTER_FDIV8_N8

fSAMPLING=fDTS/16, N=5

LL_TIM_ETR_FILTER_FDIV16_N5

fSAMPLING=fDTS/16, N=6

LL_TIM_ETR_FILTER_FDIV16_N6

fSAMPLING=fDTS/16, N=8

LL_TIM_ETR_FILTER_FDIV16_N8

fSAMPLING=fDTS/16, N=5

LL_TIM_ETR_FILTER_FDIV32_N5

fSAMPLING=fDTS/32, N=5

LL_TIM_ETR_FILTER_FDIV32_N6

fSAMPLING=fDTS/32, N=6

LL_TIM_ETR_FILTER_FDIV32_N8

fSAMPLING=fDTS/32, N=8

External Trigger Polarity

LL_TIM_ETR_POLARITY_NONINVERTED

ETR is non-inverted, active at high level or rising edge

LL_TIM_ETR_POLARITY_INVERTED

ETR is inverted, active at low level or falling edge

External Trigger Prescaler

LL_TIM_ETR_PRESCALER_DIV1

ETR prescaler OFF

LL_TIM_ETR_PRESCALER_DIV2

ETR frequency is divided by 2

LL_TIM_ETR_PRESCALER_DIV4

ETR frequency is divided by 4

LL_TIM_ETR_PRESCALER_DIV8

ETR frequency is divided by 8

Get Flags Defines

LL_TIM_SR_UIF

Update interrupt flag

LL_TIM_SR_CC1IF

Capture/compare 1 interrupt flag

LL_TIM_SR_CC2IF

Capture/compare 2 interrupt flag

LL_TIM_SR_CC3IF

Capture/compare 3 interrupt flag

LL_TIM_SR_CC4IF

Capture/compare 4 interrupt flag

LL_TIM_SR_COMIF

COM interrupt flag

LL_TIM_SR_TIF

Trigger interrupt flag

LL_TIM_SR_BIF

Break interrupt flag

LL_TIM_SR_CC1OF

Capture/Compare 1 overcapture flag

LL_TIM_SR_CC2OF

Capture/Compare 2 overcapture flag

LL_TIM_SR_CC3OF

Capture/Compare 3 overcapture flag

LL_TIM_SR_CC4OF

Capture/Compare 4 overcapture flag

Input Configuration Prescaler

LL_TIM_ICPSC_DIV1

No prescaler, capture is done each time an edge is detected on the capture input

LL_TIM_ICPSC_DIV2

Capture is done once every 2 events

LL_TIM_ICPSC_DIV4

Capture is done once every 4 events

LL_TIM_ICPSC_DIV8

Capture is done once every 8 events

Input Configuration Filter

LL_TIM_IC_FILTER_FDIV1

No filter, sampling is done at fDTS

LL_TIM_IC_FILTER_FDIV1_N2

fSAMPLING=fCK_INT, N=2

LL_TIM_IC_FILTER_FDIV1_N4

fSAMPLING=fCK_INT, N=4

LL_TIM_IC_FILTER_FDIV1_N8

fSAMPLING=fCK_INT, N=8

LL_TIM_IC_FILTER_FDIV2_N6

fSAMPLING=fDTS/2, N=6

LL_TIM_IC_FILTER_FDIV2_N8

fSAMPLING=fDTS/2, N=8

LL_TIM_IC_FILTER_FDIV4_N6

fSAMPLING=fDTS/4, N=6

LL_TIM_IC_FILTER_FDIV4_N8

fSAMPLING=fDTS/4, N=8

LL_TIM_IC_FILTER_FDIV8_N6

fSAMPLING=fDTS/8, N=6

LL_TIM_IC_FILTER_FDIV8_N8

fSAMPLING=fDTS/8, N=8

LL_TIM_IC_FILTER_FDIV16_N5

fSAMPLING=fDTS/16, N=5

LL_TIM_IC_FILTER_FDIV16_N6

fSAMPLING=fDTS/16, N=6

LL_TIM_IC_FILTER_FDIV16_N8

fSAMPLING=fDTS/16, N=8

LL_TIM_IC_FILTER_FDIV32_N5

fSAMPLING=fDTS/32, N=5

LL_TIM_IC_FILTER_FDIV32_N6

fSAMPLING=fDTS/32, N=6

LL_TIM_IC_FILTER_FDIV32_N8

fSAMPLING=fDTS/32, N=8

Input Configuration Polarity

LL_TIM_IC_POLARITY_RISING

The circuit is sensitive to TlxFP1 rising edge, TlxFP1 is not inverted

LL_TIM_IC_POLARITY_FALLING

The circuit is sensitive to TlxFP1 falling edge, TlxFP1 is inverted

LL_TIM_IC_POLARITY_BOTHEDGE

The circuit is sensitive to both TlxFP1 rising and falling edges, TlxFP1 is not inverted

IT Defines

LL_TIM_DIER_UIE

Update interrupt enable

LL_TIM_DIER_CC1IE

Capture/compare 1 interrupt enable

LL_TIM_DIER_CC2IE

Capture/compare 2 interrupt enable

LL_TIM_DIER_CC3IE

Capture/compare 3 interrupt enable

LL_TIM_DIER_CC4IE

Capture/compare 4 interrupt enable

LL_TIM_DIER_COMIE

COM interrupt enable

LL_TIM_DIER_TIE

Trigger interrupt enable

LL_TIM_DIER_BIE

Break interrupt enable

Lock Level

LL_TIM_LOCKLEVEL_OFF

LOCK OFF - No bit is write protected

LL_TIM_LOCKLEVEL_1

LOCK Level 1

LL_TIM_LOCKLEVEL_2

LOCK Level 2

LL_TIM_LOCKLEVEL_3

LOCK Level 3

Output Configuration Idle State

LL_TIM_OCIDLESTATE_LOW

OCx=0 (after a dead-time if OC is implemented) when MOE=0

LL_TIM_OCIDLESTATE_HIGH

OCx=1 (after a dead-time if OC is implemented) when MOE=0

Output Configuration Mode

LL_TIM_OCMODE_FROZEN

The comparison between the output compare register TIMx_CCRy and the counter TIMx_CNT has no effect on the output channel level

LL_TIM_OCMODE_ACTIVE

OCyREF is forced high on compare match

LL_TIM_OCMODE_INACTIVE

OCyREF is forced low on compare match

LL_TIM_OCMODE_TOGGLE

OCyREF toggles on compare match

LL_TIM_OCMODE_FORCED_INACTIVE

OCyREF is forced low

LL_TIM_OCMODE_FORCED_ACTIVE

OCyREF is forced high

LL_TIM_OCMODE_PWM1

In upcounting, channel y is active as long as $TIMx_CNT < TIMx_CCRx$ else inactive. In downcounting, channel y is inactive as long as $TIMx_CNT > TIMx_CCRx$ else active.

LL_TIM_OCMODE_PWM2

In upcounting, channel y is inactive as long as $TIMx_CNT < TIMx_CCRx$ else active. In downcounting, channel y is active as long as $TIMx_CNT > TIMx_CCRx$ else inactive

Output Configuration Polarity

LL_TIM_OCPOLARITY_HIGH

OCxactive high

LL_TIM_OCPOLARITY_LOW

OCxactive low

Output Configuration State

LL_TIM_OCSTATE_DISABLE

OCx is not active

LL_TIM_OCSTATE_ENABLE

OCx signal is output on the corresponding output pin

One Pulse Mode

LL_TIM_ONEPULSEMODE_SINGLE

Counter is not stopped at update event

LL_TIM_ONEPULSEMODE_REPETITIVE

Counter stops counting at the next update event

OSSI

LL_TIM_OSSI_DISABLE

When inactive, OCx/OCxN outputs are disabled

LL_TIM_OSSI_ENABLE

When inactive, OCx/OCxN outputs are first forced with their inactive level then forced to their idle level after the deadline

OSSR

LL_TIM_OSSR_DISABLE

When inactive, OCx/OCxN outputs are disabled

LL_TIM_OSSR_ENABLE

When inactive, OC/OCN outputs are enabled with their inactive level as soon as $CCxE=1$ or $CCxNE=1$

Slave Mode

LL_TIM_SLAVEMODE_DISABLED

Slave mode disabled

LL_TIM_SLAVEMODE_RESET

Reset Mode - Rising edge of the selected trigger input (TRGI) reinitializes the counter

LL_TIM_SLAVEMODE_GATED

Gated Mode - The counter clock is enabled when the trigger input (TRGI) is high

LL_TIM_SLAVEMODE_TRIGGER

Trigger Mode - The counter starts at a rising edge of the trigger TRGI

TIM11 External Input Capture 1 Remap

LL_TIM_TIM11_TI1_RMP_GPIO

TIM11 channel 1 is connected to GPIO

LL_TIM_TIM11_TI1_RMP_GPIO1

TIM11 channel 1 is connected to GPIO

LL_TIM_TIM11_TI1_RMP_GPIO2

TIM11 channel 1 is connected to GPIO

LL_TIM_TIM11_TI1_RMP_HSE_RTC

TIM11 channel 1 is connected to HSE_RTC

TIM2 Internal Trigger1 Remap TIM8

LL_TIM_TIM2_ITR1_RMP_TIM8_TRGO

TIM2_ITR1 is connected to TIM8_TRGO

LL_TIM_TIM2_ITR1_RMP_ETH_PTP

TIM2_ITR1 is connected to ETH_PTP

LL_TIM_TIM2_ITR1_RMP_OTG_FS_SOF

TIM2_ITR1 is connected to OTG_FS SOF

LL_TIM_TIM2_ITR1_RMP_OTG_HS_SOF

TIM2_ITR1 is connected to OTG_HS SOF

TIM5 External Input Ch4 Remap

LL_TIM_TIM5_TI4_RMP_GPIO

TIM5 channel 4 is connected to GPIO

LL_TIM_TIM5_TI4_RMP_LSI

TIM5 channel 4 is connected to LSI internal clock

LL_TIM_TIM5_TI4_RMP_LSE

TIM5 channel 4 is connected to LSE

LL_TIM_TIM5_TI4_RMP_RTC

TIM5 channel 4 is connected to RTC wakeup interrupt

Trigger Output

LL_TIM_TRGO_RESET

UG bit from the TIMx_EGR register is used as trigger output

LL_TIM_TRGO_ENABLE

Counter Enable signal (CNT_EN) is used as trigger output

LL_TIM_TRGO_UPDATE

Update event is used as trigger output

LL_TIM_TRGO_CC1IF

CC1 capture or a compare match is used as trigger output

LL_TIM_TRGO_OC1REF

OC1REF signal is used as trigger output

LL_TIM_TRGO_OC2REF

OC2REF signal is used as trigger output

LL_TIM_TRGO_OC3REF

OC3REF signal is used as trigger output

LL_TIM_TRGO_OC4REF

OC4REF signal is used as trigger output

Trigger Selection**LL_TIM_TS_ITR0**

Internal Trigger 0 (ITR0) is used as trigger input

LL_TIM_TS_ITR1

Internal Trigger 1 (ITR1) is used as trigger input

LL_TIM_TS_ITR2

Internal Trigger 2 (ITR2) is used as trigger input

LL_TIM_TS_ITR3

Internal Trigger 3 (ITR3) is used as trigger input

LL_TIM_TS_TI1F_ED

TI1 Edge Detector (TI1F_ED) is used as trigger input

LL_TIM_TS_TI1FP1

Filtered Timer Input 1 (TI1FP1) is used as trigger input

LL_TIM_TS_TI2FP2

Filtered Timer Input 2 (TI2FP2) is used as trigger input

LL_TIM_TS_ETRF

Filtered external Trigger (ETRF) is used as trigger input

Update Source**LL_TIM_UPDATESOURCE_REGULAR**

Counter overflow/underflow, Setting the UG bit or Update generation through the slave mode controller generates an update request

LL_TIM_UPDATESOURCE_COUNTER

Only counter overflow/underflow generates an update request

Exported_Macros

__LL_TIM_CALC_DEADTIME

Description:

- HELPER macro calculating DTG[0:7] in the TIMx_BDTR register to achieve the requested dead time duration.

Parameters:

- __TIMCLK__: timer input clock frequency (in Hz)
- __CKD__: This parameter can be one of the following values:
 - LL_TIM_CLOCKDIVISION_DIV1
 - LL_TIM_CLOCKDIVISION_DIV2
 - LL_TIM_CLOCKDIVISION_DIV4
- __DT__: deadtime duration (in ns)

Return value:

- DTG[0:7]

Notes:

- ex: __LL_TIM_CALC_DEADTIME (80000000, LL_TIM_GetClockDivision (), 120);

__LL_TIM_CALC_PSC

Description:

- HELPER macro calculating the prescaler value to achieve the required counter clock frequency.

Parameters:

- __TIMCLK__: timer input clock frequency (in Hz)
- __CNTCLK__: counter clock frequency (in Hz)

Return value:

- Prescaler: value (between Min_Data=0 and Max_Data=65535)

Notes:

- ex: __LL_TIM_CALC_PSC (80000000, 1000000);

__LL_TIM_CALC_ARR

Description:

- HELPER macro calculating the auto-reload value to achieve the required output signal frequency.

Parameters:

- __TIMCLK__: timer input clock frequency (in Hz)
- __PSC__: prescaler
- __FREQ__: output signal frequency (in Hz)

Return value:

- Auto-reload: value (between Min_Data=0 and Max_Data=65535)

Notes:

- ex: __LL_TIM_CALC_ARR (1000000, LL_TIM_GetPrescaler (), 10000);

__LL_TIM_CALC_DELAY

Description:

- HELPER macro calculating the compare value required to achieve the required timer output compare active/inactive delay.

Parameters:

- __TIMCLK__: timer input clock frequency (in Hz)
- __PSC__: prescaler
- __DELAY__: timer output compare active/inactive delay (in us)

Return value:

- Compare: value (between Min_Data=0 and Max_Data=65535)

Notes:

- ex: __LL_TIM_CALC_DELAY (1000000, LL_TIM_GetPrescaler (), 10);

__LL_TIM_CALC_PULSE

Description:

- HELPER macro calculating the auto-reload value to achieve the required pulse duration (when the timer operates in one pulse mode).

Parameters:

- __TIMCLK__: timer input clock frequency (in Hz)
- __PSC__: prescaler
- __DELAY__: timer output compare active/inactive delay (in us)
- __PULSE__: pulse duration (in us)

Return value:

- Auto-reload: value (between Min_Data=0 and Max_Data=65535)

Notes:

- ex: __LL_TIM_CALC_PULSE (1000000, LL_TIM_GetPrescaler (), 10, 20);

__LL_TIM_GET_ICPSC_RATIO

Description:

- HELPER macro retrieving the ratio of the input capture prescaler.

Parameters:

- __ICPSC__: This parameter can be one of the following values:
 - LL_TIM_ICPSC_DIV1
 - LL_TIM_ICPSC_DIV2
 - LL_TIM_ICPSC_DIV4
 - LL_TIM_ICPSC_DIV8

Return value:

- Input: capture prescaler ratio (1, 2, 4 or 8)

Notes:

- ex: __LL_TIM_GET_ICPSC_RATIO (LL_TIM_IC_GetPrescaler ());

Common Write and read registers Macros

LL_TIM_WriteReg

Description:

- Write a value in TIM register.

Parameters:

- `__INSTANCE__`: TIM Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_TIM_ReadReg

Description:

- Read a value in TIM register.

Parameters:

- `__INSTANCE__`: TIM Instance
- `__REG__`: Register to be read

Return value:

- Register: value

66 LL USART Generic Driver

66.1 USART Firmware driver registers structures

66.1.1 LL_USART_InitTypeDef

LL_USART_InitTypeDef is defined in the `stm32f2xx_ll_usart.h`

Data Fields

- `uint32_t BaudRate`
- `uint32_t DataWidth`
- `uint32_t StopBits`
- `uint32_t Parity`
- `uint32_t TransferDirection`
- `uint32_t HardwareFlowControl`
- `uint32_t OverSampling`

Field Documentation

- `uint32_t LL_USART_InitTypeDef::BaudRate`
This field defines expected Usart communication baud rate. This feature can be modified afterwards using unitary function `LL_USART_SetBaudRate()`.
- `uint32_t LL_USART_InitTypeDef::DataWidth`
Specifies the number of data bits transmitted or received in a frame. This parameter can be a value of `USART_LL_EC_DATAWIDTH`. This feature can be modified afterwards using unitary function `LL_USART_SetDataWidth()`.
- `uint32_t LL_USART_InitTypeDef::StopBits`
Specifies the number of stop bits transmitted. This parameter can be a value of `USART_LL_EC_STOPBITS`. This feature can be modified afterwards using unitary function `LL_USART_SetStopBitsLength()`.
- `uint32_t LL_USART_InitTypeDef::Parity`
Specifies the parity mode. This parameter can be a value of `USART_LL_EC_PARITY`. This feature can be modified afterwards using unitary function `LL_USART_SetParity()`.
- `uint32_t LL_USART_InitTypeDef::TransferDirection`
Specifies whether the Receive and/or Transmit mode is enabled or disabled. This parameter can be a value of `USART_LL_EC_DIRECTION`. This feature can be modified afterwards using unitary function `LL_USART_SetTransferDirection()`.
- `uint32_t LL_USART_InitTypeDef::HardwareFlowControl`
Specifies whether the hardware flow control mode is enabled or disabled. This parameter can be a value of `USART_LL_EC_HWCONTROL`. This feature can be modified afterwards using unitary function `LL_USART_SetHWFlowCtrl()`.
- `uint32_t LL_USART_InitTypeDef::OverSampling`
Specifies whether USART oversampling mode is 16 or 8. This parameter can be a value of `USART_LL_EC_OVERSAMPLING`. This feature can be modified afterwards using unitary function `LL_USART_SetOverSampling()`.

66.1.2 LL_USART_ClockInitTypeDef

LL_USART_ClockInitTypeDef is defined in the `stm32f2xx_ll_usart.h`

Data Fields

- `uint32_t ClockOutput`
- `uint32_t ClockPolarity`
- `uint32_t ClockPhase`
- `uint32_t LastBitClockPulse`

Field Documentation

- **`uint32_t LL_USART_ClockInitTypeDef::ClockOutput`**
Specifies whether the USART clock is enabled or disabled. This parameter can be a value of [USART_LL_EC_CLOCK](#). USART HW configuration can be modified afterwards using unitary functions `LL_USART_EnableSCLKOutput()` or `LL_USART_DisableSCLKOutput()`. For more details, refer to description of this function.
- **`uint32_t LL_USART_ClockInitTypeDef::ClockPolarity`**
Specifies the steady state of the serial clock. This parameter can be a value of [USART_LL_EC_POLARITY](#). USART HW configuration can be modified afterwards using unitary functions `LL_USART_SetClockPolarity()`. For more details, refer to description of this function.
- **`uint32_t LL_USART_ClockInitTypeDef::ClockPhase`**
Specifies the clock transition on which the bit capture is made. This parameter can be a value of [USART_LL_EC_PHASE](#). USART HW configuration can be modified afterwards using unitary functions `LL_USART_SetClockPhase()`. For more details, refer to description of this function.
- **`uint32_t LL_USART_ClockInitTypeDef::LastBitClockPulse`**
Specifies whether the clock pulse corresponding to the last transmitted data bit (MSB) has to be output on the SCLK pin in synchronous mode. This parameter can be a value of [USART_LL_EC_LASTCLKPULSE](#). USART HW configuration can be modified afterwards using unitary functions `LL_USART_SetLastClkPulseOutput()`. For more details, refer to description of this function.

66.2 USART Firmware driver API description

The following section lists the various functions of the USART library.

66.2.1 Detailed description of functions

`LL_USART_Enable`

Function name

`__STATIC_INLINE void LL_USART_Enable (USART_TypeDef * USARTx)`

Function description

USART Enable.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 UE `LL_USART_Enable`

`LL_USART_Disable`

Function name

`__STATIC_INLINE void LL_USART_Disable (USART_TypeDef * USARTx)`

Function description

USART Disable (all USART prescalers and outputs are disabled)

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- When USART is disabled, USART prescalers and outputs are stopped immediately, and current operations are discarded. The configuration of the USART is kept, but all the status flags, in the USARTx_SR are set to their default values.

Reference Manual to LL API cross reference:

- CR1 UE LL_USART_Disable
- LL_USART_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabled (USART_TypeDef * USARTx)

Function description

Indicate if USART is enabled.

Parameters

- USARTx:** USART Instance

Return values

- State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 UE LL_USART_IsEnabled
- LL_USART_EnableDirectionRx

Function name

__STATIC_INLINE void LL_USART_EnableDirectionRx (USART_TypeDef * USARTx)

Function description

Receiver Enable (Receiver is enabled and begins searching for a start bit)

Parameters

- USARTx:** USART Instance

Return values

- None:**

Reference Manual to LL API cross reference:

- CR1 RE LL_USART_EnableDirectionRx
- LL_USART_DisableDirectionRx

Function name

__STATIC_INLINE void LL_USART_DisableDirectionRx (USART_TypeDef * USARTx)

Function description

Receiver Disable.

Parameters

- USARTx:** USART Instance

Return values

- None:**

Reference Manual to LL API cross reference:

- CR1 RE LL_USART_DisableDirectionRx

```
LL_USART_EnableDirectionTx
```

Function name

```
__STATIC_INLINE void LL_USART_EnableDirectionTx (USART_TypeDef * USARTx)
```

Function description

Transmitter Enable.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TE LL_USART_EnableDirectionTx

```
LL_USART_DisableDirectionTx
```

Function name

```
__STATIC_INLINE void LL_USART_DisableDirectionTx (USART_TypeDef * USARTx)
```

Function description

Transmitter Disable.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TE LL_USART_DisableDirectionTx

```
LL_USART_SetTransferDirection
```

Function name

```
__STATIC_INLINE void LL_USART_SetTransferDirection (USART_TypeDef * USARTx, uint32_t TransferDirection)
```

Function description

Configure simultaneously enabled/disabled states of Transmitter and Receiver.

Parameters

- **USARTx:** USART Instance
- **TransferDirection:** This parameter can be one of the following values:
 - LL_USART_DIRECTION_NONE
 - LL_USART_DIRECTION_RX
 - LL_USART_DIRECTION_TX
 - LL_USART_DIRECTION_TX_RX

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RE LL_USART_SetTransferDirection
- CR1 TE LL_USART_SetTransferDirection

```
LL_USART_GetTransferDirection
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetTransferDirection (USART_TypeDef * USARTx)
```

Function description

Return enabled/disabled states of Transmitter and Receiver.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_DIRECTION_NONE
 - LL_USART_DIRECTION_RX
 - LL_USART_DIRECTION_TX
 - LL_USART_DIRECTION_TX_RX

Reference Manual to LL API cross reference:

- CR1 RE LL_USART_GetTransferDirection
- CR1 TE LL_USART_GetTransferDirection

```
LL_USART_SetParity
```

Function name

```
__STATIC_INLINE void LL_USART_SetParity (USART_TypeDef * USARTx, uint32_t Parity)
```

Function description

Configure Parity (enabled/disabled and parity mode if enabled).

Parameters

- **USARTx:** USART Instance
- **Parity:** This parameter can be one of the following values:
 - LL_USART_PARITY_NONE
 - LL_USART_PARITY_EVEN
 - LL_USART_PARITY_ODD

Return values

- **None:**

Notes

- This function selects if hardware parity control (generation and detection) is enabled or disabled. When the parity control is enabled (Odd or Even), computed parity bit is inserted at the MSB position (9th or 8th bit depending on data width) and parity is checked on the received data.

Reference Manual to LL API cross reference:

- CR1 PS LL_USART_SetParity
- CR1 PCE LL_USART_SetParity

```
LL_USART_GetParity
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetParity (USART_TypeDef * USARTx)
```

Function description

Return Parity configuration (enabled/disabled and parity mode if enabled)

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_PARITY_NONE
 - LL_USART_PARITY_EVEN
 - LL_USART_PARITY_ODD

Reference Manual to LL API cross reference:

- CR1 PS LL_USART_GetParity
- CR1 PCE LL_USART_GetParity

LL_USART_SetWakeUpMethod

Function name

__STATIC_INLINE void LL_USART_SetWakeUpMethod (USART_TypeDef * USARTx, uint32_t Method)

Function description

Set Receiver Wake Up method from Mute mode.

Parameters

- **USARTx:** USART Instance
- **Method:** This parameter can be one of the following values:
 - LL_USART_WAKEUP_IDLELINE
 - LL_USART_WAKEUP_ADDRESSMARK

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 WAKE LL_USART_SetWakeUpMethod

LL_USART_GetWakeUpMethod

Function name

__STATIC_INLINE uint32_t LL_USART_GetWakeUpMethod (USART_TypeDef * USARTx)

Function description

Return Receiver Wake Up method from Mute mode.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_WAKEUP_IDLELINE
 - LL_USART_WAKEUP_ADDRESSMARK

Reference Manual to LL API cross reference:

- CR1 WAKE LL_USART_GetWakeUpMethod

LL_USART_SetDataWidth

Function name

__STATIC_INLINE void LL_USART_SetDataWidth (USART_TypeDef * USARTx, uint32_t DataWidth)

Function description

Set Word length (i.e.

Parameters

- **USARTx:** USART Instance
- **DataWidth:** This parameter can be one of the following values:
 - LL_USART_DATAWIDTH_8B
 - LL_USART_DATAWIDTH_9B

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 M LL_USART_SetDataWidth

LL_USART_GetDataWidth

Function name

__STATIC_INLINE uint32_t LL_USART_GetDataWidth (USART_TypeDef * USARTx)

Function description

Return Word length (i.e.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_DATAWIDTH_8B
 - LL_USART_DATAWIDTH_9B

Reference Manual to LL API cross reference:

- CR1 M LL_USART_GetDataWidth

LL_USART_SetOverSampling

Function name

__STATIC_INLINE void LL_USART_SetOverSampling (USART_TypeDef * USARTx, uint32_t OverSampling)

Function description

Set Oversampling to 8-bit or 16-bit mode.

Parameters

- **USARTx:** USART Instance
- **OverSampling:** This parameter can be one of the following values:
 - LL_USART_OVERSAMPLING_16
 - LL_USART_OVERSAMPLING_8

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OVER8 LL_USART_SetOverSampling

LL_USART_GetOverSampling

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetOverSampling (USART_TypeDef * USARTx)
```

Function description

Return Oversampling mode.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_OVERSAMPLING_16
 - LL_USART_OVERSAMPLING_8

Reference Manual to LL API cross reference:

- CR1 OVER8 LL_USART_GetOverSampling

LL_USART_SetLastClkPulseOutput

Function name

```
__STATIC_INLINE void LL_USART_SetLastClkPulseOutput (USART_TypeDef * USARTx, uint32_t LastBitClockPulse)
```

Function description

Configure if Clock pulse of the last data bit is output to the SCLK pin or not.

Parameters

- **USARTx:** USART Instance
- **LastBitClockPulse:** This parameter can be one of the following values:
 - LL_USART_LASTCLKPULSE_NO_OUTPUT
 - LL_USART_LASTCLKPULSE_OUTPUT

Return values

- **None:**

Notes

- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBCL LL_USART_SetLastClkPulseOutput

LL_USART_GetLastClkPulseOutput

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetLastClkPulseOutput (USART_TypeDef * USARTx)
```

Function description

Retrieve Clock pulse of the last data bit output configuration (Last bit Clock pulse output to the SCLK pin or not)

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_LASTCLKPULSE_NO_OUTPUT
 - LL_USART_LASTCLKPULSE_OUTPUT

Notes

- Macro `IS_USART_INSTANCE(USARTx)` can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBCL `LL_USART_GetLastClkPulseOutput`
`LL_USART_SetClockPhase`

Function name

__STATIC_INLINE void LL_USART_SetClockPhase (USART_TypeDef * USARTx, uint32_t ClockPhase)

Function description

Select the phase of the clock output on the SCLK pin in synchronous mode.

Parameters

- USARTx:** USART Instance
- ClockPhase:** This parameter can be one of the following values:
 - `LL_USART_PHASE_1EDGE`
 - `LL_USART_PHASE_2EDGE`

Return values

- None:**

Notes

- Macro `IS_USART_INSTANCE(USARTx)` can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CPHA `LL_USART_SetClockPhase`
`LL_USART_GetClockPhase`

Function name

__STATIC_INLINE uint32_t LL_USART_GetClockPhase (USART_TypeDef * USARTx)

Function description

Return phase of the clock output on the SCLK pin in synchronous mode.

Parameters

- USARTx:** USART Instance

Return values

- Returned:** value can be one of the following values:
 - `LL_USART_PHASE_1EDGE`
 - `LL_USART_PHASE_2EDGE`

Notes

- Macro `IS_USART_INSTANCE(USARTx)` can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CPHA `LL_USART_GetClockPhase`
`LL_USART_SetClockPolarity`

Function name

__STATIC_INLINE void LL_USART_SetClockPolarity (USART_TypeDef * USARTx, uint32_t ClockPolarity)

Function description

Select the polarity of the clock output on the SCLK pin in synchronous mode.

Parameters

- **USARTx:** USART Instance
- **ClockPolarity:** This parameter can be one of the following values:
 - LL_USART_POLARITY_LOW
 - LL_USART_POLARITY_HIGH

Return values

- **None:**

Notes

- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CPOL LL_USART_SetClockPolarity
- LL_USART_GetClockPolarity

Function name

__STATIC_INLINE uint32_t LL_USART_GetClockPolarity (USART_TypeDef * USARTx)

Function description

Return polarity of the clock output on the SCLK pin in synchronous mode.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_POLARITY_LOW
 - LL_USART_POLARITY_HIGH

Notes

- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CPOL LL_USART_GetClockPolarity
- LL_USART_ConfigClock

Function name

__STATIC_INLINE void LL_USART_ConfigClock (USART_TypeDef * USARTx, uint32_t Phase, uint32_t Polarity, uint32_t LBCPOutput)

Function description

Configure Clock signal format (Phase Polarity and choice about output of last bit clock pulse)

Parameters

- **USARTx:** USART Instance
- **Phase:** This parameter can be one of the following values:
 - LL_USART_PHASE_1EDGE
 - LL_USART_PHASE_2EDGE
- **Polarity:** This parameter can be one of the following values:
 - LL_USART_POLARITY_LOW
 - LL_USART_POLARITY_HIGH
- **LBCPOutput:** This parameter can be one of the following values:
 - LL_USART_LASTCLKPULSE_NO_OUTPUT
 - LL_USART_LASTCLKPULSE_OUTPUT

Return values

- **None:**

Notes

- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clock Phase configuration using LL_USART_SetClockPhase() functionClock Polarity configuration using LL_USART_SetClockPolarity() functionOutput of Last bit Clock pulse configuration using LL_USART_SetLastClkPulseOutput() function

Reference Manual to LL API cross reference:

- CR2 CPHA LL_USART_ConfigClock
- CR2 CPOL LL_USART_ConfigClock
- CR2 LBCL LL_USART_ConfigClock

LL_USART_EnableSCLKOutput

Function name

__STATIC_INLINE void LL_USART_EnableSCLKOutput (USART_TypeDef * USARTx)

Function description

Enable Clock output on SCLK pin.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CLKEN LL_USART_EnableSCLKOutput

LL_USART_DisableSCLKOutput

Function name

__STATIC_INLINE void LL_USART_DisableSCLKOutput (USART_TypeDef * USARTx)

Function description

Disable Clock output on SCLK pin.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_USART_INSTANCE(USARTx)` can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CLKEN `LL_USART_DisableSCLKOutput`

`LL_USART_IsEnabledSCLKOutput`

Function name

`__STATIC_INLINE uint32_t LL_USART_IsEnabledSCLKOutput (USART_TypeDef * USARTx)`

Function description

Indicate if Clock output on SCLK pin is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_USART_INSTANCE(USARTx)` can be used to check whether or not Synchronous mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 CLKEN `LL_USART_IsEnabledSCLKOutput`

`LL_USART_SetStopBitsLength`

Function name

`__STATIC_INLINE void LL_USART_SetStopBitsLength (USART_TypeDef * USARTx, uint32_t StopBits)`

Function description

Set the length of the stop bits.

Parameters

- **USARTx:** USART Instance
- **StopBits:** This parameter can be one of the following values:
 - `LL_USART_STOPBITS_0_5`
 - `LL_USART_STOPBITS_1`
 - `LL_USART_STOPBITS_1_5`
 - `LL_USART_STOPBITS_2`

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 STOP `LL_USART_SetStopBitsLength`

`LL_USART_GetStopBitsLength`

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetStopBitsLength (USART_TypeDef * USARTx)
```

Function description

Retrieve the length of the stop bits.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_STOPBITS_0_5
 - LL_USART_STOPBITS_1
 - LL_USART_STOPBITS_1_5
 - LL_USART_STOPBITS_2

Reference Manual to LL API cross reference:

- CR2 STOP LL_USART_GetStopBitsLength

LL_USART_ConfigCharacter

Function name

```
__STATIC_INLINE void LL_USART_ConfigCharacter (USART_TypeDef * USARTx, uint32_t DataWidth,
uint32_t Parity, uint32_t StopBits)
```

Function description

Configure Character frame format (Datawidth, Parity control, Stop Bits)

Parameters

- **USARTx:** USART Instance
- **DataWidth:** This parameter can be one of the following values:
 - LL_USART_DATAWIDTH_8B
 - LL_USART_DATAWIDTH_9B
- **Parity:** This parameter can be one of the following values:
 - LL_USART_PARITY_NONE
 - LL_USART_PARITY_EVEN
 - LL_USART_PARITY_ODD
- **StopBits:** This parameter can be one of the following values:
 - LL_USART_STOPBITS_0_5
 - LL_USART_STOPBITS_1
 - LL_USART_STOPBITS_1_5
 - LL_USART_STOPBITS_2

Return values

- **None:**

Notes

- Call of this function is equivalent to following function call sequence : Data Width configuration using LL_USART_SetDataWidth() functionParity Control and mode configuration using LL_USART_SetParity() functionStop bits configuration using LL_USART_SetStopBitsLength() function

Reference Manual to LL API cross reference:

- CR1 PS LL_USART_ConfigCharacter
- CR1 PCE LL_USART_ConfigCharacter
- CR1 M LL_USART_ConfigCharacter
- CR2 STOP LL_USART_ConfigCharacter

LL_USART_SetNodeAddress

Function name

__STATIC_INLINE void LL_USART_SetNodeAddress (USART_TypeDef * USARTx, uint32_t NodeAddress)

Function description

Set Address of the USART node.

Parameters

- **USARTx:** USART Instance
- **NodeAddress:** 4 bit Address of the USART node.

Return values

- **None:**

Notes

- This is used in multiprocessor communication during Mute mode or Stop mode, for wake up with address mark detection.

Reference Manual to LL API cross reference:

- CR2 ADD LL_USART_SetNodeAddress

LL_USART_GetNodeAddress

Function name

__STATIC_INLINE uint32_t LL_USART_GetNodeAddress (USART_TypeDef * USARTx)

Function description

Return 4 bit Address of the USART node as set in ADD field of CR2.

Parameters

- **USARTx:** USART Instance

Return values

- **Address:** of the USART node (Value between Min_Data=0 and Max_Data=255)

Notes

- only 4bits (b3-b0) of returned value are relevant (b31-b4 are not relevant)

Reference Manual to LL API cross reference:

- CR2 ADD LL_USART_GetNodeAddress

LL_USART_EnableRTSHWFlowCtrl

Function name

__STATIC_INLINE void LL_USART_EnableRTSHWFlowCtrl (USART_TypeDef * USARTx)

Function description

Enable RTS HW Flow Control.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 RTSE LL_USART_EnableRTSHWFlowCtrl
LL_USART_DisableRTSHWFlowCtrl

Function name

__STATIC_INLINE void LL_USART_DisableRTSHWFlowCtrl (USART_TypeDef * USARTx)

Function description

Disable RTS HW Flow Control.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 RTSE LL_USART_DisableRTSHWFlowCtrl
LL_USART_EnableCTSHWFlowCtrl

Function name

__STATIC_INLINE void LL_USART_EnableCTSHWFlowCtrl (USART_TypeDef * USARTx)

Function description

Enable CTS HW Flow Control.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 CTSE LL_USART_EnableCTSHWFlowCtrl
LL_USART_DisableCTSHWFlowCtrl

Function name

__STATIC_INLINE void LL_USART_DisableCTSHWFlowCtrl (USART_TypeDef * USARTx)

Function description

Disable CTS HW Flow Control.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 CTSE `LL_USART_DisableCTSHWFlowCtrl`

`LL_USART_SetHWFlowCtrl`

Function name

`__STATIC_INLINE void LL_USART_SetHWFlowCtrl (USART_TypeDef * USARTx, uint32_t HardwareFlowControl)`

Function description

Configure HW Flow Control mode (both CTS and RTS)

Parameters

- **USARTx:** USART Instance
- **HardwareFlowControl:** This parameter can be one of the following values:
 - `LL_USART_HWCONTROL_NONE`
 - `LL_USART_HWCONTROL_RTS`
 - `LL_USART_HWCONTROL_CTS`
 - `LL_USART_HWCONTROL_RTS_CTS`

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 RTSE `LL_USART_SetHWFlowCtrl`
- CR3 CTSE `LL_USART_SetHWFlowCtrl`

`LL_USART_GetHWFlowCtrl`

Function name

`__STATIC_INLINE uint32_t LL_USART_GetHWFlowCtrl (USART_TypeDef * USARTx)`

Function description

Return HW Flow Control configuration (both CTS and RTS)

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_HWCONTROL_NONE
 - LL_USART_HWCONTROL_RTS
 - LL_USART_HWCONTROL_CTS
 - LL_USART_HWCONTROL_RTS_CTS

Notes

- Macro IS_UART_HWFLOW_INSTANCE(USARTx) can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 RTSE LL_USART_GetHWFlowCtrl
- CR3 CTSE LL_USART_GetHWFlowCtrl

LL_USART_EnableOneBitSamp

Function name

__STATIC_INLINE void LL_USART_EnableOneBitSamp (USART_TypeDef * USARTx)

Function description

Enable One bit sampling method.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 ONEBIT LL_USART_EnableOneBitSamp

LL_USART_DisableOneBitSamp

Function name

__STATIC_INLINE void LL_USART_DisableOneBitSamp (USART_TypeDef * USARTx)

Function description

Disable One bit sampling method.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 ONEBIT LL_USART_DisableOneBitSamp

LL_USART_IsEnabledOneBitSamp

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledOneBitSamp (USART_TypeDef * USARTx)

Function description

Indicate if One bit sampling method is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR3 ONEBIT LL_USART_IsEnabledOneBitSamp

LL_USART_SetBaudRate

Function name

__STATIC_INLINE void LL_USART_SetBaudRate (USART_TypeDef * USARTx, uint32_t PeriphClk, uint32_t OverSampling, uint32_t BaudRate)

Function description

Configure USART BRR register for achieving expected Baud Rate value.

Parameters

- **USARTx:** USART Instance
- **PeriphClk:** Peripheral Clock
- **OverSampling:** This parameter can be one of the following values:
 - LL_USART_OVERSAMPLING_16
 - LL_USART_OVERSAMPLING_8
- **BaudRate:** Baud Rate

Return values

- **None:**

Notes

- Compute and set USARTDIV value in BRR Register (full BRR content) according to used Peripheral Clock, Oversampling mode, and expected Baud Rate values
- Peripheral clock and Baud rate values provided as function parameters should be valid (Baud rate value != 0)

Reference Manual to LL API cross reference:

- BRR BRR LL_USART_SetBaudRate

LL_USART_GetBaudRate

Function name

__STATIC_INLINE uint32_t LL_USART_GetBaudRate (USART_TypeDef * USARTx, uint32_t PeriphClk, uint32_t OverSampling)

Function description

Return current Baud Rate value, according to USARTDIV present in BRR register (full BRR content), and to used Peripheral Clock and Oversampling mode values.

Parameters

- **USARTx:** USART Instance
- **PeriphClk:** Peripheral Clock
- **OverSampling:** This parameter can be one of the following values:
 - LL_USART_OVERSAMPLING_16
 - LL_USART_OVERSAMPLING_8

Return values

- **Baud:** Rate

Notes

- In case of non-initialized or invalid value stored in BRR register, value 0 will be returned.

Reference Manual to LL API cross reference:

- BRR BRR LL_USART_GetBaudRate
LL_USART_EnableIrda

Function name

```
__STATIC_INLINE void LL_USART_EnableIrda (USART_TypeDef * USARTx)
```

Function description

Enable IrDA mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_IRDA_INSTANCE(USARTx) can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 IREN LL_USART_EnableIrda
LL_USART_DisableIrda

Function name

```
__STATIC_INLINE void LL_USART_DisableIrda (USART_TypeDef * USARTx)
```

Function description

Disable IrDA mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_IRDA_INSTANCE(USARTx) can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 IREN LL_USART_DisableIrda
LL_USART_IsEnabledIrda

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledIrda (USART_TypeDef * USARTx)
```

Function description

Indicate if IrDA mode is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_IRDA_INSTANCE(USARTx)` can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 IREN `LL_USART_IsEnabledIrda`

`LL_USART_SetIrdaPowerMode`

Function name

__STATIC_INLINE void LL_USART_SetIrdaPowerMode (USART_TypeDef * USARTx, uint32_t PowerMode)

Function description

Configure IrDA Power Mode (Normal or Low Power)

Parameters

- **USARTx:** USART Instance
- **PowerMode:** This parameter can be one of the following values:
 - `LL_USART_IRDA_POWER_NORMAL`
 - `LL_USART_IRDA_POWER_LOW`

Return values

- **None:**

Notes

- Macro `IS_IRDA_INSTANCE(USARTx)` can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 IRLP `LL_USART_SetIrdaPowerMode`

`LL_USART_GetIrdaPowerMode`

Function name

__STATIC_INLINE uint32_t LL_USART_GetIrdaPowerMode (USART_TypeDef * USARTx)

Function description

Retrieve IrDA Power Mode configuration (Normal or Low Power)

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - `LL_USART_IRDA_POWER_NORMAL`
 - `LL_USART_PHASE_2EDGE`

Notes

- Macro `IS_IRDA_INSTANCE(USARTx)` can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 IRLP LL_USART_GetIrdaPowerMode
- LL_USART_SetIrdaPrescaler

Function name

__STATIC_INLINE void LL_USART_SetIrdaPrescaler (USART_TypeDef * USARTx, uint32_t PrescalerValue)

Function description

Set Irda prescaler value, used for dividing the USART clock source to achieve the Irda Low Power frequency (8 bits value)

Parameters

- **USARTx:** USART Instance
- **PrescalerValue:** Value between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Notes

- Macro IS_IRDA_INSTANCE(USARTx) can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR PSC LL_USART_SetIrdaPrescaler
- LL_USART_GetIrdaPrescaler

Function name

__STATIC_INLINE uint32_t LL_USART_GetIrdaPrescaler (USART_TypeDef * USARTx)

Function description

Return Irda prescaler value, used for dividing the USART clock source to achieve the Irda Low Power frequency (8 bits value)

Parameters

- **USARTx:** USART Instance

Return values

- **Irda:** prescaler value (Value between Min_Data=0x00 and Max_Data=0xFF)

Notes

- Macro IS_IRDA_INSTANCE(USARTx) can be used to check whether or not IrDA feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR PSC LL_USART_GetIrdaPrescaler
- LL_USART_EnableSmartcardNACK

Function name

__STATIC_INLINE void LL_USART_EnableSmartcardNACK (USART_TypeDef * USARTx)

Function description

Enable Smartcard NACK transmission.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_SMARTCARD_INSTANCE(USARTx)` can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 NACK `LL_USART_EnableSmartcardNACK`

`LL_USART_DisableSmartcardNACK`

Function name

`__STATIC_INLINE void LL_USART_DisableSmartcardNACK (USART_TypeDef * USARTx)`

Function description

Disable Smartcard NACK transmission.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_SMARTCARD_INSTANCE(USARTx)` can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 NACK `LL_USART_DisableSmartcardNACK`

`LL_USART_IsEnabledSmartcardNACK`

Function name

`__STATIC_INLINE uint32_t LL_USART_IsEnabledSmartcardNACK (USART_TypeDef * USARTx)`

Function description

Indicate if Smartcard NACK transmission is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_SMARTCARD_INSTANCE(USARTx)` can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 NACK `LL_USART_IsEnabledSmartcardNACK`

`LL_USART_EnableSmartcard`

Function name

`__STATIC_INLINE void LL_USART_EnableSmartcard (USART_TypeDef * USARTx)`

Function description

Enable Smartcard mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 SCEN LL_USART_EnableSmartcard
LL_USART_DisableSmartcard

Function name

__STATIC_INLINE void LL_USART_DisableSmartcard (USART_TypeDef * USARTx)

Function description

Disable Smartcard mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 SCEN LL_USART_DisableSmartcard
LL_USART_IsEnabledSmartcard

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledSmartcard (USART_TypeDef * USARTx)

Function description

Indicate if Smartcard mode is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 SCEN LL_USART_IsEnabledSmartcard
LL_USART_SetSmartcardPrescaler

Function name

```
__STATIC_INLINE void LL_USART_SetSmartcardPrescaler (USART_TypeDef * USARTx, uint32_t PrescalerValue)
```

Function description

Set Smartcard prescaler value, used for dividing the USART clock source to provide the SMARTCARD Clock (5 bits value)

Parameters

- **USARTx:** USART Instance
- **PrescalerValue:** Value between Min_Data=0 and Max_Data=31

Return values

- **None:**

Notes

- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR_PSC_LL_USART_SetSmartcardPrescaler

LL_USART_GetSmartcardPrescaler

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetSmartcardPrescaler (USART_TypeDef * USARTx)
```

Function description

Return Smartcard prescaler value, used for dividing the USART clock source to provide the SMARTCARD Clock (5 bits value)

Parameters

- **USARTx:** USART Instance

Return values

- **Smartcard:** prescaler value (Value between Min_Data=0 and Max_Data=31)

Notes

- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR_PSC_LL_USART_GetSmartcardPrescaler

LL_USART_SetSmartcardGuardTime

Function name

```
__STATIC_INLINE void LL_USART_SetSmartcardGuardTime (USART_TypeDef * USARTx, uint32_t GuardTime)
```

Function description

Set Smartcard Guard time value, expressed in nb of baud clocks periods (GT[7:0] bits : Guard time value)

Parameters

- **USARTx:** USART Instance
- **GuardTime:** Value between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Notes

- Macro `IS_SMARTCARD_INSTANCE(USARTx)` can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR GT `LL_USART_SetSmartcardGuardTime`
`LL_USART_GetSmartcardGuardTime`

Function name

__STATIC_INLINE uint32_t LL_USART_GetSmartcardGuardTime (USART_TypeDef * USARTx)

Function description

Return Smartcard Guard time value, expressed in nb of baud clocks periods (GT[7:0] bits : Guard time value)

Parameters

- **USARTx:** USART Instance

Return values

- **Smartcard:** Guard time value (Value between Min_Data=0x00 and Max_Data=0xFF)

Notes

- Macro `IS_SMARTCARD_INSTANCE(USARTx)` can be used to check whether or not Smartcard feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- GTPR GT `LL_USART_GetSmartcardGuardTime`
`LL_USART_EnableHalfDuplex`

Function name

__STATIC_INLINE void LL_USART_EnableHalfDuplex (USART_TypeDef * USARTx)

Function description

Enable Single Wire Half-Duplex mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HALFDUPLEX_INSTANCE(USARTx)` can be used to check whether or not Half-Duplex mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 HDSEL `LL_USART_EnableHalfDuplex`
`LL_USART_DisableHalfDuplex`

Function name

__STATIC_INLINE void LL_USART_DisableHalfDuplex (USART_TypeDef * USARTx)

Function description

Disable Single Wire Half-Duplex mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HALFDUPLEX_INSTANCE(USARTx)` can be used to check whether or not Half-Duplex mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 HDSEL `LL_USART_DisableHalfDuplex`

`LL_USART_IsEnabledHalfDuplex`

Function name

`__STATIC_INLINE uint32_t LL_USART_IsEnabledHalfDuplex (USART_TypeDef * USARTx)`

Function description

Indicate if Single Wire Half-Duplex mode is enabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro `IS_UART_HALFDUPLEX_INSTANCE(USARTx)` can be used to check whether or not Half-Duplex mode is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 HDSEL `LL_USART_IsEnabledHalfDuplex`

`LL_USART_SetLINBrkDetectionLen`

Function name

`__STATIC_INLINE void LL_USART_SetLINBrkDetectionLen (USART_TypeDef * USARTx, uint32_t LINBDLength)`

Function description

Set LIN Break Detection Length.

Parameters

- **USARTx:** USART Instance
- **LINBDLength:** This parameter can be one of the following values:
 - `LL_USART_LINBREAK_DETECT_10B`
 - `LL_USART_LINBREAK_DETECT_11B`

Return values

- **None:**

Notes

- Macro `IS_UART_LIN_INSTANCE(USARTx)` can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBDL `LL_USART_SetLINBrkDetectionLen`

```
LL_USART_GetLINBrkDetectionLen
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_GetLINBrkDetectionLen (USART_TypeDef * USARTx)
```

Function description

Return LIN Break Detection Length.

Parameters

- **USARTx:** USART Instance

Return values

- **Returned:** value can be one of the following values:
 - LL_USART_LINBREAK_DETECT_10B
 - LL_USART_LINBREAK_DETECT_11B

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBDL LL_USART_GetLINBrkDetectionLen

```
LL_USART_EnableLIN
```

Function name

```
__STATIC_INLINE void LL_USART_EnableLIN (USART_TypeDef * USARTx)
```

Function description

Enable LIN mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_EnableLIN

```
LL_USART_DisableLIN
```

Function name

```
__STATIC_INLINE void LL_USART_DisableLIN (USART_TypeDef * USARTx)
```

Function description

Disable LIN mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_LIN_INSTANCE(USARTx)` can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_DisableLIN
- LL_USART_IsEnabledLIN

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledLIN (USART_TypeDef * USARTx)

Function description

Indicate if LIN mode is enabled.

Parameters

- USARTx:** USART Instance

Return values

- State:** of bit (1 or 0).

Notes

- Macro `IS_UART_LIN_INSTANCE(USARTx)` can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_IsEnabledLIN
- LL_USART_ConfigAsyncMode

Function name

__STATIC_INLINE void LL_USART_ConfigAsyncMode (USART_TypeDef * USARTx)

Function description

Perform basic configuration of USART for enabling use in Asynchronous Mode (UART)

Parameters

- USARTx:** USART Instance

Return values

- None:**

Notes

- In UART mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register, CLKEN bit in the USART_CR2 register, SCEN bit in the USART_CR3 register, IREN bit in the USART_CR3 register, HDSEL bit in the USART_CR3 register.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using `LL_USART_DisableLIN()` function Clear CLKEN in CR2 using `LL_USART_DisableSCLKOutput()` function Clear SCEN in CR3 using `LL_USART_DisableSmartcard()` function Clear IREN in CR3 using `LL_USART_DisableIrda()` function Clear HDSEL in CR3 using `LL_USART_DisableHalfDuplex()` function
- Other remaining configurations items related to Asynchronous Mode (as Baud Rate, Word length, Parity, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigAsyncMode
- CR2 CLKEN LL_USART_ConfigAsyncMode
- CR3 SCEN LL_USART_ConfigAsyncMode
- CR3 IREN LL_USART_ConfigAsyncMode
- CR3 HDSEL LL_USART_ConfigAsyncMode

LL_USART_ConfigSyncMode

Function name

__STATIC_INLINE void LL_USART_ConfigSyncMode (USART_TypeDef * USARTx)

Function description

Perform basic configuration of USART for enabling use in Synchronous Mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In Synchronous mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register, SCEN bit in the USART_CR3 register, IREN bit in the USART_CR3 register, HDSEL bit in the USART_CR3 register. This function also sets the USART in Synchronous mode.
- Macro IS_USART_INSTANCE(USARTx) can be used to check whether or not Synchronous mode is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using LL_USART_DisableLIN() functionClear IREN in CR3 using LL_USART_DisableIrda() functionClear SCEN in CR3 using LL_USART_DisableSmartcard() functionClear HDSEL in CR3 using LL_USART_DisableHalfDuplex() functionSet CLKEN in CR2 using LL_USART_EnableSCLKOutput() function
- Other remaining configurations items related to Synchronous Mode (as Baud Rate, Word length, Parity, Clock Polarity, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigSyncMode
- CR2 CLKEN LL_USART_ConfigSyncMode
- CR3 SCEN LL_USART_ConfigSyncMode
- CR3 IREN LL_USART_ConfigSyncMode
- CR3 HDSEL LL_USART_ConfigSyncMode

LL_USART_ConfigLINMode

Function name

__STATIC_INLINE void LL_USART_ConfigLINMode (USART_TypeDef * USARTx)

Function description

Perform basic configuration of USART for enabling use in LIN Mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In LIN mode, the following bits must be kept cleared: STOP and CLKEN bits in the USART_CR2 register, SCEN bit in the USART_CR3 register, IREN bit in the USART_CR3 register, HDSEL bit in the USART_CR3 register. This function also set the UART/USART in LIN mode.
- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clear CLKEN in CR2 using LL_USART_DisableSCLKOutput() functionClear STOP in CR2 using LL_USART_SetStopBitsLength() functionClear SCEN in CR3 using LL_USART_DisableSmartcard() functionClear IREN in CR3 using LL_USART_DisableIrda() functionClear HDSEL in CR3 using LL_USART_DisableHalfDuplex() functionSet LINEN in CR2 using LL_USART_EnableLIN() function
- Other remaining configurations items related to LIN Mode (as Baud Rate, Word length, LIN Break Detection Length, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 CLKEN LL_USART_ConfigLINMode
- CR2 STOP LL_USART_ConfigLINMode
- CR2 LINEN LL_USART_ConfigLINMode
- CR3 IREN LL_USART_ConfigLINMode
- CR3 SCEN LL_USART_ConfigLINMode
- CR3 HDSEL LL_USART_ConfigLINMode

LL_USART_ConfigHalfDuplexMode

Function name

__STATIC_INLINE void LL_USART_ConfigHalfDuplexMode (USART_TypeDef * USARTx)

Function description

Perform basic configuration of USART for enabling use in Half Duplex Mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In Half Duplex mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register, CLKEN bit in the USART_CR2 register, SCEN bit in the USART_CR3 register, IREN bit in the USART_CR3 register, This function also sets the UART/USART in Half Duplex mode.
- Macro IS_UART_HALFDUPLEX_INSTANCE(USARTx) can be used to check whether or not Half-Duplex mode is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using LL_USART_DisableLIN() functionClear CLKEN in CR2 using LL_USART_DisableSCLKOutput() functionClear SCEN in CR3 using LL_USART_DisableSmartcard() functionClear IREN in CR3 using LL_USART_DisableIrda() functionSet HDSEL in CR3 using LL_USART_EnableHalfDuplex() function
- Other remaining configurations items related to Half Duplex Mode (as Baud Rate, Word length, Parity, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigHalfDuplexMode
- CR2 CLKEN LL_USART_ConfigHalfDuplexMode
- CR3 HDSEL LL_USART_ConfigHalfDuplexMode
- CR3 SCEN LL_USART_ConfigHalfDuplexMode
- CR3 IREN LL_USART_ConfigHalfDuplexMode

```
LL_USART_ConfigSmartcardMode
```

Function name

```
__STATIC_INLINE void LL_USART_ConfigSmartcardMode (USART_TypeDef * USARTx)
```

Function description

Perform basic configuration of USART for enabling use in Smartcard Mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In Smartcard mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register, IREN bit in the USART_CR3 register, HDSEL bit in the USART_CR3 register. This function also configures Stop bits to 1.5 bits and sets the USART in Smartcard mode (SCEN bit). Clock Output is also enabled (CLKEN).
- Macro IS_SMARTCARD_INSTANCE(USARTx) can be used to check whether or not Smartcard feature is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using LL_USART_DisableLIN() functionClear IREN in CR3 using LL_USART_DisableIrda() functionClear HDSEL in CR3 using LL_USART_DisableHalfDuplex() functionConfigure STOP in CR2 using LL_USART_SetStopBitsLength() functionSet CLKEN in CR2 using LL_USART_EnableSCLKOutput() functionSet SCEN in CR3 using LL_USART_EnableSmartcard() function
- Other remaining configurations items related to Smartcard Mode (as Baud Rate, Word length, Parity, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigSmartcardMode
- CR2 STOP LL_USART_ConfigSmartcardMode
- CR2 CLKEN LL_USART_ConfigSmartcardMode
- CR3 HDSEL LL_USART_ConfigSmartcardMode
- CR3 SCEN LL_USART_ConfigSmartcardMode

```
LL_USART_ConfigIrdaMode
```

Function name

```
__STATIC_INLINE void LL_USART_ConfigIrdaMode (USART_TypeDef * USARTx)
```

Function description

Perform basic configuration of USART for enabling use in Irda Mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In IrDA mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register, STOP and CLKEN bits in the USART_CR2 register, SCEN bit in the USART_CR3 register, HDSEL bit in the USART_CR3 register. This function also sets the UART/USART in IrDA mode (IREN bit).
- Macro IS_IRDA_INSTANCE(USARTx) can be used to check whether or not IrDA feature is supported by the USARTx instance.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using LL_USART_DisableLIN() functionClear CLKEN in CR2 using LL_USART_DisableSCLKOutput() functionClear SCEN in CR3 using LL_USART_DisableSmartcard() functionClear HDSEL in CR3 using LL_USART_DisableHalfDuplex() functionConfigure STOP in CR2 using LL_USART_SetStopBitsLength() functionSet IREN in CR3 using LL_USART_EnableIrda() function
- Other remaining configurations items related to Irda Mode (as Baud Rate, Word length, Power mode, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigIrdaMode
- CR2 CLKEN LL_USART_ConfigIrdaMode
- CR2 STOP LL_USART_ConfigIrdaMode
- CR3 SCEN LL_USART_ConfigIrdaMode
- CR3 HDSEL LL_USART_ConfigIrdaMode
- CR3 IREN LL_USART_ConfigIrdaMode

LL_USART_ConfigMultiProcessMode

Function name

__STATIC_INLINE void LL_USART_ConfigMultiProcessMode (USART_TypeDef * USARTx)

Function description

Perform basic configuration of USART for enabling use in Multi processor Mode (several USARTs connected in a network, one of the USARTs can be the master, its TX output connected to the RX inputs of the other slaves USARTs).

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- In MultiProcessor mode, the following bits must be kept cleared: LINEN bit in the USART_CR2 register,CLKEN bit in the USART_CR2 register,SCEN bit in the USART_CR3 register,IREN bit in the USART_CR3 register,HDSEL bit in the USART_CR3 register.
- Call of this function is equivalent to following function call sequence : Clear LINEN in CR2 using LL_USART_DisableLIN() functionClear CLKEN in CR2 using LL_USART_DisableSCLKOutput() functionClear SCEN in CR3 using LL_USART_DisableSmartcard() functionClear IREN in CR3 using LL_USART_DisableIrda() functionClear HDSEL in CR3 using LL_USART_DisableHalfDuplex() function
- Other remaining configurations items related to Multi processor Mode (as Baud Rate, Wake Up Method, Node address, ...) should be set using dedicated functions

Reference Manual to LL API cross reference:

- CR2 LINEN LL_USART_ConfigMultiProcessMode
- CR2 CLKEN LL_USART_ConfigMultiProcessMode
- CR3 SCEN LL_USART_ConfigMultiProcessMode
- CR3 HDSEL LL_USART_ConfigMultiProcessMode
- CR3 IREN LL_USART_ConfigMultiProcessMode

```
LL_USART_IsActiveFlag_PE
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_PE (USART_TypeDef * USARTx)
```

Function description

Check if the USART Parity Error Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR PE LL_USART_IsActiveFlag_PE

```
LL_USART_IsActiveFlag_FE
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_FE (USART_TypeDef * USARTx)
```

Function description

Check if the USART Framing Error Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR FE LL_USART_IsActiveFlag_FE

```
LL_USART_IsActiveFlag_NE
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_NE (USART_TypeDef * USARTx)
```

Function description

Check if the USART Noise error detected Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR NF LL_USART_IsActiveFlag_NE

```
LL_USART_IsActiveFlag_ORE
```

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_ORE (USART_TypeDef * USARTx)
```

Function description

Check if the USART OverRun Error Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR ORE LL_USART_IsActiveFlag_ORE
LL_USART_IsActiveFlag_IDLE

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_IDLE (USART_TypeDef * USARTx)
```

Function description

Check if the USART IDLE line detected Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR IDLE LL_USART_IsActiveFlag_IDLE
LL_USART_IsActiveFlag_RXNE

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_RXNE (USART_TypeDef * USARTx)
```

Function description

Check if the USART Read Data Register Not Empty Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR RXNE LL_USART_IsActiveFlag_RXNE
LL_USART_IsActiveFlag_TC

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_TC (USART_TypeDef * USARTx)
```

Function description

Check if the USART Transmission Complete Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR TC LL_USART_IsActiveFlag_TC
LL_USART_IsActiveFlag_TXE

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_TXE (USART_TypeDef * USARTx)
```

Function description

Check if the USART Transmit Data Register Empty Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR TXE LL_USART_IsActiveFlag_TXE
LL_USART_IsActiveFlag_LBD

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_LBD (USART_TypeDef * USARTx)
```

Function description

Check if the USART LIN Break Detection Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- SR LBD LL_USART_IsActiveFlag_LBD
LL_USART_IsActiveFlag_nCTS

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_nCTS (USART_TypeDef * USARTx)
```

Function description

Check if the USART CTS Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_UART_HWFLOW_INSTANCE(USARTx) can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- SR CTS LL_USART_IsActiveFlag_nCTS
LL_USART_IsActiveFlag_SBK

Function name

__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_SBK (USART_TypeDef * USARTx)

Function description

Check if the USART Send Break Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 SBK LL_USART_IsActiveFlag_SBK
LL_USART_IsActiveFlag_RWU

Function name

__STATIC_INLINE uint32_t LL_USART_IsActiveFlag_RWU (USART_TypeDef * USARTx)

Function description

Check if the USART Receive Wake Up from mute mode Flag is set or not.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 RWU LL_USART_IsActiveFlag_RWU
LL_USART_ClearFlag_PE

Function name

__STATIC_INLINE void LL_USART_ClearFlag_PE (USART_TypeDef * USARTx)

Function description

Clear Parity Error Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the USARTx_SR register followed by a read access to the USARTx_DR register.
- Please also consider that when clearing this flag, other flags as NE, FE, ORE, IDLE would also be cleared.

Reference Manual to LL API cross reference:

- SR PE LL_USART_ClearFlag_PE

```
LL_USART_ClearFlag_FE
```

Function name

```
__STATIC_INLINE void LL_USART_ClearFlag_FE (USART_TypeDef * USARTx)
```

Function description

Clear Framing Error Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the USARTx_SR register followed by a read access to the USARTx_DR register.
- Please also consider that when clearing this flag, other flags as PE, NE, ORE, IDLE would also be cleared.

Reference Manual to LL API cross reference:

- SR FE LL_USART_ClearFlag_FE

```
LL_USART_ClearFlag_NE
```

Function name

```
__STATIC_INLINE void LL_USART_ClearFlag_NE (USART_TypeDef * USARTx)
```

Function description

Clear Noise detected Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the USARTx_SR register followed by a read access to the USARTx_DR register.
- Please also consider that when clearing this flag, other flags as PE, FE, ORE, IDLE would also be cleared.

Reference Manual to LL API cross reference:

- SR NF LL_USART_ClearFlag_NE

```
LL_USART_ClearFlag_ORE
```

Function name

```
__STATIC_INLINE void LL_USART_ClearFlag_ORE (USART_TypeDef * USARTx)
```

Function description

Clear OverRun Error Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the USARTx_SR register followed by a read access to the USARTx_DR register.
- Please also consider that when clearing this flag, other flags as PE, NE, FE, IDLE would also be cleared.

Reference Manual to LL API cross reference:

- SR ORE LL_USART_ClearFlag_ORE
LL_USART_ClearFlag_IDLE

Function name

__STATIC_INLINE void LL_USART_ClearFlag_IDLE (USART_TypeDef * USARTx)

Function description

Clear IDLE line detected Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Clearing this flag is done by a read access to the USARTx_SR register followed by a read access to the USARTx_DR register.
- Please also consider that when clearing this flag, other flags as PE, NE, FE, ORE would also be cleared.

Reference Manual to LL API cross reference:

- SR IDLE LL_USART_ClearFlag_IDLE
LL_USART_ClearFlag_TC

Function name

__STATIC_INLINE void LL_USART_ClearFlag_TC (USART_TypeDef * USARTx)

Function description

Clear Transmission Complete Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR TC LL_USART_ClearFlag_TC
LL_USART_ClearFlag_RXNE

Function name

__STATIC_INLINE void LL_USART_ClearFlag_RXNE (USART_TypeDef * USARTx)

Function description

Clear RX Not Empty Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR RXNE LL_USART_ClearFlag_RXNE
LL_USART_ClearFlag_LBD

Function name

__STATIC_INLINE void LL_USART_ClearFlag_LBD (USART_TypeDef * USARTx)

Function description

Clear LIN Break Detection Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- SR LBD LL_USART_ClearFlag_LBD
LL_USART_ClearFlag_nCTS

Function name

__STATIC_INLINE void LL_USART_ClearFlag_nCTS (USART_TypeDef * USARTx)

Function description

Clear CTS Interrupt Flag.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_UART_HWFLOW_INSTANCE(USARTx) can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- SR CTS LL_USART_ClearFlag_nCTS
LL_USART_EnableIT_IDLE

Function name

__STATIC_INLINE void LL_USART_EnableIT_IDLE (USART_TypeDef * USARTx)

Function description

Enable IDLE Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 IDLEIE LL_USART_EnableIT_IDLE
LL_USART_EnableIT_RXNE

Function name

__STATIC_INLINE void LL_USART_EnableIT_RXNE (USART_TypeDef * USARTx)

Function description

Enable RX Not Empty Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RXNEIE LL_USART_EnableIT_RXNE
LL_USART_EnableIT_TC

Function name

__STATIC_INLINE void LL_USART_EnableIT_TC (USART_TypeDef * USARTx)

Function description

Enable Transmission Complete Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TCIE LL_USART_EnableIT_TC
LL_USART_EnableIT_TXE

Function name

__STATIC_INLINE void LL_USART_EnableIT_TXE (USART_TypeDef * USARTx)

Function description

Enable TX Empty Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TXEIE LL_USART_EnableIT_TXE
LL_USART_EnableIT_PE

Function name

```
__STATIC_INLINE void LL_USART_EnableIT_PE (USART_TypeDef * USARTx)
```

Function description

Enable Parity Error Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 PEIE LL_USART_EnableIT_PE

```
LL_USART_EnableIT_LBD
```

Function name

```
__STATIC_INLINE void LL_USART_EnableIT_LBD (USART_TypeDef * USARTx)
```

Function description

Enable LIN Break Detection Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBDIE LL_USART_EnableIT_LBD

```
LL_USART_EnableIT_ERROR
```

Function name

```
__STATIC_INLINE void LL_USART_EnableIT_ERROR (USART_TypeDef * USARTx)
```

Function description

Enable Error Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- When set, Error Interrupt Enable Bit is enabling interrupt generation in case of a framing error, overrun error or noise flag (FE=1 or ORE=1 or NF=1 in the USARTx_SR register). 0: Interrupt is inhibited 1: An interrupt is generated when FE=1 or ORE=1 or NF=1 in the USARTx_SR register.

Reference Manual to LL API cross reference:

- CR3 EIE LL_USART_EnableIT_ERROR

```
LL_USART_EnableIT_CTS
```

Function name

```
__STATIC_INLINE void LL_USART_EnableIT_CTS (USART_TypeDef * USARTx)
```

Function description

Enable CTS Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro IS_UART_HWFLOW_INSTANCE(USARTx) can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 CTSIE LL_USART_EnableIT_CTS

LL_USART_DisableIT_IDLE

Function name

```
__STATIC_INLINE void LL_USART_DisableIT_IDLE (USART_TypeDef * USARTx)
```

Function description

Disable IDLE Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 IDLEIE LL_USART_DisableIT_IDLE

LL_USART_DisableIT_RXNE

Function name

```
__STATIC_INLINE void LL_USART_DisableIT_RXNE (USART_TypeDef * USARTx)
```

Function description

Disable RX Not Empty Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RXNEIE LL_USART_DisableIT_RXNE

LL_USART_DisableIT_TC

Function name

```
__STATIC_INLINE void LL_USART_DisableIT_TC (USART_TypeDef * USARTx)
```

Function description

Disable Transmission Complete Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TCIE LL_USART_DisableIT_TC

LL_USART_DisableIT_TXE

Function name

__STATIC_INLINE void LL_USART_DisableIT_TXE (USART_TypeDef * USARTx)

Function description

Disable TX Empty Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 TXEIE LL_USART_DisableIT_TXE

LL_USART_DisableIT_PE

Function name

__STATIC_INLINE void LL_USART_DisableIT_PE (USART_TypeDef * USARTx)

Function description

Disable Parity Error Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 PEIE LL_USART_DisableIT_PE

LL_USART_DisableIT_LBD

Function name

__STATIC_INLINE void LL_USART_DisableIT_LBD (USART_TypeDef * USARTx)

Function description

Disable LIN Break Detection Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_LIN_INSTANCE(USARTx)` can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBDIE `LL_USART_DisableIT_LBD`
`LL_USART_DisableIT_ERROR`

Function name

__STATIC_INLINE void LL_USART_DisableIT_ERROR (USART_TypeDef * USARTx)

Function description

Disable Error Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- When set, Error Interrupt Enable Bit is enabling interrupt generation in case of a framing error, overrun error or noise flag (FE=1 or ORE=1 or NF=1 in the USARTx_SR register). 0: Interrupt is inhibited 1: An interrupt is generated when FE=1 or ORE=1 or NF=1 in the USARTx_SR register.

Reference Manual to LL API cross reference:

- CR3 EIE `LL_USART_DisableIT_ERROR`
`LL_USART_DisableIT_CTS`

Function name

__STATIC_INLINE void LL_USART_DisableIT_CTS (USART_TypeDef * USARTx)

Function description

Disable CTS Interrupt.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Notes

- Macro `IS_UART_HWFLOW_INSTANCE(USARTx)` can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 CTSIE `LL_USART_DisableIT_CTS`
`LL_USART_IsEnabledIT_IDLE`

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_IDLE (USART_TypeDef * USARTx)

Function description

Check if the USART IDLE Interrupt source is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 IDLEIE LL_USART_IsEnabledIT_IDLE
LL_USART_IsEnabledIT_RXNE

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_RXNE (USART_TypeDef * USARTx)
```

Function description

Check if the USART RX Not Empty Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 RXNEIE LL_USART_IsEnabledIT_RXNE
LL_USART_IsEnabledIT_TC

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_TC (USART_TypeDef * USARTx)
```

Function description

Check if the USART Transmission Complete Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 TCIE LL_USART_IsEnabledIT_TC
LL_USART_IsEnabledIT_TXE

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_TXE (USART_TypeDef * USARTx)
```

Function description

Check if the USART TX Empty Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 TXEIE LL_USART_IsEnabledIT_TXE
LL_USART_IsEnabledIT_PE

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_PE (USART_TypeDef * USARTx)

Function description

Check if the USART Parity Error Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 PEIE LL_USART_IsEnabledIT_PE
LL_USART_IsEnabledIT_LBD

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_LBD (USART_TypeDef * USARTx)

Function description

Check if the USART LIN Break Detection Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_UART_LIN_INSTANCE(USARTx) can be used to check whether or not LIN feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR2 LBDIE LL_USART_IsEnabledIT_LBD
LL_USART_IsEnabledIT_ERROR

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_ERROR (USART_TypeDef * USARTx)

Function description

Check if the USART Error Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR3 EIE LL_USART_IsEnabledIT_ERROR
LL_USART_IsEnabledIT_CTS

Function name

__STATIC_INLINE uint32_t LL_USART_IsEnabledIT_CTS (USART_TypeDef * USARTx)

Function description

Check if the USART CTS Interrupt is enabled or disabled.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Notes

- Macro IS_UART_HWFLOW_INSTANCE(USARTx) can be used to check whether or not Hardware Flow control feature is supported by the USARTx instance.

Reference Manual to LL API cross reference:

- CR3 CTSIE LL_USART_IsEnabledIT_CTS
LL_USART_EnableDMAReq_RX

Function name

__STATIC_INLINE void LL_USART_EnableDMAReq_RX (USART_TypeDef * USARTx)

Function description

Enable DMA Mode for reception.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 DMAR LL_USART_EnableDMAReq_RX
LL_USART_DisableDMAReq_RX

Function name

__STATIC_INLINE void LL_USART_DisableDMAReq_RX (USART_TypeDef * USARTx)

Function description

Disable DMA Mode for reception.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 DMAR LL_USART_DisableDMAReq_RX
LL_USART_IsEnabledDMAReq_RX

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledDMAReq_RX (USART_TypeDef * USARTx)
```

Function description

Check if DMA Mode is enabled for reception.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR3 DMAR LL_USART_IsEnabledDMAReq_RX

LL_USART_EnableDMAReq_TX

Function name

```
__STATIC_INLINE void LL_USART_EnableDMAReq_TX (USART_TypeDef * USARTx)
```

Function description

Enable DMA Mode for transmission.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 DMAT LL_USART_EnableDMAReq_TX

LL_USART_DisableDMAReq_TX

Function name

```
__STATIC_INLINE void LL_USART_DisableDMAReq_TX (USART_TypeDef * USARTx)
```

Function description

Disable DMA Mode for transmission.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR3 DMAT LL_USART_DisableDMAReq_TX

LL_USART_IsEnabledDMAReq_TX

Function name

```
__STATIC_INLINE uint32_t LL_USART_IsEnabledDMAReq_TX (USART_TypeDef * USARTx)
```

Function description

Check if DMA Mode is enabled for transmission.

Parameters

- **USARTx:** USART Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR3 DMAT LL_USART_IsEnabledDMAReq_TX
LL_USART_DMA_GetRegAddr

Function name

__STATIC_INLINE uint32_t LL_USART_DMA_GetRegAddr (USART_TypeDef * USARTx)

Function description

Get the data register address used for DMA transfer.

Parameters

- **USARTx:** USART Instance

Return values

- **Address:** of data register

Notes

- Address of Data Register is valid for both Transmit and Receive transfers.

Reference Manual to LL API cross reference:

- DR DR LL_USART_DMA_GetRegAddr
LL_USART_ReceiveData8

Function name

__STATIC_INLINE uint8_t LL_USART_ReceiveData8 (USART_TypeDef * USARTx)

Function description

Read Receiver Data register (Receive Data value, 8 bits)

Parameters

- **USARTx:** USART Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Reference Manual to LL API cross reference:

- DR DR LL_USART_ReceiveData8
LL_USART_ReceiveData9

Function name

__STATIC_INLINE uint16_t LL_USART_ReceiveData9 (USART_TypeDef * USARTx)

Function description

Read Receiver Data register (Receive Data value, 9 bits)

Parameters

- **USARTx:** USART Instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x1FF

Reference Manual to LL API cross reference:

- DR DR LL_USART_ReceiveData9
LL_USART_TransmitData8

Function name

__STATIC_INLINE void LL_USART_TransmitData8 (USART_TypeDef * USARTx, uint8_t Value)

Function description

Write in Transmitter Data Register (Transmit Data value, 8 bits)

Parameters

- **USARTx:** USART Instance
- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_USART_TransmitData8
LL_USART_TransmitData9

Function name

__STATIC_INLINE void LL_USART_TransmitData9 (USART_TypeDef * USARTx, uint16_t Value)

Function description

Write in Transmitter Data Register (Transmit Data value, 9 bits)

Parameters

- **USARTx:** USART Instance
- **Value:** between Min_Data=0x00 and Max_Data=0x1FF

Return values

- **None:**

Reference Manual to LL API cross reference:

- DR DR LL_USART_TransmitData9
LL_USART_RequestBreakSending

Function name

__STATIC_INLINE void LL_USART_RequestBreakSending (USART_TypeDef * USARTx)

Function description

Request Break sending.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 SBK LL_USART_RequestBreakSending

```
LL_USART_RequestEnterMuteMode
```

Function name

```
__STATIC_INLINE void LL_USART_RequestEnterMuteMode (USART_TypeDef * USARTx)
```

Function description

Put USART in Mute mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RWU LL_USART_RequestEnterMuteMode

```
LL_USART_RequestExitMuteMode
```

Function name

```
__STATIC_INLINE void LL_USART_RequestExitMuteMode (USART_TypeDef * USARTx)
```

Function description

Put USART in Active mode.

Parameters

- **USARTx:** USART Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RWU LL_USART_RequestExitMuteMode

```
LL_USART_DeInit
```

Function name

```
ErrorStatus LL_USART_DeInit (USART_TypeDef * USARTx)
```

Function description

De-initialize USART registers (Registers restored to their default values).

Parameters

- **USARTx:** USART Instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: USART registers are de-initialized
 - ERROR: USART registers are not de-initialized

```
LL_USART_Init
```

Function name

```
ErrorStatus LL_USART_Init (USART_TypeDef * USARTx, LL_USART_InitTypeDef * USART_InitStruct)
```

Function description

Initialize USART registers according to the specified parameters in USART_InitStruct.

Parameters

- **USARTx:** USART Instance
- **USART_InitStruct:** pointer to a LL_USART_InitTypeDef structure that contains the configuration information for the specified USART peripheral.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: USART registers are initialized according to USART_InitStruct content
 - ERROR: Problem occurred during USART Registers initialization

Notes

- As some bits in USART configuration registers can only be written when the USART is disabled (USART_CR1_UE bit =0), USART IP should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.
- Baud rate value stored in USART_InitStruct BaudRate field, should be valid (different from 0).

```
LL_USART_StructInit
```

Function name

```
void LL_USART_StructInit (LL_USART_InitTypeDef * USART_InitStruct)
```

Function description

Set each LL_USART_InitTypeDef field to default value.

Parameters

- **USART_InitStruct:** Pointer to a LL_USART_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

```
LL_USART_ClockInit
```

Function name

```
ErrorStatus LL_USART_ClockInit (USART_TypeDef * USARTx, LL_USART_ClockInitTypeDef * USART_ClockInitStruct)
```

Function description

Initialize USART Clock related settings according to the specified parameters in the USART_ClockInitStruct.

Parameters

- **USARTx:** USART Instance
- **USART_ClockInitStruct:** Pointer to a LL_USART_ClockInitTypeDef structure that contains the Clock configuration information for the specified USART peripheral.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: USART registers related to Clock settings are initialized according to USART_ClockInitStruct content
 - ERROR: Problem occurred during USART Registers initialization

Notes

- As some bits in USART configuration registers can only be written when the USART is disabled (USART_CR1_UE bit =0), USART IP should be in disabled state prior calling this function. Otherwise, ERROR result will be returned.

```
LL_USART_ClockStructInit
```

Function name

void LL_USART_ClockStructInit (LL_USART_ClockInitTypeDef * USART_ClockInitStruct)

Function description

Set each field of a LL_USART_ClockInitTypeDef type structure to default value.

Parameters

- **USART_ClockInitStruct:** Pointer to a LL_USART_ClockInitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

66.3 USART Firmware driver defines

The following section lists the various define and macros of the module.

66.3.1 USART

USART

Clock Signal

LL_USART_CLOCK_DISABLE

Clock signal not provided

LL_USART_CLOCK_ENABLE

Clock signal provided

Datawidth

LL_USART_DATAWIDTH_8B

8 bits word length : Start bit, 8 data bits, n stop bits

LL_USART_DATAWIDTH_9B

9 bits word length : Start bit, 9 data bits, n stop bits

Communication Direction

LL_USART_DIRECTION_NONE

Transmitter and Receiver are disabled

LL_USART_DIRECTION_RX

Transmitter is disabled and Receiver is enabled

LL_USART_DIRECTION_TX

Transmitter is enabled and Receiver is disabled

LL_USART_DIRECTION_TX_RX

Transmitter and Receiver are enabled

Get Flags Defines

LL_USART_SR_PE

Parity error flag

LL_USART_SR_FE

Framing error flag

LL_USART_SR_NE

Noise detected flag

LL_USART_SR_ORE

Overrun error flag

LL_USART_SR_IDLE

Idle line detected flag

LL_USART_SR_RXNE

Read data register not empty flag

LL_USART_SR_TC

Transmission complete flag

LL_USART_SR_TXE

Transmit data register empty flag

LL_USART_SR_LBD

LIN break detection flag

LL_USART_SR_CTS

CTS flag

Hardware Control

LL_USART_HWCONTROL_NONE

CTS and RTS hardware flow control disabled

LL_USART_HWCONTROL_RTS

RTS output enabled, data is only requested when there is space in the receive buffer

LL_USART_HWCONTROL_CTS

CTS mode enabled, data is only transmitted when the nCTS input is asserted (tied to 0)

LL_USART_HWCONTROL_RTS_CTS

CTS and RTS hardware flow control enabled

IrDA Power

LL_USART_IRDA_POWER_NORMAL

IrDA normal power mode

LL_USART_IRDA_POWER_LOW

IrDA low power mode

IT Defines

LL_USART_CR1_IDLEIE

IDLE interrupt enable

LL_USART_CR1_RXNEIE

Read data register not empty interrupt enable

LL_USART_CR1_TCIE

Transmission complete interrupt enable

LL_USART_CR1_TXEIE

Transmit data register empty interrupt enable

LL_USART_CR1_PEIE

Parity error

LL_USART_CR2_LBDIE

LIN break detection interrupt enable

LL_USART_CR3_EIE

Error interrupt enable

LL_USART_CR3_CTSIE

CTS interrupt enable

Last Clock Pulse

LL_USART_LASTCLKPULSE_NO_OUTPUT

The clock pulse of the last data bit is not output to the SCLK pin

LL_USART_LASTCLKPULSE_OUTPUT

The clock pulse of the last data bit is output to the SCLK pin

LIN Break Detection Length

LL_USART_LINBREAK_DETECT_10B

10-bit break detection method selected

LL_USART_LINBREAK_DETECT_11B

11-bit break detection method selected

Oversampling

LL_USART_OVERSAMPLING_16

Oversampling by 16

LL_USART_OVERSAMPLING_8

Oversampling by 8

Parity Control

LL_USART_PARITY_NONE

Parity control disabled

LL_USART_PARITY_EVEN

Parity control enabled and Even Parity is selected

LL_USART_PARITY_ODD

Parity control enabled and Odd Parity is selected

Clock Phase

LL_USART_PHASE_1EDGE

The first clock transition is the first data capture edge

LL_USART_PHASE_2EDGE

The second clock transition is the first data capture edge

Clock Polarity

LL_USART_POLARITY_LOW

Steady low value on SCLK pin outside transmission window

LL_USART_POLARITY_HIGH

Steady high value on SCLK pin outside transmission window

Stop Bits

LL_USART_STOPBITS_0_5

0.5 stop bit

LL_USART_STOPBITS_1

1 stop bit

LL_USART_STOPBITS_1_5

1.5 stop bits

LL_USART_STOPBITS_2

2 stop bits

Wakeup

LL_USART_WAKEUP_IDLELINE

USART wake up from Mute mode on Idle Line

LL_USART_WAKEUP_ADDRESSMARK

USART wake up from Mute mode on Address Mark

Exported_Macros_Helper

__LL_USART_DIV_SAMPLING8_100

Description:

- Compute USARTDIV value according to Peripheral Clock and expected Baud Rate in 8 bits sampling mode (32 bits value of USARTDIV is returned)

Parameters:

- __PERIPHCLK__: Peripheral Clock frequency used for USART instance
- __BAUDRATE__: Baud rate value to achieve

Return value:

- USARTDIV: value to be used for BRR register filling in OverSampling_8 case

__LL_USART_DIVMANT_SAMPLING8

__LL_USART_DIVFRAQ_SAMPLING8

__LL_USART_DIV_SAMPLING8

__LL_USART_DIV_SAMPLING16_100

Description:

- Compute USARTDIV value according to Peripheral Clock and expected Baud Rate in 16 bits sampling mode (32 bits value of USARTDIV is returned)

Parameters:

- __PERIPHCLK__: Peripheral Clock frequency used for USART instance
- __BAUDRATE__: Baud rate value to achieve

Return value:

- USARTDIV: value to be used for BRR register filling in OverSampling_16 case

__LL_USART_DIVMANT_SAMPLING16

__LL_USART_DIVFRAQ_SAMPLING16

__LL_USART_DIV_SAMPLING16

Common Write and read registers Macros

LL_USART_WriteReg

Description:

- Write a value in USART register.

Parameters:

- `__INSTANCE__`: USART Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_USART_ReadReg

Description:

- Read a value in USART register.

Parameters:

- `__INSTANCE__`: USART Instance
- `__REG__`: Register to be read

Return value:

- Register: value

67 LL UTILS Generic Driver

67.1 UTILS Firmware driver registers structures

67.1.1 LL_UTILS_PLLInitTypeDef

LL_UTILS_PLLInitTypeDef is defined in the stm32f2xx_ll_utils.h

Data Fields

- **uint32_t PLLM**
- **uint32_t PLLN**
- **uint32_t PLLP**

Field Documentation

- **uint32_t LL_UTILS_PLLInitTypeDef::PLLM**
Division factor for PLL VCO input clock. This parameter can be a value of [RCC_LL_EC_PLLM_DIV](#). This feature can be modified afterwards using unitary function [LL_RCC_PLL_ConfigDomain_SYS\(\)](#).
- **uint32_t LL_UTILS_PLLInitTypeDef::PLLN**
Multiplication factor for PLL VCO output clock. This parameter must be a number between Min_Data = 8 and Max_Data = 86. This feature can be modified afterwards using unitary function [LL_RCC_PLL_ConfigDomain_SYS\(\)](#).
- **uint32_t LL_UTILS_PLLInitTypeDef::PLLP**
Division for the main system clock. This parameter can be a value of [RCC_LL_EC_PLLP_DIV](#). This feature can be modified afterwards using unitary function [LL_RCC_PLL_ConfigDomain_SYS\(\)](#).

67.1.2 LL_UTILS_ClkInitTypeDef

LL_UTILS_ClkInitTypeDef is defined in the stm32f2xx_ll_utils.h

Data Fields

- **uint32_t AHBCLKDivider**
- **uint32_t APB1CLKDivider**
- **uint32_t APB2CLKDivider**

Field Documentation

- **uint32_t LL_UTILS_ClkInitTypeDef::AHBCLKDivider**
The AHB clock (HCLK) divider. This clock is derived from the system clock (SYSCLK). This parameter can be a value of [RCC_LL_EC_SYSCLK_DIV](#). This feature can be modified afterwards using unitary function [LL_RCC_SetAHBPrescaler\(\)](#).
- **uint32_t LL_UTILS_ClkInitTypeDef::APB1CLKDivider**
The APB1 clock (PCLK1) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of [RCC_LL_EC_APB1_DIV](#). This feature can be modified afterwards using unitary function [LL_RCC_SetAPB1Prescaler\(\)](#).
- **uint32_t LL_UTILS_ClkInitTypeDef::APB2CLKDivider**
The APB2 clock (PCLK2) divider. This clock is derived from the AHB clock (HCLK). This parameter can be a value of [RCC_LL_EC_APB2_DIV](#). This feature can be modified afterwards using unitary function [LL_RCC_SetAPB2Prescaler\(\)](#).

67.2 UTILS Firmware driver API description

The following section lists the various functions of the UTILS library.

67.2.1 System Configuration functions

System, AHB and APB buses clocks configuration

- The maximum frequency of the SYSCLK, HCLK, PCLK1 and PCLK2 is 120000000 Hz.

This section contains the following APIs:

-
-
-

67.2.2 Detailed description of functions

LL_GetUID_Word0

Function name

__STATIC_INLINE uint32_t LL_GetUID_Word0 (void)

Function description

Get Word0 of the unique device identifier (UID based on 96 bits)

Return values

- **UID[31:0]:**

LL_GetUID_Word1

Function name

__STATIC_INLINE uint32_t LL_GetUID_Word1 (void)

Function description

Get Word1 of the unique device identifier (UID based on 96 bits)

Return values

- **UID[63:32]:**

LL_GetUID_Word2

Function name

__STATIC_INLINE uint32_t LL_GetUID_Word2 (void)

Function description

Get Word2 of the unique device identifier (UID based on 96 bits)

Return values

- **UID[95:64]:**

LL_GetFlashSize

Function name

__STATIC_INLINE uint32_t LL_GetFlashSize (void)

Function description

Get Flash memory size.

Return values

- **FLASH_SIZE[15:0]:** Flash memory size

Notes

- This bitfield indicates the size of the device Flash memory expressed in Kbytes. As an example, 0x040 corresponds to 64 Kbytes.

LL_InitTick

Function name

__STATIC_INLINE void LL_InitTick (uint32_t HCLKFrequency, uint32_t Ticks)

Function description

This function configures the Cortex-M SysTick source of the time base.

Parameters

- **HCLKFrequency:** HCLK frequency in Hz (can be calculated thanks to RCC helper macro)
- **Ticks:** Number of ticks

Return values

- **None:**

Notes

- When a RTOS is used, it is recommended to avoid changing the SysTick configuration by calling this function, for a delay use rather osDelay RTOS service.

LL_Init1msTick

Function name

void LL_Init1msTick (uint32_t HCLKFrequency)

Function description

This function configures the Cortex-M SysTick source to have 1ms time base.

Parameters

- **HCLKFrequency:** HCLK frequency in Hz

Return values

- **None:**

Notes

- When a RTOS is used, it is recommended to avoid changing the Systick configuration by calling this function, for a delay use rather osDelay RTOS service.
- HCLK frequency can be calculated thanks to RCC helper macro or function LL_RCC_GetSystemClocksFreq

LL_mDelay

Function name

void LL_mDelay (uint32_t Delay)

Function description

This function provides accurate delay (in milliseconds) based on SysTick counter flag.

Parameters

- **Delay:** specifies the delay time length, in milliseconds.

Return values

- **None:**

Notes

- When a RTOS is used, it is recommended to avoid using blocking delay and use rather osDelay service.
- To respect 1ms timebase, user should call LL_Init1msTick function which will configure Systick to 1ms

LL_SetSystemCoreClock

Function name

void LL_SetSystemCoreClock (uint32_t HCLKFrequency)

Function description

This function sets directly SystemCoreClock CMSIS variable.

Parameters

- **HCLKFrequency:** HCLK frequency in Hz (can be calculated thanks to RCC helper macro)

Return values

- **None:**

Notes

- Variable can be calculated also through SystemCoreClockUpdate function.

```
LL_PLL_ConfigSystemClock_HSI
```

Function name

ErrorStatus LL_PLL_ConfigSystemClock_HSI (LL_UTILS_PLLInitTypeDef * UTILS_PLLInitStruct, LL_UTILS_ClkInitTypeDef * UTILS_ClkInitStruct)

Function description

This function configures system clock at maximum frequency with HSI as clock source of the PLL.

Parameters

- **UTILS_PLLInitStruct:** pointer to a LL_UTILS_PLLInitTypeDef structure that contains the configuration information for the PLL.
- **UTILS_ClkInitStruct:** pointer to a LL_UTILS_ClkInitTypeDef structure that contains the configuration information for the BUS prescalers.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: Max frequency configuration done
 - ERROR: Max frequency configuration not done

Notes

- The application need to ensure that PLL is disabled.
- Function is based on the following formula: PLL output frequency = (((HSI frequency / PLLM) * PLLN) / PLLP) PLLM: ensure that the VCO input frequency ranges from 0.95 to 2.10 MHz (PLLVC0_input = HSI frequency / PLLM) PLLN: ensure that the VCO output frequency is between 192 and 432 MHz (PLLVC0_output = PLLVC0_input * PLLN) PLLP: ensure that max frequency at 120000000 Hz is reached (PLLVC0_output / PLLP)

```
LL_PLL_ConfigSystemClock_HSE
```

Function name

ErrorStatus LL_PLL_ConfigSystemClock_HSE (uint32_t HSEFrequency, uint32_t HSEBypass, LL_UTILS_PLLInitTypeDef * UTILS_PLLInitStruct, LL_UTILS_ClkInitTypeDef * UTILS_ClkInitStruct)

Function description

This function configures system clock with HSE as clock source of the PLL.

Parameters

- **HSEFrequency:** Value between Min_Data = 4000000 and Max_Data = 26000000
- **HSEBypass:** This parameter can be one of the following values:
 - LL_UTILS_HSEBYPASS_ON
 - LL_UTILS_HSEBYPASS_OFF
- **UTILS_PLLInitStruct:** pointer to a LL_UTILS_PLLInitTypeDef structure that contains the configuration information for the PLL.
- **UTILS_ClkInitStruct:** pointer to a LL_UTILS_ClkInitTypeDef structure that contains the configuration information for the BUS prescalers.

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: Max frequency configuration done
 - ERROR: Max frequency configuration not done

Notes

- The application need to ensure that PLL is disabled.
- Function is based on the following formula: PLL output frequency = (((HSE frequency / PLLM) * PLLN) / PLLP) PLLM: ensure that the VCO input frequency ranges from 0.95 to 2.10 MHz (PLLVC0_input = HSE frequency / PLLM) PLLN: ensure that the VCO output frequency is between 192 and 432 MHz (PLLVC0_output = PLLVC0_input * PLLN) PLLP: ensure that max frequency at 120000000 Hz is reached (PLLVC0_output / PLLP)

67.3 UTILS Firmware driver defines

The following section lists the various define and macros of the module.

67.3.1 UTILS

UTILS

HSE Bypass activation

LL_UTILS_HSEBYPASS_OFF

HSE Bypass is not enabled

LL_UTILS_HSEBYPASS_ON

HSE Bypass is enabled

68 LL WWDG Generic Driver

68.1 WWDG Firmware driver API description

The following section lists the various functions of the WWDG library.

68.1.1 Detailed description of functions

LL_WWDG_Enable

Function name

__STATIC_INLINE void LL_WWDG_Enable (WWDG_TypeDef * WWDGx)

Function description

Enable Window Watchdog.

Parameters

- **WWDGx:** WWDG Instance

Return values

- **None:**

Notes

- It is enabled by setting the WDGA bit in the WWDG_CR register, then it cannot be disabled again except by a reset. This bit is set by software and only cleared by hardware after a reset. When WDGA = 1, the watchdog can generate a reset.

Reference Manual to LL API cross reference:

- CR WDGA LL_WWDG_Enable

LL_WWDG_IsEnabled

Function name

__STATIC_INLINE uint32_t LL_WWDG_IsEnabled (WWDG_TypeDef * WWDGx)

Function description

Checks if Window Watchdog is enabled.

Parameters

- **WWDGx:** WWDG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR WDGA LL_WWDG_IsEnabled

LL_WWDG_SetCounter

Function name

__STATIC_INLINE void LL_WWDG_SetCounter (WWDG_TypeDef * WWDGx, uint32_t Counter)

Function description

Set the Watchdog counter value to provided value (7-bits T[6:0])

Parameters

- **WWDGx:** WWDG Instance
- **Counter:** 0..0x7F (7 bit counter value)

Return values

- **None:**

Notes

- When writing to the WWDG_CR register, always write 1 in the MSB b6 to avoid generating an immediate reset This counter is decremented every (4096 x 2expWDGTB) PCLK cycles A reset is produced when it rolls over from 0x40 to 0x3F (bit T6 becomes cleared) Setting the counter lower then 0x40 causes an immediate reset (if WWDG enabled)

Reference Manual to LL API cross reference:

- CR T LL_WWDG_SetCounter

LL_WWDG_GetCounter

Function name

__STATIC_INLINE uint32_t LL_WWDG_GetCounter (WWDG_TypeDef * WWDGx)

Function description

Return current Watchdog Counter Value (7 bits counter value)

Parameters

- **WWDGx:** WWDG Instance

Return values

- **7:** bit Watchdog Counter value

Reference Manual to LL API cross reference:

- CR T LL_WWDG_GetCounter

LL_WWDG_SetPrescaler

Function name

__STATIC_INLINE void LL_WWDG_SetPrescaler (WWDG_TypeDef * WWDGx, uint32_t Prescaler)

Function description

Set the time base of the prescaler (WDGTB).

Parameters

- **WWDGx:** WWDG Instance
- **Prescaler:** This parameter can be one of the following values:
 - LL_WWDG_PRESCALER_1
 - LL_WWDG_PRESCALER_2
 - LL_WWDG_PRESCALER_4
 - LL_WWDG_PRESCALER_8

Return values

- **None:**

Notes

- Prescaler is used to apply ratio on PCLK clock, so that Watchdog counter is decremented every (4096 x 2expWDGTB) PCLK cycles

Reference Manual to LL API cross reference:

- CFR WDG TB LL_WWDG_SetPrescaler

LL_WWDG_GetPrescaler

Function name

__STATIC_INLINE uint32_t LL_WWDG_GetPrescaler (WWDG_TypeDef * WWDGx)

Function description

Return current Watchdog Prescaler Value.

Parameters

- WWDGx:** WWDG Instance

Return values

- Returned:** value can be one of the following values:
 - LL_WWDG_PRESCALER_1
 - LL_WWDG_PRESCALER_2
 - LL_WWDG_PRESCALER_4
 - LL_WWDG_PRESCALER_8

Reference Manual to LL API cross reference:

- CFR WDG TB LL_WWDG_GetPrescaler

LL_WWDG_SetWindow

Function name

__STATIC_INLINE void LL_WWDG_SetWindow (WWDG_TypeDef * WWDGx, uint32_t Window)

Function description

Set the Watchdog Window value to be compared to the downcounter (7-bits W[6:0]).

Parameters

- WWDGx:** WWDG Instance
- Window:** 0x00..0x7F (7 bit Window value)

Return values

- None:**

Notes

- This window value defines when write in the WWDG_CR register to program Watchdog counter is allowed. Watchdog counter value update must occur only when the counter value is lower than the Watchdog window register value. Otherwise, a MCU reset is generated if the 7-bit Watchdog counter value (in the control register) is refreshed before the downcounter has reached the watchdog window register value. Physically is possible to set the Window lower then 0x40 but it is not recommended. To generate an immediate reset, it is possible to set the Counter lower than 0x40.

Reference Manual to LL API cross reference:

- CFR W LL_WWDG_SetWindow

LL_WWDG_GetWindow

Function name

__STATIC_INLINE uint32_t LL_WWDG_GetWindow (WWDG_TypeDef * WWDGx)

Function description

Return current Watchdog Window Value (7 bits value)

Parameters

- **WWDGx:** WWDG Instance

Return values

- **7:** bit Watchdog Window value

Reference Manual to LL API cross reference:

- CFR W LL_WWDG_GetWindow
LL_WWDG_IsActiveFlag_EWKUP

Function name

__STATIC_INLINE uint32_t LL_WWDG_IsActiveFlag_EWKUP (WWDG_TypeDef * WWDGx)

Function description

Indicates if the WWDG Early Wakeup Interrupt Flag is set or not.

Parameters

- **WWDGx:** WWDG Instance

Return values

- **State:** of bit (1 or 0).

Notes

- This bit is set by hardware when the counter has reached the value 0x40. It must be cleared by software by writing 0. A write of 1 has no effect. This bit is also set if the interrupt is not enabled.

Reference Manual to LL API cross reference:

- SR EWIF LL_WWDG_IsActiveFlag_EWKUP
LL_WWDG_ClearFlag_EWKUP

Function name

__STATIC_INLINE void LL_WWDG_ClearFlag_EWKUP (WWDG_TypeDef * WWDGx)

Function description

Clear WWDG Early Wakeup Interrupt Flag (EWIF)

Parameters

- **WWDGx:** WWDG Instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR EWIF LL_WWDG_ClearFlag_EWKUP
LL_WWDG_EnableIT_EWKUP

Function name

__STATIC_INLINE void LL_WWDG_EnableIT_EWKUP (WWDG_TypeDef * WWDGx)

Function description

Enable the Early Wakeup Interrupt.

Parameters

- **WWDGx:** WWDG Instance

Return values

- **None:**

Notes

- When set, an interrupt occurs whenever the counter reaches value 0x40. This interrupt is only cleared by hardware after a reset

Reference Manual to LL API cross reference:

- CFR EWI LL_WWDG_EnableIT_EWKUP
LL_WWDG_IsEnabledIT_EWKUP

Function name

__STATIC_INLINE uint32_t LL_WWDG_IsEnabledIT_EWKUP (WWDG_TypeDef * WWDGx)

Function description

Check if Early Wakeup Interrupt is enabled.

Parameters

- **WWDGx:** WWDG Instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CFR EWI LL_WWDG_IsEnabledIT_EWKUP

68.2 WWDG Firmware driver defines

The following section lists the various define and macros of the module.

68.2.1 WWDG

WWDG

IT Defines

LL_WWDG_CFR_EWI

PRESCALER

LL_WWDG_PRESCALER_1

WWDG counter clock = (PCLK1/4096)/1

LL_WWDG_PRESCALER_2

WWDG counter clock = (PCLK1/4096)/2

LL_WWDG_PRESCALER_4

WWDG counter clock = (PCLK1/4096)/4

LL_WWDG_PRESCALER_8

WWDG counter clock = (PCLK1/4096)/8

Common Write and read registers macros

LL_WWDG_WriteReg

Description:

- Write a value in WWDG register.

Parameters:

- `__INSTANCE__`: WWDG Instance
- `__REG__`: Register to be written
- `__VALUE__`: Value to be written in the register

Return value:

- None

LL_WWDG_ReadReg

Description:

- Read a value in WWDG register.

Parameters:

- `__INSTANCE__`: WWDG Instance
- `__REG__`: Register to be read

Return value:

- Register: value

69 FAQs

General subjects

Why should I use the HAL drivers?

There are many advantages in using the HAL drivers:

- Ease of use: you can use the HAL drivers to configure and control any peripheral embedded within your STM32 MCU without prior in-depth knowledge of the product.
- HAL drivers provide intuitive and ready-to-use APIs to configure the peripherals and support polling, interrupt and DMA programming model to accommodate all application requirements, thus allowing the end-user to build a complete application by calling a few APIs.
- Higher level of abstraction than a standard peripheral library allowing to transparently manage:
 - Data transfers and processing using blocking mode (polling) or non-blocking mode (interrupt or DMA)
 - Error management through peripheral error detection and timeout mechanism.
- Generic architecture speeding up initialization and porting, thus allowing customers to focus on innovation.
- Generic set of APIs with full compatibility across the STM32 Series/lines, to ease the porting task between STM32 MCUs.
- The APIs provided within the HAL drivers are feature-oriented and do not require in-depth knowledge of peripheral operation.
- The APIs provided are modular. They include initialization, IO operation and control functions. The end-user has to call init function, then start the process by calling one IO operation functions (write, read, transmit, receive, ...). Most of the peripherals have the same architecture.
- The number of functions required to build a complete and useful application is very reduced. As an example, to build a UART communication process, the user only has to call HAL_UART_Init() then HAL_UART_Transmit() or HAL_UART_Receive().

Which devices are supported by the HAL drivers?

The HAL drivers are developed to support all STM32F2 devices. To ensure compatibility between all devices and portability with others Series and lines, the API is split into the generic and the extension APIs. For more details, please refer to *section Devices supported by the HAL drivers*.

What is the cost of using HAL drivers in term of code size and performance?

Like generic architecture drivers, the HAL drivers may induce firmware overhead.

This is due to the high abstraction level and ready-to-use APIs which allow data transfers, errors management and offloads the user application from implementation details.

Architecture

How many files should I modify to configure the HAL drivers?

Only one file needs to be modified: stm32f2xx_hal_conf.h. You can modify this file by disabling unused modules, or adjusting some parameters (i.e. HSE value, System configuration...)

A template is provided in the HAL drivers folders (stm32f2xx_hal_conf_template.c).

Which header files should I include in my application to use the HAL drivers?

Only stm32f2xx_hal.h file has to be included.

What is the difference between xx_hal_ppp.c/h and xx_hal_ppp_ex.c/h?

The HAL driver architecture supports common features across STM32 Series/lines. To support specific features, the drivers are split into two groups.

- The generic APIs (stm32f2xx_hal_ppp.c): It includes the common set of APIs across all the STM32 product lines

- The extension APIs (stm32f2xx_hal_ppp_ex.c): It includes the specific APIs for specific device part number or family.

Initialization and I/O operation functions

How do I configure the system clock?

Unlike the standard library, the system clock configuration is not performed in CMSIS drivers file (system_stm32f2xx.c) but in the main user application by calling the two main functions, HAL_RCC_OscConfig() and HAL_RCC_ClockConfig(). It can be modified in any user application section.

What is the purpose of the *PPP_HandleTypeDef *pHandle* structure located in each driver in addition to the Initialization structure

*PPP_HandleTypeDef *pHandle* is the main structure implemented in the HAL drivers. It handles the peripheral configuration and registers, and embeds all the structures and variables required to follow the peripheral device flow (pointer to buffer, Error code, State,...)

However, this structure is not required to service peripherals such as GPIO, SYSTICK, PWR, and RCC.

What is the purpose of HAL_PPP_MspInit() and HAL_PPP_MspDeInit() functions?

These functions are called within HAL_PPP_Init() and HAL_PPP_DeInit(), respectively. They are used to perform the low level Initialization/de-initialization related to the additional hardware resources (RCC, GPIO, NVIC and DMA).

These functions are declared in stm32f2xx_hal_msp.c. A template is provided in the HAL driver folders (stm32f2xx_hal_msp_template.c).

When and how should I use callbacks functions (functions declared with the attribute *__weak*)?

Use callback functions for the I/O operations used in DMA or interrupt mode. The PPP process complete callbacks are called to inform the user about process completion in real-time event mode (interrupts).

The Errors callbacks are called when a processing error occurs in DMA or interrupt mode. These callbacks are customized by the user to add user proprietary code. They can be declared in the application. Note that the same process completion callbacks are used for DMA and interrupt mode.

Is it mandatory to use HAL_Init() function at the beginning of the user application?

It is mandatory to use HAL_Init() function to enable the system configuration (Prefetch, Data instruction cache,...), configure the SysTick and the NVIC priority grouping and the hardware low level initialization.

The SysTick configuration shall be adjusted by calling **HAL_RCC_ClockConfig()** function, to obtain 1 ms whatever the system clock.

Why do I need to configure the SysTick timer to use the HAL drivers?

The SysTick timer is configured to be used to generate variable increments by calling *HAL_IncTick()* function in SysTick ISR and retrieve the value of this variable by calling *HAL_GetTick()* function.

The call HAL_GetTick() function is mandatory when using HAL drivers with Polling Process or when using HAL_Delay().

Why is the SysTick timer configured to have 1 ms?

This is mandatory to ensure correct IO operation in particular for polling mode operation where the 1 ms is required as timebase.

Could HAL_Delay() function block my application under certain conditions?

Care must be taken when using HAL_Delay() since this function provides accurate delay based on a variable incremented in SysTick ISR. This implies that if HAL_Delay() is called from a peripheral ISR process, then the SysTick interrupt must have higher priority (numerically lower) than the peripheral interrupt, otherwise the caller ISR process will be blocked. Use HAL_NVIC_SetPriority() function to change the SysTick interrupt priority.

What programming model sequence should I follow to use HAL drivers ?

Follow the sequence below to use the APIs provided in the HAL drivers:

1. Call HAL_Init() function to initialize the system (data cache, NVIC priority,...).
2. Initialize the system clock by calling HAL_RCC_OscConfig() followed by HAL_RCC_ClockConfig().
3. Add HAL_IncTick() function under SysTick_Handler() ISR function to enable polling process when using HAL_Delay() function
4. Start initializing your peripheral by calling HAL_PPP_Init().
5. Implement the hardware low level initialization (Peripheral clock, GPIO, DMA,...) by calling HAL_PPP_MspInit() instm32f2xx_hal_msp.c
6. Start your process operation by calling IO operation functions.

What is the purpose of HAL_PPP_IRQHandler() function and when should I use it?

HAL_PPP_IRQHandler() is used to handle interrupt process. It is called under PPP_IRQHandler() function in stm32f2xx_it.c. In this case, the end-user has to implement only the callbacks functions (prefixed by __weak) to perform the appropriate action when an interrupt is detected. Advanced users can implement their own code in PPP_IRQHandler() without calling HAL_PPP_IRQHandler().

Can I use directly the macros defined in xx_hal_ppp.h ?

Yes, you can: a set of macros is provided with the APIs. They allow accessing directly some specific features using peripheral flags.

Where must PPP_HandleTypeDef structure peripheral handler be declared?

PPP_HandleTypeDef structure peripheral handler must be declared as a global variable, so that all the structure fields are set to 0 by default. In this way, the peripheral handler default state are set to HAL_PPP_STATE_RESET, which is the default state for each peripheral after a system reset.

When should I use HAL versus LL drivers?

HAL drivers offer high-level and function-oriented APIs, with a high level of portability. Product/IPs complexity is hidden for end users. LL drivers offer low-level APIs at registers level, with a better optimization but less portability. They require a deep knowledge of product/IPs specifications.

How can I include LL drivers in my environment? Is there any LL configuration file as for HAL?

There is no configuration file. Source code shall directly include the necessary stm32f2xx_ll_ppp.h file(s).

Can I use HAL and LL drivers together? If yes, what are the constraints?

It is possible to use both HAL and LL drivers. One can handle the IP initialization phase with HAL and then manage the I/O operations with LL drivers. The major difference between HAL and LL is that HAL drivers require to create and use handles for operation management while LL drivers operates directly on peripheral registers. Mixing HAL and LL is illustrated in Examples_MIX example.

Is there any LL APIs which are not available with HAL?

Yes, there are. A few Cortex® APIs have been added in stm32f2xx_ll_cortex.h e.g. for accessing SCB or SysTick registers.

Why are SysTick interrupts not enabled on LL drivers?

When using LL drivers in standalone mode, you do not need to enable SysTick interrupts because they are not used in LL APIs, while HAL functions requires SysTick interrupts to manage timeouts.

Revision history

Table 25. Document revision history

Date	Revision	Changes
29-Sep-2015	1	Initial release.
10-Mar-2017	2	Removed list of reference documents in Introduction. Updated Table 1 to add new supported part numbers and LL drivers. Added description of LL Generic drivers.
03-Feb-2020	3	Minor update of Section Introduction . Added Section 1 General information . List of acronyms made generic in Section 2 Acronyms and definitions . Removed stm32f2xx_ll_ppp.c and stm32f2xx_ll_ppp.c from Table 2. HAL driver files as well as note related to low-layer drivers. Added note related to family specific APIs and IRQ handlers in Section 3.3 API classification . Updated Section 3.7.2 HAL extension model cases . Updated Section 69 FAQs .

Contents

1	General information	3
2	Acronyms and definitions	4
3	Overview of HAL drivers	7
3.1	HAL and user-application files	7
3.1.1	HAL driver files	7
3.1.2	User-application files	8
3.2	HAL data structures	9
3.2.1	Peripheral handle structures	9
3.2.2	Initialization and configuration structure	10
3.2.3	Specific process structures	11
3.3	API classification	12
3.4	Devices supported by HAL drivers	13
3.5	HAL driver rules	15
3.5.1	HAL API naming rules	15
3.5.2	HAL general naming rules	16
3.5.3	HAL interrupt handler and callback functions	17
3.6	HAL generic APIs	18
3.7	HAL extension APIs	19
3.7.1	HAL extension model overview	19
3.7.2	HAL extension model cases	19
3.8	File inclusion model	22
3.9	HAL common resources	22
3.10	HAL configuration	23
3.11	HAL system peripheral handling	24
3.11.1	Clock	24
3.11.2	GPIOs	25
3.11.3	Cortex® NVIC and SysTick timer	26
3.11.4	PWR	26
3.11.5	EXTI	26
3.11.6	DMA	27

3.12	How to use HAL drivers	28
3.12.1	HAL usage models	28
3.12.2	HAL initialization	29
3.12.3	HAL I/O operation process	31
3.12.4	Timeout and error management	34
4	Overview of low-layer drivers	38
4.1	Low-layer files	38
4.2	Overview of low-layer APIs and naming rules	40
4.2.1	Peripheral initialization functions	40
4.2.2	Peripheral register-level configuration functions	42
5	Cohabiting of HAL and LL	44
5.1	Low-layer driver used in Standalone mode	44
5.2	Mixed use of low-layer APIs and HAL drivers	44
6	HAL System Driver	45
6.1	HAL Firmware driver API description	45
6.1.1	How to use this driver	45
6.1.2	Initialization and de-initialization functions	45
6.1.3	HAL Control functions	45
6.1.4	Detailed description of functions	46
6.2	HAL Firmware driver defines	52
6.2.1	HAL	52
7	HAL ADC Generic Driver	55
7.1	ADC Firmware driver registers structures	55
7.1.1	ADC_InitTypeDef	55
7.1.2	ADC_ChannelConfTypeDef	56
7.1.3	ADC_AnalogWDGConfTypeDef	57
7.1.4	ADC_HandleTypeDef	57
7.2	ADC Firmware driver API description	58
7.2.1	ADC Peripheral features	58
7.2.2	How to use this driver	58
7.2.3	Initialization and de-initialization functions	61

7.2.4	IO operation functions	61
7.2.5	Peripheral Control functions	62
7.2.6	Peripheral State and errors functions	62
7.2.7	Detailed description of functions	62
7.3	ADC Firmware driver defines.	69
7.3.1	ADC	69
8	HAL ADC Extension Driver.	76
8.1	ADCEX Firmware driver registers structures	76
8.1.1	ADC_InjectionConfTypeDef	76
8.1.2	ADC_MultiModeTypeDef.	77
8.2	ADCEX Firmware driver API description.	77
8.2.1	How to use this driver	78
8.2.2	Extended features functions	79
8.2.3	Detailed description of functions	79
8.3	ADCEX Firmware driver defines	83
8.3.1	ADCEX	83
9	HAL CAN Generic Driver.	86
9.1	CAN Firmware driver registers structures	86
9.1.1	CAN_InitTypeDef	86
9.1.2	CAN_FilterTypeDef.	86
9.1.3	CAN_TxHeaderTypeDef	87
9.1.4	CAN_RxHeaderTypeDef.	88
9.1.5	__CAN_HandleTypeDef	89
9.2	CAN Firmware driver API description	89
9.2.1	How to use this driver	89
9.2.2	Initialization and de-initialization functions.	90
9.2.3	Configuration functions	91
9.2.4	Control functions	91
9.2.5	Interrupts management.	91
9.2.6	Peripheral State and Error functions	92
9.2.7	Detailed description of functions	92
9.3	CAN Firmware driver defines.	102

9.3.1	CAN	102
10	HAL CORTEX Generic Driver.....	111
10.1	CORTEX Firmware driver registers structures	111
10.1.1	MPU_Region_InitTypeDef	111
10.2	CORTEX Firmware driver API description	112
10.2.1	How to use this driver	112
10.2.2	Initialization and de-initialization functions	112
10.2.3	Peripheral Control functions	113
10.2.4	Detailed description of functions	113
10.3	CORTEX Firmware driver defines	118
10.3.1	CORTEX	118
11	HAL CRC Generic Driver.....	122
11.1	CRC Firmware driver registers structures	122
11.1.1	CRC_HandleTypeDef	122
11.2	CRC Firmware driver API description	122
11.2.1	How to use this driver	122
11.2.2	Initialization and de-initialization functions	122
11.2.3	Peripheral Control functions	122
11.2.4	Peripheral State functions	123
11.2.5	Detailed description of functions	123
11.3	CRC Firmware driver defines	125
11.3.1	CRC	125
12	HAL DAC Generic Driver.....	127
12.1	DAC Firmware driver registers structures	127
12.1.1	DAC_HandleTypeDef	127
12.1.2	DAC_ChannelConfTypeDef	127
12.2	DAC Firmware driver API description	127
12.2.1	DAC Peripheral features	127
12.2.2	How to use this driver	129
12.2.3	Initialization and de-initialization functions	130
12.2.4	IO operation functions	130

12.2.5	Peripheral Control functions	131
12.2.6	Peripheral State and Errors functions	131
12.2.7	Detailed description of functions	131
12.3	DAC Firmware driver defines	138
12.3.1	DAC	138
13	HAL DAC Extension Driver	142
13.1	DACEx Firmware driver API description	142
13.1.1	How to use this driver	142
13.1.2	Extended features functions	142
13.1.3	Peripheral Control functions	142
13.1.4	Detailed description of functions	142
13.2	DACEx Firmware driver defines	146
13.2.1	DACEx	146
14	HAL DCMI Generic Driver	148
14.1	DCMI Firmware driver registers structures	148
14.1.1	DCMI_CodesInitTypeDef	148
14.1.2	DCMI_SyncUnmaskTypeDef	148
14.1.3	DCMI_InitTypeDef	148
14.1.4	__DCMI_HandleTypeDef	149
14.2	DCMI Firmware driver API description	150
14.2.1	How to use this driver	150
14.2.2	Initialization and Configuration functions	150
14.2.3	IO operation functions	151
14.2.4	Peripheral Control functions	151
14.2.5	Peripheral State and Errors functions	151
14.2.6	Detailed description of functions	151
14.3	DCMI Firmware driver defines	156
14.3.1	DCMI	156
15	HAL DMA Generic Driver	162
15.1	DMA Firmware driver registers structures	162
15.1.1	DMA_InitTypeDef	162
15.1.2	__DMA_HandleTypeDef	163

15.2	DMA Firmware driver API description.	164
15.2.1	How to use this driver	164
15.2.2	Initialization and de-initialization functions.	165
15.2.3	IO operation functions.	165
15.2.4	State and Errors functions.	165
15.2.5	Detailed description of functions	166
15.3	DMA Firmware driver defines	169
15.3.1	DMA	169
16	HAL DMA Extension Driver	174
16.1	DMAEx Firmware driver API description	174
16.1.1	How to use this driver	174
16.1.2	Extended features functions	174
16.1.3	Detailed description of functions	174
17	HAL ETH Generic Driver	176
17.1	ETH Firmware driver registers structures.	176
17.1.1	ETH_InitTypeDef.	176
17.1.2	ETH_MACInitTypeDef.	176
17.1.3	ETH_DMAInitTypeDef.	179
17.1.4	ETH_DMADescTypeDef	180
17.1.5	ETH_DMARxFramelInfos	181
17.1.6	ETH_HandleTypeDef	181
17.2	ETH Firmware driver API description	182
17.2.1	How to use this driver	182
17.2.2	Initialization and de-initialization functions.	182
17.2.3	IO operation functions.	183
17.2.4	Peripheral Control functions	183
17.2.5	Peripheral State functions	183
17.2.6	Detailed description of functions	183
17.3	ETH Firmware driver defines	189
17.3.1	ETH	189
18	HAL EXTI Generic Driver.	218
18.1	EXTI Firmware driver registers structures	218

18.1.1	EXTI_HandleTypeDef	218
18.1.2	EXTI_ConfigTypeDef	218
18.2	EXTI Firmware driver API description.	218
18.2.1	EXTI Peripheral features	218
18.2.2	How to use this driver	219
18.2.3	Configuration functions	219
18.2.4	Detailed description of functions	219
18.3	EXTI Firmware driver defines	222
18.3.1	EXTI	222
19	HAL FLASH Generic Driver	225
19.1	FLASH Firmware driver registers structures	225
19.1.1	FLASH_ProcessTypeDef	225
19.2	FLASH Firmware driver API description.	225
19.2.1	FLASH peripheral features	225
19.2.2	How to use this driver	225
19.2.3	Programming operation functions	226
19.2.4	Peripheral Control functions	226
19.2.5	Peripheral Errors functions	226
19.2.6	Detailed description of functions	226
19.3	FLASH Firmware driver defines	229
19.3.1	FLASH	229
20	HAL FLASH Extension Driver	235
20.1	FLASHEx Firmware driver registers structures	235
20.1.1	FLASH_EraseInitTypeDef	235
20.1.2	FLASH_OBProgramInitTypeDef	235
20.2	FLASHEx Firmware driver API description	236
20.2.1	Flash Extension features.	236
20.2.2	How to use this driver	236
20.2.3	Extended programming operation functions	236
20.2.4	Detailed description of functions	236
20.3	FLASHEx Firmware driver defines	238
20.3.1	FLASHEx	238

21	HAL GPIO Generic Driver	243
21.1	GPIO Firmware driver registers structures	243
21.1.1	GPIO_InitTypeDef	243
21.2	GPIO Firmware driver API description	243
21.2.1	GPIO Peripheral features	243
21.2.2	How to use this driver	243
21.2.3	Initialization and de-initialization functions	244
21.2.4	IO operation functions	244
21.2.5	Detailed description of functions	244
21.3	GPIO Firmware driver defines	247
21.3.1	GPIO	247
22	HAL GPIO Extension Driver	252
22.1	GPIOEx Firmware driver defines	252
22.1.1	GPIOEx	252
23	HAL HASH Generic Driver	253
23.1	HASH Firmware driver registers structures	253
23.1.1	HASH_InitTypeDef	253
23.1.2	HASH_HandleTypeDef	253
23.2	HASH Firmware driver API description	254
23.2.1	How to use this driver	254
23.2.2	Initialization and de-initialization functions	256
23.2.3	Polling mode HASH processing functions	257
23.2.4	Interrupt mode HASH processing functions	257
23.2.5	DMA mode HASH processing functions	257
23.2.6	Polling mode HMAC processing functions	258
23.2.7	Interrupt mode HMAC processing functions	258
23.2.8	DMA mode HMAC processing functions	258
23.2.9	Peripheral State methods	258
23.2.10	Detailed description of functions	259
23.3	HASH Firmware driver defines	274
23.3.1	HASH	274

24	HAL HCD Generic Driver	278
24.1	HCD Firmware driver registers structures	278
24.1.1	HCD_HandleTypeDef	278
24.2	HCD Firmware driver API description	278
24.2.1	How to use this driver	278
24.2.2	Initialization and de-initialization functions	279
24.2.3	IO operation functions	279
24.2.4	Peripheral Control functions	279
24.2.5	Peripheral State functions	279
24.2.6	Detailed description of functions	279
24.3	HCD Firmware driver defines	286
24.3.1	HCD	286
25	HAL I2C Generic Driver	287
25.1	I2C Firmware driver registers structures	287
25.1.1	I2C_InitTypeDef	287
25.1.2	__I2C_HandleTypeDef	287
25.2	I2C Firmware driver API description	288
25.2.1	How to use this driver	288
25.2.2	Initialization and de-initialization functions	293
25.2.3	IO operation functions	294
25.2.4	Peripheral State, Mode and Error functions	295
25.2.5	Detailed description of functions	295
25.3	I2C Firmware driver defines	312
25.3.1	I2C	312
26	HAL I2S Generic Driver	318
26.1	I2S Firmware driver registers structures	318
26.1.1	I2S_InitTypeDef	318
26.1.2	I2S_HandleTypeDef	318
26.2	I2S Firmware driver API description	319
26.2.1	How to use this driver	319
26.2.2	Initialization and de-initialization functions	322

26.2.3	IO operation functions	322
26.2.4	Peripheral State and Errors functions	323
26.2.5	Detailed description of functions	323
26.3	I2S Firmware driver defines	329
26.3.1	I2S	329
27	HAL IRDA Generic Driver	335
27.1	IRDA Firmware driver registers structures	335
27.1.1	IRDA_InitTypeDef	335
27.1.2	IRDA_HandleTypeDef	335
27.2	IRDA Firmware driver API description	336
27.2.1	How to use this driver	336
27.2.2	Callback registration	338
27.2.3	Initialization and Configuration functions	339
27.2.4	IO operation functions	339
27.2.5	Peripheral State and Errors functions	342
27.2.6	Detailed description of functions	342
27.3	IRDA Firmware driver defines	351
27.3.1	IRDA	351
28	HAL IWDG Generic Driver	358
28.1	IWDG Firmware driver registers structures	358
28.1.1	IWDG_InitTypeDef	358
28.1.2	IWDG_HandleTypeDef	358
28.2	IWDG Firmware driver API description	358
28.2.1	IWDG Generic features	358
28.2.2	How to use this driver	359
28.2.3	Initialization and Start functions	359
28.2.4	IO operation functions	359
28.2.5	Detailed description of functions	359
28.3	IWDG Firmware driver defines	360
28.3.1	IWDG	360
29	HAL NAND Generic Driver	362

29.1	NAND Firmware driver registers structures	362
29.1.1	NAND_IDTypeDef	362
29.1.2	NAND_AddressTypeDef	362
29.1.3	NAND_DeviceConfigTypeDef	362
29.1.4	NAND_HandleTypeDef	363
29.2	NAND Firmware driver API description	363
29.2.1	How to use this driver	363
29.2.2	NAND Initialization and de-initialization functions	364
29.2.3	NAND Input and Output functions	364
29.2.4	NAND Control functions	365
29.2.5	NAND State functions	365
29.2.6	Detailed description of functions	365
29.3	NAND Firmware driver defines	372
29.3.1	NAND	372
30	HAL NOR Generic Driver	374
30.1	NOR Firmware driver registers structures	374
30.1.1	NOR_IDTypeDef	374
30.1.2	NOR_CFTypeDef	374
30.1.3	NOR_HandleTypeDef	374
30.2	NOR Firmware driver API description	375
30.2.1	How to use this driver	375
30.2.2	NOR Initialization and de_initialization functions	376
30.2.3	NOR Input and Output functions	376
30.2.4	NOR Control functions	376
30.2.5	NOR State functions	376
30.2.6	Detailed description of functions	376
30.3	NOR Firmware driver defines	382
30.3.1	NOR	382
31	HAL PCCARD Generic Driver	383
31.1	PCCARD Firmware driver registers structures	383
31.1.1	PCCARD_HandleTypeDef	383
31.2	PCCARD Firmware driver API description	383

31.2.1	How to use this driver	383
31.2.2	PCCARD Initialization and de-initialization functions	384
31.2.3	PCCARD Input and Output functions	384
31.2.4	PCCARD State functions	384
31.2.5	Detailed description of functions	384
31.3	PCCARD Firmware driver defines	389
31.3.1	PCCARD	389
32	HAL PCD Generic Driver	390
32.1	PCD Firmware driver registers structures	390
32.1.1	PCD_HandleTypeDef	390
32.2	PCD Firmware driver API description	390
32.2.1	How to use this driver	390
32.2.2	Initialization and de-initialization functions	391
32.2.3	IO operation functions	391
32.2.4	Peripheral Control functions	391
32.2.5	Peripheral State functions	392
32.2.6	Detailed description of functions	392
32.3	PCD Firmware driver defines	400
32.3.1	PCD	400
33	HAL PCD Extension Driver	402
33.1	PCDEx Firmware driver API description	402
33.1.1	Extended features functions	402
33.1.2	Detailed description of functions	402
34	HAL PWR Generic Driver	404
34.1	PWR Firmware driver registers structures	404
34.1.1	PWR_PVDTypeDef	404
34.2	PWR Firmware driver API description	404
34.2.1	Initialization and de-initialization functions	404
34.2.2	Peripheral Control functions	404
34.2.3	Detailed description of functions	406
34.3	PWR Firmware driver defines	411

34.3.1	PWR	411
35	HAL PWR Extension Driver	416
35.1	PWREx Firmware driver API description	416
35.1.1	Peripheral extended features functions	416
35.1.2	Detailed description of functions	416
35.2	PWREx Firmware driver defines	417
35.2.1	PWREx	417
36	HAL RCC Generic Driver	418
36.1	RCC Firmware driver registers structures	418
36.1.1	RCC_PLLInitTypeDef	418
36.1.2	RCC_OscInitTypeDef	418
36.1.3	RCC_ClkInitTypeDef	419
36.2	RCC Firmware driver API description	419
36.2.1	RCC specific features	419
36.2.2	RCC Limitations	420
36.2.3	Initialization and de-initialization functions	420
36.2.4	Peripheral Control functions	421
36.2.5	Detailed description of functions	421
36.3	RCC Firmware driver defines	426
36.3.1	RCC	426
37	HAL RCC Extension Driver	458
37.1	RCCEx Firmware driver registers structures	458
37.1.1	RCC_PLLI2SInitTypeDef	458
37.1.2	RCC_PeriphCLKInitTypeDef	458
37.2	RCCEx Firmware driver API description	458
37.2.1	Extended Peripheral Control functions	458
37.2.2	Detailed description of functions	459
37.3	RCCEx Firmware driver defines	459
37.3.1	RCCEx	459
38	HAL RNG Generic Driver	463
38.1	RNG Firmware driver registers structures	463

38.1.1	RNG_HandleTypeDef	463
38.2	RNG Firmware driver API description	463
38.2.1	How to use this driver	463
38.2.2	Callback registration	463
38.2.3	Initialization and configuration functions	464
38.2.4	Peripheral Control functions	464
38.2.5	Peripheral State functions	464
38.2.6	Detailed description of functions	464
38.3	RNG Firmware driver defines	468
38.3.1	RNG	468
39	HAL RTC Generic Driver	472
39.1	RTC Firmware driver registers structures	472
39.1.1	RTC_InitTypeDef	472
39.1.2	RTC_TimeTypeDef	472
39.1.3	RTC_DateTypeDef	473
39.1.4	RTC_AlarmTypeDef	473
39.1.5	RTC_HandleTypeDef	474
39.2	RTC Firmware driver API description	474
39.2.1	Backup Domain Operating Condition	474
39.2.2	Backup Domain Reset	474
39.2.3	Backup Domain Access	474
39.2.4	How to use this driver	475
39.2.5	RTC and low power modes	475
39.2.6	Initialization and de-initialization functions	476
39.2.7	RTC Time and Date functions	476
39.2.8	RTC Alarm functions	476
39.2.9	Peripheral Control functions	477
39.2.10	Peripheral State functions	477
39.2.11	Detailed description of functions	477
39.3	RTC Firmware driver defines	483
39.3.1	RTC	483
40	HAL RTC Extension Driver	493

40.1	RTCEx Firmware driver registers structures	493
40.1.1	RTC_TamperTypeDef	493
40.2	RTCEx Firmware driver API description	493
40.2.1	How to use this driver	493
40.2.2	RTC TimeStamp and Tamper functions	494
40.2.3	RTC Wake-up functions	494
40.2.4	Extension Peripheral Control functions	494
40.2.5	Extended features functions	495
40.2.6	Detailed description of functions	495
40.3	RTCEx Firmware driver defines	503
40.3.1	RTCEx	503
41	HAL SMARTCARD Generic Driver	516
41.1	SMARTCARD Firmware driver registers structures	516
41.1.1	SMARTCARD_InitTypeDef	516
41.1.2	__SMARTCARD_HandleTypeDef	517
41.2	SMARTCARD Firmware driver API description	518
41.2.1	How to use this driver	518
41.2.2	Callback registration	519
41.2.3	Initialization and Configuration functions	520
41.2.4	IO operation functions	521
41.2.5	Peripheral State and Errors functions	523
41.2.6	Detailed description of functions	523
41.3	SMARTCARD Firmware driver defines	531
41.3.1	SMARTCARD	531
42	HAL SPI Generic Driver	540
42.1	SPI Firmware driver registers structures	540
42.1.1	SPI_InitTypeDef	540
42.1.2	__SPI_HandleTypeDef	540
42.2	SPI Firmware driver API description	542
42.2.1	How to use this driver	542
42.2.2	Initialization and de-initialization functions	543
42.2.3	IO operation functions	544

42.2.4	Peripheral State and Errors functions	545
42.2.5	Detailed description of functions	545
42.3	SPI Firmware driver defines	553
42.3.1	SPI	553
43	HAL SRAM Generic Driver	559
43.1	SRAM Firmware driver registers structures	559
43.1.1	SRAM_HandleTypeDef	559
43.2	SRAM Firmware driver API description	559
43.2.1	How to use this driver	559
43.2.2	SRAM Initialization and de_initialization functions	560
43.2.3	SRAM Input and Output functions	560
43.2.4	SRAM Control functions	561
43.2.5	SRAM State functions	561
43.2.6	Detailed description of functions	561
43.3	SRAM Firmware driver defines	566
43.3.1	SRAM	566
44	HAL TIM Generic Driver	567
44.1	TIM Firmware driver registers structures	567
44.1.1	TIM_Base_InitTypeDef	567
44.1.2	TIM_OC_InitTypeDef	567
44.1.3	TIM_OnePulse_InitTypeDef	568
44.1.4	TIM_IC_InitTypeDef	569
44.1.5	TIM_Encoder_InitTypeDef	569
44.1.6	TIM_ClockConfigTypeDef	570
44.1.7	TIM_ClearInputConfigTypeDef	570
44.1.8	TIM_MasterConfigTypeDef	571
44.1.9	TIM_SlaveConfigTypeDef	571
44.1.10	TIM_BreakDeadTimeConfigTypeDef	571
44.1.11	TIM_HandleTypeDef	572
44.2	TIM Firmware driver API description	572
44.2.1	TIMER Generic features	573
44.2.2	How to use this driver	573

44.2.3	Time Base functions	575
44.2.4	TIM Output Compare functions	575
44.2.5	TIM PWM functions	576
44.2.6	TIM Input Capture functions	576
44.2.7	TIM One Pulse functions	576
44.2.8	TIM Encoder functions	577
44.2.9	TIM Callbacks functions	577
44.2.10	Detailed description of functions	578
44.3	TIM Firmware driver defines	611
44.3.1	TIM	611
45	HAL TIM Extension Driver	635
45.1	TIMEx Firmware driver registers structures	635
45.1.1	TIM_HallSensor_InitTypeDef	635
45.2	TIMEx Firmware driver API description	635
45.2.1	TIMER Extended features	635
45.2.2	How to use this driver	635
45.2.3	Timer Hall Sensor functions	636
45.2.4	Timer Complementary Output Compare functions	636
45.2.5	Timer Complementary PWM functions	637
45.2.6	Timer Complementary One Pulse functions	637
45.2.7	Peripheral Control functions	637
45.2.8	Extended Callbacks functions	638
45.2.9	Extended Peripheral State functions	638
45.2.10	Detailed description of functions	638
45.3	TIMEx Firmware driver defines	651
45.3.1	TIMEx	651
46	HAL UART Generic Driver	652
46.1	UART Firmware driver registers structures	652
46.1.1	UART_InitTypeDef	652
46.1.2	__UART_HandleTypeDef	652
46.2	UART Firmware driver API description	653
46.2.1	How to use this driver	653

46.2.2	Callback registration	654
46.2.3	Initialization and Configuration functions	656
46.2.4	IO operation functions	656
46.2.5	Peripheral Control functions	657
46.2.6	Peripheral State and Errors functions	657
46.2.7	Detailed description of functions	657
46.3	UART Firmware driver defines	670
46.3.1	UART	670
47	HAL USART Generic Driver	679
47.1	USART Firmware driver registers structures	679
47.1.1	USART_InitTypeDef	679
47.1.2	__USART_HandleTypeDef	679
47.2	USART Firmware driver API description	680
47.2.1	How to use this driver	680
47.2.2	Callback registration	682
47.2.3	Initialization and Configuration functions	683
47.2.4	IO operation functions	683
47.2.5	Peripheral State and Errors functions	685
47.2.6	Detailed description of functions	685
47.3	USART Firmware driver defines	694
47.3.1	USART	694
48	HAL WWDG Generic Driver	701
48.1	WWDG Firmware driver registers structures	701
48.1.1	WWDG_InitTypeDef	701
48.1.2	WWDG_HandleTypeDef	701
48.2	WWDG Firmware driver API description	701
48.2.1	Initialization and Configuration functions	701
48.2.2	IO operation functions	701
48.2.3	Detailed description of functions	702
48.3	WWDG Firmware driver defines	703
48.3.1	WWDG	703
49	LL ADC Generic Driver	706

49.1	ADC Firmware driver registers structures	706
49.1.1	LL_ADC_CommonInitTypeDef	706
49.1.2	LL_ADC_InitTypeDef	706
49.1.3	LL_ADC_REG_InitTypeDef	706
49.1.4	LL_ADC_INJ_InitTypeDef	707
49.2	ADC Firmware driver API description	708
49.2.1	Detailed description of functions	708
49.3	ADC Firmware driver defines	772
49.3.1	ADC	772
50	LL BUS Generic Driver	804
50.1	BUS Firmware driver API description	804
50.1.1	Detailed description of functions	804
50.2	BUS Firmware driver defines	838
50.2.1	BUS	838
51	LL CORTEX Generic Driver	842
51.1	CORTEX Firmware driver API description	842
51.1.1	Detailed description of functions	842
51.2	CORTEX Firmware driver defines	850
51.2.1	CORTEX	850
52	LL CRC Generic Driver	855
52.1	CRC Firmware driver API description	855
52.1.1	Detailed description of functions	855
52.2	CRC Firmware driver defines	857
52.2.1	CRC	857
53	LL DAC Generic Driver	858
53.1	DAC Firmware driver registers structures	858
53.1.1	LL_DAC_InitTypeDef	858
53.2	DAC Firmware driver API description	858
53.2.1	Detailed description of functions	858
53.3	DAC Firmware driver defines	877
53.3.1	DAC	877

54	LL DMA Generic Driver	882
54.1	DMA Firmware driver registers structures	882
54.1.1	LL_DMA_InitTypeDef	882
54.2	DMA Firmware driver API description	883
54.2.1	Detailed description of functions	883
54.3	DMA Firmware driver defines	945
54.3.1	DMA	945
55	LL EXTI Generic Driver	951
55.1	EXTI Firmware driver registers structures	951
55.1.1	LL_EXTI_InitTypeDef	951
55.2	EXTI Firmware driver API description	951
55.2.1	Detailed description of functions	951
55.3	EXTI Firmware driver defines	968
55.3.1	EXTI	968
56	LL GPIO Generic Driver	971
56.1	GPIO Firmware driver registers structures	971
56.1.1	LL_GPIO_InitTypeDef	971
56.2	GPIO Firmware driver API description	971
56.2.1	Detailed description of functions	971
56.3	GPIO Firmware driver defines	991
56.3.1	GPIO	991
57	LL I2C Generic Driver	995
57.1	I2C Firmware driver registers structures	995
57.1.1	LL_I2C_InitTypeDef	995
57.2	I2C Firmware driver API description	995
57.2.1	Detailed description of functions	995
57.3	I2C Firmware driver defines	1032
57.3.1	I2C	1032
58	LL IWDG Generic Driver	1037
58.1	IWDG Firmware driver API description	1037
58.1.1	Detailed description of functions	1037

58.2	IWDG Firmware driver defines	1040
58.2.1	IWDG	1040
59	LL PWR Generic Driver	1042
59.1	PWR Firmware driver API description	1042
59.1.1	Detailed description of functions	1042
59.2	PWR Firmware driver defines	1049
59.2.1	PWR	1050
60	LL RCC Generic Driver	1052
60.1	RCC Firmware driver registers structures	1052
60.1.1	LL_RCC_ClocksTypeDef	1052
60.2	RCC Firmware driver API description	1052
60.2.1	Detailed description of functions	1052
60.3	RCC Firmware driver defines	1090
60.3.1	RCC	1090
61	LL RNG Generic Driver	1109
61.1	RNG Firmware driver API description	1109
61.1.1	Detailed description of functions	1109
61.2	RNG Firmware driver defines	1113
61.2.1	RNG	1113
62	LL RTC Generic Driver	1115
62.1	RTC Firmware driver registers structures	1115
62.1.1	LL_RTC_InitTypeDef	1115
62.1.2	LL_RTC_TimeTypeDef	1115
62.1.3	LL_RTC_DateTypeDef	1115
62.1.4	LL_RTC_AlarmTypeDef	1116
62.2	RTC Firmware driver API description	1116
62.2.1	Detailed description of functions	1116
62.3	RTC Firmware driver defines	1180
62.3.1	RTC	1180
63	LL SPI Generic Driver	1190
63.1	SPI Firmware driver registers structures	1190

63.1.1	LL_SPI_InitTypeDef	1190
63.1.2	LL_I2S_InitTypeDef.	1191
63.2	SPI Firmware driver API description	1191
63.2.1	Detailed description of functions	1191
63.3	SPI Firmware driver defines.	1228
63.3.1	SPI	1228
64	LL SYSTEM Generic Driver	1232
64.1	SYSTEM Firmware driver API description	1232
64.1.1	Detailed description of functions	1232
64.2	SYSTEM Firmware driver defines.	1245
64.2.1	SYSTEM.	1245
65	LL TIM Generic Driver.	1250
65.1	TIM Firmware driver registers structures	1250
65.1.1	LL_TIM_InitTypeDef	1250
65.1.2	LL_TIM_OC_InitTypeDef	1250
65.1.3	LL_TIM_IC_InitTypeDef	1251
65.1.4	LL_TIM_ENCODER_InitTypeDef	1251
65.1.5	LL_TIM_HALLSENSOR_InitTypeDef	1252
65.1.6	LL_TIM_BDTR_InitTypeDef	1253
65.2	TIM Firmware driver API description.	1254
65.2.1	Detailed description of functions	1254
65.3	TIM Firmware driver defines	1329
65.3.1	TIM	1329
66	LL USART Generic Driver.	1344
66.1	USART Firmware driver registers structures	1344
66.1.1	LL_USART_InitTypeDef	1344
66.1.2	LL_USART_ClockInitTypeDef.	1344
66.2	USART Firmware driver API description	1345
66.2.1	Detailed description of functions	1345
66.3	USART Firmware driver defines	1396
66.3.1	USART	1396

67	LL UTILS Generic Driver	1401
67.1	UTILS Firmware driver registers structures	1401
67.1.1	LL_UTILS_PLLInitTypeDef	1401
67.1.2	LL_UTILS_ClkInitTypeDef	1401
67.2	UTILS Firmware driver API description	1401
67.2.1	System Configuration functions	1401
67.2.2	Detailed description of functions	1402
67.3	UTILS Firmware driver defines	1405
67.3.1	UTILS	1405
68	LL WWDG Generic Driver	1406
68.1	WWDG Firmware driver API description	1406
68.1.1	Detailed description of functions	1406
68.2	WWDG Firmware driver defines	1410
68.2.1	WWDG	1410
69	FAQs	1412
	Revision history	1415

List of tables

Table 1.	Acronyms and definitions	4
Table 2.	HAL driver files	7
Table 3.	User-application files	8
Table 4.	API classification	12
Table 5.	List of devices supported by HAL drivers	13
Table 6.	HAL API naming rules	15
Table 7.	Macros handling interrupts and specific clock configurations	17
Table 8.	Callback functions	18
Table 9.	HAL generic APIs	18
Table 10.	HAL extension APIs	19
Table 11.	Define statements used for HAL configuration	23
Table 12.	Description of GPIO_InitTypeDef structure	25
Table 13.	Description of EXTI configuration macros	27
Table 14.	MSP functions	31
Table 15.	Timeout values	34
Table 16.	LL driver files	38
Table 17.	Common peripheral initialization functions	40
Table 18.	Optional peripheral initialization functions	41
Table 19.	Specific Interrupt, DMA request and status flags management	42
Table 20.	Available function formats	42
Table 21.	Peripheral clock activation/deactivation management	42
Table 22.	Peripheral activation/deactivation management	43
Table 23.	Peripheral configuration management	43
Table 24.	Peripheral register management	43
Table 25.	Document revision history	1415

List of figures

Figure 1.	Example of project template	9
Figure 2.	Adding device-specific functions	20
Figure 3.	Adding family-specific functions	20
Figure 4.	Adding new peripherals	21
Figure 5.	Updating existing APIs.	21
Figure 6.	File inclusion model.	22
Figure 7.	HAL driver model	29
Figure 8.	Low-layer driver folders	39
Figure 9.	Low-layer driver CMSIS files	40

IMPORTANT NOTICE – PLEASE READ CAREFULLY

STMicroelectronics NV and its subsidiaries ("ST") reserve the right to make changes, corrections, enhancements, modifications, and improvements to ST products and/or to this document at any time without notice. Purchasers should obtain the latest relevant information on ST products before placing orders. ST products are sold pursuant to ST's terms and conditions of sale in place at the time of order acknowledgement.

Purchasers are solely responsible for the choice, selection, and use of ST products and ST assumes no liability for application assistance or the design of Purchasers' products.

No license, express or implied, to any intellectual property right is granted by ST herein.

Resale of ST products with provisions different from the information set forth herein shall void any warranty granted by ST for such product.

ST and the ST logo are trademarks of ST. For additional information about ST trademarks, please refer to www.st.com/trademarks. All other product or service names are the property of their respective owners.

Information in this document supersedes and replaces information previously supplied in any prior versions of this document.

© 2020 STMicroelectronics – All rights reserved